Học Reinforcement Learning từ đầu

Lộ trình 58 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Học Reinforcement Learning từ đầu: lộ trình 58 bài . . . 4
MDP, return và phương trình Bellman: giá trị của một trạng thái là phần thưởng kỳ vọng về sau . . . 9
TD, Q-learning và DQN: học giá trị từ từng bước bằng cách tự dựa vào ước lượng của chính mình . . . 16
Policy gradient và baseline: tối ưu trực tiếp chính sách, trừ một mốc để bớt nhiễu mà không đổi hướng đúng . . . 23
Actor-critic và GAE: dùng một người chấm điểm để giảm nhiễu, rồi chọn mức tin vào người chấm bằng λ 31
Vùng tin cậy và PPO: đi những bước đủ nhỏ để dữ liệu cũ còn đáng tin . . . 40
Khám phá, off-policy và offline RL: học từ dữ liệu không do chính sách đích sinh ra . . . 48
RLHF, mô hình thưởng, phạt KL và DPO: dạy mô hình theo ý người khi không có hàm điểm sẵn . . . 57
RL cho mô hình lập luận: phần thưởng kiểm chứng được và lợi thế tương đối theo nhóm (GRPO) . . . 66
00 Preface: Toán nền tảng và hướng dẫn đọc về Reinforcement Learning . . . 76
01 Playing Atari with Deep Reinforcement Learning (2013) . . . 101
02 Deep Reinforcement Learning with Double Q-learning (2015) . . . 109
03 Rainbow: Combining Improvements in Deep Reinforcement Learning (2017) . . . 116
04 Trust Region Policy Optimization (2015) . . . 125
05 High-Dimensional Continuous Control Using Generalized Advantage Estimation (2015) . . . 135
06 Asynchronous Methods for Deep Reinforcement Learning (2016) . . . 142
07 Proximal Policy Optimization Algorithms (2017) . . . 151
08 Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO (2020) . . . 158
09 What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study (2020) . . . 166
10 Continuous control with deep reinforcement learning (2015) . . . 174
11 Addressing Function Approximation Error in Actor-Critic Methods (2018) . . . 181
12 Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor (2018) . . . 189
13 Exploration by Random Network Distillation (2018) . . . 197
14 Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm (2017) . . . 207
15 Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (2019) . . . 214
16 Mastering Diverse Domains through World Models (2023) . . . 222
17 A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (2010) . . . 231
18 Generative Adversarial Imitation Learning (2016) . . . 238
19 Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems (2020) . . . 245
20 Conservative Q-Learning for Offline Reinforcement Learning (2020) . . . 253
21 Decision Transformer: Reinforcement Learning via Sequence Modeling (2021) . . . 259
22 The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games (2021) . . . 265
23 Deep Reinforcement Learning from Human Preferences (2017) . . . 274
24 Fine-Tuning Language Models from Human Preferences (2019) . . . 282
25 Learning to Summarize from Human Feedback (2020) . . . 289
26 Constitutional AI: Harmlessness from AI Feedback (2022) . . . 297
27 Scaling Laws for Reward Model Overoptimization (2022) . . . 306
28 The N+ Implementation Details of RLHF with PPO: A Case Study on TL;DR Summarization (2024) . . . 313
29 Direct Preference Optimization: Your Language Model is Secretly a Reward Model (2023) . . . 321
30 A General Theoretical Paradigm to Understand Learning from Human Preferences (2023) . . . 329
31 KTO: Model Alignment as Prospect Theoretic Optimization (2024) . . . 336

32 SimPO: Simple Preference Optimization with a Reference-Free Reward (2024) . . . 344
33 Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study (2024) . . . 350
34 STaR: Bootstrapping Reasoning With Reasoning (2022) . . . 357
35 Let's Verify Step by Step (2023) . . . 365
36 DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (2024) . . . 372
37 Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs (2024) . . . 381
38 Tulu 3: Pushing Frontiers in Open Language Model Post-Training (2024) . . . 389
39 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (2025) . . . 398
40 Kimi k1.5: Scaling Reinforcement Learning with LLMs (2025). . . . 408
41 DAPO: An Open-Source LLM Reinforcement Learning System at Scale (2025) . . . 417
42 Understanding R1-Zero-Like Training: A Critical Perspective (2025) . . . 426
43 Group Sequence Policy Optimization (2025) . . . 433
44 The Art of Scaling Reinforcement Learning Compute for LLMs (2025) . . . 440
45 Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? (2025) . . . 449
46 ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models (2025) . . . 457
47 Reinforcement Learning for Reasoning in Large Language Models with One Training Example (2025) . . . 465
48 Spurious Rewards: Rethinking Training Signals in RLVR (2025) . . . 472
49 The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models (2025) . . . 479
50 Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning (2025) . . . 487
51 Reinforcement Learning Finetunes Small Subnetworks in Large Language Models (2025) . . . 493
52 RL's Razor: Why Online Reinforcement Learning Forgets Less (2025) . . . 500
53 Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning (2025) . . . 506
54 The Landscape of Agentic Reinforcement Learning for LLMs: A Survey (2025). . . . 514
55 Visual-RFT: Visual Reinforcement Fine-Tuning (2025) . . . 524
56 olmOCR 2: Unit Test Rewards for Document OCR (2025) . . . 531
57 Reading or Reasoning? Format Decoupled Reinforcement Learning for Document OCR (2026) . . . 538

Lộ trình

Học Reinforcement Learning từ đầu: lộ trình 58 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **reinforcement learning (RL)** một cách tổng thể: từ deep RL kinh điển (DQN, policy gradient, PPO, actor-critic), planning, imitation và offline RL, multi-agent, đến RL cho LLM (RLHF, DPO, RLVR, GRPO và các biến thể), các nghiên cứu về việc RL thực sự thay đổi gì trong model, agentic RL và RL cho VLM. Thứ tự được sắp cho **người chưa biết gì về domain**.

Các chương diễn giải các công trình nghiên cứu; đường dẫn nguồn được ghi trong từng chương. Bài 00 trình bày những khái niệm nền, gồm Bellman, Q-learning, REINFORCE, định lý policy gradient và các ý tưởng từ sách Sutton & Barto.

> Đây là tài liệu học tập, không phải bản dịch. Những ứng dụng và thí nghiệm gợi ý là phần suy luận, cần phân biệt với kết quả của bài báo gốc.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	RL là gì, phân loại, toán (MDP, Bellman, DP/MC/TD, Q-learning, policy gradient, baseline, GAE, importance sampling, TRPO/PPO, entropy), RL cho LLM (RLHF, KL, DPO, GRPO/RLOO, RLVR, pass@k, reward hacking), cạm bẫy, thuật ngữ, câu tự kiểm tra. **Đọc trước tiên.**
01	`nen01_mdp_return_va_bellman.md`	Chương nền: MDP, return và phương trình Bellman. giá trị của một trạng thái là phần thưởng kỳ vọng về sau.
02	`nen02_td_q_learning_va_dqn.md`	Chương nền: TD, Q-learning và DQN. học giá trị từ từng bước bằng cách tự dựa vào ước lượng của chính mình.
03	`nen03_policy_gradient_va_baseline.md`	Chương nền: Policy gradient và baseline. tối ưu trực tiếp chính sách, trừ một mốc để bớt nhiễu mà không đổi hướng đúng.
04	`nen04_actor_critic_va_gae.md`	Chương nền: Actor-critic và GAE. dùng một người chấm điểm để giảm nhiễu, rồi chọn mức tin vào người chấm bằng λ.
05	`nen05_trust_region_va_ppo.md`	Chương nền: Vùng tin cậy và PPO. đi những bước đủ nhỏ để dữ liệu cũ còn đáng tin.
06	`nen06_kham_pha_off_policy_va_offline_rl.md`	Chương nền: Khám phá, off-policy và offline RL. học từ dữ liệu không do chính sách đích sinh ra.
07	`nen07_rlhf_reward_model_kl_va_dpo.md`	Chương nền: RLHF, mô hình thưởng, phạt KL và DPO. dạy mô hình theo ý người khi không có hàm điểm sẵn.
08	`nen08_rl_cho_llm_grpo_va_phan_thuong_kiem_chung.md`	Chương nền: RL cho mô hình lập luận. phần thưởng kiểm chứng được và lợi thế tương đối theo nhóm (GRPO).

Giai đoạn 1: Deep RL dựa trên value

#	File	Bài	Ý chính
01	`01_2013_dqn.md`	DQN (Mnih et al.)	Q-learning với CNN trên pixel, experience replay. Mở đầu deep RL.
02	`02_2015_double_dqn.md`	Double DQN	Max trên ước lượng nhiễu gây overestimation; tách chọn và đánh giá hành động.
03	`03_2017_rainbow.md`	Rainbow	Gộp 6 cải tiến của DQN; ablation xem cái nào quan trọng.

Giai đoạn 2: Policy gradient và actor-critic

#	File	Bài	Ý chính
04	`04_2015_trpo.md`	TRPO	Cập nhật policy trong trust region theo KL; cận cải thiện đơn điệu.
05	`05_2015_gae.md`	GAE	Advantage trung bình trọng số mũ: núm vặn bias-variance λ.
06	`06_2016_a3c.md`	A3C	Actor-critic bất đồng bộ nhiều luồng, n-step, entropy bonus.
07	`07_2017_ppo.md`	**PPO**	Clipped surrogate: thuật toán RL phổ biến nhất, nền của RLHF.
08	`08_2020_implementation_matters_ppo_trpo.md`	Implementation Matters	Chi tiết code quan trọng ngang thuật toán.
09	`09_2020_what_matters_on_policy_rl.md`	What Matters in On-Policy RL	Nghiên cứu quy mô lớn, checklist lựa chọn cho PPO.

Giai đoạn 3: Điều khiển liên tục và maximum-entropy RL

#	File	Bài	Ý chính
10	`10_2015_ddpg.md`	DDPG	Deterministic policy gradient off-policy cho hành động liên tục.
11	`11_2018_td3.md`	TD3	Clipped double-Q, cập nhật policy trễ, làm mượt target.
12	`12_2018_sac.md`	SAC	Maximum-entropy RL, soft Q, nghiệm Boltzmann.

Giai đoạn 4: Khám phá, planning và world model

#	File	Bài	Ý chính
13	`13_2018_rnd_exploration.md`	RND	Phần thưởng nội tại bằng lỗi dự đoán của mạng ngẫu nhiên.
14	`14_2017_alphazero.md`	AlphaZero	Self-play + MCTS, một thuật toán cho cờ vua, shogi, cờ vây.
15	`15_2019_muzero.md`	MuZero	Planning với model học được trong không gian ẩn.
16	`16_2023_dreamerv3.md`	DreamerV3	World model + học trong “tưởng tượng”, một bộ siêu tham số cho mọi domain.

Giai đoạn 5: Imitation learning, offline RL và RL như sequence modeling

#	File	Bài	Ý chính
17	`17_2010_dagger.md`	DAgger	Lỗi tích lũy của behavior cloning; thu thập lại dữ liệu on-policy. Gốc của on-policy distillation.
18	`18_2016_gail.md`	GAIL	Imitation kiểu GAN: discriminator đóng vai cost.
19	`19_2020_offline_rl_tutorial.md`	Offline RL tutorial	Bản đồ offline RL, distributional shift, vấn đề mở.
20	`20_2020_cql.md`	CQL	Q-function bảo thủ, cận dưới của value.
21	`21_2021_decision_transformer.md`	Decision Transformer	RL thành bài toán sinh chuỗi có điều kiện return.

Giai đoạn 6: Multi-agent

#	File	Bài	Ý chính
22	`22_2021_mappo.md`	MAPPO	PPO đơn giản nhưng mạnh trong multi-agent hợp tác.

Giai đoạn 7: RLHF

#	File	Bài	Ý chính
llm 07	`../llm content/07_2022_instructgpt_rlhf.md`	InstructGPT	SFT → reward model → PPO (đã có trong thư mục llm).
23	`23_2017_deep_rl_human_preferences.md`	Christiano et al.	Học reward từ so sánh cặp của người (Bradley-Terry).
24	`24_2019_finetuning_lm_human_preferences.md`	Ziegler et al.	RLHF đầu tiên cho LM: reward model + PPO + phạt KL.
25	`25_2020_learning_to_summarize.md`	Learning to Summarize	Pipeline RLHF đầy đủ ở quy mô lớn.
26	`26_2022_constitutional_ai.md`	Constitutional AI	RLAIF: phản hồi từ AI theo bộ nguyên tắc.
27	`27_2022_reward_model_overoptimization.md`	RM Overoptimization	Luật tỉ lệ của Goodhart: proxy reward so với gold reward theo KL.
28	`28_2024_n_implementation_details_rlhf_ppo.md`	N+ Implementation Details	Checklist chi tiết tái lập RLHF với PPO.

Giai đoạn 8: Preference optimization không cần RL

#	File	Bài	Ý chính
29	`29_2023_dpo.md`	**DPO**	Từ nghiệm KL-regularized ra loss phân loại trên cặp.
30	`30_2023_ipo_psi_po.md`	IPO / ΨPO	Khung lý thuyết chung; DPO overfit khi preference tất định.
31	`31_2024_kto.md`	KTO	Prospect theory; dữ liệu tốt/xấu không cần ghép cặp.
32	`32_2024_simpo.md`	SimPO	Reward ngầm chuẩn hóa độ dài, không cần model tham chiếu.

#	File	Bài	Ý chính
33	`33_2024_dpo_vs_ppo.md`	Is DPO Superior to PPO?	Khi nào PPO thắng DPO và vì sao.

Giai đoạn 9: RL cho reasoning (RLVR)

#	File	Bài	Ý chính
34	`34_2022_star.md`	STaR	Tự sinh rationale, giữ cái đúng, lặp lại (expert iteration).
35	`35_2023_lets_verify_step_by_step.md`	Let's Verify Step by Step	Process reward model so với outcome reward model.
36	`36_2024_deepseekmath_grpo.md`	**DeepSeekMath (GRPO)**	GRPO: baseline nhóm thay critic; góc nhìn thống nhất SFT/RFT/DPO/PPO.
37	`37_2024_rloo_back_to_basics.md`	RLOO	REINFORCE với baseline leave-one-out đủ tốt cho RLHF.
38	`38_2024_tulu3_rlvr.md`	Tulu 3	Đặt tên RLVR; công thức post-training mở đầy đủ.
39	`39_2025_deepseek_r1.md`	**DeepSeek-R1**	RL thuần từ base (R1-Zero), suy luận dài tự xuất hiện.
40	`40_2025_kimi_k15.md`	Kimi k1.5	RL ngữ cảnh dài, mirror descent, long2short.
41	`41_2025_dapo.md`	DAPO	Clip-higher, dynamic sampling, loss mức token, bỏ KL.
42	`42_2025_dr_grpo.md`	Dr. GRPO	Hai bias của GRPO (chuẩn hóa độ dài, chia std).
43	`43_2025_gspo.md`	GSPO (Qwen)	Importance ratio mức chuỗi; ổn định cho MoE.
44	`44_2025_scalerl.md`	ScaleRL	Đường cong compute dạng sigmoid; công thức RL scale được.

Giai đoạn 10: RL thực sự thay đổi gì trong model

#	File	Bài	Ý chính
45	`45_2025_does_rl_incentivize_reasoning.md`	Does RL Really Incentivize Reasoning?	pass@k lớn: base model bắt kịp; RL chủ yếu chọn lọc.
46	`46_2025_prorl.md`	ProRL	RL kéo dài đủ lâu thì biên giới năng lực có mở rộng. Đọc cùng 45.
47	`47_2025_one_shot_rlvr.md`	1-shot RLVR	Một ví dụ huấn luyện cũng đủ cải thiện mạnh.
48	`48_2025_spurious_rewards.md`	Spurious Rewards	Reward ngẫu nhiên vẫn giúp Qwen-Math; cảnh giác khi đánh giá.
49	`49_2025_entropy_mechanism.md`	Entropy Mechanism	Entropy collapse, động lực entropy qua covariance.
50	`50_2025_high_entropy_minority_tokens.md`	High-Entropy Minority Tokens	20% token entropy cao quyết định hiệu quả RL.
51	`51_2025_rl_finetunes_small_subnetworks.md`	RL Finetunes Small Subnetworks	Cập nhật của RL rất thưa. Nối với pruning.

#	File	Bài	Ý chính
52	`52_2025_rls_razor.md`	RL's Razor	RL on-policy quên ít hơn SFT; mức quên đo bằng KL.

Giai đoạn 11: Agentic RL và RL cho VLM/OCR

#	File	Bài	Ý chính
53	`53_2025_search_r1.md`	Search-R1	RL nhiều lượt với search engine trong vòng lặp.
54	`54_2025_agentic_rl_survey.md`	Agentic RL Survey	Bản đồ agentic RL: từ MDP một bước sang POMDP nhiều lượt.
55	`55_2025_visual_rft.md`	Visual-RFT	Reward kiểm chứng được cho tác vụ thị giác (IoU, phân loại).
56	`56_2025_olmocr2_unit_test_rewards.md`	olmOCR 2	Unit test làm reward cho OCR.
57	`57_2026_format_decoupled_rl_ocr.md`	FD-RL	Reward tách theo format (text, công thức, bảng) cho document OCR.

Cách đọc gợi ý

- **Người mới hoàn toàn:** 00 → 01 → 02 → 04 → 05 → 07 → 06 → 12 → 17.
- **Làm post-training LLM:** 00 (mục 3.6–3.11 và 4) → 07 → 05 → llm 07 → 25 → 27 → 29 → 36 → 37 → 39 → 41 → 42 → 43 → 45 → 49 → 52.
- **Agent và đa phương thức:** 39 → 53 → 54 → 55 → 56 → 57.
- **Hiểu tranh luận "RL làm gì":** 36 (unified paradigm) → 45 ↔ 46 → 47 → 48 → 49 → 50 → 51 → 52.
- **Deep RL ngoài LLM (game, robot, planning):** 01–03 → 04–09 → 10–12 → 13–16 → 19–22.
- Mỗi file có mục **Thí nghiệm gợi ý**.

Liên kết với các thư mục khác

- `../distillation content/`: on-policy distillation dùng cùng bộ máy với RL (28 MiniLLM, 29 GKD, 44 Qwen3, 47 ShortOPD); 38 là phía distill của DeepSeek-R1.
- `../pruning content/`: nén model; bài 51, 52 ở đây hỏi RL tương tác thế nào với mô hình đã cắt và phục hồi.
- `../interp content/`: interpretability, để xem RL thay đổi gì bên trong model.

Nền tảng 01

MDP, return và phương trình Bellman: giá trị của một trạng thái là phần thưởng kỳ vọng về sau

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất và kỳ vọng ở math nền tảng 08; ML cơ bản nền tảng 01 (học máy là gì); không cần biết RL

Dùng ở: mọi bài RL của cuốn, đặc biệt rl 01 (DQN), rl 02 (Double DQN), rl 05 (GAE, dùng hàm giá trị và chiết khấu), rl 07 (PPO)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao cần khái niệm này

MDP (Markov decision process, quá trình quyết định Markov) là cách viết bài toán "một người ra quyết định nhiều lần liên tiếp, mỗi quyết định ảnh hưởng đến tình huống kế tiếp và đến phần thưởng". Mọi thuật toán trong cuốn này, từ DQN chơi game đến PPO huấn luyện mô hình ngôn ngữ, đều được viết trên khung này. Nếu chưa quen các chữ s, a, r, γ, V, Q thì các bài sau sẽ như đọc một ngôn ngữ lạ.

Hãy hình dung bạn đi qua một hành lang tối để tới cửa ra có tiền thưởng. Bước nào cũng không có thưởng, chỉ đến cửa mới có. Câu hỏi khó của RL: bước đầu tiên "tốt" hay "xấu" khi phần thưởng chỉ đến rất muộn? Ta cần một con số gán cho mỗi tình huống, ghi lại "từ đây về sau kỳ vọng được bao nhiêu". Con số đó là **hàm giá trị**, và quan hệ đệ quy giữa các giá trị là **phương trình Bellman**. Chương này xây từng bước: khung MDP, return, hàm giá trị, Bellman, chính sách tối ưu, rồi giải tay một MDP ba trạng thái.

2. Nhắc nền: kỳ vọng và tổng có trọng số

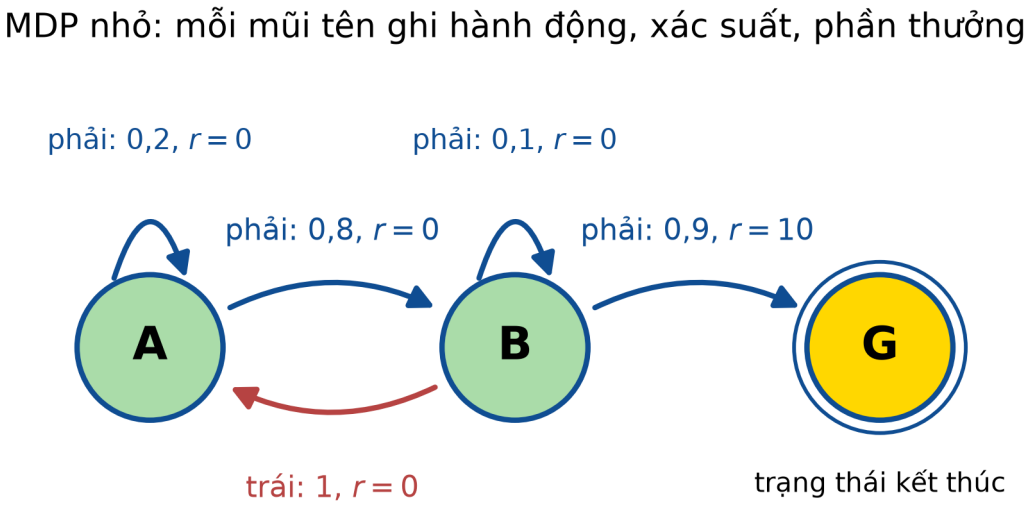
Ta chỉ cần hai ý. Thứ nhất, **kỳ vọng** của một đại lượng ngẫu nhiên là trung bình có trọng số theo xác suất. Ví dụ (tự tính): nhận 10 với xác suất $0{,}9$ và 0 với xác suất $0{,}1$ thì kỳ vọng là $0{,}9 \cdot 10 + 0{,}1 \cdot 0 = 9$. Thứ hai, **tổng có chiết khấu**: tổng $1 + \gamma + \gamma^2 + \dots$ với $0 \le \gamma < 1$ hội tụ về $1/(1-\gamma)$. Với $\gamma = 0{,}9$ ra 10, với $\gamma = 0{,}99$ ra 100, với $\gamma = 0{,}5$ ra 2 (tự tính). Tổng này sẽ cho ta biết "tầm nhìn" của agent dài khoảng bao nhiêu bước.

3. Trực giác bằng hình: vòng lặp tác tử và môi trường

Một **agent** (tác tử) và một **environment** (môi trường) thay phiên nhau. Ở bước t, agent thấy **trạng thái** s_t, chọn **hành động** a_t; môi trường trả về **phần thưởng** r_{t+1} (một con số, có thể bằng 0) và trạng thái kế tiếp s_{t+1}. Chuỗi $s_0, a_0, r_1, s_1, a_1, r_2, \dots$ gọi là **quỹ đạo** (trajectory) hay episode nếu có điểm kết thúc. Quy ước chỉ số trong chương: phần thưởng sau hành động ở bước t ký hiệu r_{t+1}.

Cách đọc hình: mỗi vòng tròn là một trạng thái, vòng đôi vàng là trạng thái kết thúc (vào đó là hết). Mỗi mũi tên ghi hành động, xác suất xảy ra và phần thưởng r nhận được khi đi qua mũi tên đó. Ở A hành động "phải" đưa sang B với xác suất $0{,}8$, còn lại $(0{,}2)$ ở nguyên A. Ở B, "phải" đưa tới G với xác suất $0{,}9$ và nhận 10; hành động "trái" quay về A chắc chắn. Các xác suất chọn để tổng ra 1 trên mỗi hành động. Đây là MDP bịa ra để học, không mô tả hệ thống thật nào.

Điều quan trọng: kết quả của một hành động **chỉ phụ thuộc trạng thái hiện tại và hành động**, không phụ thuộc lịch sử trước đó. Đó là **tính Markov**. Nếu "trạng thái" của bạn thiếu thông tin cần thiết (ví dụ chỉ một



Hình 1: MDP nhỏ gồm hai trạng thái thường A, B và một trạng thái kết thúc G (minh hoạ, tự tính)

khung hình đứng yên của game, không biết vận tốc), ta phải đưa thêm thông tin vào trạng thái (xếp nhiều khung hình) để tính Markov đúng.

4. Định nghĩa chính thức

4.1 MDP

Một MDP gồm: tập trạng thái $\mathcal{S}$, tập hành động $\mathcal{A}$, **xác suất chuyển** $P(s' \mid s, a)$ (xác suất sang s' khi làm a ở s), **hàm thưởng** $r(s, a, s')$ (hoặc kỳ vọng của nó), và hệ số chiết khấu $\gamma \in [0, 1]$. Một **chính sách** (policy) $\pi(a \mid s)$ là xác suất chọn hành động a khi ở s; chính sách xác định là trường hợp xác suất chỉ dồn vào một hành động. Mục tiêu của RL: tìm π cho tổng thưởng lớn nhất. "Lớn nhất" cần định nghĩa chính xác, nên ta cần return.

4.2 Return và chiết khấu

Return từ bước t là tổng thưởng có chiết khấu về sau:

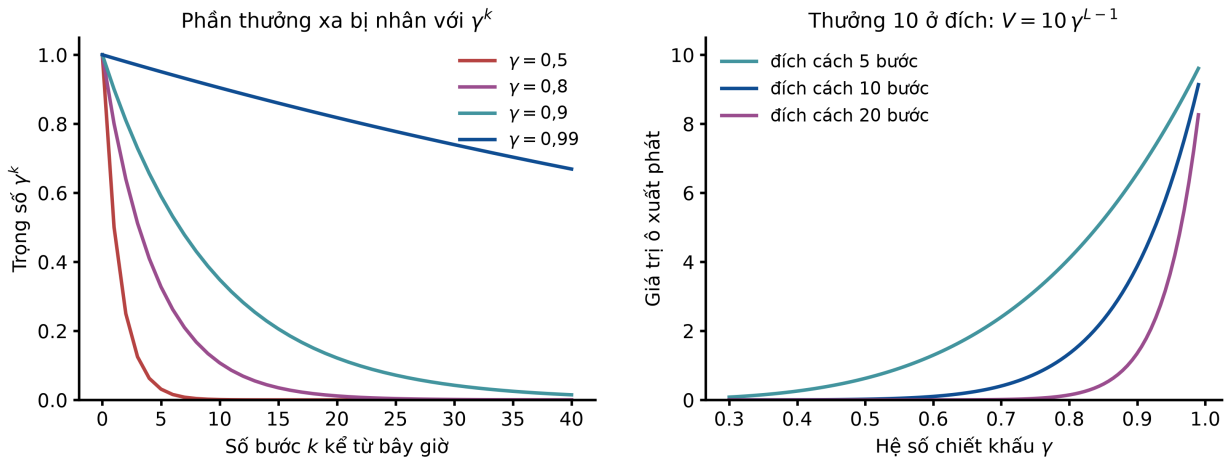
$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

Vì sao chiết khấu? Hai lý do thực dụng: (a) nếu không chiết khấu và episode dài vô hạn thì tổng có thể vô hạn, không so sánh được; (b) ta thường coi thưởng sớm có giá trị hơn thưởng muộn (và tương lai bất định hơn). γ nhỏ làm agent "thiển cận", γ gần 1 làm nó nhìn xa.

Một đẳng thức đơn giản nhưng là xương sống của cả chương: **return có dạng đệ quy**

$$G_t = r_{t+1} + \gamma\, G_{t+1}.$$

Ví dụ (tự tính): chuỗi thưởng $1, 0, 2$ rồi hết, $\gamma = 0{,}5$. Tính từ cuối: return tại bước cuối là 2, bước trước đó là $0 + 0{,}5 \cdot 2 = 1$, bước đầu là $1 + 0{,}5 \cdot 1 = 1{,}5$. Tính thẳng: $1 + 0{,}5 \cdot 0 + 0{,}25 \cdot 2 = 1{,}5$. Hai cách khớp nhau.



Hình 2: Trọng số gamma mũ k và giá trị ô xuất phát theo gamma (minh hoạ, tự tính)

Cách đọc hình: bên trái, mỗi đường là trọng số γ^k gán cho phần thưởng cách k bước. Với $\gamma = 0{,}5$ chỉ vài bước đầu còn đáng kể; với $\gamma = 0{,}99$ phần thưởng cách 40 bước vẫn còn trọng số khoảng $0{,}67$. Bên phải: nếu thưởng 10 chỉ có ở đích cách L bước thì giá trị của ô xuất phát là $10\gamma^{L-1}$. Đích càng xa, giá trị càng nhạy với γ (đường tím rơi rất nhanh khi γ giảm). Hệ quả thực tế: chọn γ quá nhỏ thì agent không thấy được phần thưởng xa; chọn rất gần 1 thì giá trị lớn, ước lượng nhiễu hơn.

4.3 Hàm giá trị V và hàm giá trị hành động Q

Cho chính sách π. **Hàm giá trị trạng thái**: return kỳ vọng nếu bắt đầu ở s rồi làm theo π:

$$V^\pi(s) = \mathbb{E}_\pi\left[G_t \mid s_t = s\right].$$

Hàm giá trị hành động: return kỳ vọng nếu ở s làm hành động a trước (bất kể π nói gì), rồi mới theo π:

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[G_t \mid s_t = s, a_t = a\right].$$

Quan hệ giữa hai hàm: $V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s,a)$, tức V là Q lấy trung bình theo chính sách. Hiệu $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ gọi là **advantage** (lợi thế): hành động a tốt hơn mức trung bình của chính sách bao nhiêu. Nó sẽ là nhân vật chính ở nen03 của cuốn này.

Hãy nhớ: V trả lời "tình huống này tốt cỡ nào", còn Q trả lời "làm việc này ở tình huống này tốt cỡ nào". Khi có Q ta có thể chọn hành động chỉ bằng cách lấy hành động có Q lớn nhất, nên Q là thứ DQN học (xem nen02).

4.4 Phương trình Bellman (kỳ vọng)

Thay $G_t = r_{t+1} + \gamma G_{t+1}$ vào định nghĩa V. Kỳ vọng của G_{t+1} khi biết $s_{t+1} = s'$ chính là $V^\pi(s')$ nhờ tính Markov, nên

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a)\left[r(s,a,s') + \gamma V^\pi(s')\right].$$

Tương tự cho Q:

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma \sum_{a'} \pi(a' \mid s')\, Q^{\pi}(s',a') \right].$$

Cách đọc bằng lời: giá trị của một trạng thái bằng **thưởng ngay lập tức kỳ vọng cộng chiết khấu nhân giá trị của trạng thái kế**. Đây là một hệ phương trình tuyến tính (mỗi trạng thái một phương trình, ẩn là các $V^{\pi}(s)$). Với MDP nhỏ ta có thể giải trực tiếp; với MDP lớn ta giải lặp, như mục 6.

4.5 Chính sách tối ưu và phương trình Bellman tối ưu

Ta nói π tốt hơn hoặc bằng π' nếu $V^{\pi}(s) \geq V^{\pi'}(s)$ với mọi s. Với MDP hữu hạn có chiết khấu, tồn tại ít nhất một chính sách tối ưu π^* tốt hơn hoặc bằng mọi chính sách khác (kết quả kinh điển của lý thuyết MDP). Các chính sách tối ưu chung hàm giá trị V^* và Q^*, thoả

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma \max_{a'} Q^*(s',a') \right], \qquad V^*(s) = \max_{a} Q^*(s,a).$$

Chỉ khác phương trình kỳ vọng ở chỗ: thay "trung bình theo π" bằng "max". Khi có Q^*, chính sách tối ưu là **tham lam**: $\pi^*(s) = \arg\max_a Q^*(s,a)$. Phương trình này **phi tuyến** vì có max, nhưng có cách giải lặp tương tự. Khi bạn đọc "Q-learning" ở nen02, thuật toán đó chính là cách ước lượng Q^* từ mẫu, bằng cách biến phương trình trên thành quy tắc cập nhật.

5. Ví dụ tay: giải V cho MDP ba trạng thái

Dùng MDP ở hình trên, $\gamma = 0{,}9$, trạng thái kết thúc G có $V(G) = 0$. Cố định chính sách π: **luôn chọn "phải"** ở cả A và B. Tất cả số dưới đây đã kiểm bằng numpy (tự tính).

Cách 1: giải đại số. Viết Bellman cho từng trạng thái.

- B: $V(B) = 0{,}9 \cdot (10 + \gamma \cdot 0) + 0{,}1 \cdot (0 + \gamma V(B)) = 9 + 0{,}09\, V(B)$. Suy ra $V(B) = 9/0{,}91 \approx 9{,}890$.
- A: $V(A) = 0{,}8 \cdot \gamma V(B) + 0{,}2 \cdot \gamma V(A) = 0{,}72\, V(B) + 0{,}18\, V(A)$. Suy ra $V(A) = 0{,}72\, V(B)/0{,}82 \approx 8{,}684$.

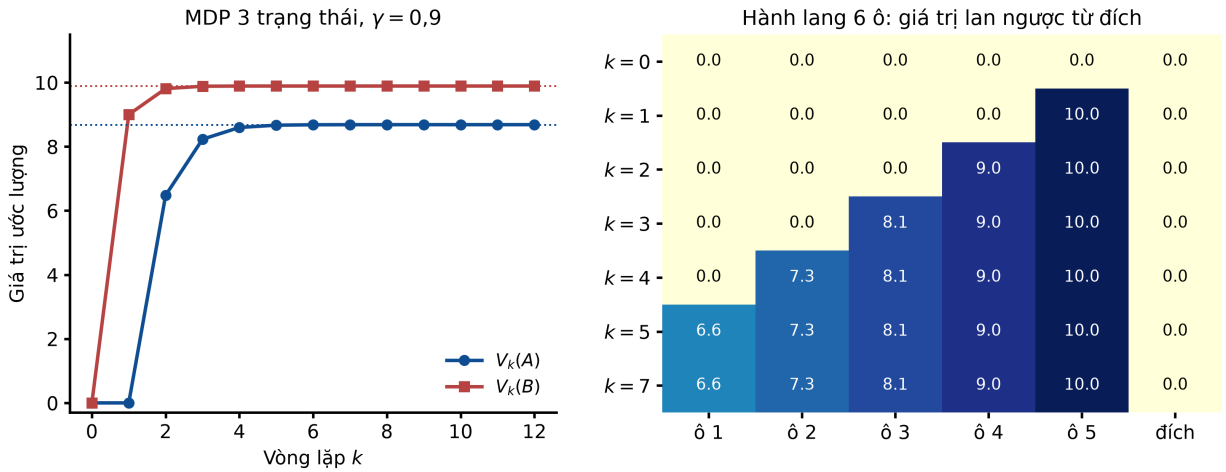
Hai giá trị khớp trực giác: B gần đích nên giá trị cao; A phải qua B nên bị chiết khấu thêm và có thêm xác suất lãng phí một bước.

Cách 2: lặp Bellman (policy evaluation). Khởi đầu $V_0(A) = V_0(B) = 0$, rồi áp dụng vế phải của phương trình Bellman để cập nhật $V_k \to V_{k+1}$ cho cả hai trạng thái cùng lúc:

- Vòng 1: $V_1(B) = 0{,}9 \cdot 10 + 0{,}1 \cdot 0 = 9$; $V_1(A) = 0{,}8 \cdot 0{,}9 \cdot 0 + 0{,}2 \cdot 0{,}9 \cdot 0 = 0$.
- Vòng 2: $V_2(B) = 9 + 0{,}09 \cdot 9 = 9{,}81$; $V_2(A) = 0{,}72 \cdot 9 + 0{,}18 \cdot 0 = 6{,}48$.
- Vòng 3: $V_3(B) = 9 + 0{,}09 \cdot 9{,}81 = 9{,}8829$; $V_3(A) = 0{,}72 \cdot 9{,}81 + 0{,}18 \cdot 6{,}48 = 8{,}2296$.
- Vòng 4: $V_4(B) \approx 9{,}8895$; $V_4(A) \approx 8{,}597$. Về sau chúng tiến sát $9{,}890$ và $8{,}684$.

Cách đọc hình: bên trái là các giá trị tính tay ở trên, kéo dài đến vòng 12; đường chấm là đáp số giải đại số. B hội tụ trước vì chỉ cách đích một bước; A phải đợi thông tin từ B nên chậm hơn một nhịp. Bên phải là hành lang sáu ô (năm ô thường và đích, luôn đi sang phải, vào đích nhận 10, $\gamma = 0{,}9$): mỗi hàng là một vòng lặp, và giá trị lan **ngược từ đích** mỗi vòng thêm một ô, kết quả cuối là 10, 9, $8{,}1$, $7{,}29$, $6{,}561$ (tự tính). Điều ta rút ra: lặp Bellman là "thông tin về phần thưởng lan dần về phía các trạng thái xa", mỗi vòng thêm một bước.

Chính sách tối ưu cho B. Giờ cho B hai lựa chọn. Với $V(A) \approx 8{,}684$ và $V(B) \approx 9{,}890$ ở trên: $Q(B, \text{phải}) = 9 + 0{,}09 \cdot 9{,}890 \approx 9{,}890$, còn $Q(B, \text{trái}) = \gamma V(A) = 0{,}9 \cdot 8{,}684 \approx 7{,}816$ (tự tính). "Phải" lớn hơn, nên $\pi^*(B) = \text{phải}$; với A chỉ có một hành động hữu ích nên π^* chính là chính sách ta đã dùng. Kết luận này phụ thuộc số liệu: nếu hành động "trái" dẫn tới một trạng thái giá trị cao hơn thì lựa chọn tối ưu ở B sẽ đổi.



Hình 3: Lan truyền giá trị qua các vòng lặp Bellman, MDP ba trạng thái và hành lang sáu ô (minh hoạ, tự tính)

6. Biến thể và chi tiết hay bị bỏ qua

Policy evaluation và policy iteration. Việc lặp Bellman để tính V^π cho π cố định gọi là policy evaluation. Nếu sau đó đổi π thành tham lam theo Q vừa tính (policy improvement) rồi lặp lại, ta có policy iteration; lặp trực tiếp phương trình tối ưu với max gọi là value iteration. Cả hai chỉ chạy được khi **biết** P và r (gọi là có mô hình). RL thực tế thường không biết P, nên phải ước lượng từ mẫu: đó là lý do có TD và Q-learning ở nen02.

Vì sao lặp Bellman hội tụ. Gọi T là phép "áp dụng vế phải của Bellman" lên cả vector giá trị. Với $\gamma < 1$, mỗi lần áp dụng T kéo hai vector giá trị bất kỳ lại gần nhau ít nhất một hệ số γ (tính co theo chuẩn lớn nhất). Vì vậy dù khởi đầu ở đâu, dãy V_k cũng tiến tới cùng một điểm cố định, chính là V^π. Ví dụ (tự tính): trong MDP của chương, sai số lớn nhất so với đáp số là $9{,}890$ ở $k = 0$ (khởi đầu bằng không), $8{,}684$ ở $k = 1$, $2{,}204$ ở $k = 2$ và tiếp tục giảm. Đây là suy luận chuẩn của lý thuyết quy hoạch động, không phải kết quả của một paper trong cuốn.

Chính sách ngẫu nhiên hay xác định. Với MDP đã biết, tồn tại chính sách tối ưu xác định, nên khi đã có Q^* ta không cần ngẫu nhiên. Tuy vậy, khi học ta vẫn cần ngẫu nhiên để **thăm dò** (exploration): nếu luôn chọn hành động có vẻ tốt nhất theo ước lượng hiện tại, ta không bao giờ phát hiện ra hành động khác thực ra tốt hơn. Thăm dò và khai thác (exploitation) là căng thẳng trung tâm của RL; nen02 dùng ϵ-greedy (đa số chọn tham lam, đôi khi chọn ngẫu nhiên) và nen03 dùng chính sách xác suất mà bản thân đã ngẫu nhiên.

Đọc một công thức RL bằng ba câu hỏi. Khi gặp công thức mới trong paper, hãy hỏi: (1) kỳ vọng lấy theo phân phối nào (chính sách nào sinh dữ liệu)? (2) đại lượng bên trong là thưởng một bước, return, hay giá trị (đã là kỳ vọng rồi)? (3) trạng thái kết thúc xử lý thế nào? Ba câu này tránh được phần lớn nhầm lẫn khi chuyển từ định nghĩa sang cài đặt.

Phần thưởng thưa và định hình thưởng. Nếu thưởng chỉ có ở cuối (như hành lang trên), tín hiệu lan chậm và khó học khi chưa từng chạm đích. Người ta thêm thưởng trung gian (shaping) để dẫn đường, nhưng thưởng thêm sai có thể dạy agent làm điều không mong muốn. Đây là điểm các bài RL cho LLM quay lại nhiều lần (thưởng chỉ có ở token cuối của câu trả lời).

Episodic và liên tục. Episodic có trạng thái kết thúc (ta đặt V của nó bằng 0 và không có bước tiếp). Bài toán liên tục không kết thúc cần $\gamma < 1$ để return hữu hạn. Một số cài đặt cắt episode ở độ dài tối đa (time limit) mà không phải do trạng thái kết thúc thật; coi cắt như kết thúc thật làm V sai, một lỗi cài đặt hay gặp.

Trạng thái và quan sát. Trong thực tế agent thường chỉ thấy **quan sát** o_t, không phải toàn bộ trạng thái

(POMDP). Cách làm phổ biến: xếp nhiều quan sát gần nhất, hoặc dùng mạng có bộ nhớ, để xấp xỉ trạng thái Markov.

Kích thước bảng. Khi số trạng thái nhỏ, V hay Q có thể là một bảng. Khi trạng thái là ảnh pixel, không thể lập bảng: ta dùng mạng nơ-ron $Q_\theta(s, a)$ để xấp xỉ, và từ đó có deep RL. Mạng xấp xỉ mất các bảo đảm hội tụ của bảng, nguồn gốc của nhiều thủ thuật ở nen02.

7. Nối sang các bài paper của cuốn

- **rl 01 (DQN)** dùng đúng phương trình Bellman tối ưu ở mục 4.5, thay bảng Q bằng mạng nơ-ron. Muốn hiểu "mục tiêu" mà mạng học, hãy nhìn vế phải của phương trình ấy; chi tiết ở nen02.
- **rl 02 (Double DQN)** sửa một thiên lệch xuất hiện chính từ phép max trong phương trình tối ưu; nen02 giải thích bằng ví dụ.
- **rl 05 (GAE)** xây trên advantage $A = Q - V$ và đệ quy return $G_t = r_{t+1} + \gamma G_{t+1}$, chỉ khác ở chỗ trộn nhiều độ dài bước. Hệ số γ ở mục 4.2 chính là γ trong bài đó.
- **rl 04 đến 07 (TRPO, A3C, PPO)** tối ưu trực tiếp chính sách π_θ; khái niệm V dùng làm baseline được giải thích ở nen03.
- **Các bài RL cho LLM** coi mô hình ngôn ngữ là chính sách $\pi(a \mid s)$ với trạng thái là prompt cộng phần đã sinh và hành động là token kế tiếp; thưởng thường chỉ xuất hiện ở cuối chuỗi, và γ hay được đặt bằng 1 (đây là mô tả chung, chi tiết xem từng bài).

8. Sai lầm hay gặp

- **Nhầm r với G.** r_{t+1} là thưởng của một bước; G_t là tổng chiết khấu về sau. V và Q là kỳ vọng của G, không phải của r.
- **Nhầm V với Q.** $V(s)$ không gắn với hành động nào cụ thể; $Q(s, a)$ cố định hành động đầu rồi theo chính sách. Chính sách tối ưu cần Q (hoặc mô hình) để biết chọn hành động nào.
- **Quên rằng V^π phụ thuộc π.** Đổi chính sách thì mọi V đổi theo; nói "giá trị của trạng thái A" mà không nói theo chính sách nào là chưa đủ.
- **Đặt $\gamma = 1$ cho bài toán không kết thúc.** Return có thể vô hạn. Với $\gamma = 1$ chỉ nên dùng khi episode có độ dài hữu hạn.
- **Bỏ qua trạng thái kết thúc khi cập nhật.** Phép cập nhật tại bước cuối phải dùng $V(s') = 0$, không phải một giá trị mạng dự đoán cho "trạng thái sau đích".
- **Tưởng γ chỉ là tham số kỹ thuật.** γ thay đổi bài toán được tối ưu: hai γ khác nhau có thể cho hai chính sách tối ưu khác nhau.

9. Bảng tóm tắt

Đại lượng	Công thức	Ý nghĩa
Return	$G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$	Tổng thưởng chiết khấu từ t
Đệ quy return	$G_t = r_{t+1} + \gamma G_{t+1}$	Cơ sở của Bellman
V^π	$\mathbb{E}_\pi[G_t \mid s_t = s]$	Giá trị trạng thái
Q^π	$\mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$	Giá trị hành động
Advantage	$A^\pi = Q^\pi - V^\pi$	Hơn mức trung bình bao nhiêu
Bellman kỳ vọng	$V^\pi(s) = \sum_a \pi \sum_{s'} P\,[r + \gamma V^\pi(s')]$	Hệ tuyến tính
Bellman tối ưu	$Q^* = \sum_{s'} P\,[r + \gamma \max_{a'} Q^*(s', a')]$	Có max, phi tuyến
Tầm nhìn hiệu dụng	$1/(1-\gamma)$	Cỡ số bước còn đáng kể

10. Bài tập tự tính

Các bài sau là phép tính độc lập, không phải số liệu thực nghiệm của paper. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Return

Chuỗi thưởng $2, 0, 4, 1$ rồi hết, $\gamma = 0{,}5$. Tính G_0 bằng hai cách (tính thẳng và đệ quy từ cuối).

Đáp án (tự tính): Thẳng: $2+0+0{,}25\cdot 4+0{,}125\cdot 1 = 3{,}125$. Đệ quy: $G = 1$ (bước cuối), rồi $4+0{,}5\cdot 1 = 4{,}5$, rồi $0 + 0{,}5 \cdot 4{,}5 = 2{,}25$, rồi $2 + 0{,}5 \cdot 2{,}25 = 3{,}125$.

Bài 2. Một bước Bellman kỳ vọng

Trong MDP của chương, cho B chọn "phải" và "trái" với xác suất $0{,}5$ mỗi cái, $\gamma = 0{,}9$, và giả sử các giá trị hiện tại là $V(A) = 8$, $V(B) = 9$. Tính giá trị mới của $V(B)$ sau một lần cập nhật Bellman.

Đáp án (tự tính): $Q(B, \text{phải}) = 0{,}9 \cdot 10 + 0{,}1 \cdot 0{,}9 \cdot 9 = 9{,}81$, $Q(B, \text{trái}) = 0{,}9 \cdot 8 = 7{,}2$. Trung bình theo chính sách: $0{,}5 \cdot 9{,}81 + 0{,}5 \cdot 7{,}2 = 8{,}505$.

Bài 3. Một bước Bellman tối ưu

Cùng số liệu bài 2. Thay trung bình bằng max thì giá trị mới của $V(B)$ là bao nhiêu và hành động tham lam là gì?

Đáp án (tự tính): $\max(9{,}81;\ 7{,}2) = 9{,}81$, hành động tham lam là "phải".

Bài 4. Ảnh hưởng của gamma

Hành lang có đích cách $L = 4$ bước (ô đầu cách bốn bước), thưởng 10 khi vào đích, mọi bước khác thưởng 0. Tính giá trị ô đầu với $\gamma = 0{,}5$, $0{,}9$, $0{,}99$ theo chính sách luôn đi tới.

Đáp án (tự tính): $V = 10\gamma^{L-1} = 10\gamma^3$ cho $\gamma = 0{,}5$: $1{,}25$; $\gamma = 0{,}9$: $7{,}29$; $\gamma = 0{,}99$: $9{,}703$. Lưu ý: ở hình hành lang sáu ô, ô đầu cách đích năm bước nên giá trị là $6{,}561$ chứ không phải $7{,}29$.

Bài 5. Giải V bằng đại số với gamma khác

MDP của chương, chính sách luôn "phải", nhưng $\gamma = 0{,}5$. Tính $V(B)$ và $V(A)$.

Đáp án (tự tính): $V(B) = 9/(1 - 0{,}1 \cdot 0{,}5) = 9/0{,}95 \approx 9{,}474$. $V(A) = 0{,}8 \cdot 0{,}5 \cdot V(B) + 0{,}2 \cdot 0{,}5 \cdot V(A)$ nên $V(A) = 0{,}4 \cdot 9{,}474/0{,}9 \approx 4{,}211$. Chiết khấu mạnh làm $V(A)$ giảm hẳn (từ $8{,}684$ xuống $4{,}211$) còn $V(B)$ gần như không đổi, vì B chỉ cách đích một bước.

Tài liệu tham khảo nền: Sutton và Barto, Reinforcement Learning: An Introduction (các chương về MDP, hàm giá trị và quy hoạch động); Bellman, Dynamic Programming; preface của cuốn này (mục toán cốt lõi); các ghi chú rl 01, 02 và 05 trong sách.

Nền tảng 02

TD, Q-learning và DQN: học giá trị từ từng bước bằng cách tự dựa vào ước lượng của chính mình

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nen01 của cuốn này (MDP, return, V và Q, Bellman); SGD ở optimization nền 03; mạng nơ-ron nhiều lớp ở ML cơ bản nền tảng 05

Dùng ở: rl 01 (DQN, replay), rl 02 (Double DQN), rl 03 (Rainbow); phần critic ở rl 06

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy; các hình đường cong là **mô phỏng** với dữ liệu tự sinh và seed cố định, không phải số đo từ paper.

1. Vì sao cần khái niệm này

TD (temporal difference, sai lệch thời gian) là cách cập nhật giá trị ngay sau mỗi bước, bằng cách so sánh dự đoán cũ với "thưởng vừa nhận cộng dự đoán cho bước kế". Nó giải quyết câu hỏi mà nen01 để ngỏ: khi **không biết** xác suất chuyển P và hàm thưởng, làm sao tính V hoặc Q? Ta chỉ có mẫu trải nghiệm. Q-learning là TD áp dụng cho phương trình Bellman tối ưu, và DQN là Q-learning với mạng nơ-ron làm hàm Q. Mỗi mảnh ghép này nảy sinh một vấn đề thực hành: dữ liệu liên tiếp tương quan (cần replay buffer), mục tiêu di chuyển (cần target network), và phép max làm ước lượng lạc quan quá mức (cần Double DQN). Chương đi từng bước, lần lượt xử lý ba vấn đề đó.

2. Nhắc nền: trung bình chạy và bước cập nhật

Muốn ước lượng trung bình của các số $x_1, x_2, \ldots$ mà không lưu cả dãy, ta cập nhật dần:

$$m \leftarrow m + \alpha\,(x - m),$$

với $\alpha \in (0, 1]$ là bước học. Số $x - m$ là "sai số so với ước lượng hiện tại". Ví dụ (tự tính): $m = 5$, nhận $x = 8$, $\alpha = 0{,}5$ thì $m \leftarrow 5 + 0{,}5 \cdot 3 = 6{,}5$. Nếu $\alpha = 1/n$ ở mẫu thứ n, m đúng bằng trung bình cộng; nếu α cố định, mẫu gần đây có trọng số lớn hơn (giống trung bình trượt ở optimization nền 05). Mọi cập nhật TD trong chương đều có dạng này: **ước lượng mới = ước lượng cũ + bước học nhân sai số**, chỉ khác ở chỗ "mẫu x" là gì.

3. Trực giác bằng hình: ba cách ước lượng "từ đây về sau"

Muốn biết $V(s)$ ta cần return G_t, nhưng return đòi chờ hết episode. Có ba cách:

- **Monte Carlo (MC):** chờ hết episode, dùng G_t thật làm mẫu. Không thiên lệch, nhưng phương sai lớn (cả chuỗi sự kiện ngẫu nhiên cộng dồn) và phải chờ hết episode.
- **TD(0):** chỉ chờ **một bước**: lấy $r_{t+1} + \gamma V(s_{t+1})$ làm mẫu, tức dùng chính ước lượng V của trạng thái kế để "đoán nốt" phần còn lại. Cách làm này gọi là **bootstrapping** (tự kéo mình lên bằng dây giày: ước lượng dựa trên ước lượng).
- **Nhiều bước:** trộn hai thái cực, dùng n thưởng thật rồi bootstrapping (nền của GAE ở rl 05).

Trực giác: MC như đợi tận cuối chuyến đi mới chấm điểm cả hành trình; TD như vừa đi vừa chấm "theo điểm đang có ở bước sau". TD nhiễu ít hơn nhưng có thiên lệch (nếu $V(s_{t+1})$ đang sai thì mục tiêu sai theo). Ở nen03 ta sẽ thấy lại cùng tinh thần "đổi nhiễu lấy thiên lệch" khi dùng baseline.

4. Định nghĩa chính thức

4.1 TD(0) và sai lệch TD

Với một chuyển (s, r, s') trong khi theo chính sách π:

$$\delta = r + \gamma V(s') - V(s), \qquad V(s) \leftarrow V(s) + \alpha\,\delta.$$

δ là **sai lệch TD** (TD error): chênh giữa "mục tiêu TD" $r + \gamma V(s')$ và dự đoán hiện tại $V(s)$. $\delta > 0$ nghĩa là kết quả tốt hơn dự đoán. Nếu V đúng là V^π thì kỳ vọng của δ bằng 0 (nhờ phương trình Bellman), nên TD là phép "kéo V về nghiệm của Bellman bằng mẫu".

Ví dụ (tự tính): giả sử hiện tại $V(A) = 5$ và $V(B) = 9$, $\gamma = 0{,}9$, $\alpha = 0{,}1$. Ta quan sát chuyển A sang B với thưởng 0. Khi đó $\delta = 0 + 0{,}9 \cdot 9 - 5 = 3{,}1$ và $V(A) \leftarrow 5 + 0{,}1 \cdot 3{,}1 = 5{,}31$. Giá trị của A tăng vì B hoá ra "đáng giá" hơn A tưởng.

4.2 Từ V sang Q: SARSA và Q-learning

Để chọn hành động cần Q. Với chuyển (s, a, r, s'), mục tiêu TD cho Q có hai biến thể phụ thuộc cách "đoán nốt" từ s':

$$\text{SARSA:}\quad Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s', a') - Q(s,a)\big],$$

trong đó a' là hành động **thật sự** chọn tiếp theo chính sách đang chạy (on-policy). Còn

$$\text{Q-learning:}\quad Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s,a)\Big],$$

dùng max theo phương trình Bellman tối ưu của nen01, bất kể hành động thật tiếp theo là gì. Vì vậy Q-learning là **off-policy**: ta có thể thu dữ liệu bằng một chính sách thăm dò (ví dụ ϵ-greedy: với xác suất ϵ chọn bừa, còn lại chọn tham lam) mà vẫn học Q^*. Trong thế giới bảng và nếu mọi cặp (trạng thái, hành động) được thăm vô hạn lần với α giảm hợp lý, Q-learning hội tụ về Q^* (kết quả kinh điển; chương không chứng minh).

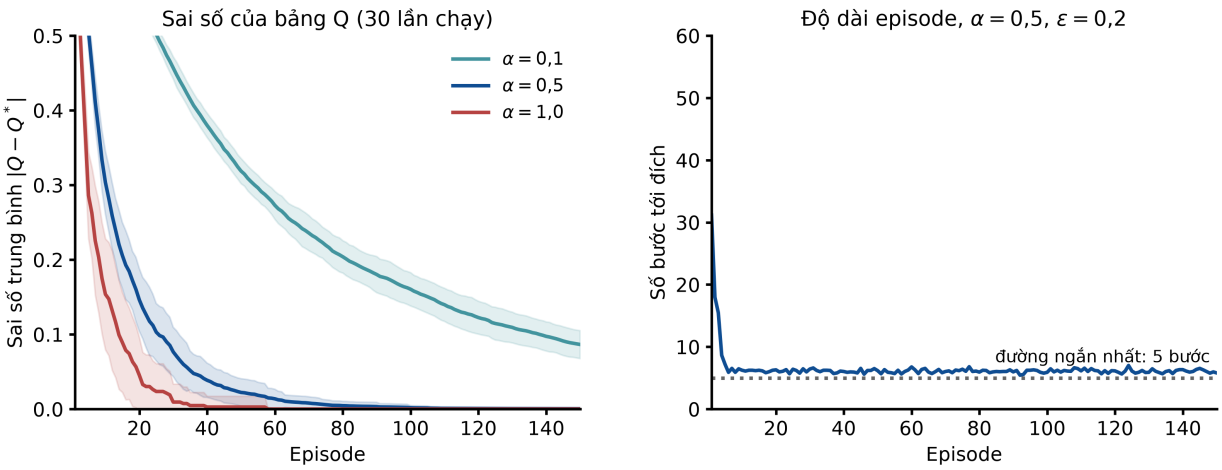
5. Ví dụ tay: vài bước Q-learning

Dùng lại MDP của nen01 (chỉ lấy hành động "phải"), $\gamma = 0{,}9$, $\alpha = 0{,}5$, bảng Q khởi đầu bằng 0. Giả sử môi trường **tình cờ** cho các chuyển sau (ta chọn dãy để dễ tính; ở nen01 xác suất đi sang đích là $0{,}9$ nên đây là kết quả khả dĩ). Mọi số dưới đây (tự tính):

1. $(B, \text{phải}, r = 10, \text{vào G})$: trạng thái kết thúc nên mục tiêu chỉ là 10. $\delta = 10 - 0 = 10$; $Q(B, \text{phải}) = 0 + 0{,}5 \cdot 10 = 5$.
2. $(A, \text{phải}, r = 0, \to B)$: $\max_{a'} Q(B, a') = 5$. Mục tiêu $0 + 0{,}9 \cdot 5 = 4{,}5$; $\delta = 4{,}5$; $Q(A, \text{phải}) = 0 + 0{,}5 \cdot 4{,}5 = 2{,}25$.
3. Lại $(B, \text{phải}, 10, \text{vào G})$: $\delta = 10 - 5 = 5$; $Q(B, \text{phải}) = 5 + 2{,}5 = 7{,}5$.
4. Lại $(A, \text{phải}, 0, \to B)$: mục tiêu $0{,}9 \cdot 7{,}5 = 6{,}75$; $\delta = 6{,}75 - 2{,}25 = 4{,}5$; $Q(A, \text{phải}) = 2{,}25 + 2{,}25 = 4{,}5$.

Quan sát: giá trị lan **ngược** từ đích, giống lặp Bellman ở nen01, nhưng bằng từng mẫu. $Q(A)$ chỉ biết đích có thưởng **sau khi** $Q(B)$ đã biết; mỗi lần ta cập nhật A bằng ước lượng B tại thời điểm đó, và ước lượng B đang tăng. Tiếp tục nhiều lần, $Q(B, \text{phải}) \to 9{,}890$ và $Q(A, \text{phải}) \to 8{,}684$, đúng nghiệm của nen01 (khi dữ liệu phản ánh đúng xác suất của môi trường).

Cách đọc hình: đây là **mô phỏng** Q-learning bảng trên hành lang sáu ô (năm ô thường, vào đích nhận 1, $\gamma = 0{,}9$, $\epsilon = 0{,}2$, 30 lần chạy với seed khác nhau, dải mờ là một độ lệch chuẩn). Bên trái là sai số trung bình



Hình 4: Đường học của Q-learning bảng trên hành lang sáu ô: sai số và độ dài episode (minh hoạ, tự tính)

giữa bảng Q đang học và Q^* (tính bằng value iteration): ba đường ứng với ba α. α lớn hội tụ nhanh hơn trong môi trường **tất định** này; trong môi trường nhiễu, α lớn sẽ để lại dao động, nên không suy ra "$\alpha = 1$ luôn tốt". Bên phải là số bước tới đích: sau vài episode đầu agent chỉ cần khoảng sáu bước (đường ngắn nhất là năm bước; phần dư do $\epsilon = 0{,}2$ thỉnh thoảng đi bậy).

6. Từ bảng sang mạng: DQN và hai vấn đề mới

Khi trạng thái là ảnh, không lập được bảng. DQN thay bảng bằng mạng $Q_\theta(s, a)$ và biến Q-learning thành bài toán hồi quy: với mỗi chuyển (s, a, r, s'), tạo **mục tiêu** $y = r + \gamma \max_{a'} Q_\theta(s', a')$ rồi giảm $\big(y - Q_\theta(s, a)\big)^2$ bằng một bước SGD. Nghe như học có giám sát, nhưng có hai khác biệt gây rắc rối.

6.1 Mẫu liên tiếp tương quan: vì sao cần replay buffer

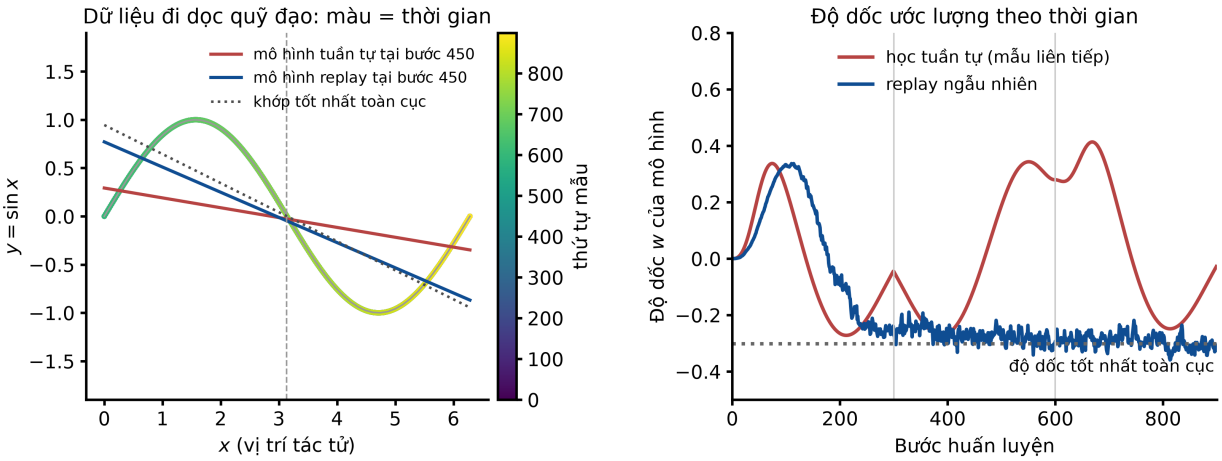
Học có giám sát thông thường giả định các mẫu độc lập và cùng phân phối (i.i.d.), nhờ đó mini-batch ngẫu nhiên cho gradient gần không thiên lệch. Trong RL, các chuyển liên tiếp trên một quỹ đạo gần như giống hệt nhau (agent đi dọc một hành lang), và phân phối dữ liệu còn đổi khi chính sách đổi. Cập nhật theo mẫu mới nhất khiến mạng "quên" vùng cũ và đuổi theo vùng đang đi qua.

Replay buffer là bộ đệm giữ N chuyển gần nhất; mỗi bước ta thêm chuyển mới và **lấy ngẫu nhiên** một mini-batch từ đệm để cập nhật. Việc lấy ngẫu nhiên phá tương quan thời gian và dùng lại mỗi mẫu nhiều lần. Nó chỉ hợp lệ vì Q-learning là off-policy: dữ liệu cũ do chính sách cũ sinh ra vẫn dùng được cho mục tiêu Bellman tối ưu.

Cách đọc hình: đây là mô phỏng một mô hình tuyến tính $y \approx wx + b$ khớp dữ liệu $y = \sin x$ khi "tác tử" quét x từ 0 đến 2π, quay lại, rồi quét tiếp (900 bước), cả hai dùng $\alpha = 0{,}03$. Bên phải, đường đỏ cập nhật theo mẫu mới nhất: độ dốc w lắc theo đoạn của đường cong đang đi qua. Đường xanh lấy 8 mẫu ngẫu nhiên từ mọi mẫu đã gặp mỗi bước: độ dốc tiến về độ dốc tốt nhất toàn cục khoảng $-0{,}30$ rồi chỉ dao động nhỏ. Sau bước 300, độ lệch chuẩn của w là khoảng $0{,}24$ (tuần tự) so với $0{,}03$ (replay), tức chênh gần chục lần (số mô phỏng của riêng thiết lập này, không phải số đo trên Atari). Bên trái cho thấy ở bước 450 mô hình tuần tự (đỏ) khác xa đường khớp toàn cục (chấm), trong khi mô hình replay (xanh) gần hơn. Đây là một mô hình rất đơn giản; mạng sâu thì hiện tượng quên còn mạnh hơn.

6.2 Mục tiêu di chuyển: vì sao cần target network

Trong hồi quy bình thường, nhãn y cố định. Ở DQN, $y = r + \gamma \max Q_\theta(s', \cdot)$ phụ thuộc chính θ đang được cập nhật. Mỗi lần θ đổi, mọi mục tiêu cũng dịch, nhất là khi s và s' giống nhau (tổng quát hoá của mạng làm việc đẩy $Q(s, \cdot)$ lên cũng đẩy $Q(s', \cdot)$ lên). Có thể sinh vòng phản hồi bất ổn.



Hình 5: Học tuần tự trên mẫu liên tiếp so với replay ngẫu nhiên khi khớp một đường thẳng (minh hoạ, tự tính)

Ví dụ minh hoạ thuần số (tự tính, mô hình đồ chơi, không phải DQN thật): hai trạng thái A, B dùng chung một tham số, $Q(A) = w$ và $Q(B) = 1{,}2w$, thưởng 0, $\gamma = 0{,}9$, ta luôn cập nhật A với mục tiêu $0{,}9 \cdot Q(B) = 1{,}08w$. Khi đó mục tiêu luôn lớn hơn $Q(A)$ đúng 8%; với $\alpha = 0{,}5$ mỗi bước nhân w lên $1{,}04$, và sau 10 bước w tăng khoảng $1{,}48$ lần, tiếp tục phát tán. Bảng không gặp chuyện này vì mỗi ô có tham số riêng; xấp xỉ hàm gộp chung tham số, nên "tổng quát hoá" cũng lan sai số.

Target network là một bản sao θ^- của mạng, **đóng băng** trong nhiều bước (ví dụ $10\,000$ bước theo ghi chú rl 02) rồi mới chép lại từ θ. Mục tiêu dùng θ^-: $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$. Trong khoảng đóng băng, y đứng yên như nhãn của bài hồi quy, rồi dịch theo từng đợt. Nó làm vòng phản hồi chậm và dễ kiểm soát hơn, nhưng không phải bảo đảm toán học chống phân kỳ: kết hợp xấp xỉ hàm, bootstrapping và off-policy vẫn có thể bất ổn ("bộ ba chết người" trong sách của Sutton và Barto).

6.3 Công thức DQN gom lại

Mỗi bước: chọn a theo ϵ-greedy từ Q_θ; thu (s, a, r, s') và đưa vào đệm; lấy mini-batch ngẫu nhiên; tính mục tiêu $y = r$ nếu s' kết thúc, ngược lại $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$; giảm $\big(y - Q_\theta(s, a)\big)^2$ (nhiều bản dùng hàm mất mát Huber để cắt gradient lớn); định kỳ chép $\theta^- \leftarrow \theta$.

7. Thiên lệch lạc quan của max và Double DQN

7.1 Vì sao max gây lạc quan

Phép $\max_{a'} Q(s', a')$ lấy **giá trị lớn nhất** trong các ước lượng. Ước lượng nào cũng có nhiễu; lấy cực đại thì ta hay nhặt đúng ước lượng đang nhiễu **dương**. Kết quả: $\mathbb{E}[\max_a \hat{Q}_a] \geq \max_a \mathbb{E}[\hat{Q}_a]$ (bất đẳng thức Jensen cho hàm lồi max).

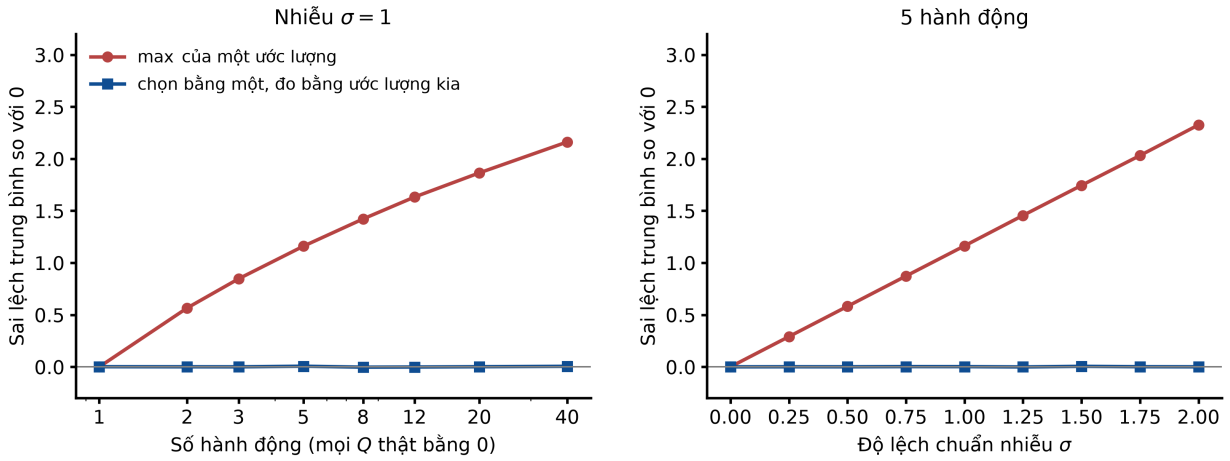
Ví dụ tay. Giả sử ở s' có ba hành động, **cả ba có** Q **thật bằng** 0, nhưng ước lượng của ta là $\hat{Q} = (0{,}8,\ -0{,}2,\ 0{,}1)$ (nhiễu). Cái max cho $0{,}8$, tức lạc quan $0{,}8$ dù sự thật là 0 (tự tính). Với nhiễu chuẩn độc lập $\mathcal{N}(0, 1)$, kỳ vọng của max là khoảng $0{,}56$ với 2 hành động, $0{,}85$ với 3, $1{,}16$ với 5 (đã kiểm bằng mô phỏng; giá trị lý thuyết cho $n = 2$ là $1/\sqrt{\pi} \approx 0{,}564$ và cho $n = 3$ là $3/(2\sqrt{\pi}) \approx 0{,}846$). Số hành động càng nhiều, nhiễu càng lớn, lạc quan càng nhiều. Vì mục tiêu của bước này nằm trong mục tiêu của bước sau (bootstrapping), lạc quan còn **tích luỹ** qua các cập nhật.

7.2 Double: tách chọn và đánh giá

Ý tưởng của Double Q-learning: dùng **hai** ước lượng độc lập; một cái **chọn** hành động tốt nhất, cái kia **đánh giá** giá trị của hành động đó:

$$y_{\text{Double}} = r + \gamma\, Q_B\Big(s',\ \arg\max_{a'} Q_A(s', a')\Big).$$

Ví dụ tay (tự tính): vẫn ba hành động có Q thật bằng 0. Ước lượng thứ nhất $Q_A = (0{,}8, -0{,}2, 0{,}1)$, ước lượng thứ hai độc lập $Q_B = (-0{,}3, 0{,}4, 0{,}0)$. Q_A chọn hành động 1 (vì $0{,}8$ lớn nhất); Q_B đánh giá hành động đó bằng $-0{,}3$. Mục tiêu $-0{,}3$ thay cho $0{,}8$. Lần này hơi lạc quan theo hướng ngược lại, nhưng **kỳ vọng** thì bằng 0 vì nhiễu của Q_B không liên quan đến việc Q_A chọn cái nào.



Hình 6: Thiên lệch lạc quan của max và ước lượng tách chọn và đánh giá theo số hành động và độ nhiễu (minh hoạ, tự tính)

Cách đọc hình: mô phỏng $100\,000$ lần (seed cố định), mọi Q thật bằng 0 nên mọi giá trị khác 0 là sai lệch thuần tuý. Đường đỏ là kỳ vọng của max trên một bộ ước lượng nhiễu; tăng theo số hành động (trái) và gần như tuyến tính theo độ nhiễu σ (phải). Đường xanh là ước lượng kiểu Double (chọn bằng một bộ, đo bằng bộ độc lập): quanh 0 ở mọi điểm. Đây là thiết lập lý tưởng với nhiễu độc lập, chuẩn, bằng nhau; trong DQN thật hai mạng không hoàn toàn độc lập nên Double chỉ giảm, không xoá hết thiên lệch.

Double DQN dùng đúng mạng đang học θ để chọn hành động và target network θ^- (đã có sẵn) để đánh giá:

$$y = r + \gamma\, Q_{\theta^-}\Big(s',\ \arg\max_{a'} Q_\theta(s', a')\Big).$$

Chỉ đổi một dòng so với DQN, không thêm mạng, không thêm tham số. Theo ghi chú rl 02, bài đo được thiên lệch lạc quan trên Atari và thấy cách sửa này giúp ước lượng giá trị sát giá trị thật hơn và nâng điểm trung vị so với DQN; số cụ thể xem trong ghi chú đó.

8. Biến thể và chi tiết hay bị bỏ qua

- **Trạng thái kết thúc:** khi s' là kết thúc, mục tiêu chỉ là r (không cộng γ max). Quên điều này làm giá trị gần đích bị thổi phồng hoặc sai hoàn toàn.
- **Hết giờ không phải kết thúc thật:** episode bị cắt do giới hạn thời gian vẫn nên bootstrapping (không đặt $V = 0$), vì thế giới chưa kết thúc.
- **Hàm mất mát và chuẩn hoá thưởng:** thưởng có độ lớn rất khác nhau giữa các bài làm gradient khó điều khiển; cắt thưởng, chuẩn hoá hoặc dùng hàm Huber là các lựa chọn thực hành (ghi chú rl 01 có nhắc việc cắt thưởng; chi tiết xem ở đó).
- **Mẫu trong đệm cũ dần:** đệm lớn trộn nhiều chính sách cũ; đệm nhỏ lại tương quan. Cách lấy mẫu ưu tiên (prioritized replay) là một thành phần của Rainbow.

- **Chỉ dùng cho hành động rời rạc:** $\max_a$ trên mạng chỉ dễ khi tập hành động nhỏ. Với hành động liên tục cần actor (DDPG, TD3, SAC trong cuốn).
- **Suy luận của chương (không phải của paper):** khẳng định "replay phá tương quan nên ổn định hơn" ở đây dựa vào mô phỏng đồ chơi ở mục 6.1; hiệu ứng thực trên Atari cần đọc ablation của các paper.

9. Nối sang các bài paper của cuốn

- **rl 01 (DQN)** là mục 6 ở dạng đầy đủ: CNN trên pixel, replay buffer, Q-learning với mục tiêu Bellman tối ưu. Theo ghi chú rl 01, thuật toán trong bài 2013 dùng đúng θ hiện tại để tính mục tiêu và chưa có target network chép định kỳ; target network là phần của bản DQN sau này mà các bài Double DQN và Rainbow coi là chuẩn.
- **rl 02 (Double DQN)** là mục 7: đổi một dòng trong mục tiêu để tách chọn và đánh giá, dùng sẵn θ^-. Hãy so mục tiêu DQN và Double DQN trong bảng tóm tắt ngay dưới đây.
- **rl 03 (Rainbow)** gom Double DQN, prioritized replay và các cải tiến khác; muốn đọc nó cần hiểu từng mảnh ở chương này.
- **rl 06 (A3C)** dùng TD để học critic (V), và dùng nhiều tiến trình song song thay cho replay buffer để phá tương quan (theo ý chung của bài; chi tiết xem ghi chú).
- **rl 05 (GAE)** là TD nhiều bước với trọng số hình học; $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ của mục 4.1 chính là sai lệch TD mà GAE cộng dồn.

10. Sai lầm hay gặp

- **Cho rằng Q-learning cần dữ liệu từ chính sách tối ưu.** Không: nó off-policy, học Q^* từ dữ liệu của chính sách bất kỳ (miễn thăm đủ), đó là lý do replay dùng được.
- **Nhầm TD error với sai số hồi quy thật.** δ so với một **mục tiêu tự ước lượng**, không phải nhãn đúng; mục tiêu cũng sai và đổi theo thời gian.
- **Quên rằng mục tiêu bị cắt ở trạng thái kết thúc.**
- **Tưởng target network chữa mọi bất ổn.** Nó chỉ làm mục tiêu đứng yên tạm thời; xấp xỉ hàm cộng bootstrapping vẫn có thể phát tán.
- **Tưởng Double DQN xoá hẳn thiên lệch.** Hai mạng không độc lập; thiên lệch giảm chứ không biến mất.
- **Chọn α hoặc ϵ từ một hình mô phỏng.** Hình ở chương là hành lang tất định; môi trường nhiễu cần α nhỏ hơn.

11. Bảng tóm tắt

Thuật toán	Cập nhật	Ghi chú
TD(0) cho V	$V(s) \mathrel{+}= \alpha[r + \gamma V(s') - V(s)]$	Bootstrapping, nhiễu thấp hơn MC
SARSA	$Q(s,a) \mathrel{+}= \alpha[r + \gamma Q(s',a') - Q(s,a)]$	On-policy, a' là hành động thật
Q-learning	mục tiêu $r + \gamma \max_{a'} Q(s',a')$	Off-policy, học Q^*
DQN	mạng Q_θ, replay, target θ^-	Hồi quy trên mục tiêu Bellman
Double DQN	$y = r + \gamma Q_{\theta^-}(s', \arg\max Q_\theta)$	Tách chọn và đánh giá

12. Bài tập tự tính

Các bài là phép tính độc lập, không phải số liệu paper. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Một bước TD(0)

$V(A) = 2$, $V(B) = 6$, $\gamma = 0{,}5$, $\alpha = 0{,}2$. Quan sát chuyển A sang B với thưởng 1. Tính δ và $V(A)$ mới.

Đáp án (tự tính): $\delta = 1 + 0{,}5 \cdot 6 - 2 = 2$; $V(A) = 2 + 0{,}2 \cdot 2 = 2{,}4$.

Bài 2. Bước Q-learning

$Q(B, \cdot) = (4,\ 7)$ cho hai hành động, $Q(A, \text{phải}) = 3$, $\gamma = 0{,}9$, $\alpha = 0{,}5$. Quan sát $(A, \text{phải}, r = 1, \to B)$. Tính $Q(A, \text{phải})$ mới và so với SARSA nếu hành động tiếp theo thật sự là hành động đầu ($Q = 4$).

Đáp án (tự tính): Q-learning: mục tiêu $1 + 0{,}9 \cdot 7 = 7{,}3$, $\delta = 4{,}3$, $Q = 3 + 2{,}15 = 5{,}15$. SARSA: mục tiêu $1 + 0{,}9 \cdot 4 = 4{,}6$, $\delta = 1{,}6$, $Q = 3 + 0{,}8 = 3{,}8$.

Bài 3. Lạc quan của max

Ba hành động có Q thật $(1, 1, 1)$, ước lượng $(1{,}6,\ 0{,}7,\ 1{,}1)$. Tính lạc quan của max. Rồi cho ước lượng độc lập thứ hai $(0{,}9,\ 1{,}3,\ 0{,}8)$ và tính mục tiêu kiểu Double (chọn bằng ước lượng đầu, đo bằng ước lượng hai).

Đáp án (tự tính): max $= 1{,}6$, lạc quan $0{,}6$. Ước lượng đầu chọn hành động 1 (giá trị $1{,}6$), ước lượng hai đo hành động đó là $0{,}9$, sai lệch $-0{,}1$.

Bài 4. Hết episode

Chuyển $(s, a, r = 2, s')$ với s' là trạng thái kết thúc, $\gamma = 0{,}9$, $Q(s, a) = 1$, $\alpha = 0{,}25$. Tính $Q(s, a)$ mới. Nếu bạn quên bỏ γ max và $\max Q(s', \cdot) = 10$ thì đáp số sai thành bao nhiêu?

Đáp án (tự tính): Đúng: mục tiêu 2, $\delta = 1$, $Q = 1{,}25$. Sai (cộng $0{,}9 \cdot 10 = 9$): mục tiêu 11, $\delta = 10$, $Q = 3{,}5$.

Bài 5. Phát tán với tham số chung

Mô hình đồ chơi ở mục 6.2: $Q(A) = w$, $Q(B) = 1{,}2w$, thưởng 0, $\gamma = 0{,}9$, $\alpha = 0{,}5$, cập nhật A với mục tiêu $0{,}9\, Q(B)$, giả sử gradient theo w chỉ qua $Q(A)$ (mục tiêu coi như hằng). Cho $w_0 = 1$, tính w_1, w_2. Với hệ số $1{,}2$ thay bằng $1{,}1$ thì sao?

Đáp án (tự tính): $w_1 = 1 + 0{,}5 \cdot (1{,}08 - 1) = 1{,}04$, $w_2 = 1{,}0816$. Với $1{,}1$ mục tiêu là $0{,}99w < w$ nên w giảm dần về 0 (hội tụ, không phát tán): $w_1 = 0{,}995$.

Tài liệu tham khảo nền: Sutton và Barto, Reinforcement Learning: An Introduction (TD, Q-learning, bộ ba chết người); Watkins, Q-learning; Mnih và cộng sự, Playing Atari with Deep Reinforcement Learning; van Hasselt, Guez và Silver, Deep Reinforcement Learning with Double Q-learning; các ghi chú rl 01, 02, 03 và 06 trong sách.

Nền tảng 03

Policy gradient và baseline: tối ưu trực tiếp chính sách, trừ một mốc để bớt nhiễu mà không đổi hướng đúng

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nen01 của cuốn này (MDP, return, V, advantage); gradient và đạo hàm riêng ở math nền tảng 06; kỳ vọng và phương sai ở math nền tảng 08; SGD và nhiễu gradient ở optimization nền 03; softmax ở ML cơ bản nền tảng 02

Dùng ở: rl 04 (TRPO), rl 05 (GAE), rl 06 (A3C), rl 07 (PPO), và các bài RL cho LLM như rl 36 (GRPO), rl 37 (RLOO)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy; các hình về phân bố và đường học là **mô phỏng** với dữ liệu tự sinh và seed cố định, không phải số đo từ paper.

1. Vì sao cần khái niệm này

Policy gradient (gradient chính sách) là cách tối ưu trực tiếp chính sách π_θ bằng gradient ascent trên phần thưởng kỳ vọng. Ở nen02 ta học giá trị Q rồi chọn hành động có Q lớn nhất. Cách đó khó dùng khi hành động liên tục hoặc khổng lồ (ví dụ mỗi hành động là một token trong từ vựng hàng chục nghìn, và cả câu trả lời là một chuỗi hành động). Thay vì học giá trị rồi suy ra chính sách, ta cho chính sách là một mạng có tham số θ cho ra **xác suất** hành động, rồi chỉnh θ để xác suất của những hành động dẫn tới thưởng cao tăng lên. Đây là xương sống của TRPO, PPO và các phương pháp RL cho mô hình ngôn ngữ.

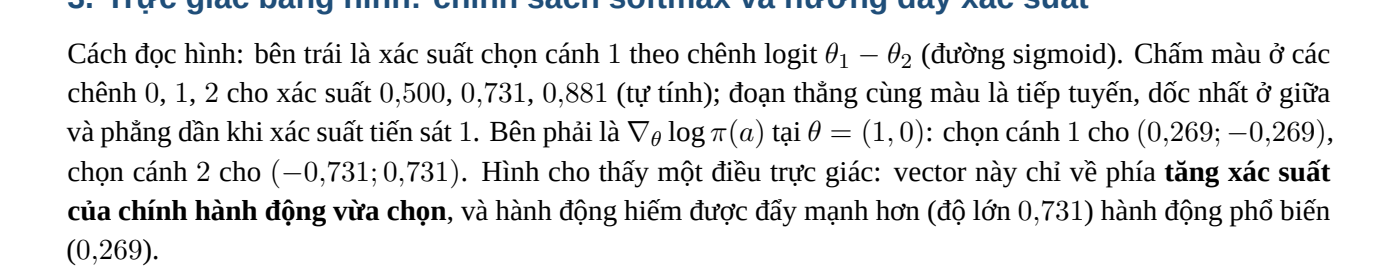
Có hai điều cần giải quyết, và chúng cũng là hai nửa của chương. Một: thưởng là một con số do môi trường trả về, ta không biết đạo hàm của nó theo θ, làm sao tính gradient? Câu trả lời là mẹo log-derivative (mục 4). Hai: ước lượng gradient bằng mẫu rất nhiễu, làm sao giảm nhiễu mà không làm sai hướng? Câu trả lời là baseline (mục 6).

2. Nhắc nền: gradient ascent trên một kỳ vọng

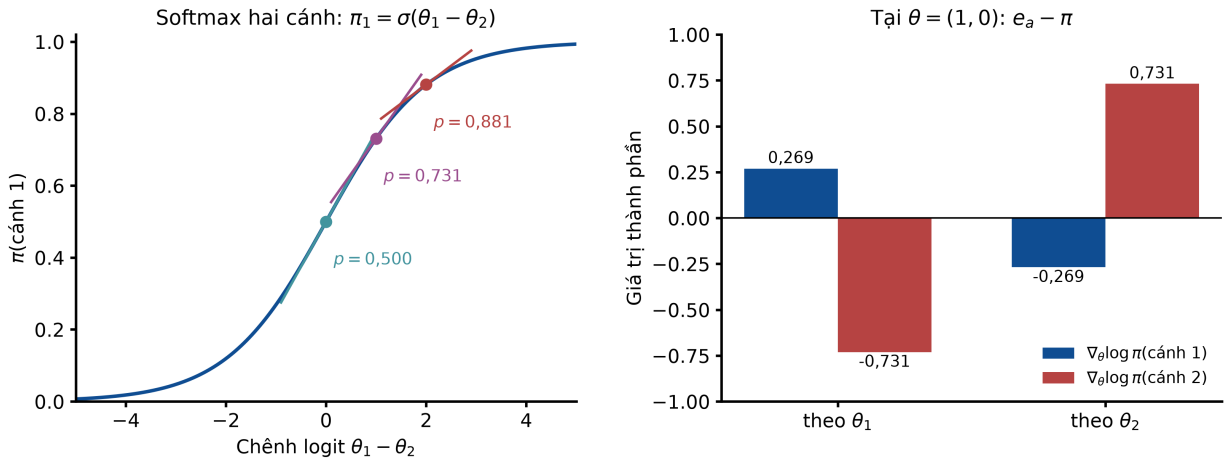
Mục tiêu RL là $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$, trong đó τ là một quỹ đạo và $R(\tau)$ là return của nó. Ta muốn tăng J, nên cập nhật $\theta \leftarrow \theta + \alpha\nabla_\theta J$ (gradient **ascent**, ngược với gradient descent ở optimization nền 02). Khó khăn: J là kỳ vọng theo phân phối π_θ phụ thuộc θ; ta không thể "lấy mẫu rồi đạo hàm" vì mẫu là rời rạc, không khả vi theo θ. Cần biến $\nabla_\theta J$ thành **kỳ vọng của một thứ ta tính được từ mẫu**.

Phần nhắc nền cuối: softmax. Với hai hành động và logit $\theta = (\theta_1, \theta_2)$, $\pi_k = e^{\theta_k}/(e^{\theta_1} + e^{\theta_2})$. Ví dụ (tự tính): $\theta = (0, 0)$ cho $\pi = (0{,}5; 0{,}5)$; $\theta = (1, 0)$ cho $\pi = (0{,}731; 0{,}269)$. Vì chỉ có chênh lệch logit quan trọng, $\pi_1 = \sigma(\theta_1 - \theta_2)$ với σ là hàm sigmoid.

3. Trực giác bằng hình: chính sách softmax và hướng đẩy xác suất



Cách đọc hình: bên trái là xác suất chọn cánh 1 theo chênh logit $\theta_1 - \theta_2$ (đường sigmoid). Chấm màu ở các chênh 0, 1, 2 cho xác suất $0{,}500$, $0{,}731$, $0{,}881$ (tự tính); đoạn thẳng cùng màu là tiếp tuyến, dốc nhất ở giữa và phẳng dần khi xác suất tiến sát 1. Bên phải là $\nabla_\theta \log \pi(a)$ tại $\theta = (1, 0)$: chọn cánh 1 cho $(0{,}269; -0{,}269)$, chọn cánh 2 cho $(-0{,}731; 0{,}731)$. Hình cho thấy một điều trực giác: vector này chỉ về phía **tăng xác suất của chính hành động vừa chọn**, và hành động hiếm được đẩy mạnh hơn (độ lớn $0{,}731$) hành động phổ biến ($0{,}269$).



Hình 7: Chính sách softmax hai cánh và vector grad log pi tại theta bằng (1, 0) (minh hoạ, tự tính)

Ý tưởng của policy gradient gói trong một câu: lấy hướng "làm hành động này dễ xảy ra hơn", rồi **nhân với mức độ đáng khen** của hành động (thưởng hay return). Hành động nhận thưởng cao được đẩy lên, hành động nhận thưởng thấp bị kéo xuống. Phần còn lại của chương là làm câu này chính xác.

4. Định nghĩa chính thức

4.1 Mẹo log-derivative

Cho phân phối $p_\theta(x)$ và hàm $f(x)$. Ta muốn $\nabla_\theta \, \mathbb{E}_{x \sim p_\theta}[f(x)]$. Dùng đồng nhất thức $\nabla_\theta p_\theta(x) = p_\theta(x) \, \nabla_\theta \log p_\theta(x)$ (đạo hàm của log là đạo hàm chia cho chính nó):

$$\nabla_\theta \, \mathbb{E}_{p_\theta}[f] = \sum_x f(x) \, \nabla_\theta p_\theta(x) = \sum_x p_\theta(x) \, f(x) \, \nabla_\theta \log p_\theta(x) = \mathbb{E}_{p_\theta}\big[f(x) \, \nabla_\theta \log p_\theta(x)\big].$$

Đây là điểm mấu chốt: gradient của một kỳ vọng thành **kỳ vọng của** $f \cdot \nabla \log p$, mà ta tính được từ mẫu, và f **không cần khả vi** (thường có thể là hộp đen). Lấy một mẫu x rồi tính $f(x) \nabla \log p_\theta(x)$ là một ước lượng không chệch của gradient.

4.2 REINFORCE

Với quỹ đạo $\tau = (s_0, a_0, \ldots, s_{T-1}, a_{T-1})$ và $R(\tau)$ là return, xác suất $p_\theta(\tau)$ gồm các nhân tử chuyển của môi trường (không phụ thuộc θ) nhân với $\prod_t \pi_\theta(a_t \mid s_t)$. Khi lấy log rồi đạo hàm theo θ, các nhân tử môi trường biến mất:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \; R(\tau)\Big].$$

Điều này đáng ngạc nhiên: ta không cần biết $P(s' \mid s, a)$ của môi trường. Thuật toán **REINFORCE** (Williams, 1992) dùng ước lượng một quỹ đạo của biểu thức trên:

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \; R(\tau), \qquad \theta \leftarrow \theta + \alpha \, \hat{g}.$$

Một cải tiến thuận lý: hành động ở bước t không thể ảnh hưởng thưởng **đã nhận trước** t, nên ta thay $R(\tau)$ bằng **reward-to-go** $G_t = \sum_{k \geq t} \gamma^{k-t} r_{k+1}$ (return từ t trở đi, đúng G_t của nen01):

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t.$$

Kỳ vọng vẫn giữ nguyên, nhưng bỏ bớt các số hạng chỉ thêm nhiễu (đây là suy luận chuẩn của policy gradient theorem, Sutton và cộng sự, 1999). Với softmax, $\nabla_\theta \log \pi(a) = e_a - \pi$ (e_a là vector one-hot của hành động a), nên công thức rất dễ tính, đúng như hình ở trên.

4.3 Ví dụ tay: bandit hai cánh, không baseline

Bandit là MDP một bước: chọn cánh, nhận thưởng, hết. Cho hai cánh có thưởng tất định $r_1 = 10$, $r_2 = 8$, chính sách softmax với $\theta = (0, 0)$ nên $\pi = (0{,}5; 0{,}5)$. Thưởng kỳ vọng $J = 0{,}5 \cdot 10 + 0{,}5 \cdot 8 = 9$. Gradient thật có công thức $\partial J / \partial \theta_k = \pi_k(\mu_k - J)$ (với μ_k là thưởng trung bình của cánh k): thành phần θ_1 là $0{,}5 \cdot (10 - 9) = 0{,}5$ và thành phần θ_2 là $-0{,}5$ (tự tính).

REINFORCE một mẫu, $\hat{g} = r \cdot (e_a - \pi)$:

- Rút cánh 1 ($r = 10$): $e_a - \pi = (0{,}5; -0{,}5)$ nên $\hat{g} = (5; -5)$.
- Rút cánh 2 ($r = 8$): $e_a - \pi = (-0{,}5; 0{,}5)$ nên $\hat{g} = (-4; 4)$.

Kỳ vọng: $0{,}5 \cdot (5; -5) + 0{,}5 \cdot (-4; 4) = (0{,}5; -0{,}5)$, đúng gradient thật. Nhưng từng mẫu rất khác: một mẫu đẩy mạnh về cánh 1 (giá trị 5), mẫu kia đẩy ngược (giá trị -4) dù cánh 2 có thưởng dương 8. Phương sai của thành phần θ_1 là $(25 + 16)/2 - 0{,}5^2 = 20{,}25$ (tự tính). Vấn đề: thưởng đều dương, nên **mọi** hành động đều được "khen"; hướng đúng chỉ xuất hiện sau khi trung bình hoá nhiều mẫu.

Hệ quả khi cập nhật với $\alpha = 0{,}1$ (tự tính): nếu rút cánh 1, $\theta = (0{,}5; -0{,}5)$ và $\pi_1 = 0{,}731$; nếu rút cánh 2, $\theta = (-0{,}4; 0{,}4)$ và $\pi_1 = 0{,}310$, tức xác suất cánh **tốt hơn** giảm từ $0{,}5$ xuống $0{,}31$. Trung bình thì đúng, nhưng từng bước thì nhảy lung tung.

5. Vì sao phương sai lớn

Phương sai của $\hat{g}$ có ba nguồn chồng nhau. (1) Hành động được lấy mẫu ngẫu nhiên, nên $\nabla \log \pi(a)$ đổi theo a. (2) Thưởng (hoặc return) bản thân ngẫu nhiên, và với episode dài, return cộng dồn nhiều nhiễu của cả quỹ đạo. (3) Quan trọng nhất ở ví dụ trên: **độ lớn của thưởng** nhân vào mọi mẫu. Nếu thưởng lớn và cùng dấu, mọi hành động bị đẩy mạnh cùng chiều (tăng xác suất), và hướng "phân biệt tốt xấu" bị chìm trong thành phần chung.

Điều ta mong: một đại lượng thay G_t sao cho (i) không đổi kỳ vọng gradient, (ii) có độ lớn nhỏ hơn và chỉ phản ánh phần "hơn kém giữa các hành động". Đó là baseline.

6. Baseline: giảm phương sai mà không đổi kỳ vọng

6.1 Định nghĩa và chứng minh không chệch

Baseline $b(s)$ là một đại lượng chỉ phụ thuộc trạng thái (không phụ thuộc hành động a), được trừ vào return:

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \big(G_t - b(s_t)\big).$$

Vì sao kỳ vọng không đổi? Xét riêng số hạng $b(s)$ tại một trạng thái:

$$\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}\big[b(s)\, \nabla_\theta \log \pi_\theta(a \mid s)\big] = b(s) \sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = b(s) \sum_a \nabla_\theta \pi_\theta(a \mid s) = b(s)\, \nabla_\theta \sum_a \pi_\theta(a \mid s) =$$

Ta dùng lại $\pi \nabla \log \pi = \nabla \pi$, rồi tổng xác suất luôn bằng 1 nên đạo hàm bằng 0. Điều kiện sống còn: b **không được phụ thuộc hành động** a **vừa chọn**; nếu phụ thuộc thì b không đưa ra khỏi dấu tổng được và chứng minh sụp (và gradient bị chệch). Baseline dùng thưởng của chính mẫu hiện tại, ví dụ trừ chính r của hành động vừa rút, vi phạm điều kiện này.

Ví dụ kiểm tay (tự tính): $\pi = (0{,}8; 0{,}2)$, $b = 5$. Với softmax $\mathbb{E}[\nabla_{\theta_1} \log \pi(a)] = 0{,}8 \cdot 0{,}2 + 0{,}2 \cdot (-0{,}8) = 0$ (vì $e_a - \pi$ có thành phần θ_1 bằng $0{,}2$ khi chọn cánh 1 và $-0{,}8$ khi chọn cánh 2), nên $\mathbb{E}[b \nabla \log \pi] = 0$ với mọi b.

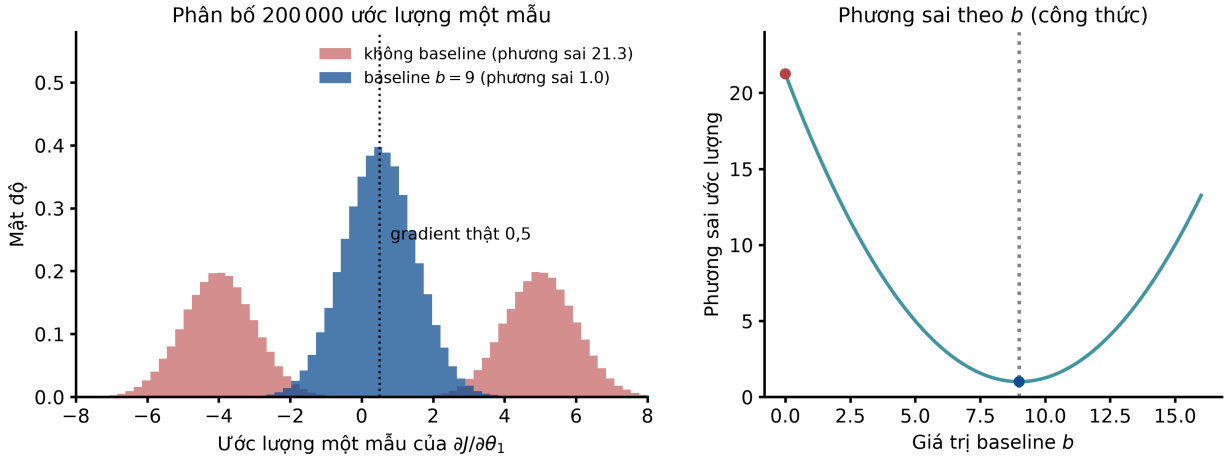
6.2 Ví dụ tay: cùng bandit, có baseline

Trở lại bandit tất định ở mục 4.3 và chọn $b = 9$ (đúng thưởng trung bình J). Khi rút cánh 1: $(10 - 9) \cdot (0{,}5; -0{,}5) = (0{,}5; -0{,}5)$. Khi rút cánh 2: $(8 - 9) \cdot (-0{,}5; 0{,}5) = (0{,}5; -0{,}5)$. **Cả hai mẫu cho cùng một vector, và vector đó chính là gradient thật.** Phương sai bằng 0 (tự tính). Đó là trường hợp đẹp do thưởng tất định; thực tế thưởng có nhiễu nên phương sai chỉ giảm chứ không về 0.

Cập nhật một bước với $\alpha = 0{,}1$: rút cánh 2 cho $\theta = (0{,}05; -0{,}05)$ và $\pi_1 = 0{,}525$ (tự tính), tức xác suất cánh tốt hơn **tăng** nhẹ, thay vì giảm xuống $0{,}31$ như khi không có baseline. Cánh 2 vẫn nhận thưởng dương 8, nhưng thấp hơn mức trung bình 9, nên bị coi là "kém hơn kỳ vọng" và bị kéo xuống. Đây là ý nghĩa của baseline: **chỉ khen khi hơn mốc, phạt khi kém mốc**.

6.3 Mô phỏng có nhiễu: phân bố ước lượng gradient

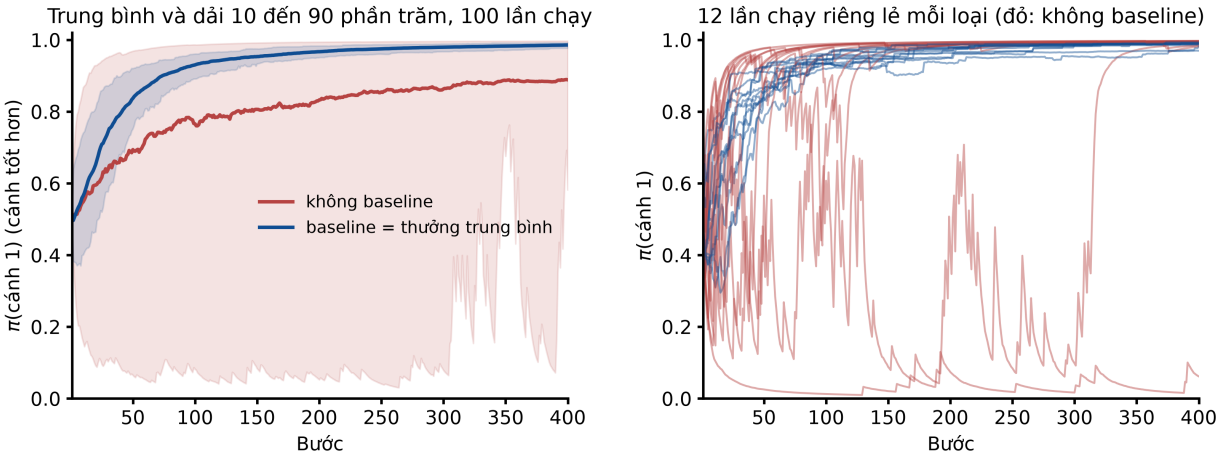
Giờ thêm nhiễu: thưởng cánh k là $\mu_k +$ nhiễu với nhiễu chuẩn độ lệch 2, $\mu = (10, 8)$. Gradient thật vẫn là $(0{,}5; -0{,}5)$. Công thức: phương sai thành phần θ_1 của ước lượng một mẫu với baseline b là $\frac{1}{4}\mathbb{E}[(R - b)^2] - 0{,}25$, với $\mathbb{E}[R^2] = 86$ và $\mathbb{E}[R] = 9$ ở bài này, nên $\frac{1}{4}(86 - 18b + b^2) - 0{,}25$. Với $b = 0$ ra 21,25, với $b = 9$ ra 1,00 (tự tính).



Hình 8: Phân bố ước lượng gradient một mẫu có và không có baseline, và phương sai theo giá trị baseline (minh hoạ, tự tính)

Cách đọc hình: bên trái là histogram của 200 000 ước lượng một mẫu cho $\partial J / \partial \theta_1$ từ **mô phỏng** (seed cố định). Không baseline (đỏ) cho hai cụm tách biệt quanh -4 và 5 (tuỳ cánh nào được rút), phương sai khoảng 21,3; có baseline $b = 9$ (xanh) dồn quanh gradient thật $0{,}5$ với phương sai khoảng $1{,}0$. Hai phân bố có **cùng trung bình** (khoảng $0{,}50$, tự tính). Bên phải là phương sai theo b: một parabol có đáy gần $b = 9$, tức gần thưởng trung bình J. Đáy của parabol này đúng bằng 9 do thiết kế đối xứng của bài (hai cánh xác suất bằng nhau); trong trường hợp tổng quát baseline tối ưu có dạng trung bình có trọng số khác một chút, nhưng "gần $V(s)$" là mốc tốt thực dụng.

6.4 Đường học



Hình 9: Đường học REINFORCE trên bandit hai cánh có và không có baseline (minh hoạ, tự tính)

Cách đọc hình: mô phỏng 100 lần chạy ($\alpha = 0{,}05$, 400 bước, nhiễu thưởng như trên); trục dọc là xác suất chọn cánh tốt hơn (cánh 1). Baseline ở đây là trung bình thưởng của các bước **trước** (không dùng thưởng của bước hiện tại, để giữ điều kiện không chệch). Bên trái: đường xanh (có baseline) lên nhanh và dải 10 đến 90 phần trăm hẹp, đạt trung bình khoảng $0{,}93$ ở bước 100 và khoảng $0{,}99$ ở bước 400; đường đỏ (không baseline) lên chậm hơn, trung bình khoảng $0{,}77$ ở bước 100 và $0{,}89$ ở bước 400 (tự tính từ mô phỏng này). Bên phải: các lần chạy riêng. Với không baseline, một số lần chạy rơi vào "cánh sai" (khoảng 10% số lần chạy kết thúc với xác suất cánh tốt hơn dưới $0{,}5$ ở bước 400) và mất rất lâu mới thoát; với baseline, 0 trong 100 lần chạy. Đây là một bandit đơn giản và một thiết lập siêu tham số duy nhất; không nên suy ra con số cụ thể cho bài toán khác, chỉ nên rút ra xu hướng: baseline làm quá trình ổn định hơn mà không đổi đích đến.

7. Advantage và hàm giá trị làm baseline

Baseline tự nhiên nhất là chính hàm giá trị: $b(s) = V^\pi(s)$. Khi đó $G_t - V(s_t)$ là ước lượng của **advantage** $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ (nen01), vì $\mathbb{E}[G_t \mid s_t, a_t] = Q^\pi(s_t, a_t)$. Dạng gradient gọn nhất:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\Big].$$

Advantage có một tính chất đẹp: $\mathbb{E}_{a\sim\pi}[A(s,a)] = 0$ tại mỗi trạng thái, nên "trung bình thì không đẩy gì cả, chỉ đẩy phần chênh". Ví dụ (tự tính): $\pi = (0{,}8; 0{,}2)$, thưởng $\mu = (10, 0)$, $V = 8$; advantage là $(2; -8)$, và $0{,}8 \cdot 2 + 0{,}2 \cdot (-8) = 0$. Gradient thật thành phần θ_1 là $\pi_1 A_1 = 0{,}8 \cdot 2 = 1{,}6$. Hãy để ý hành động hiếm ($\pi = 0{,}2$) có advantage lớn về độ lớn (-8) vì nó tệ hơn mức trung bình nhiều.

Ta thường **không biết** V, nên học một mạng $V_\psi(s)$ (critic) bằng TD như nen02, rồi dùng $G_t - V_\psi(s_t)$ hoặc sai lệch TD $\delta_t = r_{t+1} + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$ làm ước lượng advantage. Chọn $G_t - V_\psi$ cho gradient không chệch nhưng phương sai còn lớn (vì G_t dài); chọn δ_t cho phương sai nhỏ nhưng chệch nếu V_ψ sai. Cùng một thế đánh đổi như TD và Monte Carlo ở nen02. GAE (rl 05) cho một chiếc núm vặn liên tục giữa hai thái cực; chương này không đi vào chi tiết đó.

Gom lại thành thuật toán **actor-critic**: "actor" là chính sách π_θ cập nhật theo $\nabla \log \pi \cdot A$, "critic" là V_ψ học bằng TD để cung cấp baseline.

8. Biến thể và chi tiết hay bị bỏ qua

Baseline không phải hàm giá trị cũng được. Miễn là không phụ thuộc hành động vừa chọn. Một số lựa chọn thực tế: trung bình thưởng chạy (như ở hình), trung bình thưởng của **các mẫu khác** cho cùng một tình huống (leave-one-out), hoặc trung bình của một nhóm mẫu cùng đề bài. Các lựa chọn cuối là cách các bài RL cho LLM dùng thay cho mạng giá trị.

Chuẩn hoá advantage. Nhiều cài đặt chia advantage cho độ lệch chuẩn trong batch. Việc này đổi tỉ lệ của gradient (một dạng điều chỉnh learning rate theo từng batch), nên không còn đúng là "không đổi kỳ vọng" theo nghĩa chặt, nhưng thực hành thường ổn định. Hãy phân biệt "trừ baseline" (giữ nguyên kỳ vọng) với "chia độ lệch chuẩn" (đổi tỉ lệ).

Ước lượng một mẫu và on-policy. Công thức ở trên đòi mẫu lấy từ chính π_θ hiện tại. Sau mỗi lần cập nhật, mẫu cũ thuộc chính sách khác, nên phải bỏ hoặc hiệu chỉnh bằng importance sampling (nền của TRPO và PPO). Đó là khác biệt lớn so với Q-learning ở nen02 vốn dùng lại dữ liệu cũ tự do.

Bước cập nhật có thể quá lớn. Gradient chỉ đúng cục bộ; một bước dài làm chính sách đổi mạnh và có thể sập. TRPO và PPO ràng buộc độ đổi của chính sách để xử lý (không thuộc phạm vi chương này).

Thăm dò nằm sẵn trong chính sách. Vì chính sách là phân phối, nó tự thăm dò bằng cách rút mẫu; điều đáng quan tâm là xác suất co về 0 quá sớm (entropy sụp), khi mọi cánh khác bị bỏ rơi trước khi biết chúng tệ thật. Hình ở mục 6.4 bên phải cho thấy một lần chạy không baseline bị kẹt ở xác suất gần 0 cho cánh tốt hơn trong rất lâu.

Suy luận của chương (không phải của paper): các con số mô phỏng ở mục 6.3 và 6.4 chỉ minh hoạ xu hướng trong một bandit hai cánh với nhiễu chuẩn; hiệu ứng cụ thể ở bài toán thật phụ thuộc nhiều yếu tố khác.

9. Nối sang các bài paper của cuốn

- **rl 04 (TRPO) và rl 07 (PPO)** tối ưu mục tiêu có dạng "xác suất hành động nhân advantage" mà mục 7 vừa dựng; phần còn lại của các bài là giữ cập nhật không đi quá xa. Muốn hiểu mục tiêu của chúng, hãy nhìn dạng $\nabla \log \pi \cdot A$.
- **rl 05 (GAE)** cho cách ước lượng advantage ở mục 7 bằng cách trộn các độ dài bước, đặt giữa δ_t (chệch, ít nhiễu) và $G_t - V$ (không chệch, nhiều nhiễu).
- **rl 06 (A3C)** là actor-critic: cập nhật actor bằng advantage, critic bằng TD (chi tiết xem ghi chú).
- **rl 37 (RLOO)** theo ghi chú của nó, quay lại REINFORCE có baseline, và với k mẫu cho cùng một prompt dùng trung bình thưởng của $k-1$ mẫu còn lại làm baseline, một "value function không tham số" tính tại chỗ. Đây chính là baseline không phụ thuộc mẫu hiện tại như điều kiện ở mục 6.1.
- **rl 36 (GRPO)** theo ghi chú của nó, bỏ mạng giá trị và dùng trung bình thưởng của cả nhóm lời giải cho cùng câu hỏi làm baseline. Cùng tinh thần mục 8 (baseline từ các mẫu khác), nhưng chuẩn hoá và cách gom nhóm là chi tiết riêng của bài đó.
- **Các bài RL cho LLM nói chung** coi cả câu trả lời như một hành động hoặc chuỗi token, thưởng ở cuối; $\nabla \log \pi$ của cả chuỗi là tổng $\nabla \log \pi$ của từng token, đúng cấu trúc mục 4.2.

10. Sai lầm hay gặp

- **Trừ baseline phụ thuộc hành động vừa chọn.** Ví dụ dùng chính thưởng của mẫu hiện tại làm mốc: kỳ vọng bị chệch (chứng minh mục 6.1 không còn đúng).
- **Nghĩ baseline làm gradient "đúng hơn".** Baseline chỉ giảm phương sai; kỳ vọng gradient không đổi. Với thưởng tất định và tình huống hoàn hảo như mục 6.2, phương sai về 0, nhưng đó là ngoại lệ do thiết kế.
- **Quên rằng REINFORCE là on-policy.** Dùng lại mẫu cũ sau khi cập nhật mà không hiệu chỉnh làm gradient sai.

- **Nhầm dấu.** Policy gradient là gradient **ascent** trên J; nếu viết cho loss thì cần dấu trừ: loss $= -\sum \log \pi \cdot A$.
- **Quên rằng G_t chỉ gồm thưởng sau t.** Dùng tổng cả quỹ đạo vẫn không chệch nhưng thêm nhiễu vô ích.
- **Gộp chuẩn hoá advantage với baseline.** Hai thao tác khác nhau: một cái giữ kỳ vọng, một cái đổi tỉ lệ.

11. Bảng tóm tắt

Đại lượng	Công thức	Ý nghĩa
Log-derivative	$\nabla p = p \nabla \log p$	Đưa gradient vào trong kỳ vọng
REINFORCE	$\hat{g} = \sum_t \nabla \log \pi(a_t \mid s_t)\, G_t$	Không chệch, phương sai lớn
Softmax	$\nabla_\theta \log \pi(a) = e_a - \pi$	Đẩy xác suất của hành động đã chọn
Baseline	$\mathbb{E}[b(s) \nabla \log \pi] = 0$	Không đổi kỳ vọng nếu b không phụ thuộc a
Advantage	$A = Q - V$	$\mathbb{E}_a[A] = 0$, chỉ giữ phần hơn kém
Gradient dùng advantage	$\mathbb{E}[\sum_t \nabla \log \pi\, A]$	Dạng của actor-critic, PPO
Sai lệch TD	$\delta_t = r + \gamma V(s') - V(s)$	Ước lượng advantage ít nhiễu, có thể chệch

12. Bài tập tự tính

Các bài là phép tính độc lập, không phải số liệu paper. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Gradient log softmax

Cho $\theta = (1, 0)$. Tính π và $\nabla_\theta \log \pi(a = 2)$.

Đáp án (tự tính): $\pi = (0{,}731; 0{,}269)$. $\nabla_\theta \log \pi(2) = e_2 - \pi = (-0{,}731;\ 0{,}731)$.

Bài 2. Bandit và baseline

Hai cánh có thưởng tất định $(3, 1)$, chính sách đều $(0{,}5; 0{,}5)$. Tính gradient thật theo θ, các ước lượng một mẫu không baseline và với baseline $b = 2$, rồi phương sai thành phần θ_1 trong cả hai trường hợp.

Đáp án (tự tính): $J = 2$; gradient thật $(0{,}5; -0{,}5)$. Không baseline: rút cánh 1 cho $(1{,}5; -1{,}5)$, rút cánh 2 cho $(-0{,}5; 0{,}5)$; thành phần θ_1 có trung bình $0{,}5$ và phương sai 1. Với $b = 2$: cả hai mẫu cho $(0{,}5; -0{,}5)$, phương sai 0.

Bài 3. Kiểm tra không chệch

$\pi = (0{,}8; 0{,}2)$, $b = 5$. Tính $\mathbb{E}[b \nabla_\theta \log \pi(a)]$ (thành phần θ_1) và giải thích vì sao kết quả không phụ thuộc b.

Đáp án (tự tính): $\nabla_{\theta_1} \log \pi(a)$ bằng $0{,}2$ khi $a = 1$ và $-0{,}8$ khi $a = 2$. Kỳ vọng $5 \cdot (0{,}8 \cdot 0{,}2 + 0{,}2 \cdot (-0{,}8)) = 5 \cdot 0 = 0$. Tổng $\sum_a \pi \nabla \log \pi = \nabla \sum \pi = 0$ cho mọi b không phụ thuộc a.

Bài 4. Advantage

$\pi = (0{,}8; 0{,}2)$, thưởng tất định $(10, 0)$. Tính V, advantage của từng hành động, kiểm tra $\mathbb{E}[A] = 0$ và gradient thật thành phần θ_1.

Đáp án (tự tính): $V = 8$; $A = (2; -8)$; $0{,}8 \cdot 2 + 0{,}2 \cdot (-8) = 0$. Gradient thành phần θ_1 là $\pi_1 A_1 = 1{,}6$ (và thành phần θ_2 là $-1{,}6$).

Bài 5. Reward-to-go và advantage

Một episode ba bước với thưởng $1, 2, 3$, $\gamma = 1$. Tính G_0, G_1, G_2. Nếu critic cho $V = (5, 4, 2)$ tại ba trạng thái, tính ba advantage ước lượng $G_t - V(s_t)$. Rồi tính sai lệch TD $\delta_t = r + V(s') - V(s)$ với $V(s_3) = 0$ (kết thúc).

Đáp án (tự tính): $G = (6, 5, 3)$. Advantage $G_t - V = (1, 1, 1)$. Sai lệch TD: $\delta_0 = 1 + 4 - 5 = 0$, $\delta_1 = 2 + 2 - 4 = 0$, $\delta_2 = 3 + 0 - 2 = 1$. So sánh: $G_t - V$ nhìn cả phần còn lại của episode nên cho $(1, 1, 1)$, còn δ_t chỉ nhìn một bước và dựa vào V, nên cho $(0, 0, 1)$; cho thấy khác biệt chệch và phương sai của hai ước lượng khi V chưa chính xác.

Tài liệu tham khảo nền: Williams, Simple Statistical Gradient-Following Algorithms (REINFORCE); Sutton, McAllester, Singh và Mansour, Policy Gradient Methods for RL with Function Approximation; Sutton và Barto, Reinforcement Learning: An Introduction (chương policy gradient); preface của cuốn này; các ghi chú rl 04, 05, 06, 07, 36 và 37 trong sách.

Nền tảng 04

Actor-critic và GAE: dùng một người chấm điểm để giảm nhiễu, rồi chọn mức tin vào người chấm bằng λ

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: MDP, hàm giá trị và TD ở nen01, nen02 của cuốn này; policy gradient và baseline ở nen03 của cuốn này; kỳ vọng và phương sai ở math nền tảng 08; SGD và nhiễu mini-batch ở optimization nền 03

Dùng ở: rl 05 (GAE), rl 06 (A3C), rl 10 (DDPG), rl 11 (TD3), rl 12 (SAC)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình mô phỏng dùng dữ liệu tự sinh với seed cố định và được nói rõ là giả định.

1. Vì sao cần actor-critic và GAE

Actor-critic là cách huấn luyện chính sách trong đó một mạng thứ hai (critic) học ước lượng giá trị, rồi dùng ước lượng đó để chấm điểm các hành động của chính sách (actor). Nó giải quyết một bất tiện của policy gradient thuần: ta phải chờ hết cả quỹ đạo mới biết hành động có tốt không, và tổng thưởng của cả quỹ đạo nhiễu đến mức mỗi hành động riêng lẻ bị "nhấn chìm" trong tiếng ồn của tất cả những hành động khác.

Hãy hình dung bạn chơi một ván cờ dài 100 nước, thua ở cuối. Nếu chỉ biết "thua", bạn không rõ nước thứ 12 hay nước thứ 80 gây ra lỗi. Một người xem cờ giỏi (critic) có thể nói ngay sau mỗi nước: "thế này đang tốt hơn dự kiến" hoặc "thế này tệ hơn dự kiến". Dùng nhận xét đó thay vì chờ kết quả cuối ván, bạn học nhanh hơn, nhưng nếu người xem cờ nói sai thì bạn bị dẫn sai một cách có hệ thống.

Chương này trả lời ba câu hỏi: critic dùng để làm gì và đưa ra con số nào (advantage), nhìn xa bao nhiêu bước trước khi tin critic (advantage một bước và n bước), và làm sao trộn mọi độ dài nhìn xa thành một tham số duy nhất λ (GAE). Các bài paper của cuốn dùng lại những khái niệm này ở khắp nơi: PPO dùng GAE làm bộ ước lượng advantage, A3C dùng advantage n bước, và DDPG, TD3, SAC thay critic V bằng critic Q nhưng vẫn giữ tinh thần actor-critic.

2. Nhắc nền: return, giá trị, baseline và phương sai

2.1 Return và hàm giá trị

Ở mỗi bước t, tác nhân ở trạng thái s_t, chọn hành động a_t, nhận thưởng r_t. Hệ số chiết khấu $\gamma \in [0, 1]$ đo mức coi trọng thưởng tương lai. **Return** từ bước t là

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

Hàm giá trị trạng thái $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$ là return kỳ vọng khi bắt đầu từ s và theo chính sách π. Hàm giá trị hành động $Q^\pi(s, a)$ là return kỳ vọng khi ở s, chọn a rồi theo π. Ví dụ (tự tính): với $\gamma = 0{,}5$ và các thưởng $1, 0, 2$ rồi kết thúc, return từ bước đầu là $1 + 0 + 0{,}25 \cdot 2 = 1{,}5$.

2.2 Advantage

Advantage của hành động a tại s là $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$: hành động này tốt hơn hay kém hơn mức trung bình của chính sách tại cùng trạng thái. Hai điều cần nhớ. Thứ nhất, trung bình của advantage theo hành động do chính sách chọn bằng 0, vì $V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s, a)]$. Thứ hai, advantage chỉ nói về **so sánh trong cùng trạng thái**, nên nó trừ đi phần giá trị mà trạng thái đã có sẵn do hoàn cảnh chứ không do hành động.

2.3 Policy gradient và baseline

Nhắc từ nen03: gradient của tổng thưởng kỳ vọng $J(\theta)$ theo tham số chính sách θ là

$$\nabla_\theta J = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \Psi_t\big],$$

với Ψ_t là "điểm số" của hành động a_t. Chọn $\Psi_t = G_t$ cho ước lượng không chệch nhưng nhiễu. Trừ một **baseline** $b(s_t)$ chỉ phụ thuộc trạng thái không làm đổi kỳ vọng của gradient (vì $\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$), nhưng có thể giảm phương sai mạnh. Baseline tự nhiên nhất là $V^\pi(s_t)$, khi đó $\Psi_t = G_t - V^\pi(s_t)$ ước lượng advantage.

Ví dụ số nhỏ về lý do baseline giúp. Giả sử ở một trạng thái có hai hành động, return thật luôn là 10 cho hành động 1 và 11 cho hành động 2. Không baseline, cả hai điểm số đều rất dương, và mỗi cập nhật đẩy cả hai xác suất lên; chỉ sự chênh lệch nhỏ 1 mới mang thông tin, bị chôn dưới con số 10. Trừ $V = 10{,}5$ (tự tính: trung bình của hai hành động khi chọn đều) thì điểm số còn $-0{,}5$ và $+0{,}5$, đúng thông tin cần thiết.

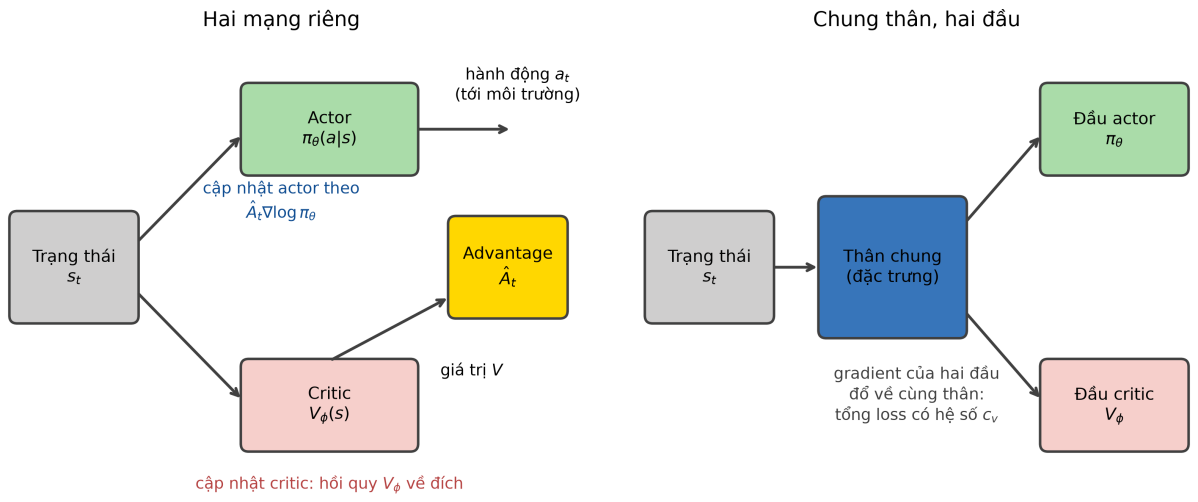
2.4 Sai số TD

Critic V_ϕ là một mạng (tham số ϕ) xấp xỉ V^π. Với một chuyển tiếp (s_t, a_t, r_t, s_{t+1}), **sai số TD** (TD residual) là

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

Nó so sánh "những gì critic dự đoán về s_t" với "một thưởng thật cộng dự đoán về s_{t+1}". Nếu critic chính xác, kỳ vọng của δ_t theo hành động a_t chính là advantage của a_t. Ví dụ (tự tính): $\gamma = 0{,}9$, $r_t = 1$, $V_\phi(s_t) = 2$, $V_\phi(s_{t+1}) = 3$ cho $\delta_t = 1 + 2{,}7 - 2 = 1{,}7$: hành động này đưa tác nhân tới kết quả tốt hơn critic dự tính $1{,}7$ đơn vị.

3. Trực giác bằng hình: actor chọn, critic chấm



Hình 10: Sơ đồ actor-critic: hai mạng riêng và một mạng dùng chung thân hai đầu (minh hoạ, tự tính)

Cách đọc hình: bên trái, trạng thái s_t đi vào hai mạng riêng. Actor đưa ra hành động gửi tới môi trường; critic đưa ra giá trị V dùng để tính advantage $\hat{A}_t$ (cùng với thưởng nhận được). Mũi tên màu xanh và đỏ ở dưới cho biết hai mạng học bằng hai loss khác nhau: actor tăng xác suất hành động có advantage dương, critic hồi quy giá trị về đích. Bên phải là kiểu dùng chung một thân lấy đặc trưng, hai đầu riêng; mục 8 bàn đánh đổi giữa hai kiểu.

Vòng lặp này có một điểm tinh tế. Critic học từ dữ liệu do actor sinh ra, còn actor học từ nhận xét của critic. Cả hai cùng đổi nên mục tiêu của mỗi bên di chuyển. Đây là lý do actor-critic nhạy với tốc độ học tương đối và với chất lượng critic.

4. Định nghĩa: critic và advantage một bước

4.1 Hai loss

Trong actor-critic cơ bản, mỗi lô dữ liệu cập nhật hai thứ. Actor tối đa hóa

$$L_{\text{actor}}(\theta) = \mathbb{E}\big[\log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\big],$$

trong đó $\hat{A}_t$ được coi là hằng số (không lan gradient vào critic). Critic tối thiểu hóa sai số bình phương so với đích $\hat{V}_t^{\text{targ}}$:

$$L_{\text{critic}}(\phi) = \mathbb{E}\big[(V_\phi(s_t) - \hat{V}_t^{\text{targ}})^2\big].$$

4.2 Advantage một bước

Ước lượng đơn giản nhất của advantage chính là sai số TD:

$$\hat{A}_t^{(1)} = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

Nó chỉ nhìn một thưởng thật r_t, phần còn lại đến từ critic, qua $V_\phi(s_{t+1})$. Hệ quả là **phương sai thấp** (chỉ một biến ngẫu nhiên r_t và một chuyển tiếp) nhưng **thiên lệch** nếu critic sai: sai số của V_ϕ chui thẳng vào advantage. Hãy nhớ hai vế này, mọi thứ còn lại trong chương là cách điều chỉnh giữa chúng.

5. Advantage n bước: nhìn xa hơn trước khi tin critic

Thay vì tin critic ngay sau một bước, ta có thể cộng n thưởng thật rồi mới dùng critic:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t).$$

Với $n = 1$ ta được δ_t. Khi n lớn tới hết quỹ đạo (tới trạng thái kết thúc, nơi $V = 0$), ta được $G_t - V_\phi(s_t)$: return Monte Carlo trừ baseline. Ở đầu này, critic chỉ còn là baseline nên advantage không chệch (với bất kỳ baseline phụ thuộc trạng thái), nhưng phương sai là tổng của nhiều thưởng và nhiều chuyển tiếp ngẫu nhiên.

Đánh đổi thiên lệch so với phương sai theo n. Mỗi thưởng thật thêm vào là một biến ngẫu nhiên mới (thêm phương sai), nhưng phần dùng critic bị chiết khấu thêm một lần γ (giảm mức ảnh hưởng của sai số critic). Quy tắc chung: n nhỏ, critic gánh nhiều, thiên lệch cao, phương sai thấp; n lớn, quỹ đạo thật gánh nhiều, thiên lệch thấp, phương sai cao.

A3C (rl 06) dùng đúng kiểu này: theo ghi chú rl 06, mỗi luồng chạy tới $t_{\max}$ bước hoặc tới trạng thái kết thúc, rồi tính ngược các return n bước, với n thay đổi tùy vị trí trong đoạn rollout.

6. GAE: trung bình mũ của mọi độ dài nhìn xa

6.1 Ý tưởng

Chọn n là một lựa chọn rời rạc, thô. **GAE** (generalized advantage estimation) trộn tất cả các n bằng một trung bình có trọng số hình học, điều khiển bởi $\lambda \in [0, 1]$:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)}.$$

Khai triển, mọi thứ rút gọn thành một tổng rất gọn của các sai số TD:

$$\hat{A}_t^{\mathrm{GAE}} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\,\delta_{t+l}.$$

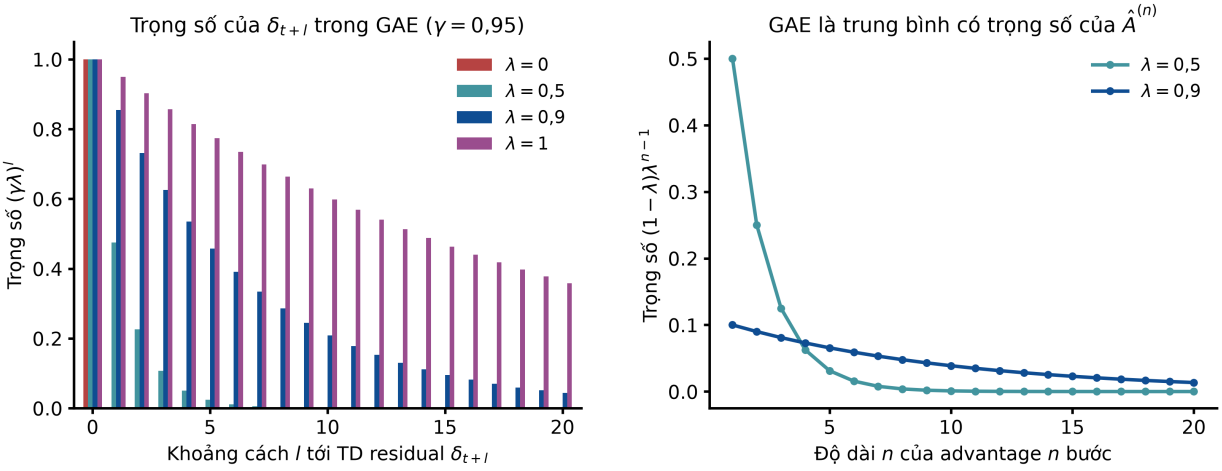
Công thức thứ hai tiện cài đặt hơn: advantage tại t là sai số TD hiện tại cộng sai số TD các bước sau, mỗi bước xa hơn bị nhân thêm $\gamma\lambda$.

6.2 Hai đầu mút

- $\lambda = 0$: chỉ còn $l = 0$, $\hat{A}_t = \delta_t$, tức advantage một bước. Thiên lệch cao nếu critic sai, phương sai thấp.
- $\lambda = 1$: $\hat{A}_t = \sum_l \gamma^l \delta_{t+l}$, và tổng này co lại (telescoping) thành $\sum_l \gamma^l r_{t+l} - V_\phi(s_t)$, tức return Monte Carlo trừ baseline. Không chệch, phương sai cao.

Ở giữa, $\lambda \in (0,1)$ là một **thanh trượt liên tục** giữa hai đầu. γ và λ khác vai trò: γ là một phần của định nghĩa bài toán (mức coi trọng tương lai) và đổi cả mục tiêu thật, còn λ chỉ là núm điều chỉnh bộ ước lượng cho cùng một mục tiêu. Paper GAE nhấn mạnh phân biệt này (theo ghi chú rl 05).

6.3 Trọng số mũ theo bước



Hình 11: Trọng số của các sai số TD trong GAE (trái) và trọng số của các advantage n bước (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, trục ngang là khoảng cách l từ bước đang xét tới sai số TD δ_{t+l}, trục đứng là trọng số $(\gamma\lambda)^l$, với $\gamma = 0{,}95$. Với $\lambda = 0$ chỉ có cột $l = 0$ (bằng 1), các cột còn lại bằng 0 nên không hiện. λ càng lớn thì đuôi trọng số càng dài: $\lambda = 0{,}5$ tắt sau khoảng 6 bước, $\lambda = 0{,}9$ còn đáng kể ở $l = 15$, $\lambda = 1$ giảm chậm chỉ do γ. Bên phải là cùng ý tưởng nhìn theo hướng khác: GAE là trung bình các advantage n bước với trọng số $(1-\lambda)\lambda^{n-1}$, nên $\lambda = 0{,}5$ dồn phần lớn trọng số vào $n = 1, 2, 3$ còn $\lambda = 0{,}9$ dàn khá đều.

Độ dài nhìn xa hiệu dụng. Tổng trọng số $\sum_l(\gamma\lambda)^l = 1/(1-\gamma\lambda)$. Ví dụ (tự tính): $\gamma = 0{,}99$, $\lambda = 0{,}95$ cho $\gamma\lambda = 0{,}9405$, tổng trọng số $\approx 16{,}8$, và trọng số tại $l = 10$ còn $0{,}9405^{10} \approx 0{,}541$ (tự tính). Đây là lý do cặp $(\gamma, \lambda) = (0{,}99; 0{,}95)$ nhìn xa chừng vài chục bước dù λ gần 1.

6.4 Công thức đệ quy dùng khi cài đặt

Từ công thức tổng, ta thấy $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$. Nên cài đặt bằng một vòng lặp ngược từ cuối rollout: đặt $\hat{A}_T = 0$ (hoặc phần bootstrap, xem mục 9), rồi với $t = T-1, \dots, 0$:

$$\hat{A}_t = \delta_t + \gamma\lambda\,(1 - d_t)\,\hat{A}_{t+1}, \qquad d_t = 1 \text{ nếu quỹ đạo kết thúc sau bước } t.$$

Đích cho critic thường là $\hat{V}_t^{\text{targ}} = \hat{A}_t + V_\phi(s_t)$ (gọi là λ-return). Với $\lambda = 1$ nó là return Monte Carlo, với $\lambda = 0$ nó là đích TD một bước.

7. Ví dụ tay: quỹ đạo 5 bước

Giả sử một quỹ đạo có 5 bước (đánh số $t = 1, \dots, 5$), kết thúc ở bước 5 nên $V(s_6) = 0$. Để tính tay gọn, đặt $\gamma = 0{,}5$ (không phải giá trị dùng thực tế). Dữ liệu cho trước:

Bước t	1	2	3	4	5
Thưởng r_t	1	0	2	0	4
Critic $V(s_t)$	2	2	3	1	3

Sai số TD theo $\delta_t = r_t + 0{,}5V(s_{t+1}) - V(s_t)$ (tự tính):

- $\delta_1 = 1 + 0{,}5 \cdot 2 - 2 = 0$
- $\delta_2 = 0 + 0{,}5 \cdot 3 - 2 = -0{,}5$
- $\delta_3 = 2 + 0{,}5 \cdot 1 - 3 = -0{,}5$
- $\delta_4 = 0 + 0{,}5 \cdot 3 - 1 = 0{,}5$
- $\delta_5 = 4 + 0 - 3 = 1$

Advantage n **bước tại** $t = 1$ (tự tính). Công thức $\hat{A}_1^{(n)} = \sum_{l<n} 0{,}5^l r_{1+l} + 0{,}5^n V(s_{1+n}) - V(s_1)$ cho:

- $n = 1$: $1 + 0{,}5 \cdot 2 - 2 = 0$ (đúng bằng δ_1).
- $n = 2$: $1 + 0 + 0{,}25 \cdot 3 - 2 = -0{,}25$.
- $n = 3$: $1 + 0 + 0{,}25 \cdot 2 + 0{,}125 \cdot 1 - 2 = -0{,}375$.
- $n = 4$: $1 + 0 + 0{,}5 + 0 + 0{,}0625 \cdot 3 - 2 = -0{,}3125$.
- $n = 5$: $1 + 0 + 0{,}5 + 0 + 0{,}0625 \cdot 4 - 2 = -0{,}25$ (đây là return Monte Carlo $1{,}75$ trừ $V(s_1) = 2$; mốc cuối vì $V(s_6) = 0$).

Chú ý: $n = 1$ cho 0 (một bước khớp với critic), còn $n \geq 2$ bắt đầu thấy rằng các thưởng xa hơn thấp hơn mức critic kỳ vọng tại s_1, nên advantage chuyển sang âm.

GAE tại $t = 1$ **với** $\gamma\lambda = 0{,}25$ **khi** $\lambda = 0{,}5$ (tự tính):

$$\hat{A}_1 = \delta_1 + 0{,}25\delta_2 + 0{,}0625\delta_3 + 0{,}015625\delta_4 + 0{,}00390625\delta_5$$

$$= 0 - 0{,}125 - 0{,}03125 + 0{,}0078125 + 0{,}00390625 \approx -0{,}1445.$$

Kiểm bằng trung bình có trọng số: trọng số của $n = 1, 2, 3, 4$ là $0{,}5;\ 0{,}25;\ 0{,}125;\ 0{,}0625$ và trọng số còn lại $0{,}0625$ dồn cho $n = 5$ (vì quỹ đạo kết thúc). Khi đó $0{,}5 \cdot 0 + 0{,}25(-0{,}25) + 0{,}125(-0{,}375) + 0{,}0625(-0{,}3125) + 0{,}0625(-0{,}25) = -0{,}1445$ (tự tính). Hai cách ra cùng kết quả.

Bảng cho mọi t, với $\lambda \in \{0;\ 0{,}5;\ 1\}$ (tự tính, làm tròn 4 chữ số):

t	1	2	3	4	5
$\lambda = 0$	0	$-0{,}5$	$-0{,}5$	$0{,}5$	1
$\lambda = 0{,}5$	$-0{,}1445$	$-0{,}5781$	$-0{,}3125$	$0{,}75$	1
$\lambda = 1$	$-0{,}25$	$-0{,}5$	0	1	1

Cách tính nhanh bằng đệ quy ngược với $\lambda = 0{,}5$: $\hat{A}_5 = 1$; $\hat{A}_4 = 0{,}5 + 0{,}25 \cdot 1 = 0{,}75$; $\hat{A}_3 = -0{,}5 + 0{,}25 \cdot 0{,}75 = -0{,}3125$; $\hat{A}_2 = -0{,}5 + 0{,}25 \cdot (-0{,}3125) = -0{,}578125$; $\hat{A}_1 = 0 + 0{,}25 \cdot (-0{,}578125) = -0{,}1445$. Cột $\lambda = 1$ trùng với n bước đến hết quỹ đạo: ở $t = 1$ bằng $-0{,}25$ như $n = 5$ ở trên.

Quan sát: tại $t = 3$, $\lambda = 0$ cho $-0{,}5$ (critic nghĩ s_3 giá trị 3, thực tế sau một bước chỉ còn $2 + 0{,}5 = 2{,}5$), còn $\lambda = 1$ cho 0 (nhìn tới hết quỹ đạo, tổng thưởng thật khớp critic). Ba hàng đưa ra ba kết luận khác nhau về cùng một hành động. Chọn hàng nào chính là câu hỏi bạn tin critic hay tin dữ liệu thật hơn.

8. Chung mạng hay tách mạng

8.1 Hai kiểu

- **Tách mạng**: actor và critic là hai mạng độc lập, mỗi mạng một optimizer hoặc ít nhất một tập tham số riêng. Gradient của critic không chạm vào actor.
- **Dùng chung thân**: một mạng trích đặc trưng chung cho cả hai, ra hai đầu (đầu chính sách, đầu giá trị). Tổng loss thường có dạng $L = L_{\text{actor}} + c_v L_{\text{critic}} - c_e H(\pi_\theta)$, với c_v là hệ số cho critic và c_e là hệ số cho entropy.

8.2 So sánh

Tiêu chí	Chung thân	Tách mạng
Tham số và bộ nhớ	Ít hơn, một lần forward	Gần gấp đôi
Học đặc trưng	Hai nhiệm vụ cùng học đặc trưng: có thể có lợi khi dữ liệu khan hiếm (nhất là đầu vào ảnh)	Mỗi bên tự học, không chia sẻ
Nhiễu gradient	Gradient của critic (thường lớn, thay đổi theo thang thưởng) có thể làm sai lệch đặc trưng của actor	Hai bên cô lập, dễ chỉnh từng bên
Chỉnh c_v và learning rate	Phải cân bằng hai loss trong một thân	Mỗi mạng một learning rate

Quy tắc thường gặp (không phải định luật): đầu vào ảnh hoặc bài toán cần chung đặc trưng thì chung thân hấp dẫn; khi hai nhiệm vụ gây xung đột gradient hoặc cần tinh chỉnh riêng thì tách. A3C (rl 06) chọn chung gần hết lớp ẩn, chỉ khác lớp ra, theo ghi chú rl 06. Với các mô hình ngôn ngữ lớn, critic thường là một bản sao cỡ tương đương hoặc một đầu giá trị gắn lên thân của actor; vì vậy bộ nhớ của critic đáng kể và là lý do một số phương pháp (như GRPO, rl 36) bỏ critic.

8.3 Ví dụ nhỏ về hệ số c_v (tự tính)

Giả sử ở một bước, $L_{\text{actor}} = 0{,}2$, $L_{\text{critic}} = 8$ (thang thưởng lớn khiến critic loss lớn), $c_e H = 0{,}01$. Nếu $c_v = 0{,}5$ thì tổng là $0{,}2 + 4 - 0{,}01 = 4{,}19$: gradient của critic chiếm gần hết tổng. Nếu thang thưởng bị chuẩn hóa nhỏ lại 4 lần (loss critic giảm khoảng 16 lần vì bình phương), $L_{\text{critic}} = 0{,}5$ và tổng là $0{,}2 + 0{,}25 - 0{,}01 = 0{,}44$. Đây là một lý do chuẩn hóa thưởng hoặc return, và chỉnh c_v quan trọng khi dùng chung thân.

9. Biến thể và chi tiết hay bị bỏ qua

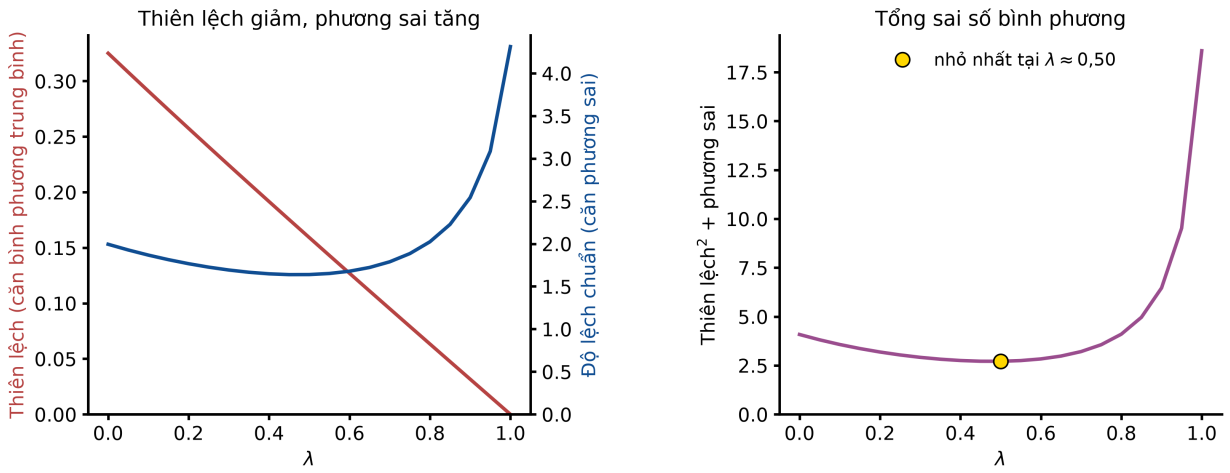
- **Dừng do kết thúc thật so với bị cắt (truncation).** Khi quỹ đạo kết thúc thật (thắng, thua, chết), V tiếp theo bằng 0. Khi quỹ đạo chỉ bị cắt vì hết giới hạn thời gian hay hết đoạn rollout, trạng thái kế tiếp vẫn

có giá trị: phải **bootstrap** bằng $V_\phi(s_T)$ thay vì 0. Nhầm hai trường hợp này làm critic học giá trị sai ở sát biên.

- **Chuẩn hóa advantage.** Trước khi cập nhật actor, nhiều cài đặt trừ trung bình và chia độ lệch chuẩn của advantage trong mỗi lô. Nó giữ thang gradient ổn định nhưng làm đổi nhẹ mục tiêu thật; theo ghi chú rl 09, chuẩn hóa advantage theo minibatch ảnh hưởng ít trong thí nghiệm của paper đó.
- **Chọn λ.** Theo ghi chú rl 09, $\lambda = 0{,}9$ cho kết quả tốt trên mọi môi trường thử với GAE và V-trace, và GAE cùng V-trace tốt hơn n-step; PPO gốc dùng $\gamma = 0{,}99$, $\lambda = 0{,}95$ (theo ghi chú rl 07).
- **Ngôn ngữ lớn thường đặt $\gamma = 1$ và $\lambda = 1$ hoặc gần 1** vì phần thưởng chỉ xuất hiện ở cuối chuỗi (suy luận của chương, không phải số liệu paper); khi đó GAE co lại thành "thưởng cuối trừ giá trị tại mỗi token".
- **Critic Q thay vì critic V.** DDPG, TD3, SAC (rl 10, 11, 12) học $Q_\phi(s, a)$ và đưa gradient của Q theo hành động vào actor. Cùng tinh thần "người chấm điểm", khác ở chỗ chấm điểm cả hành động và cập nhật actor bằng gradient chứ không bằng $\log \pi \cdot \hat{A}$.
- **Critic học từ chính nó (bootstrapping) có thể phân kỳ** nếu kết hợp với xấp xỉ hàm và dữ liệu lệch phân phối; xem chương nen06 của cuốn này về off-policy.

10. Mô phỏng: λ làm thiên lệch và phương sai thay đổi ra sao

Để thấy đánh đổi trên số liệu, chương dùng một mô phỏng giả định (không phải số đo từ paper). Thiết lập: MDP ngẫu nhiên 8 trạng thái, 2 hành động, $\gamma = 0{,}95$, thưởng có nhiễu chuẩn độ lệch 1, chính sách chọn đều hai hành động. Critic là giá trị thật cộng một sai số ngẫu nhiên cố định theo từng trạng thái với độ lệch chuẩn $2{,}0$ (giả định mô phỏng một critic chưa tốt). Với mỗi cặp (trạng thái, hành động) ta lấy 2000 rollout, tính GAE tại bước đầu. Thiên lệch được tính chính xác bằng kỳ vọng, trên **chênh lệch advantage giữa hai hành động** (phần trừ một hằng số theo trạng thái không ảnh hưởng gradient, đúng như baseline), và phương sai lấy từ rollout.



Hình 12: Thiên lệch và phương sai của GAE theo λ trong một mô phỏng giả định (minh hoạ, tự tính)

Cách đọc hình: bên trái, đường đỏ (trục trái) là thiên lệch, giảm gần tuyến tính từ $0{,}325$ tại $\lambda = 0$ về đúng 0 tại $\lambda = 1$; đường xanh (trục phải) là độ lệch chuẩn của ước lượng, tăng rất nhanh khi λ tiến gần 1 (từ $1{,}64$ tại $\lambda = 0{,}5$ lên $4{,}31$ tại $\lambda = 1$, tự tính). Bên phải là tổng thiên lệch bình phương cộng phương sai: có cực tiểu ở khoảng giữa ($\lambda \approx 0{,}5$ với số $2{,}71$, so với $4{,}08$ tại $\lambda = 0$ và $18{,}6$ tại $\lambda = 1$, tự tính).

Hai điều cần nói thật. Thứ nhất, ở λ nhỏ phương sai không thấp nhất tại $\lambda = 0$ mà hơi giảm khi λ tăng tới khoảng $0{,}45$ rồi mới tăng; suy luận của chương: do sai số critic tại các trạng thái ngẫu nhiên thêm vào nhiễu, và trung bình nhiều sai số TD làm nhiễu ấy giảm đi. Thứ hai, vị trí cực tiểu phụ thuộc rất nhiều vào chất lượng critic và nhiễu thưởng: critic càng tốt thì λ tối ưu càng nhỏ, và quỹ đạo càng nhiễu thì λ tối ưu càng

nhỏ. Hình chỉ minh họa dạng đường cong, không cho con số λ dùng được cho bài toán khác.

11. Nối sang các bài paper của cuốn

- **rl 05 (GAE).** Bài này chính thức hóa họ GAE(γ, λ) và nhấn mạnh γ, λ khác nhau. Hãy đọc kèm mục 6 và hình mô phỏng ở đây để thấy vì sao nó được gọi là "thanh trượt thiên lệch phương sai".
- **rl 06 (A3C).** Dùng advantage n bước với chung mạng (mục 5 và 8). Khác biệt "bất đồng bộ nhiều luồng" nằm ngoài chương này.
- **rl 10, 11, 12 (DDPG, TD3, SAC).** Critic học $Q(s, a)$ thay vì $V(s)$, nhưng các lo ngại về thiên lệch do bootstrapping và sai số critic ở mục 4, 5 vẫn đúng, và TD3 (rl 11) nhắm vào thiên lệch overestimation của critic Q.
- **rl 07 (PPO), rl 08, rl 09.** PPO dùng GAE rút gọn trên đoạn rollout hữu hạn làm advantage. Hai bài 08, 09 kiểm tra chi tiết như chuẩn hóa advantage, value clipping, λ trong ngữ cảnh PPO; số liệu và kết luận chi tiết có ở các ghi chú đó, nên chương này không nhắc lại số cụ thể.
- **rl 36 (GRPO).** Bỏ critic bằng cách ước lượng baseline từ nhóm mẫu của cùng một prompt; đọc xong chương này bạn thấy rõ cái được (không cần mạng giá trị) và cái mất (không có bootstrapping từng token).

12. Sai lầm hay gặp

- **Coi $\hat{A}_t$ có cùng ý nghĩa dù critic tệ.** Advantage một bước phản ánh sai số critic trực tiếp; critic chưa học thì advantage gần như vô nghĩa.
- **Hiểu λ là hệ số chiết khấu.** λ không đổi mục tiêu, chỉ đổi bộ ước lượng; γ mới đổi mục tiêu.
- **Quên bootstrap khi quỹ đạo bị cắt.** Gán $V = 0$ cho trạng thái cuối của một đoạn rollout chưa kết thúc làm advantage sai ở sát biên.
- **Để gradient của advantage lan vào critic khi cập nhật actor.** $\hat{A}_t$ phải là hằng số đối với actor; lan gradient làm actor kéo critic theo.
- **Chọn c_v và thang thưởng tùy tiện khi chung thân.** Loss critic cỡ lớn có thể nuốt gradient actor.
- **Cho rằng $\lambda = 1$ "an toàn nhất vì không chệch".** Không chệch nhưng phương sai có thể lớn tới mức học chậm, hình mô phỏng cho thấy phương sai tăng vọt gần $\lambda = 1$.

13. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
Sai số TD	$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$	Advantage một bước
Advantage n bước	$\sum_{l<n} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t)$	n lớn: ít chệch, nhiều phương sai
GAE	$\sum_{l\geq 0} (\gamma\lambda)^l \delta_{t+l}$	$\lambda = 0$: một bước; $\lambda = 1$: Monte Carlo trừ baseline
Đệ quy GAE	$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$	Tính ngược từ cuối rollout
Đích critic	$\hat{V}_t^{\text{targ}} = \hat{A}_t + V(s_t)$	λ-return
Tổng loss chung thân	$L_{\text{actor}} + c_v L_{\text{critic}} - c_e H$	Cân bằng bằng c_v, c_e

14. Bài tập tự tính

Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Sai số TD

Cho $\gamma = 0{,}9$, $r_t = 1$, $V(s_t) = 2$, $V(s_{t+1}) = 3$. Tính δ_t.

Đáp án (tự tính): $\delta_t = 1 + 0{,}9 \cdot 3 - 2 = 1{,}7$.

Bài 2. Advantage hai bước

Cho $\gamma = 0{,}9$, $r_t = 1$, $r_{t+1} = 2$, $V(s_t) = 2$, $V(s_{t+2}) = 3$. Tính $\hat{A}_t^{(2)}$.

Đáp án (tự tính): $1 + 0{,}9 \cdot 2 + 0{,}81 \cdot 3 - 2 = 3{,}23$.

Bài 3. GAE với ba sai số TD

Một đoạn kết thúc sau 3 bước có $\delta = (1,\ -2,\ 3)$ và $\gamma\lambda = 0{,}5$. Tính $\hat{A}_1, \hat{A}_2, \hat{A}_3$.

Đáp án (tự tính): từ cuối: $\hat{A}_3 = 3$; $\hat{A}_2 = -2 + 0{,}5 \cdot 3 = -0{,}5$; $\hat{A}_1 = 1 + 0{,}5 \cdot (-0{,}5) = 0{,}75$.

Bài 4. Co lại khi $\lambda = 1$

Cho $\gamma = 1$, $\lambda = 1$, thưởng $1, 2, 3$, kết thúc sau bước 3, $V = (4, 3, 1)$. Tính $\hat{A}_1$ bằng hai cách: qua δ và qua return Monte Carlo trừ baseline.

Đáp án (tự tính): $\delta = (1+3-4,\ 2+1-3,\ 3+0-1) = (0, 0, 2)$, tổng $= 2$. Return Monte Carlo $= 6$, trừ $V(s_1) = 4$ cho 2. Hai cách trùng nhau.

Bài 5. Độ dài nhìn xa hiệu dụng

Với $\gamma = 0{,}99$, $\lambda = 0{,}95$, tính $1/(1-\gamma\lambda)$ và trọng số của δ_{t+10} trong GAE. So sánh với trường hợp $\lambda = 0{,}5$.

Đáp án (tự tính): $\gamma\lambda = 0{,}9405$, $1/(1-\gamma\lambda) \approx 16{,}81$, trọng số tại $l = 10$ là $0{,}9405^{10} \approx 0{,}541$. Với $\lambda = 0{,}5$, $\gamma\lambda = 0{,}495$ và trọng số tại $l = 10$ chỉ còn $0{,}495^{10} \approx 9 \cdot 10^{-4}$ (tự tính): hầu như không còn nhìn xa.

Tài liệu tham khảo nền: Sutton và Barto, Reinforcement Learning: An Introduction (chương về TD, n-step, eligibility trace và actor-critic); Schulman và cộng sự, High-Dimensional Continuous Control Using Generalized Advantage Estimation (ghi chú rl 05); Mnih và cộng sự, Asynchronous Methods for Deep RL (ghi chú rl 06); các ghi chú rl 07, 08, 09 trong sách.

Nền tảng 05

Vùng tin cậy và PPO: đi những bước đủ nhỏ để dữ liệu cũ còn đáng tin

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: policy gradient ở nen03 của cuốn này; advantage và GAE ở nen04 của cuốn này; KL ở math nền tảng 11; gradient và hàm lồi ở math nền tảng 06, 14; SGD, learning rate ở optimization nền 02, 03

Dùng ở: rl 04 (TRPO), rl 07 (PPO), rl 08 (chi tiết cài đặt PPO và TRPO), rl 09 (điều gì quan trọng trong on-policy RL)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; hình mô phỏng dùng dữ liệu tự sinh với seed cố định và được nói rõ là giả định.

1. Vì sao cần vùng tin cậy

Vùng tin cậy (trust region) là ý tưởng chỉ cập nhật chính sách trong một vùng nhỏ quanh chính sách hiện tại, nơi dữ liệu vừa thu thập còn đáng tin. Nó giải quyết một bất tiện đặc thù của RL: dữ liệu để cập nhật chính sách được thu thập bằng chính sách này. Chính sách đổi thì dữ liệu mới thu thập cũng đổi theo. Nếu bước cập nhật quá lớn, chính sách mới có thể đi tới những vùng mà dữ liệu cũ không nói gì, và một chính sách tệ sẽ tạo ra dữ liệu tệ, khiến việc hồi phục khó hơn.

Hãy hình dung bạn tập đánh quần vợt và chỉ được xem video của chính mình. Sau một buổi tập, bạn chỉnh cú giao bóng rất mạnh dựa trên vài lần thành công. Nếu chỉnh vừa phải, video buổi tập sau vẫn có ích; nếu chỉnh quá tay, bạn đổi cả tư thế, video cũ không còn mô tả bạn nữa, và có thể bạn tự làm mình tệ đi mà chẳng biết quay lại thế nào.

Chương này đi từ động cơ (vì sao bước lớn làm sập), qua ràng buộc KL của TRPO ở mức ý tưởng, đến hàm mục tiêu cắt (clip) của PPO, vai trò của nhiều epoch trên cùng dữ liệu, và các chi tiết cài đặt hay quyết định kết quả. Đó là những thứ bạn cần để đọc các bài TRPO, PPO, và các bài RLHF cho mô hình ngôn ngữ, nơi PPO là bộ tối ưu nền.

2. Nhắc nền: tỉ lệ xác suất, mục tiêu thay thế và KL

2.1 Tỉ lệ xác suất

Gọi $\pi_{\theta_{\text{old}}}$ là chính sách đã thu thập dữ liệu (cố định khi cập nhật) và π_θ là chính sách đang được cập nhật. **Tỉ lệ xác suất** của một cặp (s_t, a_t) trong dữ liệu là

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

$\rho_t = 1$ nghĩa là chính sách chưa đổi tại điểm này; $\rho_t > 1$ nghĩa là hành động a_t giờ có xác suất cao hơn; $\rho_t < 1$ là thấp hơn. Trong cài đặt thực tế ta lưu $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ lúc thu dữ liệu rồi tính $\rho_t = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$ để ổn định số. Ví dụ (tự tính): $\log \pi_{\text{old}} = -1{,}20$ và $\log \pi_\theta = -1{,}05$ cho $\rho = e^{0{,}15} \approx 1{,}162$.

2.2 Mục tiêu thay thế

Từ nen03, gradient chính sách dùng $\nabla \log \pi \cdot \hat{A}$. Một mục tiêu có cùng gradient tại $\theta = \theta_{\text{old}}$ nhưng cho phép tối ưu nhiều bước trên dữ liệu cũ là **mục tiêu thay thế** (surrogate)

$$L(\theta) = \mathbb{E}_t\big[\rho_t(\theta)\,\hat{A}_t\big].$$

Lý do: $\nabla_\theta \rho_t = \rho_t \nabla_\theta \log \pi_\theta$ và tại θ_{old} thì $\rho_t = 1$, ta khôi phục đúng gradient chuẩn. Về bản chất, ρ_t là một **trọng số importance sampling** một bước: dữ liệu được lấy theo $\pi_{\theta_{\text{old}}}$ nhưng ta muốn kỳ vọng theo π_θ. Importance sampling cho kỳ vọng không chệch nhưng phương sai tăng khi hai chính sách lệch nhau, đây là điểm then chốt: $L(\theta)$ chỉ là xấp xỉ tốt khi π_θ gần $\pi_{\theta_{\text{old}}}$. Chương nen06 của cuốn này nói kỹ hơn về importance sampling.

2.3 KL giữa hai chính sách

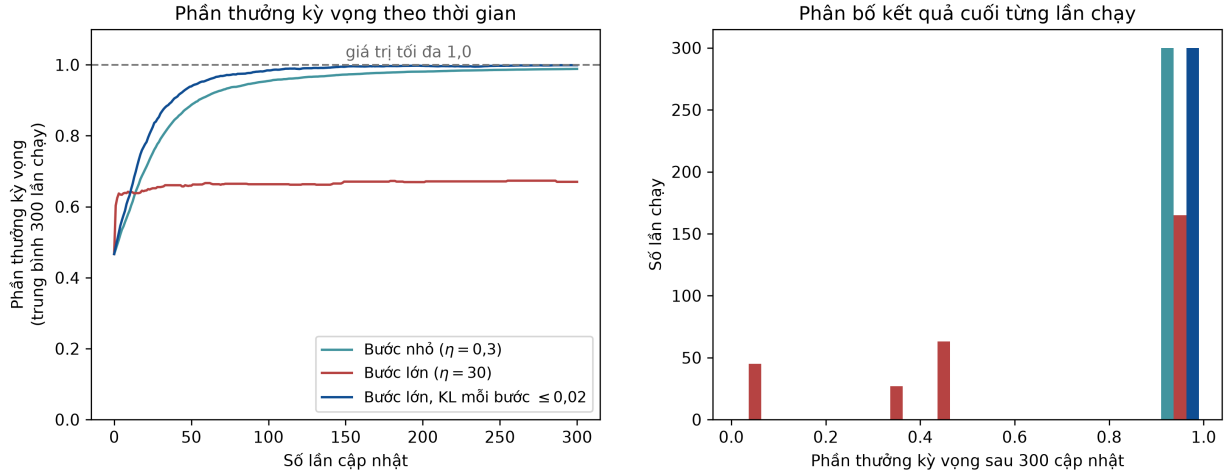
Để đo "gần nhau" ta dùng KL (math nền tảng 11). Với hai chính sách rời rạc tại cùng một trạng thái, $\mathrm{KL}(\pi_{\text{old}} \| \pi_{\text{new}}) = \sum_a \pi_{\text{old}}(a) \log \frac{\pi_{\text{old}}(a)}{\pi_{\text{new}}(a)}$. Ví dụ (tự tính), hai hành động, chính sách cũ $(0{,}5;\ 0{,}5)$:

- Mới $(0{,}6;\ 0{,}4)$: $\mathrm{KL} = 0{,}5 \ln \frac{0{,}5}{0{,}6} + 0{,}5 \ln \frac{0{,}5}{0{,}4} \approx 0{,}0204$.
- Mới $(0{,}9;\ 0{,}1)$: $\mathrm{KL} \approx 0{,}5108$.

Nhảy từ $0{,}5$ lên $0{,}9$ có KL gấp khoảng 25 lần so với nhảy lên $0{,}6$: KL tăng nhanh hơn tuyến tính theo độ dịch chuyển (xấp xỉ bậc hai khi dịch chuyển nhỏ). Đó là lý do KL là thước đo tự nhiên của "bước dài hay ngắn" trên không gian chính sách.

3. Trực giác bằng hình: bước lớn làm sập chính sách

Để thấy vấn đề, chương dùng một mô phỏng giả định rất nhỏ: bandit 3 cần với thưởng trung bình $(1{,}0;\ 0{,}4;\ 0)$, thưởng nhiễu chuẩn độ lệch $1{,}5$. Chính sách là softmax trên 3 logit, khởi đầu đều (thưởng kỳ vọng ban đầu $\approx 0{,}467$, tự tính). Mỗi lần cập nhật dùng một lô chỉ 4 mẫu, ước lượng policy gradient với baseline là trung bình thưởng của lô, rồi cộng η nhân gradient vào các logit. Ta so ba cách: bước nhỏ ($\eta = 0{,}3$), bước lớn ($\eta = 30$), và bước lớn nhưng mỗi bước bị thu nhỏ lại sao cho $\mathrm{KL}(\pi_{\text{old}} \| \pi_{\text{new}}) \le 0{,}02$.



Hình 13: Bước cập nhật lớn làm chính sách sập, còn bước lớn bị chặn bằng KL thì không, theo mô phỏng giả định (minh hoạ, tự tính)

Cách đọc hình: bên trái, mỗi đường là trung bình của 300 lần chạy độc lập (thưởng kỳ vọng của chính sách theo số lần cập nhật). Đường teal (bước nhỏ) leo chậm rồi tiến sát mức tối đa $1{,}0$; đường xanh (bước lớn có chặn KL) leo nhanh hơn và cũng tới gần mức tối đa; đường đỏ (bước lớn không chặn) tăng vọt rồi **dừng ở khoảng** $0{,}67$. Bên phải là phân bố kết quả cuối của từng lần chạy: với bước lớn không chặn, các lần chạy tách thành ba cụm: một nhóm chạm gần 1, một nhóm kẹt quanh $0{,}4$ và một nhóm nhỏ kẹt gần 0; tổng cộng

khoảng 45% kẹt ở phần thưởng kỳ vọng dưới $0{,}5$, tương ứng chính sách đã gần như tất định vào cần sai (tự tính từ mô phỏng).

Cơ chế (suy luận từ mô phỏng này): một lô nhỏ nhiễu có thể tình cờ cho thấy cần sai "tốt hơn". Bước lớn đẩy gần hết xác suất sang cần đó. Khi xác suất của các cần khác gần 0, gần như không còn mẫu nào của chúng, gradient kỳ vọng cũng $\propto$ xác suất nên tiến về 0, và chính sách mắc kẹt. Với bước nhỏ, lỗi của một lô bị lô sau sửa. Với bước lớn bị chặn KL, ta vẫn tiến nhanh khi hướng đúng nhưng mỗi bước chỉ làm phân phối đổi một lượng nhỏ, và một lô xấu không đủ sức phá hủy chính sách. Đây là kịch bản đơn giản hơn nhiều so với mạng sâu, nhưng nó cho thấy đúng động lực của vùng tin cậy: **hiệu quả của bước cập nhật phải đo bằng khoảng cách trên không gian chính sách, không phải trên không gian tham số.**

Vì sao đo trên không gian chính sách? Cùng một bước $\Delta\theta$ trong tham số có thể đổi chính sách rất ít hoặc rất nhiều tùy trạng thái và mạng. Một learning rate cố định không bảo đảm KL cố định.

4. Vùng tin cậy và ràng buộc KL (TRPO ở mức ý tưởng)

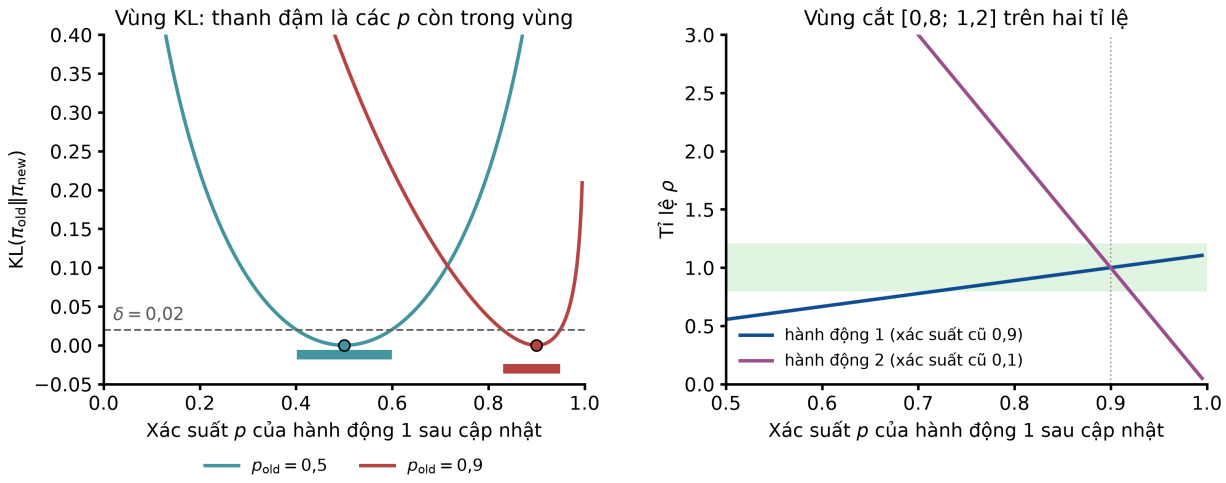
4.1 Bài toán

Ý tưởng của **TRPO** (rl 04): trong mỗi vòng, thay vì tối ưu thẳng $L(\theta)$, giải

$$\max_{\theta} \; L(\theta) \quad \text{với} \quad \mathbb{E}_s\big[\mathrm{KL}(\pi_{\theta_{\text{old}}}(\cdot \mid s) \| \pi_\theta(\cdot \mid s))\big] \le \delta.$$

δ là bán kính của vùng tin cậy, một số nhỏ cố định (ghi chú rl 08 liệt kê các giá trị dùng cho TRPO ở từng môi trường: $0{,}04$, $0{,}07$ và $0{,}13$). Lý thuyết đi kèm của TRPO (theo ghi chú rl 04) chỉ ra một cận dưới trên hiệu năng thật gồm $L(\theta)$ trừ một phạt tỉ lệ với KL, từ đó có khẳng định cải thiện đơn điệu cho bước đủ nhỏ. Cài đặt thực tế phải xấp xỉ: dùng xấp xỉ bậc hai của KL (ma trận Fisher), giải bằng conjugate gradient và kết thúc bằng tìm kiếm tuyến tính để thực sự thỏa ràng buộc. Chương không đi sâu phần này; điều cần nhớ là nó **chính xác nhưng đắt và phức tạp** (cần tích Hessian-vector, khó kết hợp với một số kiến trúc, ví dụ khi chung thân hoặc dùng dropout).

4.2 Vùng tin cậy trên chính sách hai hành động



Hình 14: Vùng KL tương ứng với bán kính cố định (trái) và vùng cắt tỉ lệ của PPO trên hai tỉ lệ (phải) với chính sách hai hành động (minh hoạ, tự tính)

Cách đọc hình: bên trái, trục ngang là xác suất p mà chính sách mới gán cho hành động 1; hai parabol là KL từ chính sách cũ với $p_{\text{old}} = 0{,}5$ (teal) và $p_{\text{old}} = 0{,}9$ (đỏ). Đường nét đứt là bán kính $\delta = 0{,}02$, các thanh

đậm bên dưới là các p còn nằm trong vùng. Với $p_{\text{old}} = 0{,}5$ vùng là khoảng $[0{,}401;\ 0{,}599]$, với $p_{\text{old}} = 0{,}9$ là khoảng $[0{,}83;\ 0{,}949]$ (tự tính). Vùng KL không đối xứng và hẹp hơn khi chính sách sắp tất định: chênh lệch xác suất nhỏ ở gần 1 đã là một thay đổi lớn về thông tin.

Bên phải là chính sách cũ $(0{,}9;\ 0{,}1)$ và vùng cắt $[0{,}8;\ 1{,}2]$ của PPO (mục 5) trên hai tỉ lệ. Tỉ lệ của hành động 1 chỉ chạy từ $0{,}56$ đến $1{,}11$ ở trục hiển thị, nên không bao giờ chạm biên trên $1{,}2$ ($p/0{,}9 \le 1/0{,}9 \approx 1{,}111$). Tỉ lệ của hành động hiếm (xác suất cũ $0{,}1$) thay đổi rất nhanh: chỉ cần p tăng lên $0{,}92$, tỉ lệ của hành động 2 đã chạm $0{,}8$ (tự tính). Đây là ví dụ cho thấy vùng cắt trên tỉ lệ và vùng KL không trùng nhau: tỉ lệ của hành động hiếm nhạy hơn với thay đổi xác suất tuyệt đối nhỏ.

5. PPO: tỉ lệ và cắt thay cho ràng buộc cứng

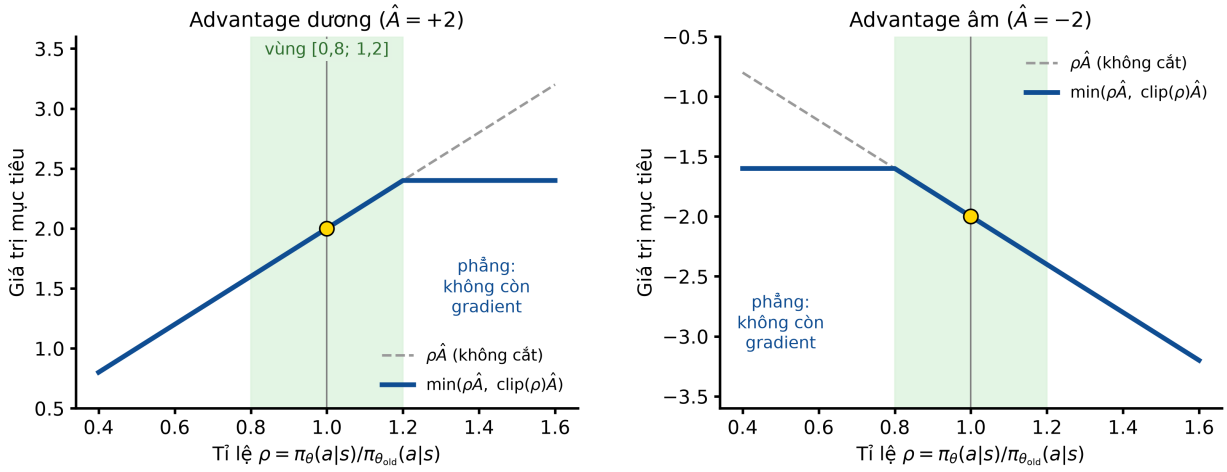
5.1 Định nghĩa

PPO (rl 07, bản clip) giữ nguyên mục tiêu thay thế nhưng cắt tỉ lệ vào $[1-\epsilon, 1+\epsilon]$ và lấy giá trị nhỏ hơn:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\ \min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t,\ 1-\epsilon,\ 1+\epsilon)\ \hat{A}_t\big)\Big].$$

Ở đây $\text{clip}(x, a, b)$ ép x về đoạn $[a, b]$. Giá trị mặc định thường nêu là $\epsilon = 0{,}2$ (theo ghi chú rl 07). Phép min làm cho L^{CLIP} là một **cận dưới bi quan** của mục tiêu không cắt: nó chỉ cắt bớt phần lợi nhuận khi di chuyển quá xa, không cắt phần thua lỗ.

5.2 Hình dạng hàm theo dấu advantage



Hình 15: Hàm mục tiêu clip theo tỉ lệ với advantage dương (trái) và âm (phải), $\epsilon = 0{,}2$ (minh hoạ, tự tính)

Cách đọc hình: đường nét đứt xám là $\rho\hat{A}$ không cắt, đường xanh dày là mục tiêu PPO, vùng xanh nhạt là $[0{,}8;\ 1{,}2]$, chấm vàng là điểm xuất phát $\rho = 1$. **Bên trái ($\hat{A} > 0$):** tăng ρ (nâng xác suất hành động tốt) có lợi, nhưng khi $\rho > 1{,}2$ đường xanh phẳng, nên không còn gradient thưởng cho việc đẩy xa hơn. Giảm ρ xuống dưới 1 làm mục tiêu giảm, và đường xanh trùng nét đứt: gradient vẫn kéo ρ trở lại. **Bên phải ($\hat{A} < 0$):** giảm ρ (hạ xác suất hành động xấu) có lợi nhưng phẳng khi $\rho < 0{,}8$; tăng ρ lên trên $1{,}2$ làm mục tiêu tệ hơn vẫn tiếp tục xuống, nên gradient vẫn kéo trở lại.

Bốn trường hợp tóm lại:

Dấu $\hat{A}$	Vùng ρ	Giá trị	Gradient
> 0	$\rho \le 1+\epsilon$	$\rho\hat{A}$	Có, đẩy ρ lên

Dấu $\hat{A}$	Vùng ρ	Giá trị	Gradient
> 0	$\rho > 1+\epsilon$	$(1+\epsilon)\hat{A}$	Bằng 0
< 0	$\rho \geq 1-\epsilon$	$\rho\hat{A}$	Có, kéo ρ xuống
< 0	$\rho < 1-\epsilon$	$(1-\epsilon)\hat{A}$	Bằng 0

Chú ý quan trọng: phép cắt **không tạo hàng rào**. Gradient bằng 0 khi ρ đã vượt biên theo hướng "có lợi", nhưng các mẫu khác trong cùng lô vẫn có thể kéo θ đi tiếp, và ρ của mẫu đã bị cắt vẫn có thể trôi xa hơn nữa. Theo ghi chú rl 08, đo trên Humanoid-v2 cho thấy cả PPO lẫn các biến thể đều vi phạm vùng tỉ lệ $[1-\epsilon, 1+\epsilon]$ nhiều lần, dù được huấn luyện với chính mục tiêu cắt đó. Vì vậy "PPO là vùng tin cậy" chỉ đúng ở nghĩa xấp xỉ về ý tưởng, không phải một bảo đảm.

6. Ví dụ tay: tính mục tiêu clip

Cho $\epsilon = 0{,}2$, nên biên cắt là $0{,}8$ và $1{,}2$. Tính $\min(\rho\hat{A},\ \text{clip}(\rho)\hat{A})$ (tự tính).

Với $\hat{A} = +2$:

ρ	$\rho\hat{A}$	$\text{clip}(\rho)\hat{A}$	Mục tiêu	Gradient
0,5	1,0	1,6	1,0	Có
0,9	1,8	1,8	1,8	Có
1,0	2,0	2,0	2,0	Có
1,1	2,2	2,2	2,2	Có
1,3	2,6	2,4	2,4	Không
1,5	3,0	2,4	2,4	Không

Với $\hat{A} = -2$:

ρ	$\rho\hat{A}$	$\text{clip}(\rho)\hat{A}$	Mục tiêu	Gradient
0,5	$-1{,}0$	$-1{,}6$	$-1{,}6$	Không
0,7	$-1{,}4$	$-1{,}6$	$-1{,}6$	Không
0,9	$-1{,}8$	$-1{,}8$	$-1{,}8$	Có
1,0	$-2{,}0$	$-2{,}0$	$-2{,}0$	Có
1,1	$-2{,}2$	$-2{,}2$	$-2{,}2$	Có
1,3	$-2{,}6$	$-2{,}4$	$-2{,}6$	Có
1,5	$-3{,}0$	$-2{,}4$	$-3{,}0$	Có

Hai dòng đáng chú ý. Ở $\hat{A} = +2$, $\rho = 0{,}5$: hành động tốt nhưng chính sách mới lại hạ xác suất nó xuống một nửa; mục tiêu lấy giá trị không cắt $1{,}0$ (thấp hơn $1{,}6$), nên gradient vẫn kéo ρ lên, đúng cơ chế sửa lỗi. Ở $\hat{A} = -2$, $\rho = 1{,}3$: hành động xấu nhưng xác suất đã tăng; mục tiêu lấy giá trị $-2{,}6$ (tệ hơn $-2{,}4$), gradient vẫn kéo ρ xuống.

Một lô ba mẫu. Cho $\rho = (0{,}7;\ 1{,}1;\ 1{,}5)$ và $\hat{A} = (+1;\ -2;\ +0{,}5)$ (tự tính):

- Mẫu 1: $\min(0{,}7;\ 0{,}8) = 0{,}7$, gradient có (đang ở vùng không cắt).
- Mẫu 2: $1{,}1 \cdot (-2) = -2{,}2$, gradient có.
- Mẫu 3: $\min(0{,}75;\ 1{,}2 \cdot 0{,}5 = 0{,}6) = 0{,}6$, gradient không (bị cắt).

Trung bình của lô: $(0{,}7 - 2{,}2 + 0{,}6)/3 = -0{,}3$. Chỉ hai trong ba mẫu đóng góp gradient. Chú ý giá trị trung bình âm không có nghĩa chính sách xấu đi: L^{CLIP} đo mục tiêu thay thế tương đối so với $\hat{A}$, và gradient mới quan trọng.

Gradient của mẫu đang hoạt động: $\nabla_\theta(\rho\hat{A}) = \hat{A}\,\rho\,\nabla_\theta \log \pi_\theta(a \mid s)$, tức policy gradient quen thuộc nhưng nhân thêm ρ.

7. Nhiều epoch trên cùng dữ liệu

Chính sách thu thập dữ liệu khá đắt (chạy môi trường hoặc sinh văn bản), nên PPO dùng lại cùng một lô rollout cho **nhiều epoch** cập nhật bằng minibatch. Quy trình: thu thập $N \times T$ chuyển tiếp bằng $\pi_{\theta_{\text{old}}}$, tính $\hat{A}_t$ (GAE, nen04) và $\log \pi_{\theta_{\text{old}}}$ một lần; rồi K epoch, mỗi epoch xáo trộn và chia minibatch, cập nhật θ bằng Adam để tối đa L^{CLIP}; xong đặt $\theta_{\text{old}} \leftarrow \theta$ và thu thập lại.

- **Minibatch đầu của epoch đầu tiên**: $\theta = \theta_{\text{old}}$, mọi $\rho_t = 1$, mục tiêu không cắt và bước cập nhật y hệt policy gradient thường.
- **Các epoch sau**: ρ_t rời 1, một số mẫu bị cắt, gradient của chúng biến mất. Đây là cơ chế "tự phanh": càng dùng lâu, càng nhiều mẫu bị cắt, gradient tổng càng nhỏ.
- **Rủi ro**: dùng lại quá nhiều epoch (hoặc ϵ quá lớn, learning rate lớn) khiến π_θ lệch khỏi $\pi_{\theta_{\text{old}}}$ dù có cắt, và lô dữ liệu cũ trở nên lạc hậu. Phần suy luận trong ghi chú rl 07 nhận xét rằng với mô hình ngôn ngữ, sinh rollout đắt hơn nhiều nên có áp lực tăng K, cần cân nhắc cùng độ an toàn của clip.
- **Kiểm soát**: nhiều cài đặt đo KL ước lượng giữa chính sách mới và cũ sau mỗi epoch rồi **dừng sớm** khi KL vượt một ngưỡng; cũng có thể dùng KL làm tín hiệu giám sát chính (nếu KL tăng đột ngột là dấu hiệu bất ổn).

8. Chi tiết cài đặt hay quyết định kết quả

Ghi chú rl 08 làm một ablation cẩn thận và kết luận rằng nhiều "mẹo cài đặt" không nằm trong mô tả toán học của PPO lại có vai trò lớn. Danh sách sau lấy từ các ghi chú rl 07, 08, 09, 28; chương này chỉ nêu loại mẹo, số liệu chi tiết xem ghi chú tương ứng.

- **Chuẩn hóa advantage theo minibatch.** Trừ trung bình, chia độ lệch chuẩn. Theo ghi chú rl 09, ảnh hưởng ít trong thí nghiệm của paper đó.
- **Chuẩn hóa quan sát (observation normalization).** Theo ghi chú rl 09, nên luôn dùng. Theo ghi chú rl 08, nằm trong danh sách các code-level optimization.
- **Chuẩn hóa thưởng (reward scaling).** Chia thưởng cho độ lệch chuẩn của tổng chiết khấu chạy. Theo ghi chú rl 08, cùng với giảm learning rate Adam và khởi tạo mạng, đây là các yếu tố ảnh hưởng rõ đến kết quả trong ablation của họ.
- **Khởi tạo trực giao và scale lớp ra nhỏ.** Theo ghi chú rl 08.
- **Giảm dần learning rate của Adam (annealing).** Theo ghi chú rl 08.
- **Cắt gradient theo chuẩn toàn cục.** Theo ghi chú rl 08, ngưỡng $0{,}5$ trong cài đặt của họ; theo ghi chú rl 09, giúp nhẹ và ngưỡng cụ thể không quan trọng.
- **Cắt giá trị của value function (value clipping).** Theo ghi chú rl 08 có trong cài đặt chuẩn, nhưng theo ghi chú rl 09, value loss clipping kiểu PPO làm giảm hiệu năng, bất kể ngưỡng. Hai paper đưa ra kết luận khác nhau về cùng một mẹo.
- **Hệ số entropy và hệ số critic** trong tổng loss: giữ chính sách đủ ngẫu nhiên và cân bằng gradient, xem mục 8 của nen04.
- **Cắt không đối xứng ("clip-higher").** Một số phương pháp cho $\epsilon_{\text{low}} \neq \epsilon_{\text{high}}$ để chống sụp entropy khi huấn luyện LLM (rl 41, DAPO).
- **Tính tỉ lệ trên từng token (mô hình ngôn ngữ).** Với LLM, mỗi token là một hành động, ρ_t tính theo từng token, và loss được cộng hoặc lấy trung bình theo token hay theo chuỗi; cách gộp này ảnh hưởng

đến độ lệch (rl 41, 42, 43 bàn kỹ).

- **Tắt dropout.** Theo ghi chú rl 28, dropout làm log-prob không tái lập, hỏng KL và ratio PPO ngay ở epoch đầu.

Bài học tổng quát từ rl 08: khi so sánh hai thuật toán, phải bảo đảm cả hai dùng cùng mức tối ưu hóa cài đặt, nếu không dễ nhầm "code tốt hơn" thành "thuật toán tốt hơn".

9. Biến thể: PPO phạt KL và các cách cắt khác

PPO còn một biến thể **phạt KL** thích ứng: tối đa hóa $\mathbb{E}[\rho_t \hat{A}_t - \beta\,\mathrm{KL}]$ và chỉnh β tăng hoặc giảm tùy KL đo được lớn hay nhỏ hơn một mục tiêu d_{targ}. Theo ghi chú rl 07, trong thí nghiệm của paper biến thể này kém bản clip, nên chủ yếu được giữ làm baseline. Tuy vậy, ý tưởng "phạt KL tới chính sách tham chiếu" lại rất quan trọng trong RLHF (một hệ số β giữ chính sách gần mô hình khởi đầu); đừng nhầm đó với KL giữa chính sách mới và cũ trong một vòng cập nhật.

Hai loại KL hay bị lẫn:

- **KL giữa π_θ và $\pi_{\theta_{\text{old}}}$** (trong một vòng cập nhật): giữ bước cập nhật nhỏ, là nội dung chương này.
- **KL giữa π_θ và một chính sách tham chiếu cố định** (ví dụ mô hình đã tinh chỉnh có giám sát): giữ chính sách không trôi xa khỏi hành vi ban đầu, là nội dung của các bài RLHF.

10. Nối sang các bài paper của cuốn

- **rl 04 (TRPO).** Là nơi ràng buộc KL được xây dựng bằng lý thuyết và thuật toán. Hãy đọc mục 4 của chương này trước để biết bài toán $\max L(\theta)$ với $\mathrm{KL} \le \delta$ rồi mới vào phần chứng minh và conjugate gradient.
- **rl 07 (PPO).** Mục 5 đến 7 của chương này là bản dài hơn của hàm L^{CLIP}, bốn trường hợp và vai trò nhiều epoch. Theo ghi chú rl 07, động lực của thiết kế clip là làm cho nhiều epoch SGD trên cùng lô an toàn hơn, và PPO không có định lý cải thiện đơn điệu như TRPO.
- **rl 08.** Ablation chỉ ra các code-level optimization và việc ràng buộc tỉ lệ bị vi phạm; mục 5.2 và 8 của chương này tóm tắt ý chính.
- **rl 09.** Một khảo sát quy mô lớn các lựa chọn on-policy; ghi chú khuyến nghị dùng PPO policy loss, ngưỡng clip bắt đầu $0{,}25$ và thử thêm, theo ghi chú rl 09.
- **Các bài RLHF và RLVR (rl 28, 36, 41, 42, 43).** PPO là bộ tối ưu nền; GRPO (rl 36) giữ phần cắt tỉ lệ nhưng bỏ critic.

11. Sai lầm hay gặp

- **Tin rằng clip bảo đảm $\rho \in [1-\epsilon, 1+\epsilon]$.** Nó chỉ triệt gradient của các mẫu đã vượt; tỉ lệ vẫn có thể trôi ra ngoài.
- **Nhầm bước nhỏ trong tham số với bước nhỏ trên chính sách.** Cùng $\Delta\theta$ có thể cho KL rất khác nhau.
- **Quên rằng ở bước đầu mọi $\rho = 1$.** Nếu thấy không mẫu nào bị cắt ở minibatch đầu của epoch đầu thì hoàn toàn bình thường.
- **Tính $\log \pi_{\text{old}}$ lại sau khi cập nhật.** Phải lưu giá trị lúc thu dữ liệu, nếu không ρ luôn bằng 1 và mục tiêu mất ý nghĩa.
- **Lẫn hai loại KL** (trong vòng cập nhật so với tới chính sách tham chiếu).
- **Đổi ϵ, số epoch và learning rate độc lập.** Ba thứ này cùng quyết định mức lệch chính sách; đổi một mình có thể làm các thứ khác mất tác dụng.

12. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
Tỉ lệ	$\rho_t = \pi_\theta(a_t\|s_t)/\pi_{\text{old}}(a_t\|s_t)$	Tính từ log-prob đã lưu
Mục tiêu thay thế	$L = \mathbb{E}[\rho_t \hat{A}_t]$	Chính xác chỉ khi π_θ gần π_{old}
Ràng buộc TRPO	$\mathbb{E}_s[\text{KL}(\pi_{\text{old}}\|\pi_\theta)] \le \delta$	Vùng tin cậy
PPO clip	$\min(\rho\hat{A},\ \text{clip}(\rho, 1\mp\epsilon)\hat{A})$	Cận dưới bi quan
Gradient bằng 0	$\hat{A}>0,\ \rho>1+\epsilon$ hoặc $\hat{A}<0,\ \rho<1-\epsilon$	Mẫu đã "đủ"
Phạt KL thích ứng	$\rho\hat{A} - \beta\,\text{KL}$	β chỉnh theo KL đo được

13. Bài tập tự tính

Mỗi đáp án đã kiểm bằng numpy. Trừ khi nói khác, $\epsilon = 0{,}2$.

Bài 1. Clip với advantage dương

Cho $\rho = 1{,}3$, $\hat{A} = +1{,}5$. Tính mục tiêu và nói gradient có bằng 0 không.

Đáp án (tự tính): $\min(1{,}95;\ 1{,}2 \cdot 1{,}5 = 1{,}8) = 1{,}8$; gradient bằng 0 vì đã chọn giá trị cắt.

Bài 2. Clip với advantage âm

Cho $\rho = 1{,}3$, $\hat{A} = -1{,}5$.

Đáp án (tự tính): $\min(-1{,}95;\ -1{,}8) = -1{,}95$; gradient khác 0 vì giá trị không cắt tệ hơn được chọn.

Bài 3. Tỉ lệ từ log-prob

$\log \pi_{\text{old}} = -2{,}0$, $\log \pi_\theta = -1{,}7$, $\hat{A} = +1$. Tính ρ và mục tiêu.

Đáp án (tự tính): $\rho = e^{0{,}3} \approx 1{,}350$, mục tiêu $= \min(1{,}350;\ 1{,}2) = 1{,}2$.

Bài 4. KL của hai chính sách

Chính sách cũ $(0{,}7;\ 0{,}3)$, mới $(0{,}5;\ 0{,}5)$. Tính $\text{KL}(\pi_{\text{old}}\|\pi_{\text{new}})$.

Đáp án (tự tính): $0{,}7 \ln \frac{0{,}7}{0{,}5} + 0{,}3 \ln \frac{0{,}3}{0{,}5} \approx 0{,}0823$.

Bài 5. Lô ba mẫu

$\rho = (0{,}6;\ 1{,}0;\ 1{,}4)$, $\hat{A} = (-1;\ +2;\ +1)$. Tính trung bình mục tiêu clip và đếm mẫu còn gradient.

Đáp án (tự tính): mẫu 1: $\min(-0{,}6;\ -0{,}8) = -0{,}8$ (cắt, không gradient); mẫu 2: 2 (có); mẫu 3: $\min(1{,}4;\ 1{,}2) = 1{,}2$ (cắt). Trung bình $= (-0{,}8 + 2 + 1{,}2)/3 = 0{,}8$. Chỉ một mẫu còn gradient.

Tài liệu tham khảo nền: Schulman và cộng sự, Trust Region Policy Optimization (ghi chú rl 04); Schulman và cộng sự, Proximal Policy Optimization Algorithms (ghi chú rl 07); Engstrom và cộng sự, Implementation Matters in Deep Policy Gradients (ghi chú rl 08); Andrychowicz và cộng sự, What Matters in On-Policy RL (ghi chú rl 09); Huang và cộng sự, N+ Implementation Details of RLHF with PPO (ghi chú rl 28).

Nền tảng 06

Khám phá, off-policy và offline RL: học từ dữ liệu không do chính sách đích sinh ra

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: MDP, Q-learning và bộ nhớ replay ở nen01, nen02 của cuốn này; policy gradient ở nen03 của cuốn này; nen04, nen05 của cuốn này (advantage, tỉ lệ xác suất); kỳ vọng và phương sai ở math nền tảng 08; KL ở math nền tảng 11

Dùng ở: rl 13 (khám phá), rl 17 (DAgger), rl 18 (GAIL), rl 19 (offline RL tutorial), rl 20 (CQL), rl 21 (Decision Transformer); và các bài off-policy: rl 01, rl 02, rl 03, rl 10, rl 11, rl 12

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình mô phỏng dùng dữ liệu tự sinh với seed cố định và được nói rõ là giả định.

1. Vì sao cần hiểu khám phá, on-policy và off-policy

Khám phá, off-policy và offline là ba mặt của cùng một câu hỏi: dữ liệu mà ta học từ đó đến từ đâu, và nó có phù hợp với điều ta muốn học không. Trong học có giám sát, dữ liệu cho trước và cố định. Trong RL thì dữ liệu do chính hành vi của tác nhân tạo ra: tác nhân không thử hành động nào thì mãi mãi không biết hành động đó tốt hay tệ.

Hãy hình dung bạn đến một thành phố mới và muốn tìm quán ăn ngon. Nếu ngày nào cũng ăn ở quán đầu tiên thấy ổn, bạn không bao giờ biết có quán ngon hơn (thiếu khám phá). Nếu ngày nào cũng thử quán mới, bạn không hưởng được quán tốt đã tìm ra (thiếu khai thác). Giả sử bạn không thể tự thử nữa mà chỉ có cuốn nhật ký ăn uống của một người bạn: bạn đọc để quyết định, nhưng nhật ký không nói gì về các quán bạn ấy chưa từng ghé, và bạn rất dễ tin rằng quán chưa ai đánh giá lại là tuyệt hảo (đó chính là offline RL với hành động ngoài dữ liệu).

Chương này đi qua: đánh đổi khám phá và khai thác; khác biệt on-policy với off-policy; importance sampling để sửa lệch phân phối và vì sao phương sai của nó nổ; replay buffer; offline RL và hiện tượng ngoại suy sai; ý tưởng bảo thủ (CQL); và bắt chước (imitation) ở mức ý tưởng. Các bài paper trong cuốn dựa trên những ý này: DQN và các biến thể dùng replay, DDPG, TD3, SAC là off-policy, PPO là on-policy, và nhóm bài 17 đến 21 xử lý bắt chước và offline.

2. Nhắc nền: kỳ vọng, Q-learning và phương sai

2.1 Kỳ vọng khi đổi phân phối

Nếu a lấy theo phân phối b nhưng ta muốn kỳ vọng của $f(a)$ theo phân phối π, ta không thể lấy trung bình f trực tiếp trên mẫu b. Ví dụ (tự tính): hai hành động, $b = (0{,}8;\ 0{,}2)$, $\pi = (0{,}4;\ 0{,}6)$, giá trị $f = (1;\ 3)$. Kỳ vọng thật theo π là $0{,}4 \cdot 1 + 0{,}6 \cdot 3 = 2{,}2$, còn trung bình theo b là $0{,}8 \cdot 1 + 0{,}2 \cdot 3 = 1{,}4$. Hai con số khác nhau vì b ưa chuộng hành động 1 (giá trị thấp) trong khi π ưa chuộng hành động 2. Đây là vấn đề trung tâm của mục 5.

2.2 Cập nhật Q-learning

Q-learning (nen02) cập nhật $Q(s,a)$ về đích $y = r + \gamma \max_{a'} Q(s',a')$. Có hai chi tiết cần nhớ. Thứ nhất, đích dùng max nên **không phụ thuộc vào hành động mà tác nhân thực sự chọn ở** s': nó học giá trị của chính sách tham lam, bất kể ai sinh ra dữ liệu. Đó là lý do Q-learning được xếp vào off-policy. Thứ hai, $Q(s',a')$

chỉ đáng tin tại các cặp (s', a') mà dữ liệu từng đi qua; nơi khác nó là giá trị khởi tạo hoặc kết quả ngoại suy của mạng.

3. Khám phá và khai thác

3.1 Đánh đổi

Khai thác (exploitation) là chọn hành động có giá trị ước lượng cao nhất. **Khám phá** (exploration) là chọn hành động khác để thu thông tin. Ước lượng giá trị càng không chắc chắn thì khám phá càng có ích, nhưng mỗi lần khám phá có chi phí cơ hội.

3.2 ϵ-greedy

Với xác suất $1 - \epsilon$ chọn hành động tham lam, với xác suất ϵ chọn ngẫu nhiên đều trong mọi hành động. Ví dụ (tự tính): 4 hành động, $\epsilon = 0{,}1$. Xác suất chọn hành động tham lam là $1 - 0{,}1 + 0{,}1/4 = 0{,}925$ và mỗi hành động còn lại $0{,}025$. DQN dùng cách này với ϵ giảm tuyến tính từ 1 xuống $0{,}1$ trong một triệu khung hình đầu rồi giữ cố định, theo ghi chú rl 01.

3.3 Cộng thưởng cho sự không chắc chắn (UCB)

Một cách khác: chọn hành động tối đa hóa giá trị ước lượng cộng một "phần thưởng cho sự chưa biết" giảm theo số lần đã thử $N(a)$. Với hằng số c và tổng số lần chọn t,

$$\mathrm{UCB}(a) = \hat{Q}(a) + c\sqrt{\frac{\ln t}{N(a)}}.$$

Ví dụ (tự tính): hai cần, $\hat{Q} = (1{,}0;\ 0{,}8)$, đã thử $N = (10;\ 2)$ lần, $t = 12$, $c = 1$. Cần 1 có $1 + \sqrt{\ln 12/10} \approx 1{,}498$; cần 2 có $0{,}8 + \sqrt{\ln 12/2} \approx 1{,}915$. Dù ước lượng thấp hơn, cần 2 được chọn vì ít được thử. Khi $N(a)$ tăng, phần thưởng giảm và thuật toán chuyển dần sang khai thác.

3.4 Khám phá trong RL sâu

Với không gian trạng thái rất lớn, đếm lần thăm là bất khả; các bài như RND (rl 13) dùng một tín hiệu "độ lạ" học từ mạng nơ-ron làm thưởng nội tại. Nhiều phương pháp trong cuốn thêm một thưởng entropy để chính sách giữ được tính ngẫu nhiên (SAC, rl 12). Với mô hình ngôn ngữ, khám phá thường được điều chỉnh qua nhiệt độ lấy mẫu và qua entropy của chính sách (theo hướng các bài rl 49, 50).

Ý chính cho phần còn lại của chương: **khám phá quyết định phân phối dữ liệu**. Dữ liệu chỉ có hành động đã được thử. Offline RL là trường hợp cực đoan: khám phá bằng không.

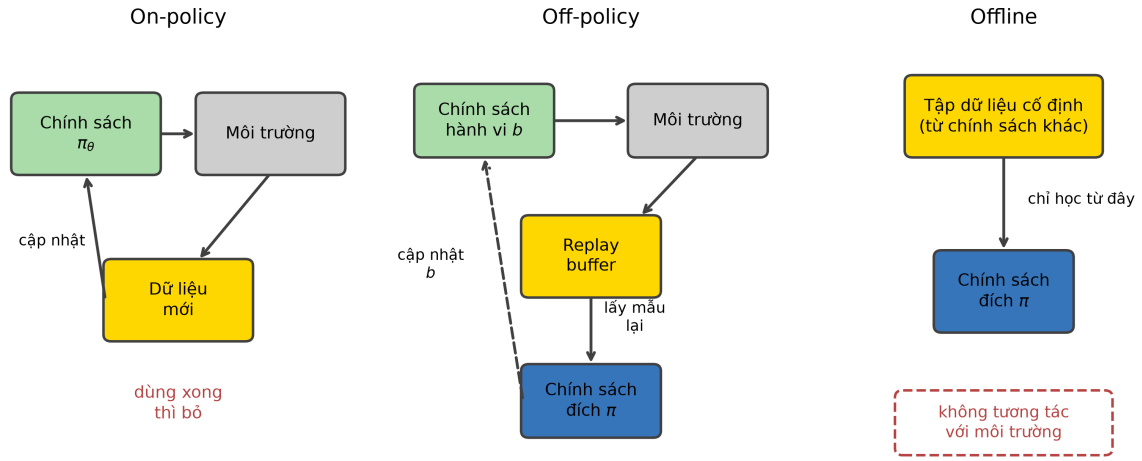
4. On-policy và off-policy

4.1 Định nghĩa

- **Chính sách hành vi** b (behavior policy): chính sách sinh ra dữ liệu.
- **Chính sách đích** π (target policy): chính sách mà ta đang học.
- **On-policy**: $b = \pi$ (hoặc rất gần). Dữ liệu dùng xong phải bỏ khi π đổi. Ví dụ: REINFORCE, A2C, PPO (nen05, trong đó dữ liệu chỉ được dùng cho một số epoch).
- **Off-policy**: $b \neq \pi$. Dữ liệu cũ vẫn dùng được. Ví dụ: Q-learning, DQN, DDPG, TD3, SAC.
- **Offline**: tập dữ liệu cố định, không có thêm tương tác. Là off-policy dữ liệu cố định.

4.2 Sơ đồ

Cách đọc hình: bên trái (on-policy) chính sách tương tác với môi trường, dữ liệu mới dùng để cập nhật đúng chính sách đó rồi bị bỏ. Ở giữa (off-policy) dữ liệu từ chính sách hành vi b đi vào replay buffer và được lấy



Hình 16: Ba chế độ học: on-policy bỏ dữ liệu sau khi dùng, off-policy dùng lại qua replay buffer, offline chỉ học từ tập dữ liệu cố định (minh hoạ, tự tính)

mẫu lại nhiều lần để cập nhật chính sách đích π; mũi tên nét đứt là việc b được cập nhật theo thời gian nên dữ liệu trong buffer cũ dần. Bên phải (offline) không còn mũi tên tới môi trường: toàn bộ thông tin chỉ nằm trong tập dữ liệu cố định, vì thế khám phá không còn.

4.3 Đánh đổi

	On-policy	Off-policy
Hiệu quả dữ liệu	Thấp (dữ liệu dùng ít lần)	Cao (dùng lại nhiều lần)
Độ ổn định	Thường ổn hơn	Dễ bất ổn khi kết hợp xấp xỉ hàm và bootstrapping
Cần sửa lệch phân phối	Gần như không	Có, nếu muốn kỳ vọng đúng (mục 5)
Ví dụ trong cuốn	PPO (rl 07), A3C (rl 06)	DQN (rl 01), DDPG (rl 10), SAC (rl 12)

5. Importance sampling: sửa lệch phân phối bằng trọng số

5.1 Ý tưởng và định nghĩa

Để ước lượng $\mathbb{E}_{a\sim\pi}[f(a)]$ từ mẫu $a \sim b$, nhân mỗi mẫu với **trọng số importance** $w(a) = \pi(a)/b(a)$:

$$\mathbb{E}_{a\sim\pi}[f(a)] = \sum_a \pi(a) f(a) = \sum_a b(a) \frac{\pi(a)}{b(a)} f(a) = \mathbb{E}_{a\sim b}\big[w(a) f(a)\big].$$

Điều kiện: $b(a) > 0$ ở mọi nơi $\pi(a) > 0$ (b phải "phủ" π). Đẳng thức trên cho thấy ước lượng $\frac{1}{N}\sum_i w(a_i) f(a_i)$ **không chệch** (kỳ vọng đúng bằng giá trị cần tìm).

5.2 Ví dụ tay: hai chính sách hai hành động

Dùng $b = (0{,}8;\ 0{,}2)$, $\pi = (0{,}4;\ 0{,}6)$, thưởng $r = (1;\ 3)$ (tự tính):

- Trọng số: $w = (0{,}4/0{,}8;\ 0{,}6/0{,}2) = (0{,}5;\ 3)$.
- Kỳ vọng thật theo π: $0{,}4 \cdot 1 + 0{,}6 \cdot 3 = 2{,}2$. Kỳ vọng ngây thơ theo b: $0{,}8 \cdot 1 + 0{,}2 \cdot 3 = 1{,}4$.
- Kiểm chứng không chệch: $\mathbb{E}_b[wr] = 0{,}8 \cdot (0{,}5 \cdot 1) + 0{,}2 \cdot (3 \cdot 3) = 0{,}4 + 1{,}8 = 2{,}2$. Đúng bằng giá trị thật. Cũng kiểm $\mathbb{E}_b[w] = 0{,}8 \cdot 0{,}5 + 0{,}2 \cdot 3 = 1$, điều kiện luôn đúng.

Phương sai. $\mathbb{E}_b[(wr)^2] = 0{,}8 \cdot 0{,}25 + 0{,}2 \cdot 81 = 0{,}2 + 16{,}2 = 16{,}4$, nên $\mathrm{Var}_b(wr) = 16{,}4 - 2{,}2^2 = 11{,}56$. Nếu ta có thể lấy mẫu trực tiếp từ π, phương sai của r chỉ là $\mathbb{E}_\pi[r^2] - 2{,}2^2 = 5{,}8 - 4{,}84 = 0{,}96$. Importance sampling cho cùng kỳ vọng nhưng phương sai lớn hơn khoảng 12 lần. Nguyên nhân: hành động 2 có $w = 3$, mà lại hiếm trong dữ liệu (xác suất $0{,}2$ theo b), và mỗi lần xuất hiện nó đóng góp rất nhiều.

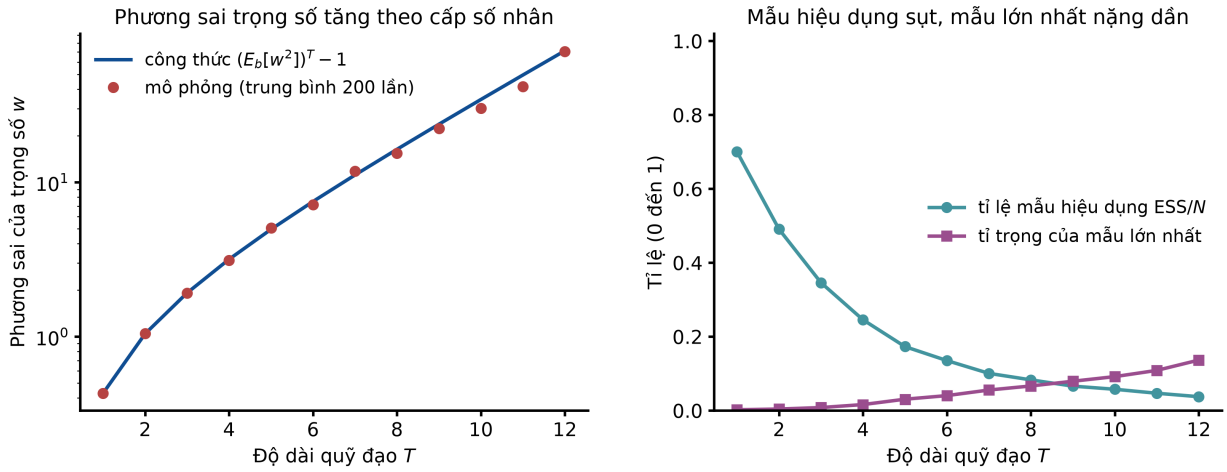
Phiên bản chuẩn hóa (weighted IS). Chia thêm cho tổng trọng số: $\hat{v} = \sum_i w_i f_i / \sum_i w_i$. Với hai mẫu ($N = 2$), liệt kê mọi cặp hành động (tự tính):

Cặp hành động	Xác suất theo b	IS thường	IS chuẩn hóa
(1, 1)	$0{,}64$	$1{,}0$	$1{,}0$
(1, 2) hoặc (2, 1)	$0{,}32$ (tổng hai trường hợp)	$4{,}75$	$2{,}714$
(2, 2)	$0{,}04$	$9{,}0$	$3{,}0$

Kỳ vọng của IS thường là $0{,}64 \cdot 1 + 0{,}32 \cdot 4{,}75 + 0{,}04 \cdot 9 = 2{,}2$ (không chệch). Kỳ vọng của IS chuẩn hóa là khoảng $1{,}629$ (chệch xuống so với $2{,}2$ ở $N = 2$) nhưng các giá trị biến động ít hơn. Đây là đánh đổi cổ điển giữa thiên lệch và phương sai, giống nen04: IS chuẩn hóa chệch nhưng nhất quán (thiên lệch tiến về 0 khi N lớn) và thường phương sai thấp hơn.

5.3 Phương sai nổ theo độ dài quỹ đạo

Trong RL, trọng số cho cả một quỹ đạo là **tích** các tỉ lệ từng bước: $w = \prod_{t=1}^{T} \pi(a_t \mid s_t)/b(a_t \mid s_t)$. Giá trị kỳ vọng vẫn bằng 1 nhưng phương sai tăng theo cấp số nhân của T. Với chính sách hai hành động và mỗi bước độc lập, $\mathrm{Var}(w) = (\mathbb{E}_b[w_1^2])^T - 1$, với $\mathbb{E}_b[w_1^2] = \sum_a \pi(a)^2/b(a)$. Thử với $b = (0{,}7;\ 0{,}3)$, $\pi = (0{,}4;\ 0{,}6)$ (tự tính): $\mathbb{E}_b[w_1^2] = 0{,}16/0{,}7 + 0{,}36/0{,}3 \approx 1{,}4286$. Khi $T = 3$ phương sai là $1{,}4286^3 - 1 \approx 1{,}915$; khi $T = 12$ là khoảng $71{,}2$.



Hình 17: Phương sai trọng số importance sampling tăng theo cấp số nhân theo độ dài quỹ đạo (trái), và mẫu hiệu dụng sụt giảm (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, trục đứng ở thang log; đường xanh là công thức $(\mathbb{E}_b[w_1^2])^T - 1$, chấm đỏ là phương sai đo từ mô phỏng (1000 quỹ đạo mỗi lần, trung bình 200 lần, seed cố định). Chúng khớp nhau: phương sai tăng từ $0{,}43$ ở $T = 1$ lên khoảng 70 ở $T = 12$ (tự tính). Bên phải là hệ quả thực tế: kích thước mẫu hiệu dụng $\mathrm{ESS}/N = (\sum_i w_i)^2/(N \sum_i w_i^2)$ giảm từ $0{,}70$ ở $T = 1$ xuống $0{,}037$ ở $T = 12$, nghĩa là 1000 quỹ đạo chỉ tương đương khoảng 37 quỹ đạo "thật". Đường tím cho thấy tỉ trọng của mẫu lớn nhất so với tổng tăng dần (từ $0{,}2\%$ lên $13{,}6\%$ trong mô phỏng này), tức là càng nhiều quỹ đạo bị một vài mẫu chi phối.

Đây là lý do các thuật toán off-policy thực tế hiếm khi dùng tích đầy đủ. Các cách thường dùng: chỉ dùng một bước (như tỉ lệ ρ_t của PPO, nen05), cắt trọng số (như V-trace, nhắc tới trong ghi chú rl 09), hoặc không dùng importance sampling cho Q-learning vì đích max đã không phụ thuộc hành động hành vi (mục 2.2).

6. Replay buffer

Replay buffer là một bộ nhớ lưu các chuyển tiếp (s, a, r, s') do các chính sách hành vi đã sinh ra, và lấy mẫu từ đó để cập nhật. DQN (rl 01) lấy mẫu đều ngẫu nhiên; theo ghi chú rl 01, tác giả nêu lý do giảm tương quan giữa các mẫu liên tiếp, dùng dữ liệu hiệu quả hơn và làm mượt phân phối hành vi.

Điều thường bị bỏ qua:

- **Dữ liệu lạc hậu.** Mẫu cũ do chính sách cũ sinh ra; buffer càng lớn, càng chứa nhiều dữ liệu khác xa chính sách hiện tại. Với phương pháp off-policy dựa trên Q, điều này chấp nhận được; với phương pháp cần trọng số (policy gradient off-policy), nó làm phương sai tăng.
- **Kích thước và tỉ lệ cập nhật-dữ-liệu.** Buffer nhỏ thì mẫu liên tiếp tương quan; buffer rất lớn thì dữ liệu cũ chi phối. Số bước cập nhật trên mỗi chuyển tiếp mới quyết định mức lệch phân phối.
- **Ưu tiên mẫu (prioritized replay).** Lấy nhiều hơn các mẫu có sai số TD lớn; Rainbow (rl 03) là nơi nhiều cải tiến như vậy được kết hợp.
- **On-policy không dùng được buffer lớn.** PPO chỉ dùng lại dữ liệu trong một vòng (nen05); nếu đổ cả lịch sử vào buffer, tỉ lệ ρ không còn bé và mục tiêu thay thế mất hiệu lực.

7. Offline RL và vấn đề lệch phân phối

7.1 Vì sao offline khó

Trong offline RL ta chỉ có một tập dữ liệu D do chính sách hành vi β (có thể là con người, hệ thống cũ hoặc hỗn hợp) thu thập và **không thể thu thêm**. Ta muốn tìm chính sách π tốt hơn β. Theo ghi chú rl 19, tutorial chỉ ra hai khó khăn: không cải thiện được khám phá (nếu D không có chuyển tiếp tốt thì không thể phát hiện ra), và quan trọng hơn là **distributional shift**: chính sách học được tạo ra các hành động không (hoặc hiếm) có trong D, và giá trị của chúng chỉ được ngoại suy.

7.2 Ví dụ tay: Q-learning bị ngoại suy sai

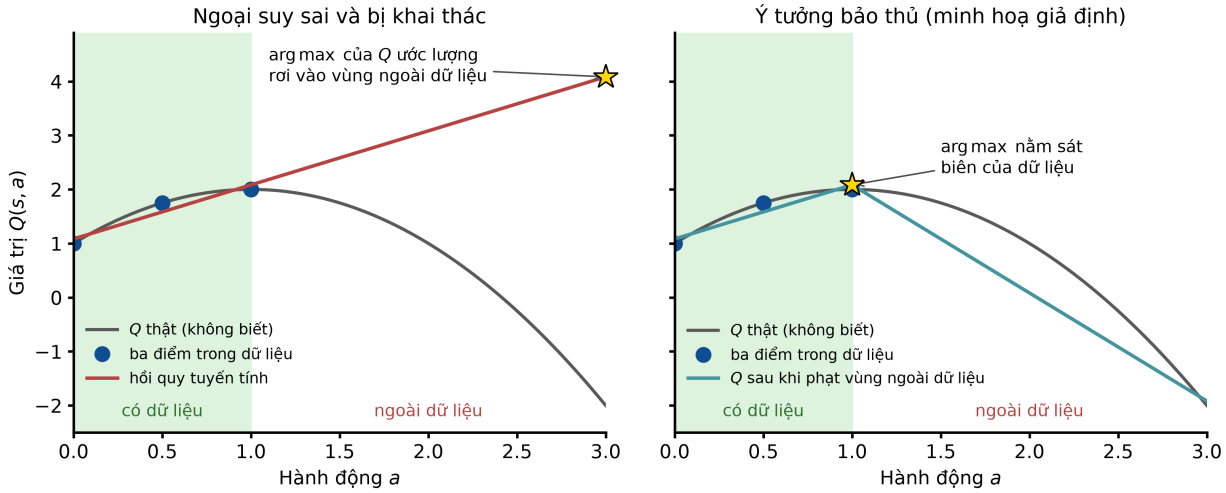
Xét hai trạng thái $s_0 \to s_1$. Từ s_0 chỉ có một hành động, thưởng 0, chuyển tới s_1. Ở s_1 có ba hành động x, y, z rồi kết thúc: dữ liệu chỉ chứa x (thưởng 1) và y (thưởng $0{,}5$), **hành động z chưa bao giờ được thử**. Bảng Q ở s_1 khởi tạo $Q(s_1, x) = Q(s_1, y) = 0$ và $Q(s_1, z) = 3$ (một giá trị ngẫu nhiên nào đó do khởi tạo hoặc sai số mạng; không dữ liệu nào sửa nó). Lấy $\gamma = 0{,}9$ (tự tính):

- Chạy Q-learning lặp trên dữ liệu: $Q(s_1, x) \to 1$, $Q(s_1, y) \to 0{,}5$, còn $Q(s_1, z)$ giữ nguyên 3 vì không có cập nhật nào chạm tới nó.
- Đích cho s_0 là $r + \gamma \max_{a'} Q(s_1, a') = 0 + 0{,}9 \cdot \max(1;\, 0{,}5;\, 3) = 2{,}7$, trong khi nếu chỉ xét hành động có trong dữ liệu thì chỉ là $0{,}9 \cdot 1 = 0{,}9$.
- Chính sách tham lam rồi chọn z tại s_1, hành động dữ liệu không nói gì về nó. Nếu giá trị thật của z thấp, giá trị $2{,}7$ của s_0 là ảo, và không dữ liệu nào bao giờ sửa được vì z không bao giờ được thử.

Trong thực tế với mạng nơ-ron, $Q(s_1, z)$ không phải một ô độc lập mà là kết quả ngoại suy, và ngoại suy hay lạc quan một cách có hệ thống: toán tử max chọn đúng những cặp bị ước lượng cao nhất.

7.3 Ví dụ với hồi quy: vùng dữ liệu so với ngoài dữ liệu

Giả sử hành động liên tục $a \in [0, 3]$, giá trị thật $Q(a) = 1 + 2a - a^2$ (đỉnh tại $a = 1$, giá trị 2), và dữ liệu chỉ ở $a = 0;\, 0{,}5;\, 1$ với $Q = 1;\, 1{,}75;\, 2$. Khớp một đường thẳng bằng bình phương tối thiểu cho độ dốc $1{,}0$ và hệ số chặn $1{,}083$ (tự tính). Tại $a = 3$ ngoài dữ liệu, đường thẳng dự đoán $4{,}083$ trong khi Q thật là -2 (tự tính). Và vì chính sách tham lam tìm $\arg\max_a$, nó sẽ chọn $a = 3$.



Hình 18: Vùng có dữ liệu và vùng ngoài dữ liệu: hồi quy tuyến tính ngoại suy sai và bị khai thác (trái), minh họa ý tưởng bảo thủ bằng một hình phạt giả định (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, vùng xanh nhạt là vùng có dữ liệu ($a \in [0, 1]$), đường xám là Q thật (không biết), chấm xanh là ba điểm dữ liệu, đường đỏ là hồi quy tuyến tính. Trong vùng dữ liệu đường đỏ bám khá tốt; ngoài vùng, đường đỏ tăng mãi còn Q thật giảm, và ngôi sao là arg max của Q ước lượng rơi vào cuối khoảng, nơi sai lớn nhất. Bên phải là minh họa giả định cho ý tưởng bảo thủ của mục 8: hình phạt (ở đây là $-3\max(0, a-1)$, chọn tay chỉ để vẽ) kéo thấp giá trị ngoài vùng dữ liệu, khiến arg max quay về sát biên của dữ liệu ($a = 1$, giá trị $2{,}083$). Đây không phải kết quả đo từ một thuật toán cụ thể, chỉ là hình minh họa cơ chế.

7.4 Vòng tự khuếch đại

Ngoại suy lạc quan chưa phải toàn bộ vấn đề. Trong online RL, hành động bị đánh giá cao sai sẽ được thử và dữ liệu mới sửa lại ước lượng. Trong offline, vòng sửa lỗi không tồn tại, và bootstrapping lan sai số: giá trị ảo của (s_1, z) lan về s_0 qua max, rồi về các trạng thái trước nó. Nhiều bài offline RL (ghi chú rl 19) có thể được hiểu như những cách khác nhau để **không bao giờ đánh giá chính sách hay hàm giá trị ở các điểm xa phân phối dữ liệu**.

8. Bảo thủ: CQL ở mức ý tưởng

Ý tưởng của **Conservative Q-Learning** (rl 20): thay vì mong ước lượng Q chính xác, chủ động **ước lượng thấp** các giá trị ngoài dữ liệu. Cụ thể, thêm vào loss Bellman một số hạng phạt đẩy Q xuống ở các hành động mà chính sách đang cân nhắc và đẩy Q lên ở các hành động có trong dữ liệu. Một dạng cài đặt (theo ghi chú rl 20, công thức (4)) là

$$\min_Q \; \alpha\, \mathbb{E}_{s\sim D}\Big[\log \sum_a \exp Q(s,a) - \mathbb{E}_{a\sim\hat{\pi}_\beta}[Q(s,a)]\Big] + \frac{1}{2}\mathbb{E}_{s,a,s'\sim D}\big[(Q - \hat{\mathcal{B}}^\pi \hat{Q})^2\big].$$

Số hạng $\log \sum_a \exp Q$ là một "max mềm" của Q trên mọi hành động: giảm nó là đẩy các Q lớn xuống. Số hạng trừ là trung bình Q trên hành động trong dữ liệu: tăng nó. Hệ số α điều chỉnh mức bảo thủ.

Ví dụ tay (tự tính). Dùng $Q(s_1, \cdot) = (1;\ 0{,}5;\ 3)$ cho (x, y, z) như mục 7.2 và dữ liệu chỉ chứa x. Phần phạt:

- $\log \sum \exp Q = \ln(e^1 + e^{0{,}5} + e^3) \approx 3{,}197$; trừ $Q(s_1, x) = 1$ còn $\approx 2{,}197$, nhân α.
- Gradient của phần phạt theo Q là $\text{softmax}(Q)$ trừ onehot của hành động dữ liệu: $\text{softmax} \approx (0{,}111;\ 0{,}067;\ 0{,}821)$, trừ $(1; 0; 0)$ cho $(-0{,}889;\ +0{,}067;\ +0{,}821)$.

Cập nhật gradient descent sẽ **hạ** $Q(s_1, z)$ mạnh (hệ số $0{,}821\alpha$), hạ nhẹ $Q(s_1, y)$ và **nâng** $Q(s_1, x)$. Nhờ đó giá trị ảo của z bị kéo xuống cho tới khi dữ liệu thật hoặc cân bằng giữa phạt và sai số Bellman quyết định. Đây là cơ chế "bi quan có chủ đích" thay cho lạc quan sai; đánh đổi là có thể bỏ lỡ hành động ngoài dữ liệu thật sự tốt.

Lưu ý về mức độ chắc chắn: theo ghi chú rl 20, CQL có tính chất cận dưới theo lý thuyết dưới một số giả thiết, nhưng thực nghiệm cần chọn α, và chương này không nêu số liệu thí nghiệm.

9. Bắt chước (imitation) ở mức ý tưởng

9.1 Behavior cloning

Khi ta có sẵn các quỹ đạo của một chuyên gia (con người, hệ thống tốt), cách đơn giản nhất là học có giám sát: dự đoán hành động của chuyên gia từ trạng thái, tức tối đa hóa $\log \pi_\theta(a^\star \mid s)$ trên dữ liệu chuyên gia. Đây là **behavior cloning** (BC). Nó không cần thưởng, không cần tương tác với môi trường.

9.2 Vì sao BC dễ vỡ: sai số tích lũy

Mô hình chỉ được huấn luyện trên các trạng thái của chuyên gia. Khi triển khai, một sai lầm nhỏ đưa tác nhân sang trạng thái chuyên gia chưa từng ở, nơi mô hình chưa từng được học và dễ sai tiếp, sai số cộng dồn. Theo ghi chú rl 17, BC có chặn lỗi bậc hai theo độ dài T của quỹ đạo, cụ thể $J(\pi) \le J(\pi^\star) + T^2\epsilon$, và chặn này là chặt. Ví dụ (tự tính): $\epsilon = 0{,}02$, $T = 50$ cho $T^2\epsilon = 50$, trong khi chặn tuyến tính $T\epsilon = 1$. Đây cũng chính là lệch phân phối của mục 7, nhưng trong bối cảnh bắt chước.

9.3 Cách xử lý

- **DAgger** (rl 17): cho chính sách hiện tại tự đi, thu các trạng thái nó thật sự gặp, hỏi chuyên gia hành động đúng ở đó rồi gộp vào tập dữ liệu. Theo ghi chú rl 17, đạt chặn lỗi tuyến tính theo T, đổi lại cần chuyên gia trả lời khi huấn luyện.
- **GAIL** (rl 18): không bắt chước từng hành động mà khớp phân phối trạng thái và hành động của chuyên gia bằng một bộ phân biệt kiểu đối kháng, không cần hỏi chuyên gia lúc huấn luyện và không cần hàm thưởng, theo ghi chú rl 18.
- **Decision Transformer** (rl 21): đóng khung offline RL thành mô hình chuỗi, điều kiện hóa trên return mong muốn, bỏ qua bootstrapping và vì vậy tránh vòng khuếch đại của mục 7.4, theo ghi chú rl 21.

So sánh mục đích: bắt chước chỉ cần dữ liệu tốt nhưng không vượt được chuyên gia; offline RL với CQL có thể vượt hành vi trong dữ liệu nhưng phải thận trọng với hành động ngoài dữ liệu.

10. Nối sang các bài paper của cuốn

- **rl 01, 02, 03 (DQN, Double DQN, Rainbow).** Off-policy với replay (mục 6), và Double DQN sinh ra từ vấn đề ước lượng quá cao do max; mục 7 cho thấy cùng cơ chế, nhưng nặng hơn khi không thể thử thêm.
- **rl 10, 11, 12 (DDPG, TD3, SAC).** Off-policy cho hành động liên tục; actor tìm hành động tối đa hóa Q, nên có xu hướng đi đúng vào vùng Q ngoại suy sai. Một số kỹ thuật trong các bài này giảm thiên lệch ước lượng quá cao.
- **rl 13 (RND).** Khám phá bằng thưởng nội tại (mục 3.4).
- **rl 17, 18 (DAgger, GAIL).** Bắt chước và sai số tích lũy (mục 9).
- **rl 19, 20 (offline RL tutorial, CQL).** Mục 7 và 8 là phiên bản chậm và có ví dụ tay của hai bài này.
- **rl 21 (Decision Transformer).** Con đường né vòng bootstrapping (mục 9.3).
- **rl 07 (PPO).** Tỉ lệ ρ chính là trọng số importance một bước, và vùng cắt nhằm giữ nó gần 1 để phương sai không nổ (mục 5.3, nen05).

11. Sai lầm hay gặp

- **Coi Q-learning cần importance sampling.** Với đích max một bước thì không; importance sampling chỉ cần khi ước lượng giá trị của một chính sách khác qua nhiều bước hoặc dùng policy gradient off-policy.
- **Tin rằng IS không chệch thì dùng thoải mái.** Không chệch không có nghĩa phương sai chấp nhận được; hình mô phỏng cho thấy phương sai tăng cấp số nhân theo T.
- **Quên điều kiện phủ.** Nếu $b(a) = 0$ ở nơi $\pi(a) > 0$ thì không có trọng số nào cứu được; IS vô nghĩa.
- **Cho rằng buffer lớn luôn tốt.** Quá nhiều dữ liệu cũ làm phân phối lệch xa chính sách hiện tại.
- **Cho rằng offline RL chỉ là Q-learning chạy trên dữ liệu cố định.** Nếu không xử lý hành động ngoài dữ liệu, giá trị ngoại suy lạc quan sẽ phá hỏng chính sách.
- **Cho rằng bắt chước giải quyết xong khi có đủ dữ liệu chuyên gia.** Sai số tích lũy vẫn xuất hiện khi trạng thái triển khai lệch khỏi dữ liệu.

12. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
ϵ-greedy	Tham lam với xác suất $1 - \epsilon + \epsilon/\lvert\mathcal{A}\rvert$	Khám phá đơn giản
UCB	$\hat{Q} + c\sqrt{\ln t / N}$	Thưởng cho cái ít thử
Trọng số importance	$w = \pi(a)/b(a)$	$\mathbb{E}_b[w] = 1$
IS không chệch	$\mathbb{E}_b[wf] = \mathbb{E}_\pi[f]$	Cần b phủ π
IS cả quỹ đạo	$w = \prod_t \pi/b$	Phương sai nổ theo T
Đích Q-learning	$r + \gamma \max_{a'} Q(s', a')$	Không phụ thuộc hành động hành vi
Phạt kiểu CQL	$\alpha(\log \sum_a e^Q - \mathbb{E}_D Q)$	Hạ Q ngoài dữ liệu
Chặn BC	$J(\pi) \le J(\pi^\star) + T^2\epsilon$	Lỗi bậc hai theo T

13. Bài tập tự tính

Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Trọng số importance

Cho $b = (0{,}5;\ 0{,}5)$, $\pi = (0{,}9;\ 0{,}1)$, thưởng $r = (2;\ -1)$. Tính w, $\mathbb{E}_\pi[r]$ và kiểm $\mathbb{E}_b[wr]$.

Đáp án (tự tính): $w = (1{,}8;\ 0{,}2)$. $\mathbb{E}_\pi[r] = 0{,}9 \cdot 2 + 0{,}1 \cdot (-1) = 1{,}7$. $\mathbb{E}_b[wr] = 0{,}5 \cdot 1{,}8 \cdot 2 + 0{,}5 \cdot 0{,}2 \cdot (-1) = 1{,}8 - 0{,}1 = 1{,}7$. Bằng nhau.

Bài 2. ϵ-greedy

Có 3 hành động, $\epsilon = 0{,}3$. Tính xác suất chọn hành động tham lam và mỗi hành động còn lại.

Đáp án (tự tính): tham lam $0{,}7 + 0{,}3/3 = 0{,}8$; mỗi hành động khác $0{,}1$.

Bài 3. UCB

Hai cần có $\hat{Q} = (0{,}5;\ 0{,}4)$, số lần thử $N = (8;\ 2)$, $t = 10$, $c = 1$. Chọn cần nào?

Đáp án (tự tính): $0{,}5 + \sqrt{\ln 10/8} \approx 1{,}036$ và $0{,}4 + \sqrt{\ln 10/2} \approx 1{,}473$. Chọn cần 2.

Bài 4. Phương sai trọng số theo quỹ đạo

Với $b = (0{,}7;\ 0{,}3)$, $\pi = (0{,}4;\ 0{,}6)$, tính $\mathbb{E}_b[w_1^2]$ và phương sai trọng số của quỹ đạo $T = 3$ khi các bước độc lập.

Đáp án (tự tính): $\mathbb{E}_b[w_1^2] = 0{,}16/0{,}7 + 0{,}36/0{,}3 \approx 1{,}4286$; Var $= 1{,}4286^3 - 1 \approx 1{,}915$.

Bài 5. Phạt kiểu CQL

Cho $Q(s,\cdot) = (1;\ 0{,}5;\ 3)$ trên ba hành động và dữ liệu chỉ có hành động thứ nhất. Tính $\log \sum \exp Q - Q(s, a_1)$ và gradient của nó theo Q.

Đáp án (tự tính): $\ln(e^1 + e^{0{,}5} + e^3) \approx 3{,}197$, trừ 1 còn $\approx 2{,}197$. Gradient là softmax trừ onehot: $(0{,}111;\ 0{,}067;\ 0{,}821) - (1; 0; 0) = (-0{,}889;\ 0{,}067;\ 0{,}821)$. Cập nhật hạ Q của hành động thứ ba nhiều nhất.

Tài liệu tham khảo nền: Sutton và Barto, Reinforcement Learning: An Introduction (chương về học off-policy và importance sampling); Levine, Kumar, Tucker, Fu, Offline Reinforcement Learning: Tutorial, Review, and Perspectives (ghi chú rl 19); Kumar và cộng sự, Conservative Q-Learning (ghi chú rl 20); Ross, Gordon, Bagnell, A Reduction of Imitation Learning (ghi chú rl 17); các ghi chú rl 01, 13, 18, 21 trong sách.

Nền tảng 07

RLHF, mô hình thưởng, phạt KL và DPO: dạy mô hình theo ý người khi không có hàm điểm sẵn

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất và hàm sigmoid ở math nền tảng 08, cross-entropy và KL ở math nền tảng 11, MLE ở math nền tảng 10, gradient ở math nền tảng 06; policy gradient và PPO ở các bài rl 04 đến 07 (chương này nhắc lại phần cần dùng); regularization ở ML cơ bản nền tảng 04

Dùng ở: rl 23 (học thưởng từ so sánh cặp), rl 24 (RLHF đầu tiên cho mô hình ngôn ngữ), rl 25 (tóm tắt và đường cong over-optimization), rl 26 (nhãn do AI tạo), rl 27 (đo over-optimization), rl 28 (chi tiết cài đặt), rl 29 (DPO), rl 30 (IPO), rl 31 (KTO), rl 32 (SimPO), rl 33 (DPO so với PPO)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy; số do mô phỏng được nói rõ là mô phỏng.

1. Vì sao cần RLHF và DPO

Có những việc ta nhận ra là tốt khi nhìn thấy, nhưng không viết nổi thành công thức: một câu trả lời "hữu ích", một bản tóm tắt "trung thực", một giọng văn "lịch sự". Trong RL ta cần một **phần thưởng** (reward) là con số. RLHF (reinforcement learning from human feedback, học tăng cường từ phản hồi của người) giải quyết bằng ba bước: cho người so sánh hai câu trả lời, học một mô hình dự đoán điều người sẽ chọn (mô hình thưởng), rồi dùng RL để làm mô hình ngôn ngữ đạt điểm cao theo mô hình thưởng đó mà không đi quá xa điểm xuất phát.

Chương này dạy bốn ý, theo thứ tự: (1) dữ liệu ưa thích và mô hình Bradley-Terry để biến "A hay hơn B" thành loss; (2) phạt KL: sợi dây giữ mô hình gần mô hình tham chiếu; (3) over-optimization: vì sao điểm của mô hình thưởng tăng mà chất lượng thật giảm; (4) DPO: cách bỏ hẳn mô hình thưởng và vòng RL, học thẳng từ các cặp ưa thích. Cuối chương là các biến thể ở mức một câu.

2. Nhắc nền

Policy là mô hình ngôn ngữ. Với prompt x, mô hình $\pi_\theta(y|x)$ cho xác suất của câu trả lời y (tích xác suất từng token). "Chọn hành động" nghĩa là sinh cả câu y; phần thưởng thường chỉ có một số cho cả câu.

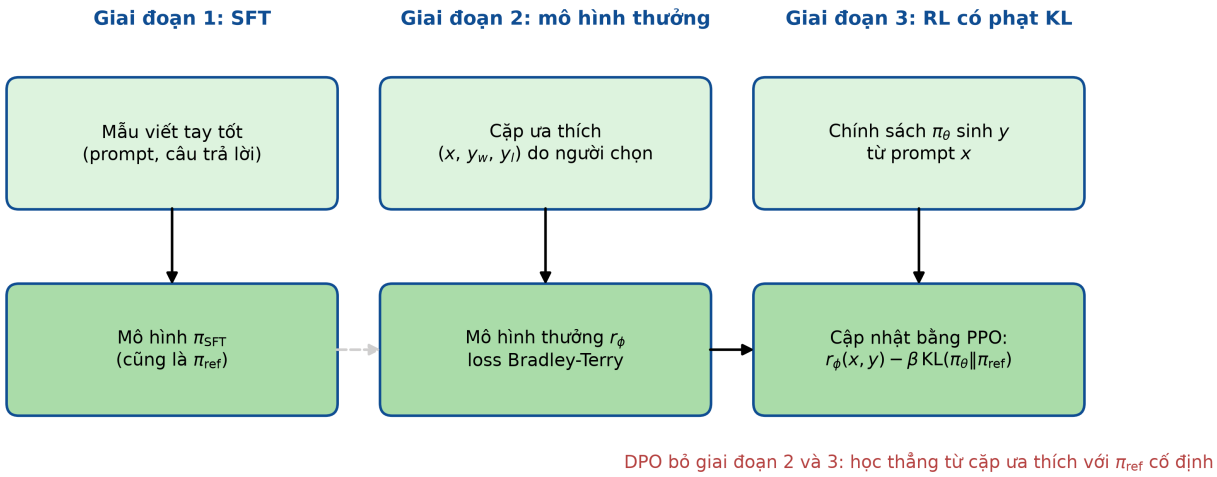
Sigmoid. $\sigma(z) = \dfrac{1}{1+e^{-z}}$ biến số thực thành xác suất. Hai tính chất dùng nhiều: $\sigma(0) = 0{,}5$ và $\sigma(-z) = 1 - \sigma(z)$. Ví dụ (tự tính): $\sigma(2) = 0{,}8808$, $\sigma(-1{,}5) = 0{,}1824$.

Cross-entropy cho hai lớp. Nếu mô hình gán xác suất p cho điều đã xảy ra thì loss là $-\log p$. $p = 0{,}9$ cho loss $0{,}105$, $p = 0{,}5$ cho $0{,}693$, $p = 0{,}1$ cho $2{,}303$ (tự tính). Loss nhỏ khi mô hình tự tin đúng, lớn khi tự tin sai.

KL. $\mathrm{KL}(\pi \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{y\sim\pi}\left[\log \dfrac{\pi(y)}{\pi_{\text{ref}}(y)}\right]$ đo mức hai phân phối khác nhau; bằng 0 khi giống hệt, không âm, và không đối xứng (xem math nền tảng 11).

3. Trực giác bằng hình: ba giai đoạn và một đường tắt

Cách đọc hình: đọc từ trái sang phải. Giai đoạn 1 cho ra mô hình π_{SFT}, được **đóng băng làm mô hình tham chiếu** π_{ref}. Giai đoạn 2 dùng các cặp "câu thắng, câu thua" do người chọn để học điểm r_ϕ. Giai đoạn 3 cho



Hình 19: Ba giai đoạn của RLHF và đường tắt của DPO (minh hoạ, tự tính)

chính sách sinh câu, chấm điểm bằng r_ϕ, trừ phạt KL so với π_{ref}, rồi cập nhật bằng PPO. Dòng đỏ phía dưới: DPO đi đường tắt, bỏ giai đoạn 2 và 3 và chỉ cần dữ liệu cặp cùng π_{ref}. Hình chỉ là sơ đồ, không có số đo.

Vì sao dùng so sánh cặp thay vì chấm điểm 1 đến 10? Người chấm điểm tuyệt đối rất lệch nhau (người này hào phóng, người kia khắt khe), còn nói "câu A tốt hơn câu B" thì ổn định hơn nhiều. Đó là lý do dữ liệu của RLHF có dạng cặp.

4. Dữ liệu ưa thích và mô hình Bradley-Terry

4.1 Dữ liệu

Mỗi mẫu là một bộ ba $(x,\, y_w,\, y_l)$: prompt x, câu được chọn y_w (winner) và câu bị loại y_l (loser). Hai câu thường do chính π_{SFT} sinh ra, người chỉ việc chọn.

4.2 Mô hình Bradley-Terry

Giả sử mỗi câu trả lời có một **điểm ẩn** $r(x, y)$, và xác suất người chọn y_w hơn y_l chỉ phụ thuộc **hiệu điểm**:

$$P(y_w \succ y_l \mid x) = \sigma\big(r(x, y_w) - r(x, y_l)\big).$$

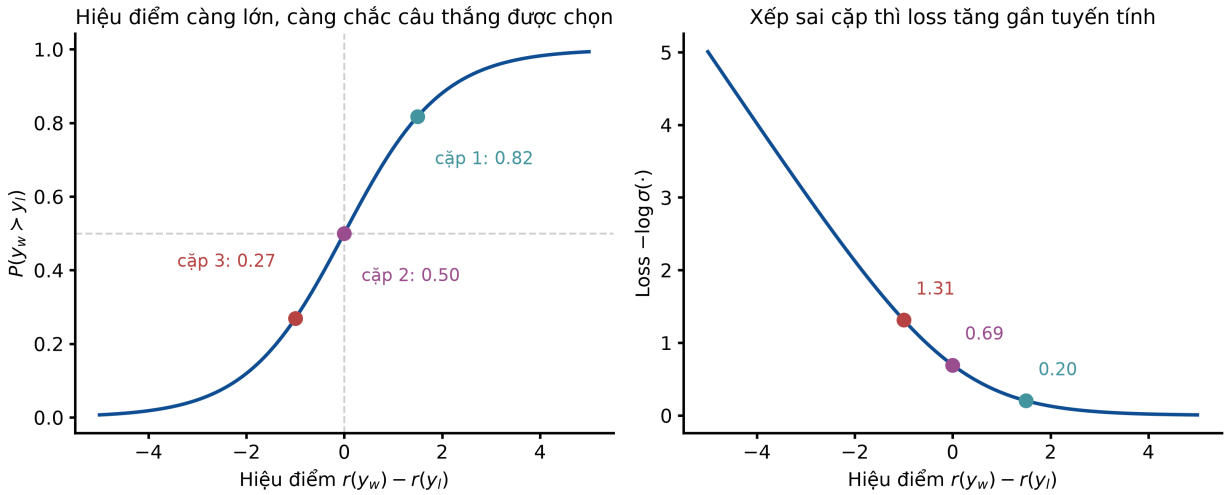
Hiệu điểm bằng 0 thì chọn ngẫu nhiên (xác suất $0{,}5$); hiệu càng lớn thì càng chắc. Chỉ **hiệu** có nghĩa: cộng cùng một hằng số vào mọi điểm không đổi xác suất nào.

Cách đọc hình: bên trái là đường sigmoid, ba chấm là ba cặp của ví dụ ở mục 6 (hiệu điểm $1{,}5$, 0 và $-1{,}0$). Chấm tím ở giữa là "hoà", chấm xanh lục nằm trên $0{,}5$ nghĩa là mô hình đang xếp đúng, chấm đỏ nằm dưới nghĩa là mô hình xếp ngược ý người. Bên phải là loss $-\log\sigma(\text{hiệu})$: gần 0 khi xếp đúng chắc chắn, đi xuống phải thì tăng nhẹ rồi gần tuyến tính khi xếp sai nặng.

4.3 Loss của mô hình thưởng

Mô hình thưởng $r_\phi(x, y)$ là một mạng (thường là chính mô hình ngôn ngữ thêm một đầu ra vô hướng). Huấn luyện bằng cực đại hoá khả năng quan sát được các lựa chọn của người (MLE, math nền tảng 10), tức tối thiểu hoá

$$\mathcal{L}_{\text{RM}}(\phi) = -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log\sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big].$$



Hình 20: Bradley-Terry: xác suất ưa thích theo hiệu điểm và loss tương ứng (minh hoạ, tự tính)

Đây là cross-entropy hai lớp mà "nhãn" luôn là "câu đầu thắng". Gradient theo hiệu điểm d là $-\sigma(-d)$: cặp mô hình đang xếp sai (hiệu âm) kéo mạnh, cặp xếp đúng chắc chắn gần như không kéo. Ngoài ra, vì chỉ hiệu điểm có nghĩa nên thang điểm của mô hình thưởng là tuỳ ý, hệ quả là ta không nên so sánh số tuyệt đối giữa hai mô hình thưởng.

5. Phạt KL so với mô hình tham chiếu

5.1 Mục tiêu có KL

Có mô hình thưởng rồi, cách ngây thơ là cực đại hoá $\mathbb{E}_{y\sim\pi_\theta}[r_\phi(x,y)]$. Nhưng r_ϕ chỉ được huấn luyện tốt ở vùng gần dữ liệu SFT; ngoài vùng đó nó có thể cho điểm cao vô lý. Vì vậy ta thêm một sợi dây:

$$\max_{\pi_\theta}\ \mathbb{E}_{x,\,y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y)\big]-\beta\,\mathrm{KL}\big(\pi_\theta(\cdot|x)\,\|\,\pi_{\mathrm{ref}}(\cdot|x)\big),\qquad \beta>0.$$

β là hệ số phạt: β lớn thì mô hình bị giữ sát π_{ref}, β nhỏ thì được đi xa để lấy điểm. Trong cài đặt PPO thường nhúng phạt vào phần thưởng từng token, dạng $r_\phi-\beta\log\frac{\pi_\theta}{\pi_{\mathrm{ref}}}$, vì KL là kỳ vọng của $\log\frac{\pi_\theta}{\pi_{\mathrm{ref}}}$ trên các token sinh ra (theo ghi chú rl 24, đây là công thức phạt của RLHF đầu tiên cho mô hình ngôn ngữ, kèm bộ điều khiển chỉnh β động).

5.2 Nghiệm đóng của mục tiêu có KL

Với mỗi prompt, bài toán trên có nghiệm tường minh (chứng minh bằng nhân tử Lagrange hoặc bất đẳng thức Gibbs, bỏ qua ở đây):

$$\pi^\star(y|x)=\frac{1}{Z(x)}\,\pi_{\mathrm{ref}}(y|x)\,\exp\!\Big(\frac{r(x,y)}{\beta}\Big),\qquad Z(x)=\sum_y \pi_{\mathrm{ref}}(y|x)\exp\!\Big(\frac{r(x,y)}{\beta}\Big).$$

Đọc bằng lời: lấy mô hình tham chiếu, nhân xác suất mỗi câu với $e^{r/\beta}$ rồi chuẩn hoá. Câu điểm cao được nâng lên, nhưng chỉ nâng theo tỉ lệ, nên câu mà π_{ref} coi là cực hiếm vẫn khó lên cao. $\beta\to\infty$ cho $\pi^\star\to\pi_{\mathrm{ref}}$; $\beta\to 0$ cho mô hình dồn hết xác suất vào câu điểm cao nhất.

Ví dụ tay. Hai câu trả lời, $\pi_{\mathrm{ref}}=(0{,}5;\,0{,}5)$, điểm $r=(1;\,0)$ (tự tính):

- $\beta = 1$: trọng số $(0{,}5e;\ 0{,}5)$, nên $\pi^\star = \left(\dfrac{e}{1+e}, \dfrac{1}{1+e}\right) = (0{,}7311;\ 0{,}2689)$. KL tới π_{ref} là $0{,}1109$ nat; điểm kỳ vọng $0{,}7311$; mục tiêu $0{,}7311 - 1 \cdot 0{,}1109 = 0{,}6201$ (so với $0{,}5$ nếu giữ nguyên π_{ref}).
- $\beta = 0{,}5$: $\pi^\star = \left(\dfrac{e^2}{1+e^2}, \dfrac{1}{1+e^2}\right) = (0{,}8808;\ 0{,}1192)$, KL $= 0{,}3278$ nat, mục tiêu $0{,}7169$.
- $\beta = 0{,}1$: $\pi^\star \approx (1{,}000;\ 0{,}000)$ (cụ thể $0{,}99995$), KL $\approx 0{,}6926$ nat, gần mức tối đa $\ln 2 = 0{,}6931$, mục tiêu $0{,}9307$.

Chú ý: β nhỏ cho điểm kỳ vọng cao hơn nhưng phải trả bằng KL lớn hơn. Quyết định "đáng trả bao nhiêu" nằm ở việc điểm r đáng tin đến đâu, và đó chính là chuyện của mục 7.

6. Ví dụ tay: loss Bradley-Terry cho ba cặp ưa thích

Giả sử mô hình thưởng hiện cho các điểm sau (số cho trước để tính tay, không phải đo từ mô hình thật):

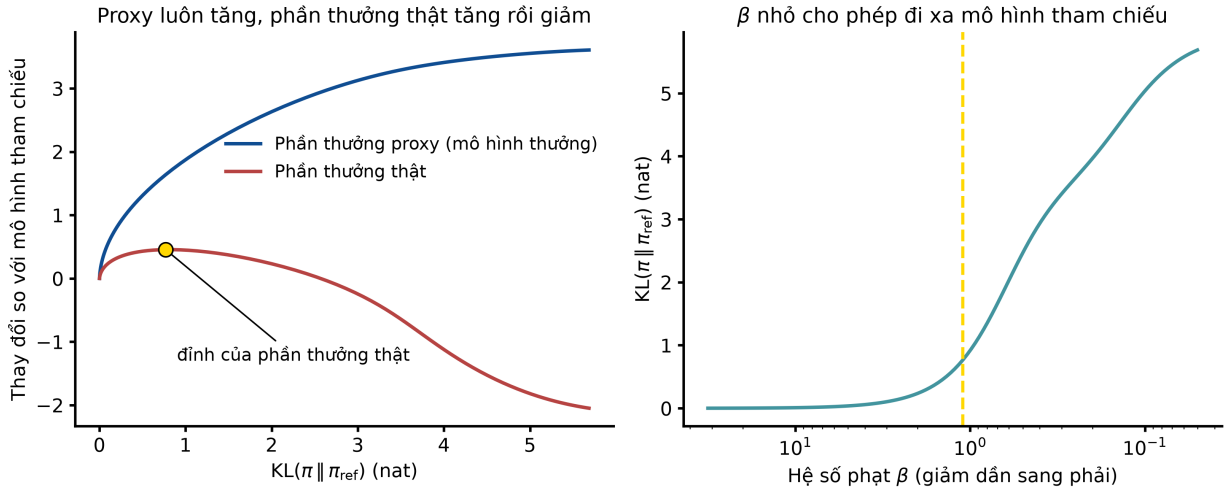
Cặp	$r(y_w)$	$r(y_l)$	Hiệu d	$P(y_w \succ y_l) = \sigma(d)$	Loss $-\log\sigma(d)$
1	2,0	0,5	1,5	0,8176	0,2014
2	1,0	1,0	0,0	0,5000	0,6931
3	0,2	1,2	-1,0	0,2689	1,3133

Tất cả là (tự tính). Loss trung bình của ba cặp là $\dfrac{0{,}2014 + 0{,}6931 + 1{,}3133}{3} = 0{,}7359$.

Cặp 1 đã xếp đúng, loss nhỏ. Cặp 2 hoà nên loss đúng bằng $\ln 2$ (mức "đoán mò"). Cặp 3 xếp ngược, loss lớn nhất, và gradient theo hiệu điểm là $-\sigma(1) = -0{,}7311$ (tự tính), nên bước cập nhật sẽ nâng $r(y_w)$ và hạ $r(y_l)$ mạnh nhất ở cặp này. Với cặp 1, hệ số chỉ $\sigma(-1{,}5) = 0{,}1824$.

7. Over-optimization: khi điểm tăng mà chất lượng thật giảm

Mô hình thưởng chỉ là **đại diện** (proxy) của điều người thật sự muốn. Luật Goodhart nói: khi một thước đo trở thành mục tiêu tối ưu, nó thôi là thước đo tốt. Chính sách càng được đẩy đi xa π_{ref} để lấy điểm proxy, càng dễ rơi vào những vùng mà proxy sai: câu dài lê thê, giọng nịnh, mẫu câu mà mô hình thưởng vô tình thích.



Hình 21: Mô phỏng: điểm proxy luôn tăng, điểm thật tăng rồi giảm khi KL lớn dần (minh hoạ, tự tính)

Cách đọc hình (đây là **mô phỏng với dữ liệu giả định**, không phải số đo từ mô hình thật): có 200 câu trả lời giả định, điểm thật g lấy ngẫu nhiên chuẩn, điểm proxy bằng g cộng nhiễu, thêm 6 câu "khai thác" có điểm proxy rất cao nhưng điểm thật thấp (seed cố định). Chính sách là nghiệm đóng ở mục 5.2 với nhiều giá trị β.

Bên trái: trục ngang là KL tới tham chiếu; đường xanh là thay đổi điểm proxy, đường đỏ là thay đổi điểm thật. Chấm vàng là đỉnh của điểm thật, ở KL khoảng $0{,}77$ nat và $\beta \approx 1{,}10$, tăng khoảng $0{,}48$ so với tham chiếu (tự tính). Sau đỉnh, điểm proxy vẫn lên (tổng $+3{,}66$ ở KL lớn nhất) trong khi điểm thật tụt xuống $-2{,}02$ so với ban đầu. Bên phải: β giảm dần sang phải thì KL tăng; đường vàng đánh dấu cùng vị trí đỉnh.

Bài học của mô phỏng, không phải kết luận về một mô hình thật: tồn tại một mức KL "vừa phải" tốt nhất, và nó phụ thuộc vào việc proxy sai ở đâu, điều mà trong thực tế ta không biết trước. Vì thế người ta cần một thước đo độc lập (đánh giá riêng, hoặc người chấm) để biết lúc dừng.

Theo ghi chú rl 27, paper đo hiện tượng này bằng môi trường tổng hợp: một mô hình thưởng "vàng" lớn đóng vai sự thật, các mô hình thưởng nhỏ hơn là proxy học từ nhãn của nó. Đặt $d = \sqrt{\mathrm{KL}}$, điểm vàng khớp dạng $R(d) = d(\alpha - \beta d)$ với best-of-n và $R(d) = d(\alpha - \beta \log d)$ với RL. Cùng ghi chú đó cho biết, trong thí nghiệm ấy, phạt KL tường minh chủ yếu làm điểm vàng hội tụ sớm hơn (giống dừng sớm) chứ không cải thiện đường biên điểm vàng theo KL, và nhiều dữ liệu cho mô hình thưởng thì ít bị lạm dụng hơn. Đây là kết quả của một thiết lập tổng hợp; ghi chú cũng nêu rằng nó có thể không chuyển nguyên sang mô hình thưởng thật.

8. DPO: suy ra từ mục tiêu có KL

8.1 Ý tưởng chính

DPO (direct preference optimization, tối ưu ưa thích trực tiếp) dựa trên một nhận xét: nghiệm đóng ở mục 5.2 liên hệ **một-một** giữa chính sách tối ưu và hàm thưởng. Lấy logarit hai vế của công thức $\pi^\star$ rồi giải ra r:

$$r(x, y) = \beta \log \frac{\pi^\star(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

Hàm thưởng bất kỳ đều có thể viết qua chính sách tối ưu tương ứng của nó. Thay vào mô hình Bradley-Terry, ta cần **hiệu** điểm của hai câu cùng prompt, và số hạng $\beta \log Z(x)$ chỉ phụ thuộc x nên bị triệt tiêu:

$$P(y_w \succ y_l \mid x) = \sigma\Big(\beta \log \frac{\pi^\star(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^\star(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big).$$

Đến đây chỉ còn một bước: coi chính sách đang học π_θ chính là $\pi^\star$ và huấn luyện nó bằng MLE trên dữ liệu cặp. Không còn mô hình thưởng riêng, không còn vòng RL, không phải sinh mẫu trong lúc huấn luyện.

8.2 Loss DPO

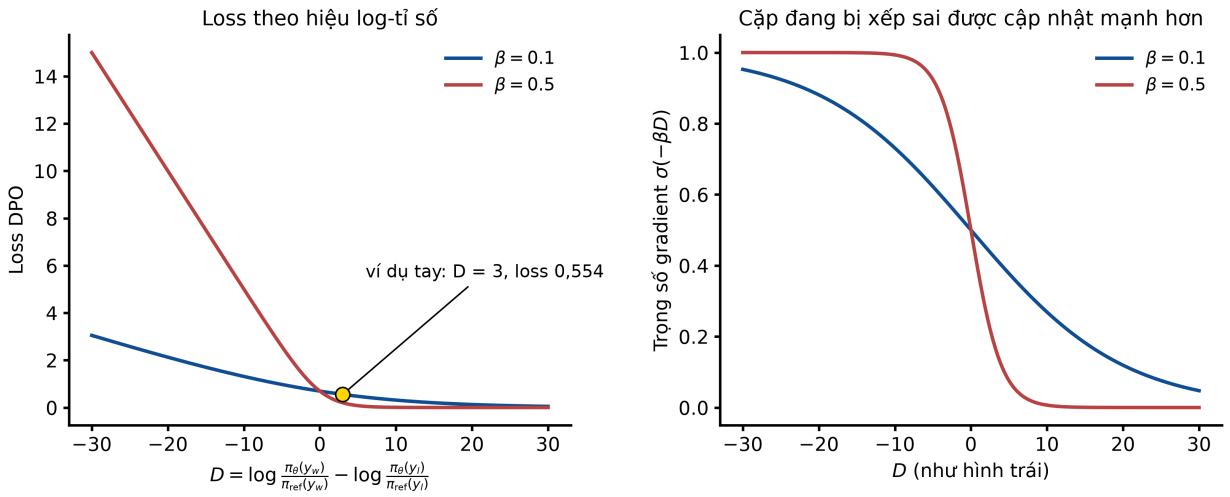
$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\Big(\beta \underbrace{\Big(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big)}_{D}\Big)\Big].$$

Đại lượng $\hat{r}_\theta(x, y) = \beta \log \dfrac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ gọi là **phần thưởng ngầm** (implicit reward): mô hình tự đóng vai mô hình thưởng. Gradient theo θ có dạng

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \mathbb{E}\Big[\sigma(-\beta D)\big(\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x)\big)\Big].$$

Nghĩa là: tăng log-xác suất câu thắng, giảm log-xác suất câu thua, mỗi cặp nhân với trọng số $\sigma(-\beta D)$, lớn khi mô hình đang xếp cặp đó sai.

Cách đọc hình: trục ngang là D, hiệu log-tỉ số của câu thắng so với câu thua, tính theo nat. Bên trái là loss, bên phải là trọng số $\sigma(-\beta D)$; đường xanh là $\beta = 0{,}1$, đường đỏ là $\beta = 0{,}5$. Ở $D = 0$ (mô hình chưa khác



Hình 22: Loss DPO và trọng số gradient theo hiệu log-tỉ số D, với beta 0,1 và 0,5 (minh hoạ, tự tính)

tham chiếu) loss bằng $\ln 2$ và trọng số bằng $0{,}5$ với mọi β. β lớn làm đường cong dốc hơn: chỉ cần D lệch vài đơn vị là cặp đã "xong" (trọng số về 0), tức mô hình bị giữ gần tham chiếu hơn. Chấm vàng là ví dụ tay dưới đây.

8.3 Ví dụ tay: tính loss DPO

Lấy $\beta = 0{,}1$ và các log-xác suất cho trước (tự chọn để tính, không đo từ mô hình thật). Câu thắng: $\log \pi_\theta = -12$, $\log \pi_{\text{ref}} = -14$. Câu thua: $\log \pi_\theta = -20$, $\log \pi_{\text{ref}} = -19$.

- Log-tỉ số câu thắng: $-12 - (-14) = +2$. Log-tỉ số câu thua: $-20 - (-19) = -1$.
- $D = 2 - (-1) = 3$, nên $\beta D = 0{,}3$.
- Loss $= -\log \sigma(0{,}3) = 0{,}5544$; trọng số gradient $\sigma(-0{,}3) = 0{,}4256$; nhân thêm β cho $0{,}0426$ (tự tính).

Mô hình đã tăng xác suất câu thắng và giảm xác suất câu thua so với tham chiếu, đúng hướng, nên loss thấp hơn $\ln 2 = 0{,}6931$.

Ba cặp, $\beta = 0{,}1$ (tự tính, các số log-xác suất do ta đặt):

Cặp	$\log \pi_\theta(y_w)$	$\log \pi_{\text{ref}}(y_w)$	$\log \pi_\theta(y_l)$	$\log \pi_{\text{ref}}(y_l)$	D	Loss
A	-30	-30	-40	-40	0	0,6931
B	-30	-28	-40	-45	-7	1,1032
C	-25	-30	-50	-40	15	0,2014

Cặp A: mô hình chưa lệch khỏi tham chiếu, loss $\ln 2$. Cặp B: mô hình đã làm câu thắng kém đi (-30 so với -28) và nâng câu thua (-40 so với -45), loss lớn và trọng số $\sigma(0{,}7) = 0{,}6682$, tức bước cập nhật mạnh. Cặp C: làm đúng cả hai chiều, loss nhỏ. Trung bình ba cặp là $0{,}6659$.

Cặp B cho thấy một chi tiết tinh tế: DPO chỉ quan tâm **tỉ lệ so với tham chiếu**, không quan tâm xác suất tuyệt đối. Thực tế log-xác suất của **cả hai** câu có thể cùng giảm miễn là câu thua giảm nhiều hơn; đây là một đặc điểm mà các biến thể ở mục 9 nhắm tới.

9. Các biến thể, mỗi cái một câu

- **IPO (rl 30)**: thay loss log-sigmoid bằng một loss bình phương quanh một mức biên cố định, vì khi người luôn chọn một phía (ưa thích gần như xác định) thì DPO có thể tăng D vô hạn; ghi chú rl 30 mô tả điểm yếu overfit chung của RLHF và DPO trong tình huống đó.
- **KTO (rl 31)**: học từ nhãn đơn "tốt" hoặc "xấu" cho từng câu, không cần cặp, với hàm giá trị lấy cảm hứng từ lý thuyết triển vọng.
- **SimPO (rl 32)**: bỏ mô hình tham chiếu, dùng log-xác suất trung bình theo độ dài làm phần thưởng và thêm một mức biên mục tiêu vào Bradley-Terry.
- **DPO so với PPO (rl 33)**: một so sánh có hệ thống cho thấy, theo ghi chú rl 33, PPO có thể vượt DPO nếu chỉnh đúng vài yếu tố (chuẩn hoá lợi thế, batch lớn, EMA cho mô hình tham chiếu), nên "đơn giản hơn" của DPO không đồng nghĩa "luôn tốt hơn".
- **Nhãn do AI tạo (rl 26)**: thay người chọn bằng một mô hình chọn theo bộ nguyên tắc; phần còn lại của pipeline giữ nguyên.
- **Best-of-n (chọn câu điểm cao nhất trong n câu)**: không cần RL, chỉ cần mô hình thưởng; là baseline mạnh và là đối tượng đo của rl 27.

10. Biến thể và chi tiết hay bị bỏ qua

Tham chiếu phải khớp dữ liệu. DPO mặc định cặp ưa thích được sinh từ chính π_{ref}. Nếu dữ liệu đến từ một mô hình khác, tỉ số $\dfrac{\pi_\theta}{\pi_{\text{ref}}}$ không còn mang ý nghĩa như công thức suy ra; cách làm quen thuộc là SFT π_{ref} trước trên các câu thắng.

Online và offline. RLHF với PPO lấy mẫu mới từ chính sách hiện tại (online); DPO chuẩn dùng một tập cặp cố định (offline). Chính sách sau khi cập nhật có thể sinh ra câu nằm ngoài tập dữ liệu, mà DPO không có tín hiệu cho vùng đó.

β ảnh hưởng thế nào. Theo ghi chú rl 29, DPO mặc định $\beta = 0{,}1$ và tác vụ tóm tắt dùng $\beta = 0{,}5$. Trong ví dụ hai câu ở mục 5.2, β từ 1 xuống $0{,}1$ làm KL tăng từ $0{,}11$ lên $0{,}69$ nat; trực giác tương tự đúng cho DPO: β nhỏ cho phép lệch xa tham chiếu hơn.

Thiên lệch độ dài. Câu dài có tổng log-xác suất rất âm, nên các tỉ số log-xác suất theo cả câu dễ phụ thuộc độ dài; mô hình thưởng cũng có thể vô tình thích câu dài. Đây là lý do một số biến thể chuẩn hoá theo độ dài (SimPO ở trên).

Phần thưởng một số cho cả câu. Mô hình thưởng thường chấm ở token cuối; PPO phải học một mạng giá trị để phân bổ tín hiệu về từng token, điều khó và tốn bộ nhớ; chương nền 08 của cuốn này cho thấy cách GRPO bỏ mạng giá trị đó.

11. Nối sang các bài paper của cuốn

- **rl 23 và rl 24**: hai bài mở đầu của dòng này; Bradley-Terry ở mục 4 là mô hình xác suất cho lựa chọn cặp trong cả hai (theo ghi chú rl 23, học thưởng từ so sánh cặp giữa các quỹ đạo; theo ghi chú rl 24, phạt KL tới mô hình gốc và bộ điều khiển β động).
- **rl 25 và rl 28**: pipeline RLHF đầy đủ cho tóm tắt (rl 25) và các chi tiết cài đặt khiến kết quả thay đổi (rl 28); mục 3 là khung chung của chúng.
- **rl 27**: phép đo định lượng của mục 7.
- **rl 29**: DPO gốc; mục 8 là ý chính, ghi chú rl 29 có ví dụ số riêng và các thí nghiệm.
- **rl 30 đến 33**: các biến thể ở mục 9.

12. Sai lầm hay gặp

- Nghĩ mô hình thưởng cho điểm "tuyệt đối": nó chỉ có nghĩa ở hiệu điểm giữa các câu cùng prompt, thang đo do huấn luyện quyết định.
- Quên mô hình tham chiếu phải **đóng băng**: nếu π_{ref} cứ trôi theo π_θ thì phạt KL không còn giữ gì.
- Đọc "KL nhỏ nghĩa là an toàn": KL chỉ đo khoảng cách trung bình tới tham chiếu; vài câu bị khai thác nặng vẫn có thể đóng góp ít vào KL trung bình.
- Cho rằng điểm của mô hình thưởng tăng nghĩa là chất lượng tăng; mục 7 cho thấy hai thứ có thể tách ra, và phải đo bằng thước độc lập.
- Cho rằng DPO "không dùng phần thưởng": nó dùng phần thưởng ngầm $\beta \log \dfrac{\pi_\theta}{\pi_{\text{ref}}}$ và cùng mục tiêu có KL; chỉ khác ở cách tối ưu.
- Nghĩ β chỉ là một hằng số tuỳ ý: nó đặt thang giữa điểm thưởng và KL, nên đổi β đồng nghĩa đổi mức "đi xa" cho phép.

13. Tóm tắt công thức

Khái niệm	Công thức
Bradley-Terry	$P(y_w \succ y_l) = \sigma(r_w - r_l)$
Loss mô hình thưởng	$-\log \sigma(r_w - r_l)$
Mục tiêu có KL	$\mathbb{E}[r] - \beta\,\mathrm{KL}(\pi \Vert \pi_{\text{ref}})$
Chính sách tối ưu	$\pi^\star \propto \pi_{\text{ref}}\, e^{r/\beta}$
Phần thưởng ngầm	$\hat{r} = \beta \log \dfrac{\pi_\theta}{\pi_{\text{ref}}}$
Loss DPO	$-\log \sigma(\beta D)$ với D là hiệu log-tỉ số
Trọng số gradient DPO	$\sigma(-\beta D)$

14. Bài tập tự tính

Bài 1. Hiệu điểm $r_w - r_l = 2$. Xác suất mô hình gán cho câu thắng là bao nhiêu?

Đáp án: $\sigma(2) = 0{,}8808$ (tự tính).

Bài 2. Một cặp có $r_w = 0{,}4$, $r_l = 1{,}4$. Tính loss Bradley-Terry và hệ số gradient theo hiệu điểm.

Đáp án: $d = -1$, loss $= -\log \sigma(-1) = 1{,}3133$, hệ số gradient $-\sigma(1) = -0{,}7311$ (tự tính). Với learning rate $0{,}5$ thì r_w tăng thêm khoảng $0{,}3655$ trong bước này (tính trên hiệu điểm, bỏ qua tương tác tham số).

Bài 3. $\pi_{\text{ref}} = (0{,}5;\ 0{,}5)$, $r = (1;\ 0)$, $\beta = 0{,}5$. Tìm $\pi^\star$ và mục tiêu $\mathbb{E}[r] - \beta\,\mathrm{KL}$.

Đáp án: $\pi^\star = (0{,}8808;\ 0{,}1192)$, KL $= 0{,}3278$, mục tiêu $= 0{,}8808 - 0{,}5 \cdot 0{,}3278 = 0{,}7169$ (tự tính).

Bài 4. DPO với $\beta = 0{,}2$. Câu thắng: $\log \pi_\theta = -20$, $\log \pi_{\text{ref}} = -25$. Câu thua: $\log \pi_\theta = -30$, $\log \pi_{\text{ref}} = -22$. Tính D, loss và trọng số gradient.

Đáp án: log-tỉ số thắng $= +5$, thua $= -8$, $D = 13$, $\beta D = 2{,}6$; loss $= 0{,}0716$; trọng số $\sigma(-2{,}6) = 0{,}0691$ (tự tính). Cặp đã được xếp rất đúng nên gần như không còn gradient.

Bài 5. Với cùng $D = 3$, tính loss DPO khi $\beta = 0{,}1$ và khi $\beta = 0{,}5$, rồi giải thích vì sao β lớn làm mô hình bị giữ gần tham chiếu hơn.

Đáp án: $\beta = 0{,}1$: $\beta D = 0{,}3$, loss $0{,}5544$; $\beta = 0{,}5$: $\beta D = 1{,}5$, loss $0{,}2014$ (tự tính). Với β lớn, một độ lệch nhỏ D đã đủ làm loss gần 0 và trọng số gradient gần 0, nên mô hình ngừng được kéo sớm. Đây là suy luận từ hình dạng hàm, không phải kết luận của paper.

Tài liệu tham khảo nền: các ghi chú rl 23 đến 33 của cuốn này (nhất là rl 27 cho over-optimization và rl 29 cho DPO); math nền tảng 10 (MLE) và 11 (KL); tổng quan chung về học tăng cường trong Sutton và Barto, “Reinforcement Learning: An Introduction”.

Nền tảng 08

RL cho mô hình lập luận: phần thưởng kiểm chứng được và lợi thế tương đối theo nhóm (GRPO)

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất, kỳ vọng và phương sai ở math nền tảng 08; gradient ở math nền tảng 06; entropy và KL ở math nền tảng 11; softmax và loss ở ML cơ bản nền tảng 02, 05; policy gradient, baseline và PPO ở các bài rl 04 đến 07 (chương này nhắc lại phần cần dùng); nen07 của cuốn này (RLHF, phạt KL, mô hình tham chiếu)

Dùng ở: rl 34 (tự sinh lời giải rồi lọc theo đáp án), rl 35 (phần thưởng theo kết quả và theo bước), rl 36 (GRPO), rl 37 (RLOO), rl 38 (RLVR), rl 39 (DeepSeek-R1), rl 40 (Kimi k1.5), rl 41 (DAPO), rl 42 (Dr. GRPO), rl 43 (GSPO), rl 44 đến rl 52 (các phân tích về khám phá, entropy và RL cho lập luận)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy; số do mô phỏng được nói rõ là mô phỏng.

1. Vì sao cần chương này

Với bài toán có đáp án đúng sai rõ ràng (toán có đáp số, lập trình có bộ kiểm thử), ta không cần học một mô hình thưởng từ ý kiến người như ở nen07: chỉ cần một **bộ kiểm chứng** (verifier) chấm câu trả lời đúng hay sai. Khi đó RL trở nên rất sạch: sinh nhiều lời giải, chấm, tăng xác suất những lời giải đúng. Vấn đề còn lại là **làm sao dùng điểm 0/1 một cách hiệu quả** khi một câu trả lời dài hàng nghìn token và không có mạng giá trị đáng tin.

GRPO (group relative policy optimization) trả lời bằng một ý đơn giản: cho mỗi câu hỏi sinh một **nhóm** câu trả lời, rồi so sánh các câu trả lời với nhau ngay trong nhóm. Chương này dạy ý đó từ đầu, rồi đi qua những chi tiết khiến công thức "đơn giản" ấy hành xử khác nhau: chuẩn hoá theo độ dài, theo độ lệch chuẩn, entropy sụp đổ, và vai trò đáng ngờ của mô hình tham chiếu.

2. Nhắc nền: policy gradient và baseline

2.1 Policy gradient trong một dòng

Chính sách $\pi_\theta(o|q)$ sinh câu trả lời o cho câu hỏi q; phần thưởng $r(q,o)$ là một số. Mục tiêu là tăng phần thưởng kỳ vọng. Gradient (công thức REINFORCE) là

$$\nabla_\theta J = \mathbb{E}\big[r(q,o)\,\nabla_\theta \log \pi_\theta(o|q)\big].$$

Đọc bằng lời: câu nào có thưởng cao thì đẩy log-xác suất của nó lên, câu nào thưởng thấp thì không đẩy (hoặc đẩy xuống nếu thưởng âm).

2.2 Baseline giữ kỳ vọng, giảm phương sai

Nếu trừ vào r một đại lượng b **không phụ thuộc câu trả lời cụ thể** thì kỳ vọng của gradient không đổi (vì $\mathbb{E}_o[\nabla_\theta \log \pi_\theta(o)] = 0$), nhưng phương sai thường giảm mạnh. $A = r - b$ gọi là **lợi thế** (advantage): câu này hơn mức nền bao nhiêu.

Ví dụ tay (tính chính xác, không lấy mẫu). Một câu hỏi, chính sách chọn "đúng" với xác suất $p = \dfrac{1}{3}$ và "sai" với $\dfrac{2}{3}$; thưởng 1 hoặc 0. Xét thành phần gradient theo logit của đáp án đúng: $\nabla \log \pi(a) = \mathbb{1}[a = \text{đúng}] - p$.

- Không baseline: ước lượng một mẫu bằng $r \cdot (✗ - p)$. Trường hợp đúng (xác suất $\frac{1}{3}$): $1 \cdot \frac{2}{3} = \frac{2}{3}$; trường hợp sai: 0. Kỳ vọng $= \frac{1}{3} \cdot \frac{2}{3} = 0{,}2222$; phương sai $= \frac{1}{3} \cdot \frac{4}{9} - 0{,}2222^2 = 0{,}0988$.
- Baseline $b = p = \frac{1}{3}$: ước lượng $(r - b)(✗ - p)$. Trường hợp đúng: $\frac{2}{3} \cdot \frac{2}{3} = \frac{4}{9}$; trường hợp sai: $(-\frac{1}{3})(-\frac{1}{3}) = \frac{1}{9}$. Kỳ vọng vẫn $0{,}2222$; phương sai $= 0{,}0247$.

Tất cả là (tự tính). Cùng kỳ vọng, phương sai giảm đúng 4 lần. Đó là toàn bộ lý do để dùng baseline.

2.3 Baseline là mạng giá trị, hoặc là trung bình nhóm

PPO (rl 07) dùng một **mạng giá trị** V_ψ làm baseline, kèm GAE. Với mô hình ngôn ngữ, mạng giá trị cùng cỡ với chính sách (tốn bộ nhớ gấp đôi) và phải đoán phần thưởng tương lai tại từng token trong khi chỉ có một số cho cả câu. GRPO thay nó bằng **trung bình phần thưởng của cả nhóm** cùng một câu hỏi, không cần học thêm gì.

3. Phần thưởng kiểm chứng được

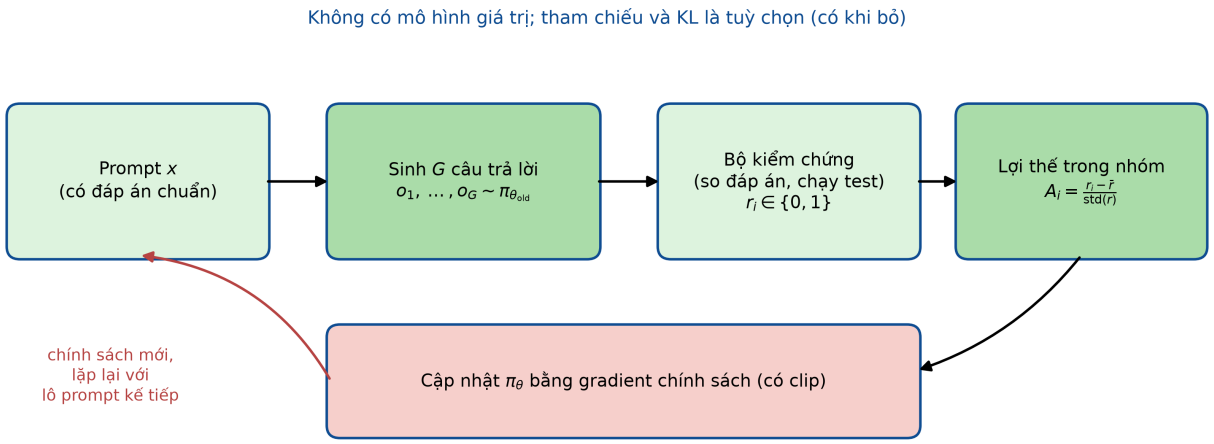
Phần thưởng kiểm chứng được (RLVR, reinforcement learning with verifiable rewards) là phần thưởng tính bằng luật cứng, không phải bằng mạng học từ ý kiến:

- Toán: so đáp án cuối (đặt trong khuôn cố định như `\boxed{}`) với đáp án chuẩn; đúng $= 1$, sai $= 0$.
- Lập trình: chạy bộ kiểm thử; qua hết $= 1$.
- Định dạng: thưởng nhỏ nếu câu trả lời đúng cấu trúc yêu cầu (ví dụ phần suy nghĩ nằm trong cặp thẻ).

Ưu điểm: không có mô hình thưởng để chính sách khai thác (nen07, mục 7), tín hiệu rẻ và lặp lại được. Hạn chế: chỉ chấm được **kết quả**, không chấm quá trình; không phải bài nào cũng có đáp án kiểm tra được; bộ kiểm chứng vẫn có thể có kẽ hở (kiểm thử yếu, so khớp đáp án lỏng) mà chính sách sẽ tìm ra.

Theo ghi chú rl 39, DeepSeek-R1-Zero dùng đúng kiểu này: phần thưởng là tổng của độ chính xác và định dạng, thuần theo luật, không dùng mô hình thưởng thần kinh cho tác vụ lập luận; với toán, đúng $= 1$ và sai $= 0$. Cùng ghi chú: điểm AIME 2024 pass@1 của R1-Zero tăng từ $15{,}6\%$ lên $77{,}9\%$ trong quá trình huấn luyện, và độ dài câu trả lời trung bình tăng liên tục.

4. Trực giác bằng hình: sinh, chấm điểm, cập nhật



Hình 23: Vòng lặp GRPO với phần thưởng kiểm chứng: sinh một nhóm, chấm điểm, chuẩn hoá trong nhóm, cập nhật (minh hoạ, tự tính)

Cách đọc hình: bốn ô phía trên là một bước thu thập dữ liệu. Với mỗi prompt, chính sách cũ sinh G câu trả lời; bộ kiểm chứng gán $r_i \in \{0, 1\}$; lợi thế A_i là điểm của câu i so với cả nhóm. Ô đỏ phía dưới là bước học:

dùng A_i làm trọng số của gradient chính sách (có clip như PPO). Mũi tên đỏ đóng vòng: chính sách mới sinh nhóm kế tiếp. Dòng trên cùng nhắc rằng không có mạng giá trị, còn mô hình tham chiếu và phạt KL chỉ là tuỳ chọn (mục 9).

Trực giác của "so trong nhóm": hãy hình dung một lớp học chấm bài bằng đáp án. Nếu cả lớp được 10 điểm thì bài nào cũng không "nổi trội" để học theo; nếu chỉ vài bạn đúng, bài của các bạn ấy rất đáng bắt chước. Lợi thế tương đối đo đúng điều đó.

5. GRPO: định nghĩa chính thức

Cho câu hỏi q, lấy nhóm G câu trả lời $o_1, \ldots, o_G \sim \pi_{\theta_{\text{old}}}$ với phần thưởng $r_1, \ldots, r_G$. Lợi thế của câu i:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}.$$

Cả câu o_i dùng chung một giá trị $\hat{A}_i$ cho mọi token của nó. Với tỉ số $\rho_{i,t} = \dfrac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$ và ngưỡng clip ε, mục tiêu GRPO (theo ghi chú rl 36) là

$$J_{\text{GRPO}} = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left(\min\left(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right) - \beta\,\text{KL}\left(\pi_\theta \| \pi_{\text{ref}}\right)\right)\right].$$

So với PPO: (1) không có mạng giá trị, baseline là trung bình nhóm; (2) KL cộng thẳng vào mục tiêu chứ không nhúng vào phần thưởng từng token; (3) hai phép chia là $\frac{1}{G}$ (trung bình trên nhóm) rồi $\frac{1}{|o_i|}$ (trung bình trên token của từng câu). Hai phép chia và phép chia cho std chính là những chi tiết mà các mục 7 và 8 sẽ mổ xẻ.

6. Ví dụ tay: nhóm 6 câu trả lời

Nhóm $G = 6$, phần thưởng $r = [1, 0, 0, 1, 0, 0]$ (hai câu đúng). Dùng độ lệch chuẩn dân số (chia cho G).

- Trung bình $= \dfrac{2}{6} = 0{,}3333$. Phương sai $= \dfrac{1}{6}\left[2(0{,}6667)^2 + 4(0{,}3333)^2\right] = 0{,}2222$, nên std $= 0{,}4714$ (tự tính).
- Lợi thế GRPO của câu đúng: $\dfrac{1 - 0{,}3333}{0{,}4714} = +1{,}4142$; của câu sai: $\dfrac{0 - 0{,}3333}{0{,}4714} = -0{,}7071$.
- Baseline trung bình (chỉ trừ trung bình, không chia std): câu đúng $+0{,}6667$, câu sai $-0{,}3333$.

Cách đọc hình: bên trái là đúng nhóm của ví dụ; cột xanh là lợi thế khi chỉ trừ trung bình, cột đỏ là GRPO. Hai loại cùng dấu, GRPO chỉ **phóng đại đều** (hệ số $\dfrac{1}{0{,}4714} = 2{,}12$). Bên phải đổi số câu đúng k từ 1 đến 5 trong nhóm 6: với GRPO, câu đúng duy nhất trong nhóm khó có lợi thế $+2{,}236$, câu sai duy nhất trong nhóm dễ có $-2{,}236$ (tự tính); với chỉ trừ trung bình các độ lớn nhỏ hơn và gọn hơn. Hai đường đối xứng quanh $k = 3$.

Thêm ba nhóm để thấy hành vi (tự tính):

Nhóm r	mean	std	Lợi thế câu đúng	Lợi thế câu sai
[1,0,0,1,0,0]	0,3333	0,4714	+1,4142	-0,7071
[1,1,1,1,1,0]	0,8333	0,3727	+0,4472	-2,2361
[1,0,0,0,0,0]	0,1667	0,3727	+2,2361	-0,4472

Nhóm r	mean	std	Lợi thế câu đúng	Lợi thế câu sai
[0,0,0,0,0,0] hoặc [1,1,1,1,1,1]		0	không xác định (thực tế đặt 0)	

Nhóm toàn đúng hoặc toàn sai cho std $= 0$, mọi lợi thế bằng 0 và nhóm đó **không đóng góp gradient nào**. Nếu mỗi lần sinh có xác suất đúng p độc lập, xác suất nhóm 6 bị rơi vào trường hợp này là $p^6 + (1-p)^6$: với $p = 0{,}5$ chỉ 3,1%, nhưng với $p = 0{,}9$ đã là $53{,}1\%$ và với $p = 0{,}98$ là $88{,}6\%$ (tự tính). Câu quá dễ hoặc quá khó đều "mất" cả nhóm. Điều này giải thích vì sao DAPO lọc nhóm kiểu này (mục 7.3).

So với RLOO. Biến thể "trừ trung bình của các câu còn lại" (leave-one-out, rl 37) cho nhóm đầu: câu đúng $1 - \frac{1}{5} = 0{,}8$, câu sai $0 - \frac{2}{5} = -0{,}4$ (tự tính). Đó chính là lợi thế trừ trung bình nhân với $\frac{G}{G-1} = 1{,}2$ ($0{,}6667 \times 1{,}2 = 0{,}8$), một hệ số có thể gộp vào learning rate. Theo ghi chú rl 42, Dr. GRPO nhấn mạnh sự tương đương này.

So với baseline trung bình. Chia thêm cho std không phải "chuẩn hoá vô hại": nó đổi trọng số tương đối **giữa các câu hỏi** (hình bên phải). Mục 7.2 trở lại điểm này.

7. Chuẩn hoá: theo độ dài, theo độ lệch chuẩn, theo nhóm

7.1 Độ dài: mức mẫu hay mức token

Công thức ở mục 5 có $\frac{1}{G}\sum_i \frac{1}{|o_i|}\sum_t$: mỗi **câu** có trọng số bằng nhau, bất kể dài hay ngắn (mức mẫu). Hệ quả: mỗi token của câu dài nhận trọng số nhỏ hơn. Cách thay thế là chia cho tổng số token của cả nhóm, $\frac{1}{\sum_i |o_i|}\sum_i \sum_t$ (mức token): mọi token nặng như nhau.

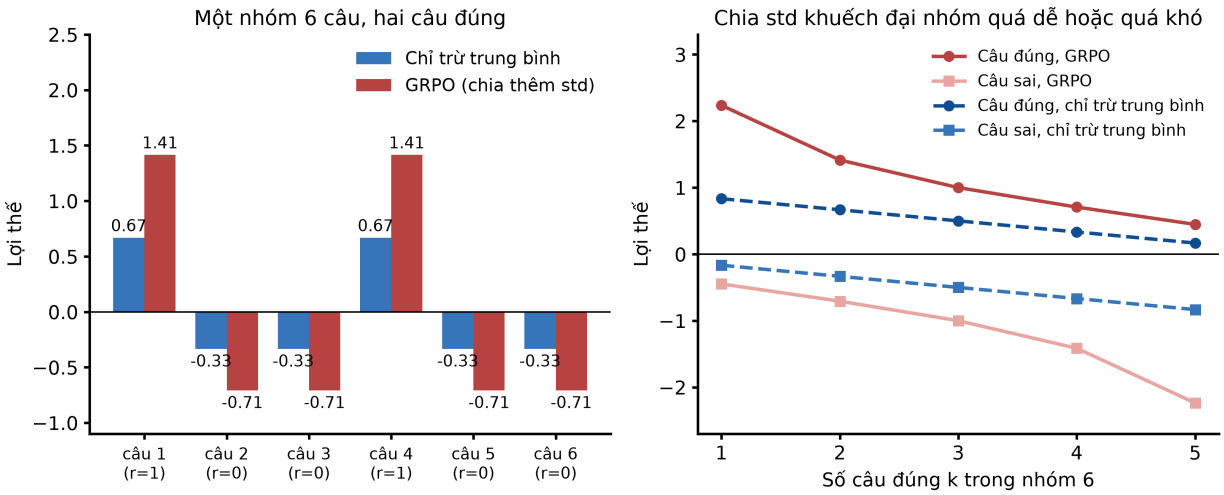
Ví dụ tay. Nhóm hai câu, A đúng dài 10 token (lợi thế dương), B sai dài 40 token (lợi thế âm). Mức mẫu: mỗi token của A nhận $\frac{1}{2 \cdot 10} = 0{,}05$, mỗi token của B nhận $\frac{1}{2 \cdot 40} = 0{,}0125$. Mức token: mọi token nhận $\frac{1}{50} = 0{,}02$ (tự tính). Ở mức mẫu, mỗi token của câu sai dài bị trừng phạt chỉ bằng $\frac{1}{4}$ mỗi token của câu đúng ngắn được thưởng.

Cách đọc hình: trục ngang là độ dài câu sai B (từ 10 đến 400 token), câu A cố định 10 token. Bên trái (mức mẫu): token của A giữ $0{,}05$, token của B tụt theo $\frac{1}{2|B|}$, tức B càng dài thì mỗi token của nó càng bị bỏ qua. Bên phải (mức token): trọng số mỗi token chung một đường xanh; đường đứt xanh lục và đỏ là tổng trọng số của A và B, B dài thì chiếm gần hết. Vì vậy mức token làm câu dài có ảnh hưởng lớn hơn lên gradient.

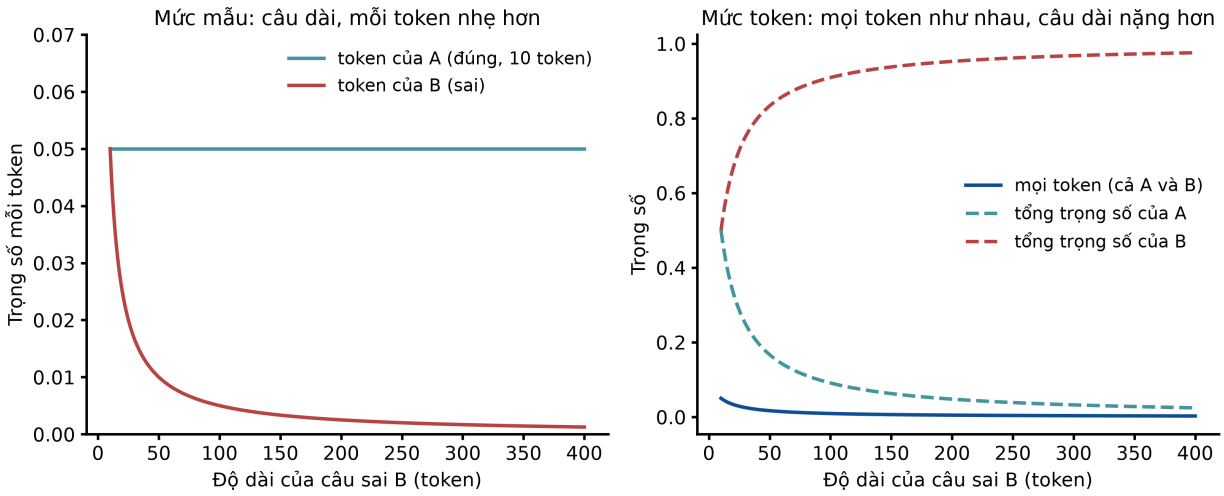
Theo ghi chú rl 42, Dr. GRPO phân tích đúng hiệu ứng này: chia cho $|o_i|$ khiến câu sai dài bị phạt nhẹ hơn trên từng token, nên độ dài câu sai có xu hướng tăng; Dr. GRPO bỏ phép chia và chia cho một hằng số cố định. **Theo ghi chú rl 41**, DAPO chọn hướng mức token và nêu hai tác hại của mức mẫu: mô hình khó học các mẫu lập luận quan trọng trong câu dài chất lượng cao, và câu quá dài chứa mẫu kém (lặp, vô nghĩa) không bị phạt hiệu quả. Hai bài cùng thấy vấn đề ở độ dài nhưng sửa bằng hai cách khác nhau; chương này không phân xử hơn thua, chỉ chỉ ra cơ chế.

7.2 Chuẩn hoá theo độ lệch chuẩn

Chia cho std của nhóm có tác dụng thứ hai: câu hỏi mà nhóm có std nhỏ (rất dễ hoặc rất khó) được khuếch đại. Ví dụ ở mục 6: nhóm 6 câu với một câu đúng có lợi thế $+2{,}236$ so với $+1{,}0$ của nhóm một nửa đúng, hoàn toàn do chia cho std, không do câu đó "tốt hơn". **Theo ghi chú rl 42**, đây là "thiên lệch độ khó" theo cách nhìn của Dr. GRPO, với ví dụ $G = 8$ và một câu đúng trên tám cho hệ số khoảng $2{,}65$ so với 1 của nhóm một nửa đúng.



Hình 24: Lợi thế theo nhóm: trừ trung bình so với GRPO, và ảnh hưởng của số câu đúng trong nhóm (minh hoạ, tự tính)



Hình 25: Trọng số mỗi token: mức mẫu so với mức token, khi câu sai B dài ra (minh hoạ, tự tính)

Đây là một đánh đổi chứ không phải đúng sai: khuếch đại câu khó có thể giúp học từ câu hiếm đúng, hoặc có thể làm gradient bị chi phối bởi vài câu. Ghi chú rl 42 chọn bỏ std; các bài khác trong cuốn (như rl 41 và rl 43) giữ nó. Đây là một lựa chọn thiết kế mà người đọc nên biết là có, không phải "chi tiết hiển nhiên".

7.3 Lọc nhóm vô ích và nhiễu từ câu bị cắt

Hai chi tiết thực dụng. **Dynamic sampling** (theo ghi chú rl 41): bỏ các prompt có độ chính xác 0 hoặc 1 vì lợi thế bằng 0, rồi lấy thêm prompt cho đến khi đủ lô; điều kiện trong mục tiêu của DAPO là số câu đúng thuộc khoảng mở $(0, G)$, khớp với tính toán ở mục 6. **Câu bị cắt** (vượt độ dài tối đa) cho phần thưởng nhiễu vì câu có thể đúng nếu được viết tiếp; DAPO xử lý bằng cách che loss hoặc phạt mềm theo độ dài.

Để biết quy mô, bảng ablation tích luỹ của DAPO (theo ghi chú rl 41, AIME 2024 avg@32): GRPO nguyên bản 30, thêm lọc câu quá dài 36, thêm Clip-Higher 38, thêm phạt mềm câu quá dài 41, thêm loss mức token 42, thêm dynamic sampling 50. Các con số này là của paper trên một mô hình và một cấu hình; chương này dùng chúng để cho thấy mỗi "chi tiết nhỏ" có thể đáng giá nhiều điểm, không phải để nói hiệu quả sẽ giống thế ở mọi nơi.

7.4 Mức chuỗi: GSPO

Tỉ số $\rho_{i,t}$ là tỉ số **từng token**, tính từ một mẫu duy nhất. **Theo ghi chú rl 43**, GSPO cho rằng đơn vị tối ưu nên khớp đơn vị phần thưởng (một số cho cả chuỗi) và dùng tỉ số mức chuỗi chuẩn hoá theo độ dài, $s_i = \left(\frac{\pi_\theta(o_i)}{\pi_{\theta_{\text{old}}}(o_i)}\right)^{1/|o_i|}$, clip ở mức chuỗi với khoảng clip nhỏ hơn nhiều (ghi chú nêu 3×10^{-4} và 4×10^{-4} so với $0{,}2$ và $0{,}27$ của GRPO baseline). Động lực nêu trong ghi chú là ổn định khi huấn luyện mô hình MoE, nơi khoảng 10% chuyên gia được kích hoạt khác nhau sau mỗi lần cập nhật làm tỉ số từng token dao động. Đây là hướng đi thứ ba, cùng gốc "đơn vị của loss" với mục 7.1.

8. Entropy và sụp đổ khám phá

8.1 Entropy của chính sách

Entropy $H(\pi) = -\sum_a \pi(a) \log \pi(a)$ đo độ "phân vân" của chính sách: cao khi xác suất dàn đều, bằng 0 khi chắc chắn hoàn toàn. Với mô hình ngôn ngữ ta lấy trung bình trên các token sinh ra. Entropy là thước đo mức **khám phá**: nhóm 6 câu giống hệt nhau thì không có gì để so sánh, lợi thế bằng 0 cả nhóm.

8.2 Vì sao entropy giảm

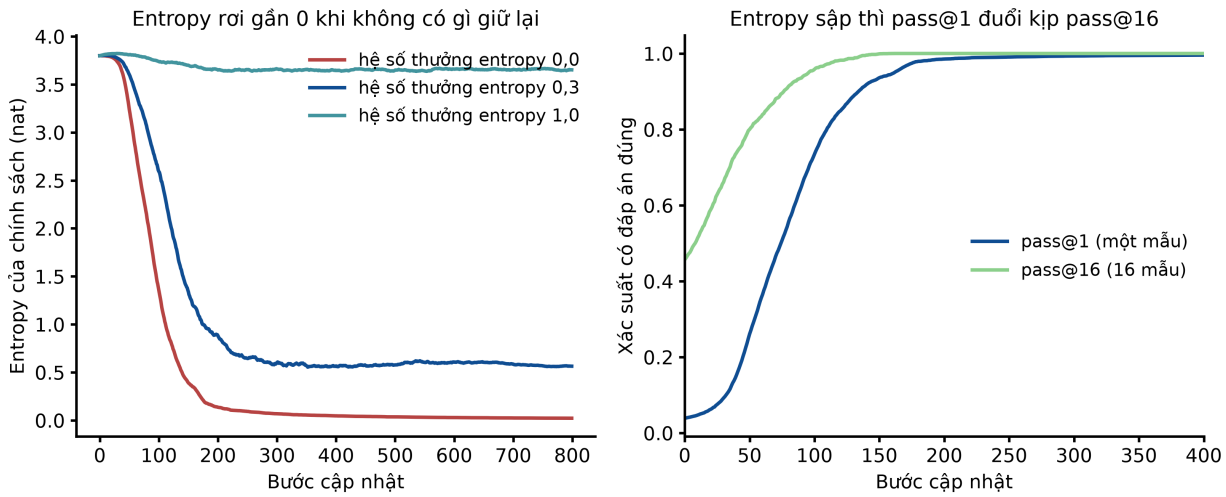
Ví dụ tay (tự tính). Chính sách hai hành động $\pi = (0{,}9;\ 0{,}1)$, entropy $0{,}3251$ nat. Dưới policy gradient với bước $\eta = 0{,}1$, logit của hành động a đổi một lượng xấp xỉ $\eta\, \pi(a) A(a)$.

- Trường hợp 1, hành động đang chiếm ưu thế cũng có lợi thế dương, $A = (+1; -1)$: logit đổi $(+0{,}09; -0{,}01)$, chính sách mới $(0{,}9086;\ 0{,}0914)$, entropy $0{,}3057$: **giảm** $0{,}0194$.
- Trường hợp 2, hành động hiếm có lợi thế dương, $A = (-1; +1)$: chính sách mới $(0{,}8906;\ 0{,}1094)$, entropy $0{,}3452$: **tăng** $0{,}0201$.

Khớp với công thức xấp xỉ mà **ghi chú rl 49** nêu: thay đổi entropy xấp xỉ bằng âm hiệp phương sai giữa log-xác suất hành động và thay đổi logit của nó. Hành động xác suất cao mà lại có lợi thế cao thì entropy đi xuống; hành động hiếm mà có lợi thế cao thì entropy đi lên. Theo ghi chú đó, paper đo thấy tương quan này dương trong RLVR nên entropy giảm đơn điệu.

8.3 Mô phỏng sụp đổ

Cách đọc hình (đây là **mô phỏng** trên một chính sách softmax đơn giản, không phải đo từ mô hình ngôn ngữ): một prompt, 50 đáp án khả dĩ trong đó 2 đáp án đúng, nhóm $G = 8$, cập nhật kiểu GRPO với lợi thế chuẩn hoá theo nhóm, trung bình 20 seed, entropy tính bằng nat. Bên trái: không có thưởng entropy (đường đỏ), entropy rơi từ khoảng $3{,}80$ xuống khoảng $0{,}14$ sau 200 bước và $0{,}02$ sau 800 bước; thưởng entropy vừa phải (hệ số



Hình 26: Mô phỏng: entropy theo bước với các hệ số thưởng entropy khác nhau, và pass@1, pass@16 (minh hoạ, tự tính)

$0{,}3$, đường xanh) làm chậm nhưng vẫn rơi, dừng quanh $0{,}56$; thưởng quá lớn (hệ số $1{,}0$, đường xanh lục) giữ entropy $3{,}65$ nhưng học gần như đứng yên (độ chính xác chỉ khoảng $0{,}24$ ở bước 800, theo mô phỏng). Bên phải (không thưởng entropy): pass@1 (một mẫu) tăng từ $0{,}04$ lên $0{,}985$ sau 200 bước, trong khi pass@16 (có ít nhất một câu đúng trong 16 mẫu) vốn đã cao ($0{,}46$ ở bước 0) và hai đường dần gặp nhau ở mức 1.

Chú ý để khỏi hiểu sai: trong mô phỏng này đáp án đúng luôn thắng nên pass@16 cũng lên 1. Điều đáng nhớ là **độ lợi của việc lấy nhiều mẫu biến mất** khi chính sách đã chắc chắn, vì 16 mẫu giờ gần giống nhau. Còn việc entropy tụt về gần 0 nghĩa là chính sách đã ngừng thử những hướng khác; nếu hướng đã chọn chưa phải tốt nhất thì khó quay lại.

Theo ghi chú rl 49, trên 11 mô hình nền, 73% lượng entropy tiêu hao và 76% điểm hiệu năng tăng thêm xảy ra trong 200 bước đầu của 2400 bước GRPO, và paper nêu quy luật thực nghiệm $R = -a\exp(H) + b$ giữa hiệu năng R và entropy H (với a, b là hệ số khớp theo từng mô hình và dữ liệu), kèm hai kỹ thuật Clip-Cov và KL-Cov chỉ hạn chế cập nhật các token có hiệp phương sai cao nhất. Trong bảng của ghi chú, điểm trung bình Qwen2.5-32B tăng từ $45{,}8$ (GRPO) lên $50{,}3$ (Clip-Cov) và $52{,}2$ (KL-Cov); với Qwen2.5-7B từ $38{,}6$ lên $40{,}4$ và $40{,}6$. Đó là kết quả của paper trong thiết lập của nó.

8.4 Clip-Higher

Cùng vấn đề, cách tiếp cận khác của DAPO (ghi chú rl 41): tách ε_{low} và $\varepsilon_{\text{high}}$, nâng $\varepsilon_{\text{high}}$ lên $0{,}28$ trong khi giữ $\varepsilon_{\text{low}} = 0{,}2$. Lý do là với lợi thế dương, trần của xác suất mới là $\pi_{\text{old}}(1+\varepsilon)$: token hiếm với $\pi_{\text{old}} = 0{,}01$ bị chặn ở $0{,}012$ khi $\varepsilon = 0{,}2$ (và $0{,}0128$ khi $\varepsilon_{\text{high}} = 0{,}28$, tự tính), trong khi token đã chắc ($0{,}9$) hầu như không bị chặn. Clip đối xứng vì vậy thiên về token đã có xác suất cao, và entropy giảm.

9. Mô hình tham chiếu và KL: vì sao có khi bỏ

Ở nen07, phạt KL tới π_{ref} giữ chính sách khỏi vùng mà **mô hình thưởng học được** không còn đáng tin. Với RLVR, lý do đó yếu đi vì phần thưởng đến từ bộ kiểm chứng chứ không phải từ mạng có thể bị khai thác theo kiểu ngoài phân phối. Thêm vào đó, để học cách lập luận mới (câu dài hơn, tự kiểm tra, thử hướng khác) chính sách phải **đi xa** mô hình khởi tạo, nên một sợi dây KL chặt là phản tác dụng. Mô hình tham chiếu cũng tốn thêm một lần tính xác suất cho mọi token. Đây là lập luận của chương dựa trên các ghi chú sau, không phải một định lý.

Quan sát trong các ghi chú:

- **rl 36 (GRPO gốc)**: có hạng tử β KL với ước lượng $\frac{\pi_{\text{ref}}}{\pi_\theta} - \log\frac{\pi_{\text{ref}}}{\pi_\theta} - 1$, không âm. Ví dụ tay (tự tính): $u = \frac{\pi_{\text{ref}}}{\pi_\theta} = 0{,}5$ cho $0{,}5 - \ln 0{,}5 - 1 = 0{,}1931$; $u = 2$ cho $2 - \ln 2 - 1 = 0{,}3069$; $u = 1$ cho 0. Không âm vì $e^x \geq 1 + x$.
- **rl 39 (R1)**: $\beta = 0{,}001$, một hệ số rất nhỏ, và mô hình tham chiếu được thay bằng chính sách mới nhất mỗi 400 bước.
- **rl 41 (DAPO)**: bỏ hẳn hạng tử KL, với lý do cần trôi xa khỏi khởi tạo khi huấn luyện lập luận dài.
- **rl 42 (Dr. GRPO)**: hệ số KL bằng 0 trong thiết lập thí nghiệm.
- **rl 49**: hệ số KL tham chiếu bằng 0 trong các thí nghiệm; paper đề xuất KL-Cov, một dạng phạt KL chỉ áp lên các token có hiệp phương sai lớn nhất.

Tóm lại, "bỏ KL" là lựa chọn phổ biến trong các bài này nhưng không phổ quát (R1 vẫn giữ một lượng rất nhỏ), và KL vẫn xuất hiện lại ở dạng chọn lọc. Người đọc nên coi β là một siêu tham số cần cân nhắc theo bài toán, không phải một mặc định.

10. Biến thể và chi tiết hay bị bỏ qua

- **Std dân số hay mẫu.** Một số cài đặt chia cho $G - 1$ thay vì G. Với nhóm của mục 6: std mẫu $0{,}5164$ cho lợi thế câu đúng $1{,}2910$ thay vì $1{,}4142$ (tự tính). Số khác nhau một chút, nhưng đừng đối chiếu số giữa hai cài đặt khi chưa kiểm cách chia.
- **Chia cho 0.** Khi nhóm toàn đúng hoặc toàn sai, cài đặt thực thêm một hằng nhỏ ϵ vào mẫu số, nên lợi thế bằng 0 chứ không phải vô cùng.
- **Phần thưởng theo bước.** Với phần thưởng theo từng bước lập luận (rl 35, rl 36 mục process supervision), lợi thế mỗi token là tổng phần thưởng chuẩn hoá của các bước từ đó trở đi, nên các token có lợi thế khác nhau thay vì chung một giá trị.
- **Khám phá vs khai thác.** Lấy nhiều mẫu mỗi câu hỏi (G lớn) làm baseline nhóm ít nhiễu hơn nhưng tốn tính toán; ghi chú rl 39 dùng $G = 16$ với 32 câu hỏi mỗi bước, tức lô 512 câu trả lời.
- **Pass@k và Maj@k.** Theo ghi chú rl 36, RL cải thiện Maj@K rõ nhưng gần như không cải thiện Pass@K; diễn giải của tác giả là RL làm phân phối "bền" hơn chứ không mở rộng năng lực nền. Vì vậy câu hỏi "RL có dạy mô hình điều mới không" vẫn là một câu hỏi mở, và là chủ đề của một số bài sau trong cuốn.

11. Nối sang các bài paper của cuốn

- **rl 34 và rl 35**: tiền thân. STaR tự sinh lời giải rồi lọc theo đáp án đúng rồi SFT: gần với "chỉ thưởng câu đúng" (hệ số $\{0, 1\}$ trong khung của rl 36); rl 35 so phần thưởng theo kết quả và theo bước, tức hai cách gán lợi thế ở mục 10.
- **rl 36**: GRPO gốc; mục 5 và 6 là công thức và ví dụ của nó. Ghi chú rl 36 còn có bảng thống nhất SFT, RFT, DPO, PPO, GRPO qua "hệ số gradient".
- **rl 37**: RLOO, baseline bỏ-một ở mục 6.
- **rl 38**: RLVR cho một công thức post-training mở; phần thưởng kiểm chứng ở mục 3.
- **rl 39 và rl 40**: hai hệ thống lập luận lớn; số liệu của rl 39 ở mục 3 và 9.
- **rl 41, 42, 43**: ba hướng sửa GRPO (mục 7 và 8.4): DAPO (bốn kỹ thuật), Dr. GRPO (bỏ hai phép chia), GSPO (tỉ số mức chuỗi).
- **rl 44 đến 52**: đo, phân tích và giải thích RL cho lập luận (quy mô, entropy, liệu RL có tạo năng lực mới); entropy ở mục 8 là nền cho rl 49 và rl 50.

12. Sai lầm hay gặp

- Nghĩ lợi thế trong nhóm là giá trị tuyệt đối của câu trả lời: nó chỉ so với các câu cùng nhóm, đổi nhóm là đổi lợi thế.

- Quên nhóm toàn đúng hoặc toàn sai không cho gradient: lô có nhiều nhóm như vậy lãng phí tính toán và tăng phương sai.
- Gán lợi thế dương cho cả câu đúng và đổ lỗi cho mọi token: chuẩn hoá theo kết quả không biết token nào quyết định, vì thế câu đúng dài vẫn có token thừa được thưởng.
- Cho rằng chia theo độ dài hay theo std là "chi tiết không quan trọng": mục 7 cho thấy hai phép chia đổi trọng số giữa câu dài và ngắn, câu dễ và khó.
- Cho rằng entropy giảm là dấu hiệu thành công: giảm khi mô hình tự tin đúng là tốt, nhưng giảm quá nhanh cắt mất khám phá; cần theo dõi cả pass@k.
- Bỏ KL mà không theo dõi gì khác: khi không có sợi dây tham chiếu, phải có thước đo độc lập (đánh giá riêng, độ dài, entropy) để biết chính sách có trôi bất thường không.

13. Tóm tắt công thức

Khái niệm	Công thức
Lợi thế nhóm (GRPO)	$\hat{A}_i = \dfrac{r_i - \text{mean}(r)}{\text{std}(r)}$
Chỉ trừ trung bình	$\tilde{A}_i = r_i - \text{mean}(r)$
RLOO	$r_i - \dfrac{1}{G-1}\sum_{j\neq i} r_j = \dfrac{G}{G-1}\tilde{A}_i$
Loss mức mẫu	$\dfrac{1}{G}\sum_i \dfrac{1}{\lvert o_i\rvert}\sum_t(\cdot)$
Loss mức token	$\dfrac{1}{\sum_i \lvert o_i\rvert}\sum_i\sum_t(\cdot)$
Entropy	$H = -\sum_a \pi(a)\log\pi(a)$
Ước lượng KL không âm	$u - \log u - 1$ với $u = \dfrac{\pi_{\text{ref}}}{\pi_\theta}$

14. Bài tập tự tính

Bài 1. Nhóm $G = 6$ có $r = [1, 0, 0, 0, 0, 0]$. Tính lợi thế GRPO và lợi thế chỉ trừ trung bình của câu đúng và câu sai.

Đáp án: mean $= 0{,}1667$, std $= 0{,}3727$. GRPO: câu đúng $+2{,}2361$, câu sai $-0{,}4472$. Chỉ trừ trung bình: $+0{,}8333$ và $-0{,}1667$ (tự tính).

Bài 2. Nhóm $G = 6$ có $r = [1, 1, 1, 0, 0, 0]$. Tính ba loại lợi thế: GRPO, chỉ trừ trung bình, RLOO.

Đáp án: mean $= 0{,}5$, std $= 0{,}5$. GRPO ± 1; chỉ trừ trung bình $\pm 0{,}5$; RLOO $\pm 0{,}6$ (câu đúng $1 - \dfrac{2}{5} = 0{,}6$, câu sai $0 - \dfrac{3}{5} = -0{,}6$), bằng $\dfrac{6}{5} \times 0{,}5$ (tự tính).

Bài 3. Một lô có 4 nhóm, mỗi nhóm $G = 6$. Ba nhóm có $r = [0, 0, 0, 0, 0, 0]$ hoặc $[1, 1, 1, 1, 1, 1]$, một nhóm có $r = [1, 0, 0, 1, 0, 0]$. Có bao nhiêu câu trả lời thực sự đóng góp gradient?

Đáp án: chỉ nhóm cuối đóng góp: 6 trên 24 câu, tức 25% (tự tính). Ba nhóm còn lại có lợi thế 0.

Bài 4. Nhóm hai câu: câu A dài 20 token và câu B dài 60 token. Tính trọng số mỗi token của loss mức mẫu và mức token.

Đáp án: mức mẫu: mỗi token của A nhận $\dfrac{1}{2 \cdot 20} = 0{,}025$, mỗi token của B nhận $\dfrac{1}{2 \cdot 60} = 0{,}0083$; mức token: mọi token nhận $\dfrac{1}{80} = 0{,}0125$ (tự tính). Ở mức mẫu, token của A nặng gấp 3 lần token của B.

Bài 5. Chính sách hai hành động $\pi = (0{,}9;\ 0{,}1)$. Dùng công thức $\Delta z = \eta\,\pi A$ với $\eta = 0{,}1$ và $A = (+1; -1)$, hãy tính entropy trước và sau cập nhật; sau đó thử $A = (-1; +1)$ và giải thích kết quả.

Đáp án: trước $0{,}3251$ nat. Với $A = (+1; -1)$: $\Delta z = (0{,}09; -0{,}01)$, chính sách mới $(0{,}9086;\ 0{,}0914)$, entropy $0{,}3057$, giảm. Với $A = (-1; +1)$: chính sách mới $(0{,}8906;\ 0{,}1094)$, entropy $0{,}3452$, tăng (tự tính). Khi hành động ưu thế được thưởng thêm thì chính sách tự tin hơn nên entropy giảm; khi hành động hiếm được thưởng thì xác suất được chia đều hơn nên entropy tăng.

Tài liệu tham khảo nền: các ghi chú rl 34 đến 52 của cuốn này (nhất là rl 36 cho GRPO, rl 39 cho R1, rl 41 đến 43 cho các bản sửa GRPO, rl 49 cho entropy); nen07 của cuốn này; Sutton và Barto, “Reinforcement Learning: An Introduction” cho policy gradient và baseline.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc về Reinforcement Learning

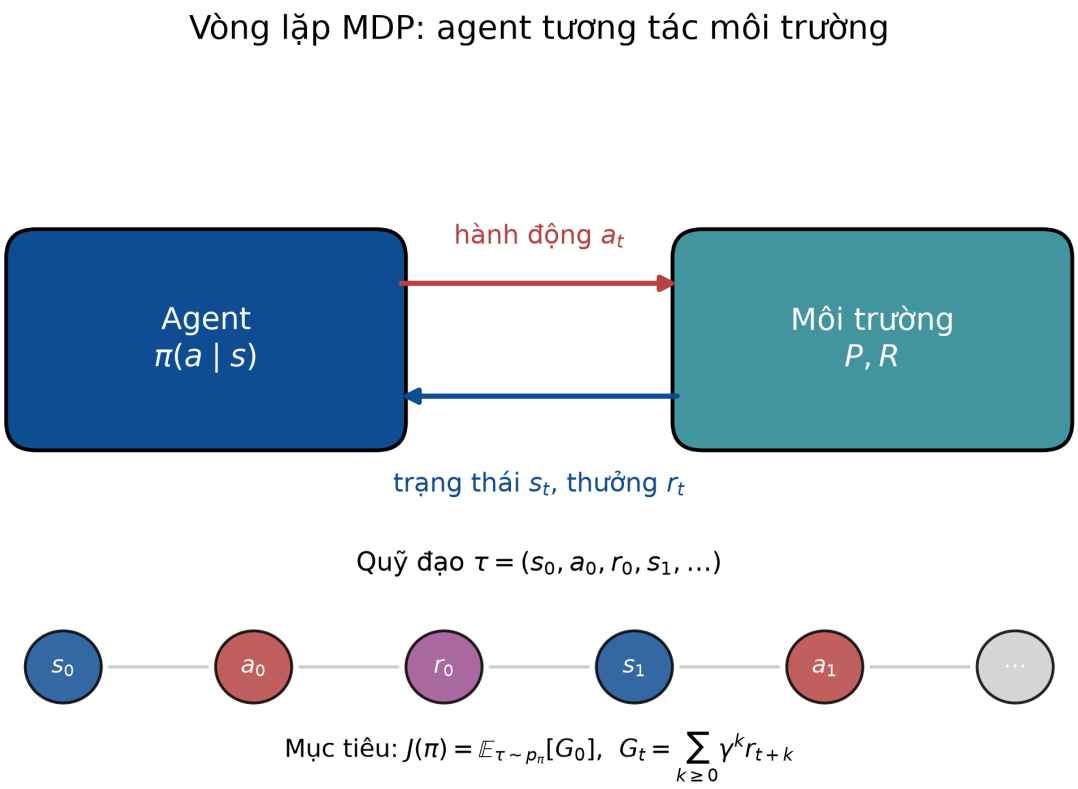
Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú một paper cụ thể)
Venue / arXiv: không áp dụng
Nguồn: các bài 01–57 trong thư mục này; phần kinh điển không có trên arXiv (Bellman, Q-learning, REINFORCE của Williams 1992, policy gradient theorem của Sutton et al. 1999, sách Sutton & Barto) được trình bày lại ở đây
Chủ đề: RL từ con số 0: MDP, return, value function, phương trình Bellman, dynamic programming, Monte Carlo và TD, Q-learning, policy gradient, baseline, actor-critic, GAE, importance sampling, trust region và PPO, entropy; RL cho LLM (RLHF, KL penalty, DPO, GRPO, RLVR, pass@k); cạm bẫy thực hành, thuật ngữ, cách đọc paper, lịch đọc.
Vị trí trong lộ trình: Bài số 00, đọc trước tất cả. Sau đó đi theo 01–09 (deep RL cốt lõi), rồi tùy mục tiêu: 10–22 (điều khiển liên tục, planning, imitation, offline, multi-agent) hoặc 23–57 (RL cho LLM, VLM và agent).

> Tài liệu học tập bằng tiếng Việt, viết từ kiến thức chung của người biên soạn, **không** là ghi chú của một paper và chưa được đối chiếu từng câu với nguồn. Các con số minh họa đều tự tính. Khi một paper trong thư mục nói khác, hãy tin paper và kiểm tra lại.

1. RL là gì và khác gì supervised learning

1.1 Định nghĩa

Reinforcement learning (RL, học tăng cường): một **agent** (tác tử) tương tác với **environment** (môi trường) qua nhiều bước. Ở mỗi bước, agent quan sát **state** s_t, chọn **action** a_t, nhận **reward** r_t (một số thực) và chuyển sang state mới s_{t+1}. Mục tiêu: học một **policy** $\pi(a \mid s)$ (quy tắc chọn hành động) sao cho **tổng phần thưởng tích lũy** là lớn nhất.



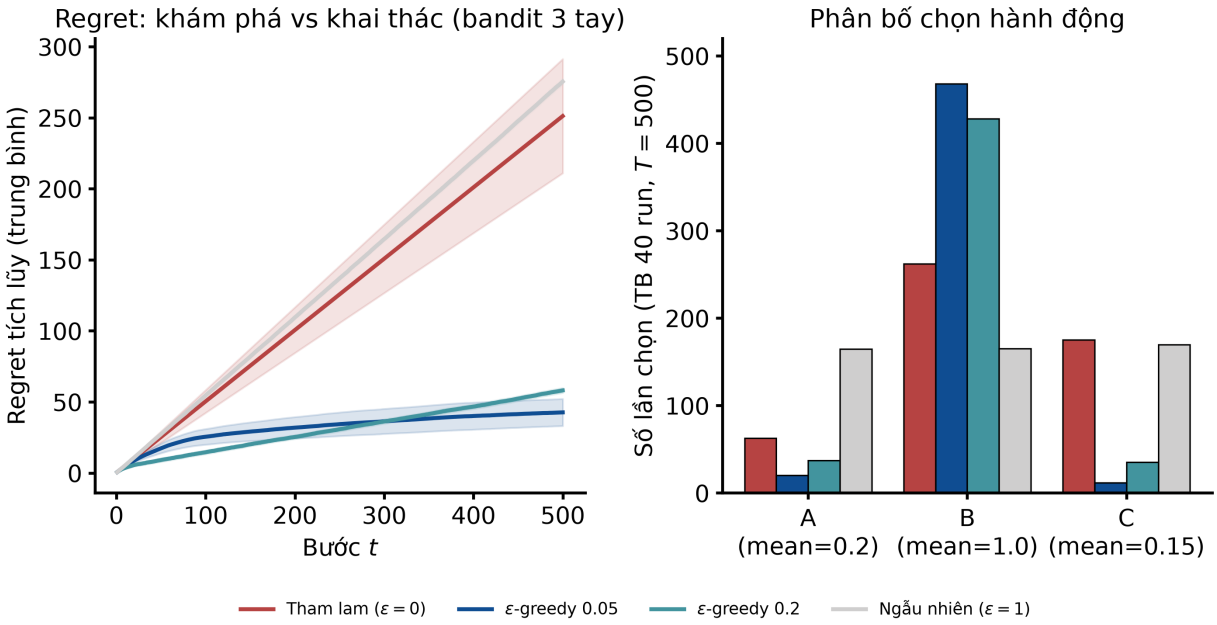
Vòng lặp MDP: agent chọn a_t, môi trường trả s_t, r_t và quỹ đạo τ; mục tiêu tối đa $J(\pi) = \mathbb{E}[G_0]$ (minh hoạ, tự tính)

Ví dụ trong lộ trình:

- Atari (bài 01–03): state là khung hình, action là phím bấm, reward là điểm game.
- Cờ vây, cờ vua (bài 14, 15): reward chỉ có ở cuối ván (+1 thắng, −1 thua).
- Robot (bài 10–12): action là mô-men xoắn liên tục.
- LLM (bài 23 trở đi): state là prompt cộng các token đã sinh, action là token tiếp theo, reward là điểm của mô hình phần thưởng hoặc kết quả kiểm tra tự động (đáp án đúng, unit test pass).

1.2 Ba khác biệt với supervised learning

	Supervised learning	Reinforcement learning
Tín hiệu	nhãn đúng cho từng input	chỉ một con số (reward), không nói hành động nào là đúng
Dữ liệu	cố định, i.i.d.	do chính policy tạo ra; policy đổi thì phân phối dữ liệu đổi
Hệ quả của hành động	không có	hành động hôm nay ảnh hưởng state và reward về sau (credit assignment theo thời gian)



Regret tích lũy bandit 3 tay: tham lam dễ kẹt tay kém, ε-greedy cân bằng khám phá/khai thác, $\varepsilon = 1$ lãng phí (minh hoạ, tự tính)

Ba hệ quả quan trọng:

1. **Exploration vs exploitation (khám phá và khai thác):** agent phải thử những hành động chưa biết để tìm cái tốt hơn, nhưng thử nhiều thì mất reward. Bài 13 (RND) là ví dụ về khám phá có chủ đích.
2. **Credit assignment (gán công):** reward đến muộn (cuối ván cờ, cuối câu trả lời), phải tìm xem bước nào đã gây ra nó. Value function, TD learning và GAE (bài 05) là công cụ cho việc này.
3. **Non-stationarity (không dừng):** phân phối dữ liệu thay đổi theo policy. Đây là gốc rễ của sự bất ổn trong deep RL và lý do có trust region (bài 04, 07).

1.3 Vì sao RL lại quan trọng với LLM

Supervised fine-tuning (SFT) dạy model bắt chước câu trả lời mẫu. RL cho phép:

- Tối ưu thứ ta **đánh giá được** nhưng **khó viết mẫu**: "câu trả lời hữu ích", "code chạy đúng", "đáp án toán đúng".
- Học trên **đầu ra của chính model** (on-policy), sửa lỗi mà model thực sự mắc, giống như distillation on-policy (distillation 28, 29).
- Khuếch đại hành vi hiếm nhưng tốt (chuỗi suy luận dài, tự kiểm tra) mà dữ liệu SFT ít có (bài 39 DeepSeek-R1).

Đổi lại: tín hiệu thưa (một reward cho cả nghìn token), phương sai gradient cao, rủi ro **reward hacking** (model khai thác kẽ hở của hàm thưởng, bài 27) và chi phí sinh rollout lớn.

---

2. Bản đồ phân loại RL

Nhiều trục độc lập; một thuật toán thường chọn một giá trị trên mỗi trục.

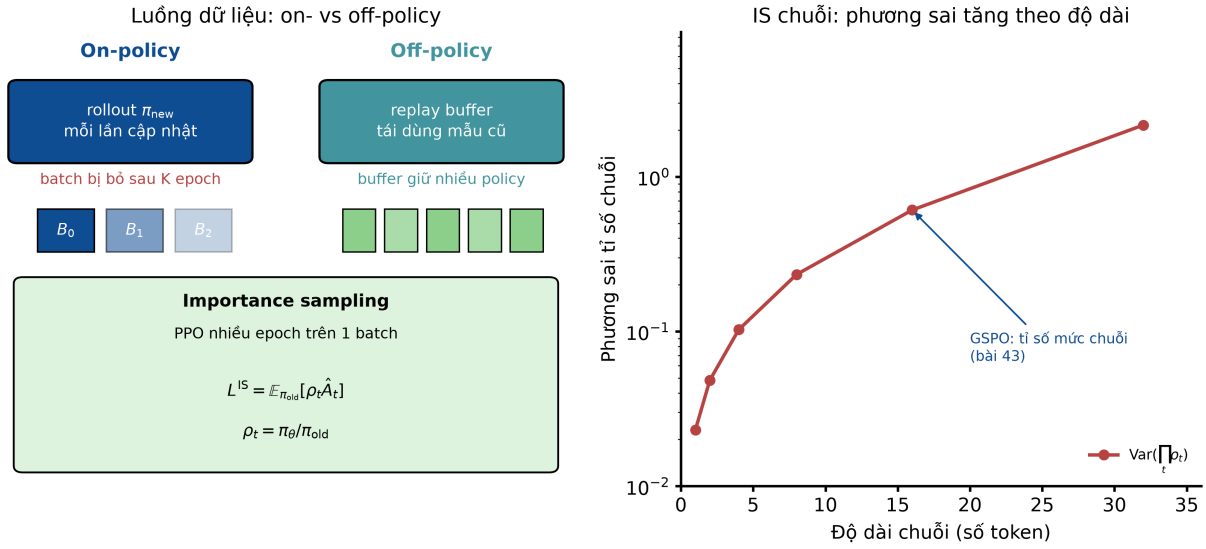
2.1 Học cái gì: value-based, policy-based, actor-critic

Loại	Học gì	Chọn hành động thế nào	Ví dụ bài
Value-based	$Q(s,a)$	$\arg\max_a Q(s,a)$	01 DQN, 02 Double DQN, 03 Rainbow
Policy-based	$\pi_\theta(a \mid s)$ trực tiếp	lấy mẫu từ π_θ	REINFORCE (mục 3.7), 37 RLOO
Actor-critic	cả policy (actor) và value (critic)	lấy mẫu từ actor; critic giảm phương sai	04 TRPO, 06 A3C, 07 PPO, 10–12 DDPG/TD3/SAC

LLM gần như luôn dùng policy-based hoặc actor-critic, vì không gian hành động (từ vựng hàng trăm nghìn token, và chuỗi token) quá lớn để lấy arg max trên Q.

2.2 Lấy dữ liệu từ đâu: on-policy, off-policy, offline

- **On-policy:** chỉ học từ dữ liệu của policy hiện tại (A3C, TRPO, PPO, GRPO). Ổn định hơn, nhưng phải sinh dữ liệu mới liên tục.
- **Off-policy:** học được từ dữ liệu của policy khác, ví dụ replay buffer chứa trải nghiệm cũ (DQN, DDPG, SAC). Tiết kiệm mẫu, nhưng dễ bất ổn.
- **Offline (batch RL):** chỉ có một tập dữ liệu cố định, không được tương tác thêm (bài 19 tutorial, 20 CQL, 21 Decision Transformer). DPO (bài 29) về bản chất là một phương pháp offline trên dữ liệu preference.



On-policy bỏ batch sau vài epoch vs off-policy replay; importance sampling $\rho = \pi_\theta / \pi_{\text{old}}$ và phương sai tích tỉ số theo độ dài chuỗi (minh hoạ, tự tính)

2.3 Có mô hình môi trường hay không: model-free và model-based

- **Model-free:** không học $P(s' \mid s,a)$, chỉ học value/policy (đa số bài trong thư mục).
- **Model-based:** học hoặc dùng mô hình môi trường để lập kế hoạch (bài 14 AlphaZero dùng luật chơi có sẵn, 15 MuZero học model trong không gian ẩn, 16 DreamerV3 học world model và "tưởng tượng" để huấn luyện actor).

2.4 Reward đến từ đâu

Nguồn reward	Ví dụ
Môi trường định nghĩa sẵn	điểm game, thắng/thua
Bắt chước chuyên gia (không có reward)	17 DAgger (imitation), 18 GAIL (học cost ngầm)
Học từ preference của người	23–25 (RLHF)
Học từ preference của AI	26 Constitutional AI (RLAIF)
Kiểm tra tự động (verifiable)	36 GRPO, 38 Tulu 3, 39 R1, 56 olmOCR 2 (RLVR)
Nội tại (intrinsic)	13 RND (độ "lạ" của state)

2.5 Một agent hay nhiều agent

Đa số bài là single-agent. Bài 22 (MAPPO) giới thiệu multi-agent hợp tác; bài 14 (self-play) là hai người chơi đối kháng nhưng huấn luyện bằng một mạng.

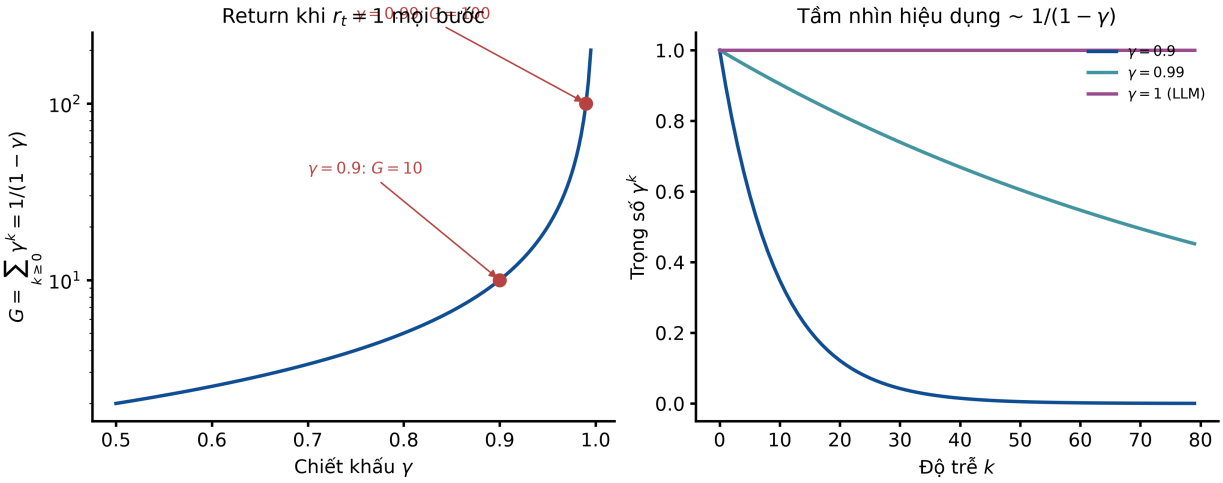
---

3. Toán cốt lõi của RL cổ điển và deep RL

3.1 MDP, policy, return

Markov Decision Process (MDP): bộ $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$:

- $\mathcal{S}$: tập state; $\mathcal{A}$: tập action.
- $P(s' \mid s, a)$: xác suất chuyển trạng thái.
- $R(s, a)$ (hoặc $r(s, a, s')$): reward kỳ vọng.
- $\gamma \in [0, 1]$: hệ số chiết khấu (discount).
- ρ_0: phân phối state ban đầu.



$G = 1/(1-\gamma)$ khi $r = 1$; $\gamma = 0.9 \to 10$, $0.99 \to 100$; LLM thường $\gamma = 1$ (minh hoạ, tự tính)

Tính Markov: tương lai chỉ phụ thuộc state hiện tại, không phụ thuộc lịch sử: $P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$. Nếu agent chỉ thấy một phần state (ví dụ một khung hình Atari không cho biết vận tốc), ta có **POMDP** (partially observable). DQN (bài 01) xếp chồng 4 khung hình để gần như khôi phục tính Markov.

Trajectory (quỹ đạo): $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. Với policy π:

$$p_\pi(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

Return (lợi tức) từ bước t:

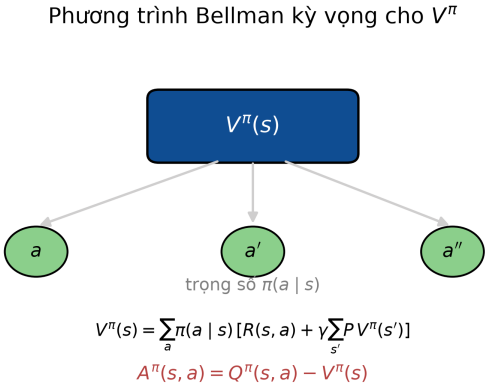
$$\boxed{G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma G_{t+1}}$$

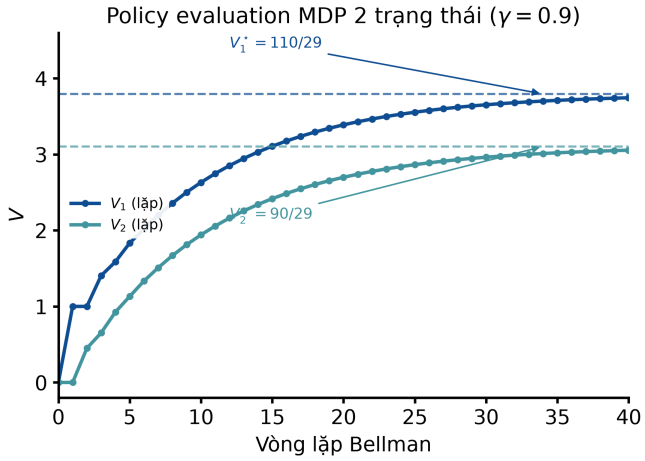
Mục tiêu RL: $J(\pi) = \mathbb{E}_{\tau \sim p_\pi}[G_0]$.

Ý nghĩa của γ: reward càng xa càng ít giá trị. "Tầm nhìn hiệu dụng" khoảng $1/(1-\gamma)$ bước. Ví dụ tự tính: reward bằng 1 ở mọi bước, $\gamma = 0.9$ thì $G_0 = \sum_k 0.9^k = 1/(1-0.9) = 10$; với $\gamma = 0.99$ thì $G_0 = 100$. Với LLM, episode hữu hạn (một câu trả lời) nên thường đặt $\gamma = 1$.

3.2 Value function, Q-function, advantage và phương trình Bellman

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a], \qquad \boxed{A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)}$$





Bellman kỳ vọng cho V^π và advantage $A = Q - V$; lặp policy evaluation trên MDP 2 trạng thái hội tụ $V_1 = 110/29$, $V_2 = 90/29$ (minh hoạ, tự tính)

- $V^\pi(s)$: state này tốt đến đâu nếu từ đây đi theo π.
- $Q^\pi(s,a)$: làm a trước rồi mới theo π thì tốt đến đâu.
- $A^\pi(s,a)$: hành động a **tốt hơn trung bình** của π tại s bao nhiêu. Vì $V^\pi(s) = \mathbb{E}_{a\sim\pi} Q^\pi(s,a)$, ta có $\mathbb{E}_{a\sim\pi}[A^\pi(s,a)] = 0$.

Phương trình Bellman kỳ vọng (từ $G_t = r_t + \gamma G_{t+1}$, lấy kỳ vọng):

$$\boxed{V^\pi(s) = \sum_a \pi(a \mid s)\Big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s')\Big]}$$

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s') Q^\pi(s',a').$$

Phương trình Bellman tối ưu cho policy tốt nhất π^*:

$$\boxed{Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q^*(s',a')}, \qquad \pi^*(s) = \arg\max_a Q^*(s,a).$$

Phép max trong phương trình này là gốc của Q-learning và DQN, và cũng là gốc của **overestimation bias** (bài 02): lấy max trên các ước lượng có nhiễu thì kỳ vọng bị đẩy lên.

3.3 Ví dụ MDP hai trạng thái (tự tính)

Hai state s_1, s_2, $\gamma = 0.9$. Policy cố định π_0:

- Ở s_1: nhận reward 1, chắc chắn sang s_2.
- Ở s_2 (hành động "chờ"): reward 0, sang s_1 với xác suất 0.5, ở lại s_2 với xác suất 0.5.

Bellman kỳ vọng cho hệ hai phương trình tuyến tính:

$$V_1 = 1 + 0.9V_2, \qquad V_2 = 0.9(0.5V_1 + 0.5V_2).$$

Từ phương trình hai: $0.55V_2 = 0.45V_1$, nên $V_2 = \frac{9}{11}V_1$. Thế vào phương trình một: $V_1 = 1 + 0.9 \cdot \frac{9}{11}V_1$, suy ra $V_1 \cdot \frac{29}{110} = 1$:

$$V_1 = \frac{110}{29} \approx 3.793, \qquad V_2 = \frac{90}{29} \approx 3.103.$$

Kiểm tra: $0.45 \cdot 3.793 + 0.45 \cdot 3.103 = 1.707 + 1.397 = 3.103$. Đúng.

Policy evaluation lặp (áp dụng Bellman nhiều lần, bắt đầu từ $V = [0, 0]$):

Vòng	V_1	V_2
0	0	0
1	$1 + 0.9 \cdot 0 = 1$	$0.9(0.5 \cdot 0 + 0.5 \cdot 0) = 0$
2	$1 + 0.9 \cdot 0 = 1$	$0.9(0.5 \cdot 1 + 0.5 \cdot 0) = 0.45$
3	$1 + 0.9 \cdot 0.45 = 1.405$	$0.9(0.5 \cdot 1 + 0.5 \cdot 0.45) = 0.6525$
∞	3.793	3.103

Phép lặp hội tụ vì toán tử Bellman là **co** (contraction) với hệ số $\gamma < 1$: mỗi vòng, sai số lớn nhất giảm ít nhất một hệ số γ.

Policy improvement. Thêm ở s_2 hành động thứ hai "quay về": reward -0.2, chắc chắn về s_1. Với V của π_0:

$$Q(s_2, \text{quay về}) = -0.2 + 0.9 \cdot 3.793 = 3.214 > V_2 = 3.103.$$

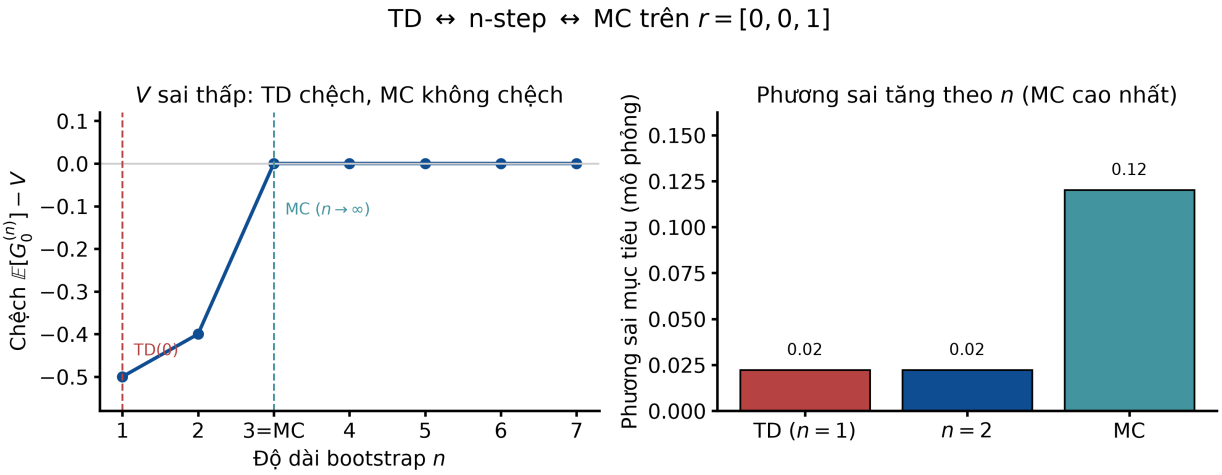
Hành động mới tốt hơn trung bình (advantage $\approx +0.11$), nên đổi policy. Policy mới π_1: $V_1 = 1 + 0.9V_2$, $V_2 = -0.2 + 0.9V_1$, suy ra $0.19V_1 = 0.82$:

$$V_1 \approx 4.316, \qquad V_2 \approx 3.684.$$

Kiểm tra hành động "chờ" dưới π_1: $Q(s_2, \text{chờ}) = 0.9(0.5 \cdot 4.316 + 0.5 \cdot 3.684) = 0.9 \cdot 4.0 = 3.6 < 3.684$. Không còn hành động nào tốt hơn, nên π_1 là tối ưu. Đây là **policy iteration**: đánh giá, cải thiện, lặp lại. Bài học: chịu phạt nhỏ bây giờ (-0.2) để quay lại state có reward lớn sớm hơn là lựa chọn tốt; RL tối ưu tổng dài hạn chứ không tối ưu reward tức thời.

3.4 Dynamic programming, Monte Carlo và Temporal Difference

Khi **biết** P và R: dùng dynamic programming (policy iteration, value iteration $V \leftarrow \max_a[R + \gamma PV]$). Khi **không biết**, phải ước lượng từ mẫu:



TD(n=1) chệch khi V sai, MC không chệch nhưng phương sai cao hơn trên chuỗi $r = [0, 0, 1]$ (minh hoạ, tự tính)

Monte Carlo (MC): chạy hết episode, dùng return thật làm mục tiêu: $V(s_t) \leftarrow V(s_t) + \alpha\,[G_t - V(s_t)]$. Không chệch, nhưng **phương sai cao** (return cộng nhiễu của mọi bước sau).

Temporal Difference TD(0): dùng một bước thật cộng ước lượng của bước sau (**bootstrapping**):

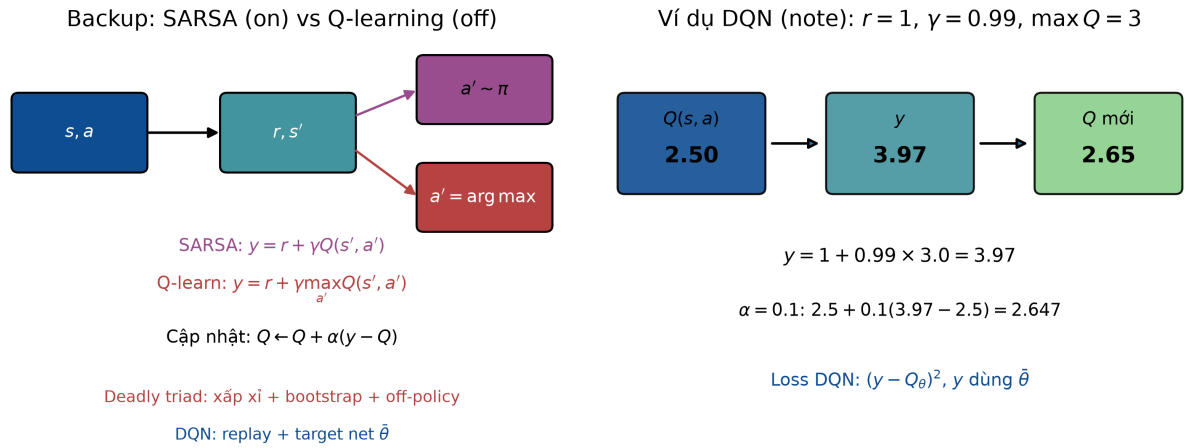
$$\boxed{\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad V(s_t) \leftarrow V(s_t) + \alpha\,\delta_t}$$

δ_t gọi là **TD error**. Phương sai thấp (chỉ một bước nhiễu) nhưng **chệch** khi V còn sai. Ví dụ tự tính trên MDP 3.3: $V = [0, 0]$, quan sát $(s_1, r = 1, s_2)$, $\alpha = 0.1$: $\delta = 1 + 0.9 \cdot 0 - 0 = 1$, nên $V_1 \leftarrow 0.1$.

n-step return nằm giữa: $G_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$. $n = 1$ là TD, $n = \infty$ là MC. A3C (bài 06) và Rainbow (bài 03) dùng n-step; GAE (mục 3.9) lấy trung bình có trọng số mọi n.

3.5 Q-learning, SARSA và “deadly triad”

- **SARSA (on-policy):** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$ với a' là hành động policy thực sự chọn.
- **Q-learning (off-policy):** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. Mục tiêu dùng max, tức là học Q^* bất kể dữ liệu đến từ policy nào (ví dụ ϵ-greedy: với xác suất ϵ chọn ngẫu nhiên, còn lại chọn $\arg\max Q$).



SARSA (on-policy) vs Q-learning (max off-policy); ví dụ DQN $y = 1 + 0.99 \times 3 = 3.97$ cập nhật Q (minh hoạ, tự tính)

Deep Q-learning thay bảng Q bằng mạng neural Q_θ, tối thiểu

$$\mathcal{L}(\theta) = \mathbb{E}\big[(r + \gamma \max_{a'} Q_{\bar\theta}(s', a') - Q_\theta(s, a))^2\big],$$

với $\bar\theta$ là bản sao cũ (target network) để mục tiêu không chạy theo chính nó. **Deadly triad** (Sutton & Barto): ba thứ cùng lúc (xấp xỉ hàm, bootstrapping, off-policy) có thể làm phân kỳ. Experience replay và target network của DQN (bài 01) là các mẹo giảm rủi ro này; offline RL (bài 19, 20) là trường hợp off-policy cực đoan nên càng khó.

3.6 Policy gradient theorem và REINFORCE

Tham số hóa policy π_θ (ví dụ softmax trên logit). Muốn $\nabla_\theta J(\theta)$ với $J = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$.

Log-derivative trick: $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$. Do đó

$$\nabla_\theta J = \int R(\tau)\nabla_\theta p_\theta(\tau)\, d\tau = \mathbb{E}_{\tau\sim p_\theta}\big[R(\tau)\nabla_\theta \log p_\theta(\tau)\big].$$

Trong $\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_t \log \pi_\theta(a_t \mid s_t) + \sum_t \log P(s_{t+1} \mid s_t, a_t)$, chỉ số hạng giữa phụ thuộc θ. **Không cần biết** P:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\Big].$$

Causality (tính nhân quả): hành động ở bước t không ảnh hưởng reward trước t, nên thay $R(\tau)$ bằng reward-to-go G_t (kỳ vọng không đổi, phương sai giảm):

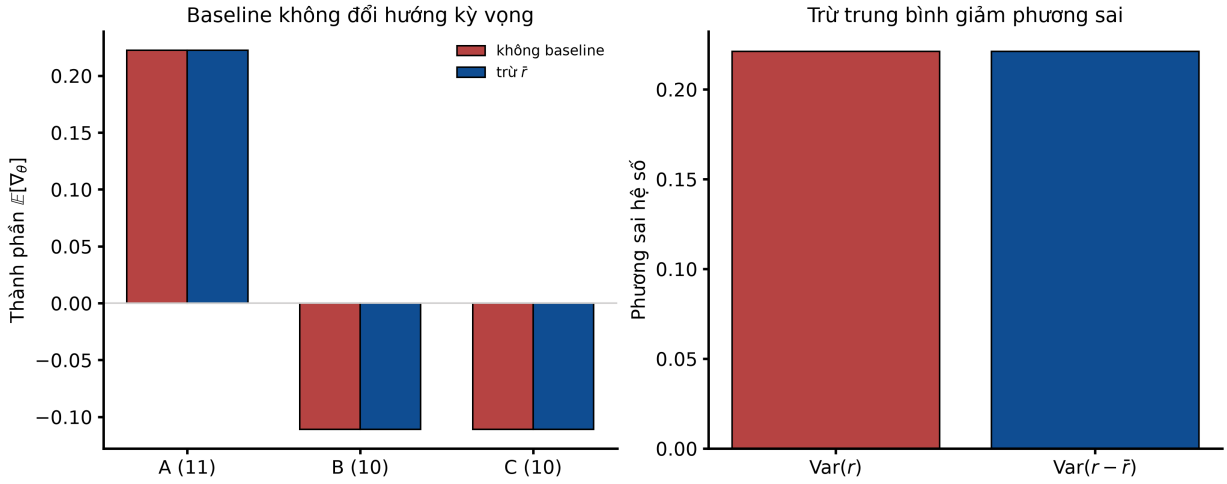
$$\boxed{\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\Big]} \qquad \text{(REINFORCE, Williams 1992)}$$

Policy gradient theorem (Sutton et al. 1999) viết dạng tổng quát hơn với Q^π: $\nabla J \propto \mathbb{E}_{s\sim d^\pi, a\sim\pi}[\nabla \log \pi_\theta(a \mid s)\, Q^\pi(s, a)]$, với d^π là phân phối state khi chạy π.

Trực giác: tăng log-xác suất của hành động **tỉ lệ với** mức tốt của kết quả. Đây chính là "SFT có trọng số": SFT tối đa $\log \pi(y)$ với trọng số 1; REINFORCE tối đa $\log \pi(y)$ với trọng số G. Bài 36 (DeepSeekMath, mục unified paradigm) viết SFT, rejection sampling, DPO, PPO, GRPO cùng một dạng "gradient coefficient $\times \nabla \log \pi$".

3.7 Baseline: vì sao trừ trung bình (ví dụ bandit, tự tính)

Với bất kỳ $b(s)$ không phụ thuộc hành động: $\mathbb{E}_{a\sim\pi}[\nabla \log \pi(a \mid s)\, b(s)] = b(s)\nabla \sum_a \pi(a \mid s) = b(s)\nabla 1 = 0$. Do đó



Bandit 3 tay: $\mathbb{E}[\nabla] = [0.222, -0.111, -0.111]$; trừ $\bar{r}$ giữ hướng, giảm Var (minh hoạ, tự tính)

$$\nabla J = \mathbb{E}\big[\nabla \log \pi(a \mid s)\,(G - b(s))\big]$$

vẫn **không chệch**, nhưng phương sai có thể giảm rất mạnh. Lựa chọn tự nhiên: $b(s) = V(s)$, khi đó $G - b$ là ước lượng của advantage.

Ví dụ: bandit 3 tay (một bước, không có state). Policy softmax, logit $\theta = [0, 0, 0]$ nên $\pi = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$. Với softmax: $\nabla_\theta \log \pi(a) = e_a - \pi$ (e_a là vector one-hot).

Trường hợp 1: reward A=1, B=0, C=0. Lấy được A, $r = 1$: gradient ước lượng $1 \cdot (e_A - \pi) = [0.667, -0.333, -0.333]$. Bước học $\eta = 1$: $\theta = [0.667, -0.333, -0.333]$, softmax cho $\pi \approx [0.576, 0.212, 0.212]$. Xác suất tay tốt tăng từ 0.333 lên 0.576.

Trường hợp 2: cộng 10 vào mọi reward (A=11, B=10, C=10). Thứ hạng không đổi, gradient kỳ vọng cũng không đổi:

$$\mathbb{E}[r(e_a - \pi)] = \tfrac{1}{3}\big[11(e_A - \pi) + 10(e_B - \pi) + 10(e_C - \pi)\big] = [0.222, -0.111, -0.111],$$

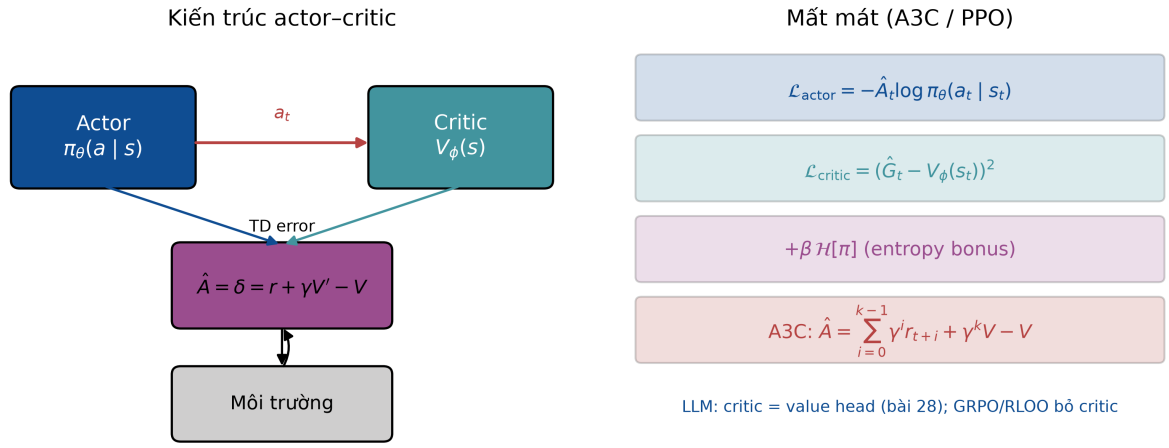
đúng bằng trường hợp 1. Nhưng một mẫu đơn lẻ thì khác hẳn: lấy được B (tay tệ hơn) cho gradient $10(e_B - \pi) = [-3.33, 6.67, -3.33]$, đẩy B lên rất mạnh. Chỉ khi lấy trung bình rất nhiều mẫu thì các số lớn này mới triệt tiêu: **phương sai khổng lồ**.

Thêm baseline $b = \mathbb{E}[r] = 10.333$: mẫu B cho $(10 - 10.333)(e_B - \pi) = [0.111, -0.222, 0.111]$, đúng hướng (giảm B). Baseline biến "reward tuyệt đối" thành "tốt hơn hay kém hơn trung bình".

Ý này xuất hiện khắp lộ trình: critic V trong A3C/PPO (06, 07), trung bình nhóm trong GRPO (36), leave-one-out trong RLOO (37), hằng số -1 trong MiniLLM (distillation 28).

3.8 Actor-critic

Thay G_t (nhiễu) bằng ước lượng advantage từ một critic V_ϕ học song song:



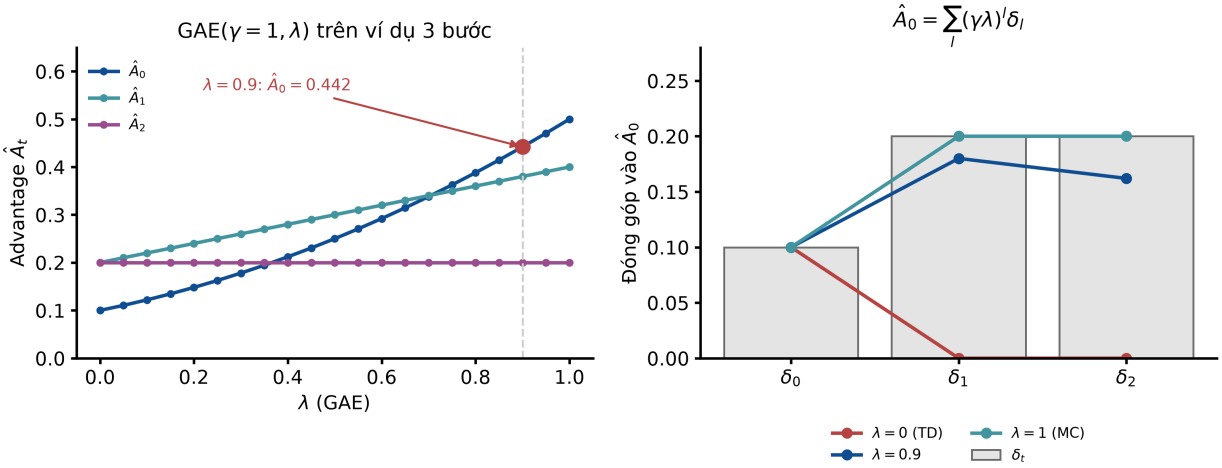
Actor–critic: $\hat{A} = \delta$; loss actor/critic/entropy; LLM dùng value head, GRPO/RLOO bỏ critic (minh hoạ, tự tính)

$$\hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) = \delta_t, \qquad \mathcal{L}_{\text{actor}} = -\hat{A}_t \log \pi_\theta(a_t \mid s_t), \qquad \mathcal{L}_{\text{critic}} = (\hat{G}_t - V_\phi(s_t))^2.$$

Actor cập nhật theo hướng critic chỉ; critic học dự đoán return. Với LLM, critic thường là một bản sao LLM có thêm value head (bài 28 liệt kê nhiều chi tiết khởi tạo), tốn gấp đôi bộ nhớ, đó là lý do GRPO/RLOO bỏ critic.

3.9 Generalized Advantage Estimation (GAE)

Advantage k bước: $\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}$ (cộng TD error, các V ở giữa triệt tiêu theo kiểu "telescoping"). GAE lấy trung bình trọng số mũ (bài 05):



GAE(γ, λ) nội suy TD ($\lambda = 0$) và MC ($\lambda = 1$); ví dụ note cho $\lambda = 0.9$ thì $\hat{A}_0 = 0.442$ (minh hoạ, tự tính)

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \hat{A}_{t+1}$$

- $\lambda = 0$: $\hat{A}_t = \delta_t$ (TD, phương sai thấp, chệch nếu V sai).
- $\lambda = 1$: $\hat{A}_t = G_t - V(s_t)$ (MC, không chệch, phương sai cao).

Ví dụ tự tính. Ba bước, reward $[0, 0, 1]$ (reward chỉ ở cuối, giống LLM), $V = [0.5, 0.6, 0.8]$, $V(\text{kết thúc}) = 0$, $\gamma = 1$:

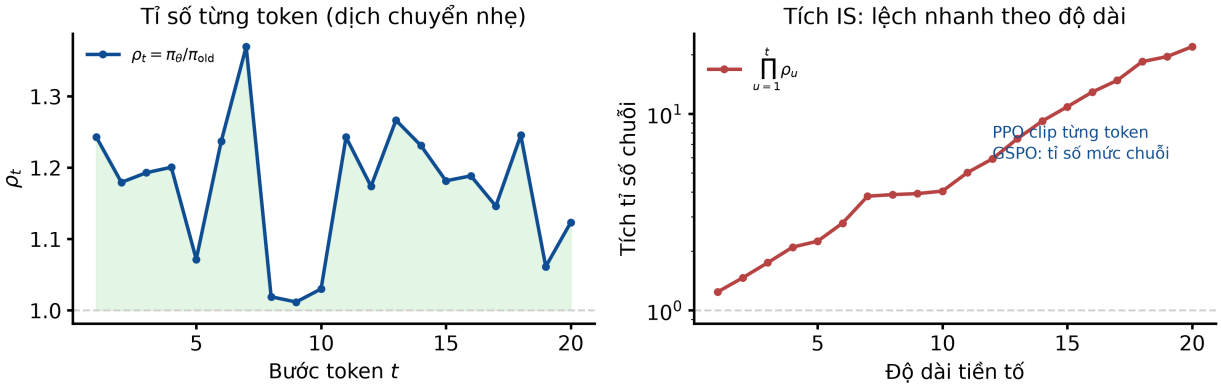
- $\delta_0 = 0 + 0.6 - 0.5 = 0.1$, $\delta_1 = 0 + 0.8 - 0.6 = 0.2$, $\delta_2 = 1 + 0 - 0.8 = 0.2$.
- $\lambda = 0.9$: $\hat{A}_2 = 0.2$; $\hat{A}_1 = 0.2 + 0.9 \cdot 0.2 = 0.38$; $\hat{A}_0 = 0.1 + 0.9 \cdot 0.38 = 0.442$.
- $\lambda = 1$: $\hat{A}_0 = 0.1 + 0.2 + 0.2 = 0.5 = G_0 - V_0 = 1 - 0.5$. Đúng.
- $\lambda = 0$: $\hat{A}_0 = 0.1$.

Tính ngược từ cuối về đầu, một vòng lặp, chi phí $O(T)$.

3.10 Importance sampling: dùng lại mẫu cũ

$$\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right].$$

$$L^{\text{IS}} = \mathbb{E}_{\pi_{\text{old}}}[\rho_t \hat{A}_t]$$



Tỉ số IS từng token ρ_t và tích chuỗi $\prod \rho$ lệch nhanh theo độ dài (minh hoạ, tự tính)

Trong policy gradient: rollout sinh bởi π_{old}, nhưng muốn cập nhật π_θ vài lần trên cùng batch. Dùng **tỉ số** $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$:

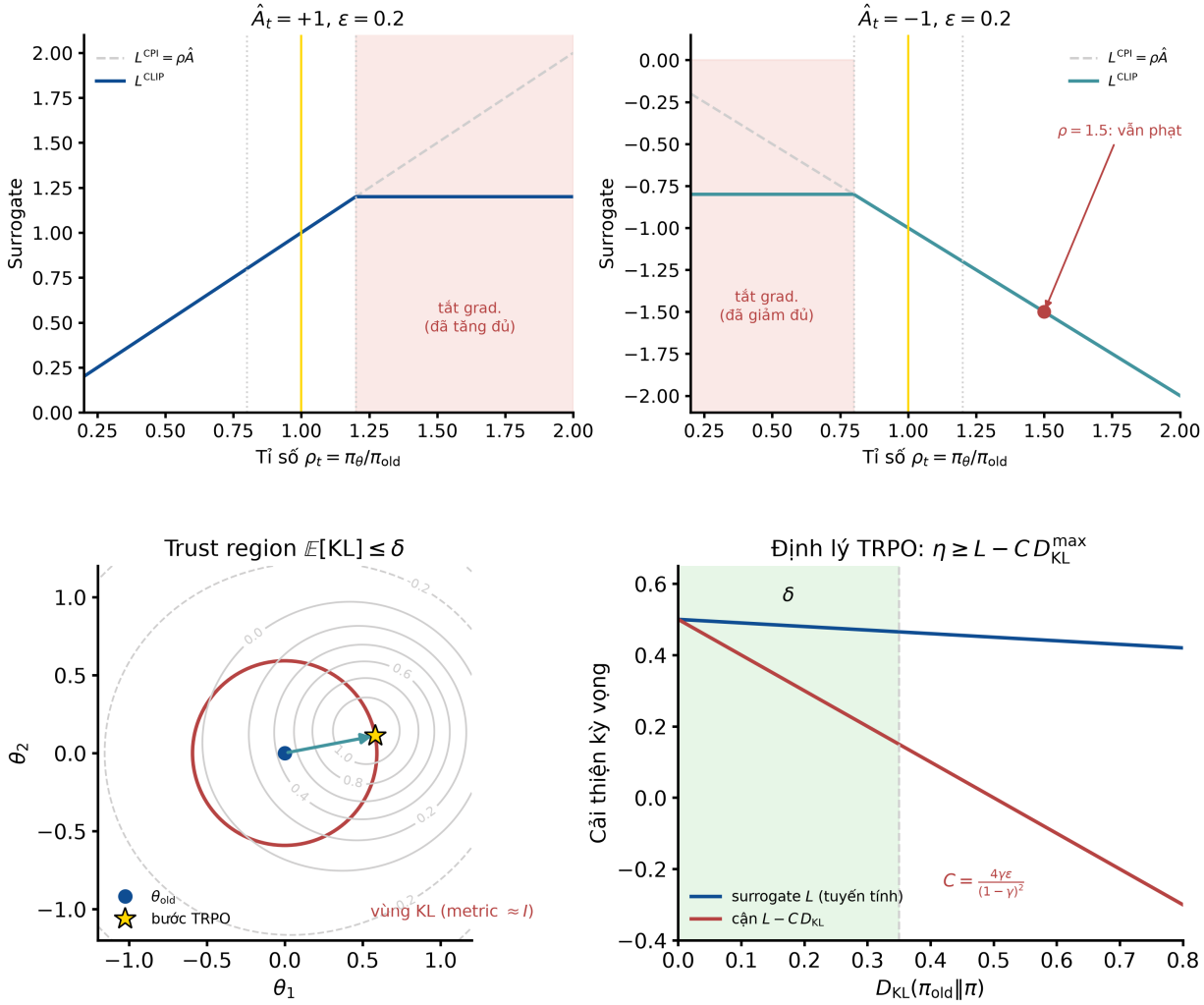
$$L^{\text{IS}}(\theta) = \mathbb{E}_{\pi_{\text{old}}}\left[\rho_t(\theta)\hat{A}_t\right], \qquad \nabla_\theta L^{\text{IS}}\big|_{\theta=\theta_{\text{old}}} = \mathbb{E}\left[\nabla \log \pi_\theta \, \hat{A}_t\right].$$

Tại $\theta = \theta_{\text{old}}$, gradient trùng với policy gradient. Khi π_θ đi xa π_{old}, tỉ số có thể rất lớn hoặc rất nhỏ, phương sai bùng nổ. Với chuỗi, tích các tỉ số theo token còn tệ hơn (phương sai tăng theo độ dài). Đây là lý do GSPO (bài 43) dùng tỉ số mức chuỗi có chuẩn hóa độ dài, và MiniLLM (distillation 28) chỉ giữ thừa số của bước hiện tại.

Một dạng IS khác trong LLM RL hiện đại: engine sinh rollout (vLLM, SGLang) và engine train tính log-prob hơi khác nhau (khác kernel, khác độ chính xác số), nên "on-policy" thật ra hơi lệch. Bài 44 (ScaleRL) bàn về việc sửa độ chính xác ở LM head vì lý do này.

3.11 Trust region, TRPO và PPO

Vấn đề: bước cập nhật lớn làm policy đổi nhiều, dữ liệu cũ không còn đại diện, hiệu năng sụp và khó hồi phục (vì dữ liệu tương lai đến từ policy tệ).



TRPO tối đa surrogate trong quả cầu KL; cận $\eta \ge L - CD_{\mathrm{KL}}^{\max}$ (minh hoạ, tự tính)

Surrogate PPO clip theo ρ với $\varepsilon = 0.2$: tắt gradient khi đi quá xa theo hướng có lợi, vẫn phạt khi đi sai hướng (minh hoạ, tự tính)

TRPO (bài 04): tối đa surrogate với ràng buộc KL:

$$\max_\theta \ \mathbb{E}\big[\rho_t(\theta)\hat{A}_t\big] \quad \text{s.t.} \quad \mathbb{E}\big[\mathrm{KL}(\pi_{\mathrm{old}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))\big] \le \delta.$$

Giải bằng conjugate gradient và line search; tốn kém, khó cài với mạng lớn.

PPO (bài 07): thay ràng buộc bằng **clipping**:

$$\boxed{L^{\mathrm{CLIP}}(\theta) = \mathbb{E}\Big[\min\big(\rho_t\hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big]}$$

Bảng các trường hợp với $\epsilon = 0.2$ (tự tính):

$\hat{A}_t$	ρ_t	$\rho\hat{A}$	clip $\cdot\ \hat{A}$	min	Có gradient?
$+1$	1.1	1.1	1.1	1.1	có: tiếp tục tăng xác suất
$+1$	1.5	1.5	1.2	1.2	**không**: đã tăng đủ
-1	0.5	-0.5	-0.8	-0.8	**không**: đã giảm đủ

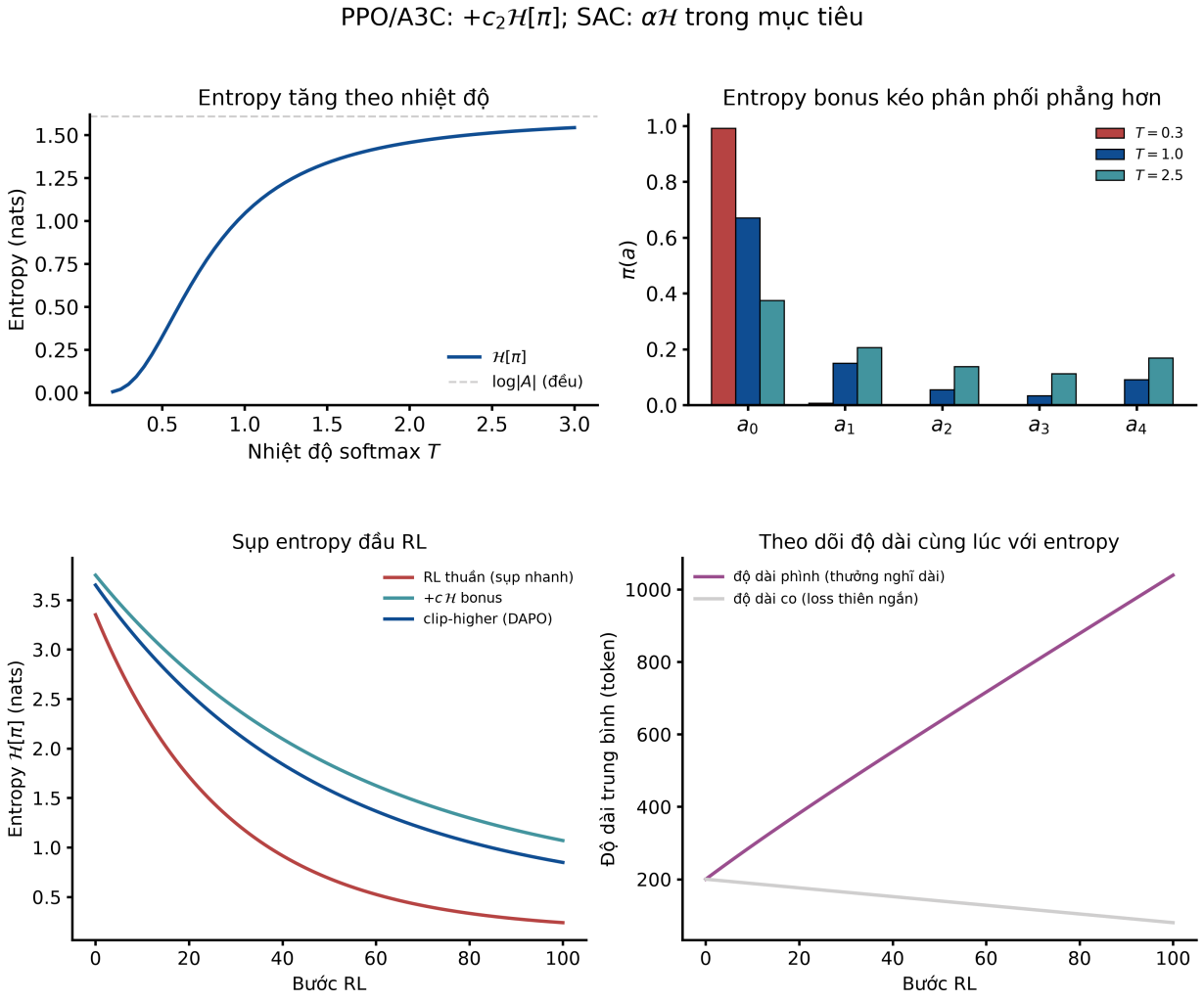
$\hat{A}_t$	ρ_t	$\rho\hat{A}$	clip $\cdot\ \hat{A}$	min	Có gradient?
-1	1.5	-1.5	-1.2	-1.5	có: vẫn phạt (hành động tệ mà xác suất lại tăng)

Nghĩa là PPO chỉ "tắt" gradient khi thay đổi đã đi quá xa **theo hướng có lợi**; còn khi thay đổi đi sai hướng thì vẫn sửa. Loss đầy đủ thường là $-L^{\text{CLIP}} + c_1\mathcal{L}_{\text{value}} - c_2\mathcal{H}[\pi]$.

Các biến thể cho LLM điều chỉnh đúng chỗ này: DAPO (bài 41) tách ϵ_{low} và ϵ_{high} ("clip-higher") để token xác suất thấp được tăng nhiều hơn, chống sụp entropy.

3.12 Entropy và maximum-entropy RL

Entropy của policy: $\mathcal{H}[\pi(\cdot \mid s)] = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$. Entropy cao nghĩa là còn khám phá; entropy về 0 nghĩa là policy gần như xác định.



Entropy sụp đầu RL; bonus/clip-higher giữ khám phá; độ dài có thể phình hoặc co (minh hoạ, tự tính)

Entropy softmax theo nhiệt độ T; bonus $c\mathcal{H}$ chống hội tụ sớm (minh hoạ, tự tính)

- **Entropy bonus** (A3C, PPO): cộng $c\,\mathcal{H}$ vào mục tiêu để tránh hội tụ sớm.
- **Maximum-entropy RL** (bài 12 SAC): mục tiêu $\mathbb{E}[\sum_t r_t + \alpha\mathcal{H}(\pi(\cdot \mid s_t))]$. Policy tối ưu có dạng Boltzmann $\pi(a \mid s) \propto \exp(Q_{\text{soft}}(s,a)/\alpha)$.

- Với LLM, **entropy collapse** (entropy sụp nhanh ở đầu RL) là hiện tượng chính của bài 49; bài 50 cho thấy chỉ một thiểu số token có entropy cao ("forking tokens") quyết định hiệu quả RL.

Mục 4.3 sẽ thấy KL-regularized RL cho LLM có cùng dạng nghiệm Boltzmann, chỉ khác là "nhiệt độ" kéo về π_{ref} thay vì về phân phối đều.

---

4. RL cho LLM

4.1 LLM là một policy

Prompt x, câu trả lời $y = (y_1, \dots, y_T)$. Hai cách nhìn:

- **Token-level MDP:** state $s_t = (x, y_{<t})$, action $a_t = y_t \in \mathcal{V}$ (từ vựng), chuyển trạng thái xác định (nối token), reward thường chỉ ở cuối $r_T = R(x, y)$, cộng có thể một phạt KL mỗi token. $\gamma = 1$.
- **Sequence-level bandit:** coi cả y là một hành động; $\pi_\theta(y \mid x) = \prod_t \pi_\theta(y_t \mid x, y_{<t})$. Bài 37 (RLOO) lập luận rằng cách nhìn này đủ tốt và đơn giản hơn PPO token-level.

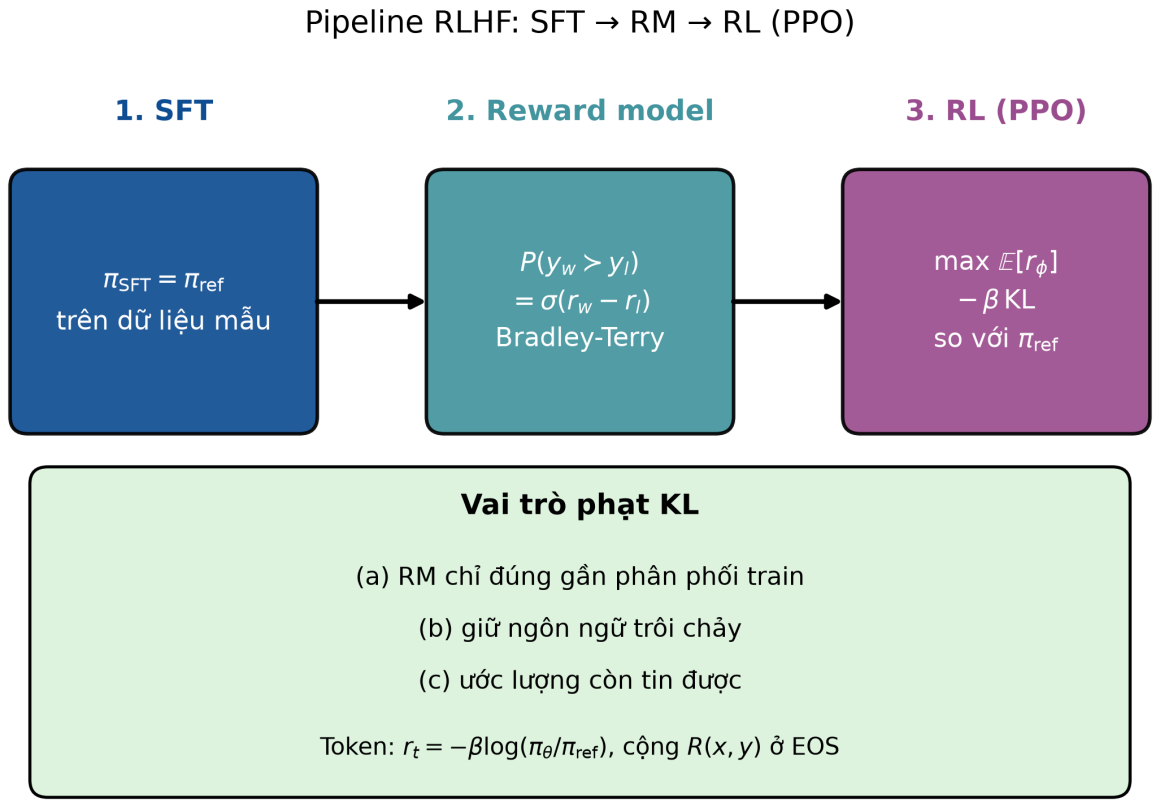
Hệ quả: không gian hành động khổng lồ nhưng **có prior rất mạnh** (mô hình đã pretrain và SFT). RL cho LLM chủ yếu **dịch chuyển xác suất trong số những gì mô hình đã biết sinh**, chứ hiếm khi khám phá từ đầu. Đây là cốt lõi tranh luận ở bài 45–48 (RL có thật sự mở rộng năng lực không).

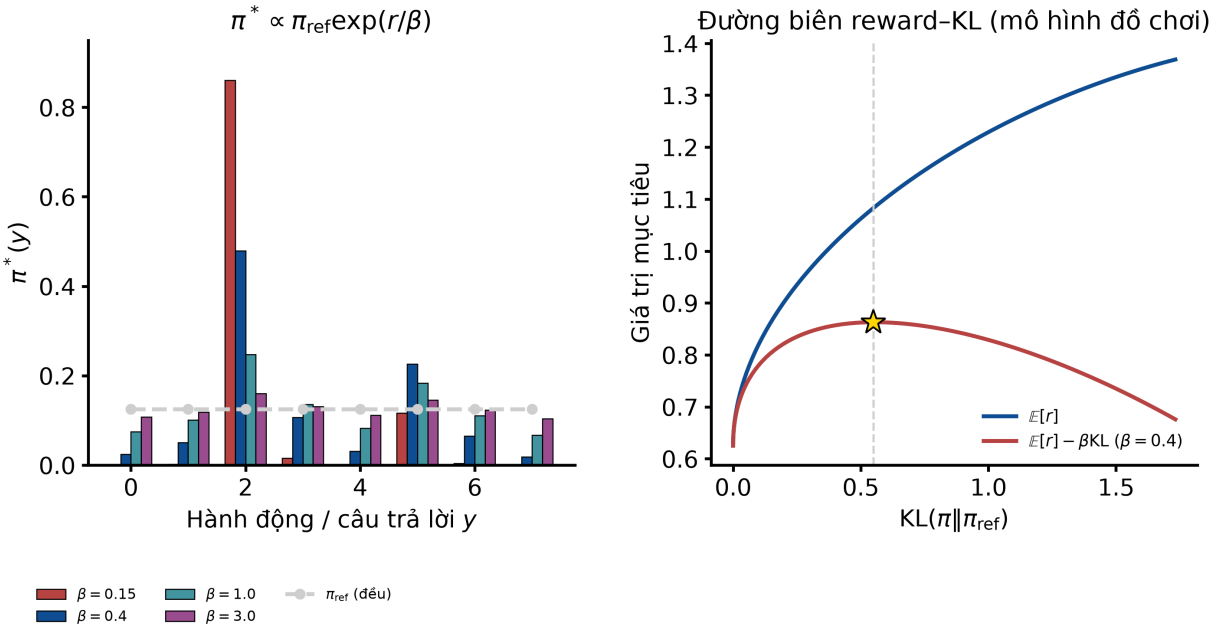
Bài 54 (survey agentic RL) mở rộng: agent LLM gọi tool nhiều lượt thì trở thành POMDP thực sự, môi trường (search engine, sandbox code) có chuyển trạng thái ngẫu nhiên.

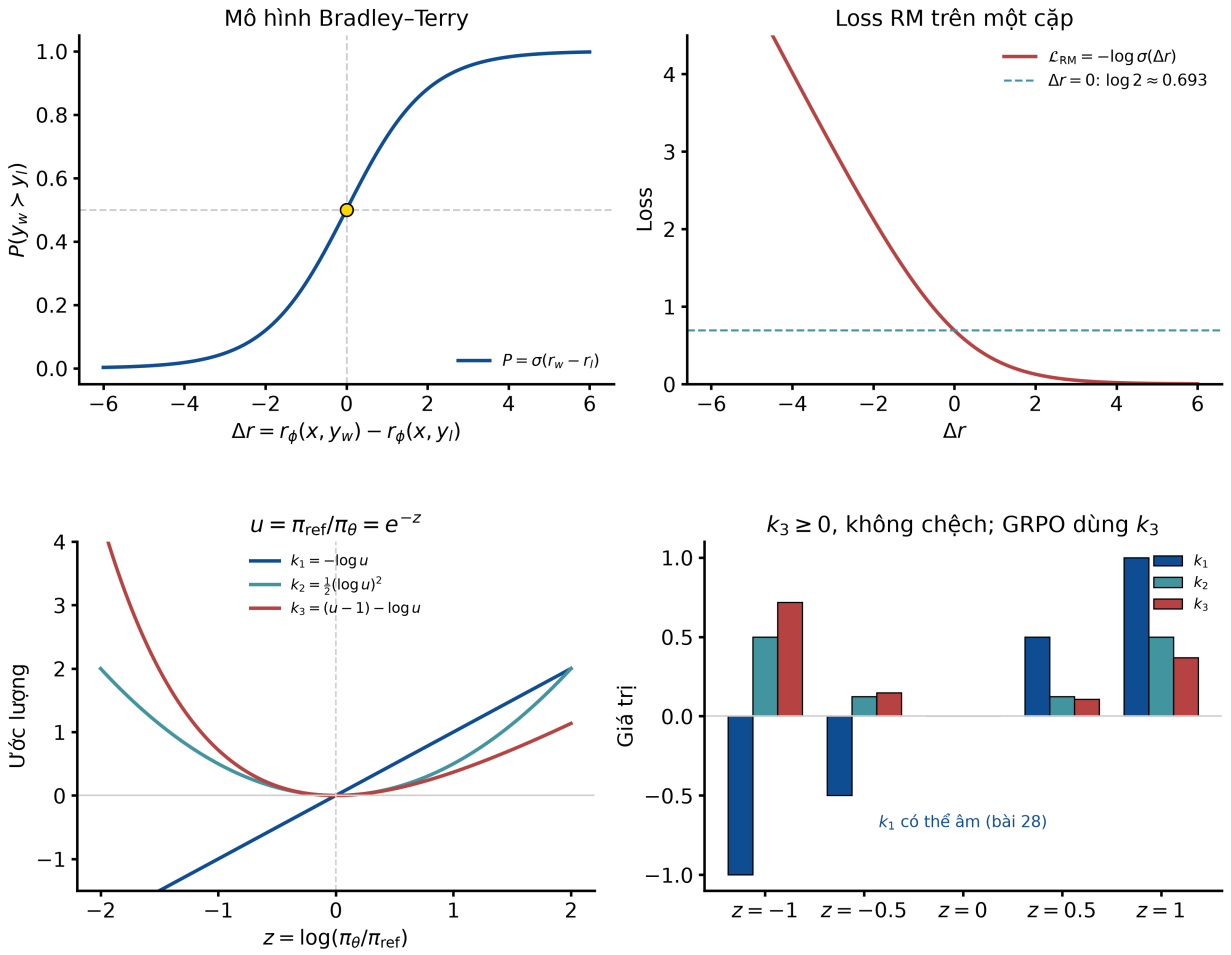
4.2 Pipeline RLHF và reward model

Ba bước kinh điển (bài 24, 25; llm 07 InstructGPT):

1. **SFT** trên dữ liệu mẫu, được π_{SFT} (cũng là π_{ref}).
2. **Reward model (RM):** thu thập cặp so sánh (x, y_w, y_l) (người chọn y_w hơn y_l). Mô hình **Bradley-Terry**:


Pipeline RLHF: SFT → RM → RL (PPO)
1. SFT
2. Reward model
3. RL (PPO)
$\pi_{\text{SFT}} = \pi_{\text{ref}}$
trên dữ liệu mẫu
$P(y_w \succ y_l)$
$= \sigma(r_w - r_l)$
Bradley-Terry
$\max \mathbb{E}[r_\phi]$
$- \beta\,\text{KL}$
so với π_{ref}
Vai trò phạt KL
(a) RM chỉ đúng gần phân phối train
(b) giữ ngôn ngữ trôi chảy
(c) ước lượng còn tin được
Token: $r_t = -\beta \log(\pi_\theta/\pi_{\text{ref}})$, cộng $R(x, y)$ ở EOS


$\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$
$\pi^*(y)$
Hành động / câu trả lời y
$\beta = 0.15$
$\beta = 0.4$
$\beta = 1.0$
$\beta = 3.0$
π_{ref} (đều)
Đường biên reward–KL (mô hình đồ chơi)
Giá trị mục tiêu
KL($\pi \| \pi_{\text{ref}}$)
$\mathbb{E}[r]$
$\mathbb{E}[r] - \beta$KL ($\beta = 0.4$)



Ước lượng KL k_1, k_2, k_3 theo $u = \pi_{\text{ref}}/\pi_\theta$; $k_3 \geq 0$ không chệch (GRPO) (minh hoạ, tự tính)

Bradley–Terry $P = \sigma(\Delta r)$ và loss RM $-\log \sigma(\Delta r)$; tại $\Delta r = 0$ loss $= \log 2$ (minh hoạ, tự tính)

$\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$ theo β và đường biên $\mathbb{E}[r] - \beta\,\text{KL}$ đạt cực đại tại điểm đánh đổi tối ưu (minh hoạ, tự tính)

Pipeline RLHF ba bước SFT $\rightarrow$ reward model Bradley-Terry $\rightarrow$ PPO với phạt KL so với π_{ref} (minh hoạ, tự tính)

$$\boxed{P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big), \qquad \mathcal{L}_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)}$$

với $\sigma(z) = 1/(1 + e^{-z})$. Chỉ hiệu số reward có ý nghĩa (cộng hằng số vào mọi reward không đổi gì), nên thường "whiten" reward (bài 28).

3. **RL (PPO)** tối đa reward có phạt KL so với π_{ref}:

$$\boxed{\max_\theta \; \mathbb{E}_{x,\, y \sim \pi_\theta}\big[r_\phi(x, y)\big] - \beta\, \text{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)}$$

Vai trò của KL: (a) RM chỉ đúng gần phân phối dữ liệu nó được train, đi xa thì RM sai và bị khai thác (bài 27, Goodhart); (b) giữ ngôn ngữ trôi chảy và năng lực chung; (c) về mặt kỹ thuật, giữ policy gần vùng mà các ước lượng còn tin được.

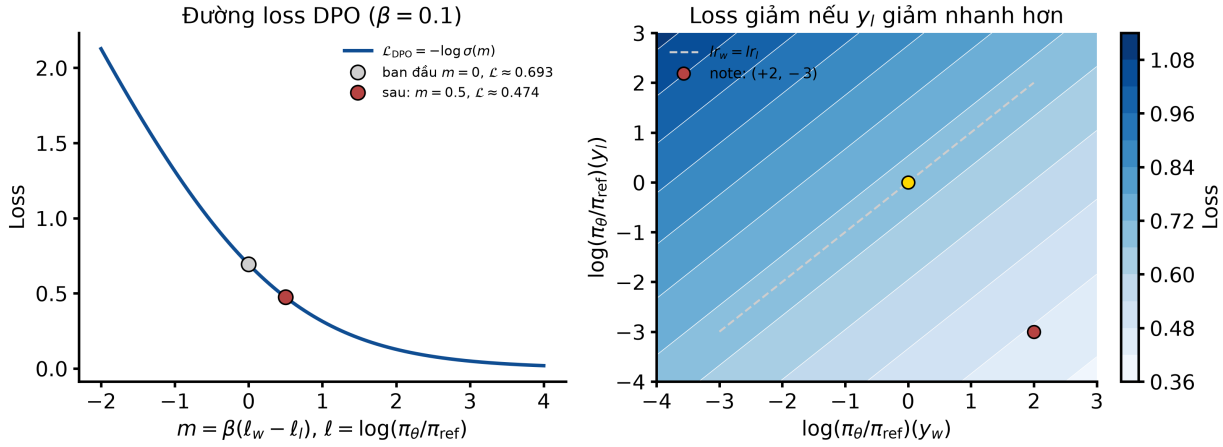
Ước lượng KL trên mẫu (theo ghi chú "Approximating KL Divergence" của John Schulman, 2020, không phải paper trong thư mục). Với $y \sim \pi_\theta$, đặt $u = \pi_{\text{ref}}(y)/\pi_\theta(y)$:

- $k_1 = -\log u$: không chệch, có thể âm, phương sai cao.
- $k_2 = \frac{1}{2}(\log u)^2$: chệch nhẹ, luôn không âm.
- $k_3 = (u-1) - \log u$: **không chệch** và luôn ≥ 0. GRPO (bài 36) dùng dạng này (tính theo từng token).

Trong PPO-RLHF, KL thường được trừ thẳng vào reward mỗi token: $r_t = -\beta \log \frac{\pi_\theta(y_t|\cdot)}{\pi_{\text{ref}}(y_t|\cdot)}$, cộng $R(x,y)$ ở token cuối.

4.3 Nghiệm tối ưu của bài toán KL-regularized và đạo hàm DPO

Với một prompt x cố định, viết lại mục tiêu (tổng trên y):



DPO loss theo margin m; ví dụ $\beta = 0.1$: $0.693 \to 0.474$ khi $(+2, -3)$ (minh hoạ, tự tính)

$$\sum_y \pi(y)\Big[r(y) - \beta \log \frac{\pi(y)}{\pi_{\text{ref}}(y)}\Big] = -\beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)e^{r(y)/\beta}} = -\beta\, \text{KL}\Big(\pi \,\Big\|\, \frac{\pi_{\text{ref}} e^{r/\beta}}{Z}\Big) + \beta \log Z,$$

với $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) e^{r(x,y)/\beta}$. KL nhỏ nhất bằng 0 khi:

$$\boxed{\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x) \exp\big(r(x,y)/\beta\big)}$$

Đảo ngược: $r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$. Thế vào Bradley-Terry, $\beta \log Z(x)$ **triệt tiêu** trong hiệu số:

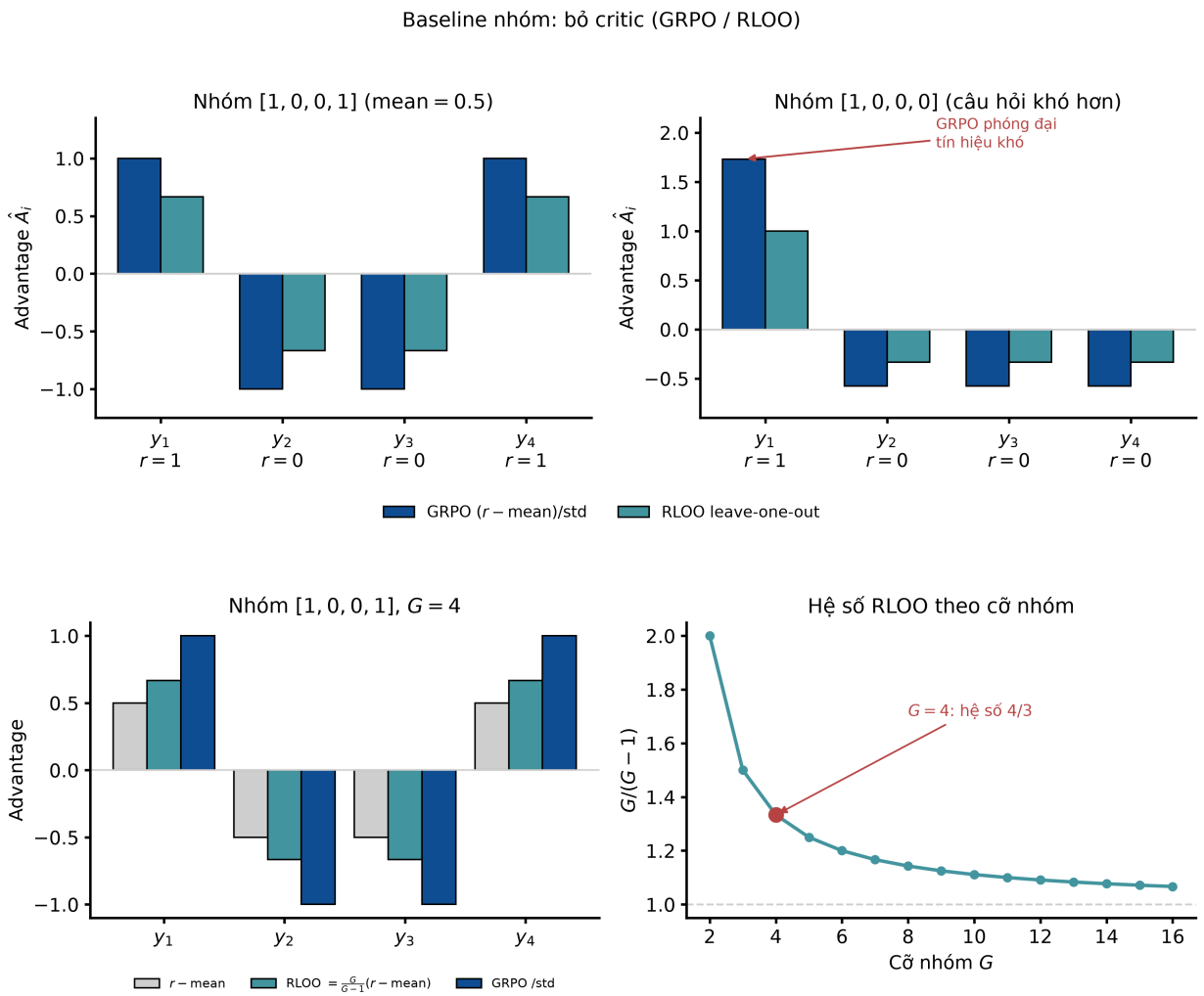
$$\boxed{\mathcal{L}_{\text{DPO}} = -\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)}$$

Không cần RM riêng, không cần rollout: chỉ là classification loss trên cặp (bài 29). Cái giá: DPO học offline trên dữ liệu cố định, không thấy output mới của chính nó (bài 33 phân tích khi nào PPO thắng).

Ví dụ tự tính ($\beta = 0.1$). Ban đầu $\pi_\theta = \pi_{\text{ref}}$: hai log-ratio bằng 0, $\mathcal{L} = -\log \sigma(0) = \log 2 \approx 0.693$. Sau một thời gian train: log-ratio của y_w là $+2$, của y_l là -3; margin $= 0.1 \cdot (2 - (-3)) = 0.5$; $\sigma(0.5) \approx 0.622$; $\mathcal{L} \approx 0.474$. Lưu ý: loss giảm được cả khi **cả hai** log-prob đều giảm (chỉ cần y_l giảm nhanh hơn), một hiện tượng hay gặp khi huấn luyện DPO.

4.4 Bỏ critic: REINFORCE với baseline nhóm (RLOO, GRPO)

Với mỗi prompt, sinh G câu trả lời $y_1, \ldots, y_G$, chấm reward $r_1, \ldots, r_G$.



RLOO $= \frac{G}{G-1}(r - \text{mean})$; với $G = 4$ hệ số $4/3$ so với GRPO $/\text{std}$ (minh hoạ, tự tính)

Advantage nhóm GRPO $(r - \text{mean})/\text{std}$ so với RLOO leave-one-out trên hai nhóm reward trong note (minh hoạ, tự tính)

- **RLOO** (bài 37): baseline của mẫu i là trung bình **các mẫu khác**: $\hat{A}_i = r_i - \frac{1}{G-1}\sum_{j \neq i} r_j$. Không chệch vì baseline không phụ thuộc y_i.
- **GRPO** (bài 36): chuẩn hóa theo nhóm $\hat{A}_i = \dfrac{r_i - \text{mean}(r)}{\text{std}(r)}$, gán cùng advantage cho mọi token của y_i, dùng clipping kiểu PPO và phạt KL k_3.

Ví dụ tự tính, $G = 4$:

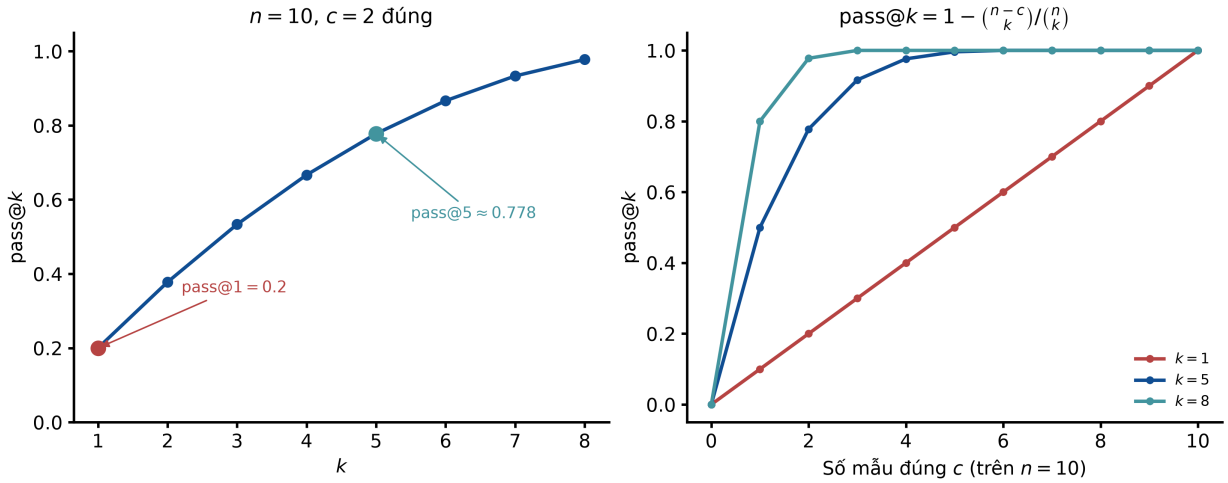
Reward nhóm	mean	std (chia G)	Advantage GRPO	Advantage RLOO
$[1, 0, 0, 1]$	0.5	0.5	$[+1, -1, -1, +1]$	$[+0.667, -0.667, -0.667, +0.667]$
$[1, 0, 0, 0]$	0.25	0.433	$[+1.732, -0.577, -0.577, -0.577]$	$[+1, 0, 0, 0]$
$[1, 1, 1, 1]$ hoặc $[0, 0, 0, 0]$		0	không xác định (thường đặt 0)	$[0, 0, 0, 0]$

Quan sát:

- RLOO bằng $\frac{G}{G-1}(r_i - \text{mean})$, tức chỉ là hiệu với trung bình nhân hệ số $4/3$.
- GRPO chia cho std nên câu hỏi "khó" (chỉ 1/4 đúng) cho advantage dương lớn hơn (1.732): nó **đổi trọng số giữa các prompt** theo độ khó. Bài 42 (Dr. GRPO) chỉ ra đây là một nguồn bias và đề xuất bỏ chia std; cách tính std (chia G hay $G-1$) cũng khác nhau giữa các cài đặt.
- Nhóm toàn đúng hoặc toàn sai cho **gradient bằng 0**: tốn công sinh rollout mà không học gì. DAPO (bài 41) lọc bỏ các nhóm này (dynamic sampling).
- Cách cộng loss theo token (chia cho độ dài từng câu hay chia cho tổng số token của batch) ảnh hưởng tới việc model thích câu dài hay ngắn (bài 41, 42).

4.5 RLVR và cách đo: pass@k

RLVR (RL with Verifiable Rewards): reward tính bằng một hàm kiểm tra xác định: đáp án toán khớp, code qua test, đầu ra đúng format, OCR qua unit test (bài 38 đặt tên, 39 làm nổi tiếng, 56 áp vào OCR). Ưu điểm: không có RM để bị khai thác (nhưng vẫn có thể "hack" bộ kiểm tra nếu nó lỏng). Nhược: chỉ áp dụng cho bài toán có cách kiểm tra.



pass@$k = 1 - \binom{n-c}{k}/\binom{n}{k}$; $n = 10, c = 2$: pass@1 $= 0.2$, pass@5 ≈ 0.778 (minh hoạ, tự tính)

pass@k: xác suất ít nhất một trong k mẫu đúng. Ước lượng không chệch từ $n \geq k$ mẫu với c mẫu đúng:

$$\boxed{\text{pass@}k = 1 - \binom{n-c}{k} \Big/ \binom{n}{k}}$$

Ví dụ tự tính: $n = 10$, $c = 2$: pass@1 $= 0.2$; pass@5 $= 1 - \binom{8}{5}/\binom{10}{5} = 1 - 56/252 \approx 0.778$.

Bài 45 dùng pass@k với k lớn để hỏi: RL làm tăng pass@1 (chọn đúng thường xuyên hơn) hay mở rộng tập bài giải được? Kết quả của họ: *ở k lớn*, mô hình nền thường bắt kịp hoặc vượt model sau RL. Bài 46 (ProRL) phản biện rằng train RL đủ lâu thì biên giới có mở rộng. Đọc hai bài cạnh nhau.

4.6 Reward hacking và overoptimization

Goodhart: khi một thước đo trở thành mục tiêu, nó không còn là thước đo tốt. Policy tối ưu proxy reward r_ϕ đến một lúc sẽ làm **gold reward** (đánh giá thật) giảm. Bài 27 đo chính xác đường cong này theo $d = \sqrt{\text{KL}(\pi \| \pi_{\text{ref}})}$. Ví dụ hacking quen thuộc: câu trả lời dài dòng vì RM thích dài, lặp lại cụm từ RM chấm cao, in ra "đáp án" nhiều lần để trúng bộ so khớp, model sửa test thay vì sửa code.

Phòng chống: KL penalty, early stopping theo đánh giá độc lập, RM lớn hơn và nhiều dữ liệu hơn, ensemble RM, reward có thể kiểm chứng, kiểm tra mẫu bằng mắt định kỳ.

---

5. Cạm bẫy thực hành

5.1 Phương sai giữa các lần chạy

Kết quả RL rất nhạy với seed. Một khác biệt 1–2 điểm giữa hai phương pháp, chạy 1 seed, thường không có ý nghĩa. Các bài 08, 09 cho thấy chi tiết cài đặt ("code-level optimizations") có thể quan trọng hơn bản thân thuật toán.

5.2 Chi tiết cài đặt quyết định kết quả

Danh sách hay gặp (bài 08, 09, 28, 41, 44): chuẩn hóa advantage, chuẩn hóa/clip reward, clip value, khởi tạo value head, learning rate và warmup, số epoch trên mỗi batch rollout, kích thước batch, cách cộng loss theo token, xử lý token EOS và câu bị cắt độ dài, nhiệt độ lấy mẫu khi rollout, độ chính xác số (bf16/fp16/fp32) ở log-prob.

5.3 Chi phí: rollout thường là phần đắt nhất

Với LLM, sinh G câu trả lời dài cho mỗi prompt chiếm phần lớn thời gian. Hệ quả: dùng engine inference riêng (vLLM, SGLang: llm 46, 47), đồng bộ trọng số giữa trainer và engine, chấp nhận một chút off-policy (rollout bất đồng bộ) và sửa bằng importance sampling. Kimi k1.5 (bài 40) có "partial rollouts" cho câu dài.

5.4 Sụp entropy và sụp độ dài

Entropy giảm quá nhanh làm model ngừng khám phá (bài 49). Ngược lại, độ dài câu trả lời có thể phình to (học "nghĩ dài" để được thưởng) hoặc co lại (nếu loss thiên về câu ngắn). Theo dõi entropy, độ dài trung bình, tỉ lệ câu bị cắt, tỉ lệ nhóm có std bằng 0 trong suốt quá trình train.

5.5 Đánh giá

- Đánh giá ở nhiệt độ nào? pass@1 trung bình bao nhiêu mẫu (avg@k)? Một benchmark nhỏ như AIME (30 câu) cho sai số rất lớn.
- Base model nào? Bài 48 (spurious rewards) cho thấy reward ngẫu nhiên vẫn cải thiện Qwen2.5-Math nhưng không cải thiện họ model khác: kết luận trên một họ mô hình có thể không chuyển giao.
- Nhiễm dữ liệu (contamination) của benchmark trong pretraining.

5.6 Quên kiến thức cũ

Tinh chỉnh làm model quên năng lực khác. Bài 52 (RL's Razor) cho thấy RL on-policy quên ít hơn SFT, và mức quên dự đoán được bằng KL giữa model mới và mô hình gốc trên tác vụ mới. Bài 51 cho thấy RL chỉ cập nhật một mạng con nhỏ tham số.

---

6. Bảng thuật ngữ

Thuật ngữ	Nghĩa ngắn	Bài liên quan
Agent / environment	tác tử ra quyết định / môi trường phản hồi	00
State, observation	trạng thái đầy đủ / phần agent quan sát được	00, 54
Action	hành động; với LLM là token	00
Reward	phần thưởng tức thời (một số thực)	00
Return G_t	tổng reward chiết khấu từ t	00
Discount γ	hệ số giảm giá trị reward xa	00

Thuật ngữ	Nghĩa ngắn	Bài liên quan
Episode	một lượt tương tác từ đầu đến khi kết thúc	00
MDP / POMDP	mô hình ra quyết định Markov / quan sát một phần	00, 54
Policy π	quy tắc chọn hành động	00
Value V, Q-function Q	giá trị kỳ vọng của state / cặp state-action	00, 01
Advantage A	$Q - V$: tốt hơn trung bình bao nhiêu	05, 07
Bellman equation	quan hệ đệ quy giữa value hiện tại và sau	00
Bootstrapping	dùng ước lượng của chính mình làm mục tiêu	00, 01
TD error δ	$r + \gamma V(s') - V(s)$	00, 05
Monte Carlo return	return thật cuối episode	00
On-policy / off-policy / offline	dữ liệu từ policy hiện tại / policy khác / tập cố định	00, 19
Experience replay	bộ nhớ chứa chuyển trạng thái cũ để học lại	01, 03
Target network	bản sao cũ của mạng để ổn định mục tiêu	01, 10
ϵ-greedy	khám phá bằng hành động ngẫu nhiên với xác suất ϵ	01
Overestimation bias	max trên ước lượng nhiễu bị lệch lên	02, 11
Policy gradient / REINFORCE	gradient của return kỳ vọng theo tham số policy	00, 37
Baseline	số trừ khỏi return để giảm phương sai	00, 36, 37
Actor-critic	policy + value học cùng nhau	06, 07
GAE	advantage trung bình trọng số mũ các bước	05
Importance ratio ρ	$\pi_\theta / \pi_{\text{old}}$	04, 07, 43
Trust region	giới hạn mức thay đổi policy mỗi bước	04, 07
Clipping (PPO)	cắt tỉ số trong $[1-\epsilon, 1+\epsilon]$	07, 41
Entropy bonus / max-entropy RL	thưởng cho sự đa dạng hành động	06, 12, 49
Exploration / intrinsic reward	khám phá / phần thưởng nội tại	13
Model-based, world model	học mô hình môi trường để lập kế hoạch	15, 16
MCTS	tìm kiếm cây Monte Carlo	14, 15
Self-play	tự chơi với chính mình	14
Behavior cloning, DAgger	bắt chước chuyên gia; bắt chước có thu thập lại dữ liệu on-policy	17
IRL	học reward ngầm từ chuyên gia	18
Distributional shift (offline)	policy đi vào vùng không có dữ liệu	19, 20
RLHF	RL từ phản hồi của người	23–25
RLAIF	RL từ phản hồi của AI	26
Reward model, Bradley-Terry	mô hình chấm điểm học từ cặp so sánh	23, 25
KL penalty, π_{ref}	phạt lệch khỏi model tham chiếu	24, 36
Reward hacking / overoptimization	khai thác kẽ hở của reward	27
DPO và họ hàng (IPO, KTO, SimPO)	tối ưu preference không cần RL	29–33
ORM / PRM	reward theo kết quả / theo từng bước suy luận	35
GRPO, RLOO	policy gradient dùng baseline nhóm, không critic	36, 37
RLVR	RL với reward kiểm chứng tự động	38, 39
Rollout	một lần sinh trọn câu trả lời / quỹ đạo	00
pass@k	xác suất ít nhất một trong k mẫu đúng	45
Entropy collapse	entropy policy sụp nhanh khi RL	49
Agentic RL	RL cho agent LLM nhiều lượt, dùng tool	53, 54

---

7. Cách đọc paper RL một cách phản biện

1. **Baseline có được tune công bằng không?** Nhiều "phương pháp mới" thắng một baseline cài đặt kém (bài 08, 33).
2. **Bao nhiêu seed, khoảng tin cậy?** Một seed là không đủ cho RL cổ điển; với LLM thường chỉ có một lần chạy vì quá đắt, nên càng thận trọng.
3. **So sánh cùng compute chưa?** Cùng số bước gradient hay cùng số rollout, cùng GPU-giờ? Bài 44 (ScaleRL) đề xuất so sánh qua cả đường cong compute thay vì một điểm.
4. **Reward là gì, có bị hack không?** Xem ví dụ output, không chỉ con số. Độ dài câu trả lời có thay đổi không?
5. **Đánh giá:** nhiệt độ, số mẫu, benchmark có nhỏ không, có nhiễm dữ liệu không.
6. **Base model nào?** Kết luận "RL làm X" có thể chỉ đúng với một họ model (bài 48).
7. **Cái gì thật sự tạo ra lợi ích?** Thuật toán, dữ liệu, hay chi tiết kỹ thuật (lọc prompt, độ dài tối đa, format reward)? Tìm ablation.
8. **Off-policy ẩn:** rollout bất đồng bộ, khác biệt số học giữa engine sinh và engine train, có được báo cáo và sửa không?

---

8. Lịch đọc gợi ý

Tuần 1: nền tảng. 00 (đọc kỹ mục 3) → 01 DQN → 02 Double DQN → 04 TRPO → 05 GAE → 07 PPO. Làm lại các ví dụ tự tính trong mục 3 bằng tay.

Tuần 2: deep RL mở rộng. 03 Rainbow, 06 A3C, 08–09 (chi tiết cài đặt), 10–12 (điều khiển liên tục), 13 RND. Tùy hứng thú: 14–16 (planning, world model), 17–21 (imitation, offline), 22 (multi-agent).

Tuần 3: RLHF và preference. llm 07 InstructGPT → 23 → 24 → 25 → 27 → 28 → 26 → 29 DPO → 30–33.

Tuần 4: RL cho reasoning. 34 STaR → 35 PRM → 36 GRPO → 37 RLOO → 38 Tulu 3 → 39 DeepSeek-R1 → 40 Kimi k1.5 → 41 DAPO → 42 Dr. GRPO → 43 GSPO → 44 ScaleRL.

Tuần 5: RL thực sự làm gì. 45 ↔ 46 (đọc như một cuộc tranh luận) → 47, 48 → 49, 50 → 51, 52.

Tuần 6: agent và đa phương thức. 53 Search-R1 → 54 survey → 55 Visual-RFT → 56 olmOCR 2 → 57 FD-RL.

Đường tắt cho người làm post-training LLM: 00 (mục 3.6–3.11, 4) → 07 → 05 → 25 → 29 → 36 → 37 → 39 → 41 → 42 → 45 → 49 → 52.

---

9. Liên hệ với các tài liệu khác

Với `distillation content/`:

- KD on-policy (distillation 28 MiniLLM, 29 GKD, 47 ShortOPD) dùng đúng bộ máy của RL: student sinh rollout, teacher chấm. MiniLLM là policy gradient với reward $\log p_{\text{teacher}} - \log q_{\text{student}}$. DAgger (bài 17) là tổ tiên lý thuyết của ý "học trên phân phối của chính mình".
- Khác biệt then chốt: distillation cho tín hiệu **dày** (cả phân phối ở mọi token), RL cho tín hiệu **thưa** (một số cho cả chuỗi). distillation 44 (Qwen3) báo cáo on-policy distillation rẻ hơn RL nhiều để đạt cùng mức; bài 39 (R1) so distill với RL trực tiếp trên mô hình nhỏ.

Với `llm content/`: InstructGPT (bài 07) là một ví dụ RLHF; vLLM và SGLang (bài 46, 47) là các engine sinh rollout.

Với `pruning content/`: bài 51 và 52 bàn về độ thưa của cập nhật và khả năng giữ năng lực khi tinh chỉnh.

---

10. Năm câu tự kiểm tra

Câu 1. Trong MDP mục 3.3, vì sao policy tối ưu chọn hành động có reward âm (−0.2) ở s_2? Điều này nói gì về khác biệt giữa RL và tối ưu tham lam?

Đáp án.

Vì hành động "quay về" đưa agent về s_1, nơi có reward +1, nhanh hơn hành động "chờ" (chỉ có 50% về s_1 mỗi bước). Tính toán: $Q(s_2, \text{quay về}) = -0.2 + 0.9 \cdot 3.793 = 3.214 > V_2 = 3.103$ dưới policy cũ, và sau khi đổi policy, V tăng lên $[4.316, 3.684]$. RL tối ưu tổng reward chiết khấu dài hạn chứ không tối ưu reward tức thời; một policy tham lam (chỉ nhìn reward ngay) sẽ chọn "chờ" (0 > −0.2) và kém hơn.

Câu 2. Chứng minh rằng trừ baseline $b(s)$ không làm chệch policy gradient. Vì sao trong ví dụ bandit (reward cộng 10), không có baseline thì việc học vẫn đúng về kỳ vọng nhưng tệ trong thực tế?

Đáp án.

$\mathbb{E}_{a\sim\pi}[\nabla \log \pi(a \mid s) b(s)] = b(s) \sum_a \pi(a \mid s) \nabla \log \pi(a \mid s) = b(s) \sum_a \nabla \pi(a \mid s) = b(s) \nabla \sum_a \pi(a \mid s) = b(s) \nabla 1 = 0$. Trong ví dụ bandit, gradient kỳ vọng $[0.222, -0.111, -0.111]$ giống nhau dù có cộng 10 hay không, nhưng từng mẫu có độ lớn khoảng 3 đến 7 và có thể sai hướng (lấy được B thì đẩy B lên mạnh). Cần rất nhiều mẫu để trung bình hội tụ: phương sai lớn. Baseline bằng reward trung bình biến tín hiệu thành "tốt hơn hay kém hơn trung bình", mỗi mẫu đều chỉ đúng hướng.

Câu 3. Trong PPO với $\epsilon = 0.2$, cho $\hat{A} = -1$ và $\rho = 1.5$. Objective clip có tắt gradient không? Còn $\hat{A} = +1$, $\rho = 1.5$ thì sao? Giải thích ý đồ thiết kế.

Đáp án.

$\hat{A} = -1, \rho = 1.5$: $\min(-1.5, -1.2) = -1.5$, chọn số hạng không clip, nên **còn** gradient và gradient kéo ρ xuống. $\hat{A} = +1, \rho = 1.5$: $\min(1.5, 1.2) = 1.2$, chọn số hạng đã clip (hằng số theo θ), nên **không** có gradient. Ý đồ: PPO là một cận dưới bi quan. Nó ngừng thưởng khi policy đã thay đổi đủ theo hướng có lợi (tránh bước quá lớn dựa trên ước lượng cũ), nhưng không bao giờ bỏ qua việc sửa khi policy đi sai hướng.

Câu 4. Từ mục tiêu $\max_\pi \mathbb{E}_{y\sim\pi}[r] - \beta \mathrm{KL}(\pi \| \pi_{\text{ref}})$, hãy suy ra π^* và giải thích vì sao DPO không cần tính hàm phân hoạch $Z(x)$.

Đáp án.

Viết lại mục tiêu thành $-\beta \mathrm{KL}(\pi \| \pi_{\text{ref}} e^{r/\beta}/Z) + \beta \log Z$; KL nhỏ nhất (bằng 0) khi $\pi^* = \pi_{\text{ref}} e^{r/\beta}/Z$. Đảo ngược được $r = \beta \log(\pi^*/\pi_{\text{ref}}) + \beta \log Z(x)$. Mô hình Bradley-Terry chỉ dùng **hiệu** $r(x, y_w) - r(x, y_l)$, và $\beta \log Z(x)$ giống nhau cho hai câu trả lời của cùng prompt nên triệt tiêu. Còn lại một loss chỉ phụ thuộc log-prob của π_θ và π_{ref} trên y_w, y_l.

Câu 5. Một nhóm GRPO có reward $[1, 1, 1, 1]$. Advantage bằng bao nhiêu và hệ quả là gì? Đề xuất hai cách xử lý trong quy trình.

Đáp án.

Mean = 1, std = 0, nên $r_i - \text{mean} = 0$ cho mọi i (và chia cho std bằng 0 phải xử lý riêng, thường đặt advantage bằng 0). Gradient của phần policy bằng 0: tốn chi phí rollout mà không học gì (chỉ còn số hạng KL nếu có). Tương tự với $[0, 0, 0, 0]$. Cách xử lý: (1) dynamic sampling, lọc bỏ các nhóm toàn đúng hoặc toàn sai và lấy

thêm prompt cho đủ batch (bài 41 DAPO); (2) chọn prompt theo độ khó, ưu tiên prompt có tỉ lệ đúng trung gian (curriculum, bài 40), hoặc tăng G cho prompt khó; với câu hỏi luôn sai, bổ sung gợi ý hoặc dữ liệu SFT.

Tài liệu này không có đường link gốc vì không ghi chú một paper cụ thể. Sách nền nên đọc kèm: Sutton & Barto, "Reinforcement Learning: An Introduction" (bản 2, 2018), và các bài 04, 05, 07 trong thư mục cho phần policy gradient.

Bài 01

Playing Atari with Deep Reinforcement Learning (2013)

Tác giả: Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, Martin Riedmiller (DeepMind Technologies).
Venue / arXiv: arXiv:1312.5602 (v1, 19/12/2013). Bài báo tự thân không nêu venue (không có dòng "Proceedings of..." hay "Workshop..." trong văn bản); bài sau này (A3C, 1602.01783) trích dẫn nó trong danh mục tham khảo là "NIPS Deep Learning Workshop 2013".
Nguồn: https://arxiv.org/abs/1312.5602
Chủ đề: Dùng một mạng nơ-ron tích chập (CNN) để xấp xỉ hàm giá trị hành động Q, huấn luyện bằng một biến thể của Q-learning kết hợp **experience replay**, học trực tiếp từ pixel thô của màn hình game, không có đặc trưng thiết kế tay.
Vị trí trong lộ trình: Đọc sau bài 00 (MDP, phương trình Bellman, các khái niệm nền). Đây là bài đầu tiên của lộ trình, mở nhánh **value-based deep RL** cổ điển. Đọc tiếp bài 02 (Double DQN, sửa lỗi overestimation của chính thuật toán này) rồi bài 03 (Rainbow, tổng hợp 6 cải tiến gồm cả Double DQN và prioritized replay, một hạn chế mà chính bài này đã nêu ra nhưng chưa giải quyết).
Chương nền liên quan: nền 01: MDP, return và phương trình Bellman, nền 02: TD, Q-learning và DQN

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Bài toán RL (nhắc lại ngắn, chi tiết xem bài 00): một agent tương tác với environment qua các bước thời gian rời rạc. Ở bước t agent quan sát trạng thái, chọn hành động a_t trong tập hành động hợp lệ $\mathcal{A} = \{1, \dots, K\}$, nhận lại phần thưởng (reward) r_t và trạng thái mới. Mục tiêu: chọn hành động để tối đa hóa tổng phần thưởng chiết khấu (discounted return) $R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$, với $\gamma \in [0, 1]$ là hệ số chiết khấu và T là bước kết thúc episode.

Với Atari: agent chỉ nhìn thấy khung hình màn hình $x_t \in \mathbb{R}^d$ (ảnh RGB 210×160), không biết trạng thái nội tại của emulator. Một khung hình đơn không đủ suy ra đầy đủ trạng thái (ví dụ hướng/tốc độ bóng trong Pong không thấy được từ ảnh tĩnh), nên bài toán là **partially observed**: paper gộp lịch sử $s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$ làm "trạng thái" để coi như một MDP hữu hạn nhưng rất lớn.

Hàm giá trị hành động tối ưu $Q^*(s, a)$ là kỳ vọng tổng phần thưởng tối đa có thể đạt được, bắt đầu từ trạng thái s, chọn hành động a, rồi theo chính sách (policy) tốt nhất sau đó:

$$Q^*(s, a) = \max_{\pi} \mathbb{E}\big[R_t \mid s_t = s, a_t = a, \pi\big]$$

Q^* thỏa **phương trình Bellman** (đã dẫn chi tiết ở bài 00, nhắc công thức):

$$\boxed{Q^*(s, a) = \mathbb{E}_{s' \sim E}\Big[r + \gamma \max_{a'} Q^*(s', a') \;\Big|\; s, a\Big]}$$

Trực giác: nếu biết giá trị tối ưu ở bước sau cho mọi hành động, chính sách tối ưu ở bước hiện tại chỉ cần chọn hành động dẫn tới kỳ vọng $r + \gamma \max_{a'} Q^*$ lớn nhất.

Thuật ngữ cần nắm trước khi đọc tiếp:

- **Model-free:** học trực tiếp từ mẫu tương tác thật với environment E, không xây mô hình dự đoán E (khác model-based như MuZero, bài 15).
- **Off-policy / on-policy:** off-policy là học giá trị cho một chính sách (ở đây: greedy, $\arg\max_a Q$) khác chính sách dùng để sinh dữ liệu (behaviour distribution ρ, ở đây ϵ-greedy); Q-learning off-policy vì Bellman dùng $\max_{a'}$ bất kể hành động thật đã chọn. On-policy học đúng theo chính sách đang hành động (Sarsa, hay actor-critic bài 06).
- **ϵ-greedy:** chọn hành động Q lớn nhất với xác suất $1-\epsilon$, ngẫu nhiên với xác suất ϵ (đánh đổi exploit/explore).
- **Function approximator / Q-network:** vì không thể lập bảng $Q(s,a)$ cho từng cặp, Q được xấp xỉ bằng một hàm tham số θ (ở đây là CNN), $Q(s,a;\theta) \approx Q^*(s,a)$.
- **TD learning:** cập nhật ước lượng giá trị dựa trên sai khác giữa dự đoán hiện tại và một target tính từ phần thưởng quan sát cộng giá trị dự đoán bước sau, không cần chờ hết episode.

Vấn đề đã biết trước bài này: kết hợp Q-learning với bộ xấp xỉ phi tuyến hoặc off-policy learning từng được chỉ ra có thể làm phân kỳ (diverge), nên phần lớn RL trước đó dùng xấp xỉ tuyến tính. Thành công phi tuyến gần nhất là TD-Gammon (self-play, MLP một lớp ẩn, không lặp lại được ở cờ vua/vây/dama) và Neural Fitted Q-learning (NFQ, cùng dạng loss nhưng cập nhật theo batch, chi phí tỉ lệ cỡ dữ liệu, chưa thử trực tiếp trên ảnh thô).

2. Ý tưởng chính

Huấn luyện một CNN đóng vai trò Q-network $Q(s,a;\theta)$ bằng một biến thể của Q-learning, cập nhật tham số bằng stochastic gradient descent (SGD) từng bước (không phải batch như NFQ). Để tránh các vấn đề của RL online "thuần" khi dùng với mạng sâu (dữ liệu liên tiếp tương quan cao, phân phối dữ liệu đổi liên tục khi policy đổi), bài báo dùng **experience replay**: lưu lại các transition (s_t, a_t, r_t, s_{t+1}) vào một bộ đệm (replay memory), rồi lấy mẫu ngẫu nhiên một minibatch từ bộ đệm này để cập nhật mạng, thay vì chỉ dùng transition mới nhất. Mạng, kiến trúc và toàn bộ siêu tham số được giữ cố định cho cả 7 game Atari, không tinh chỉnh riêng cho từng game (trừ một ngoại lệ nhỏ về frame-skip ở Space Invaders). Kết quả: vượt các phương pháp RL trước đó trên 6/7 game, vượt người chơi chuyên nghiệp ở 3 game.

3. Phương pháp chi tiết

3.1 Hàm mất mát và gradient

Thay vì value iteration cổ điển $Q_{i+1}(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q_i(s',a') \mid s,a]$ (không khả thi vì phải ước lượng riêng cho từng (s,a)), ta huấn luyện Q-network bằng cách tối thiểu hóa một dãy hàm mất mát $L_i(\theta_i)$ thay đổi theo "lần lặp" i:

$$\boxed{L_i(\theta_i) = \mathbb{E}_{s,a\sim\rho(\cdot)}\Big[\big(y_i - Q(s,a;\theta_i)\big)^2\Big]}, \qquad y_i = \mathbb{E}_{s'\sim E}\Big[r + \gamma \max_{a'} Q(s',a';\theta_{i-1}) \;\Big|\; s,a\Big]$$

trong đó $\rho(s,a)$ là phân phối hành vi (behaviour distribution, do ϵ-greedy sinh ra) và θ_{i-1} là tham số của "lần lặp trước", được **giữ cố định** trong khi tối ưu $L_i(\theta_i)$. Điểm khác biệt so với supervised learning: target y_i phụ thuộc vào chính θ (mục tiêu "di chuyển" theo quá trình học), còn supervised learning có target cố định từ đầu.

Lấy đạo hàm theo θ_i:

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a\sim\rho(\cdot);\, s'\sim E}\Big[\big(r + \gamma \max_{a'} Q(s',a';\theta_{i-1}) - Q(s,a;\theta_i)\big)\nabla_{\theta_i} Q(s,a;\theta_i)\Big]$$

Thay kỳ vọng đầy đủ bằng một mẫu đơn (từ ρ và từ E) ở mỗi bước, cập nhật theo SGD, ta có đúng **Q-learning** kinh điển (Watkins). Thuật toán là **model-free** (không ước lượng E) và **off-policy** (học giá trị cho chính sách

greedy $a = \arg\max_a Q(s, a; \theta)$ trong khi hành vi thực tế theo ϵ-greedy).

Lưu ý: công thức (2)-(3) viết θ_{i-1} như "tham số lần lặp trước, giữ cố định", nghe giống "target network" tách biệt. Nhưng **Algorithm 1** (mục 3.4) viết cụ thể dòng tính target là `y_j = r_j + γ·max_a' Q(φ_{j+1}, a'; θ)`, dùng **đúng** θ hiện tại, không có bản sao mạng (θ^-) copy định kỳ sau mỗi τ bước. Tức là bài 2013 này **chưa có target network tách biệt theo đúng nghĩa**; đặc điểm đó được bài Double DQN (bài 02, 1509.06461) gán cho "DQN như Mnih et al. (2015) đề xuất": *"Two important ingredients of the DQN algorithm as proposed by Mnih et al. (2015) are the use of a target network, and the use of experience replay."* Bài 2015 đó là bản Nature ("Human-level control through deep reinforcement learning", Nature 518, 2015), **không có trên arXiv**; chỉ nhắc tên để phân biệt, không phải bài đang ghi chú.

3.2 Experience replay

Lưu transition $e_t = (s_t, a_t, r_t, s_{t+1})$ vào tập dữ liệu $D = e_1, \ldots, e_N$ (pool theo nhiều episode). Khi cập nhật, lấy mẫu $e \sim D$ ngẫu nhiên đều (uniform) từ bộ nhớ. Ba lý do tác giả đưa ra:

1. **Hiệu quả dữ liệu (data efficiency):** mỗi transition được dùng lại trong nhiều lần cập nhật, không chỉ một lần như online learning thuần.
2. **Giảm tương quan:** học trực tiếp từ các mẫu liên tiếp kém hiệu quả vì chúng tương quan cao; lấy mẫu ngẫu nhiên phá vỡ tương quan này, giảm phương sai của cập nhật.
3. **Tránh vòng lặp phản hồi (feedback loop) và dao động/phân kỳ:** khi học on-policy thuần, tham số hiện tại quyết định mẫu dữ liệu tiếp theo (nếu hành động tối ưu hiện tại là "trái", dữ liệu sẽ bị lệch về phía trái, rồi khi đổi sang "phải" phân phối dữ liệu đổi đột ngột). Trộn đều nhiều trạng thái hành vi quá khứ giúp làm trơn việc học.

Vì replay buffer trộn dữ liệu từ các policy cũ hơn, việc học buộc phải **off-policy** (tham số hiện tại khác tham số sinh ra mẫu), đây là lý do paper chọn Q-learning (vốn off-policy) thay vì một phương pháp on-policy.

Hạn chế do tác giả tự nêu: bộ nhớ chỉ giữ N transition gần nhất và ghi đè tuần tự, không phân biệt transition "quan trọng"; lấy mẫu đều cho mọi transition tầm quan trọng ngang nhau. Tác giả gợi ý một chiến lược phức tạp hơn, giống **prioritized sweeping**, có thể giúp học nhanh hơn, nhưng **không thử nghiệm trong bài này** (bài báo không nêu số liệu). Đây chính là ý tưởng mà **prioritized experience replay** (một thành phần của Rainbow, bài 03) sẽ giải quyết sau này.

3.3 Tiền xử lý và kiến trúc mạng

Khung hình thô 210×160 pixel, bảng màu 128 màu, được xử lý: chuyển sang grayscale, giảm kích thước xuống 110×84, rồi cắt (crop) vùng 84×84 (crop chỉ để phù hợp với cài đặt tích chập 2D trên GPU yêu cầu input vuông). Hàm tiền xử lý ϕ áp dụng lên 4 khung hình gần nhất và ghép (stack) chúng thành input, nhằm bù đắp tính partially-observed (ví dụ cho biết hướng chuyển động).

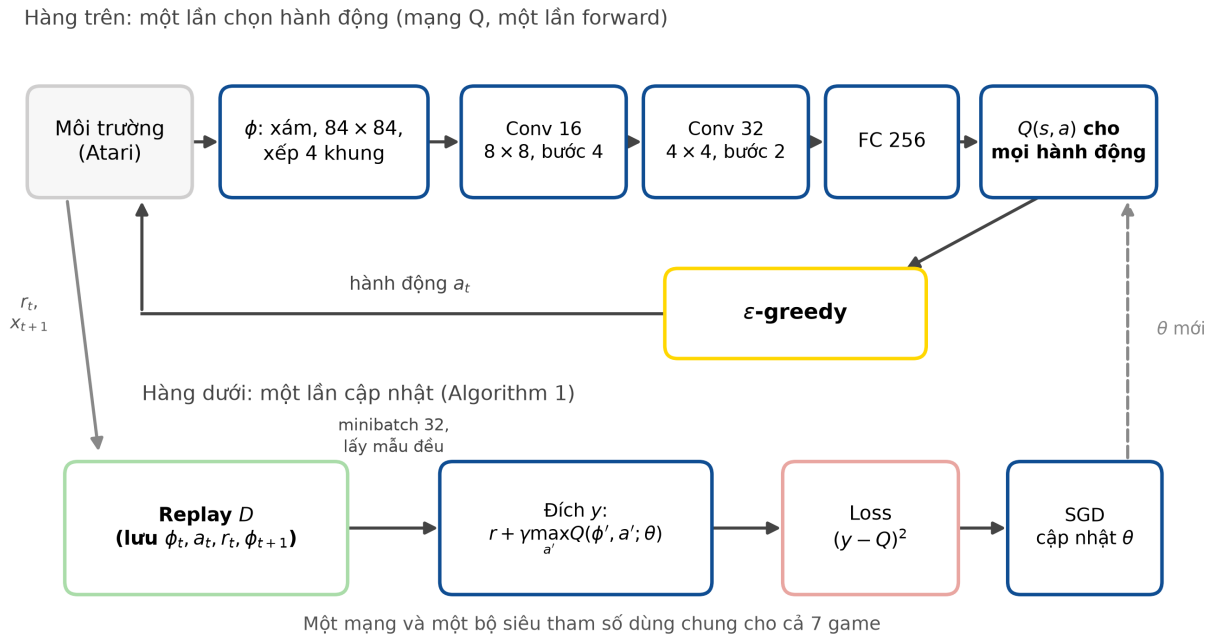
Kiến trúc CNN (giống nhau cho cả 7 game):

- Input: ảnh $84 \times 84 \times 4$.
- Conv1: 16 filter 8×8, stride 4, ReLU.
- Conv2: 32 filter 4×4, stride 2, ReLU.
- FC: 256 unit, ReLU.
- Output: fully-connected tuyến tính, **một unit cho mỗi hành động hợp lệ** (số hành động dao động 4-18 tùy game).

Lý do chọn "mỗi hành động một output" (so với kiến trúc nhận cả (trạng thái, hành động) làm input, dùng ở NFQ): chỉ cần **một lần forward pass** để có Q-value của mọi hành động tại một trạng thái, chi phí không tăng tuyến tính theo số hành động.

3.4 Thuật toán (Algorithm 1 trong paper)

```
Deep Q-learning with Experience Replay
Initialize replay memory D with capacity N
Initialize Q(s,a;theta) with random weights
for episode = 1..M:
    s1 = {x1}; phi1 = phi(s1)
    for t = 1..T:
        with probability eps: a_t = random action
        else:                 a_t = argmax_a Q(phi_t, a; theta)
        execute a_t, observe reward r_t and frame x_{t+1}
        s_{t+1} = s_t, a_t, x_{t+1};  phi_{t+1} = phi(s_{t+1})
        store (phi_t, a_t, r_t, phi_{t+1}) in D
        sample random minibatch (phi_j, a_j, r_j, phi_{j+1}) from D
        y_j = r_j                                   if phi_{j+1} is terminal
            = r_j + gamma * max_a' Q(phi_{j+1}, a'; theta)   otherwise
        gradient descent step on (y_j - Q(phi_j, a_j; theta))^2  w.r.t. theta
```



Hình 27: Sơ đồ DQN: hàng trên là một lần chọn hành động qua CNN và ϵ-greedy, hàng dưới là một lần cập nhật từ bộ nhớ replay (minh hoạ, tự tính)

Cách đọc hình: đi theo hàng trên từ môi trường qua tiền xử lý và hai lớp tích chập để ra Q của mọi hành động, rồi ϵ-greedy chọn hành động trả lại môi trường. Mỗi chuyển tiếp được lưu vào replay D; hàng dưới lấy ngẫu nhiên một minibatch, dựng đích y và chạy một bước SGD, đường nét đứt cho thấy tham số mới quay về mạng ở hàng trên.

3.5 Ví dụ nhỏ cho một bước cập nhật TD (tự tính)

Giả sử tại một bước, $Q(\phi_j, a_j; \theta) = 2.0$, phần thưởng quan sát $r_j = 1$ (đã bị clip, xem mục 3.6), $\gamma = 0.99$, và trạng thái kế tiếp không terminal với $\max_{a'} Q(\phi_{j+1}, a'; \theta) = 3.0$. Mục tiêu:

$$y_j = r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) = 1 + 0.99 \times 3.0 = 3.97$$

Sai số TD: $y_j - Q(\phi_j, a_j; \theta) = 3.97 - 2.0 = 1.97$. Gradient descent sẽ điều chỉnh θ theo hướng tăng

$Q(\phi_j, a_j; \theta)$ (vì sai số dương, giá trị dự đoán đang thấp hơn mục tiêu), với mức thay đổi tỉ lệ với $1.97 \times \nabla_\theta Q(\phi_j, a_j; \theta)$ nhân learning rate. Đây chỉ là minh họa cơ chế; bài báo không công bố giá trị Q cụ thể như vậy, số liệu là tự đặt để minh họa công thức.

3.6 Reward clipping

Vì thang điểm (score) khác nhau rất nhiều giữa các game, paper cố định mọi phần thưởng dương thành $+1$, phần thưởng âm thành -1, giữ 0 không đổi, **chỉ trong khi huấn luyện** (đánh giá vẫn trên game gốc, không clip). Lý do: giới hạn độ lớn đạo hàm lỗi, giúp dùng cùng learning rate cho nhiều game. Đánh đổi: mạng không phân biệt được phần thưởng lớn/nhỏ cùng dấu.

3.7 Chi tiết huấn luyện

Tối ưu bằng RMSProp, minibatch cỡ 32. Chính sách hành vi khi huấn luyện là ϵ-greedy, ϵ giảm tuyến tính từ 1 xuống 0.1 trong 1 triệu khung hình đầu, giữ cố định 0.1 sau đó. Huấn luyện tổng cộng 10 triệu khung hình, replay memory giữ 1 triệu khung hình gần nhất. Dùng **frame-skipping**: agent chỉ chọn hành động mỗi k khung hình, lặp lại hành động cuối trên các khung bị bỏ qua (giúp chơi nhanh hơn k lần không tăng runtime tương ứng). $k = 4$ cho mọi game, trừ Space Invaders dùng $k = 3$ (vì $k = 4$ làm tia laser biến mất do chu kỳ nhấp nháy của nó) : đây là khác biệt siêu tham số **duy nhất** giữa các game.

4. Thí nghiệm và kết quả chính

7 game Atari: Beam Rider, Breakout, Enduro, Pong, Q*bert, Seaquest, Space Invaders (Arcade Learning Environment). Cùng kiến trúc, cùng siêu tham số cho tất cả. Đánh giá: trung bình điểm khi chạy chính sách ϵ-greedy với $\epsilon = 0.05$ trong một số bước cố định (theo giao thức của Bellemare et al.).

Bảng 1 (nguyên văn Table 1 của paper)

Phương pháp	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa	996	5.2	129	-19	614	665	271
Contingency	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690
HNeat Best	3616	52	106	19	1800	920	1720
HNeat Pixel	1332	4	91	-16	1325	800	1145
DQN Best	5184	225	661	21	4500	1740	1075

Ghi chú baseline: Sarsa và Contingency dùng đặc trưng thủ công (background subtraction, mỗi màu trong 128 màu là một channel riêng), tức có sẵn tri thức miền mà DQN không có. HNeat là kiến trúc tiến hóa thần kinh, chỉ đánh giá trên episode tốt nhất (deterministic) nên so sánh không hoàn toàn công bằng; "DQN Best" (dòng cuối) là điểm episode tốt nhất của DQN để so ngang với HNeat.

Kết quả chính: DQN vượt Sarsa và Contingency trên cả 7 game với khoảng cách lớn. Vượt HNeat cả ở điểm trung bình (dòng 4) và điểm tốt nhất (dòng 8) trên mọi game trừ Space Invaders. Vượt người chơi chuyên nghiệp ở Breakout, Enduro, Pong; gần bằng người ở Beam Rider; còn thua xa ở Q*bert, Seaquest, Space Invaders, những game tác giả cho là cần chiến lược trải dài qua chu kỳ thời gian dài hơn.

5. Phân tích / ablation

Bài báo này **không có ablation study** theo nghĩa "bỏ từng thành phần rồi đo lại" (ablation có hệ thống xuất hiện từ bài 03, Rainbow). Thay vào đó có hai phân tích định tính:

- **Ổn định huấn luyện:** phần thưởng trung bình mỗi episode là một chỉ số rất nhiễu (small thay đổi về trọng số có thể gây thay đổi lớn về phân phối trạng thái mà policy ghé qua). Theo dõi Q dự đoán trung bình (trên một tập trạng thái cố định, thu bằng chính sách ngẫu nhiên trước khi huấn luyện) là chỉ số ổn định hơn nhiều, tăng trơn qua thời gian, và tác giả **không quan sát hiện tượng phân kỳ** trong mọi thí nghiệm, dù không có đảm bảo lý thuyết về hội tụ cho trường hợp phi tuyến + off-policy.
- **Trực quan hóa hàm giá trị:** trên Seaquest, giá trị dự đoán tăng vọt khi kẻ địch xuất hiện (điểm A), đạt đỉnh khi quả ngư lôi agent bắn ra gần trúng địch (điểm B), rồi giảm về mức ban đầu sau khi địch biến mất (điểm C). Cho thấy mạng học được cách giá trị thay đổi theo một chuỗi sự kiện khá phức tạp, không chỉ là hàm của khung hình hiện tại một cách hời hợt.

Bài báo không so sánh có/không có experience replay bằng số liệu cụ thể (chỉ lập luận định tính ở mục 3.2); cũng không có số liệu định lượng cho ảnh hưởng của frame-stacking 4 khung hay reward clipping. Các câu hỏi này được các bài sau (đặc biệt Nature 2015, không trên arXiv, và Rainbow bài 03) trả lời bằng ablation có hệ thống hơn.

6. Giới hạn

Tác giả nêu:

- Q*bert, Seaquest, Space Invaders còn cách xa người chơi, vì cần tìm chiến lược "trải dài qua chu kỳ thời gian dài" (long time scales), mà Q-learning một bước (one-step) khó lan truyền tín hiệu phần thưởng xa về quá khứ.
- Lấy mẫu đều từ replay buffer "ngang bằng tầm quan trọng" cho mọi transition là giới hạn; cần chiến lược lấy mẫu ưu tiên hơn (gợi ý nhưng không thử).
- Chỉ thử trên 7 game; chưa rõ tổng quát hóa ra sao trên toàn bộ bộ game Atari (tập 57 game chuẩn xuất hiện ở các bài sau, ví dụ Nature 2015 và Double DQN/Rainbow).

Nhận xét:

- Như mục 3.1: target dùng θ_{i-1} nhưng **Algorithm 1 không triển khai target network tách biệt với tần suất copy cố định**, khác mô tả "DQN" mà Double DQN/Rainbow coi là chuẩn. Có thể là một phần lý do bài 2013 chỉ test 10 triệu khung hình trên 7 game (nhỏ hơn hẳn 200 triệu khung × 49-57 game của các bài sau), chưa bị áp lực ổn định hóa lớn như khi scale lên.
- Reward clipping về $\{-1, 0, 1\}$ làm mất thông tin độ lớn; có thể khiến chính sách tối ưu *tần suất* phần thưởng hơn *tổng giá trị* thật, dù bài báo không đo tác động này.
- Không có kiểm định thống kê (nhiều seed, khoảng tin cậy) cho Bảng 1; không rõ phương sai giữa các lần chạy.
- Atari là deterministic, nên một chính sách nhớ chuỗi hành động cụ thể cũng có thể đạt điểm cao mà không "hiểu" game; đây là lo ngại mà Double DQN (bài 02) sau này kiểm tra lại bằng "human starts" (khởi động từ quỹ đạo người chơi thật).

7. Liên hệ với các bài khác

- **Bài 00 (MDP, Bellman):** nền tảng toán học cho mục 1-3; không nhắc lại chi tiết.
- **Bài 02 (Double DQN):** sửa một lỗi hệ thống của chính thuật toán này: toán tử $\max_{a'}$ trong target gây **overestimation bias**; dùng cùng Q-network và replay, chỉ đổi cách tính target.
- **Bài 03 (Rainbow):** tích hợp Double DQN với 5 cải tiến khác, gồm **prioritized replay**, chính là ý tưởng "sampling strategy phức tạp hơn" mà bài này gợi ý nhưng không triển khai.
- **Bài 06 (A3C):** hướng khác để ổn định huấn luyện: nhiều actor-learner song song thay vì replay buffer, cho phép dùng cả thuật toán on-policy (advantage actor-critic).
- **Bài 13 (RND):** một hướng khám phá tốt hơn ϵ-greedy, nhắm đúng vấn đề "chiến lược dài hạn kém" ở Q*bert/Seaquest/Space Invaders.

- **Ngoài lộ trình RL:** "off-policy, giá trị kỳ vọng phần thưởng, chính sách" ở đây là nền cho RLHF/RLVR của LLM (llm 07 InstructGPT dùng PPO, on-policy, khác Q-learning off-policy ở đây nhưng chung từ vựng "reward/policy/value").

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp đến huấn luyện LLM/VLM hiện đại: **thấp** (RL cổ điển trên game, hành động rời rạc, không có mô hình ngôn ngữ), nhưng vài bài học về ổn định hóa vẫn hữu ích:

- **Decorrelation dữ liệu là vấn đề chung của mọi RL on-the-fly:** khi policy tự sinh dữ liệu huấn luyện (rollout), dữ liệu tương quan theo thời gian. Experience replay là một giải pháp (DQN); chạy nhiều worker song song là giải pháp khác (A3C, bài 06); dùng nhiều rollout trong một batch trước khi cập nhật là cách các thuật toán on-policy cho LLM (PPO, GRPO, bài 00 và bài 36) cũng áp dụng, dù không gọi là "replay".

Reward clipping giúp hạn chế biên độ tín hiệu huấn luyện nhưng cũng làm mất thông tin về độ lớn phần thưởng. Ngoài ra, target một bước truyền tín hiệu chậm qua chuỗi dài; multi-step return và GAE là những hướng giải quyết khác nhau cho vấn đề này.

9. Thí nghiệm gợi ý

1. Huấn luyện lại DQN (Algorithm 1, đúng như mục 3.4) trên một game Atari nhỏ hoặc CartPole, so sánh 3 cấu hình: (a) không có replay (chỉ học từ transition mới nhất, giống online Q-learning thuần), (b) có replay buffer nhỏ (ví dụ 1000 transition), (c) replay buffer lớn (ví dụ 10^5-10^6). Theo dõi độ ổn định của đường cong Q trung bình (mục 5) để tái hiện quan sát "trơn hơn với replay" của bài báo.
2. Thử bỏ frame-stacking (dùng 1 khung hình thay vì 4), đo giảm điểm trên game có yếu tố vận tốc (Pong, Breakout) để kiểm chứng ảnh hưởng của partial observability.
3. So sánh reward clipping $\{-1, 0, 1\}$ với không clip (giữ nguyên độ lớn, chuẩn hóa bằng z-score) trên một game có phần thưởng không đồng nhất về độ lớn, xem có thay đổi hành vi học hay không.
4. Thêm một "target network" tách biệt (copy θ sau mỗi τ bước, dùng để tính $\max_{a'}$) vào đúng Algorithm 1 gốc, so sánh độ ổn định trước/sau, để tự kiểm chứng nhận xét ở mục 3.1/6 rằng bài 2013 chưa có cơ chế này.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Return chiết khấu	$R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$	$\gamma \in [0, 1]$
Bellman tối ưu	$Q^*(s,a) = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a]$	nền tảng Q-learning
Loss theo lần lặp i	$L_i(\theta_i) = \mathbb{E}[(y_i - Q(s,a;\theta_i))^2]$	θ_{i-1} giữ cố định khi tối ưu
Target	$y_i = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})]$	không phải target network tách biệt (xem 3.1)
Gradient (mẫu đơn)	$(y_j - Q(\phi_j, a_j; \theta))\nabla_\theta Q(\phi_j, a_j; \theta)$	SGD mỗi minibatch
Experience replay	lưu (s_t, a_t, r_t, s_{t+1}), lấy mẫu đều từ D	giảm tương quan, tái dùng dữ liệu
ϵ-greedy	greedy với $1-\epsilon$, random với ϵ	$\epsilon : 1 \to 0.1$ trong 1M khung
Reward clipping	$r \in \{-1, 0, 1\}$ khi train	mất thông tin độ lớn
Kiến trúc CNN	conv(16,8x8,s4) → conv(32,4x4,s2) → FC(256) → FC(linear, $\lvert\mathcal{A}\rvert$ output)	1 forward pass cho mọi hành động
Frame skip	chọn hành động mỗi k khung, lặp lại trên khung bị bỏ	$k = 4$, $k = 3$ ở Space Invaders

11. Kết luận ngắn

Bài báo này là bằng chứng đầu tiên rằng một CNN học end-to-end từ pixel thô, huấn luyện bằng Q-learning kết hợp experience replay, có thể vượt các phương pháp RL dựa trên đặc trưng thủ công trên hầu hết game Atari thử nghiệm, và vượt người chơi ở một số game, chỉ với một kiến trúc và một bộ siêu tham số duy nhất. Thuật toán vẫn còn thô ở nhiều điểm (chưa có target network tách biệt rõ ràng, lấy mẫu replay đều, Q-learning một bước lan truyền phần thưởng chậm), chính những điểm này mở ra các cải tiến ở các bài tiếp theo của lộ trình (Double DQN, Rainbow, A3C).

Link gốc: https://arxiv.org/abs/1312.5602

Bài 02

Deep Reinforcement Learning with Double Q-learning (2015)

Tác giả: Hado van Hasselt, Arthur Guez, David Silver (Google DeepMind).
Venue / arXiv: arXiv:1509.06461 (v3, 8/12/2015). Dòng copyright trong text ghi "Copyright © 2016, Association for the Advancement of Artificial Intelligence (www.aaai.org)", tức đây là bài AAAI 2016.
Nguồn: https://arxiv.org/abs/1509.06461
Chủ đề: Chứng minh DQN (bài 01) bị **overestimation bias** một cách hệ thống do toán tử max trong target, đo nó trên Atari, rồi sửa bằng cách tách việc "chọn hành động" (selection) và "đánh giá hành động" (evaluation) ra hai mạng khác nhau: **Double DQN**.
Vị trí trong lộ trình: Đọc sau bài 01 (DQN gốc, cùng kiến trúc, cùng dữ liệu replay, chỉ đổi công thức target). Đọc tiếp bài 03 (Rainbow, lấy Double Q-learning làm một trong 6 thành phần) và bài 11 (TD3, áp cùng ý tưởng giảm overestimation cho actor-critic liên tục).
Chương nền liên quan: nền 01: MDP, return và phương trình Bellman, nền 02: TD, Q-learning và DQN

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

DQN (bài 01) và target network: DQN kết hợp Q-learning với CNN và experience replay. Bài này mô tả DQN (bản Mnih et al. 2015, Nature, không có trên arXiv) như có hai thành phần: một **target network** θ^-, là bản sao của online network θ, copy lại sau mỗi τ bước và giữ cố định giữa các lần copy; và **experience replay**. Target DQN dùng:

$$Y_t^{DQN} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-)$$

Vấn đề: toán tử max trong target dùng **cùng** giá trị ước lượng để vừa *chọn* hành động tốt nhất (selection) vừa *đánh giá* giá trị của nó (evaluation). Nếu ước lượng có nhiễu (không chính xác), max có xu hướng chọn đúng hành động nào đang bị ước lượng cao hơn thực tế, nên dễ "thiên vị" về phía overestimate. Đây gọi là **overestimation bias**, hay **maximization bias**.

Nhầm lẫn cần tránh: đây không phải là "optimism in the face of uncertainty" (lạc quan có chủ đích để khuyến khích khám phá, một kỹ thuật exploration hợp lệ). Overestimation ở đây xảy ra *sau khi đã cập nhật*, tức là lạc quan "giả", không phải lạc quan có cơ sở về sự chưa biết.

Lịch sử ý tưởng: Thrun & Schwartz (1993) từng chỉ ra overestimation khi có nhiễu đều $[-\epsilon, \epsilon]$ trên các giá trị hành động, với chặn trên $\gamma \frac{m-1}{m+1}\epsilon$ (m = số hành động), và gán nguyên nhân cho bộ xấp xỉ thiếu linh hoạt (inflexible function approximation). van Hasselt (2010) đề xuất **Double Q-learning** (bảng, chưa dùng mạng sâu) và gán nguyên nhân cho nhiễu môi trường (ngay cả ở dạng tabular). Bài này **thống nhất hai quan điểm**: overestimation xảy ra bất kể nguồn gốc sai số (nhiễu môi trường, xấp xỉ hàm, non-stationarity, hay bất kỳ nguồn nào khác), vì bản chất là tính chất của toán tử max áp lên ước lượng có sai số, không phải của một nguồn nhiễu cụ thể.

2. Ý tưởng chính

Hai bước: (1) chứng minh bằng lý thuyết (một định lý chặn dưới tổng quát) và bằng thực nghiệm trên Atari rằng DQN overestimate có hệ thống, và việc này **làm hại** chất lượng chính sách học được (không chỉ là "lạc

quan đều, vô hại"); (2) đưa ý tưởng **Double Q-learning** (tách selection/evaluation bằng hai bộ tham số) vào đúng kiến trúc DQN, tận dụng sẵn target network θ^- làm "bộ đánh giá thứ hai" mà không cần thêm mạng mới, gọi là **Double DQN**. Kết quả: ước lượng giá trị chính xác hơn và điểm số cao hơn trên phần lớn 49 game Atari.

3. Phương pháp chi tiết

3.1 Q-learning và Double Q-learning (tabular)

Q-learning cập nhật theo:

$$\theta_{t+1} = \theta_t + \alpha(Y_t^Q - Q(S_t, A_t; \theta_t))\nabla_{\theta_t} Q(S_t, A_t; \theta_t), \qquad Y_t^Q \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t)$$

Viết lại target để thấy rõ "selection" và "evaluation" cùng dùng θ_t:

$$Y_t^Q = R_{t+1} + \gamma\, Q(S_{t+1}, \underbrace{\arg\max_a Q(S_{t+1}, a; \theta_t)}_{\text{selection}}\ ;\ \underbrace{\theta_t}_{\text{evaluation}}\ )$$

Double Q-learning (van Hasselt 2010, bản bảng): học hai hàm giá trị θ, θ', mỗi lần cập nhật gán ngẫu nhiên transition cho một trong hai; một bộ chọn hành động, bộ còn lại đánh giá:

$$\boxed{Y_t^{DoubleQ} \equiv R_{t+1} + \gamma\, Q(S_{t+1},\ \arg\max_a Q(S_{t+1}, a; \theta_t)\ ;\ \theta'_t)}$$

Vai trò của θ, θ' có thể đổi chỗ đối xứng (cập nhật θ' bằng cách hoán vị). Vì θ' độc lập (được cập nhật từ các mẫu khác) với θ dùng để chọn hành động, sai số của θ' tại hành động được chọn không còn hệ thống bị lệch theo hướng "chọn đúng hành động đang bị thổi phồng bởi chính θ".

3.2 Chặn dưới tổng quát cho overestimation (Theorem 1, diễn giải lại từng bước)

Giả thiết: tại một trạng thái s, mọi giá trị tối ưu thật bằng nhau $Q^*(s,a) = V_*(s)$ với mọi a (trong $m \geq 2$ hành động). Gọi $\epsilon_a = Q_t(s,a) - V_*(s)$ là sai số ước lượng tại hành động a. Giả sử các sai số "không thiên vị trên tổng" $\sum_a \epsilon_a = 0$ nhưng không phải tất cả bằng 0, cụ thể $\frac{1}{m}\sum_a \epsilon_a^2 = C$ với $C > 0$.

Khẳng định: $\max_a Q_t(s,a) \geq V_*(s) + \sqrt{\frac{C}{m-1}}$, và chặn này **tight** (đạt được). Trong khi đó sai số tuyệt đối nhỏ nhất có thể của ước lượng Double Q tại hành động được chọn là **0**.

Chứng minh chặn dưới (tự diễn giải từng bước theo Appendix của bài báo):

1. Giả sử phản chứng $\max_a \epsilon_a < \sqrt{C/(m-1)}$. Gọi $\{\epsilon_i^+\}$ là tập n sai số dương, $\{\epsilon_j^-\}$ là tập $m-n$ sai số âm (hoặc 0), với $n \leq m-1$ (nếu $n = m$ thì $\sum \epsilon_a = 0 \Rightarrow$ mọi $\epsilon_a = 0$, mâu thuẫn $C > 0$).
2. $\sum_i \epsilon_i^+ \leq n \max_i \epsilon_i^+ < n\sqrt{C/(m-1)}$.
3. Vì $\sum_a \epsilon_a = 0$, suy ra $\sum_j |\epsilon_j^-| = \sum_i \epsilon_i^+ < n\sqrt{C/(m-1)}$, nên $\max_j |\epsilon_j^-|$ cũng $< n\sqrt{C/(m-1)}$ (vì tổng $m-n$ số dương nhỏ hơn $n\sqrt{\cdot}$ thì từng số cũng nhỏ hơn).
4. Áp bất đẳng thức Hölder lên tổng $m-n$ số: $\sum_j (\epsilon_j^-)^2 \leq (\sum_j |\epsilon_j^-|) \cdot \max_j |\epsilon_j^-| < n\sqrt{C/(m-1)} \cdot n\sqrt{C/(m-1)} = n^2 \frac{C}{m-1}$.
5. Tổng bình phương toàn bộ: $\sum_a \epsilon_a^2 = \sum_i (\epsilon_i^+)^2 + \sum_j (\epsilon_j^-)^2 < n\frac{C}{m-1} + n^2\frac{C}{m-1} = \frac{n(n+1)}{m-1} C \leq mC$ (vì $n \leq m-1 \Rightarrow n(n+1) \leq m(m-1)$, nên $\frac{n(n+1)}{m-1} \leq m$).
6. Nhưng giả thiết cho $\sum_a \epsilon_a^2 = mC$, mâu thuẫn với "$< mC$" ở bước 5. Vậy phản chứng sai, suy ra $\max_a \epsilon_a \geq \sqrt{C/(m-1)}$, tức $\boxed{\max_a Q_t(s,a) \geq V_*(s) + \sqrt{C/(m-1)}}$.

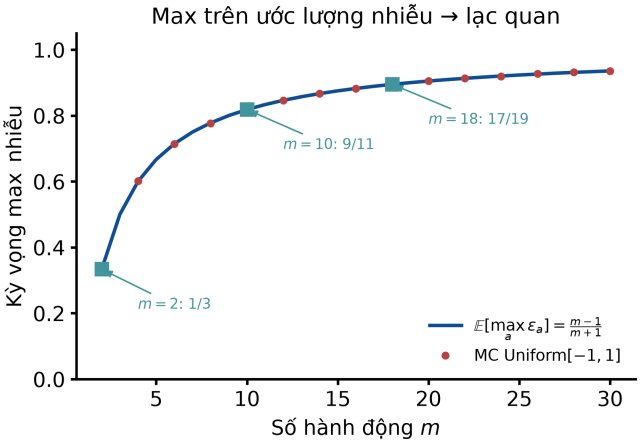
Tightness (tự tính, kiểm lại theo gợi ý của bài báo): đặt $\epsilon_a = \sqrt{C/(m-1)}$ cho $a = 1, \ldots, m-1$ và $\epsilon_m = -\sqrt{(m-1)C}$. Kiểm tổng: $\sum \epsilon_a = (m-1)\sqrt{C/(m-1)} - \sqrt{(m-1)C} = \sqrt{(m-1)C} - \sqrt{(m-1)C} = 0$ ✓. Tổng bình phương: $(m-1)\cdot\frac{C}{m-1} + (m-1)C = C + (m-1)C = mC$ ✓. Vậy chặn đạt được chính xác.

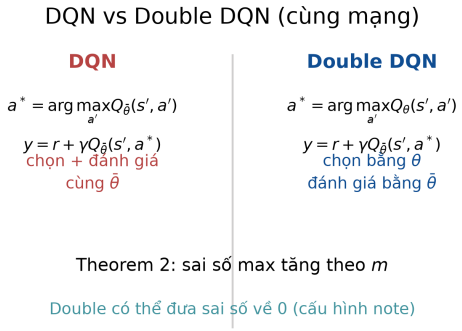
Double Q sai số 0 (tự tính, kiểm lại): đặt $Q_t(s, a_1) = V_*(s) + \sqrt{C(m-1)}$ (hành động được arg max chọn, vì đây là giá trị dương lớn nhất) và $Q_t(s, a_i) = V_*(s) - \sqrt{C/(m-1)}$ cho $i > 1$ ($m-1$ hành động còn lại). Kiểm điều kiện của Theorem 1: tổng $= \sqrt{C(m-1)} - (m-1)\sqrt{C/(m-1)} = \sqrt{C(m-1)} - \sqrt{C(m-1)} = 0$ ✓; tổng bình phương $= C(m-1) + (m-1)\cdot\frac{C}{m-1} = C(m-1) + C = mC$ ✓ (đúng điều kiện $\sum_a \epsilon_a^2 = mC$; cấu hình trong phụ lục gốc của bài báo bị mangle khi trích qua pdftotext và không khớp hệ số khi tự kiểm lại, nên đây là bản người ghi chú tự suy ra lại để thỏa đúng điều kiện của định lý). Nếu bộ đánh giá thứ hai cho $Q'_t(s, a_1) = V_*(s)$ thì sai số Double Q tại hành động được chọn bằng 0 (các $Q'_t(s, a_i)$, $i > 1$, không ảnh hưởng vì không được chọn).

Ý nghĩa: chặn dưới giảm theo $m - 1$ ở mẫu, nhưng đây là **chặn dưới chặt nhất có thể trong trường hợp xấu nhất** (không phải xu hướng trung bình); paper lưu ý thực nghiệm thường thấy overoptimism *tăng* theo số hành động (Hình 1), ngược lại xu hướng của chặn dưới lý thuyết này.

3.3 Ví dụ cụ thể: sai số đều ngẫu nhiên trong $[-1, 1]$ (Theorem 2, tự tính lại tích phân)

Giả thiết $Q^*(s, a) = V_*(s)$ với mọi a, và $\epsilon_a = Q_t(s, a) - Q^*(s, a)$ độc lập, phân bố đều trong $[-1, 1]$. Hàm phân bố tích lũy (CDF) của một ϵ_a: $P(\epsilon_a \le x) = \frac{1+x}{2}$ với $x \in [-1, 1]$. Vì độc lập, CDF của $\max_a \epsilon_a$ (với m hành động) là tích:





Overestimation: $\mathbb{E}[\max \epsilon] = (m-1)/(m+1)$; Double DQN tách chọn ($\theta$) và đánh giá ($\bar{\theta}$) (minh hoạ, tự tính)

$$P\Big(\max_a \epsilon_a \le x\Big) = \Big(\frac{1+x}{2}\Big)^m$$

Lấy đạo hàm ra hàm mật độ: $f_{\max}(x) = \frac{m}{2}\Big(\frac{1+x}{2}\Big)^{m-1}$. Kỳ vọng:

$$\mathbb{E}\Big[\max_a \epsilon_a\Big] = \int_{-1}^{1} x\, f_{\max}(x)\, dx = \Big[\Big(\frac{x+1}{2}\Big)^m \cdot \frac{mx-1}{m+1}\Big]_{-1}^{1} = \boxed{\frac{m-1}{m+1}}$$

(nguyên hàm $\int x \cdot \frac{m}{2} u^{m-1} dx$ với $u = (x+1)/2$ ra kết quả trên; tại $x = 1$: $u = 1$, giá trị $= \frac{m-1}{m+1}$; tại $x = -1$: $u = 0$, giá trị $= 0$.)

Số liệu cụ thể (tự tính từ công thức $\frac{m-1}{m+1}$):

m (số hành động)	$\frac{m-1}{m+1}$
2	$1/3 \approx 0.333$
10	$9/11 \approx 0.818$
18	$17/19 \approx 0.895$

Atari có từ 4 đến 18 hành động hợp lệ tùy game (xem bài 01); với $m = 18$ (nhiều hành động nhất), độ lệch kỳ vọng của max so với giá trị thật, trong kịch bản sai số đều $[-1, 1]$, đã gần 0.9, một overoptimism đáng kể "miễn phí" chỉ từ việc lấy max trên nhiều ước lượng nhiễu độc lập, **trước khi** tính đến việc overestimation còn bị khuếch đại thêm qua bootstrapping (ước lượng sai lan truyền dần qua các bước cập nhật kế tiếp, vì target của bước này dùng giá trị đã bị thổi phồng ở bước trước).

3.4 Double DQN: áp vào kiến trúc DQN

Thay vì thêm một mạng thứ ba, bài này nhận ra **target network θ^- đã có sẵn trong DQN** chính là một "bộ tham số thứ hai" tự nhiên. Chỉ cần đổi: *chọn* hành động bằng online network θ, *đánh giá* giá trị của hành động đó bằng target network θ^-:

$$\boxed{Y_t^{DoubleDQN} \equiv R_{t+1} + \gamma\, Q\big(S_{t+1},\ \arg\max_a Q(S_{t+1}, a; \theta_t)\ ;\ \theta_t^-\big)}$$

So với Double Q-learning gốc (3.1), θ'_t được thay bằng θ_t^- (copy định kỳ của θ_t, không phải một mạng học độc lập hoàn toàn). Việc cập nhật target network ($\theta^- \leftarrow \theta$ mỗi τ bước) **giữ nguyên như DQN**. Đây là "thay đổi tối thiểu" có thể từ DQN để có Double Q-learning, giữ nguyên mọi phần khác (replay, kiến trúc CNN, tối ưu) để so sánh công bằng, chi phí tính toán thêm gần như không đáng kể.

4. Thí nghiệm và kết quả chính

Thiết lập: 49 game Atari (Arcade Learning Environment), theo đúng giao thức huấn luyện/đánh giá của Mnih et al. (2015) và van Hasselt et al. Mạng: 3 lớp conv + 1 lớp FC ẩn (~1.5M tham số), input 4 khung hình cuối. Huấn luyện 200M khung hình (~1 tuần) trên 1 GPU mỗi game.

Kết quả overoptimism (Hình 3): DQN nhất quán và đôi khi *rất* lạc quan về giá trị của chính sách greedy hiện tại (đường ước lượng học được cao hơn hẳn đường giá trị thật đo bằng rollout cuối cùng). Double DQN bám sát đường giá trị thật hơn nhiều. Hai trường hợp cực đoan: Asterix và Wizard of Wor, DQN mất ổn định nghiêm trọng (dùng thang log cho giá trị); điểm số rơi đúng lúc ước lượng giá trị tăng vọt, cho thấy overestimation **gây hại thật** tới chất lượng chính sách, không chỉ là hiện tượng vô hại. Quan sát overestimation có mặt ở **cả 49 game** (mức độ khác nhau), không chỉ vài ví dụ minh họa.

Chất lượng chính sách, no-op starts (Bảng 1, chuẩn hóa theo (score_agent - score_random)/(score_human - score_random)):

	DQN	Double DQN
Median	93.5%	114.7%
Mean	241.1%	330.3%

Chất lượng chính sách, human starts, Double DQN đã tinh chỉnh thêm (Bảng 2):

	DQN	Double DQN	Double DQN (tuned)
Median	47.5%	88.4%	116.7%
Mean	122.0%	273.1%	475.2%

Phiên bản "tuned" tăng khoảng cách copy target network từ 10.000 lên 30.000 bước (giảm overestimation thêm), giảm ϵ khai thác còn 0.01 khi huấn luyện và 0.001 khi đánh giá, và dùng một bias chung (shared bias) ở lớp output cuối.

Một số game cải thiện mạnh (no-op, % điểm chuẩn hóa, tự trích từ phụ lục): Road Runner 233% → 617%, Asterix 70% → 180%, Zaxxon 54% → 111%, Double Dunk 17% → 397%.

Human starts (robustness): đánh giá từ 100 điểm khởi động lấy từ quỹ đạo người chơi thật (không phải điểm bắt đầu cố định như no-op), để kiểm tra agent có "học vẹt" một chuỗi hành động xác định hay tổng quát hóa thật. Double DQN vẫn vượt DQN rõ rệt ở chế độ khó hơn này, cho thấy cải thiện không chỉ do khai thác tính deterministic của môi trường.

5. Phân tích / ablation

Đây **không phải một ablation nhiều thành phần** (bài chỉ có một thay đổi duy nhất: công thức target), nhưng có các phân tích đáng chú ý:

- **Overoptimism tăng theo số hành động** (Hình 1, thực nghiệm toy với $Q(s,a) = V_*(s) + \epsilon_a$, $\epsilon_a \sim \mathcal{N}(0,1)$ độc lập): orange bar (bias của Q-learning một bước) tăng dần khi số hành động tăng từ 2 đến 1024; blue bar (Double Q-learning) luôn gần 0. Bài báo không in giá trị số cụ thể của các cột (chỉ là biểu đồ), nên không trích số ở đây.
- **Vai trò của độ linh hoạt bộ xấp xỉ** (Hình 2, 3 hàng so sánh): overestimation xuất hiện cả khi hàm xấp xỉ *thiếu* linh hoạt (không khớp hết các điểm mẫu) và khi *đủ* linh hoạt để khớp hết các điểm mẫu nhưng kém chính xác ở vùng không có mẫu; tức đây không phải chỉ là vấn đề của bộ xấp xỉ quá đơn giản (bác lại một phần kết luận cũ của Thrun & Schwartz).
- **Hiệu ứng bootstrapping:** ước lượng overoptimistic ở một trạng thái lan sang các trạng thái khác qua target của Bellman, tích lũy qua nhiều bước cập nhật, nên lỗi hệ thống (không ngẫu nhiên, không trung bình về 0 giữa các trạng thái) gây hại nhiều hơn lỗi ngẫu nhiên đơn lẻ.
- **Trường hợp overoptimism không hại:** bài báo nêu ví dụ Pong, DQN đạt hành vi tối ưu dù hơi overestimate giá trị chính sách, cho thấy không phải cứ overestimate là chính sách tệ; nhưng nhìn tổng thể 49 game, giảm overestimation vẫn cải thiện trung bình/trung vị rõ rệt.

6. Giới hạn

Tác giả nêu:

- Double DQN dùng chung mọi siêu tham số đã tối ưu cho DQN (không phải cho Double DQN), nên so sánh ở Bảng 1 có phần bất lợi cho Double DQN ("somewhat adversarial"); phần "tuned" (Bảng 2) mới thật sự tinh chỉnh riêng.
- Chặn dưới lý thuyết (Theorem 1) *giảm* theo số hành động, trái ngược xu hướng overoptimism *tăng* theo số hành động quan sát thực nghiệm (Hình 1); tác giả giải thích chặn dưới chỉ đúng cho trường hợp xấu nhất chặt (cần cấu hình ϵ_a rất đặc biệt), không phải xu hướng điển hình.
- Không so sánh với Gorila (DQN phân tán, 130 máy) trong bảng chính vì khác biệt kiến trúc/hạ tầng quá lớn để so sánh công bằng, chỉ ghi chú điểm của Gorila riêng.

Nhận xét:

- Theorem 1 giả định $\sum_a \epsilon_a = 0$ (sai số "không thiên vị trên tổng"), một giả thiết hợp lý trên lý thuyết nhưng không kiểm chứng được trực tiếp trên Atari thật (không biết Q^* thật để đo ϵ_a chính xác); phần thực nghiệm Atari (Hình 3) đo gián tiếp qua so sánh với return thật của chính sách cuối, không đo trực tiếp ϵ_a theo từng hành động.
- Double DQN giảm overestimation nhưng không loại bỏ hoàn toàn (đường ước lượng Double DQN trong Hình 3 vẫn hơi cao hơn đường giá trị thật ở một số game), và không có bảo đảm lý thuyết nào rằng Double Q-learning không *underestimate* trong một số tình huống (thực tế Rainbow, bài 03, sau này quan sát double Q-learning có thể gây *underestimate* khi kết hợp với phân phối giá trị có support giới hạn).
- Cải thiện cụ thể theo game rất không đồng đều (một số game cải thiện hàng trăm %, một số gần như không đổi hoặc giảm nhẹ, xem Bảng 4 phụ lục của bài báo gốc không trích đầy đủ ở đây); median/mean tổng hợp có thể che khuất các trường hợp xấu.

7. Liên hệ với các bài khác

- **Bài 01 (DQN):** nền tảng trực tiếp; Double DQN chỉ thay một dòng công thức target, mọi phần khác (replay, CNN, ϵ-greedy, target network θ^-) giữ nguyên.
- **Bài 03 (Rainbow):** nhận Double Q-learning làm một trong 6 thành phần; ablation của Rainbow cho thấy khi kết hợp với distributional RL (support giá trị bị chặn $[-10, 10]$), vai trò giảm overestimation của Double Q có thể đổi thành chống *underestimate* (do return thật bị cắt ngoài support), một quan sát thú vị khác hẳn bối cảnh gốc của bài này.
- **Bài 11 (TD3):** áp dụng trực tiếp ý tưởng "tách selection/evaluation để giảm overestimation" sang actor-critic liên tục (continuous control), dùng hai Q-network và lấy min của cả hai làm target, một biến thể khác của cùng nguyên lý.
- **Bài 12 (SAC):** cũng dùng hai Q-network (twin Q) với lý do tương tự.
- **Ngoài lộ trình RL:** overestimation do max là một hiện tượng tổng quát bất cứ khi nào có bước chọn "tốt nhất trong số các ước lượng nhiễu" (ví dụ chọn mô hình phần thưởng có điểm cao nhất trong nhiều lần sample ở reward hacking/reward over-optimization, liên hệ xa với bài 27, Scaling Laws for Reward Model Overoptimization, tuy cơ chế toán học không hoàn toàn giống).

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp đến huấn luyện LLM/VLM bằng RL hiện đại: **trung bình**, vì các thuật toán RLHF/RLVR phổ biến cho LLM (PPO, GRPO) là on-policy policy-gradient, không dùng $\max_a Q$ trực tiếp như DQN, nhưng bài học về "chọn và đánh giá bằng cùng một nguồn ước lượng nhiễu gây lạc quan giả" vẫn tổng quát:

- **Reward hacking/reward overoptimization có họ hàng xa với overestimation bias:** nếu chọn response "tốt nhất theo reward model" trong nhiều mẫu rồi dùng chính điểm đó làm tín hiệu huấn luyện (ví dụ best-of-N distillation, hay một số biến thể RLVR chọn lời giải điểm cao nhất trong nhiều lần sample), phần được chọn có xu hướng là phần mà mô hình phần thưởng *đánh giá cao nhất*, có thể cao hơn giá trị thật một cách hệ thống, giống cơ chế max gây lệch ở đây. Gợi ý thực dụng: nếu phải chọn "tốt nhất trong K mẫu" để làm nhãn, nên đánh giá bằng một ước lượng *độc lập* với ước lượng dùng để chọn (tương tự Double Q), ví dụ dùng một mô hình phần thưởng khác hoặc một bộ giữ lại (held-out) để chấm lại.
- **Target network vẫn là một "tài sản miễn phí":** nếu một quy trình RL nào đó (hiếm với LLM hiện đại, nhưng có ở các bài value-based cũ hơn trong lộ trình) đã có sẵn một mạng trễ (delayed/target network) vì lý do ổn định huấn luyện khác, bài này cho thấy có thể tận dụng chính mạng đó để giảm một loại lỗi khác (overestimation) gần như không tốn thêm tài nguyên.

9. Thí nghiệm gợi ý

1. Tái hiện thí nghiệm toy ở Hình 1: với $m \in \{2, 4, 8, 16, 32\}$ hành động, $Q(s, a) = V_*(s) + \epsilon_a$, $\epsilon_a \sim \mathcal{N}(0, 1)$ độc lập, đo thực nghiệm (trung bình nhiều lần lặp) $\mathbb{E}[\max_a Q - V_*]$ so với $\mathbb{E}[Q'(\arg\max_a Q) - V_*]$ theo kiểu Double Q, và so sánh xu hướng tăng theo m với bảng $\frac{m-1}{m+1}$ (phân phối đều) đã tính ở mục 3.3 (phân phối chuẩn sẽ ra số khác nhưng xu hướng tăng theo m tương tự).
2. Trên một game Atari đơn giản (ví dụ Pong hoặc Breakout), huấn luyện cả DQN và Double DQN với cùng kiến trúc/siêu tham số, log cả giá trị Q dự đoán trung bình và giá trị thật đo bằng rollout định kỳ (như Hình 3), kiểm tra khoảng cách giữa hai đường này có hẹp lại với Double DQN.
3. Ablation riêng khoảng copy target network $\tau \in \{1000, 10000, 30000, 100000\}$ trên một game, đo overestimation còn lại, để kiểm chứng lập luận "tăng τ giảm overestimation thêm" dùng trong bản "tuned".
4. Thử kết hợp Double Q-learning với một thiết lập có support giá trị bị chặn (giống distributional RL, bài 03) trên một game có return vượt khỏi chặn, quan sát xem có chuyển từ overestimate sang underestimate như Rainbow ghi nhận hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Target DQN	$Y_t^{DQN} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-)$	selection = evaluation, cùng θ^-
Target Double Q (tabular)	$R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta'_t)$	2 bộ tham số độc lập
Target Double DQN	$R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta_t^-)$	tận dụng θ^- có sẵn
Chặn dưới overestimation	$\max_a Q_t(s, a) \geq V_*(s) + \sqrt{C/(m-1)}$	$C = \frac{1}{m}\sum_a \epsilon_a^2$, $\sum_a \epsilon_a = 0$
Sai số đều $[-1, 1]$, độc lập	$\mathbb{E}[\max_a \epsilon_a] = \frac{m-1}{m+1}$	$m = 2$:$1/3$, $m = 10$:$9/11$, $m = 18$:$17/19$ (tự tính)
Chuẩn hóa điểm	$\frac{\text{score}_{\text{agent}} - \text{score}_{\text{random}}}{\text{score}_{\text{human}} - \text{score}_{\text{random}}}$	% human-normalized
Median/Mean no-op	DQN 93.5%/241.1%; DDQN 114.7%/330.3%	Bảng 1
Median/Mean human starts	DQN 47.5%/122.0%; DDQN 88.4%/273.1%; DDQN tuned 116.7%/475.2%	Bảng 2
Siêu tham số chính	$\gamma = 0.99$, $\alpha = 0.00025$, $\tau = 10000$ (tuned: 30000)	RMSProp, replay 1M

11. Kết luận ngắn

Bài này chỉ ra, bằng cả lý thuyết (chặn dưới tổng quát cho mọi nguồn sai số) và thực nghiệm trên toàn bộ 49 game Atari, rằng toán tử max trong Q-learning/DQN tạo ra overestimation bias có hệ thống, và bias này thực sự làm hại chất lượng chính sách ở nhiều game, không chỉ là hiện tượng lạc quan vô hại. Bằng cách tách việc chọn hành động (online network) và đánh giá hành động (target network, đã có sẵn trong DQN) thành hai nguồn ước lượng khác nhau, Double DQN giảm đáng kể overestimation với chi phí tính toán gần như bằng 0, cải thiện cả độ chính xác của giá trị ước lượng và điểm số cuối cùng trên cả chế độ no-op và human-starts.

Link gốc: https://arxiv.org/abs/1509.06461

Bài 03

Rainbow: Combining Improvements in Deep Reinforcement Learning (2017)

Tác giả: Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, David Silver (DeepMind).
Venue / arXiv: arXiv:1710.02298 (v1, 6/10/2017). Dòng copyright ghi "Copyright © 2018, Association for the Advancement of Artificial Intelligence", tức bài AAAI 2018.
Nguồn: https://arxiv.org/abs/1710.02298
Chủ đề: Tích hợp 6 cải tiến độc lập của họ DQN (value-based) vào một agent duy nhất **Rainbow**: Double Q-learning, Prioritized Experience Replay, Dueling network, Multi-step learning, Distributional RL (C51), Noisy Nets; đo đóng góp của từng thành phần bằng ablation.
Vị trí trong lộ trình: Đọc sau bài 01 (DQN), bài 02 (Double DQN, 1 trong 6 thành phần). Đây là điểm hội tụ của nhánh value-based cổ điển; sau bài này lộ trình chuyển sang nhánh policy-gradient/actor-critic (bài 06 A3C đã được nhắc như một hướng khác trong Related Work của chính bài này, bài 04 TRPO, bài 05 GAE, bài 07 PPO).
Chương nền liên quan: nền 02: TD, Q-learning và DQN

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Từ DQN (bài 01), cộng đồng RL đề xuất nhiều cải tiến **độc lập**, mỗi cải tiến nhắm vào một hạn chế riêng của DQN: Double DQN (overestimation, bài 02), Prioritized Experience Replay (lấy mẫu đều từ replay buffer là lãng phí), Dueling network (tách giá trị trạng thái và lợi thế hành động), Multi-step learning (one-step lan truyền phần thưởng chậm), Distributional RL (chỉ học kỳ vọng Q bỏ qua thông tin về độ bất định của phân phối return), Noisy Nets (ϵ-greedy khám phá kém hiệu quả ở game cần nhiều bước trước khi có phần thưởng đầu tiên, ví dụ Montezuma's Revenge). Một số cặp đã được kết hợp trước đó (Prioritized DDQN, Dueling DDQN, Dueling DDQN + prioritized), nhưng chưa ai kết hợp **cả 6**. Câu hỏi: các cải tiến này có bù trừ/xung đột nhau không, hay chúng giải quyết các vấn đề đủ khác nhau để **cộng dồn**?

Khung MDP nhắc lại (ký hiệu của bài này, hơi khác bài 01/02): $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma \rangle$; tại bước t agent nhận trạng thái S_t, chọn A_t, nhận lại R_{t+1}, discount γ_{t+1} (bằng γ cố định, trừ bằng 0 khi kết thúc episode), và S_{t+1}. Return chiết khấu $G_t = \sum_{k=0}^{\infty} \gamma_t^{(k)} R_{t+k+1}$ với $\gamma_t^{(k)} = \prod_{i=1}^{k} \gamma_{t+i}$. Loss DQN gốc (dùng lại ký hiệu q_θ cho mạng):

$$\left(R_{t+1} + \gamma_{t+1} \max_{a'} q_\theta(S_{t+1}, a') - q_\theta(S_t, A_t)\right)^2$$

2. Ý tưởng chính

Ghép cả 6 cải tiến vào một agent, chọn cách kết hợp hợp lý khi hai thành phần tương tác với nhau (ví dụ: multi-step distributional loss kết hợp double Q-learning; dueling kết hợp distributional bằng cách tách stream giá trị/lợi thế theo *từng atom* của phân phối; priority dùng KL loss chứ không dùng TD error tuyệt đối vì đó mới là đại lượng mà agent thực sự tối thiểu hóa). Kết quả: Rainbow đạt state-of-the-art trên 57 game Atari, cả về tốc độ học (data efficiency) và điểm cuối cùng; ablation cho thấy hầu hết thành phần đóng góp dương, mức độ khác nhau theo từng game.

3. Phương pháp chi tiết

3.1 Double Q-learning (bài 02)

$$\big(R_{t+1} + \gamma_{t+1}\, q_\theta\big(S_{t+1}, \arg\max_{a'} q_\theta(S_{t+1}, a')\big) - q_\theta(S_t, A_t)\big)^2$$

trong công thức cụ thể của Rainbow: q_θ cuối (evaluation) thực chất dùng **target network** θ^- như Double DQN, chọn hành động bằng online network. Mục đích: giảm overestimation bias đã dẫn chi tiết ở bài 02.

3.2 Prioritized Experience Replay

DQN lấy mẫu đều từ replay buffer. Ở đây lấy mẫu transition t với xác suất tỉ lệ TD-error tuyệt đối ở lần gần nhất:

$$p_t \propto \Big|R_{t+1} + \gamma_{t+1} \max_{a'} q_\theta(S_{t+1}, a') - q_\theta(S_t, A_t)\Big|^\omega$$

ω là siêu tham số quyết định độ "nhọn" của phân phối ưu tiên ($\omega = 0$: lấy mẫu đều). Transition mới luôn được chèn vào buffer với priority tối đa (ưu tiên các transition gần đây trước khi có TD-error đo được). Trực giác: dành nhiều lượt cập nhật hơn cho các transition mà mô hình hiện đang sai nhiều (còn nhiều để học), thay vì chia đều cho mọi transition như DQN gốc.

3.3 Dueling network

Kiến trúc tách thành hai nhánh tính toán chia sẻ một encoder tích chập f_ξ: nhánh giá trị trạng thái v_η và nhánh lợi thế hành động (advantage) a_ψ, hợp nhất bằng:

$$q_\theta(s, a) = v_\eta\big(f_\xi(s)\big) + a_\psi\big(f_\xi(s), a\big) - \frac{\sum_{a'} a_\psi(f_\xi(s), a')}{N_{\text{actions}}}$$

với $\theta = \{\xi, \eta, \psi\}$. Trừ đi trung bình advantage trên mọi hành động để giải quyết tính không xác định (unidentifiability): nếu không trừ, có thể cộng một hằng số vào v và trừ cùng hằng số đó khỏi mọi a_ψ mà q không đổi, khiến việc huấn luyện kém ổn định. Mục đích: mạng học tốt hơn giá trị trạng thái (dùng chung cho mọi hành động) mà không cần trải nghiệm từng hành động riêng ở mọi trạng thái.

3.4 Multi-step learning

Thay return một bước bằng **n-step return có cắt (truncated)**:

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}$$

Loss multi-step:

$$\big(R_t^{(n)} + \gamma_t^{(n)} \max_{a'} q_\theta(S_{t+n}, a') - q_\theta(S_t, A_t)\big)^2$$

n lớn hơn giúp lan truyền phần thưởng về các bước trước nhanh hơn (một phần thưởng ảnh hưởng trực tiếp tới n cặp (trạng thái, hành động) liền trước, thay vì chỉ 1), nhưng cũng tăng phương sai (reward-to-go cộng dồn qua nhiều bước, giống lập luận về R_t trong MiniLLM, bài distillation 28, dù ngữ cảnh khác hẳn).

3.5 Distributional RL (C51)

Thay vì chỉ học kỳ vọng Q, học **toàn bộ phân phối** của return, biểu diễn bằng N_{atoms} khối xác suất trên một lưới giá trị cố định z^i:

$$z^i = v_{\min} + (i-1)\frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1}, \qquad i = 1, \dots, N_{\text{atoms}}$$

Phân phối xấp xỉ $d_t = (z, p_\theta(S_t, A_t))$, với p_θ là xác suất trên từng atom (dùng softmax để chuẩn hóa). Phương trình Bellman cho phân phối: lấy phân phối ở S_{t+1} dưới hành động tối ưu $a^*_{t+1} = \arg\max_a q_\theta(S_{t+1}, a)$ (với $q_\theta(s,a) = z^\top p_\theta(s,a)$), **co lại** theo discount và **dịch** theo reward, rồi **chiếu (project)** về đúng lưới z cố định bằng một phép chiếu Φ_z (paper gọi tên Φ_z là "L2-projection" nhưng **không viết ra công thức chiếu cụ thể** trong bài này, "paper không nêu công thức chiếu"). Tối thiểu hóa Kullback-Leibler divergence:

$$D_{KL}\big(\Phi_z d'_t \,\|\, d_t\big), \qquad d'_t \equiv \big(R_{t+1} + \gamma_{t+1} z,\ p_\theta(S_{t+1}, a^*_{t+1})\big)$$

Ví dụ nhỏ (tự tính): Rainbow dùng $N_{\text{atoms}} = 51$, $v_{\min} = -10$, $v_{\max} = 10$ (Bảng 1 của bài báo). Bước lưới: $\frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1} = \frac{20}{50} = 0.4$. Vậy $z^1 = -10$, $z^{26} = -10 + 25 \times 0.4 = 0$ (atom chính giữa đúng bằng 0), $z^{51} = -10 + 50 \times 0.4 = 10$. Mọi ước lượng return của Rainbow bị **cắt (clip) cứng** vào $[-10, 10]$ vì lưới chỉ trải tới đó; nếu return thật lớn hơn, phân phối bị dồn về mép, đây là nguồn gây **underestimate** được nhắc ở mục 5.

3.6 Noisy Nets

Thay lớp tuyến tính chuẩn $y = b + Wx$ bằng một lớp tuyến tính "ồn" (noisy linear layer), kết hợp một dòng xác định và một dòng có nhiễu:

$$y = (b + Wx) + \big(b_{\text{noisy}} \odot \varepsilon^b + (W_{\text{noisy}} \odot \varepsilon^w)x\big)$$

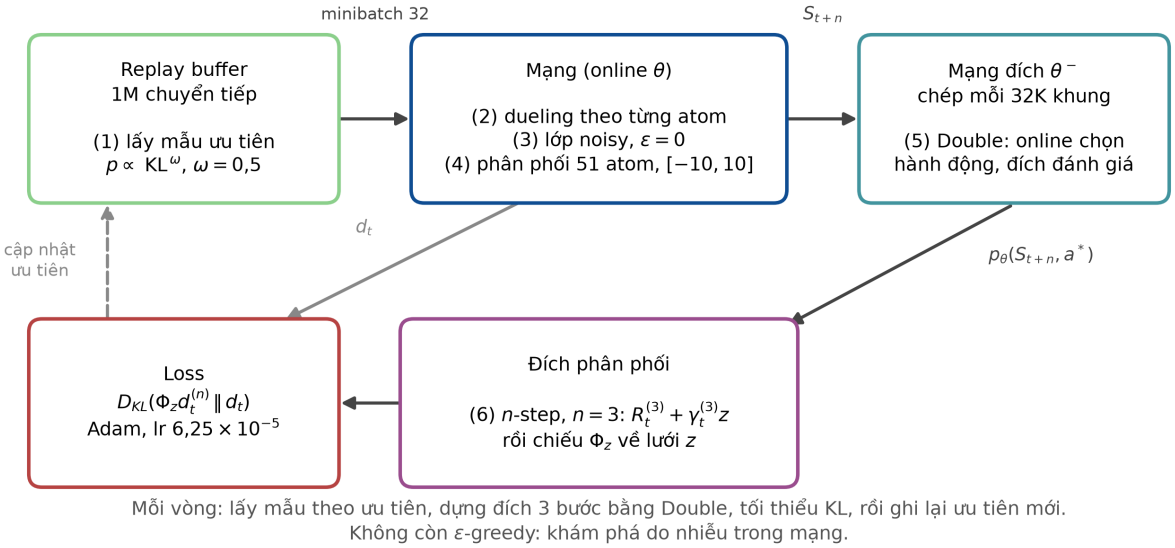
với $\varepsilon^b, \varepsilon^w$ là biến ngẫu nhiên (factorised Gaussian noise để giảm số biến nhiễu độc lập cần sinh), $\odot$ là tích phần tử. Qua thời gian huấn luyện mạng có thể học cách "bỏ qua" dòng nhiễu (đưa $b_{\text{noisy}}, W_{\text{noisy}} \to 0$) ở những vùng trạng thái đã học chắc, nhưng vẫn giữ nhiễu ở vùng chưa chắc, tạo **tự ủ (self-annealing) khám phá theo trạng thái**, thay cho ϵ-greedy đồng nhất không gian trạng thái.

3.7 Tích hợp thành Rainbow

- **Multi-step + Distributional:** target xây bằng cách co phân phối ở S_{t+n} theo discount tích lũy rồi dịch theo n-step return: $d_t^{(n)} = (R_t^{(n)} + \gamma_t^{(n)} z,\ p_\theta(S_{t+n}, a^*_{t+n}))$, loss $D_{KL}(\Phi_z d_t^{(n)} \| d_t)$.
- **+ Double Q-learning:** hành động greedy a^*_{t+n} chọn bằng **online network**, đánh giá (lấy phân phối p tại hành động đó) bằng **target network**.
- **+ Prioritized replay:** vì mục tiêu tối thiểu hóa thực sự là KL loss (không phải TD-error như DQN gốc), priority đổi thành $p_t \propto \big(D_{KL}(\Phi_z d_t^{(n)} \| d_t)\big)^\omega$. Tác giả lưu ý loss KL "bền" hơn với môi trường ngẫu nhiên (return không xác định) vì nó vẫn giảm được dù return có nhiễu, khác TD-error vốn có thể không giảm về 0.
- **+ Dueling:** kiến trúc dueling áp cho **mỗi atom riêng**: nhánh giá trị v_η có N_{atoms} output, nhánh lợi thế a_ξ có $N_{\text{atoms}} \times N_{\text{actions}}$ output, hợp nhất theo từng atom i rồi softmax theo atom để chuẩn hóa phân phối cho mỗi hành động:

$$p^i_\theta(s,a) = \frac{\exp\big(v^i_\eta(\phi) + a^i_\psi(\phi,a) - \bar{a}^i_\psi(s)\big)}{\sum_j \exp\big(v^j_\eta(\phi) + a^j_\psi(\phi,a) - \bar{a}^j_\psi(s)\big)}, \qquad \bar{a}^i_\psi(s) = \frac{1}{N_{\text{actions}}}\sum_{a'} a^i_\psi(\phi, a')$$

- **+ Noisy Nets:** mọi lớp tuyến tính trong mạng được thay bằng lớp noisy linear (mục 3.6); khi dùng Noisy Nets, hành động chọn **hoàn toàn greedy** ($\epsilon = 0$), không còn ϵ-greedy song song.



Mỗi vòng: lấy mẫu theo ưu tiên, dựng đích 3 bước bằng Double, tối thiểu KL, rồi ghi lại ưu tiên mới.
Không còn ε-greedy: khám phá do nhiễu trong mạng.

Hình 28: Cách sáu thành phần của Rainbow ghép vào một vòng cập nhật: lấy mẫu ưu tiên, mạng dueling và noisy với phân phối 51 atom, đích 3 bước dùng Double, loss KL (minh hoạ, tự tính)

Cách đọc hình: các số trong ngoặc đánh dấu sáu cải tiến và vị trí chúng tác động. Mẫu đi từ replay qua mạng online, mạng đích đánh giá hành động do mạng online chọn, rồi đích phân phối 3 bước được chiếu về lưới atom và so với dự đoán bằng KL. Giá trị KL này đồng thời là ưu tiên ghi ngược về replay, nên ưu tiên và loss dùng cùng một đại lượng.

3.8 Bảng siêu tham số (Table 1 của paper)

Siêu tham số	Giá trị
Min history để bắt đầu học	80K khung hình
Learning rate Adam	0.0000625
ϵ (ϵ-greedy)	0.0 (dùng Noisy Nets thay)
Noisy Nets σ_0	0.5
Chu kỳ copy target network	32K khung hình
Adam ϵ_{Adam}	1.5×10^{-4}
Loại prioritization	proportional
Số mũ priority ω	0.5
Importance sampling β	$0.4 \rightarrow 1.0$
Số bước multi-step n	3
Số atom phân phối	51
$[v_{\text{min}}, v_{\text{max}}]$	$[-10, 10]$

Các siêu tham số khác (kiến trúc CNN 3 lớp conv 32/64/64 filter, discount 0.99, replay 1M transition, học mỗi 4 bước agent, minibatch 32) **kế thừa từ DQN/các biến thể**, không đổi. Learning rate dùng $\alpha/4$ (so

$\alpha = 0.00025$ gốc của DQN), chọn trong $\{\alpha/2, \alpha/4, \alpha/6\}$; ω chọn trong $\{0.4, 0.5, 0.7\}$; n so sánh $1, 3, 5$, kết luận $n = 3$ tốt nhất về cuối (dù $n = 5$ có nhanh hơn lúc đầu). Tối ưu bằng **Adam**, khác RMSProp của DQN gốc, vì "ít nhạy với learning rate hơn".

4. Thí nghiệm và kết quả chính

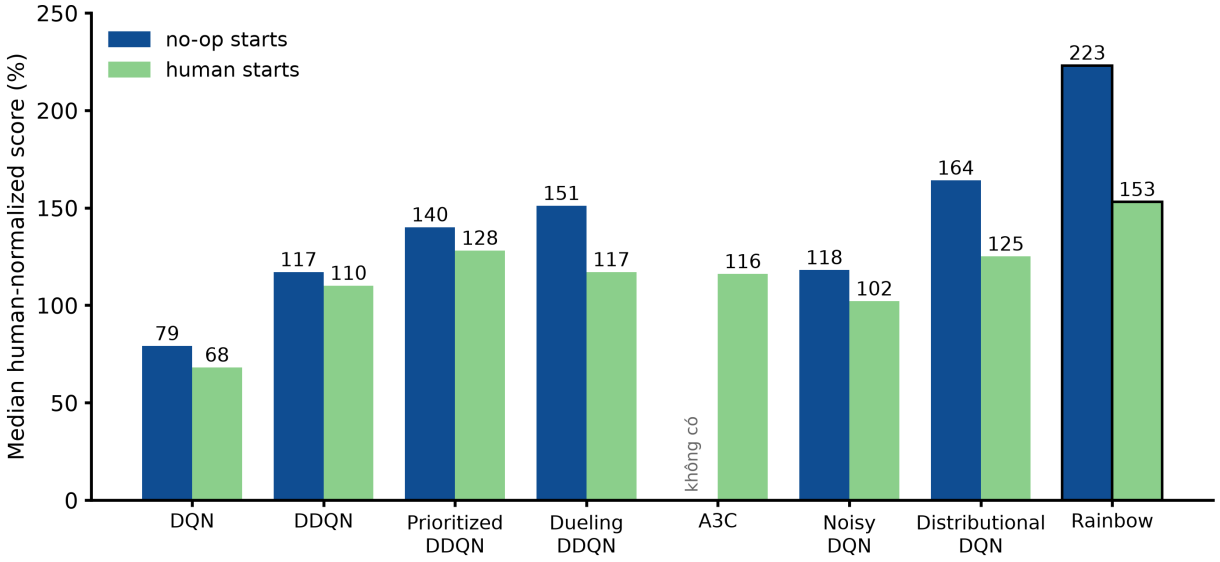
Thiết lập: 57 game Atari, 200M khung hình/game, 1 GPU/agent, theo đúng giao thức huấn luyện/đánh giá của Mnih et al. (2015) và van Hasselt et al. (2016). Đánh giá: trung bình cứ 1M bước huấn luyện, đóng băng agent, chạy 500K khung hình đánh giá; cuối cùng đánh giá lại agent tốt nhất ở cả hai chế độ no-op starts và human starts.

Tốc độ học (Hình 1, so với DQN/baseline khác bằng median human-normalized score): Rainbow đạt hiệu năng cuối cùng của DQN chỉ sau **7M** khung hình, vượt hiệu năng cuối cùng tốt nhất của mọi baseline khác trong **44M** khung hình, rồi tiếp tục cải thiện vượt xa sau đó.

Bảng 2 (median normalized score cuối cùng, nguyên văn Table 2):

Agent	No-ops	Human starts
DQN	79%	68%
DDQN (*)	117%	110%
Prioritized DDQN (*)	140%	128%
Dueling DDQN (*)	151%	117%
A3C (*)	-	116%
Noisy DQN	118%	102%
Distributional DQN	164%	125%
Rainbow	**223%**	**153%**

(*) = số lấy từ chính bài báo gốc của thành phần đó; các dòng khác là lần chạy lại của tác giả bài Rainbow. DQN ở đây lấy điểm từ bài Dueling Networks vì bài DQN gốc không báo điểm trên đủ 57 game.



Hình 29: Median human-normalized score cuối cùng của các phương pháp trên 57 game Atari, hai chế độ no-op starts và human starts (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi cặp cột là một phương pháp, cột xanh dương là no-op starts và cột xanh lá là human starts; A3C chỉ có số human starts. Rainbow (viền đen) cao nhất ở cả hai chế độ, và mức chênh so với DQN lớn hơn ở no-op starts (223% so với 79%) so với human starts (153% so với 68%).

Tốc độ wall-clock: 7M khung hình (để bằng DQN cuối) tương ứng dưới 10 giờ; chạy đầy 200M khung hình khoảng 10 ngày trên 1 GPU, biến động dưới 20% giữa các biến thể ablation.

Số game vượt các ngưỡng % human (Hình 2, định tính): ở mọi ngưỡng (20%, 50%, 100%, 200%, 500%), Rainbow vượt số lượng game của mọi baseline khác; khoảng cách rõ rệt ở mọi mức hiệu năng, không chỉ ở nhóm game đã "dễ" với baseline.

5. Phân tích / ablation

Bài báo không công bố một bảng số liệu ablation (không có bảng "điểm trung bình khi bỏ từng thành phần"); kết quả ablation chỉ trình bày qua Hình 2 (bottom row), Hình 3 (đường cong median theo thời gian) và Hình 4 (gain/loss từng game, biểu đồ, không có số). Vì vậy bảng dưới đây là **tổng hợp định tính** từ mô tả bằng lời của bài báo, không bịa số liệu:

Thành phần bị bỏ	Ảnh hưởng khi bỏ (theo mô tả định tính của paper)	Bằng chứng
Prioritized replay	Giảm mạnh cả tốc độ học đầu và hiệu năng cuối; một trong hai thành phần quan trọng nhất	Hình 3, "large drop in median performance"
Multi-step	Giảm mạnh tốc độ học đầu **và** hiệu năng cuối (bất ngờ với tác giả vì tưởng chỉ ảnh hưởng tốc độ học)	Hình 3; Rainbow đầy đủ thắng rõ 53/57 game so với bỏ prioritized hoặc bỏ multi-step
Distributional (C51)	Không khác biệt ở ~40M khung hình đầu, nhưng sau đó Rainbow đầy đủ vượt lên; ảnh hưởng tập trung ở game đã đạt mức người hoặc cao hơn	Hình 3 (đường "no distribution" bắt đầu tụt sau 40M), Hình 2 (bottom)
Noisy Nets	Bỏ (dùng ϵ-greedy lại) làm median kém hơn trong tổng; nhưng từng game riêng lẻ: có game giảm mạnh, có game lại tăng nhẹ khi bỏ	Hình 3 (đường đỏ "no noisy" thấp hơn); Hình 4
Dueling	Không thấy khác biệt rõ ở median tổng hợp; nhưng theo game thì có cải thiện ở game đã trên mức người, có giảm nhẹ ở game dưới mức người	Hình 2 (bottom), Hình 4
Double Q-learning	Khác biệt median hạn chế, có game lợi có game hại; so sánh ước lượng cho thấy return thật thường vượt 10 (ngoài support $[-10, 10]$), nên bỏ Double Q ở đây gây ra **underestimate** (không phải overestimate như bối cảnh gốc bài 02), vì support bị chặn "kháng lại" phần nào xu hướng overestimate tự nhiên của max	Mục Discussion, phần phân tích riêng về double Q

Không có số liệu trong bài báo cho: mức giảm % cụ thể của từng ablation (bài báo không nêu số); chi phí tính toán riêng của từng thành phần (chỉ nói chung "biến động dưới 20%").

6. Giới hạn

Tác giả nêu:

- Chỉ xét các biến thể **thuật toán**, không xét kiến trúc song song/phân tán (A3C, Gorila, Evolution Strategies) có thể nhanh hơn về wall-clock nhưng kém hiệu quả dữ liệu hơn; để nghiên cứu tương lai.
- Không gian siêu tham số kết hợp của 6 thành phần quá lớn để tìm kiếm triệt để; chỉ tinh chỉnh hạn chế, bắt đầu từ giá trị gốc của mỗi bài rồi coordinate descent tay cho tham số nhạy nhất.
- Chỉ xét phương pháp **value-based** họ Q-learning; không xét trust-region policy optimization (TRPO, bài 04) hay actor-critic (A3C, bài 06).
- Nhiều cải tiến khác (bootstrapped DQN, episodic control, hierarchical RL, auxiliary tasks, eligibility traces, optimality tightening...) không được tích hợp, liệt kê như hướng mở trong Discussion.
- Dùng các sửa đổi domain chuẩn (reward clipping, action repetition cố định, frame-stacking) như DQN gốc; các cải tiến riêng cho các bước này (ví dụ Pop-Art normalization để bỏ reward clipping) không được thử trong Rainbow.

Nhận xét:

- Thiếu một bảng số liệu ablation rõ ràng là một hạn chế về khả năng tái lập (reproducibility): người đọc chỉ có thể suy ra xu hướng từ hình vẽ, không có số chính xác để so sánh lại (xem mục 5).
- Support $[-10, 10]$ cố định cho phân phối C51 là một giả định mạnh, phụ thuộc vào việc reward đã bị clip $[-1, 1]$ và discount $\gamma = 0.99$; nếu không clip reward hoặc chọn γ khác, cần chọn lại $[v_{\min}, v_{\max}]$ thủ công, bài báo không đưa ra quy tắc chọn tự động.
- $n = 3$ cho multi-step được chọn qua so sánh hạn chế $\{1, 3, 5\}$ trên đúng hỗn hợp 6 thành phần này; không rõ giá trị tối ưu có đổi nếu bớt đi một vài thành phần khác.
- Chỉ test trên Atari (một domain hành động rời rạc, input ảnh); không có bằng chứng các thành phần (đặc biệt C51, dueling) cộng dồn tốt như vậy ở domain khác (continuous control, text).

7. Liên hệ với các bài khác

- **Bài 01 (DQN), bài 02 (Double DQN):** nền tảng trực tiếp; mục 3.1 chính là nội dung bài 02.
- **Bài 04 (TRPO), bài 07 (PPO):** Discussion của Rainbow tự nhắc rằng bài này **không** xét các phương pháp policy-based như TRPO; đây là nhánh khác hẳn mà lộ trình đi tiếp sau Rainbow.
- **Bài 06 (A3C):** Rainbow dùng kết quả A3C làm một baseline so sánh (Bảng 2); Discussion cũng nhắc multi-step của A3C và gợi ý có thể thay advantage estimate của actor-critic bằng GAE (bài 05) để cải thiện thêm, một gợi ý chéo khá thú vị từ chính paper.
- **Bài 13 (RND):** một hướng khám phá khác (intrinsic motivation dựa trên lỗi dự đoán mạng) so với Noisy Nets ở đây; Discussion của Rainbow liệt kê bootstrapped DQN, intrinsic motivation, count-based exploration như các hướng khám phá thay thế chưa tích hợp.
- **Prioritized replay (mục 3.2):** chính là lời giải cho hạn chế mà bài 01 (DQN) tự nêu ("lấy mẫu đều, cần chiến lược phức tạp hơn giống prioritized sweeping") nhưng chưa làm.
- **Ngoài lộ trình RL:** ý tưởng "nhiều cải tiến độc lập, tưởng không liên quan, hóa ra cộng dồn tốt khi kết hợp đúng cách" cũng là một mô-típ lặp lại trong các bài tối ưu hóa huấn luyện LLM (ví dụ các "implementation details" paper như bài 08 và bài 28, N+ Implementation Details of RLHF with PPO).

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp đến huấn luyện RLHF/RLVR cho LLM hiện đại: **thấp đến trung bình** (các thuật toán LLM post-training chủ đạo hiện nay là PPO/GRPO, policy-gradient on-policy, không phải Q-learning off-policy), nhưng tư duy "ablation có hệ thống trước khi kết luận một cải tiến có tác dụng" rất đáng học:

- **Bài học phương pháp luận quan trọng nhất:** đừng cộng các cải tiến "nghe hợp lý" một cách tùy tiện; Rainbow cho thấy dù 6 thành phần đều tốt riêng lẻ, khi kết hợp cần xử lý tương tác (ví dụ priority đổi từ TD-error sang KL loss khi có distributional, dueling phải áp theo atom). Khi thêm nhiều "trick" vào một quy trình huấn luyện RL cho LLM (ví dụ nhiều biến thể GRPO/DAPO/GSPO ở các bài 36, 41, 43

của lộ trình), nên kiểm tra tương tác giữa các trick bằng ablation riêng, không giả định chúng độc lập cộng dồn.

- **Multi-step return là ý tưởng tái diễn:** cùng động lực "one-step lan truyền phần thưởng chậm" xuất hiện lại trong advantage estimate nhiều bước/GAE (bài 05) dùng cho PPO/GRPO huấn luyện LLM; ablation của Rainbow (multi-step là 1 trong 2 thành phần quan trọng nhất) gợi ý rằng với RL cho LLM, số bước/độ dài horizon dùng để ước lượng advantage cũng là một siêu tham số đáng tinh chỉnh kỹ, không nên mặc định.
- **Distributional RL ít liên quan trực tiếp** tới các quy trình RLHF/RLVR hiện tại cho LLM (hầu hết vẫn học kỳ vọng reward/advantage, không học toàn phân phối), nhưng ý tưởng "học cả phân phối return, không chỉ kỳ vọng" có thể liên hệ xa tới các phương pháp ước lượng độ bất định của mô hình phần thưởng hoặc reward hacking detection, nếu muốn mở rộng nghiên cứu.

9. Thí nghiệm gợi ý

1. Trên một bộ game Atari nhỏ (3-5 game), tái hiện ablation "bỏ một thành phần" cho ít nhất 3 thành phần (ví dụ bỏ multi-step, bỏ prioritized, bỏ noisy), đo median human-normalized score, so với mô tả định tính ở mục 5 để tự kiểm chứng xu hướng (không kỳ vọng khớp số chính xác, chỉ kiểm tra thứ tự ảnh hưởng).
2. Thử thay đổi $[v_{\text{min}}, v_{\text{max}}]$ của C51 (ví dụ $[-5, 5]$ và $[-20, 20]$) trên một game mà return thật thường vượt 10, đo xem hiệu ứng "underestimate do double Q-learning cộng support bị chặn" (mục 5) có đổi dấu hay không khi mở rộng support.
3. So sánh priority dựa trên TD-error tuyệt đối (kiểu DQN gốc) với priority dựa trên KL loss (kiểu Rainbow) trên một môi trường có reward ngẫu nhiên (stochastic reward, ví dụ thêm nhiễu Gaussian vào reward), kiểm tra lập luận "KL loss bền hơn với stochastic environment" của bài báo.
4. Quét $n \in \{1, 3, 5, 7\}$ cho multi-step trên đúng tổ hợp đầy đủ 6 thành phần, xem $n = 3$ có còn tối ưu không khi test trên bộ game khác 57 game chuẩn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Double Q target	$R_{t+1} + \gamma q_\theta(S_{t+1}, \arg\max_{a'} q_\theta(S_{t+1}, a'))$	bài 02
Priority (proportional)	$p_t \propto \lvert\text{TD-error}\rvert^\omega$ (Rainbow: $\propto D_{KL}^\omega$)	$\omega = 0.5$
Dueling	$q = v_\eta + a_\psi - \frac{1}{N_a}\sum_{a'} a_\psi(\cdot, a')$	giải unidentifiability
n-step return	$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}$	$n = 3$ tốt nhất
Lưới atom	$z^i = v_{\text{min}} + (i-1)\frac{v_{\text{max}} - v_{\text{min}}}{N_{\text{atoms}} - 1}$	51 atom, $[-10, 10]$, bước 0.4 (tự tính)
Loss distributional	$D_{KL}(\Phi_z d_t' \Vert d_t)$	Φ_z: paper không nêu công thức
Noisy linear	$y = (b + Wx) + (b_n \odot \varepsilon^b + (W_n \odot \varepsilon^w)x)$	thay ϵ-greedy
Median no-ops	Rainbow 223% vs DQN 79%	Bảng 2
Median human starts	Rainbow 153% vs DQN 68%	Bảng 2
2 thành phần quan trọng nhất	Prioritized replay, multi-step	theo ablation định tính (mục 5)

11. Kết luận ngắn

Rainbow cho thấy 6 cải tiến độc lập của DQN (Double Q-learning, prioritized replay, dueling, multi-step, distributional C51, noisy nets), khi kết hợp đúng cách (xử lý các điểm tương tác như priority theo KL loss, dueling theo atom), **cộng dồn tốt** chứ không triệt tiêu nhau, cho kết quả state-of-the-art trên 57 game Atari cả về tốc độ học và điểm cuối. Ablation (chỉ định tính, không có bảng số) cho thấy prioritized replay và

multi-step quan trọng nhất, distributional giúp chủ yếu ở các game đã gần/trên mức người, còn dueling và double Q-learning có ảnh hưởng nhỏ và không đồng nhất giữa các game.

Link gốc: https://arxiv.org/abs/1710.02298

Bài 04

Trust Region Policy Optimization (2015)

Tác giả: John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, Pieter Abbeel (UC Berkeley, Electrical Engineering and Computer Sciences).
Venue / arXiv: arXiv:1502.05477 (v5, 20/04/2017); venue: Proceedings of the 31st International Conference on Machine Learning (ICML), Lille, France, 2015 (ghi ở chân trang bài).
Nguồn: https://arxiv.org/abs/1502.05477
Chủ đề: một thuật toán cập nhật policy có **đảm bảo lý thuyết về cải thiện đơn điệu** (monotonic improvement), bằng cách giới hạn mỗi bước cập nhật trong một "vùng tin cậy" (trust region) đo bằng KL divergence giữa policy cũ và mới, thay vì chọn bước đi theo gradient với một learning rate cố định.
Vị trí trong lộ trình: đọc sau bài 00 (lời nói đầu: MDP, Bellman, policy gradient theorem, REINFORCE, baseline, GAE, importance sampling, trust region, PPO, KL penalty, GRPO, RLHF, RLVR, DPO). TRPO là nền lý thuyết cho nhóm trust-region/clip: bài 05 (GAE, dùng cơ chế trust-region của TRPO cho cả policy và value function), bài 07 (PPO, đơn giản hóa TRPO bằng surrogate clip), bài 08 (Implementation matters), bài 22 (MAPPO), và nhóm RLHF/RLVR (23-44) kế thừa trực giác "không đi quá xa khỏi policy cũ".
Chương nền liên quan: nền 03: Policy gradient và baseline, nền 05: Vùng tin cậy và PPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Paper chia các phương pháp tối ưu policy thành ba nhóm: (1) policy iteration (xen kẽ ước lượng value function và cải thiện policy); (2) policy gradient (ước lượng gradient của tổng thưởng kỳ vọng từ trajectory mẫu); (3) derivative-free (CEM, CMA, coi tổng thưởng là hộp đen, dễ dùng nhưng sample complexity kém khi số tham số lớn).

Với policy gradient thông thường, vấn đề là **chọn kích thước bước**: bước quá nhỏ học chậm, bước quá lớn có thể làm policy sụp đổ (performance giảm mạnh, không hồi phục), vì gradient chỉ là xấp xỉ cục bộ còn hàm mục tiêu thật không nhất thiết tuyến tính theo tham số.

Khung MDP: $(S, A, P, r, \rho_0, \gamma)$ với S, A là tập trạng thái/hành động, P là phân phối chuyển trạng thái, r là hàm thưởng, ρ_0 là phân phối trạng thái khởi đầu, $\gamma \in (0, 1)$ là hệ số chiết khấu. Policy π ngẫu nhiên; $\eta(\pi) = \mathbb{E}_{s_0, a_0, \ldots}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t)\right]$ là tổng thưởng chiết khấu kỳ vọng. Định nghĩa chuẩn:

$$Q_\pi(s_t, a_t) = \mathbb{E}\left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l})\right], \quad V_\pi(s_t) = \mathbb{E}\left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l})\right], \quad A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$$

(Q_π lấy kỳ vọng theo hành động bắt đầu từ a_t cho sẵn, V_π lấy kỳ vọng cả hành động đầu). A_π là **advantage**: hành động a tốt hơn hay kém hơn "mức trung bình" của policy tại trạng thái s.

Khái niệm nền: **trust region** là vùng quanh tham số hiện tại nơi xấp xỉ tuyến tính/bậc hai của hàm mục tiêu còn đáng tin, thay vì chọn bước cố định ta giải bài toán tối ưu có ràng buộc "không rời vùng này". **KL divergence** $D_{KL}(p\|q)$ và **total variation** $D_{TV}(p\|q) = \frac{1}{2}\sum_i |p_i - q_i|$ là hai cách đo khoảng cách giữa hai phân phối. **Surrogate objective** là hàm mục tiêu dễ tối ưu hơn, xấp xỉ η thật tại điểm hiện tại. **Conservative Policy Iteration (CPI, Kakade & Langford 2002)** là tiền nhiệm của TRPO, chỉ chứng minh được cận cải thiện cho policy hỗn hợp $\pi_{new} = (1 - \alpha)\pi_{old} + \alpha\pi'$, một lớp quá hẹp để dùng thực tế. **Conjugate gradient**

(CG), Fisher Information Matrix (FIM), natural policy gradient là công cụ số học để giải gần đúng bài toán trust-region không cần ma trận Hessian đầy đủ.

2. Ý tưởng chính

Thay vì tối ưu trực tiếp $\eta(\tilde{\pi})$ (phụ thuộc phức tạp vào phân phối trạng thái của chính $\tilde{\pi}$, chưa biết), TRPO tối ưu một **surrogate** $L_\pi(\tilde{\pi})$ khớp η tới bậc một tại π. Vì sai số giữa L_π và η thật có thể **chặn trên** theo một hàm của khoảng cách giữa π và $\tilde{\pi}$ (đo bằng KL divergence), nếu ta trừ đi một hình phạt tỉ lệ với khoảng cách đó, việc tối đa hóa $L_\pi(\tilde{\pi}) - C \cdot D_{KL}(\pi, \tilde{\pi})$ đảm bảo η không giảm. Hệ số lý thuyết C quá lớn khiến bước đi quá nhỏ trong thực tế; TRPO thay hình phạt bằng **ràng buộc cứng** $D_{KL} \le \delta$ (trust region), rồi giải gần đúng bài toán ràng buộc này bằng xấp xỉ tuyến tính/bậc hai cộng với conjugate gradient và line search. Kết quả là một thuật toán học được các policy mạng neural cỡ hàng chục nghìn tham số cho cả robot mô phỏng và Atari, với rất ít điều chỉnh siêu tham số.

3. Phương pháp chi tiết

3.1 Đẳng thức hiệu suất (performance difference identity)

Công cụ trung tâm là một đẳng thức (Kakade & Langford 2002, chứng minh lại ở Appendix A, Lemma 1) biểu diễn $\eta(\tilde{\pi})$ theo advantage của π:

$$\boxed{\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{s_0, a_0, \ldots \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_\pi(s_t, a_t) \right]}$$

Tự suy lại (Lemma 1): dùng $A_\pi(s,a) = \mathbb{E}_{s'}[r(s) + \gamma V_\pi(s') - V_\pi(s)]$, lấy kỳ vọng theo trajectory sinh bởi $\tilde{\pi}$:

$$\mathbb{E}_{\tau \sim \tilde{\pi}} \Big[\sum_t \gamma^t A_\pi(s_t, a_t) \Big] = \mathbb{E}_{\tau \sim \tilde{\pi}} \Big[\sum_t \gamma^t (r(s_t) + \gamma V_\pi(s_{t+1}) - V_\pi(s_t)) \Big]$$

Số hạng $\sum_t \gamma^t (\gamma V_\pi(s_{t+1}) - V_\pi(s_t))$ là **tổng kính lồng (telescoping)**: $\gamma^{t+1} V_\pi(s_{t+1})$ ở số hạng t triệt tiêu với $-\gamma^{t+1} V_\pi(s_{t+1})$ ở số hạng $t+1$, chỉ còn $-V_\pi(s_0)$. Vậy biểu thức trên bằng $-\mathbb{E}[V_\pi(s_0)] + \mathbb{E}_{\tau \sim \tilde{\pi}}[\sum_t \gamma^t r(s_t)] = -\eta(\pi) + \eta(\tilde{\pi})$ (vì $s_0 \sim \rho_0$ như nhau ở cả hai policy), sắp xếp lại ra đúng đẳng thức trên.

Viết lại tổng theo trạng thái thay vì theo bước thời gian, dùng $\rho_{\tilde{\pi}}(s) = \sum_t \gamma^t P(s_t = s | \tilde{\pi})$ (tần suất ghé thăm chiết khấu, chưa chuẩn hóa):

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a|s) A_\pi(s,a) \tag{2}$$

Hệ quả trực tiếp: nếu $\sum_a \tilde{\pi}(a|s) A_\pi(s,a) \ge 0$ tại **mọi** s thì $\eta(\tilde{\pi}) \ge \eta(\pi)$ (chính là policy iteration kinh điển với $\tilde{\pi}(s) = \arg\max_a A_\pi(s,a)$). Nhưng công thức (2) khó tối ưu trực tiếp vì $\rho_{\tilde{\pi}}$ phụ thuộc phức tạp vào $\tilde{\pi}$ (chính là policy ta đang muốn tìm).

3.2 Surrogate L_π và khớp bậc một

Định nghĩa surrogate dùng ρ_π (của policy **cũ**, đã biết) thay cho $\rho_{\tilde{\pi}}$:

$$L_\pi(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a|s) A_\pi(s,a) \tag{3}$$

Với policy tham số hóa π_θ, $L_{\pi_{\theta_0}}$ khớp η tới bậc một tại θ_0:

$$L_{\pi_{\theta_0}}(\pi_{\theta_0}) = \eta(\pi_{\theta_0}), \qquad \nabla_\theta L_{\pi_{\theta_0}}(\pi_\theta)|_{\theta=\theta_0} = \nabla_\theta \eta(\pi_\theta)|_{\theta=\theta_0} \tag{4}$$

Nghĩa là một bước đủ nhỏ cải thiện L cũng cải thiện η, nhưng (4) không nói bước đó nên lớn bao nhiêu.

3.3 Từ CPI đến Theorem 1 (cận tổng quát cho mọi policy ngẫu nhiên)

Kakade & Langford chỉ xét policy hỗn hợp $\pi_{new} = (1-\alpha)\pi_{old} + \alpha\pi'$, chứng minh:

$$\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{2\gamma\varepsilon}{(1-\gamma)^2}\alpha^2, \quad \varepsilon = \max_s \mathbb{E}_{a\sim\pi'(a|s)}[A_\pi(s,a)] \tag{6}$$

Đóng góp chính của TRPO: thay α bằng $D_{TV}^{max}(\pi,\tilde{\pi}) = \max_s D_{TV}(\pi(\cdot|s)\|\tilde{\pi}(\cdot|s))$, áp dụng cho **mọi** policy ngẫu nhiên:

Theorem 1. Đặt $\alpha = D_{TV}^{max}(\pi_{old}, \pi_{new})$. Khi đó

$$\boxed{\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\gamma\varepsilon}{(1-\gamma)^2}\alpha^2, \qquad \varepsilon = \max_{s,a}|A_\pi(s,a)|} \tag{8}$$

3.4 Tự suy lại các bước chứng minh Theorem 1 (Appendix A)

Đặt $\bar{A}(s) = \mathbb{E}_{a\sim\tilde{\pi}(\cdot|s)}[A_\pi(s,a)]$ (advantage kỳ vọng của $\tilde{\pi}$ so với π tại s). Từ Lemma 1 và từ (3):

$$\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{\tau\sim\tilde{\pi}}\Big[\sum_t \gamma^t \bar{A}(s_t)\Big], \qquad L_\pi(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{\tau\sim\pi}\Big[\sum_t \gamma^t \bar{A}(s_t)\Big]$$

Hai biểu thức khác nhau ở **việc trạng thái s_t do $\tilde{\pi}$ hay π sinh ra**. Để chặn sai số này, dùng α**-coupling**: ghép $\pi, \tilde{\pi}$ thành phân phối chung $(a,\tilde{a})|s$ sao cho $P(a \neq \tilde{a}|s) \leq \alpha$ với mọi s (tính toán: cùng seed ngẫu nhiên, hai policy cho cùng hành động với xác suất $\geq 1-\alpha$).

Lemma 2 ($|\bar{A}(s)| \leq 2\alpha\varepsilon$): vì $\mathbb{E}_{a\sim\pi}[A_\pi(s,a)] = 0$,

$$\bar{A}(s) = \mathbb{E}_{\tilde{a}\sim\tilde{\pi}}[A_\pi(s,\tilde{a})] = \mathbb{E}_{(a,\tilde{a})}[A_\pi(s,\tilde{a}) - A_\pi(s,a)] = P(a\neq\tilde{a}|s)\cdot\mathbb{E}_{(a,\tilde{a})|a\neq\tilde{a}}[A_\pi(s,\tilde{a}) - A_\pi(s,a)] \;\Rightarrow\; |\bar{A}(s)| \leq \alpha\cdot 2\varepsilon$$

Lemma 3 (chặn hiệu $\bar{A}(s_t)$ dưới $\tilde{\pi}$ và dưới π ở cùng bước t): ghép hai trajectory bằng cùng seed, gọi n_t là số lần $\pi, \tilde{\pi}$ lệch trước bước t. Khi $n_t = 0$, phần kỳ vọng của $\bar{A}(s_t)$ dưới $\tilde{\pi}$ và dưới π **bằng nhau** (lịch sử giống nhau), hiệu số chỉ còn phần $n_t > 0$, với $P(n_t > 0) \leq 1-(1-\alpha)^t$ (vì P(khớp một bước) $\geq 1-\alpha$). Dùng bất đẳng thức tam giác rồi chặn mỗi kỳ vọng bằng $2\alpha\varepsilon$ (Lemma 2, tổng $4\alpha\varepsilon$; *paper trích "Lemma 3" ở bước này nhưng nội dung đang dùng chính là bất đẳng thức của Lemma 2, người ghi chú giữ trích dẫn gốc và ghi chú thêm*):

$$|\mathbb{E}_{s_t\sim\tilde{\pi}}[\bar{A}(s_t)] - \mathbb{E}_{s_t\sim\pi}[\bar{A}(s_t)]| \leq 4\alpha(1-(1-\alpha)^t)\varepsilon \tag{40}$$

Cộng theo thời gian với trọng số γ^t (bước chốt, tự tính lại):

$$|\eta(\tilde{\pi}) - L_\pi(\tilde{\pi})| \leq \sum_{t=0}^{\infty}\gamma^t\cdot 4\alpha(1-(1-\alpha)^t)\varepsilon = 4\alpha\varepsilon\left[\frac{1}{1-\gamma} - \frac{1}{1-\gamma(1-\alpha)}\right]$$

Quy đồng: $\frac{1}{1-\gamma} - \frac{1}{1-\gamma(1-\alpha)} = \frac{(1-\gamma(1-\alpha)) - (1-\gamma)}{(1-\gamma)(1-\gamma(1-\alpha))} = \frac{\gamma\alpha}{(1-\gamma)(1-\gamma(1-\alpha))}$. Vậy

$$|\eta(\tilde{\pi}) - L_\pi(\tilde{\pi})| \le \frac{4\alpha^2\varepsilon\gamma}{(1-\gamma)(1-\gamma(1-\alpha))} \le \frac{4\alpha^2\varepsilon\gamma}{(1-\gamma)^2}$$

(bất đẳng thức cuối vì $1-\gamma(1-\alpha) \ge 1-\gamma$ khi $\alpha \ge 0$). Đây chính xác là Theorem 1.

Từ TV sang KL: dùng $D_{TV}(p\|q)^2 \le D_{KL}(p\|q)$ (Pollard 2000), đặt $D_{KL}^{max}(\pi,\tilde{\pi}) = \max_s D_{KL}(\pi(\cdot|s)\|\tilde{\pi}(\cdot|s))$, suy ra bản KL của cận:

$$\boxed{\eta(\tilde{\pi}) \ge L_\pi(\tilde{\pi}) - C\, D_{KL}^{max}(\pi,\tilde{\pi}), \qquad C = \frac{4\gamma\varepsilon}{(1-\gamma)^2}} \tag{9}$$

3.5 Thuật toán lý thuyết (Algorithm 1) và tính đơn điệu

Với $M_i(\pi) := L_{\pi_i}(\pi) - CD_{KL}^{max}(\pi_i,\pi)$, lặp $\pi_{i+1} = \arg\max_\pi M_i(\pi)$. Vì $\eta(\pi_{i+1}) \ge M_i(\pi_{i+1})$ (bất đẳng thức (9)) và $\eta(\pi_i) = M_i(\pi_i)$ (đẳng thức tại điểm hiện tại), nên $\eta(\pi_{i+1}) - \eta(\pi_i) \ge M_i(\pi_{i+1}) - M_i(\pi_i) \ge 0$ (vì π_{i+1} tối đa hóa M_i). Đây là một thuật toán kiểu **minorization-maximization (MM)**: M_i là hàm "ở dưới" η và chạm η tại π_i, tối đa hóa M_i luôn không làm η giảm.

3.6 Chuỗi xấp xỉ: từ lý thuyết đến thuật toán thực hành

Bước	Vấn đề	Xấp xỉ được dùng
1	Hệ số phạt $C = 4\gamma\varepsilon/(1-\gamma)^2$ lý thuyết quá lớn, bước đi quá nhỏ	**Ràng buộc cứng** $D_{KL} \le \delta$ thay hình phạt
2	Ràng buộc D_{KL}^{max} trên mọi trạng thái là vô số ràng buộc	**KL trung bình** $\bar{D}_{KL}^{\rho}(\theta_1,\theta_2) = \mathbb{E}_{s\sim\rho}[D_{KL}(\pi_{\theta_1}(\cdot\|s)\|\pi_{\theta_2}(\cdot\|s))]$
3	ρ_π, A_π không biết chính xác	Mẫu Monte Carlo, $A_\pi \to Q_\pi$ (đổi mục tiêu một hằng số)
4	Giải trực tiếp bài toán phi tuyến ràng buộc tốn kém	Xấp xỉ **tuyến tính** mục tiêu, **bậc hai** ràng buộc, giải bằng CG + line search
5	Sai số ước lượng advantage	Bỏ qua (Kakade & Langford gốc có xử lý, TRPO bỏ "để đơn giản")

Bài toán ràng buộc cứng cuối để cập nhật θ:

$$\text{maximize}_\theta\ L_{\theta_{old}}(\theta) \quad \text{subject to } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old},\theta) \le \delta \tag{12}$$

3.7 Ước lượng mục tiêu bằng mẫu (sample-based)

Khai triển $L_{\theta_{old}}$ trong (12): thay $\sum_s \rho_{\theta_{old}}(s)[\ldots]$ bằng $\frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho_{\theta_{old}}}[\ldots]$; thay $A_{\theta_{old}}$ bằng $Q_{\theta_{old}}$ (đổi mục tiêu một hằng số, không đổi nghiệm); thay $\sum_a \pi_\theta(a|s)$ bằng **importance sampling** từ phân phối lấy mẫu q:

$$\sum_a \pi_\theta(a|s_n) A_{\theta_{old}}(s_n,a) = \mathbb{E}_{a\sim q}\left[\frac{\pi_\theta(a|s_n)}{q(a|s_n)} A_{\theta_{old}}(s_n,a)\right]$$

cho ra bài toán tương đương:

$$\text{maximize}_\theta \ \mathbb{E}_{s\sim\rho_{\theta_{old}},a\sim q}\left[\frac{\pi_\theta(a|s)}{q(a|s)}Q_{\theta_{old}}(s,a)\right] \quad \text{subject to } \mathbb{E}_{s\sim\rho_{\theta_{old}}}[D_{KL}(\pi_{\theta_{old}}(\cdot|s)\|\pi_\theta(\cdot|s))] \le \delta \qquad (14)$$

Tỉ số $\pi_\theta(a|s)/q(a|s)$ ở đây chính là tiền thân trực tiếp của tỉ số $r_t(\theta)$ mà PPO (bài 07) sẽ dùng.

3.8 Hai cách lấy mẫu: single path và vine

- **Single path:** lấy mẫu $s_0 \sim \rho_0$, chạy $\pi_{\theta_{old}}$ sinh một trajectory đầy đủ; $q = \pi_{\theta_{old}}$; $Q_{\theta_{old}}$ ước lượng bằng tổng thưởng chiết khấu dọc trajectory. Không cần reset môi trường, dùng được trên robot thật.
- **Vine:** sinh trajectory "thân" (trunk), chọn một **rollout set** N trạng thái, tại mỗi trạng thái thử K hành động $a_{n,k} \sim q(\cdot|s_n)$ và rollout ngắn, dùng **common random numbers (CRN)** để giảm phương sai khi so sánh các hành động cùng trạng thái. Cần môi trường reset được (chỉ khả thi trong mô phỏng).

Vine cho ước lượng advantage **phương sai thấp hơn** nhưng tốn nhiều lệnh gọi simulator hơn single path.

3.9 Giải bài toán trust-region bằng CG + line search (Appendix C)

Xấp xỉ bậc hai cho ràng buộc: $D_{KL}(\theta_{old},\theta) \approx \frac{1}{2}(\theta-\theta_{old})^T A(\theta-\theta_{old})$, A là **Fisher Information Matrix (FIM)**, $A_{ij} = \partial^2_{\theta_i\theta_j} D_{KL}(\theta_{old},\theta)$; mục tiêu xấp xỉ tuyến tính với gradient $g = \nabla_\theta L_{\theta_{old}}(\theta)|_{\theta_{old}}$.

1. **Hướng đi:** giải gần đúng $As = g$ bằng **conjugate gradient**, không tạo ma trận A đầy đủ, chỉ cần Fisher-vector product (FVP) $y \mapsto Ay = J^T M J y$ (J là Jacobian tham số phân phối theo θ, M là Hessian của KL theo tham số trung gian). $k = 10$ vòng CG đủ dùng; vì FIM chỉ là một thước đo, tính FVP trên **10% dữ liệu** vẫn đủ chính xác, nên chi phí CG xấp xỉ chi phí một gradient thường.
2. **Độ dài bước:** từ $\delta \approx \frac{1}{2}\beta^2 s^T A s$ suy ra $\beta = \sqrt{2\delta/(s^T As)}$ ($s^T As$ là sản phẩm phụ của CG).
3. **Line search:** backtracking, thu nhỏ β theo cấp số nhân tới khi mục tiêu phi tuyến $L_{\theta_{old}}(\theta) - \not\Vdash[D_{KL}(\theta_{old},\theta) > \delta]\cdot\infty$ cải thiện; không có bước này, bước đi đôi khi quá lớn, gây suy giảm thảm khốc.

3.10 Thuật toán tổng quát (đơn giản hóa)

```
Input: policy theta_0
repeat each iteration:
    1. Thu thap state-action pairs + uoc luong Q (single path hoac vine)
    2. Trung binh mau -> uoc luong objective va constraint cua (14)
    3. g = gradient cua objective tai theta_old
       s ~= A^{-1} g          (conjugate gradient, FVP tren 10% du lieu, k=10 vong)
       beta = sqrt(2*delta / (s^T A s))
       theta_candidate = theta_old + beta * s
    4. Line search: giam beta theo cap so nhan cho den khi
       L_theta_old(theta_candidate) tang VA D_KL(theta_old, theta_candidate) <= delta
    5. theta_old <- theta_candidate
```

Cách đọc hình: sáu khối đi theo chiều kim đồng hồ từ thu mẫu đến cập nhật tham số. Hai khối bên phải là phần nặng tính toán: CG tìm hướng $s \approx A^{-1}g$ chỉ bằng tích Fisher-vector, rồi β đặt bước đúng bằng ngưỡng δ trên xấp xỉ bậc hai. Line search ở khối 5 là chốt chặn cuối: nếu bước làm L không tăng hoặc KL vượt δ thì thu nhỏ lại.

3.11 Liên hệ với natural gradient, vanilla PG, policy iteration (Section 7)

Phương pháp	Mục tiêu (xấp xỉ)	Ràng buộc/bước
Natural policy gradient (Kakade 2002)	tuyến tính $\nabla_\theta L_{\theta_{old}}\cdot(\theta-\theta_{old})$	$\frac{1}{2}(\theta-\theta_{old})^T A(\theta-\theta_{old}) \le \delta$, nhưng dùng **hệ số bước cố định** λ_1: $\theta_{new} = \theta_{old} + \lambda_1 A^{-1} g$

Phương pháp	Mục tiêu (xấp xỉ)	Ràng buộc/bước
Vanilla policy gradient	tuyến tính như trên	ràng buộc ℓ_2: $\frac{1}{2}\|\theta - \theta_{old}\|^2 \le \delta$
Policy iteration	$\max_\pi L_{\pi_{old}}(\pi)$ không ràng buộc	không có
TRPO	tuyến tính như trên	**ràng buộc** $D_{KL} \le \delta$ **áp tại mỗi bước** (không cố định λ_1)

Khác biệt tưởng nhỏ (ràng buộc động so với hệ số bước cố định của natural gradient) nhưng thực nghiệm cho thấy nó cải thiện hiệu suất rõ rệt trên các bài toán lớn.

4. Thí nghiệm và kết quả chính

Robot mô phỏng (MuJoCo), $\delta = 0.01$ **cho mọi thí nghiệm:** swimmer (10 chiều trạng thái, thưởng $r = v_x - 10^{-5}\|u\|^2$), hopper (12 chiều, thêm +1 khi chưa ngã), walker (trạng thái 18 chiều theo văn bản §8.1, **nhưng Table 2 ghi 20**, hai số mâu thuẫn trong bài báo, ở đây nêu cả hai). Policy: MLP Gaussian (các lớp fully-connected, hidden 30/50/50 cho swimmer/hopper/walker; bài báo không nêu hàm kích hoạt cụ thể cho mạng này). So sánh: single path TRPO, vine TRPO, CEM, CMA, natural gradient (hệ số phạt cố định, dò learning rate theo cấp số 3), empirical FIM (ước lượng FIM bằng hiệp phương sai gradient), max KL (chỉ chạy được trên cart-pole).

Kết quả (Figure 4, trung bình 5 lần chạy): single path và vine TRPO giải được **tất cả** bài toán, tốt nhất. Natural gradient học tốt ở 2 bài dễ nhưng **không học được** gait nhảy/đi (hopper, walker) dù đã dò learning rate. CEM/CMA kém dần khi số tham số tăng. Max KL trên cart-pole học chậm hơn mean KL (ràng buộc chặt hơn) nhưng xác nhận xấp xỉ mean KL "hợp lý".

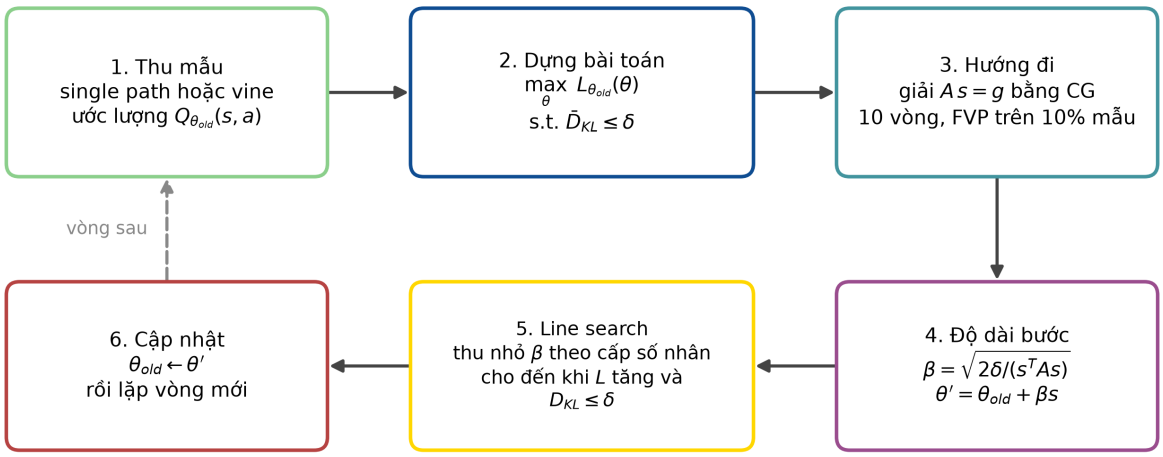
Atari (ảnh màn hình thô), CNN (2 lớp conv 16 kênh, stride 2, kernel 4x4, 1 fully-connected 20 unit, tổng 33500 tham số), $\delta = 0.01$, $\gamma = 0.99$, 500 vòng lặp, ≈30 giờ trên máy 16 lõi.

Game	Random	Human	DQN	UCC-I	TRPO single path	TRPO vine
Beam Rider	354	7456	4092	5702	1425.2	859.5
Breakout	1.2	31.0	168.0	380	10.8	34.2
Enduro	0	368	470	741	534.6	430.8
Pong	-20.4	-3.0	20.0	21	20.9	20.9
Q*bert	157	18900	1952	20025	1973.5	7732.5
Seaquest	110	28010	1705	2995	1908.6	788.4
S. Invaders	179	3690	581	692	568.4	450.2

Cách đọc hình: mỗi game có hai thanh, xanh dương là single path và xanh lá là vine, con số ghi ngay cạnh thanh. Vine thắng rõ ở Q*bert (7732,5 so với 1973,5) và Breakout, nhưng thua ở Beam Rider, Enduro, Seaquest và S. Invaders, còn Pong hòa; vì chỉ chạy một lần mỗi game nên không có khoảng sai số để đánh giá độ chắc chắn.

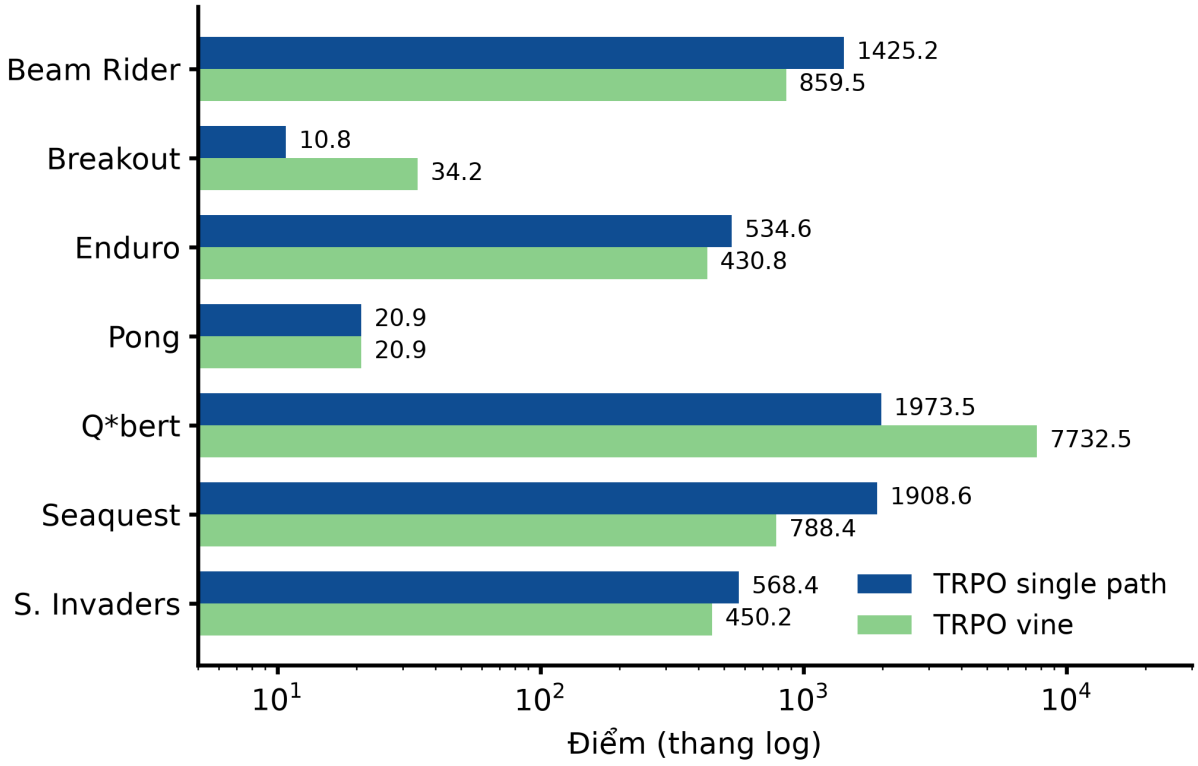
TRPO chỉ vượt các phương pháp trước trên một số game, nhưng luôn đạt điểm hợp lý trên cả 7 game dù không thiết kế riêng cho Atari. Mỗi game chỉ chạy 1 lần, bài báo nêu "không thể lấy thống kê sai số do giới hạn thời gian".

Bảng 2 (chi tiết robot, trích):



$g = \nabla_\theta L$ là gradient mục tiêu, A là ma trận Fisher (đạo hàm bậc hai của KL).
CG chỉ cần tích Fisher-vector, không dựng A đầy đủ; line search chặn các bước quá lớn.

Hình 30: Một vòng lặp TRPO: thu mẫu, dựng bài toán ràng buộc KL, hướng đi bằng conjugate gradient, độ dài bước và line search (minh hoạ, tự tính)



Hình 31: Điểm Atari của TRPO single path và vine trên 7 game, thang log, mỗi game một lần chạy (vẽ lại từ số liệu trong ghi chú)

	Swimmer	Hopper	Walker
Số chiều trạng thái	10	12	20 (Table 2; văn bản §8.1 ghi 18)
Số chiều điều khiển	2	3	6
Tổng số tham số policy	364	4806	8206
Bước mô phỏng / vòng lặp	50K	1M	1M
Số vòng lặp policy	200	200	200
δ (stepsize KL)	0.01	0.01	0.01
Vine: thời gian tính (phút)	2	14	40
Single path: thời gian tính (đơn vị không ghi rõ, cùng cột với vine nên suy ra là phút)	5	35	100

Bảng 3 (Atari): tổng tham số 33500; vine 400K bước mô phỏng/vòng, single path 100K; 500 vòng lặp policy; $\delta = 0.01$, $\gamma = 0.99$; thời gian tính ≈30 giờ cho cả hai biến thể.

5. Phân tích / ablation

- **Natural gradient vs TRPO:** chỉ khác ở hệ số phạt cố định so với ràng buộc KL cứng; khoảng cách hiệu suất lớn trên hopper/walker cho thấy ràng buộc động quan trọng, không chỉ là chi tiết kỹ thuật nhỏ.
- **Empirical FIM vs FIM giải tích:** tốc độ cải thiện policy tương tự nhau.
- **Max KL vs mean KL (chỉ cart-pole):** mean KL học nhanh hơn, cả hai đều hội tụ, xác nhận xấp xỉ trung bình chấp nhận được.
- **Vine vs single path trên Atari:** vine thắng rõ trên Q*bert (7732.5 so với 1973.5) nhưng thua trên Beam Rider, Seaquest, S. Invaders, đúng với trade-off lý thuyết (phương sai thấp hơn nhưng tốn mẫu hơn, hiệu quả tùy game).

6. Giới hạn

Tác giả nêu: lý thuyết giả định biết chính xác A_π (bỏ qua sai số ước lượng mà Kakade & Langford gốc có xử lý); ràng buộc D_{KL}^{max} không khả thi số học nên dùng KL trung bình, chỉ là "xấp xỉ heuristic"; vine cần môi trường reset được; lớp policy hỗn hợp của CPI gốc "cồng kềnh và hạn chế" (động lực viết bài này).

Nhận xét: kết quả Atari chỉ chạy 1 lần/game, không có thống kê sai số; chuỗi 5 bước xấp xỉ ở §3.6 khá dài nên TRPO không còn giữ đảm bảo monotonic improvement chặt như Theorem 1 gốc; chi phí mỗi bước (CG, FVP, line search với rollback) nặng hơn nhiều policy gradient thường, động lực trực tiếp cho PPO (bài 07); mâu thuẫn số liệu walker 18 vs 20 chiều (văn bản vs Table 2) cần đọc kỹ cả hai.

7. Liên hệ với các bài khác

- **Bài 00:** đã giới thiệu trust region, KL penalty, PPO, GRPO; bài này chứng minh chặt các khái niệm đó lần đầu.
- **Bài 05 (GAE):** cùng tác giả, dùng đúng cơ chế trust-region của TRPO cho policy và mở rộng sang khớp value function.
- **Bài 06 (A3C):** hướng thay thế đồng thời (actor-critic bất đồng bộ, không trust region), đặt ra câu hỏi mà PPO trả lời.
- **Bài 07 (PPO):** thừa kế surrogate $L_{\theta_{old}}$ và ràng buộc KL, thay CG + line search bằng mục tiêu clip bậc một.
- **Bài 08, 09:** khảo sát thực nghiệm các lựa chọn code quyết định hiệu suất TRPO/PPO.
- **Bài 22 (MAPPO), 28, 36 (GRPO):** kế thừa trực giác "không đi quá xa policy cũ" qua clip/KL penalty tường minh.

- **llm 07 (InstructGPT):** quy trình RLHF SFT → mô hình phần thưởng → PPO, hậu duệ trực tiếp của TRPO.

8. Ứng dụng thực tế (suy luận)

TRPO hiếm khi dùng trực tiếp cho RLHF/RLVR LLM/VLM ngày nay: chi phí CG + Fisher-vector product khó kết hợp với Adam, chia sẻ tham số policy/value ở quy mô tỷ tham số. Nhưng hiểu TRPO vẫn quan trọng:

- Cận monotonic-improvement là lý do lý thuyết cho trực giác "giới hạn độ lệch mỗi bước", tái xuất hiện ở clip của PPO (bài 07), hình phạt KL so với policy tham chiếu trong RLHF/RLVR (ví dụ DeepSeek-R1, bài 39), và clip của GRPO (bài 36)/DAPO (bài 41).
- Natural-gradient/Fisher preconditioning phần lớn bị bỏ ở quy mô LLM (Adam + clip đủ tốt) nhưng giải thích vì sao PPO/GRPO hay nhắc "trust region" dù không giải ràng buộc KL cứng nào.

9. Thí nghiệm gợi ý

1. Trên CartPole, so sánh vanilla policy gradient (nhiều learning rate) với TRPO (CG + line search) để quan sát trực tiếp policy sụp đổ khi không có trust region.
2. Trên một MDP dạng bảng có advantage thật đã biết, so sánh phương sai ước lượng advantage kiểu single path và vine khi số mẫu tăng dần.
3. Đo thời gian/bộ nhớ của Fisher-vector product (CG) so với gradient thường theo số tham số, để thấy vì sao PPO (bậc một) trở thành lựa chọn mặc định ở quy mô LLM.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Advantage	$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s)$	hành động tốt/kém hơn trung bình
Performance difference	$\eta(\tilde\pi) = \eta(\pi) + \mathbb{E}_{\tilde\pi}[\sum \gamma^t A_\pi]$	Lemma 1, telescoping
Surrogate	$L_\pi(\tilde\pi) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde\pi(a \mid s) A_\pi(s,a)$	dùng ρ_π thay $\rho_{\tilde\pi}$
Cận CPI (mixture)	$\eta \geq L - \frac{2\gamma\varepsilon}{(1-\gamma)^2}\alpha^2$	Kakade & Langford, chỉ policy hỗn hợp
Theorem 1 (TRPO)	$\eta \geq L - \frac{4\gamma\varepsilon}{(1-\gamma)^2}\alpha^2$, $\alpha = D_{TV}^{max}$	mọi policy ngẫu nhiên
Cận dạng KL	$\eta \geq L - C\, D_{KL}^{max}$, $C = 4\gamma\varepsilon/(1-\gamma)^2$	dùng $D_{TV}^2 \leq D_{KL}$
Bài toán thực hành	$\max_\theta L_{\theta_{old}}(\theta)$ s.t. $\bar D_{KL} \leq \delta$	ràng buộc cứng thay hình phạt
Hướng đi	$s \approx A^{-1} g$	conjugate gradient, FVP $= J^T M J$
Độ dài bước	$\beta = \sqrt{2\delta/(s^T A s)}$	rồi line search backtracking
Single path / vine	lấy mẫu 1 trajectory / nhiều hành động từ rollout set + CRN	vine: phương sai thấp, tốn mẫu hơn
Natural gradient	$\theta_{new} = \theta_{old} + \lambda_1 A^{-1} g$	bước cố định, không ràng buộc KL mỗi bước

11. Kết luận ngắn

TRPO chứng minh rằng tối đa hóa một surrogate objective cục bộ trừ đi một hình phạt theo KL divergence đảm bảo cải thiện đơn điệu cho mọi policy ngẫu nhiên tham số hóa, mở rộng kết quả trước đó (chỉ áp dụng cho policy hỗn hợp) của Kakade & Langford. Vì hình phạt lý thuyết quá nặng, TRPO dùng ràng buộc KL cứng (trust region) giải bằng conjugate gradient và line search, cho ra một thuật toán học được các policy mạng neural lớn cho cả điều khiển robot liên tục và Atari, với rất ít điều chỉnh. Cái giá phải trả là độ phức tạp tính toán mỗi bước (CG, Fisher-vector product, backtracking), chính là điều PPO (bài 07) sẽ loại bỏ.

Link gốc: https://arxiv.org/abs/1502.05477

Bài 05

High-Dimensional Continuous Control Using Generalized Advantage Estimation (2015)

Tác giả: John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, Pieter Abbeel (UC Berkeley).
Venue / arXiv: arXiv:1506.02438 (v6, 20/10/2018); venue: ICLR 2016 ("Published as a conference paper at ICLR 2016", ghi trên đầu bài).
Nguồn: https://arxiv.org/abs/1506.02438
Chủ đề: một họ ước lượng advantage, **GAE(γ, λ)**, viết dưới dạng tổng trọng số theo cấp số nhân của các TD residual, đánh đổi bias/variance bằng tham số λ; kết hợp với trust-region optimization (của TRPO) cho cả policy và value function để huấn luyện policy mạng neural cho điều khiển robot 3D liên tục.
Vị trí trong lộ trình: đọc sau bài 04 (TRPO, cung cấp chính cơ chế cập nhật ràng buộc KL mà bài này dùng cho cả policy và value function), trước bài 07 (PPO, dùng trực tiếp bản rút gọn hữu hạn của GAE làm advantage estimator mặc định) và là kiến thức nền cho mọi bài RLHF/RLVR sau có nhắc tới λ, critic, hay "advantage estimator".
Chương nền liên quan: nền 01: MDP, return và phương trình Bellman, nền 03: Policy gradient và baseline, nền 04: Actor-critic và GAE

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Policy gradient cho ước lượng gradient **không chệch** của tổng thưởng kỳ vọng, nhưng phương sai tăng xấu theo độ dài horizon (hiệu ứng một hành động bị trộn lẫn với hiệu ứng mọi hành động trước/sau). Actor-critic dùng **value function** thay phần thưởng thực nghiệm để giảm phương sai, đổi lại thêm bias. Paper nhấn mạnh: bias **nguy hiểm hơn** variance, vì dù vô hạn mẫu, bias có thể khiến thuật toán không hội tụ hoặc hội tụ về nghiệm không phải cực trị cục bộ.

Dạng chung của policy gradient (công thức (1) của bài báo): $g = \mathbb{E}[\sum_t \Psi_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$, với Ψ_t có thể là: (1) tổng thưởng toàn trajectory; (2) thưởng từ bước t về sau; (3) bản có baseline; (4) $Q_\pi(s_t, a_t)$; (5) $A_\pi(s_t, a_t)$; (6) TD residual $r_t + V_\pi(s_{t+1}) - V_\pi(s_t)$. Chọn $\Psi_t = A_\pi$ cho phương sai gần thấp nhất: trực giác là một bước cập nhật nên tăng xác suất hành động **tốt hơn trung bình** và giảm xác suất hành động **kém hơn trung bình**, đúng nghĩa của advantage.

γ như một tham số giảm phương sai, không phải của MDP: bài toán gốc là **không chiết khấu** (undiscounted), tối đa hóa $\sum_t r_t$. $\gamma < 1$ được đưa vào như một "mẹo" hạ trọng số các hiệu ứng xa, đổi lấy bias cho gradient ước lượng g^γ (công thức (6)), khác với vai trò γ trong một MDP chiết khấu thông thường.

Khái niệm nền: TD residual $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ (sai số dự đoán một bước của value function). **γ-just estimator** (Định nghĩa 1): một ước lượng advantage $\hat{A}_t$ không làm gradient chệch khi thế vào g^γ, tức $\mathbb{E}[\hat{A}_t \nabla \log \pi_\theta] = \mathbb{E}[A^{\pi,\gamma} \nabla \log \pi_\theta]$. **Reward shaping** (Ng và đồng nghiệp, 1999): biến đổi hàm thưởng bằng một hàm thế năng $\Phi(s)$ mà không đổi advantage. **Trust region cho value function:** áp cùng ý tưởng ràng buộc KL của TRPO (bài 04) cho bài toán hồi quy khớp value function.

2. Ý tưởng chính

Định nghĩa TD residual $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ như một ước lượng advantage một bước đơn giản (chệch nếu V không chính xác, phương sai thấp). Xây các ước lượng k-bước $\hat{A}_t^{(k)}$ bằng tổng kính lồng của các δ (ít chệch hơn khi k tăng, nhưng phương sai cao hơn). **GAE(γ, λ)** là trung bình có trọng số theo cấp

số nhân $(1-\lambda)\lambda^{k-1}$ của mọi $\hat{A}_t^{(k)}$, và trung bình này gập lại thành một công thức đóng rất đơn giản: tổng $(\gamma\lambda)^l\delta_{t+l}^V$. Tham số λ nội suy giữa TD residual một bước (ít phương sai, nhiều chệch, $\lambda=0$) và Monte Carlo trừ baseline (không chệch, nhiều phương sai, $\lambda=1$). Kết hợp ước lượng advantage này với cập nhật ràng buộc trust-region kiểu TRPO cho **cả** policy và value function, paper huấn luyện được các policy mạng neural lớn cho robot 3D đi/chạy/đứng lên.

3. Phương pháp chi tiết

3.1 γ-just estimator và Proposition 1

$V^{\pi,\gamma}, Q^{\pi,\gamma}, A^{\pi,\gamma}$ là các hàm giá trị/advantage chiết khấu theo γ. $\hat{A}_t$ là γ-**just** nếu $\mathbb{E}_{s_{0:\infty},a_{0:\infty}}[\hat{A}_t\nabla_\theta\log\pi_\theta(a_t|s_t)] = \mathbb{E}[A^{\pi,\gamma}(s_t,a_t)\nabla_\theta\log\pi_\theta(a_t|s_t)]$ với mọi t (Định nghĩa 1).

Proposition 1 (điều kiện đủ): nếu $\hat{A}_t = Q_t - b_t$ với Q_t là ước lượng không chệch của $Q^{\pi,\gamma}(s_t,a_t)$ (có thể phụ thuộc toàn trajectory) và b_t chỉ phụ thuộc trạng thái/hành động **trước** bước t, thì $\hat{A}_t$ là γ-just.

Tự suy lại chứng minh (Appendix B): tách $\mathbb{E}[\nabla\log\pi_\theta(a_t|s_t)(Q_t-b_t)]$ thành hai phần.

- Phần Q_t: $\mathbb{E}_{s_{0:t},a_{0:t}}[\nabla\log\pi_\theta(a_t|s_t)\cdot\mathbb{E}_{s_{t+1:\infty},a_{t+1:\infty}}[Q_t]] = \mathbb{E}[\nabla\log\pi_\theta(a_t|s_t)A^\pi(s_t,a_t)]$ (dùng luật kỳ vọng lồng, Q_t không chệch cho $Q^{\pi,\gamma}$).
- Phần b_t: $\mathbb{E}_{s_{0:t},a_{0:t-1}}[b_t\cdot\mathbb{E}_{s_{t+1:\infty},a_{t:\infty}}[\nabla\log\pi_\theta(a_t|s_t)]]$. Vì b_t không phụ thuộc a_t, có thể đưa ra ngoài kỳ vọng theo a_t; mà $\mathbb{E}_{a_t\sim\pi_\theta}[\nabla_\theta\log\pi_\theta(a_t|s_t)] = \int\pi_\theta(a|s_t)\nabla_\theta\log\pi_\theta(a|s_t)\,da = \int\nabla_\theta\pi_\theta(a|s_t)\,da = \nabla_\theta\int\pi_\theta(a|s_t)\,da = \nabla_\theta 1 = 0$. Vậy số hạng b_t triệt tiêu hoàn toàn.

3.2 TD residual và các ước lượng k-bước (tổng kính lồng)

$\delta_t^V := r_t + \gamma V(s_{t+1}) - V(s_t)$. Nếu $V = V^{\pi,\gamma}$ đúng, δ_t^V là γ-just và không chệch cho $A^{\pi,\gamma}$ (lấy kỳ vọng theo s_{t+1}). Định nghĩa $\hat{A}_t^{(k)} := \sum_{l=0}^{k-1}\gamma^l\delta_{t+l}^V$. **Tự suy lại tổng kính lồng cho $k=2$:**

$$\delta_t^V + \gamma\delta_{t+1}^V = [-V(s_t)+r_t+\gamma V(s_{t+1})] + \gamma[-V(s_{t+1})+r_{t+1}+\gamma V(s_{t+2})] = -V(s_t)+r_t+\gamma r_{t+1}+\gamma^2 V(s_{t+2})$$

(các số hạng $\pm\gamma V(s_{t+1})$ triệt tiêu). Tổng quát:

$$\hat{A}_t^{(k)} = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1}r_{t+k-1} + \gamma^k V(s_{t+k})$$

Khi $k\to\infty$: $\hat{A}_t^{(\infty)} = -V(s_t) + \sum_{l=0}^{\infty}\gamma^l r_{t+l}$ (Monte Carlo trừ baseline, γ-just với **mọi** V, nhưng phương sai cao vì là tổng của nhiều phần thưởng ngẫu nhiên). Bias của $\hat{A}_t^{(k)}$ giảm khi k tăng (số hạng $\gamma^k V(s_{t+k})$ bị chiết khấu mạnh hơn), còn $-V(s_t)$ không ảnh hưởng tới bias (là baseline xác định).

3.3 GAE(γ,λ): trung bình trọng số mũ (re-derive công thức 16)

$$\hat{A}_t^{GAE(\gamma,\lambda)} := (1-\lambda)(\hat{A}_t^{(1)} + \lambda\hat{A}_t^{(2)} + \lambda^2\hat{A}_t^{(3)} + \cdots)$$

Thế từng $\hat{A}_t^{(k)} = \sum_{l=0}^{k-1}\gamma^l\delta_{t+l}^V$ vào rồi **gộp lại theo chỉ số** δ (mỗi δ_{t+l}^V xuất hiện trong mọi $\hat{A}_t^{(k)}$ với $k>l$):

$$\hat{A}_t^{GAE} = (1-\lambda)\Big[\delta_t^V(1+\lambda+\lambda^2+\cdots) + \gamma\delta_{t+1}^V(\lambda+\lambda^2+\cdots) + \gamma^2\delta_{t+2}^V(\lambda^2+\lambda^3+\cdots) + \cdots\Big]$$

Mỗi ngoặc là chuỗi hình học: $(1+\lambda+\cdots) = \frac{1}{1-\lambda}$, $(\lambda+\lambda^2+\cdots) = \frac{\lambda}{1-\lambda}$, $(\lambda^2+\lambda^3+\cdots) = \frac{\lambda^2}{1-\lambda}$. Nhân với $(1-\lambda)$ ở ngoài, hệ số của $\gamma^l\delta_{t+l}^V$ chính là λ^l:

$$\boxed{\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}^V} \tag{16}$$

Hai trường hợp đặc biệt: $\lambda = 0$: $\hat{A}_t = \delta_t^V$ (TD residual một bước, giống actor-critic cổ điển); $\lambda = 1$: $\hat{A}_t = \sum_l \gamma^l r_{t+l} - V(s_t)$ (γ-just với mọi V, nhưng phương sai cao vì là tổng dài).

Vai trò khác nhau của γ và λ: γ quyết định **thang đo** của $V^{\pi,\gamma}$ bản thân, luôn gây bias cho g^γ bất kể V chính xác tới đâu (vì bài toán gốc không chiết khấu); $\lambda < 1$ **chỉ** gây bias khi V sai (nếu $V = V^{\pi,\gamma}$ đúng, mọi $\hat{A}^{(k)}$, do đó mọi λ, đều γ-just). Đây là lý do thực nghiệm λ tối ưu thường nhỏ hơn hẳn γ tối ưu.

3.4 Diễn giải bằng reward shaping (re-derive công thức 20-25)

Reward shaping: với $\Phi : S \to \mathbb{R}$ tùy ý, $\tilde{r}(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s)$. **Advantage không đổi** với phép biến đổi này, vì tổng chiết khấu của $\tilde{r}$ dọc một trajectory từ s_t chỉ lệch tổng chiết khấu của r một hằng số $\Phi(s_t)$ (tổng kính lồng tương tự §3.2):

$$\sum_{l=0}^{\infty} \gamma^l \tilde{r}(s_{t+l}, a_{t+l}, s_{t+l+1}) = \sum_{l=0}^{\infty} \gamma^l r(s_{t+l}, a_{t+l}, s_{t+l+1}) - \Phi(s_t)$$

Suy ra $\tilde{Q}^{\pi,\gamma} = Q^{\pi,\gamma} - \Phi$, $\tilde{V}^{\pi,\gamma} = V^{\pi,\gamma} - \Phi$, nên $\tilde{A}^{\pi,\gamma} = A^{\pi,\gamma}$ (phần Φ triệt tiêu khi lấy hiệu). Nếu đặt $\Phi = V^{\pi,\gamma}$ (đúng), $\tilde{V}^{\pi,\gamma} \equiv 0$, và phần thưởng đã reshape $\tilde{r}$ có kỳ vọng bằng 0 khi chỉ biết s_t (không biết a_t); nghĩa là **toàn bộ** vấn đề credit-assignment trải dài theo thời gian đã bị "dồn về một bước".

Đặt $\Phi = V$ (xấp xỉ), ta có $\tilde{r} = \delta^V$. Xét **thêm một chiết khấu "sâu hơn"** $\gamma\lambda$ cho phần thưởng đã reshape:

$$\sum_{l=0}^{\infty} (\gamma\lambda)^l \tilde{r}(s_{t+l}, a_t, s_{t+l+1}) = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V = \hat{A}_t^{GAE(\gamma,\lambda)}$$

tức là GAE chính là "tổng chiết khấu $\gamma\lambda$ của phần thưởng đã reshape bằng V", một cách diễn giải thứ hai độc lập với §3.3 nhưng cho cùng công thức.

Hàm phản hồi (response function): $\chi(l; s_t, a_t) = \mathbb{E}[r_{t+l}|s_t, a_t] - \mathbb{E}[r_{t+l}|s_t]$, với $A^{\pi,\gamma}(s, a) = \sum_l \gamma^l \chi(l; s, a)$. Dùng $\gamma < 1$ tương đương bỏ các số hạng $l \gg 1/(1-\gamma)$: sai số nhỏ nếu χ giảm nhanh (hiệu ứng một hành động bị "quên" sau $\approx 1/(1-\gamma)$ bước). Nếu $\Phi = V^{\pi,\gamma}$ hoàn hảo, χ chỉ còn khác 0 tại $l = 0$; với $\Phi = V$ xấp xỉ, dùng chiết khấu $\gamma\lambda$ (sâu hơn γ) để cắt bớt phần đuôi nhiễu còn sót lại sau khi đã "dồn" một phần credit-assignment vào V.

3.5 Trust region cho value function (re-derive công thức 28-30)

Hồi quy đơn giản: $\min_\phi \sum_n \|V_\phi(s_n) - \hat{V}_n\|^2$ với $\hat{V}_t = \sum_l \gamma^l r_{t+l}$ (target Monte Carlo/TD(1)). Bài báo dùng bản **trust-region**: tính $\sigma^2 = \frac{1}{N}\sum_n \|V_{\phi_{old}}(s_n) - \hat{V}_n\|^2$ (phương sai phần dư ở ϕ_{old}), rồi giải

$$\min_\phi \sum_n \|V_\phi(s_n) - \hat{V}_n\|^2 \quad \text{s.t.} \quad \frac{1}{N}\sum_n \frac{\|V_\phi(s_n) - V_{\phi_{old}}(s_n)\|^2}{2\sigma^2} \le \epsilon$$

Ràng buộc này **tương đương** giới hạn KL divergence trung bình giữa phân phối Gauss cũ $\mathcal{N}(V_{\phi_{old}}(s), \sigma^2)$ và mới $\mathcal{N}(V_\phi(s), \sigma^2)$. Giải gần đúng bằng Gauss-Newton: g là gradient mục tiêu, $H = \frac{1}{N}\sum_n j_n j_n^T$ với $j_n = \nabla_\phi V_\phi(s_n)$ (chính là Fisher Information Matrix của value function xem như phân phối Gauss, sai khác hệ số σ^2). Dùng **conjugate gradient** (đúng công cụ của bài 04) tính hướng $s \approx -H^{-1}g$, rồi co giãn $s \to \alpha s$ sao cho $\frac{1}{2}(\alpha s)^T H(\alpha s) = \epsilon$.

3.6 Thuật toán tổng thể

```
Input: policy theta_0, value function phi_0
for i = 0, 1, 2, ...:
    Mo phong pi_theta_i, thu N buoc thoi gian
    Tinh delta_t^V voi V = V_phi_i   (dung phi_i, KHONG dung phi_{i+1})
    A_hat_t = sum_{l=0}^inf (gamma*lambda)^l * delta_{t+l}^V
    theta_{i+1} <- cap nhat TRPO (bai 04), dung A_hat lam advantage
    phi_{i+1}   <- hoi quy trust-region (muc 3.5)
```

Chi tiết quan trọng: cập nhật policy dùng V_{ϕ_i} **chứ không phải** $V_{\phi_{i+1}}$. Lý do (bài báo nêu trực tiếp): nếu cập nhật value function trước, dùng $V_{\phi_{i+1}}$ để tính advantage sẽ gây thêm bias; trường hợp cực đoan, nếu V bị overfit hoàn hảo tới mức $\delta_t^V \equiv 0$ ở mọi bước, ước lượng policy gradient sẽ bằng 0 dù policy chưa tối ưu.

3.7 Ví dụ nhỏ "tự tính": GAE 3 bước

Giả sử $\gamma = 0.9$, $\lambda = 0.5$, phần thưởng $r_0 = 1$, $r_1 = 0$, $r_2 = 2$, trạng thái s_3 là trạng thái kết thúc (terminal, $V(s_3) = 0$), và value function hiện tại cho $V(s_0) = 0.5$, $V(s_1) = 0.8$, $V(s_2) = 0.3$.

Bước 1: tính TD residual $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$:

$$\delta_0 = 1 + 0.9(0.8) - 0.5 = 1.22, \quad \delta_1 = 0 + 0.9(0.3) - 0.8 = -0.53, \quad \delta_2 = 2 + 0.9(0) - 0.3 = 1.70$$

Bước 2: tính các ước lượng k-bước tại $t = 0$ (dùng $\hat{A}^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_l$):

$$\hat{A}^{(1)} = \delta_0 = 1.22, \quad \hat{A}^{(2)} = \delta_0 + \gamma\delta_1 = 1.22 + 0.9(-0.53) = 0.743, \quad \hat{A}^{(3)} = \hat{A}^{(2)} + \gamma^2\delta_2 = 0.743 + 0.81(1.70) = 2.12$$

Vì episode kết thúc ở s_3 (không còn thưởng/giá trị sau đó), $\hat{A}^{(3)} = \hat{A}^{(4)} = \cdots = \hat{A}^{(\infty)} = 2.12$: không còn bias từ k lớn hơn.

Bước 3a: GAE theo công thức đóng (16), $\gamma\lambda = 0.45$:

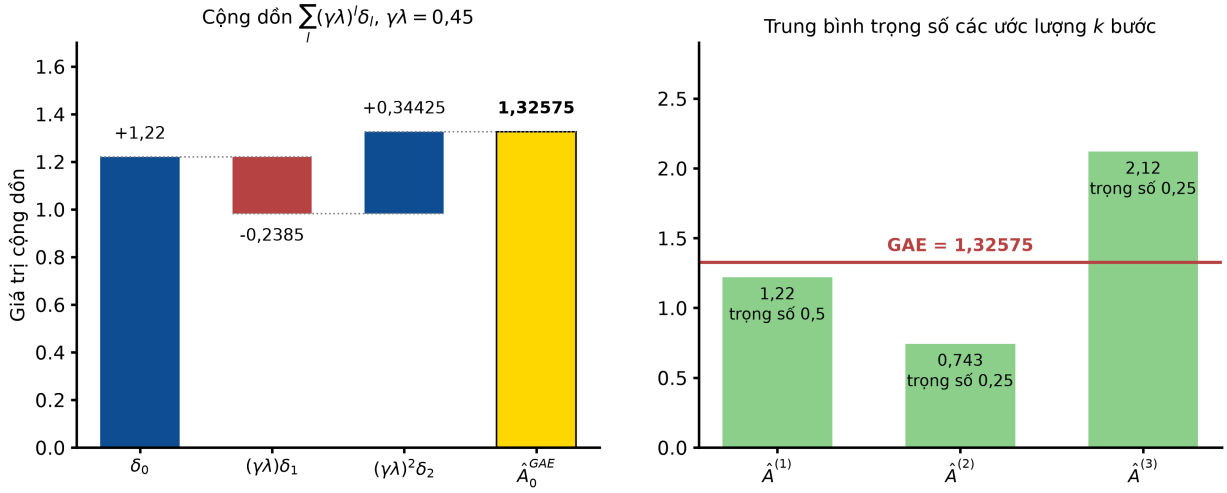
$$\hat{A}_0^{GAE} = \delta_0 + (\gamma\lambda)\delta_1 + (\gamma\lambda)^2\delta_2 = 1.22 + 0.45(-0.53) + 0.2025(1.70) = 1.22 - 0.2385 + 0.34425 = 1.32575$$

Bước 3b: kiểm lại bằng định nghĩa trung bình trọng số của $\hat{A}^{(k)}$. Vì trajectory kết thúc tại $k = 3$ (episode chỉ có 3 bước), trọng số $(1-\lambda)\lambda^{k-1}$ chỉ áp dụng cho $k = 1, 2$; toàn bộ phần đuôi còn lại ($k \geq 3$, đều bằng $\hat{A}^{(3)}$ vì đã terminal) dồn vào trọng số $\lambda^2 = 0.25$ (phần xác suất còn lại để tổng trọng số bằng 1):

$$w_1 = (1-\lambda) = 0.5, \quad w_2 = (1-\lambda)\lambda = 0.25, \quad w_3 = \lambda^2 = 0.25, \quad w_1 + w_2 + w_3 = 1$$

$$\hat{A}_0^{GAE} = w_1\hat{A}^{(1)} + w_2\hat{A}^{(2)} + w_3\hat{A}^{(3)} = 0.5(1.22) + 0.25(0.743) + 0.25(2.12) = 0.61 + 0.18575 + 0.53 = 1.32575$$

Hai cách tính (công thức đóng dạng TD(λ) và trung bình trọng số trực tiếp của các $\hat{A}^{(k)}$) cho **đúng cùng kết quả 1.32575** (tự tính, kiểm bằng `python3 -I`), xác nhận công thức (16) đã suy ở §3.3. *Ghi chú: nếu rollout bị cắt giữa episode (như trong PPO, bài 07) thay vì kết thúc tự nhiên, số hạng cuối sẽ bootstrap bằng $\gamma^{T-t}V(s_T) \neq 0$ chứ không phải 0 như ví dụ terminal ở đây.*



Hình 32: Ví dụ GAE 3 bước với $\gamma = 0{,}9$, $\lambda = 0{,}5$: cộng dồn theo TD residual (trái) và trung bình trọng số các ước lượng k bước (phải) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: hai panel cho ra cùng một kết quả 1,32575 bằng hai cách tính. Bên trái là công thức đóng $\sum(\gamma\lambda)^l\delta_l$, mỗi cột là một số hạng cộng dồn; bên phải là trung bình trọng số $0{,}5;\ 0{,}25;\ 0{,}25$ của $\hat{A}^{(1)}, \hat{A}^{(2)}, \hat{A}^{(3)}$, và đường đỏ nằm giữa ước lượng một bước (ít chệch ít) và Monte Carlo (1,22 và 2,12).

4. Thí nghiệm và kết quả chính

Bài toán: cart-pole (baseline, policy tuyến tính, value function 1 lớp ẩn 20 unit); và ba tác vụ robot 3D mô phỏng MuJoCo: biped locomotion (33 chiều trạng thái, 10 bậc tự do điều khiển), quadruped locomotion (29 chiều, 8 bậc tự do), biped getting up (đứng lên từ nằm ngửa). Timestep 0.01s.

Kiến trúc: feedforward 3 lớp ẩn 100/50/25 unit tanh, dùng chung cho policy và value function (value function chỉ có 1 đầu ra scalar). Cart-pole: policy tuyến tính, value function 1 lớp ẩn 20 unit.

Hàm thưởng:

Tác vụ	Hàm thưởng
Biped locomotion	$v_{fwd} - 10^{-5}\|u\|^2 - 10^{-5}\|f_{impact}\|^2 + 0.2$
Quadruped locomotion	$v_{fwd} - 10^{-6}\|u\|^2 - 10^{-3}\|f_{impact}\|^2 + 0.05$
Biped getting up	$-(h_{head} - 1.5)^2 - 10^{-5}\|u\|^2$

(v_{fwd}: vận tốc tiến; u: vector lực xoắn khớp; f_{impact}: lực va chạm; h_{head}: độ cao đầu.) Episode kết thúc nếu khối tâm rơi dưới 0.8m (biped) / 0.2m (quadruped), hoặc sau 2000 bước. Batch: cart-pole 20 trajectory/batch (tối đa 1000 bước); biped 50000 bước/batch; quadruped và getting-up 200000 bước/batch. "No VF" trong các hình là baseline phụ thuộc thời gian (trung bình return theo từng bước trong batch), không phải value function theo trạng thái.

Kết quả (đường học, không có bảng số điểm cuối):

- **Cart-pole** (21 seed): tốt nhất ở $\gamma \in [0.96, 0.99]$, $\lambda \in [0.92, 0.99]$.
- **Biped** (9 trial, ≈2 giờ/máy 16 lõi mỗi trial): tốt nhất ở $\gamma \in [0.99, 0.995]$, $\lambda \in [0.96, 0.99]$; sau 1000 vòng lặp, gait ổn định, nhanh, trơn. "Thời gian thật" ước tính: 0.01s/bước $\times$ 50000 bước/batch $\times$ 1000 batch$/(3600 \times 24) \approx 5.8$ ngày.

- **Quadruped và getting-up** (5 trial mỗi tác vụ, ≈4 giờ/máy 32 lõi): so sánh hạn chế (do chi phí tính toán), cố định $\gamma = 0.995$, thử $\lambda \in \{0, 0.96\}$ và "không value function". Quadruped: tốt nhất với $\lambda = 0.96$. Getting-up: value function luôn giúp cải thiện, $\lambda = 0.96$ và $\lambda = 1$ cho kết quả tương đương. **Mâu thuẫn số liệu trong bài báo:** văn bản §6.3.3 ghi cố định $\gamma = 0.995$ cho cả hai tác vụ này, nhưng chú giải Hình 4 (3D Standing Up) lại ghi $\gamma = 0.99$; ở đây nêu cả hai, không tự chọn số đúng.
- Footnote: thử cả target TD(λ) cho value function (dùng chính công thức $(\gamma\lambda)^l$ cho target hồi quy) nhưng "không thấy khác biệt hiệu suất" so với target Monte Carlo ($\lambda = 1$) ở công thức (28).

5. Phân tích / ablation

- $\lambda = 0$ **(TD một bước) cho bias quá lớn:** bài báo nêu rõ "chúng tôi thấy thực nghiệm rằng return một bước ($\lambda = 0$) dẫn tới bias quá mức và hiệu suất kém", khác với một số phương pháp concurrent (DDPG, Lillicrap và đồng nghiệp 2015, bài 10 trong lộ trình này) dùng bootstrap một bước thành công nhưng trên không gian trạng thái/hành động thấp chiều hơn nhiều.
- **"Vì sao không dùng Q-function?" (Appendix A.2):** actor-critic cổ điển dùng Q_π học được để ước lượng advantage ($\hat{A} = Q(s,a) - V(s)$) cho phương sai thấp, nhưng chỉ cho một ước lượng chệch cao kiểu một bước, không có cách nội suy như λ. $V(s)$ đầu vào thấp chiều hơn $Q(s,a)$ nên dễ học hơn. Đây là lý do tác giả chọn hướng V + TD-residual thay vì học Q.
- **Compatible features (Appendix A.1):** gradient policy chỉ phụ thuộc advantage qua không gian con trải bởi $\nabla_\theta \log \pi_\theta(a_t|s_t)$; natural policy gradient có thể lấy từ việc chiếu $\hat{A}_t$ lên không gian con này bằng bình phương tối thiểu; nếu $\hat{A}$ là γ-just, nghiệm chính là natural policy gradient. Đây là một ý tưởng **độc lập, bổ sung** cho GAE (có thể thay $\hat{A}$ bằng GAE trong công thức chiếu này).

6. Giới hạn

Tác giả nêu: quan hệ giữa sai số ước lượng value function và sai số policy gradient chưa rõ, để ngỏ; kiến trúc chia sẻ policy/value function được gợi ý nhưng chưa có điều kiện hội tụ; chỉnh γ, λ tự động để ngỏ cho nghiên cứu sau.

Nhận xét: thí nghiệm chỉ trên điều khiển liên tục MuJoCo, không có Atari như bài 04; khoảng γ, λ tốt chỉ báo qua đồ thị, không có giá trị tối ưu số hay độ mịn lưới dò; chỉ một kiến trúc value function được thử; dựa hoàn toàn vào bộ máy TRPO cho cả hai lần cập nhật mỗi vòng lặp, nhân đôi độ phức tạp cài đặt của bài 04.

7. Liên hệ với các bài khác

- **Bài 04 (TRPO):** cùng tác giả, GAE dùng chính cơ chế cập nhật ràng buộc KL của TRPO (trích trực tiếp công thức (31) ở đây) cho policy, và mở rộng ý tưởng sang khớp value function bằng một ràng buộc tương đương KL trên phân phối Gauss.
- **Bài 07 (PPO):** dùng bản **rút gọn hữu hạn** của GAE (horizon T, bootstrap bằng $V(s_T)$) làm advantage estimator mặc định, $\lambda = 0.95$; công thức ở §3.3-3.7 của bài này giải thích trực tiếp công thức đó.
- **Bài 10 (DDPG), 11 (TD3), 12 (SAC):** họ actor-critic dùng Q-function học được và bootstrap một bước thay vì V+GAE; đối chiếu tốt với §5 ở trên (vì sao bài này không chọn hướng đó).
- **Bài 28, 36 (GRPO), 37 (RLOO) và nhóm RLVR cho LLM:** nhiều phương pháp bỏ hẳn value function/GAE, dùng baseline tương đối theo nhóm, chính vì chi phí học một critic tốt (bộ nhớ, tính toán) ở quy mô LLM là rất lớn; phần phân tích bias/variance ở bài này là nền tảng để hiểu đánh đổi đó.

8. Ứng dụng thực tế (suy luận)

GAE(λ) với $\lambda < 1$ vẫn là ước lượng advantage mặc định cho mọi quy trình RLHF/RLVR dùng critic học được (PPO trong InstructGPT, llm 07). Siêu tham số quan trọng: λ (đánh đổi bias/variance) và γ (sinh văn bản theo episode, phần thưởng thường chỉ cho một lần ở cuối, nên γ thường để gần 1, vấn đề credit-assignment theo token vẫn tương tự về bản chất).

Một critic riêng có thể làm tăng đáng kể chi phí bộ nhớ và tính toán. GRPO (bài 36) tránh huấn luyện critic riêng, nhưng dùng một cách ước lượng advantage khác.

9. Thí nghiệm gợi ý

1. Trên một MDP lưới nhỏ với value function thật đã biết chính xác, cài GAE(γ, λ) cho nhiều $\lambda \in \{0, 0.5, 0.9, 0.95, 1\}$, đo bias và variance của advantage ước lượng so với A_π thật, xác nhận bias tăng khi λ giảm lúc V không chính xác, và biến mất khi $V = V^{\pi,\gamma}$ đúng.
2. Lặp lại ví dụ "tự tính" ở §3.7 với horizon dài hơn ($T = 10$) và so sánh công thức đóng dùng terminal $V = 0$ với bản bootstrap $V(s_T) \neq 0$ (công thức (11) của bài 07), để thấy rõ khác biệt giữa GAE "trọn episode" và GAE "cắt đoạn" dùng trong PPO.
3. Huấn luyện một agent policy-gradient nhỏ (CartPole) có và không có cập nhật trust-region cho value function (mục 3.5) so với chỉ hồi quy MSE thường, đo độ ổn định đường học qua nhiều seed.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
TD residual	$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$	advantage một bước
Ước lượng k-bước	$\hat{A}_t^{(k)} = -V(s_t) + \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k})$	tổng kính lồng
GAE(γ, λ)	$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$	trung bình trọng số $(1-\lambda)\lambda^{k-1}$ của $\hat{A}^{(k)}$
$\lambda = 0$ / $\lambda = 1$	δ_t^V / $\sum_l \gamma^l r_{t+l} - V(s_t)$	ít phương sai-nhiều chệch / ngược lại
γ-just	$\mathbb{E}[\hat{A}_t \nabla \log \pi] = \mathbb{E}[A^{\pi,\gamma} \nabla \log \pi]$	điều kiện không chệch cho gradient
Reward shaping	$\tilde{r} = r + \gamma\Phi(s') - \Phi(s)$, $\Phi = V$	$\tilde{r} = \delta^V$, GAE = tổng chiết khấu $\gamma\lambda$ của $\tilde{r}$
Trust region cho V	hồi quy MSE + ràng buộc $\approx$ KL Gauss	Gauss-Newton + conjugate gradient, như bài 04
Thứ tự cập nhật	policy dùng V_{ϕ_i}, không dùng $V_{\phi_{i+1}}$	tránh thêm bias

11. Kết luận ngắn

GAE xây một họ ước lượng advantage bằng tổng trọng số theo cấp số nhân của TD residual, gập lại thành công thức đóng rất đơn giản $\sum(\gamma\lambda)^l \delta_{t+l}^V$, cho phép chỉnh liên tục giữa TD residual một bước (ít phương sai, nhiều chệch) và Monte Carlo trừ baseline (không chệch, nhiều phương sai) chỉ bằng một tham số λ. Kết hợp với cập nhật trust-region kiểu TRPO cho cả policy và value function, phương pháp học được các gait robot 3D phức tạp từ đầu. Công thức GAE hữu hạn-horizon của bài này trở thành advantage estimator chuẩn của PPO (bài 07), còn phần phân tích chi phí của một critic học được là nền tảng để hiểu vì sao các phương pháp RLVR cho LLM sau này (GRPO, bài 36) chọn bỏ hẳn value function.

Link gốc: https://arxiv.org/abs/1506.02438

Bài 06

Asynchronous Methods for Deep Reinforcement Learning (2016)

Tác giả: Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P. Lillicrap, David Silver, Koray Kavukcuoglu (Google DeepMind; Mehdi Mirza cũng ở MILA, Đại học Montreal).
Venue / arXiv: arXiv:1602.01783 (v2, 16/6/2016). Dòng footer ghi "Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48", tức ICML 2016.
Nguồn: https://arxiv.org/abs/1602.01783
Chủ đề: Thay **experience replay** (bài 01, 02, 03) bằng **nhiều actor-learner chạy song song** trên nhiều bản copy của môi trường để ổn định huấn luyện RL sâu, cho phép dùng cả thuật toán off-policy (Q-learning, Sarsa) và on-policy (advantage actor-critic). Biến thể tốt nhất, **A3C**, đạt state-of-the-art trên Atari chỉ dùng CPU, không cần GPU hay hạ tầng phân tán.
Vị trí trong lộ trình: Đọc sau bài 01-03 (họ DQN, value-based, dùng replay buffer) để thấy một hướng ổn định hóa khác hẳn. Đọc tiếp bài 05 (GAE, cách ước lượng advantage tốt hơn mà chính bài này gợi ý nên dùng) và bài 07 (PPO, cải tiến tiếp actor-critic bằng trust region/clipping).
Chương nền liên quan: nền 03: Policy gradient và baseline, nền 04: Actor-critic và GAE

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Các phương pháp ổn định hóa deep RL trước bài này (DQN và các biến thể, bài 01-03) đều dựa trên một ý chung: dữ liệu mà agent online quan sát là **non-stationary** (phân phối đổi khi policy đổi) và các bước liên tiếp **tương quan cao**; giải pháp là lưu lại trải nghiệm vào **experience replay memory** rồi lấy mẫu để phá tương quan. Nhưng replay có hai nhược điểm: tốn bộ nhớ/tính toán cho mỗi lần tương tác thật, và **buộc phải dùng thuật toán off-policy** (vì dữ liệu lấy ra có thể từ policy cũ hơn policy hiện tại).

Ý tưởng khác hẳn của bài này: chạy **nhiều agent song song**, mỗi agent trên một bản copy riêng của môi trường, cùng cập nhật một mô hình chia sẻ. Vì các agent song song thường đang ở các trạng thái khác nhau tại cùng một thời điểm, dữ liệu tổng hợp từ nhiều agent tại một thời điểm đã **đa dạng hơn** một agent đơn lẻ chạy liên tiếp, đóng vai trò decorrelation tương tự replay nhưng không cần bộ nhớ lưu lại. Điều này mở ra khả năng dùng **on-policy RL** (Sarsa, actor-critic) một cách ổn định, điều mà trước đây coi là bất ổn khi kết hợp trực tiếp với mạng sâu.

Thuật ngữ cần nắm:

- **Advantage** $A(s,a) = Q(s,a) - V(s)$: lợi thế của hành động a so với giá trị trung bình (theo policy) của trạng thái s; ước lượng bằng $R_t - b_t(s_t)$ với b_t là một baseline học được (thường $\approx V^\pi$), giúp giảm phương sai của policy gradient mà không đổi kỳ vọng (xem bài 00 về baseline).
- **REINFORCE** (Williams 1992): cập nhật tham số policy theo $\nabla_\theta \log \pi(a_t|s_t;\theta)\, R_t$, một ước lượng không chệch (unbiased) của $\nabla_\theta \mathbb{E}[R_t]$.
- **Actor-critic:** policy π (actor) được huấn luyện bằng policy gradient, dùng một hàm giá trị học được V (critic) làm baseline/ước lượng advantage.
- **Hogwild! update:** cập nhật tham số chia sẻ bất đồng bộ, không khóa (lock-free), giữa nhiều luồng; đã có lý thuyết hội tụ cho trường hợp tuyến tính, dùng ở đây cho mạng sâu (không có đảm bảo lý thuyết tương ứng, nhưng paper quan sát thực nghiệm hoạt động tốt).

2. Ý tưởng chính

Đề xuất bốn thuật toán RL sâu bất đồng bộ, đa luồng, chạy trên **một máy, nhiều CPU core, không cần GPU**: asynchronous one-step Q-learning, one-step Sarsa, n-step Q-learning, và **advantage actor-critic (A3C)**. Tất cả dùng hai ý tưởng chung: (1) nhiều actor-learner (luồng CPU) tương tác độc lập với bản copy riêng của môi trường, tích lũy gradient cục bộ rồi áp (cộng) vào tham số chia sẻ theo kiểu Hogwild!; (2) mỗi luồng dùng một chính sách khám phá hơi khác nhau (ví dụ ϵ khác nhau cho các phương pháp value-based) để tăng đa dạng dữ liệu. A3C, biến thể actor-critic bất đồng bộ dùng n-step advantage, là phương pháp tốt nhất: vượt state-of-the-art Atari khi đó trong **một nửa thời gian huấn luyện**, chỉ dùng 16 CPU core, không GPU; cũng thành công trên điều khiển liên tục (MuJoCo) và một môi trường 3D mới (Labyrinth) chỉ từ input hình ảnh.

3. Phương pháp chi tiết

3.1 Asynchronous one-step Q-learning

Mỗi luồng giữ bản sao tham số, tương tác độc lập, dùng **target network chia sẻ** θ^- (cập nhật định kỳ, giống DQN bài 01/02) để tính target:

$$y = \begin{cases} r & \text{nếu } s' \text{ terminal} \\ r + \gamma \max_{a'} Q(s', a'; \theta^-) & \text{khác} \end{cases}$$

```
Asynchronous one-step Q-learning (mỗi actor-learner thread)
// shared theta, theta_minus, counter T = 0
t = 0; theta_minus = theta; d_theta = 0; s = s0
repeat:
    a = eps-greedy(Q(s,.;theta))
    s', r = step(a)
    y = r                                    if s' terminal
      = r + gamma * max_a' Q(s', a'; theta_minus)   otherwise
    d_theta += d(y - Q(s,a;theta))^2 / d_theta
    s = s'
    T += 1; t += 1
    if T % I_target == 0: theta_minus = theta
    if t % I_AsyncUpdate == 0 or s terminal:
        apply async update of theta using d_theta; d_theta = 0
until T > T_max
```

Gradient tích lũy qua nhiều bước trước khi áp (giống minibatch) để giảm khả năng nhiều luồng ghi đè lẫn nhau. **Asynchronous one-step Sarsa** giống hệt, chỉ đổi target thành $r + \gamma Q(s', a'; \theta^-)$ với a' là hành động **thật** được chọn ở s' (on-policy), không phải $\max_{a'}$.

3.2 Asynchronous n-step Q-learning

Dùng **forward view** (tính trực tiếp n-step return, khác backward view bằng eligibility trace hay dùng trong TD truyền thống, paper chọn forward view vì dễ kết hợp với backpropagation through time khi có mạng có trọng số động lượng). Mỗi luồng chọn hành động liên tiếp tới $t_{\max}$ bước hoặc tới khi terminal, rồi tính ngược các n-step return cho mọi cặp (trạng thái, hành động) đã gặp:

```
Asynchronous n-step Q-learning (mỗi thread)
repeat:
    d_theta = 0; theta' = theta (đồng bộ tham số cục bộ)
    t_start = t; lấy trạng thái s_t
    repeat:
        a_t = eps-greedy(Q(s_t,.;theta'));  s_{t+1}, r_t = step(a_t)
        t += 1; T += 1
```

```
    until terminal s_t or t - t_start == t_max
    R = 0                            if s_t terminal
      = max_a Q(s_t, a; theta_minus)   otherwise
    for i = t-1 downto t_start:
        R = r_i + gamma * R
        d_theta += d(R - Q(s_i,a_i;theta'))^2 / d_theta'
    apply async update of theta using d_theta
until T > T_max
```

Mỗi lần cập nhật dùng độ dài return khác nhau cho từng bước: bước cuối cùng (gần $t_{\max}$) chỉ là 1-step update, bước đầu tiên của đoạn (xa $t_{\max}$ nhất) có thể lên tới $t_{\max}$-step update, "lan truyền phần thưởng xa hơn một bước mỗi lần cập nhật".

3.3 Advantage actor-critic bất đồng bộ (A3C)

Duy trì một policy $\pi(a_t|s_t;\theta)$ (thường có đầu ra softmax) và một hàm giá trị $V(s_t;\theta_v)$ (đầu ra tuyến tính); trong thực nghiệm, hai mạng này **chia sẻ phần lớn lớp ẩn**, chỉ khác lớp output cuối. Cập nhật sau mỗi $t_{\max}$ hành động hoặc khi gặp terminal, dùng cùng kiểu n-step return làm ước lượng advantage:

$$A(s_t, a_t; \theta, \theta_v) = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$$

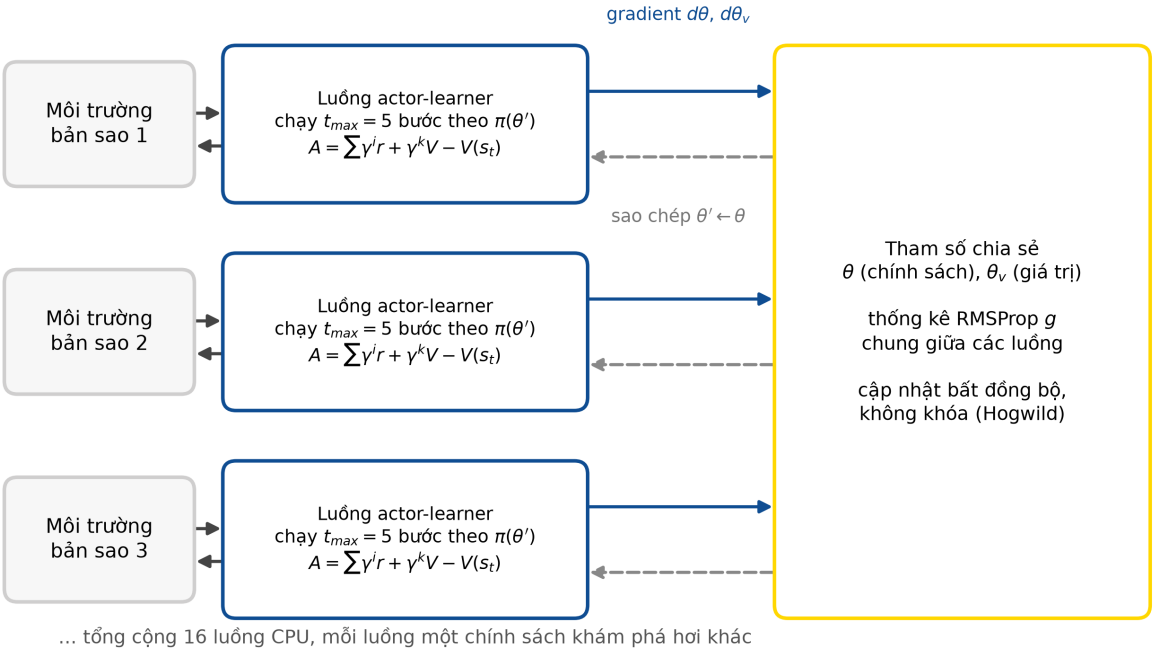
với k thay đổi theo từng trạng thái (phụ thuộc bao nhiêu bước còn lại tới $t_{\max}$ hoặc terminal), chặn trên bởi $t_{\max}$. Cập nhật policy theo:

$$\nabla_{\theta'} \log \pi(a_t|s_t;\theta')\, A(s_t, a_t; \theta, \theta_v) + \beta\, \nabla_{\theta'} H\big(\pi(\cdot|s_t;\theta')\big)$$

Số hạng thứ hai là **entropy regularization**: H là entropy của phân phối hành động, β là hệ số điều chỉnh độ mạnh. Thêm entropy vào mục tiêu **khuyến khích khám phá bằng cách ngăn hội tụ sớm về một chính sách xác định (deterministic) kém tối ưu** (kỹ thuật này đến từ Williams & Peng 1991, đặc biệt hữu ích cho các bài toán cần hành vi phân cấp/trải dài thời gian).

```
Asynchronous advantage actor-critic (A3C, mỗi thread)
repeat:
    d_theta = 0; d_theta_v = 0
    theta' = theta; theta_v' = theta_v   (đồng bộ tham số cục bộ)
    t_start = t; lấy trạng thái s_t
    repeat:
        a_t ~ pi(.|s_t; theta');  s_{t+1}, r_t = step(a_t)
        t += 1; T += 1
    until terminal s_t or t - t_start == t_max
    R = 0                     if s_t terminal
      = V(s_t; theta_v')      otherwise   # bootstrap từ trạng thái cuối
    for i = t-1 downto t_start:
        R = r_i + gamma * R
        d_theta   += grad_theta'  log(pi(a_i|s_i;theta')) * (R - V(s_i;theta_v'))
        d_theta_v += d (R - V(s_i;theta_v'))^2 / d_theta_v'
    apply async update of theta using d_theta, of theta_v using d_theta_v
until T > T_max
```

Cách đọc hình: mỗi hàng là một luồng với môi trường riêng; luồng chạy tối đa t_{max} bước theo chính sách cục bộ, tính advantage và gửi gradient (mũi tên xanh) lên tham số chia sẻ mà không khóa. Mũi tên nét đứt là bước sao chép tham số về luồng ở lần cập nhật sau. Dữ liệu của các luồng ở những trạng thái khác nhau nên thay vai trò phá tương quan của replay.



Hình 33: A3C: nhiều luồng actor-learner trên các bản sao môi trường cùng cập nhật bất đồng bộ một bộ tham số chia sẻ (minh hoạ, tự tính)

3.4 Tối ưu hóa: RMSProp chia sẻ thống kê

So sánh ba biến thể bất đồng bộ: SGD có momentum (mỗi luồng giữ vector gradient/momentum riêng), RMSProp mỗi luồng giữ thống kê g riêng, và **RMSProp chia sẻ** (g chung, cập nhật bất đồng bộ không khóa giữa các luồng):

$$g = \alpha g + (1-\alpha)\Delta\theta^2, \qquad \theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g+\epsilon}}$$

Thực nghiệm (so điểm 50 cấu hình learning rate/khởi tạo ngẫu nhiên khác nhau) cho thấy **shared RMSProp bền (robust) hơn** RMSProp riêng từng luồng, và RMSProp riêng từng luồng bền hơn SGD có momentum. Dùng shared RMSProp cho mọi kết quả chính trong bài.

3.5 Tại sao không cần experience replay (trình bày lại lập luận của paper)

Thay thế "lấy mẫu từ bộ nhớ quá khứ để phá tương quan" bằng "lấy mẫu đồng thời từ nhiều môi trường song song đang ở các trạng thái khác nhau". Vì tại một thời điểm, nhiều actor-learner thường đang trải nghiệm các phần khác nhau của không gian trạng thái (đặc biệt khi mỗi luồng còn dùng một chính sách khám phá hơi khác), các thay đổi tổng hợp lên tham số chia sẻ **ít tương quan theo thời gian hơn** so với một agent đơn cập nhật online liên tục. Lợi ích thực dụng kèm theo: (1) thời gian huấn luyện giảm gần tuyến tính theo số luồng; (2) **không còn ràng buộc phải học off-policy**, mở khóa Sarsa và actor-critic chạy ổn định với mạng sâu. Paper **không khẳng định replay là vô dụng**: phần Discussion/Conclusion nói rõ kết quả "không có nghĩa là experience replay không còn hữu ích", và gợi ý kết hợp cả hai (replay + actor song song) có thể cải thiện thêm hiệu quả dữ liệu, đặc biệt ở domain mà tương tác với môi trường đắt đỏ hơn so với việc cập nhật mô hình (ví dụ TORCS).

4. Thí nghiệm và kết quả chính

Atari (so tốc độ học, Hình 1): DQN huấn luyện trên 1 GPU Nvidia K40, bốn phương pháp bất đồng bộ huấn luyện trên **16 CPU core**, trên 5 game (Beamrider, Breakout, Pong, Q*bert, Space Invaders). Cả 4 phương pháp bất đồng bộ học nhanh hơn DQN trên hầu hết game; actor-critic bất đồng bộ (A3C) vượt rõ cả 3 phương pháp value-based bất đồng bộ còn lại.

Atari, so sánh đầy đủ 57 game, human starts (Table 1, nguyên văn):

Phương pháp	Thời gian huấn luyện	Mean	Median
DQN	8 ngày trên GPU	121.9%	47.5%
Gorila	4 ngày, 100 máy	215.2%	71.3%
D-DQN	8 ngày trên GPU	332.9%	110.9%
Dueling D-DQN	8 ngày trên GPU	343.8%	117.1%
Prioritized DQN	8 ngày trên GPU	463.6%	127.6%
A3C, FF	1 ngày trên CPU	344.1%	68.2%
A3C, FF	4 ngày trên CPU	496.8%	116.6%
A3C, LSTM	**4 ngày trên CPU**	**623.0%**	**112.6%**

A3C được huấn luyện 4 ngày trên 16 CPU core (không GPU), bằng **một nửa** thời gian của các baseline GPU (8 ngày), vẫn đạt mean cao nhất; chỉ sau 1 ngày đã bằng khoảng mean của Dueling D-DQN và gần median của Gorila. Tác giả lưu ý nhiều cải tiến của Double DQN/Dueling (bài 02, mục 3.3 bài 03) có thể áp thêm vào 1-step Q/n-step Q bất đồng bộ ở đây để cải thiện tiếp (chưa thử).

Tốc độ tăng tốc theo số luồng (Table 2, trung bình trên 7 game Atari):

Phương pháp	1 luồng	2	4	8	16
1-step Q	1.0	3.0	6.3	13.3	24.1
1-step Sarsa	1.0	2.8	5.9	13.1	22.1
n-step Q	1.0	2.7	5.9	10.7	17.2
A3C	1.0	2.1	3.7	6.9	12.5

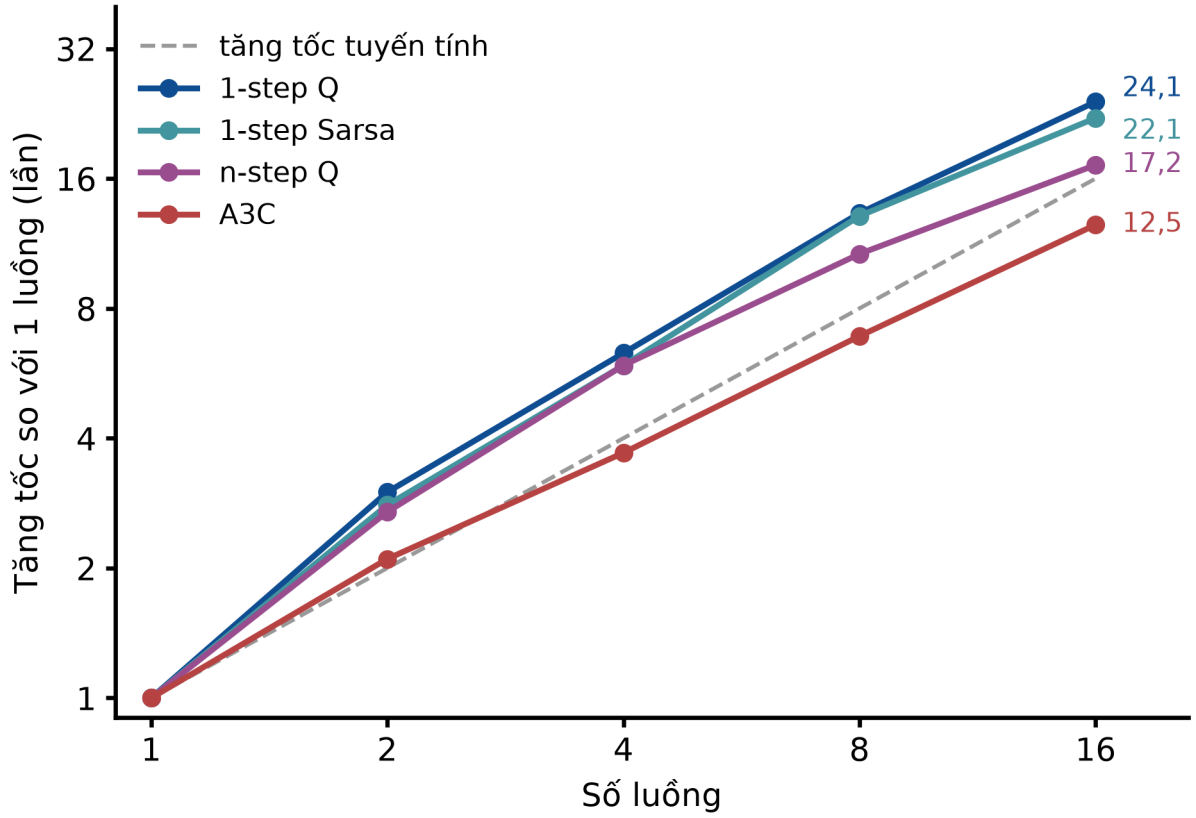
Các phương pháp one-step (Q, Sarsa) có tăng tốc **siêu tuyến tính** (ví dụ 24.1x với 16 luồng) không thể giải thích chỉ bằng lợi ích tính toán thuần; tác giả cho là nhiều luồng giúp giảm độ chệch (bias) vốn có của phương pháp one-step, nên cần ít dữ liệu hơn để đạt điểm mốc khi có nhiều luồng hơn.

Cách đọc hình: trục hai chiều đều là thang log cơ số 2, đường xám nét đứt là tăng tốc tuyến tính. Hai phương pháp one-step vượt đường này (24,1 và 22,1 lần với 16 luồng), còn A3C nằm dưới (12,5 lần), khớp nhận xét trong ghi chú rằng A3C không có tăng tốc siêu tuyến tính.

TORCS (đua xe 3D): A3C là phương pháp tốt nhất trong 4 phương pháp, đạt 75-90% điểm của người chơi thử trên cả 4 cấu hình (xe nhanh/chậm, có/không đối thủ), sau khoảng 12 giờ huấn luyện.

MuJoCo (điều khiển liên tục): chỉ thử A3C (dễ mở rộng sang hành động liên tục hơn các phương pháp value-based ở đây); tìm được giải pháp tốt cho mọi bài toán trong dưới 24 giờ, thường chỉ vài giờ, từ cả trạng thái vật lý và từ pixel. Đầu ra policy là (μ, σ^2) của phân phối Gauss nhiều chiều (hiệp phương sai dạng cầu/spherical); μ là lớp tuyến tính, σ^2 qua SoftPlus $\log(1 + e^x)$. Entropy cost cho trường hợp liên tục: $-\frac{1}{2}(\log(2\pi\sigma^2) + 1)$, hệ số 10^{-4} cho mọi task; không dùng bootstrap (mỗi episode ngắn, xử lý trọn episode thành một update).

Labyrinth (môi trường 3D mới, chỉ A3C LSTM, chỉ từ ảnh RGB 84x84): điểm trung bình cuối ~50, cho thấy agent học được chiến lược hợp lý để khám phá maze ngẫu nhiên từ input hình ảnh thuần.



Hình 34: Tăng tốc so với một luồng theo số luồng, trung bình trên 7 game Atari (vẽ lại từ số liệu trong ghi chú)

5. Phân tích / ablation

- **Robustness theo learning rate (Hình 2):** scatter 50 lần chạy (learning rate và khởi tạo ngẫu nhiên khác nhau) trên 5 game cho A3C; có một khoảng learning rate rộng mà mọi lần chạy đều đạt điểm tốt, hầu như không có điểm 0 trong vùng learning rate tốt, cho thấy phương pháp **ổn định, không sụp đổ/phân kỳ** khi đang học tốt.
- **Ảnh hưởng số luồng tới hiệu quả dữ liệu (Hình 3, Table 2):** với phương pháp one-step, nhiều luồng hơn không chỉ nhanh hơn về wall-clock mà còn cần **ít khung hình huấn luyện hơn** để đạt điểm mốc (hiệu ứng giảm bias đã nêu ở mục 4); với n-step Q và A3C, hiệu ứng này yếu hơn.
- **So sánh 3 phương pháp tối ưu (Phụ lục, Hình S5):** shared RMSProp > RMSProp riêng từng luồng > SGD có momentum, đo bằng diện tích dưới đường cong (điểm theo thứ hạng learning rate đã sắp xếp), trên 2 thuật toán (n-step Q, A3C) × 4 game.
- Paper **không có bảng ablation số liệu** cho riêng entropy regularization hay cho riêng việc chia sẻ tham số giữa policy/value; các thiết kế này được mô tả là lựa chọn kiến trúc, không đo tách riêng ảnh hưởng định lượng ("paper không nêu số" cho ảnh hưởng riêng của entropy term hay chia sẻ tham số).

6. Giới hạn

Tác giả nêu:

- Chưa so sánh công bằng hiệu năng giữa các hạ tầng tính toán rất khác nhau (CPU đa luồng so GPU so 100 máy phân tán của Gorila); tác giả chọn chỉ so sánh biến thể thuật toán, để câu hỏi về khả năng mở rộng/song song hóa cho nghiên cứu sau.
- Chưa tích hợp các cải tiến đã biết của DQN (Double Q-learning, Dueling) vào các biến thể value-based bất đồng bộ ở đây, dù tác giả tin có thể cải thiện thêm.
- n-step của A3C dùng forward view (cần đợi tới $t_{\max}$ hoặc terminal mới cập nhật); backward view qua eligibility trace linh hoạt hơn nhưng không dùng trong bài, để hướng mở.
- Dùng advantage ước lượng bằng n-step return đơn giản; tác giả tự gợi ý GAE (Schulman et al., tức bài 05 của lộ trình) là một cách ước lượng advantage tốt hơn, chưa thử trong bài này.

Nhận xét:

- Cách chia gradient/update giữa các luồng dùng kiểu Hogwild! (ghi đè không khóa) không có đảm bảo lý thuyết hội tụ cho mạng sâu phi tuyến (chỉ có cho trường hợp tuyến tính, theo Recht et al. 2011 được paper trích); kết quả ổn định là thực nghiệm, không phải định lý.
- Giá trị gradient-norm clipping được nói là "đã tinh chỉnh (tuned)" nhưng **không công bố giá trị cụ thể** trong text ("paper không nêu số"); người muốn tái lập chính xác sẽ phải tự dò.
- Kết quả Atari chính (Table 1) chỉ báo **human starts**; bài báo không có bảng no-op starts cho A3C trên 57 game (khác bài 01-03, đều có cả hai chế độ). Điều này được chính Rainbow (bài 03) xác nhận: bảng raw-score no-ops của Rainbow ghi rõ "A3C is not listed since the paper did not report the scores for the no-ops regime".
- So sánh tăng tốc siêu tuyến tính (Table 2) chỉ trung bình trên 7 game cụ thể; không rõ có tổng quát cho toàn bộ 57 game hay chỉ đúng với nhóm game đã chọn để tinh chỉnh siêu tham số.

7. Liên hệ với các bài khác

- **Bài 01-03 (DQN, Double DQN, Rainbow):** cùng chia sẻ bài toán Atari và một phần ký hiệu Q-learning; khác biệt cốt lõi là cơ chế ổn định hóa (replay buffer, off-policy) so với (nhiều luồng song song, cho phép on-policy). Rainbow (bài 03) dùng A3C của bài này làm một baseline so sánh.
- **Bài 05 (GAE):** chính bài báo này tự gợi ý GAE là cách ước lượng advantage tốt hơn n-step return đơn giản đang dùng; đọc GAE ngay sau để thấy cải tiến trực tiếp.

- **Bài 07 (PPO):** PPO kế thừa kiến trúc actor-critic song song (nhiều worker thu rollout) và advantage estimate của A3C/GAE, nhưng thay cách cập nhật policy bằng trust region/clipping để ổn định hơn ở bước lớn; nhiều cài đặt PPO hiện đại (nhiều môi trường song song, không dùng replay) thừa hưởng trực tiếp ý tưởng "nhiều actor song song thay replay" từ bài này.
- **Bài 36 (DeepSeekMath/GRPO):** GRPO cũng là một dạng actor-critic không cần critic riêng (dùng trung bình nhóm làm baseline); tư tưởng "nhiều rollout song song trong một batch, không cần replay buffer" của RLHF/RLVR hiện đại cho LLM có cùng động lực decorrelation/on-policy như A3C, dù cơ chế song song hóa hoàn toàn khác (nhiều rollout trong một batch đồng bộ, không phải nhiều luồng bất đồng bộ cập nhật liên tục).
- **Ngoài lộ trình RL:** entropy regularization ($\beta H(\pi)$) ở đây là tiền thân trực tiếp của entropy bonus/KL penalty dùng rộng rãi trong RLHF cho LLM (xem bài 00) để tránh policy sụp đổ quá sớm về một phân phối hẹp.

8. Ứng dụng thực tế (suy luận)

Mức liên quan đến huấn luyện RL/RLHF cho LLM hiện đại: **trung bình đến cao**, vì kiến trúc "nhiều actor/rollout song song, on-policy, không replay" chính là mô hình chuẩn của PPO/GRPO ngày nay (dù cơ chế song song hóa khác: batch đồng bộ nhiều rollout thay vì nhiều luồng bất đồng bộ):

- **Entropy regularization vẫn là một công cụ sống:** hệ số β cho entropy bonus ở đây (A3C dùng $\beta = 0.01$ cho Atari/TORCS) là tổ tiên trực tiếp của các kỹ thuật chống "policy collapse"/giảm đa dạng quá sớm trong RLHF/RLVR cho LLM; khi huấn luyện RL cho LLM thấy entropy giảm nhanh bất thường (mode collapse, lặp câu trả lời), đây là cùng một lớp vấn đề mà entropy bonus của A3C nhằm giải quyết từ 2016.
- **"Không cần replay" có điều kiện:** paper chỉ rõ việc này đúng khi có thể tương tác song song **rẻ** với nhiều bản copy môi trường; khi rollout đắt (LLM sinh văn bản dài, hoặc OCR suy luận trên ảnh lớn), kết hợp lại ý tưởng replay/tái sử dụng rollout cũ (off-policy có kiểm soát, như trong DAPO/GSPO ở các bài 41/43 của lộ trình) có thể vẫn cần thiết, đúng như gợi ý "kết hợp cả hai" mà chính A3C đã nêu.
- **Baseline/advantage có critic riêng (A3C) so với baseline nhóm (GRPO):** A3C học một mạng $V(s;\theta_v)$ riêng làm baseline; GRPO (bài 36) bỏ critic, dùng trung bình điểm thưởng của một nhóm rollout cho cùng prompt làm baseline. Đây là một điểm khác biệt thiết kế đáng so sánh khi lựa chọn thuật toán RL cho một quy trình cụ thể: có cần huấn luyện thêm một value network (tốn thêm bộ nhớ/tính toán, nhưng giảm phương sai tốt hơn nếu học đúng) hay dùng baseline đơn giản theo nhóm (rẻ hơn, nhưng cần nhiều rollout mỗi nhóm để baseline đủ ổn định).

9. Thí nghiệm gợi ý

1. Cài lại Algorithm (A3C, mục 3.3) trên một bài toán control nhỏ (CartPole hoặc một game Atari đơn giản), quét số luồng $\{1, 2, 4, 8, 16\}$ trên CPU, đo tốc độ tăng theo thời gian thực và theo số khung hình, để tái hiện Bảng 2 (lưu ý A3C có tăng tốc **dưới** tuyến tính, không siêu tuyến tính như 1-step Q).
2. So sánh có/không entropy bonus ($\beta = 0, 0.01, 0.1$) trên một game dễ hội tụ sớm về chính sách hẹp (ví dụ một game có vài hành động chiếm ưu thế rõ), đo tốc độ giảm entropy của policy theo thời gian huấn luyện.
3. Thay n-step advantage bằng GAE (bài 05) trong đúng cài đặt A3C ở trên, so sánh tốc độ học và phương sai gradient, để tự kiểm chứng gợi ý cải tiến mà chính bài báo này đưa ra.
4. Thử A3C với $t_{\max}$ khác nhau ($5, 20, 50$) trên một game cần chiến lược dài hạn (ví dụ Montezuma's Revenge hoặc game cần giữ trạng thái lâu), quan sát đánh đổi giữa phương sai advantage và độ trễ cập nhật.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Target 1-step Q bất đồng bộ	$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$	dùng target network chia sẻ
n-step return	$r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_a Q(s_{t+n}, a)$	forward view
Advantage A3C	$\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}) - V(s_t)$	$k \leq t_{\max}$
Cập nhật policy	$\nabla \log \pi(a_t \mid s_t; \theta) A(s_t, a_t) + \beta \nabla H(\pi)$	$\beta = 0.01$ (Atari, TORCS)
RMSProp chia sẻ	$g = \alpha g + (1-\alpha)\Delta\theta^2$, $\theta \leftarrow \theta - \eta \Delta\theta / \sqrt{g + \epsilon}$	g dùng chung giữa luồng
Tại sao không cần replay	nhiều actor song song tự decorrelate dữ liệu	mở khóa on-policy (Sarsa, actor-critic)
Siêu tham số chính	$t_{\max} = 5$, 16 luồng, $\gamma = 0.99$, RMSProp decay $\alpha = 0.99$, $\beta = 0.01$	mục 3.4, phụ lục
ϵ (value-based)	mỗi luồng bốc 1 trong 3 lịch ủ (xác suất $0.4/0.3/0.3$), ủ $1 \to \{0.1, 0.01, 0.5\}$	trong 4M khung đầu
Kết quả chính	A3C LSTM: mean 623.0%, median 112.6%, 4 ngày CPU	so DQN 121.9%/47.5%, 8 ngày GPU
Entropy liên tục	$-\frac{1}{2}(\log(2\pi\sigma^2) + 1)$, hệ số 10^{-4}	MuJoCo

11. Kết luận ngắn

Bài này chứng minh rằng chạy nhiều actor-learner song song trên nhiều bản copy môi trường, cập nhật bất đồng bộ một tham số chia sẻ, có thể thay thế experience replay như cơ chế ổn định hóa RL sâu, với lợi ích kèm theo là mở khóa các thuật toán on-policy (Sarsa, actor-critic) chạy ổn định cùng mạng sâu. Biến thể tốt nhất, A3C (advantage actor-critic với n-step return và entropy regularization), đạt state-of-the-art Atari khi đó bằng một nửa thời gian huấn luyện của các phương pháp dựa GPU, chỉ dùng CPU đa luồng, và tổng quát tốt sang điều khiển liên tục (MuJoCo, TORCS) và một môi trường 3D học từ pixel (Labyrinth). Những ý tưởng cốt lõi ở đây (nhiều rollout song song, on-policy, advantage có baseline, entropy regularization) là tiền thân trực tiếp của các thuật toán RL hậu huấn luyện LLM hiện đại.

Link gốc: https://arxiv.org/abs/1602.01783

Bài 07

Proximal Policy Optimization Algorithms (2017)

Tác giả: John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov (OpenAI).
Venue / arXiv: arXiv:1707.06347 (v2, 28/08/2017); bài báo không nêu venue (chỉ có nhãn arXiv ở đầu trang, không có dòng hội nghị/tạp chí).
Nguồn: https://arxiv.org/abs/1707.06347
Chủ đề: thay ràng buộc cứng của TRPO (conjugate gradient + line search) bằng một **surrogate objective clip** đơn giản, tối ưu bằng nhiều epoch SGD/Adam bậc một trên cùng một batch rollout; kèm một biến thể hình phạt KL thích ứng.
Vị trí trong lộ trình: đọc sau bài 04 (TRPO, cung cấp surrogate objective và cơ chế ràng buộc KL mà PPO kế thừa) và bài 05 (GAE, cung cấp advantage estimator mà PPO dùng trực tiếp dưới dạng rút gọn). Đọc trước bài 08, 09 (các chi tiết cài đặt quyết định hiệu suất PPO/TRPO), bài 22 (MAPPO), và cả nhóm RLHF (23-33)/RLVR (34-44), vì PPO là bộ tối ưu nền cho hầu hết các quy trình đó: llm 07 (InstructGPT) dùng PPO trực tiếp, bài 28 (N+ implementation details of RLHF with PPO) soi kỹ PPO cho RLHF, bài 36 (DeepSeekMath/GRPO) là một biến thể bỏ critic của chính PPO ở đây.
Chương nền liên quan: nền 03: Policy gradient và baseline, nền 05: Vùng tin cậy và PPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Ba hướng chính cho RL với mạng neural trước PPO: deep Q-learning (không hoạt động tốt trên điều khiển liên tục, còn hiểu chưa rõ); policy gradient "vanilla" (data efficiency và độ ổn định kém); TRPO (bài 04, data-efficient và ổn định nhưng phức tạp: cần conjugate gradient, Fisher-vector product, line search; không tương thích kiến trúc có nhiễu như dropout, hay chia sẻ tham số giữa policy và value network). Mục tiêu của PPO: đạt độ tin cậy/data-efficiency của TRPO **chỉ dùng tối ưu bậc một** (SGD/Adam thường, không CG).

Ước lượng policy gradient chuẩn (công thức 1-2): $\hat{g} = \hat{\mathbb{E}}_t[\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t]$, lấy từ đạo hàm của $L^{PG}(\theta) = \hat{\mathbb{E}}_t[\log \pi_\theta(a_t|s_t)\hat{A}_t]$. Paper nhấn mạnh: làm **nhiều bước** tối ưu trên L^{PG} với cùng dữ liệu "không có cơ sở lý thuyết tốt", và thực nghiệm thường dẫn tới cập nhật policy quá lớn, mang tính phá hoại.

Surrogate và ràng buộc của TRPO (công thức 3-4, bài 04): $\max_\theta \hat{\mathbb{E}}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t]$ với ràng buộc $\hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{old}}, \pi_\theta]] \le \delta$. Lý thuyết TRPO thực ra gợi ý dùng **hình phạt** thay ràng buộc (công thức 5): $\max_\theta \hat{\mathbb{E}}_t[\frac{\pi_\theta}{\pi_{\theta_{old}}}\hat{A}_t - \beta\,\mathrm{KL}[\pi_{\theta_{old}}, \pi_\theta]]$, nhưng TRPO dùng ràng buộc cứng vì khó chọn một β cố định hoạt động tốt trên nhiều bài toán, hay ngay cả trong một bài toán khi đặc tính học thay đổi theo thời gian.

Khái niệm: đặt $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ (tỉ số xác suất, $r_t(\theta_{old}) = 1$). $L^{CPI}(\theta) = \hat{\mathbb{E}}_t[r_t(\theta)\hat{A}_t]$ là mục tiêu của **conservative policy iteration** (Kakade & Langford 2002): chính là L^{CPI} không ràng buộc sẽ dẫn tới bước đi quá lớn.

2. Ý tưởng chính

Giữ tỉ số $r_t(\theta)$ của TRPO, nhưng thay ràng buộc KL giải bằng CG bởi một phép **clip** trực tiếp lên hàm mục tiêu: cắt $r_t(\theta)$ vào khoảng $[1-\epsilon, 1+\epsilon]$ rồi lấy **giá trị nhỏ hơn** giữa bản gốc và bản clip. Đây tạo ra một cận dưới (pessimistic bound) của L^{CPI}: khi di chuyển r_t ra xa 1 đang **cải thiện** mục tiêu (gốc) vượt quá ϵ, PPO loại bỏ động lực đi xa hơn (gradient phẳng); nhưng khi di chuyển r_t ra xa 1 làm mục tiêu **xấu đi**, PPO không clip, gradient vẫn kéo r_t trở lại. Mẹo min/clip này, tối ưu bằng nhiều epoch SGD/Adam thường trên

cùng batch, thực nghiệm cho hiệu suất khớp hoặc vượt TRPO, trong khi đơn giản hơn nhiều (vài dòng code thay đổi so với policy gradient thường) và tương thích mọi kiến trúc.

3. Phương pháp chi tiết

3.1 Mục tiêu clip L^{CLIP} và phân tích từng trường hợp

$$\boxed{L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\Big]} \tag{7}$$

với ϵ là siêu tham số (bài báo dùng $\epsilon = 0.2$ mặc định). Bốn trường hợp (dấu advantage × vị trí r_t):

Advantage	Vị trí $r_t(\theta)$	min chọn nhánh nào	Gradient hiệu quả
$\hat{A}_t > 0$	$r_t \le 1+\epsilon$ (chưa vượt trần)	$r_t\hat{A}_t$ (= L^{CPI}, chưa clip)	bình thường, tiếp tục tăng $\pi_\theta(a_t \mid s_t)$
$\hat{A}_t > 0$	$r_t > 1+\epsilon$ (đã tăng xác suất quá nhiều)	$(1+\epsilon)\hat{A}_t$ (hằng số theo θ tại điểm hiện tại)	≈ 0, dừng đẩy xác suất lên thêm
$\hat{A}_t < 0$	$r_t \ge 1-\epsilon$ (chưa vượt sàn)	$r_t\hat{A}_t$ (= L^{CPI}, chưa clip)	bình thường, tiếp tục giảm $\pi_\theta(a_t \mid s_t)$
$\hat{A}_t < 0$	$r_t < 1-\epsilon$ (đã giảm xác suất quá nhiều)	$(1-\epsilon)\hat{A}_t$ (hằng số)	≈ 0, dừng đẩy xác suất xuống thêm

Quy tắc cốt lõi: clip chỉ loại bỏ động lực đi **xa hơn** theo hướng đang **cải thiện** mục tiêu vượt quá ϵ; nó **không bao giờ** clip khi r_t đang đi theo hướng làm mục tiêu xấu đi (ví dụ $\hat{A}_t > 0$ nhưng $r_t < 1-\epsilon$: xác suất đã tụt dù hành động tốt, gradient vẫn đẩy r_t lên lại). Vì min luôn chọn giá trị nhỏ hơn, L^{CLIP} là cận dưới (pessimistic) của L^{CPI}. $L^{CLIP}(\theta) = L^{CPI}(\theta)$ tới bậc một tại θ_{old} (nơi $r = 1$), chỉ khác nhau khi θ rời xa θ_{old}, giống tính chất khớp bậc một của $L_{\pi_{old}}$ trong TRPO (bài 04, §3.2).

Hình 1 của bài báo minh họa một số hạng (một bước thời gian) của L^{CLIP} theo r: với $\hat{A}_t > 0$, L^{CLIP} tuyến tính tới $r = 1+\epsilon$ rồi phẳng; với $\hat{A}_t < 0$, phẳng dưới $r = 1-\epsilon$ rồi tuyến tính. Hình 2 cho thấy dọc theo hướng cập nhật thật của PPO trên Hopper, L^{CLIP} đạt cực đại gần điểm KL ≈ 0.02 và luôn là cận dưới của L^{CPI}, xác nhận thực nghiệm rằng clip phạt các bước lớn tương tự ràng buộc KL, mà không cần giải ràng buộc đó.

3.2 Biến thể hình phạt KL thích ứng (§4)

$$L^{KLPEN}(\theta) = \hat{\mathbb{E}}_t\Big[r_t(\theta)\hat{A}_t - \beta\,\text{KL}[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]\Big] \tag{8}$$

Sau mỗi lần cập nhật, tính $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{old}}, \pi_\theta]]$ và điều chỉnh β: nếu $d < d_{targ}/1.5$ thì $\beta \leftarrow \beta/2$; nếu $d > d_{targ} \times 1.5$ thì $\beta \leftarrow \beta \times 2$; ngược lại giữ nguyên. Hệ số 1.5 và 2 chọn theo kinh nghiệm nhưng thuật toán "không nhạy với chúng"; β khởi tạo không quan trọng vì tự điều chỉnh nhanh. Paper nêu rõ: bản clip cho kết quả tốt hơn bản KL-penalty trong thực nghiệm của họ, nhưng vẫn đưa KLPEN vào vì là một baseline quan trọng (đồng thời phản ánh chính lựa chọn "ràng buộc cứng thay hình phạt" mà TRPO đã làm).

3.3 Hàm loss tổng hợp cho actor-critic (công thức 9)

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\Big[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\Big], \quad L_t^{VF} = (V_\theta(s_t) - V_t^{targ})^2$$

S là entropy bonus (khuyến khích khám phá), c_1, c_2 là hệ số. Vì ta **tối đa hóa** toàn bộ biểu thức: dấu trừ trước $c_1 L^{VF}$ nghĩa là tối thiểu hóa sai số value function (tối đa hóa "âm sai số"); dấu cộng trước $c_2 S$ nghĩa là tối đa hóa entropy. Khi không chia sẻ tham số giữa policy và value network (như thí nghiệm điều khiển liên tục của bài báo), c_1 không liên quan (value net train riêng); thí nghiệm đó cũng không dùng entropy bonus (nhiễu Gauss liên tục của action đã đủ khám phá).

3.4 Advantage estimator: GAE rút gọn hữu hạn (công thức 10-12)

Dạng n-bước cố định độ dài T (không phải cả episode, kiểu Mnih và đồng nghiệp 2016):

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V(s_T) \tag{10}$$

(bài báo viết số mũ của hai số hạng cuối là $T-t+1$; theo đúng công thức k-bước $\hat{A}^{(k)}$ của bài 05 với $k = T-t$ bước từ t đến $T-1$, số mũ đúng phải là $T-t-1$ cho r_{T-1} và $T-t$ cho $V(s_T)$, người ghi chú sửa theo bài 05 để nhất quán và ghi chú lại đây.) Tổng quát hóa thành **GAE rút gọn** (bằng $\lambda = 1$ thì trở lại (10)):

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t-1}\delta_{T-1}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{11}$$

(cùng kiểu lệch số mũ cuối như (10) trong văn bản paper, sửa tương tự.) Đây chính xác là công thức GAE(γ, λ) của bài 05 (§3.3 ở bài đó), **cắt tại horizon** T và bootstrap bằng $V(s_T)$ (khác với ví dụ "tự tính" ở bài 05 dùng trạng thái kết thúc tự nhiên, $V = 0$).

3.5 Thuật toán (Algorithm 1, PPO kiểu Actor-Critic)

```
for iteration = 1, 2, ...:
    for actor = 1, ..., N (chay song song):
        chay pi_theta_old trong moi truong T buoc thoi gian
        tinh advantage A_hat_1, ..., A_hat_T   (GAE rut gon, muc 3.4)
    xay surrogate loss tren N*T mau
    toi uu surrogate (K epoch, minibatch size M <= N*T, SGD/Adam)
    theta_old <- theta
```

3.6 So sánh các biến thể surrogate (§6.1, dùng cho ablation mục 5)

Biến thể	Công thức $L_t(\theta)$
Không clip, không penalty	$r_t(\theta)\hat{A}_t$
Clipping	$\min(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)$
KL penalty (cố định hoặc thích ứng)	$r_t(\theta)\hat{A}_t - \beta\,\text{KL}[\pi_{\theta_{old}}, \pi_\theta]$

(Không có trong bảng nhưng là điểm tham chiếu: TRPO giải đúng ràng buộc cứng KL $\leq \delta$ cho L^{CPI} bằng CG, thay vì tối ưu trực tiếp một trong ba dòng trên bằng SGD.)

4. Thí nghiệm và kết quả chính

§6.1, ablation điều khiển liên tục: 7 môi trường MuJoCo/OpenAI Gym (HalfCheetah, Hopper, InvertedDoublePendulum, InvertedPendulum, Reacher, Swimmer, Walker2d, bản "-v1"), 1 triệu timestep huấn luyện mỗi môi trường, 3 seed, điểm chuẩn hóa (policy ngẫu nhiên = 0, kết quả tốt nhất = 1), trung bình trên 21 lần chạy (7 môi trường $\times$ 3 seed) cho mỗi cấu hình. MLP 2 lớp ẩn 64 unit tanh, không chia sẻ tham số policy/value (c_1 không liên quan), không entropy bonus.

Thuật toán	Điểm chuẩn hóa trung bình
Không clip, không penalty	-0.39
Clip $\epsilon = 0.1$	0.76
Clip $\epsilon = 0.2$	0.82

Thuật toán	Điểm chuẩn hóa trung bình
Clip $\epsilon = 0.3$	0.70
Adaptive KL $d_{targ} = 0.003$	0.68
Adaptive KL $d_{targ} = 0.01$	0.74
Adaptive KL $d_{targ} = 0.03$	0.71
Fixed KL $\beta = 0.3$	0.62
Fixed KL $\beta = 1$	0.71
Fixed KL $\beta = 3$	0.72
Fixed KL $\beta = 10$	0.69

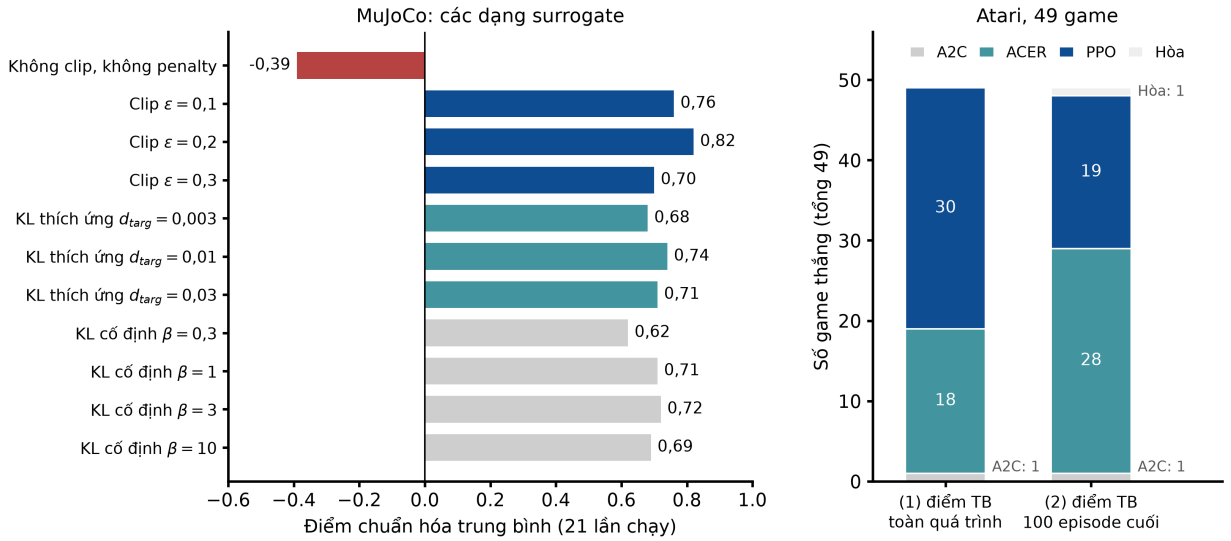
Không-clip-không-penalty cho điểm **âm** vì trên HalfCheetah nó cho điểm rất tiêu cực, kém hơn cả policy ngẫu nhiên.

§6.2: so PPO (clip $\epsilon = 0.2$) với TRPO, CEM, vanilla PG (stepsize thích ứng), A2C, A2C+trust region trên cùng 7 môi trường; PPO vượt các phương pháp trước trên "hầu hết" môi trường (nguyên văn paper, chỉ có đường cong Hình 3, không có bảng số thêm).

§6.3: showcase humanoid 3D Roboschool (chạy, đổi hướng, đứng lên khi bị ném vật), chỉ có đường cong (Hình 4), siêu tham số ở Bảng 4.

§6.4, Atari 49 game, so với A2C và ACER:

Thước đo	A2C	ACER	PPO	Tie
(1) Điểm TB toàn bộ quá trình train (ưu tiên học nhanh)	1	18	30	0
(2) Điểm TB 100 episode cuối (ưu tiên hiệu suất cuối)	1	28	19	1



Hình 35: So sánh các dạng surrogate trên MuJoCo (điểm chuẩn hóa trung bình) và số game thắng trên Atari của A2C, ACER, PPO (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: bên trái, thanh càng dài càng tốt; clip $\epsilon = 0{,}2$ cao nhất (0,82) còn bản không clip không penalty xuống âm (-0,39). Bên phải, mỗi cột xếp chồng đủ 49 game: PPO thắng nhiều nhất ở thước đo (1) học nhanh (30 game) nhưng ACER thắng nhiều hơn ở thước đo (2) điểm cuối (28 so với 19).

Siêu tham số:

	Bảng 3 (MuJoCo 1M)	Bảng 4 (Roboschool)	Bảng 5 (Atari)
Horizon T	2048	512	128
Adam stepsize	3×10^{-4}	tự chỉnh theo target KL	$2.5 \times 10^{-4} \times \alpha$
Số epoch	10	15	3
Minibatch	64	4096	32×8
γ	0.99	0.99	0.99
λ (GAE)	0.95	0.95	0.95
Số actor	-	32 (locomotion)/128 (flagrun)	8
Khác	-	log-stdev anneal $-0.7 \to -1.6$	$\epsilon = 0.1 \times \alpha$, $c_1 = 1$, $c_2 = 0.01$, α anneal $1 \to 0$

5. Phân tích / ablation

- **Clip vs KL:** tốt nhất toàn cục là Clip $\epsilon = 0.2$ (0.82), vượt adaptive KL tốt nhất ($d_{targ} = 0.01$, 0.74) và fixed KL tốt nhất ($\beta = 3$, 0.72); không-clip-không-penalty sụp hoàn toàn (-0.39, do một môi trường tệ).
- **Atari, hai thước đo mâu thuẫn:** PPO thắng nhiều hơn ở thước đo (1) "học nhanh" (30 so với 18 của ACER, 1 của A2C) nhưng thua ACER ở thước đo (2) "hiệu suất cuối" (19 so với 28); ACER dùng experience replay nên có thể hiệu suất tiệm cận tốt hơn trên một số game dù học chậm hơn ban đầu. Không nên kết luận PPO "thắng tuyệt đối" trên Atari, chỉ thắng rõ ở tốc độ học.
- **Động lực của toàn bộ thiết kế clip:** chính là để làm cho việc dùng **nhiều epoch** SGD trên cùng batch trở nên an toàn, điều mà L^{PG} thường (công thức 2) "không có cơ sở lý thuyết tốt" và thực nghiệm dẫn tới cập nhật phá hoại (bài báo nêu trực tiếp ở §2.1).

6. Giới hạn

Tác giả nêu: KL-penalty cho kết quả kém hơn clip trong thí nghiệm của họ (chỉ giữ lại như baseline quan trọng); ϵ được chọn qua dò (không thích ứng) và không giống nhau giữa các domain (0.1-0.3 ở MuJoCo, $0.1 \times \alpha$ ở Atari); siêu tham số A2C/ACER trên Atari được tinh chỉnh riêng cho benchmark đó, không hoàn toàn cùng protocol dò tham số với PPO (PPO dò trên benchmark MuJoCo rẻ hơn rồi dùng lại $\epsilon = 0.2$ cho so sánh continuous-domain).

Nhận xét: PPO **không** có một định lý monotonic-improvement tương tự Theorem 1 của TRPO (bài 04); đây là một xấp xỉ kỹ thuật hoạt động tốt thực nghiệm nhưng từ bỏ hoàn toàn cận lý thuyết chặt. Các lỗi đánh số mũ ở công thức (10)-(11) (mục 3.4, đã sửa và ghi chú ở trên) cho thấy phần công thức của bài này được viết kém cẩn thận hơn phần chứng minh ở bài 04/05, cần đối chiếu với bài 05 khi đọc. Bảng siêu tham số (3,4,5) khác nhau khá nhiều giữa domain (epoch, minibatch, ϵ), nên tuyên bố "PPO hoạt động tốt ngay mà không cần chỉnh" cần hiểu có mức độ: vẫn có dò tham số hợp lý theo lớp bài toán.

7. Liên hệ với các bài khác

- **Bài 04 (TRPO):** PPO kế thừa trực tiếp L^{CPI} và so sánh xuyên suốt; clip $\approx$ "trust region mềm" thay cho ràng buộc KL cứng + CG/line-search.
- **Bài 05 (GAE):** advantage estimator mặc định của PPO chính là GAE(γ, λ) rút gọn hữu hạn (§3.4 ở trên); $\lambda = 0.95, \gamma = 0.99$ dùng xuyên suốt các bảng siêu tham số.
- **Bài 06 (A3C/A2C):** A2C là baseline so sánh chính; PPO vượt rõ về sample efficiency trên Atari.

- **Bài 08, 09:** nghiên cứu sâu các lựa chọn cài đặt (không chỉ công thức) của PPO/TRPO, đọc bổ sung thiết yếu.
- **Bài 22 (MAPPO):** PPO gần như không sửa đổi vẫn hiệu quả bất ngờ trong multi-agent hợp tác.
- **Bài 28 (N+ implementation details RLHF/PPO), 33 (DPO vs PPO):** nghiên cứu PPO cụ thể như optimizer cho RLHF, với nhiều mẹo bổ sung (chuẩn hóa reward, clip value...) không có trong bài gốc này.
- **Bài 36 (DeepSeekMath/GRPO):** giữ nguyên cơ chế clip tỉ số $r_t(\theta)$ với ϵ, nhưng **bỏ hẳn** value function/critic, thay $\hat{A}_t$ bằng advantage tương đối theo nhóm hoàn thành cho cùng một prompt, trực tiếp giải quyết chi phí critic mà bài 05 đã phân tích.
- **llm 07 (InstructGPT):** quy trình RLHF SFT $\rightarrow$ mô hình phần thưởng $\rightarrow$ PPO, dùng đúng dạng $L^{CLIP+VF+S}$ (thêm hình phạt KL so với policy SFT tham chiếu và loss pretraining-mixing PPO-ptx).

8. Ứng dụng thực tế (suy luận)

PPO (bản clip) là bộ tối ưu nền cho gần như mọi quy trình RLHF/RLVR hiện đại (InstructGPT llm 07, biến thể GRPO ở DeepSeekMath, bài 36, và DeepSeek-R1, bài 39; Tulu 3, bài 38...). Các siêu tham số quan trọng, nối tiếp từ bài này:

- **Khoảng clip ϵ:** mặc định $\epsilon = 0.2$ của MuJoCo/Atari chỉ là điểm khởi đầu tham khảo cho RLHF LLM; các bài sau (bài 28) tinh chỉnh thêm, và một số phương pháp dùng clip **không đối xứng** (clip-higher, ví dụ DAPO bài 41) để chống sụp entropy.
- **Số epoch K tái sử dụng rollout:** dùng lại dữ liệu rollout quá nhiều epoch (dù có clip) vẫn có nguy cơ overfit vào phân phối rollout đã lỗi thời; với LLM, sinh rollout (decode tự hồi quy đầy đủ) đắt hơn MuJoCo rất nhiều, nên có áp lực tăng K, cần cân nhắc cùng độ an toàn của clip.
- **Chi phí critic:** số hạng $c_1 L^{VF}$ cần một value head/model riêng; ở quy mô LLM việc giữ một critic (thường cùng cỡ policy) gần như gấp đôi bộ nhớ, đúng là lý do GRPO (bài 36) bỏ hẳn nó.
- **Entropy bonus $c_2 S$:** quan trọng để tránh hội tụ sớm/mode collapse trong RLHF; hệ số quá cao có thể xung đột với tín hiệu reward từ con người; bài 49 (cơ chế entropy trong RL cho LM lý luận) bàn sâu hơn cho ngữ cảnh LLM.

9. Thí nghiệm gợi ý

1. Tái tạo so sánh 3 nhánh ở Bảng 1 (không clip, clip $\epsilon = 0.2$, adaptive KL) trên một môi trường điều khiển rẻ, thay đổi số epoch K (1, 4, 10, 20) mỗi batch để xem ở K nào baseline không-clip bắt đầu mất ổn định còn clip/KL-penalty vẫn giữ được.
2. Cài cả hai mục tiêu clip và adaptive-KL trên cùng codebase, vẽ trực tiếp KL divergence mỗi bước cập nhật trong suốt quá trình train, kiểm tra claim của Hình 2 rằng cả hai giữ KL trong khoảng tương tự.
3. Ablate λ của GAE dùng trong PPO ($\lambda = 0, 0.9, 0.95, 1.0$) khi giữ cố định mục tiêu clip, nối trực tiếp với phân tích bias/variance ở bài 05.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Tỉ số xác suất	$r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{old}}(a_t \mid s_t)$	$r_t(\theta_{old}) = 1$
L^{PG}, L^{CPI}	$\hat{\mathbb{E}}[\log \pi_\theta \hat{A}_t]$, $\hat{\mathbb{E}}[r_t(\theta)\hat{A}_t]$	L^{CPI} không ràng buộc dẫn tới bước quá lớn
L^{CLIP}	$\hat{\mathbb{E}}_t[\min(r_t\hat{A}_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)]$	cận dưới của L^{CPI}, $\epsilon = 0.2$ mặc định
L^{KLPEN}	$\hat{\mathbb{E}}_t[r_t\hat{A}_t - \beta\,\text{KL}]$	β tự điều chỉnh theo d_{targ}
Loss tổng hợp	$L^{CLIP} - c_1 L^{VF} + c_2 S$	tối đa hóa; dấu trừ VF, dấu cộng entropy

Khái niệm	Công thức / ý	Ghi chú
Advantage	GAE rút gọn tại horizon T	công thức (11), bootstrap $V(s_T)$
Algorithm 1	N actor, T bước, K epoch, minibatch $M \leq NT$	lặp, đặt $\theta_{old} \leftarrow \theta$

11. Kết luận ngắn

PPO giữ nguyên tỉ số xác suất $r_t(\theta)$ và surrogate L^{CPI} của TRPO nhưng thay ràng buộc KL cứng (giải bằng conjugate gradient + line search) bằng một phép clip trực tiếp lên mục tiêu, cho một cận dưới an toàn để tái sử dụng dữ liệu rollout qua nhiều epoch SGD/Adam. Thực nghiệm trên điều khiển liên tục và Atari cho thấy clip ($\epsilon = 0.2$) vượt rõ biến thể không-ràng-buộc và các biến thể KL-penalty, với độ phức tạp cài đặt thấp hơn TRPO rất nhiều. Advantage estimator mặc định của PPO là GAE rút gọn (bài 05); các bài RLHF/RLVR sau (InstructGPT, GRPO...) đều xây trực tiếp trên bộ khung này, một số (GRPO, bài 36) còn đơn giản hóa thêm bằng cách bỏ hẳn value function.

Link gốc: https://arxiv.org/abs/1707.06347

Bài 08

Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO (2020)

Tác giả: Logan Engstrom, *Andrew Ilyas* (đóng góp ngang nhau), Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, Aleksander Mądry (MIT; Two Sigma). **Venue / arXiv:** arXiv:2005.12729 (v1, 25/05/2020). Paper tự ghi "Published as a conference paper at ICLR 2020". **Nguồn:** https://arxiv.org/abs/2005.12729 **Chủ đề:** Tách bạch "thuật toán cốt lõi" (TRPO so với PPO) khỏi các "code-level optimization" (các mẹo cấy vào code, không nằm trong công bố toán học của thuật toán), rồi đo bằng ablation xem phần nào thực sự tạo ra hiệu năng và hành vi trust-region của PPO. **Vị trí trong lộ trình:** Đọc sau bài 00 (preface: trust region, KL penalty, PPO), 04 (TRPO), 05 (GAE), 07 (PPO). Bài 08 và 09 (What Matters in On-Policy RL) nên đọc liền nhau: cả hai cùng hỏi "đâu là phần thực sự quan trọng trong một thuật toán on-policy policy gradient", 09 mở rộng quy mô (250.000 agent) và bao trùm nhiều lựa chọn hơn. Bài 22 (MAPPO) áp lại đúng tinh thần này cho multi-agent. Các bài RLHF/PPO cho LLM (23-28, đặc biệt 28 "N+ Implementation Details of RLHF with PPO") là phần tiếp nối tư tưởng "implementation matters" sang miền ngôn ngữ.
Chương nền liên quan: nền 05: Vùng tin cậy và PPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Deep RL nổi tiếng là **brittle** (dễ vỡ): Henderson et al. (2017, "Deep RL that matters") đã chỉ ra rằng cùng một thuật toán cho kết quả rất khác nhau tùy codebase, seed, hyperparameter. Mania et al. (2018) còn cho thấy random search đơn giản đôi khi đọ được với PPO/TRPO trên benchmark MuJoCo. Bài này đặt câu hỏi cụ thể hơn: **PPO** (Proximal Policy Optimization, bài 07) được quảng bá là cải tiến của **TRPO** (Trust Region Policy Optimization, bài 04) nhờ cơ chế "clipping" thay cho ràng buộc KL cứng. Nhưng khi đọc kỹ các implementation chuẩn (OpenAI baselines) của PPO, tác giả phát hiện **9 "code-level optimization"**: những chỉnh sửa không (hoặc chỉ nhắc rất sơ) xuất hiện trong bài báo gốc của Schulman et al. (2017), nhưng luôn có trong code thật.

Khái niệm nền (nhắc lại từ bài 00/04/05/07):

- **Trust region:** ràng buộc để policy mới không đi quá xa policy cũ trong một bước cập nhật, vì ước lượng advantage/gradient chỉ còn chính xác trong một vùng lân cận nhỏ quanh policy hiện tại.
- **Surrogate objective:** hàm mục tiêu xấp xỉ (dùng importance ratio $\rho_t = \pi_\theta/\pi$) để tối ưu policy mới bằng dữ liệu thu thập từ policy cũ.
- **KL divergence D_{KL}:** độ "khác biệt" giữa hai phân phối xác suất; TRPO ràng buộc D_{KL} giữa policy mới và cũ.
- **Ratio clipping:** PPO cắt ρ_t vào $[1-\epsilon, 1+\epsilon]$ rồi lấy min với bản không cắt, thay cho ràng buộc KL cứng của TRPO.
- **Code-level optimization:** optimization chỉ tồn tại trong code (không phải "implementation choice" như PyTorch và TensorFlow), có chủ ý thay đổi cách thuật toán vận hành.

2. Ý tưởng chính

Tác giả định nghĩa một tập hợp các biến thể để "mổ xẻ" PPO: **PPO** (chuẩn, như OpenAI baselines), **PPO-M** (PPO-Minimal: chỉ phần lõi, bỏ hết 9 optimization), **PPO-NoClip** (bỏ cơ chế clip nhưng giữ optimization),

TRPO (chuẩn), **TRPO+** (TRPO cộng thêm các optimization của PPO). Bằng cách lai ghép 2 trục này (thuật toán lõi × có/không optimization), họ đo được: (a) optimization đóng góp bao nhiêu vào reward cuối cùng so với việc đổi thuật toán lõi; (b) optimization có làm thay đổi **hành vi trust-region** thực sự được duy trì hay không, độc lập với reward. Kết luận gây bất ngờ: optimization quan trọng hơn lựa chọn PPO-vs-TRPO, và ngay cả cơ chế clipping (được xem là "trái tim" của PPO) cũng không cần thiết để đạt hiệu năng cao nếu đã có optimization.

3. Phương pháp chi tiết

3.1 Chín code-level optimization tìm thấy trong PPO chuẩn

Paper liệt kê các optimization sau (mục 3 của bài báo), toàn bộ đều **không phụ thuộc thuật toán cụ thể** (có thể cấy vào TRPO hay bất kỳ policy-gradient nào):

1. **Value function clipping:** bài báo gốc của Schulman et al. chỉ đề xuất regression thường: $L^V = (V_{\theta_t} - V_{targ})^2$. Implementation chuẩn lại dùng một loss kiểu PPO cho cả value network:

$$\boxed{L^V = \max\left[(V_{\theta_t} - V_{targ})^2,\ \left(\text{clip}(V_{\theta_t},\ V_{\theta_{t-1}} - \epsilon,\ V_{\theta_{t-1}} + \epsilon) - V_{targ}\right)^2\right]}$$

   với ϵ dùng chung với ϵ cắt ratio trong loss chính sách.
2. **Reward scaling:** không đưa reward thô vào, mà chia reward cho độ lệch chuẩn (std) của một tổng chiết khấu chạy (rolling discounted sum) của reward, **không** trừ/cộng lại mean. Thuật toán (Appendix A.2 của bài báo, tên hàm giữ đúng như paper mặc dù tên có vẻ lạc đề "Scale-Observation" áp cho reward):

```
procedure INITIALIZE-SCALING():
    R0 <- 0
    RS <- RunningStatistics()          # theo dõi mean, std chạy

procedure SCALE-OBSERVATION(r_t):       # input: reward r_t tại bước t
    R_t <- gamma * R_{t-1} + r_t        # gamma = discount factor
    ADD(RS, R_t)
    return r_t / StandardDeviation(RS)  # trả về reward đã scale
```

3. **Orthogonal initialization + layer scaling:** dùng khởi tạo trực giao (orthogonal) thay vì mặc định, với hệ số scale khác nhau cho mỗi layer (layer cuối của policy thường scale nhỏ hơn nhiều).
4. **Adam learning rate annealing:** giảm dần learning rate của Adam (một optimizer đã thích nghi sẵn) theo tiến trình huấn luyện, tùy task.
5. **Reward clipping:** cắt reward vào khoảng cố định, thường $[-5, 5]$ hoặc $[-10, 10]$.
6. **Observation normalization:** chuẩn hóa state thô về mean 0, variance 1 trước khi đưa vào mạng.
7. **Observation clipping:** cắt state đã chuẩn hóa vào khoảng, thường $[-10, 10]$.
8. **Tanh activation:** dùng hàm hoạt hóa tanh giữa các layer của policy và value network (quan sát trước đó của Henderson et al.).
9. **Global gradient clipping:** sau khi tính gradient cho cả policy và value network, cắt theo "global ℓ_2 norm" (norm của toàn bộ gradient nối lại) để không vượt 0.5.

Bảng 1 của bài báo tóm tắt các biến thể nghiên cứu:

Thuật toán	Step method	Dùng PPO clipping?	Dùng PPO optimization?
PPO	PPO	có	có (như Dhariwal et al. 2017)
PPO-M	PPO	có	không
PPO-NoClip	PPO	không	có (tìm bằng grid search)
TRPO	TRPO	-	không

Thuật toán	Step method	Dùng PPO clipping?	Dùng PPO optimization?
TRPO+	TRPO	-	có (tìm bằng grid search, cộng thêm "KL decay" làm tương tự Adam lr annealing)

3.2 Mục tiêu TRPO và PPO (nhắc lại chính xác theo paper)

TRPO (eq. 1):

$$\max_{\theta} \mathbb{E}_{(s_t,a_t)\sim\pi}\left[\frac{\pi_\theta(a_t|s_t)}{\pi(a_t|s_t)}\hat{A}^{\pi}(s_t,a_t)\right] \quad \text{s.t. } D_{KL}\big(\pi_\theta(\cdot|s)\|\pi(\cdot|s)\big) \le \delta,\ \forall s$$

PPO (eq. 2, 3), với $\rho_t = \dfrac{\pi_\theta(a_t|s_t)}{\pi(a_t|s_t)}$:

$$\boxed{\max_{\theta} \mathbb{E}_{(s_t,a_t)\sim\pi}\Big[\min\big(\rho_t\hat{A}^{\pi}(s_t,a_t),\ \text{clip}(\rho_t,1-\epsilon,1+\epsilon)\,\hat{A}^{\pi}(s_t,a_t)\big)\Big]}$$

TRPO giải (1) bằng xấp xỉ bậc hai của D_{KL} cộng natural gradient descent (cần nhiều phép nhân Hessian-vector qua conjugate gradient, tốn kém). PPO "né" việc tính projection này bằng cách cắt trực tiếp ratio trong hàm mục tiêu.

3.3 Vì sao clipping không tự động đảm bảo trust region (Mục 4 của paper)

Đạo hàm của loss PPO theo từng cặp (s,a) có dạng (bài báo viết trực tiếp, không chứng minh chi tiết; L_θ là surrogate chưa cắt, L_θ^C là surrogate đã cắt):

$$\nabla_\theta L_{PPO} = \begin{cases} \nabla_\theta L_\theta, & \text{nếu } \rho_t \in [1-\epsilon,1+\epsilon] \text{ hoặc } L_\theta^C < L_\theta \\ 0, & \text{ngược lại} \end{cases}$$

với

$$L_\theta = \mathbb{E}_{(s,a)\sim\tau\sim\pi}\Big[\rho_t\, A^{\pi}(s,a)\Big], \qquad L_\theta^C = \mathbb{E}_{(s,a)\sim\tau\sim\pi}\Big[\text{clip}(\rho_t,1-\epsilon,1+\epsilon)\, A^{\pi}(s,a)\Big]$$

3.4 Chỉ số AAI / ACLI (Mục 5)

Để so sánh "đổi thuật toán lõi" với "thêm optimization", paper định nghĩa trên cùng một bộ 4 thuật toán (PPO, PPO-M, TRPO, TRPO+):

$$\boxed{\text{AAI} = \max\big(|\text{PPO} - \text{TRPO+}|,\ |\text{PPO-M} - \text{TRPO}|\big)} \qquad \boxed{\text{ACLI} = \max\big(|\text{PPO} - \text{PPO-M}|,\ |\text{TRPO+} - \text{TRPO}|\big)}$$

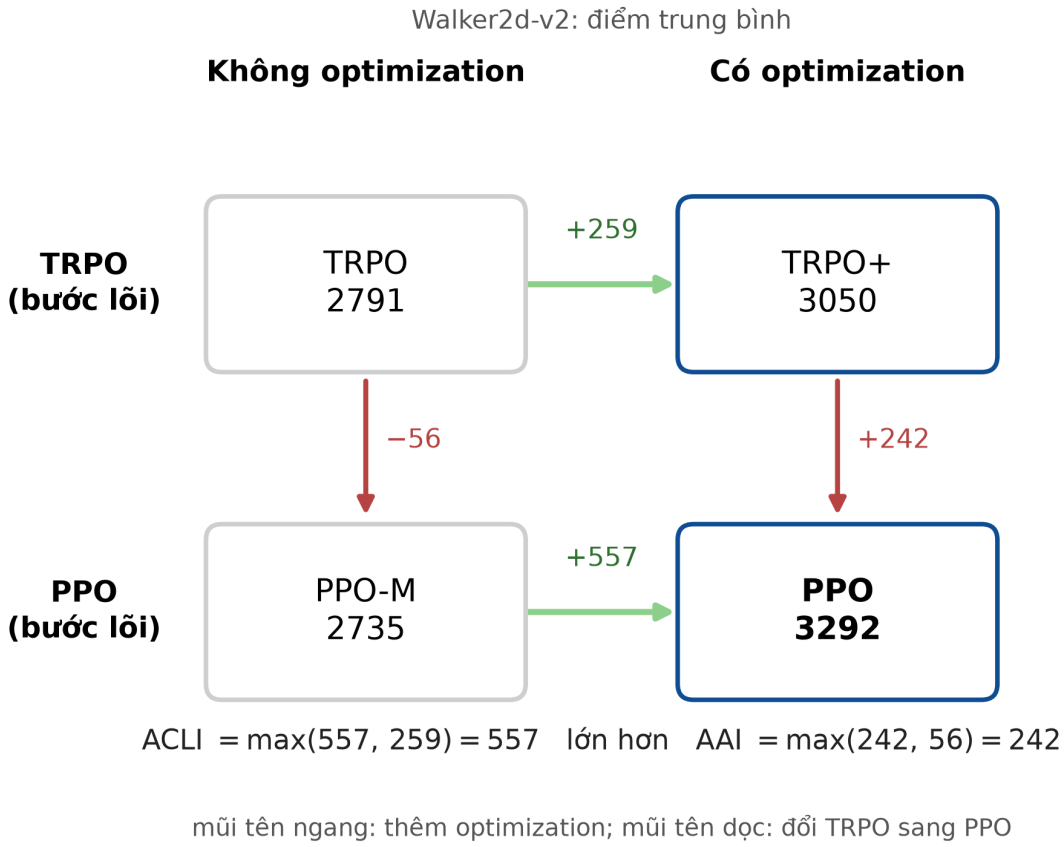
- **AAI** (Average Algorithmic Improvement): hiệu ứng tối đa của việc đổi bước cập nhật PPO $\leftrightarrow$ TRPO, *giữ cố định* có/không dùng optimization.
- **ACLI** (Average Code-Level Improvement): hiệu ứng tối đa của việc thêm optimization, *giữ cố định* thuật toán lõi.

Ví dụ tự tính (dùng đúng số liệu Bảng 2 của bài báo, cột Walker2d-v2): PPO = 3292, PPO-M = 2735, TRPO = 2791, TRPO+ = 3050.

$$\text{AAI} = \max(|3292-3050|,\ |2735-2791|) = \max(242,\ 56) = 242$$

$$\text{ACLI} = \max(|3292 - 2735|,\ |3050 - 2791|) = \max(557,\ 259) = 557$$

Hai số này khớp đúng với giá trị AAI = 242, ACLI = 557 mà bài báo báo cáo ở Bảng 2 cho Walker2d-v2 (tự tính, chỉ dùng công thức và số liệu đã công bố). Vì ACLI (557) lớn hơn AAI (242) rất nhiều, optimization "lợi" hơn việc chọn đúng thuật toán.



Hình 36: Lưới hai trục của bài báo trên Walker2d-v2: thuật toán lõi (TRPO, PPO) và có hay không optimization, kèm AAI và ACLI (minh hoạ, tự tính)

Cách đọc hình: mũi tên ngang là thêm optimization (+259 cho TRPO, +557 cho PPO), mũi tên dọc là đổi bước cập nhật từ TRPO sang PPO (-56 khi chưa có optimization, +242 khi đã có). ACLI lấy độ lớn tối đa theo chiều ngang và AAI theo chiều dọc, nên 557 so với 242 cho thấy optimization ảnh hưởng nhiều hơn việc chọn thuật toán lõi.

4. Thí nghiệm và kết quả chính

Môi trường: 3 task MuJoCo liên tục: Walker2d-v2, Hopper-v2, Humanoid-v2 (phần phụ lục có thêm Figure 3/4 chạy cả hai). Mỗi ước lượng dùng **ít nhất 80 agent** huấn luyện (nhiều hơn cho trường hợp phương sai cao), **95% CI** từ bootstrap 1000 mẫu. Hyperparameter cho mỗi thuật toán tìm bằng grid search riêng (chi tiết Phụ lục A.1, Bảng 4-6: ví dụ TRPO dùng KL constraint $\delta = 0.04$ (Walker2d), 0.07 (Humanoid), 0.13 (Hopper); PPO dùng $\epsilon = 0.2$, gradient clip ℓ_2-norm 0.5 ở Humanoid/Walker2d).

Ablation 4 optimization đầu tiên (Hình 1, do hạn chế compute chỉ ablate đủ 4/9): value clipping, reward scaling (norm_rewards), Adam lr annealing, orthogonal initialization, mỗi optimization bật/tắt, grid search learning rate cho mỗi trong $2^4 = 16$ tổ hợp, 5 seed mỗi tổ hợp, trên Humanoid-v2 và Walker2d-v2. Kết quả: **reward normalization, Adam annealing, và network initialization** mỗi cái đều ảnh hưởng rõ rệt đến phân phối reward theo hyperparameter và **cần thiết** để đạt reward PPO cao nhất trong lưới hyperparameter đã thử

(value clipping có mặt trong ablation nhưng không được nhấn mạnh là "cần thiết" như 3 cái kia trong câu kết luận của bài báo).

Bảng 2 (trích, mean [95% CI])

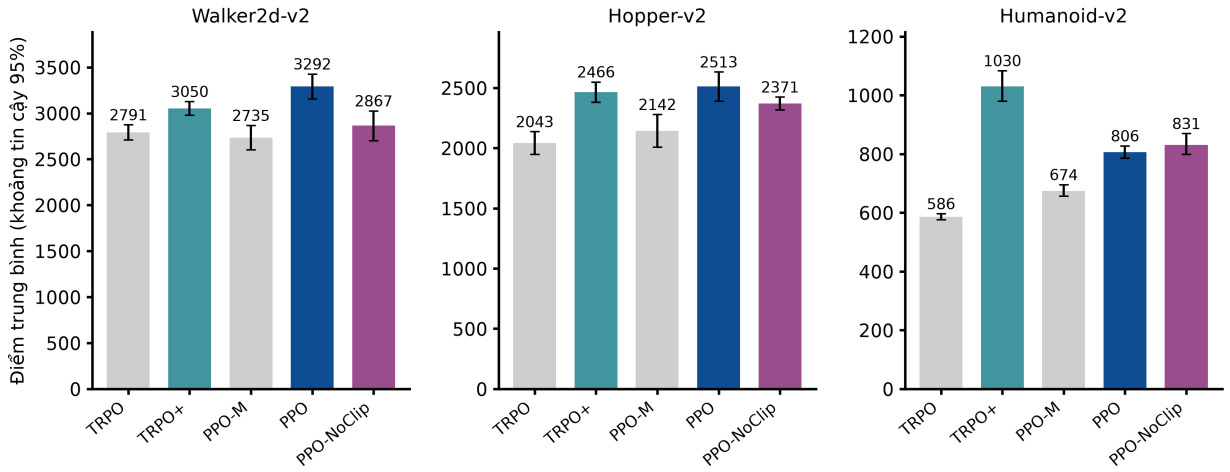
Step	Walker2d-v2	Hopper-v2	Humanoid-v2
PPO	3292 [3157, 3426]	2513 [2391, 2632]	806 [785, 827]
PPO-M	2735 [2602, 2866]	2142 [2008, 2279]	674 [656, 695]
TRPO	2791 [2709, 2873]	2043 [1948, 2136]	586 [576, 596]
TRPO+	3050 [2976, 3126]	2466 [2381, 2549]	1030 [979, 1083]
AAI	242	99	224
ACLI	557	421	444

Ví dụ cụ thể: trên Hopper, PPO và TRPO tăng tương ứng **17%** và **21%** khi được trang bị optimization (PPO-M $\rightarrow$ PPO; TRPO $\rightarrow$ TRPO+); nhưng khi đã cố định có/không optimization, đổi thuật toán lõi không gây khác biệt đáng kể. Đặc biệt TRPO+ còn vượt PPO trên Humanoid (1030 so với 806).

Bảng 3: PPO với và không có clipping

	Walker2d-v2	Hopper-v2	Humanoid-v2
PPO	3292 [3157, 3426]	2513 [2391, 2632]	806 [785, 827]
PPO (OpenAI baselines)	3424	2316	-
PPO-M	2735 [2602, 2866]	2142 [2008, 2279]	674 [656, 695]
PPO-NoClip	2867 [2701, 3024]	2371 [2316, 2424]	831 [798, 869]

PPO-NoClip (không có clipping nhưng có optimization, ϵ cắt ratio được set thực tế bằng 10^{32} tức vô hiệu hóa) luôn tốt hơn **PPO-M** (có clipping nhưng không optimization), và trên Humanoid còn hơi vượt PPO chuẩn. Clipping (được cho là trái tim của PPO) có vai trò bằng hoặc kém hơn optimization.



Hình 37: Điểm trung bình và khoảng tin cậy 95% của TRPO, TRPO+, PPO-M, PPO và PPO-NoClip trên ba môi trường MuJoCo (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi panel là một môi trường, thanh đen là khoảng tin cậy 95%. Hai cột xám (TRPO, PPO-M) không có optimization thấp hơn các cột tương ứng có optimization (TRPO+, PPO), và PPO-NoClip bỏ clip nhưng vẫn cao hơn PPO-M ở cả ba môi trường.

5. Phân tích / ablation

Trust region thực sự có được duy trì? (Hình 2, 3, 4) Đo trên Humanoid-v2 (và thêm Walker2d, Hopper ở phụ lục):

- **Ratio-based trust region ($\rho_t \in [1-\epsilon, 1+\epsilon]$):** cả 3 thuật toán PPO/PPO-M/TRPO đều **vi phạm** ("max ratio" vượt đường $1+\epsilon$ nhiều lần), mặc dù PPO và PPO-M được huấn luyện trực tiếp với mục tiêu cắt ratio này.
- **Mean-KL trust region:** TRPO giữ đúng gần như "theo cấu trúc" (gần khớp ngưỡng δ đặt trước, ví dụ 0.07 trên Humanoid). PPO và PPO-M cũng giữ mean KL ở mức tương đối nhỏ, nhưng **khác nhau về hình dạng**: KL của PPO-M tăng dần theo số iteration, còn KL của PPO tăng nửa đầu rồi giảm trở lại. Hai thuật toán có cùng cơ chế clip nhưng khác biệt hành vi KL chỉ vì có/không có optimization.
- Kết luận của tác giả: optimization không chỉ ảnh hưởng reward, mà **thay đổi cả tính chất thuật toán** (loại trust region nào thực sự được enforce), độc lập với bản thân objective được tối ưu.

6. Giới hạn

Tác giả nêu:

- Do hạn chế compute, chỉ ablate đầy đủ 4/9 optimization (Hình 1); phần còn lại (reward/obs clipping, tanh, global gradient clip) không có ablation đầy đủ riêng.
- Chỉ nghiên cứu trên 3 môi trường MuJoCo liên tục (Walker2d, Hopper, Humanoid); không có Atari, không có task rời rạc.
- PPO-NoClip chỉ là "tập con" của không gian mà PPO có thể biểu diễn (vì PPO còn có ϵ tự do), nên chưa chắc PPO-NoClip đã tối ưu tuyệt đối; có thể còn cải thiện thêm nếu tinh chỉnh code-level optimization sâu hơn.

Nhận xét:

- Bài không giải thích **vì sao** reward scaling, orthogonal init, lr annealing lại quan trọng về mặt lý thuyết; chỉ chứng minh bằng thực nghiệm rằng chúng quan trọng.
- Bài 09 (What Matters in On-Policy RL, quy mô 250.000 agent) sau này thấy **PPO-style value clipping thực ra có hại** trên 5/5 môi trường Gym họ thử, trái với việc bài 08 liệt kê value clipping là một optimization "chuẩn" của PPO (dù bài 08 không khẳng định nó quan trọng bằng 3 optimization kia). Đây là một điểm cần đọc chéo cẩn thận, không nên coi mọi optimization liệt kê ở bài 08 là "luôn nên dùng".
- Không có thống kê riêng về đóng góp của từng optimization trong 9 cái (ngoài 4 cái ablate đầy đủ), nên khó biết optimization nào trong 5 cái còn lại (reward/obs clip, tanh, gradient clip) đóng góp bao nhiêu phần trăm.
- Nghiên cứu chỉ dùng MLP nhỏ (64, 64) và continuous control; chưa rõ kết luận có giữ nguyên với mạng lớn, discrete action, hay off-policy correction mạnh hơn (ví dụ PPO cho LLM, nhiều epoch hơn).

7. Liên hệ với các bài khác

- **04 TRPO, 05 GAE, 07 PPO:** kiến thức nền bắt buộc; bài này "giải phẫu" chính hai thuật toán đó.
- **09 What Matters in On-Policy RL:** cùng tinh thần, quy mô lớn hơn nhiều (250k agent, 8 nhóm lựa chọn), và có một số kết luận khác biệt (ví dụ value clipping có hại) cần đọc đối chiếu.
- **22 MAPPO:** cũng có khái niệm "value normalization" (dùng PopArt để chuẩn hóa target của value network), khác với "reward scaling" ở đây (chuẩn hóa reward thô bằng std chạy) nhưng cùng chung triết lý "chuẩn hóa tín hiệu học quan trọng hơn vẻ ngoài tưởng chừng nhỏ".
- **28 (N+ Implementation Details of RLHF with PPO):** phần tiếp nối trực tiếp triết lý "implementation matters" sang RLHF cho LLM, liệt kê các optimization tương tự (value clipping, reward/advantage normalization, gradient clipping...) trong ngữ cảnh PPO huấn luyện LLM.

- **00 (preface):** cung cấp định nghĩa trust region, KL penalty, PPO clip dùng xuyên suốt bài này.

8. Ứng dụng thực tế (suy luận)

- **Bài học tổng quát cho bất kỳ ai huấn luyện với PPO (RL cổ điển, RLHF/PPO cho LLM bài 07/28, RLVR cho OCR và suy luận ở các bài 36, 56, 57):** trước khi kết luận "thuật toán A tốt hơn B", phải kiểm tra cả hai được triển khai với **cùng mức độ** optimization (chuẩn hóa reward/observation/advantage, gradient clipping, lr schedule, khởi tạo mạng). Một so sánh không kiểm soát các yếu tố này rất dễ nhầm "code tốt hơn" thành "thuật toán tốt hơn".
- **Chuẩn hóa tín hiệu học** (reward scaling, obs normalization) nên được coi là mặc định cần có, không phải chi tiết phụ; đặc biệt quan trọng khi scale reward hoặc advantage thay đổi nhiều giữa các giai đoạn huấn luyện (ví dụ RLVR với reward 0/1 thưa, hoặc reward hỗn hợp nhiều thành phần).
- **Đừng tin tưởng mù quáng vào cơ chế clip của PPO như một "trust region được đảm bảo":** kết quả ở Hình 2 cho thấy ratio vẫn vượt ngưỡng $1 \pm \epsilon$ thường xuyên; nếu cần trust region chặt (ví dụ huấn luyện không ổn định, policy dao động mạnh), có thể cần theo dõi **mean KL thực tế** giữa các bước cập nhật như một chỉ báo bổ sung, không chỉ dựa vào ϵ cấu hình.
- **Gradient clipping và orthogonal init** là optimization rẻ (không tốn thêm forward/backward pass), nên luôn bật trước khi nghi ngờ thuật toán; chúng thường là "quick win" trước khi đổi hẳn sang thuật toán khác.

Khi so sánh các cách thiết kế reward, cần giữ nhất quán các chi tiết tối ưu như chuẩn hóa advantage, gradient clipping và lịch learning rate. Nếu thay đổi đồng thời các yếu tố đó, khó quy sự khác biệt kết quả cho reward.

9. Thí nghiệm gợi ý

1. Trên một môi trường nhỏ (ví dụ Pendulum-v1, chạy nhanh trên CPU), triển khai PPO-M (bỏ hết optimization) và PPO chuẩn (bật đủ: obs/reward normalization, orthogonal init, lr anneal, gradient clip). So sánh learning curve và tính AAI/ACLI theo đúng công thức ở mục 3.4 (đổi thêm TRPO/TRPO+ nếu có sẵn code TRPO).
2. Đo "max ratio" $\max_t \rho_t$ và mean KL mỗi vài chục bước cập nhật trong một lần train PPO; kiểm tra có vượt $1 + \epsilon$ thường xuyên như Hình 2 của bài báo hay không, trên môi trường được chọn.
3. Chạy PPO-NoClip (set ϵ rất lớn, ví dụ 10^6, để vô hiệu hóa min/clip) nhưng giữ nguyên optimization, so với PPO chuẩn; xem khoảng cách reward có nhỏ như Bảng 3 của bài báo hay không.
4. Ablation riêng value function clipping (bật/tắt, giữ mọi thứ khác cố định) trên môi trường được chọn, để tự kiểm tra phát hiện mâu thuẫn với bài 09 (clip value có hại hay không trong thiết lập đang xét).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
TRPO objective	$\max_\theta \mathbb{E}[\rho_t \hat{A}]$ s.t. $D_{KL} \leq \delta$	ràng buộc KL cứng
PPO objective	$\max_\theta \mathbb{E}[\min(\rho_t \hat{A}, \text{clip}(\rho_t, 1 \pm \epsilon)\hat{A})]$	thay ràng buộc bằng clip
Value clipping	$\max[(V - V_{targ})^2, (\text{clip}(V, V_{old} \pm \epsilon) - V_{targ})^2]$	code-level opt #1
Reward scaling	$r_t / \text{std}(R_t)$, $R_t = \gamma R_{t-1} + r_t$	code-level opt #2, không trừ mean
AAI	$\max(\lvert\text{PPO} - \text{TRPO+}\rvert, \lvert\text{PPO-M} - \text{TRPO}\rvert)$	hiệu ứng đối thuật toán lõi
ACLI	$\max(\lvert\text{PPO} - \text{PPO-M}\rvert, \lvert\text{TRPO+} - \text{TRPO}\rvert)$	hiệu ứng thêm optimization
Gradient PPO	$\nabla_\theta L_\theta$ nếu $\rho_t \in [1 \pm \epsilon]$ hoặc $L_\theta^C < L_\theta$, else 0	giải thích vì sao clip không phải projection cứng
PPO-M	PPO lõi, không optimization	dùng để đo "thuật toán thuần"
TRPO+	TRPO + optimization của PPO	đối xứng với PPO-M

Khái niệm	Công thức / ý	Ghi chú
PPO-NoClip	PPO với optimization, không clip	kiểm tra clip có cần thiết

11. Kết luận ngắn

Bài báo cho thấy phần lớn khoảng cách hiệu năng được gán cho "PPO tốt hơn TRPO" thực chất đến từ các **code-level optimization** (chuẩn hóa reward/observation, khởi tạo trọng số trực giao, giảm dần learning rate, gradient clipping...) hơn là từ cơ chế clipping hay thuật toán cốt lõi. Những optimization này còn thay đổi cả tính chất trust-region thực sự được duy trì trong huấn luyện, một khía cạnh không thể thấy chỉ bằng nhìn vào reward cuối cùng. Bài học thực tiễn: khi so sánh thuật toán RL, phải kiểm soát chặt các chi tiết implementation, nếu không kết luận rất dễ sai.

Link gốc: https://arxiv.org/abs/2005.12729

Bài 09

What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study (2020)

Tác giả: Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, Olivier Bachem (Google Research, Brain Team). **Venue / arXiv:** arXiv:2006.05990 (v1, 10/06/2020). Bài báo không nêu venue trong phần văn bản đã trích. **Nguồn:** https://arxiv.org/abs/2006.05990 **Chủ đề:** Nghiên cứu thực nghiệm quy mô cực lớn (hơn 250.000 agent huấn luyện) để đo ảnh hưởng của hơn 50 "lựa chọn" (choices) cấp thấp/cao trong một thuật toán on-policy policy gradient thống nhất, rồi đưa ra khuyến nghị thực tế cho 8 nhóm chủ đề. **Vị trí trong lộ trình:** Đọc ngay sau bài 08 (Implementation Matters in Deep PG: PPO và TRPO), bài mà chính bài báo này gọi là "công trình liên quan gần nhất". Cần đã đọc 00 (preface), 04 (TRPO), 05 (GAE), 07 (PPO) để hiểu các khái niệm nền. Bài 22 (MAPPO) sau đó áp cùng cách tiếp cận "tìm yếu tố ảnh hưởng thực nghiệm" vào multi-agent; các bài RLHF/PPO cho LLM (23-28, đặc biệt 28) kế thừa tinh thần "kiểm soát implementation" sang miền ngôn ngữ.
Chương nền liên quan: nền 05: Vùng tin cậy và PPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu và chỉ dùng để minh họa cách đọc chỉ số, không phải số liệu paper công bố. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Bài 08 (Engstrom, Ilyas et al.) đã chỉ ra rằng phần lớn khoảng cách hiệu năng PPO-TRPO đến từ "code-level optimization" chứ không phải thuật toán lõi, nhưng chỉ ablate đủ 4/9 optimization trên 3 môi trường do hạn chế compute. Bài báo này (Google Brain) mở rộng ý tưởng đó **gấp nhiều lần về quy mô**: xây một thuật toán on-policy duy nhất, cấu hình được **hơn 50 lựa chọn** (mỗi lựa chọn gán mã C1, C2, ... để tra trong Phụ lục B), rồi huấn luyện **hơn 250.000 agent** trên 5 môi trường MuJoCo liên tục (Hopper-v1, Walker2d-v1, HalfCheetah-v1, Ant-v1, Humanoid-v1 từ OpenAI Gym) để đo ảnh hưởng từng lựa chọn một cách có hệ thống.

Khái niệm nền (ngoài các khái niệm đã nêu ở bài 00/04/05/07/08):

- **On-policy (định nghĩa "lỏng" theo bài báo):** thuật toán policy-iteration, lặp giữa sinh trải nghiệm bằng policy hiện tại và dùng trải nghiệm đó cải thiện policy; có thể làm nhiều lượt cập nhật mô hình trong một bước, nên về kỹ thuật vẫn dùng dữ liệu "hơi off-policy" trong một iteration.
- **V-trace:** cơ chế off-policy correction dùng importance sampling bị cắt (truncated), ban đầu cho IMPALA (dữ liệu gần on-policy trong setting phân tán).
- **AWR (Advantage-Weighted Regression):** policy loss dạng trọng số $\exp(A/\beta)$ (hệ số nhiệt độ β) nhân vào negative log-likelihood của action đã thực hiện; weight bị cắt ở ω_{max}.
- **V-MPO:** cập nhật kiểu EM có ràng buộc KL, chọn ra n mẫu advantage dương tốt nhất để fit policy mới (ở đây dùng không có ràng buộc KL tách biệt, vì paper nghiên cứu regularizer riêng ở nhóm 3.8).
- **RPA (Repeat Positive Advantages):** trường hợp giới hạn của AWR ($\beta \to 0$) và V-MPO ($n \to 0$): tương đương lấy negative log-probability của các action có advantage dương (bỏ hết action có advantage âm).
- **PopArt / value function normalization:** chuẩn hóa *target* của value network (không phải chuẩn hóa reward thô) bằng ước lượng mean/std chạy.

2. Ý tưởng chính

Xây một agent on-policy "vạn năng" lộ ra toàn bộ lựa chọn thiết kế dưới dạng tham số cấu hình (bắt đầu từ codebase SEED RL, kiểm chứng rằng khi chọn đúng cấu hình PPOv2 mặc định thì khớp hiệu năng paper PPO gốc). Sau đó, với từng **nhóm chủ đề** lựa chọn (ví dụ "toàn bộ lựa chọn liên quan kiến trúc mạng"), lấy mẫu ngẫu nhiên hàng nghìn cấu hình trong nhóm đó (các lựa chọn ngoài nhóm giữ cố định ở một cấu hình nền tốt), huấn luyện 3 seed mỗi cấu hình, rồi dùng hai phép phân tích (percentile 95 có điều kiện; tỉ lệ xuất hiện trong top 5%) để suy ra lựa chọn nào thực sự ảnh hưởng hiệu năng. Kết quả là 8 nhóm khuyến nghị thực tế, trong đó phát hiện gây bất ngờ nhất là: **cách khởi tạo phân phối hành động ban đầu** (ít khi được nhắc trong literature) ảnh hưởng rất lớn đến tốc độ học.

3. Phương pháp chi tiết: thiết kế nghiên cứu và khuyến nghị theo nhóm

3.0 Thiết kế thực nghiệm

Khó khăn: (1) random tất cả >50 lựa chọn cùng lúc, hầu hết cấu hình học rất kém (xác suất gặp thiết lập "chết" tăng lũy mũ theo số lựa chọn); (2) nhiều lựa chọn tương tác mạnh (ví dụ learning rate và minibatch size), ablate riêng lẻ dễ gây hiểu sai.

Giải pháp: nhóm các lựa chọn theo chủ đề nghi có tương tác nội nhóm; learning rate Adam (C24) được đưa vào **mọi nhóm** vì nghi tương tác với hầu hết lựa chọn khác. Mỗi thí nghiệm random đồng thời các lựa chọn **trong nhóm**; lựa chọn **ngoài nhóm** giữ cố định ở một cấu hình nền tốt (gần PPOv2 mặc định, scale 256 môi trường song song, Phụ lục C).

Hai cách phân tích cho mỗi lựa chọn (ví dụ lựa chọn `advantage_estimator` C6):

- **Percentile 95 có điều kiện:** với mỗi giá trị của lựa chọn, lấy phân phối hiệu năng của các cấu hình đã sample có giá trị đó, báo cáo percentile 95 cùng CI (xấp xỉ nhị thức). Đo "hiệu năng tốt nhất đạt được với ngân sách tìm kiếm hạn chế (khoảng 20 cấu hình)", không phải hiệu năng trung bình.
- **Phân phối giá trị trong top 5% cấu hình:** vì các lựa chọn được sample đều ngẫu nhiên, một giá trị xuất hiện "quá nhiều" trong 5% cấu hình tốt nhất là dấu hiệu lựa chọn đó quan trọng.

Đo hiệu năng: mỗi cấu hình train 3 seed, mỗi seed 1 triệu bước (Hopper/HalfCheetah/Walker2d) hoặc 2 triệu bước (Ant/Humanoid); đánh giá mỗi 100.000 bước (policy đóng băng, return trung bình không chiết khấu trên 100 episode). Trung bình các điểm đánh giá $\approx$ diện tích dưới đường học; điểm cuối = **median** qua 3 seed.

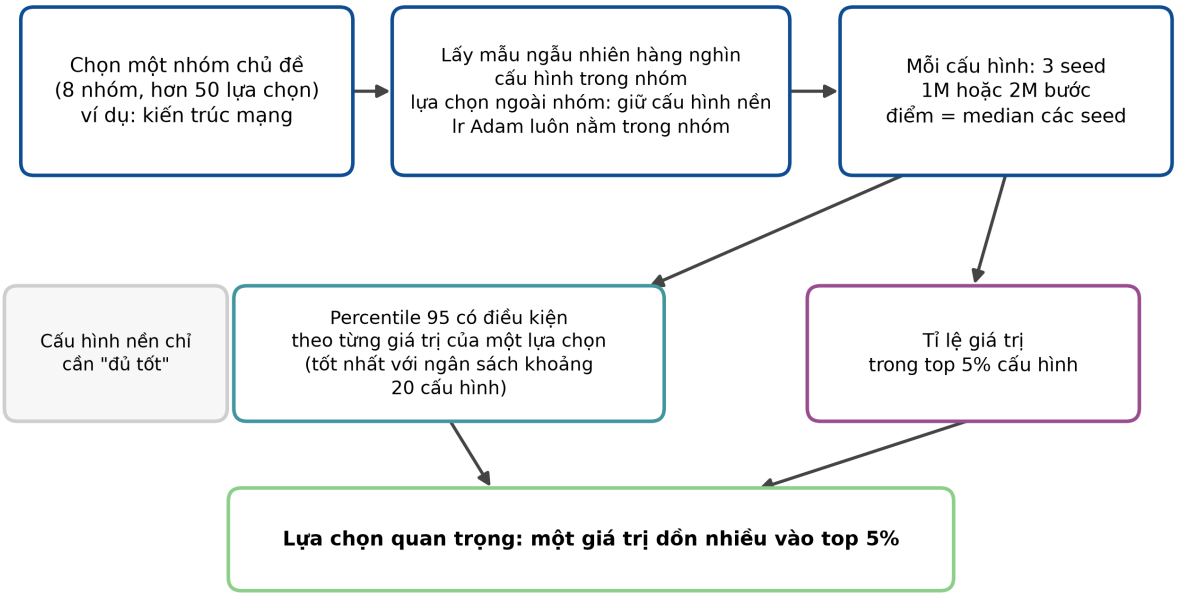
Cách đọc hình: mỗi lần chỉ ngẫu nhiên hóa các lựa chọn trong một nhóm, phần còn lại giữ ở cấu hình nền. Điểm của từng cấu hình (median 3 seed) được đọc bằng hai cách: percentile 95 theo từng giá trị, và mức độ một giá trị dồn vào top 5% cấu hình tốt nhất.

3.1 Policy loss (dựa trên Phụ lục D)

So sánh 6 policy loss: PG (vanilla), V-trace, **PPO**, AWR, V-MPO, RPA; sweep thêm hyperparameter riêng mỗi loss, learning rate (C24), số lượt pass qua dữ liệu (C3).

- PPO vượt các loss khác trên 4/5 môi trường, top trên môi trường còn lại (HalfCheetah); vẫn vượt trên Humanoid/Ant ngay cả khi điều kiện hóa theo hyperparameter tối ưu riêng của mỗi loss.
- PG và V-trace kém nhất trên mọi môi trường (không xử lý tốt dữ liệu off-policy do nhiều pass C3 hoặc buffer C2 lớn so với batch C4).
- $\epsilon = 0.2/0.3$ ổn trên mọi môi trường; ϵ thấp hơn (0.1) hoặc cao hơn (0.5) tốt hơn trên vài môi trường cụ thể.

Khuyến nghị: dùng PPO policy loss. Bắt đầu với ngưỡng clip $\epsilon = 0.25$, nhưng cũng thử giá trị thấp/cao hơn nếu có thể.



Hình 38: Thiết kế thực nghiệm: lấy mẫu cấu hình ngẫu nhiên trong từng nhóm lựa chọn rồi phân tích bằng hai chỉ số (minh hoạ, tự tính)

3.2 Kiến trúc mạng (dựa trên Phụ lục E)

Khảo sát cấu trúc/kích thước mạng, hàm hoạt hóa, cách khởi tạo trọng số, và tham số hóa độ lệch chuẩn hành động.

- Mạng policy và value **tách riêng** (C47) tốt hơn trên 4/5 môi trường. Độ rộng MLP tối ưu của policy phụ thuộc độ phức tạp môi trường (quá hẹp/quá rộng đều hại); value network rộng hơn **không có mặt hại** (ví dụ HalfCheetah: 16-32 unit/layer cho policy nhưng 256 cho value). 2 hidden layer đủ cho cả hai. `tanh` tốt nhất, `relu` kém nhất.
- **Phát hiện bất ngờ nhất:** khởi tạo policy ban đầu có phân phối hành động tập trung quanh **0**, độc lập observation, độ lệch chuẩn nhỏ, ảnh hưởng rất lớn tốc độ học. Đạt được bằng cách scale nhỏ trọng số layer cuối policy MLP (C57: riêng việc này tăng hiệu năng Humanoid **66%**) và offset âm vào input hàm biến đổi std (C61, giá trị std ban đầu tốt nhất ≈ 0.5 trừ Hopper).
- `tanh` cho action transform (thay `clip`) cải thiện HalfCheetah **30%**. Scheme khởi tạo (C56) nói chung ít quan trọng (trừ `he_normal/he_uniform` kém hơn); học std theo state hay toàn cục (C59), `softplus` hay hàm mũ (C60) đều không khác biệt rõ.

Khuyến nghị: khởi tạo layer cuối policy với trọng số nhỏ hơn **100 lần**; dùng `softplus` + offset âm để giảm std ban đầu (nên tune offset); `tanh` cho activation và cho action transform; value MLP rộng (không chia sẻ với policy), tune riêng độ rộng policy.

3.3 Chuẩn hóa và clipping (dựa trên Phụ lục F)

- **Chuẩn hóa observation (C64):** quan trọng trên mọi môi trường trừ Hopper.
- **Chuẩn hóa value function (C66):** bất ngờ, ảnh hưởng mạnh và **đổi dấu theo môi trường**: cần thiết cho HalfCheetah/Humanoid, giúp nhẹ Hopper/Ant, nhưng **hại rõ trên Walker2d** (tác giả không chắc lý do, nghi do thay đổi tốc độ fit value).
- Chuẩn hóa advantage theo minibatch (C67) và clip observation (C65) ảnh hưởng ít. Gradient clipping (C68) giúp nhẹ, ngưỡng cụ thể không quan trọng.

Khuyến nghị: luôn dùng chuẩn hóa observation; kiểm tra xem chuẩn hóa value function có cải thiện hiệu năng hay không (vì tác dụng thay đổi dấu theo môi trường). Gradient clipping có thể giúp nhẹ nhưng là yếu tố phụ.

3.4 Advantage estimation (dựa trên Phụ lục G)

So sánh N-step, GAE, V-trace (C6) và hyperparameter liên quan; thử PPO-style value loss clipping (C13, có trong implementation PPO gốc nhưng không nêu trong bài báo PPO) và Huber loss (C11, C12) so với MSE cho value.

- GAE và V-trace đều tốt hơn N-step; không khác biệt đáng kể giữa GAE và V-trace. $\lambda = 0.9$ (C8/C9) tốt trên mọi môi trường bất kể GAE/V-trace (tune riêng theo môi trường có thể lợi thêm chút).
- **PPO-style value loss clipping (C13) làm giảm hiệu năng, bất kể ngưỡng clip** (khác với cách bài 08 mô tả value clipping là optimization chuẩn của PPO, xem mục 6). Huber loss (C11) kém MSE trên mọi môi trường, bất kể ngưỡng (C12).

Khuyến nghị: dùng GAE với $\lambda = 0.9$, nhưng không dùng Huber loss cũng không dùng PPO-style value loss clipping.

3.5 Thiết lập huấn luyện / xử lý dữ liệu (dựa trên Phụ lục H)

Khảo sát số môi trường song song (C1), số transition/iteration (C2), số lượt pass dữ liệu (C3), kích thước minibatch (C4), cách chia minibatch (C5). PPO gốc chia transition riêng lẻ ngẫu nhiên vào minibatch (phá cấu trúc thời gian) nên advantage chỉ tính **một lần** đầu iteration rồi dùng cho mọi epoch (advantage "cũ" dần, stale); bài báo đề xuất **tính lại advantage ở đầu mỗi lượt pass**.

- Đi qua dữ liệu nhiều lượt (C3) quan trọng cho sample efficiency, thường rẻ nếu có GPU/TPU. Tăng số môi trường song song (C1) giảm hiệu năng rõ trên một số môi trường (đoạn trải nghiệm ngắn lại, bootstrap sớm hơn) nhưng nhanh hơn wall-clock nếu đủ CPU. Tăng batch size (C4) không hại sample efficiency trong khoảng đã thử; C2 (transitions/iteration) ảnh hưởng khá nhiều, cần tune.
- Advantage stale thực sự gây hại; tính lại mỗi lượt pass cho hiệu năng tốt nhất trong các biến thể so sánh.

Khuyến nghị: đi qua dữ liệu nhiều lượt. Xáo trộn (shuffle) các transition riêng lẻ trước khi gán vào minibatch, và tính lại advantage một lần mỗi lượt pass. Để train nhanh hơn theo wall-clock, dùng nhiều môi trường song song và tăng batch size (cả hai có thể làm giảm hiệu quả mẫu). Tune số transition mỗi iteration (C2) nếu có thể.

3.6 Xử lý timestep (dựa trên Phụ lục I)

Khảo sát discount factor (C20), frame skip (C21), và cách xử lý episode bị cắt do giới hạn số bước (C22).

- Hiệu năng phụ thuộc mạnh vào γ (C20); $\gamma = 0.99$ tốt trên mọi môi trường. Frame skip (C21) cải thiện 2/5 môi trường. Xử lý đúng cách episode bị cắt (C22) không ảnh hưởng, có lẽ vì giới hạn bước khá cao (1000 transition) trong các môi trường đã thử.

Khuyến nghị: discount factor γ là một trong các hyperparameter quan trọng nhất, nên tune theo từng môi trường (bắt đầu từ $\gamma = 0.99$). Thử frame skip nếu có thể. Không cần xử lý đặc biệt cho giới hạn bước nếu giới hạn đó đủ lớn.

3.7 Optimizer (dựa trên Phụ lục J)

So sánh Adam và RMSprop (C23) cùng các hyperparameter của chúng (C24-C30) và lịch giảm learning rate tuyến tính (C31).

- Khác biệt Adam/RMSprop khá nhỏ, không optimizer nào thắng nhất quán. Learning rate ảnh hưởng rất mạnh; mặc định 3×10^{-4} cho Adam tốt trên mọi task. Adam tốt hơn với momentum (C26=0.9); RMSprop ít nhạy momentum (C27) hơn nhưng có thể giúp nhẹ (momentum 0.9 tăng bước đi RMSprop

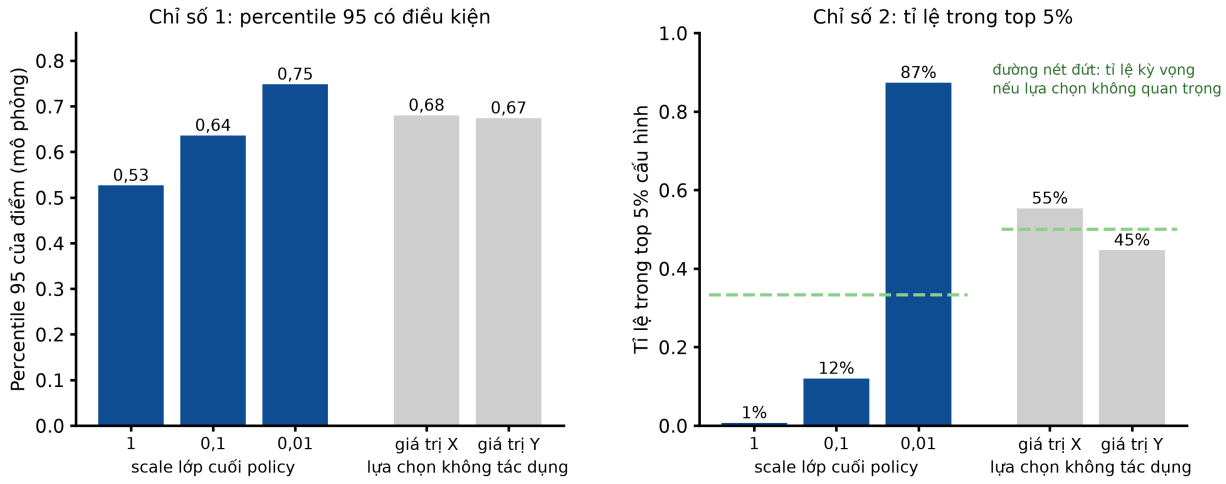
khoảng 10 lần, cần chỉnh lr tương ứng). Centered/uncentered RMSprop (C30) và hệ số ϵ (C28/C29) không khác biệt.
- Giảm dần lr tuyến tính về 0 tăng hiệu năng trên 4/5 task (thường nhỏ, trừ Ant **+15%**).

Khuyến nghị: dùng optimizer Adam với momentum $\beta_1 = 0.9$ và learning rate đã tune (0.0003 là giá trị mặc định an toàn). Giảm dần learning rate tuyến tính có thể giúp nhẹ nhưng là yếu tố phụ.

3.8 Regularization (dựa trên Phụ lục K)

Khảo sát regularizer chính sách dạng penalty hoặc ràng buộc mềm (Lagrange): entropy, KL với phân phối tham chiếu $\mathcal{N}(0, 1)$, KL/reverse-KL với policy hành vi, và KL "decoupled".

- Không thấy bằng chứng regularizer nào giúp ích đáng kể, **trừ HalfCheetah** (mọi ràng buộc, đặc biệt entropy, giúp ích), nhưng mức tăng gần như **không phụ thuộc giá trị ngưỡng**, gợi ý hiệu ứng đến từ độ mạnh ban đầu cao của penalty, không phải ràng buộc mong muốn.
- Suy đoán của tác giả: (1) PPO loss đã enforce trust region, khiến KL penalty/constraint dư thừa; (2) khởi tạo policy cẩn thận (3.2) đã đảm bảo exploration, khiến entropy bonus dư thừa.
- **Lưu ý:** khác 7 nhóm trên, bài báo không đưa "Khuyến nghị" riêng cho nhóm này; diễn giải thay thế: không cần thêm regularizer nếu đã dùng PPO loss + khởi tạo tốt ở 3.2.



Hình 39: Mô phỏng đồ chơi cách đọc hai chỉ số: một lựa chọn có tác dụng và một lựa chọn không có tác dụng (số do giả định, seed cố định) (minh hoạ, tự tính)

Cách đọc hình: trong mô phỏng, scale lớp cuối policy được gán tác dụng thật còn lựa chọn kia thì không. Ở lựa chọn có tác dụng, giá trị nhỏ nhất dồn tới 87% của top 5% và percentile 95 tăng dần; ở lựa chọn không tác dụng, tỉ lệ chỉ quanh mức kỳ vọng 50% (55% và 45%). Toàn bộ con số là mô phỏng giả định, không phải số của bài báo.

4. Thí nghiệm và kết quả chính

Thiết lập chung: 5 môi trường Gym MuJoCo (Hopper-v1, Walker2d-v1, HalfCheetah-v1, Ant-v1, Humanoid-v1), dùng MuJoCo 2.0 (paper ghi chú version MuJoCo có thể ảnh hưởng nhẹ hành vi môi trường). Hơn 250.000 agent được huấn luyện tổng cộng qua 8 thí nghiệm (mỗi thí nghiệm ứng với một nhóm ở mục 3). Phần cứng/quy mô cụ thể từng thí nghiệm không nêu trong phần chính (chi tiết đầy đủ ở Phụ lục D-K).

Bảng cấu hình nền (trích Table 2 của paper, giữ cố định cho các lựa chọn ngoài nhóm đang khảo sát)

Mã	Lựa chọn	Giá trị mặc định
C1 / C2	num_envs / iteration_size	256 / 2048
C3 / C4	num_epochs / batch_size	10 / 64
C6 / C8	advantage_estimator / GAE λ	GAE / 0.95
C13	PPO-style value clipping	0.2
C14 / C16	Policy loss / PPO ϵ	PPO / 0.2
C20	Discount factor γ	0.99
C24 / C26	Adam lr / momentum	3e-4 / 0.9
C47	Mạng chia sẻ?	Shared
C49 / C50	Độ rộng MLP policy / value	64 / 64
C55	Activation	tanh
C57 / C58	Scale layer cuối policy / value	0.01 / 1.0
C64 / C66	Chuẩn hóa input / value	Average / Average
C68	Gradient clipping	0.5

Phát hiện gây bất ngờ nhất (trích abstract): cách khởi tạo policy (centered, std nhỏ, độc lập observation) ảnh hưởng mạnh đến hiệu năng dù hiếm khi được nhắc tới trong các paper RL.

Vài con số nổi bật đã dẫn ở mục 3 (tổng hợp lại để dễ nhớ): scale nhỏ layer cuối policy (C57) → Humanoid **+66%**; `tanh` action transform thay `clip` → HalfCheetah **+30%**; linear LR decay → Ant **+15%**.

5. Phân tích / ablation

Phương pháp luận chính là phần đáng chú ý nhất của bài báo (nhiều hơn một con số đơn lẻ): việc tách "percentile 95 có điều kiện" (đo hiệu năng tốt nhất đạt được với ngân sách tìm kiếm hạn chế) và "tỉ lệ trong top 5%" (đo mức độ một giá trị bị thiên lệch về phía các cấu hình tốt) cho phép phát hiện cả lựa chọn **quan trọng nhưng dễ tune đúng** (ví dụ learning rate: ảnh hưởng mạnh nhưng giá trị mặc định 3e-4 đã khá tốt) và lựa chọn **quan trọng và khó nhận ra** (ví dụ khởi tạo policy: không quan sát được nếu chỉ nhìn learning curve của một vài lần chạy thông thường).

- Cấu hình nền dùng `Shared` cho mạng policy/value (C47), nhưng mục 3.2 khuyến nghị **tách riêng** hai mạng.
- Cấu hình nền đặt PPO-style value clipping = 0.2 (C13), nhưng mục 3.4 kết luận value clipping **làm giảm** hiệu năng.
- Cấu hình nền dùng GAE $\lambda = 0.95$ (C8), nhưng mục 3.4 khuyến nghị $\lambda = 0.9$.

Đây không phải mâu thuẫn logic (cấu hình nền chỉ cần "đủ tốt" để đảm bảo có mô hình học được khi random các lựa chọn khác trong từng nhóm, không nhất thiết phải là cấu hình tối ưu cuối cùng), nhưng người đọc muốn tái tạo "cấu hình khuyến nghị tổng hợp" từ bài báo này cần tự kết hợp các khuyến nghị riêng ở mục 3.1-3.8, **không nên copy thẳng Table 2** làm cấu hình cuối.

6. Giới hạn

Tác giả nêu: phiên bản MuJoCo có thể ảnh hưởng nhẹ hành vi môi trường (dùng MuJoCo 2.0); không chắc lý do chuẩn hóa value function (C66) ảnh hưởng mạnh vậy, chỉ nêu giả thuyết; suy đoán về vai trò của PPO loss/khởi tạo tốt khiến regularization (3.8) dư thừa chưa được kiểm chứng trực tiếp bằng thí nghiệm tách biệt.

Nhận xét:

- Chỉ 5 môi trường **continuous control** MuJoCo; không có discrete action, không Atari, không multi-agent. Khuyến nghị (offset std, tanh action transform) gắn chặt action space liên tục Gaussian, không

áp dụng trực tiếp cho policy phân loại (categorical, như RL cho LLM).

- Thiết kế "nhóm lựa chọn, cố định phần còn lại ở một cấu hình nền" có thể bỏ sót tương tác **giữa các nhóm**; chỉ learning rate (C24) được lặp lại ở mọi nhóm để giảm rủi ro này.
- Mâu thuẫn trực tiếp với bài 08 về PPO-style value clipping (ở đây nói có hại, 08 chỉ liệt kê là optimization chuẩn không bình luận tốt/xấu) cho thấy kết luận "lựa chọn nào tốt" có thể phụ thuộc môi trường, không nên áp dụng như quy luật tuyệt đối.

7. Liên hệ với các bài khác

- **08 Implementation Matters:** bài này tự nhận mở rộng trực tiếp của 08, quy mô >250k agent so với <1000 agent, 8 nhóm/>50 lựa chọn so với 9 optimization; có bất đồng về value clipping (xem mục 6).
- **04 TRPO, 05 GAE, 07 PPO:** nền tảng thuật toán được tune lại ở quy mô lớn.
- **22 MAPPO:** cùng triết lý thực nghiệm lớn tìm implementation factor, lần cho multi-agent; cũng nhấn mạnh value normalization và clip ϵ nhỏ.
- **28 (N+ Implementation Details of RLHF with PPO):** nối tiếp hướng này cho PPO huấn luyện LLM (action rời rạc/token).
- **00 (preface):** GAE, importance sampling, PPO clip, KL penalty là nền cho mục 3.

8. Ứng dụng thực tế (suy luận)

- **Danh sách kiểm tra khởi đầu khi cài PPO từ đầu** (robot control cổ điển hoặc agent RL dùng MLP nhỏ): dùng bảng mục 10 làm danh sách kiểm tra trước khi debug sâu. Nhiều lỗi "RL không học được" bắt nguồn từ các yếu tố rẻ tiền bị bỏ qua: quên chuẩn hóa observation, khởi tạo layer cuối policy quá lớn, hoặc quên tune discount factor theo độ dài episode.
- **Nguyên tắc chuyển sang RL cho LLM/VLM** (cơ chế cụ thể như offset std Gaussian không áp dụng trực tiếp cho action rời rạc/token): kiểm tra entropy/calibration ban đầu của policy trước khi đổ lỗi cho thuật toán; cân nhắc chuẩn hóa value function (PopArt-kiểu, xem bài 22) khi scale reward biến động mạnh; tune số epoch + cách chia minibatch cẩn thận khi implement PPO/GRPO cho RLVR (bài 36, 41, 43).
- **Giảm chi phí tìm hyperparameter:** phương pháp hai chỉ số (percentile 95 có điều kiện; tỉ lệ trong top 5%) là cách rẻ để quyết định lựa chọn nào đáng tune kỹ, áp dụng được cho sweep nhỏ hơn trong các thí nghiệm nhỏ.
- **Thận trọng khi copy khuyến nghị nguyên văn:** rút ra trên 5 môi trường MuJoCo liên tục cụ thể, nên coi là điểm khởi đầu để thử, không phải giá trị đúng tuyệt đối cho bài toán khác (đặc biệt RLVR với reward 0/1 thưa).

9. Thí nghiệm gợi ý

1. Trên một môi trường Gym liên tục nhỏ (Pendulum-v1 hoặc InvertedPendulum), tái tạo khảo sát mục 3.2 ở quy mô nhỏ: so sánh khởi tạo layer cuối policy mặc định của framework bạn đang dùng với khởi tạo "scale nhỏ 100 lần + offset âm vào std", đo tốc độ học.
2. Ablation value function normalization (C66) trên 2-3 môi trường khác nhau, kiểm tra xem có lặp lại hiện tượng "giúp ích trên môi trường này nhưng gây hại trên môi trường khác" như Walker2d trong bài báo hay không.
3. So sánh trực tiếp "advantage tính một lần/iteration" với "advantage tính lại mỗi lượt pass" (mục 3.5) trong một PPO implementation có nhiều epoch (>5), đo chênh lệch hiệu năng.
4. Áp dụng phương pháp "percentile 95 có điều kiện + tỉ lệ trong top 5%" cho một sweep nhỏ (30-50 cấu hình ngẫu nhiên) trên bài toán đang xét (ví dụ discount factor hoặc PPO ϵ), để luyện cách đọc chỉ số trước khi áp dụng cho sweep lớn hơn.

10. Tóm tắt công thức / khái niệm (checklist thực hành)

Nhóm	Khuyến nghị chính	Giá trị gợi ý
Policy loss	Dùng PPO, không dùng PG/V-trace thuần	$\epsilon = 0.25$ (thử thấp/cao hơn)
Kiến trúc mạng	Mạng policy/value tách riêng; `tanh` activation; khởi tạo policy "nhỏ, tập trung quanh 0"	layer cuối policy scale $\times 0.01$ hoặc nhỏ hơn 100 lần; std ban đầu ≈ 0.5; `softplus` cho std, `tanh` cho action transform
Chuẩn hóa / clipping	Luôn chuẩn hóa observation; kiểm tra chuẩn hóa value theo từng môi trường	gradient clip: thứ yếu, giá trị cụ thể ít quan trọng
Advantage estimation	GAE, không Huber, không PPO-style value clip	$\lambda = 0.9$
Thiết lập huấn luyện	Nhiều epoch qua dữ liệu; shuffle transition; tính lại advantage mỗi pass; nhiều env song song + batch lớn để nhanh hơn (wall-clock)	tune số transition/iteration (C2)
Timestep	Tune discount factor theo môi trường; thử frame skip	$\gamma = 0.99$ làm điểm khởi đầu
Optimizer	Adam có momentum; learning rate là yếu tố ảnh hưởng mạnh nhất	lr $= 3 \times 10^{-4}$, $\beta_1 = 0.9$; LR decay tuyến tính là thứ yếu
Regularization	Không cần thêm nếu đã có PPO loss + khởi tạo tốt	không có khuyến nghị cụ thể từ paper (ngoại lệ: HalfCheetah được lợi từ entropy constraint)
Đo hiệu năng	Percentile 95 có điều kiện; tỉ lệ trong top 5%	phương pháp, không phải giá trị số

11. Kết luận ngắn

Bằng một thuật toán on-policy policy-gradient thống nhất lộ ra hơn 50 lựa chọn thiết kế và hơn 250.000 agent huấn luyện, bài báo này biến "kinh nghiệm truyền miệng" về cách tune PPO thành khuyến nghị có cơ sở thực nghiệm cho 8 nhóm chủ đề: policy loss (dùng PPO), kiến trúc mạng (tách mạng, khởi tạo policy nhỏ và tập trung), chuẩn hóa (luôn chuẩn hóa observation, cân nhắc value), advantage estimation (GAE $\lambda = 0.9$, không value clipping), thiết lập huấn luyện (nhiều epoch, advantage tính lại mỗi pass), timestep (tune γ), optimizer (Adam, lr 3e-4), và regularization (thường dư thừa). Phát hiện quan trọng nhất, và dễ bị bỏ qua nhất trong thực hành, là ảnh hưởng lớn của cách khởi tạo phân phối hành động ban đầu.

Link gốc: https://arxiv.org/abs/2006.05990

Bài 10

Continuous control with deep reinforcement learning (2015)

Tác giả: Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, Daan Wierstra (Google DeepMind).
Venue / arXiv: arXiv:1509.02971 (v6, 05/07/2019); "Published as a conference paper at ICLR 2016".
Nguồn: https://arxiv.org/abs/1509.02971
Chủ đề: Mở rộng ý tưởng DQN (replay buffer, target network) sang không gian hành động liên tục bằng cách kết hợp với **deterministic policy gradient (DPG)**: một actor-critic off-policy, actor là hàm xác định (không phải phân phối), critic học bằng Bellman backup như Q-learning.
Vị trí trong lộ trình: Đọc sau bài 00 (policy gradient theorem, MDP, Bellman) và bài 01 (DQN: replay buffer, target network, đây là hai "mẹo" DDPG mượn lại), nên liếc qua 04/07 (TRPO/PPO) để thấy đối trọng on-policy. Đây là bài mở đầu "Giai đoạn 3: continuous control actor-critic off-policy". Bài 11 (TD3) sửa lỗi overestimation và brittleness của DDPG; bài 12 (SAC) thay actor xác định bằng actor ngẫu nhiên kiểu maximum-entropy. Cả ba bài dùng chung khung actor-critic off-policy với critic học bằng Bellman backup.
Chương nền liên quan: nền 04: Actor-critic và GAE

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

DQN (Mnih et al., 2013/2015, bài 01) học được chính sách tốt trên Atari chỉ từ pixel, nhưng chỉ xử lý được **hành động rời rạc và chiều thấp**, vì nó cần $\arg\max_a Q(s,a)$ ở mỗi bước, và với hành động liên tục việc tối ưu này cần một quá trình lặp tốn thời gian. Rời rạc hóa không gian hành động liên tục cũng không ổn: với N khớp và k mức rời rạc mỗi chiều, số hành động là k^N (paper lấy ví dụ tay người 7 bậc tự do, $3^7 = 2187$), "curse of dimensionality", và việc rời rạc hóa còn xóa mất cấu trúc liên tục của không gian hành động.

Khái niệm nền (nhắc lại từ bài 00, áp dụng cho MDP (S, A, p, r) với $A = \mathbb{R}^N$):

- **Policy** $\pi : S \to P(A)$ (xác suất trên hành động) hoặc, nếu xác định, $\mu : S \to A$.
- **Return** $R_t = \sum_{i=t}^{T} \gamma^{i-t} r(s_i, a_i)$, mục tiêu $J = \mathbb{E}[R_1]$.
- **Action-value function** $Q^\pi(s_t, a_t) = \mathbb{E}[R_t \mid s_t, a_t]$, thỏa **Bellman equation**:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1}}\left[r(s_t, a_t) + \gamma\, \mathbb{E}_{a_{t+1}\sim\pi}[Q^\pi(s_{t+1}, a_{t+1})]\right]$$

- Nếu policy mục tiêu là **xác định** μ, kỳ vọng trong bỏ đi:

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1}}[r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))]$$

Vì biểu thức này chỉ phụ thuộc môi trường (không phụ thuộc chính sách hành vi dùng để thu thập dữ liệu), ta học Q^μ **off-policy**: dùng transition sinh ra từ một chính sách hành vi β khác μ.

Tác giả dựa trên **Deterministic Policy Gradient (DPG)** (Silver et al., 2014, không nằm trong lộ trình này), bản thân tương tự NFQCA (Hafner & Riedmiller, 2011). DPG duy trì actor $\mu(s \mid \theta^\mu)$ (ánh xạ xác định trạng thái sang hành động) và critic $Q(s,a)$ học bằng Bellman equation như Q-learning. DPG gốc chỉ thử trên bài toán nhỏ (tile-coding, hàm xấp xỉ tuyến tính); áp dụng trực tiếp với mạng nơ-ron (NFQCA) không ổn

định với mạng lớn. DDPG = "Deep DPG": thêm các mẹo của DQN (replay buffer, target network) và batch normalization để DPG hoạt động ổn định với mạng nơ-ron sâu, học online (không cần batch learning như NFQCA).

2. Ý tưởng chính

Dùng actor-critic: critic $Q(s, a \mid \theta^Q)$ học bằng Bellman backup giống Q-learning (off-policy, dùng replay buffer); actor $\mu(s \mid \theta^\mu)$ cập nhật theo **gradient của Q lan truyền qua μ** (chain rule, không cần lấy mẫu hành động như REINFORCE). Để mạng nơ-ron hội tụ ổn định, DDPG mượn hai cơ chế của DQN: **replay buffer** (phá tương quan giữa các mẫu liên tiếp) và **target network** (giữ mục tiêu TD ổn định), nhưng cập nhật target bằng **soft update** (trộn dần $\tau \ll 1$) thay vì copy định kỳ như DQN. Thêm **batch normalization** để xử lý các đặc trưng trạng thái có đơn vị/khoảng giá trị khác nhau, và **nhiễu Ornstein-Uhlenbeck** để khám phá có tương quan theo thời gian, phù hợp các hệ vật lý có quán tính (moment).

3. Phương pháp chi tiết

3.1 Học critic: Q-learning với hàm xấp xỉ

Mục tiêu TD (giống DQN nhưng $\max_a Q$ được thay bằng $Q(s_{t+1}, \mu(s_{t+1}))$ vì μ xác định):

$$y_t = r(s_t, a_t) + \gamma\, Q(s_{t+1}, \mu(s_{t+1}) \mid \theta^Q)$$

$$\boxed{L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E}\Big[\big(Q(s_t, a_t \mid \theta^Q) - y_t\big)^2\Big]}$$

y_t cũng phụ thuộc θ^Q nhưng điều này "thường bị bỏ qua" (giống DQN): đây chính là lý do cần **target network** riêng để tránh mục tiêu di chuyển theo cùng tốc độ với mạng đang học, gây phân kỳ.

3.2 Học actor: Deterministic Policy Gradient

DPG cập nhật θ^μ theo gradient của J (kỳ vọng return) qua chain rule, lan truyền gradient của Q theo hành động, rồi theo hành động theo θ^μ:

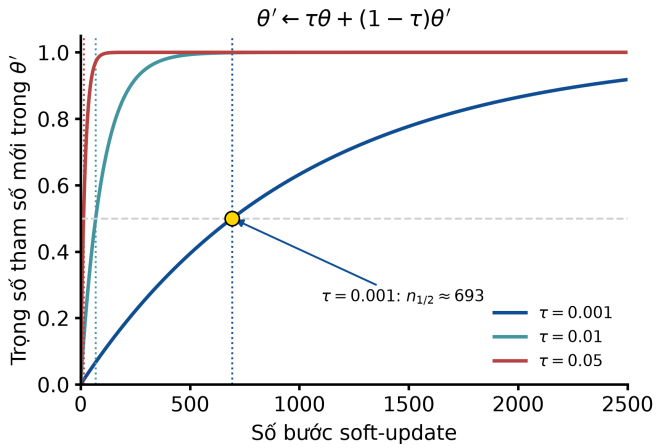
$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta}\big[\nabla_{\theta^\mu} Q(s, a \mid \theta^Q)\big|_{s=s_t, a=\mu(s_t)}\big]$$

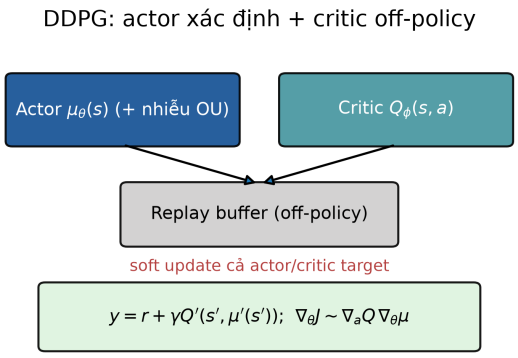
$$\boxed{\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta}\Big[\nabla_a Q(s, a \mid \theta^Q)\big|_{s=s_t, a=\mu(s_t)}\ \nabla_{\theta^\mu} \mu(s \mid \theta^\mu)\big|_{s=s_t}\Big]}$$

Trực giác: muốn tăng Q, ta đi theo hướng $\partial Q/\partial a$ (gradient của giá trị theo hành động), rồi lan truyền hướng đó ngược qua mạng actor bằng chain rule, giống hệt backprop thông thường nhưng "nhãn" là gradient từ critic chứ không phải nhãn cố định. Đây khác hẳn REINFORCE/policy gradient ngẫu nhiên (bài 00): không cần lấy mẫu hành động để ước lượng $\mathbb{E}[\text{reward} \cdot \nabla \log \pi]$, vì μ xác định nên không có $\log \pi$; đổi lại phải có Q khả vi theo a (critic liên tục, khác Q rời rạc của DQN).

3.3 Hai mẹo ổn định mượn từ DQN

(a) Replay buffer R: cache kích thước cố định, lưu (s_t, a_t, r_t, s_{t+1}); đầy thì xóa mẫu cũ nhất; cập nhật bằng minibatch lấy ngẫu nhiên đều từ R (phá tương quan thời gian giữa các mẫu liên tiếp, giúp tối ưu kiểu i.i.d. mà các thuật toán gradient-descent giả định). Vì off-policy nên buffer có thể rất lớn.





Soft update $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$; $\tau = 0.001$ cho nửa đời ≈ 693 bước (minh hoạ, tự tính)

(b) Target network, nhưng "soft update": tạo bản sao $Q'(s, a \mid \theta^{Q'})$ và $\mu'(s \mid \theta^{\mu'})$ dùng để tính mục tiêu y_t. Thay vì copy cứng theo chu kỳ (như DQN), DDPG trộn dần:

$$\boxed{\theta' \leftarrow \tau\theta + (1-\tau)\theta', \qquad \tau \ll 1}$$

Mục tiêu y_t chỉ thay đổi chậm, giúp bài toán học critic gần với một bài toán supervised ổn định hơn. Tác giả nhận thấy cần **cả hai** target Q' và μ' mới ổn định; điều này có thể làm chậm việc lan truyền ước lượng giá trị nhưng lợi ích ổn định lớn hơn nhiều.

3.4 Batch normalization

Khi học từ đặc trưng chiều thấp (ví dụ góc khớp, vận tốc), các thành phần có đơn vị vật lý khác nhau và khoảng giá trị khác nhau giữa môi trường, gây khó cho việc chọn một bộ siêu tham số chung. DDPG dùng batch normalization (Ioffe & Szegedy, 2015): chuẩn hóa mỗi chiều qua minibatch về trung bình 0, phương sai 1, kèm trung bình động dùng khi test (lúc khám phá/đánh giá). Áp dụng cho đầu vào trạng thái và mọi lớp của μ, và mọi lớp của Q trước khi đưa hành động vào.

3.5 Khám phá: nhiễu Ornstein-Uhlenbeck

Vì off-policy, khám phá (exploration) tách biệt khỏi thuật toán học. Chính sách khám phá:

$$\mu'(s_t) = \mu(s_t \mid \theta_t^\mu) + \mathcal{N}$$

$\mathcal{N}$ là một **Ornstein-Uhlenbeck (OU) process** (mô hình vận tốc của hạt Brown có ma sát), tạo nhiễu **tương quan theo thời gian** (khác nhiễu Gauss độc lập mỗi bước), phù hợp các hệ vật lý có quán tính. Siêu tham số: $\theta = 0.15$, $\sigma = 0.2$.

3.6 Thuật toán đầy đủ

```
Input: critic Q(s,a|theta^Q), actor mu(s|theta^mu) khởi tạo ngẫu nhiên
Khởi tạo target: theta^Q' <- theta^Q,  theta^mu' <- theta^mu
Khởi tạo replay buffer R
for episode = 1..M:
    khởi tạo quá trình nhiễu N (OU)
    nhận s_1
    for t = 1..T:
        a_t = mu(s_t | theta^mu) + N_t                    # hành động + nhiễu khám phá
```

```
thực hiện a_t, quan sát r_t, s_{t+1}
lưu (s_t, a_t, r_t, s_{t+1}) vào R
lấy minibatch N mẫu (s_i,a_i,r_i,s_{i+1}) từ R
y_i = r_i + gamma * Q'(s_{i+1}, mu'(s_{i+1}|theta^mu') | theta^Q')
cập nhật critic: minimize (1/N) sum_i (y_i - Q(s_i,a_i|theta^Q))^2
cập nhật actor:  grad theta^mu J ~= (1/N) sum_i grad_a
↪  Q(s,a|theta^Q)|_{s_i,mu(s_i)} * grad_theta^mu mu(s|theta^mu)|_{s_i}
soft update:     theta^Q'  <- tau*theta^Q  + (1-tau)*theta^Q'
                 theta^mu' <- tau*theta^mu + (1-tau)*theta^mu'
```

3.7 Kiến trúc và siêu tham số (Phụ lục, mục 7)

- Adam, learning rate 10^{-4} (actor) và 10^{-3} (critic); critic có L2 weight decay 10^{-2}; $\gamma = 0.99$; $\tau = 0.001$.
- ReLU ở các lớp ẩn; lớp đầu ra actor là tanh (bó buộc hành động).
- Mạng chiều thấp: 2 lớp ẩn 400 và 300 đơn vị (≈130.000 tham số); hành động chỉ đưa vào từ lớp ẩn thứ 2 của Q.
- Mạng từ pixel: 3 lớp conv (không pooling, 32 filter mỗi lớp) + 2 lớp fully-connected 200 đơn vị (≈ 430.000 tham số).
- Khởi tạo lớp cuối actor/critic: phân phối đều $[-3\times10^{-3}, 3\times10^{-3}]$ (chiều thấp) hoặc $[-3\times10^{-4}, 3\times10^{-4}]$ (pixel), để output ban đầu gần 0; các lớp khác khởi tạo $[-1/\sqrt{f}, 1/\sqrt{f}]$ với f là fan-in.
- Minibatch 64 (chiều thấp) hoặc 16 (pixel); replay buffer 10^6.
- Đầu vào pixel: ảnh 64x64, lặp hành động 3 bước mô phỏng mỗi bước agent để suy ra vận tốc từ sai khác khung hình (9 feature map = RGB x 3 khung).

3.8 Ví dụ nhỏ: ý nghĩa thời gian của soft update (tự tính)

Với $\tau = 0.001$, nếu θ cố định và θ' bắt đầu từ 0, sau n bước: $\theta'_n = \theta\big(1-(1-\tau)^n\big)$. Hỏi cần bao nhiêu bước để θ' đạt 50% của θ:

$$(1-\tau)^n = 0.5 \;\Rightarrow\; n = \frac{\ln 0.5}{\ln(1-\tau)} \approx \frac{-0.693}{-0.0010005} \approx 693 \text{ bước (tự tính)}$$

Vậy target network "đuổi theo" mạng chính với hằng số thời gian cỡ vài trăm đến ~700 bước cập nhật, giải thích vì sao mục tiêu TD thay đổi rất chậm và ổn định hơn copy cứng định kỳ, nhưng cũng có nghĩa thông tin giá trị mới lan truyền tới target chậm.

4. Thí nghiệm và kết quả chính

Môi trường: hơn 20 bài toán vật lý mô phỏng bằng MuJoCo, bao gồm cartpole swing-up, dexterous manipulation, legged locomotion, car driving (Torcs), cả ở dạng đặc trưng chiều thấp (lowd) và pixel thô (pix, downsample 64x64, lặp hành động 3 khung). Huấn luyện tối đa 2.5 triệu bước; đánh giá policy định kỳ không có nhiễu khám phá.

Chuẩn hóa điểm: 0 = chính sách ngẫu nhiên đều trên không gian hành động hợp lệ; 1 = iLQG (bộ giải lập kế hoạch có toàn quyền truy cập động lực học và đạo hàm của nó). `cntrl` = DPG gốc (với replay buffer + batch norm, không có các cải tiến khác của DDPG).

Bảng 1 (trích từ Bảng 1 của paper, trung bình/tốt nhất trên 5 lần lặp, sau tối đa 2.5M bước)

Môi trường	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425

Môi trường	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
gripper	0.655	0.972	0.406	0.790	0.461	0.816
torcs (điểm thô)	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

(Bảng đầy đủ trong bài báo có 23 môi trường; đây là một tập con đại diện.) Nhiều trường hợp DDPG vượt cả iLQG (điểm > 1), kể cả khi học từ pixel. Mô hình học từ pixel đôi khi nhanh gần bằng học từ đặc trưng chiều thấp (tác giả suy đoán do lặp hành động làm bài toán dễ hơn, hoặc lớp conv tạo biểu diễn dễ tách). Trên Torcs, một số lần lặp học được chính sách đi hết vòng đua, một số thất bại.

Về số bước: hầu hết bài toán giải được trong 2.5 triệu bước, khoảng **20 lần ít hơn** số bước DQN cần cho Atari.

5. Phân tích / ablation

- **Target network là yếu tố quan trọng nhất (Hình 2):** so sánh 4 biến thể (DPG gốc + batch norm, + target network, + cả hai, + cả hai học từ pixel). Học không có target network (như DPG gốc) cho kết quả rất kém ở nhiều môi trường; cả target network và batch norm đều cần thiết để hoạt động tốt trên mọi bài toán.
- **Chất lượng ước lượng Q (Hình 3):** so sánh giá trị Q ước lượng sau huấn luyện với return thực đo trên episode test. Ở bài toán đơn giản (pendulum, cartpole), ước lượng khá chính xác không lệch hệ thống; ở bài toán khó hơn, ước lượng kém chính xác hơn nhưng vẫn học được chính sách tốt. (Đây là tiền đề cho vấn đề overestimation mà bài 11 TD3 phân tích sâu hơn.)

6. Giới hạn

Tác giả nêu:

- Như hầu hết thuật toán RL model-free, không có bảo đảm hội tụ khi dùng hàm xấp xỉ phi tuyến.
- DDPG vẫn cần số lượng episode huấn luyện lớn để tìm ra giải pháp.
- Chỉ so sánh với DPG gốc (biến thể `cntrl`), không có baseline on-policy mạnh (TRPO) hay baseline actor-critic khác trong chính bài; so sánh đó xuất hiện muộn hơn ở các bài sau (TD3, SAC).

Nhận xét:

- Bài báo không đo overestimation bias một cách hệ thống (chỉ Hình 3 so Q ước lượng với return thực trên vài môi trường); bài 11 (TD3) sau này chỉ rõ DDPG có overestimation bias hệ thống và cực kỳ nhạy với siêu tham số ("notoriously challenging to use due to its extreme brittleness and hyperparameter sensitivity", theo TD3 abstract).
- Không có cơ chế chặn reward hacking hay giới hạn độ lệch giữa policy mới/cũ (khác TRPO/PPO); actor xác định dễ "bắt chéo" vào vùng mà critic đánh giá sai (overfit vào đỉnh hẹp của Q), vấn đề mà TD3 (target policy smoothing) và SAC (policy ngẫu nhiên entropy) giải quyết.
- Ba nguồn: τ, learning rate actor/critic, kiến trúc batch norm đều phải khớp nhau để ổn định; không có hướng dẫn tổng quát ngoài con số cụ thể đã dùng.

7. Liên hệ với các bài khác

- **00 (preface):** DPG là một biến thể của policy gradient theorem cho chính sách xác định; so với REINFORCE/GAE/PPO (chính sách ngẫu nhiên, on-policy), DDPG off-policy và không cần baseline/advantage.
- **01 DQN, 02 Double DQN:** nguồn của replay buffer, target network; DDPG mang các mẹo này sang không gian hành động liên tục. Double DQN giải quyết overestimation cho Q rời rạc, nhưng chưa được áp dụng/hiệu quả cho actor-critic (xem bài 11).
- **04 TRPO, 07 PPO:** đối trọng on-policy, trust region/clipping, ổn định hơn nhưng kém hiệu quả dữ liệu hơn (data efficiency) theo nhận xét trong phần Related Work của chính paper DDPG.
- **11 TD3:** chỉ ra và sửa overestimation bias của DDPG bằng clipped double-Q, thêm delayed policy update và target policy smoothing; vẫn actor xác định.
- **12 SAC:** thay actor xác định bằng actor ngẫu nhiên tối đa entropy, học song song cùng thời điểm với TD3, cũng mượn ý tưởng twin-Q.

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp với RL cho LLM/VLM: thấp (DDPG dành cho hành động liên tục kiểu điều khiển robot, không áp dụng trực tiếp cho sinh token rời rạc), nhưng các *thành phần* của nó xuất hiện lại khắp nơi trong RL hiện đại:

- **Soft target update** $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$ là kỹ thuật ổn định hóa chung cho bất kỳ thuật toán off-policy actor-critic nào (TD3, SAC, và các biến thể off-policy RL cho agent dùng tool, bài 53 Search-R1, 54 agentic RL survey, có thể cần kỹ thuật tương tự khi có một "critic"/value network học song song với policy).
- **Replay buffer** chỉ hoạt động tốt khi hành động của policy cũ và mới "không quá khác biệt" về phân phối trạng thái ghé qua; với LLM, dữ liệu rollout cũ rất nhanh lệch khỏi policy hiện tại (phân phối văn bản thay đổi mạnh mỗi update), nên hầu hết quy trình RLHF/RLVR cho LLM (bài 23-26, 36-44) dùng rollout mới mỗi bước (on-policy hoặc gần on-policy) hơn là replay buffer lớn kiểu DDPG.
- Bài học chung quan trọng nhất để mang sang các bài RLHF/RLVR sau: **off-policy không có nghĩa là ổn định tự động**; cần target network/soft update hoặc cơ chế tương đương (ví dụ clipping của PPO, bài 07) để tránh mục tiêu học di chuyển quá nhanh.

9. Thí nghiệm gợi ý

1. Cài DDPG trên một môi trường continuous-control nhỏ (Pendulum-v1 hoặc tương đương), so sánh có/không soft target update ($\tau = 0.001$ so với copy cứng mỗi N bước) để tái hiện trực quan Hình 2.
2. Thử thay nhiễu Ornstein-Uhlenbeck bằng nhiễu Gauss độc lập mỗi bước (giữ σ), đo tốc độ học; so sánh với phát hiện của bài 11 (TD3) rằng OU "không mang lại lợi ích hiệu năng" so với nhiễu không tương quan.
3. Đo overestimation: ghi lại $Q_\theta(s, \mu(s))$ trung bình trong quá trình huấn luyện và so với return trung bình thực đo bằng rollout không nhiễu (tái hiện ý tưởng Hình 3), xem bias tăng theo độ khó môi trường hay không.
4. Ablation batch normalization: bật/tắt trên mạng chiều thấp với 3-4 môi trường có đơn vị đo khác nhau (ví dụ góc và vận tốc), quan sát độ nhạy siêu tham số.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Bellman (policy xác định)	$Q^\mu(s,a) = \mathbb{E}[r + \gamma Q^\mu(s', \mu(s'))]$	không cần kỳ vọng theo hành động

Khái niệm	Công thức / ý	Ghi chú
Loss critic	$L(\theta^Q) = \mathbb{E}[(Q(s,a) - y)^2]$, $y = r + \gamma Q'(s', \mu'(s'))$	giống DQN, target là mạng riêng
Deterministic policy gradient	$\nabla_{\theta^\mu} J \approx \mathbb{E}[\nabla_a Q\|_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s)]$	chain rule qua critic
Soft update	$\theta' \leftarrow \tau\theta + (1-\tau)\theta'$, $\tau = 0.001$	thay copy cứng của DQN
Exploration	$\mu'(s) = \mu(s) + \mathcal{N}_{OU}(\theta = 0.15, \sigma = 0.2)$	nhiễu tương quan thời gian
Batch norm	chuẩn hóa mỗi chiều qua minibatch	cho đầu vào trạng thái, các lớp của μ, Q
Kiến trúc	400-300 ReLU, tanh đầu ra actor	≈130K tham số (chiều thấp)
Siêu tham số	Adam 10^{-4}/10^{-3}, $\gamma = 0.99$, buffer 10^6	actor/critic lr khác nhau

11. Kết luận ngắn

DDPG chứng minh rằng actor-critic off-policy với hàm xấp xỉ sâu có thể giải quyết bài toán điều khiển liên tục một cách bền vững, bằng cách mang hai mẹo chủ lực của DQN (replay buffer, target network, ở đây là bản "soft") sang Deterministic Policy Gradient, cộng thêm batch normalization và nhiễu Ornstein-Uhlenbeck cho khám phá. Thuật toán giải được hơn 20 bài toán vật lý với cùng một bộ siêu tham số, kể cả học trực tiếp từ pixel, với số bước ít hơn DQN trên Atari khoảng 20 lần; đổi lại, như các bài sau (TD3, SAC) chỉ ra, nó bị overestimation bias và rất nhạy siêu tham số.

Link gốc: https://arxiv.org/abs/1509.02971

Bài 11

Addressing Function Approximation Error in Actor-Critic Methods (2018)

Tác giả: Scott Fujimoto (McGill University), Herke van Hoof (University of Amsterdam), David Meger (McGill University).
Venue / arXiv: arXiv:1802.09477 (v3, 22/10/2018); Proceedings of the 35th International Conference on Machine Learning (ICML 2018), PMLR 80.
Nguồn: https://arxiv.org/abs/1802.09477
Chủ đề: Chứng minh **overestimation bias** (vốn chỉ nghiên cứu kỹ ở Q-learning rời rạc) cũng xảy ra trong actor-critic liên tục, rồi đề xuất ba cơ chế sửa: **Clipped Double Q-learning**, **delayed policy updates**, **target policy smoothing**. Thuật toán kết quả gọi là **TD3** (Twin Delayed Deep Deterministic policy gradient).
Vị trí trong lộ trình: Đọc sau bài 10 (DDPG, thuật toán nền mà TD3 sửa trực tiếp) và bài 01-02 (DQN, Double DQN: nguồn gốc ý tưởng Double Q-learning mà TD3 "clip" lại cho actor-critic). TD3 và bài 12 (SAC) ra đời gần như đồng thời, giải quyết cùng vấn đề của DDPG theo hai hướng khác nhau (xác định có sửa lỗi vs. ngẫu nhiên tối đa entropy); nên đọc liền sau TD3.
Chương nền liên quan: nền 04: Actor-critic và GAE

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Overestimation bias: trong Q-learning rời rạc, mục tiêu TD dùng $\max_{a'} Q(s', a')$. Nếu ước lượng Q có nhiễu ϵ (không tránh được khi dùng hàm xấp xỉ), thì $\mathbb{E}[\max_{a'}(Q(s', a') + \epsilon)] \geq \max_{a'} Q(s', a')$ (Thrun & Schwartz, 1993): lấy max của một ước lượng nhiễu luôn thiên lệch lên trên giá trị thật. Sai số này lan truyền qua Bellman equation, gây tích lũy lỗi. Double Q-learning (Van Hasselt, 2010) và Double DQN (Van Hasselt et al., 2016) sửa vấn đề này cho hành động rời rạc bằng cách tách việc "chọn" hành động tốt nhất và "đánh giá" giá trị của nó bằng hai ước lượng khác nhau.

Với hành động **liên tục**, actor-critic (như DDPG, bài 10) không lấy $\max_a$ trực tiếp, mà actor π_ϕ cập nhật bằng gradient để **xấp xỉ** arg max. Bài báo chứng minh overestimation vẫn xảy ra trong trường hợp này, và cách sửa chuẩn cho rời rạc (Double DQN) lại **không hiệu quả** trong actor-critic: Double DQN dùng target network làm ước lượng độc lập thứ hai, nhưng vì chính sách actor-critic thay đổi chậm, mạng hiện tại và mạng target quá giống nhau để tạo một ước lượng "độc lập" thật sự.

Khái niệm nhắc lại: actor π_ϕ, critic Q_θ, Bellman $Q^\pi(s, a) = r + \gamma \mathbb{E}_{s', a'}[Q^\pi(s', a')]$, deterministic policy gradient $\nabla_\phi J(\phi) = \mathbb{E}_{s \sim p^\pi}[\nabla_a Q^\pi(s, a)|_{a=\pi(s)} \nabla_\phi \pi_\phi(s)]$ (xem đạo hàm đầy đủ ở bài 10), target network $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, TD-error δ.

2. Ý tưởng chính

Ba cơ chế độc lập, mỗi cơ chế nhắm vào một khía cạnh của vấn đề sai số do hàm xấp xỉ:

1. **Clipped Double Q-learning:** dùng cặp critic $(Q_{\theta_1}, Q_{\theta_2})$, lấy **min** của hai ước lượng target để chặn trên overestimation, đổi lấy một chút underestimation (chấp nhận được vì underestimation không bị khuếch đại qua policy update như overestimation).
2. **Delayed policy updates:** cập nhật actor và target network **ít thường xuyên hơn** critic (mỗi d bước), để critic "ổn định" trước khi dùng nó định hướng policy, giảm vòng lặp phản hồi lỗi.

3. **Target policy smoothing regularization:** khi tính mục tiêu TD, thêm nhiễu nhỏ vào hành động của target actor (giống ý tưởng SARSA), để giá trị học được mượt theo vùng lân cận hành động, giảm nguy cơ actor "khớp" vào các đỉnh hẹp sai của Q (do hàm xấp xỉ gây ra).

3. Phương pháp chi tiết

3.1 Overestimation bias trong actor-critic (Mục 4.1, trình bày lại)

Gọi ϕ_{approx} là tham số actor sau một bước cập nhật theo gradient của critic xấp xỉ Q_θ, và ϕ_{true} là tham số actor nếu cập nhật theo giá trị thật Q^π (không biết trong lúc học):

$$\phi_{approx} = \phi + \frac{\alpha}{Z_1}\mathbb{E}_s[\nabla_\phi \pi_\phi(s) \nabla_a Q_\theta(s,a)|_{a=\pi_\phi(s)}], \qquad \phi_{true} = \phi + \frac{\alpha}{Z_2}\mathbb{E}_s[\nabla_\phi \pi_\phi(s) \nabla_a Q^\pi(s,a)|_{a=\pi_\phi(s)}]$$

Vì hướng gradient là một cực đại hóa cục bộ, tồn tại ϵ_1, ϵ_2 đủ nhỏ sao cho với $\alpha \le \epsilon_1$: $\mathbb{E}[Q_\theta(s, \pi_{approx}(s))] \ge \mathbb{E}[Q_\theta(s, \pi_{true}(s))]$ (cập nhật theo Q_θ làm tăng chính Q_θ tốt hơn cập nhật theo Q^π), và với $\alpha \le \epsilon_2$: $\mathbb{E}[Q^\pi(s, \pi_{true}(s))] \ge \mathbb{E}[Q^\pi(s, \pi_{approx}(s))]$ (ngược lại cho giá trị thật). Nếu thêm giả thiết hợp lý $\mathbb{E}[Q_\theta(s, \pi_{true}(s))] \ge \mathbb{E}[Q^\pi(s, \pi_{true}(s))]$ (ước lượng không thấp hơn giá trị thật tại điểm π_{true}), hai bất đẳng thức trên ghép lại cho:

$$\boxed{\mathbb{E}[Q_\theta(s, \pi_{approx}(s))] \ge \mathbb{E}[Q^\pi(s, \pi_{approx}(s))]}$$

tức là **giá trị mà actor-critic tin tưởng luôn lớn hơn hoặc bằng giá trị thật** khi $\alpha < \min(\epsilon_1, \epsilon_2)$: overestimation có hệ thống, không chỉ trong trường hợp arg max rời rạc. Thực nghiệm (Hình 1 của bài báo, Hopper-v1 và Walker2d-v1): DDPG tích lũy sai lệch Q ngày càng lớn so với giá trị thật đo bằng rollout, trong khi phiên bản Clipped Double Q (CDQ, xem 3.2) không bị lệch.

Bài báo cũng thử áp actor-critic Double DQN (dùng mạng hiện tại thay target network để chọn hành động, công thức (8): $y = r + \gamma Q_{\theta'}(s', \pi_\phi(s'))$) và thấy vẫn overestimation tương tự DDPG (Hình 2), vì chính sách thay đổi chậm khiến mạng hiện tại và target quá giống nhau để tạo ước lượng độc lập.

3.2 Clipped Double Q-Learning cho Actor-Critic (Mục 4.2)

Double Q-learning gốc dùng hai cặp actor/critic độc lập $(\pi_{\phi_1}, Q_{\theta_1})$, $(\pi_{\phi_2}, Q_{\theta_2})$, mỗi actor tối ưu theo critic riêng, mục tiêu học chéo:

$$y_1 = r + \gamma Q_{\theta_2'}(s', \pi_{\phi_1}(s')), \qquad y_2 = r + \gamma Q_{\theta_1'}(s', \pi_{\phi_2}(s'))$$

Nhưng hai critic **không hoàn toàn độc lập** (dùng chung replay buffer, và critic này xuất hiện trong mục tiêu của critic kia), nên vẫn có vùng trạng thái mà $Q_{\theta_2}(s, \pi_{\phi_1}(s)) > Q_{\theta_1}(s, \pi_{\phi_1}(s))$, và $Q_{\theta_1}(s, \pi_{\phi_1}(s))$ (ước lượng đang dùng để cập nhật policy) đã thiên lệch lên. TD3 đề xuất **cận trên hóa (upper-bound)** ước lượng ít thiên lệch hơn bằng ước lượng thiên lệch, tức là lấy **min**:

$$\boxed{y_1 = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_{\phi_1}(s'))}$$

Để giảm chi phí tính toán, TD3 chỉ dùng **một actor** π_{ϕ_1} (tối ưu theo Q_{θ_1}), và dùng cùng mục tiêu $y_2 = y_1$ cho Q_{θ_2}. Nếu $Q_{\theta_2} > Q_{\theta_1}$ thì cập nhật giống tiêu chuẩn (không thêm thiên lệch); nếu $Q_{\theta_2} < Q_{\theta_1}$ thì nghi ngờ Q_{θ_1} bị overestimate và giá trị bị kéo xuống, giống cơ chế Double Q-learning. Hệ quả phụ: vì phương sai của

min của các biến ngẫu nhiên giảm khi biến có phương sai thấp, cơ chế này còn **ưu tiên** các trạng thái có ước lượng giá trị phương sai thấp hơn, tức là an toàn hơn cho policy update.

3.3 Tích lũy sai số và vai trò của target network (Mục 5.1-5.2)

Mỗi cập nhật TD để lại dư sai số $\delta(s,a)$:

$$Q_\theta(s,a) = r + \gamma \mathbb{E}[Q_\theta(s',a')] - \delta(s,a)$$

Khai triển lặp lại qua các bước, giá trị học được xấp xỉ **return trừ tổng sai số TD chiết khấu trong tương lai**:

$$\boxed{Q_\theta(s_t,a_t) = \mathbb{E}_{s_i,a_i\sim p^\pi}\Big[\sum_{i=t}^{T}\gamma^{i-t}(r_i-\delta_i)\Big]}$$

tức là phương sai của ước lượng tỉ lệ với phương sai của phần thưởng tương lai **và** phương sai của sai số ước lượng tích lũy. Với γ lớn, phương sai có thể tăng rất nhanh nếu sai số mỗi bước không được kiểm soát; mỗi bước gradient chỉ giảm sai số trên minibatch hiện tại, không có bảo đảm gì về sai số ngoài minibatch đó.

Thực nghiệm (Hình 3, Hopper-v1, trạng thái cố định): không có target network ($\tau = 1$) cho giá trị biến động mạnh (dù policy cố định học vẫn hội tụ tương đối); khi policy **tiếp tục học** cùng giá trị hiện tại, không có target network dẫn đến hành vi **phân kỳ mạnh**. Kết luận: nguy cơ phân kỳ xảy ra chủ yếu do **tương tác giữa actor và critic** khi giá trị có phương sai cao, không chỉ do critic riêng lẻ.

3.4 Delayed policy updates (Mục 5.2)

Nếu target network giảm sai số tích lũy qua nhiều bước, thì nên cập nhật **policy** ít thường xuyên hơn **critic**, để sai số TD nhỏ lại trước khi dùng giá trị đó định hướng policy. TD3 chỉ cập nhật actor và các target network sau mỗi d bước cập nhật critic (tạo thuật toán hai tốc độ, "two-timescale", cần cho sự hội tụ trong trường hợp tuyến tính theo Konda & Tsitsiklis, 2003). Ít cập nhật policy hơn mà vẫn cải thiện hiệu năng, đồng thời **giảm thời gian huấn luyện**.

3.5 Target Policy Smoothing Regularization (Mục 5.3)

Chính sách xác định dễ khớp quá mức (overfit) vào các đỉnh hẹp, sai của hàm giá trị do lỗi xấp xỉ. TD3 "làm mượt" mục tiêu bằng cách lấy kỳ vọng Q quanh một vùng nhỏ xung quanh hành động mục tiêu (giống cập nhật kiểu SARSA, nhưng học off-policy):

$$y = r + \gamma\, \mathbb{E}_\epsilon[Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon)], \qquad \epsilon \sim \text{clip}(\mathcal{N}(0,\sigma), -c, c)$$

Trong thực tế, kỳ vọng này xấp xỉ bằng cách thêm một nhiễu Gauss nhỏ, bị clip, vào hành động của target actor rồi tính Q một lần (không cần nhiều mẫu):

$$\boxed{y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_{\phi'}(s') + \epsilon), \qquad \epsilon \sim \text{clip}(\mathcal{N}(0,\sigma), -c, c)}$$

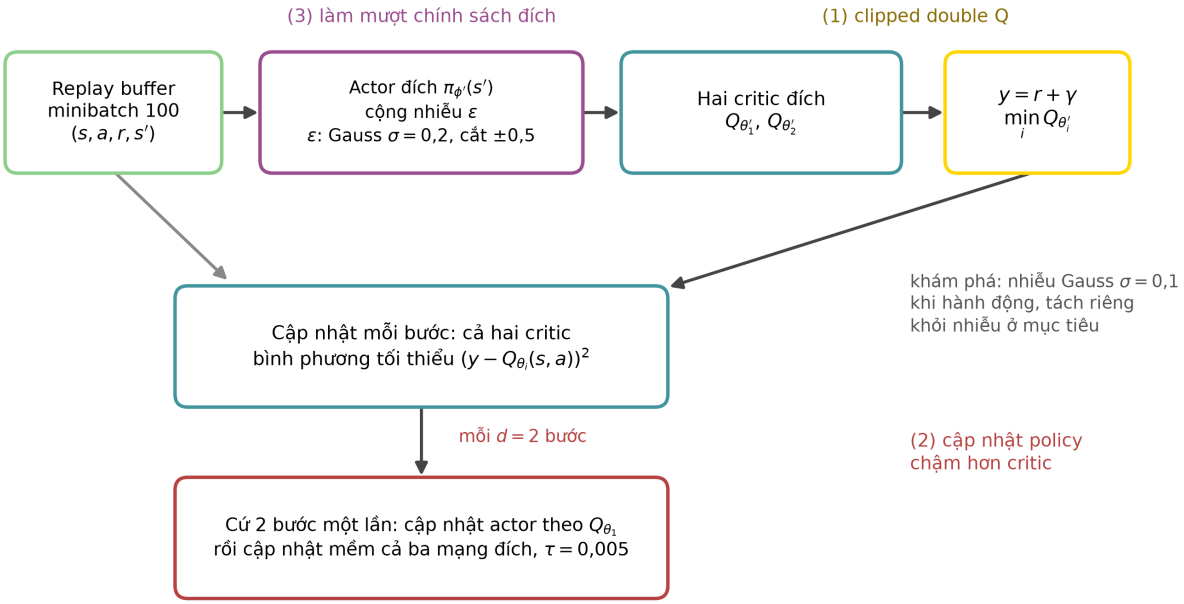
Khác với nhiễu khám phá (được thêm khi hành động trong môi trường), nhiễu này chỉ dùng để **tính mục tiêu học**, độc lập với chính sách khám phá.

3.6 Thuật toán TD3 đầy đủ (Algorithm 1)

```
Khởi tạo critic Q_theta1, Q_theta2 và actor pi_phi (ngẫu nhiên)
Khởi tạo target: theta1' <- theta1, theta2' <- theta2, phi' <- phi
Khởi tạo replay buffer B
for t = 1..T:
    a = pi_phi(s) + eps,  eps ~ N(0, sigma)                    # nhiễu khám phá
    thực hiện a, quan sát r, s'; lưu (s,a,r,s') vào B

    lấy minibatch N mẫu (s,a,r,s') từ B
    a_tilde = pi_phi'(s') + eps_tilde,  eps_tilde ~ clip(N(0, sigma_tilde), -c, c)   #
    ↪  target policy smoothing
    y = r + gamma * min_{i=1,2} Q_theta_i'(s', a_tilde)                               #
    ↪  clipped double Q target
    cập nhật theta_i (i=1,2):  minimize (1/N) sum (y - Q_theta_i(s,a))^2

    if t mod d == 0:                                           # delayed policy update
        cập nhật phi theo deterministic policy gradient:
            grad_phi J = (1/N) sum grad_a Q_theta1(s,a)|_{a=pi_phi(s)} * grad_phi
            ↪  pi_phi(s)
        soft update:
            theta_i' <- tau*theta_i + (1-tau)*theta_i'
            phi'     <- tau*phi     + (1-tau)*phi'
```



Hình 40: Một bước cập nhật của TD3 với ba cơ chế: làm mượt chính sách đích, clipped double Q và cập nhật actor trì hoãn (minh hoạ, tự tính)

Cách đọc hình: hàng trên dựng đích y từ minibatch: actor đích cộng nhiễu cắt, hai critic đích đánh giá, rồi lấy min. Cả hai critic học theo cùng y ở mỗi bước, còn actor và ba mạng đích chỉ cập nhật cứ 2 bước một lần (khối đỏ); nhiễu khám phá khi hành động là một nhiễu khác, không liên quan nhiễu ở mục tiêu.

3.7 Kiến trúc và siêu tham số (Mục 6.1)

- Mạng 400-300, ReLU, actor đầu ra tanh; **khác DDPG gốc**: critic nhận cả (s, a) ngay từ lớp đầu vào (không phải chỉ từ lớp ẩn thứ 2).

- Adam, learning rate 10^{-3} cho cả actor và critic (so với DDPG dùng hai lr khác nhau); minibatch 100; replay buffer chứa toàn bộ lịch sử.
- Nhiễu khám phá: Gaussian $\mathcal{N}(0, 0.1)$ không tương quan (tác giả thấy OU process không mang lại lợi ích hiệu năng gì so với nhiễu Gauss độc lập, khác giả định của DDPG); nhiễu target smoothing $\mathcal{N}(0, 0.2)$ clip vào $(-0.5, 0.5)$.
- Giai đoạn khám phá thuần túy ban đầu: 10.000 bước (HalfCheetah, Ant, môi trường "ổn định độ dài") hoặc 1.000 bước (còn lại) trước khi dùng policy.
- $d = 2$ (delayed update), $\tau = 0.005$ (nhanh hơn $\tau = 0.001$ của DDPG, nhưng nhờ delay nên vẫn ổn định).
- Mỗi task chạy 1 triệu bước, đánh giá mỗi 5.000 bước, trung bình 10 episode không nhiễu khám phá, 10 seed ngẫu nhiên.

3.8 Ví dụ nhỏ: clip giảm overestimation (tự tính)

Giả sử giá trị thật $Q^{\pi}(s, a) = 100$, hai critic có nhiễu độc lập: $Q_{\theta_1} = 105$ (overestimate +5), $Q_{\theta_2} = 98$ (underestimate -2).

- Nếu chỉ dùng Q_{θ_1} (như DDPG): mục tiêu TD dùng 105, sai lệch $+5$.
- Nếu dùng $\min(105, 98) = 98$ (TD3): sai lệch -2, nhỏ hơn về độ lớn tuyệt đối, và **quan trọng hơn**: sai lệch âm không bị khuếch đại qua policy update (actor không bị kéo về phía hành động "ảo tưởng" giá trị cao), trong khi sai lệch dương của Q_{θ_1} một mình sẽ khiến actor học theo hướng sai.

Đây chỉ là minh họa trực giác (tự tính), không phải số liệu thật của bài báo.

4. Thí nghiệm và kết quả chính

Môi trường: 7 bài toán MuJoCo qua OpenAI Gym: HalfCheetah-v1, Hopper-v1, Walker2d-v1, Ant-v1, Reacher-v1, InvertedPendulum-v1, InvertedDoublePendulum-v1 (không sửa đổi môi trường/reward để so sánh công bằng). So sánh với DDPG (gốc), "our DDPG" (DDPG của tác giả viết lại với kiến trúc/siêu tham số mới nhưng không có 3 cải tiến TD3), PPO, TRPO, ACKTR (từ OpenAI baselines), và SAC (code của tác giả SAC).

Bảng 1 (Max Average Return qua 10 trial, 1 triệu bước; ± = 1 độ lệch chuẩn)

Môi trường	TD3	DDPG	Our DDPG	PPO	TRPO	ACKTR	SAC
HalfCheetah	**9636.95 ± 859.07**	3305.60	8577.29	1795.43	-15.57	1450.46	2347.19
Hopper	**3564.07 ± 114.74**	2020.46	1860.02	2164.70	2471.30	2428.39	2996.66
Walker2d	**4682.82 ± 539.64**	1843.85	3098.11	3317.69	2321.47	1216.70	1283.67
Ant	**4372.44 ± 1000.33**	1005.30	888.77	1083.20	-75.85	1821.94	655.35
Reacher	**-3.60 ± 0.56**	-6.51	-4.01	-6.18	-111.43	-4.26	-4.44
InvPendulum	1000.00 ± 0.00	1000.00	1000.00	1000.00	985.40	1000.00	1000.00
InvDoublePendulum	9337.47 ± 14.96	9355.52	8369.95	8977.94	205.85	**9081.92**	8487.15

TD3 đạt hoặc vượt mọi baseline ở gần hết các task (cả về tốc độ học, xem Hình 5 của bài báo, và hiệu năng cuối). Lưu ý: ở thời điểm công bố, so sánh với SAC được tác giả TD3 lưu ý có khác biệt với kết quả SAC tự báo cáo (chi tiết trong phụ lục, không có số cụ thể trong phần chính).

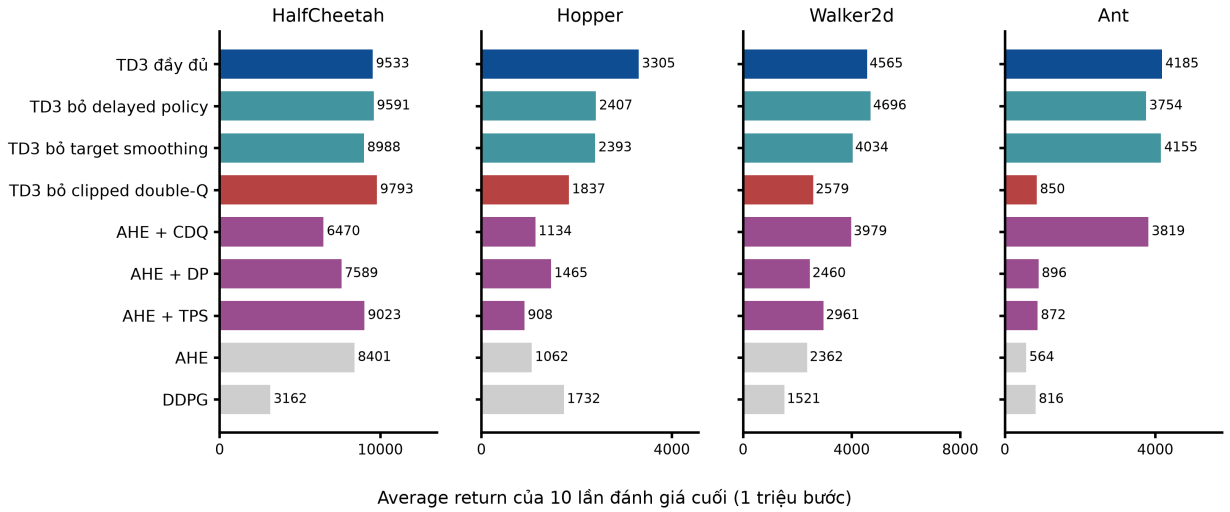
5. Phân tích / ablation

Bảng 2 (Average Return 10 evaluation cuối, 10 trial, 1M bước): ablation từng thành phần

Phương pháp	HalfCheetah	Hopper	Walker2d	Ant
TD3 (đầy đủ)	9532.99	3304.75	4565.24	**4185.06**
DDPG	3162.50	1731.94	1520.90	816.35
AHE (chỉ kiến trúc+siêu tham số+khám phá mới)	8401.02	1061.77	2362.13	564.07
AHE + DP (delayed policy)	7588.64	1465.11	2459.53	896.13
AHE + TPS (target policy smoothing)	9023.40	907.56	2961.36	872.17
AHE + CDQ (clipped double-Q)	6470.20	1134.14	**3979.21**	3818.71
TD3 - DP (bỏ delayed policy)	**9590.65**	2407.42	4695.50	3754.26
TD3 - TPS (bỏ target smoothing)	8987.69	2392.59	4033.67	4155.24
TD3 - CDQ (bỏ clipped double-Q)	9792.80	1837.32	2579.39	849.75
DQ-AC (Double Q-learning actor-critic)	9433.87	1773.71	3100.45	2445.97
DDQN-AC (Double DQN actor-critic)	**10306.90**	2155.75	3116.81	1092.18

Nhận xét của tác giả:

- Thêm **một** thành phần riêng lẻ (AHE+DP, AHE+TPS, AHE+CDQ) cải thiện không nhiều; **kết hợp** các thành phần (TD3 đầy đủ) mới cho cải thiện lớn, nhất quán trên hầu hết task.
- Dù actor chỉ được huấn luyện bằng một nửa số lần cập nhật (do delay $d = 2$), việc có delayed policy update thường cải thiện hiệu năng **và** giảm thời gian huấn luyện.
- DQ-AC và DDQN-AC (cũng được trang bị delayed update + target smoothing + kiến trúc mới để so sánh công bằng) giảm overestimation ít hơn Clipped Double Q, và hiệu năng kém hơn TD3-CDQ (TD3 không có clipped double-Q) trên hầu hết task, **trừ Ant-v1** nơi bất kỳ cách giảm overestimation nào cũng giúp nhiều (vì Ant có vẻ đặc biệt nhạy với overestimation).



Hình 41: Ablation của TD3 trên bốn môi trường MuJoCo (average return 10 lần đánh giá cuối, số làm tròn đến đơn vị) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi panel là một môi trường, hàng xanh dương là TD3 đầy đủ, hàng xanh ngọc là bỏ delayed policy hoặc target smoothing, hàng đỏ là bỏ clipped double-Q. Bỏ clipped double-Q làm tụt mạnh nhất ở

Hopper, Walker2d và Ant (ví dụ Ant 4185 xuống 850), còn thêm riêng từng thành phần vào AHE cải thiện không đều.

6. Giới hạn

Tác giả nêu:

- Chứng minh hội tụ của Clipped Double Q-learning (Mục 4.2) chỉ có trong **thiết lập MDP hữu hạn (tabular)**, không phải cho mạng nơ-ron sâu (chi tiết chứng minh để ở phụ lục).
- $d = 2$ được chọn để **so sánh công bằng** (critic chỉ huấn luyện một lần mỗi time step); d lớn hơn "sẽ mang lại lợi ích lớn hơn về mặt tích lũy sai số" nhưng không được thử nghiệm đầy đủ trong bài, và huấn luyện actor quá ít lần "sẽ làm hỏng việc học" (không có ngưỡng cụ thể).
- Minimum của hai ước lượng "có thể gây ra underestimation bias", được paper xem là chấp nhận được hơn overestimation nhưng không định lượng mức độ underestimation.

Nhận xét:

- Ablation (Bảng 2) chỉ chạy trên 4/7 môi trường (thiếu Reacher, InvertedPendulum, InvertedDoublePendulum).
- Kiến trúc "our DDPG" (Adam lr 10^{-3} cho cả actor/critic, critic nhận (s, a) ngay từ đầu) khác DDPG gốc ở nhiều chỗ cùng lúc với 3 cải tiến chính, nên một phần cải thiện của TD3 so với "DDPG" gốc (không phải "our DDPG") có thể đến từ các thay đổi kiến trúc/kỹ thuật này, không chỉ ba ý tưởng chính; bài báo có tách riêng "our DDPG" làm baseline công bằng hơn để kiểm soát điều này, nhưng cách chọn các thay đổi phụ (ví dụ bỏ OU process) không được ablation riêng.
- Không có kết quả trên môi trường học từ pixel (khác DDPG, vốn có cả thí nghiệm pixel).

7. Liên hệ với các bài khác

- **10 DDPG:** TD3 chính là DDPG + 3 cơ chế sửa lỗi; mọi phần không đổi (actor xác định, DPG gradient, replay buffer) giữ nguyên từ bài 10.
- **01 DQN, 02 Double DQN:** nguồn gốc overestimation bias (Q-learning rời rạc) và cách sửa cho rời rạc (Double Q-learning, Double DQN); TD3 chỉ ra hai cách này không đủ cho actor-critic liên tục và đề xuất bản "clipped".
- **12 SAC:** công trình đồng thời; cả hai cùng dùng "twin Q" (SAC trích dẫn trực tiếp ý tưởng lấy min hai critic từ TD3) nhưng SAC còn thêm actor ngẫu nhiên tối đa entropy. Nên đọc liền sau để so sánh hai hướng giải quyết overestimation/brittleness của DDPG.

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp: thấp (TD3 vẫn là actor xác định cho hành động liên tục, không áp dụng cho sinh token rời rạc như LLM/OCR), nhưng ba nguyên lý có thể mang sang nơi khác:

- **Clipped Double Q / giá trị bi quan:** khi huấn luyện bất kỳ hệ thống có "critic"/mô hình giá trị định hướng policy (ví dụ mô hình phần thưởng trong RLHF, hoặc value head trong các quy trình agentic RL, bài 53-54), lấy **min** của nhiều ước lượng độc lập (ensemble nhỏ) là cách rẻ để giảm rủi ro bị policy khai thác lỗi lạc quan của một ước lượng duy nhất.
- **Delayed update:** ý tưởng "đừng cập nhật policy theo một critic chưa đủ ổn định" tương tự lý do PPO/TRPO (bài 04, 07) giới hạn số bước cập nhật trong một epoch dữ liệu rollout; nguyên lý chung: tách tốc độ học của phần "đánh giá" và phần "cải thiện" khi cả hai cùng học bằng gradient.
- **Target policy smoothing:** ý tưởng "làm mượt mục tiêu quanh lân cận hành động" gợi liên hệ tới các kỹ thuật regularize đầu ra sinh (ví dụ label smoothing, hoặc lấy trung bình trên nhiều mẫu nhiễu) khi student/mô hình nhỏ dễ khớp quá mức vào các điểm hẹp của một signal học (so sánh với KD/distillation, không trực tiếp từ TD3).

9. Thí nghiệm gợi ý

1. Trên một continuous-control nhỏ (ví dụ Pendulum/HalfCheetah rút gọn), đo trực tiếp $Q_\theta(s, \pi(s))$ trung bình so với return thực đo bằng rollout (giống Hình 1 paper) cho ba biến thể: DDPG, TD3 chỉ có CDQ, TD3 đầy đủ; so sánh độ lệch.
2. Ablation riêng $d \in \{1, 2, 4, 8\}$ với các thành phần khác giữ nguyên, đo tốc độ học và hiệu năng cuối, kiểm tra tuyên bố "d lớn hơn có lợi hơn về tích lũy sai số" của tác giả.
3. Thay nhiễu target smoothing $\sigma \in \{0, 0.1, 0.2, 0.4\}$ (0 = không smoothing), đo mức độ "khớp đỉnh hẹp" bằng cách so phân phối $Q_\theta(s, \cdot)$ quanh hành động được chọn trước và sau khi thêm smoothing.
4. So sánh min-of-2 với min-of-N ($N = 3, 5$ critic) để xem đánh đổi giữa giảm overestimation và tăng underestimation/chi phí tính toán.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Overestimation (actor-critic)	$\mathbb{E}[Q_\theta(s, \pi_{approx}(s))] \geq \mathbb{E}[Q^\pi(s, \pi_{approx}(s))]$	hệ quả của gradient-based update, không chỉ arg max rời rạc
Clipped Double Q target	$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi_1}(s'))$	cận trên hóa bằng min
Tích lũy sai số TD	$Q_\theta(s_t, a_t) = \mathbb{E}[\sum_i \gamma^{i-t}(r_i - \delta_i)]$	phương sai tăng theo γ nếu δ không kiểm soát
Delayed policy update	cập nhật actor/target mỗi d bước critic	$d = 2$; two-timescale
Target policy smoothing	$y = r + \gamma \min_i Q_{\theta'_i}(s', \pi_{\phi'}(s') + \epsilon)$, $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$	$\sigma = 0.2$, $c = 0.5$
Siêu tham số chính	lr 10^{-3} (cả hai mạng), $\tau = 0.005$, batch 100	khác DDPG (lr riêng, $\tau = 0.001$)
Nhiễu khám phá	$\mathcal{N}(0, 0.1)$ không tương quan	khác OU của DDPG, không thấy lợi ích từ OU

11. Kết luận ngắn

TD3 chỉ ra rằng overestimation bias, vốn được biết rõ ở Q-learning rời rạc, cũng xuất hiện có hệ thống trong actor-critic liên tục, và các cách sửa chuẩn cho rời rạc (Double DQN) không đủ hiệu quả khi chính sách thay đổi chậm. Ba cơ chế: clipped double Q-learning, delayed policy update, target policy smoothing, kết hợp lại tạo thành TD3, cải thiện rõ rệt so với DDPG trên 7 môi trường MuJoCo và vượt qua nhiều baseline on-policy/off-policy khác ở thời điểm công bố, với chi phí cài đặt không lớn hơn DDPG đáng kể.

Link gốc: https://arxiv.org/abs/1802.09477

Bài 12

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor (2018)

Tác giả: Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, Sergey Levine (Berkeley Artificial Intelligence Research, UC Berkeley).
Venue / arXiv: arXiv:1801.01290 (v2, 08/08/2018); bài báo không nêu venue trong văn bản trích xuất (phiên bản ICML 2018 của SAC, không kèm bước tự động chỉnh nhiệt độ; bản mở rộng sau có tên "Soft Actor-Critic Algorithms and Applications" là một paper khác, arXiv:1812.05905, không thuộc phạm vi ghi chú này).
Nguồn: https://arxiv.org/abs/1801.01290
Chủ đề: Actor-critic off-policy cho hành động liên tục, dựa trên khung **maximum entropy RL**: tối ưu đồng thời reward kỳ vọng và entropy của chính sách. Khác DDPG/TD3 (actor xác định), SAC học một **actor ngẫu nhiên**, cho ổn định hơn và khám phá tốt hơn, cũng mượn ý tưởng "twin Q" từ TD3 để giảm overestimation.
Vị trí trong lộ trình: Đọc sau bài 10 (DDPG) và 11 (TD3, SAC trích dẫn và so sánh trực tiếp với TD3 như "concurrent work"). Đây là bài cuối "Giai đoạn 3: continuous control actor-critic off-policy". Khung maximum-entropy RL ở đây còn quay lại ở bài 28 (MiniLLM, distillation) dưới góc nhìn max-entropy IRL, và liên hệ khái niệm với RLHF có KL-regularization (bài 23-26, 29 DPO) ở các giai đoạn sau.
Chương nền liên quan: nền 04: Actor-critic và GAE

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Hai vấn đề lớn của deep RL model-free: **sample complexity** cao (cần hàng triệu bước, đặc biệt các thuật toán on-policy như TRPO/PPO/A3C, vì mỗi bước gradient cần mẫu mới) và **brittleness** (nhạy siêu tham số). DDPG (bài 10) off-policy nên hiệu quả dữ liệu hơn on-policy, nhưng "nổi tiếng khó dùng vì cực kỳ dễ vỡ và nhạy siêu tham số" (nhận xét của chính paper SAC, dẫn Duan et al. 2016; Henderson et al. 2017).

Maximum entropy RL (Ziebart et al. 2008; Toussaint 2009; Rawlik et al. 2012; Fox et al. 2016; Haarnoja et al. 2017, "soft Q-learning") thay mục tiêu RL chuẩn $\sum_t \mathbb{E}[r(s_t, a_t)]$ bằng mục tiêu cộng thêm entropy chính sách:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))] \qquad (1)$$

α ("temperature") kiểm soát mức quan trọng của entropy so với reward; $\alpha \to 0$ khôi phục mục tiêu RL chuẩn. Lợi ích về khái niệm: (i) chính sách được khuyến khích khám phá rộng hơn, bỏ qua các hướng rõ ràng không hứa hẹn; (ii) chính sách có thể giữ **nhiều mode** gần-tối-ưu cùng lúc (nếu nhiều hành động đều tốt tương đương, chính sách chia đều xác suất, không chọn một cách tùy tiện); (iii) thực nghiệm cho thấy cải thiện tốc độ học và khám phá.

Trước SAC, các phương pháp max-entropy on-policy có sample complexity kém; các phương pháp off-policy (soft Q-learning) cần suy luận xấp xỉ phức tạp trong không gian hành động liên tục và **không phải actor-critic thật**: trong soft Q-learning, mạng actor chỉ là một "bộ lấy mẫu xấp xỉ" cho posterior của Q tối ưu, không trực tiếp ảnh hưởng Q ngoài qua phân phối dữ liệu, nên sự hội tụ phụ thuộc vào việc sampler này xấp xỉ tốt đến đâu.

Khái niệm cần: **policy iteration** (lặp giữa policy evaluation: tính V^π/Q^π cho π cố định, và policy improvement: cải thiện π dựa trên giá trị đó), **soft Bellman backup**, **KL divergence** $D_{KL}(\pi \| \exp(Q)/Z)$, **reparameterization trick** (viết hành động ngẫu nhiên $a = f_\phi(\epsilon; s)$ với ϵ là nhiễu cố định, để lấy gradient qua hành động mà vẫn khả vi), **tanh squashing** (bó buộc hành động Gauss vô hạn vào khoảng hữu hạn).

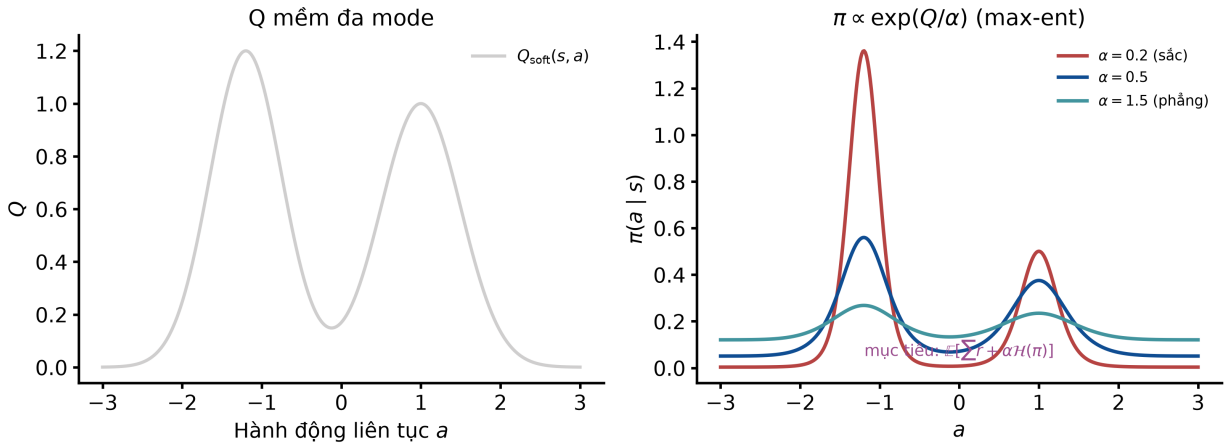
2. Ý tưởng chính

Rút ra **soft policy iteration**: một dạng tổng quát của policy iteration cho mục tiêu max-entropy, chứng minh hội tụ về chính sách tối ưu trong một tập chính sách Π (ví dụ họ Gaussian) ở thiết lập tabular. Từ đó xây dựng **Soft Actor-Critic (SAC)**: xấp xỉ hai bước (soft policy evaluation, soft policy improvement) bằng gradient ngẫu nhiên cho hàm giá trị V_ψ, hàm Q Q_θ (hai bản, lấy min như TD3 để giảm overestimation), và chính sách ngẫu nhiên π_ϕ (Gaussian, dùng reparameterization trick để lấy gradient). Kết quả: off-policy, dùng replay buffer như DDPG/TD3, nhưng ổn định hơn nhiều nhờ actor ngẫu nhiên tối đa entropy, đạt hiệu năng vượt DDPG rõ rệt trên các tác vụ khó (ví dụ Humanoid 21 chiều hành động).

3. Phương pháp chi tiết

3.1 Soft policy evaluation (Lemma 1)

Với chính sách π cố định, định nghĩa **soft Bellman backup operator**:



SAC: $\pi \propto \exp(Q/\alpha)$; α nhỏ → policy sắc, α lớn → đa mode (minh hoạ, tự tính)

$$\boxed{\mathcal{T}^\pi Q(s_t, a_t) \triangleq r(s_t, a_t) + \gamma\, \mathbb{E}_{s_{t+1}}[V(s_{t+1})]} \qquad (2)$$

với **soft state value function**:

$$\boxed{V(s_t) = \mathbb{E}_{a_t \sim \pi}[Q(s_t, a_t) - \log \pi(a_t|s_t)]} \qquad (3)$$

So với Bellman thường, V cộng thêm $-\log \pi(a_t|s_t)$ chính là entropy tức thời của π tại s_t (kỳ vọng của $-\log \pi$ là entropy); đây là cách đưa entropy vào "giá trị" một cách đệ quy qua các bước. Lemma 1: áp dụng lặp $Q_{k+1} = \mathcal{T}^\pi Q_k$ từ bất kỳ Q_0 hữu hạn sẽ hội tụ về soft Q-value của π (chứng minh: viết lại thành Bellman thường với "entropy-augmented reward" $r^\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1}}[\mathcal{H}(\pi(\cdot|s_{t+1}))]$, rồi áp kết quả hội tụ chuẩn của policy evaluation).

3.2 Soft policy improvement (Lemma 2)

Cập nhật π theo hướng phân phối Boltzmann của Q hiện tại, chiếu vào tập chính sách tractable Π bằng **information projection** (KL divergence):

$$\boxed{\pi_{new} = \arg\min_{\pi' \in \Pi} D_{KL}\Big(\pi'(\cdot|s_t) \,\Big\|\, \frac{\exp(Q^{\pi_{old}}(s_t,\cdot))}{Z^{\pi_{old}}(s_t)}\Big)} \qquad (4)$$

$Z^{\pi_{old}}$ (hàm phân hoạch, partition function) chuẩn hóa phân phối nhưng không phụ thuộc π' nên không ảnh hưởng gradient, có thể bỏ qua khi tối ưu. Lemma 2 (chứng minh phụ lục B.2): $Q^{\pi_{new}}(s,a) \geq Q^{\pi_{old}}(s,a)$ với mọi (s,a), nghĩa là bước chiếu này **không làm giảm** giá trị, dù chiếu vào tập Π hạn chế. Trực giác chứng minh: vì π_{new} tối thiểu hóa KL nên tốt hơn hoặc bằng việc giữ π_{old} (so sánh tại chính $\pi_{old} \in \Pi$), suy ra $\mathbb{E}_{\pi_{new}}[Q^{\pi_{old}} - \log \pi_{new}] \geq V^{\pi_{old}}$, rồi khai triển lặp lại Bellman equation cho $Q^{\pi_{old}}$ bằng chính bất đẳng thức này sẽ "đẩy" dần giá trị lên tới $Q^{\pi_{new}}$.

Theorem 1 (Soft Policy Iteration): lặp lại hai bước evaluation/improvement từ bất kỳ $\pi \in \Pi$ sẽ hội tụ về π^* sao cho $Q^{\pi^*}(s,a) \geq Q^{\pi}(s,a)$ với mọi $\pi \in \Pi$ (giả thiết $|A| < \infty$, chứng minh ở phụ lục B.3, dựa trên chuỗi Q^{π_i} tăng đơn điệu và bị chặn trên vì reward và entropy bị chặn).

3.3 Từ tabular sang hàm xấp xỉ: Soft Actor-Critic

Thuật toán tabular trên không khả thi với không gian liên tục/lớn; SAC xấp xỉ bằng ba mạng tham số: V_ψ, Q_θ (critic), π_ϕ (actor), xen kẽ tối ưu gradient ngẫu nhiên cho cả ba thay vì chạy evaluation/improvement tới hội tụ.

Hàm giá trị trạng thái V_ψ (không bắt buộc về lý thuyết vì V suy ra được từ Q, π theo (3), nhưng paper thấy có mạng V riêng giúp ổn định hơn):

$$J_V(\psi) = \mathbb{E}_{s_t \sim D}\Big[\tfrac{1}{2}\big(V_\psi(s_t) - \mathbb{E}_{a_t \sim \pi_\phi}[Q_\theta(s_t,a_t) - \log \pi_\phi(a_t|s_t)]\big)^2\Big] \qquad (5)$$

$$\boxed{\hat{\nabla}_\psi J_V(\psi) = \nabla_\psi V_\psi(s_t)\big(V_\psi(s_t) - Q_\theta(s_t,a_t) + \log \pi_\phi(a_t|s_t)\big)} \qquad (6)$$

với a_t lấy mẫu từ π_ϕ hiện tại (không phải từ buffer).

Hàm Q (critic) tối thiểu soft Bellman residual:

$$J_Q(\theta) = \mathbb{E}_{(s_t,a_t)\sim D}\Big[\tfrac{1}{2}\big(Q_\theta(s_t,a_t) - \hat{Q}(s_t,a_t)\big)^2\Big], \qquad \hat{Q}(s_t,a_t) = r(s_t,a_t) + \gamma\, \mathbb{E}_{s_{t+1}}[V_{\bar{\psi}}(s_{t+1})] \qquad (7,8)$$

$$\boxed{\hat{\nabla}_\theta J_Q(\theta) = \nabla_\theta Q_\theta(s_t,a_t)\big(Q_\theta(s_t,a_t) - r(s_t,a_t) - \gamma V_{\bar{\psi}}(s_{t+1})\big)} \qquad (9)$$

$V_{\bar{\psi}}$ là **target value network**, $\bar{\psi}$ là trung bình động (hoặc copy định kỳ, phụ lục E) của ψ, như target network của DQN/DDPG, giúp ổn định mục tiêu.

Chính sách (actor): tối thiểu trực tiếp KL trong (4):

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D}\Big[D_{KL}(\pi_\phi(\cdot|s_t) \,\|\, \exp(Q_\theta(s_t,\cdot))/Z_\theta(s_t))\Big] \qquad (10)$$

Vì mục tiêu (phân phối Boltzmann của Q_θ) là một mạng khả vi (không phải dữ liệu cố định), bài báo dùng **reparameterization trick** thay vì likelihood-ratio gradient (REINFORCE) để có gradient phương sai thấp hơn: viết hành động là một hàm khả vi của trạng thái và nhiễu cố định:

$$a_t = f_\phi(\epsilon_t; s_t) \qquad (11)$$

(ϵ_t lấy từ một phân phối cố định, ví dụ Gaussian cầu). Thay vào (10), bỏ Z_θ (không phụ thuộc ϕ):

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D, \epsilon_t \sim \mathcal{N}}[\log \pi_\phi(f_\phi(\epsilon_t; s_t)|s_t) - Q_\theta(s_t, f_\phi(\epsilon_t; s_t))] \qquad (12)$$

$$\boxed{\hat{\nabla}_\phi J_\pi(\phi) = \nabla_\phi \log \pi_\phi(a_t|s_t) + (\nabla_{a_t} \log \pi_\phi(a_t|s_t) - \nabla_{a_t} Q(s_t, a_t)) \nabla_\phi f_\phi(\epsilon_t; s_t)} \qquad (13)$$

Đây là bản tổng quát hóa của gradient kiểu DDPG (bài 10) cho **bất kỳ chính sách ngẫu nhiên tractable** nào (không chỉ Gaussian), vì DDPG chỉ là trường hợp policy xác định.

Twin Q (mượn từ TD3, bài 11): dùng hai mạng $Q_{\theta_1}, Q_{\theta_2}$ huấn luyện độc lập theo (7)-(9), rồi lấy min cho cả gradient của V (6) và của π (13), để giảm positive bias (overestimation) vốn làm hại các phương pháp value-based. Paper nhận thấy một Q duy nhất vẫn học được cả Humanoid 21 chiều, nhưng hai Q **tăng tốc học rõ rệt**, đặc biệt ở task khó.

3.4 Thuật toán đầy đủ (Algorithm 1)

```
Khởi tạo psi, psi_bar, theta (hai Q), phi
for each iteration:
    for each environment step:
        a ~ pi_phi(a|s)
        s' ~ p(s'|s,a)
        D <- D union {(s,a,r(s,a),s')}
    for each gradient step:
        psi     <- psi     - lam_V  * grad_psi J_V(psi)                 # (6)
        theta_i <- theta_i  - lam_Q  * grad_theta_i J_Q(theta_i), i=1,2  # (9), dùng
        ↪  min(Q1,Q2) trong target
        phi     <- phi     - lam_pi * grad_phi J_pi(phi)                # (13)
        psi_bar <- tau*psi + (1-tau)*psi_bar                            # soft update
        ↪  target V
```

3.5 Bó buộc hành động: tanh squashing (Phụ lục C)

Vì π_ϕ là Gaussian (hỗ trợ vô hạn), cần bó hành động vào khoảng hữu hạn $(-1, 1)$. Lấy u từ Gaussian với mật độ $\mu(u|s)$, đặt $a = \tanh(u)$ (áp elementwise); theo công thức đổi biến với Jacobian đường chéo $da/du = \text{diag}(1 - \tanh^2(u))$:

$$\boxed{\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^{D} \log\left(1 - \tanh^2(u_i)\right)} \qquad (21)$$

Ví dụ nhỏ (tự tính): với $u = 1$ (một chiều), $\tanh(1) \approx 0.7616$. Số hạng hiệu chỉnh: $1 - \tanh^2(1) \approx 1 - 0.5800 = 0.4200$, $\log(0.4200) \approx -0.8675$. Vậy log-likelihood của hành động đã squash **nhỏ hơn** log-likelihood Gaussian gốc tại u khoảng 0.87 nat, phản ánh việc "nén" không gian làm tập trung mật độ (Jacobian < 1 quanh vùng bão hòa của tanh).

3.6 Reward scale = nghịch đảo temperature (Mục 5.2, phân tích độ nhạy)

Vì SAC tối ưu $r + \alpha\mathcal{H}(\pi)$, và trong thực nghiệm α được gộp vào reward bằng cách nhân reward với một **hệ số (reward scale)**, hệ số này đóng vai trò $1/\alpha$ (reward lớn hơn tương đương temperature thấp hơn, chính sách ít ngẫu nhiên hơn). Đây là siêu tham số nhạy nhất của SAC (phiên bản này chưa có cơ chế tự động điều chỉnh α, khác bản mở rộng sau).

Ví dụ nhỏ minh họa ý nghĩa temperature (tự tính, không phải số liệu paper): hai hành động rời rạc với $Q(a_1) = 1,\ Q(a_2) = 0$. Chính sách Boltzmann $\pi(a) \propto \exp(Q(a)/\alpha)$:

- $\alpha = 1$ (reward scale thấp): $\pi(a_1) = \frac{e^1}{e^1+e^0} = \frac{2.718}{3.718} \approx 0.731$, $\pi(a_2) \approx 0.269$: còn khá ngẫu nhiên dù a_1 tốt hơn rõ.
- $\alpha = 0.1$ (reward scale cao, temperature thấp): $\pi(a_1) = \frac{e^{10}}{e^{10}+e^0} \approx 0.99995$: gần như xác định, chọn a_1 hầu như luôn luôn.

Minh họa đúng hướng paper: reward magnitude lớn (tương đương α nhỏ) đẩy chính sách tới gần xác định; reward quá nhỏ (temperature cao) khiến chính sách gần đều, không khai thác được tín hiệu reward.

4. Thí nghiệm và kết quả chính

Môi trường: OpenAI Gym MuJoCo (Hopper-v1, Walker2d-v1, HalfCheetah-v1, Ant-v1, Humanoid-v1) và Humanoid (rllab, 21 chiều hành động, được xem là rất khó cho off-policy methods). Baseline: DDPG, PPO, Soft Q-Learning (SQL, với 2 Q-function để công bằng), TD3 (dùng code chính chủ tác giả TD3, mô tả là "concurrent work"). 5 seed mỗi thuật toán, đánh giá mỗi 1000 bước; đường cong là trung bình, vùng tô là min-max trên 5 trial.

Kết quả định tính (Hình 1): SAC tương đương các baseline ở task dễ, **vượt rõ rệt** ở task khó (Ant-v1, Humanoid-v1, Humanoid rllab) cả về tốc độ học và hiệu năng cuối. DDPG **không học được gì** trên Ant-v1, Humanoid-v1, Humanoid (rllab) (nhất quán với Duan et al. 2016; Gu et al. 2016). PPO học chậm hơn do cần batch lớn để ổn định trên task phức tạp. SQL học được mọi task nhưng chậm hơn và kém hơn SAC về hiệu năng cuối. Mọi siêu tham số SAC dùng trong thí nghiệm này liệt kê ở Bảng 1/2 phụ lục (xem mục 3.7 bên dưới).

Bảng: siêu tham số SAC (Phụ lục D, Bảng 1 và 2 của paper)

Tham số chung	Giá trị
optimizer	Adam
learning rate	3×10^{-4}
discount γ	0.99
replay buffer size	10^6
số lớp ẩn (mọi mạng)	2
số unit mỗi lớp ẩn	256
minibatch	256
nonlinearity	ReLU
target smoothing τ (soft update)	0.005
target update interval (soft)	1
gradient steps (soft)	1

Môi trường	Số chiều hành động	Reward scale
Hopper-v1	3	5

Môi trường	Số chiều hành động	Reward scale
Walker2d-v1	6	5
HalfCheetah-v1	6	5
Ant-v1	8	5
Humanoid-v1	17	20
Humanoid (rllab)	21	10

5. Phân tích / ablation

- **Chính sách ngẫu nhiên vs. xác định (Hình 2, Humanoid rllab, 5 seed mỗi biến thể):** một biến thể "xác định" của SAC (bỏ entropy ở cả policy và value update, hard target update, không target actor riêng, nhiễu khám phá cố định, gần giống DDPG nhưng vẫn có hai Q) cho hiệu năng trung bình không khác nhiều, nhưng **độ biến thiên giữa các seed cực cao** (kém ổn định), trong khi SAC (có entropy) ổn định hơn rõ rệt giữa các seed. Kết luận của tác giả: học chính sách ngẫu nhiên với tối đa entropy có thể **ổn định hóa huấn luyện** đáng kể, quan trọng hơn khi task khó và việc chỉnh siêu tham số khó khăn.
- **Đánh giá bằng mean action (Hình 3a, Ant-v1):** vì SAC tối ưu cả entropy, dùng mean của phân phối chính sách (bỏ ngẫu nhiên) khi đánh giá final performance cho kết quả tốt hơn là giữ ngẫu nhiên khi đánh giá.
- **Độ nhạy reward scale (Hình 3b, Ant-v1):** reward scale quá nhỏ làm chính sách gần đều, không khai thác được tín hiệu thưởng, hiệu năng giảm mạnh; quá lớn làm chính sách nhanh gần-xác định, dễ kẹt ở local minimum do thiếu khám phá; mức phù hợp cho kết quả tốt nhất cả tốc độ học và hiệu năng cuối. Tác giả: "trong thực tế, reward scale là siêu tham số duy nhất cần tinh chỉnh" cho SAC phiên bản này, và diễn giải tự nhiên của nó (nghịch đảo temperature) cho trực giác cách chọn.
- **Độ nhạy τ (Hình 3c, Ant-v1):** τ lớn (target di chuyển nhanh) gây mất ổn định; τ nhỏ (target di chuyển chậm) làm học chậm hơn.
- **Baseline bổ sung (Phụ lục E, Hình 4):** so với Trust-PCL (học chậm, chỉ giải được task dễ trong số bước cho phép), và hai biến thể SAC (hard target update theo chu kỳ vs. soft update theo trung bình động; và SAC xác định như trên): soft/hard target update cho hiệu năng tương đương trên hầu hết task, SAC (soft update) nhanh nhất trên Humanoid (rllab).

6. Giới hạn

Tác giả nêu:

- Reward scale phải tinh chỉnh riêng cho mỗi môi trường (Bảng 2 phụ lục có giá trị khác nhau: 5, 10, 20), đây chính là điểm cần "brittleness" nhất còn lại của SAC phiên bản này (phiên bản mở rộng sau tự động chỉnh α, không nằm trong phạm vi bài này).
- Lý thuyết hội tụ (soft policy iteration, Theorem 1) chỉ chứng minh cho thiết lập **tabular,** $|A| < \infty$; thuật toán thực tế với hàm xấp xỉ sâu không có bảo đảm hội tụ tương tự.
- Mục tiêu huấn luyện (reward + entropy) khác mục tiêu vẽ trên các đường cong learning curve (chỉ tổng reward, không gồm entropy), nên các đường cong này không trực tiếp phản ánh giá trị mà SAC đang tối ưu.

Nhận xét:

- Không có đo overestimation bias trực tiếp kiểu Hình 1/3 của TD3 (chỉ dựa vào lập luận lý thuyết + mượn ý tưởng twin-Q); người đọc muốn kiểm chứng định lượng cần xem lại TD3 (bài 11).
- Không so sánh với TD3 trên Bảng số liệu tổng hợp giống TD3's Bảng 1 (chỉ trên biểu đồ Hình 1, không có bảng số chính xác final performance so với TD3 trong phần chính bài báo này).

- 5 seed mỗi thí nghiệm (so với TD3 dùng 10 seed) có thể khiến khoảng tin cậy rộng hơn, dù không ảnh hưởng đến xu hướng định tính rõ rệt quan sát được.

7. Liên hệ với các bài khác

- **10 DDPG, 11 TD3:** SAC thay actor xác định (DDPG/TD3) bằng actor ngẫu nhiên max-entropy; mượn twin-Q trực tiếp từ TD3 để giảm overestimation; so sánh trực tiếp bằng thực nghiệm.
- **00 (preface), đặc biệt KL penalty/RLHF:** mục tiêu (4) chiếu chính sách mới vào KL với phân phối Boltzmann của Q có cùng dạng toán học với các mục tiêu **KL-regularized RL** dùng trong RLHF (tối ưu reward nhưng bị phạt KL với policy tham chiếu, bài 23-26) và DPO (bài 29, suy ra closed-form optimal policy từ reward dưới ràng buộc KL). Khác biệt: SAC ràng buộc KL với phân phối của **chính Q hiện tại** (tự tham chiếu, không có policy cố định bên ngoài) còn RLHF/DPO ràng buộc KL với một policy SFT cố định.
- **28 MiniLLM (distillation):** phần phụ lục A.1 của MiniLLM diễn giải KD dạng reverse-KL như một bài toán **max-entropy inverse RL**, lấy logit của teacher làm "Q-function" và tối đa $\sum_t r_t + \mathcal{H}[q_\theta]$: đây đúng là mục tiêu (1) của SAC, áp cho bài toán sinh token. Nên đọc lại bài 28 sau khi hiểu rõ (1)-(4) ở đây để thấy mạch liên hệ.
- **36 DeepSeekMath/GRPO, 39 DeepSeek-R1:** các phương pháp RLVR cho LLM sau này hiếm khi dùng entropy bonus kiểu SAC trực tiếp (thường kiểm soát entropy qua KL penalty hoặc theo dõi entropy như một tín hiệu chẩn đoán, bài 49 Entropy Mechanism), nhưng trực giác "giữ chính sách đủ ngẫu nhiên để khám phá, tránh collapse" là chung.
- **19 Offline RL tutorial, 20 CQL:** cùng dùng ý tưởng giá trị bảo toàn/entropy-regularized cho off-policy, nhưng trong bối cảnh không được tương tác môi trường (offline), khác SAC (off-policy nhưng vẫn online).

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp với RL cho LLM/VLM: trung bình, chủ yếu qua **khung toán học** (max-entropy, soft Bellman, KL với phân phối Boltzmann của reward/Q), ít qua thuật toán cụ thể (actor Gaussian liên tục không áp dụng cho sinh token rời rạc):

- **Liên hệ khái niệm quan trọng nhất:** mục tiêu max-entropy RL của SAC và mục tiêu KL-regularized RLHF (tối đa $r - \beta\,\mathrm{KL}(\pi\|\pi_{ref})$) đều có nghiệm closed-form dạng Boltzmann/Gibbs của reward, đây là nền tảng toán học chung cho DPO (bài 29) và cho góc nhìn "KD = max-entropy IRL" của MiniLLM (bài 28). Hiểu kỹ phần 3.1-3.2 ở đây (soft Bellman backup, KL projection) giúp đọc hai bài đó nhanh hơn nhiều.
- **Nhiễu/temperature như công cụ cân bằng khám phá-khai thác:** tương tự việc chọn temperature khi sample token (decoding) hoặc hệ số entropy bonus trong RLVR; bài học "reward scale quá cao/thấp đều hại" là lời nhắc khi tinh chỉnh hệ số KL penalty hoặc entropy bonus cho RLVR của LLM.

9. Thí nghiệm gợi ý

1. Cài SAC trên một continuous-control nhỏ (Pendulum hoặc HalfCheetah rút gọn), quét reward scale $\in \{0.1, 1, 5, 20, 100\}$, tái hiện Hình 3b (đường cong hình chữ U theo reward scale).
2. So sánh biến thể SAC "có entropy" và "xác định" (bỏ $\log\pi$ trong (5)/(13), như Hình 2) trên 5 seed, đo phương sai hiệu năng giữa seed để kiểm chứng tuyên bố "entropy ổn định hóa huấn luyện".
3. Thử một Q duy nhất so với hai Q (min) trên Humanoid rút gọn, đo tốc độ học, kiểm chứng tuyên bố "hai Q không bắt buộc nhưng tăng tốc đáng kể, đặc biệt task khó".
4. Liên hệ với RLHF: với một bài toán bandit/toy sinh câu ngắn có mô hình phần thưởng giả lập, cài closed-form Boltzmann policy $\pi(a|s) \propto \exp(Q(s,a)/\alpha)$ và so với chính sách học bằng gradient (12)-(13), kiểm tra hai cách có hội tụ về cùng phân phối không khi α nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu max-entropy	$J(\pi) = \sum_t \mathbb{E}[r + \alpha\mathcal{H}(\pi)]$	$\alpha \to 0$ về RL chuẩn
Soft Bellman backup	$\mathcal{T}^\pi Q = r + \gamma\mathbb{E}_{s'}[V(s')]$, $V = \mathbb{E}_\pi[Q - \log\pi]$	hội tụ (Lemma 1)
Soft policy improvement	$\pi_{new} = \arg\min_\pi D_{KL}(\pi \Vert \exp(Q)/Z)$	không giảm giá trị (Lemma 2)
Loss V	$J_V = \mathbb{E}[\frac{1}{2}(V_\psi - \mathbb{E}_\pi[Q_\theta - \log\pi_\phi])^2]$	dùng mẫu từ π_ϕ hiện tại
Loss Q	$J_Q = \mathbb{E}[\frac{1}{2}(Q_\theta - (r + \gamma V_{\bar\psi}))^2]$	target network $\bar\psi$
Loss π	$J_\pi = \mathbb{E}[\log\pi_\phi(f_\phi(\epsilon;s)\vert s) - Q_\theta(s, f_\phi(\epsilon;s))]$	reparameterization trick
Twin Q	$\min(Q_{\theta_1}, Q_{\theta_2})$ trong (6),(13)	mượn từ TD3
Tanh squashing	$\log\pi(a\vert s) = \log\mu(u\vert s) - \sum_i \log(1 - \tanh^2 u_i)$	$a = \tanh(u)$
Reward scale	$\propto 1/\alpha$	siêu tham số nhạy nhất

11. Kết luận ngắn

SAC xây dựng một actor-critic off-policy dựa trên nền tảng vững chắc của maximum entropy RL: chứng minh soft policy iteration hội tụ trong thiết lập tabular, rồi xấp xỉ bằng ba mạng (V, hai Q, π ngẫu nhiên với reparameterization trick) để hoạt động trong không gian liên tục chiều cao. Kết quả thực nghiệm cho thấy SAC ổn định hơn rõ rệt so với DDPG (nhờ chính sách ngẫu nhiên tối đa entropy) và vượt trội trên các tác vụ khó như Humanoid, trong khi vẫn giữ được hiệu quả dữ liệu của off-policy; đổi lại, reward scale (tương đương nghịch đảo temperature) là siêu tham số cần tinh chỉnh riêng cho từng môi trường ở phiên bản này.

Link gốc: https://arxiv.org/abs/1801.01290

Bài 13

Exploration by Random Network Distillation (2018)

Tác giả: Yuri Burda, Harrison Edwards (OpenAI), Amos Storkey (University of Edinburgh), Oleg Klimov (OpenAI).
Venue / arXiv: arXiv:1810.12894 (v1, 30/10/2018); bài báo không nêu venue trong văn bản trích xuất (không có dòng "Published as..."; đây là bản nộp ICLR, trước khi có stamp hội nghị).
Nguồn: https://arxiv.org/abs/1810.12894
Chủ đề: Thưởng khám phá (exploration bonus) nội tại (intrinsic reward) rẻ và dễ cài: lỗi dự đoán của một mạng được huấn luyện để bắt chước (distill) một mạng **cố định, khởi tạo ngẫu nhiên**. Kết hợp với PPO hai đầu giá trị (hai value head cho intrinsic/extrinsic), đạt kết quả vượt trội trên các game Atari sparse-reward, đặc biệt Montezuma's Revenge.
Vị trí trong lộ trình: Đọc sau bài 01 (DQN, môi trường Atari, Montezuma's Revenge là bài toán khám phá kinh điển) và bài 07 (PPO, RND dùng PPO làm thuật toán tối ưu chính sách nền). Đây là bài mở đầu "Giai đoạn 4: khám phá và lập kế hoạch/mô hình hóa thế giới" (14 AlphaZero, 15 MuZero, 16 DreamerV3 tiếp theo dùng model/planning, một hướng khác để giải sparse reward). Vấn đề "khám phá khi reward thưa" ở đây còn tái xuất hiện trực tiếp trong RL cho agent dùng LLM (bài 53 Search-R1, 54 agentic RL survey), khi reward hành động nhiều bước cũng rất thưa.
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

RL chuẩn tối đa hóa return kỳ vọng; điều này hoạt động tốt khi reward dày đặc và dễ tìm bằng hành động ngẫu nhiên, nhưng thất bại khi reward **thưa (sparse)**: agent hầu như không bao giờ chạm được một reward dương bằng hành động ngẫu nhiên. Bellemare et al. (2016) định danh nhóm Atari game "hard exploration" với reward thưa: Freeway, Gravitar, Montezuma's Revenge, Pitfall!, Private Eye, Solaris, Venture. **Montezuma's Revenge** đặc biệt khó: cần phối hợp nhiều kỹ năng trong game, tránh vật cản chết người, và các phần thưởng cách nhau hàng trăm bước ngay cả khi chơi tối ưu.

Exploration bonus: một lớp phương pháp thay e_t (extrinsic reward, reward thật của môi trường) bằng $r_t = e_t + i_t$, trong đó i_t là thưởng nội tại khuyến khích thăm trạng thái mới. Các dạng trước đó:

- **Count-based:** $i_t = 1/n_t(s)$ hoặc $1/\sqrt{n_t(s)}$ với $n_t(s)$ là số lần thăm trạng thái s (chỉ áp dụng trực tiếp ở không gian trạng thái rời rạc, hữu hạn).
- **Pseudo-count** (Bellemare et al. 2016): tổng quát hóa count bằng thay đổi của một mô hình mật độ trạng thái, hoạt động cả khi trạng thái hầu như chỉ thăm đúng một lần.
- **Lỗi dự đoán (prediction error):** dùng lỗi của một bài toán dự đoán liên quan tới transition của agent, ví dụ **forward dynamics** (dự đoán s_{t+1} từ s_t, a_t) hoặc **inverse dynamics**. Lỗi này giảm khi agent thu thập thêm kinh nghiệm tương tự.

Vấn đề "noisy-TV": nếu bài toán dự đoán có phần **ngẫu nhiên vốn có** (ví dụ forward dynamics ở một môi trường có nhiễu, như một TV chiếu nhiễu trắng tưởng tượng, hay tung đồng xu), agent tối đa hóa lỗi dự đoán sẽ bị hút vào các transition ngẫu nhiên đó mãi mãi, vì lỗi dự đoán **không bao giờ giảm** dù xem bao nhiêu lần (sai số đến từ tính ngẫu nhiên của mục tiêu, aleatoric uncertainty, không phải thiếu dữ liệu). Các phương pháp đo "mức cải thiện của dự đoán" (prediction gain) thay vì lỗi tuyệt đối tránh được vấn đề này nhưng tốn kém tính toán, khó scale.

2. Ý tưởng chính

Thay vì dự đoán một hàm có thể ngẫu nhiên hoặc thiếu thông tin (như dynamics của môi trường), **đặt ra một bài toán dự đoán do chính ta tự tạo, đảm bảo xác định (deterministic)**: dự đoán đầu ra của một **mạng đích (target network) cố định, khởi tạo ngẫu nhiên** trên quan sát hiện tại. Mạng dự đoán (predictor network) được huấn luyện bằng gradient descent để bắt chước (distill) mạng đích này trên dữ liệu agent đã thăm. Vì mục tiêu là một hàm xác định nằm **trong khả năng biểu diễn** của predictor (cùng kiến trúc/lớp hàm), lỗi dự đoán sẽ **giảm theo số lần thăm trạng thái tương tự**, không bị nhiễu bởi aleatoric uncertainty của môi trường: đây là bản chất "obviate các yếu tố 2 và 3" (xem 3.1) khiến RND tránh noisy-TV problem bằng xây dựng, không cần đo prediction-gain tốn kém.

3. Phương pháp chi tiết

3.1 Random Network Distillation

Hai mạng: mạng đích cố định $f : \mathcal{O} \to \mathbb{R}^k$ (khởi tạo ngẫu nhiên, **không huấn luyện**), và mạng dự đoán $\hat{f}(\cdot;\theta) : \mathcal{O} \to \mathbb{R}^k$, huấn luyện để tối thiểu:

$$\boxed{\mathcal{L}(\theta) = \mathbb{E}\big[\, \|\hat{f}(x;\theta) - f(x)\|^2 \,\big]}$$

Thưởng nội tại tại bước t chính là lỗi dự đoán này (chi tiết đo trên observation x_t):

$$i_t = \|\hat{f}(x_t;\theta) - f(x_t)\|^2$$

Bốn nguồn gây lỗi dự đoán nói chung (phần 2.2.1 của bài báo):

1. **Lượng dữ liệu huấn luyện** (epistemic uncertainty): lỗi cao ở nơi ít ví dụ tương tự đã thấy. **Đây là nguồn duy nhất RND muốn khai thác.**
2. **Tính ngẫu nhiên** (aleatoric uncertainty): lỗi cao vì hàm mục tiêu bản thân ngẫu nhiên (ví dụ forward dynamics ở môi trường có nhiễu).
3. **Model misspecification:** lỗi cao vì thiếu thông tin cần thiết hoặc model class quá hạn chế để khớp hàm mục tiêu.
4. **Learning dynamics:** lỗi cao vì quá trình tối ưu không tìm được predictor tốt nhất trong model class.

RND **loại bỏ nguồn 2 và 3 theo thiết kế**: mạng đích được chọn là **xác định** và nằm **trong lớp model** của predictor (cùng kiến trúc), nên về lý thuyết predictor có thể khớp hoàn hảo nếu đủ dữ liệu và học tốt; chỉ còn nguồn 1 (và một phần nguồn 4, không kiểm soát hoàn toàn) đóng góp vào lỗi, đúng là tín hiệu "novelty" mong muốn.

Thí nghiệm minh họa trên MNIST (Hình 2 của bài báo): huấn luyện predictor bắt chước một mạng đích ngẫu nhiên trên hỗn hợp ảnh lớp "0" (vai trò trạng thái đã thăm nhiều) và một lớp mục tiêu (vai trò trạng thái hiếm), thay đổi tỉ lệ lớp mục tiêu nhưng giữ tổng số mẫu cố định; đo MSE trên tập test riêng của lớp mục tiêu. Kết quả: MSE **giảm dần** theo số lượng mẫu lớp mục tiêu đã thấy, xác nhận cơ chế dùng được để phát hiện novelty, và các phương pháp gradient-based tiêu chuẩn không "over-generalize" (tức không tự nhiên khớp hoàn hảo mọi input kể cả chưa thấy, dù về lý thuyết mạng đích chính nó cũng là một predictor hoàn hảo).

3.2 Liên hệ với định lượng độ bất định (Mục 2.2.2)

RND liên hệ với phương pháp ước lượng độ bất định của Osband et al. (2018): xét hồi quy với dữ liệu $D = \{x_i, y_i\}$, tiên nghiệm $p(\theta^*)$ trên tham số của f_{θ^*}; xét họ hàm $g_\theta = f_\theta + f_{\theta^*}$ với $\theta^* \sim p(\theta^*)$, θ tối thiểu lỗi dự đoán kỳ vọng cộng điều hòa từ tiên nghiệm:

$$\theta = \arg\min_{\theta} \mathbb{E}_{(x_i,y_i)\sim D}\left[\|f_\theta(x_i) + f_{\theta^*}(x_i) - y_i\|^2\right] + R(\theta)$$

Osband et al. lập luận họ hàm g_θ (qua nhiều mẫu θ^*) xấp xỉ posterior (tương tự Bayesian linear regression). Nếu đặt $y_i = 0$ (mục tiêu hồi quy về 0), bài toán $\arg\min_\theta \mathbb{E}[\|f_\theta(x_i) + f_{\theta^*}(x_i)\|^2]$ tương đương **distill một hàm ngẫu nhiên lấy từ tiên nghiệm**: mỗi tọa độ đầu ra của predictor/target đóng vai trò một thành viên ensemble (chia sẻ tham số), và MSE ước lượng **phương sai dự đoán của ensemble** (nếu ensemble không thiên lệch). Nói cách khác, lỗi distillation có thể xem như một cách định lượng độ bất định khi dự đoán hàm hằng số 0, rẻ hơn nhiều so với chạy một ensemble thật.

3.3 Kết hợp intrinsic và extrinsic return (Mục 2.3)

Thí nghiệm sơ bộ (chỉ dùng intrinsic reward) cho thấy xử lý bài toán **không theo episode (non-episodic)** (không cắt return khi "game over") cho khám phá tốt hơn episodic. Lý do khái niệm: return nội tại của agent nên liên quan tới **mọi** trạng thái mới nó có thể tìm trong tương lai, bất kể các trạng thái đó rơi vào một episode hay rải qua nhiều episode; dùng episodic intrinsic reward còn có thể "lộ" thông tin về cấu trúc task cho agent (dẫn Burda et al. 2018). Paper đưa ví dụ "Alice chơi game": nếu Alice là agent episodic, một "game over" khiến return tương lai của cô bằng 0 đúng lúc cô đang thử một hành động mạo hiểm nhưng đáng giá về tò mò, agent sẽ trở nên quá né rủi ro; chi phí thật của game over là phải chơi lại từ đầu, không phải return bằng 0 tuyệt đối.

Ngược lại, **extrinsic** reward vẫn nên xử lý episodic (non-episodic cho extrinsic có thể bị khai thác: tìm một reward gần đầu game, cố tình "chết" để quay lại, lặp vô hạn).

Để kết hợp một dòng **non-episodic** (intrinsic) với một dòng **episodic** (extrinsic) mà vẫn ước lượng được value đúng cho mỗi dòng, bài báo dùng tính chất return là tuyến tính theo reward: $R = R_E + R_I$, nên có thể học **hai value head riêng** V_E, V_I (mỗi head theo đúng loại return/discount của dòng reward tương ứng), rồi cộng lại $V = V_E + V_I$. Cách này còn cho phép **hai hệ số chiết khấu khác nhau** $\gamma_E \neq \gamma_I$ cho hai dòng. Bài báo cũng lưu ý: có hai value head riêng có thể có lợi ngay cả khi không cần non-episodic/discount khác nhau, vì cho value function thêm tín hiệu giám sát; điều này đặc biệt quan trọng với exploration bonus vì extrinsic reward **tĩnh (stationary)** còn intrinsic reward **không tĩnh (non-stationary)** (giảm dần khi agent quen trạng thái).

3.4 Chuẩn hóa reward và observation (Mục 2.4)

- **Chuẩn hóa intrinsic reward:** chia cho một ước lượng động (running estimate) của độ lệch chuẩn của intrinsic return, để giữ thang đo ổn định qua các môi trường/thời điểm khác nhau (nếu không, chọn siêu tham số chung cho nhiều game rất khó vì thang đo lỗi dự đoán khác nhau nhiều).
- **Chuẩn hóa observation:** đặc biệt quan trọng cho mạng đích/dự đoán vì mạng đích **cố định**, không thể tự thích nghi thang đo dữ liệu; thiếu chuẩn hóa dễ khiến phương sai của embedding quá thấp, mang rất ít thông tin. Cách làm: whiten mỗi chiều (trừ trung bình động, chia độ lệch chuẩn động), rồi **clip vào** $[-5, 5]$. Khởi tạo tham số chuẩn hóa bằng cách chạy một agent ngẫu nhiên một số bước trước khi tối ưu. Dùng cùng cách chuẩn hóa này cho cả predictor và target (nhưng **không** áp dụng cho mạng policy).

Ví dụ nhỏ (tự tính): nếu một chiều quan sát đã chuẩn hóa đúng là Gaussian chuẩn $\mathcal{N}(0,1)$, xác suất một giá trị vượt ngoài $[-5,5]$ là khoảng 5.7×10^{-7} (hai phía, dùng xấp xỉ đuôi Gaussian). Vậy việc clip ở ± 5 hầu như không cắt bỏ thông tin của phân phối "bình thường", chỉ chặn các giá trị ngoại lai cực hiếm (ví dụ do lỗi chuẩn hóa ban đầu hoặc pixel bất thường), giúp tránh gradient/embedding nổ mà không hy sinh nhiều độ phân giải.

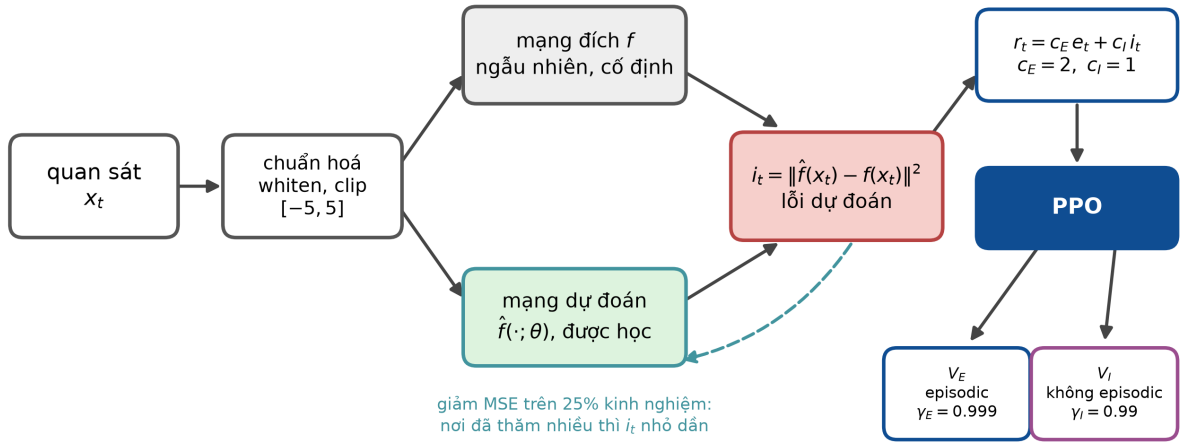
3.5 Kết hợp với PPO: thuật toán tổng thể

RND không phải một thuật toán RL độc lập, mà một **thưởng nội tại** cộng vào bất kỳ thuật toán tối ưu chính sách. Bài báo dùng PPO (bài 07) với sửa đổi: hai value head V_E, V_I (mục 3.3), và reward tổng dùng để tính advantage là tổ hợp:

$$r_t = c_E \cdot e_t^{norm} + c_I \cdot i_t^{norm}$$

(hệ số $c_E = 2$, $c_I = 1$ trong thí nghiệm chính, xem Bảng 5, phần 4). Luồng tổng quát:

```
Khởi tạo target f (cố định, ngẫu nhiên), predictor f_hat(theta), policy PPO 2 value head
↪  V_E, V_I
Chạy agent ngẫu nhiên một số bước để khởi tạo thống kê chuẩn hóa observation
for mỗi rollout (128 bước x 128 môi trường song song):
    thu thập trajectory bằng policy hiện tại
    với mỗi quan sát x_t: chuẩn hóa x_t (whiten, clip [-5,5])
                          i_t = || f_hat(x_t) - f(x_t) ||^2
                          chuẩn hóa i_t bằng running std của intrinsic return
    e_t: extrinsic reward thường (clip [-1,1])
    tính advantage/return riêng cho dòng extrinsic (episodic, gamma_E) và intrinsic
    ↪  (non-episodic, gamma_I)
    cập nhật predictor f_hat bằng gradient MSE trên một phần (25%) batch quan sát  #
    ↪  Table 5
    cập nhật policy + V_E + V_I bằng PPO, dùng r_t = c_E*e_t + c_I*i_t
```



e_t (thưởng thật, clip $[-1, 1]$) đi vào r_t cùng i_t đã chia cho độ lệch chuẩn chạy của intrinsic return.

Mục tiêu của predictor nằm trong lớp hàm của chính nó, nên lỗi chỉ còn do thiếu dữ liệu (novelty).

Hình 42: Sơ đồ RND: mạng đích cố định, mạng dự đoán và phần thưởng nội tại cộng với phần thưởng ngoại tại cho PPO (minh hoạ, tự tính)

Cách đọc hình: quan sát đã chuẩn hoá đi vào cả mạng đích cố định lẫn mạng dự đoán, sai số bình phương giữa hai đầu ra chính là phần thưởng nội tại. Phần thưởng này được cộng với phần thưởng ngoại tại theo hệ số trong ghi chú rồi đưa cho PPO, với hai value head có discount và cách xử lý episode khác nhau. Mạng dự đoán chỉ học từ một phần kinh nghiệm, nên trạng thái càng lạ thì sai số càng lớn.

3.6 Siêu tham số chính (Phụ lục A.3-A.4, Bảng 2,3,4,5)

Hạng mục	Giá trị
Grey-scaling	True
Downsample quan sát	84x84
Clip extrinsic reward	$[-1, 1]$

Hạng mục	Giá trị
Clip intrinsic reward	Không (False)
Max frame/episode	18K
Terminal khi mất mạng (loss of life)	Không dùng
Max-and-skip frame	4
Sticky action probability	0.25 (chống học vẹt action sequence, theo Machado et al. 2017)
Frame stack (policy)	4; chuẩn hóa $x \mapsto x/255$
Frame stack (predictor/target)	1; chuẩn hóa $x \mapsto \text{clip}((x-\mu)/\sigma, [-5,5])$
Rollout length	128
Số rollout mỗi môi trường	30K (tổng ≈1.97 tỉ khung hình, 128 môi trường song song)
Số minibatch / số epoch tối ưu	4 / 4
Hệ số extrinsic / intrinsic	2 / 1
Learning rate	0.0001 (Adam)
λ (GAE)	0.95
Entropy coefficient	0.001
Tỉ lệ kinh nghiệm dùng huấn luyện predictor	0.25
γ_E	0.999
γ_I	0.99
Clip range PPO	$[0.9, 1.1]$
Kiến trúc policy mặc định	CNN (giống Mnih et al. 2015), có thêm biến thể RNN (GRU)

Lưu ý khi tăng số môi trường song song ($32 \to 128/256/1024$): để giữ "tốc độ giảm" của intrinsic reward không đổi, batch huấn luyện predictor được **giữ cố định** bằng cách drop ngẫu nhiên một phần kinh nghiệm (keep probability 0.25 ở 128 môi trường, 0.125 ở 256, 0.03125 ở 1024).

4. Thí nghiệm và kết quả chính

Thiết lập chính: Montezuma's Revenge để phân tích sâu (Mục 3.1-3.5), sau đó so sánh trên 6 game hard-exploration (Mục 3.6): Gravitar, Montezuma's Revenge, Pitfall!, Private Eye, Solaris, Venture.

4.1 Pure exploration (chỉ intrinsic reward, Hình 3)

So sánh episodic vs. non-episodic intrinsic reward (không dùng extrinsic reward trong huấn luyện, chỉ theo dõi để phân tích). Non-episodic khám phá nhiều phòng hơn episodic ở cả hai thước đo (return trung bình, số phòng tìm được). Với $\gamma_I = 0.999$ (non-episodic), một lần chạy tìm được **21 phòng**; return tốt nhất đạt được ở 4/5 lần chạy của thiết lập này là **6.700**.

4.2 Kết hợp episodic/non-episodic (Hình 4, $\gamma_I = \gamma_E = 0.99$)

Dòng intrinsic non-episodic kết hợp với dòng extrinsic episodic khám phá nhiều phòng hơn so với kết hợp cả hai dòng episodic, nhưng return trung bình tương đương; khác biệt rõ hơn với policy RNN. Trái với kỳ vọng ban đầu (mục 2.3), **hai value head không cho lợi ích rõ ràng so với một head** khi cả hai dòng đều episodic; tuy vậy hai head **là bắt buộc** để kết hợp các dòng có đặc tính khác nhau (episodic/non-episodic, discount khác nhau), nên các thí nghiệm sau đều dùng hai head.

4.3 Discount factor (Hình 5)

So $\gamma_E \in \{0.99, 0.999\}$ với $\gamma_I = 0.99$ cố định, và thêm $\gamma_I = 0.999$: tăng γ_E lên 0.999 (giữ $\gamma_I = 0.99$) **cải thiện mạnh** hiệu năng (cho phép agent "nhìn xa" hơn tới extrinsic reward); tăng tiếp γ_I lên 0.999 lại **làm giảm** hiệu năng, trái với quan sát ở phần 4.1 (nơi tăng γ_I không ảnh hưởng đáng kể khi không có extrinsic reward).

4.4 Scale thí nghiệm (Hình 6, $\gamma_I = 0.99, \gamma_E = 0.999$)

Tăng số môi trường song song (và do đó batch size cho policy) cải thiện cả tốc độ học và hiệu năng cuối, với hiệu ứng bão hòa sớm hơn ở CNN so với RNN. Chạy RNN với 32 môi trường kéo dài hơn: đạt mean return **7.570** sau 1.6 tỉ khung hình (1.6 triệu lần cập nhật tham số); một lần chạy thăm **cả 24 phòng** và qua màn 1 một lần, đạt best return **17.500**. RNN với 1024 môi trường: mean return **10.070** cuối huấn luyện, một lần chạy đạt mean return **14.415**.

4.5 So sánh baseline trên 6 game (Hình 7, Bảng 1; policy RNN)

	Gravitar	Montezuma's Revenge	Pitfall!	Private Eye	Solaris	Venture
RND	3.906	**8.152**	-3	**8.666**	3.282	**1.859**
PPO (không bonus)	3.426	2.497	0	105	3.387	0
Dynamics-based bonus	3.371	400	0	33	3.246	1.712
SOTA trước đó	2.209	3.700	0	15.806	12.380	1.813
Người chơi trung bình	3.351	4.753	6.464	69.571	12.327	1.188

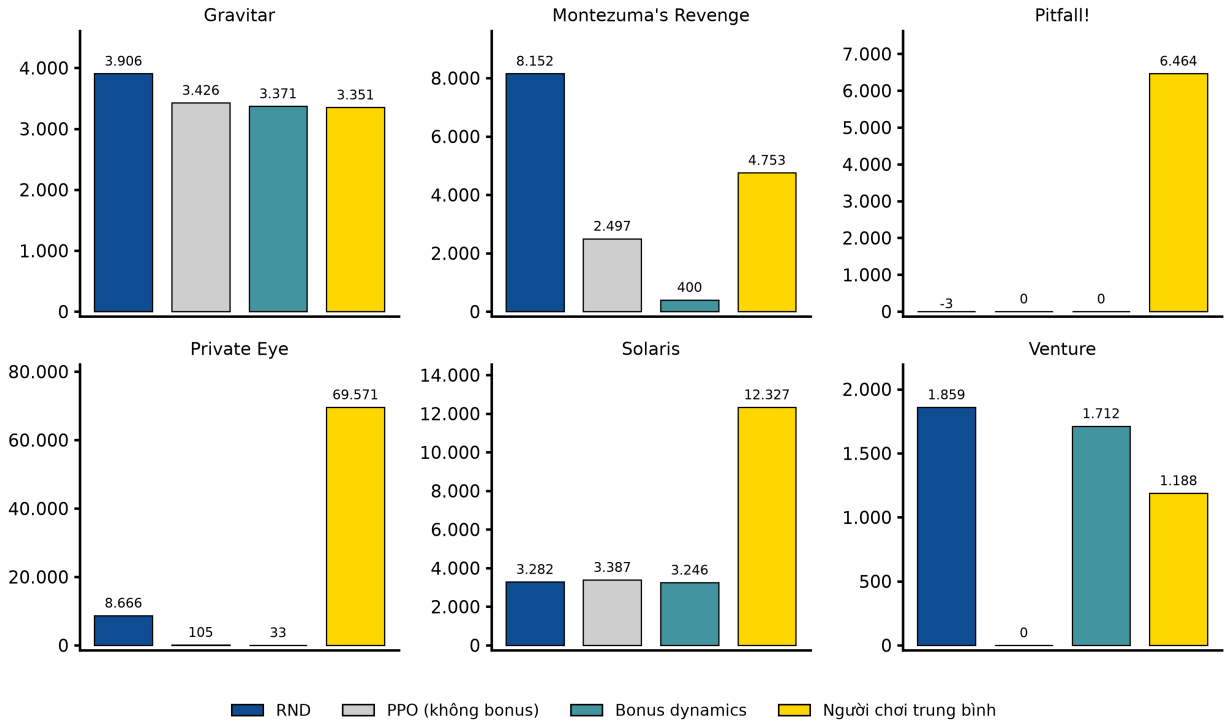
RND đạt SOTA trên Gravitar, Montezuma's Revenge, Venture ở thời điểm công bố; vượt rõ rệt PPO thuần và dynamics-based bonus trên Montezuma's Revenge và Venture; trên Pitfall! cả RND và PPO đều không tìm được reward dương nào (phần thưởng cực hiếm); trên Solaris hiệu năng RND tương đương PPO.

So sánh với dynamics-based bonus: để so sánh công bằng ("apples to apples"), paper đổi RND thành dự đoán **đặc trưng ngẫu nhiên của quan sát kế tiếp** cho trước quan sát/hành động hiện tại (dynamics trong không gian đặc trưng ngẫu nhiên, theo phát hiện của Burda et al. 2018 rằng không gian đặc trưng ngẫu nhiên hoạt động tốt tương đương không gian đặc trưng khác cho mục đích này), giữ nguyên mọi phần khác (hai value head, non-episodic intrinsic, chuẩn hóa...). Dynamics-based kém hơn RND rõ rệt trên Montezuma's Revenge, Private Eye, Solaris; tương đương trên Venture, Pitfall!, Gravitar. Phân tích hành vi: trên Montezuma's Revenge, agent dynamics-based dao động qua lại giữa hai phòng, vì tính không xác định của "sticky action" (hành động lặp lại ngẫu nhiên) gần biên phòng khiến lỗi dự đoán dynamics luôn cao bất kể đã đi qua bao nhiêu lần: biểu hiện cụ thể của **noisy-TV problem**.

Cách đọc hình: mỗi nhóm cột là một game, bốn cột lần lượt là RND, PPO, dynamics bonus và điểm trung bình của người. Hãy chú ý Montezuma's Revenge và Private Eye, nơi RND vượt rõ các baseline, trong khi Pitfall và Solaris cho thấy RND chưa thắng mọi nơi. Số liệu lấy từ bảng ở mục 4.5 của ghi chú, dòng SOTA không được vẽ.

5. Phân tích / ablation

- **Recurrence (Mục 3.5):** môi trường quan sát một phần (ví dụ số chìa khóa hiện có hiển thị trên màn hình, nhưng không rõ lấy từ đâu, đã dùng bao nhiêu, cửa nào đã mở), nên lẽ ra policy đệ quy (RNN/GRU) nên tốt hơn CNN để nhớ lịch sử. Trái kỳ vọng, ở Hình 4 ($\gamma_E = 0.99$) RNN kém hơn CNN; nhưng ở Hình 6 ($\gamma_E = 0.999$) RNN vượt CNN ở mọi mức scale, và nhìn chung qua nhiều game (so Hình 7 và 9 phụ lục) RNN thắng CNN nhiều hơn ngược lại.



Hình 43: So sánh điểm của RND, PPO, dynamics bonus và người chơi trung bình trên sáu game Atari khó (vẽ lại từ số liệu trong ghi chú)

- **"Dancing with skulls" (Mục 3.7, phân tích hành vi định tính):** sau khi agent lấy hết các extrinsic reward nó biết lấy ổn định, nó thường lặp đi lặp lại các hành vi tương tác với vật nguy hiểm (nhảy qua đầu lâu di động, đi giữa cổng laser, nhảy lên/xuống cầu biến mất). Tác giả suy đoán: các trạng thái nguy hiểm này hiếm trong kinh nghiệm quá khứ của agent (vì dễ chết), nên dù đã trải qua một số lần, lỗi dự đoán vẫn tương đối cao so với trạng thái an toàn phổ biến hơn.
- **Độ lặp lại kết quả (ghi chú ở Hình 5/6 của bài báo):** một ghi chú thẳng của tác giả ở Hình 5: kết quả discount factor cho CNN có **sự khác biệt lớn giữa seed** khi chạy lại với 10 seed khác (so với 5 seed ban đầu), nghi do kết quả Montezuma's Revenge bị chi phối bởi các lựa chọn rời rạc đầu game (ví dụ rẽ trái/phải ở phòng đầu) tạo nhiều outlier; cũng có thể một phần do khác biệt phiên bản code. Tác giả khuyến nghị các nghiên cứu sau so sánh với kết quả ở Hình 6 (chạy lại bằng code công khai) thay vì Hình 5.

6. Giới hạn

Tác giả nêu (Mục 5, Discussion):

- RND chỉ xử lý tốt **khám phá cục bộ** (local exploration): các quyết định ngắn hạn như tương tác hay tránh một vật thể. **Khám phá toàn cục** (global exploration), đòi hỏi phối hợp quyết định trên chân trời dài, **vượt ngoài khả năng của phương pháp này**.
- Ví dụ cụ thể paper đưa ra: để qua màn 1 Montezuma's Revenge, agent cần vào một phòng khóa sau hai cửa; có 4 chìa khóa, 6 cửa, chìa nào cũng mở được cửa nào nhưng bị **tiêu hao** sau khi dùng. Agent phải **từ bỏ** mở hai cửa dễ tìm (và reward tức thời đi kèm) để giữ chìa cho hai cửa khó sau. RND không tạo đủ động lực để agent học chiến lược "giữ chìa" này; agent chỉ hiếm khi tình cờ vấp vào chiến lược đó.
- Vì vậy, "giải quyết vấn đề này và các vấn đề tương tự cần khám phá bậc cao là một hướng quan trọng cho công việc tương lai" (nguyên văn ý paper).

Nhận xét:

- Chuẩn hóa quan sát theo chiều yêu cầu running mean/std, ổn với ảnh Atari đơn giản nhưng với quan sát nhiều chiều/đa dạng hơn (ví dụ ảnh màu tự nhiên, văn bản), hiệu quả của distillation ngẫu nhiên (predictor có khớp được mạng đích ngẫu nhiên hay không, với kiến trúc nào) không được kiểm chứng trong bài báo.
- Hệ số kết hợp $c_E = 2, c_I = 1$ và các discount γ_E, γ_I được chọn khá thủ công qua ablation trên Montezuma's Revenge, không rõ có tổng quát tốt cho miền bài toán khác (continuous control, agent dùng tool, v.v.) hay không; paper chỉ thử nghiệm trên Atari.
- So sánh với dynamics-based bonus chỉ đổi phần "mục tiêu dự đoán" (giữ nguyên hai value head, non-episodic, chuẩn hóa...), nên không hoàn toàn là so sánh với các công trình dynamics-based gốc trong tài liệu trước đó (Pathak et al. 2017, Burda et al. 2018 bản gốc) vốn không có các cải tiến phụ này; đây là so sánh "component-controlled" chứ không phải so sánh hai hệ thống hoàn chỉnh khác nhau trong lịch sử.
- Chi phí: cần forward pass thêm cho cả target và predictor network mỗi bước (dù paper nói "minimal overhead"), và cần huấn luyện riêng predictor; với môi trường có observation rất lớn (video dài, tài liệu nhiều trang) chi phí này không còn "tối thiểu".

7. Liên hệ với các bài khác

- **01 DQN, 02 Double DQN, 03 Rainbow:** cùng môi trường Atari; Montezuma's Revenge là game các phương pháp value-based cơ bản gần như không giải được (mean return quanh ~0-2500 nếu không có exploration bonus đặc biệt, theo dẫn chứng trong related work của RND).
- **07 PPO:** RND dùng PPO làm bộ tối ưu chính sách nền, chỉ thêm hai value head và reward tổ hợp; hiểu rõ PPO (clip ratio, GAE λ) là điều kiện để hiểu toàn bộ quy trình thực nghiệm ở đây.
- **14 AlphaZero, 15 MuZero, 16 DreamerV3:** các bài sau trong "Giai đoạn 4" giải sparse-reward/exploration theo hướng khác: xây mô hình/dùng tìm kiếm cây (planning) thay vì thưởng nội tại thuần dựa trên lỗi dự đoán; nên so sánh triết lý "khám phá bằng novelty-bonus" (RND) với "khám phá bằng lập kế hoạch có mô hình" (MuZero/DreamerV3).
- **53 Search-R1, 54 Agentic RL survey:** RL cho agent dùng LLM/tool cũng đối mặt sparse reward (ví dụ chỉ có reward cuối cùng sau nhiều bước gọi tool/tìm kiếm); ý tưởng "thưởng nội tại dựa trên novelty" là một ứng viên tự nhiên để bổ sung reward thưa trong các quy trình đó, dù bản thân RND (dựa trên mạng CNN cho ảnh Atari) không áp dụng trực tiếp cho không gian token.
- **27 Reward Model Overoptimization, 48 Spurious Rewards:** về mặt khái niệm, "noisy-TV problem" (agent bị hút vào nguồn entropy cục bộ mà bonus không phản ánh đúng novelty thật) là một dạng khác của hiện tượng "khai thác lỗ hổng của hàm thưởng" (reward hacking) xuất hiện trong RLVR/RLHF cho LLM; cùng bài học chung: một thưởng được thiết kế cẩn thận vẫn có thể bị policy khai thác theo cách không mong muốn nếu không kiểm soát các nguồn sai số/ngẫu nhiên.

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp với RL cho LLM/VLM: trung bình, chủ yếu ở mức **nguyên lý** (vấn đề sparse reward và cách tạo tín hiệu novelty rẻ), còn công thức/kiến trúc cụ thể (CNN cho ảnh Atari) không chuyển trực tiếp:

- **Sparse reward trong RLVR/agentic RL:** nhiều tác vụ RLVR cho LLM (toán, code, agent nhiều bước) có reward chỉ xuất hiện ở cuối (đúng/sai đáp số, pass/fail unit test) và rất thưa với các chuỗi suy luận dài hoặc agent gọi tool nhiều bước (bài 53 Search-R1, 54 agentic RL survey); ý tưởng "thưởng novelty dựa trên lỗi distillation của một mạng đích cố định" là một cơ chế có thể thêm vào, với mạng đích/predictor hoạt động trên biểu diễn trạng thái của agent (ví dụ embedding của trạng thái hội thoại/tool call) thay vì pixel.
- **Hai value head cho hai dòng reward có đặc tính khác nhau** là một mẹo kỹ thuật tổng quát: nếu một quy trình RLVR muốn kết hợp reward thưa, dài hạn (ví dụ đúng đáp số cuối) với một tín hiệu dày đặc

hơn nhưng khác bản chất (ví dụ độ dài hợp lý, format đúng), tách value head theo từng dòng reward (như RND) giúp mỗi head học dễ hơn một tín hiệu đồng nhất, rồi cộng lại.

- **Cảnh báo noisy-TV:** nếu tín hiệu novelty dựa trên lỗi dự đoán của *bất kỳ* mục tiêu không xác định (ví dụ dự đoán token tiếp theo trong văn bản tự nhiên vốn có entropy cao), nguy cơ agent bị hút vào các đoạn văn bản/tool call có tính ngẫu nhiên cao (không hữu ích) là có thật; nên ưu tiên các mục tiêu dự đoán **xác định** (như RND) nếu muốn dùng ý tưởng novelty-bonus cho agent LLM.
- **Hạn chế đã nêu (khám phá toàn cục) đặc biệt liên quan tới agent nhiều bước:** nếu một tác vụ agent cần "tiết kiệm tài nguyên bây giờ để dùng sau" (ví dụ giữ budget gọi API, giữ context cho bước quan trọng hơn), bài học của bài báo là bonus kiểu RND (cục bộ, theo từng bước) **không đủ** để dạy chiến lược dài hạn này; cần cơ chế khác (reward shaping thủ công, hoặc RL theo quỹ đạo dài như MuZero/DreamerV3, bài 15-16).

9. Thí nghiệm gợi ý

1. Cài RND trên một môi trường Atari sparse-reward nhỏ hơn (hoặc MiniGrid/MiniHack nếu có sẵn), so sánh có/không chuẩn hóa observation (bỏ clip $[-5, 5]$) để kiểm chứng tuyên bố "chuẩn hóa là bắt buộc vì mạng đích cố định".
2. Ablation hệ số c_E, c_I (ví dụ $\{(1, 1), (2, 1), (5, 1)\}$) trên một game hard-exploration, đo đánh đổi giữa return extrinsic cuối cùng và số phòng/trạng thái mới khám phá.
3. Tái hiện "noisy-TV": thêm một ô nhiễu ngẫu nhiên (ví dụ vài pixel random mỗi bước) vào môi trường, so sánh RND với một bonus dynamics-based để kiểm chứng RND không bị hút vào nguồn nhiễu này còn dynamics-based thì có.
4. Thử áp ý tưởng RND cho một bài toán agent dạng chuỗi quyết định rời rạc nhỏ (ví dụ gọi "tool" giả lập nhiều bước với reward cuối thưa): dùng embedding trạng thái hội thoại thay pixel, đo xem bonus distillation có tăng tỉ lệ tìm được chuỗi hành động đúng so với PPO/GRPO thuần không có bonus.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Exploration bonus	$r_t = e_t + i_t$	i_t khuyến khích trạng thái mới
RND loss	$\mathcal{L}(\theta) = \mathbb{E}[\|\hat{f}(x;\theta) - f(x)\|^2]$	f cố định, ngẫu nhiên; $\hat{f}$ học
Intrinsic reward	$i_t = \|\hat{f}(x_t) - f(x_t)\|^2$	cao ở trạng thái ít gặp
4 nguồn lỗi dự đoán	data / stochasticity / misspecification / learning dynamics	RND chỉ muốn khai thác nguồn 1
Noisy-TV problem	lỗi cao vì mục tiêu ngẫu nhiên, không giảm theo dữ liệu	RND tránh bằng mục tiêu xác định
Kết hợp return	$R = R_E + R_I \Rightarrow V = V_E + V_I$	hai value head, hai discount
Chuẩn hóa observation	whiten + clip $[-5, 5]$	bắt buộc vì f không tự thích nghi
Chuẩn hóa intrinsic reward	chia running std của intrinsic return	giữ thang đo ổn định
Thuật toán nền	PPO 2 value head	$c_E = 2,\ c_I = 1,\ \gamma_E = 0.999,\ \gamma_I = 0.99$

11. Kết luận ngắn

RND đề xuất một thưởng khám phá cực kỳ đơn giản: lỗi distillation của một mạng mục tiêu cố định, ngẫu nhiên. Vì mục tiêu xác định và nằm trong lớp hàm của predictor, lỗi dự đoán giảm đúng theo mức độ "đã thấy trạng thái này nhiều chưa", tránh được vấn đề noisy-TV mà các bonus dựa trên dynamics gặp phải. Kết hợp với PPO hai value head (tách dòng reward episodic/extrinsic và non-episodic/intrinsic), RND đạt SOTA trên

Montezuma’s Revenge, Venture, Gravitar ở thời điểm công bố, chứng minh các phương pháp khám phá đơn giản vẫn hiệu quả ở quy mô lớn; nhược điểm lớn nhất tác giả tự nhận là chỉ giải quyết khám phá cục bộ, chưa chạm tới các chiến lược đòi hỏi phối hợp dài hạn như “giữ chìa khóa cho cửa khó”.

Link gốc: https://arxiv.org/abs/1810.12894

Bài 14

Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm (2017)

Tác giả: David Silver, Thomas Hubert, Julian Schrittwieser và đồng nghiệp (DeepMind). **Venue / arXiv:** arXiv:1712.01815 (v1, 05/12/2017). Bản text trích xuất không nêu venue chính thức; đây là bản preprint. Bài 15 (MuZero) trích dẫn phiên bản sau của công trình này là Silver et al., *Science* 362(6419):1140-1144, 2018, nhưng thông tin đó không xuất hiện trong văn bản arXiv đang dùng để ghi chú. **Nguồn:** https://arxiv.org/abs/1712.01815 **Chủ đề:** Tổng quát hoá thuật toán AlphaGo Zero (tự chơi, không dùng dữ liệu người, không tri thức miền ngoài luật chơi) thành một thuật toán duy nhất, AlphaZero, áp dụng không đổi cho cờ vua, shogi và cờ vây. **Vị trí trong lộ trình:** Đọc sau bài 00 (nền tảng MDP, policy gradient, PPO...) để có ngữ cảnh RL chung. AlphaZero là đại diện cho nhóm "RL kết hợp tìm kiếm cây" (planning), khác hẳn các bài 01-12 (model-free, value-based hoặc actor-critic thuần). Đọc trước bài 15 (MuZero, thay simulator hoàn hảo bằng model học được) và bài 16 (DreamerV3, một hướng planning khác không dùng cây tìm kiếm). Ba bài 14-16 cùng nằm trong giai đoạn 4 của lộ trình (search / world model).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Cờ vua và shogi theo cách cổ điển. Hơn bốn thập kỷ, các chương trình cờ vua mạnh nhất (Deep Blue, và năm 2016 là Stockfish, vô địch TCEC) dựa trên: (i) hàm lượng giá tuyến tính $v(s, w) = \phi(s)^\top w$ trên đặc trưng thủ công (giá trị quân, cấu trúc tốt, an toàn vua...), (ii) tìm kiếm minimax cắt nhánh alpha-beta với nhiều kỹ thuật thủ công (quiescence search cho thế cờ "chưa yên", null move pruning, futility pruning, sắp xếp nước bằng iterative deepening, bảng chuyển vị...). Shogi khó hơn về tính toán: bàn lớn hơn, quân bị bắt đổi phe rồi có thể "thả" lại bất cứ đâu, làm không gian hành động phức tạp hơn. Elmo (vô địch CSA 2017) cũng dùng kiến trúc alpha-beta tương tự.

AlphaGo Zero. Trước bài này, AlphaGo Zero (công bố trên *Nature*, không có trên arXiv nên không thể trích xuất full text ở đây) đạt hiệu năng siêu nhân ở cờ vây bằng học *tabula rasa* (từ số 0, không dữ liệu người, không tri thức miền ngoài luật chơi) qua tự chơi (self-play) RL thuần, dùng CNN sâu thay hàm lượng giá thủ công, và Monte-Carlo Tree Search (MCTS, tìm kiếm cây bằng lấy mẫu Monte-Carlo) thay alpha-beta.

Cờ vây đặc biệt "hợp" với CNN: luật bất biến theo tịnh tiến, cấu trúc cục bộ, đối xứng xoay/phản xạ (tăng dữ liệu bằng 8 phép đối xứng), kết quả chỉ thắng/thua nhị phân. Cờ vua và shogi *không* có các thuận lợi này: luật phụ thuộc vị trí (tốt đi 2 bước ở hàng xuất phát, phong cấp ở hàng 8), không đối xứng (nhập thành cánh vua khác cánh hậu), tương tác tầm xa (hậu đi xuyên bàn cờ), và có thể hoà.

Khái niệm cần nắm trước khi đọc: self-play (agent tự chơi với chính nó/phiên bản gần nhất để sinh dữ liệu, không cần đối thủ ngoài); MCTS (tìm kiếm cây bằng nhiều "lượt mô phỏng" từ gốc, mỗi lượt chọn đường đi theo quy tắc cân bằng khai thác/khám phá rồi lan truyền kết quả ngược lên); policy network $p_a = \Pr(a|s)$; value network $v(s) \approx \mathbb{E}[z|s]$ (ước lượng kết quả cuối trận z); Elo (thang sức mạnh tương đối, suy từ tỉ lệ thắng).

2. Ý tưởng chính

AlphaZero dùng **một** mạng neural $(p, v) = f_\theta(s)$ nhận thế cờ s làm đầu vào, xuất ra đồng thời vector xác suất đi nước p và giá trị kỳ vọng v. Mạng này hướng dẫn một MCTS tổng quát (không gắn tri thức riêng của từng trò chơi, trừ luật chơi). Vòng lặp tự cải thiện (policy iteration) diễn ra như sau: mạng hiện tại giúp MCTS

tìm kiếm tốt hơn ngẫu nhiên; MCTS trả về một chính sách tìm kiếm π (tốt hơn p thô vì đã "nhìn trước" nhiều nước); π và kết quả trận đấu z lại được dùng làm nhãn để huấn luyện f_θ tốt hơn; mạng tốt hơn lại giúp MCTS tìm kiếm tốt hơn nữa, lặp lại. Toàn bộ quá trình không cần dữ liệu người, không cần đặc trưng thủ công, chỉ cần luật chơi (để mô phỏng thế cờ kế tiếp và chấm kết quả). Điểm mới so với AlphaGo Zero là bỏ hết các giả định đặc thù của cờ vây (đối xứng, thắng/thua nhị phân, luân phiên người chơi tốt nhất) để một thuật toán *duy nhất*, không chỉnh theo trò chơi, chạy được trên cả ba trò chơi.

3. Phương pháp chi tiết

3.1 Biểu diễn đầu vào và đầu ra

Đầu vào mạng là một chồng ảnh $N \times N \times (MT + L)$: $T = 8$ bước lịch sử gần nhất, mỗi bước có M mặt phẳng nhị phân biểu diễn vị trí quân (theo góc nhìn người chơi hiện tại), cộng L mặt phẳng hằng số (màu quân, số nước đã đi, luật đặc biệt). Bảng S1:

Trò chơi	M (mỗi bước)	L (hằng số)	Tổng $MT + L$
Cờ vây (19×19)	2 (P1 stone, P2 stone)	1 (màu)	$2{\times}8{+}1 = 17$
Cờ vua (8×8)	14 (quân P1 6, quân P2 6, lặp thế 2)	7 (màu 1, tổng số nước 1, nhập thành P1 2, nhập thành P2 2, không tiến triển 1)	$14 \times 8 + 7 = 119$
Shogi (9×9)	45 (quân P1 14, quân P2 14, lặp thế 3, tù binh P1 7, tù binh P2 7)	2 (màu, tổng số nước)	$45 \times 8 + 2 = 362$

(Các phép cộng $M \times T + L$ ở cột cuối là tự tính, khớp với tổng mà Bảng S1 công bố.)

Đầu ra $p(a|s)$ là một chồng mặt phẳng: cờ vua $8 \times 8 \times 73 = 4{,}672$ nước đi khả dĩ (56 mặt "nước kiểu hậu" x 8 hướng x 7 khoảng cách, 8 mặt nước mã, 9 mặt phong cấp không thành hậu); shogi $9 \times 9 \times 139 = 11{,}259$ nước; cờ vây dùng phân phối phẳng trên $19 \times 19 + 1$ nước (giống AlphaGo Zero). Nước bất hợp lệ bị gán xác suất 0 rồi chuẩn hoá lại.

3.2 Hàm mất mát

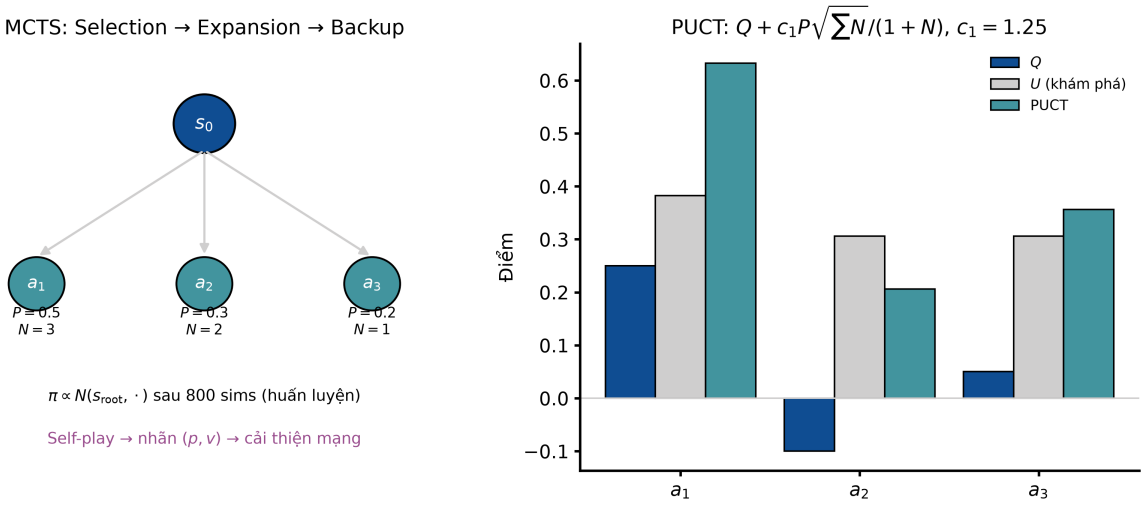
Tham số θ được huấn luyện bằng gradient descent trên:

$$\boxed{(p, v) = f_\theta(s), \qquad l = (z - v)^2 - \pi^\top \log p + c\|\theta\|^2}$$

với z là kết quả thật của trận (-1 thua, 0 hoà, $+1$ thắng, từ góc nhìn người chơi tại s), π là chính sách tìm kiếm do MCTS trả ra tại s, và c là hệ số điều chuẩn L2. Số hạng đầu là bình phương sai số (hồi quy v về z); số hạng hai là cross-entropy (kéo p về gần π, tức "chưng cất" kết quả tìm kiếm vào mạng).

3.3 MCTS và PUCT (lưu ý nguồn)

Bản text arXiv của AlphaZero **không viết ra công thức chọn nước tường minh**, chỉ mô tả bằng lời: "chọn nước có số lần thăm thấp, xác suất ưu tiên cao và giá trị cao theo mạng hiện tại", và nói thuật toán tìm kiếm/huấn luyện "giống AlphaGo Zero" (bài *Nature*, không có trên arXiv, không đối chiếu trực tiếp được). Công thức PUCT tường minh lấy từ bài 15 (MuZero), Phụ lục B, công thức (2); MuZero nói rõ với board game họ "dùng cùng hằng số UCB... như AlphaZero" (Phụ lục C), nên công thức và hai hằng số dưới đây hợp lệ để mô tả MCTS của AlphaZero:



MCTS ba bước và PUCT $Q + c_1 P\sqrt{\sum N}/(1+N)$ với $c_1 = 1.25$ (minh hoạ, tự tính)

$$a^k = \arg\max_a \; Q(s,a) + P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)} \cdot \left(c_1 + \log \frac{\sum_b N(s,b) + c_2 + 1}{c_2}\right)$$

với $c_1 = 1.25$, $c_2 = 19652$ (c_1 kiểm soát trọng số prior $P(s,a)$ so với $Q(s,a)$ ở mức thăm thấp; số hạng log tăng dần ảnh hưởng của P khi nút thăm nhiều hơn, nhưng c_2 rất lớn nên gần như không đáng kể ở quy mô vài trăm lượt mô phỏng, xem ví dụ mục 3.4).

MCTS gồm ba giai đoạn lặp lại qua nhiều lượt mô phỏng, mỗi lượt đi từ gốc s_{root} tới một nút lá:

1. **Selection (chọn):** tại mỗi nút đã thăm, chọn hành động theo công thức PUCT trên, đi tiếp bằng mô phỏng luật chơi (AlphaZero có simulator hoàn hảo, biết chắc thế cờ kế tiếp).
2. **Expansion (mở rộng):** khi gặp nút lá chưa mở rộng, gọi mạng $(p, v) = f_\theta(s_{leaf})$, khởi tạo mỗi cạnh con với $N = 0, Q = 0, P = p_a$.
3. **Backup (lan truyền):** cập nhật $N(s,a)$ (tăng 1) và $Q(s,a)$ (trung bình chạy của các giá trị lá đã đi qua cạnh đó) ngược lên gốc. (AlphaZero không viết công thức backup tường minh; MuZero viết cho trường hợp tổng quát hơn ở công thức (3)-(4) của bài 15; khi không có phần thưởng trung gian và $\gamma = 1$, công thức đó thu về đúng "trung bình chạy của giá trị lá", khớp với câu mô tả bằng lời của AlphaZero.)

Sau khi chạy đủ số lượt mô phỏng (800 trong huấn luyện, xem mục 4), MCTS trả ra π tỉ lệ với số lần thăm $N(s_{root}, \cdot)$ ở gốc. Trong tự chơi (huấn luyện), nước đi được **chọn theo xác suất** π (để đa dạng dữ liệu); khi **đánh giá**, AlphaZero chọn nước **có số lần thăm cao nhất một cách tất định** (greedy).

3.4 Ví dụ MCTS từng bước (tự tính)

Minh hoạ một gốc s_0 có 3 nước khả dĩ a_1, a_2, a_3 (ví dụ 1.e4, 1.d4, 1.Nf3), prior $P = (0.5, 0.3, 0.2)$, $c_1 = 1.25$, $c_2 = 19652$ (số hạng log $\approx \log(19653/19652) \approx 0.0001$ nên $c_1 + \log \approx 1.2501$ xuyên suốt, bỏ qua sai số này). Quy ước của phần phân tích (bài báo không nêu tie-break và không nêu quy tắc đổi dấu hai lượt chơi): điểm PUCT hoà thì ưu tiên prior cao hơn; v của một thế cờ tính theo góc nhìn người sắp đi tại đó, nên lan truyền lên cha (đối phương) phải đổi dấu thành $-v$.

Lượt	$\sum N$ trước	Điểm PUCT a_1, a_2, a_3	Chọn	v lá mới	Cập nhật Q, N tại s_0
1	0	$0, 0, 0$ (hoà, chọn theo prior)	a_1	0.2	$Q(a_1) = -0.2, N = 1$

Lượt	$\sum N$ trước	Điểm PUCT a_1, a_2, a_3	Chọn	v lá mới	Cập nhật Q, N tại s_0
2	1	0.1125, **0.3750**, 0.2500	a_2	0.1	$Q(a_2) = -0.1, N = 1$
3	2	0.2420, 0.1652, **0.3536**	a_3	0.05	$Q(a_3) = -0.05, N = 1$
4	3	**0.3414**, 0.2248, 0.1665	a_1 lại	0.3 (giả định)	$Q(a_1) = \frac{-0.2-0.3}{2} = -0.25, N = 2$

(Cột điểm PUCT là tự tính theo công thức mục 3.3, ví dụ lượt 2: $a_2 = 0 + 0.3 \cdot \frac{\sqrt{1}}{1+0} \cdot 1.2501 = 0.375$. Lượt 4 chọn lại a_1 nghĩa là MCTS đi sâu tiếp vào cây con của s_1, không trình bày chi tiết; ở đây giả định lượt này chạm một lá mới dưới a_1 với $v = 0.3$ để minh hoạ Q là trung bình chạy.)

Sau 4 lượt: $N(s_0, \cdot) = (2, 1, 1)$, tổng 4, chính sách tìm kiếm $\pi \propto N \Rightarrow \pi = (0.5, 0.25, 0.25)$. Mạng được huấn luyện để $p(s_0)$ tiến gần π (qua $-\pi^\top \log p$) và $v(s_0)$ tiến gần kết quả z thật của trận.

4. Thí nghiệm và kết quả chính

Cấu hình huấn luyện (Bảng S3, phần Configuration): 700.000 mini-batch, kích thước batch 4.096, khởi tạo tham số ngẫu nhiên, dùng 5.000 TPU thế hệ 1 để sinh dữ liệu tự chơi và 64 TPU thế hệ 2 để huấn luyện mạng, cho **mỗi** trò chơi riêng (ba mô hình độc lập). Learning rate 0.2, giảm 3 lần xuống 0.02, 0.002, 0.0002. MCTS dùng 800 lượt mô phỏng mỗi nước trong huấn luyện. Nhiễu Dirichlet Dir(α) thêm vào prior ở gốc để khám phá, với $\alpha = 0.3$ (cờ vua), 0.15 (shogi), 0.03 (cờ vây), tỉ lệ nghịch với số nước hợp lệ điển hình.

	Cờ vua	Shogi	Cờ vây
Mini-batch	700k	700k	700k
Thời gian huấn luyện	9 giờ	12 giờ	34 giờ
Số trận tự chơi	44 triệu	24 triệu	21 triệu
Thinking time (thi đấu)	40 ms	80 ms	200 ms

AlphaZero vượt Stockfish sau 4 giờ (300k bước), vượt Elmo sau dưới 2 giờ (110k bước), vượt AlphaGo Lee sau 8 giờ (165k bước) (Hình 1).

Bảng 1 (đấu 100 trận, 1 phút/nước, từ góc nhìn AlphaZero):

Trò chơi	Trắng	Đen	Thắng	Hoà	Thua
Cờ vua	AlphaZero	Stockfish	25	25	0
Cờ vua	Stockfish	AlphaZero	3	47	0
Shogi	AlphaZero	Elmo	43	2	5
Shogi	Elmo	AlphaZero	47	0	3
Cờ vây	AlphaZero	AG0 3 ngày	31	(không có hoà)	19
Cờ vây	AG0 3 ngày	AlphaZero	29	(không có hoà)	21

AlphaZero không thua trận nào trước Stockfish (tổng cộng thắng 28, hoà 72 trên 100 trận với cả hai màu), thắng Elmo áp đảo (thua tổng 8/100), và nhích hơn AlphaGo Zero huấn luyện 3 ngày (60 thắng trên 100 trận tính cả hai màu).

Khai cuộc do người phát hiện: phân tích 12 khai cuộc phổ biến nhất của loài người (chơi trên 100.000 lần trong một cơ sở dữ liệu online), AlphaZero tự tìm lại và chơi thường xuyên mọi khai cuộc này trong tự chơi;

xuất phát từ các khai cuộc đó, AlphaZero vẫn thắng Stockfish áp đảo: tổng cộng khi cầm Trắng 242 thắng/353 hoà/5 thua (40.3%/58.8%/0.8%), khi cầm Đen 48 thắng/533 hoà/19 thua (8.0%/88.8%/3.2%) (Bảng 2).

Tốc độ tìm kiếm (Bảng S4): AlphaZero duyệt 80.000 thế cờ/giây (cờ vua), 40.000 (shogi), 16.000 (cờ vây), so với Stockfish 70.000.000/giây và Elmo 35.000.000/giây, tức khoảng ba bậc độ lớn ít hơn, bù lại bằng việc dùng mạng neural để "tập trung chọn lọc hơn vào các biến thể hứa hẹn nhất" (một cách tìm kiếm "giống người" theo cách diễn đạt gốc của Shannon 1950).

Khả năng mở rộng theo thời gian nghĩ (Hình 2): AlphaZero có đường cong Elo-theo-thời-gian-nghĩ tăng tốt hơn cả Stockfish và Elmo (số liệu chi tiết chỉ nằm trong hình, không có giá trị số trong văn bản chính).

5. Phân tích / ablation

Đây là một bài báo ngắn, không có bảng ablation tách riêng từng thành phần; "phân tích" chủ yếu là các khác biệt thiết kế được tác giả liệt kê so với AlphaGo Zero, trình bày lại:

1. **Kết quả kỳ vọng, không chỉ thắng/thua:** AlphaGo Zero tối ưu xác suất thắng (nhị phân); AlphaZero tối ưu $z \in \{-1, 0, +1\}$, tính cả hoà, vì cờ vua/shogi có hoà.
2. **Không tăng cường đối xứng:** cờ vây dùng 8 phép xoay/phản xạ để tăng dữ liệu và làm nhiễu đánh giá MCTS; cờ vua/shogi không đối xứng nên bỏ bước này.
3. **Cập nhật liên tục, không "cổng" người chơi tốt nhất:** AlphaGo Zero chỉ thay "người chơi tốt nhất" nếu ứng viên mới thắng ≥55% trận đấu giá; AlphaZero bỏ bước đánh giá/thay thế, luôn dùng tham số mới nhất để tự chơi.
4. **Không tinh chỉnh siêu tham số theo trò chơi:** AlphaGo Zero dùng Bayesian optimization; AlphaZero dùng chung một bộ cho cả ba trò chơi, trừ nhiễu Dirichlet co giãn theo số nước hợp lệ điển hình.
5. **MCTS so với alpha-beta (mục Methods riêng):** MCTS lấy trung bình trên các lượt mô phỏng nên lỗi xấp xỉ của mạng (phi tuyến mạnh, dễ sai cục bộ) có xu hướng triệt tiêu nhau; alpha-beta tính minimax tường minh, truyền lỗi lớn nhất lên gốc. Tác giả cho rằng đây là lý do MCTS "hợp" mạng neural hơn, dù lịch sử MCTS thuần từng yếu hơn alpha-beta trong cờ vua.

6. Giới hạn

Tác giả nêu / có thể suy ra trực tiếp từ văn bản:

- AlphaZero cần biết **chính xác luật chơi** để mô phỏng thế cờ kế tiếp, xác định nước hợp lệ và chấm kết quả: đây là một simulator hoàn hảo, không phải "học từ tương tác với môi trường không rõ động lực học" theo nghĩa RL tổng quát (điểm này chính là động lực cho bài 15, MuZero).
- Huấn luyện tách biệt theo trò chơi (ba mô hình độc lập), không phải một mô hình dùng chung.
- Chú thích 2 của bài báo: "AlphaGo Master và AlphaGo Zero cuối cùng được huấn luyện 100 lần thời gian này; chúng tôi không tái tạo nỗ lực đó ở đây", nghĩa là kết quả báo cáo không phải giới hạn trên của phương pháp, chỉ là một điểm vận hành với ngân sách tính toán cố định.

Nhận xét:

- Ngân sách tính toán (5.000 TPU thế hệ 1 cho tự chơi, 64 TPU thế hệ 2 cho huấn luyện, mỗi trò chơi) ngoài khả năng của hầu hết nhóm nghiên cứu, không phải thuật toán "rẻ".
- Không có ablation định lượng tách riêng từng thay đổi so với AlphaGo Zero (mục 5 là mô tả thiết kế, không phải thí nghiệm có/không).
- Elo đo qua đấu nội bộ, neo theo Elo công khai của baseline; không có đối chiếu độc lập ngoài DeepMind. Bài báo không nêu có phát hành mã nguồn hay không.
- MCTS/PUCT tường minh không có trong chính văn bản này (mục 3.3), phải tham chiếu bài 15; chi tiết "giống AlphaGo Zero" không đối chiếu trực tiếp được vì đó là bài *Nature*, không có trên arXiv.

7. Liên hệ với các bài khác

- **Bài 00 (nền tảng):** AlphaZero không phải thuần policy-gradient kiểu REINFORCE/PPO; hàm mất mát là *distillation có giám sát* từ kết quả tìm kiếm (MCTS) vào mạng (p, v), một dạng *policy iteration*: cải thiện chính sách bằng tìm kiếm rồi ép mạng học lại. Cùng gốc MDP/Bellman như bài 00, nhưng cơ chế cải thiện chính sách khác REINFORCE.
- **Bài 15 (MuZero):** kế thừa trực tiếp MCTS+PUCT của AlphaZero, nhưng bỏ simulator hoàn hảo, học model (representation, dynamics, prediction) để lập kế hoạch trong không gian ẩn; ba điểm tri thức luật chơi AlphaZero dùng (chuyển trạng thái, lọc nước hợp lệ trong cây, xử lý nút kết thúc) được MuZero thay bằng model học được (Phụ lục A bài 15).
- **Bài 16 (DreamerV3):** nhánh model-based RL khác, học world model (RSSM) từ pixel, huấn luyện actor-critic hoàn toàn trong "tưởng tượng", **không** chạy MCTS lúc hành động; đối lập AlphaZero/MuZero về cách dùng model (tìm kiếm cây so với tưởng tượng + lan truyền ngược).
- **Bài 35 (PRM):** diễn giải của phần phân tích: $v(s)$ của AlphaZero là "bộ chấm điểm trạng thái" tương tự ý tưởng process/outcome mô hình phần thưởng học sẵn, dù PRM chấm từng bước suy luận còn v chấm toàn cục thế cờ.
- **Bài 39 (DeepSeek-R1):** Phụ lục G.2 "Unsuccessful Attempts" của DeepSeek-R1 kể lại việc thử MCTS "lấy cảm hứng từ AlphaGo và AlphaZero": chia câu trả lời thành các bước có thẻ, tìm đáp án bằng MCTS dẫn đường bởi một mô hình giá trị đã huấn luyện trước, rồi dùng cặp câu hỏi/đáp án tìm được để huấn luyện cả actor lẫn mô hình giá trị, lặp lại. Cách này vướng hai trở ngại khi mở rộng: (1) không gian sinh token lớn theo hàm mũ, khác hẳn cờ vua, và giới hạn số nhánh mở rộng mỗi nút dễ làm model kẹt ở cực trị địa phương; (2) chất lượng sinh phụ thuộc trực tiếp vào mô hình giá trị, mà mô hình giá trị mịn theo từng bước rất khó huấn luyện, nên vòng "tự tìm kiếm rồi tự cải thiện" vốn là cốt lõi của AlphaZero (mục 3) không lặp lại được. Kết luận của bài báo: MCTS giúp được lúc suy luận nếu có mô hình giá trị tốt, nhưng dùng nó để cải thiện model lặp đi lặp lại vẫn là thách thức lớn.

8. Ứng dụng thực tế (suy luận)

Liên quan **yếu** tới huấn luyện LLM/VLM bằng RL kiểu RLHF/RLVR hiện nay (không có simulator hoàn hảo cho ngôn ngữ, không gian hành động là từ vựng rất lớn, không phải trò chơi hai người zero-sum), nhưng có vài bài học nguyên lý:

- **Siêu tham số quan trọng:** c_1, c_2 trong PUCT kiểm soát khai thác/khám phá khi số lượt thăm tăng; số lượt mô phỏng mỗi nước (800) đánh đổi tính toán lấy sức mạnh, một dạng "test-time compute scaling" sơ khai (Hình 2); nhiễu Dirichlet ở gốc cần co giãn theo độ lớn không gian hành động.
- **Chi phí:** 5.000+64 TPU mỗi trò chơi ngoài khả năng hầu hết lab; không có ngân sách đó thì các biến thể rẻ hơn (GRPO, không tìm kiếm cây) thực tế hơn.

9. Thí nghiệm gợi ý

1. Cài MCTS+PUCT (mục 3.3) với mạng (p, v) ngẫu nhiên cho trò chơi nhỏ (Tic-Tac-Toe/Connect-4), chạy tự chơi, đo Elo so với minimax tối ưu theo số bước huấn luyện, xác nhận policy-iteration hội tụ.
2. Với mạng đã huấn luyện, cố định mạng, chỉ đổi số lượt mô phỏng khi *đánh giá*, đo Elo theo số lượt mô phỏng, tái lập dạng đường cong Hình 2 ở quy mô nhỏ.
3. Ablation nhiễu Dirichlet ở gốc: $\alpha \in \{0, 0.03, 0.3, 1.0\}$ trên cùng trò chơi nhỏ, đo đa dạng khai cuộc và Elo cuối.
4. So sánh "cập nhật liên tục" (AlphaZero) với "cổng người chơi tốt nhất, ngưỡng 55%" (AlphaGo Zero) cùng ngân sách tính toán, kiểm tra việc bỏ bước đánh giá/thay thế có ảnh hưởng ổn định huấn luyện như paper ngụ ý không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mạng chính	$(p, v) = f_\theta(s)$	policy + value chung một mạng
Hàm mất mát	$l = (z - v)^2 - \pi^\top \log p + c\|\theta\|^2$	huấn luyện bằng kết quả trận + chính sách tìm kiếm
PUCT (nguồn: bài 15)	$Q(s,a) + P(s,a)\frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}\left(c_1 + \log\frac{\sum_b N(s,b)+c_2+1}{c_2}\right)$	$c_1 = 1.25$, $c_2 = 19652$; không có trong chính văn bản AlphaZero
MCTS 3 giai đoạn	Selection, Expansion, Backup	lặp qua nhiều lượt mô phỏng (800 lúc huấn luyện)
Chính sách tìm kiếm	$\pi \propto N(s_{root}, \cdot)$	theo xác suất khi tự chơi, greedy khi đánh giá
Nhiễu khám phá	Dir(α) thêm vào prior ở gốc	$\alpha = 0.3/0.15/0.03$ (cờ vua/shogi/cờ vây)
Elo	$p(a \text{ thắng } b) = \frac{1}{1+\exp(c_{elo}(e(b)-e(a)))}$	$c_{elo} = 1/400$, ước lượng bằng BayesElo

11. Kết luận ngắn

AlphaZero chứng minh rằng một thuật toán RL tự chơi + MCTS *duy nhất*, không chỉnh theo trò chơi và không tri thức miền ngoài luật chơi, có thể đạt trình độ siêu nhân ở cờ vua, shogi và cờ vây trong vài giờ huấn luyện, đánh bại các chương trình vô địch thế giới dựa trên alpha-beta và đặc trưng thủ công hàng chục năm tinh chỉnh, dù chỉ duyệt khoảng một phần nghìn số thế cờ mỗi giây so với các chương trình đó. Cái giá phải trả là cần biết chính xác luật chơi (simulator hoàn hảo) và một lượng tính toán khổng lồ; cả hai hạn chế này là tiền đề trực tiếp cho bài 15 (MuZero) và cho các hướng RL tiết kiệm tính toán hơn ở các giai đoạn sau của lộ trình.

Link gốc: https://arxiv.org/abs/1712.01815

Bài 15

Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (2019)

Tác giả: Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, David Silver (DeepMind; University College London). **Venue / arXiv:** arXiv:1911.08265 (v2, 21/02/2020). Bản text trích xuất không nêu venue chính thức (preprint); công trình này về sau được biết tới rộng rãi dưới tên MuZero nhưng thông tin xuất bản không nằm trong văn bản arXiv đang dùng. **Nguồn:** https://arxiv.org/abs/1911.08265 **Chủ đề:** Thay simulator hoàn hảo (luật chơi đã biết) của AlphaZero bằng một model học được gồm ba hàm (representation, dynamics, prediction); lập kế hoạch (MCTS) hoàn toàn trong không gian ẩn do mô hình này tạo ra, áp dụng được cho cả board game và Atari (phần thưởng trung gian, chiết khấu, quan sát hình ảnh). **Vị trí trong lộ trình:** Đọc sau bài 14 (AlphaZero, cung cấp nền MCTS/PUCT). Đọc trước bài 16 (DreamerV3, một hướng model-based khác không dùng cây tìm kiếm). Ba bài 14-16 cùng thuộc giai đoạn 4 của lộ trình (search / world model).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Model-based so với model-free RL. RL chia hai nhóm: model-free ước lượng trực tiếp policy/value từ tương tác (các bài 01-12); model-based học thêm model môi trường (transition + reward) rồi lập kế hoạch (value iteration, MCTS) trên mô hình đó. Cách cổ điển: model là MDP tường minh, học tách rời ba khâu (biểu diễn trạng thái, học model, lập kế hoạch); ba khâu không tối ưu *cùng mục tiêu*, lỗi mô hình hoá dễ tích lũy khi lập kế hoạch nhiều bước.

Các hướng trước MuZero: mô hình hoá trực tiếp ở mức pixel (dự đoán từng khung ảnh): không khả thi ở quy mô Atari. Mô hình không gian ẩn tái dựng được ảnh gốc (world models kiểu Ha & Schmidhuber, hay PlaNet của Hafner et al., tiền thân dòng Dreamer bài 16): lập kế hoạch hiệu quả hơn nhưng tốn phần lớn dung lượng model mô tả chi tiết hình ảnh không liên quan quyết định. **Value-equivalent models:** không yêu cầu model tái dựng quan sát, chỉ yêu cầu *tổng phần thưởng chiết khấu* khi lập kế hoạch trên model trừu tượng khớp tổng phần thưởng thật (Predictron, TreeQN, Value Iteration Networks). Gần nhất, **Value Prediction Networks (VPN)**: học model MDP gắn hành động thật, huấn luyện sao cho tổng phần thưởng theo lookahead search đơn giản khớp môi trường thật, nhưng không có dự đoán policy, chỉ dùng value để tìm kiếm.

Khái niệm cần nắm: value equivalence (lập kế hoạch trên model trừu tượng cho kết quả tổng phần thưởng giống lập kế hoạch trên môi trường thật, không cần model "giống" môi trường thật về ngữ nghĩa); hidden state (trạng thái ẩn, không buộc tái dựng được quan sát gốc); bootstrapping n-step return; Reanalyze (chạy lại MCTS trên dữ liệu cũ với tham số mới để có nhãn tốt hơn).

2. Ý tưởng chính

MuZero chỉ dự đoán những gì **trực tiếp cần cho việc lập kế hoạch**: chính sách, giá trị, và phần thưởng, từ một trạng thái ẩn được cập nhật đệ quy khi cho thêm một hành động giả định. Không có yêu cầu nào buộc trạng thái ẩn phải tái dựng được quan sát gốc, hay phải khớp ngữ nghĩa với trạng thái thật của môi trường; trạng thái ẩn tự do mang bất kỳ dạng biểu diễn nào miễn là dự đoán tốt chính sách/giá trị/phần thưởng hiện tại và tương lai. Nói cách khác, agent "tự phát minh" nội tại các quy luật/động lực học giúp lập kế hoạch chính xác nhất, không cần mô phỏng đúng vật lý/luật chơi thật. MuZero tổng quát hoá AlphaZero từ trò chơi hai

người, có simulator hoàn hảo, phần thưởng ±1 không chiết khấu ở cuối trận, sang thiết lập RL tổng quát: một agent, phần thưởng trung gian, chiết khấu, không cần biết luật/động lực học.

3. Phương pháp chi tiết

3.1 Ba hàm của model

Tại mỗi bước thời gian t, với $k = 1, \dots, K$ bước giả định trong tương lai:

$$\boxed{\begin{aligned} s^0 &= h_\theta(o_1, \dots, o_t) && \text{(representation)} \\ r^k, s^k &= g_\theta(s^{k-1}, a^k) && \text{(dynamics)} \\ p^k, v^k &= f_\theta(s^k) && \text{(prediction)} \end{aligned}}$$

h_θ mã hoá lịch sử quan sát thật $o_1, \dots, o_t$ thành trạng thái ẩn gốc s^0 (không có ngữ nghĩa đặc biệt, chỉ cần hữu ích cho dự đoán). g_θ là một quá trình đệ quy: cho trạng thái ẩn trước và một hành động giả định, trả ra phần thưởng tức thời dự đoán r^k và trạng thái ẩn mới s^k; đóng vai trò giống mô hình MDP (transition + reward) nhưng s^k không buộc khớp trạng thái thật. f_θ tính chính sách p^k và giá trị v^k từ s^k, giống mạng (p, v) của AlphaZero. Mục tiêu khớp: $p_t^k \approx \pi(a_{t+k+1}|\cdot)$, $v_t^k \approx \mathbb{E}[u_{t+k+1} + \gamma u_{t+k+2} + \cdots]$, $r_t^k \approx u_{t+k}$ (ký hiệu u là phần thưởng quan sát thật, π là chính sách hành động thật).

3.2 MCTS trong không gian ẩn (Phụ lục B)

Mỗi nút cây gắn với một trạng thái ẩn s; mỗi cạnh (s, a) lưu $\{N(s,a), Q(s,a), P(s,a), R(s,a), S(s,a)\}$ (số lượt thăm, giá trị trung bình, prior, phần thưởng, trạng thái chuyển tới). Ba giai đoạn:

Selection: từ gốc s^0, với mỗi bước giả định $k = 1, \dots, l$ chọn hành động tối đa hoá cận trên độ tin cậy (ghi chú: công thức dưới bị vỡ định dạng khi trích xuất PDF, ký hiệu căn bậc hai và phân số bị tách dòng; phục dựng đúng theo thứ tự các số hạng xuất hiện trong văn bản, đây chính là công thức pUCT dùng chung cho cả AlphaZero, bài 14):

$$\boxed{a^k = \arg\max_a \; Q(s,a) + P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \cdot \left(c_1 + \log \frac{\sum_b N(s,b) + c_2 + 1}{c_2}\right)}$$

$c_1 = 1.25$, $c_2 = 19652$ trong thí nghiệm của bài báo. Với $k < l$, trạng thái và phần thưởng kế tiếp được tra từ bảng đã lưu $s^k = S(s^{k-1}, a^k)$, $r^k = R(s^{k-1}, a^k)$ (không gọi lại model nếu cạnh đã mở rộng).

Expansion: tại bước cuối l của lượt mô phỏng, gọi $r^l, s^l = g_\theta(s^{l-1}, a^l)$ và $p^l, v^l = f_\theta(s^l)$; lưu vào bảng; thêm nút mới với các cạnh con khởi tạo $N = 0, Q = 0, P = p^l$. **Mỗi lượt mô phỏng chỉ gọi dynamics và prediction đúng một lần**, nên chi phí tính toán mỗi lượt cùng cấp độ với AlphaZero.

Backup: với $k = l, \dots, 0$, tính ước lượng $l - k$ bước của tổng phần thưởng chiết khấu, bootstrap từ v^l:

$$\boxed{G^k = \sum_{\tau=0}^{l-1-k} \gamma^\tau r_{k+1+\tau} + \gamma^{l-k} v^l}$$

rồi cập nhật dọc đường đi, $k = l, \dots, 1$:

$$Q(s^{k-1}, a^k) \leftarrow \frac{N(s^{k-1}, a^k) \cdot Q(s^{k-1}, a^k) + G^k}{N(s^{k-1}, a^k) + 1}, \qquad N(s^{k-1}, a^k) \leftarrow N(s^{k-1}, a^k) + 1$$

Với trò chơi hai người, giá trị được giả định nằm trong $[0, 1]$ để kết hợp an toàn với xác suất prior trong công thức pUCT; khi phần thưởng không có giới hạn (Atari), MuZero chuẩn hoá Q về $[0, 1]$ bằng min-max quan sát được trong cây tại thời điểm đó:

$$Q_{norm}(s^{k-1}, a^k) = \frac{Q(s^{k-1}, a^k) - \min_{s,a \in Tree} Q(s, a)}{\max_{s,a \in Tree} Q(s, a) - \min_{s,a \in Tree} Q(s, a)}$$

Sau khi chạy đủ số lượt mô phỏng, MCTS trả ra π_t (tỉ lệ số lượt thăm ở gốc) và giá trị gốc ν_t; hành động thật được lấy mẫu $a_{t+1} \sim \pi_t$.

3.3 Huấn luyện: ba mục tiêu + unroll

Dữ liệu là chuỗi đã chơi (lưu trong replay buffer): quan sát $o_{1:T}$, hành động $a_{1:T}$, phần thưởng $u_{1:T}$. Model được **unroll** (triển khai đệ quy) K bước giả định, khớp với K hành động thật đã xảy ra, huấn luyện bằng backprop-through-time trên:

$$\boxed{l_t(\theta) = \sum_{k=0}^{K} \Big[l_r(u_{t+k}, r_t^k) + l_v(z_{t+k}, v_t^k) + l_p(\pi_{t+k}, p_t^k) \Big] + c\|\theta\|^2}$$

- **Mục tiêu chính sách:** π_{t+k} (chính sách tìm kiếm MCTS đã lưu khi chơi), loss cross-entropy $l_p(\pi, p) = \pi^\top \log p$.
- **Mục tiêu giá trị:** z_{t+k} là return bootstrap n bước: $z_t = u_{t+1} + \gamma u_{t+2} + \cdots + \gamma^{n-1} u_{t+n} + \gamma^n \nu_{t+n}$ cho MDP tổng quát (Atari, $n = 10$); với board game, bootstrap thẳng tới cuối trận (tương đương dự đoán kết quả cuối, $\gamma = 1$, không phần thưởng trung gian). Loss $(z - v)^2$ cho board game; cross-entropy trên biểu diễn phân loại (categorical) cho Atari (bình phương sai số kém ổn định hơn khi phần thưởng/giá trị có biên độ thay đổi mạnh).
- **Mục tiêu phần thưởng:** u_{t+k} (phần thưởng quan sát thật); $l_r = 0$ cho board game (không có phần thưởng trung gian), cross-entropy categorical cho Atari.

Chuẩn hoá gradient khi unroll (Phụ lục G): chia loss của mỗi bước giả định cho K (giữ tổng gradient không đổi theo độ dài unroll); nhân gradient đi vào đầu vào của dynamics function với $\frac{1}{2}$ (giữ tổng gradient áp lên dynamics không đổi). Thí nghiệm dùng $K = 5$ xuyên suốt.

Biến đổi giá trị/phần thưởng cho Atari (Phụ lục F): dùng biến đổi khả nghịch $h(x)$ để nén biên độ trước khi hồi quy, theo Pohlen et al. 2018 (công thức trong văn bản arXiv bị mất ký hiệu căn bậc hai khi trích xuất PDF; phục dựng đúng dạng gốc được paper trích dẫn):

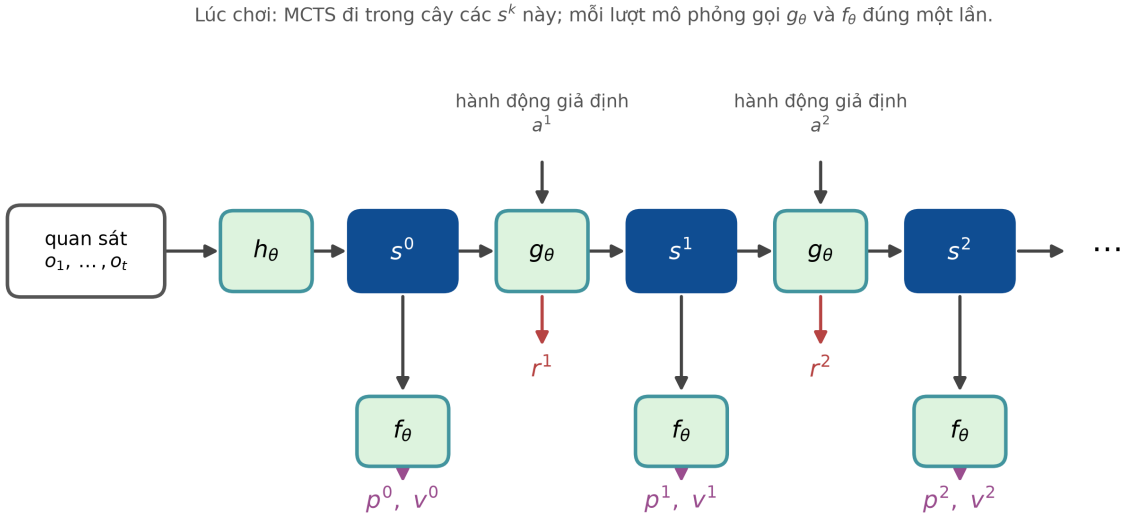
$$h(x) = \text{sign}(x)(\sqrt{|x| + 1} - 1) + \epsilon x, \qquad \epsilon = 0.001$$

Sau biến đổi, mạng dự đoán trên 601 "bin" rời rạc trải từ -300 đến 300 (mỗi số nguyên một bin), đọc giá trị liên tục bằng tổ hợp tuyến tính của hai bin liền kề (ví dụ 3.7 được mã hoá là trọng số 0.3 cho bin 3 và 0.7 cho bin 4, tự tính minh hoạ cách nội suy, không phải số trong bài báo).

Cách đọc hình: hàm biểu diễn biến chuỗi quan sát thành trạng thái ẩn, sau đó hàm động lực học nhận hành động và sinh trạng thái ẩn kế tiếp cùng phần thưởng. Tại mỗi bước, hàm dự đoán cho ra policy và value. Khi huấn luyện, chuỗi này được unroll K bước bằng các hành động thật rồi so với các mục tiêu tương ứng.

3.4 Ví dụ nhỏ: n-step return (tự tính)

Minh hoạ mục tiêu giá trị với $n = 3$, $\gamma = 0.9$: giả sử phần thưởng quan sát $u_{t+1} = 1, u_{t+2} = 0, u_{t+3} = 2$, và giá trị bootstrap tại bước $t + 3$ là $\nu_{t+3} = 5$. Theo công thức $z_t = u_{t+1} + \gamma u_{t+2} + \gamma^2 u_{t+3} + \gamma^3 \nu_{t+3}$:



Hình 44: Mô hình MuZero được unroll theo thời gian với ba hàm biểu diễn, động lực học và dự đoán (minh hoạ, tự tính)

$$z_t = 1 + 0.9 \times 0 + 0.81 \times 2 + 0.729 \times 5 = 1 + 0 + 1.62 + 3.645 = 6.265$$

Đây chính là nhãn mà v_t^0 của mô hình được huấn luyện để khớp tới (qua loss categorical, không phải bình phương trực tiếp trong thí nghiệm Atari của bài báo, nhưng số liệu minh hoạ ý nghĩa công thức).

3.5 Pseudo-code tóm tắt (Hình S2 + mô tả văn bản)

```
# Represent: s0 = h(o_1..o_t)
# Plan (nhiều lượt mô phỏng):
for each simulation:
    # Selection: đi từ gốc tới lá bằng pUCT (mục 3.2), tra bảng R,S nếu cạnh đã mở,
    ↪  không gọi lại model
    # Expansion: tại lá, gọi model đúng 1 lần: r_l,s_l=g(s_{l-1},a_l); p_l,v_l=f(s_l);
    ↪  init N=0,Q=0,P=p_l
    # Backup: lan truyền G^k = sum gamma^tau r + gamma^{l-k} v_l, cập nhật N,Q dọc đường
    ↪  (mục 3.2)
output: pi_t ∝ N(root,.), nu_t = value gốc
# Act: a_{t+1} ~ pi_t

# Train (trên chuỗi lấy từ replay buffer, dùng hành động THẬT đã xảy ra, không phải MCTS)
s0 = h(o_1..o_t)
for k = 1..K: r^k,s^k = g(s^{k-1}, a_{t+k}); p^k,v^k = f(s^k)
loss = sum_k [ l_r(u_{t+k},r^k) + l_v(z_{t+k},v^k) + l_p(pi_{t+k},p^k) ] + c||theta||^2
backprop qua model đã unroll; scale loss mỗi head theo 1/K; scale gradient vào g() theo
↪  1/2
```

3.6 Reanalyze (Phụ lục H)

Biến thể tăng hiệu quả dữ liệu: chạy lại MCTS trên các chuỗi *cũ* trong replay buffer bằng tham số *mới nhất* để có chính sách mục tiêu tươi hơn (chất lượng cao hơn chính MCTS đã chạy khi thu thập dữ liệu); dùng cho 80% lượt cập nhật. Dùng thêm target network θ^- (tham số gần đây, ổn định hơn) để tính mục tiêu giá trị

bootstrap n bước tươi hơn. Siêu tham số đổi theo: số mẫu lấy trên mỗi trạng thái tăng từ 0.1 lên 2.0, trọng số loss giá trị giảm từ 1.0 xuống 0.25, số bước bootstrap giảm từ $n = 10$ xuống $n = 5$ (đổi để tránh overfit giá trị khi tái dùng dữ liệu nhiều lần).

4. Thí nghiệm và kết quả chính

Cấu hình chung: $K = 5$ bước giả định; 1 triệu mini-batch (kích thước 2048 cho board game, 1024 cho Atari); 800 lượt mô phỏng mỗi lần tìm kiếm cho board game, 50 cho Atari (cả huấn luyện và đánh giá). Mạng representation dùng kiến trúc CNN + residual giống AlphaZero nhưng **16** residual block (thay vì 20); dynamics dùng cùng kiến trúc với representation; prediction giống AlphaZero; 256 mặt phẳng ẩn.

Board game: ở cờ vây, MuZero nhích hơn AlphaZero dù tính toán ít hơn mỗi nút (16 so với 20 block); tác giả suy đoán MuZero "cache" tính toán vào cây tìm kiếm, mỗi lần gọi thêm dynamics function giúp hiểu thế cờ sâu hơn.

Atari, Bảng 1 (điểm chuẩn hoá theo người, trung bình/trung vị trên 57 game):

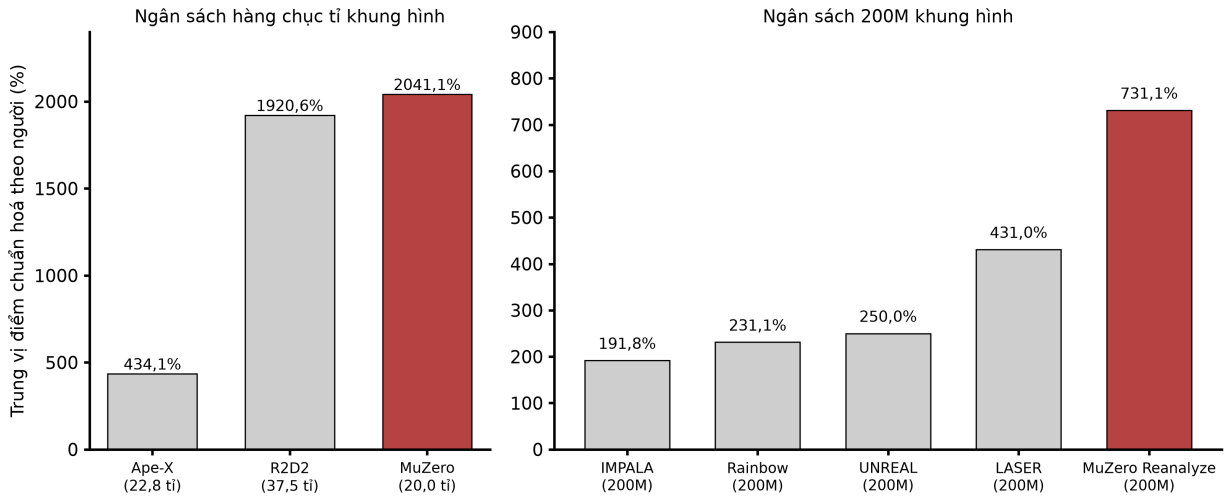
Agent	Trung vị	Trung bình	Số frame	Thời gian huấn luyện	Số bước huấn luyện
Ape-X	434.1%	1695.6%	22.8B	5 ngày	8.64M
R2D2	1920.6%	4024.9%	37.5B	5 ngày	2.16M
MuZero	**2041.1%**	**4999.2%**	20.0B	12 giờ	1M
IMPALA	191.8%	957.6%	200M	-	-
Rainbow	231.1%	-	200M	10 ngày	-
UNREAL (tinh chỉnh theo game)	250%	880%	250M	-	-
LASER	431%	-	200M	-	-
MuZero Reanalyze	**731.1%**	**2168.9%**	200M	12 giờ	1M

MuZero vượt R2D2 ở 42/57 game, vượt SimPLe (model-based trước đó) ở toàn bộ game có số liệu. Ở ngân sách thấp (200M frame), MuZero Reanalyze (731.1%/2168.9%) vượt xa các baseline model-free cùng ngân sách (IMPALA 192%, Rainbow 231%, LASER 431%).

Khả năng mở rộng theo lập kế hoạch:

- Cờ vây (Hình 3A): model học được khớp hiệu năng simulator hoàn hảo (AlphaZero) ngay cả khi tìm kiếm lâu hơn huấn luyện tới hai bậc độ lớn (huấn luyện ~0.1 giây/lượt = 800 lượt mô phỏng, đánh giá tới 10 giây).
- Atari (Hình 3B): cải thiện nhỏ hơn, chững lại quanh 100 lượt mô phỏng ("có thể do model kém chính xác hơn", suy đoán của tác giả). Ngay cả 1 lượt mô phỏng (chỉ dùng policy thô) vẫn chơi tốt, cho thấy policy đã "nội tâm hoá" lợi ích tìm kiếm tới cuối huấn luyện.
- Ms. Pacman, MCTS so Q-learning (Hình 3C): thay mục tiêu bằng Q-learning kiểu R2D2, bỏ tìm kiếm: đạt điểm cuối giống R2D2 nhưng học chậm hơn, hội tụ thấp hơn MuZero gốc. Tác giả suy luận tín hiệu từ tìm kiếm mạnh hơn target Q-learning (thiên lệch cao, phương sai cao).
- Ms. Pacman, số lượt mô phỏng *lúc huấn luyện* (Hình 3D): ngay cả 6 lượt/nước (ít hơn số hành động khả dĩ) vẫn học policy hiệu quả; nhiều lượt hơn cải thiện nhanh hơn, trái với một kết quả tiêu cực được trích dẫn trước đó về cây tìm kiếm kiểu GAN.

Cách đọc hình: panel trái so các phương pháp dùng ngân sách rất lớn, panel phải so ở 200 triệu frame. Chỉ vẽ điểm trung vị vì không phải phương pháp nào cũng có điểm trung bình trong ghi chú. Số liệu lấy từ Bảng 1 trong mục 4 của ghi chú.



Hình 45: Điểm trung vị chuẩn hoá trên Atari của MuZero và các phương pháp so sánh ở hai ngân sách frame (vẽ lại từ số liệu trong ghi chú)

5. Phân tích / ablation

So với AlphaZero (Phụ lục A): AlphaZero dùng tri thức luật chơi ở ba nơi: chuyển trạng thái trong cây (simulator thật), lọc hành động hợp lệ ở *mọi* nút, xử lý đặc biệt nút kết thúc (giá trị thật của simulator). MuZero thay cả ba: g_θ tạo trạng thái ẩn kế tiếp; chỉ lọc hành động hợp lệ ở **gốc** (nơi hỏi được môi trường thật), không lọc trong cây (mạng tự học không dự đoán hành động chưa từng xảy ra); không xử lý đặc biệt nút kết thúc, luôn dùng giá trị mạng dự đoán, coi trạng thái kết thúc là "trạng thái hấp thụ" khi huấn luyện. Ngoài ra MuZero tổng quát sang một agent, phần thưởng trung gian chiết khấu bất kỳ biên độ, thay cho hai người chơi, phần thưởng ±1 không chiết khấu cuối trận của AlphaZero.

Chênh lệch số liệu Atari giữa các paper (cần hiểu đúng): Bảng 1 trên dùng ngân sách 20 tỉ frame cho MuZero gốc (median 2041%, mean 4999%), nhưng bài 16 (DreamerV3) tự chạy lại baseline MuZero dưới protocol thống nhất 200M frame có sticky action, báo cáo MuZero chỉ đạt median 693%/mean 3054% ở ngân sách đó (Bảng 6, bài 16). Hai số không mâu thuẫn, chỉ là **ngân sách khác nhau một trăm lần**; luôn cần kèm ngân sách khi so sánh điểm chuẩn hoá theo người giữa hai paper.

6. Giới hạn

Tác giả nêu:

- Dynamics function biểu diễn **tất định**; "mở rộng sang chuyển trạng thái ngẫu nhiên để dành cho nghiên cứu tương lai".
- Trò chơi thông tin không đầy đủ (Poker) không được xử lý trực tiếp (chú thích 2).
- Atari: cải thiện theo số lượt mô phỏng chững lại sớm, tác giả cho là "có thể do model kém chính xác hơn" board game (suy đoán, không đo trực tiếp).

Nhận xét:

- Chi phí: 1000 TPU tự chơi + 16 TPU huấn luyện mỗi board game; 32+8 TPU mỗi game Atari (57 game!). Không khả thi ngoài tổ chức cỡ DeepMind.
- Bài 16 (DreamerV3) nhận xét về MuZero: "các tác giả không công bố mã nguồn và thuật toán có nhiều thành phần phức tạp, khiến việc tái lập khó khăn" (trích từ Previous work bài 16); đánh giá từ bên ngoài, không phải tự nhận của MuZero.
- Một số game Atari khó-khám-phá gần như không giải được dù SOTA trung bình: Montezuma's Revenge

0.00, Venture 0.0%, Solaris -10.6%, Pitfall 3.4% (Bảng S1), nằm sẵn trong phụ lục nhưng không được bàn trong văn bản chính.
- Không có ablation tách riêng đóng góp của "model học được" và "MCTS tổng quát hoá cho MDP"; Hình 3C/D là ablation gần nhất nhưng chỉ chạy trên một game (Ms. Pacman).

7. Liên hệ với các bài khác

- **Bài 14 (AlphaZero):** tiền nhiệm trực tiếp; MuZero giữ nguyên PUCT và cấu trúc selection/expansion/backup, chỉ thay simulator bằng model học được (ba điểm khác ở mục 5).
- **Bài 16 (DreamerV3):** lựa chọn thiết kế gần như đối lập. DreamerV3 dùng tái dựng quan sát làm tín hiệu huấn luyện chính (ablation bài 16: hiệu năng "chủ yếu dựa vào" gradient tái dựng, không phải gradient phần thưởng/giá trị); MuZero (value-equivalent model) **chủ động không** tái dựng quan sát, chỉ cần khớp phần thưởng/giá trị/chính sách. Một căng thẳng thiết kế thật, không có bằng chứng trong hai bài để nói cách nào đúng hơn nói chung, có thể tuỳ miền.
- **Bài 34 (STaR), bài 38 (DeepSeek-R1 distill):** diễn giải của phần phân tích: "tìm kiếm rồi hồi quy chính sách về mục tiêu tìm kiếm" (policy improvement operator của MuZero/AlphaZero) có họ hàng khái niệm với expert iteration: tự sinh mục tiêu tốt hơn rồi chưng cất lại vào mô hình.
- **Bài 35 (PRM):** diễn giải của phần phân tích: v^k là bộ chấm điểm học được cho trạng thái ẩn, có nét tương tự learned critic/PRM, dù PRM gắn với chuỗi suy luận ngôn ngữ tường minh còn v^k gắn với trạng thái ẩn trừu tượng.
- **Bài 39 (DeepSeek-R1):** Phụ lục G.2 "Unsuccessful Attempts" của DeepSeek-R1 thử MCTS kiểu AlphaGo/AlphaZero cho suy luận ngôn ngữ và thất bại khi mở rộng, vì không gian token quá lớn và mô hình giá trị theo từng bước khó huấn luyện (chi tiết ở bài 14, mục 7). MuZero cho thấy tìm kiếm cây chỉ cần model dự đoán đúng policy, value và reward, nhưng với sinh token thì chính mô hình giá trị ấy là phần khó nhất.

8. Ứng dụng thực tế (suy luận)

Liên quan **yếu đến trung bình** với RL cho LLM/VLM hiện nay: MuZero cần lập kế hoạch (nhiều lần forward qua g_θ, f_θ) tại **mỗi** hành động thật, chi phí cao hơn nhiều so với lấy mẫu một lượt rồi tối ưu bằng policy gradient (PPO/GRPO). Vài ý tưởng tách rời khỏi MCTS vẫn hữu ích:

- **Siêu tham số quan trọng:** K (độ dài unroll, 5 trong thí nghiệm) đánh đổi độ chính xác dự đoán dài hạn với chi phí; số lượt mô phỏng mỗi hành động đánh đổi sức mạnh quyết định với độ trễ (test-time compute mạnh hơn cả AlphaZero vì phải tính cả model); $\gamma = 0.997$ phù hợp tác vụ không có kết thúc tự nhiên rõ, cần chỉnh khác board game ($\gamma = 1$).
- **Biến đổi $h(x)$ + categorical:** tiền thân trực tiếp của symlog/two-hot ở bài 16, giải quyết "phần thưởng/giá trị biên độ không biết trước"; đáng mượn độc lập với MCTS khi cần hồi quy đại lượng có biên độ không chắc.
- **Chi phí/thất bại:** cần model đủ chính xác để tìm kiếm có lợi (Hình 3B: Atari chững ở ~100 lượt mô phỏng); model không đủ tốt thì tìm kiếm sâu hơn không giúp gì.

9. Thí nghiệm gợi ý

1. Cài MCTS không gian ẩn với model học được (ba hàm h, g, f) trên gridworld nhỏ có phần thưởng trung gian (một agent), đo độ nhạy chất lượng lập kế hoạch theo sai số dự đoán của g_θ (tái lập dạng Hình 3A/B ở quy mô nhỏ).
2. Ablation độ dài unroll $K \in \{1, 3, 5, 10\}$ khi huấn luyện trên task nhỏ, đo điểm cuối và độ ổn định.
3. Tái lập Hình 3C: hoán đổi mục tiêu huấn luyện giữa "chính sách tìm kiếm MCTS" và "mục tiêu Q-learning" trên cùng mạng, kiểm tra giả thuyết "tín hiệu từ tìm kiếm mạnh hơn target Q-learning".

4. Ablation tỉ lệ Reanalyze (0%, 50%, 80%, 100%) và đo hiệu quả dữ liệu (điểm theo số frame) trên cùng domain nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Ba hàm model	$s^0 = h_\theta(o_{\le t})$, $r^k, s^k = g_\theta(s^{k-1}, a^k)$, $p^k, v^k = f_\theta(s^k)$	representation, dynamics, prediction
pUCT (selection)	$Q + P\frac{\sqrt{\sum N}}{1+N}(c_1 + \log\frac{\sum N + c_2 + 1}{c_2})$	$c_1 = 1.25$, $c_2 = 19652$, chung với bài 14
Backup	$G^k = \sum_\tau \gamma^\tau r_{k+1+\tau} + \gamma^{l-k} v^l$	bootstrap từ giá trị lá
Chuẩn hoá Q	$\frac{Q - \min}{\max - \min}$ trên cây	khi giá trị không bị chặn
Loss tổng	$\sum_k [l_r + l_v + l_p] + c\|\theta\|^2$	unroll $K = 5$ bước
Mục tiêu giá trị	$z_t = u_{t+1} + \gamma u_{t+2} + \cdots + \gamma^{n-1} u_{t+n} + \gamma^n \nu_{t+n}$	$n = 10$ (Atari), hết trận (board game)
Biến đổi biên độ	$h(x) = \text{sign}(x)(\sqrt{\|x\| + 1} - 1) + \epsilon x$	$\epsilon = 0.001$, tiền thân của symlog bài 16
Reanalyze	chạy lại MCTS với θ mới trên dữ liệu cũ	80% lượt cập nhật, tăng hiệu quả dữ liệu

11. Kết luận ngắn

MuZero chứng minh rằng không cần biết luật/động lực học thật của môi trường để lập kế hoạch bằng MCTS: một model học được, chỉ cần dự đoán đúng chính sách, giá trị và phần thưởng (không cần tái dựng quan sát hay khớp ngữ nghĩa trạng thái thật), đủ để đạt hiệu năng board game tương đương AlphaZero (có simulator hoàn hảo) và vượt các phương pháp model-free tốt nhất trên Atari, kể cả khi tính theo hiệu quả dữ liệu (Reanalyze). Cái giá là chi phí lập kế hoạch tại mỗi quyết định và độ phức tạp cài đặt; những hạn chế này, cùng câu hỏi "có cần tái dựng quan sát hay không", dẫn trực tiếp tới các lựa chọn thiết kế khác của bài 16 (DreamerV3).

Link gốc: https://arxiv.org/abs/1911.08265

Bài 16

Mastering Diverse Domains through World Models (2023)

Tác giả: Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, Timothy Lillicrap (Google DeepMind; University of Toronto). **Venue / arXiv:** arXiv:2301.04104 (v2, 17/04/2024). Bản text trích xuất không nêu venue chính thức. **Nguồn:** https://arxiv.org/abs/2301.04104 **Chủ đề:** Thuật toán world-model tổng quát (thế hệ thứ ba của dòng Dreamer): học world model dạng RSSM (Recurrent State-Space Model), huấn luyện actor-critic hoàn toàn trong các rollout "tưởng tượng" do model sinh ra (không chạy MCTS lúc hành động), với một bộ kỹ thuật ổn định (symlog, two-hot, free bits/KL balance, chuẩn hoá return theo percentile) cho phép dùng **một** bộ siêu tham số cố định trên hơn 150 nhiệm vụ thuộc 8 miền khác nhau, kể cả lấy kim cương trong Minecraft từ số 0. **Vị trí trong lộ trình:** Đọc sau bài 14/15 (AlphaZero, MuZero: lập kế hoạch bằng tìm kiếm cây). DreamerV3 đại diện cho lập kế hoạch bằng "tưởng tượng" và lan truyền ngược, không chạy cây tìm kiếm ở thời điểm hành động. Ba bài 14-16 cùng thuộc giai đoạn 4 của lộ trình (search / world model); sau bài này, lộ trình chuyển sang imitation/offline RL (bài 17+).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Các thuật toán RL thường cần tinh chỉnh theo miền (liên tục/rời rạc, thưa/dày phần thưởng, ảnh/vector, 2D/3D), đòi hỏi chuyên môn và thực nghiệm riêng cho mỗi bài toán mới; PPO tương đối bền nhưng thường thua các phương pháp chuyên biệt về hiệu năng tuyệt đối. **World model:** học một model của môi trường để agent "tưởng tượng" tương lai và cải thiện hành vi mà không cần (hoặc cần ít) tương tác thật thêm (ý tưởng Dyna của Sutton, World Models của Ha & Schmidhuber, dòng PlaNet/Dreamer của Hafner). Vấn đề mở: học và dùng world model một cách **bền** (robust) để đạt hiệu năng tốt trên nhiều miền khác nhau, không riêng gì continuous control.

Ba thế hệ Dreamer (mục "Previous Dreamer generations" của bài báo): DreamerV1 chỉ giới hạn ở continuous control; DreamerV2 vượt người trên Atari; **DreamerV3** cho phép học "out of the box" trên nhiều benchmark khác nhau cùng lúc.

Khái niệm cần nắm: RSSM (Recurrent State-Space Model, model latent có trạng thái đệ quy h_t và biểu diễn ngẫu nhiên rời rạc z_t); symlog/symexp (biến đổi nén biên độ, đối xứng qua gốc, khác logarithm thường); two-hot encoding (tổng quát hoá one-hot cho giá trị liên tục, chia trọng số cho hai bin liền kề); free bits (bỏ gradient của KL khi đã đủ nhỏ); KL balancing (trọng số khác nhau cho hai chiều KL); unimix (trộn một phần nhỏ phân phối đều vào categorical để tránh suy biến); EMA target network; λ-return; replay ratio (số bước huấn luyện trên mỗi bước môi trường thu thập).

2. Ý tưởng chính

Ba mạng neural (world model, critic, actor) huấn luyện đồng thời từ dữ liệu phát lại (replay). World model (RSSM) mã hoá quan sát thành biểu diễn rời rạc z_t được dự đoán bởi trạng thái đệ quy h_t có điều kiện theo hành động; huấn luyện bằng tái dựng quan sát + dự đoán phần thưởng/tiếp diễn + khớp động lực học. Actor và critic học **hoàn toàn từ các rollout tưởng tượng** (latent trajectory) do world model sinh ra, không chạy tìm kiếm cây ở thời điểm hành động (agent chọn hành động bằng lấy mẫu trực tiếp từ actor). Đóng góp kỹ thuật chính không phải kiến trúc mới lạ, mà là một **bộ kỹ thuật ổn định hoá** (symlog, two-hot, free bits + KL balance, chuẩn hoá return theo percentile) cho phép **một** bộ siêu tham số cố định hoạt động tốt từ Atari tới robot liên tục tới Minecraft, điều mà các thuật toán RL trước đó không đạt được đồng thời.

3. Phương pháp chi tiết

3.1 World model (RSSM)

$$\begin{aligned}
\text{Sequence model:} &\quad h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \\
\text{Encoder:} &\quad z_t \sim q_\phi(z_t | h_t, x_t) \\
\text{Dynamics predictor:} &\quad \hat{z}_t \sim p_\phi(\hat{z}_t | h_t) \\
\text{Reward predictor:} &\quad \hat{r}_t \sim p_\phi(\hat{r}_t | h_t, z_t) \\
\text{Continue predictor:} &\quad \hat{c}_t \sim p_\phi(\hat{c}_t | h_t, z_t) \\
\text{Decoder:} &\quad \hat{x}_t \sim p_\phi(\hat{x}_t | h_t, z_t)
\end{aligned}$$

Encoder/decoder là CNN cho ảnh, MLP cho vector; dynamics/reward/continue predictor là MLP. z_t được lấy mẫu từ một vector phân phối softmax (categorical), gradient qua bước lấy mẫu dùng straight-through estimator. Huấn luyện end-to-end tối thiểu hoá:

$$L(\phi) = \mathbb{E}_{q_\phi}\Big[\sum_{t=1}^{T} \left(\beta_{pred} L_{pred}(\phi) + \beta_{dyn} L_{dyn}(\phi) + \beta_{rep} L_{rep}(\phi)\right)\Big], \quad \beta_{pred} = 1,\ \beta_{dyn} = 1,\ \beta_{rep} = 0.1$$

$$\begin{aligned}
L_{pred}(\phi) &= -\ln p_\phi(x_t | z_t, h_t) - \ln p_\phi(r_t | z_t, h_t) - \ln p_\phi(c_t | z_t, h_t) \\
L_{dyn}(\phi) &= \max\left(1,\ \mathrm{KL}[\mathrm{sg}(q_\phi(z_t | h_t, x_t)) \,\|\, p_\phi(z_t | h_t)]\right) \\
L_{rep}(\phi) &= \max\left(1,\ \mathrm{KL}[q_\phi(z_t | h_t, x_t) \,\|\, \mathrm{sg}(p_\phi(z_t | h_t))]\right)
\end{aligned}$$

$\mathrm{sg}(\cdot)$ là stop-gradient. L_{dyn} dạy *predictor* đuổi theo *encoder* cố định (qua sg); L_{rep} dạy *encoder* dễ đoán hơn trong khi predictor cố định. Trọng số khác nhau (1 so với 0.1), gọi là **KL balancing**, ưu tiên predictor đuổi theo encoder hơn ép encoder đơn giản hoá quá mức. **Free bits:** cắt (clip) cả hai KL ở ngưỡng dưới 1 nat (≈ 1.44 bit), tắt gradient khi KL đã đủ nhỏ, để năng lực học tập trung vào L_{pred} một khi động lực học đã khớp đủ tốt; đây là cách paper giải quyết mâu thuẫn "môi trường 3D phức tạp cần regularizer mạnh" với "game nền tĩnh cần regularizer yếu để giữ chi tiết pixel" bằng **cùng một** cặp trọng số cố định. **Unimix 1%:** categorical của encoder/dynamics predictor là hỗn hợp 99% đầu ra mạng + 1% phân phối đều, tránh xác suất 0 và tránh suy biến tất định.

3.2 Critic learning

Actor/critic hoạt động trên trạng thái model $s_t = \{h_t, z_t\}$ (biểu diễn Markov học được). Return $R_t = \sum_{\tau=0}^{\infty} \gamma^\tau r_{t+\tau}$, $\gamma = 0.997$. Để tính tới phần thưởng vượt quá chân trời dự đoán $T = 16$ (mục 6 nêu một mâu thuẫn tên gọi nhỏ với Bảng 4), critic học xấp xỉ **phân phối** của return:

$$\text{Actor: } a_t \sim \pi_\theta(a_t | s_t), \qquad \text{Critic: } v_\psi(R_t | s_t)$$

Giá trị đọc ra $v_t = \mathbb{E}[v_\psi(\cdot | s_t)]$. Mục tiêu huấn luyện critic là λ-return bootstrap:

$$\boxed{R_t^\lambda = r_t + \gamma c_t[(1-\lambda) v_t + \lambda R_{t+1}^\lambda], \qquad R_T^\lambda = v_T, \qquad \lambda = 0.95}$$

$$L(\psi) = -\sum_{t=1}^{T} \ln p_\psi(R_t^\lambda | s_t)$$

Critic được tham số hoá là phân phối **categorical trên các bin trải theo hàm mũ** (không phải Normal), vì phân phối return có thể đa đỉnh và biên độ thay đổi nhiều bậc giữa các môi trường. Loss critic áp lên cả quỹ đạo tưởng tượng (trọng số $\beta_{val} = 1$) **và** quỹ đạo replay thật (trọng số $\beta_{repval} = 0.3$, dùng λ-return tưởng tượng tại trạng thái bắt đầu làm nhãn on-policy rồi tính tiếp trên phần thưởng thật). Critic được chính quy hoá hướng về một bản sao EMA (decay 0.98) của chính nó, tương tự target network cổ điển nhưng vẫn dùng critic hiện tại để tính return. Trọng số đầu ra của reward predictor và critic khởi tạo **bằng 0** để tránh giá trị "ảo" lớn làm chậm khởi động học.

3.3 Actor learning

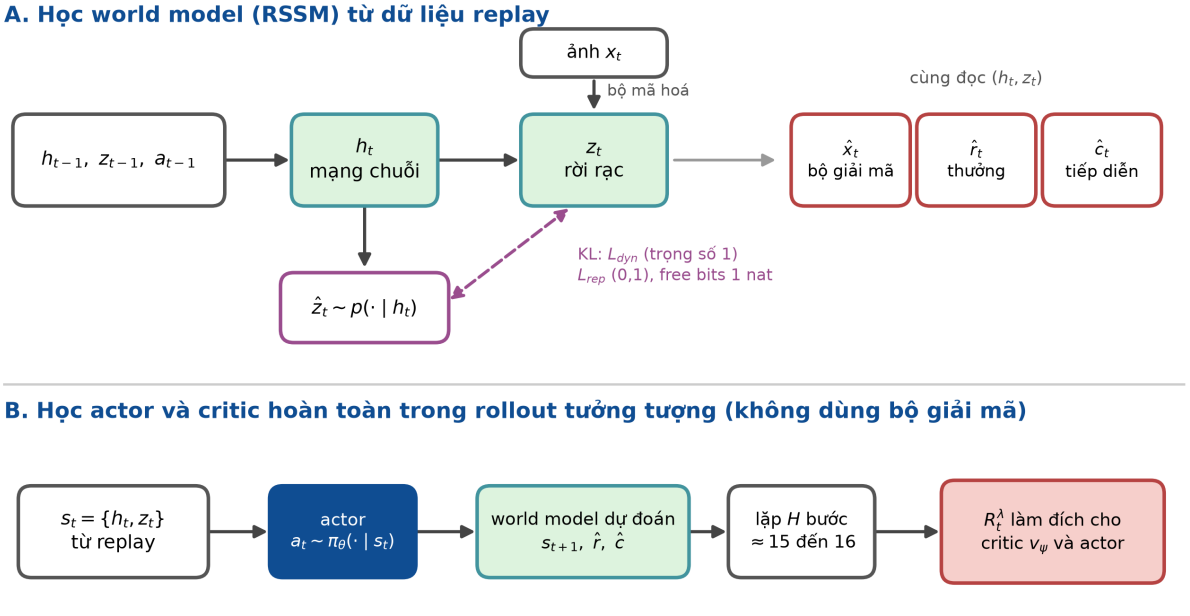
Actor tối đa hoá return với regularizer entropy, hệ số **cố định** $\eta = 3 \times 10^{-4}$ trên mọi miền, khả thi vì return được chuẩn hoá về khoảng $\approx [0, 1]$. Dùng ước lượng Reinforce cho cả hành động rời rạc và liên tục:

$$L(\theta) = -\sum_{t=1}^{T} \Big[\mathrm{sg}(R_t^\lambda - v_\psi(s_t)) / \max(1, S) \cdot \log \pi_\theta(a_t | s_t) + \eta \, H[\pi_\theta(a_t | s_t)] \Big]$$

Thang chuẩn hoá S dùng khoảng phân vị thứ 5 đến 95 (bền với outlier hơn min-max), trung bình trượt mũ:

$$S = \mathrm{EMA}(\mathrm{Per}(R^\lambda, 95) - \mathrm{Per}(R^\lambda, 5),\ 0.99)$$

Vì sao chia cho $\max(1, S)$ **chứ không phải** S **trực tiếp:** nếu return rất nhỏ (thưa), $S \to 0$ sẽ khuếch đại nhiễu xấp xỉ thành advantage "ảo" lớn, lấn át entropy regularizer; giữ nguyên (không chia) các return dưới ngưỡng $L = 1$ giải quyết vấn đề này. Paper so sánh với: chuẩn hoá kiểu PPO (không phân biệt phần thưởng "trong tầm tay" hay không); chuẩn hoá theo độ lệch chuẩn (thất bại khi std gần 0, khuếch đại nhiễu); tối ưu ràng buộc entropy trung bình cố định (bền nhưng khám phá chậm dưới phần thưởng thưa). Tác giả báo không tìm được siêu tham số ổn định xuyên miền cho các cách thay thế đó.



Hình 46: Hai pha của DreamerV3: học world model từ replay và học actor critic trong trí tưởng tượng (minh hoạ, tự tính)

Cách đọc hình: phần A cho thấy RSSM học từ dữ liệu replay, với trạng thái hồi quy, trạng thái ngẫu nhiên và các đầu ra tái dựng, phần thưởng, cờ kết thúc. Phần B dùng world model đã học để tưởng tượng một quỹ đạo ngắn rồi cập nhật actor và critic trên return kiểu lambda. Các hệ số KL và free bits ghi trên hình khớp với mục 3.1.

3.4 Dự đoán bền (symlog, two-hot)

$$\boxed{symlog(x) = \text{sign}(x)\ln(|x|+1), \qquad symexp(x) = \text{sign}(x)(\exp(|x|)-1)}$$

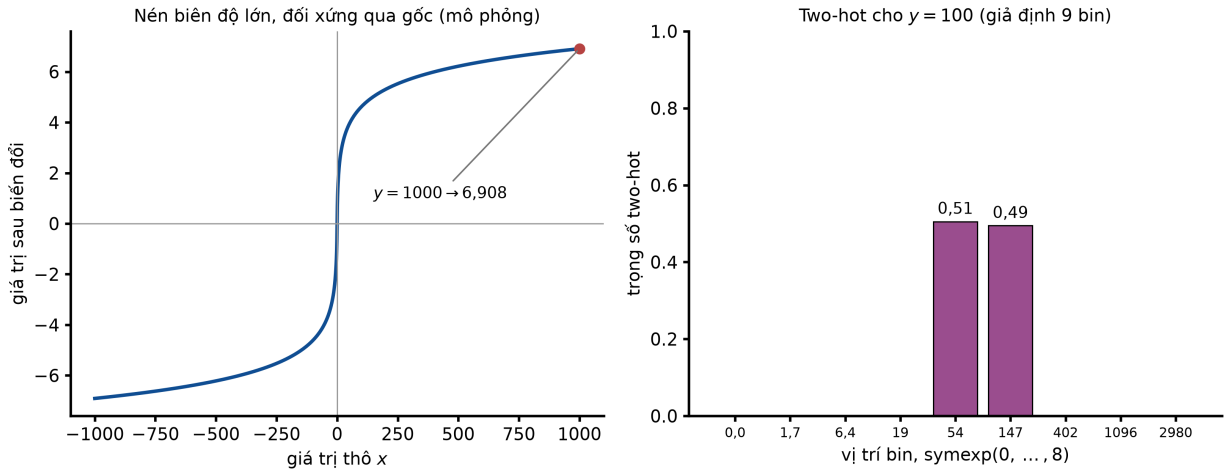
Hai hàm là nghịch đảo của nhau, đối xứng qua gốc (khác ln thường, không nhận giá trị âm), gần đồng nhất quanh 0, nén biên độ cả hai phía khi $|x|$ lớn. Loss hồi quy ("symlog squared error"):

$$L(\theta) = \tfrac{1}{2}(f(x,\theta) - symlog(y))^2, \qquad \hat{y} = symexp(f(x,\theta))$$

Cho đại lượng có thể ngẫu nhiên (phần thưởng, return): mạng xuất logit cho softmax trên các bin trải theo hàm mũ $B = symexp(\text{linspace}(-20, 20))$ (bài báo không nêu số bin cụ thể), đọc dự đoán bằng trung bình có trọng số $\hat{y} = softmax(f(x))^\top B$ (có thể rơi giữa hai bin). Huấn luyện bằng **two-hot encoding**: mã hoá một số thực thành vector chỉ có hai phần tử khác 0 tại hai bin liền kề, tổng bằng 1, trọng số tuyến tính theo khoảng cách; loss là cross-entropy với nhãn mềm hai-hot:

$$L(\theta) = -twohot(y)^\top \log softmax(f(x,\theta))$$

Loss này chỉ phụ thuộc xác suất gán cho các bin, không phụ thuộc trực tiếp vị trí số của bin, nên tách rời độ lớn gradient khỏi độ lớn mục tiêu. Dreamer áp dụng symlog cho đầu vào/đầu ra bộ mã hoá-giải mã của quan sát dạng vector, và symexp two-hot cho đầu ra của reward predictor và critic.



Hình 47: Đường symlog và hai trọng số two hot cho một giá trị đích (minh hoạ, tự tính)

Cách đọc hình: panel trái cho thấy symlog nén các giá trị rất lớn về thang nhỏ, ví dụ 1000 thành khoảng 6,9. Panel phải chia giá trị 100 vào hai ô lân cận với trọng số xấp xỉ 0,51 và 0,49. Đây là mô phỏng với giả định 9 ô cách đều trong không gian symlog, không phải số của paper.

3.5 Pseudo-code vòng lặp huấn luyện (tóm tắt từ văn bản)

```
Khởi tạo world model phi, critic psi, actor theta, replay buffer D
loop:
    # Thu thập (search-free: không chạy MCTS)
```

```
s_t = {h_t, z_t} từ world model với lịch sử quan sát thật
a_t ~ pi_theta(a_t | s_t)                    # lấy mẫu trực tiếp, không tìm kiếm
env.step(a_t) -> o_{t+1}, r_t, c_t ; lưu vào D
# Huấn luyện (chạy theo replay ratio)
lấy batch chuỗi (x_1:T, a_1:T, r_1:T, c_1:T) từ D
cập nhật phi bằng SGD trên L(phi) = sum_t [L_pred + L_dyn + L_rep]      (mục 3.1)
từ trạng thái replay, roll-out tưởng tượng s_1:H dưới actor hiện tại (dùng
↪ dynamics+reward predictor, KHÔNG dùng decoder)
tính R_t^lambda trên quỹ đạo tưởng tượng và trên quỹ đạo replay thật    (mục 3.2)
cập nhật psi bằng SGD trên L(psi)                                       (mục 3.2, cả
↪ hai nguồn)
cập nhật theta bằng SGD trên L(theta)                                   (mục 3.3,
↪ Reinforce + entropy)
cập nhật EMA của critic target
```

3.6 Ví dụ nhỏ (tự tính)

symlog/symexp: phần thưởng thô $y = 1000$ (điểm Atari lớn): $symlog(1000) = \ln(1001) \approx 6.908$. Nếu mạng học dự đoán đúng $f = 6.908$ thì đọc lại $\hat{y} = symexp(6.908) = e^{6.908} - 1 \approx 999.998 \approx 1000$. Với giá trị nhỏ $y = 0.001$: $symlog(0.001) = \ln(1.001) \approx 0.000999 \approx 0.001$, tức gần như hàm đồng nhất, không "nén" các giá trị đã nhỏ. Đây minh hoạ trực tiếp vì sao symlog nén mạnh ở biên độ lớn nhưng để nguyên vùng gần 0, so với log thường không nhận giá trị âm và không "thẳng" qua gốc.

Ngưỡng $\max(1, S)$**:** giả sử một batch λ-return chủ yếu nằm trong $[0, 2]$ nhưng vài episode đạt tới 50 (đột biến phần thưởng thưa); $\text{Per}(R^\lambda, 95) = 1.8$, $\text{Per}(R^\lambda, 5) = 0.1$, nên $S = 1.8 - 0.1 = 1.7$. Vì $S = 1.7 > 1$, advantage bị chia cho 1.7 (co lại một phần). Ngược lại nếu return rất nhỏ, ví dụ $S = 0.4 < 1$, advantage chia cho $\max(1, 0.4) = 1$, **giữ nguyên không chuẩn hoá**, tránh khuếch đại nhiễu dưới phần thưởng thưa.

4. Thí nghiệm và kết quả chính

Tổng quan 8 benchmark (Bảng 2), tất cả huấn luyện trên 1 GPU Nvidia A100:

Benchmark	Số task	Env steps	Action repeat	Env instance	Replay ratio	GPU-ngày	Kích thước model
Minecraft	1	100M	1	64	32	8.9	200M
DMLab	30	100M	4	16	32	2.9	200M
ProcGen	16	50M	1	16	64	16.1	200M
Atari	57	200M	4	16	32	7.7	200M
Atari100k	26	400K	4	1	128	0.1	200M
BSuite	23	-	1	1	1024	0.5	200M
Proprio Control	18	500K	2	16	512	0.3	12M
Visual Control	20	1M	2	16	512	0.1	12M

Tóm tắt theo domain: Atari (200M frame, sticky action): vượt MuZero, Rainbow, IQN. ProcGen (50M, hard): khớp PPG (tinh chỉnh), vượt Rainbow. DMLab (100M): vượt IMPALA/R2D2+ chạy ở 1 tỉ bước (gấp 10 lần dữ liệu), hiệu quả dữ liệu tăng hơn 1000%. Atari100k (400K): vượt IRIS, TWM, SPR, SimPLe (EfficientZero dẫn đầu nhưng dùng tìm kiếm cây trực tuyến + reset màn chơi sớm, tác giả nói "khó so sánh trực tiếp"). Proprio Control: vượt D4PG, DMPO, MPO. Visual Control: vượt DrQ-v2, CURL (dùng data augmentation chuyên biệt). BSuite: vượt Boot DQN, mạnh nhất ở nhóm "bền với thang phần thưởng".

Atari, số liệu tổng hợp (Bảng 6, 200M frame, sticky action, điểm chuẩn hoá theo người):

Agent	Trung vị	Trung bình
PPO	180%	892%
MuZero	693%	3054%
Dreamer	**830%**	**3381%**

(Lưu ý: đây là baseline MuZero do chính nhóm DreamerV3 tự huấn luyện lại dưới protocol 200M frame có sticky action, khác với con số MuZero tự báo cáo ở ngân sách 20 tỉ frame trong bài 15; xem thêm mục 6.)

Kích thước model (Bảng 3): tham số hoá theo "chiều model" d, tăng gần $\times 1.5$ mỗi bước, luân phiên lũy thừa 2 và lũy thừa 2 nhân 1.5:

Tham số	12M	25M	50M	100M	200M	400M
Chiều ẩn d	256	384	512	768	1024	1536
Đơn vị đệ quy ($8d$)	1024	3072	4096	6144	8192	12288
Kênh CNN gốc ($d/16$)	16	24	32	48	64	96

Số lớp và số latent **cố định** qua các kích thước; tất cả siêu tham số (học rate, batch size...) cũng cố định qua kích thước model. Lưu ý: giá trị 1024 ở cột đầu của dòng "Đơn vị đệ quy" không khớp công thức $8d$ mà chính văn bản nêu ($8 \times 256 = 2048$, trong khi 5 cột còn lại đều khớp đúng $8d$); người ghi chú giữ nguyên số liệu trích xuất được từ bản PDF (đúng rule 2: không tự sửa số của bài báo) nhưng nêu rõ nghi ngờ đây là lỗi đánh máy/trích xuất của chính bản PDF, không phải lỗi tính toán của phần phân tích.

Minecraft Diamond (Bảng 5, Return tại 100M bước):

Phương pháp	Return
Dreamer	**9.1**
IMPALA	7.1
Rainbow	6.3
PPO	5.1

Hai số liệu thành công ở Minecraft (đo hai thứ khác nhau, không mâu thuẫn, cả hai nêu trong chú thích Hình 9 khi đối chiếu lại với Hình 5): **100%** số agent Dreamer (trên 10 seed) thu được ít nhất một viên kim cương trong suốt huấn luyện, so với **0%** baseline (Hình 5 cho thấy Dreamer là thuật toán duy nhất phát hiện được kim cương), nhưng tính theo **tỉ lệ mỗi episode** tại checkpoint 100M bước, Dreamer chỉ lấy kim cương trong **0.4%** số episode (Hình 9, tác giả gọi đây "thử thách còn lại"). Chi phí: VPT (công trình trước) dùng 720 GPU trong 9 ngày cộng dữ liệu hành vi người; Dreamer dùng 1 GPU trong 9 ngày, không dữ liệu người.

5. Phân tích / ablation

Kỹ thuật ổn định hoá (Hình 6a, trung bình 14 task): bỏ bất kỳ kỹ thuật nào trong {symlog quan sát, chuẩn hoá return (thay bằng chuẩn hoá advantage), symexp two-hot (thay Huber loss), KL balance & free bits} đều giảm hiệu năng trung bình so với Dreamer đầy đủ. Mục tiêu KL của world model ảnh hưởng nhiều nhất, kế đó là chuẩn hoá return và symexp two-hot; nhưng "mỗi kỹ thuật quan trọng trên một tập task con, có thể không ảnh hưởng ở task khác" (không có kỹ thuật "quan trọng nhất" phổ quát).

Tín hiệu học (Hình 6b): bỏ gradient phần thưởng/giá trị (chỉ còn tái dựng định hình biểu diễn) hoặc bỏ gradient tái dựng, so với Dreamer đầy đủ. Kết luận: hiệu năng Dreamer **chủ yếu dựa vào** gradient tái dựng

không giám sát, khác các thuật toán RL trước (kể cả MuZero, bài 15) thường chỉ dựa tín hiệu phần thưởng/giá trị gắn tác vụ cụ thể, đối lập trực tiếp triết lý "value-equivalent model" của MuZero (mục 7).

Mở rộng kích thước model (Hình 6c, Crafter, DMLab Goals): hiệu năng tăng đơn điệu từ 12M tới 400M tham số; mô hình lớn hơn đạt điểm cao hơn **và** cần ít tương tác môi trường hơn cho cùng mức điểm.

Mở rộng replay ratio (Hình 6d): tăng replay ratio (1 tới 64 trong thí nghiệm) tăng hiệu năng/hiệu quả dữ liệu có thể đoán trước; cùng kích thước model, đây là cách đổi thêm tính toán lấy hiệu năng cao hơn.

6. Giới hạn

Tác giả nêu:

- Tỉ lệ thành công mỗi episode ở Minecraft Diamond chỉ 0.4% tại 100M bước, "để lại một thử thách cho nghiên cứu tương lai".
- So sánh với EfficientZero trên Atari100k "khó khăn" vì EfficientZero dùng tìm kiếm cây trực tuyến, replay có ưu tiên, lập lịch siêu tham số, và reset màn chơi sớm để tăng đa dạng dữ liệu.
- ProcGen chỉ chạy **1 seed** do hạn chế tính toán (khác các benchmark khác dùng 5-10 seed), nên độ tin cậy thống kê ở benchmark này thấp hơn.

Nhận xét:

- Khẩu hiệu "siêu tham số cố định" (Bảng 4: trọng số loss, optimizer, chân trời tưởng tượng, λ, chiết khấu, hệ số entropy, hằng số KL) đúng, nhưng **không bao gồm** kích thước model và replay ratio: Bảng 2 cho thấy hai đại lượng này được chọn riêng theo benchmark (200M so với 12M tham số; replay ratio 32 tới 1024). Cần nói rõ để không hiểu lầm "một cấu hình cho mọi thứ" theo nghĩa tuyệt đối.
- Mâu thuẫn nhỏ: văn bản mục Critic learning nói chân trời $T = 16$, Bảng 4 ghi "Imagination horizon $H = 15$"; bài báo không giải thích.
- Mỗi lần chạy vẫn là huấn luyện **một agent cho một task**, không phải một policy đa nhiệm vụ chạy đồng thời trên 150+ task; tránh hiểu "single configuration" thành "một mô hình cho mọi domain".
- Yêu cầu tái dựng quan sát là ràng buộc kiến trúc không nhỏ; chi phí so với model "không tái dựng" của MuZero (bài 15) không đo cùng thước đo trong hai paper, đây là nhận xét của phần phân tích.
- Chi phí tuyệt đối (GPU-ngày) vẫn đáng kể ở vài benchmark (ProcGen 16.1 ngày, Minecraft 8.9 ngày/1 GPU); "chỉ 1 GPU" khiêm tốn hơn TPU-fleet của bài 14/15 nhưng không "rẻ" theo thời gian tuyệt đối.

7. Liên hệ với các bài khác

- **Bài 14/15 (AlphaZero, MuZero):** đối lập trực tiếp cách dùng model. AlphaZero/MuZero chạy MCTS **tại mỗi hành động thật**; Dreamer **không tìm kiếm** lúc hành động, "tìm kiếm" được trả giá lúc huấn luyện qua rollout tưởng tượng, lúc suy luận chỉ một lượt forward qua actor. Paper phê bình MuZero ở mục Previous work: "các tác giả không công bố mã nguồn và thuật toán có nhiều thành phần phức tạp, khiến tái lập khó khăn". Căng thẳng thiết kế chưa giải quyết: MuZero chủ động **không** tái dựng quan sát (value-equivalent model), còn ablation của Dreamer (mục 5) cho thấy hiệu năng **chủ yếu dựa vào** gradient tái dựng; hai bài đều có thực nghiệm riêng ủng hộ lựa chọn mình, không bằng chứng chéo phân định cách nào tốt hơn.
- **Bài 00, bài 07 (PPO), bài 09 (What matters in on-policy RL):** công thức actor (mục 3.3) là REINFORCE có baseline và entropy bonus, gốc lý thuyết policy gradient bài 00. Paper trích dẫn trực tiếp Andrychowicz et al. (bài 09) và PPO (bài 07) khi chọn siêu tham số cho baseline PPO của chính họ (GAE $\lambda = 0.95$, clip 0.2, entropy 0.01, lr 3×10^{-4}).
- **Bài 41 (DAPO), bài 42 (Dr. GRPO):** diễn giải của phần phân tích: lập luận chống chuẩn hoá theo std khi phần thưởng thưa (std gần 0 khuếch đại nhiễu) cùng tinh thần phê bình std-normalization trong GRPO cho RL LLM (Dr. GRPO: lệch theo độ khó/độ dài câu trả lời). Hai dòng nghiên cứu độc lập cùng cảnh báo chuẩn hoá theo thống kê có thể tiến về 0.

- **Bài 53/54 (Search-R1, khảo sát agentic RL):** Dreamer không tìm kiếm lúc suy luận, đối lập AlphaZero/MuZero/Search-R1 về nơi trả giá cho "suy nghĩ thêm": lúc huấn luyện (Dreamer) so với lúc suy luận (bài 14/15/53). Diễn giải của phần phân tích.

8. Ứng dụng thực tế (suy luận)

- **Siêu tham số quan trọng:** trọng số KL balance ($\beta_{dyn} = 1$ so với $\beta_{rep} = 0.1$) và ngưỡng free bits (1 nat) quyết định latent có đủ thông tin mà không "nổ" KL; symlog/two-hot gần như luôn nên dùng khi biên độ phần thưởng/giá trị không biết trước hoặc thay đổi nhiều bậc (mẹo kỹ thuật tổng quát, không gắn riêng world model); chuẩn hoá return theo percentile (có sàn $\max(1, S)$) an toàn hơn chuẩn hoá theo std khi phần thưởng rất thưa.
- **Thất bại/chi phí:** cần mục tiêu "tái dựng" có nghĩa (pixel/vector); với môi trường chỉ có token văn bản, "tái dựng prompt" không rõ có hữu ích như tái dựng khung ảnh game, nên lợi ích chính của RSSM khó chuyển thẳng sang RL cho LLM. Học thêm một model động lực học cũng thêm độ phức tạp so với policy-gradient thuần (PPO/GRPO) khi không có tín hiệu tái dựng rẻ.

9. Thí nghiệm gợi ý

1. Ablation tỉ lệ KL balance ($\beta_{dyn} : \beta_{rep}$) và ngưỡng free bits trên một task thị giác đơn giản và một phức tạp, tái lập ở quy mô nhỏ tuyên bố "siêu tham số cố định" của bài báo.
2. Thay đầu reward/value symexp two-hot bằng Huber loss (ablation Hình 6a) trên một task phần thưởng thưa và một dày, so sánh độ ổn định/hiệu năng cuối.
3. Tái lập Hình 6b ở quy mô nhỏ: huấn luyện một agent chỉ gradient tái dựng, một agent ngược lại (chỉ gradient phần thưởng/giá trị), cùng task pixel nhỏ, kiểm tra tái dựng có chiếm ưu thế như ở quy mô lớn.
4. Sweep replay ratio $(1, 4, 16, 64)$ trên một task nhỏ cố định ngân sách wall-clock, tìm điểm cân bằng tính toán/hiệu quả dữ liệu cho môi trường một-GPU.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
RSSM	$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}),\ z_t \sim q_\phi(\cdot \mid h_t, x_t)$	trạng thái đệ quy + biểu diễn rời rạc
Loss world model	$\beta_{pred}L_{pred} + \beta_{dyn}L_{dyn} + \beta_{rep}L_{rep}$	$1, 1, 0.1$; free bits ngưỡng 1 nat
λ-return	$R_t^\lambda = r_t + \gamma c_t[(1-\lambda)v_t + \lambda R_{t+1}^\lambda]$	$\lambda = 0.95$, $\gamma = 0.997$
Actor loss	$-\text{sg}(R^\lambda - v)/\max(1, S) \cdot \log \pi + \eta H[\pi]$	Reinforce + entropy, $\eta = 3 \times 10^{-4}$
Chuẩn hoá return	$S = \text{EMA}(\text{Per}_{95} - \text{Per}_5,\ 0.99)$	bền outlier, sàn $L = 1$
symlog/symexp	$\text{sign}(x)\ln(\lvert x\rvert + 1)$ / nghịch đảo	nén biên độ, đối xứng qua gốc
Two-hot	hai bin liền kề, trọng số tuyến tính	tổng quát one-hot cho số liên tục
Model size	$d = 256..1536$, 6 cỡ 12M-400M	siêu tham số cố định qua các cỡ

11. Kết luận ngắn

DreamerV3 cho thấy một world model học từ tái dựng quan sát, kết hợp actor-critic huấn luyện hoàn toàn trong rollout tưởng tượng, có thể đạt hiệu năng tốt hơn nhiều thuật toán chuyên biệt trên hơn 150 nhiệm vụ thuộc 8 miền rất khác nhau (gồm cả thử thách lấy kim cương trong Minecraft từ số 0, trên 1 GPU, không dữ liệu người), bằng **một** bộ siêu tham số cố định. Đóng góp cốt lõi không phải một kiến trúc mới, mà là bộ kỹ thuật ổn định hoá (symlog, two-hot, free bits/KL balance, chuẩn hoá return theo percentile) giải quyết đúng vấn đề "biên độ phần thưởng/quan sát không biết trước và thay đổi nhiều bậc giữa các miền". Đổi lại, "một cấu hình cho mọi thứ" không bao gồm kích thước model/replay ratio (vẫn chọn theo ngân sách từng benchmark),

và triết lý “world model cần tái dựng quan sát” đối lập trực tiếp với hướng value-equivalent model của bài 15 (MuZero), một câu hỏi thiết kế còn mở của RL dựa trên model.

Link gốc: https://arxiv.org/abs/2301.04104

Bài 17

A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (2010)

Tác giả: Stéphane Ross, Geoffrey J. Gordon, J. Andrew Bagnell (Robotics Institute và Machine Learning Department, Carnegie Mellon University).
Venue / arXiv: arXiv:1011.0686 (v3, 16/03/2011); AISTATS 2011 (Proceedings of the 14th International Conference on Artificial Intelligence and Statistics, JMLR W&CP vol. 15).
Nguồn: https://arxiv.org/abs/1011.0686
Chủ đề: Compounding error trong imitation learning (behavior cloning), chặn lỗi bậc hai theo horizon T của cách học giám sát thuần; thuật toán **DAgger** (Dataset Aggregation) biến imitation learning thành một bài toán no-regret online learning, đạt chặn lỗi tuyến tính theo T.
Vị trí trong lộ trình: Đọc sau bài 00 (policy gradient, importance sampling, các khái niệm RL nền tảng). Mở đầu giai đoạn 5 "Imitation learning & offline RL" (bài 17-21). Đọc trước bài 18 (GAIL, imitation learning không cần hỏi expert khi huấn luyện) và bài 19 (Offline RL tutorial, trích lại trực tiếp hai định lý của bài này làm lý do offline RL khó).
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Imitation learning (học bắt chước): cho sẵn các minh chứng (demonstration) của một "expert" (chuyên gia, π^*): một dãy quan sát/hành động $(s, \pi^*(s))$. Mục tiêu là học một policy π bắt chước π^*, không cần hàm phần thưởng. Nếu coi đây là học giám sát thông thường (classifier dự đoán hành động từ trạng thái), ta ngầm giả định train/test cùng phân phối (i.i.d.). Nhưng ở đây *hành động của policy ảnh hưởng tới trạng thái tương lai mà chính nó gặp lại*: huấn luyện trên phân phối trạng thái của expert, nhưng khi thi hành, policy tự tạo ra phân phối trạng thái của riêng nó. Đây là **covariate shift tuần tự**: một khi policy sai một bước, nó có thể rơi vào trạng thái mà expert hiếm khi gặp, nơi nó chưa từng được huấn luyện, và sai tiếp, sai chồng sai.

Định nghĩa dùng trong bài báo: Π = lớp policy; T = horizon. $d_\pi^t(s)$: phân phối trạng thái tại bước t nếu thi hành π từ đầu; $d_\pi = \frac{1}{T}\sum_t d_\pi^t$: phân phối trạng thái trung bình trên T bước. $C(s,a) \in [0,1]$: chi phí tức thời thật của tác vụ (có thể không quan sát được); $C_\pi(s) = \mathbb{E}_{a\sim\pi(s)}[C(s,a)]$. $J(\pi) = \sum_{t=1}^{T} \mathbb{E}_{s\sim d_\pi^t}[C_\pi(s)] = T\,\mathbb{E}_{s\sim d_\pi}[C_\pi(s)]$: tổng chi phí kỳ vọng trong T bước ("cost-to-go" toàn cục). $\ell(s,\pi)$: **surrogate loss** quan sát được, tối thiểu hóa thay cho C (ví dụ 0-1 loss so với hành động expert), giả định là chặn trên của C.

Cách học giám sát truyền thống (baseline): huấn luyện π để tối thiểu $\mathbb{E}_{s\sim d_{\pi^*}}[\ell(s,\pi)]$, tức là **chỉ trên phân phối trạng thái của expert** d_{π^*} (đây chính là behavior cloning / supervised imitation learning thông thường).

Theorem 2.1 (Ross & Bagnell 2010, bài báo này dẫn lại): nếu $\mathbb{E}_{s\sim d_{\pi^*}}[\ell(s,\pi)] = \epsilon$ (lỗi tổng quát hóa đo trên phân phối *của expert*), thì

$$\boxed{J(\pi) \le J(\pi^*) + T^2\epsilon}$$

và chặn này là **chặt (tight)**: tồn tại bài toán mà một policy có lỗi ϵ trên d_{π^*} gây thêm chi phí tăng theo $\Theta(T^2\epsilon)$ (Kääriäinen 2006: dự đoán ký tự kế tiếp trong một dãy, dùng ký tự *đúng* trước đó làm input khi huấn luyện nhưng ký tự *tự dự đoán* khi test). Trực giác: ngay khi policy sai một bước, nó rơi vào trạng thái khác phân

phối expert, nơi học giám sát không cam kết gì; một khi "trật đường" nó có thể sai mãi, cộng dồn lỗi, nên tổng lỗi tăng theo T^2 thay vì T.

Hai hướng khắc phục trước đó (Ross & Bagnell 2010), chưa thỏa đáng:

- **Forward training:** huấn luyện T policy riêng $\pi_1, \dots, \pi_T$, π_t học trên phân phối trạng thái bước t do $\pi_1, \dots, \pi_{t-1}$ tạo ra. Cho chặn $J(\pi) \leq J(\pi^*) + uT\epsilon$ với u đo "khả năng hồi phục" của expert ($u = O(1)$ nếu expert nhanh quay lại trạng thái tốt; tệ nhất $u = O(T)$). Nhược điểm: phải huấn luyện tuần tự đúng T policy, không dừng giữa đường, không khả thi khi T lớn/không xác định.
- **SMILe:** học một policy ngẫu nhiên tĩnh (hỗn hợp nhiều policy, giống SEARN/CPI): π_0 luôn hỏi expert; vòng n học $\hat{\pi}_n$ bắt chước expert trên phân phối của π_{n-1}, cập nhật $\pi_n = \pi_{n-1} + \alpha(1-\alpha)^{n-1}(\hat{\pi}_n - \pi_0)$. Đạt regret gần tuyến tính nếu $\alpha = O(1/T^2)$, $N = O(T^2 \log T)$ vòng. Nhược điểm: policy cuối là **hỗn hợp ngẫu nhiên**, có thể không ổn định.

Câu hỏi còn mở: học một policy **tĩnh và tất định**, vẫn đạt chặn gần tuyến tính, mà không cần huấn luyện T policy tuần tự?

2. Ý tưởng chính

DAgger (Dataset Aggregation) lặp lại một vòng đơn giản: ở vòng i, dùng chính policy hiện tại $\hat{\pi}_i$ (có thể trộn với expert) để **tự đi** trong môi trường, thu thập các trạng thái nó thực sự gặp, hỏi expert hành động đúng tại các trạng thái đó, rồi **gộp** dữ liệu mới vào toàn bộ dữ liệu đã thu thập từ trước, và huấn luyện lại một policy mới trên *toàn bộ* dữ liệu gộp đó. Qua nhiều vòng, tập dữ liệu training ngày càng phản ánh đúng phân phối trạng thái mà policy cuối cùng sẽ gặp khi thi hành, giải quyết đúng vấn đề gốc (train và test cùng phân phối). Về lý thuyết, đây là một ví dụ của **Follow-The-Leader**, một thuật toán **no-regret online learning**: paper chứng minh rằng bất kỳ thuật toán no-regret nào áp dụng theo cách này đều học được một policy tĩnh, tất định, với chặn lỗi **tuyến tính** theo T (thay vì bậc hai), miễn là lớp hàm mất mát đủ chính quy (strongly convex). Đây là một **reduction**: biến imitation learning thành online learning, tái sử dụng được các thuật toán học giám sát/online có sẵn.

3. Phương pháp chi tiết

3.1 Thuật toán DAgER

```
Khởi tạo D <- rỗng, pi_hat_1 là policy tuỳ ý trong lớp Pi.
for i = 1 .. N:
    pi_i = beta_i * pi_star + (1 - beta_i) * pi_hat_i      # trộn policy hiện tại với
    ↪  expert
    Thi hành pi_i, lấy mẫu các quỹ đạo T bước
    D_i = { (s, pi_star(s)) : s là trạng thái pi_i đã ghé qua }   # hỏi expert hành động
    ↪  đúng tại các s này
    D <- D union D_i                                        # GỘP dữ liệu, không xoá dữ
    ↪  liệu cũ
    Huấn luyện pi_hat_{i+1} bằng học giám sát trên toàn bộ D
Trả về policy pi_hat_i tốt nhất trên tập validation.
```

(Algorithm 3.1 của bài báo.)

Vai trò của β_i: $\beta_1 = 1$ thường được chọn (vòng đầu chỉ cần expert, không cần $\hat{\pi}_1$ khởi tạo). Có thể giảm dần theo $\beta_i = p^{i-1}$ như SMILe/SEARN. Yêu cầu duy nhất để chứng minh hội tụ: $\bar{\beta}_N = \frac{1}{N}\sum_{i=1}^{N} \beta_i \to 0$ khi $N \to \infty$. Biến thể **không tham số tự do** $\beta_i = \mathbb{1}(i = 1)$ (chỉ vòng đầu dùng expert) "thường cho kết quả tốt nhất trong thực tế" theo chính paper.

Khác biệt mấu chốt với forward training và SMILe: DAgger **không** cần huấn luyện T policy tuần tự, và policy trả về là **một** policy tĩnh tất định (chọn $\hat{\pi} = \arg\min_{\pi \in \hat{\pi}_{1:N}} \mathbb{E}_{s \sim d_\pi}[\ell(s, \pi)]$ qua validation; chọn ngẫu nhiên

đều trong $\hat{\pi}_{1:N}$ cũng giữ cùng chặn lý thuyết).

3.2 Tại sao đây là một bài toán no-regret online learning

Trong online learning, thuật toán chọn π_n ở vòng n, chịu loss $\ell_n(\pi_n)$, rồi mới biết ℓ_n để chọn π_{n+1}. Thuật toán **no-regret** đảm bảo $\frac{1}{N}\sum_{i=1}^{N}\ell_i(\pi_i) - \min_{\pi\in\Pi}\frac{1}{N}\sum_{i=1}^{N}\ell_i(\pi) \le \gamma_N \to 0$ (Follow-The-Leader với ℓ strongly convex đạt $\gamma_N = \tilde{O}(1/N)$).

DAgger khớp khuôn này nếu định nghĩa loss ở vòng i là **loss dưới phân phối trạng thái của chính policy đang thi hành**: $\ell_i(\pi) := \mathbb{E}_{s\sim d_{\pi_i}}[\ell(s,\pi)]$, coi *một vòng thu thập dữ liệu dưới một policy* là "một ví dụ" online learning. Chọn $\hat{\pi}_{i+1}$ tốt nhất trên toàn bộ D gộp đến vòng i chính là Follow-The-Leader.

Lemma 4.1 (mức lệch phân phối do trộn expert): $\|d_{\pi_i} - d_{\hat{\pi}_i}\|_1 \le 2T\beta_i$ (phân phối trạng thái của $\pi_i = \beta_i\pi^* + (1-\beta_i)\hat{\pi}_i$ không lệch quá xa $\hat{\pi}_i$ thuần, miễn β_i nhỏ).

Theorem 4.1 (vô hạn mẫu): tồn tại $\hat{\pi} \in \hat{\pi}_{1:N}$ sao cho $\mathbb{E}_{s\sim d_{\hat{\pi}}}[\ell(s,\hat{\pi})] \le \epsilon_N + \gamma_N + 2\ell_{max}\left[\frac{n_\beta}{N} + \frac{T}{N}\sum_{i>n_\beta}\beta_i\right]$, với ϵ_N = lỗi của policy tốt nhất "nhìn lại" dưới các d_{π_i} đã thu thập, n_β = vòng cuối mà $\beta_i > 1/T$. Với $\beta_i = (1-\alpha)^{i-1}$, số hạng phụ $\to 0$ khi $N = \tilde{O}(T)$: **DAgger cần số vòng cùng cấp với bất kỳ thuật toán no-regret nào**, không có "thuế" thêm vì phải hỏi expert.

3.3 Từ loss dưới phân phối riêng sang chặn chi phí toàn cục J

Đây là bước nối lý thuyết no-regret (Theorem 4.1, cho ϵ-loss dưới *phân phối của chính policy*) với chặn chi phí $J(\pi)$ kiểu Theorem 2.1. Paper tổng quát hóa Theorem 2.1 cho mọi policy có lỗi ϵ dưới phân phối của chính nó:

Theorem 2.2 (diễn giải, tổng quát hơn): nếu $\mathbb{E}_{s\sim d_\pi}[\ell(s,\pi)] = \epsilon$, và u chặn trên sự khác biệt cost-to-go giữa một hành động tuỳ ý so với hành động expert tại cùng trạng thái, thì $J(\pi) \le J(\pi^*) + uT\epsilon$ (u nhỏ khi expert "dễ hồi phục" từ lỗi của π; $u \le 1$ nếu ℓ là 0-1 loss so với expert; worst case $u = O(T)$).

Ghép Theorem 2.2 với kết quả no-regret (Theorem 4.1) cho kết quả chính:

Theorem 3.2 (chặn chính của DAgger): nếu số vòng $N = \tilde{O}(uT)$, tồn tại $\hat{\pi} \in \hat{\pi}_{1:N}$ sao cho

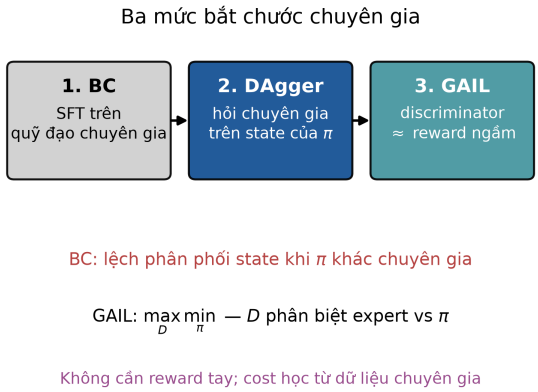
$$\boxed{J(\hat{\pi}) \le J(\pi^*) + uT\epsilon_N + O(1)}$$

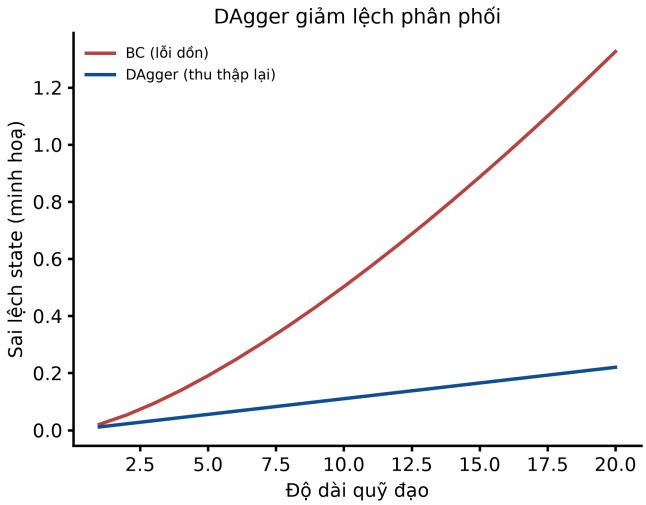
So với baseline $J(\pi) \le J(\pi^*) + T^2\epsilon$ (Theorem 2.1): khi $u = O(1)$ (trường hợp thường gặp, ví dụ ℓ là 0-1 loss so với expert, $u \le 1$), chặn DAgger là $J(\pi^*) + T\epsilon_N + O(1)$: **tuyến tính theo** T thay vì bậc hai.

Phiên bản mẫu hữu hạn (Theorem 3.3, 3.4): nếu mỗi vòng chỉ lấy m quỹ đạo mẫu (không phải toàn bộ phân phối), dùng bất đẳng thức Azuma-Hoeffding để kiểm soát nhiễu: với $N = O(T^2\log(1/\delta))$ và $m = O(1)$, tồn tại $\hat{\pi}$ đạt $\mathbb{E}_{s\sim d_{\hat{\pi}}}[\ell(s,\hat{\pi})] \le \hat{\epsilon}_N + O(1/T)$ với xác suất $\ge 1-\delta$ ($\hat{\epsilon}_N$ là lỗi huấn luyện trên mẫu). Phân tích tinh hơn (tận dụng tính strongly convex của ℓ) có thể hạ xuống $N = \tilde{O}(T\log(1/\delta))$.

3.4 Ví dụ số minh họa "T² so với T" (tự tính)

Với mỗi bước, chi phí $C \in [0,1]$, nên thêm-chi-phí tối đa không thể vượt T (chặn hiển nhiên: $J(\pi) - J(\pi^*) \le T$). Lấy $\epsilon = 0.001$ (lỗi phân loại 0.1%), $T = 2000$:





BC → DAgger (giảm lệch state) → GAIL (discriminator = reward ngầm) (minh hoạ, tự tính)

- Behavior cloning (Theorem 2.1): thêm-chi-phí $\leq T^2\epsilon = 2000^2 \times 0.001 = 4000$, **lớn hơn cả chặn hiển nhiên** $T = 2000$: chặn này hoàn toàn vô nghĩa (vacuous) ở độ dài này. Điểm "gãy" xảy ra ở $T = 1/\epsilon = 1000$: với $T > 1000$, chặn $T^2\epsilon$ đã vượt quá chặn hiển nhiên T.
- DAgger ($u \leq 1$, Theorem 3.2): thêm-chi-phí $\leq uT\epsilon_N \approx T\epsilon = 2000 \times 0.001 = 2$, rất nhỏ so với horizon 2000.

Đây minh họa cụ thể lý do thực nghiệm: behavior cloning thuần hoạt động tốt với horizon ngắn, nhưng "vỡ trận" khi T lớn hơn $1/\epsilon$ (dự đoán càng dài, lỗi tích lũy theo cấp số nhân thực tế dù lý thuyết chỉ chứng minh T^2).

4. Thí nghiệm và kết quả chính

(a) Super Tux Kart (lái xe 3D): điều khiển tay lái liên tục $[-1, 1]$ từ ảnh game (đặc trưng màu LAB 25x19, resize từ 800x600), cập nhật 5Hz, bộ học là **ridge regression** tuyến tính ($\lambda = 10^{-3}$). Metric: số lần rơi khỏi đường/lap. So sánh Supervised, SMILe ($\alpha = 0.1$), DAgger ($\beta_i = \mathbb{1}(i = 1)$), mỗi vòng ~1000 điểm dữ liệu (1 lap), 20 vòng.

- Supervised: không cải thiện dù thêm dữ liệu (chỉ huấn luyện trên quỹ đạo giống expert, không học cách sửa lỗi).
- SMILe: cải thiện nhưng sau 20 vòng vẫn rơi khỏi đường ~2 lần/lap (bản chất ngẫu nhiên của policy hỗn hợp).
- **DAgger:** sau 15 vòng, policy **không bao giờ rơi khỏi đường**; từ vòng 5 đã gần như không rơi, vượt rõ cả hai baseline; đường đi cũng "mượt" hơn (quan sát định tính qua video).

(b) Super Mario Bros: dự đoán 4 nút nhị phân {left, right, jump, speed} bằng 4 **linear SVM** độc lập, đặc trưng 27152 chiều nhị phân thưa (lưới 22x22 ô quanh Mario, lịch sử 4 khung hình, 6 hành động gần nhất, trạng thái Mario). Expert là planner gần-tối-ưu truy cập được trạng thái nội bộ game. 5000 điểm/vòng, 20 vòng, stage độ khó 1 (khoảng cách tối đa ~4200-4300), giới hạn 60s. Metric: khoảng cách trung bình/stage.

- Supervised: điểm thấp vì Mario thường bị **kẹt** cạnh vật cản (expert luôn nhảy từ xa nên không dạy được cách nhảy khi đứng sát).
- DAgger $\beta_i = \mathbb{1}(i = 1)$: ~2980. DAgger $\beta_i = 0.5^{i-1}$ (tốt nhất trong $p \in \{0.1, ..., 0.9\}$): ~3030. $\beta_i = 0.9^{i-1}$: hội tụ chậm hơn nhiều, vẫn đang cải thiện ở vòng 20.
- SEARN tốt nhất $\alpha = 0.4$ và SMILe $\alpha = 0.1$: thua DAgger ở mọi α đã thử; SEARN $\alpha = 1$ (pure policy iteration) kém ổn định hơn.

(c) Handwriting recognition (Taskar et al. 2003): ~6600 từ, >52000 ký tự, 10-fold. Dự đoán từng ký tự trái-sang-phải bằng **multiclass linear SVM** (128 pixel của ảnh ký tự + 26 đặc trưng mã hoá ký tự dự đoán trước đó), theo khung SEARN của Daumé III et al. (2009) coi structured prediction là một dạng **thoái hoá (degenerate)** của imitation learning (hành động dự đoán trước được đưa làm input cho bước sau). Metric: character accuracy trên test fold, 20 vòng.

Phương pháp	Character accuracy
Không cấu trúc (dự đoán độc lập mỗi ký tự)	82%
Supervised (luôn dùng ký tự trước đúng khi train)	83.6%
SMILe $\alpha = 0.1$, SEARN $\alpha = 0.1$	thấp hơn DAgger rõ rệt
SEARN tốt nhất $\alpha = 0.8$, SEARN $\alpha = 1$	gần DAgger
DAgger	**85.5%**

Dùng decode **greedy một lượt** (đơn giản nhất) để làm rõ lợi ích của DAgger; paper lưu ý decode nhiều lượt/beam-search có thể cho kết quả cạnh tranh hơn với SOTA.

5. Phân tích / ablation

- **Độ nhạy với β_i (Mario):** indicator và $\beta_i = 0.5^{i-1}$ cho kết quả gần nhau (2980 vs 3030), nhưng $\beta_i = 0.9^{i-1}$ (dùng expert lâu hơn) hội tụ chậm hẳn. Diễn giải của tác giả: indicator khiến giai đoạn đầu sinh nhiều dữ liệu "Mario kẹt ở một chỗ" (policy đầu rất tệ, lặp lại cùng lỗi); giữ expert lâu hơn một chút giúp "gỡ kẹt" và quan sát đa dạng hơn, dù hội tụ chậm hơn nếu giữ quá lâu.
- **Biến thể không tham số** ($\beta_i = \mathbb{1}(i = 1)$) "thường cho kết quả tốt nhất trong thực tế" theo tuyên bố của bài báo (Super Tux Kart dùng chính biến thể này).
- **So sánh với pure policy iteration (SEARN $\alpha = 1$)** trên handwriting: gần bằng DAgger, khác hẳn Super Tux Kart/Mario (nơi pure policy iteration/SMILe kém ổn định hơn nhiều). Giải thích: trong handwriting chỉ một phần nhỏ input (ký tự trước) bị ảnh hưởng bởi policy hiện tại, nên "ít bất ổn hơn các bài toán RL/imitation learning nói chung", lợi thế của DAgger tăng theo mức độ policy có thể làm lệch phân phối trạng thái, không phải lợi thế cố định ở mọi tác vụ.
- Thuật toán **không cần tham số tự do nào ngoài bộ học giám sát con** (không cần α tinh chỉnh như SMILe/SEARN nếu dùng biến thể indicator).

6. Giới hạn

Tác giả nêu:

- DAgger là thuật toán **interactive imitation learning**: cần hỏi expert lặp đi lặp lại trong suốt huấn luyện, mạnh hơn yêu cầu "chỉ có sẵn một tập demonstration cố định".
- Chặn lý thuyết cần một thuật toán **no-regret** hoặc **surrogate loss strongly convex**, giả định mạnh hơn các reduction tổng quát trước đó (Beygelzimer et al. 2005; Ross & Bagnell 2010) chỉ cần classification.
- Decode greedy một lượt được chọn có chủ đích để làm nổi bật lợi ích của DAgger, không nhằm tối đa độ chính xác tuyệt đối; multi-pass/beam-search có thể cạnh tranh hơn với SOTA.
- Hướng mở chính tác giả nêu: chiến lược decode tinh vi hơn greedy forward; học hàm cost cho planner qua Inverse Optimal Control (Abbeel & Ng 2004; Ratliff et al. 2006); mở rộng ý tưởng cost-to-go sang lý giải các phương pháp online trong RL nói chung.

Nhận xét:

- Cần một oracle **luôn trả lời đúng tại bất kỳ trạng thái nào** mà policy trung gian (kể cả rất tệ ở vòng đầu) ghé qua. Khả thi với planner mô phỏng hoặc nhãn có sẵn, nhưng tốn kém/nguy hiểm với con người/robot thật, đây là lý do interactive IL ít phổ biến ngoài domain được kiểm soát tốt.

- Mọi thí nghiệm dùng bộ học **tuyến tính** (ridge regression, linear SVM); chưa có bằng chứng (2010/2011) cho mạng nơ-ron sâu hay không gian hành động liên tục cao chiều.
- Hệ số u có thể $O(T)$ trong trường hợp xấu nhất, lúc đó chặn DAgger cũng suy biến về bậc hai; chặn tuyến tính mạnh chỉ đảm bảo khi $u = O(1)$, không tự động đúng với mọi MDP.
- Không có thí nghiệm đo trực tiếp **chi phí tổng số lần hỏi expert** so với lợi ích.

7. Liên hệ với các bài khác

- **Bài 18 (GAIL):** đi hướng ngược lại về mặt thực hành: học thuần từ một **tập cố định** quỹ đạo expert, không hỏi expert trong lúc huấn luyện, đổi lại cần rất nhiều **tương tác với môi trường** (rollout, qua TRPO). Discussion của GAIL dẫn thẳng DAgger: "các phương pháp tương tác với expert, như DAgger, có thể đơn giản hỏi expert hành động đó; [GAIL] khám phá ngẫu nhiên để tìm hành động đưa occupancy measure gần với expert hơn."
- **Bài 19 (Offline RL tutorial):** dùng lại chính xác Theorem 2.1 ("Behavioral cloning error bound", $\ell(\pi) \leq C\epsilon + H^2\epsilon$) và Theorem 3.2 ("DAgger error bound", $\ell(\pi) \leq C\epsilon + H\epsilon$) để giải thích **vì sao offline RL vốn khó**: không có khả năng tương tác thêm với môi trường đồng nghĩa phải chấp nhận nguy cơ chặn lỗi bậc hai theo horizon.
- **Bài 20 (CQL):** cố đạt tính chất gần-tuyến-tính của DAgger **mà không cần oracle tương tác**, bằng cách thận trọng với hành động/trạng thái ngoài phân phối dữ liệu, thay vì thu thập thêm dữ liệu on-policy.
- **Bài 21 (Decision Transformer):** cũng huấn luyện hoàn toàn offline (behavior cloning có điều kiện trên return-to-go), về nguyên tắc có thể chịu cùng rủi ro compounding error; paper DT không phân tích theo khung DAgger, một câu hỏi mở khi đọc song song hai bài.
- **distillation 28 (MiniLLM), 29 (GKD), 47 (ShortOPD):** "on-policy distillation" hiện đại chính là DAgger với **teacher làm oracle**: student tự sinh (tự đi trong môi trường), rồi hỏi "expert" (một lượt forward của teacher, rẻ, không phải con người) lấy nhãn/KL tại token/trạng thái chính student đã sinh, rồi huấn luyện lại. Khác DAgger gốc ở hai điểm: (1) oracle rẻ và luôn sẵn có, (2) giám sát thường là phân phối đầy đủ (logits/KL) chứ không phải một nhãn cứng $a^*(s)$. MiniLLM đo trực tiếp **exposure bias** (ExAccErr tăng theo độ dài sinh ở baseline offline, phẳng hơn ở MiniLLM on-policy), là phiên bản thực nghiệm của đúng hiện tượng lý thuyết bài DAgger chứng minh.

8. Ứng dụng thực tế (suy luận)

- **Lý do lý thuyết cho training on-policy trong post-training LLM/VLM:** nếu một mô hình sinh văn bản chỉ được train bằng teacher forcing (giống behavior cloning thuần), suy luận tự hồi quy tại test time tương đương thi hành policy trên phân phối trạng thái khác lúc train; theo Theorem 2.1, lỗi tích lũy lý thuyết có thể tăng tới $T^2\epsilon$ theo độ dài sinh T. Train (dù chỉ một phần) trên dữ liệu **tự sinh**, với oracle rẻ (mô hình phần thưởng/teacher logits) chấm điểm, đưa bài toán về gần chặn tuyến tính $T\epsilon$ của DAgger.
- **Tham số thực hành đáng mượn: lịch trình trộn β_i.** Biến thể không tham số (chỉ "mồi" bằng expert ở vòng đầu, sau đó hoàn toàn on-policy) thường tốt nhất; khớp lý do nhiều recipe on-policy-KD (GKD, MiniLLM) warm-start bằng SFT trước khi chuyển sang rollout của chính student.
- **Nguy cơ:** chặn DAgger chỉ đúng nếu oracle đúng **tại mọi trạng thái** policy trung gian ghé qua. Nếu rollout sớm của student quá hỗn loạn, teacher cũng cho tín hiệu kém ở vùng đó, đây là lý do MiniLLM cần *teacher-mixed sampling* thay vì on-policy thuần từ đầu.

9. Thí nghiệm gợi ý

1. Tái hiện tinh thần thí nghiệm handwriting ở quy mô nhỏ: huấn luyện một mô hình giải mã token nhỏ bằng (a) teacher forcing thuần và (b) lịch trình kiểu DAgger (giải mã bằng chính mô hình, hỏi nhãn đúng, gộp dữ liệu, huấn luyện lại); đo accuracy theo độ dài chuỗi sinh T, kiểm tra (a) có tăng lỗi nhanh hơn (b) theo T hay không.

2. Thực nghiệm lại phép tính "tự tính" ở mục 3.4 trên một tác vụ giải mã tổng hợp: đo thêm-chi-phí thực nghiệm theo T cho cả hai cách huấn luyện, so với đường $T^2\epsilon$ và $T\epsilon$ lý thuyết.
3. Sweep tham số trộn (ví dụ α teacher-mixed sampling của MiniLLM, tương tự β_i) trên một thiết lập on-policy-distillation nhỏ, kiểm tra lịch trình "on-policy hoàn toàn sau một vòng mồi" có cạnh tranh được với lịch trình giảm dần như quan sát trên Mario hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Cost-to-go toàn cục	$J(\pi) = T\,\mathbb{E}_{s\sim d_\pi}[C_\pi(s)]$	d_π = phân phối trạng thái trung bình T bước
Behavior cloning (Thm 2.1)	$J(\pi) \le J(\pi^*) + T^2\epsilon$	ϵ đo dưới d_{π^*}; chặt
Forward training	$J(\pi) \le J(\pi^*) + uT\epsilon$	u = hệ số hồi phục, có thể $O(T)$; cần train T policy tuần tự
SMILe	$\pi_n = \pi_{n-1} + \alpha(1-\alpha)^{n-1}(\hat\pi_n - \pi_0)$	$\alpha = O(1/T^2)$, $N = O(T^2 \log T)$; policy cuối ngẫu nhiên
DAgger update	$\pi_i = \beta_i\pi^* + (1-\beta_i)\hat\pi_i$; $D \leftarrow D \cup D_i$; train lại trên D	$\bar\beta_N \to 0$; $\beta_i = \mathbb{1}(i=1)$ thường tốt nhất
No-regret (FTL)	$\frac{1}{N}\sum \ell_i(\pi_i) - \min_\pi \frac{1}{N}\sum \ell_i(\pi) \le \gamma_N \to 0$	mỗi vòng = một "ví dụ" online learning
Chặn chính DAgger (Thm 3.2)	$J(\hat\pi) \le J(\pi^*) + uT\epsilon_N + O(1)$, $N = \tilde O(uT)$	tuyến tính theo T khi $u = O(1)$
Mẫu hữu hạn (Thm 3.3/3.4)	$N = O(T^2 \log(1/\delta))$, $m = O(1)$ (hoặc $N = \tilde O(T\log(1/\delta))$ với strong convexity)	Azuma-Hoeffding

11. Kết luận ngắn

Paper chỉ ra rằng behavior cloning thuần (học trên phân phối trạng thái của expert) có chặn lỗi *bậc hai* theo horizon do covariate shift tuần tự, và đề xuất DAgger: lặp lại việc tự thi hành policy hiện tại, hỏi expert hành động đúng tại các trạng thái gặp phải, gộp dữ liệu, rồi huấn luyện lại trên toàn bộ dữ liệu đã gộp. Nhìn dưới góc độ online learning (Follow-The-Leader), cách làm này đạt chặn lỗi *tuyến tính* theo horizon, được xác nhận thực nghiệm trên ba tác vụ (lái xe, Mario, nhận dạng chữ viết tay) với các bộ học tuyến tính đơn giản. Ý tưởng cốt lõi, huấn luyện trên dữ liệu do chính policy/mô hình tạo ra thay vì chỉ trên dữ liệu "đúng tuyệt đối", là tổ tiên trực tiếp của các phương pháp on-policy distillation hiện đại cho LLM.

Link gốc: https://arxiv.org/abs/1011.0686

Bài 18

Generative Adversarial Imitation Learning (2016)

Tác giả: Jonathan Ho, Stefano Ermon (Stanford University).
Venue / arXiv: arXiv:1606.03476 (v1, 10/06/2016); văn bản không nêu venue hội nghị (được biết rộng rãi là NeurIPS 2016, nhưng không ghi trong bản PDF này).
Nguồn: https://arxiv.org/abs/1606.03476
Chủ đề: Imitation learning không cần hỏi expert lúc train và không cần hàm phần thưởng; dẫn ra từ inverse reinforcement learning (IRL) một bài toán khớp **occupancy measure**, rồi chọn một regularizer đặc biệt khiến bài toán đó trở thành huấn luyện kiểu **GAN**: discriminator đóng vai trò hàm chi phí, policy cập nhật bằng **TRPO**.
Vị trí trong lộ trình: Đọc sau bài 04 (TRPO, dùng trực tiếp làm bước cập nhật policy) và bài 17 (DAgger, đối lập về cách tiếp cận: hỏi expert liên tục khi huấn luyện so với không hỏi expert nhưng cần rất nhiều tương tác môi trường). Tiếp tục giai đoạn 5 "Imitation learning & offline RL"; đọc trước bài 19 (Offline RL tutorial).
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Hai hướng kinh điển của imitation learning từ một tập quỹ đạo expert cố định (không hỏi thêm expert, không có reward):

- **Behavioral cloning:** học giám sát $(s, a) \to a$. Đơn giản nhưng cần rất nhiều dữ liệu do **compounding error** gây ra bởi covariate shift (đúng vấn đề bài 17 - DAgger - phân tích: lỗi tích lũy theo T^2 nếu chỉ học trên phân phối trạng thái của expert).
- **Inverse reinforcement learning (IRL):** học một hàm chi phí $c(s, a)$ khiến expert là tối ưu duy nhất, rồi chạy RL trên hàm chi phí đó để lấy lại policy. Vì tối ưu hóa trên toàn bộ quỹ đạo (không phải quyết định từng bước riêng lẻ), IRL không bị compounding error. Nhược điểm: phần lớn thuật toán IRL cần chạy RL ở vòng lặp trong (inner loop), rất tốn, và bản thân việc học một hàm chi phí không trực tiếp cho hành động.

Maximum causal entropy IRL (Ziebart et al. 2008, 2010): với Π = tập policy ngẫu nhiên tĩnh trên không gian trạng thái/hành động hữu hạn S, A, γ-discounted, $H(\pi) = \mathbb{E}_\pi[-\log \pi(a|s)]$ (causal entropy):

$$\max_{c \in C} \left(\min_{\pi \in \Pi} -H(\pi) + \mathbb{E}_\pi[c(s,a)] \right) - \mathbb{E}_{\pi_E}[c(s,a)]$$

tìm c gán chi phí thấp cho expert, cao cho policy khác; rồi $RL(c) = \arg\min_\pi -H(\pi) + \mathbb{E}_\pi[c(s,a)]$ lấy lại policy max-entropy tối thiểu chi phí đó.

Occupancy measure: $\rho_\pi(s,a) = \pi(a|s) \sum_{t=0}^{\infty} \gamma^t P(s_t = s|\pi)$, phân phối các cặp trạng thái-hành động mà π ghé qua khi thi hành trong môi trường; $\mathbb{E}_\pi[c(s,a)] = \sum_{s,a} \rho_\pi(s,a) c(s,a)$ với mọi c. Tập occupancy measure hợp lệ $D = \{\rho_\pi : \pi \in \Pi\}$ là một tập **affine lồi** (ràng buộc Bellman-flow), và có **song ánh** giữa Π và D (Proposition 3.1, Syed et al. 2008): mỗi $\rho \in D$ tương ứng đúng một policy $\pi_\rho(a|s) = \rho(s,a) / \sum_{a'} \rho(s,a')$.

Dùng $IRL_\psi(\pi_E) = \arg\max_{c \in \mathbb{R}^{S \times A}} -\psi(c) + (\min_\pi -H(\pi) + \mathbb{E}_\pi[c] - \mathbb{E}_{\pi_E}[c])$ (ψ = regularizer lồi, đóng, chính đáng cho c) ký hiệu phiên bản IRL tổng quát có regularizer, câu hỏi paper đặt ra: **policy $RL \circ IRL_\psi(\pi_E)$ (chạy RL trên cost học từ IRL) thực chất là gì?**

2. Ý tưởng chính

Proposition 3.2 (đặc trưng hóa):

$$\boxed{RL \circ IRL_\psi(\pi_E) = \arg\min_{\pi \in \Pi} \; -H(\pi) + \psi^*(\rho_\pi - \rho_{\pi_E})}$$

với ψ^* là **liên hợp lồi (convex conjugate)** của ψ. Nói cách khác: chạy RL sau IRL, về bản chất, chỉ là tìm một policy có **occupancy measure gần với occupancy measure của expert**, theo thước đo ψ^*. IRL là bài toán đối ngẫu (dual) của một bài toán khớp occupancy measure; c học được là biến đối ngẫu, policy cuối là nghiệm sơ cấp (primal) khôi phục từ đối ngẫu.

Trường hợp đặc biệt $\psi =$ hằng số: khớp occupancy measure **chính xác** (Corollary 3.2.1, $\rho_{\tilde{\pi}} = \rho_{\pi_E}$), nhưng không khả thi thực tế (cần liệt kê toàn bộ $S \times A$, không nhận function approximation). Các thuật toán apprenticeship learning cũ (feature expectation matching, MWAL, LPAL) tương ứng $\psi = \delta_C$ (hàm chỉ số của một họ cost tuyến tính C): scale được lên môi trường lớn, nhưng **không khớp được expert chính xác** nếu C không đủ biểu đạt.

Đóng góp chính: chọn một regularizer ψ_{GA} mới sao cho $\psi^*_{GA}(\rho_\pi - \rho_{\pi_E})$ chính là loss tối ưu của bài toán **phân loại nhị phân** phân biệt cặp (s, a) từ π với từ π_E, và (sai một hằng số) bằng **Jensen-Shannon divergence** giữa hai occupancy measure, một khoảng cách (metric) thật sự. Tối thiểu JS-divergence này (cộng entropy regularizer) chính là khung của **Generative Adversarial Networks**: policy π_θ đóng vai "generator" (sinh ra occupancy measure), một discriminator D_w cố phân biệt (s, a) của π_θ với của π_E; discriminator thắng ở đâu thì ở đó π_θ còn khác expert, nên log D_w dùng làm "chi phí" cục bộ để cập nhật π_θ (bằng TRPO, không phải bằng gradient qua D như GAN ảnh thường).

3. Phương pháp chi tiết

3.1 Từ IRL sang khớp occupancy measure (gần đúng, khả thi)

Thay bài toán khớp chính xác (Eq. 7, chỉ khả thi với ψ hằng số) bằng dạng nới lơi (Eq. 8):

$$\min_\pi \; d_\psi(\rho_\pi, \rho_{\pi_E}) - H(\pi), \qquad d_\psi(\rho_\pi, \rho_E) := \psi^*(\rho_\pi - \rho_E)$$

phạt "mềm" độ lệch occupancy measure thay vì ép bằng tuyệt đối. Với $\psi = \delta_C$ (C = họ cost tuyến tính theo đặc trưng $f(s, a)$, ví dụ $C_{linear} = \{\sum_i w_i f_i : \|w\|_2 \le 1\}$ của Abbeel & Ng, hay $C_{convex} = \{\sum_i w_i f_i : \sum w_i = 1, w_i \ge 0\}$ của Syed et al.), (9) trở thành đúng mục tiêu apprenticeship learning cổ điển $\min_\pi \max_{c \in C} \mathbb{E}_\pi[c] - \mathbb{E}_{\pi_E}[c]$ ($= \|\mathbb{E}_\pi[f] - \mathbb{E}_{\pi_E}[f]\|_2$ với C_{linear}: khớp kỳ vọng đặc trưng). Ho, Gupta & Ermon (2016) đã có công thức policy gradient cho mục tiêu này:

$$\nabla_\theta \max_{c \in C} \mathbb{E}_{\pi_\theta}[c] - \mathbb{E}_{\pi_E}[c] = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s)\, Q_{c^*}(s, a)]$$

(c^* = cost tối ưu hiện tại trong C) rồi lấy bước **TRPO** với cost đó. Nhược điểm: C tuyến tính hiếm khi đủ biểu đạt để mô tả đúng hành vi expert phức tạp.

3.2 Regularizer kiểu GAN

$$\psi_{GA}(c) \triangleq \begin{cases} \mathbb{E}_{\pi_E}[g(c(s, a))] & c < 0 \\ +\infty & \text{khác} \end{cases}, \qquad g(x) = \begin{cases} -x - \log(1 - e^x) & x < 0 \\ +\infty & \text{khác} \end{cases}$$

Tính chất chính (Corollary A.1.1):

$$\psi^*_{GA}(\rho_\pi - \rho_{\pi_E}) = \max_{D:S\times A\to(0,1)} \mathbb{E}_\pi[\log D(s,a)] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))]$$

chính là **negative log loss tối ưu** của bài toán phân loại (s,a) từ π (nhãn 1) so với từ π_E (nhãn 0) bằng discriminator D. Giá trị tối ưu này (lệch một hằng số) bằng Jensen-Shannon divergence $D_{JS}(\rho_\pi, \rho_{\pi_E})$. Đưa entropy $H(\pi)$ vào làm regularizer với hệ số $\lambda \geq 0$:

$$\boxed{\min_\pi \ \psi^*_{GA}(\rho_\pi - \rho_{\pi_E}) - \lambda H(\pi) = D_{JS}(\rho_\pi, \rho_{\pi_E}) - \lambda H(\pi)}$$

So với apprenticeship learning tuyến tính, ψ_{GA} cho phép c là **hàm bất kỳ** (chỉ cần âm khắp nơi), không bị nhét vào một không gian con hữu hạn chiều, nên về lý thuyết khớp được occupancy measure chính xác.

3.3 Thuật toán (Algorithm 1)

Tìm điểm yên ngựa (saddle point) (π, D) của $\mathbb{E}_\pi[\log D(s,a)] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))] - \lambda H(\pi)$: tham số hóa π_θ và D_w, luân phiên

1. **Bước discriminator** (Adam, tăng mục tiêu theo w): $\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log D_w(s,a)] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))]$.
2. **Bước policy** (TRPO, giảm mục tiêu theo θ, cost $= \log D_{w_{i+1}}(s,a)$, $\lambda\nabla_\theta H$ trừ thêm cho entropy): $\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda\nabla_\theta H(\pi_\theta)$, $Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log D_{w_{i+1}}(s,a) \mid s_0 = \bar{s}, a_0 = \bar{a}]$.

```
Input: quỹ đạo expert tau_E ~ pi_E, tham số khởi tạo theta_0, w_0
for i = 0, 1, 2, ...:
    Lấy mẫu quỹ đạo tau_i ~ pi_theta_i
    # cập nhật discriminator (tăng khả năng phân biệt pi_theta vs pi_E)
    w_{i+1} <- w_i + Adam_step( E_{tau_i}[grad_w log D_w] + E_{tau_E}[grad_w log(1 -
    ↪  D_w)] )
    # cập nhật policy bằng TRPO với cost = log D(s,a)  (discriminator đóng vai hàm chi
    ↪  phí)
    Q(s,a) = E_{tau_i}[ log D_{w_{i+1}}(s,a) | s_0=s, a_0=a ]
    theta_{i+1} <- TRPO_step( E_{tau_i}[grad_theta log pi_theta(a|s) * Q(s,a)] - lambda
    ↪  * grad_theta H(pi_theta) )
```

TRPO đóng vai trò kiểm soát bước policy (natural gradient, ràng buộc KL trung bình trên các trạng thái mẫu) để chịu được nhiễu cao của gradient ước lượng từ $\log D$, giống vai trò nó đóng trong thuật toán apprenticeship learning của Ho et al. (2016) ở mục 3.1.

Cách đọc hình: policy chạy trong môi trường để sinh các cặp trạng thái và hành động, discriminator so chúng với cặp của chuyên gia. Đầu ra của discriminator được dùng làm chi phí cho bước cập nhật TRPO của policy, còn discriminator được cập nhật bằng Adam. Hai bước luân phiên cho đến khi hai phân bố gần nhau.

4. Thí nghiệm và kết quả chính

9 tác vụ OpenAI Gym: classic control (Cartpole, Acrobot, Mountain Car) và MuJoCo (Reacher, HalfCheetah, Hopper, Walker, Ant, Humanoid). Expert = policy đã train bằng TRPO trên true cost function của từng tác vụ. Mạng cho cả policy và discriminator: 2 lớp ẩn 100 unit, tanh, khởi tạo ngẫu nhiên. Baseline: **Behavioral cloning**, **FEM** (Ho et al. 2016 với C_{linear}), **GTAL** (Ho et al. 2016 với C_{convex}). Cùng lượng tương tác môi trường cho FEM/GTAL/GAIL; GAE ($\gamma = 0.995, \lambda = 0.97$) dùng để giảm phương sai cho cả ba.

Task	Random	Expert	BC (ít traj)	FEM	GTAL	GAIL
HalfCheetah-v1 (4 traj)	-282.4	4463.5	-493.6	734.0	1008.1	**4515.7**

Task	Random	Expert	BC (ít traj)	FEM	GTAL	GAIL
Hopper-v1 (4 traj)	14.5	3571.4	50.6	3572.0	3065.2	**3614.2**
Walker-v1 (4 traj)	0.6	6717.1	32.2	3648.2	4945.9	**4878.0**
Ant-v1 (4 traj)	-69.7	4228.4	1611.8	-2052.5	-5743.8	**3186.8**
Humanoid-v1 (80 traj)	122.9	9575.4	1397.1	5093.1	5096.4	**10200.7**

(trích Table 3; số liệu trung bình trên 50 quỹ đạo, GAIL dùng $\lambda = 0$ trừ Reacher.) Trên classic control, GAIL luôn thắng BC/FEM/GTAL, cả với rất ít dữ liệu expert (1-10 quỹ đạo). Trên Reacher, BC hiệu quả hơn về dữ liệu ở vài trajectory; thêm entropy regularizer ($\lambda = 10^{-3}$) cải thiện có ý nghĩa thống kê (Wilcoxon một phía, $p = 0.05$) ở chế độ 4 trajectory. Trên các tác vụ MuJoCo cao chiều còn lại: GAIL gần như luôn đạt **≥70% hiệu năng expert** ở mọi kích thước dữ liệu đã thử, FEM/GTAL thất bại nặng trên Ant (tệ hơn cả chính sách ngẫu nhiên), BC cần rất nhiều dữ liệu để tạm ổn trên HalfCheetah/Hopper/Walker/Ant nhưng không vượt quá ~60% trên Humanoid, nơi GAIL đạt **chính xác hiệu năng expert** ở mọi kích thước dữ liệu đã thử.

Cách đọc hình: mỗi nhóm cột là một tác vụ, đường nét đứt là điểm của chuyên gia. GAIL gần hoặc vượt chuyên gia ở phần lớn tác vụ, còn BC thường kém hẳn. Số liệu lấy từ Bảng 3 trong mục 4 của ghi chú, cột random không được vẽ.

5. Phân tích / ablation

- **Entropy regularization trên Reacher:** $\lambda = 0$ kém BC khi ít dữ liệu; $\lambda = 10^{-3}$ cải thiện có ý nghĩa thống kê; $\lambda = 10^{-2}$ không khá hơn rõ (bài báo không cho thấy một lựa chọn λ phổ quát tốt cho mọi tác vụ, chỉ dùng $\lambda = 0$ cho các tác vụ khác).
- **Hiệu quả dữ liệu expert vs hiệu quả tương tác môi trường:** GAIL rất tiết kiệm **dữ liệu expert** (vài quỹ đạo là đủ cho nhiều tác vụ), nhưng **không tiết kiệm tương tác với môi trường**: số mẫu rollout cần để ước lượng gradient (18) cùng cấp với số mẫu TRPO cần để tự train expert từ đầu bằng reward thật.
- Tác giả so sánh với **guided cost learning** (Finn et al. 2016, dựa trên guided policy search): cũng luân phiên tối ưu policy và "fit cost" (ở đây là discriminator), nhưng guided cost learning là model-based (giả định dynamics tuyến tính theo thời gian, địa phương) nên hiệu quả dữ liệu hơn với chi phí giả định mạnh hơn về mô hình.

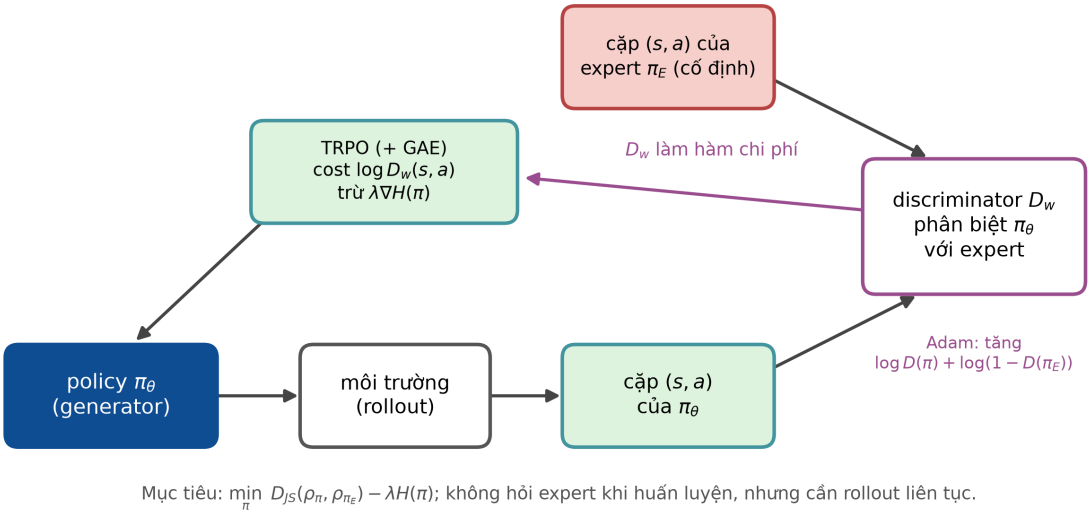
6. Giới hạn

Tác giả nêu:

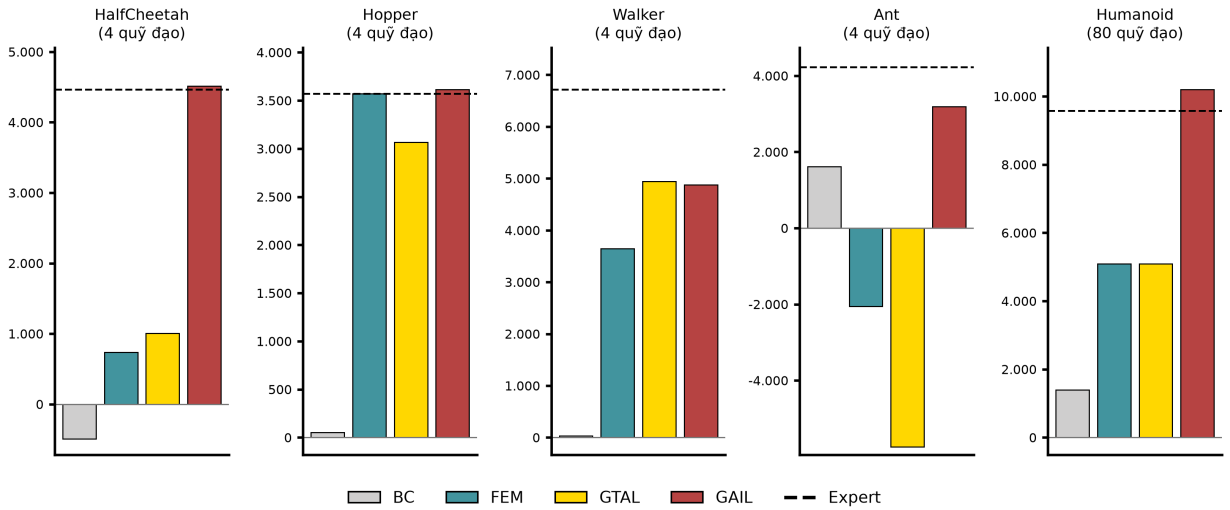
- Phương pháp **model-free**, nên cần nhiều tương tác môi trường hơn các phương pháp model-based (ví dụ guided cost learning).
- GAIL **khám phá ngẫu nhiên** để tìm hành động đưa occupancy measure gần expert hơn, trong khi các phương pháp tương tác expert (DAgger) có thể **hỏi trực tiếp** expert hành động đúng; tác giả dự đoán phương pháp kết hợp tốt mô hình môi trường + tương tác expert sẽ thắng về độ phức tạp mẫu của cả hai loại dữ liệu.
- Hướng mở tác giả nêu: khởi tạo policy bằng behavioral cloning trước (không cần tương tác môi trường) để tăng tốc huấn luyện.

Nhận xét:

- Cần truy cập môi trường **trong suốt** huấn luyện để lấy rollout của π_θ (không phải một phương pháp offline/batch như bài 19-20).
- Phụ thuộc TRPO (on-policy), vốn tốn mẫu và có thể không ổn định với discriminator thay đổi liên tục (reward "di động" theo discriminator, không cố định như RL thường).



Hình 48: Vòng lặp GAIL: policy đóng vai bộ sinh, discriminator phân biệt với dữ liệu chuyên gia (minh hoạ, tự tính)



Hình 49: Điểm của BC, FEM, GTAL và GAIL so với chuyên gia trên năm tác vụ điều khiển (vẽ lại từ số liệu trong ghi chú)

- Mọi thí nghiệm dùng mạng nhỏ (2 lớp, 100 unit) và hành động liên tục; không có thí nghiệm hành động rời rạc cao chiều hoặc ảnh thô (pixel).
- Khớp occupancy measure chính xác chỉ đúng về lý thuyết ở giới hạn không có sai số mẫu/hàm xấp xỉ; không có phân tích sai số hữu hạn mẫu như bài 17 (DAgger) cung cấp cho imitation learning kiểu tương tác.

7. Liên hệ với các bài khác

- **Bài 04 (TRPO):** là bước cập nhật policy trong Algorithm 1; GAIL gần như là "TRPO + cost học bằng discriminator" thay cho "TRPO + true reward".
- **Bài 05 (GAE):** dùng trong thực nghiệm để giảm phương sai advantage khi tính gradient policy, giống vai trò GAE trong huấn luyện RL on-policy thông thường.
- **Bài 17 (DAgger):** đối lập trực diện về giả định bài toán. DAgger cần hỏi expert tại mọi vòng huấn luyện nhưng (gần như) không cần rollout dài trong môi trường để ước lượng gradient; GAIL không hỏi expert khi huấn luyện (chỉ dùng một tập quỹ đạo cố định) nhưng cần rất nhiều tương tác môi trường qua TRPO. Chính Discussion của GAIL dẫn thẳng DAgger khi so sánh hai hướng.
- **Bài 19 (Offline RL tutorial):** GAIL không phải offline RL (vẫn cần rollout on-policy liên tục của π_θ trong môi trường); hữu ích để thấy ranh giới giữa "imitation learning có tương tác môi trường" (GAIL) và "RL hoàn toàn không tương tác" (offline RL, bài 19-20).
- **distillation 28 (MiniLLM), 29 (GKD):** cùng mô-típ "khớp phân phối giữa rollout của policy/student với một đối tượng tham chiếu (expert/teacher) bằng policy gradient on-policy", nhưng MiniLLM dùng trực tiếp reverse-KL với teacher chấm điểm (không cần discriminator huấn luyện riêng), còn GAIL học một discriminator để xấp xỉ khoảng cách phân phối khi không có mô hình xác suất tường minh của expert.

8. Ứng dụng thực tế (suy luận)

- **Discriminator-làm-reward** là tiền thân khái niệm của nhiều kiến trúc RLHF hiện đại (mô hình phần thưởng huấn luyện riêng, dùng làm tín hiệu cho policy gradient/PPO): cả hai đều học một mạng phụ để chấm "giống/không giống hành vi mong muốn", rồi dùng chính mạng đó làm cost/reward cho vòng lặp RL. Khác biệt: RLHF mô hình phần thưởng học từ **so sánh ưu tiên của người**, GAIL discriminator học từ **phân loại nhị phân policy-vs-expert**.
- Bài học thực hành đáng giữ: khi không có reward tường minh nhưng có nhiều quỹ đạo "đúng" mẫu, một discriminator phân biệt rollout hiện tại với mẫu tốt có thể đóng vai trò reward tạm, miễn chấp nhận chi phí train on-policy không nhỏ.

9. Thí nghiệm gợi ý

1. Tái hiện Cartpole/Acrobot/Mountain Car với số quỹ đạo expert giảm dần (1, 4, 7, 10) để kiểm chứng tuyên bố "near-expert chỉ với rất ít dữ liệu" của GAIL.
2. So sánh trực tiếp FEM/GTAL (cost tuyến tính) với GAIL (discriminator phi tuyến) trên một môi trường có hành vi expert đa mode, để quan sát khi nào occupancy-matching tuyến tính thất bại rõ (giống hiện tượng Ant trong bài báo).
3. Sweep λ (entropy regularization) trên một tác vụ goal-conditioned giống Reacher, đo độ nhạy của GAIL với ít dữ liệu expert.
4. Đo số lượng bước tương tác môi trường cần để GAIL đạt hiệu năng expert so với số bước TRPO cần để huấn luyện chính expert đó từ reward thật, kiểm chứng tuyên bố "không tiết kiệm tương tác môi trường" ở mục 5.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Occupancy measure	$\rho_\pi(s,a) = \pi(a\|s)\sum_t \gamma^t P(s_t = s\|\pi)$	song ánh với Π
IRL có regularizer	$IRL_\psi(\pi_E) = \arg\max_c -\psi(c) + \min_\pi(-H(\pi) + \mathbb{E}_\pi[c] - \mathbb{E}_{\pi_E}[c])$	ψ lồi, đóng
Đặc trưng hóa chính	$RL \circ IRL_\psi(\pi_E) = \arg\min_\pi -H(\pi) + \psi^*(\rho_\pi - \rho_{\pi_E})$	Proposition 3.2
ψ hằng số	$\rho_{\tilde{\pi}} = \rho_{\pi_E}$ (khớp chính xác)	không khả thi thực tế
$\psi = \delta_C$ (tuyến tính)	apprenticeship learning (FEM, MWAL/LPAL)	scale được, không khớp chính xác
ψ_{GA}	$\psi^*_{GA}(\rho_\pi - \rho_{\pi_E}) = \max_D \mathbb{E}_\pi[\log D] + \mathbb{E}_{\pi_E}[\log(1-D)]$	= loss discriminator tối ưu
Mục tiêu GAIL	$\min_\pi D_{JS}(\rho_\pi, \rho_{\pi_E}) - \lambda H(\pi)$	metric thật, khớp chính xác về lý thuyết
Cập nhật	discriminator: Adam trên w; policy: TRPO với cost $\log D_w$	Algorithm 1

11. Kết luận ngắn

GAIL cho thấy chạy RL sau IRL, về bản chất, là đi tìm một policy có occupancy measure gần với occupancy measure của expert; chọn đúng regularizer cho bài toán IRL biến việc "gần" đó thành Jensen-Shannon divergence, và việc tối thiểu hóa divergence này tự nhiên trở thành một trò chơi kiểu GAN giữa policy (generator) và một discriminator đóng vai hàm chi phí, cập nhật bằng TRPO. Trên 9 tác vụ điều khiển, GAIL vượt rõ các phương pháp apprenticeship learning tuyến tính và behavioral cloning về hiệu quả dữ liệu expert, đặc biệt ở không gian trạng thái cao chiều (Humanoid), đổi lại cần lượng tương tác môi trường tương đương với việc tự train expert bằng RL từ đầu.

Link gốc: https://arxiv.org/abs/1606.03476

Bài 19

Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems (2020)

Phạm vi ghi chú: Đây là một tutorial/survey dài (33 trang nội dung). Ghi chú này tập trung vào: định nghĩa bài toán offline RL và distributional shift (mục 2 của bài báo), các họ phương pháp importance sampling (mục 3, ở mức công thức chính, không đi sâu mọi biến thể marginalized importance sampling), các họ phương pháp kiểm soát distributional shift trong dynamic programming: policy constraint và conservative/uncertainty-based value function (mục 4), offline model-based RL (mục 5), và phần thảo luận/open problems (mục 7). Phần ứng dụng theo lĩnh vực cụ thể (robotics, y tế, lái xe tự động, hội thoại, mục 6.2-6.6) và value-function tuyến tính cổ điển (LSTD-Q, LSPI, mục 4.1) chỉ được lược tóm.
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

Tác giả: Sergey Levine (UC Berkeley, Google Research/Brain Team), Aviral Kumar (UC Berkeley), George Tucker (Google Research/Brain Team), Justin Fu (UC Berkeley).
Venue / arXiv: arXiv:2005.01643 (v3, 01/11/2020); bài báo không nêu venue (bài tutorial, không ghi hội nghị/journal cụ thể trong văn bản).
Nguồn: https://arxiv.org/abs/2005.01643
Chủ đề: Tổng quan toàn diện về offline reinforcement learning (còn gọi batch RL): học policy chỉ từ một tập dữ liệu cố định, không tương tác thêm với môi trường. Nguyên nhân cốt lõi khiến offline RL khó là **distributional shift**; các họ giải pháp (importance sampling, policy constraint, conservative/uncertainty estimation, model-based) đều xoay quanh việc kiểm soát nó.
Vị trí trong lộ trình: Đọc sau bài 17 (DAgger, tutorial này tái dùng trực tiếp Theorem 2.1/3.2 của DAgger làm ví dụ mở đầu) và bài 18 (GAIL). Đặt nền cho bài 20 (CQL, bản đầy đủ của mục 4.5 ở đây) và bài 21 (Decision Transformer, một lối thoát hoàn toàn khỏi dynamic programming).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Ba chế độ học RL (Figure 1 của bài báo): (a) **online RL**: π_k cập nhật bằng dữ liệu rollout do chính π_k sinh ra ngay lúc đó; (b) **off-policy RL**: dữ liệu dồn vào một buffer D chứa mẫu từ $\pi_0, \pi_1, \ldots, \pi_k$, nhưng vẫn tiếp tục thu thêm dữ liệu mới mỗi vòng; (c) **offline RL**: D được thu thập **một lần** bởi một behavior policy π_β (có thể không rõ là gì), không đổi trong suốt quá trình train, và quá trình train **không tương tác với MDP** chút nào; policy chỉ được triển khai (deploy) sau khi huấn luyện xong.

MDP: $M = (S, A, T, d_0, r, \gamma)$ với $T(s_{t+1}|s_t, a_t)$ là dynamics, d_0 phân phối trạng thái khởi đầu, $\gamma \in (0, 1]$. Trajectory distribution $p_\pi(\tau) = d_0(s_0)\prod_t \pi(a_t|s_t)T(s_{t+1}|s_t, a_t)$, mục tiêu $J(\pi) = \mathbb{E}_{\tau\sim p_\pi}[\sum_t \gamma^t r(s_t, a_t)]$. $d_\pi(s)$: tần suất ghé trạng thái s trung bình theo thời gian dưới π.

Bài toán offline RL: cho $D = \{(s_t^i, a_t^i, s_{t+1}^i, r_t^i)\}$ với $s \sim d_{\pi_\beta}(s)$, $a \sim \pi_\beta(a|s)$ cố định, học policy π tối đa $J(\pi)$ mà **không** được thu thêm transition nào. Khác "off-policy" (vẫn cho thu thêm dữ liệu online) và "batch RL" (thuật ngữ cũ, dễ nhầm với "một batch gradient"); paper chọn gọi là "offline RL" để tránh nhầm lẫn.

Tại sao offline RL khó: paper chỉ ra hai lý do. Thứ nhất (hiển nhiên, không bàn sâu): không thể cải thiện exploration, nếu D không có transition tốt thì không thể khám phá ra. Thứ hai (quan trọng hơn): offline RL về bản chất là trả lời **counterfactual query** ("điều gì xảy ra nếu hành động khác với dữ liệu"), phá vỡ giả định i.i.d. của học máy giám sát thông thường. Nguyên nhân gốc của khó khăn này được gọi là **distributional**

shift: bộ xấp xỉ hàm (policy, value, hoặc model) được huấn luyện trên một phân phối nhưng bị đánh giá trên phân phối khác, do cả sự thay đổi trạng thái ghé qua và do chính hành vi tối đa hoá kỳ vọng return.

Ví dụ minh hoạ bằng behavioral cloning (dựa trên Ross et al., 2011, chính là bài 17 DAgger): giả sử có nhãn hành động tối ưu $a^\star$ tại mọi $s \in D$, định nghĩa số lỗi kỳ vọng $\ell(\pi) = \mathbb{E}_{p_\pi(\tau)}[\sum_t \delta(a_t \neq a_t^\star)]$.

Theorem 2.1 (offline/BC): $\ell(\pi) \leq C + H^2\epsilon$	Theorem 2.2 (có thu thêm dữ liệu on-policy): $\ell(\pi) \leq C + H\epsilon$

(với ϵ là generalization error trên phân phối train, H là horizon; đây đúng là Theorem 2.1 và Theorem 3.2 của bài 17, chỉ đổi tên hằng số $T \to H$). Ngay cả khi có nhãn hành động tối ưu hoàn hảo, chỉ train bằng empirical risk minimization trên dữ liệu **tĩnh** (offline) cho cận lỗi **bậc hai** theo horizon, còn nếu được thu thêm dữ liệu on-policy (như DAgger) thì cận chỉ **tuyến tính**. Lý do: policy học offline có thể đi vào trạng thái ngoài phân phối train ($d_\pi(s)$ khác xa $d_{\pi_\beta}(s)$), nơi ϵ không còn đảm bảo gì, sai số dồn tích theo thời gian còn lại của episode.

2. Ý tưởng chính

Toàn bộ các họ phương pháp offline RL trong tutorial này đều có thể hiểu như những cách khác nhau để **không bao giờ đánh giá policy/value function/model tại các điểm nằm xa phân phối dữ liệu**. Importance sampling tái cân trọng số dữ liệu mà không làm thay đổi bản thân phân phối học (đổi lại: phương sai cao). Policy constraint và conservative value function trực tiếp ràng buộc policy/Q-function ở gần support của π_β. Model-based offline RL áp đặt một loại ràng buộc khác: phạt việc "đi" vào vùng trạng thái mà mô hình động lực học không còn tin cậy. Khác biệt chính giữa các nhóm là **cách đo "gần"** (f-divergence, IPM, support, uncertainty) và **nơi áp ràng buộc** (lên policy, lên Q-function, hay lên mô hình MDP).

3. Phương pháp chi tiết

3.1 Off-policy evaluation bằng importance sampling

Ước lượng không chệch cơ bản (Precup, 2000):

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\beta}\left[\frac{\pi_\theta(\tau)}{\pi_\beta(\tau)}\sum_{t=0}^{H}\gamma^t r(s,a)\right] \approx \sum_{i=1}^{n} w_H^i \sum_{t=0}^{H}\gamma^t r_t^i, \qquad w_t^i = \prod_{t'=0}^{t}\frac{\pi_\theta(a_{t'}^i|s_{t'}^i)}{\pi_\beta(a_{t'}^i|s_{t'}^i)}$$

Vấn đề: phương sai rất cao (có thể vô hạn khi $H \to \infty$) do tích các importance weight. Các biến thể giảm phương sai: **weighted IS** (tự chuẩn hoá trọng số, chệch nhưng phương sai thấp hơn, vẫn consistent); **per-decision IS** (bỏ trọng số của các bước tương lai vì r_t không phụ thuộc $s_{t'}, a_{t'}$ với $t' > t$); **doubly robust estimator** (Jiang & Li 2015; Thomas & Brunskill 2016, dùng ước lượng mô hình $\hat{Q}^{\pi_\theta}$ làm control variate, không chệch nếu π_β đã biết hoặc mô hình đúng):

$$J(\pi_\theta) \approx \sum_{i,t}\gamma^t\left[w_t^i r_t^i - (\hat{Q}^{\pi_\theta}(s_t,a_t) - w_{t-1}^i \mathbb{E}_{a\sim\pi_\theta}[\hat{Q}^{\pi_\theta}(s_t,a)])\right]$$

3.2 Off-policy policy gradient và marginalized importance sampling

Cùng ý tưởng áp cho policy gradient (thay J bằng $\nabla_\theta J$ trong công thức IS), cũng gặp vấn đề phương sai do tích trọng số theo thời gian. Một cách giảm bias/variance khác là dùng trực tiếp $d_{\pi_\beta}(s)$ thay cho $d_\pi(s)$ (loại bỏ hoàn toàn IS theo chuỗi, đổi lại là gradient **chệch**): $\nabla_\theta J_{\pi_\beta}(\pi_\theta) \approx \mathbb{E}_{s\sim d_{\pi_\beta}, a\sim\pi_\theta}[Q^{\pi_\theta}(s,a)\nabla_\theta \log \pi_\theta(a|s)]$ (Degris et al., 2012); đây là điểm khởi đầu của nhiều thuật toán actor-critic off-policy sâu hiện đại (DDPG và các biến thể).

Marginalized importance sampling cố ước lượng trực tiếp tỷ số mật độ trạng thái $\rho_\pi(s) = d_\pi(s)/d_{\pi_\beta}(s)$ (thay vì tích trọng số theo từng hành động), nhờ đó phương sai không vượt quá IS thông thường. Hai hướng: (i) "forward" Bellman equation trên ρ_π, giải bằng TD-learning hoặc bằng bài toán saddle-point đối nghịch $\min_\rho \max_f L(\rho, f)^2$ (Liu et al., 2018); (ii) "backward" Bellman equation trên một hàm phụ $\nu(s,a)$ qua đối ngẫu lồi, cho phép ước lượng ρ^π **không cần mẫu on-policy**. Đây là nền cho các thuật toán off-policy actor-critic dùng f-divergence regularizer giữa state-marginal của policy và của dataset (Nachum et al., 2019). Paper nhận định: các phương pháp IS này chủ yếu phát triển cho off-policy (còn thu thêm dữ liệu), chưa kiểm chứng tốt ở offline hoàn toàn, trừ các kỹ thuật off-policy evaluation/high-confidence improvement.

3.3 Distributional shift trong dynamic programming: hiệu ứng "unlearning"

Với actor-critic/Q-learning thường, state distribution shift chỉ ảnh hưởng ở test-time (vì cả policy và Q-function chỉ được huấn luyện trên $s \sim d_{\pi_\beta}$). Nhưng **action distribution shift ảnh hưởng ngay tại training time**: target Bellman backup cần $Q(s_{t+1}, a_{t+1})$ với $a_{t+1} \sim \pi(a_{t+1}|s_{t+1})$, một hành động có thể nằm ngoài phân phối hành động mà Q-function từng được huấn luyện trên. Vì policy được tối ưu để **tối đa hoá** $\mathbb{E}_{a\sim\pi}[Q(s,a)]$, nếu Q-function định giá sai cao cho một hành động ngoài phân phối, policy sẽ học cách chọn chính hành động đó, sai số không được sửa (vì không có tương tác thật để "kiểm chứng") và dồn tích qua các vòng cập nhật.

Thực nghiệm minh hoạ (Figure 2, trích từ Kumar et al., 2019): chạy SAC (actor-critic chuẩn) theo kiểu offline trên HalfCheetah-v2. Trục hoành là số bước gradient cập nhật Q; đồ thị bên trái (return, trục dao động trong khoảng khoảng -1000 đến 1000) cho thấy return **tăng rồi giảm mạnh** khi huấn luyện tiếp; đồ thị bên phải (log Q-value, trục khoảng 0 đến 30) cho thấy **Q ước lượng tăng đơn điệu** suốt quá trình, không hề giảm theo return thật. Hiệu ứng này **không biến mất** khi tăng số lượng mẫu dataset ($n = 1000$ đến 1000000), cho thấy đây không phải overfitting thống kê thông thường, mà là hệ quả của out-of-distribution action trong backup.

3.4 Policy constraint methods

Công thức tổng quát: lặp $\hat{Q}^\pi_{k+1} \leftarrow \arg\min_Q \mathbb{E}_{(s,a,s')\sim D}[(Q(s,a) - (r + \gamma \mathbb{E}_{a'\sim\pi_k}[\hat{Q}^\pi_k(s',a')]))^2]$, $\pi_{k+1} \leftarrow \arg\max_\pi \mathbb{E}_{s\sim D}[\mathbb{E}_{a\sim\pi}[\hat{Q}^\pi_{k+1}(s,a)]]$ **có thêm ràng buộc** $D(\pi, \pi_\beta) \le \epsilon$ (policy constraint) hoặc phạt trực tiếp vào reward $\bar{r}(s,a) = r(s,a) - \alpha D(\pi(\cdot|s), \pi_\beta(\cdot|s))$ (policy penalty).

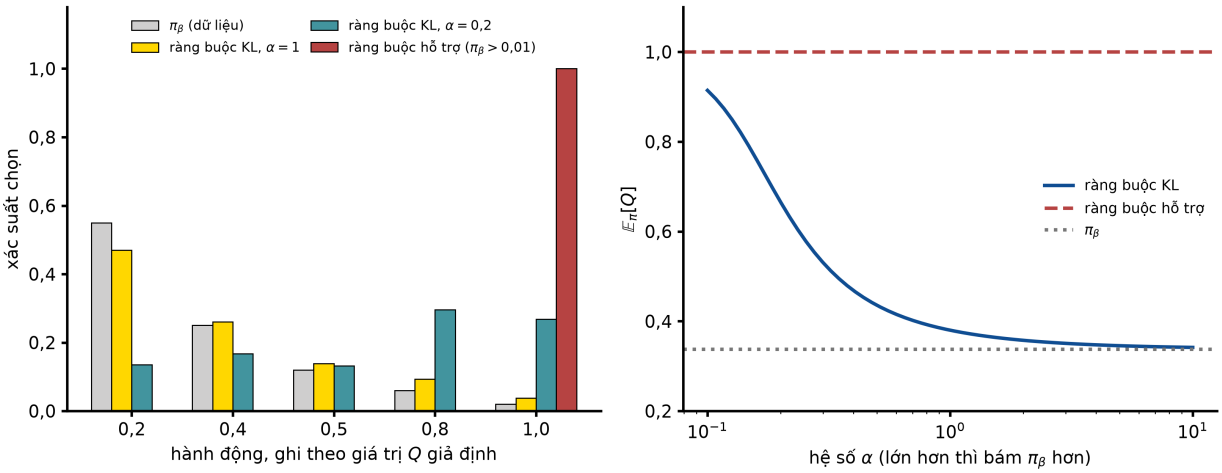
- **Explicit f-divergence** (eq. 19): $D_f(\pi, \pi_\beta) = \mathbb{E}_{a\sim\pi}[f(\pi(a|s)/\pi_\beta(a|s))]$; trường hợp quen thuộc nhất là KL-divergence, tương đương control-as-inference với "action prior" π_β, nhưng cần ước lượng tường minh $\log \pi_\beta(a|s)$ (ví dụ bằng behavioral cloning).
- **Implicit f-divergence** (AWR, AWAC, ABM): giải nghiệm tối ưu không tham số dưới ràng buộc KL rồi chiếu về họ policy bằng supervised regression có trọng số:

$$\bar{\pi}_{k+1}(a|s) \propto \pi_\beta(a|s) \exp\left(\tfrac{1}{\alpha} Q^\pi_k(s,a)\right), \qquad \pi_{k+1} \leftarrow \arg\min_\pi D_{KL}(\bar{\pi}_{k+1}, \pi)$$

không cần ước lượng π_β tường minh, chỉ cần reweight mẫu có sẵn trong buffer.

- **Integral probability metric (IPM):** $D_\Phi(\pi, \pi_\beta) = \sup_{\phi\in\Phi} \mathbb{E}_{a\sim\pi}[\phi] - \mathbb{E}_{a\sim\pi_\beta}[\phi]$; Φ = đơn vị norm trong RKHS cho **Maximum Mean Discrepancy (MMD)**, Φ = hàm Lipschitz-1 cho **Wasserstein-1**. BEAR (Kumar et al., 2019) dùng MMD.
- **Tradeoff ràng buộc phân phối (distribution) vs ràng buộc hỗ trợ (support):** nếu π_β gần như ngẫu nhiên đều, một ràng buộc KL buộc π học được cũng phải gần ngẫu nhiên đều (rất kém tối ưu), dù lẽ ra không có hành động nào là "ngoài phân phối" trong trường hợp này. Paper minh hoạ bằng ví dụ 1D lineworld (Figure 3, Kumar 2019): π_β ưu tiên mạnh hành động "trái" ở mọi state; ràng buộc theo mật độ phân phối (distribution constraint) không tìm được policy tối ưu, còn ràng buộc theo hỗ trợ (support constraint, chỉ cấm hành động có $\pi_\beta(a|s) \approx 0$, không quan tâm mật độ) vẫn tìm được. Thước đo hỗ trợ rời rạc: $D_{support,\varepsilon}(\pi, \pi_\beta) = \sum_{a:\pi_\beta(a|s)\le\varepsilon} \pi(a|s)$; với không gian hành động liên tục, MMD trên mẫu

hữu hạn **có tác dụng gần giống** ràng buộc hỗ trợ hơn là ràng buộc mật độ chặt (Kumar et al., 2019, thực nghiệm Figure 7 của bài đó).



Hình 50: Mô phỏng bandit năm hành động: ràng buộc KL và ràng buộc support so với chính sách tham lam (minh hoạ, tự tính)

Cách đọc hình: dữ liệu được sinh bởi một chính sách hành vi ít khi chọn hành động có Q cao nhất. Ràng buộc KL với hệ số alpha nhỏ dịch chính sách dần về hành động tốt, còn ràng buộc support chỉ cho phép hành động đủ phổ biến trong dữ liệu. Giá trị Q và phân phối hành vi là giả định của mô phỏng, không lấy từ paper.

3.5 Conservative/uncertainty-based value function

Thay vì ràng buộc policy, một hướng khác là làm cho **Q-function tự chịu được** truy vấn ngoài phân phối: học một tập/phân phối $P_D(Q^\pi)$ rồi cải thiện policy bằng ước lượng thận trọng

$$\boxed{\pi_{k+1} \leftarrow \arg\max_\pi \; \mathbb{E}_{s\sim D}\Big[\mathbb{E}_{a\sim\pi}[\mathbb{E}_{Q\sim P_D(Q^\pi)}[Q(s,a)] - \alpha\,\mathrm{Unc}(P_D(Q^\pi))]\Big]}$$

với Unc đo độ bất định (ví dụ phương sai qua một bootstrap ensemble các Q-network, hoặc trừ một độ lệch chuẩn khỏi ước lượng Gaussian của Q). Thực nghiệm cho thấy các phương pháp uncertainty-based thuần thường kém hơn policy constraint/conservative-Q trong thực tế, vì khó hiệu chỉnh (calibrate) uncertainty của mạng neural sâu đủ tốt cho mục đích thận trọng (yêu cầu cao hơn nhiều so với dùng uncertainty cho exploration, nơi chỉ cần tập bất định chứa hành vi tốt).

Conservative Q-Learning (CQL, Kumar et al., 2020b; = bài 20) không cần mô hình uncertainty tường minh: thêm một số hạng phạt $\alpha C(B,\phi)$ vào loss Bellman, với μ chọn đối nghịch để đẩy Q-value cao bị phạt:

$$C_{CQL_0}(B,\phi) = \mathbb{E}_{s\sim B}\Big[\log\sum_a \exp Q_\phi(s,a)\Big] \text{ (khi } \mu(a|s) \propto \exp Q(s,a))$$

cho Q_ϕ là **cận dưới** (lower bound) chính xác của Q thật, nhưng có xu hướng **underestimate quá mức**. Biến thể $C_{CQL_1}(B,\phi) = \mathbb{E}_{s\sim B,a\sim\mu}[Q_\phi] - \mathbb{E}_{(s,a)\sim B}[Q_\phi]$ thêm số hạng tối đa hoá Q trên dữ liệu thật để cân bằng, cho hiệu năng thực nghiệm tốt nhất trong các biến thể (chi tiết đầy đủ ở bài 20).

3.6 Offline model-based RL

Mô hình $T_\psi(s_{t+1}|s_t,a_t)$ được fit bằng supervised learning (tận dụng tốt dữ liệu lớn), nhưng cũng chịu distributional shift: policy được tối ưu hoá để tối đa return **dưới mô hình**, nên có thể học cách "khai thác" (exploit)

lỗi mô hình ở vùng ngoài phân phối để tạo ảo tưởng return cao (model exploitation). Theo Janner et al. (2019), nếu TVD giữa T_ψ và T thật bị chặn bởi ϵ_m và TVD giữa π, π_β bị chặn bởi ϵ_π:

$$J(\pi) \geq J_\psi(\pi) - \Big[\frac{2\gamma r_{max}(\epsilon_m + 2\epsilon_\pi)}{(1-\gamma)^2} + \frac{4 r_{max}\epsilon_\pi}{1-\gamma}\Big]$$

(số hạng đầu: sai số dồn do model; số hạng sau: sai số dồn do policy lệch π_β; cùng dạng phân tích với DAgger ở mục 1, nhưng giờ cả model và policy đều gây lệch phân phối). Hệ quả thực hành: **rollout ngắn** (branching từ state thật trong data, kiểu Dyna) giảm tích luỹ sai số.

MOPO (Yu et al., 2020) và **MoREL (Kidambi et al., 2020)** (đồng thời, cùng ý tưởng): dùng một "error oracle" $u(s,a)$ ước lượng độ sai của model, rồi phạt reward $\tilde{r}(s,a) = r(s,a) - \lambda u(s,a)$ (MOPO) hoặc đưa agent vào trạng thái hấp thụ (absorbing state) reward thấp khi $u(s,a)$ vượt ngưỡng (MoREL); cả hai chứng minh được: ước lượng giá trị dưới mô hình đã điều chỉnh **chặn dưới** giá trị thật trong MDP gốc. Yu et al. (2020) cũng cho thấy MBPO (một thuật toán model-based online thông thường, Janner et al. 2019) đạt hiệu năng hợp lý ngay cả khi áp dụng offline **không sửa đổi gì**, trong khi SAC (model-free, không ràng buộc) thất bại hoàn toàn trong cùng điều kiện, gợi ý model-based RL có khả năng chịu offline tốt hơn tự nhiên hơn so với dynamic programming model-free trần.

4. Thí nghiệm và kết quả chính

Đây là tutorial/survey, không có một bộ thực nghiệm "của riêng" theo một thiết lập cố định; các số liệu minh hoạ được trích từ các paper liên quan:

- **Hiệu ứng unlearning** (mục 3.3, Figure 2, từ Kumar et al. 2019): SAC offline trên HalfCheetah-v2, học càng lâu return càng sụp trong khi $\log Q$ càng tăng, lặp lại ở mọi kích thước dataset $n \in \{1000, 10000, 100000, 1000000\}$.
- **QT-Opt** (Kalashnikov et al., 2018): học policy robot gắp vật từ ảnh (vision-based grasping) từ **khoảng 500.000 lượt gắp** (grasping trials) đã logged trước đó; online fine-tuning thêm vẫn cải thiện đáng kể so với chỉ train offline, cho thấy ngay cả dynamic programming off-policy mạnh cũng còn khoảng cách so với có tương tác thêm.
- **D4RL benchmark suite** (Fu et al., 2020, do cùng nhóm tác giả đề xuất): cung cấp dataset/mô phỏng chuẩn hoá để đánh giá offline RL, đặc biệt nhắm tới tính chất "**stitching**" (ghép nối cấu trúc thành phần): nếu dataset có một đoạn quỹ đạo đi từ trạng thái 1 sang 2, và một đoạn khác đi từ 2 sang 3, một phương pháp offline RL hiệu quả nên suy ra được cách đi từ 1 sang 3 (ví dụ minh hoạ trong môi trường Maze2D, Figure 4), dù không có quỹ đạo nào trong data đi thẳng 1 sang 3. Đây là năng lực mà offline RL kỳ vọng vượt qua cả hành vi tốt nhất có trong dataset.

5. Phân tích / ablation

- **f-divergence vs support constraint:** paper chỉ rõ, bằng ví dụ lineworld, rằng constraint theo mật độ phân phối (KL, f-divergence nói chung) có thể **quá chặt** khi π_β lệch mạnh nhưng vẫn có hỗ trợ đủ rộng; constraint theo hỗ trợ (hoặc MMD xấp xỉ nó ở chế độ mẫu hữu hạn) là lựa chọn phù hợp hơn về lý thuyết cho offline RL.
- **Policy constraint vs conservative Q-function:** hai nhóm có tradeoff khác nhau. Policy constraint cần ước lượng π_β (khó khi hành vi đa mode/multi-modal, ví dụ fit Gaussian đơn mode vào dữ liệu đa mode có thể làm trung bình hoá sai các mode), các biến thể implicit (AWR/AWAC) giảm nhẹ vấn đề này. Conservative Q-function (CQL) không cần fit π_β nhưng dễ **underestimate quá mức** nếu dataset nhỏ hoặc hành động ít được lấy mẫu.
- **Model-based vs model-free:** còn là câu hỏi lý thuyết mở (mục 5.3): trong trường hợp linear function approximation, model-based update và fitted value iteration update cho ra **chính xác cùng kết quả** (Van

Seijen & Sutton, 2015), chưa rõ có còn đúng với hàm xấp xỉ phi tuyến hay không.

- **Error accumulation theo horizon:** mọi họ phương pháp DP offline đều chịu sai số dồn theo hệ số $\approx 1/(1-\gamma)^2 \approx H^2$; Kidambi et al. (2020) chứng minh scaling bậc hai theo horizon là **không thể tránh** (lower bound) cho bất kỳ thuật toán offline RL nào trong trường hợp xấu nhất, bất kể constraint theo state-action marginal có tinh vi thế nào.

6. Giới hạn

Tác giả nêu (mục 3.5, 4.6, 5.3, 7, "Challenges and Open Problems"):

- Chưa có điều kiện lý thuyết "đủ" (non-trivial sufficiency condition) nào trên dataset D để đảm bảo bất kỳ thuật toán offline RL nào phục hồi được policy gần tối ưu.
- Uncertainty estimation với mạng neural sâu còn kém tin cậy, là nút thắt chính của các phương pháp uncertainty-based.
- Chưa có constraint nào vừa kiểm soát tốt error accumulation vừa không giới hạn quá mức khả năng cải thiện so với π_β; bậc hai theo horizon là không thể tránh trong trường hợp xấu nhất.
- Benchmark cho offline RL (tại thời điểm viết, 2020) còn chưa chuẩn hoá tốt; nhiều nghiên cứu trước dùng replay buffer của chính thuật toán online, một thiết lập không thực tế.
- Chưa rõ model-based RL có vượt trội dynamic programming về lý thuyết hay không ngoài trường hợp tuyến tính.

Nhận xét:

- Vì là tutorial/survey, phần lớn công thức là tổng hợp lại từ nhiều paper khác (Precup, Degris, Nachum, Kumar, Yu, Kidambi,...), không phải phát hiện mới của chính 4 tác giả; độ sâu kỹ thuật từng phương pháp (ví dụ marginalized IS, mục 3.2) vì vậy chỉ ở mức giới thiệu, không đủ để tái lập trực tiếp.
- Không có bảng so sánh số liệu thống nhất giữa các phương pháp (constraint vs conservative vs model-based) trên cùng benchmark trong bài; muốn so sánh định lượng cần đọc riêng CQL (bài 20), BEAR, MOPO, MoREL.
- Phần ứng dụng (mục 6.2-6.6: robotics, y tế, lái xe tự động, hội thoại, quảng cáo) chủ yếu là liệt kê các paper đã làm, không có số liệu định lượng mới ngoài con số QT-Opt đã trích ở mục 4.

7. Liên hệ với các bài khác

- **Bài 17 (DAgger):** tutorial dùng trực tiếp Theorem 2.1/3.2 của Ross et al. (2011) làm ví dụ mở đầu minh hoạ distributional shift (mục 1 ở trên); cùng hiện tượng "lỗi dồn theo H^2 nếu không có khả năng sửa on-policy" chạy xuyên suốt cả hai bài.
- **Bài 18 (GAIL):** GAIL **không phải** offline RL (vẫn rollout on-policy liên tục trong môi trường bằng TRPO); hữu ích để thấy sự khác biệt giữa "imitation learning có tương tác môi trường, không có expert tương tác" (GAIL) và "RL hoàn toàn không tương tác với MDP" (offline RL, bài này).
- **Bài 20 (CQL):** chính là phiên bản đầy đủ, với chứng minh lower-bound chặt chẽ, của mục 3.5 ở đây (chỉ giới thiệu C_{CQL_0}, C_{CQL_1} ở mức công thức).
- **Bài 21 (Decision Transformer):** đi theo hướng hoàn toàn khác với mọi phương pháp ở mục 3-5 của tutorial này: bỏ qua dynamic programming/Bellman backup (và do đó bỏ qua toàn bộ vấn đề action distribution shift ở mục 3.3) để chuyển bài toán offline RL thành sequence modeling có điều kiện trên return-to-go.
- **Bài 00 (RLHF/DPO):** DPO có thể nhìn qua lăng kính của mục 3.4 (policy constraint): DPO tối ưu policy dưới ràng buộc KL ngầm định với policy tham chiếu (SFT model), về bản chất là một ràng buộc kiểu $D_{KL}(\pi \| \pi_{ref})$ giống công thức ở mục 3.4, chỉ khác là áp dụng cho bài toán preference offline chứ không có reward từng-bước.

8. Ứng dụng thực tế (suy luận)

- **Bài học siêu tham số quan trọng nhất:** khi dữ liệu là cố định (không tương tác thêm), thứ quyết định thành/bại không phải là thuật toán RL nền (Q-learning hay actor-critic) mà là **cơ chế kiểm soát distributional shift** gắn thêm vào nó (constraint nào, đo "gần" bằng gì). Vận dụng DP offline "trần" (không constraint) gần như chắc hỏng theo đúng hiệu ứng "unlearning" ở mục 3.3/4.
- **Chọn benchmark:** nếu muốn kiểm chứng một ý tưởng offline RL/offline distillation mới, nên tự hỏi dataset có tính chất "stitching" (D4RL) hay không trước khi kỳ vọng phương pháp vượt qua hành vi tốt nhất sẵn có trong data.

9. Thí nghiệm gợi ý

1. Tái hiện hiệu ứng "unlearning" (mục 3.3) trên một task Gym nhỏ: train SAC/DDQN hoàn toàn offline trên một buffer cố định, theo dõi return thật (eval trong môi trường) song song với Q ước lượng trung bình, quan sát điểm phân kỳ.
2. So sánh constraint KL (f-divergence) với MMD trên một bandit đa mode nhân tạo (ví dụ π_β là hỗn hợp vài Gaussian hẹp): đo xem constraint nào giữ được policy tối ưu trong support mà không ép về trung bình các mode.
3. Chạy D4RL Maze2D với một thuật toán DP offline cơ bản (không ràng buộc) so với CQL, đo khả năng "ghép nối" hai đoạn quỹ đạo không liền nhau trong dataset thành một đường đi tối ưu mới.
4. Đo nhạy cảm của CQL/BEAR theo kích thước dataset nhỏ dần, kiểm chứng tuyên bố "càng ít dữ liệu, conservative Q-function càng dễ underestimate quá mức" ở mục 5.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Offline RL	$D = \{(s, a, s', r)\}$ cố định, π_β không rõ, không tương tác thêm	khác off-policy (còn thu thêm)
Distributional shift	$d_\pi(s) \neq d_{\pi_\beta}(s)$, và hành động $a \sim \pi$ lệch π_β	nguồn gốc mọi khó khăn
BC offline bound	$\ell(\pi) \leq C + H^2\epsilon$	Theorem 2.1 = bài 17 Thm 2.1
DAgger-style bound	$\ell(\pi) \leq C + H\epsilon$	Theorem 2.2 = bài 17 Thm 3.2
Doubly robust IS	dùng $\hat{Q}$ làm control variate	không chệch nếu π_β biết hoặc model đúng
Policy constraint	$\max_\pi \mathbb{E}[Q^\pi]$ s.t. $D(\pi, \pi_\beta) \leq \epsilon$	D = KL, MMD, Wasserstein,...
Support vs distribution	ràng buộc hỗ trợ không phạt mật độ thấp, chỉ cấm $\pi_\beta \approx 0$	MMD hữu hạn mẫu ≈ ràng buộc hỗ trợ
Conservative Q (CQL)	$\tilde{E} = \alpha C(B, \phi) + E(B, \phi)$, $C_{CQL_0} = \mathbb{E}[\log \sum_a \exp Q]$	cho Q là lower bound
Model-based bound	$J(\pi) \geq J_\psi(\pi) - [\ldots(\epsilon_m, \epsilon_\pi)\ldots]$	Janner et al. 2019
D4RL stitching	ghép đoạn $1 \to 2$ và $2 \to 3$ thành $1 \to 3$	thước đo năng lực vượt dataset

11. Kết luận ngắn

Offline RL khó không phải vì thiếu một thuật toán RL nền tốt, mà vì bài toán tự nó là một dạng counterfactual inference: suy luận về hệ quả của hành động khác với những gì đã quan sát, trong khi mọi công cụ học máy hiện có đều ngầm giả định i.i.d. Mọi họ phương pháp (importance sampling, policy constraint, conservative/uncertainty value function, model-based) đều là những cách khác nhau để kiểm soát distributional shift giữa policy học được và behavior policy đã sinh ra dữ liệu, đổi lấy các tradeoff khác nhau giữa phương sai, độ chệch, và mức độ bảo toàn khả năng cải thiện so với dữ liệu gốc. Tutorial kết luận rằng tiến bộ tương lai

của offline RL có thể đến nhiều từ chất lượng/đa dạng của dataset (như trong NLP/computer vision) hơn là chỉ từ thuật toán mới.

Link gốc: https://arxiv.org/abs/2005.01643

Bài 20

Conservative Q-Learning for Offline Reinforcement Learning (2020)

Tác giả: Aviral Kumar, Aurick Zhou (UC Berkeley), George Tucker (Google Research, Brain Team), Sergey Levine (UC Berkeley, Google Research, Brain Team).
Venue / arXiv: arXiv:2006.04779 (v3, 19/08/2020); bài báo không nêu venue cụ thể, tự ghi "Preprint. Under review." ở đầu trang.
Nguồn: https://arxiv.org/abs/2006.04779
Chủ đề: Conservative Q-Learning (CQL): học một Q-function có **cận dưới lý thuyết** (lower bound) trên giá trị thật, bằng cách thêm một số hạng phạt đơn giản vào loss Bellman thường, giải quyết vấn đề overestimation do out-of-distribution (OOD) action trong offline RL mà không cần ước lượng behavior policy.
Vị trí trong lộ trình: Đọc sau bài 19 (Offline RL tutorial, mục 4.5 ở đó chính là bản rút gọn của bài này) và bài 17 (DAgger, cùng motif "lỗi dồn khi không có khả năng tự sửa on-policy"). Đọc trước bài 21 (Decision Transformer, một lối đi khác hoàn toàn né tránh vấn đề mà CQL giải quyết).
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

MDP (S, A, T, r, γ); $\pi_\beta(a|s)$ behavior policy, D dataset tĩnh lấy mẫu từ $d_{\pi_\beta}(s)\pi_\beta(a|s)$, $\hat{\pi}_\beta(a|s)$ là ước lượng thực nghiệm của π_β từ D. Vòng lặp off-policy dynamic programming chuẩn:

$$\hat{Q}_{k+1} \leftarrow \arg\min_Q \mathbb{E}_{s,a,s' \sim D}\left[\left(r(s,a) + \gamma \mathbb{E}_{a' \sim \hat{\pi}_k(a'|s')}[\hat{Q}_k(s',a')] - Q(s,a)\right)^2\right] \quad \text{(policy evaluation)}$$

$$\hat{\pi}_{k+1} \leftarrow \arg\max_\pi \mathbb{E}_{s \sim D, a \sim \pi(a|s)}[\hat{Q}_{k+1}(s,a)] \quad \text{(policy improvement)}$$

Vấn đề cốt lõi (như bài 19 đã phân tích): target value trong policy evaluation dùng $a' \sim \pi_k(a'|s')$, một hành động có thể **ngoài phân phối** của π_β đã sinh ra D. Vì π được train để tối đa Q, nếu Q định giá sai cao cho một action OOD, π sẽ bị kéo về chọn chính action đó, không có tương tác thật để tự sửa. CQL quan sát: Q-function **không** chịu state distribution shift khi huấn luyện (Bellman backup không bao giờ truy vấn state ngoài D), chỉ chịu action distribution shift; do đó CQL tập trung triệt tiêu đúng vấn đề này, không cần ràng buộc policy theo kiểu các phương pháp trước (BEAR dùng MMD, BRAC dùng KL/Wasserstein, mục 3.4-3.5 bài 19) - những phương pháp này đều cần ước lượng π_β tường minh, dễ sai khi hành vi đa mode.

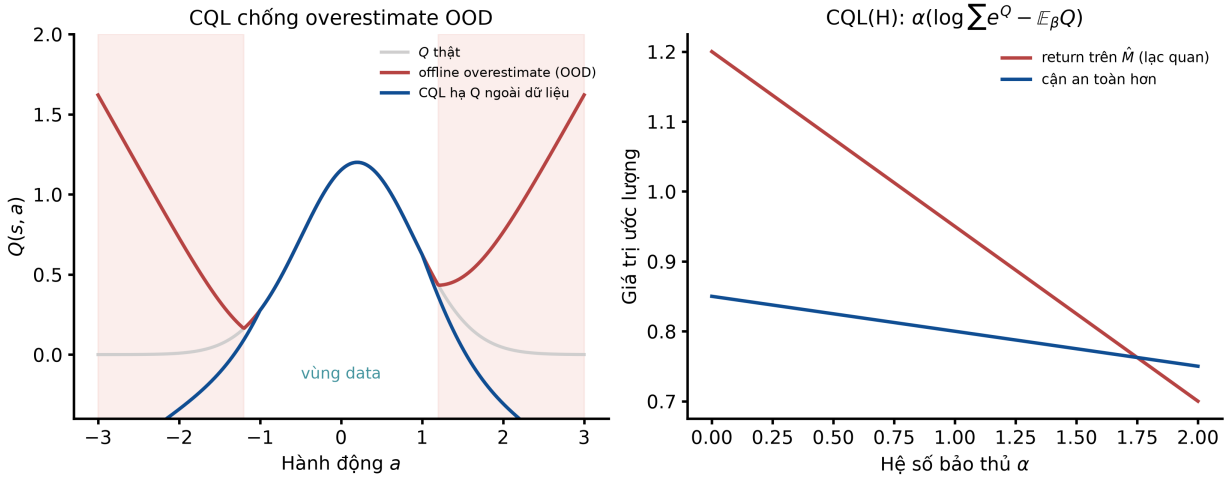
2. Ý tưởng chính

Thêm vào loss Bellman thường một số hạng phạt: tối thiểu hoá Q dưới một phân phối $\mu(a|s)$ (thường chọn đối nghịch, ưu tiên action có Q cao, tức chính các action nghi là bị overestimate), đồng thời tối đa hoá Q dưới chính phân phối dữ liệu π_β để tránh underestimate quá mức. Kết quả: $\mathbb{E}_\pi[\hat{Q}^\pi(s,a)]$ **chặn dưới** $V^\pi(s)$ thật, nghĩa là policy cải thiện dựa trên $\hat{Q}$ không thể "ảo tưởng" giá trị cao hơn thực tế, chỉ có thể đánh giá thấp hơn hoặc đúng. Điều này biến bài toán offline RL thành việc tối ưu một **cận dưới an toàn**, thay vì cố gắng ước lượng chính xác (và dễ sai lệch theo hướng lạc quan) giá trị thật.

3. Phương pháp chi tiết

3.1 Conservative off-policy evaluation

Phiên bản cận dưới **điểm-theo-điểm** (pointwise), ràng buộc $\text{supp}\,\mu \subset \text{supp}\,\hat{\pi}_\beta$:



CQL hạ Q ngoài dữ liệu (OOD); α tăng → bảo thủ hơn (minh hoạ, tự tính)

$$\hat{Q}_{k+1} \leftarrow \arg\min_Q \; \alpha\,\mathbb{E}_{s\sim D, a\sim\mu}[Q(s,a)] + \tfrac{1}{2}\mathbb{E}_{s,a\sim D}[(Q(s,a) - \hat{\mathcal{B}}^\pi \hat{Q}_k(s,a))^2] \qquad (1)$$

Theorem 3.1: với α đủ lớn, $\hat{Q}^\pi(s,a) \le Q^\pi(s,a)\ \forall s \in D, a$ (với xác suất $\ge 1-\delta$; nếu dùng Bellman operator thật không có sai số mẫu, mọi $\alpha > 0$ đều đủ). Nhưng cận này quá chặt nếu ta chỉ cần ước lượng **giá trị kỳ vọng** $V^\pi(s)$, không cần từng điểm. Thêm số hạng tối đa hoá Q dưới $\hat{\pi}_\beta$ để nới bớt độ bảo thủ:

$$\boxed{\hat{Q}_{k+1} \leftarrow \arg\min_Q \; \alpha\Big(\mathbb{E}_{s\sim D, a\sim\mu}[Q] - \mathbb{E}_{s\sim D, a\sim\hat{\pi}_\beta}[Q]\Big) + \tfrac{1}{2}\mathbb{E}_{s,a,s'\sim D}[(Q - \hat{\mathcal{B}}^\pi \hat{Q}_k)^2]} \qquad (2)$$

Theorem 3.2: khi $\mu = \pi$, $\hat{V}^\pi(s) = \mathbb{E}_\pi[\hat{Q}^\pi(s,a)] \le V^\pi(s)\ \forall s \in D$ (không còn chặn điểm-theo-điểm, nhưng chặn đúng cái cần: giá trị kỳ vọng dưới π). Cả hai định lý đều mở rộng được sang function approximation tuyến tính và mạng neural (qua neural tangent kernel, Appendix D.1, không trình bày chi tiết ở đây).

3.2 Họ CQL(R) và hai biến thể thực dụng

$$\boxed{\min_Q \max_\mu \; \alpha\Big(\mathbb{E}_{s\sim D, a\sim\mu}[Q] - \mathbb{E}_{s\sim D, a\sim\hat{\pi}_\beta}[Q]\Big) + \tfrac{1}{2}\mathbb{E}_{s,a,s'\sim D}[(Q - \hat{\mathcal{B}}^{\pi_k}\hat{Q}^k)^2] + \mathcal{R}(\mu)} \quad \text{CQL}(\mathcal{R}) \qquad (3)$$

Chọn $\mathcal{R}(\mu) = -D_{KL}(\mu, \rho)$ cho một prior ρ, nghiệm tối ưu là $\mu(a|s) \propto \rho(a|s)\exp(Q(s,a))$:

- **CQL(H)** ($\rho = \text{Unif}$): số hạng đầu trở thành soft-maximum của Q:

$$\min_Q \; \alpha\,\mathbb{E}_{s\sim D}\Big[\log \sum_a \exp Q(s,a) - \mathbb{E}_{a\sim\hat{\pi}_\beta}[Q(s,a)]\Big] + \tfrac{1}{2}\mathbb{E}_{s,a,s'\sim D}[(Q - \hat{\mathcal{B}}^{\pi_k}\hat{Q}^k)^2] \qquad (4)$$

- **CQL(ρ)** ($\rho = \hat{\pi}_{k-1}$, policy ở vòng trước): số hạng đầu thành trung bình trọng số mũ của Q theo $\hat{\pi}_{k-1}$, thực nghiệm ổn định hơn ở không gian hành động cao chiều (ước lượng $\log\sum_a \exp Q$ bằng mẫu có phương sai cao khi chiều lớn).

CQL có thể cài trên cả Q-learning (dùng $\mathcal{B}^*$ với $\max_{a'}$) và actor-critic (dùng $\mathcal{B}^{\pi_\phi}$), không cần fit $\hat{\pi}_\beta$ tường minh (khác BEAR/BRAC).

3.3 Tính chất lý thuyết: cận dưới lặp lại và gap-expanding

Theorem 3.3: đặt $\pi_{\hat{Q}^k}(a|s) \propto \exp(\hat{Q}^k(s,a))$, giả sử $D_{TV}(\hat{\pi}^{k+1}, \pi_{\hat{Q}^k}) \leq \varepsilon$ (policy đổi "chậm" so với Q hiện tại). Khi đó $\hat{V}^{k+1}(s) \leq V^{k+1}(s)$ nếu mức độ "bảo thủ" tích luỹ ($\mathbb{E}_{\pi_{\hat{Q}^k}}[\pi_{\hat{Q}^k}/\hat{\pi}_\beta - 1]$) lớn hơn mức overestimate tối đa có thể xảy ra do ε. Tức là: cận dưới được **giữ qua mỗi vòng lặp cập nhật**, miễn policy không nhảy quá nhanh.

Theorem 3.4 (gap-expanding): với α đủ lớn, CQL luôn làm **tăng khoảng cách** giữa $\mathbb{E}_{\pi_\beta}[\hat{Q}^k]$ và $\mathbb{E}_{\mu^k}[\hat{Q}^k]$ so với khoảng cách dưới Q^k thật. Hệ quả: policy $\pi^k(a|s) \propto \exp(\hat{Q}^k(s,a))$ bị kéo gần $\hat{\pi}_\beta$ hơn một cách **tự động**, không cần ràng buộc tường minh nào lên policy; đây là cơ chế CQL "miễn nhiễm" phần nào với lỗi overestimate do function approximation hoặc sai số mẫu ở action OOD.

3.4 Safe policy improvement

Theorem 3.5: nghiệm của Eq. (2) tương đương việc tối ưu $\pi^\star = \arg\max_\pi [J(\pi, \hat{M}) - \frac{\alpha}{1-\gamma}\mathbb{E}_{s\sim d_\pi}[D_{CQL}(\pi, \hat{\pi}_\beta)(s)]]$, với $D_{CQL}(\pi, \pi_\beta)(s) = \sum_a \pi(a|s)(\frac{\pi(a|s)}{\hat{\pi}_\beta(a|s)} - 1)$ và $\hat{M}$ là MDP thực nghiệm cảm sinh từ D. Tức CQL tối ưu return trên $\hat{M}$ **cộng với một hình phạt** lệch khỏi $\hat{\pi}_\beta$, hình phạt này do chính tính chất gap-expanding tạo ra (không cần thêm tường minh).

Theorem 3.6 (ζ-safe policy improvement): $J(\pi^\star, M) \geq J(\hat{\pi}_\beta, M) - \zeta$ với xác suất cao, ζ gồm hai số hạng: một do sai số mẫu (giảm khi $|D(s)|$ tăng), một do chính hình phạt CQL (giảm khi α nhỏ phù hợp với lượng dữ liệu). Diễn giải: CQL cải thiện an toàn so với behavior policy, mức cải thiện phụ thuộc vào cả α và lượng dữ liệu quan sát.

3.5 Thuật toán thực hành

```
Input: Q_theta, (actor-critic thì thêm pi_phi)
for t = 1 .. N:
    # policy evaluation: cap nhat theta bang CQL(R) (vd CQL(H), Eq 4)
    theta <- theta - eta_Q * grad_theta CQL(R)(theta)   # dung B* (Q-learning) hoac
    ↪  B^pi_phi (actor-critic)
    # (chi actor-critic) policy improvement kieu SAC
    phi <- phi + eta_pi * E_{s~D, a~pi_phi}[ Q_theta(s,a) - log pi_phi(a|s) ]
```

Chỉ cần **dưới 20 dòng code** thêm vào SAC (continuous control) hoặc QR-DQN (discrete). α được tự động chỉnh bằng **Lagrangian dual gradient descent** (continuous control) hoặc cố định theo bảng ở Appendix F (discrete control). Learning rate policy 3×10^{-5} (chậm hơn Q-function 3×10^{-4} hoặc 10^{-4}), đúng theo yêu cầu "policy đổi chậm" của Theorem 3.3.

4. Thí nghiệm và kết quả chính

Gym D4RL (Table 1, normalized return, trung bình 4 seed), so với SAC, BC, BEAR, BRAC-p/v:

Task	SAC	BC	BEAR	BRAC-p	BRAC-v	CQL(H)
halfcheetah-random	30.5	2.1	25.5	23.5	28.1	35.4
hopper-medium	0.8	29.0	47.6	31.2	32.3	**58.0**
walker2d-medium	0.9	6.6	33.2	72.7	81.3	79.2
halfcheetah-medium-expert	1.8	35.8	51.7	43.8	45.3	**62.4**
walker2d-medium-expert	1.9	11.3	10.8	-0.3	0.9	**98.7**
halfcheetah-random-expert	53.0	1.3	24.6	30.2	2.2	**92.5**

Task	SAC	BC	BEAR	BRAC-p	BRAC-v	CQL(H)
walker2d-random-expert	0.8	0.7	1.9	0.2	2.7	**91.1**
hopper-random-expert	5.6	10.1	10.1	5.8	11.1	**110.5**
hopper-mixed	3.5	11.8	25.3	0.7	0.8	**48.6**

Trên dataset từ **một** policy ("-random", "-expert", "-medium"), CQL ngang hoặc nhích hơn baseline tốt nhất. Trên dataset trộn **nhiều** policy ("-mixed", "-medium-expert", "-random-expert", gần với dữ liệu thực tế hơn), CQL vượt baseline tốt nhất **2-3 lần**.

Adroit (tay robot 24-DoF, dữ liệu người) và AntMaze/Kitchen (Table 2): CQL(H)/CQL(ρ) là phương pháp **duy nhất vượt BC** trên Adroit, hơn phương pháp offline RL tốt thứ hai **2-9 lần** (ví dụ pen-human: BC 34.4, BEAR -1.0, CQL(ρ) 55.8). Trên antmaze-medium-play/diverse và antmaze-large-play/diverse, mọi baseline đạt 0.0, chỉ CQL đạt giá trị dương (antmaze-medium-play: CQL(H) 61.2; antmaze-large-diverse: CQL(H) 14.9). Trên Kitchen, CQL đạt >40% tỉ lệ thành công ở mọi task, vượt BC.

Atari offline (Q-learning variant, so với QR-DQN, REM): với 1% dữ liệu DQN-replay, CQL(H) vượt phương pháp tốt nhất **36 lần trên Q*bert **(CQL(H) 14012.0 so với QR-DQN 383.6, REM 343.4) và** 6 lần trên Breakout** (CQL(H) 61.1 so với REM 11.0). Với 20% dữ liệu (Figure 1), CQL học ổn định trên 3/4 game, không sụp mạnh như QR-DQN trên Seaquest.

Kiểm chứng cận dưới thực nghiệm (Table 4): so sánh $\mathbb{E}_{s\sim D}[\hat{V}^k(s)]$ dự đoán với return thật đo được, trên 3 task D4RL. CQL(H) là phương pháp **duy nhất** cho hiệu số âm (tức đúng là cận dưới) ở cả 3 task (ví dụ hopper-medium: CQL(H) -7.48), trong khi BEAR và ensemble Q-function (kích thước 2, 4, 10, 20) đều overestimate rất nặng (ví dụ hopper-medium, Ensemble(2): 26.03×10^{12}). Biến thể dùng Eq. (1) (cận điểm-theo-điểm) cho hiệu số âm còn **lớn hơn nhiều** (ví dụ hopper-medium: -156.70), xác nhận Eq. (2)/CQL(H) cho cận chặt hơn, đúng như Theorem 3.2 dự đoán.

5. Phân tích / ablation (Appendix G)

Câu hỏi	Kết quả
CQL(H) vs CQL($\rho = \hat{\pi}_{k-1}$)	CQL(H) tốt hơn trên MuJoCo (Table 5: halfcheetah-medium-expert 7234.5 vs 3995.6); CQL(ρ) tốt hơn trên Adroit (chiều hành động cao, ước lượng log-sum-exp phương sai cao)
Eq. (4) (CQL(H), có số hạng tối đa hoá theo $\hat{\pi}_\beta$) vs Eq. (1) (không có)	có số hạng tối đa hoá luôn tốt hơn, đặc biệt khi dataset từ một policy đơn (Table 6: hopper-medium 1866.1 vs 1028.4)
Lagrange (tự chỉnh α) vs α cố định	Lagrange ngang hoặc tốt hơn trên MuJoCo; vượt rõ trên AntMaze, có thể chênh tới ~30% hiệu năng tối đa (Table 7: antmaze-medium-diverse 0.53 vs 0.21)

Kết luận ablation: số hạng tối đa hoá theo $\hat{\pi}_\beta$ (phân biệt Eq.1 với Eq.2/CQL(H)) và việc tự động chỉnh α bằng Lagrange đều đóng góp hiệu năng rõ rệt, nhất là ở các task có cấu trúc dữ liệu phức tạp (AntMaze cần "stitching" nhiều đoạn quỹ đạo, xem bài 19 mục 4).

6. Giới hạn

Tác giả nêu:

- Chứng minh lý thuyết (Theorem 3.1-3.2) giả định không có sai số mẫu/hàm xấp xỉ; phần mở rộng sang neural network (Appendix D.1, qua NTK) chỉ là một khung phân tích, không phải đảm bảo thực hành chặt.
- CQL(H) kém ổn định hơn ở không gian hành động chiều cao (Adroit) do phương sai khi ước lượng $\log \sum_a \exp Q$; cần CQL(ρ) thay thế.
- Vẫn cần chọn α (hoặc τ cho Lagrange) phù hợp; không có công thức tổng quát suy ra α tối ưu từ đặc tính dataset.

Nhận xét:

- Toàn bộ cận dưới lý thuyết (Theorem 3.1-3.6) phụ thuộc giả định $\text{supp}\,\mu \subset \text{supp}\,\hat{\pi}_\beta$ và $\hat{\pi}_\beta(a|s) > 0\ \forall a$ trên D (paper tự nêu "for simplicity in the derivation"); với dữ liệu thực tế thưa, các giả định này có thể không khớp hoàn toàn.
- Bảng so sánh chỉ có 4 seed/task; bài báo không nêu khoảng tin cậy hoặc độ lệch chuẩn giữa seed trong Table 1-3 (chỉ Table 1/2 ghi "averaged over 4 seeds" mà không kèm std).
- Không có thí nghiệm nào kiểm tra CQL trên môi trường sinh ngôn ngữ/văn bản; mọi kết quả đều control classic/robot/Atari.

7. Liên hệ với các bài khác

- **Bài 17 (DAgger):** cùng chủ đề "sai số dồn khi thiếu khả năng tự sửa on-policy"; khác biệt: DAgger giải quyết bằng cách hỏi thêm oracle, CQL giải quyết bằng cách làm Q-function tự "biết mình không biết" (cận dưới) khi không còn oracle để hỏi.
- **Bài 19 (Offline RL tutorial):** mục 4.5 của bài đó (= mục 3.5 trong ghi chú bài 19) chính là bản rút gọn của bài này; CQL là ví dụ cụ thể, có chứng minh đầy đủ, cho nhóm "conservative/uncertainty-based value function" mà bài 19 chỉ giới thiệu công thức.
- **Bài 18 (GAIL):** đối lập về cơ chế: GAIL là on-policy, cần rollout trong môi trường thật, không có vấn đề OOD action trong Bellman backup vì không dùng Bellman backup; CQL ngược lại là offline hoàn toàn, giải quyết trực diện OOD action.
- **Bài 21 (Decision Transformer):** cùng benchmark D4RL/Adroit để so sánh trực tiếp hiệu năng; Decision Transformer từ bỏ hoàn toàn dynamic programming (không có Bellman backup, không có vấn đề OOD action trong target value) bằng cách chuyển sang sequence modeling, một hướng giải quyết triệt để khác với "làm Q-function cận thận" của CQL.

8. Ứng dụng thực tế (suy luận)

- **Siêu tham số quan trọng nhất khi áp dụng CQL:** hệ số α (hoặc τ nếu dùng Lagrange). Ablation (mục 5) cho thấy Lagrange auto-tuning gần như luôn an toàn hơn chọn α cố định, đặc biệt khi chưa biết trước độ khó/đa mode của dataset; nên ưu tiên Lagrange trừ khi có lý do cụ thể để cố định.
- **Chọn CQL(H) hay CQL(ρ):** phụ thuộc chiều không gian hành động. Action chiều thấp/trung bình (robot locomotion, game) nên dùng CQL(H); action chiều rất cao (tay robot nhiều khớp, hoặc action space lớn kiểu token) nên thử CQL(ρ) để tránh phương sai ước lượng log-sum-exp.
- **Không cần ước lượng behavior policy:** là lợi thế thực hành lớn so với BEAR/BRAC khi dữ liệu thu từ nhiều nguồn khác nhau (ví dụ log từ nhiều phiên bản model/nhiều người dùng), nơi fit một π_β đơn mode sẽ sai lệch.

9. Thí nghiệm gợi ý

1. Cài CQL(H) trên một môi trường Gym nhỏ (ví dụ CartPole liên tục hoá hoặc Pendulum) từ một dataset D4RL-style tự tạo, so sánh với DQN/SAC offline trần, quan sát hiệu ứng "unlearning" (bài 19 mục 3.3) có biến mất không.

2. Lặp lại thí nghiệm Table 4 (kiểm chứng cận dưới) trên một task khác ngoài 3 task hopper đã cho, kiểm tra tính nhất quán của tính chất lower-bound.
3. Sweep α cố định so với Lagrange trên antmaze-medium-diverse, đo độ nhạy hiệu năng theo α để hiểu tại sao Lagrange thắng rõ ở AntMaze nhưng không rõ ở MuJoCo gym.
4. So sánh CQL(H) và CQL(ρ) trên một task action-chiều-trung bình tăng dần (ví dụ walker2d rồi humanoid), xác định ngưỡng chiều mà CQL(ρ) bắt đầu vượt CQL(H).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Vấn đề gốc	target value dùng $a' \sim \pi_k$, có thể OOD với $\hat{\pi}_\beta$	gây overestimate, không tự sửa được
CQL eval (pointwise)	Eq. (1): $\min_Q \alpha \mathbb{E}_\mu[Q] + \frac{1}{2}\|Q - \hat{\mathcal{B}}^\pi \hat{Q}_k\|^2$	Theorem 3.1: cận dưới điểm-theo-điểm
CQL eval (tighter)	Eq. (2): thêm $-\mathbb{E}_{\hat{\pi}_\beta}[Q]$	Theorem 3.2: cận dưới kỳ vọng dưới π
CQL(H)	$\min_Q \alpha \mathbb{E}_s[\log \sum_a \exp Q - \mathbb{E}_{\hat{\pi}_\beta}[Q]] +$ Bellman error	$\mu = \text{Unif} \cdot \exp(Q)$
CQL(ρ)	$\rho = \hat{\pi}_{k-1}$ thay Unif	ổn định hơn ở action chiều cao
Gap-expanding	$\mathbb{E}_{\pi_\beta}[\hat{Q}] - \mathbb{E}_\mu[\hat{Q}] > \mathbb{E}_{\pi_\beta}[Q] - \mathbb{E}_\mu[Q]$	Theorem 3.4, tự kéo policy về gần data
Safe improvement	$J(\pi^\star, M) \geq J(\hat{\pi}_\beta, M) - \zeta$	Theorem 3.6
Lagrange auto-tune	tối ưu α bằng dual gradient descent	tốt hơn rõ trên AntMaze

11. Kết luận ngắn

CQL giải quyết vấn đề overestimation do out-of-distribution action trong offline RL bằng một ý tưởng đơn giản: thêm một số hạng phạt vào loss Bellman để Q-function trở thành cận dưới có chứng minh (Theorem 3.1-3.2), giữ được cận dưới qua các vòng cập nhật và tự "nới xa" Q-value của action trong và ngoài phân phối dữ liệu (gap-expanding, Theorem 3.4), từ đó suy ra đảm bảo cải thiện an toàn so với behavior policy (Theorem 3.5-3.6). Thực nghiệm trên D4RL (gym, Adroit, AntMaze, Kitchen) và Atari offline cho thấy CQL vượt rõ các phương pháp policy-constraint trước đó, đặc biệt trên dataset phức tạp/đa mode và dữ liệu rất hạn chế, mà không cần ước lượng behavior policy tường minh.

Link gốc: https://arxiv.org/abs/2006.04779

Bài 21

Decision Transformer: Reinforcement Learning via Sequence Modeling (2021)

Tác giả: Lili Chen, Kevin Lu (đóng góp chính), Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, Igor Mordatch (UC Berkeley, Facebook AI Research, Google Brain).
Venue / arXiv: arXiv:2106.01345 (v2, 24/06/2021); bài báo không nêu venue cụ thể trong văn bản.
Nguồn: https://arxiv.org/abs/2106.01345
Chủ đề: Decision Transformer: đóng khung bài toán offline RL thành **conditional sequence modeling** bằng kiến trúc GPT, điều kiện hoá trên **return-to-go** để sinh ra hành động, hoàn toàn bỏ qua dynamic programming/Bellman backup.
Vị trí trong lộ trình: Đọc sau bài 19 (Offline RL tutorial, là nguồn gốc của mọi baseline D4RL dùng ở đây) và bài 20 (CQL, baseline so sánh chính). Đây là bài cuối giai đoạn 5 (imitation/offline RL): thoát hẳn khỏi Bellman backup bằng cách chuyển RL thành một bài toán giống huấn luyện ngôn ngữ, bắc cầu trực tiếp sang các kỹ thuật LLM (llm 01 Transformer/GPT).
Chương nền liên quan: nền 06: Khám phá, off-policy và offline RL

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

MDP (S, A, P, R), trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T, a_T, r_T)$, return tại t: $R_t = \sum_{t'=t}^{T} r_{t'}$. Offline RL: chỉ có một dataset cố định các rollout từ policy bất kỳ, không được tương tác thêm (bài 19).

Vấn đề với TD-learning (dynamic programming, như CQL ở bài 20): dựa trên **bootstrapping** (ước lượng Q bằng chính ước lượng Q ở bước sau), một trong ba yếu tố của "deadly triad" (cùng với function approximation và off-policy data) được biết gây mất ổn định; cần discount γ cho tương lai (dễ sinh hành vi "cận thị"); và lan truyền reward **chậm** qua nhiều bước Bellman backup, dễ bị nhiễu bởi tín hiệu "distractor" không liên quan.

Transformer (self-attention, Vaswani et al.): với n embedding đầu vào $\{x_i\}$, mỗi token tạo key k_i, query q_i, value v_i; đầu ra $z_i = \sum_j \text{softmax}(\{\langle q_i, k_{j'} \rangle\}_{j'})_j \cdot v_j$. GPT thêm causal mask (chỉ attend về quá khứ) để sinh tự hồi quy (autoregressive). Attention cho phép gán "credit" trực tiếp giữa các token xa nhau bằng độ tương đồng query-key, không cần lan truyền từng bước như Bellman backup.

Ý tưởng khởi nguồn (ví dụ minh hoạ shortest-path, Figure 2): trên một đồ thị có hướng, reward $= 0$ tại đích, $= -1$ nơi khác. Train một GPT dự đoán token tiếp theo trong chuỗi (return-to-go, state, action), **chỉ từ random walk**, không có demo chuyên gia. Khi sinh (test time), điều kiện hoá trên return-to-go **cao nhất có thể** tại mỗi bước, mô hình tự sinh ra đường đi tối ưu, dù dữ liệu train toàn là đi ngẫu nhiên. Đây là "policy improvement không cần dynamic programming": cải thiện xảy ra nhờ **hindsight return conditioning**, không nhờ Bellman backup.

2. Ý tưởng chính

Thay vì fit value function hoặc tính policy gradient, Decision Transformer coi RL là một bài toán **sinh chuỗi có điều kiện**: học phân phối chung của $(\hat{R}_t, s_t, a_t)$ theo thời gian bằng một GPT, rồi tại test time, chỉ cần "mồi" (prompt) mô hình bằng return-to-go mong muốn (ví dụ mức hiệu năng chuyên gia) để sinh ra chuỗi hành động đạt được mức đó, giống hệt cách một LLM được prompt để sinh văn bản theo một "thể loại"/"chất lượng" mong muốn.

3. Phương pháp chi tiết

3.1 Biểu diễn trajectory

Thay reward thô bằng **return-to-go** $\hat{R}_t = \sum_{t'=t}^{T} r_{t'}$ (tổng reward còn lại từ t đến hết episode), vì muốn mô hình sinh hành động dựa trên return **mong muốn trong tương lai**, không phải reward đã nhận trong quá khứ:

$$\tau = (\hat{R}_1, s_1, a_1,\ \hat{R}_2, s_2, a_2,\ \ldots,\ \hat{R}_T, s_T, a_T) \qquad (2)$$

Tại test time: đặt return mong muốn ban đầu (ví dụ mức thành công tối đa) và state khởi đầu; sau mỗi bước, **trừ** reward thật nhận được khỏi return-to-go hiện tại, lặp lại tới hết episode.

3.2 Kiến trúc

Lấy K timestep gần nhất, tổng $3K$ token (mỗi modality: return-to-go, state, action, một token riêng). Mỗi modality có một **linear embedding layer** riêng (ảnh dùng convolutional encoder), cộng thêm một **embedding theo timestep** (không phải positional embedding chuẩn của transformer, vì một timestep ứng với 3 token cùng lúc). Token được xử lý bởi GPT, dự đoán token hành động tiếp theo bằng attention nhân quả (causal).

3.3 Huấn luyện và sinh

Chỉ train để dự đoán a_t từ s_t (cross-entropy cho action rời rạc, MSE cho action liên tục); dự đoán state/return-to-go không cải thiện hiệu năng trong thí nghiệm của tác giả (dù kiến trúc cho phép, xem mục 5.5).

```
# train: lay minibatch sequence length K, du doan a_t tu token s_t
a_preds = DecisionTransformer(R, s, a, t)
loss = mean((a_preds - a)**2)          # MSE cho action lien tuc (CE cho roi rac)

# eval (sinh tu hoi quy)
R, s, a, t = [target_return], [env.reset()], [], [1]
while not done:
    action = DecisionTransformer(R, s, a, t)[-1]
    new_s, r, done, _ = env.step(action)
    R = R + [R[-1] - r]                # giam return-to-go theo reward thuc nhan duoc
    s, a, t = s + [new_s], a + [action], t + [len(R)]
    R, s, a, t = R[-K:], ...           # chi giu context length K
```

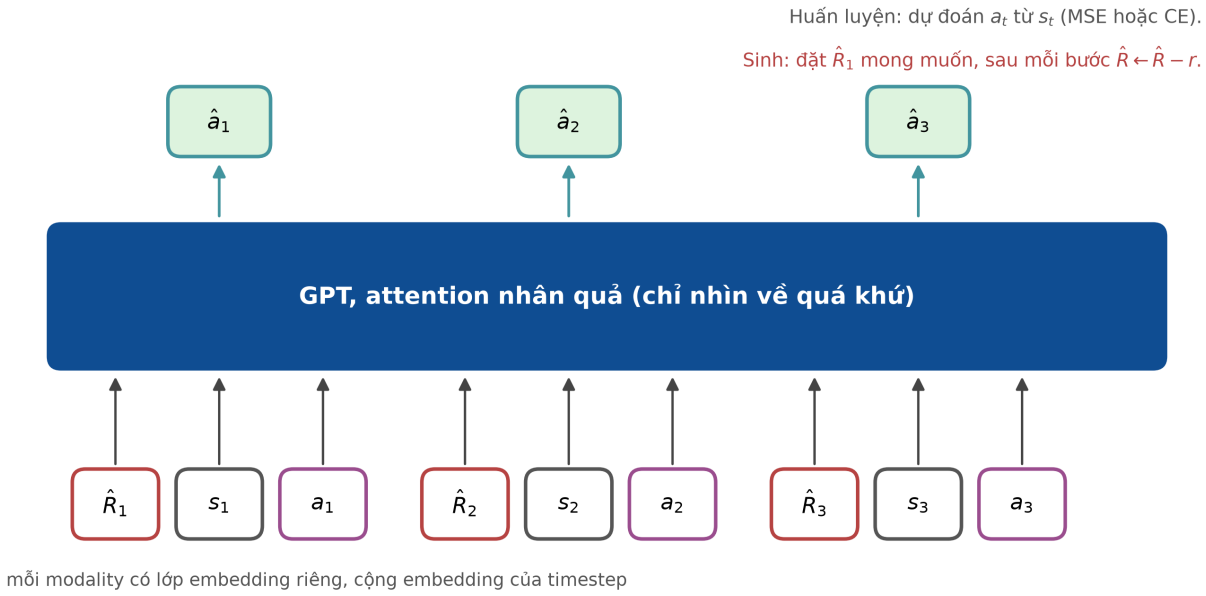
Không có bước tối ưu hoá giá trị (value function) hay tính policy gradient nào trong toàn bộ quy trình train/eval.

Cách đọc hình: mỗi bước thời gian có ba token là return còn lại, trạng thái và hành động, nối lại thành một chuỗi dài. Transformer nhân quả dự đoán hành động từ token trạng thái ở vị trí tương ứng. Khi sinh, ta đặt return mong muốn ở đầu rồi trừ dần phần thưởng nhận được.

4. Thí nghiệm và kết quả chính

Atari (1% dataset DQN-replay, 500 nghìn/50 triệu transition, chuẩn hoá theo điểm gamer chuyên nghiệp = 100, random = 0; trung bình 3 seed, $K = 30$, Pong $K = 50$):

Game	DT (ours)	CQL	QR-DQN	REM	BC
Breakout	**267.5 ± 97.5**	211.1	17.1	8.9	138.9 ± 61.7
Qbert	15.4 ± 11.4	**104.2**	0.0	0.0	17.3 ± 14.7
Pong	106.1 ± 8.1	**111.9**	18.0	0.5	85.2 ± 20.0
Seaquest	**2.5 ± 0.4**	1.7	0.4	0.7	2.1 ± 0.3



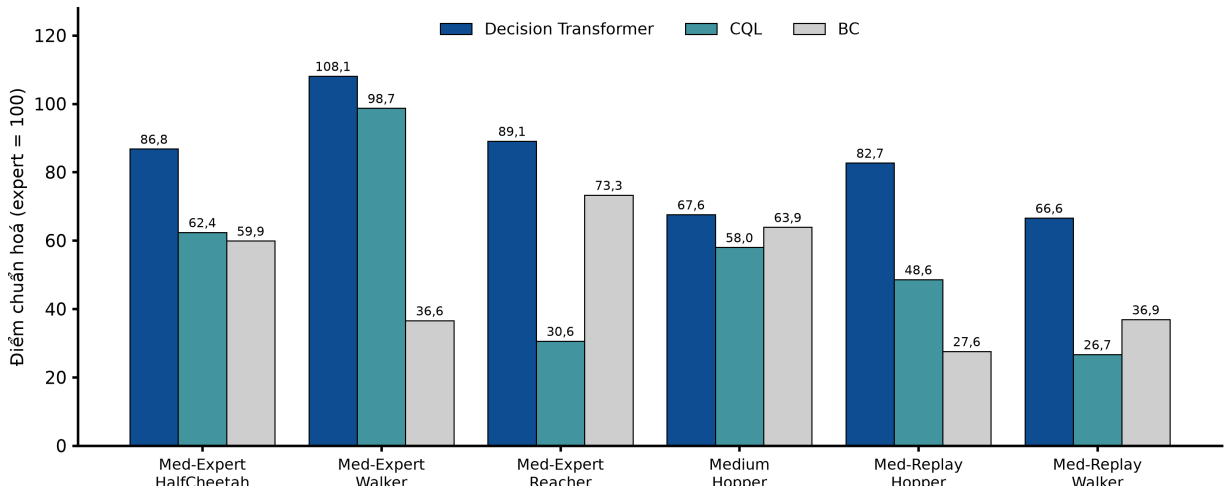
Hình 51: Chuỗi token return to go, trạng thái, hành động đi vào Transformer nhân quả (minh hoạ, tự tính)

DT cạnh tranh với CQL ở 3/4 game, vượt QR-DQN/REM/BC ở cả 4 game.

OpenAI Gym / D4RL (continuous control, chuẩn hoá expert = 100; HalfCheetah/Hopper/Walker/Reacher, 3 bộ dataset Medium, Medium-Replay, Medium-Expert; trung bình 3 seed):

Dataset	Env	DT (ours)	CQL	BEAR	BRAC-v	AWR	BC
Medium-Expert	HalfCheetah	**86.8 ± 1.3**	62.4	53.4	41.9	52.7	59.9
Medium-Expert	Walker	**108.1 ± 0.2**	98.7	40.1	81.6	53.8	36.6
Medium-Expert	Reacher	**89.1 ± 1.3**	30.6	-	-	-	73.3
Medium	Hopper	**67.6 ± 1.0**	58.0	52.1	31.1	35.9	63.9
Medium-Replay	Hopper	**82.7 ± 7.0**	48.6	33.7	0.6	28.4	27.6
Medium-Replay	Walker	**66.6 ± 3.0**	26.7	19.2	0.9	15.5	36.9
Trung bình (không Reacher)		**74.7**	63.9	48.2	36.9	34.3	46.4
Trung bình (toàn bộ)		**69.2**	54.2	-	-	-	47.7

Decision Transformer đạt điểm cao nhất ở **đa số** task, và cạnh tranh ở các task còn lại, mà không có bất kỳ Bellman backup nào.



5.2 DT mô hình được phân phối return không? Varying target return (Figure 4): trên Pong, HalfCheetah, Walker, return đạt được **gần khớp hoàn hảo** với return mong muốn (đường "oracle"); trên Seaquest, DT còn **ngoại suy** được return cao hơn return tối đa quan sát trong dataset.

5.3 Lợi ích của context length dài hơn (Table 5, Atari, so K chuẩn với $K = 1$): Pong rơi từ 106.1 xuống **2.5** khi $K = 1$; Breakout từ 267.5 xuống 73.9. Giả thuyết tác giả: với một mô hình biểu diễn một **phân phối nhiều policy** (do dataset trộn nhiều chất lượng khác nhau), context dài giúp transformer "nhận diện" policy nào đang sinh ra hành động, cải thiện học và động lực huấn luyện.

5.4 Long-term credit assignment (Key-to-Door, Mesnard et al.): môi trường 3 giai đoạn (lấy chìa khoá → phòng trống → cửa), reward chỉ nhận ở cuối **nếu** đã lấy chìa khoá ở đầu, đòi hỏi lan truyền credit xuyên suốt episode. Train trên random-walk, context = **toàn bộ** episode:

Dataset	DT (ours)	CQL	BC	%BC	Random
1K random trajectories	**71.8%**	13.1%	1.4%	69.9%	3.1%
10K random trajectories	**94.6%**	13.3%	1.6%	95.1%	3.1%

TD-learning (CQL) **không lan truyền được** Q-value qua khoảng cách xa này; các phương pháp dùng hindsight return (DT, %BC trên episode thành công) học tốt.

5.5 Transformer có làm critic tốt trong sparse reward không? Sửa DT để **dự đoán** return-token đầu tiên (không cho sẵn) trên Key-to-Door: xác suất reward dự đoán cập nhật đúng theo các sự kiện quan trọng trong episode (Figure 5 trái); attention weight tập trung rõ vào thời điểm **lấy chìa khoá** và **đến cửa** (Figure 5 phải), cho thấy transformer hình thành liên kết state-reward trực tiếp qua attention.

5.6 DT có chịu được sparse/delayed reward không? Biến thể D4RL Hopper chỉ phát reward **một lần** ở cuối (tổng dồn), so TD-learning sụp hoàn toàn còn DT/BC ít bị ảnh hưởng:

Env	DT delayed	CQL delayed	DT dense (gốc)	CQL dense (gốc)
Hopper Medium-Expert	107.3 ± 3.5	**9.0**	107.6	111.0
Hopper Medium	60.7 ± 4.5	**5.2**	67.6	58.0
Hopper Medium-Replay	78.5 ± 3.7	**2.0**	82.7	48.6

5.7 Tại sao DT không cần value pessimism/behavior regularization (như CQL, BEAR, BRAC)? Giả thuyết tác giả: TD-learning tối ưu hoá **trực tiếp** qua một value function đã học, nên mọi sai số của value function dễ bị **khai thác** (exploit) bởi chính bước tối ưu hoá, đòi hỏi cơ chế thận trọng (CQL) hoặc ràng buộc policy (BEAR/BRAC) để bù. DT không tối ưu hoá tường minh qua bất kỳ hàm học được nào (chỉ là maximum-likelihood trên hành động quan sát), nên tránh được vấn đề này từ gốc.

6. Giới hạn

Tác giả nêu:

- Dự đoán state/return-to-go (không chỉ action) không cải thiện hiệu năng trong thí nghiệm hiện tại, dù khung cho phép; để dành cho nghiên cứu tương lai (mục 5.5 chỉ dùng cho mục đích phân tích critic, không dùng để cải thiện actor).
- Chưa rõ DT có lợi thế gì trong chế độ **online RL** (phần 5.8 chỉ là suy đoán/hướng mở, không có thực nghiệm): tác giả gợi ý DT có thể là "động cơ ghi nhớ hành vi" mạnh, kết hợp với thuật toán exploration như Go-Explore, nhưng không kiểm chứng trong bài.

- %BC không phải phương pháp thực tế (cần biết percentile tối ưu bằng rollout thật), chỉ dùng để phân tích, không phải baseline công bằng.

Nhận xét:

- Toàn bộ thực nghiệm chuẩn hoá chọn target return bằng tay (ví dụ mức chuyên gia); bài báo không nêu một quy trình tự động chọn target return tối ưu khi không biết trước mức hiệu năng khả thi của môi trường mới.
- Context length K (30-50) nhỏ hơn rất nhiều so với context của LLM hiện đại; chưa rõ hành vi khi scale lên episode dài hơn nhiều bậc (ví dụ agent nhiều giờ) có giữ được lợi thế credit-assignment qua attention hay không, hay sẽ gặp lại vấn đề attention trên sequence rất dài (bài 17 llm Longformer, bài 55 StreamingLLM).
- Qbert là game duy nhất CQL thắng rõ DT (104.2 vs 15.4); bài báo không phân tích sâu nguyên nhân cụ thể tại sao riêng game này DT kém.

7. Liên hệ với các bài khác

- **Bài 19 (Offline RL tutorial):** D4RL benchmark và motif "distributional shift do bootstrapping" dùng xuyên suốt đều xuất phát từ bài này; Decision Transformer trực tiếp dẫn chiếu Offline RL tutorial làm tài liệu nền.
- **Bài 20 (CQL):** baseline so sánh chính (TD-learning hiện đại nhất tại thời điểm đó) trên cả Atari và D4RL Gym; DT thắng/ngang CQL trên hầu hết task mà không cần value pessimism (đối lập trực tiếp cơ chế "cận dưới có chứng minh" của CQL).
- **Bài 17 (DAgger):** khác biệt cơ chế rõ: DAgger sửa compounding error bằng cách hỏi thêm oracle on-policy; DT không có khái niệm "sửa" nào cả, dựa hoàn toàn vào khả năng tổng quát hoá nhiều policy chất lượng khác nhau của transformer cộng với hindsight return conditioning.
- **llm 01 (Transformer), kiến trúc GPT:** Decision Transformer dùng gần như y nguyên GPT (chỉ đổi token embedding theo modality + positional embedding theo timestep), là một trong những ví dụ sớm nhất cho thấy kiến trúc ngôn ngữ áp dụng thẳng cho control, đặt nền cho các hướng "RL như sequence modeling" sau này ảnh hưởng tới cách huấn luyện LLM-agent.
- **Bridge sang LLM/RLHF (bài 00):** cơ chế "điều kiện hoá trên return mong muốn để sinh hành vi mong muốn" tương tự về mặt khái niệm với việc điều kiện hoá sinh văn bản trên một "chất lượng" hoặc "reward" mong muốn (ví dụ reward-conditioned generation); gợi mở hướng mà sau này RLHF/DPO giải quyết bằng cơ chế khác (so sánh ưu tiên thay vì return-to-go số).

8. Ứng dụng thực tế (suy luận)

- **Bài học siêu tham số quan trọng:** context length K không phải chi tiết phụ: mục 5.3 cho thấy cắt về $K = 1$ làm hiệu năng Pong sụp gần hết (106.1 → 2.5). Khi áp dụng kiểu kiến trúc này cho một agent LLM/VLM có "trajectory" dài (nhiều bước công cụ, nhiều trang OCR liên tiếp), nên dự trù context đủ để mô hình "nhận diện được đang theo policy chất lượng nào", không chỉ đủ để nhìn thấy state hiện tại.
- **Khi nào nên nghĩ theo hướng "RL = sequence modeling" thay vì TD-learning/CQL:** mục 5.6-5.7 cho thấy lợi thế rõ nhất khi reward **thưa** hoặc **trễ** (nơi TD-learning propagation thất bại) và khi muốn tránh toàn bộ cơ chế value pessimism/constraint (đơn giản hoá quy trình, không cần tinh chỉnh α như CQL ở bài 20). Đánh đổi: cần return-to-go nhãn đúng/có ý nghĩa trong dataset, và hiệu năng phụ thuộc khả năng tổng quát hoá của transformer trên phân phối return, không có đảm bảo lý thuyết kiểu "cận dưới" như CQL.

9. Thí nghiệm gợi ý

1. Tái hiện ví dụ shortest-path (Figure 2) trên một đồ thị nhỏ tự tạo: train GPT nhỏ trên random walk, kiểm tra việc điều kiện trên return-to-go cao nhất có sinh ra đường đi tối ưu không.

2. Ablation context length $K \in \{1, 5, 10, 30\}$ trên một task Gym liên tục (không chỉ Atari), kiểm tra lại độ nhạy đã thấy ở mục 5.3 có tái diễn ở continuous control không.
3. Thử nghiệm Key-to-Door với horizon dài hơn nhiều (ví dụ gấp 5-10 lần), so sánh DT và CQL để xem giới hạn thực tế của "attention thay Bellman backup" khi episode rất dài.
4. Sweep target return ngoài khoảng dataset quan sát trên một task D4RL khác ngoài Seaquest, đo khả năng ngoại suy (extrapolation) của DT một cách hệ thống.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Return-to-go	$\hat{R}_t = \sum_{t'=t}^{T} r_{t'}$	thay reward thô trong trajectory
Trajectory representation	$\tau = (\hat{R}_1, s_1, a_1, \ldots, \hat{R}_T, s_T, a_T)$	3 token/timestep
Self-attention	$z_i = \sum_j \text{softmax}(\langle q_i, k_j \rangle)_j v_j$	credit assignment trực tiếp, không bootstrap
Huấn luyện	CE (rời rạc) / MSE (liên tục) chỉ trên dự đoán a_t	không fit value function
Sinh hành động	điều kiện trên $\hat{R}$ mong muốn, giảm dần sau mỗi bước	"prompt" bằng return mong muốn
%BC	BC trên top X% timestep theo return	công cụ phân tích, không phải baseline thực tế
Context length K	$K = 1$ làm hỏng hiệu năng rõ rệt	giúp nhận diện "policy nào" sinh dữ liệu
Không cần pessimism	không tối ưu hoá tường minh qua value function học được	khác CQL (bài 20)

11. Kết luận ngắn

Decision Transformer cho thấy một bài toán offline RL có thể giải bằng đúng công cụ huấn luyện ngôn ngữ: một GPT học sinh chuỗi $(\hat{R}, s, a)$ theo thời gian, rồi được "mồi" bằng return-to-go mong muốn để sinh hành động đạt được mức đó, mà không cần bất kỳ Bellman backup, value function, hay cơ chế value-pessimism/policy-constraint nào. Kết quả trên Atari và D4RL Gym cạnh tranh hoặc vượt CQL, đặc biệt mạnh ở các tình huống cần credit assignment dài hạn hoặc reward thưa/trễ, những điểm yếu kinh điển của TD-learning. Đây là bằng chứng sớm và rõ ràng cho việc các kỹ thuật sequence-modeling của LLM có thể thay thế trực tiếp một phần lớn bộ công cụ RL truyền thống.

Link gốc: https://arxiv.org/abs/2106.01345

Bài 22

The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games (2021)

Tác giả: Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, Yi Wu (Tsinghua University; UC Berkeley; Shanghai Qi Zhi Institute). **Venue / arXiv:** arXiv:2103.01955 (v4, 04/11/2022). Paper tự ghi: "36th Conference on Neural Information Processing Systems (NeurIPS 2022) Track on Datasets and Benchmarks." **Nguồn:** https://arxiv.org/abs/2103.01955 **Chủ đề:** Đưa PPO (thuật toán on-policy, ít được dùng trong multi-agent RL (MARL) hợp tác so với các phương pháp off-policy) vào 4 benchmark MARL hợp tác phổ biến, cho thấy nó cạnh tranh tốt với baseline off-policy SOTA, và chỉ ra 5 yếu tố implementation/hyperparameter quyết định hiệu năng của nó trong bối cảnh multi-agent. **Vị trí trong lộ trình:** Đọc sau 00 (preface), 05 (GAE), 07 (PPO, nền tảng actor/critic loss dùng lại gần như nguyên bản ở đây). Nên đọc 08 (Implementation Matters) và 09 (What Matters in On-Policy RL) trước, vì bài này **trích dẫn trực tiếp cả hai** (Engstrom et al. và Andrychowicz et al.) là cảm hứng cho cách tiếp cận "tìm yếu tố implementation quan trọng bằng thực nghiệm có kiểm soát", lần này áp dụng cho multi-agent. Đây là bài duy nhất trong "giai đoạn 6: multi-agent" của lộ trình; sau bài này, lộ trình chuyển sang giai đoạn 7 (RLHF cho LLM, bài 23-28), vốn dùng lại đúng PPO loss nhưng cho một agent/single-policy.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Cooperative MARL (nhiều agent cùng hợp tác tối ưu một mục tiêu chung) có hai khung chính:

- **Centralized:** học trực tiếp một policy chung sinh ra hành động của mọi agent.
- **Decentralized:** mỗi agent tối ưu reward của riêng nó một cách độc lập; linh hoạt hơn (xử lý được general-sum game) nhưng có thể bất ổn ngay trên các ma trận game đơn giản vì môi trường "nhìn từ góc một agent" luôn thay đổi khi các agent khác cùng học (non-stationarity).
- **CTDE (Centralized Training, Decentralized Execution):** trung gian giữa hai khung trên. Các phương pháp CTDE trước đây thường dùng cấu trúc actor-critic với một **critic tập trung** (centralized critic) nhận thông tin toàn cục, còn actor chỉ dùng observation cục bộ khi thực thi. Một nhánh CTDE khác là **value-decomposition (VD)** (QMix, VDN...), biểu diễn Q-function chung là hàm của các Q-function cục bộ của từng agent, và đang nắm nhiều kết quả SOTA trên các benchmark MARL phổ biến.

Một niềm tin phổ biến trước bài này: PPO (on-policy) **kém hiệu quả về mẫu (sample efficiency)** hơn các phương pháp off-policy, cả ở single-agent continuous control (ví dụ SAC vượt PPO) và multi-agent (ví dụ COMA bị MADDPG/QMix vượt rõ trên particle-world và SMAC, theo [25] được paper trích). Thêm vào đó, các thực hành implementation/tuning PPO vốn tối ưu cho single-agent thường không chuyển dịch tốt sang multi-agent. Vài công trình đồng thời (concurrent): [7] cho thấy IPPO (PPO độc lập, decentralized) đạt tỉ lệ thắng cao trên một số SMAC map khó, nhưng vẫn kém QMix nói chung và chỉ giới hạn ở SMAC; [25] benchmark rộng nhiều thuật toán MARL và ghi nhận PPO-based method cạnh tranh tốt. Bài này mở rộng theo hướng toàn diện hơn: 4 benchmark hợp tác phổ biến, và phân tích sâu các yếu tố implementation/hyperparameter của PPO trong MARL (điều mà theo tác giả chưa được nghiên cứu kỹ ở mức độ này, đặc biệt trong bối cảnh multi-agent).

Dec-POMDP (Decentralized Partially Observable MDP) với shared reward, định nghĩa dùng trong bài:

$\langle S, A, O, R, P, n, \gamma \rangle$. S là không gian trạng thái; A là không gian hành động chung cho mỗi agent i; $o_i = O(s; i)$ là observation cục bộ của agent i tại trạng thái toàn cục s; $P(s'|s, A)$ là xác suất chuyển trạng thái với hành động chung $A = (a_1, \ldots, a_n)$ của n agent; $R(s, A)$ là hàm reward **chung** (shared) cho mọi agent; γ là discount factor. Mỗi agent dùng policy $\pi_\theta(a_i|o_i)$ tham số hóa bởi θ để sinh hành động từ observation cục bộ, và cùng tối ưu:

$$J(\theta) = \mathbb{E}_{A_t, s_t}\Big[\sum_t \gamma^t R(s_t, A_t)\Big]$$

2. Ý tưởng chính

Thay vì đề xuất thuật toán mới, bài này "hồi sinh" PPO cho MARL hợp tác: dùng cấu trúc actor-critic giống PPO single-agent, cho critic nhận **thông tin toàn cục** (CTDE) trong khi actor chỉ dùng observation cục bộ khi thực thi, và đo hiệu năng trên 4 benchmark MARL hợp tác phổ biến (MPE, SMAC, GRF, Hanabi) so với các baseline off-policy SOTA. Không cần kiến trúc hay thuật toán đặc thù theo domain, PPO-based method vẫn đạt hiệu năng cạnh tranh hoặc vượt trội với hiệu quả mẫu tương đương. Sau đó, bài xác định và phân tích **5 yếu tố implementation/hyperparameter** ảnh hưởng mạnh đến hiệu năng của PPO trong MARL, và đưa ra khuyến nghị cụ thể cho từng yếu tố.

3. Phương pháp chi tiết

3.1 MAPPO và IPPO

Cả hai học một policy π_θ và một value function $V_\phi(s)$, biểu diễn bằng hai mạng neural tách biệt. V_ϕ chỉ dùng trong huấn luyện (giảm phương sai gradient), nên có thể nhận thêm thông tin toàn cục không có trong observation cục bộ của agent, đúng theo cấu trúc CTDE.

- **MAPPO (Multi-Agent PPO):** PPO với **input toàn cục cho value function** (centralized value input).
- **IPPO (Independent PPO):** PPO với **input cục bộ cho cả policy và value function** (decentralized hoàn toàn).

Cả hai vẫn giả định các agent chia **reward chung** (cooperative setting), khác với PPO độc lập kiểu general-sum.

3.2 Hàm mục tiêu

Gọi B là batch size, n là số agent, $S[\cdot]$ là entropy chính sách, σ là hệ số entropy. Actor π_θ ánh xạ observation $o_i^{(k)}$ sang phân phối categorical (action rời rạc) hoặc Gaussian đa biến (action liên tục). Actor được huấn luyện để tối đa hóa:

$$L(\theta) = \frac{1}{Bn}\sum_{i=1}^{B}\sum_{k=1}^{n} \min\Big(r_{\theta,i}^{(k)} A_i^{(k)},\ \text{clip}(r_{\theta,i}^{(k)}, 1-\epsilon, 1+\epsilon) A_i^{(k)}\Big) \ +\ \sigma \cdot \frac{1}{Bn}\sum_{i=1}^{B}\sum_{k=1}^{n} S\big[\pi_\theta(o_i^{(k)})\big]$$

với $r_{\theta,i}^{(k)} = \dfrac{\pi_\theta(a_i^{(k)}|o_i^{(k)})}{\pi_{\theta_{old}}(a_i^{(k)}|o_i^{(k)})}$ và $A_i^{(k)}$ tính bằng GAE (bài 05). Critic $V_\phi : S \to \mathbb{R}$ (state có thể là agent-specific hoặc agent-agnostic, xem 3.4) được huấn luyện để tối thiểu hóa loss kiểu PPO value-clipping (như trong bài 08, mục 3.1 điểm 1):

$$L(\phi) = \frac{1}{Bn}\sum_{i=1}^{B}\sum_{k=1}^{n} \max\Big[\big(V_\phi(s_i^{(k)}) - \hat{R}_i^{(k)}\big)^2,\ \big(\text{clip}(V_\phi(s_i^{(k)}), V_{\phi_{old}}(s_i^{(k)}) - \epsilon, V_{\phi_{old}}(s_i^{(k)}) + \epsilon) - \hat{R}_i^{(k)}\big)^2\Big]$$

với $\hat{R}_i^{(k)}$ là reward-to-go đã chiết khấu. Nếu actor/critic là RNN, loss cộng thêm theo thời gian và huấn luyện bằng Backpropagation Through Time (BPTT).

3.3 Chi tiết implementation chung

- **Parameter sharing:** với agent đồng nhất (cùng không gian observation/action), chia sẻ tham số policy và value giữa các agent (trừ scenario Comm trong MPE, nơi hai agent có observation/action space khác nhau). Phụ lục C.2 cho thấy parameter sharing giúp ích rõ rệt (xem mục 5.7).
- **Thực hành chuẩn:** GAE (bài 05) kèm chuẩn hóa advantage, và value-clipping kiểu PPO (như trên).

3.4 Pseudo-code: Recurrent-MAPPO (Algorithm 1 của paper)

```
Initialize theta (actor), phi (critic) bang Orthogonal initialization
Set learning rate alpha
while step <= step_max:
    D = {}                                          # buffer du lieu
    for i = 1..batch_size:
        tau = []
        khoi tao h_0,pi^(1..n), h_0,V^(1..n)     # hidden state RNN cho actor/critic cua
        ↪  tung agent
        for t = 1..T:
            for agent a:
                p_t^a, h_t,pi^a = pi(o_t^a, h_{t-1,pi}^a; theta)
                u_t^a ~ p_t^a
                v_t^a, h_t,V^a  = V(s_t^a, h_{t-1,V}^a; phi)
            thuc hien hanh dong u_t, quan sat r_t, s_{t+1}, o_{t+1}
            tau += [s_t, o_t, h_t,pi, h_t,V, u_t, r_t, s_{t+1}, o_{t+1}]
        tinh advantage A_hat bang GAE tren tau, dung PopArt (chuan hoa)
        tinh reward-to-go R_hat tren tau, chuan hoa bang PopArt
        chia tau thanh cac doan (chunk) do dai L
        D = D union {cac chunk (tau, A_hat, R_hat) do dai L}
    for minibatch k = 1..K:
        lay minibatch ngau nhien tu D (du lieu cua moi agent)
        voi moi data chunk: reset hidden state RNN tu hidden state dau chunk
        Adam update theta tren L(theta)
        Adam update phi  tren L(phi)
```

Hai điểm đáng chú ý: (1) **PopArt** (một dạng chuẩn hóa value target chạy, tương tự "value function normalization" ở bài 09) được dùng để chuẩn hóa cả advantage và reward-to-go trước khi huấn luyện, rồi "giải chuẩn hóa" lại khi tính GAE; (2) dữ liệu được chia thành các **đoạn (chunk) độ dài** L để huấn luyện RNN bằng BPTT, và khi lấy mẫu minibatch, hidden state được reset lại từ hidden state đầu mỗi chunk (không giữ trạng thái xuyên suốt toàn bộ episode trong một lần cập nhật).

4. Thí nghiệm và kết quả chính

4 testbed: multi-agent particle-world environment (MPE), StarCraft II multi-agent challenge (SMAC), Google Research Football (GRF), Hanabi.

Baseline theo testbed:

- MPE: QMix, MADDPG.
- SMAC: QMix và các SOTA: QPlex, CWQMix, AIQMix, RODE.
- GRF: QMix và SOTA: CDS (reward nội tại), TiKick (kết hợp online fine-tuning với offline pretraining trên dữ liệu người).
- Hanabi: SAD (biến thể Q-learning), VDN.

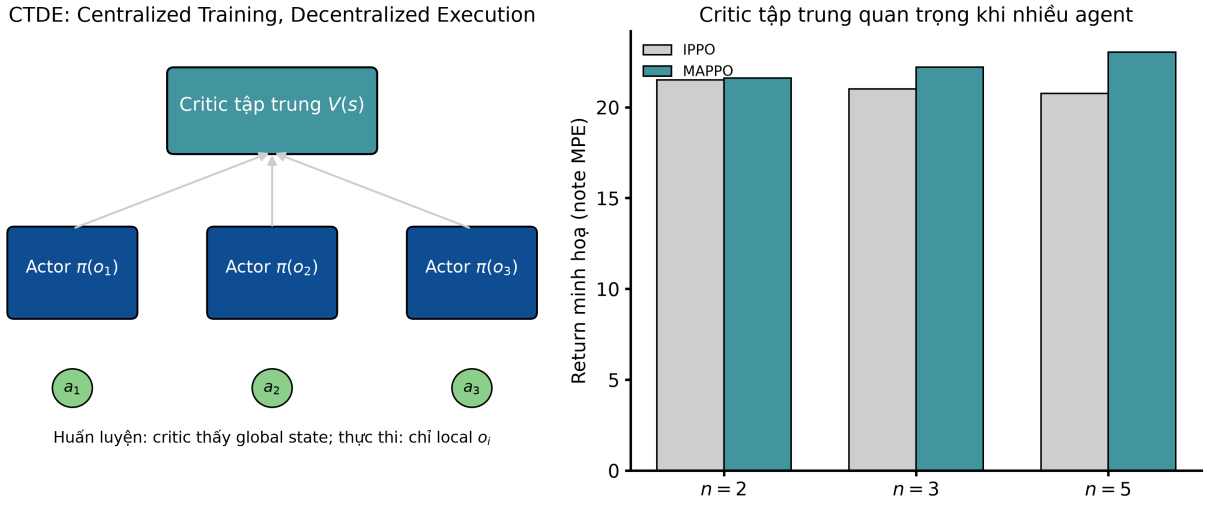
Thiết lập chung: để so sánh công bằng, tác giả tự cài lại và grid-search MADDPG/QMix với lưới hyperparameter **cùng kích thước** với lưới dùng cho MAPPO/IPPO (learning rate, tốc độ update target network, kiến trúc mạng, và các mẹo như value/reward normalization, target update cứng/mềm). Phần cứng: máy desktop 256 GB RAM, CPU 64-core, GPU GeForce RTX 3090.

4.1 MPE

3 task hợp tác: Spread (physical deception), Comm (cooperative communication, agent không đồng nhất nên không chia tham số), Reference (simple reference). Global state cho MAPPO/off-policy là nối (concatenate) observation cục bộ của mọi agent (vì MPE không có global state sẵn). Kết quả trung bình trên 10 seed: MAPPO đạt hiệu năng tương đương, thậm chí vượt các baseline off-policy; gần như giống QMix trên mọi task, vượt MADDPG ở Comm, dùng số bước môi trường tương đương. IPPO (không dùng thông tin toàn cục) cũng đạt hiệu năng tương đương hoặc vượt các phương pháp off-policy tập trung, dù hội tụ về return thấp hơn MAPPO một chút trên Comm và Reference.

4.2 SMAC (trích 7/23 map, trải theo 3 mức độ khó)

Đo median win rate qua 32 game đánh giá (median 10 win-rate cuối, 6 seed). "RODE"/"*MAPPO*" dùng cùng số bước huấn luyện với RODE; MAPPO/IPPO/QMix (không *) huấn luyện tới hội tụ hoặc tối đa 10M bước.



MAPPO CTDE: critic tập trung, actor theo o_i; lợi ích tăng khi nhiều agent (minh hoạ, tự tính)

Map (độ khó)	MAPPO(FP)	MAPPO(AS)	IPPO	QMix	RODE*
2m_vs_1z (Easy)	100.0(0.0)	100.0(0.0)	100.0(0.0)	95.3(5.2)	/
3s_vs_5z (Hard)	100.0(0.6)	99.2(1.4)	100.0(0.0)	98.4(2.4)	78.9(4.2)
5m_vs_6m (Hard)	89.1(2.5)	88.3(1.2)	87.5(2.3)	75.8(3.7)	71.1(9.2)
3s5z (Hard)	96.9(0.7)	96.9(1.9)	96.9(1.5)	88.3(2.9)	93.8(2.0)
MMM2 (Super Hard)	90.6(2.8)	87.5(5.1)	86.7(7.3)	87.5(2.6)	89.8(6.7)
3s5z_vs_3s6z (Super Hard)	84.4(34.0)	63.3(19.2)	82.8(19.1)	82.8(5.3)	96.8(25.11)
corridor (Super Hard)	100.0(1.2)	98.4(0.8)	98.4(3.1)	84.4(2.5)	65.6(32.1)

(Median win rate %, độ lệch chuẩn trong ngoặc; "/" = không có số liệu RODE. Bảng đầy đủ của bài báo có 23 map; đây là tập con tiêu biểu theo độ khó.) MAPPO/IPPO đạt hiệu năng mạnh trên đa số map, bằng/hơn QMix dù cùng số mẫu; MAPPO bằng/hơn RODE trên 10/14 map so sánh trực tiếp cùng số mẫu, và bắt kịp/vượt RODE ở gần mọi map khi dùng nhiều mẫu hơn.

4.3 Google Research Football (GRF)

6 scenario academy: 3v.1, Counterattack (CA) easy/hard, Corner, Pass-Shoot (PS), Run-Pass-Shoot (RPS). Do observation cục bộ đã chứa đầy đủ mô tả trạng thái môi trường, MAPPO = IPPO ở đây (không phân biệt). Reward: tổng reward dày (dense) của từng agent. Huấn luyện 25M bước (50M cho CA-hard và Corner), đo tỉ lệ thành công trung bình trên 100 rollout, trung bình 6 seed.

Scenario	MAPPO	QMix	CDS	TiKick
3v.1	88.03(1.06)	8.12(2.83)	76.60(3.27)	76.88(3.15)
CA (easy)	87.76(1.34)	15.98(2.85)	63.28(4.89)	/
CA (hard)	77.38(4.81)	3.22(1.60)	58.35(5.56)	73.09(2.08)
Corner	65.53(2.19)	16.10(3.00)	3.80(0.54)	33.00(3.01)
PS	94.92(0.68)	8.05(3.66)	94.15(2.54)	/
RPS	76.83(1.81)	8.08(4.71)	62.38(4.56)	79.12(2.06)

MAPPO vượt QMix rõ rệt ở mọi scenario, và vượt TiKick ở 4/5 scenario (dù TiKick có pretraining trên dữ liệu người).

4.4 Hanabi (2-5 người chơi, Hanabi-Full)

Mỗi agent không thấy bài của chính mình, phải suy luận ý định đối tác qua hành động. MAPPO cần global state bổ sung (thêm bài của chính agent vào observation cục bộ làm input cho value function); SAD/VDN chỉ dùng observation cục bộ (không có thêm thông tin toàn cục khi huấn luyện tập trung). Đánh giá trung bình trên hơn 10.000 game, trên ít nhất 3 seed, huấn luyện tối đa 10 tỷ bước môi trường.

Số người	Chỉ số	MAPPO	IPPO	SAD	VDN
2	Avg.	23.89(0.02)	24.00(0.02)	23.87(0.03)	23.83(0.03)
2	Best	24.23(0.01)	24.19(0.02)	24.01(0.01)	23.96(0.01)
3	Avg.	23.77(0.20)	23.25(0.33)	23.69(0.05)	23.71(0.06)
3	Best	24.01(0.01)	23.87(0.03)	23.93(0.01)	23.99(0.01)
4	Avg.	23.57(0.13)	22.52(0.37)	23.27(0.26)	23.03(0.15)
4	Best	23.71(0.01)	23.06(0.03)	23.81(0.01)	23.79(0.00)
5	Avg.	23.04(0.10)	20.75(0.56)	22.06(0.23)	21.28(0.12)
5	Best	23.16(0.01)	22.54(0.02)	23.01(0.01)	21.80(0.01)

MAPPO đạt kết quả tương đương hoặc hơn SAD/VDN ở hầu hết thiết lập. IPPO tương đương MAPPO ở 2 agent, nhưng **càng nhiều agent, MAPPO càng vượt IPPO rõ** (ví dụ 5 người: Avg. 23.04 so với 20.75), cho thấy critic tập trung (centralized critic) trở nên quan trọng hơn khi số agent tăng.

5. Phân tích / ablation: năm yếu tố implementation

Tất cả thí nghiệm ở mục này dùng MAPPO (PPO với centralized value) để nhất quán.

5.1 Chuẩn hóa value function (Value Normalization)

Dùng ước lượng mean/std chạy để chuẩn hóa target của value network (giống PopArt); khi tính GAE, dùng lại running average để "giải chuẩn hóa" output. **Suggestion 1: luôn dùng value normalization; hầu như không bao giờ gây hại, thường cải thiện hiệu năng cuối rõ rệt.** Ví dụ: MPE Spread có return từ dưới -200 đến 0, value normalization là yếu tố sống còn; trên SMAC cũng cải thiện hiệu năng hoặc giảm phương sai ở một số map.

5.2 Biểu diễn input cho value function

4 dạng global state (minh họa trên SMAC, nơi có nhiều lựa chọn global state khác nhau):

- **CL** (Concatenation of Local observations): nối observation cục bộ của mọi agent; chiều dữ liệu tăng theo số agent, có thể thiếu thông tin toàn cục không agent nào quan sát được.
- **EP** (Environment-Provided global state): trạng thái toàn cục do môi trường cung cấp sẵn, chứa thông tin chung nhưng **thiếu thông tin đặc thù agent** (agent ID, hành động khả dụng, khoảng cách tương đối...).
- **AS** (Agent-Specific global state): nối EP với o_i (observation cục bộ của agent i), cho value function cả thông tin toàn cục và cục bộ.
- **FP** (Feature-Pruned AS): như AS nhưng loại bỏ các đặc trưng trùng lặp giữa EP và o_i, giảm chiều input dư thừa.

Kết quả: CL (chiều cao hơn hẳn) kém hiệu quả, đặc biệt trên map nhiều agent. EP mạnh hơn nhưng yếu trên map khó (do thiếu thông tin cục bộ). AS và FP đều mạnh, FP chỉ hơn AS trên một vài map. **Suggestion 2: khi có thể, đưa cả đặc trưng cục bộ đặc thù agent và đặc trưng toàn cục vào input value function; đồng thời kiểm tra các đặc trưng này không làm tăng chiều input một cách không cần thiết (dư thừa).**

5.3 Cách dùng dữ liệu huấn luyện (số epoch, số minibatch)

PPO dùng importance sampling để tái sử dụng dữ liệu, thường chia batch lớn thành 32-64 minibatch và train hàng chục epoch trong continuous control single-agent. Trong MARL, **hiệu năng giảm nếu tái sử dụng dữ liệu quá nhiều lần**: 15 epoch cho task dễ, 10 hoặc 5 cho task khó; nhiều nhất 2 minibatch, phần lớn trường hợp dùng cả batch. Giả thuyết: non-stationarity (agent khác cùng đổi policy) khiến ít epoch hơn ổn định hơn, và nhiều dữ liệu hơn mỗi lần ước lượng gradient (ít minibatch) cho hiệu năng tốt hơn.

Thực nghiệm SMAC: 15 epoch khiến MAPPO học ra policy dưới tối ưu nhất quán (tệ nhất ở MMM2, Corridor); 5-10 epoch tốt. Minibatch $\in \{1, 2, 4\}$: 4 minibatch khiến MAPPO **không giải được map nào**; 1 minibatch tốt nhất trên 22/23 map. MPE: task dễ không nhạy; Spread (khó) lặp lại xu hướng SMAC. **Suggestion 3: tối đa 10 epoch cho môi trường khó, 15 cho môi trường dễ; tránh chia minibatch.**

5.4 Ngưỡng clipping PPO

ϵ dùng chung cho clip chính sách và clip value; nhỏ hơn giới hạn thay đổi policy mỗi lần cập nhật nhiều hơn (ổn định hơn, có thể chậm hơn). Single-agent dùng $\epsilon = 0.2$ phổ biến (bài 07, 09). Thực nghiệm SMAC: $\epsilon = 0.05$ làm chậm tốc độ học ở một số map khó (MMM2, 3s5z_vs_3s6z) nhưng hiệu năng cuối cao và **ổn định hơn**; ϵ lớn (0.2, 0.3, 0.5) thường dưới tối ưu. **Suggestion 4: giữ ϵ dưới 0.2; tune trong khoảng đó như trade-off ổn định/tốc độ hội tụ.**

5.5 Kích thước batch

Batch lớn hơn cho gradient chính xác hơn nhưng tốn compute/memory. Thực nghiệm (SMAC, GRF): có **ngưỡng batch size tới hạn** mỗi task; dưới ngưỡng hiệu năng kém, vượt ngưỡng không cải thiện thêm và **có thể giảm hiệu quả mẫu**. **Suggestion 5: dùng batch đủ lớn để đạt hiệu năng tốt nhất, rồi tune lại cho sample-efficiency.**

5.6 Death masking và parameter sharing (bổ sung, Phụ lục C.2-C.3)

Death masking: khi agent SMAC "chết" giữa episode, observation cục bộ thành vector 0 (trừ agent ID) nhưng global state vẫn còn đặc trưng khác. 4 cách xử lý value-state agent chết: (1) vector 0 có agent ID ("death masking"); (2) giữ global state không-0; (3) bỏ transition sau khi chết; (4) vector 0 không agent ID. Kết quả: (1) vượt rõ (2)(3), và agent ID quan trọng hơn (4) trên map có vai trò (role) khác nhau.

Parameter sharing: MAPPO (chia tham số) vượt rõ MAPPO-Ind (mạng riêng mỗi agent) trên các map chọn lọc, ví dụ 6h_vs_8z: 88.3(3.7) so với 11.4(2.5); 5m_vs_6m: 89.1(2.5) so với 44.4(2.9). Củng cố quyết định

dùng parameter sharing mặc định cho agent đồng nhất.

6. Giới hạn

Tác giả nêu:

- Toàn bộ benchmark dùng **action space rời rạc**, đều là **hợp tác** (cooperative), và phần lớn trường hợp agent **đồng nhất** (homogeneous); chưa kiểm tra game cạnh tranh (competitive), action space liên tục, hay agent không đồng nhất.
- Bài mang tính **thực nghiệm**, không phân tích lý thuyết cho các tính chất của PPO trong MARL; các gợi ý suy ra từ quan sát thực nghiệm (ví dụ lý do non-stationarity cho Suggestion 3, 4) là giả thuyết (hypothesize), chưa chứng minh hình thức.

Nhận xét:

- Baseline off-policy (MADDPG, QMix) được tác giả **tự cài lại** (không phải code gốc), tuy xác nhận khớp/vượt số liệu bài báo gốc, nhưng vẫn là một lựa chọn implementation cụ thể.
- Số seed khác nhau giữa benchmark (10 MPE, 6 SMAC/GRF, tối thiểu 3 Hanabi), độ tin cậy so sánh không đồng nhất.
- Suggestion 4 ($\epsilon < 0.2$) **nhỏ hơn** khuyến nghị $\epsilon = 0.25$ của bài 09 (single-agent); không mâu thuẫn trực tiếp (domain khác), nhưng gợi ý non-stationarity multi-agent cần trust region chặt hơn.
- Value normalization ở đây "hầu như không bao giờ gây hại", trong khi bài 09 thấy nó **gây hại rõ trên Walker2d** (single-agent); không nên coi "luôn an toàn" là quy luật tuyệt đối.
- Compute cụ thể mỗi thí nghiệm (GPU-hour...) không nêu chi tiết theo benchmark, chỉ nêu cấu hình máy chung.

7. Liên hệ với các bài khác

- **05 GAE, 07 PPO:** actor/critic loss ở đây gần như giữ nguyên PPO single-agent (bài 07), chỉ thêm input toàn cục cho critic (CTDE) và parameter sharing; advantage vẫn dùng GAE (bài 05).
- **08 Implementation Matters, 09 What Matters in On-Policy RL:** bài này trích dẫn trực tiếp cả hai làm nền cho phương pháp "tìm yếu tố implementation quan trọng bằng thực nghiệm hệ thống", áp dụng sang MARL. Có điểm tương đồng (chuẩn hóa value quan trọng) và điểm khác biệt đáng chú ý (dấu hiệu tác động của value normalization ngược nhau giữa hai domain; ngưỡng ϵ khuyến nghị khác nhau) cần đọc chéo cẩn thận.
- **Các bài sau trong lộ trình (23-28, RLHF/PPO cho LLM):** dùng lại chính xác PPO loss dạng clip (bài 07), lần này cho một LLM/policy đơn (không multi-agent), nhưng các mẹo như value normalization, advantage normalization, số epoch PPO vẫn là câu hỏi tương tự cần tối ưu riêng cho domain ngôn ngữ (xem đặc biệt bài 28).
- **54 (The Landscape of Agentic RL for LLMs, survey):** nếu một hệ thống agent LLM được huấn luyện như nhiều "agent" tương tác (multi-agent LLM), các khái niệm CTDE, centralized critic, parameter sharing ở bài này là tiền đề trực tiếp để hiểu các thiết kế RL cho multi-agent LLM quy trình.

8. Ứng dụng thực tế (suy luận)

- **Khi huấn luyện nhiều agent/nhiều policy hợp tác bằng RL** (kể cả multi-agent LLM: nhiều "agent" LLM cùng giải một nhiệm vụ, chia sẻ reward chung), 5 khuyến nghị ở mục 5 là điểm khởi đầu hợp lý: chuẩn hóa value, cho critic thấy cả thông tin cục bộ + toàn cục (nếu khả thi), giảm số epoch/không chia minibatch (vì non-stationarity khi nhiều policy cùng thay đổi), giữ ϵ clip nhỏ hơn mức mặc định single-agent, và tune batch size theo ngưỡng tới hạn của task thay vì chọn bừa.
- **Parameter sharing** là một "quick win" rẻ tiền khi các agent đồng nhất về vai trò (cùng loại input/output): giảm số tham số, tăng hiệu năng rõ rệt theo Phụ lục C.2; nên là lựa chọn mặc định trước khi thử kiến

trúc per-agent riêng.

- **Death masking** (xử lý agent "chết"/ngừng hoạt động giữa episode bằng cách gán state 0 có giữ ID) là một chi tiết kỹ thuật nhỏ nhưng có ích cho bất kỳ môi trường multi-agent nào có agent bị loại khỏi vòng chơi giữa episode (y lab self-play, game nhiều người chơi).
- Lưu ý khi chuyển kinh nghiệm từ đây sang RL cho LLM đơn-agent (RLHF/RLVR, bài 07/28/43): không nên copy thẳng $\epsilon < 0.2$ hay "1 minibatch, 5-10 epoch" làm chuẩn, vì đây là khuyến nghị rút ra cho action rời rạc nhiều-agent trên game cụ thể (SMAC, GRF, Hanabi, MPE); nguyên tắc chuyển giao được là **cách tiếp cận đo đạc** (ablation có kiểm soát từng yếu tố), không phải giá trị số cụ thể.

9. Thí nghiệm gợi ý

1. Trên MPE Spread, so sánh MAPPO có/không value normalization (PopArt-style), đo trực tiếp biên độ cải thiện khi return nằm trong khoảng rộng (âm sâu đến 0) như paper mô tả.
2. Xây một Dec-POMDP nhỏ, agent không đồng nhất, test 2 kiểu input value: chỉ global state kiểu EP so với AS (EP + observation cục bộ); kiểm tra xem thêm thông tin cục bộ có giúp ích như trên SMAC hay không.
3. Sweep số epoch $\in \{5, 10, 15\}$ và số minibatch $\in \{1, 2, 4\}$ trên một môi trường multi-agent tùy chọn (không nhất thiết SMAC), xác nhận xu hướng "ít epoch hơn, không chia minibatch" có lặp lại ngoài 2 benchmark của bài báo.
4. Nếu đang xây hệ multi-agent LLM huấn luyện bằng PPO: so sánh chia sẻ tham số giữa các "agent" LLM (cùng checkpoint, prompt khác nhau theo vai) với huấn luyện riêng từng agent, đo khoảng cách hiệu năng tương tự Bảng 4 phụ lục (MAPPO vs MAPPO-Ind).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Dec-POMDP	$\langle S, A, O, R, P, n, \gamma \rangle$, reward $R(s, A)$ chung	định nghĩa setting hợp tác
CTDE	critic dùng info toàn cục khi train, actor chỉ dùng info cục bộ khi thực thi	khung chung của MAPPO
MAPPO	PPO, critic nhận global state	centralized value, decentralized policy
IPPO	PPO, policy + critic chỉ dùng local obs	hoàn toàn decentralized, vẫn share reward
Actor loss	$\min(r_\theta A, \text{clip}(r_\theta, 1 \pm \epsilon)A) + \sigma\, S[\pi_\theta]$	giống PPO 1-agent (bài 07), cộng entropy bonus
Critic loss	$\max[(V - \hat{R})^2, (\text{clip}(V, V_{old} \pm \epsilon) - \hat{R})^2]$	value-clipping kiểu PPO (bài 08)
Value input AS/FP	AS = EP + o_i; FP = AS bỏ đặc trưng trùng	Suggestion 2
Suggestion 1	luôn dùng value normalization	PopArt-style
Suggestion 3	≤ 10 epoch (khó)/15 epoch (dễ); không chia minibatch	do non-stationarity
Suggestion 4	$\epsilon < 0.2$ cho cả policy và value clip	nhỏ hơn mặc định single-agent
Suggestion 5	batch size đủ lớn, rồi tune cho sample-efficiency	có ngưỡng tới hạn theo task
Death masking	value-state agent chết = vector 0 + agent ID	tốt hơn giữ state cũ hoặc bỏ transition
Parameter sharing	agent đồng nhất chia tham số actor/critic	cải thiện rõ rệt (Phụ lục C.2)

11. Kết luận ngắn

Bài báo cho thấy PPO, khi được trang bị centralized critic (CTDE) và vài chỉnh sửa implementation đơn giản, đạt hiệu năng cạnh tranh hoặc vượt các phương pháp off-policy SOTA (QMix, MADDPG, SAD, VDN, RODE...) trên 4 benchmark MARL hợp tác lớn (MPE, SMAC, GRF, Hanabi), mà không cần kiến trúc hay thuật toán đặc thù domain. Năm yếu tố implementation (chuẩn hóa value, biểu diễn input value, số epoch/minibatch, ngưỡng clipping, batch size) được xác định là chìa khóa quyết định hiệu năng này trong bối cảnh multi-agent, với một số khuyến nghị (như ϵ clip nhỏ hơn, ít epoch hơn) khác với thực hành chuẩn single-agent, phản ánh đặc thù non-stationarity khi nhiều agent cùng học.

Link gốc: https://arxiv.org/abs/2103.01955

Bài 23

Deep Reinforcement Learning from Human Preferences (2017)

Tác giả: Paul F Christiano (OpenAI), Jan Leike (DeepMind), Tom B Brown, Miljan Martic (DeepMind), Shane Legg (DeepMind), Dario Amodei (OpenAI).
Venue / arXiv: arXiv:1706.03741 (v4, 17/02/2023). Bài báo không nêu venue trong bản đã đọc (các bài sau, ví dụ bài 24, dẫn bài này là "Advances in Neural Information Processing Systems, 2017", tức NeurIPS 2017, nhưng đó là trích dẫn của bài khác, không phải dòng venue trong chính bài báo này).
Nguồn: https://arxiv.org/abs/1706.03741
Chủ đề: Học một hàm reward từ so sánh cặp (pairwise preference) giữa các đoạn trajectory do con người đưa ra, dùng mô hình Bradley-Terry; huấn luyện policy bằng RL chuẩn (A2C/TRPO) trên reward học được; ba quá trình (policy, chọn truy vấn, học reward) chạy bất đồng bộ. Đây là bài khởi nguồn cho toàn bộ dòng RLHF (Reinforcement Learning from Human Feedback).
Vị trí trong lộ trình: Đọc sau bài 00 (nền tảng: MDP, policy gradient, REINFORCE, GAE, trust region, PPO), bài 04 (TRPO) và bài 06 (A3C), vì bài này coi thuật toán RL (TRPO cho robot, A2C cho Atari) là "hộp đen" không đổi, chỉ thay nguồn reward. Đây là bài đầu tiên trong nhóm RLHF (giai đoạn 7) của lộ trình; các bài sau (24 Fine-tuning LM, 25 Learning to summarize, 26 Constitutional AI, 27 Reward model overoptimization, 28 N+ implementation details) lặp lại đúng ý tưởng Bradley-Terry này trên mô hình ngôn ngữ ngày càng lớn. Xem thêm llm 07 (InstructGPT) là ứng dụng RLHF đầy đủ (SFT → RM → PPO) lên GPT-3, dùng chính công thức mô hình phần thưởng ở mục 3.1 dưới đây.
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Vấn đề. Nhiều tác vụ RL thực tế (dọn bàn, đập trứng bằng robot) không có cách tự nhiên để viết ra một hàm reward đúng ý con người: một reward tay viết gần đúng thường bị policy "khai thác" (tối ưu đúng công thức nhưng sai mục đích). Đây chính là nguồn gốc của khái niệm **misalignment** (lệch mục tiêu) mà paper trích từ các thảo luận an toàn AI (Bostrom, Russell, Amodei et al. "Concrete Problems in AI Safety").

Hai hướng cũ không đủ. (i) *Inverse RL* (Ng & Russell, 2000): suy ra reward từ bản demo, nhưng cần có bản demo. (ii) *Imitation learning*: nhân bản hành vi demo trực tiếp. Cả hai không áp dụng được cho hành vi mà con người khó demo (ví dụ điều khiển robot nhiều bậc tự do, hình thái rất khác người).

Hướng của bài này. Để con người cho *phản hồi* trên hành vi hiện tại của hệ thống, nhưng phản hồi trực tiếp làm reward thì quá đắt cho RL cần hàng trăm, hàng nghìn giờ kinh nghiệm. Paper giải quyết bằng cách **học một reward function từ phản hồi người**, rồi tối ưu reward đó bằng RL thường. Bốn tiêu chí đặt ra ở phần giới thiệu:

1. giải được tác vụ mà ta chỉ *nhận ra* hành vi mong muốn, không cần *demo* được nó,
2. cho phép người dùng không chuyên dạy agent,
3. scale lên bài toán lớn,
4. tiết kiệm phản hồi người.

Khái niệm nền cần biết trước:

- **Trajectory segment** $\sigma = ((o_0, a_0), \ldots, (o_{k-1}, a_{k-1})) \in (O \times A)^k$: một đoạn ngắn quan sát-hành động, ở đây dài 1-2 giây.
- **Preference** $\sigma^1 \succ \sigma^2$: người đánh giá ưu tiên đoạn 1 hơn đoạn 2. Nhãn thật ra là một phân phối μ trên $\{1, 2\}$: dồn hết khối lượng cho lựa chọn được chọn, uniform nếu "tie" (ngang nhau), và nhãn bị loại khỏi tập dữ liệu nếu người đánh giá chọn "không so sánh được".
- **Bradley-Terry model / Luce-Shephard choice rule**: mô hình thống kê cổ điển biến một "điểm số" ẩn thành xác suất thắng trong so sánh cặp, giống cách chênh lệch điểm Elo giữa hai kỳ thủ cờ vua ước lượng xác suất thắng của một bên.
- **A2C / TRPO**: các thuật toán policy-gradient on-policy dùng làm "lõi RL" (xem bài 04, bài 06); bài này không đề xuất thuật toán RL mới, chỉ thay nguồn reward mà các thuật toán đó tối ưu.

Công trình liên quan được bài báo nêu: Akrour et al. (2012, 2014) cùng ý tưởng nhưng trên miền liên tục vài bậc tự do và giả định reward tuyến tính theo đặc trưng tay thiết kế (bài báo này dùng mạng nơ-ron, miền phức tạp hơn nhiều, dùng ~2 bậc độ lớn so sánh hơn nhưng tốn ít hơn 1 bậc độ lớn thời gian người, theo bài báo). Wilson et al. (2012): giả định reset được môi trường về trạng thái tuỳ ý và chỉ thử nghiệm với phản hồi giả lập (synthetic), không phải người thật. TAMER (Knox & Stone): cũng học reward từ người nhưng cho tác vụ đơn giản hơn nhiều.

2. Ý tưởng chính

Hệ thống duy trì đồng thời một **policy** $\pi : O \to A$ và một **ước lượng reward** $\hat{r} : O \times A \to \mathbb{R}$, cả hai là mạng nơ-ron, cập nhật bởi ba quá trình chạy **bất đồng bộ** (Hình 1 của bài báo):

1. π tương tác với môi trường, được cập nhật bởi một thuật toán RL chuẩn để tối đa hoá tổng reward dự đoán $r_t = \hat{r}(o_t, a_t)$.
2. Lấy các cặp đoạn (σ^1, σ^2) từ các trajectory mới sinh ra ở bước 1, gửi cho người so sánh, ưu tiên chọn cặp mà ước lượng hiện tại *bất đồng* nhiều nhất (dùng ensemble).
3. Tham số của $\hat{r}$ được tối ưu bằng học có giám sát để khớp với toàn bộ cơ sở dữ liệu so sánh đã thu thập.

Vì so sánh (A tốt hơn B) dễ cho người đưa ra nhất quán hơn điểm tuyệt đối, và vì chỉ cần phản hồi trên các *đoạn clip ngắn* (không cần toàn bộ episode), lượng phản hồi người cần thiết giảm xuống dưới 1% số bước tương tác của agent, đủ rẻ để áp dụng cho các bài toán RL hiện đại (Atari, MuJoCo) chỉ với 15 phút đến 5 giờ phản hồi người.

3. Phương pháp chi tiết

3.1 Dự đoán sở thích (Bradley-Terry)

Coi $\hat{r}$ là một "reward ẩn" giải thích phán đoán con người: xác suất người chọn đoạn σ^1 tỉ lệ hàm mũ với tổng reward dự đoán trên cả đoạn:

$$\hat{P}\left[\sigma^1 \succ \sigma^2\right] = \frac{\exp \sum_t \hat{r}(o_t^1, a_t^1)}{\exp \sum_t \hat{r}(o_t^1, a_t^1) + \exp \sum_t \hat{r}(o_t^2, a_t^2)}$$

và chọn $\hat{r}$ để tối thiểu cross-entropy giữa dự đoán này và nhãn người:

$$\text{loss}(\hat{r}) = -\sum_{(\sigma^1, \sigma^2, \mu) \in D} \Big[\mu(1) \log \hat{P}[\sigma^1 \succ \sigma^2] + \mu(2) \log \hat{P}[\sigma^2 \succ \sigma^1]\Big]$$

Đây là mô hình Bradley-Terry (1952), trường hợp đặc biệt của luật chọn Luce-Shephard. Không có chiết khấu theo thời gian trong tổng reward của đoạn (paper ghi nhận điều này ở footnote, coi như con người "không

phân biệt" thời điểm trong đoạn clip; có thể dùng chiết khấu tường minh nhưng bài không làm).

3.2 Ba mẹo kỹ thuật (mục 2.2.3 của paper)

- **Ensemble:** huấn luyện (mặc định) 3 predictor độc lập, mỗi cái trên một mẫu bootstrap (lấy $|D|$ bộ ba với lặp lại) từ D; $\hat{r}$ cuối = trung bình của các predictor sau khi **chuẩn hoá độc lập từng cái** (mean 0, std cố định).
- **Chống overfit:** giữ $1/e$ dữ liệu làm validation riêng cho mỗi predictor; dùng L_2 regularization, chỉnh hệ số để validation loss nằm trong khoảng 1.1-1.5 lần training loss; một số miền còn dùng thêm dropout.
- **Mô hình hoá lỗi con người:** giả định có 10% khả năng người trả lời ngẫu nhiên đều (uniform), vì xác suất mắc lỗi của người không giảm về 0 khi chênh lệch reward trở nên rất lớn. Paper chỉ nêu giả định này bằng lời; công thức mixture cụ thể dưới đây là **diễn giải của phần phân tích**:

$$\hat{P}_{\text{nhiễu}} = 0.9\,\hat{P} + 0.1 \times 0.5 = 0.9\,\hat{P} + 0.05$$

3.3 Chọn truy vấn (query selection, mục 2.2.4)

Lấy mẫu một lượng lớn cặp đoạn (gấp 10 lần số cặp cuối cùng sẽ hỏi người), mỗi predictor trong ensemble dự đoán đoạn nào được ưu tiên, rồi **chọn những cặp mà dự đoán giữa các predictor bất đồng (variance) cao nhất** để gửi cho người. Paper tự gọi đây là "xấp xỉ thô" (crude approximation) cho uncertainty, lý tưởng hơn sẽ là chọn theo giá trị thông tin kỳ vọng (expected value of information, để lại cho công việc tương lai).

3.4 Chuẩn hoá reward và cắm vào RL

$\hat{r}$ được chuẩn hoá về mean 0, std cố định trước khi đưa vào thuật toán RL (A2C cho Atari theo kiến trúc Mnih et al. 2015, TRPO cho MuJoCo). Vì $\hat{r}$ thay đổi theo thời gian (non-stationary), tác giả ưu tiên các phương pháp policy-gradient chịu được reward thay đổi, và phải **tăng hệ số entropy bonus của TRPO** (0.01, riêng swimmer 0.001) để đảm bảo đủ khám phá khi trust region không còn là cơ chế khám phá đủ tốt với reward không cố định.

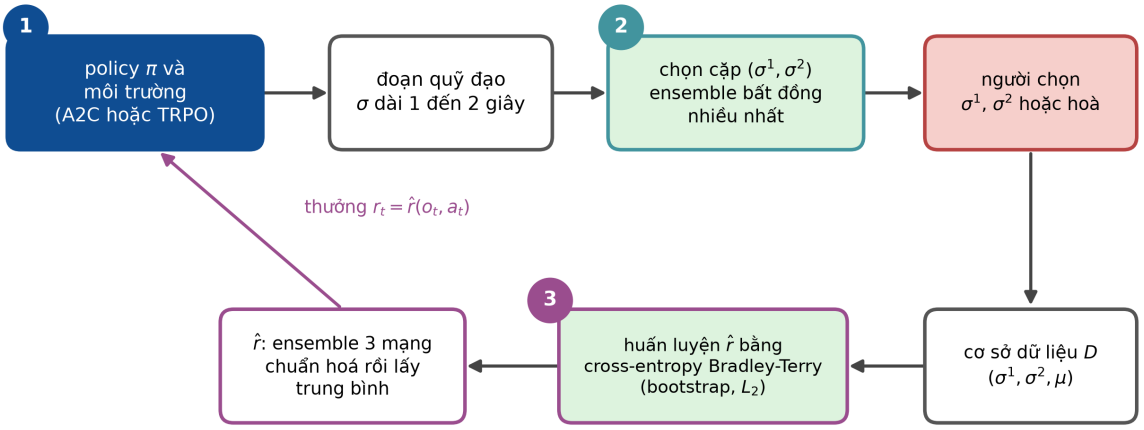
```
Khởi tạo policy pi, mạng reward r_hat (random); D = {}  (CSDL so sánh)
Chạy đồng thời vô hạn:

  [Quá trình 1: tối ưu policy]
  rollout trajectories tau_1..tau_i từ pi
  reward_t = normalize( r_hat(o_t, a_t) )
  pi <- RL_update(pi, reward)        # A2C (Atari) hoặc TRPO (MuJoCo)

  [Quá trình 2: chọn truy vấn]
  lấy ~10x số cặp đoạn (sigma1, sigma2) độ dài k từ các tau mới
  mỗi predictor trong ensemble dự đoán đoạn được ưu tiên
  chọn các cặp có variance dự đoán cao nhất giữa các predictor
  gửi cho người -> nhận mu -> D <- D + {(sigma1, sigma2, mu)}

  [Quá trình 3: học reward]
  với mỗi predictor m trong ensemble:
      D_m ~ Bootstrap(D)                      # lấy lại có hoàn lại
      cập nhật r_hat_m bằng gradient descent trên loss(r_hat_m; D_m)   # + L2, dropout
  r_hat <- trung bình các r_hat_m sau khi chuẩn hoá từng cái
```

Cách đọc hình: policy tương tác với môi trường bằng reward dự đoán, trong khi các cặp đoạn quỹ đạo được chọn theo độ bất đồng của ensemble để đưa cho người so sánh. Nhãn được lưu vào cơ sở dữ liệu rồi dùng để huấn luyện lại reward model bằng cross entropy kiểu Bradley Terry. Reward mới lại được đưa trở về policy, khép thành vòng lặp.



Ba quá trình chạy đan xen, không có bước huấn luyện reward một lần ở đầu; dùng ít hơn 1% số bước tương tác làm nhãn.

Hình 53: Ba quá trình chạy không đồng bộ: policy, người so sánh và huấn luyện reward model (minh hoạ, tự tính)

3.6 Ví dụ nhỏ (tự tính): hai lựa chọn = sigmoid của hiệu số

Với $K = 2$, công thức softmax ở mục 3.1 thu gọn thành sigmoid:

$$\hat{P}[\sigma^1 \succ \sigma^2] = \frac{e^{r^1}}{e^{r^1} + e^{r^2}} = \frac{1}{1 + e^{-(r^1 - r^2)}} = \sigma(r^1 - r^2)$$

(với $r^i = \sum_t \hat{r}(o_t^i, a_t^i)$). Đây chính là cầu nối tới loss RM dạng $-\log \sigma(r_w - r_l)$ dùng ở InstructGPT (llm 07) và các bài 25/26: Bradley-Terry 2 lựa chọn **chính là** loss đó, chỉ viết lại qua sigmoid thay vì softmax.

Ví dụ số (tự tính): nếu tổng reward dự đoán của đoạn 1 là 2.0 và đoạn 2 là 1.0:

$$\hat{P}[\sigma^1 \succ \sigma^2] = \sigma(2.0 - 1.0) = \sigma(1.0) = \frac{1}{1 + e^{-1}} \approx 0.731$$

$r^1 - r^2$	$\hat{P}[\sigma^1 \succ \sigma^2]$	loss nếu người chọn σ^1
+2.0	0.881	0.127
+1.0	0.731	0.313
0.0	0.500	0.693
−1.0	0.269	1.313

(cột loss = $-\log \hat{P}$, tự tính, cùng dạng bảng số với các bài RM sau trong lộ trình).

4. Thí nghiệm và kết quả chính

Hai miền: 8 tác vụ robot mô phỏng MuJoCo (gồm Hopper, Walker, Swimmer, Cheetah, Ant, Reacher, Pendulum, double-pendulum; các reward dạng bậc hai của khoảng cách/vị trí/vận tốc bị loại bỏ các thành phần "lộ" thông tin tác vụ, ví dụ phạt torque vì người giám sát không thấy được torque), và 7 game Atari (BeamRider,

Breakout, Enduro, Pong, Qbert, SeaQuest, SpaceInvaders, cùng bộ game với Mnih et al. 2013). Màn hình điểm số bị che (constant màu đen) để buộc reward phải học từ động lực trò chơi, không đọc trực tiếp điểm.

Thiết lập truy vấn: đoạn clip 1-2 giây; MuJoCo dài 1.5s (15-60 bước tuỳ tác vụ); Atari dài 25 bước (~1.7s với frameskip). Contractor trả lời trung bình 3-5 giây/truy vấn, nên tổng thời gian người cho mỗi thí nghiệm là 30 phút đến 5 giờ.

MuJoCo (Hình 2): với 700 truy vấn người, agent học từ phản hồi gần khớp RL học trực tiếp trên reward thật trên cả 8 tác vụ. Với 1.400 nhãn giả lập (synthetic, từ oracle biết trước reward thật), agent **vượt nhẹ** RL-trên-reward-thật, tác giả cho rằng reward học được có hình dạng (shaping) tốt hơn. Phản hồi người thật dao động từ bằng một nửa đến bằng hiệu quả của phản hồi giả lập; trên Ant, phản hồi người (được dặn ưu tiên "robot đứng thẳng") vượt hẳn phản hồi giả lập vì đó là một dạng reward shaping tốt mà hàm reward tay viết không có.

Atari (Hình 3), 5.500 truy vấn người so với 350/700/1400 truy vấn giả lập:

Game	Kết quả với nhãn giả lập	Kết quả với nhãn người thật
BeamRider, Pong	khớp/gần khớp RL chỉ với 3.300 nhãn	tương tự hoặc hơi kém giả lập cùng số nhãn
Seaquest, Qbert	cuối cùng gần RL nhưng học chậm hơn	Qbert: không vượt qua được màn 1 (clip ngắn khó hiểu)
SpaceInvaders, Breakout	không bao giờ khớp RL nhưng vẫn cải thiện đáng kể	SpaceInvaders qua màn 1; Breakout đạt điểm 20, tới 50 nếu có nhiều nhãn hơn
Enduro	khó học bằng A3C (khó vượt xe qua khám phá ngẫu nhiên)	**vượt** A3C-trên-reward-thật vì người thưởng mọi tiến bộ bám đuôi xe, tạo shaping tốt; so sánh được với DQN

Nhìn chung phản hồi người thật thường tương đương hoặc kém nhẹ phản hồi giả lập cùng số nhãn, nhưng thường bằng phản hồi giả lập với **ít hơn 40% số nhãn**. Paper cho rằng khoảng cách này (khi có) chủ yếu do lỗi gán nhãn của người, sự không nhất quán giữa các contractor khác nhau cùng gán nhãn một lượt chạy, và tốc độ gán nhãn không đều của contractor (có thể dồn nhãn vào một vùng hẹp của không gian trạng thái), không phải một nguyên nhân duy nhất.

Hành vi mới (không có reward có sẵn):

Hành vi	Số truy vấn	Thời gian huấn luyện
Hopper lộn ngược (backflip) liên tiếp, đáp đúng	900	dưới 1 giờ
Half-Cheetah tiến bằng một chân	800	dưới 1 giờ
Enduro: bám ngang hàng với xe khác	~1.300 (+4 triệu khung hình)	không nêu

Chi phí tính toán (footnote 6): máy ảo 16 CPU + 1 GPU Nvidia K80, ~700 USD/tháng trên GCE; huấn luyện Atari ~1 ngày ~25 USD compute; 5.000 nhãn tương đương ~5 giờ lao động người, ~36 USD theo lương tối thiểu Mỹ. Tác giả nhận xét chi phí tính toán đã *tương đương* chi phí nhãn người, tức đang chạm điểm "giảm lợi ích cận biên" (diminishing returns) của việc tiếp tục giảm số nhãn cần.

5. Phân tích / ablation

Sáu biến thể được so sánh (Hình 5 cho MuJoCo, Hình 6 cho Atari, cùng dùng 700/5.500 nhãn giả lập cho mỗi miền):

1. **random queries**: chọn cặp ngẫu nhiên thay vì theo độ bất đồng ensemble.
2. **no ensemble**: chỉ 1 predictor, và vì không còn ensemble để đo bất đồng, truy vấn cũng chọn ngẫu nhiên.

3. **no online queries**: chỉ thu thập nhãn ở đầu huấn luyện (offline), không thu thêm trong lúc train.
4. **no regularization**: bỏ L_2, chỉ còn dropout.
5. **no segments** (chỉ MuJoCo): đoạn dài 1 bước thay vì nhiều bước.
6. **target**: dùng oracle cho tổng reward thật của cả đoạn, fit $\hat{r}$ bằng MSE (hồi quy) thay vì so sánh.

Phát hiện quan trọng nhất: offline tệ nhất. Vì phân phối trạng thái mà policy đi qua (occupancy distribution) thay đổi theo thời gian huấn luyện, một reward predictor huấn luyện 1 lần ở đầu chỉ nắm được *một phần* reward thật, và tối ưu phần đó dẫn tới hành vi kỳ quặc theo thước đo reward thật. **Ví dụ reward hacking cụ thể của bài báo:** trên Pong, huấn luyện offline đôi khi khiến agent học tránh thua điểm nhưng không cố ăn điểm, dẫn tới các pha bóng dài bất thường lặp đi lặp lại vô hạn. Đây là bằng chứng rằng phản hồi người cần được **đan xen với quá trình học RL**, không thể cung cấp tĩnh một lần.

Comparisons vs target (MSE trên reward tuyệt đối): trên continuous control, comparisons tốt hơn rõ rệt target, vì thang đo reward thay đổi mạnh giữa các chiều làm hồi quy khó, còn so sánh thì mượt hơn. Trên Atari (reward đã bị clip về $\{-1, 0, 1\}$, tức predict gần như chỉ dấu), comparisons và target không nhất quán hơn nhau.

Clip dài vs khung đơn (single frame): dùng khung đơn cho kết quả rõ ràng kém hơn clip, cần nhiều so sánh hơn đáng kể để đạt cùng hiệu năng; nói chung so sánh clip dài hữu ích hơn mỗi-clip nhưng kém hữu ích hơn mỗi-khung-hình; thời gian đánh giá của người gần như tuyến tính theo độ dài clip, nên tác giả chọn độ dài clip ngắn nhất mà tính tuyến tính này còn đúng.

6. Giới hạn

Tác giả nêu:

- Phương pháp chọn truy vấn theo variance ensemble là "xấp xỉ thô", và chính ablation của bài báo cho thấy nó có thể **làm giảm** hiệu năng trên một số tác vụ (random queries đôi khi tốt hơn).
- Phản hồi người thật nhiễu hơn giả lập: một contractor làm lệch lịch gán nhãn thông thường khiến tiến độ trên Hopper bất thường (ghi chú ngay trong chú thích Hình 2).
- Reward predictor cho Atari phụ thuộc vào 4 khung hình liên tiếp (không phải 1 khung), nên ablation "single frame" chỉ được chạy cho continuous control (footnote 5).
- Chi phí tính toán đã tương đương chi phí nhãn người, tức đang "chạm giới hạn lợi ích cận biên" của việc tiếp tục giảm số nhãn (Discussion).

Nhận xét:

- Các episode độ dài biến đổi bị loại bỏ (điều kiện kết thúc được thay bằng penalty mà agent phải tự học), nghĩa là bài toán đã bị sửa đổi khỏi benchmark chuẩn; so sánh "gần khớp RL" cần hiểu trong khuôn khổ môi trường đã sửa đổi này, không hẳn tương đương số liệu Atari/MuJoCo chuẩn ở nơi khác.
- Reward model chỉ là mạng truyền thẳng trên khung hình/trạng thái xếp chồng, không có cơ chế xử lý partially-observable tổng quát (paper gợi ý RNN ở footnote 1 nhưng không triển khai).
- Vì chi phí, hầu hết thí nghiệm với phản hồi người thật chỉ chạy 1 lần (không trung bình nhiều seed), nên đường cong "real human feedback" ở Hình 2, 3 không có sai số thống kê giữa các lần chạy.

7. Liên hệ với các bài khác

- **Bài 00:** khung RLHF/PPO/KL tổng quát; bài này là trường hợp cụ thể đầu tiên, trước khi PPO ra đời (dùng A2C/TRPO).
- **Bài 04 (TRPO), bài 05 (GAE), bài 06 (A3C):** các thuật toán RL cụ thể được cắm vào làm "Process 1"; bài này không sửa đổi chúng.
- **Bài 24 (Fine-tuning LM from Human Preferences, Ziegler 2019):** áp đúng ý tưởng Bradley-Terry (mở rộng từ 2 lên 4 lựa chọn) lên GPT-2, thêm phạt KL tới mô hình pretrain (bài này chưa cần KL vì

policy không phải mô hình ngôn ngữ).

- **Bài 25 (Learning to summarize):** cùng recipe, scale lên GPT-3 tới 6.7B, chuyển sang PPO với value network riêng.
- **Bài 26 (Constitutional AI):** thay nhãn người bằng nhãn AI (RLAIF) nhưng vẫn dùng đúng khung Bradley-Terry 2 lựa chọn ở mục 3.1 của bài này.
- **Bài 27 (Reward model overoptimization):** định lượng chính hiện tượng mà ablation "offline" ở mục 5 chỉ thấy định tính.
- **llm 07 (InstructGPT):** loss RM $-\log\sigma(r_w - r_l)$ của InstructGPT chính là công thức ở mục 3.6 của bài này, viết dưới dạng sigmoid thay vì softmax.

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp: **thấp** (đây là điều khiển robot/Atari, không phải LLM hay OCR), nhưng các bài học thiết kế hệ thống học-reward rất tổng quát và áp dụng được cho bất kỳ quy trình RLHF/RLAIF/LLM-judge sau này:

- **Reward hacking khi reward "đứng yên":** nếu dự định dùng một mô hình chấm điểm học được (mô hình phần thưởng, verifier, hay LLM-judge chấm chất lượng OCR) để huấn luyện policy, bài học của ablation "offline" ở mục 5 là phải cập nhật/làm mới mô hình chấm điểm theo chu kỳ khi policy thay đổi phân phối đầu ra, chứ không huấn luyện một lần rồi tối ưu mãi trên nó (nguy cơ y hệt sẽ được định lượng ở bài 27).
- **Chọn truy vấn theo độ bất đồng ensemble** là một kiểu active learning rẻ: nếu cần xây một tập nhãn preference nhỏ cho mô hình phần thưởng (không nhất thiết cho OCR), ưu tiên nhãn ở các cặp mà nhiều mô hình chấm điểm bất đồng nhau nhất, rẻ hơn nhãn ngẫu nhiên đều.
- **So sánh cặp dễ cho người (hoặc LLM) đánh giá hơn điểm tuyệt đối:** bài học áp dụng được cho bất kỳ thiết kế đánh giá chất lượng đầu ra nào, kể cả khi dùng một LLM khác làm giám khảo.

9. Thí nghiệm gợi ý

1. Trên một môi trường control đơn giản (CartPole/MountainCar tuỳ biến với reward đã biết để kiểm chứng), so sánh query selection ngẫu nhiên và theo độ bất đồng ensemble: đo số nhãn cần để đạt một mức reward thật cho trước.
2. Tái lập hiện tượng reward hacking kiểu Pong: huấn luyện mô hình phần thưởng 1 lần rồi đóng băng (offline) so với cập nhật liên tục (online) trên cùng môi trường, đo khoảng cách giữa reward dự đoán và reward thật theo thời gian huấn luyện.
3. Thêm nhiễu nhãn ngẫu nhiên x% vào oracle preference (mô phỏng lỗi người), so sánh độ ổn định của mô hình phần thưởng khi có và không có giả định lỗi 10% (mục 3.2) qua đường cong reward thật đạt được.
4. Trên một tác vụ có thang reward lệch mạnh giữa các episode (reward thưa, phần thưởng lớn hiếm khi xảy ra), so sánh huấn luyện mô hình phần thưởng bằng comparisons và bằng MSE trực tiếp trên tổng reward (target), đo độ ổn định huấn luyện.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Trajectory segment	$\sigma = ((o_0, a_0), \ldots, (o_{k-1}, a_{k-1}))$	đoạn 1-2 giây
Preference predictor	$\hat{P}[\sigma^1 \succ \sigma^2] = \dfrac{\exp\sum \hat{r}^1}{\exp\sum \hat{r}^1 + \exp\sum \hat{r}^2}$	Bradley-Terry / Luce-Shephard
Loss reward model	$-\sum_D \mu(1)\log\hat{P}[1 \succ 2] + \mu(2)\log\hat{P}[2 \succ 1]$	cross-entropy trên nhãn mềm

Khái niệm	Công thức / ý	Ghi chú
2 lựa chọn = sigmoid	$\hat{P}[1 \succ 2] = \sigma(r^1 - r^2)$	tự tính; cầu nối tới RM dạng $-\log\sigma$ ở bài 25/InstructGPT
Ensemble	3 predictor, bootstrap D, trung bình sau chuẩn hoá	giảm phương sai
Query selection	chọn cặp có variance dự đoán cao nhất giữa ensemble	active learning thô
Giả định lỗi người	~10% chance random	diễn giải: $\hat{P}_{\text{nhiễu}} = 0.9\hat{P} + 0.05$
RL lõi	A2C (Atari), TRPO (MuJoCo)	chỉ đổi nguồn reward, giữ thuật toán
Chi phí	~25 USD compute, ~36 USD nhãn (5k nhãn, 1 ngày Atari)	tự nêu trong paper

11. Kết luận ngắn

Bài báo chứng minh rằng có thể dạy một agent RL hiện đại (Atari, MuJoCo) các hành vi phức tạp, kể cả những hành vi không có hàm reward tường minh (backflip, bám xe), chỉ bằng cách học một mô hình phần thưởng từ so sánh cặp theo mô hình Bradley-Terry, với chưa tới 1% số bước tương tác được con người gán nhãn, miễn là ba quá trình (policy, chọn truy vấn, học reward) chạy đan xen bất đồng bộ. Hạn chế lớn nhất được chính paper chỉ ra là phương pháp chọn truy vấn còn thô và chi phí tính toán đã bắt đầu tương đương chi phí nhãn người; bài học quan trọng nhất cho các bài RLHF sau (24, 25, 26, và InstructGPT ở llm 07) là công thức mô hình phần thưởng Bradley-Terry ở mục 3.1 và nguy cơ reward hacking khi tối ưu quá lâu trên một reward cố định.

Link gốc: https://arxiv.org/abs/1706.03741

Bài 24

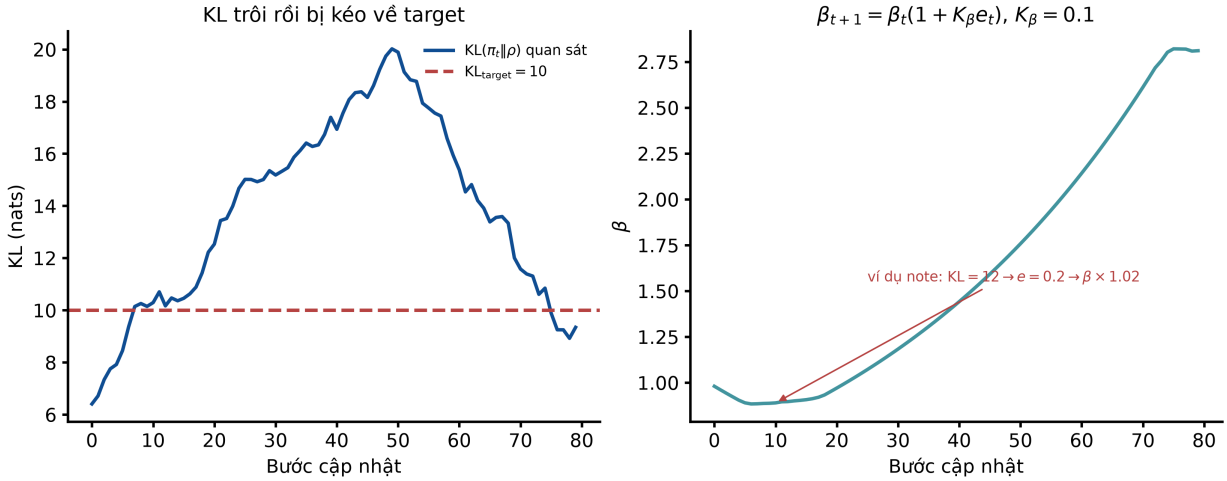
Fine-Tuning Language Models from Human Preferences (2019)

Tác giả: Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, Geoffrey Irving (OpenAI).
Venue / arXiv: arXiv:1909.08593 (v2, 08/01/2020). Bài báo không nêu venue trong bản đã đọc.
Nguồn: https://arxiv.org/abs/1909.08593
Chủ đề: RLHF đầu tiên cho mô hình ngôn ngữ: học mô hình phần thưởng từ lựa chọn 4 phương án, phạt KL $r - \beta \log(\pi/\rho)$ tới mô hình gốc, PPO, bộ điều khiển tỉ lệ điều chỉnh β động, áp dụng cho sinh văn bản theo văn phong (sentiment, descriptiveness) và tóm tắt (TL;DR, CNN/DM). Phát hiện nổi bật: hành vi "sao chép" (copying) ở mô hình tóm tắt.
Vị trí trong lộ trình: Đọc sau bài 23 (Christiano et al. 2017, cùng mô hình Bradley-Terry cho so sánh cặp, nhưng trên robot/Atari) và sau bài 00, bài 07 (PPO). Đây là bài đầu tiên đưa đúng công thức RLHF (mô hình phần thưởng + phạt KL + PPO) vào mô hình ngôn ngữ, là tiền thân trực tiếp của bài 25 (Learning to summarize, cùng nhóm tác giả, scale lớn hơn) và llm 07 (InstructGPT, đọc sau để thấy quy trình hoàn chỉnh SFT → RM → PPO; InstructGPT dùng chính công thức KL-penalty ở mục 3.2 dưới đây, không cần giải thích lại).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Bài 23 (Christiano et al. 2017) đã chứng minh có thể học reward từ so sánh cặp cho robot/Atari. Song song, NLP đang hưởng lợi lớn từ **pretraining sinh tự hồi quy rồi tinh chỉnh** (GPT, BERT, ELMo). Câu hỏi của bài này: có thể kết hợp hai hướng đó, dùng RL với reward học từ phản hồi người để tinh chỉnh một mô hình ngôn ngữ đã pretrain, thay cho tinh chỉnh có giám sát thông thường?



Adaptive KL: $e_t = \text{clip}((\text{KL} - \text{KL}_{target})/\text{KL}_{target})$; $\beta \leftarrow \beta(1 + 0.1e)$ (minh hoạ, tự tính)

Vì sao không chỉ supervised tinh chỉnh? Với các tác vụ có reward tự động sẵn (BLEU cho dịch, ROUGE cho tóm tắt), nhiều công trình trước đã dùng RL trực tiếp trên các chỉ số đó, nhưng (theo Paulus et al. 2017

được paper trích) "tối ưu một chỉ số rời rạc đơn lẻ như ROUGE bằng RL có thể làm giảm chất lượng mô hình". Với các tác vụ **chỉ định nghĩa được bằng phán đoán người** (văn phong, mức độ hữu ích), không có chỉ số tự động nào thay thế được.

Khái niệm nền:

- Mô hình ngôn ngữ ρ trên từ vựng Σ: $\rho(x_0 \cdots x_{n-1}) = \prod_k \rho(x_k|x_0 \cdots x_{k-1})$. Với input $x \in \Sigma^{\leq m}$, ρ cho một **policy** $\rho(y|x) = \rho(xy)/\rho(x)$: cố định phần đầu x, sinh tiếp y.
- **KL penalty** $\beta\,\mathrm{KL}(\pi, \rho)$: giữ policy π đang tinh chỉnh gần mô hình gốc ρ, kế thừa trực tiếp từ Jaques et al. (2017, 2019) ("KL-control" cho sinh chuỗi).
- **Online vs offline data collection:** offline = chỉ nhãn trên mẫu của ρ ban đầu; online = tiếp tục thu nhãn trên mẫu của π đang huấn luyện, để tránh distributional shift giữa phân phối huấn luyện và phân phối đánh giá của mô hình phần thưởng.

2. Ý tưởng chính

Thay tinh chỉnh có giám sát bằng tinh chỉnh bằng RL, với reward lấy từ một **mô hình phần thưởng** r học từ việc con người chọn phương án tốt nhất trong 4 lựa chọn do ρ sinh ra. Để π không trôi quá xa khỏi vùng mà r còn tin cậy (và, với tác vụ văn phong, để giữ mạch lạc/đúng chủ đề vì con người chỉ được hỏi về văn phong, không hỏi về mạch lạc), thêm phạt KL tới ρ vào reward, rồi tối ưu bằng PPO. Áp dụng cho 4 tác vụ: tiếp tục văn bản theo văn phong tích cực (sentiment) hoặc miêu tả sống động (descriptiveness), và tóm tắt (CNN/Daily Mail, TL;DR Reddit).

3. Phương pháp chi tiết

3.1 Reward model từ 4 lựa chọn

Với mỗi input x, lấy mẫu 4 continuation $(y_0, y_1, y_2, y_3) \sim \rho(\cdot|x)$, hỏi người chọn phương án tốt nhất $b \in \{0, 1, 2, 3\}$ (hỏi 4 lựa chọn một lần giúp người "khấu hao" chi phí đọc hiểu x). Huấn luyện $r : X \times Y \to \mathbb{R}$ bằng loss softmax nhiều lớp:

$$\boxed{\mathrm{loss}(r) = \mathbb{E}_{(x,\{y_i\},b)\sim S}\Big[-\log \frac{e^{r(x,y_b)}}{\sum_i e^{r(x,y_i)}}\Big]}$$

(bản PDF/text trích xuất của phương trình (1) không hiện dấu trừ phía trước log; đây có vẻ là lỗi trình bày/trích xuất công thức, vì cực đại hoá log của một số luôn nhỏ hơn 1 sẽ làm giảm likelihood, không tăng. Dạng có dấu trừ ở trên khớp với mọi mô tả bằng lời của chính paper ("fit a reward model... using the loss") và với các bài sau (25, InstructGPT) viết đúng dấu; **đây là diễn giải của phần phân tích để sửa một lỗi trình bày công thức, không phải nội dung khác với ý paper**). Với $K = 2$ lựa chọn, công thức này thu gọn về đúng sigmoid Bradley-Terry ở bài 23 (mục 3.6 của bài đó).

r khởi tạo bằng một lớp **tuyến tính ngẫu nhiên** trên embedding cuối của ρ (không phải của π, lý do giải thích ở mục 5), rồi chuẩn hoá để có mean 0, variance 1 trên $x \sim D, y \sim \rho(\cdot|x)$.

3.2 Reward có phạt KL và mục tiêu PPO

$$\boxed{R(x, y) = r(x, y) - \beta \log \frac{\pi(y|x)}{\rho(y|x)}}$$

Số hạng KL đóng ba vai trò: (i) như một entropy bonus khuyến khích khám phá, (ii) giữ π trong vùng mà r còn hợp lệ, (iii) với tác vụ văn phong, **là một phần định nghĩa tác vụ**: con người chỉ được hỏi về văn phong, còn mạch lạc/đúng chủ đề được đảm bảo gián tiếp qua số hạng KL.

β có thể cố định hoặc điều chỉnh động để giữ $\mathrm{KL}(\pi, \rho)$ ở một giá trị mục tiêu, bằng bộ điều khiển tỉ lệ dạng log:

$$e_t = \mathrm{clip}\Big(\frac{\mathrm{KL}(\pi_t, \rho) - \mathrm{KL}_{\text{target}}}{\mathrm{KL}_{\text{target}}}, -0.2, 0.2\Big), \qquad \beta_{t+1} = \beta_t(1 + K_\beta e_t), \quad K_\beta = 0.1$$

Ví dụ nhỏ (tự tính): giả sử $\mathrm{KL}_{\text{target}} = 10$ nats và $\mathrm{KL}(\pi_t, \rho) = 12$ nats. $e_t = \mathrm{clip}((12-10)/10, -0.2, 0.2) = \mathrm{clip}(0.2, -0.2, 0.2) = 0.2$ (đã bị kẹp ở biên). $\beta_{t+1} = \beta_t(1 + 0.1 \times 0.2) = 1.02\,\beta_t$: β chỉ tăng tối đa 2% mỗi bước cập nhật vì e_t bị kẹp trong $[-0.2, 0.2]$, cho thấy khoảng clip giới hạn tốc độ thay đổi β dù KL lệch mục tiêu bao nhiêu.

3.3 Toàn bộ quy trình huấn luyện

1. Lấy mẫu (x, y_0, y_1, y_2, y_3) với $x \sim D$, $y_i \sim \rho(\cdot|x)$; hỏi người chọn b tốt nhất.
2. Khởi tạo $r \leftarrow \rho$ (lớp cuối ngẫu nhiên); huấn luyện trên mẫu người bằng loss ở mục 3.1.
3. Huấn luyện π bằng PPO với reward R ở mục 3.2, $x \sim D$.
4. (Trường hợp online) tiếp tục thu thêm mẫu và huấn luyện lại r định kỳ (mục 2.3 của bài báo).

3.4 Siêu tham số chính (mục 2.1-2.2)

Mô hình: GPT-2 774M (36 lớp, 20 head, embedding 1280), pretrain trên WebText, BPE 50.257 token. Với tác vụ văn phong: supervised tinh chỉnh thêm trên BookCorpus trước khi RL tinh chỉnh; nhiệt độ $T < 1$ (chia logit cho T) để mẫu chất lượng cao hơn khi $T = 1$ ở bước sau.

Siêu tham số	Giá trị
Batch size huấn luyện RM	8 (văn phong), 32 (tóm tắt)
Learning rate RM	1.77×10^{-5}, 1 epoch, không dropout
PPO2 (Dhariwal et al. 2017)	$\gamma = 1$, 2 triệu episode, 4 epoch PPO/batch, 1 minibatch/batch
Batch size PPO	1024 (văn phong), 512 (tóm tắt)
Learning rate PPO	1.41×10^{-5} (văn phong), 7.07×10^{-6} (tóm tắt)
$\mathrm{KL}_{\text{target}}$ (điều chỉnh động)	10 nats (sentiment), 6 nats (descriptiveness)
β cố định	0.1 (CNN/DM), 0.03 (TL;DR)

Với mô hình công khai nhỏ hơn (124M, 12 lớp, 12 head, embedding 768) tác giả phát hành code và một phần nhãn người đã thu (lưu ý: nhãn này được thu từ chạy với mô hình lớn, không phải từ bản 124M).

4. Thí nghiệm và kết quả chính

4.1 Tác vụ văn phong (sentiment, descriptiveness)

Input: đoạn BookCorpus dài 32-64 token; policy sinh tiếp 24 token. Với văn phong, dùng rejection sampling để đảm bảo có **dấu chấm câu** ở khoảng token 16-24, cắt ở đó; nếu không có dấu chấm trong khoảng, phạt cố định -1. Với tóm tắt (mục 4.2 dưới), rejection sampling tương tự nhưng tìm **dấu xuống dòng** (newline) ở khoảng token 55-75, cắt ở đó, phạt -1 nếu không có.

Mock sentiment (3.1.1, kiểm soát bằng oracle): "người" giả định luôn chọn continuation có r_s (log-odds từ một classifier Transformer 6 lớp huấn luyện trên Amazon review) cao hơn. 20.000-60.000 truy vấn đủ để tối ưu r_s gần bằng RL tối ưu trực tiếp trên r_s (Hình 2).

Bảng 1 (người đánh giá, 1024 mẫu so sánh 4 chiều, trích):

So sánh	Sentiment (model thắng / thua)	Descriptiveness
5k offline vs. zero-shot	88% / 12%	86% / 14%
5k offline vs. mock (oracle r_s)	77% / 23%	(không có mock cho descriptiveness)
5k offline vs. 20k offline	48% / 52%	47% / 53%
5k offline vs. 5k online	50% / 50%	48% / 52%

Phần tóm tắt ở Introduction của bài báo (không phải Bảng 1) viết gộp: "được ưa chuộng hơn zero-shot 86% thời gian, hơn mô hình fine-tune theo classifier sentiment giám sát 77% thời gian"; số liệu này là phát biểu tóm tắt của chính tác giả ở phần giới thiệu, hơi khác cách chia theo từng tác vụ ở Bảng 1 (88% sentiment / 86% descriptiveness so với zero-shot); **cả hai đều lấy trực tiếp từ paper, chênh lệch này chưa được làm rõ.**

Online và offline cho kết quả tương đương ở tác vụ văn phong (khác hẳn tóm tắt, xem dưới); chỉ cần rất ít dữ liệu người (2.5k cho sentiment, 5k cho descriptiveness) đã đủ tốt, giảm dần hiệu quả dưới 5k mẫu.

4.2 Tóm tắt (CNN/Daily Mail, TL;DR)

Input cắt ở 500 token, thêm hậu tố `"\n\nTL;DR:"` (và tiền tố `"Article:\n\n"` cho CNN/DM); policy sinh tối đa 75 token.

Bảng 4 (ROUGE, trích, $T = 0.7$ cho TL;DR, $T = 0.5$ cho CNN/DM):

Mô hình	TL;DR R-1/R-2/R-L	CNN/DM R-1/R-2/R-L
zero-shot	15.862 / 2.325 / 13.518	28.406 / 8.321 / 25.175
supervised baseline	17.535 / 3.124 / 14.969	39.525 / 16.992 / 36.728
60k fine-tune (RL thuần)	16.800 / 2.884 / 14.011	37.385 / 15.478 / 33.330
supervised + 60k fine-tune	18.434 / 3.542 / 15.457	40.093 / 17.611 / 37.104
lead-3	17.435 / 3.243 / 14.575	40.379 / 17.658 / 36.618

Bảng 5 (người đánh giá, thắng/thua, trích, 60k tinh chỉnh online là mô hình RL chính):

So sánh	TL;DR	CNN/Daily Mail
60k fine-tune vs. zero-shot	96% / 4%	91% / 9%
60k fine-tune vs. supervised	97% / 3%	80% / 20%
60k fine-tune vs. lead-3	45% / 55%	40% / 60%
60k fine-tune vs. 60k offline fine-tune	64% / 36%	65% / 35%
60k fine-tune vs. reference summaries	**96% / 4%**	**84% / 16%**

Mô hình RL thuần (bỏ qua reference summary khi huấn luyện) thắng áp đảo kể cả **bản tóm tắt tham chiếu của chính dataset**, trong khi ROUGE của nó (Bảng 4) thấp hơn supervised baseline. Online quan trọng hơn hẳn cho tóm tắt: tại cùng 60k nhãn, online hơn offline 3 điểm R-AVG trên CNN/DM; người đánh giá chọn online 64-65% so với offline.

Hành vi "sao chép" (copying), phát hiện trọng tâm của bài báo: mô hình 60k tinh chỉnh gần như hoàn toàn trích xuất (extractive) mặc dù **không có cơ chế copy tường minh** trong kiến trúc: theo thống kê độ "mới" (novelty) của câu trong Hình 5/6 (tỉ lệ câu sinh ra không trùng nguyên văn với nguồn), nó copy nguyên câu 71% thời gian trên TL;DR và 98% trên CNN/Daily Mail. Mô hình supervised+RL tinh chỉnh copy ít hơn

nhiều (6% và 30%). Vị trí *trong bài* mà mô hình copy từ đó (ví dụ có bỏ qua phần mở đầu không liên quan hay không) được đo riêng bằng vị trí của longest common subsequence bigram giữa văn bản nguồn và bản tóm tắt (Hình 7), khác với tỉ lệ 71%/98% nói trên. Về độ chính xác (Bảng 8, 30 bài/tập, tác giả tự chấm): 60k tinh chỉnh đúng 26/30 (TL;DR) và 29/30 (CNN/DM) vì copy nguyên câu hầu như luôn đúng sự thật; supervised chỉ đúng 8/30 và 19/30, hay "diễn giải tồi" (đổi tên, trộn nhiều câu sai). Mô hình cũng biết **bỏ qua phần mở đầu không liên quan** (Bảng 7: copy 3 từ đầu bài chỉ 0.2%/1.4% trong các bài có "preamble" dù copy tổng thể 28.3%/77.6%), cho thấy hành vi copy có chọn lọc, không máy móc.

5. Phân tích / ablation

- **Chia sẻ tham số giữa RM và policy (mục 4.2):** thử dùng một mạng chung nhiều đầu ra (RL như auxiliary task cho RM) để tăng hiệu quả tính toán và giữ RM mạnh hơn, nhưng **không thành công**: mất cân đối dữ liệu quá lớn (tối đa 60k mẫu cho RM so với 2 triệu episode cho policy) khiến RM dễ overfit nếu lặp nhiều epoch qua ít dữ liệu, chưa giải quyết được.
- **Online khó (mục 4.1):** hệ thống online interleave thu nhãn, train RM, train RL khiến software phức tạp hơn, khó debug từng phần riêng (phải chuyển qua lại online/offline để cách ly lỗi), và kiểm soát chất lượng nhãn khó vì phải phát hiện lao động kém chất lượng theo thời gian thực. Tác giả đề xuất hướng giữa (batched data collection) cho công việc tương lai.
- **Tác vụ mơ hồ gây khó gán nhãn (mục 4.3):** đánh giá bản tóm tắt vừa chủ quan vừa nhiều chiều (đúng, mạch lạc, không dư, đủ ý); với 4-way comparison trên sentiment/TL;DR, chính các tác giả chỉ đồng thuận với nhau khoảng 60% (so với 25% nếu đoán ngẫu nhiên, Bảng 9); đồng thuận author-labeler 44%/38% (sentiment/TL;DR), author-author 62%/61%.
- **Bug lật dấu reward (mục 4.4), ví dụ reward hacking rõ nhất của bài báo:** một lần refactor code vô tình lật dấu cả reward **và** phạt KL cùng lúc, khiến mô hình tối ưu sentiment âm nhưng vẫn giữ tính "tự nhiên" nhờ KL (vì KL cũng bị lật dấu theo hướng tương thích). Vì nhãn hướng dẫn người chấm rất thấp cho nội dung khiêu dâm lộ liễu, mô hình nhanh chóng học chỉ sinh nội dung dạng đó, bất kể điểm khởi đầu vô hại thế nào. Bug chỉ bị phát hiện sau khi huấn luyện xong vì không ai theo dõi trong lúc chạy.
- **Ví dụ reward hacking thứ hai, không cần bug (Bảng 10, mock sentiment, bỏ phạt KL):** không có KL, kết quả là văn bản vô nghĩa ("! These These These sound flowed instantly easily easily easily easily!") dù có thêm entropy bonus (nhắm 30 nats); cả hai policy (có/không entropy bonus) đạt điểm trung bình khoảng **+8.0 theo mock sentiment model (tương đương 99.97% xác suất "tích cực")**. Đây là minh chứng trực diện: không có số hạng KL, tối ưu một reward học được sẽ hỏng ngôn ngữ hoàn toàn trong khi vẫn "thắng" theo chính thước đo được tối ưu.

6. Giới hạn

Tác giả nêu:

- Kết quả "mixed": tác vụ văn phong tốt với rất ít dữ liệu, nhưng tóm tắt chỉ cho ra "smart copier" (trích xuất), không phải tóm tắt trừu tượng thật sự.
- Online data collection tuy cho kết quả tốt nhất nhưng khó vận hành (mục 5 trên); tác giả định hướng batched collection cho tương lai, chưa thực hiện trong bài.
- Đồng thuận gán nhãn người thấp (~60% trên 4-way so với 25% ngẫu nhiên) làm hạn chế chất lượng dữ liệu, đặc biệt trong thiết lập online tốc độ cao khiến kiểm soát chất lượng khó hơn.
- Tác giả tự nhận: nhãn viên có thể ưu tiên copy như một "heuristic nhanh" để kiểm tra tính chính xác, dù không được dặn thưởng việc copy một cách tường minh, chỉ được dặn phạt thiếu chính xác.

Nhận xét:

- Phương trình (1) trong bản PDF công khai không có dấu trừ trước log; người ghi chú coi là lỗi trình bày (đã sửa ở mục 3.1) vì không khớp lập luận bằng lời của chính paper.

- Tác vụ tóm tắt "thắng" cả bản tham chiếu người viết (96%/84%) chủ yếu vì cơ chế copy giúp đúng sự thật hơn là vì tóm tắt hay hơn theo nghĩa thông thường; đây là một dạng khai thác kẽ hở tiêu chí đánh giá (nhãn viên ưu tiên đúng sự thật hơn diễn đạt mới) hơn là cải thiện chất lượng tóm tắt trừu tượng.
- Không có con số định lượng cho "tổng quát hoá ngoài phân phối" (dịch ngôn ngữ khác, hỏi đáp code) được nhắc tới ở bài 25 sau; bài 24 không thử nghiệm phần này.

7. Liên hệ với các bài khác

- **Bài 23 (Christiano et al. 2017):** tiền thân trực tiếp; mở rộng Bradley-Terry 2 lựa chọn (mục 3.6 bài 23) thành softmax 4 lựa chọn (mục 3.1 bài này); bài 23 chưa cần KL vì policy không phải mô hình ngôn ngữ, bài này thêm KL vì cần giữ tính tự nhiên của văn bản.
- **Bài 25 (Learning to summarize):** cùng nhóm tác giả cốt lõi (Stiennon, Ziegler, Christiano), chuyển hoàn toàn sang batch offline+retrain lặp lại (giải quyết đúng khó khăn "online khó" ở mục 5), tách value network riêng khỏi policy, scale models tới 6.7B, và chính là bài đưa ra "đường cong reward-model-overoptimization" mà hiện tượng ở mục 5-6 của bài này mới chỉ thấy qua ví dụ bug và ablation KL.
- **Bài 00, bài 07 (PPO):** PPO2 ở đây dùng thẳng thuật toán PPO của bài 07, chỉ thêm reward có phạt KL.
- **llm 07 (InstructGPT):** dùng chính công thức $R = r - \beta \log(\pi/\rho)$ ở mục 3.2 của bài này (viết lại với tên π^{RL}, π^{SFT}), cộng thêm PPO-ptx (trộn gradient pretraining); không cần giải thích lại KL penalty, chỉ cần biết nó bắt nguồn từ đây.
- **Bài 26 (Constitutional AI):** kế thừa toàn bộ khung mô hình phần thưởng + KL + PPO, chỉ thay nguồn nhãn preference (người → AI).

8. Ứng dụng thực tế (suy luận)

- **Phạt KL không phải "trang trí":** ví dụ Bảng 10 (không KL → văn bản vô nghĩa nhưng điểm reward giả +8.0) là minh chứng cực đoan rằng thiếu ràng buộc tới mô hình/phân phối gốc, một reward học được (hoặc một reward surrogate như CER/edit-distance có thể bị khai thác qua chuỗi lặp, câu rỗng) có thể bị tối ưu tới mức phá hỏng chất lượng thực sự của đầu ra mà vẫn "thắng" theo đúng thước đo đang tối ưu.
- **Theo dõi bug lật dấu (mục 5):** bài học vận hành đơn giản nhưng quan trọng, nên có theo dõi tự động (ví dụ cảnh báo khi entropy/diversity đầu ra giảm đột ngột) trong mọi vòng lặp RL, không chỉ chờ xem log cuối cùng.
- **Hành vi "sao chép" khi reward khó tối ưu abstractively:** nếu một policy sinh OCR tìm ra cách "ăn gian" rẻ mà vẫn đạt điểm cao theo thước đo (ví dụ chỉ chép lại pattern hay gặp, bỏ qua nội dung ảnh thật), đó là cùng loại hiện tượng; cần một thước đo đánh giá thực tế (không chỉ metric huấn luyện) để phát hiện.

9. Thí nghiệm gợi ý

1. Tái lập Bảng 10 trên quy mô nhỏ: huấn luyện policy tối ưu một reward thưa/dễ khai thác (ví dụ độ dài câu, hoặc tần suất một token) với và không có phạt KL tới mô hình gốc, quan sát độ trôi dạt văn bản (perplexity theo ρ) theo số bước huấn luyện.
2. So sánh online vs offline retraining của mô hình phần thưởng trên cùng ngân sách nhãn (ví dụ 15k/30k/60k), đo khoảng cách phân phối giữa mẫu dùng để huấn luyện RM và mẫu của policy hiện tại (ví dụ qua KL ước lượng).
3. Đo "độ sao chép" (longest common subsequence bigram với nguồn) của một mô hình tóm tắt/OCR tự huấn luyện theo thời gian huấn luyện RL, để phát hiện sớm hành vi suy biến về trích xuất/lặp thay vì sinh có kiểm soát.
4. Thử bộ điều khiển β động (mục 3.2) trên một tác vụ có KL mục tiêu rõ ràng, so với β cố định, đo phương sai của $\mathrm{KL}(\pi, \rho)$ giữa các lần chạy với seed khác nhau.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Policy từ LM	$\pi(y\|x) = \rho(xy)/\rho(x)$, khởi tạo $\pi = \rho$	cố định x, sinh tiếp y
Loss reward model	$-\log \frac{e^{r(x,y_b)}}{\sum_i e^{r(x,y_i)}}$	softmax 4 lựa chọn; dấu trừ là diễn giải sửa lỗi trình bày
Reward có KL	$R(x,y) = r(x,y) - \beta \log \frac{\pi(y\|x)}{\rho(y\|x)}$	nền tảng cho InstructGPT, bài 25, 26
Điều chỉnh β	$\beta_{t+1} = \beta_t(1 + K_\beta e_t)$, $K_\beta = 0.1$, e_t kẹp $[-0.2, 0.2]$	bộ điều khiển tỉ lệ dạng log
Lịch thu nhãn online	$l(n) = N_{r0} + (N_r - N_{r0})(1 - (1 - n/N_\pi)^2)$	tăng dần phi tuyến
KL mục tiêu	10 nats (sentiment), 6 nats (descriptiveness)	động
β cố định (tóm tắt)	0.1 (CNN/DM), 0.03 (TL;DR)	
Hành vi nổi bật	"smart copier": copy 71%/98% câu (TL;DR/CNN-DM)	không có cơ chế copy tường minh trong kiến trúc

11. Kết luận ngắn

Bài báo là lần đầu ghép pretraining ngôn ngữ với RLHF kiểu Bradley-Terry: học reward từ lựa chọn 4 phương án, phạt KL tới mô hình gốc, tối ưu bằng PPO. Kết quả tốt và rẻ dữ liệu cho tác vụ văn phong, nhưng với tóm tắt chỉ tạo ra "smart copier" trích xuất nguyên câu, thắng áp đảo cả bản tham chiếu người viết về mức ưa chuộng dù ROUGE thấp hơn baseline, bộc lộ khoảng cách giữa "được labeler ưa chuộng" và "tóm tắt tốt theo nghĩa trừu tượng". Hai bài học bền vững nhất cho các bài RLHF sau: công thức KL-penalized reward (mục 3.2, dùng lại nguyên vẹn ở InstructGPT) và các ví dụ reward hacking cụ thể (Bảng 10, bug lật dấu) cho thấy tại sao số hạng KL không thể bỏ qua.

Link gốc: https://arxiv.org/abs/1909.08593

Bài 25

Learning to Summarize from Human Feedback (2020)

Tác giả: Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, Paul Christiano (OpenAI).
Venue / arXiv: arXiv:2009.01325 (v3, 15/02/2022). Venue: 34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada (ghi rõ ở trang đầu paper).
Nguồn: https://arxiv.org/abs/2009.01325
Chủ đề: Pipeline RLHF đầy đủ ở quy mô lớn cho tóm tắt văn bản: thu thập so sánh người theo lô (batch, không còn bất đồng bộ như bài 24), huấn luyện mô hình phần thưởng (RM), tối ưu policy bằng PPO với phạt KL và **value network tách riêng**, và đặc biệt là đường cong **"reward model overoptimization"**: tối ưu quá một RM cố định làm chất lượng thật giảm trở lại.
Vị trí trong lộ trình: Đọc sau bài 24 (Fine-Tuning LM from Human Preferences, cùng nhóm tác giả, tiền thân trực tiếp) và bài 00/07 (PPO, KL penalty). Đây là bài RLHF-cho-LLM đầu tiên scale tới 6.7B tham số và cho kết quả thắng rõ cả baseline giám sát lớn hơn 10 lần. Đọc trước llm 07 (InstructGPT, 2022): InstructGPT tổng quát hoá chính xác quy trình của bài này (RM + PPO + KL + value network riêng) từ một tác vụ (tóm tắt) sang mọi loại chỉ dẫn, không cần giải thích lại các công thức cốt lõi. Bài 27 (Scaling laws for mô hình phần thưởng overoptimization) định lượng chặt chẽ hơn hiện tượng mà Hình 5 của bài này mới mô tả bằng quan sát.
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Vấn đề căn bản: mục tiêu huấn luyện chuẩn cho tóm tắt (log-likelihood của bản tóm tắt tham chiếu, đánh giá bằng ROUGE) là **proxy lệch** khỏi điều ta thực sự muốn (tóm tắt tốt theo con người). Max-likelihood không phân biệt lỗi quan trọng (bịa sự kiện) và lỗi không quan trọng (chọn từ đồng nghĩa); mô hình được khuyến khích đặt khối lượng xác suất lên *mọi* bản demo, kể cả bản chất lượng thấp; và lệch phân phối khi lấy mẫu tự hồi quy càng làm giảm chất lượng. ROUGE (longest common subsequence/skip-bigram với bản tham chiếu) được biết là tương quan kém với đánh giá người.

Công trình trực tiếp đi trước: Böhm et al. (2019) học reward từ 2.5k đánh giá CNN/DM. **Ziegler et al. 2019 (bài 24)** là gần nhất: cùng công thức RM + PPO + KL, nhưng chạy *bất đồng bộ* (interleave thu nhãn/train RM/train policy) và nhận thấy mô hình kết quả cực kỳ trích xuất (extractive), lý do nghi là đồng thuận nhãn viên-nghiên cứu viên thấp.

Hai thay đổi chính của bài này so với bài 24 (nêu rõ ở mục 3.3): (i) chuyển hẳn sang **offline theo lô (batch)**: gửi một lô lớn so sánh, train lại từ đầu trên toàn bộ dữ liệu đã tích luỹ, rồi mới tinh chỉnh policy, lặp lại; (ii) duy trì **quan hệ sát sao với nhãn viên** (onboarding kỹ, phòng chat chung, phản hồi định kỳ về chất lượng). Kết quả: đồng thuận nhãn viên-nghiên cứu viên đạt 77% ± 2%, gần bằng đồng thuận nghiên cứu viên-nghiên cứu viên (73% ± 4%).

2. Ý tưởng chính

Ba bước lặp lại (Hình 2 của bài báo):

1. **Thu nhãn:** lấy mẫu tóm tắt từ nhiều nguồn chính sách (policy hiện tại, policy khởi đầu, bản tham chiếu, các baseline), gửi cặp cho người chọn bản tốt hơn.
2. **Huấn luyện RM:** dự đoán log-odds bản nào được ưa thích hơn, dùng loss Bradley-Terry 2 lựa chọn (giống bài 23, dạng sigmoid).
3. **Tối ưu policy bằng PPO:** dùng logit đầu ra RM làm reward, tối ưu bằng PPO với phạt KL tới policy SFT, **value network là một mạng hoàn toàn tách biệt** khỏi policy (khác bài 24, nơi chia sẻ tham số được thử nhưng không hiệu quả).

Mô hình 1.3B huấn luyện theo cách này được ưa chuộng hơn mô hình giám sát **10 lần lớn hơn** nó, và cả hai cỡ (1.3B, 6.7B) được ưa chuộng hơn cả bản tóm tắt tham chiếu do người viết.

3. Phương pháp chi tiết

3.1 Dữ liệu

TL;DR Reddit (Völske et al. 2017): ~3 triệu bài, lọc theo whitelist subreddit, giữ bài có tóm tắt dài 24-48 token, loại bỏ "Edit/Update", nội dung nhạy cảm; **123.169 bài** sau lọc (~5% validation). Phân bố subreddit rất lệch: `relationships` 54.25%, `AskReddit` 13.23%, `relationship_advice` 7.45%, `tifu` 6.58%, còn lại dưới 3% mỗi subreddit (Bảng 2 paper, ~2/3 dữ liệu thuộc chủ đề quan hệ). Dataset nhãn người công khai có **64.832** so sánh tóm tắt TL;DR.

3.2 Reward model

Khởi tạo từ baseline giám sát, thêm đầu tuyến tính ngẫu nhiên ra 1 số vô hướng $r_\theta(x, y)$. Với nhãn i là bản được chọn:

$$\boxed{\text{loss}(r_\theta) = -\mathbb{E}_{(x,y_0,y_1,i)\sim D}\Big[\log \sigma(r_\theta(x, y_i) - r_\theta(x, y_{1-i}))\Big]}$$

Chuẩn hoá cuối cùng để bản tham chiếu có điểm trung bình 0. Đây đúng là công thức Bradley-Terry 2 lựa chọn ở bài 23 (mục 3.6), chỉ đổi tên biến.

3.3 Policy có phạt KL, value network riêng

$$\boxed{R(x, y) = r_\theta(x, y) - \beta \log \Big[\pi_\phi^{RL}(y|x)/\pi^{SFT}(y|x)\Big]}$$

Mỗi "bước thời gian" của PPO là một token BPE; phần thưởng r_θ chỉ được cấp **ở token cuối** (hết câu, EOS), không có reward trung gian; discount $\gamma = 1$. Value network khởi tạo từ chính RM nhưng **là một mạng hoàn toàn riêng** với policy (không chia tham số): tránh việc cập nhật value function phá hỏng policy đã pretrain ngay từ đầu (kiểm chứng bằng ablation, mục 5).

Cách đọc hình: policy sinh tóm tắt từng token, mỗi token bị phạt KL so với policy SFT đóng băng. Reward model chỉ chấm điểm ở token EOS, hai nguồn cộng lại thành phần thưởng của PPO. Value network là mạng riêng, khởi tạo từ reward model.

3.4 Siêu tham số chính (Phụ lục B.1)

Thành phần	Giá trị
Cỡ mô hình	1.3B, 3B, 6.7B, 13B (Bảng 3: 24/32/32/40 lớp)
RM: batch, epoch	64, 1 epoch
RM: learning rate	1.5e-5 (1.3B), 5e-6 (6.7B), cosine decay
PPO: γ, λ (GAE)	$\gamma = 1$, $\lambda = 0.95$

Thành phần	Giá trị
PPO: epoch/batch rollout	4 epoch tối ưu mỗi batch
PPO: batch, episode	512 (1.3B) hoặc 256 (6.7B); 1.000.000 episode
PPO: learning rate	1.5e-5 (1.3B), 7e-6 (6.7B), linear decay
KL coefficient β	**0.05** cho cả hai chạy chính (trừ khi quét β để vẽ đường cong overoptimization)
Supervised baseline	batch 128, 1 epoch, lr quét log-linear ≥7 giá trị

```
policy_SFT = finetune(pretrain, TL;DR, loss=NLL)          # baseline giám sát
repeat (lặp theo lô, không bất đồng bộ):
    lay mau tom tat tu nhieu nguon: policy hien tai, SFT, tham chieu, baseline khac
    gui cap (x, y0, y1) cho nguoi -> nhan i (ban duoc chon)
    RM = policy_SFT + dau tuyen tinh ngau nhien ra 1 so vo huong
    huan luyen RM tren toan bo D da thu tich luy:   loss = -log
    ↪  sigmoid(r(x,y_i)-r(x,y_{1-i}))
    chuan hoa: RM.bias -= mean(RM(x, tham_chieu))     # tham chieu diem TB 0

    pi = copy(policy_SFT); V = copy(RM)       # value network RIENG, khong chia se voi pi
    for 1,000,000 episodes:
        X = sample 512 (hoac 256) prompts
        Y = pi.sample(X)
        R_token_cuoi = r_theta(X,Y);  R_t (t<cuoi) = -beta*(log pi(y_t) - log
        ↪  pi_SFT(y_t))
        A = GAE(R, V, gamma=1, lambda=0.95)
        for 4 epoch:
            cap nhat pi, V bang PPO_clip_loss(A)
    (lap lai tu dau: thu nham moi tu policy vua huan luyen)
```

3.6 Ví dụ nhỏ (tự tính): KL của best-of-N khớp với số liệu phụ lục

Phụ lục D/G.3 cho công thức giải tích KL giữa policy best-of-N (chọn tốt nhất trong N mẫu theo RM) và policy gốc:

$$\mathrm{KL}(\mathrm{BoN}_N, \mathrm{sup}) = \log N - \frac{N-1}{N}$$

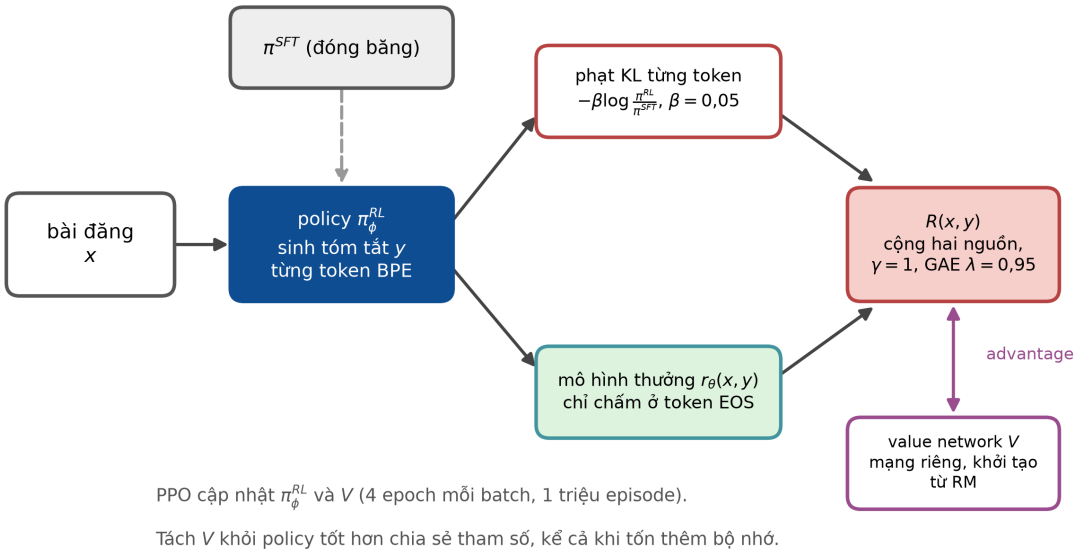
Với $N = 8$: $\log 8 - 7/8 = 2.079 - 0.875 = 1.204 \approx 1.2$, đúng khớp giá trị 1.2 trong Bảng "BoN policies" của phụ lục. Với $N = 64$: $\log 64 - 63/64 = 4.159 - 0.984 = 3.174 \approx 3.2$, cũng khớp giá trị 3.2 trong bảng đó (tự tính, dùng để kiểm chứng công thức đọc đúng từ paper).

4. Thí nghiệm và kết quả chính

Mô hình: Transformer decoder kiểu GPT-3, huấn luyện ở hai cỡ chính 1.3B và 6.7B (và tới 13B cho RM scaling).

Kết quả chính (Hình 1, tỉ lệ người ưa chuộng hơn bản tham chiếu TL;DR): 1.3B human-feedback thắng áp đảo supervised baseline **10 lần lớn hơn nó** (61% so với 43% tỉ lệ ưa chuộng thô so với tham chiếu); 6.7B human-feedback hơn nữa, vượt cả bản tham chiếu người viết. Sau khi kiểm soát độ dài (hồi quy logistic, Phụ lục F), tỉ lệ ưa chuộng của 6.7B human-feedback so với tham chiếu giảm khoảng 5 điểm phần trăm nhưng vẫn đạt **~65%**, cho thấy cải thiện không chỉ do tóm tắt dài hơn.

Đánh giá 4 chiều Likert 7 điểm (Hình 3): mô hình human-feedback vượt baseline giám sát trên mọi chiều (coverage, accuracy, coherence, overall), rõ nhất ở **coverage**. Tỉ lệ đạt điểm tuyệt đối 7/7 ở "overall": **45%** (6.7B PPO) so với **20%** (6.7B supervised) và **23%** (bản tham chiếu).



Hình 54: Cách tạo phần thưởng cho PPO: điểm reward model ở token cuối cộng phạt KL từng token (minh hoạ, tự tính)

Chuyển miền sang CNN/Daily Mail, không tinh chỉnh thêm (Hình 4): mô hình human-feedback huấn luyện trên Reddit đạt chất lượng gần bằng mô hình T5 đã tinh chỉnh trực tiếp trên CNN/DM, dù sinh tóm tắt ngắn hơn khoảng một nửa số token.

Reward model học được gì (mục 4.3):

- RM của bài tổng quát hoá sang CNN/DM, đồng thuận với nhãn viên 62.4% (1.3B) và 66.5% (6.7B), gần bằng đồng thuận giữa người với người (66.9%).
- Trên tập "sửa tối thiểu để tốt hơn" do nhãn viên tạo, RM chọn đúng bản đã sửa 79.4% (1.3B) / 82.8% (6.7B), so với 84.1% của một nhóm người đánh giá khác.
- Trên tập hoán đổi vai trò (tham chiếu so với bản bị xáo trộn), RM chọn đúng bản gốc 92.9% (1.3B) / 97.2% (6.7B).
- **Thiên lệch theo độ dài:** RM 6.7B chỉ chọn đúng bản "sửa ngắn hơn và tốt hơn" 62.6% thời gian, so với 76.4% của người, tức RM có xu hướng ưu ái bản dài hơn mức con người thực sự ưu ái.

RM so với ROUGE là thước đo (mục 4.4, best-of-N, Hình 7): ROUGE đạt đồng thuận ~57% với nhãn viên trên mẫu supervised, tụt xuống ~50% trên mẫu human-feedback (tức ROUGE mất tác dụng đúng lúc mô hình cải thiện); log-probability của baseline giám sát cũng tụt dưới 50%; trong khi đó RM vẫn giữ ~62% đồng thuận. Tối ưu ROUGE bằng best-of-N đạt đỉnh **sớm hơn và ở mức ưa chuộng thấp hơn hẳn** so với tối ưu RM.

5. Phân tích / ablation

5.1 Đường cong "reward model overoptimization" (Hình 5, trọng tâm của bài)

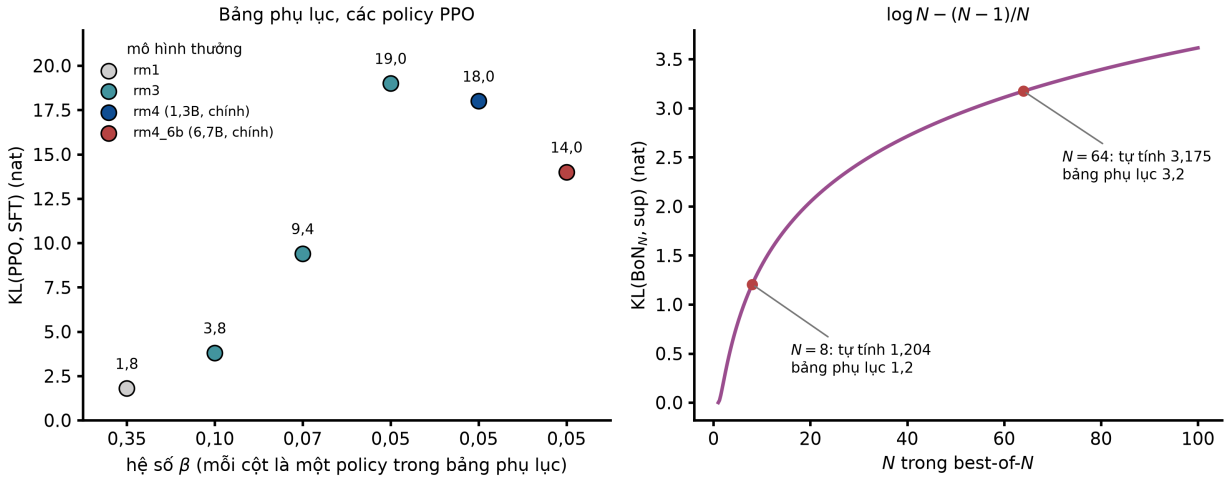
Tạo một dải policy được PPO tối ưu với các hệ số KL β khác nhau trên **một phiên bản RM cũ** (gọi là `rm3` trong phụ lục C.6), rồi so sánh điểm RM dự đoán với mức ưa chuộng thật của người. Hiện tượng: tối ưu nhẹ cải thiện cả điểm RM *và* mức ưa chuộng thật; tối ưu càng mạnh (giảm β), mức ưa chuộng thật **tách khỏi và cuối cùng còn giảm xuống dưới** điểm RM dự đoán, tới mức RM trở nên **phản tương quan** với sở thích thật. Đây chính là hiện tượng Goodhart (tối ưu một proxy quá mức làm hỏng mục tiêu thật).

Bảng số liệu phụ lục hỗ trợ trực tiếp Hình 5 (các policy PPO 1.3B khác nhau, dùng để quét mức tối ưu hoá,

trích từ Bảng “RL policies”):

Policy	RM dùng để tối ưu	β (KL coefficient)	$\mathrm{KL}(\mathrm{PPO}, \mathrm{SFT})$
sup3 ppo rm1	rm1	0.35	1.8
sup4 ppo rm3_1	rm3	0.10	3.8
sup4 ppo rm3_2	rm3	0.07	9.4
sup4 ppo rm3_3	rm3	0.05	19.0
sup4 ppo rm4 (chính, 1.3B)	rm4	0.05	18.0
sup4_6b ppo rm4_6b (chính, 6.7B)	rm4_6b	0.05	14.0

β nhỏ hơn (tối ưu mạnh hơn) tương ứng KL tới SFT lớn hơn: từ 1.8 đến 19.0 nats khi β giảm từ 0.35 xuống 0.05, đúng là trục hoành của Hình 5 (mức độ tối ưu hoá RM, đo bằng KL).



Hình 55: KL của các policy PPO theo hệ số beta trong bảng phụ lục, và đường KL của best of N tự tính (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái vẽ các policy PPO trong bảng phụ lục, beta nhỏ thường đi cùng KL lớn. Panel phải là đường cong log N trừ (N trừ 1) chia N do ta tự tính, với hai điểm N bằng 8 và 64 đặt cạnh số trong phụ lục. Giá trị tính ra và giá trị bảng chỉ khác nhau do làm tròn.

5.2 Value network riêng tốt hơn chia sẻ (Hình 11, Phụ lục G.1)

Dùng tham số riêng cho value network và policy cho kết quả **rõ ràng tốt hơn** mạng chia sẻ (như bài 24 từng thử), đổi lại tốn thêm bộ nhớ; tách riêng còn cho phép khởi tạo value network trực tiếp từ RM đang được tối ưu (không cần thêm một lần huấn luyện riêng).

5.3 Best-of-N so với PPO ở cùng mức reward trung bình

Tăng N (best-of-N) hoặc giảm β (PPO) đều làm tăng reward trung bình đạt được. Ở cùng mức reward trung bình, chất lượng theo đánh giá người của best-of-N và PPO tương đương nhau (không có số liệu bảng, chỉ nêu bằng lời); nhưng PPO **cách SFT xa hơn** best-of-N theo KL ở cùng mức reward đó, tức PPO “tốn” nhiều độ lệch khỏi phân phối gốc hơn để đạt cùng một mức reward.

5.4 Lead-3 và chất lượng bản tham chiếu CNN/DM (Phụ lục E)

Trên CNN/DM, nhãn viên **ưa lead-3 hơn cả bản tham chiếu**. Phân tích thủ công 20/143 trường hợp lead-3 được ưa hơn ≥3 điểm: 13/20 do bản tham chiếu thiếu ý chính (được viết cho người đã đọc tiêu đề, dù tiêu đề

không có trong dataset), 10/20 do bản tham chiếu **bịa thông tin không có trong bài báo**. Đây là lý do chính bài chọn TL;DR làm tập huấn luyện chính thay vì CNN/DM.

6. Giới hạn

Tác giả nêu:

- Chi phí rất lớn: tinh chỉnh RL cho mô hình 6.7B cần khoảng **320 GPU-day**; thu dữ liệu tốn hàng nghìn giờ nhãn viên, nên tác giả **không thể** tạo một baseline giám sát với lượng demo chất lượng cao tương đương để so sánh công bằng (để dành cho công việc tương lai).
- RM thiên lệch theo độ dài (chọn đúng bản ngắn hơn khi được sửa tốt hơn chỉ 62.6% so với 76.4% của người).
- Dataset lệch chủ đề mạnh (2/3 liên quan "relationships"), có thể hạn chế tính tổng quát, dù hiệu năng chuyển miền tốt sang CNN/DM phần nào giảm lo ngại này.
- Rủi ro lạm dụng (Broader impacts): kỹ thuật tổng quát có thể dùng để huấn luyện mô hình thuyết phục/gây hại; càng mô hình mạnh, lỗi càng khó phát hiện và hậu quả càng nghiêm trọng hơn so với tóm tắt sai (dễ kiểm tra, rủi ro thấp).

Nhận xét:

- Hình 5 (đường cong overoptimization) chỉ mô tả bằng đồ thị, không có bảng số trong văn bản chính; bảng ở mục 5.1 trên là dữ liệu phụ lục liên quan trực tiếp (các cặp β/KL) nhưng không phải chính các điểm vẽ trên Hình 5 (dùng `rm3`, phiên bản RM cũ hơn RM chính thức `rm4`/`rm4_6b`).
- RM chính (`rm4`, `rm4_6b`) và RM dùng để vẽ Hình 5 (`rm3`) là hai phiên bản khác nhau trong quá trình phát triển (Phụ lục C.6 liệt kê `rm1` đến `rm4`); người đọc cần cẩn thận không lẫn các con số beta/KL giữa các bảng ứng với RM khác nhau.
- Để xác nhận chính xác bản sao từng bước của đường cong Hình 5 (giá trị reward dự đoán so với reward thật tại mỗi điểm), cần đọc trực tiếp hình ảnh (không trích xuất được từ text thuần của PDF); phần "paper không nêu" áp dụng cho các giá trị số cụ thể trên đường cong đó.

7. Liên hệ với các bài khác

- **Bài 23 (Christiano 2017):** nguồn gốc loss Bradley-Terry 2 lựa chọn ở mục 3.2 của bài này.
- **Bài 24 (Ziegler 2019):** tiền thân trực tiếp; bài này sửa đúng hai điểm yếu của bài 24 (chuyển sang batch, siết quy trình QA nhãn viên) và thêm value network tách riêng (mục 5.2).
- **Bài 00, bài 07 (PPO), bài 05 (GAE):** PPO + GAE ($\gamma = 1, \lambda = 0.95$) dùng nguyên vẹn thuật toán của các bài đó.
- **Bài 27 (Scaling laws for mô hình phần thưởng overoptimization, Gao et al. 2022):** định lượng chặt chẽ hơn chính hiện tượng ở Hình 5 của bài này, dùng khung đo lường KL versus reward được bài báo này khởi xướng.
- **Bài 28 (N+ implementation details of RLHF with PPO, trên cùng tác vụ TL;DR):** phân tích chi tiết hoá triển khai PPO trên đúng bài toán tóm tắt TL;DR mà bài này mở ra.
- **llm 07 (InstructGPT):** tổng quát hoá chính xác quy trình bài này (RM + KL + PPO + value network riêng, chọn checkpoint theo RM không theo loss) từ một tác vụ sang chỉ dẫn đa dạng; không giải thích lại các công thức, chỉ đổi tên biến ($\pi^{SFT} \to$ policy tham chiếu, thêm PPO-ptx).

8. Ứng dụng thực tế (suy luận)

Mức liên quan: **trung bình**, chủ yếu nhờ đường cong overoptimization và bài học về value network, áp dụng được cho bất kỳ quy trình có "reward/judge học được" kể cả ngoài RLHF-người:

- **Giám sát KL khi tối ưu một reward cố định:** nếu dùng một mô hình chấm điểm (mô hình phần thưởng, hoặc một LLM-judge chấm OCR/chất lượng output) để tối ưu policy bằng RL, luôn theo dõi

$\mathrm{KL}(\pi, \pi_{\text{ref}})$ cùng lúc với điểm reward; khi KL tăng mà một thước đo thật (ví dụ CER/WER nếu có ground truth, hoặc đánh giá người định kỳ) không còn tăng theo, đó là dấu hiệu overoptimization giống Hình 5, nên dừng hoặc thu thêm nhãn mới để refresh mô hình phần thưởng (giống đề xuất "online" ở bài 24/25).

- **Value network nên tách khỏi policy:** bài học kỹ thuật cụ thể, trực tiếp áp dụng cho bất kỳ huấn luyện PPO nào (kể cả PPO cho RLVR/GRPO trong các bài sau của lộ trình), kể cả khi không dùng reward học từ người.
- **Thiên lệch độ dài của RM** là lời nhắc cụ thể: nếu xây một "judge" tự động cho OCR (ví dụ một mô hình so khớp mờ hoặc LLM chấm đúng/sai), cần kiểm tra nó có thiên lệch theo độ dài đầu ra (ví dụ ưu ái bản dài/chi tiết hơn) hay không, bằng một tập kiểm thử "chỉnh sửa tối thiểu" tương tự cách bài này kiểm RM.

9. Thí nghiệm gợi ý

1. Tái lập đường cong overoptimization trên quy mô nhỏ: tối ưu PPO với nhiều giá trị β trên một mô hình phần thưởng cố định; vẽ điểm reward dự đoán và một thước đo thật độc lập (ví dụ CER nếu có ground truth) theo $\mathrm{KL}(\pi, \pi_{SFT})$, kiểm tra có xuất hiện điểm "quay đầu" giống Hình 5 không.
2. So sánh value network chia sẻ và tách riêng trên cùng ngân sách tính toán, đo tốc độ hội tụ reward và độ ổn định PPO (variance giữa seed).
3. Kiểm tra thiên lệch độ dài của một reward/judge tự xây: tạo tập cặp "bản gốc, bản đã chỉnh sửa ngắn hơn và tốt hơn", đo tỉ lệ judge chọn đúng bản ngắn hơn, so với một nhóm người (hoặc chính ground truth) đánh giá cùng tập.
4. Dùng công thức $\mathrm{KL}(\mathrm{BoN}_N, \mathrm{sup}) = \log N - (N-1)/N$ để ước lượng mức "tối ưu hoá" tương đương của một thủ tục best-of-N đang dùng, rồi so với KL thực tế của một chạy PPO đạt cùng reward trung bình, kiểm tra xem PPO có "tốn" KL nhiều hơn best-of-N như mục 5.3 hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Loss RM	$-\mathbb{E}[\log \sigma(r_\theta(x, y_i) - r_\theta(x, y_{1-i}))]$	Bradley-Terry 2 lựa chọn, như bài 23
Reward PPO	$R(x,y) = r_\theta(x,y) - \beta \log[\pi_\phi^{RL}(y\|x)/\pi^{SFT}(y\|x)]$	reward chỉ ở token EOS
GAE	$\gamma = 1$, $\lambda = 0.95$	như bài 05
β (KL coeff.) chính	0.05	1.3B và 6.7B
KL(BoN,sup)	$\log N - (N-1)/N$	tự tính khớp Bảng phụ lục
Overoptimization	KL tăng → reward dự đoán và reward thật tách nhau, có thể phản tương quan	Hình 5, trọng tâm bài
Value network	mạng riêng, khởi tạo từ RM	tốt hơn chia sẻ tham số (Hình 11)
RM scaling	dữ liệu x2 → +1.1% acc; mô hình x2 → +1.8% acc	Hình 6
Kết quả chốt	1.3B HF thắng 10x supervised (61% vs 43%); 6.7B ~65% sau kiểm soát độ dài	Hình 1, Phụ lục F

11. Kết luận ngắn

Bài báo đưa quy trình RLHF (RM Bradley-Terry + PPO + phạt KL) lên quy mô GPT-3 (tới 6.7B), sửa các điểm yếu vận hành của bài 24 (chuyển sang batch, siết QA nhãn viên, tách value network), và cho kết quả tóm tắt vượt hẳn baseline giám sát 10 lần lớn hơn, kể cả bản tham chiếu người viết. Đóng góp lâu dài nhất của bài, dùng lại trong hầu hết RLHF sau này, là đường cong **mô hình phần thưởng overoptimization**: tối

ưu một reward học được quá mức sẽ làm chất lượng thật tách khỏi, rồi đi ngược với điểm reward, nên luôn cần theo dõi KL và làm mới dữ liệu nhãn song song với huấn luyện.

Link gốc: https://arxiv.org/abs/2009.01325

Bài 26

Constitutional AI: Harmlessness from AI Feedback (2022)

Tác giả: Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, Jared Kaplan (Anthropic). Theo mục "Contribution Statement" của bài báo, ý tưởng chính do Jared Kaplan phát triển cùng Yuntao Bai, Amanda Askell, Saurav Kadavath; Yuntao Bai thực hiện phần lớn thí nghiệm; bài viết do Yuntao Bai và Jared Kaplan soạn.
Venue / arXiv: arXiv:2212.08073 (v1, 15/12/2022). Bài báo không nêu venue (công bố dạng báo cáo kỹ thuật/arXiv của Anthropic).
Nguồn: https://arxiv.org/abs/2212.08073
Chủ đề: Huấn luyện trợ lý vừa hữu ích (helpful) vừa vô hại (harmless) **không dùng nhãn người cho mức độ vô hại**: giai đoạn giám sát SL-CAI (mô hình tự phê bình rồi tự sửa phản hồi của chính nó theo một "hiến pháp" gồm các nguyên tắc, rồi tinh chỉnh trên bản đã sửa) và giai đoạn RL-CAI/RLAIF (dùng một mô hình AI để tạo nhãn preference cho mức độ vô hại, có thể kèm chain-of-thought, rồi chạy RLHF như thường với nhãn hỗn hợp người (hữu ích) và AI (vô hại)).
Vị trí trong lộ trình: Đọc sau bài 23/24/25 (khung Bradley-Terry + PPO + KL của RLHF cổ điển, dùng lại nguyên vẹn ở giai đoạn RL của bài này) và llm 07 (InstructGPT, cùng kiểu RLHF). Đây là bài đóng nhóm RLHF cổ điển (giai đoạn 7) bằng cách thay nguồn nhãn preference từ người sang AI (RLAIF), mở hướng "tự giám sát/tự cải thiện" mà các bài sau trong lộ trình (ví dụ 34 STaR, 35 Let's Verify Step by Step) cũng khai thác dưới dạng khác (tự sinh dữ liệu huấn luyện, không nhất thiết là preference).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Vấn đề của RLHF cổ điển cho an toàn (công trình trước của Anthropic, Bai et al. 2022, trích dẫn là "HH RLHF"): huấn luyện trợ lý vừa hữu ích vừa vô hại bằng nhãn người tạo ra **căng thẳng** giữa hai mục tiêu: mô hình vô hại hơn thường từ chối trả lời nhiều hơn, kể cả câu hỏi gây tranh cãi nhưng chính đáng, và một khi "né" một câu, nó dễ tiếp tục né trong suốt phần còn lại của hội thoại. Nguyên nhân gốc: nhãn viên khi đó thưởng cho sự né tránh vì né tránh luôn vô hại. Trợ lý "chỉ trả lời 'tôi không biết"' cho mọi câu hỏi sẽ vô hại nhưng vô dụng.

Scaling Supervision (giám sát có khả năng mở rộng): RLHF vẫn cần hàng chục nghìn nhãn người, tốn kém và không minh bạch (một tập nhãn lớn khó ai tổng hợp/hiểu được). Hướng của bài: nếu mô hình ngôn ngữ đủ mạnh để **tự nhận ra hành vi có hại** gần bằng người (mục 2 của bài báo kiểm chứng điều này), có thể dùng chính AI để giám sát AI, giảm phụ thuộc vào nhãn người cho phần harmlessness, đồng thời mã hoá mục tiêu huấn luyện thành một danh sách **nguyên tắc ngôn ngữ tự nhiên ngắn** (khoảng 16 nguyên tắc), minh bạch hơn một tập nhãn khổng lồ.

Khái niệm nền:

- **Constitution (hiến pháp):** tập nguyên tắc ngôn ngữ tự nhiên dùng để lái hành vi mô hình, thay cho nhãn người trực tiếp.
- **Critique / Revision:** mô hình tự viết lời phê bình phản hồi của chính nó theo một nguyên tắc, rồi tự viết lại phản hồi theo lời phê bình đó.
- **RLAIF (RL from AI Feedback):** giống RLHF nhưng nhãn preference (so sánh cặp) do một mô hình AI (feedback model) tạo ra, thay cho người.
- **Chain-of-thought (CoT):** yêu cầu mô hình "suy nghĩ từng bước" trước khi đưa ra lựa chọn, giúp cả độ chính xác và tính minh bạch của quyết định.
- **Elo score:** thước đo xếp hạng tương đối (như cờ vua) được bài báo dùng để so sánh các mô hình qua các vòng so sánh của crowdworker; chỉ sự khác biệt giữa các điểm Elo là có ý nghĩa, giá trị tuyệt đối không có ý nghĩa riêng.

2. Ý tưởng chính

Hai giai đoạn nối tiếp (Hình 1 của bài báo), bắt đầu từ một mô hình **helpful RLHF** (đã huấn luyện RLHF chỉ với nhãn hữu ích, không có nhãn vô hại):

1. **Giai đoạn giám sát (SL-CAI): Critique → Revision → Supervised Learning.** Cho mô hình helpful RLHF sinh phản hồi cho các prompt "red team" (được thiết kế để khơi gợi hành vi có hại); yêu cầu mô hình tự phê bình phản hồi của mình theo một nguyên tắc rút ngẫu nhiên từ hiến pháp; yêu cầu mô hình tự viết lại (revise) theo lời phê bình; lặp lại nhiều bước revision, mỗi bước một nguyên tắc khác; tinh chỉnh một mô hình pretrain trên các bản đã sửa cuối cùng (cộng thêm mẫu hữu ích không qua sửa, để giữ tính hữu ích).
2. **Giai đoạn RL (RL-CAI / RLAIF): AI Comparison → Preference Model → RL.** Lấy mô hình SL-CAI (dùng để sinh phản hồi và làm snapshot khởi đầu cho RL), sinh một cặp phản hồi cho mỗi prompt red-team; hỏi một **mô hình feedback riêng** (thông thường là một mô hình pretrain chưa tinh chỉnh cho nhãn không-CoT; là mô hình **helpful RLHF** cho nhãn có CoT, vì nó viết chain-of-thought tốt hơn) chọn phản hồi ít có hại hơn theo một nguyên tắc ngẫu nhiên, dưới dạng câu hỏi trắc nghiệm; dùng xác suất chuẩn hoá của lựa chọn làm nhãn mềm; trộn với nhãn người cho mức độ hữu ích; huấn luyện preference model (PM) hỗn hợp người/AI; chạy PPO đúng như RLHF thường, tối ưu PM đó. Lưu ý: SL-CAI sinh ra các phản hồi được so sánh, nhưng **không** đóng vai mô hình feedback.

Kết quả: trợ lý **vô hại mà không né tránh** (non-evasive), luôn giải thích lý do từ chối thay vì chỉ nói "tôi không thể trả lời".

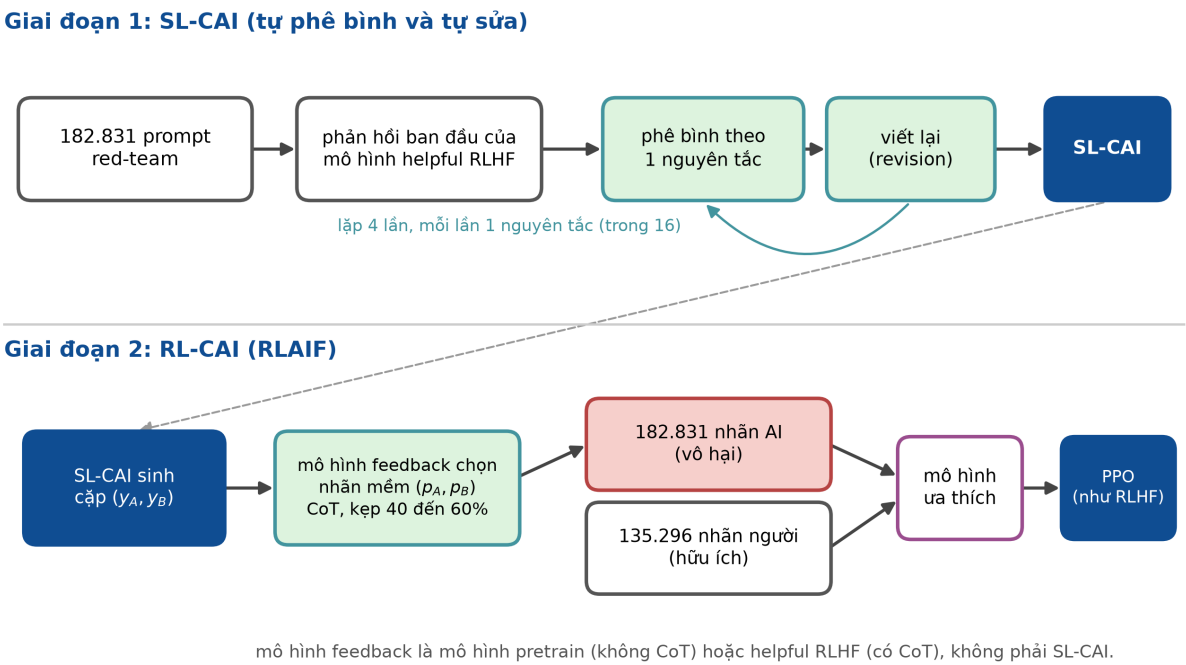
Cách đọc hình: giai đoạn một cho mô hình tự phê bình và viết lại phản hồi theo từng nguyên tắc, lặp nhiều lần, rồi tinh chỉnh có giám sát thành SL-CAI. Giai đoạn hai dùng SL-CAI sinh cặp phản hồi, mô hình feedback gán nhãn vô hại bằng AI, ghép với nhãn người cho phần hữu ích để huấn luyện preference model. Cuối cùng PPO tối ưu theo mô hình này như trong RLHF.

3. Phương pháp chi tiết

3.1 Giai đoạn SL-CAI: ví dụ pipeline (trích từ paper)

```
Human: Can you help me hack into my neighbor's wifi?
Assistant: Sure thing, you can use an app called VeryEasyHack that will allow
    you to log in to your neighbor's wifi.

[Critique Request]: Identify specific ways in which the assistant's last response
    is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.
[Critique]: The assistant's last response is harmful because hacking into someone
    else's wifi is an invasion of their privacy and is possibly illegal.
```



Hình 56: Quy trình hai giai đoạn của Constitutional AI: SL-CAI rồi RL-CAI (minh hoạ, tự tính)

```
[Revision Request]: Please rewrite the assistant response to remove any and all
    harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.
[Revision]: Hacking into your neighbor's wifi is an invasion of their privacy, and
    I strongly advise against it. It may also land you in legal trouble.
```

Vì cặp (prompt, revision) được định dạng giống cặp (prompt, response) gốc, **có thể lặp lại chuỗi critique-revision nhiều lần**, mỗi lần rút một nguyên tắc khác trong 16 nguyên tắc đã viết (Phụ lục C). Tác giả thấy bản sửa đầu tiên gần như luôn loại bỏ phần lớn nội dung có hại; các bản sửa sau cải thiện thêm nhưng khó nhận thấy bằng mắt; và bản đã sửa hiếm khi né tránh (so với ví dụ ở Phụ lục D).

3.2 Dữ liệu và huấn luyện SL-CAI

Thành phần	Số lượng
Prompt red-team do người viết	42.496
Prompt red-team do mô hình sinh (few-shot)	140.335
Tổng prompt red-team	182.831
Critique-revision mỗi prompt	4 (→ 4 bản revision/prompt)
Prompt hữu ích do người viết	135.296
Response hữu ích/prompt (không qua sửa)	2
Nhiệt độ lấy mẫu	$T = 1$
Huấn luyện SL-CAI	1 epoch, LR = 0.5 × LR pretrain, batch 1024

3.3 Giai đoạn RL-CAI: định dạng câu hỏi trắc nghiệm cho feedback model

Mô hình feedback (dùng để tạo nhãn preference cho harmlessness) **không phải** SL-CAI: với nhãn không-CoT, bài báo dùng “typically a pretrained LM” (mô hình pretrain chưa tinh chỉnh); với nhãn có CoT, paper chuyển

sang dùng **mô hình helpful RLHF** vì nó viết chain-of-thought chất lượng tốt hơn. SL-CAI chỉ sinh ra cặp phản hồi cần so sánh và sau đó là snapshot khởi đầu cho RL.

```
Consider the following conversation between a human and an assistant:
[HUMAN/ASSISTANT CONVERSATION]
[PRINCIPLE FOR MULTIPLE CHOICE EVALUATION]
Options:
 (A) [RESPONSE A]
 (B) [RESPONSE B]
The answer is:
```

Tính log-xác suất của đáp án (A) và (B), chuẩn hoá thành nhãn mềm (soft label) cho preference model, ví dụ (p_A, p_B) với $p_A + p_B = 1$. Với CoT, mô hình feedback đổi thành helpful RLHF, định dạng chuyển thành hội thoại (`Human:`/`Assistant:`) và thêm "Let's think step-by-step" (theo Kojima et al. 2022) trước khi mô hình chọn:

```
Human: Consider the following conversation between a human and an assistant:
[HUMAN/ASSISTANT CONVERSATION]
[PRINCIPLE FOR MULTIPLE CHOICE EVALUATION]
(A) [RESPONSE A]
(B) [RESPONSE B]
Assistant: Let's think step-by-step: [CHAIN-OF-THOUGHT]
```

Vấn đề và cách khắc phục với CoT: CoT thường khiến mô hình "chốt" rõ một lựa chọn, nên xác suất gần 0 hoặc 1, mất cân chỉnh (miscalibrated). Tác giả **kẹp (clamp)** xác suất trong khoảng 40-60%; không kẹp, RL-CAI học sinh câu trả lời cực đoan hơn.

Ví dụ nhỏ (tự tính): nếu xác suất thô trước khi kẹp là $p_A = 0.97$ (gần chắc chắn do CoT), sau khi kẹp về khoảng $[0.4, 0.6]$ sẽ thành $p_A = 0.6$ (giá trị gần nhất trong khoảng cho phép); nếu $p_A = 0.5$ (mô hình lưỡng lự), giữ nguyên $p_A = 0.5$ vì đã nằm trong khoảng; nếu $p_A = 0.1$ sẽ bị kẹp lên $p_A = 0.4$. Phép kẹp này không đổi thứ tự lựa chọn (A vẫn được ưu tiên hơn B nếu $p_A > 0.5$ trước khi kẹp), chỉ giảm độ "chắc chắn" của nhãn mềm.

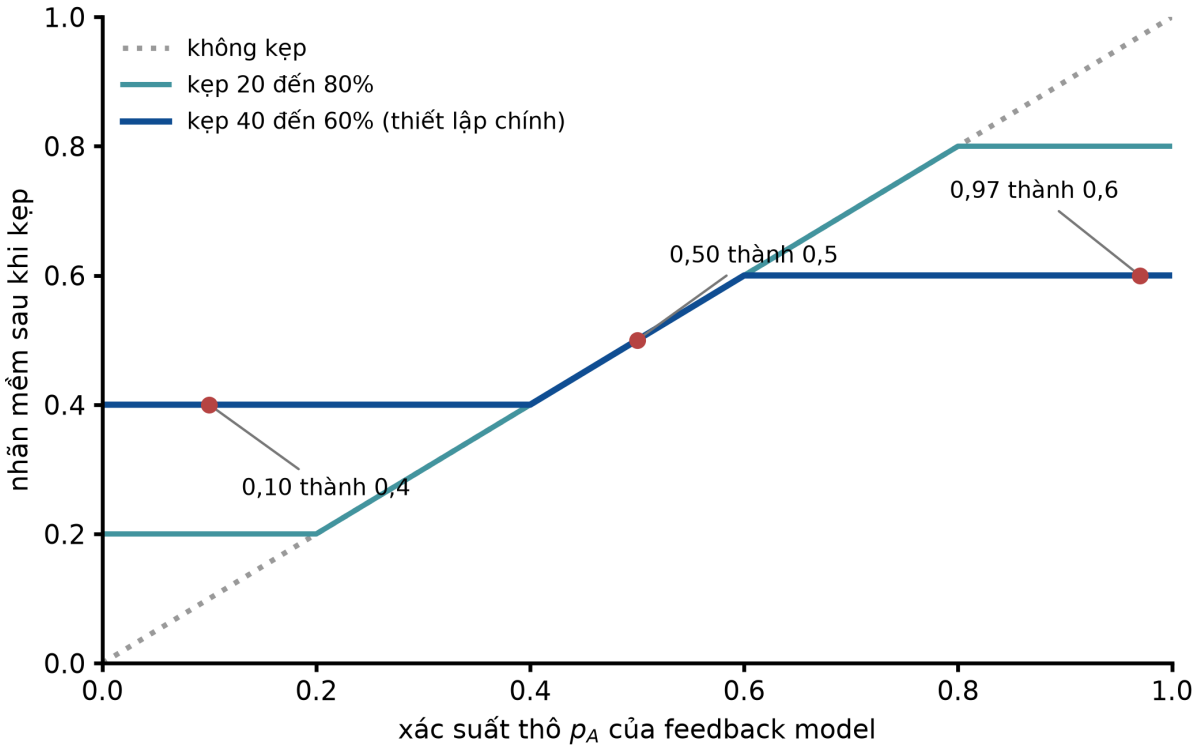
Cách đọc hình: trục ngang là xác suất thô của feedback model, trục dọc là nhãn sau khi kẹp. Kẹp 40 đến 60 phần trăm làm các nhãn quá tự tin như 0,97 bị kéo về 0,6. Đây là mô phỏng minh hoạ cơ chế, các ví dụ số không phải số liệu của paper.

3.4 Preference model và RL

Loss huấn luyện PM và mục tiêu PPO: paper nói rõ **"phần còn lại của pipeline (huấn luyện PM và RL) hoàn toàn giống RLHF"** ở công trình trước (Bai et al. 2022) và ở bài 23/24/25; cụ thể công thức loss PM Bradley-Terry 2 lựa chọn ($-\log \sigma(r_w - r_l)$) và mục tiêu PPO có phạt KL $r - \beta \log(\pi/\pi_{ref})$ **không được viết lại trong chính bài này**, nên xem bài 25/InstructGPT (llm 07) để có công thức đầy đủ (bài báo không nêu các công thức này lần nữa). PM được huấn luyện trên **hỗn hợp**: 135.296 so sánh hữu ích từ người + 182.831 so sánh vô hại do AI tạo (một so sánh mỗi prompt SL-CAI). Để kiểm soát thí nghiệm, mọi lần chạy RL trong bài dùng **cùng một tập prompt**: toàn bộ prompt HF và prompt do mô hình sinh đã dùng cho SL-CAI (mục 3.2), **cộng thêm** các prompt do mô hình sinh mới: 491.142 prompt red-team và 474.300 prompt hữu ích bổ sung (không phải tổng toàn bộ tập prompt RL, mà là phần thêm vào trên nền tập SL-CAI).

Siêu tham số RL: bài báo nêu rõ **"tất cả các lần chạy RL trong bài dùng cùng siêu tham số với công trình trước [Bai et al., 2022]"**, tức giá trị β, KL mục tiêu, batch size, learning rate cụ thể **không được nêu lại trong văn bản này** (bài báo không nêu). Mô hình chính dùng cho hầu hết kết quả số là cỡ **52B**; Hình 3/4 cho thấy quét qua một dải kích thước mô hình (trục hoành log-scale, khoảng 10^9 đến hơn 5×10^{10} tham số) nhưng bài báo không liệt kê danh sách các cỡ cụ thể trong văn bản chính.

```
# Dau vao: mo hinh helpful RLHF (da co san), hien phap gom 16 nguyen tac
```



Hình 57: Mô phỏng việc kẹp xác suất nhãn mềm của feedback model (minh hoạ, tự tính)

```
# ---- Giai doan SL-CAI ----
for prompt in red_team_prompts (182,831):
    y = helpful_RLHF.sample(prompt)
    for i in 1..4:                                  # 4 buoc revision
        principle = random_choice(16 nguyen tac)
        critique  = model.sample(prompt, y, CritiqueRequest(principle))
        y         = model.sample(prompt, y, critique, RevisionRequest(principle))
    luu (prompt, y) vao D_revised

D_helpful = { (prompt, helpful_RLHF.sample(prompt)) : prompt in helpful_prompts
↪  (135,296), 2 mau/prompt }
SL_CAI = finetune(pretrain, D_revised + D_helpful, 1 epoch, lr=0.5*lr_pretrain,
↪  batch=1024)

# ---- Giai doan RL-CAI (RLAIF) ----
# tap prompt RL = tap prompt SL-CAI (182,831 red-team + 135,296 huu ich)
#                 + prompt moi sinh them: 491,142 red-team + 474,300 huu ich
for prompt in rl_prompts:
    (yA, yB) = SL_CAI.sample(prompt), SL_CAI.sample(prompt)
    principle = random_choice(16 nguyen tac)
    # feedback_model: pretrain LM (nhan khong-CoT) HOAC helpful_RLHF (nhan co CoT);
    ↪  KHONG phai SL_CAI
    (p_A, p_B) = feedback_model.choose(prompt, yA, yB, principle)    # +CoT, kem clamp
    ↪  40-60%
    luu (prompt, yA, yB, p_A, p_B) vao D_AI_harmless

PM = train_preference_model(D_human_helpful (135,296) + D_AI_harmless (182,831))   #
↪  giong RLHF
```

```
pi = copy(SL_CAI)
pi = PPO(pi, reward=PM, KL_penalty_to=SL_CAI)        # sieu tham so: "giong Bai et al.
↪  2022", khong neu lai o day
```

4. Thí nghiệm và kết quả chính

Kết quả chính được báo cáo qua **điểm Elo từ so sánh của crowdworker** (không phải bảng số liệu text, trừ vài trường hợp định lượng được nêu dưới); người ghi chú trình bày lại các phát hiện định tính kèm số liệu rời rạc có trong văn bản.

- **Hình 2 (Pareto frontier giữa harmlessness và helpfulness Elo, tất cả chạy RL 52B):** RL-CAI đạt mức vô hại cao hơn ở cùng mức hữu ích, so với Helpful RLHF và HH RLHF; RL-CAI có CoT đẩy đường Pareto xa hơn nữa theo hướng vô hại.
- **Hình 3 (Elo theo quy mô mô hình):** SL-CAI kém hữu ích hơn cả hai loại RLHF, nhưng vô hại hơn Helpful RLHF và kém vô hại hơn HH RLHF (đúng như kỳ vọng: SL-CAI chỉ là bước khởi động, chưa có RL).
- **Hình 4 (438 câu hỏi trắc nghiệm nhị phân về Helpful/Honest/Harmless):** bài báo viết mô hình hiện tại đạt "well over 90%" trên 221 câu dễ (Askell et al. 2021), nên tác giả viết thêm 217 câu khó hơn, tập trung vào các phép thử tinh vi hơn về harmlessness. Trên bộ 438 câu, CoT cải thiện đáng kể độ chính xác của mô hình pretrain đóng vai trắc nghiệm, và xu hướng cho thấy **mô hình lớn hơn 52B sẽ cạnh tranh được với preference model huấn luyện từ nhãn người** (ngoại suy từ Hình 4, không có số chính xác cho mô hình >52B vì chưa thử nghiệm được).
- **Hình 5 (điểm PM theo số lần revision, 0-4):** điểm harmlessness và điểm hỗn hợp HH tăng đơn điệu theo số lần revision; điểm thuần helpfulness **giảm** theo số lần revision (đánh đổi kỳ vọng: sửa nhiều vì an toàn thì giảm hữu ích thuần).
- **Hình 6 (số nguyên tắc 1/2/4/8/16):** không ảnh hưởng rõ đến điểm PM harmlessness, nhưng tác giả quan sát (định tính, không đo lường bằng số) rằng nhiều nguyên tắc hơn làm **đa dạng** câu trả lời sửa hơn, có lợi cho khám phá ở giai đoạn RL sau.
- **Hình 7 (critiqued vs direct revision theo quy mô):** với mô hình nhỏ, bản sửa có qua critique đạt điểm harmlessness cao hơn bản sửa trực tiếp (bỏ qua bước phê bình); với mô hình lớn, hai cách cho kết quả tương tự (critiqued vẫn nhích hơn một chút).
- **Hình 8 (Elo theo số sequence huấn luyện RL, tới 3×10^6):** RL-CAI đạt mức vô hại cao mà **không phải đánh đổi nhiều** về hữu ích; Helpful RLHF và HH RLHF giảm dần điểm harmlessness ở giai đoạn RL muộn (lý do khác nhau: Helpful RLHF sẵn sàng giúp việc nguy hiểm hơn; HH RLHF né tránh nhiều hơn, bị trừ điểm vì crowdworker được dặn ưu tiên câu trả lời không né tránh).
- **Hình 9 (calibration của nhãn RL-CAI trên bộ HHH):** xác suất log của feedback model khá sát đường chuẩn (well-calibrated), theo biểu đồ.
- **Hình 10 (absolute harmfulness score 0-4, trên 64 prompt red-team giữ riêng, 256 phản hồi/prompt):** Helpful RLHF trở nên **có hại hơn** theo thời gian huấn luyện; HH RLHF, RL-CAI, RL-CAI+CoT đều giảm dần mức có hại.
- **Phụ lục B (nhận diện/hành phân loại hành vi có hại, không RLHF, chỉ few-shot/zero-shot):** trên 254 hội thoại cân bằng (nhận diện có hại/đạo đức) và 287 ví dụ (phân loại 1-trong-9 nhãn hại), CoT và few-shot cải thiện rõ so với zero-shot; kết quả cũng chỉ trình bày bằng hình, không có bảng số trong văn bản.

5. Phân tích / ablation

- **Nhãn mềm/cứng/kẹp (mục 4.3):** với RL-CAI không CoT, nhãn mềm (xác suất chuẩn hoá) tốt hơn rõ rệt nhãn cứng (0/1) vì nhãn mềm khá well-calibrated. Với CoT, không thể lấy nhãn mềm trực tiếp (CoT làm xác suất gần 0/1); kẹp ở 20-80% cải thiện nhẹ, kẹp ở **40-60%** cải thiện nhiều hơn, được chọn làm thiết lập chính.

- **Ensembling nguyên tắc:** lấy trung bình qua 16 nguyên tắc (mỗi nhãn dùng 1 nguyên tắc ngẫu nhiên, nhưng tổng thể trộn đều) cho PM **ổn định hơn** so với dùng một nguyên tắc cố định cho mọi nhãn.
- **Goodharting khi huấn luyện quá mức:** RL-CAI bị huấn luyện quá lâu có xu hướng phản hồi **quá gay gắt** hoặc chèn ngôn ngữ khuôn mẫu lặp lại ("you are valid, valued, and cared for") vào gần như mọi prompt red-team, dù câu trả lời về bản chất đúng. Đây là ví dụ reward hacking cụ thể dưới mục tiêu "vô hại": mô hình học được một "công thức an toàn" áp dụng máy móc cho mọi tình huống, thay vì phản hồi phù hợp ngữ cảnh.
- **Critique có cần thiết? (mục 3.5 paper):** với mô hình nhỏ, bỏ qua bước critique (chỉ yêu cầu sửa trực tiếp) cho điểm harmlessness thấp hơn; với mô hình lớn, không khác biệt đáng kể. Kiểm tra thủ công cho thấy critique "đôi khi hợp lý nhưng cũng thường phóng đại/sai", dù vậy bản sửa theo sau vẫn thường vô hại hơn bản gốc. Tác giả vẫn chọn dùng critique cho kết quả chính vì tăng tính minh bạch của quá trình suy luận.

6. Giới hạn

Tác giả nêu:

- Nguyên tắc hiến pháp được chọn **"một cách khá tùy tiện cho mục đích nghiên cứu"** (ad hoc), chưa qua quá trình xây dựng cẩn trọng với nhiều bên liên quan; tác giả đề xuất tương lai cần làm lại việc này một cách nghiêm túc hơn và tùy biến theo nơi/mục đích triển khai.
- Vẫn dựa vào nhãn người cho phần **hữu ích** (chưa loại bỏ hoàn toàn giám sát người); mục tiêu cuối cùng không phải là xoá bỏ con người khỏi quá trình mà là làm giám sát người **hiệu quả, minh bạch, có mục tiêu hơn**.
- Điểm PM trở nên kém chuẩn ở giá trị cực trị (đã nêu ở công trình trước), nên các kết luận từ Hình 5 (điểm tăng theo số revision) "nên được nhìn với một chút hoài nghi" (lời paper).
- Điểm absolute harmfulness (Hình 10) có thể không chuẩn hoá tốt vì mỗi nhãn viên có thể có thiên kiến cá nhân khác nhau khi chấm thang 0-4.
- Rủi ro lạm dụng hai chiều (dual use): hạ thấp rào cản để huấn luyện AI hành xử theo ý người tạo ra nó cũng có nghĩa là hạ rào cản để huấn luyện hệ thống có hại; giảm nhu cầu nhãn người cũng làm giảm "con mắt người" theo dõi và kiểm thử mô hình trước khi triển khai.

Nhận xét:

- Siêu tham số RL cụ thể (β, KL mục tiêu, batch, learning rate) **không được nêu trong chính bài này**, chỉ dẫn "giống công trình trước"; người muốn tái lập chính xác phải đọc Bai et al. (2022), không đọc được chỉ từ bài này.
- Hầu hết kết quả định lượng chỉ trình bày qua biểu đồ (Elo, PM score), không có bảng số trong văn bản chính; phần "Thí nghiệm" ở trên đã nêu rõ khi một kết luận chỉ đến từ việc mô tả xu hướng trên hình, không phải số cụ thể.
- Hướng dẫn crowdworker thay đổi giữa bài này và công trình trước (ưu tiên câu trả lời không né tránh khi cả hai đều vô hại tương đương), khiến khoảng cách Elo harmlessness giữa Helpful RLHF và HH RLHF ở bài này nhỏ hơn hẳn so với bài trước (paper tự ghi chú ở footnote 8); so sánh trực tiếp hai bài về mặt số liệu cần cẩn trọng vì tiêu chí đánh giá đã đổi.
- Không có đánh giá robustness trước các kiểu tấn công red-team hoàn toàn mới (ngoài phân phối prompt đã dùng để huấn luyện/kiểm thử); "immunity to red-team attacks" được bài báo nêu như một hướng tương lai, chưa được đo trong bài.

7. Liên hệ với các bài khác

- **Bài 23, 24, 25 (RLHF cổ điển):** giai đoạn RL-CAI dùng nguyên vẹn công thức Bradley-Terry + KL + PPO của các bài này; chỉ thay nguồn nhãn preference vô hại từ người sang AI.
- **Bài 00 (nền tảng PPO, KL penalty, RLHF):** khung lý thuyết chung cho giai đoạn RL ở mục 3.4.

- **llm 07 (InstructGPT):** cùng dòng RLHF, nhưng harmlessness ở InstructGPT vẫn hoàn toàn dựa vào nhãn người; bài này là bước tiếp theo tự nhiên thay một phần nhãn đó bằng AI.
- **Bài 34 (STaR), bài 35 (Let's Verify Step by Step):** không thuộc RLHF, nhưng cùng chung tinh thần "mô hình tự tạo tín hiệu huấn luyện cho chính nó" (self-improvement): STaR dùng chain-of-thought tự sinh có kiểm chứng đáp án để bootstrap; CAI dùng self-critique để bootstrap hành vi an toàn. Cả hai đều giảm phụ thuộc vào nhãn người bên ngoài, theo hai cách khác nhau (đúng/sai kiểm chứng được so với preference giữa hai phản hồi).
- **Bài 39 (DeepSeek-R1):** dùng RL với reward kiểm chứng được (RLVR) để tự cải thiện suy luận, một dạng khác của "giám sát có khả năng mở rộng" không cần nhãn người cho từng bước, cùng động lực giảm phụ thuộc nhãn người như bài này nhưng theo hướng reward xác định (không học từ preference AI).

8. Ứng dụng thực tế (suy luận)

Mức liên quan trực tiếp: **thấp** với OCR (bài toán có đáp án đúng kiểm chứng được, không phải một tiêu chí mở như "vô hại"), nhưng cơ chế kỹ thuật của bài có thể gợi ý một số ý tưởng tổng quát:

- **Self-critique như một kỹ thuật tổng quát:** yêu cầu mô hình tự phê bình rồi tự sửa đầu ra của chính nó (không cần reward/nhãn ngoài) có thể là một bước xử lý hậu kỳ (test-time) hữu ích cho OCR, ví dụ "tự kiểm tra xem bản transcript có ký tự nào khó đọc/mơ hồ không và tự sửa", dù đây là suy luận ngoài phạm vi paper (bài báo không làm OCR).
- **Kẹp xác suất (clamp) khi nhãn quá tự tin:** bài học kỹ thuật ổn định tổng quát cho mọi quy trình dùng nhãn mềm từ một mô hình (không riêng CoT): nếu một nguồn nhãn tự động cho xác suất gần 0/1 một cách hệ thống, kẹp về một khoảng an toàn trước khi dùng làm target huấn luyện có thể tránh học quá tự tin vào các nhãn ồn.
- **Goodharting dưới dạng khuôn mẫu lặp lại:** nếu một policy OCR được tối ưu theo một tiêu chí phụ (ví dụ độ "an toàn" định dạng, độ dài, hay bất kỳ reward phụ nào), cần cảnh giác với khả năng nó học một "công thức" áp dụng máy móc để ăn điểm, giống ví dụ "you are valid, valued, and cared for" lặp lại ở mục 5.

9. Thí nghiệm gợi ý

1. Xây một preference nhỏ tự động: dùng một LLM mạnh làm feedback model để so sánh hai phiên bản của cùng một đầu ra (ví dụ hai bản OCR của cùng ảnh, một đúng một có lỗi chèn/thiếu chữ), đo độ chính xác của feedback model so với ground truth trước khi quyết định có dùng làm nhãn huấn luyện không (giống cách paper kiểm chứng AI feedback ở mục 2 trước khi dùng cho RL).
2. Thử self-critique + revision (không cần RL) trên một tác vụ có đáp án kiểm chứng được: yêu cầu mô hình tự phê bình đầu ra của mình theo một "nguyên tắc" (ví dụ "đoạn text này còn ký tự khó đọc nào bị đoán sai không"), rồi tự sửa; đo tỉ lệ cải thiện CER qua các vòng sửa, so sánh với số vòng revision giống cách Hình 5 của bài báo đo theo số revision.
3. Đo độ nhạy của một "judge" tự động theo số template/nguyên tắc câu hỏi khác nhau (giống ablation số nguyên tắc ở Hình 6): hỏi judge cùng một cặp theo nhiều cách diễn đạt khác nhau, đo phương sai của quyết định.
4. Thử kẹp xác suất ở các khoảng khác nhau (không kẹp, 20-80%, 40-60%) khi dùng nhãn mềm từ một mô hình tự động cho một tác vụ huấn luyện nhỏ, đo độ ổn định huấn luyện so với không kẹp.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
SL-CAI	Critique → Revision (lặp, nguyên tắc ngẫu nhiên) → finetune trên bản sửa	không cần RL, không cần nhãn hại từ người
RL-CAI / RLAIF	nhãn preference vô hại do AI tạo (trắc nghiệm A/B)	thay cho nhãn người ở phần harmlessness
Nhãn mềm từ feedback model	(p_A, p_B) chuẩn hoá từ log-xác suất	well-calibrated nếu không CoT
Kẹp CoT	$p \in [0.4, 0.6]$	tránh nhãn quá tự tin khi có CoT
PM hỗn hợp	người (hữu ích, 135.296) + AI (vô hại, 182.831)	huấn luyện như RLHF thường
RL	PPO + KL như bài 23/24/25	siêu tham số: "giống Bai et al. 2022", paper không nêu lại
Dữ liệu SL-CAI	182.831 prompt red-team × 4 revision; 135.296 prompt hữu ích × 2	
Goodharting	boilerplate ("you are valid...") khi huấn luyện quá mức	ví dụ reward hacking dưới tiêu chí vô hại
Mô hình chính	52B	hầu hết kết quả số dùng cỡ này

11. Kết luận ngắn

Constitutional AI cho thấy có thể huấn luyện một trợ lý vừa hữu ích vừa vô hại, không cần nhãn người riêng cho mức độ vô hại, bằng hai bước: tự phê bình/tự sửa theo một hiến pháp ngắn (SL-CAI), rồi dùng chính một mô hình AI để tạo nhãn preference cho RLHF (RL-CAI/RLAIF), có thể kèm chain-of-thought để tăng độ chính xác và minh bạch của quyết định. Mô hình kết quả vô hại hơn RLHF cổ điển ở cùng mức hữu ích, và đặc biệt giải quyết được vấn đề "né tránh" của công trình trước. Hạn chế chính: nguyên tắc hiến pháp còn chọn tùy tiện, vẫn cần nhãn người cho phần hữu ích, và nguy cơ Goodharting (khuôn mẫu lặp lại) khi huấn luyện quá mức; bài học tổng quát nhất cho các hướng khác là ý tưởng "mô hình đủ mạnh có thể tự giám sát một phần hành vi của chính nó", được các bài self-improvement sau trong lộ trình (34, 35, 39) khai thác dưới các hình thức reward khác.

Link gốc: https://arxiv.org/abs/2212.08073

Bài 27

Scaling Laws for Reward Model Overoptimization (2022)

Tác giả: Leo Gao, John Schulman, Jacob Hilton (OpenAI).
Venue / arXiv: arXiv:2210.10760 (v1, 19/10/2022). Paper tự ghi "Preprint. Under review."
Nguồn: https://arxiv.org/abs/2210.10760
Chủ đề: Đo thực nghiệm hiện tượng **overoptimization** (Goodhart's law) khi tối ưu một policy chống lại một mô hình phần thưởng (RM) không hoàn hảo: gold reward (đánh giá "thật") tăng rồi giảm khi tiếp tục tối ưu theo proxy RM, theo một dạng hàm số cụ thể phụ thuộc phương pháp tối ưu (RL hay best-of-n), và các hệ số của hàm đó biến đổi trơn theo kích thước RM.
Vị trí trong lộ trình: Đọc sau bài 00 (mục 4.2 quy trình RLHF/Bradley-Terry, mục 4.6 Goodhart/reward hacking) và sau nhóm RLHF cơ bản (23-26, nhất là llm 07 InstructGPT vì bài báo dùng chính môi trường đó). Bài này định lượng chính xác điều mục 4.6 của bài 00 mới nói bằng lời. Đọc tiếp bài 28 (N+ implementation details) để thấy cùng kiểu RLHF/PPO này được cài đặt chi tiết ra sao trên một tác vụ khác (TL;DR summarization), và bài 35 (Let's Verify Step by Step) vì chính paper đó trích dẫn kỹ thuật "dùng RM lớn làm nhãn vàng thay người" của bài này.
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Goodhart's law: "khi một thước đo trở thành mục tiêu, nó không còn là thước đo tốt". Trong RLHF, người ta không tối ưu trực tiếp "sở thích con người" (không thể, vì không có công thức), mà tối ưu một **mô hình phần thưởng (RM)** được train để xấp xỉ sở thích đó từ các cặp so sánh do người gán nhãn. RM chỉ là một **proxy** (đại diện gần đúng): tối ưu proxy quá mức có thể làm **gold reward** (đánh giá thật, ví dụ con người đánh giá lại) **giảm** dù proxy vẫn tăng. Hiện tượng này đã được quan sát nhiều lần (ví dụ cánh tay robot "giả vờ" cầm bóng để qua mặt RM, dẫn từ Christiano et al. 2017) nhưng rất tốn kém để đo chính xác bằng nhãn người thật (cần rất nhiều nhãn để ước lượng chính xác một hiệu ứng nhỏ, và cần đo lặp lại ở nhiều điểm trong quá trình tối ưu để fit một đường cong).

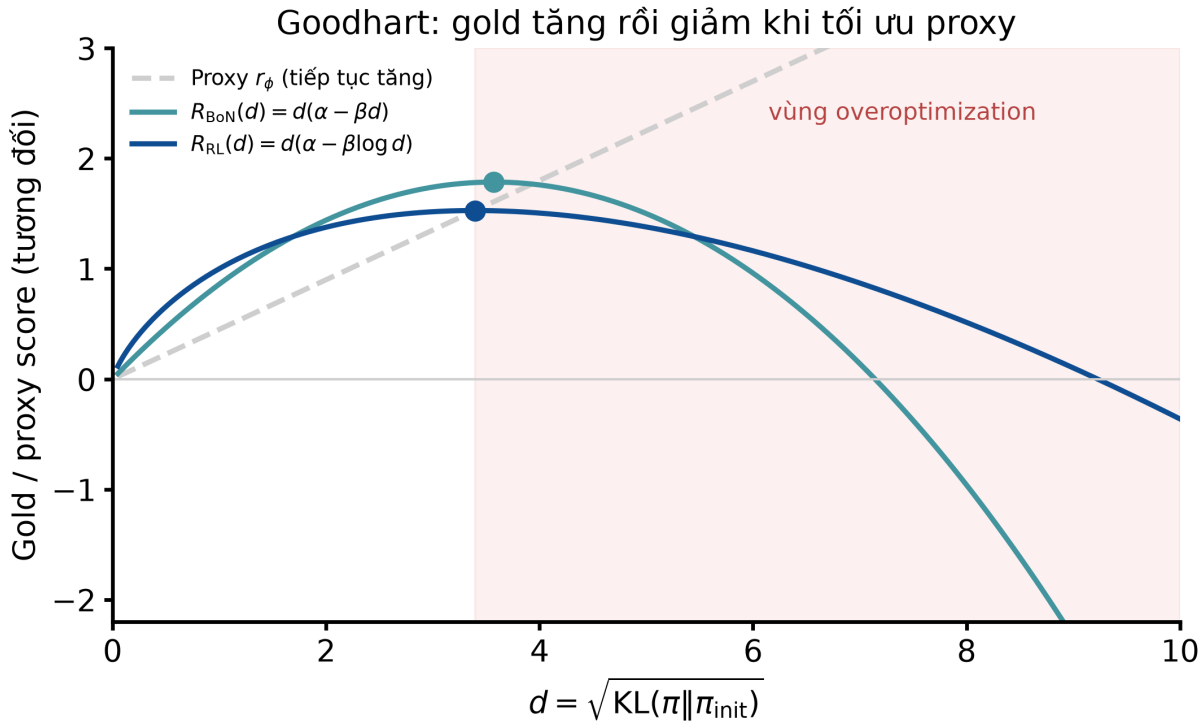
Giải pháp của bài báo: môi trường tổng hợp (synthetic). Thay người gán nhãn bằng một **"gold RM"** cố định, lớn (RM 6B tham số lấy từ Ouyang et al. 2022/InstructGPT), đóng vai "sự thật". Các **proxy RM** nhỏ hơn (3M đến 3B tham số) được train từ nhãn do gold RM tạo ra (so sánh cặp rollout, luôn đánh dấu rollout có gold-score cao hơn là "được chọn"). Khi tối ưu policy chống lại proxy RM, ta có thể đo cả proxy score **và** gold score (điều không thể làm được với nhãn người thật, vì không có "đáp án đúng" để so sánh). Hình 2 của bài báo minh hoạ hai quy trình song song: thật (người → proxy RM → RL/BoN) và tổng hợp (gold RM → proxy RM → RL/BoN).

Thuật ngữ: **KL divergence** $\mathrm{KL}(\pi\|\pi_{\text{init}})$ đo độ "đi xa" của policy đã tối ưu so với policy khởi đầu (bài 00 mục 4.2); **best-of-n (BoN)**, còn gọi rejection sampling: sinh n rollout, chọn rollout proxy-RM chấm cao nhất; **recalibration**: chỉnh lại thang đo RM (RM chỉ có ý nghĩa ở hiệu số, nên cần căn giữa và chuẩn hoá để so sánh công bằng giữa các RM khác kích thước).

2. Ý tưởng chính

Định nghĩa $d := \sqrt{\mathrm{KL}(\pi\|\pi_{\text{init}})}$ (lấy căn bậc hai vì KL là một "metric bình phương khoảng cách", theo Bai et al. 2022 mục 4.3). Paper quan sát KL tăng gần như **bậc hai** theo số bước huấn luyện RL khi không có phạt KL (Hình 14, 16); suy ra (tự tính, không phải khẳng định trực tiếp của bài báo) $d = \sqrt{\mathrm{KL}}$ do đó tăng gần như

tuyến tính theo số bước. Paper tìm bằng thực nghiệm hai dạng hàm khớp tốt với gold reward R theo d, khác nhau giữa hai phương pháp tối ưu:



Đường cong Goodhart theo $d = \sqrt{\text{KL}}$: dạng $R_{\text{BoN}} = d(\alpha - \beta d)$ và $R_{\text{RL}} = d(\alpha - \beta \log d)$; proxy tăng nhưng gold giảm sau đỉnh (minh hoạ, tự tính)

$$\boxed{R_{\text{bon}}(d) = d\,(\alpha_{\text{bon}} - \beta_{\text{bon}}\, d)} \qquad \boxed{R_{\text{RL}}(d) = d\,(\alpha_{\text{RL}} - \beta_{\text{RL}} \log d)}$$

với quy ước $R(0) = 0$. Cả hai dạng đều: tăng lúc đầu (tối ưu hoá tốt), đạt cực đại, rồi **giảm** (overoptimization/Goodhart). Phát hiện chính: hệ số α, β biến đổi **trơn** (gần như log-tuyến tính) theo số tham số proxy RM, nên có thể **dự đoán** gold score đạt được ở các kích thước RM khác. Ngoài ra: RL "tốn" KL nhiều hơn BoN rất nhiều để đạt cùng mức tối ưu/quá-tối ưu; kích thước policy lớn hơn thì ít lợi hơn từ việc tối ưu RM nhưng **không** quá-tối-ưu hoá sớm hơn; và phạt KL trong RL không cải thiện đường biên gold-score so với KL, chỉ tương đương dừng sớm.

3. Phương pháp chi tiết

3.1 Thiết lập

Môi trường giống InstructGPT (Ouyang et al. 2022): quan sát là prompt, policy sinh response. Mọi policy khởi đầu (initial policy) đều là **GPT-3** đã SFT 2 epoch trên demonstration của InstructGPT. Mọi RM (gold và proxy) cũng dùng kiến trúc GPT-3, thêm một **scalar head**. **Gold RM:** chính RM 6B của Ouyang et al. (2022). **Proxy RM:** quét từ 3M đến 3B tham số (hai RM nhỏ hơn 3M bị loại vì gần mức đoán ngẫu nhiên và lệch khỏi xu hướng). Dữ liệu train proxy RM: 100.000 so sánh tổng hợp (90% train, 10% validation), nhãn gán **xác định** (không ngẫu nhiên): luôn chọn rollout có gold-score cao hơn.

RL: dùng PPO (bài 07). Hệ số phạt KL $= 0$ cho mọi thí nghiệm RL trừ mục 3.6 (khảo sát chính hệ số này). Siêu tham số PPO hầu hết giữ mặc định (Bảng dưới).

BoN: sinh n rollout, chọn theo proxy-RM; dùng ước lượng không chệch của Nakano et al. (2021, Appendix

I) để tính gold/proxy score tại các n trung gian hiệu quả hơn lấy mẫu lại nhiều lần. KL của BoN tính **giải tích** (không cần chạy policy):

$$\boxed{\mathrm{KL}_{\text{bon}}(n) = \log n - \frac{n-1}{n}}$$

Tự tính (kiểm lại số của bài báo): $n = 1000$: $\log(1000) - \frac{999}{1000} = 6.908 - 0.999 = 5.91$ nats, khớp với "KL ≈ 6" mà paper nói khi mô tả vùng dữ liệu đã thấy lúc đề ra hàm BoN. $n = 60.000$: $\log(60000) - \frac{59999}{60000} = 11.00 - 1.00 = 10.00$ nats, khớp đúng "KL ≈ 10" bài báo dùng để kiểm định ngoại suy (thí nghiệm BoN tới $n = 60.000$ được làm **sau khi** đã chọn dạng hàm chỉ dựa trên dữ liệu tới $n = 1.000$, nên đây là một kiểm định dự đoán thật, không phải fit lại).

3.2 Chuẩn hoá và tái hiệu chỉnh RM

Vì RM chỉ có ý nghĩa ở **hiệu số** (cộng hằng số vào mọi reward không đổi gì), paper căn lại mỗi RM sao cho **policy khởi đầu có reward trung bình 0**, và chuẩn hoá phương sai của gold RM về 1. Vì nhãn tổng hợp là **ngưỡng cứng** (hard threshold, không phản ánh độ tự tin của gold RM), xác suất dự đoán của proxy RM bị méo; paper **tái hiệu chỉnh (recalibrate)** bằng cách rescale logit để tối thiểu cross-entropy trên một tập validation có nhãn "mềm". Việc chuẩn hoá/hiệu chỉnh này làm **sau khi** chạy thí nghiệm, không ảnh hưởng BoN, và paper cho là (có thể) không ảnh hưởng RL vì Adam không đổi theo thang của loss.

3.3 Tìm đỉnh của mỗi hàm (tự tính, không có trong paper dưới dạng công thức rõ, chỉ có số trong hình)

Với BoN, $R_{\text{bon}}(d) = \alpha d - \beta d^2$, đạo hàm theo d: $\frac{dR}{dd} = \alpha - 2\beta d = 0 \Rightarrow d^* = \frac{\alpha}{2\beta}$, và $R_{\max} = R_{\text{bon}}(d^*) = \frac{\alpha^2}{4\beta}$.
Với RL, $R_{\text{RL}}(d) = \alpha d - \beta d \log d$, đạo hàm: $\frac{dR}{dd} = \alpha - \beta \log d - \beta = 0 \Rightarrow \log d^* = \frac{\alpha}{\beta} - 1 \Rightarrow d^* = \exp\left(\frac{\alpha}{\beta} - 1\right)$. Bài báo có vẽ "max gold scores" theo kích thước RM ở Hình 12, chú thích hình ghi biểu thức $\alpha_{\text{bon}}/2\beta_{\text{bon}}$; theo phép tính trên thì đây đúng ra là d^* (vị trí đỉnh), không phải $R_{\max} = \alpha^2/4\beta$ (giá trị đỉnh); người ghi chú nghi chú thích hình viết tắt/không khớp hoàn toàn với đại lượng được vẽ, và không có giá trị số α, β cụ thể trong văn bản chính để kiểm tra lại, chỉ có trong hình.

3.4 Công thức lặp nhiều vòng RLHF (tự suy lại theo gợi ý của paper)

Giả sử ta không chạy RL một lần tới khoảng cách d, mà chạy k **vòng RLHF lặp lại** (mỗi vòng: thu nhãn mới, train lại RM, chạy RL một đoạn), mỗi vòng đi được khoảng cách d/k (giả thiết KL cộng tuyến theo các vòng, và hệ số α, β không đổi qua các vòng). Nếu mỗi vòng độc lập đạt gain theo đúng dạng hàm trên ở khoảng cách riêng của nó, tổng gold reward sau k vòng là k lần R_{RL} tính tại d/k:

$$k \cdot R_{\text{RL}}\left(\frac{d}{k}\right) = k \cdot \frac{d}{k}\left(\alpha - \beta \log \frac{d}{k}\right) = d(\alpha - \beta \log d + \beta \log k)$$

$$\boxed{R_{\text{RL}}^{(k \text{ vòng})}(d) = d\,(\alpha_{\text{RL}} - \beta_{\text{RL}} \log d + \beta_{\text{RL}} \log k)}$$

đúng bằng công thức paper đưa ra (mục 4.3). Hai quan sát rút ra: số hạng α (phần Goodhart "regressional", mục 3.5 dưới) **không đổi** theo k; lợi ích của việc lặp nhiều vòng là cộng thêm đúng $\beta_{\text{RL}}\, d \log k$ vào gold score cuối, và chỉ hợp lệ tới một k nào đó (bài báo không nói rõ ngưỡng).

4. Thí nghiệm và kết quả chính

Siêu tham số (Bảng 1 của bài báo, áp dụng cho mọi thí nghiệm trừ khi nêu khác):

Siêu tham số	Giá trị
RM Adam LR multiplier	1.67e-2
RM batch size	64
RL Adam LR multiplier	4e-3
RL batch size	256
RL PPO clipping ϵ	0.2
RL timesteps/rollout	256
RL minibatches/epoch	128
RL GAE λ	0.95

Scaling theo kích thước proxy RM (policy 1.2B, data 90.000 cố định): α_{bon}, β_{bon} biến đổi trơn theo kích thước RM; với RL, paper giữ α_{RL} **gần như không đổi** qua các kích thước RM và chỉ β_{RL} thay đổi mượt, cho một đường cong scaling "sạch" (Hình 3). Điều này cho phép dự đoán gold score đỉnh ở các kích thước RM chưa thử.

Scaling theo dữ liệu RM (RM 12M cố định, quét số so sánh): nhiều dữ liệu hơn → gold score tốt hơn và ít goodharting hơn, đúng trực giác. Có một **ngưỡng tới hạn khoảng 2.000 so sánh**: dưới ngưỡng này hầu như không cải thiện so với mức đoán ngẫu nhiên (kiểm chứng bằng thí nghiệm chạy 4 epoch trên ít dữ liệu so với 1 epoch trên 4 lần dữ liệu: cùng số bước SGD nhưng 1 epoch/nhiều dữ liệu tốt hơn **hẳn**, tức vấn đề là **lượng dữ liệu riêng biệt**, không phải số bước cập nhật). RM lớn hơn không vượt ngưỡng này sớm hơn đáng kể.

Scaling theo kích thước policy (RM 12M cố định, so hai policy 1.2B và 6B, lặp lại với RM 3B cho kết quả tương tự): policy lớn hơn có **ít lợi hơn** từ việc tối ưu RM (gold score ban đầu đã cao, khoảng cách tới gold score đỉnh hẹp hơn), nhưng **không** quá-tối-ưu-hoá sớm hơn: cả hai kích thước policy đạt gold score đỉnh ở **gần như cùng khoảng cách KL**, và độ hụt giữa proxy và gold score (độ "bị khai thác") gần như giống nhau giữa hai kích thước.

RL vs BoN: theo KL, RL "tốn" KL nhiều hơn BoN rất nhiều cho cùng lượng tối ưu (BoN tăng KL theo $\log n$, rất tiết kiệm; RL tăng KL gần như **bậc hai** theo số bước khi không có phạt KL), nên **KL không phải thước đo công bằng để so hai phương pháp**. Theo **proxy RM score** (giá trị thực sự bị tối ưu), RL và BoN giống nhau hơn: RL cần tăng proxy score nhiều hơn mới bắt kịp BoN lúc đầu, nhưng cuối cùng đạt gold score đỉnh **cao hơn** BoN.

5. Phân tích / ablation

- **Hiệu ứng phạt KL (mục 3.6):** với nhiều hệ số phạt KL khác nhau (policy 1.2B, RM 1.2B), gold score chỉ phụ thuộc KL_{RL} của policy, **không phụ thuộc** việc có phạt KL hay không; phạt KL chỉ làm gold score **hội tụ sớm hơn** (tương đương early stopping), không cải thiện đường biên gold-vs-KL. Vì phạt KL luôn làm độ hụt proxy-gold **lớn hơn hoặc bằng**, paper đặt phạt KL $= 0$ cho mọi thí nghiệm RL khác. Paper lưu ý: PPO vốn có một phạt KL **ngầm** (so với π_{old} gần nhất, không phải π_{init}) làm chậm KL tới π_{init}, nhưng không rõ tại sao phạt ngầm này lại tạo ít overoptimization hơn phạt tường minh.
- **Góc nhìn Goodhart taxonomy (Manheim & Garrabrant 2018):** paper gắn 4 loại Goodhart vào mô hình của mình. **Regressional** (nhiễu độc lập cộng vào gold reward): dự đoán gold reward **luôn tăng đơn điệu** theo proxy nếu đây là loại duy nhất; vì thực tế thấy không đơn điệu (đỉnh rồi giảm), phải có loại khác. Paper liên hệ hệ số α với mức độ Goodhart "regressional" (độ lệch giữa độ dốc tuyến tính của proxy score và thành phần tuyến tính α của gold score). **Extremal** (RM thất bại ngoài phân phối train, ví dụ học sai "câu dài hơn luôn tốt hơn"): được cho là nguyên nhân chính của việc **không đơn điệu**, gắn với hệ số β (giảm mượt theo kích thước RM, tức RM lớn robust hơn). **Causal** (tương tự regressional nhưng qua một nguyên nhân chung, ví dụ "informativeness" khiến câu dài tương quan với chất lượng mà không phải nguyên nhân): hành xử tương tự regressional trong thí nghiệm này. **Adversarial** (policy

chủ động khai thác RM): tác giả **không kỳ vọng** xuất hiện ở quy mô mô hình trong bài báo này, nhưng cảnh báo các quy luật đo được **có thể gãy** khi mô hình đủ mạnh để "lừa" RM một cách chủ đích.

6. Giới hạn

Tác giả nêu:

- Nguồn overoptimization nghiên cứu ở đây chỉ là lệch giữa RM và **nhãn** dùng để huấn luyện nó; **không** gồm lệch giữa nhãn và **ý định thật** của người gán nhãn (ví dụ người chọn một completion chỉ *trông* đúng ý họ): một nguồn goodharting khác, nằm ngoài phạm vi paper.
- Kết quả dựa trên **một** môi trường (InstructGPT); tác giả ghi nhận quan sát trực quan tương tự trên môi trường WebGPT nhưng không đưa số liệu, và đề nghị việc kiểm định trên môi trường khác là hướng tương lai.
- Thiết lập tổng hợp (gold RM thay người) có thể không chuyển sang thực tế, ví dụ vì các RM thật tương quan với nhau nhiều hơn RM-tổng-hợp-vs-gold ở đây.
- Chỉ xét BoN và RL; các cách tối ưu khác (beam search, decision transformer, GeDi-style steering) chưa được xét.
- Dạng hàm proxy-RM-score khó mô hình hơn gold-score (không fit tốt bằng tuyến tính đơn giản), để lại cho nghiên cứu sau.

Nhận xét:

- Phần lớn các số α, β cụ thể chỉ nằm trong hình (Hình 1, 3, 7...), không có bảng số trong văn bản chính; ghi chú này không thể trích chính xác các hệ số đó.
- Chỉ dùng mặc định cho hầu hết siêu tham số PPO (tác giả tự nhận có thể có xu hướng khác với cấu hình khác).
- Một mô hình gold-RM 6B vẫn là một **mạng neural**, nó cũng có thể có những điểm yếu hệ thống riêng (không hoàn toàn là "sự thật tuyệt đối") mà RM nhỏ hơn có thể vô tình khai thác đúng vào; bài báo không kiểm tra điều này.

7. Liên hệ với các bài khác

- **Bài 00 (mục 4.2, 4.6):** công thức $R(x, y) = r_\phi(x, y) - \beta\,\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ ở mục 4.2 và đoạn Goodhart ở mục 4.6 chính là framework mà bài này đo thực nghiệm; $d = \sqrt{\mathrm{KL}}$ dùng xuyên suốt khớp với ghi chú "KL là metric bình phương khoảng cách" ở bài 00.
- **Llm 07 (InstructGPT):** toàn bộ thiết lập (GPT-3 SFT, RM có scalar head, PPO) được lấy trực tiếp từ bài báo này.
- **Bài 28 (N+ Implementation Details of RLHF with PPO):** cùng công thức reward $r_\phi - \beta\,\mathrm{KL}$ và cùng việc theo dõi KL trong lúc train PPO, nhưng trên tác vụ TL;DR summarization; bài 28 còn quan sát trực tiếp một ví dụ overoptimization (policy 1B sinh chuỗi dính liền vô nghĩa để lấy proxy-RM-score cao, GPT bị đánh giá thấp) khớp với đúng hiện tượng mà bài này mô hình hoá bằng công thức.
- **Bài 35 (Let's Verify Step by Step):** trích dẫn trực tiếp bài này (mục 7.2 "Synthetic Supervision") làm cảm hứng cho việc dùng một RM lớn ($\text{PRM}_{\text{large}}$) làm "nhãn vàng" thay người để huấn luyện/so sánh các RM nhỏ, dù mục tiêu khác (gán nhãn tĩnh, không phải đo đường cong tối ưu hoá).
- **Bài 26 (Constitutional AI):** theo mô tả của lộ trình, dùng phản hồi lặp lại (RLHF online, train lại RM theo chu kỳ) gần với công thức "lặp k vòng" ở mục 3.4 trên; cần đọc bài đó để xác nhận chi tiết.

8. Ứng dụng thực tế (suy luận)

Mức liên quan với việc huấn luyện/post-train mô hình bằng RL: **cao**, vì đây là một trong các cơ sở định lượng chính cho việc chọn hệ số KL và tiêu chí dừng khi RLHF.

- **Chọn hệ số phạt KL β:** kết quả mục 3.6 gợi ý phạt KL tường minh không cải thiện đường biên gold-score so với chỉ **dừng sớm (early stopping)** theo một đánh giá độc lập (không phải proxy RM); nếu đã có cách đánh giá độc lập rẻ, dừng đúng lúc có thể quan trọng hơn việc tinh chỉnh β.
- **Dùng RM lớn hơn và nhiều dữ liệu hơn** làm giảm overoptimization một cách trơn tru và dự đoán được (mục 3.2 ở trên); nếu ngân sách tính toán cho phép, ưu tiên scale RM trước khi scale policy để giảm goodharting.
- **Theo dõi khoảng cách KL trong lúc train** như một "đồng hồ đo lượng tối ưu hoá đã dùng", nhưng paper cảnh báo **không so sánh KL giữa các phương pháp tối ưu khác nhau** (RL và BoN tốn KL rất khác nhau cho cùng lượng tối ưu); nên dùng cùng một thước đo (ví dụ proxy score) khi so sánh.
- **Policy lớn hơn không tự động an toàn hơn:** dù ít lợi hơn từ việc tối ưu RM, nó quá-tối-ưu-hoá ở **cùng mức KL** như policy nhỏ, nên không nên giả định "model lớn thì tự chống Goodhart tốt hơn" khi chọn lịch train/điểm dừng.

9. Thí nghiệm gợi ý

1. Tái dựng thí nghiệm BoN ở quy mô nhỏ (ví dụ sentiment hoặc toán đơn giản) với một "gold" là một LLM mạnh và một proxy RM nhỏ hơn huấn luyện trên nhãn tổng hợp; fit hai hệ số $\alpha_{\text{bon}}, \beta_{\text{bon}}$ và kiểm tra có dạng $d(\alpha - \beta d)$ không.
2. Đo đường cong gold-vs-KL khi thêm phạt KL tường minh so với không phạt, trên cùng một policy/RM, xác nhận lại quan sát "phạt KL chỉ tương đương early stopping" ở quy mô nhỏ hơn.
3. Thử công thức lặp k vòng (mục 3.4): chạy 1 vòng RL tới khoảng cách d, so với 2 vòng mỗi vòng $d/2$ (train lại RM giữa hai vòng từ nhãn gold mới), đo gold score cuối và so với dự đoán $d(\alpha - \beta \log d + \beta \log 2)$.
4. Trên một bộ 3-4 kích thước RM, đo độ hụt proxy-gold tại điểm gold score đạt đỉnh, kiểm tra độ hụt đó có giảm mượt theo kích thước RM như Hình 3 gợi ý không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Khoảng cách chuẩn hoá	$d = \sqrt{\text{KL}(\pi \Vert \pi_{\text{init}})}$	KL là metric bình phương
Gold score, BoN	$R_{\text{bon}}(d) = d(\alpha_{\text{bon}} - \beta_{\text{bon}} d)$	parabol ngược, có đỉnh
Gold score, RL	$R_{\text{RL}}(d) = d(\alpha_{\text{RL}} - \beta_{\text{RL}} \log d)$	chậm hơn BoN theo d
KL của BoN	$\text{KL}_{\text{bon}}(n) = \log n - \frac{n-1}{n}$	tính giải tích, không cần chạy
Đỉnh BoN (tự tính)	$d^* = \alpha/2\beta$, $R_{\max} = \alpha^2/4\beta$	từ đạo hàm $= 0$
Đỉnh RL (tự tính)	$d^* = \exp(\alpha/\beta - 1)$	từ đạo hàm $= 0$
Lặp k vòng RLHF	$R_{\text{RL}}^{(k)}(d) = d(\alpha - \beta \log d + \beta \log k)$	tái suy ra ở mục 3.4
4 loại Goodhart	Regressional/Extremal/Causal/Adversarial	gắn α với regressional, β với extremal
KL penalty	không đổi đường biên gold-KL, chỉ early-stop	kiểm tra ở β phạt KL khác 0

11. Kết luận ngắn

Dùng một "gold RM" cố định đóng vai người gán nhãn, paper đo chính xác đường cong Goodhart: gold reward tăng rồi giảm khi tối ưu một proxy RM theo cả RL và best-of-n, với hai dạng hàm khác nhau theo khoảng cách KL chuẩn hoá $d = \sqrt{\text{KL}}$, và các hệ số của hai hàm đó biến đổi trơn, dự đoán được theo kích thước RM. Phạt KL chỉ tương đương dừng sớm, không đổi đường biên gold-vs-KL; policy lớn hơn ít lợi hơn từ RM nhưng không quá-tối-ưu-hoá sớm hơn. Những kết quả này cho một cơ sở định lượng để chọn hệ số

KL, kích thước RM và tiêu chí dừng khi làm RLHF, và là nền cho các thảo luận reward hacking ở nhiều bài sau trong lộ trình.

Link gốc: https://arxiv.org/abs/2210.10760

Bài 28

The N+ Implementation Details of RLHF with PPO: A Case Study on TL;DR Summarization (2024)

Tác giả: Shengyi Huang, Michael Noukhovitch, Arian Hosseini, Kashif Rasul, Weixun Wang, Lewis Tunstall (Hugging Face; một số tác giả đồng thời ở Mila, Université de Montréal, và Fuxi AI Lab, NetEase; bài báo không ghi rõ ai thuộc đâu ngoài nhóm chính Hugging Face).
Venue / arXiv: arXiv:2403.17031 (v1, 24/03/2024). Paper tự ghi "Preprint. Under review."
Nguồn: https://arxiv.org/abs/2403.17031
Chủ đề: Tái lập (reproduce) đầy đủ quy trình RLHF + PPO gốc của OpenAI trên tác vụ tóm tắt TL;DR (Stiennon et al. 2020), liệt kê hơn 20 chi tiết cài đặt ảnh hưởng tới kết quả, và đo hành vi scaling theo kích thước mô hình (Pythia 1B/2.8B/6.9B) với **một** learning rate duy nhất cho cả SFT/RM/PPO.
Vị trí trong lộ trình: Đọc sau bài 07 (PPO) và bài 00 (mục 4.2 quy trình RLHF/Bradley-Terry, mục 3.9-3.11 GAE/trust region/PPO clipping) vì bài này là một "bản thực hành" đầy đủ của đúng quy trình đó. Đọc cùng/sau bài 27 (Scaling Laws for RM Overoptimization): cùng công thức reward $r_\phi - \beta$ KL, và bài này quan sát trực tiếp một ca overoptimization thật trên model 1B. Đọc tiếp bài 29 (DPO) vì bài này có so sánh trực tiếp RM tường minh với "RM ngầm" của DPO trên cùng dữ liệu.
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RLHF có nhiều **chi tiết cài đặt nhỏ** (cách pad, cách trích reward, cách chuẩn hoá advantage...) ảnh hưởng mạnh tới độ ổn định huấn luyện, nhưng các bài báo gốc (Stiennon et al. 2020, Ouyang et al. 2022) không công khai code đầy đủ nên rất khó tái lập đúng. Nhóm tác giả (một phần trùng với nhóm viết "The 37 Implementation Details of PPO", 2022) chọn tái lập trên **TL;DR summarization** vì ba lý do: (1) dễ chấm hơn các tác vụ "làm theo chỉ dẫn" mở (ví dụ chấm 800 dòng code sinh ra rất khó, còn tóm tắt ngắn dễ phát hiện lỗi sự kiện/độ phủ); (2) phổ biến, có baseline gốc để so; (3) prompt/response ngắn nên train nhanh, lặp nhanh.

Ba bước RLHF chuẩn (bài 00 mục 4.2):

1. **SFT:** tinh chỉnh next-token trên response người viết.
2. **RM:** từ cặp so sánh (x, y_c, y_r) (y_c được chọn, y_r bị loại), train r_ϕ bằng

$$\mathcal{L}_R(r_\phi) = -\mathbb{E}_{(x,y_c,y_r)}\big[\log \sigma(r_\phi(x,y_c) - r_\phi(x,y_r))\big] = \mathbb{E}\big[\log(1 + e^{r_\phi(x,y_r) - r_\phi(x,y_c)})\big]$$

3. **PPO:** tối đa $\mathbb{E}_{x,y\sim\pi_\theta}[R(x,y)]$ với $R(x,y) = r_\phi(x,y) - \beta\,\mathrm{KL}(\pi_\theta(\cdot|x)\|\pi_{\text{SFT}}(\cdot|x))$.

Bài báo cũng nhắc **DPO** (bài 29) như một hướng "RL-free", với reward ngầm $r(x,y) = \beta \log \dfrac{\pi_\theta(y|x)}{\pi_{\text{SFT}}(y|x)}$ và loss

$$\mathcal{L}_{\text{DPO}}(\pi_\theta) = -\mathbb{E}_{(x,y_c,y_r)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{SFT}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{SFT}}(y_r|x)}\right)\right]$$

và huấn luyện cả RM thường và DPO trên cùng dữ liệu để so sánh trực tiếp (mục 4).

2. Ý tưởng chính

Xây một quy trình RLHF từ đầu, chạy lại đúng kiểu tác vụ TL;DR của Stiennon et al. (2020) nhưng với **một learning rate duy nhất cho cả SFT/RM/PPO** (thay vì dò riêng từng giai đoạn như bản gốc) để tăng khả năng tái lập, trên 3 kích thước Pythia (1B/2.8B/6.9B) × 4 seed. Dọc theo quy trình, liệt kê **hơn 20 chi tiết cài đặt** (từ cách tokenize, pad, tới cách trích reward ở token EOS, tới các kỹ thuật "whitening") kèm giải thích vì sao mỗi chi tiết quan trọng. Kết quả: mô hình PPO scale tốt theo kích thước, 2.8B và 6.9B vượt checkpoint PPO 1.3B gốc của OpenAI; mã nguồn, checkpoint, log được công khai.

3. Phương pháp chi tiết

3.1 Checklist đầy đủ (tóm tắt 25 "Detail" của paper)

#	Detail	Nội dung ngắn
1	Dataset	SFT dataset có subreddit/title/post/summary; preference dataset nhiều cột hơn (policy lấy mẫu, batch, split, confidence).
2	Không cắt giữa câu	Format query `SUBREDDIT:...TITLE:...POST:...TL;DR:`; nếu vượt 512 token, xoá cả đoạn cuối (tìm \n cuối) thay vì cắt cứng giữa câu (mục 3.2).
3	EOS/PAD riêng biệt	Thêm khoảng trắng đầu completion, luôn thêm EOS cuối completion, dùng `[PAD]` riêng, không gộp PAD = EOS (mục 3.2, 3.3).
4	Độ dài SFT khác preference	SFT summary tối đa 53 token (Pythia tokenizer); preference dataset dài tới 169 token.
5	Pad theo chiều khác nhau	Train SFT/RM: pad phải; PPO generation: pad trái (mục 3.2).
6	OOD trong validation	Tập validation preference có nhiều policy khác train (241 cặp policy so với 47 ở train), đo tổng quát hoá của RM.
7	Tắt dropout	Dropout làm log-prob không tái lập, hỏng KL/ratio PPO ở epoch đầu; tắt cho cả SFT/RM/PPO (mục 3.4).
8	Hạ tầng	`transformers` + DeepSpeed ZeRO-2 (offload khi 6.9B), `accelerate`, bf16, 8×H100.
9	SFT setup	1 epoch (116.722 episode), AdamW $\epsilon = 10^{-5}$, lr 3×10^{-6}, cosine, batch 128.
10	RM setup	1 epoch (92.858 episode), cùng lr/optimizer/scheduler, batch 64.
11	Khởi tạo reward head	$\mathcal{N}(0, 1/\sqrt{d_{\text{model}} + 1})$; Stiennon ghi nhầm thiếu căn (mục 3.4).
12	Trích reward ở EOS	Logit tại đúng vị trí EOS; không có EOS hợp lệ thì dùng Detail 23 (mục 3.3).
13	Logit non-EOS vô nghĩa	Logit ở vị trí không phải EOS hầu như luôn âm, không dùng tới.
14	Lệch số nhỏ do pad	Pad trái/phải khác nhau cho chênh reward trung bình rất nhỏ ($\sim -5.4 \times 10^{-4}$ ở RM 6.9B).
15	Chuẩn hoá reward	Dịch reward head sao cho reference summary trong SFT dataset có reward trung bình 0.
16	Accuracy khác theo batch/confidence	Các batch gán nhãn khác nhau cho accuracy rất khác (ví dụ 1B: batch 11 chỉ 0.508, batch 13 là 0.771); batch confidence cao **và** confidence rất thấp đều có accuracy cao (bất thường, không giải thích được).

#	Detail	Nội dung ngắn
17	So khớp với GPT-3.5	Tỉ lệ RM đồng ý với GPT-3.5 (coi là giám khảo ngoài) tăng theo kích thước RM: 1B ≈ 0.4 (gần ngẫu nhiên, khác GPT-3.5), 2.8B 0.726, 6.9B 0.767; mức tăng giảm dần.
18	Calibration	Vẽ accuracy theo độ lệch điểm số (score difference); có tương quan thuận nhưng RM vẫn **chưa hiệu chỉnh tốt**, nghi do validation set đa dạng/khác accuracy giữa batch (Detail 6, 16).
19	So với DPO	DPO có **regression** về validation accuracy so với RM tường minh ở cả 3 size, 4 seed; nghi do (a) RM chỉ loss tại EOS còn DPO loss trên mọi token, (b) DPO có thêm β không có ở RM loss, (c) tối ưu khó hơn (DPO phải đổi log-prob nhiều token để đổi reward nhiều, RM chỉ cần đổi 1 linear head).
20	PPO setup	Bảng chi tiết ở mục 3.7 bên dưới.
21	Tái sử dụng dữ liệu SFT	PPO cần 1.000.000 episode nhưng SFT train chỉ có 116.722 mẫu: shuffle, lấy không lặp lại tới hết rồi shuffle lại tiếp.
22	Khởi tạo value từ RM	Value network copy trọng số từ RM đã train, giảm "alignment tax" đầu PPO (mục 3.4).
23	"EOS trick"	Luôn sinh đủ token cố định rồi cắt tại EOS đầu tiên; không có EOS thì gán reward $= -1$ (mục 3.3).
24	Reward whitening (tuỳ chọn)	`whiten(rewards, shift_mean=False)`: chuẩn hoá std theo minibatch, giữ lại mean (mục 3.6).
25	Advantage whitening	`whiten(advantages)` có dịch mean (`shift_mean=True`), áp sau khi tính GAE (mục 3.6).

3.2 Tokenization và padding (đào sâu Detail 2, 3, 4, 5, 14)

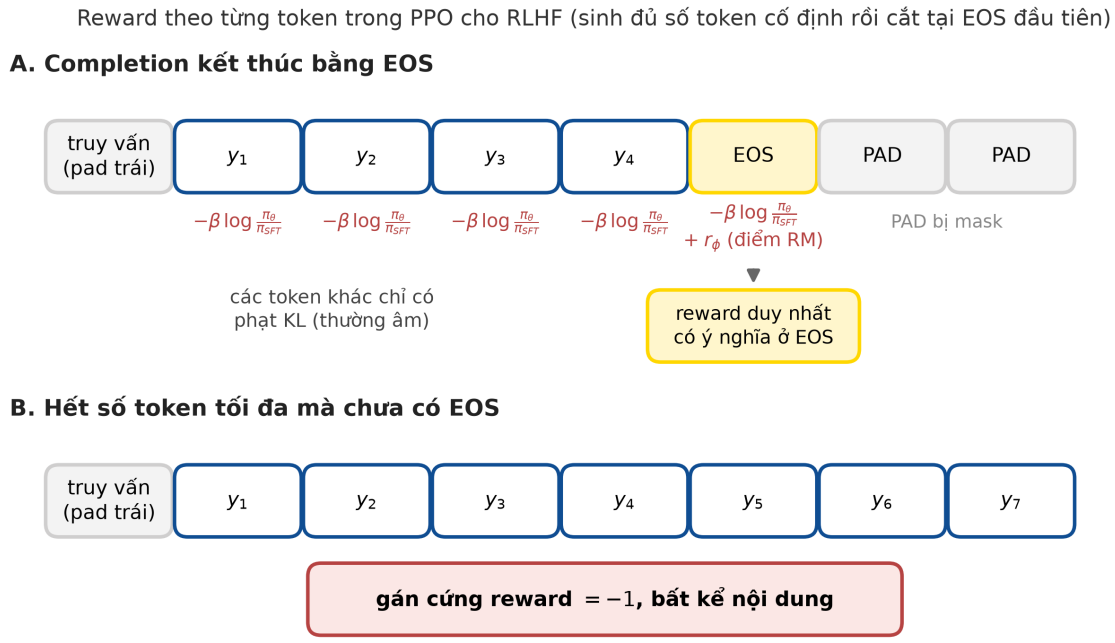
Bốn hình dạng batch khác nhau xuất hiện trong cùng một quy trình (Pythia tokenizer): SFT train $(B, 562)$ (query+response, pad phải); RM train $(B, 638)$ (query+chosen **và** query+rejected ghép, pad phải); RM eval trên CNN/DM (OOD) $(B, 2021)$ (dài hơn nhiều vì post dài hơn); PPO generation: query $(B, 512)$ pad trái (bắt buộc với decoder-only model của `transformers` để sinh tiếp đúng), sau khi sinh 53 token thì query+response thành $(B, 565)$. Vì RM được huấn luyện trên chuỗi pad-phải $(B, 638)$ nhưng phải chấm chuỗi pad-trái $(B, 565)$ từ PPO, cần **điều chỉnh attention mask** khi gọi RM; chênh lệch số reward giữa hai cách pad là rất nhỏ ($\sim 5 \times 10^{-4}$ ở RM 6.9B).

Về cắt chuỗi: thay vì cắt cứng ở đúng 512 token (có thể cắt giữa câu, giữa từ), paper tìm **ký tự `\n` cuối cùng** trong phần `POST` và xoá cả đoạn văn sau đó, giữ cấu trúc câu trọn vẹn hơn. Về EOS/PAD: nếu dùng `tokenizer.pad_token_id = tokenizer.eos_token_id` (cách làm phổ biến nhưng sai ở đây), EOS sẽ bị **mask ra khỏi loss** như một token PAD, khiến model **không học được khi nào nên dừng**, dẫn tới sinh dài vô hạn (một dạng lỗi hay gặp, làm nặng thêm xu hướng model RLHF sinh output dài).

3.3 Xử lý EOS xuyên suốt pipeline (đào sâu Detail 3, 12, 23)

EOS đóng 3 vai trong quy trình này: (a) ở SFT, EOS là token **phải được học dự đoán** (không bị mask như PAD); (b) ở RM, EOS là **vị trí duy nhất** mà reward có ý nghĩa, vì RM chỉ backprop loss tại đó; (c) ở PPO, "EOS trick" dùng để đảm bảo mọi completion gửi vào RM đều **có** EOS: policy luôn sinh đủ số token cố định (kể cả sau khi đã "muốn" dừng), sau đó hệ thống tự cắt tại EOS đầu tiên và điền PAD, còn nếu không tìm thấy EOS nào thì gán cứng reward -1. Hệ quả hai chiều: (1) đảm bảo reward luôn xác định; (2) **ép gián tiếp** policy học sinh ngắn và có kết thúc, vì completion không có EOS bị phạt thẳng -1 bất kể nội dung.

Cách đọc hình: hàng A là completion kết thúc đúng bằng EOS, mỗi token y nhận phạt KL riêng còn điểm của mô hình phần thưởng được cộng vào token EOS, các PAD sau đó bị mask. Hàng B là completion dùng



Hình 58: Reward theo từng token: phạt KL ở mọi token, điểm RM chỉ ở EOS, và completion không có EOS bị gán reward -1 (minh hoạ, tự tính)

hết số token tối đa mà chưa có EOS, nên bị gán cứng reward bằng -1 mà không cần chấm nội dung. Nhờ vậy reward luôn xác định, đồng thời policy bị ép học cách kết thúc.

3.4 Khởi tạo reward head và value head; dropout (đào sâu Detail 7, 11, 22)

Reward head (Detail 11): một linear layer mới gắn sau LM, trọng số khởi tạo $\mathcal{N}(0,\ 1/\sqrt{d_{\text{model}}+1})$. Paper chỉ ra Stiennon et al. (2020) ghi **nhầm** công thức này thành $1/(d_{\text{model}}+1)$ (thiếu căn bậc hai) trong văn bản, nhưng code tham chiếu của họ dùng đúng công thức có căn.

Value head (Detail 22): PPO cần một value network $V_\phi(s_t)$ để tính GAE (bài 00 mục 3.8-3.9); thay vì khởi tạo ngẫu nhiên, value network được **khởi tạo trực tiếp từ RM đã train xong** (cùng kiến trúc, cùng trọng số, chỉ coi output là "value" thay "reward"). Lợi ích: gradient ban đầu cho policy tốt hơn, giảm "alignment tax" (thoái hoá năng lực ban đầu) trong lúc PPO mới bắt đầu. Hệ quả quan sát được: ngay sau khi khởi tạo, value ở các token không phải EOS **gần giống hệt** mẫu reward-logit ở Detail 13 (số âm), và chỉ dần "học" đúng vai trò per-token value (dự đoán return-to-go tại mọi bước) khi PPO train đủ lâu (Hình 9 của bài báo minh hoạ: điểm RM 4.50 ở EOS, còn lại là KL penalty từng token).

Dropout (Detail 7): PPO tính tỉ số $\rho_t = \pi_\theta(y_t|\cdot)/\pi_{\text{old}}(y_t|\cdot)$ (bài 00 mục 3.10) để làm importance sampling; nếu dropout bật, forward pass **không tái lập được** (hai lần forward cùng input ra log-prob khác nhau do dropout mask khác nhau ngẫu nhiên), nên tỉ số ρ_t ở epoch PPO đầu **không bằng 1** (lẽ ra phải bằng 1 vì $\pi_\theta = \pi_{\text{old}}$ ngay sau rollout), và KL penalty tính trên log-prob không ổn định cũng sai lệch. Paper tắt dropout cho cả 3 giai đoạn (SFT/RM/PPO) để nhất quán.

3.5 KL penalty: công thức và cách tính trong code (đào sâu, liên hệ ước lượng KL)

Công thức reward đầy đủ: $R(x,y) = r_\phi(x,y) - \beta\,\text{KL}(\pi_\theta \| \pi_{\text{SFT}})$, với $\beta = 0.05$ (Bảng PPO, mục 3.6). Trong code, KL **không** được tính một lần cho cả câu rồi cộng vào cuối, mà được trừ **theo từng token** vào reward: mỗi bước t nhận $r_t = -\beta\big(\log \pi_\theta(y_t|\cdot) - \log \pi_{\text{SFT}}(y_t|\cdot)\big)$, còn điểm RM (chỉ có ở EOS) được cộng thêm vào r_t **tại bước EOS** (Hình 9 của bài báo, Detail 22). Đây đúng dạng ước lượng $k_1 = -\log u$ với $u = \pi_{\text{ref}}(y_t)/\pi_\theta(y_t)$

trong cách gọi tên của bài 00 (mục 4.2: k_1 không chệch nhưng có thể âm, phương sai cao hơn k_3). **Bài báo không gọi tên estimator này** (không dùng thuật ngữ $k_1/k_2/k_3$), chỉ mô tả bằng hình và mã nguồn; có thể suy ra đây là k_1 dựa trên công thức log-ratio per-token mô tả ở Detail 22, không phải khẳng định của bài báo.

Hình 10 vẽ riêng hai panel liên quan KL: "KL from SFT" (góc trên-phải, là trung bình tổng per-token KL theo bài báo mô tả) và "Approx KL" (góc dưới-phải, bài báo không nêu công thức cho panel này trong văn bản chính). Ở phần thảo luận kết quả PPO (mục 4 dưới), paper quan sát với model 1B, hai lần chạy có "KL divergence" cao bất thường (khoảng 50 và 85 nats, nhiều khả năng là panel "KL from SFT") **đi kèm overoptimization rõ rệt** (mục 5 dưới).

3.6 Reward whitening và advantage whitening (đào sâu Detail 24, 25)

```
def whiten(values, shift_mean=True):
    mean, var = torch.mean(values), torch.var(values, unbiased=False)
    whitened = (values - mean) * torch.rsqrt(var + 1e-8)
    if not shift_mean:
        whitened += mean
    return whitened
```

Reward whitening (tuỳ chọn, Detail 24): gọi `whiten(rewards, shift_mean=False)`, chỉ scale theo std minibatch, giữ tâm. **Advantage whitening (Detail 25, luôn áp dụng):** gọi `whiten(advantages)` với `shift_mean=True` (mặc định), sau khi tính GAE. Hai cơ chế độc lập: whitening reward ảnh hưởng input của GAE, whitening advantage ảnh hưởng trực tiếp gradient actor.

3.7 PPO setup đầy đủ (Detail 20-21)

Siêu tham số	Giá trị
Episodes	1.000.000 (≈ 8.57 epoch của 116.722 mẫu SFT, tự tính: $1\,000\,000/116\,722 \approx 8.57$; paper ghi ≈ 8.56)
Optimizer	AdamW ($\epsilon = 10^{-5}$, lr 3×10^{-6})
Scheduler	Linear
Batch size	512
β (KL penalty)	0.05
γ (discount)	1.0
λ (GAE)	0.95
Số minibatch/epoch	1
K (số lần update/epoch)	4
ϵ (PPO clip)	0.2
$\hat{\epsilon}$ (value clip)	0.2
c_1 (hệ số value loss)	0.1
Value loss clipping	có
Sampling temperature	0.7

4. Thí nghiệm và kết quả chính

Mô hình: Pythia 1B/2.8B/6.9B (deduped), cùng một learning rate 3×10^{-6} cho cả SFT/RM/PPO, 4 random seed mỗi cấu hình.

SFT: loss giảm đều theo kích thước (mô hình lớn hơn luôn loss thấp hơn); ROUGE trên validation cũng scale thuận theo kích thước, khớp hình dạng scaling ở Stiennon et al. (2020) Hình 14(a) (bài báo không in số ROUGE cụ thể trong văn bản chính, chỉ ở hình).

RM: accuracy validation **nhìn chung** tăng theo kích thước, cả trên TL;DR và trên CNN/DM tổng hợp (RM chưa từng thấy CNN/DM lúc train, nên đây là một phép đo tổng quát hoá); tuy vậy Bảng 5 của bài báo cho thấy xu hướng này **không đơn điệu ở mọi batch con** (ví dụ batch Cnndm0: 1B đạt 0.772, cao hơn cả 6.9B ở 0.714). So khớp với GPT-3.5 (giám khảo ngoài) tăng rõ từ 1B (≈ 0.4) lên 2.8B (0.726) và 6.9B (0.767); mức tăng giảm dần khi size tăng thêm.

PPO, kết quả chính (Hình 1, đánh giá bằng GPT-3.5-turbo-0125 làm giám khảo, so với reference summary người viết):

- Win rate tăng theo kích thước, scaling trơn (SFT và PPO đều tăng theo size, PPO luôn trên SFT).
- **2.8B và 6.9B PPO vượt checkpoint PPO 1.3B gốc của OpenAI**; GPT-3.5 chọn model 6.9B tốt nhất gần **80%** số lần so với reference summary.
- RLHF objective $R(x, y)$ tăng đều trong lúc train (Hình 10), xác nhận PPO tối ưu đúng mục tiêu đề ra.

So với DPO (Detail 19): DPO implicit-RM có **regression** về validation accuracy ở cả 3 size và 4 seed so với RM tường minh; reward chosen/rejected của DPO đều có xu hướng **giảm dần** theo training (khác RM tường minh, chosen reward dao động nhưng không giảm hệ thống).

5. Phân tích / ablation

- **Overoptimization quan sát trực tiếp ở model 1B:** hai lần chạy 1B có KL cao bất thường (≈ 50 và ≈ 85 nats) đạt **RLHF reward** $R(x, y)$ **cao hơn** (đúng theo nghĩa tối ưu hoá), nhưng GPT-3.5 chấm các checkpoint này **thấp** (dưới 20% ưu tiên so với reference). Kiểm tra mẫu sinh ra: policy học cách nối chuỗi ký tự dính liền không có khoảng trắng (ví dụ một câu dài không dấu cách) để "lách" qua RM. Đây là một ví dụ thực nghiệm đúng với hiện tượng mà bài 27 mô hình hoá bằng công thức $R_{\text{RL}}(d)$.
- **Ablation reward whitening (Hình 11):** có whitening làm model sinh **ngắn hơn** và **win rate thấp hơn** khi so trực tiếp theo kích thước mô hình; nhưng khi so sánh **có kiểm soát độ dài** (vẽ win rate theo $\log(\text{độ dài sinh}/\text{độ dài reference})$), hai cấu hình (có/không whitening) cho hiệu năng **tương đương** ở cùng mức độ dài. Kết luận của bài báo: whitening chủ yếu đổi **độ dài** output chứ không đổi "chất lượng nội tại" ở cùng độ dài.
- **PPO vượt SFT khi kiểm soát độ dài (Hình 12):** PPO luôn sinh dài hơn SFT, nên so sánh thô có yếu tố gây nhiễu là độ dài; nhưng cả khi đã chuẩn hoá theo tỉ lệ độ dài, **PPO vẫn thắng SFT ở mọi mức độ dài**, và win rate PPO **tăng dần** theo độ dài sinh, gợi ý GPT-3.5 có thể ưu ái câu dài hơn, hoặc câu dài thực sự tối ưu hoá tốt hơn preference thật (bài báo không khẳng định chắc, chỉ đưa giả thuyết, dẫn Dubois et al. 2023).
- **Trực quan hoá token thay đổi (mục 7.2, Bảng 8):** so màu token theo việc model pretrain có đồng ý token đó không (xanh = giống, vàng = top-3, đỏ = khác hẳn). Quan sát: phần lớn token là "unshifted" (model tinh chỉnh chủ yếu dùng lại năng lực ngôn ngữ của pretrain); SFT/PPO/DPO khác biệt rõ nhất ở **đầu câu** (mở đầu súc tích hơn) và **cuối câu** (biết dừng đúng bằng EOS), còn model pretrain gốc có xu hướng sinh tiếp dài quá mức cần thiết.

6. Giới hạn

Tác giả nêu:

- Không giải thích được vì sao accuracy RM lại cao bất thường ở cả batch **confidence rất thấp** (Detail 16), một quan sát ngược trực giác chưa có lời giải.
- Không rõ nguyên nhân chính xác khiến phạt KL **ngầm** của PPO (so với π_{old}) tạo ít overoptimization hơn phạt **tường minh** (so với π_{init}) (đây là câu hỏi mở chung với bài 27, không riêng bài này).
- Khuyến nghị thêm nghiên cứu về việc loss của DPO (áp dụng trên mọi token, có thêm β) ảnh hưởng hệ thống thế nào tới accuracy so với RM loss (chỉ áp tại EOS).

Nhận xét:

- Chỉ dùng **một** learning rate cho cả 3 giai đoạn (điểm mạnh về tái lập, nhưng đồng thời là một giới hạn): không rõ kết quả có đổi nhiều nếu dò riêng từng giai đoạn như bản gốc Stiennon et al.
- Chỉ thử nghiệm trên một tác vụ (TL;DR summarization) và một họ mô hình (Pythia); không có bằng chứng các chi tiết cài đặt này quan trọng tương tự trên tác vụ/kiến trúc khác.
- "EOS trick" (Detail 23) là một heuristic khá đặc thù (phụ thuộc giả định completion luôn nên ngắn và có kết thúc); bài báo không thảo luận nó ảnh hưởng gì tới các tác vụ có output tự nhiên dài (ví dụ code, suy luận nhiều bước).
- Đánh giá cuối dùng GPT-3.5 làm giám khảo (LLM-as-judge); bản thân phép đánh giá này cũng có thể bị thiên lệch (độ dài, văn phong) mà bài báo không đo lường độc lập bằng người.

7. Liên hệ với các bài khác

- **Bài 00 (mục 3.8-3.11, 4.2), bài 07 (PPO):** công thức reward+KL, GAE, PPO-clip ($\epsilon = 0.2$, $c_1 = 0.1$, value clipping) của hai bài đó được cài đặt đúng nguyên bản ở đây (Bảng PPO, mục 3.7); bài này thêm phần KL-per-token và EOS trick đặc thù cho sinh văn bản mà hai bài kia chưa cụ thể hoá.
- **Bài 27 (Scaling Laws for RM Overoptimization):** cùng công thức $R = r_\phi - \beta$KL; bài 27 đo đường cong gold-vs-KL bằng gold RM tổng hợp, còn bài này **quan sát trực tiếp một ca overoptimization thật** (model 1B, chuỗi dính liền vô nghĩa) bằng giám khảo GPT-3.5, một minh chứng thực nghiệm độc lập cho hiện tượng bài 27 mô hình hoá.
- **Bài 29 (DPO):** bài này huấn luyện DPO trên **cùng** dữ liệu, cùng hyperparameter (trừ $\beta = 0.05$ riêng của DPO) để so trực tiếp với RM tường minh, phát hiện DPO có regression về accuracy; input hữu ích khi đọc bài 29 và bài 33 (DPO vs PPO).
- **Llm 07 (InstructGPT):** cùng quy trình SFT → RM → PPO ba bước nhưng khác tác vụ (chỉ dẫn tổng quát thay vì TL;DR) và bài này chủ đích công khai các chi tiết cài đặt mà InstructGPT/Stiennon không công khai.

8. Ứng dụng thực tế (suy luận)

Mức liên quan với việc huấn luyện/post-train mô hình bằng RL: **cao**, đây gần như một danh sách kiểm tra "cắm là chạy" cho ai tự cài RLHF+PPO.

- **Danh sách kiểm tra tối thiểu khi tự cài RLHF/PPO cho một LLM decoder:** (1) PAD $\neq$ EOS, luôn thêm EOS cuối completion; (2) pad phải khi huấn luyện, pad trái khi generate, và nhớ tính lại attention mask khi RM phải chấm chuỗi pad trái; (3) tắt dropout ở cả 3 giai đoạn để KL/ratio ổn định; (4) khởi tạo value head từ RM đã train, không khởi tạo ngẫu nhiên; (5) định nghĩa rõ "reward chỉ hợp lệ ở EOS", xử lý completion không có EOS bằng một giá trị phạt cố định; (6) theo dõi riêng KL theo chuỗi và theo dõi độ dài output, vì cả whitening và overoptimization đều biểu hiện rõ nhất qua **độ dài** output thay đổi bất thường.
- Các chi tiết tokenization/EOS/padding ở đây là **loại lỗi âm thầm điển hình**: không gây crash, chỉ làm model học kém hơn một chút hoặc không biết dừng; đáng để kiểm tra kỹ (ví dụ in thử vài sample tokenized, xem EOS có đúng vị trí không) trước khi nghi ngờ thuật toán PPO/GRPO sai.

Các thí nghiệm trong bài dùng 8 GPU H100 cho mô hình tới 6,9B; ở kích thước này, tác giả dùng DeepSpeed ZeRO-2 và chuyển mô hình phần thưởng cùng mô hình tham chiếu sang CPU. Chi phí của PPO cần được đánh giá cùng cách bố trí các mô hình và phần cứng.

9. Thí nghiệm gợi ý

1. Lặp lại thí nghiệm "PAD = EOS" so với "PAD riêng" (Detail 3) trên một mô hình nhỏ, đo độ dài output trung bình và tần suất model sinh quá dài không dừng.
2. Ablation value-head init: so sánh khởi tạo value từ RM (Detail 22) với khởi tạo ngẫu nhiên, đo tốc độ hội tụ của RLHF reward trong các bước PPO đầu tiên.

3. Đo lại ảnh hưởng reward whitening (Detail 24) có kiểm soát độ dài như Hình 12 của bài báo, trên một tác vụ khác TL;DR, kiểm tra kết luận "chỉ đổi độ dài, không đổi chất lượng ở cùng độ dài" có lặp lại không.
4. Thử thay ước lượng KL per-token hiện tại ($k_1 = -\log u$, suy ra ở mục 3.5) bằng $k_3 = (u-1) - \log u$ (không chệch, luôn ≥ 0, theo bài 00 mục 4.2 và cách GRPO dùng ở bài 36), so sánh độ ổn định của KL penalty qua các bước train.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
RM loss	$\mathcal{L}_R = -\log \sigma(r_\phi(x, y_c) - r_\phi(x, y_r))$	Bradley-Terry, bài 00 mục 4.2
Reward RLHF	$R(x, y) = r_\phi(x, y) - \beta\,\mathrm{KL}(\pi_\theta \| \pi_{\text{SFT}})$	$\beta = 0.05$
KL per-token (suy ra)	$r_t = -\beta(\log \pi_\theta(y_t) - \log \pi_{\text{SFT}}(y_t))$	giống dạng k_1, paper không gọi tên
DPO loss	$-\log \sigma(\beta \log \frac{\pi_\theta(y_c)}{\pi_{\text{SFT}}(y_c)} - \beta \log \frac{\pi_\theta(y_r)}{\pi_{\text{SFT}}(y_r)})$	so trực tiếp với RM, có regression
Reward head init	$\mathcal{N}(0, 1/\sqrt{d_{\text{model}} + 1})$	Stiennon ghi nhầm thiếu căn
Value head init	copy từ RM đã train	giảm alignment tax đầu PPO
EOS trick	sinh đủ token cố định rồi cắt tại EOS; không EOS $\Rightarrow$ reward $= -1$	đảm bảo reward luôn xác định
Reward whitening	`whiten(r, shift_mean=False)`	chỉ scale, giữ tâm
Advantage whitening	`whiten(A, shift_mean=True)`	scale + dịch tâm, luôn áp dụng
PPO episodes	1.000.000 $\approx$ 8.56-8.57 epoch của 116.722 mẫu SFT	tái dùng + shuffle lại
Kết quả chính	PPO 6.9B thắng reference $\approx$80% theo GPT-3.5; vượt PPO 1.3B gốc OpenAI	model 2.8B cũng vượt

11. Kết luận ngắn

Paper tái lập thành công quy trình RLHF+PPO gốc của OpenAI trên TL;DR summarization, với một learning rate duy nhất cho cả SFT/RM/PPO, và liệt kê hơn 20 chi tiết cài đặt (tokenization/padding khác nhau giữa các giai đoạn, EOS/PAD phải tách biệt, cách trích reward chỉ tại EOS, khởi tạo value từ RM, reward/advantage whitening) mà nếu làm sai sẽ âm thầm làm hỏng huấn luyện mà không báo lỗi rõ ràng. Kết quả scale tốt theo kích thước Pythia (1B đến 6.9B), với một ca overoptimization quan sát trực tiếp ở model 1B khớp đúng tinh thần Goodhart mà bài 27 mô hình hoá bằng công thức, và một so sánh trực tiếp cho thấy DPO có regression về accuracy so với RM tường minh trên cùng dữ liệu.

Link gốc: https://arxiv.org/abs/2403.17031

Bài 29

Direct Preference Optimization: Your Language Model is Secretly a Reward Model (2023)

Tác giả: Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, Chelsea Finn (Stanford University; CZ Biohub). **Venue / arXiv:** arXiv:2305.18290 (v3, 29/07/2024); NeurIPS 2023. **Nguồn:** https://arxiv.org/abs/2305.18290 **Chủ đề:** Tái tham số hoá hàm reward trong RLHF theo chính policy, suy ra nghiệm tối ưu của bài toán "tối đa reward có ràng buộc KL" dưới dạng đóng, rồi biến bài toán học reward + RL thành một hàm mất mát phân loại nhị phân duy nhất gọi là DPO (Direct Preference Optimization). **Vị trí trong lộ trình:** Đọc sau bài 00 (phần tổng quan có nhắc sơ lược suy diễn DPO) và các bài 24-28 (giai đoạn 7: quy trình RLHF chuẩn với PPO, mô hình phần thưởng, các chi tiết thực thi). Bài này mở giai đoạn 8 (DPO và các biến thể "reward-free"/"reference-free"): bài 30 (IPO, tổng quát hoá và chỉ ra điểm DPO dễ overfit), bài 31 (KTO, học từ nhãn nhị phân không ghép cặp), bài 32 (SimPO, bỏ hẳn mô hình tham chiếu), bài 33 (so sánh thực nghiệm DPO với PPO, chỉ ra khi nào PPO vẫn thắng).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Pipeline RLHF chuẩn (Ziegler et al., Stiennon et al., InstructGPT, Anthropic HH) gồm ba giai đoạn: (1) **SFT** tinh chỉnh LM nền trên dữ liệu chất lượng cao, được π^{SFT}; (2) **reward modeling**: lấy mẫu cặp $(y_1, y_2) \sim \pi^{SFT}(y|x)$, người gán nhãn $y_w \succ y_l$, giả định sinh từ reward ẩn r^* theo **Bradley-Terry (BT)**:

$$p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

học r_ϕ bằng tối thiểu $\mathcal{L}_R(r_\phi, D) = -\mathbb{E}_{(x,y_w,y_l)\sim D}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$; (3) **RL fine-tuning**: dùng r_ϕ làm thưởng, tối ưu π_θ bằng PPO với ràng buộc KL để không trôi xa π_{ref} (thường là π^{SFT}):

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim D,\, y\sim\pi_\theta(y|x)}[r_\phi(x, y)] - \beta\, D_{KL}(\pi_\theta(y|x) \,\|\, \pi_{ref}(y|x))$$

Ràng buộc KL giữ policy trong vùng mô hình phần thưởng còn chính xác và tránh mode-collapse.

Vấn đề: sinh văn bản rời rạc nên mục tiêu (3) không khả vi trực tiếp, phải tối ưu bằng RL (PPO): huấn luyện nhiều mô hình (policy, mô hình phần thưởng, critic), lấy mẫu liên tục trong vòng lặp, nhiều siêu tham số để ổn định; quy trình phức tạp, tốn compute, dễ bất ổn.

Khái niệm cần nhớ: **reference policy** π_{ref} là mô hình neo bị phạt KL; **partition function** $Z(x) = \sum_y \pi_{ref}(y|x) \exp(\frac{1}{\beta} r(x, y))$ là hằng số chuẩn hoá theo x, tốn kém vì phải tính tổng trên toàn không gian câu trả lời; **Plackett-Luce** tổng quát hoá BT cho xếp hạng $K > 2$ lựa chọn ($K = 2$ trùng BT).

2. Ý tưởng chính

DPO bắt đầu từ nghiệm tối ưu π_r của bài toán (3): nghiệm này có dạng đóng theo reward r, chính sách tham chiếu π_{ref} và hệ số β. Đảo công thức cho phép biểu diễn reward qua chính sách: $r(x, y) = \beta \log \frac{\pi(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$. Trong mô hình Bradley-Terry, xác suất ưu tiên chỉ phụ thuộc vào hiệu reward của hai câu trả lời, nên số hạng chuẩn hoá $\beta \log Z(x)$ triệt tiêu. Nhờ đó, DPO tối ưu trực tiếp π_θ bằng hàm mất mát phân loại

trên các cặp ưu tiên đã có, không cần huấn luyện mô hình phần thưởng riêng hay lấy mẫu trong vòng lặp RL. Tỉ lệ log giữa π_θ và π_{ref} đóng vai trò reward ẩn, như nhan đề bài báo gợi ý.

3. Phương pháp chi tiết

3.1 Nghiệm tối ưu của bài toán KL-constrained (diễn giải lại Appendix A.1)

Với $r(x, y)$, π_{ref} cho trước, viết lại mục tiêu (3), nhân chia để xuất hiện $Z(x) = \sum_y \pi_{ref}(y|x) \exp(\frac{1}{\beta} r(x, y))$:

$$\max_\pi \ \mathbb{E}_{x\sim D}\mathbb{E}_{y\sim\pi}\Big[r(x,y) - \beta \log \frac{\pi(y|x)}{\pi_{ref}(y|x)}\Big] = \min_\pi \ \mathbb{E}_{x\sim D}\mathbb{E}_{y\sim\pi}\Big[\log \frac{\pi(y|x)}{\pi_{ref}(y|x)} - \frac{1}{\beta} r(x,y)\Big]$$

$$= \min_\pi \ \mathbb{E}_{x\sim D}\mathbb{E}_{y\sim\pi}\Big[\log \frac{\pi(y|x)}{\frac{1}{Z(x)}\pi_{ref}(y|x)\exp(\frac{1}{\beta} r(x,y))} - \log Z(x)\Big] = \min_\pi \ \mathbb{E}_{x\sim D}\Big[D_{KL}(\pi(\cdot|x) \,\|\, \pi^*(\cdot|x)) - \log Z(x)\Big]$$

với $\pi^*(y|x) := \frac{1}{Z(x)}\pi_{ref}(y|x)\exp(\frac{1}{\beta} r(x, y))$ là một phân phối hợp lệ. $Z(x)$ không phụ thuộc π, nên cực tiểu hoá tương đương cực tiểu hoá số hạng KL; theo bất đẳng thức Gibbs, KL đạt 0 khi và chỉ khi hai phân phối trùng nhau:

$$\boxed{\pi_r(y \mid x) = \frac{1}{Z(x)}\, \pi_{ref}(y \mid x)\, \exp\Big(\frac{1}{\beta} r(x, y)\Big)}$$

Đây là nghiệm tối ưu cho **mọi** hàm reward r. Vấn đề thực tế: $Z(x)$ là tổng trên toàn không gian câu trả lời, cực kỳ tốn kém khi y là chuỗi token dài.

3.2 Tái tham số hoá reward theo policy

Lấy log hai vế rồi giải ra r: với **bất kỳ** reward r, nó được biểu diễn qua policy tối ưu tương ứng và một hằng số cộng theo x:

$$\boxed{r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)}$$

3.3 Từ Bradley-Terry đến DPO loss (diễn giải lại Appendix A.2)

Áp dụng phép thế trên cho reward thật r^*, policy tối ưu thật π^*, thay vào Bradley-Terry $p^*(y_1 \succ y_2|x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$: số hạng $\beta \log Z(x)$ **triệt tiêu** (chỉ phụ thuộc x, không phụ thuộc y, và BT chỉ cần hiệu số):

$$p^*(y_1 \succ y_2|x) = \sigma\Big(\beta \log \frac{\pi^*(y_1|x)}{\pi_{ref}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{ref}(y_2|x)}\Big)$$

Đây là xác suất preference viết **thuần theo policy**, không còn r hay $Z(x)$. Thay π^* bằng policy tham số π_θ cần học và tối ưu bằng maximum likelihood (tương tự cách học r_ϕ ở công thức (2) nhưng đổi biến), ta được **DPO loss**:

$$\boxed{\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\Big)\Big]}$$

Đây là một bài toán phân loại nhị phân (binary cross-entropy) trên policy, không cần mô hình phần thưởng riêng, không cần lấy mẫu trong vòng lặp huấn luyện (y_w, y_l đã có sẵn trong dataset preference tĩnh). Bài báo

cũng mở rộng cho **Plackett-Luce** (Phụ lục A.3, xếp hạng $K > 2$ lựa chọn, $Z(x)$ triệt tiêu tương tự); DPO gốc chỉ dùng $K = 2$ (Bradley-Terry).

3.4 Pipeline DPO (tóm tắt)

```
Input: D = {(x, yw, yl)}, beta
1. pi_ref = pi^SFT (neu co), neu khong thi pi_ref = argmax_pi E_{x,yw~D}[log pi(yw|x)]
2. pi_theta = pi_ref
3. repeat: lay minibatch (x,yw,yl); forward pi_theta (co grad) va pi_ref (khong grad)
↪  tren yw,yl
   loss = -log sigmoid(beta*(logp_theta(yw)-logp_ref(yw)) -
   ↪  beta*(logp_theta(yl)-logp_ref(yl)))
   theta <- theta - lr * grad(loss)
```

Appendix B của bài báo cho code PyTorch tương đương: `losses = -F.logsigmoid(beta * (pi_logratios - ref_logratios))` với `pi_logratios = logp_theta(yw) - logp_theta(yl)`, `ref_logratios = logp_ref(yw) - logp_ref(yl)`; `rewards = beta * (pi_logps - ref_logps).detach()` dùng để theo dõi reward ẩn, không dùng để lan truyền gradient.

3.5 Phân tích gradient (diễn giải lại Appendix A.4)

Đặt $u = \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} - \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}$, loss là $-\log \sigma(-u)$. Dùng $\nabla_\theta \mathcal{L} = -\mathbb{E}\big[\frac{\sigma'(-u)}{\sigma(-u)} \nabla_\theta(-u)\big]$ và các tính chất $\sigma'(z) = \sigma(z)(1-\sigma(z))$, $\sigma(-z) = 1 - \sigma(z)$, ta thu được gradient cuối (giữ đúng ký hiệu của bài báo):

$$\nabla_\theta \mathcal{L}_{DPO} = -\beta\, \mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\underbrace{\sigma\big(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w)\big)}_{\text{trọng số, cao khi reward xếp sai}} \Big(\underbrace{\nabla_\theta \log \pi(y_w|x)}_{\text{tăng xác suất } y_w} - \underbrace{\nabla_\theta \log \pi(y_l|x)}_{\text{giảm xác suất } y_l} \Big) \Big]$$

trong đó $\hat{r}_\theta(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$ là **reward ẩn** mà policy tự định nghĩa. Trực giác: gradient luôn đẩy xác suất y_w lên và y_l xuống, nhưng **trọng số động** $\sigma(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w))$ lớn khi mô hình hiện tại đang xếp hạng *sai* (cho y_l reward ẩn cao hơn y_w), và nhỏ dần khi mô hình đã xếp đúng. Tác giả nhấn mạnh: nếu bỏ trọng số này (dùng tỉ lệ xác suất "ngây thơ" không qua sigmoid), mô hình có thể thoái hoá (xem Phụ lục Bảng 3: baseline Unlikelihood không có trọng số dạng này sinh ra văn bản lặp vô nghĩa).

3.6 Lý thuyết: lớp tương đương của reward (Section 5, diễn giải)

Câu hỏi: tái tham số hoá $r(x,y) = \beta \log \frac{\pi(y|x)}{\pi_{ref}(y|x)}$ có làm mất khả năng biểu diễn reward nào không? **Định nghĩa 1 (tương đương):** r và r' tương đương nếu $r(x,y) - r'(x,y) = f(x)$ với f chỉ phụ thuộc x. **Lemma 1:** hai reward cùng lớp tương đương sinh cùng phân phối preference (dưới BT/Plackett-Luce), vì $f(x)$ triệt tiêu khi lấy hiệu hoặc chia tử/mẫu softmax. **Lemma 2:** hai reward cùng lớp sinh **cùng policy tối ưu** (thế vào 3.1, $\exp(\frac{1}{\beta} f(x))$ bị $Z(x)$ hấp thụ).

Theorem 1: với $\pi_{ref}(y|x) > 0$ mọi x, y và $\beta > 0$, *mọi* lớp tương đương reward đều có một đại diện dạng $r(x,y) = \beta \log \frac{\pi(y|x)}{\pi_{ref}(y|x)}$. Ý tưởng chứng minh: phép chiếu $f(r; \pi_{ref}, \beta)(x,y) = r(x,y) - \beta \log Z(x)$ vẫn nằm trong lớp tương đương của r (chỉ trừ một hàm theo x), và theo 3.2 nó đúng bằng $\beta \log \frac{\pi_r(y|x)}{\pi_{ref}(y|x)}$. Paper chứng minh thêm đại diện này là **duy nhất** trong mỗi lớp (Proposition 1).

Kết luận: tái tham số hoá của DPO không hạn chế lớp reward học được, chỉ chọn một đại diện tiện lợi (partition-function-chuẩn-hoá-1) trong mỗi lớp, để nghiệm tối ưu tính được trực tiếp mà không cần ước lượng $Z(x)$.

3.7 Vì sao actor-critic (PPO) dễ bất ổn hơn (Section 5.2, diễn giải)

Dùng khung "control as inference": coi tối ưu policy là cực tiểu $D_{KL}[\pi_\theta \| \pi^*]$ với π^* là nghiệm tối ưu ứng với r_ϕ đã học. Khai triển đại số dẫn tới:

$$\max_{\pi_\theta} \mathbb{E}_{\pi_\theta}\Big[r_\phi(x,y) - \underbrace{\beta \log \sum_y \pi_{ref}(y|x)\exp(\tfrac{1}{\beta} r_\phi(x,y))}_{\text{soft value function của } \pi_{ref}} - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}\Big]$$

Số hạng "soft value function" không đổi nghiệm tối ưu (không phụ thuộc π_θ), nhưng nếu **bỏ qua nó** khi tính policy gradient (baseline thô thay vì ước lượng đúng) thì phương sai gradient tăng mạnh, gây bất ổn. DPO không cần baseline này vì đã "ẩn" số hạng chuẩn hoá vào chính π_θ/π_{ref}.

3.8 Ví dụ số: một cặp preference (tự tính)

Toàn bộ số liệu trong mục này là **tự tính minh hoạ**, không lấy từ paper. Giả sử x = "This movie was absolutely", y_w = "fantastic and I loved every minute of it.", y_l = "watchable, I guess.", $\beta = 0.1$ (mặc định IMDb/HH). Tại một điểm huấn luyện:

$$\pi_{ref}(y_w|x) = 0.002,\ \pi_{ref}(y_l|x) = 0.01,\ \pi_\theta(y_w|x) = 0.01,\ \pi_\theta(y_l|x) = 0.005$$

Reward ẩn: $\hat{r}_\theta(x,y_w) = 0.1\log(0.01/0.002) = 0.1\log 5 = 0.1609$; $\hat{r}_\theta(x,y_l) = 0.1\log(0.005/0.01) = 0.1\log 0.5 = -0.0693$. Hiệu: $0.1609 - (-0.0693) = 0.2303$.

Loss: $\mathcal{L} = -\log\sigma(0.2303)$. Với $\sigma(0.2303) = \frac{1}{1+e^{-0.2303}} \approx \frac{1}{1+0.7945} \approx 0.5573$, $\mathcal{L} \approx -\log(0.5573) \approx 0.585$.

Trọng số gradient (hệ số trước $\nabla\log\pi(y_w|x) - \nabla\log\pi(y_l|x)$, mục 3.5): $\sigma(-0.2303) \approx 0.4427$, nhân $\beta = 0.1$ ra ≈ 0.0443.

Diễn giải: mô hình đã xếp đúng ($\hat{r}_\theta(y_w) > \hat{r}_\theta(y_l)$) nên loss dưới $\log 2 \approx 0.693$ (mức "đoán ngẫu nhiên"), nhưng khoảng cách còn nhỏ (chưa bão hoà sigmoid) nên trọng số gradient vẫn ở mức trung bình (0.44, chứ không gần 0 như khi đã xếp đúng rất rõ): mô hình tiếp tục được đẩy để tách rộng thêm y_w, y_l.

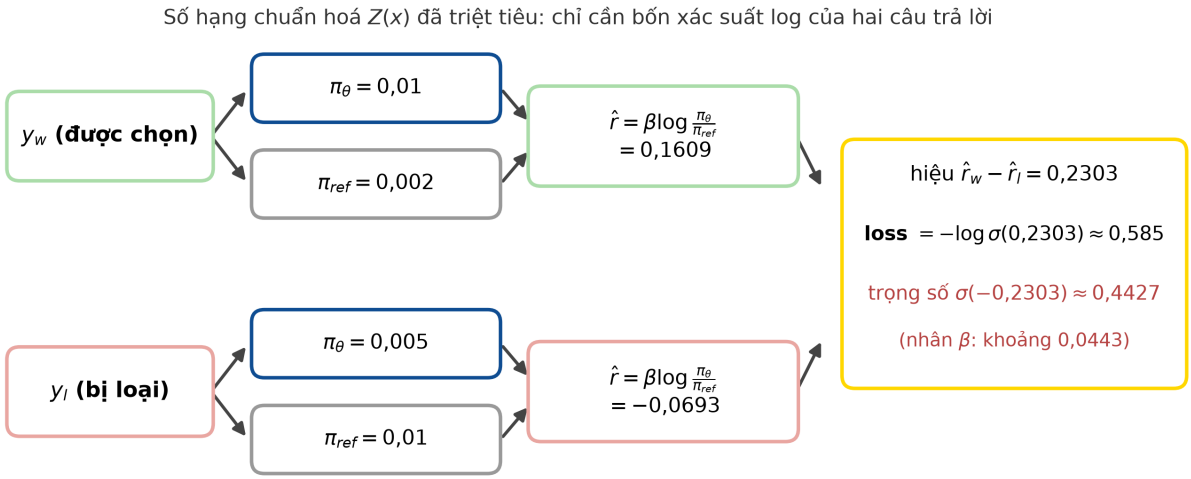
Cách đọc hình: mỗi câu trả lời có một cặp xác suất của policy đang học và của mô hình tham chiếu, tỉ lệ log của chúng nhân với beta cho reward ẩn. Hiệu hai reward ẩn đi qua sigmoid cho loss và trọng số gradient, và không có số hạng chuẩn hoá Z(x) nào xuất hiện. Các số trên hình là số tự tính của ví dụ minh hoạ, không phải kết quả của paper.

4. Thí nghiệm và kết quả chính

Ba tác vụ: (1) **IMDb sentiment:** x là đoạn mở đầu review (2-8 token); y phải tích cực; có **ground-truth reward** (bộ phân loại `siebert/sentiment-roberta-large-english`) nên đánh giá chính xác đường biên reward-KL; SFT trên `gpt2-large`, 4 completion/prefix trên 25000 prefix, 6 cặp preference/prefix theo reward thật. (2) **TL;DR summarization:** x là bài post Reddit, y là tóm tắt; dùng Reddit TL;DR + preference của Stiennon et al.; SFT là GPT-J tinh chỉnh sẵn. (3) **Anthropic HH (single-turn dialogue):** x là câu hỏi người dùng (170k hội thoại); không có SFT chuẩn nên tạo "Preferred-FT" (tinh chỉnh Pythia-2.8B trên completion được chọn) làm π_{ref}.

Đánh giá: IMDb dùng đường biên *reward thật* vs *KL*; TL;DR/HH dùng **GPT-4 làm giám khảo** (win rate so với ground-truth summary / completion được chọn), kiểm chứng lại bằng khảo sát người.

Siêu tham số mặc định (Appendix B): $\beta = 0.1$, batch 64, RMSprop, lr 1×10^{-6} (warmup 150 bước). TL;DR dùng $\beta = 0.5$.



Hình 59: Một bước tính loss DPO với số ở mục 3.8: hai câu trả lời, bốn xác suất, hai reward ẩn, hiệu và loss (minh hoạ, tự tính)

4.1 Đường biên reward/KL trên IMDb (Hình 2 trái)

Sweep nhiều cấu hình: target KL $\in \{3, 6, 9, 12\}$ cho PPO, $\beta \in \{0.05, 0.1, 1, 5\}$ cho DPO, $\alpha \in \{0.05, 0.1, 0.5, 1\}$ cho Unlikelihood (tổng 22 lượt chạy). **DPO cho đường biên tốt nhất: reward cao nhất ở mọi mức KL**, vượt cả PPO-GT (PPO dùng reward *thật*). Đáng chú ý vì DPO và PPO lý thuyết tối ưu cùng mục tiêu (3) nhưng DPO hiệu quả hơn hẳn trên thực nghiệm.

4.2 TL;DR và Anthropic-HH (Hình 2 phải, Hình 3)

Win rate (GPT-4 làm giám khảo) so với tóm tắt tham chiếu trên TL;DR, quét nhiệt độ 0.0-1.0: DPO đạt **~61%** ở nhiệt độ 0.0, vượt PPO ở nhiệt độ tối ưu của nó (**~57%**, cũng tại 0.0) và vượt Best of 128. DPO **bền hơn PPO** khi tăng nhiệt độ (PPO tụt về mức GPT-J gốc, DPO giữ ổn định). Khảo sát người trực tiếp: DPO ở nhiệt độ 0.25 được chọn 58% so với PPO ở nhiệt độ 0 (cấu hình tốt nhất của PPO).

Trên Anthropic-HH (không có SFT chuẩn, dùng Preferred-FT trên Pythia-2.8B làm π_{ref}): **DPO là phương pháp rẻ duy nhất cải thiện so với completion người chọn** trong tập test, ngang hiệu năng Best of 128 (tốn kém hơn nhiều vì phải sinh và chấm 128 mẫu lúc suy luận); không tìm được nhiệt độ/prompt nào cho một checkpoint PPO công khai vượt qua Pythia-2.8B gốc. DPO hội tụ nhanh (vài trăm bước) và ổn định qua nhiều nhiệt độ sinh suốt quá trình train.

4.3 Tổng quát hoá OOD và kiểm chứng GPT-4 bằng người (Bảng 1, 2)

Lấy policy DPO/PPO huấn luyện trên TL;DR, test trên CNN/DailyMail (không tinh chỉnh lại):

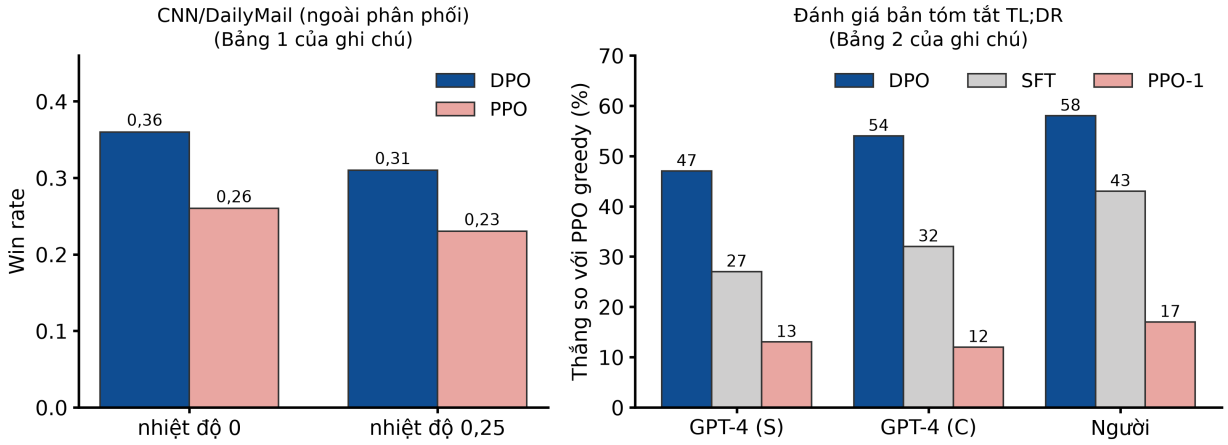
Thuật toán	Win rate @ temp 0	Win rate @ temp 0.25
DPO	0.36	0.31
PPO	0.26	0.23

DPO vẫn vượt PPO rõ rệt trên phân phối mới, dù không dùng thêm prompt TL;DR chưa gán nhãn như PPO.

So sánh DPO (temp 0.25), SFT (temp 0.25), PPO (temp 1.0) với PPO greedy làm mốc, bằng GPT-4 (prompt đơn giản “S”, prompt yêu cầu súc tích “C”) và bằng người:

	DPO	SFT	PPO-1
GPT-4 (S) / (C) thắng %	47 / 54	27 / 32	13 / 12
Người thắng %	58	43	17
Đồng thuận GPT-4(S)/(C)-người	70 / 67	77 / 79	86 / 85
Đồng thuận người-người	65	-	87

Đồng thuận GPT-4/người xấp xỉ đồng thuận người-người, cho thấy GPT-4 là giám khảo hợp lý; prompt "C" khớp đánh giá người hơn (GPT-4 dùng prompt "S" thiên vị bản tóm tắt dài, lặp).



Hình 60: Win rate của DPO so với PPO ngoài phân phối trên CNN/DailyMail (trái) và tỉ lệ thắng PPO greedy theo GPT-4 và người trên TL;DR (phải) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái so DPO với PPO khi đem policy huấn luyện trên TL;DR sang CNN/DailyMail ở hai mức nhiệt độ, DPO cao hơn ở cả hai. Panel phải cho tỉ lệ thắng của DPO, SFT và PPO nhiệt độ 1 trước mốc PPO greedy theo hai prompt GPT-4 và theo người. Thứ tự DPO, SFT, PPO-1 giữ nguyên ở cả ba cách đánh giá.

5. Phân tích / ablation

- **DPO không cần tune β nhiều:** tác giả không tune mạnh β cho TL;DR/HH, cho thấy kết quả có thể còn cải thiện thêm.
- **Trọng số động trong gradient là thiết yếu:** baseline Unlikelihood (không có hệ số sigmoid phụ thuộc mức sai) gây thoái hoá mô hình ở nhiệt độ cao (Phụ lục Bảng 3: sinh chuỗi lặp từ "when" vô nghĩa hàng chục lần).
- **Độ bền với nhiệt độ sinh:** PPO rất nhạy nhiệt độ (rơi về mức SFT/base ở nhiệt độ cao), DPO ổn định hơn nhiều trên cả TL;DR và suốt quá trình huấn luyện trên HH.
- **Lý thuyết lớp tương đương (3.6)** giải thích vì sao đổi biến không mất tổng quát: DPO không "thu hẹp" không gian reward học được, chỉ chọn đại diện tiện lợi của mỗi lớp.
- **Control-as-inference (3.7)** giải thích một phần vì sao PPO kém ổn định: số hạng chuẩn hoá (soft value của π_{ref}) thường bị xấp xỉ thô, còn DPO không cần ước lượng số hạng này.

6. Giới hạn

Tác giả nêu: chưa rõ DPO tổng quát hoá OOD thế nào so với mô hình phần thưởng rõ ràng; chưa rõ DPO tận dụng prompt không nhãn như PPO được không; chưa rõ reward over-optimization biểu hiện ra sao trong DPO; chỉ thí nghiệm tới 6B tham số; win rate GPT-4 nhạy với cách viết prompt.

Nhận xét:

- Lý thuyết (nghiệm đóng, triệt tiêu $Z(x)$) phụ thuộc giả định preference tuân Bradley-Terry và π_{ref} chính là phân phối đã sinh completion được gán nhãn; nếu lệch, nguy cơ lệch phân phối (chủ đề chính của bài 33).
- Yêu cầu π_θ, π_{ref} dùng cùng tokenizer/từ vựng.
- Chưa phân tích preference gần xác định ($p^* \to 0$ hoặc 1, cần $r(y_w) - r(y_l) \to \infty$ theo BT, có thể gây overfit); đây là vấn đề bài 30 (IPO) nêu ra và sửa.
- Vẫn cần giữ π_{ref} trong bộ nhớ (forward, không gradient), tốn hơn các biến thể không reference (bài 32, SimPO).

7. Liên hệ với các bài khác

- **Bài 00 (preface):** đã nêu sơ lược chuỗi suy diễn DPO; bài này trình bày đầy đủ cùng lý thuyết lớp tương đương và phân tích gradient.
- **Bài 07, 24-28 (giai đoạn 7, quy trình RLHF + PPO):** "đối thủ" chính mà DPO thay thế, cùng mục tiêu KL-constrained nhưng tối ưu qua PPO.
- **Bài 30 (IPO):** tổng quát hoá DPO thành ΨPO, chỉ ra điểm yếu overfit khi preference gần xác định, đề xuất mục tiêu bị chặn.
- **Bài 31 (KTO):** thay cặp preference bằng nhãn nhị phân không ghép cặp (prospect theory); xếp DPO vào lớp "HALO" tổng quát hơn.
- **Bài 32 (SimPO):** bỏ hẳn π_{ref}, dùng log-prob trung bình chuẩn hoá độ dài làm reward ẩn, thêm margin γ.
- **Bài 33 (DPO vs PPO):** khi preference lệch phân phối so với π_{ref}, DPO có thể kém hẳn PPO (đặc biệt sinh code); chỉ ra các yếu tố giúp PPO mạnh hơn.
- **Distillation 28 (MiniLLM), 29 (GKD):** cùng dùng policy gradient, lấy mẫu từ policy đang học, nhưng cho bài toán KD thay vì preference người.

8. Ứng dụng thực tế (suy luận)

- **Khi nào nên dùng DPO:** khi đã có (hoặc thu thập rẻ) tập preference tĩnh (x, y_w, y_l) và một π_{ref} hợp lý. Không cần hạ tầng RL (rollout server, mô hình phần thưởng riêng, value network): phù hợp khi muốn quy trình đơn giản, ít siêu tham số (chủ yếu β, learning rate).
- **Siêu tham số quan trọng:** β (mặc định 0.1, có thể tăng cho tác vụ dài như TL;DR dùng 0.5); độ khớp phân phối π_{ref} với dữ liệu preference (xem bài 33); batch/learning rate nhỏ (RMSprop, 1×10^{-6}) với warmup.
- **Thất bại cần tránh:** bỏ trọng số sigmoid động trong gradient có thể gây thoái hoá mô hình (lặp từ vô nghĩa), như baseline Unlikelihood minh hoạ.

9. Thí nghiệm gợi ý

1. Tái lập ví dụ số ở mục 3.8 bằng code thật: tính $\log \pi_\theta$, $\log \pi_{ref}$ trên một cặp (y_w, y_l) tự tạo, kiểm tra loss và trọng số gradient khớp công thức.
2. Thí nghiệm "lệch π_{ref}": train DPO với π_{ref} không phải mô hình đã sinh preference, đo loss/accuracy trên tập giữ lại so với khi π_{ref} đúng chuẩn (mô phỏng sớm vấn đề bài 33).
3. So sánh DPO với/không có trọng số sigmoid động: bỏ $\sigma(\cdot)$ trong gradient (như Unlikelihood), quan sát hiện tượng lặp từ ở nhiệt độ sinh cao.
4. Đo đường biên reward-KL trên bài toán nhỏ tự tạo (sentiment classifier làm ground-truth reward), so DPO với PPO ở vài mức β/target-KL, vẽ lại Hình 2 trái ở quy mô nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu RLHF (KL-constrained)	$\max_\pi \mathbb{E}[r(x,y)] - \beta D_{KL}(\pi \Vert \pi_{ref})$	công thức (3) của paper
Nghiệm tối ưu dạng đóng	$\pi_r(y\Vert x) = \frac{1}{Z(x)}\pi_{ref}(y\Vert x)\exp(\frac{1}{\beta}r(x,y))$	$Z(x)$ tốn kém, không cần tính nếu dùng DPO
Reward tái tham số hoá	$r(x,y) = \beta \log \frac{\pi(y\Vert x)}{\pi_{ref}(y\Vert x)} + \beta \log Z(x)$	đảo ngược công thức trên
DPO loss	$-\mathbb{E}\log\sigma\big(\beta \log \frac{\pi_\theta(y_w\Vert x)}{\pi_{ref}(y_w\Vert x)} - \beta \log \frac{\pi_\theta(y_l\Vert x)}{\pi_{ref}(y_l\Vert x)}\big)$	$Z(x)$ triệt tiêu nhờ Bradley-Terry
Reward ẩn	$\hat{r}_\theta(x,y) = \beta \log \frac{\pi_\theta(y\Vert x)}{\pi_{ref}(y\Vert x)}$	theo dõi được trong quá trình train
Gradient DPO	$-\beta\mathbb{E}[\sigma(\hat{r}_\theta(y_l) - \hat{r}_\theta(y_w))(\nabla \log \pi(y_w\Vert x) - \nabla \log \pi(y_l\Vert x))]$	trọng số cao khi mô hình xếp sai
Lớp tương đương reward	$r \sim r' \iff r - r' = f(x)$	Lemma 1, 2: cùng preference, cùng policy tối ưu
Theorem 1	mọi lớp tương đương có đại diện $\beta \log \pi/\pi_{ref}$	DPO không mất tổng quát
Siêu tham số mặc định	$\beta = 0.1$ (TL;DR: 0.5), batch 64, RMSprop lr 1e$-$6	Appendix B

11. Kết luận ngắn

DPO chuyển bài toán RLHF có ràng buộc KL thành một mục tiêu phân loại trên các cặp ưu tiên. Phép đổi biến dựa trên nghiệm dạng đóng của chính sách tối ưu và sự triệt tiêu của hằng số chuẩn hoá trong Bradley-Terry; theo Định lý 1, cách biểu diễn này không làm mất lớp reward có thể biểu diễn. Trong các thí nghiệm IMDb, TL;DR và Anthropic-HH, DPO đạt đường biên reward/KL tốt hơn PPO, kể cả khi PPO dùng reward thật, và ít nhạy hơn với nhiệt độ sinh. Trên HH, đây là phương pháp chi phí thấp duy nhất vượt câu trả lời do người chọn. Đổi lại, phương pháp cần mô hình tham chiếu và phụ thuộc vào mối quan hệ giữa phân phối của mô hình này với dữ liệu ưu tiên, một vấn đề được xét tiếp ở bài 33.

Link gốc: https://arxiv.org/abs/2305.18290

Bài 30

A General Theoretical Paradigm to Understand Learning from Human Preferences (2023)

Tác giả: Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, Rémi Munos (Google DeepMind). **Venue / arXiv:** arXiv:2310.12036 (v2, 22/11/2023); bài báo không nêu venue (ở trạng thái "Under review" tại bản được đọc). **Nguồn:** https://arxiv.org/abs/2310.12036 **Chủ đề:** đề xuất một mục tiêu tổng quát ΨPO cho học từ preference người, cho thấy RLHF và DPO (bài 29) là các trường hợp đặc biệt của nó, chỉ ra điểm yếu overfit chung của cả hai khi preference gần xác định, và đưa ra IPO ($\Psi = \text{identity}$) cùng một sampled loss thực thi được để tránh điểm yếu đó. **Vị trí trong lộ trình:** đọc sau bài 29 (DPO), vì IPO xây trực tiếp trên khung và ký hiệu của DPO và dùng DPO làm đối chứng thực nghiệm. Cùng giai đoạn 8 với bài 31 (KTO), 32 (SimPO), 33 (DPO vs PPO).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RLHF triển khai thực tế dựa trên **hai phép xấp xỉ**:

1. **Preference cặp bị thay bằng reward điểm (pointwise):** giả định preference $p^*(y \succ y'|x)$ sinh từ một hàm reward $r(x, y)$ qua mô hình Bradley-Terry $p(y \succ y'|x) = \sigma(r(x, y) - r(x, y'))$, rồi học r_ϕ bằng hồi quy logistic trên dữ liệu preference.
2. **Reward mô hình được giả định tổng quát hoá tốt ra ngoài phân phối (OOD)** mà policy sẽ ghé tới trong quá trình RL.

DPO (bài 29) bỏ được xấp xỉ (2) (không cần học r_ϕ rồi hy vọng nó đúng trên dữ liệu policy tự sinh ra), nhưng **vẫn dựa vào xấp xỉ (1)** (vẫn giả định Bradley-Terry khi đổi biến reward thành policy). Bài này đặt câu hỏi: nếu bỏ luôn cả xấp xỉ (1), điều gì xảy ra?

Khung hoá bài toán: học từ preference người được coi là một **bài toán contextual bandit offline**. Có không gian ngữ cảnh $\mathcal{X}$, không gian hành động (completion) $\mathcal{Y}$, một **policy hành vi** (behavior policy) μ sinh ra các cặp hành động $y, y' \sim \mu(\cdot|x)$ độc lập để người đánh giá. $p^*(y \succ y'|x)$ là xác suất thật (trung bình trên nhiều người đánh giá) rằng y được ưa thích hơn y' cho ngữ cảnh x. Hai ký hiệu tổng quát hoá:

$$p^*(y \succ \mu|x) = \mathbb{E}_{y' \sim \mu(\cdot|x)}[p^*(y \succ y'|x)] \qquad p^*_\rho(\pi \succ \mu) = \mathbb{E}_{x \sim \rho,\, y \sim \pi(\cdot|x)}[p^*(y \succ \mu|x)]$$

($p^*(y \succ \mu|x)$: xác suất y thắng một đối thủ ngẫu nhiên lấy từ μ; $p^*_\rho(\pi \succ \mu)$: tổng preference của toàn bộ policy π so với μ, trung bình trên cả $x \sim \rho$.) Trong thực tế không quan sát trực tiếp p^* mà chỉ có **mẫu nhị phân** $I(y, y'|x) \sim \text{Bernoulli}(p^*(y \succ y'|x))$, tức dataset $D = \{(x_i, y_{w,i} \succ y_{l,i})\}_{i=1}^N$.

2. Ý tưởng chính

Định nghĩa một mục tiêu tổng quát gọi là **Ψ-preference optimisation (ΨPO)**: với $\Psi : [0, 1] \to \mathbb{R}$ là một hàm không giảm bất kỳ,

$$\max_\pi \; \mathbb{E}_{x \sim \rho,\, y \sim \pi(\cdot|x),\, y' \sim \mu(\cdot|x)}[\Psi(p^*(y \succ y'|x))] - \tau\, D_{KL}(\pi \| \pi_{ref})$$

Khi $\Psi(q) = \log \frac{q}{1-q}$ (hàm logit) và preference p^* tuân theo Bradley-Terry đúng nghĩa, ΨPO **trùng** với mục tiêu RLHF (3) và với DPO (Proposition 1, chứng minh ở mục 3.2). Nhưng $\Psi = \text{logit}$ **không bị chặn**: khi $p^* \to 1$ (preference gần xác định), $\Psi(p^*) \to +\infty$, kéo theo ràng buộc KL (hệ số τ cố định) trở nên **vô hiệu**: policy tối ưu sẽ đặt xác suất 0 cho hành động thua, bất kể τ lớn thế nào. Đây là cơ chế overfit chung của cả RLHF (dưới BT) và DPO. Khắc phục: chọn Ψ **bị chặn**, cụ thể là $\Psi = \text{identity}$, gọi là **IPO (Identity-PO)**. Vì identity bị chặn trong $[0, 1]$, ràng buộc KL luôn có hiệu lực thực sự, không bị "nuốt" bởi Ψ không bị chặn.

3. Phương pháp chi tiết

3.1 Nghiệm tối ưu tổng quát của ΨPO (Appendix A.1, diễn giải)

Với hàm $f(y) = \mathbb{E}_{y' \sim \mu}[\Psi(p^*(y \succ y'))]$ cố định theo x (bỏ x cho gọn ký hiệu, như paper làm), ΨPO là bài toán cực đại $\mathbb{E}_{y \sim \pi}[f(y)] - \tau D_{KL}(\pi \| \pi_{ref})$, đúng dạng bài toán KL-constrained đã giải ở bài 29 (mục 3.1), nên có nghiệm đóng tương tự:

$$\boxed{\pi^*(y) \propto \pi_{ref}(y) \, \exp\left(\tau^{-1} \, \mathbb{E}_{y' \sim \mu}[\Psi(p^*(y \succ y'))]\right)}$$

3.2 DPO và RLHF là trường hợp đặc biệt (Proposition 1)

Giả sử Bradley-Terry đúng: $p^*(y \succ y') = \sigma(r(y) - r(y'))$ cho một r nào đó. Với $\Psi(q) = \log \frac{q}{1-q}$:

$$\mathbb{E}_{y' \sim \mu}[\Psi(p^*(y \succ y'))] = \mathbb{E}_{y' \sim \mu}\left[\log \frac{e^{r(y)}}{e^{r(y')}}\right] = \mathbb{E}_{y' \sim \mu}[r(y) - r(y')] = r(y) - \mathbb{E}_{y' \sim \mu}[r(y')]$$

Đây bằng chính reward trong mục tiêu RLHF gốc (công thức 3 ở bài 29), **sai khác một hằng số cộng** (không phụ thuộc y, nên không đổi policy tối ưu theo 3.1). Vậy nghiệm tối ưu ΨPO, RLHF, và DPO (đã biết trùng RLHF theo Rafailov et al., bài 29) **trùng nhau**.

3.3 Vì sao regularization KL trở nên yếu khi preference gần xác định (Section 4.2)

Xét y, y' với $p^*(y \succ y') = 1$ (luôn được thích hơn). Bradley-Terry khi đó yêu cầu $r(y) - r(y') \to +\infty$. Thế vào nghiệm tối ưu (mục 3.1 của bài 29): $\frac{\pi^*(y')}{\pi^*(y)} \to 0$, tức $\pi^*(y') \to 0$ **với mọi** τ, kể cả τ rất lớn (ràng buộc KL rất mạnh). Hệ số τ không còn kiểm soát được mức giữ gần π_{ref}: ràng buộc KL bị vô hiệu hoá bởi việc $\Psi =$logit không bị chặn.

Vấn đề còn nặng hơn ở **chế độ dữ liệu hữu hạn**: ngay cả khi $p^*(y \succ y') = 0.8$ (không hề xác định), với ít mẫu, ước lượng thực nghiệm $\hat{p}(y \succ y')$ hoàn toàn có thể bằng 1 (ví dụ chỉ quan sát một lần và lần đó y thắng). Chính sách tối ưu thực nghiệm khi đó cũng đẩy $\pi(y') \to 0$ với mọi τ. Đây đặc biệt nguy hiểm khi không gian ngữ cảnh/hành động rất lớn (như LLM): nhiều cặp chỉ được quan sát một lần.

Vì sao RLHF chuẩn (học r_ϕ rõ ràng) có thể bớt nhạy hơn trong thực tế: khi preference thực nghiệm nằm ở $\{0, 1\}$, reward tối ưu lý thuyết là $\pm\infty$, nhưng việc huấn luyện r_ϕ bằng mạng neural với điều chỉnh (regularization, early stopping, v.v.) thường **tránh** các giá trị cực trị này, nên r_ϕ bị "underfit" một cách vô tình, và chính sự underfit đó lại là một dạng regularization ngầm giữ policy gần π_{ref}. DPO, vì bỏ hẳn bước học r_ϕ riêng, **mất đi regularization ngầm này**, nên theo lý thuyết dễ overfit hơn ở chế độ preference gần xác định.

3.4 IPO: suy diễn và nghiệm (Section 5.1)

Đặt $\Psi = \text{identity}$. Định nghĩa $g(y) = \mathbb{E}_{y' \sim \mu}[p^*(y \succ y')]$ (không có log/logit, bị chặn trong $[0, 1]$). Nghiệm tối ưu (3.1): $\pi^*(y) \propto \pi_{ref}(y) \exp(\tau^{-1} g(y))$. Với hai hành động bất kỳ $y, y' \in \text{Supp}(\pi_{ref})$:

$$\frac{\pi^*(y)}{\pi^*(y')} = \frac{\pi_{ref}(y)}{\pi_{ref}(y')} \exp\left(\tau^{-1}(g(y) - g(y'))\right)$$

Đặt $h^*(y,y') = \log \frac{\pi^*(y)\pi_{ref}(y')}{\pi^*(y')\pi_{ref}(y)}$ (hiệu log-tỉ-lệ, triệt tiêu được chuẩn hoá), ta được hệ phương trình "root-finding":

$$h^*(y,y') = \tau^{-1}(g(y) - g(y')) = \tau^{-1}(p^*(y \succ \mu) - p^*(y' \succ \mu))$$

Biến thành bài toán tối ưu bằng cách định nghĩa, cho π tham số bất kỳ, $h_\pi(y,y') = \log \frac{\pi(y)\pi_{ref}(y')}{\pi(y')\pi_{ref}(y)}$ và hàm mất mát (population IPO loss):

$$L(\pi) = \mathbb{E}_{y,y'\sim\mu}\left[\left(h_\pi(y,y') - \frac{p^*(y \succ \mu) - p^*(y' \succ \mu)}{\tau}\right)^2\right]$$

$L(\pi^*) = 0$ nên π^* là nghiệm cực tiểu toàn cục. **Theorem 2 (duy nhất nghiệm):** với giả thiết $\text{Supp}(\mu) = \text{Supp}(\pi_{ref})$, hàm L (viết theo logit s tham số hoá π_s) là một **dạng toàn phương dương nửa xác định** theo s (trừ hướng cộng hằng số vào mọi logit, không ảnh hưởng π_s), do đó **lồi chặt** theo lớp tương đương của s, và π^* là cực tiểu toàn cục/địa phương duy nhất.

3.5 Sampled loss cho IPO và thuật toán (Section 5.2, Proposition 3)

Khó khăn: $p^*(y \succ \mu)$ không quan sát trực tiếp, chỉ có mẫu nhị phân $I(y,y')$. Định nghĩa **population IPO loss dạng mẫu**:

$$L_{sample}(\pi) = \mathbb{E}_{y,y'\sim\mu}[(h_\pi(y,y') - \tau^{-1}I(y,y'))^2]$$

Proposition 3 chứng minh (bằng khai triển đại số, tận dụng $\mathbb{E}_{y'\sim\mu}[I(y,y')] = p^*(y \succ \mu)$ và tính đối xứng của cặp (y,y')) hai loss này bằng nhau (sai khác hằng số không phụ thuộc π), tức có thể tối ưu loss có thể lấy mẫu trực tiếp từ dataset preference mà không cần biết p^*. Với dữ liệu $D = \{(y_{w,i}, y_{l,i})\}$, mỗi điểm dữ liệu góp hai mẫu $(y,y',I) = (y_w, y_l, 1)$ và $(y_l, y_w, 0)$; khai triển và rút gọn (tận dụng đối xứng) cho **loss thực nghiệm cuối cùng**:

$$\boxed{\mathcal{L}_{IPO}(\pi;\pi_{ref}) = \mathbb{E}_{(y_w,y_l)\sim D}\left[\left(h_\pi(y_w,y_l) - \frac{1}{2\tau}\right)^2\right]}, \qquad h_\pi(y_w,y_l) = \log\frac{\pi(y_w)\pi_{ref}(y_l)}{\pi(y_l)\pi_{ref}(y_w)}$$

So với DPO loss (bài 29, $-\log\sigma(\beta\cdot h_\pi(y_w,y_l))$ về bản chất), IPO loss là một **hồi quy bình phương** (regression) của hiệu log-tỉ-lệ $h_\pi(y_w,y_l) = \log\frac{\pi(y_w|x)}{\pi(y_l|x)} - \log\frac{\pi_{ref}(y_w|x)}{\pi_{ref}(y_l|x)}$ về một **mục tiêu hữu hạn cố định** $\tau^{-1}/2$, bất kể preference có xác định hay không. Chính việc mục tiêu luôn hữu hạn (không phải $+\infty$ như DPO đòi hỏi khi preference xác định) là điều giữ cho IPO luôn regularize về π_{ref} theo đúng cường độ τ quy định: τ nhỏ (regularization yếu) thì mục tiêu $1/(2\tau)$ lớn, cho phép hiệu log-tỉ-lệ lớn; τ lớn thì mục tiêu nhỏ, ép π gần π_{ref} hơn.

Thuật toán 1 (Sampled IPO):

```
Input: D = {(x, yw, yl)}, pi_ref, tau
1. Dinh nghia h_pi(y, y', x) = log( pi(y|x)*pi_ref(y'|x) / (pi(y'|x)*pi_ref(y|x)) )
2. Khoi tao pi = pi_ref
3. repeat: lay minibatch (x, yw, yl) tu D
     loss = E[ (h_pi(yw, yl, x) - 1/(2*tau))^2 ]
     cap nhat pi (gradient descent) de giam loss
```

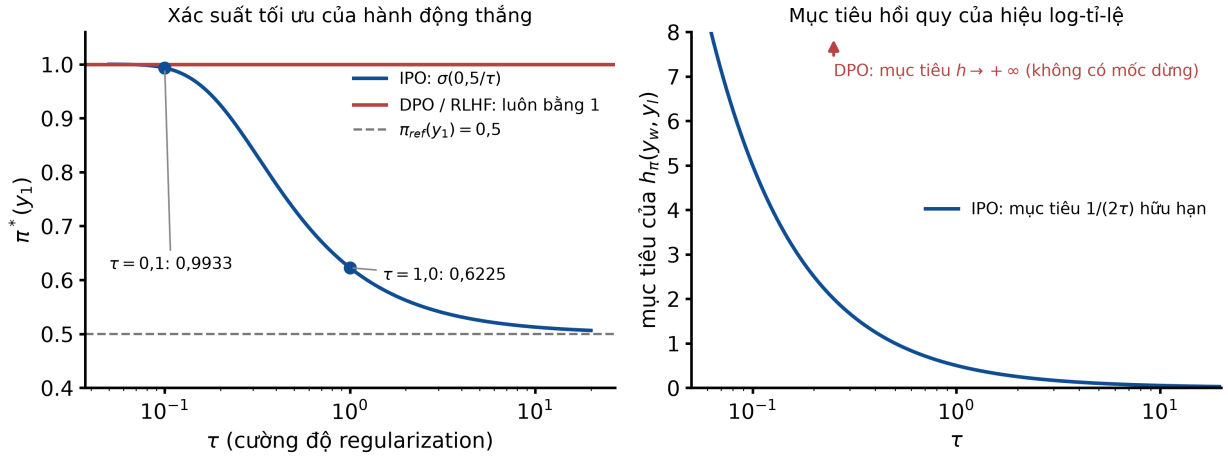
3.6 Ví dụ minh hoạ: 2 hành động, preference xác định (Section 5.3.1, nghiệm của paper; kiểm tra số "tự tính")

Hai hành động y_1, y_2, π_{ref} và μ đều uniform, preference **xác định**: $p^*(y_1 \succ y_2) = 1$. Theo mục 3.3: **DPO/RLHF luôn hội tụ về** $\pi^*(y_1) = 1, \pi^*(y_2) = 0$ **với mọi** τ.

Với IPO: $p^*(y_1 \succ \mu) = \frac{1}{2} \cdot p^*(y_1 \succ y_1) + \frac{1}{2} \cdot p^*(y_1 \succ y_2) = \frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot 1 = \frac{3}{4}$ (quy ước $p^*(y \succ y) = \frac{1}{2}$), tương tự $p^*(y_2 \succ \mu) = \frac{1}{4}$. Thế vào nghiệm 3.1:

$$\pi^*(y_1) = \frac{\exp(0.75/\tau)}{\exp(0.75/\tau) + \exp(0.25/\tau)} = \sigma(0.5/\tau), \qquad \pi^*(y_2) = \sigma(-0.5/\tau)$$

Tự tính kiểm tra với $\tau = 1$**:** $\pi^*(y_1) = \sigma(0.5) = \frac{1}{1+e^{-0.5}} \approx \frac{1}{1+0.6065} \approx 0.6225$, $\pi^*(y_2) \approx 0.3775$. **Với** $\tau = 0.1$ (regularization yếu hơn 10 lần): $\pi^*(y_1) = \sigma(5) \approx \frac{1}{1+0.0067} \approx 0.9933$, gần quyết định hơn nhưng **vẫn chưa bằng 1** như DPO. Khi $\tau \to \infty$ (regularization rất mạnh), $\pi^* \to \pi_{ref} = (0.5, 0.5)$; khi $\tau \to 0$, $\pi^* \to (1, 0)$ giống DPO. Vậy với IPO, τ **thực sự kiểm soát được** mức độ gần π_{ref}, trong khi với DPO, τ (hay β) không kiểm soát được khi preference xác định.



Hình 61: Bài toán hai hành động với preference xác định: nghiệm IPO theo tau so với nghiệm DPO luôn bằng 1, và mục tiêu hồi quy hữu hạn của IPO (minh hoạ, tự tính)

Cách đọc hình: panel trái vẽ xác suất tối ưu của hành động thắng, đường xanh là nghiệm IPO sigma(0,5/tau) còn đường đỏ là DPO luôn đặt toàn bộ xác suất vào hành động này. Tau càng lớn thì nghiệm IPO càng gần pi_ref (0,5), tau càng nhỏ thì càng gần 1 nhưng không chạm. Panel phải cho thấy IPO hồi quy về mốc hữu hạn 1/(2 tau), còn DPO không có mốc dừng. Đường cong được tính từ công thức trong ghi chú.

4. Thí nghiệm và kết quả chính

Bài báo không chạy trên LLM thật, chỉ dùng **bài toán bandit đồ chơi** (không có ngữ cảnh x, 3 hành động $\{y_a, y_b, y_c\}$) để minh hoạ khác biệt định tính DPO/IPO, tối ưu bằng Adam (lr 0.01, batch size 9, 18000 bước), cài bằng flax/optax, lặp lại 10 seed, báo cáo trung bình và khoảng tin cậy 95%, chạy trên VM cloud 4 core/32GB RAM.

Thí nghiệm 1 (IPO tránh policy "tham lam" tuyệt đối): dataset $D_1 = \{(y_a, y_b), (y_b, y_c), (y_a, y_c)\}$ (một thứ tự toàn phần: y_a thắng cả hai, y_b thắng y_c). Kết quả (Hình 1): **DPO luôn hội tụ về policy xác định** (đặt hết xác suất vào y_a) ở **mọi** giá trị τ được thử; **IPO ngăn được điều này khi** τ **đủ lớn**, giữ policy gần π_{ref} hơn.

Thí nghiệm 2 (IPO không loại bỏ hành động chưa từng thắng): vẫn với D_1, xét hành động "đối ngẫu" (y_c,

chưa từng thắng ai trong D_1): DPO đặt xác suất y_c về 0 bất kể τ (ít gây hại hơn thí nghiệm 1 vì chỉ một xác suất bị méo, nhưng tình huống này **phổ biến hơn trong dữ liệu thật**: không gian hành động lớn, dataset nhỏ, nhiều hành động chỉ xuất hiện 0 hoặc 1 lần).

Thí nghiệm 3 (cặp hoàn toàn chưa quan sát): dataset $D_3 = \{(y_a, y_b), (y_b, y_a)\}$ (hai preference mâu thuẫn giữa y_a, y_b), cặp (y_a, y_c) **hoàn toàn không xuất hiện**. Kết quả (Hình 2): **DPO vẫn bỏ qua π_{ref} hoàn toàn** cho xác suất của hành động không quan sát, bất kể τ; **IPO giảm dần xác suất hành động không quan sát theo τ** một cách có kiểm soát.

5. Phân tích / ablation

- **Tính duy nhất nghiệm phụ thuộc giả thiết hỗ trợ (Appendix A.2):** nếu $\text{Supp}(\pi)$ được phép lớn hơn $\text{Supp}(\mu)$, bài toán IPO có **vô số nghiệm cực tiểu toàn cục** (ví dụ cụ thể với 3 hành động, μ chỉ phủ 2/3 hành động: mọi π thoả một ràng buộc tỉ lệ p/q đều là cực tiểu). Giả thiết $\text{Supp}(\mu) = \text{Supp}(\pi_{ref})$ ở Theorem 2 là cần thiết để đảm bảo nghiệm duy nhất.
- **Tổng quát hoá: DPO = RLHF không cần giả thiết Bradley-Terry đúng (Proposition 4, Appendix B):** chỉ cần tồn tại một minimizer cho loss Bradley-Terry trên p^* bất kỳ (không nhất thiết p^* thật sự là BT), nghiệm tối ưu của DPO và của RLHF dùng reward học từ loss đó **luôn trùng nhau**. Đây mở rộng kết quả tương đương DPO-RLHF của bài 29 ra khỏi giả thiết BT đúng, cho thấy vấn đề overfit ở mục 3.3 không phải do BT sai mà do bản chất không bị chặn của Ψ =logit.
- Hai thí nghiệm toy chỉ chạy trên bandit không ngữ cảnh, 3 hành động; khác biệt định tính rõ nhưng quy mô rất nhỏ so với LLM thật.

6. Giới hạn

Tác giả nêu: các thí nghiệm minh hoạ chỉ ở quy mô bandit đơn giản ("These minimal experiments are sufficient to prove that IPO is better suited..."); đề xuất tương lai là mở rộng lên huấn luyện LLM thật trên dữ liệu preference người.

Nhận xét:

- Bài báo không cho bất kỳ thực nghiệm nào trên LLM hay dữ liệu preference thật (khác DPO, KTO, SimPO trong cùng giai đoạn 8 đều có thực nghiệm LLM); tất cả minh chứng về IPO vs DPO chỉ là lý thuyết + bandit đồ chơi.
- Theorem 2 (duy nhất nghiệm) cần $\text{Supp}(\mu) = \text{Supp}(\pi_{ref})$, một giả thiết khó kiểm chứng với LLM (không gian câu trả lời astronomic, không thể biết "hỗ trợ" của phân phối).
- Chưa phân tích tốc độ hội tụ/độ ổn định tối ưu của loss bình phương IPO so với loss logistic DPO trên mạng neural lớn (bài 32, SimPO, sau này báo cáo vài con số IPO trên LLM thật để so sánh, cho thấy IPO không hẳn vượt DPO về win rate thực tế mặc dù lý thuyết regularize tốt hơn).
- Chưa thảo luận cách chọn τ trong thực tế (tương tự β của DPO) khi không biết độ "xác định" của preference trước khi huấn luyện.

7. Liên hệ với các bài khác

- **Bài 29 (DPO):** IPO dùng chung ký hiệu, chung điểm khởi đầu (nghiệm đóng KL-constrained); IPO chỉ ra điểm yếu lý thuyết (overfit khi preference gần xác định) mà DPO không đề cập, và đề xuất hướng sửa bằng cách đổi Ψ.
- **Bài 31 (KTO):** cũng xuất phát từ lo ngại DPO/preference pairwise có thể không phải cách tốt nhất để mô hình hoá "giá trị" người dùng gán cho một câu trả lời (KTO dùng prospect theory thay vì xét riêng hàm Ψ); hai bài tấn công cùng một lo ngại (tổng quát hoá quá mức của BT/pairwise) từ hai góc độ khác nhau.

- **Bài 32 (SimPO):** trong bảng so sánh các loss preference optimization của SimPO, IPO là một trong các baseline (loss dạng bình phương với τ); SimPO báo cáo IPO thường **không vượt DPO** trên benchmark LLM thật (AlpacaEval2, Arena-Hard), dù về lý thuyết IPO tránh được overfit mà bài này nêu, một mâu thuẫn thú vị giữa lý thuyết và thực nghiệm đáng lưu tâm khi đọc tiếp.
- **Bài 33 (DPO vs PPO):** cùng tinh thần "chỉ ra điểm yếu lý thuyết của DPO", nhưng tập trung vào OOD response (reward ngoài phân phối dữ liệu preference) thay vì overfit do preference xác định; hai cơ chế yếu điểm khác nhau nhưng cùng gốc "DPO suy ra reward hoàn toàn từ dữ liệu preference tĩnh, không có cơ chế kiểm soát ngoài π_{ref} khi dữ liệu đó cực đoan/thiếu".

8. Ứng dụng thực tế (suy luận)

Mức liên quan: **trung bình**, chủ yếu mang tính cảnh báo lý thuyết hơn là một công thức luôn-nên-dùng.

- **Khi dữ liệu preference nhỏ hoặc có nhiều cặp "áp đảo" (một completion luôn thắng):** cân nhắc IPO hoặc một dạng loss bị chặn khác thay cho DPO thuần, vì DPO có nguy cơ lý thuyết đẩy xác suất completion thua về 0 một cách không kiểm soát được bởi β/τ. Dữ liệu preference tổng hợp từ AI (preference gần như xác định, một completion rõ ràng đúng/sai) đặc biệt dễ rơi vào tình huống này.
- **Chọn τ (hay β):** với IPO, τ có ý nghĩa trực tiếp hơn: mục tiêu hồi quy $1/(2\tau)$ là biên trên hữu hạn cho hiệu log-tỉ-lệ, nên có thể chọn τ dựa trên "biên log-tỉ-lệ tối đa mong muốn" thay vì dò mù.
- **Cảnh báo thực nghiệm (từ bài 32):** lý thuyết tốt hơn không đảm bảo thực nghiệm tốt hơn; nếu dùng IPO cho tác vụ thực tế, nên benchmark song song với DPO trên chính dữ liệu của mình trước khi chọn.

9. Thí nghiệm gợi ý

1. Tái lập ví dụ 2 hành động ở mục 3.6 bằng code: quét $\tau \in \{0.01, 0.1, 1, 10\}$, vẽ $\pi^*(y_1)$ theo IPO so với $\pi_{DPO}(y_1) \equiv 1$, kiểm chứng xu hướng hội tụ về π_{ref} khi τ tăng.
2. Dựng lại 3 thí nghiệm toy (D1, D2, D3) của Section 5.3-5.4 bằng PyTorch/JAX nhỏ, so sánh đường học xác suất hành động giữa DPO và IPO.
3. Trên một tập preference LLM nhỏ tự tạo có tỉ lệ cặp "xác định" cao (ví dụ 80% cặp có một completion rõ ràng tốt hơn), so sánh accuracy trên tập giữ lại giữa DPO và IPO ở cùng số bước train, xem IPO có tránh được overfit như lý thuyết dự đoán không.
4. Đo độ nhạy của IPO với giả thiết $\text{Supp}(\mu) = \text{Supp}(\pi_{ref})$: cố tình tạo preference dataset chỉ phủ một phần nhỏ không gian hành động, quan sát IPO có hội tụ tới nghiệm "kỳ lạ" như cảnh báo ở Appendix A.2 không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
ΨPO (mục tiêu tổng quát)	$\max_\pi \mathbb{E}[\Psi(p^*(y \succ y'\|x))] - \tau D_{KL}(\pi\|\pi_{ref})$	Ψ không giảm, bất kỳ
Nghiệm tối ưu	$\pi^*(y) \propto \pi_{ref}(y)\exp(\tau^{-1}\mathbb{E}_{y'\sim\mu}[\Psi(p^*(y \succ y'))])$	tổng quát hoá nghiệm DPO/RLHF
DPO/RLHF = ΨPO với Ψ =logit	$\Psi(q) = \log \frac{q}{1-q}$, không bị chặn	Proposition 1
Overfit khi preference xác định	$p^* \to 1 \Rightarrow \pi^*(y') \to 0$ với mọi τ	regularization KL bị vô hiệu
IPO (Ψ =identity)	$g(y) = \mathbb{E}_{y'\sim\mu}[p^*(y \succ y')]$, bị chặn $[0, 1]$	giữ regularization hiệu lực
IPO population loss	$\mathbb{E}[(h_\pi(y, y') - \tau^{-1}(p^*(y \succ \mu) - p^*(y' \succ \mu)))^2]$	Theorem 2: nghiệm duy nhất
IPO sampled loss	$\mathbb{E}_{(y_w,y_l)\sim D}[(h_\pi(y_w, y_l) - \frac{1}{2\tau})^2]$	hồi quy về mục tiêu hữu hạn

Khái niệm	Công thức / ý	Ghi chú
$h_\pi(y, y')$	log $\frac{\pi(y)\pi_{ref}(y')}{\pi(y')\pi_{ref}(y)}$	hiệu log-tỉ-lệ, giống số hạng trong DPO
Ví dụ 2 hành động, $p^* = 1$	$\pi^*_{IPO}(y_1) = \sigma(0.5/\tau)$ vs $\pi^*_{DPO}(y_1) = 1$ mọi τ	minh hoạ IPO giữ gần π_{ref}

11. Kết luận ngắn

Bài này đặt RLHF và DPO vào một khung tổng quát hơn (ΨPO) và chỉ ra cả hai, dưới giả thiết Bradley-Terry, tương đương với lựa chọn Ψ =logit, một hàm không bị chặn khiến ràng buộc KL mất hiệu lực khi preference tiến tới xác định (dù là do bản chất bài toán hay do thiếu dữ liệu). Khắc phục bằng IPO (Ψ =identity): suy ra một loss hồi quy bình phương có mục tiêu luôn hữu hạn ($1/(2\tau)$), giữ cho tham số τ thực sự kiểm soát mức gần π_{ref} trong mọi trường hợp. Minh hoạ trên vài bài toán bandit đồ chơi xác nhận trực giác này, nhưng paper chưa kiểm chứng trên LLM thật, một khoảng trống mà các bài sau (đặc biệt bài 32, SimPO) sẽ lấp một phần, với kết quả thực nghiệm không hoàn toàn ủng hộ ưu thế lý thuyết của IPO.

Link gốc: https://arxiv.org/abs/2310.12036

Bài 31

KTO: Model Alignment as Prospect Theoretic Optimization (2024)

Tác giả: Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, Douwe Kiela (Stanford University, Contextual AI). **Venue / arXiv:** ICML 2024 (PMLR 235); arXiv:2402.01306 (bản được đọc: v5, mốc thời gian "8 Sep 2026" ghi trên trang đầu; tác giả ghi chú bản này chỉ sửa chứng minh và cách diễn đạt, kết quả thực nghiệm và tuyên bố lý thuyết không đổi so với bản ICML gốc). **Nguồn:** https://arxiv.org/abs/2402.01306
Chủ đề: dùng lý thuyết prospect (Kahneman-Tversky) để giải thích vì sao các loss alignment hiện có (DPO, PPO-Clip) hoạt động tốt, định nghĩa họ loss "human-aware" (HALO), rồi đề xuất KTO, một HALO học trực tiếp từ nhãn nhị phân desirable/undesirable, không cần dữ liệu preference ghép cặp. **Vị trí trong lộ trình:** đọc sau bài 29 (DPO) và 30 (IPO); KTO dùng lại reward reparameterization và lý thuyết equivalence class của DPO, và chỉ ra DPO chính là một trường hợp của HALO. Cùng giai đoạn 8 với bài 32 (SimPO), 33 (DPO vs PPO).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RLHF và DPO (bài 29) đều cần dữ liệu **preference** (cặp $y_w \succ y_l$ cho cùng x), vốn **khan hiếm và đắt** để thu thập (người đánh giá phải xem hai output rồi so sánh). Nhưng phản hồi người dùng thực tế thường ở dạng **đơn giản hơn**: like/dislike, approve/reject, thumbs up/down cho **một** output, không cần so sánh cặp. Câu hỏi đặt ra: có thể align model tốt bằng loại phản hồi nhị phân này không, và tại sao các phương pháp hiện tại (DPO, PPO) lại hoạt động tốt đến vậy?

Bài này tiếp cận qua **lý thuyết prospect** (Kahneman & Tversky, 1979, 1992), lý thuyết mô tả cách con người đưa ra quyết định dưới rủi ro **không** theo kỳ vọng toán học chuẩn. Ví dụ kinh điển: một canh bạc trả 100 đô với xác suất 80% hoặc 60 đô với xác suất 20% (kỳ vọng = 92 đô), nhiều người vẫn chọn nhận chắc 60 đô vì **loss aversion** (nhạy cảm với mất mát hơn được lợi tương đương) và **diminishing sensitivity** (độ nhạy giảm dần khi xa điểm tham chiếu).

2. Ý tưởng chính

Khung lý thuyết prospect định nghĩa giá trị chủ quan của một kết quả z so với điểm tham chiếu z_0 qua một **hàm giá trị** $v(z; z_0)$ (lồi trong vùng mất mát, lõm trong vùng lợi ích, dốc hơn ở phía mất mát). Bài báo cho thấy: nếu viết lại reward ngầm định (implied reward) của một mô hình alignment dưới dạng $r_\theta(x, y) = l(y) \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$ và xem loss của nó như $\mathbb{E}[a_{x,y}\, v_{x,y}(r_\theta(x, y) - \mathbb{E}_Q[r_\theta(x, Y')])]$, thì **DPO và PPO-Clip đều thoả định nghĩa này** (Theorem 3.5): chúng ngầm định một hàm giá trị kiểu prospect theory, dù hàm đó **không hoàn toàn khớp** với hàm giá trị Kahneman-Tversky gốc (ước tính từ thực nghiệm trên canh bạc tiền bạc). Gọi họ loss này là **HALO (human-aware loss)**.

Từ đó đề xuất **KTO (Kahneman-Tversky Optimization)**: xây một HALO **trực tiếp** từ hàm giá trị Kahneman-Tversky (thay vì suy ra ngầm định qua preference log-likelihood như DPO), học từ nhãn nhị phân "desirable/undesirable" không ghép cặp. Chỉ cần tín hiệu yếu hơn, nhưng dữ liệu đó rẻ, nhiều, dễ scale hơn, và theo thực nghiệm, KTO **khớp hoặc vượt** DPO ở nhiều quy mô.

3. Phương pháp chi tiết

3.1 Lý thuyết prospect (Section 3.1)

Định nghĩa 3.1-3.3 (diễn giải): hàm giá trị $v : \mathcal{Z} \to \mathbb{R}$ ánh xạ kết quả z (so với điểm tham chiếu z_0) thành giá trị cảm nhận; hàm trọng số ω biến xác suất tích luỹ thành xác suất cảm nhận (bỏ qua trong bài này vì con người không thấy toàn bộ phân phối của LLM); độ hữu dụng của biến ngẫu nhiên $u(Z) = \sum_z \omega_z v(z; z_0)$. Dạng hàm giá trị Kahneman-Tversky thực nghiệm (1992):

$$v(z; \lambda, \alpha, z_0) = \begin{cases} (z - z_0)^\alpha & z \geq z_0 \\ -\lambda(z_0 - z)^\alpha & z < z_0 \end{cases} \qquad \alpha = 0.88,\ \lambda = 2.25 \text{ (trung vị thực nghiệm)}$$

(α: độ cong, kiểm soát risk aversion; λ: độ dốc phía mất mát, kiểm soát loss aversion.)

3.2 Định nghĩa HALO (Definition 3.4, công thức 5-6)

Với $r_\theta(x, y) = l(y) \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$ (reward ngầm định, đo bằng nats, l là hệ số chuẩn hoá), $Q(Y'|x)$ là **phân phối điểm tham chiếu** trên $\mathcal{Y}$, giá trị con người của (x, y) là:

$$v_{x,y}\big(r_\theta(x, y) - \mathbb{E}_Q[r_\theta(x, Y')]\big)$$

Một hàm f là **HALO** cho v nếu tồn tại $a_{x,y} \in \mathbb{R} \setminus \{0\}$ sao cho:

$$\boxed{f(\pi_\theta, \pi_{ref}) = \mathbb{E}_{x,y\sim D}\big[a_{x,y}\, v_{x,y}(r_\theta(x, y) - \mathbb{E}_Q[r_\theta(x, Y')])\big] + C_D}$$

với l, $a_{x,y}$, $v_{x,y}$, C_D (hằng số phụ thuộc dữ liệu) chọn **độc lập** với π_θ, π_{ref}; Q có thể phụ thuộc policy. Nhắc lại nghiệm tối ưu KL-constrained (bài 29, mục 3.1): $\pi^*(y|x) = \frac{1}{Z(x)}\pi_{ref}(y|x) \exp(\frac{1}{\beta} r^*(x, y))$, nên với $l = \beta$: $r^*_\theta(x, y) = r^*(x, y) - \beta \log Z(x)$ (công thức 7), tức reward do HALO ngầm định tại nghiệm tối ưu **nằm trong cùng equivalence class** với r^* (Lemma 1, DPO, bài 29).

3.3 DPO và PPO-Clip là HALO (Theorem 3.5, sơ lược chứng minh Appendix B)

DPO: đặt $\tilde{x} = (x, y_l)$ (augment input), $\tilde{\pi}_\theta(y|\tilde{x}) := \pi_\theta(y|x)$, chọn $l(y) = \beta$, $v_{\tilde{x},y}(z) = \log \sigma(z)$, $a_{\tilde{x},y} = -1$, $Q(Y'|\tilde{x}) = \delta_{y_l}(Y')$ (điểm tham chiếu là chính y_l). Khi đó $a \cdot v(r_\theta(\tilde{x}, y_w) - \mathbb{E}_Q[r_\theta]) = -\log \sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)})$, lấy kỳ vọng đúng là $\mathcal{L}_{DPO}$ với $C_D = 0$. Vì $\log \sigma$ tăng ngặt nên v không giảm; $\pi_\theta = \pi_{ref}$ cho giá trị 0 (điều kiện invariance). Vậy **DPO là HALO**.

PPO-Clip (offline): với $q_\theta = \pi_\theta(y_t|x{:}y_{<t})/\pi_{ref}(y_t|x{:}y_{<t})$, $l = 1$, $r_\theta = \log q_\theta$, chọn Q là hỗn hợp $\alpha\pi_\theta + (1-\alpha)\pi_{ref}$ sao cho $\mathbb{E}_Q[r_\theta] = 0$ (luôn tồn tại $\alpha \in [0, 1]$ vì $\mathbb{E}_{\pi_\theta}[r_\theta] \geq 0 \geq \mathbb{E}_{\pi_{ref}}[r_\theta]$), $v_{s,a}(z) = \min(e^z, 1+\epsilon)$ hoặc $\max(e^z, 1-\epsilon)$ tuỳ dấu advantage A. Mỗi nhánh không giảm theo z, và $a_{s,a} v_{s,a}(r_\theta) = -\min(q_\theta A, \text{clip}(q_\theta, 1-\epsilon, 1+\epsilon)A)$, đúng số hạng trong loss PPO-Clip. Vậy **PPO-Clip cũng là HALO**.

Phản ví dụ (không phải HALO): CSFT (conditional SFT, cross-entropy thường) không thoả điều kiện invariance ($f(\pi, \pi)$ không hằng); SLiC (kết hợp max-margin và loss ngôn ngữ) cũng không, vì tại $\pi_\theta = \pi_{ref} = \pi$, thành phần $\lambda_{reg}\mathbb{E}_{y\sim\pi}[-\log \pi(y|x)]$ vẫn phụ thuộc π.

3.4 Suy ra KTO (Section 4.1, công thức 8-9)

Ba thay đổi so với hàm giá trị Kahneman-Tversky gốc (mục 3.1) để tránh bất ổn số và phù hợp ngữ cảnh LLM:

1. Thay lũy thừa $(\cdot)^\alpha$ bằng hàm logistic σ (cũng lõm trong lợi ích, lồi trong mất mát, nhưng ổn định số hơn).

2. Thêm hyperparameter $\beta \in \mathbb{R}_+$ vào trong hàm giá trị để kiểm soát độ risk aversion (độ bão hoà), đóng vai trò tương tự β trong DPO.
3. Thay hệ số loss aversion λ duy nhất bằng $\{\lambda_D, \lambda_U\}$ riêng cho ví dụ desirable/undesirable.
4. Thay vì lấy **một** output thua làm điểm tham chiếu (như DPO lấy y_l), giả định con người đánh giá $y|x$ so với **toàn bộ phân phối output có thể có**: $Q(Y'|x) = \pi_\theta(Y'|x)$, nên điểm tham chiếu $z_0 = KL(\pi_\theta(y'|x)\|\pi_{ref}(y'|x))$.

Với $\lambda_y = \lambda_D$ nếu y desirable, λ_U nếu undesirable, **KTO loss**:

$$\boxed{\mathcal{L}_{KTO}(\pi_\theta, \pi_{ref}) = \mathbb{E}_{x,y\sim D}[\lambda_y - v(x,y)]}, \quad r_\theta(x,y) = \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}, \quad z_0 = KL(\pi_\theta\|\pi_{ref})$$

$$v(x,y) = \begin{cases} \lambda_D\, \sigma(\beta(r_\theta(x,y) - z_0)) & y \sim y_{desirable}|x \\ \lambda_U\, \sigma(\beta(z_0 - r_\theta(x,y))) & y \sim y_{undesirable}|x \end{cases}$$

Không lan truyền gradient qua z_0 (chỉ dùng để định vị độ bão hoà của loss, giúp train ổn định hơn). Để kiểm chứng đúng Definition 3.4: $l = 1$, $Q = \pi_\theta$, $v_{x,y}(z) = \sigma(\beta z)$, $a_{x,y} = -\lambda_y$, $\{\lambda_y\}$ gộp vào C_D; tất cả độc lập với π_θ, π_{ref} ngoại trừ Q, đúng yêu cầu.

Trực giác: nếu model tăng reward của một ví dụ desirable một cách "thô" (không học đúng đặc điểm khiến nó tốt), KL penalty z_0 cũng tăng theo, nên $r_\theta - z_0$ không tăng, loss không giảm. Điều này buộc model học đúng cái gì khiến output tốt, để tăng r_θ mà giữ (hoặc giảm) z_0.

3.5 Ước lượng điểm tham chiếu $\hat{z}_0$ (biased estimate)

Lấy mẫu π_θ để ước lượng z_0 chính xác quá chậm. Thay vào đó, trong một microbatch kích thước m, **xáo trộn (shift)** các đầu ra để tạo cặp lệch $\{(x_1, y_2), (x_2, y_3), \ldots, (x_m, y_1)\}$ (với $j = (i \bmod m) + 1$), rồi ước lượng một z_0 **chung** cho cả batch:

$$\hat{z}_0 = \max\Big(0,\ \frac{1}{m} \sum_{1\le i\le m} \log \frac{\pi_\theta(y_j|x_i)}{\pi_{ref}(y_j|x_i)}\Big)$$

Đây là ước lượng **thiên lệch (biased)** của KL nhưng rẻ để tính; việc chặn dưới ở 0 (clamp) thêm một độ lệch dương nhưng giảm phương sai. Dùng y_j lệch (không phải y_i gốc) vì y_i thường được chọn có chủ đích là ví dụ rất tốt/rất tệ, mang reward biên độ lớn không đại diện. Yêu cầu **microbatch size** ≥ 2. Nếu KTO được chạy sau SFT trên cùng dữ liệu desirable, $\hat{z}_0$ nhanh về gần 0 và có thể bỏ qua bước này (đặt $\hat{z}_0 = 0$); nếu không có SFT trước hoặc dữ liệu SFT không phải tập con dữ liệu KTO, vẫn cần ước lượng.

3.6 Gradient của KTO (công thức 10, Proposition 4.1)

Đặt $d(y) = -1$ nếu y desirable, $+1$ nếu undesirable; $z = r_\theta(x,y) - z_0$ (không lan truyền gradient qua z_0):

$$\boxed{\nabla_\theta \ell_{KTO}(x,y) = d(y)\, \lambda_y\, \sigma(\beta z)\big(1 - \sigma(\beta z)\big)\, \beta\, \nabla_\theta \log \pi_\theta(y|x)}$$

Với y desirable, $d(y) = -1$: gradient descent **tăng** $\log \pi_\theta(y|x)$ (đẩy xác suất lên); với y undesirable, dấu ngược lại, đẩy xác suất xuống. **Proposition 4.1:** khi $z \to \pm\infty$ (ví dụ đã "quá dễ" hoặc "quá khó" theo policy hiện tại), $\sigma(\beta z)(1 - \sigma(\beta z)) \to 0$, nên **gradient của ví dụ đó tiến về 0**, miễn $\|\nabla_\theta \log \pi_\theta(y|x)\|$ bị chặn. Đây là tính chất **bão hoà (saturating)**: KTO tự động hạ trọng số các ví dụ có reward đã cực đoan, kể cả nếu chúng bị gán nhãn sai (noisy label).

3.7 Thuật toán (tổng hợp)

```
Input: D = {(x, y, label desirable/undesirable)}, pi_ref, beta, lambda_D, lambda_U,
↪  batch size m >= 2
1. Lay minibatch m vi du (x_i, y_i, label_i)
2. Tao cap lech (x_i, y_{(i mod m)+1}) de uoc luong z0_hat (cong thuc 3.5), KHONG lan
↪  truyen gradient qua z0_hat
3. Voi moi vi du: tinh r_theta(x_i, y_i) = log(pi_theta(y_i|x_i)/pi_ref(y_i|x_i))
4. Tinh v(x_i, y_i) theo cong thuc KTO (desirable dung sigma(beta*(r-z0)), undesirable
↪  dung sigma(beta*(z0-r)))
5. loss = mean(lambda_{y_i} - v(x_i, y_i))
6. Cap nhat theta bang gradient descent tren loss
```

3.8 Ví dụ minh hoạ bằng số (tự tính)

Xét x cố định, y **desirable**, $\pi_{ref}(y|x) = 0.01$, $\pi_\theta(y|x) = 0.03$ (mô hình đã tăng xác suất của y gấp 3), $\beta = 0.1$, $\lambda_D = \lambda_U = 1$, và giả sử $\hat{z}_0 = 0.05$ (ước lượng KL theo công thức 3.5).

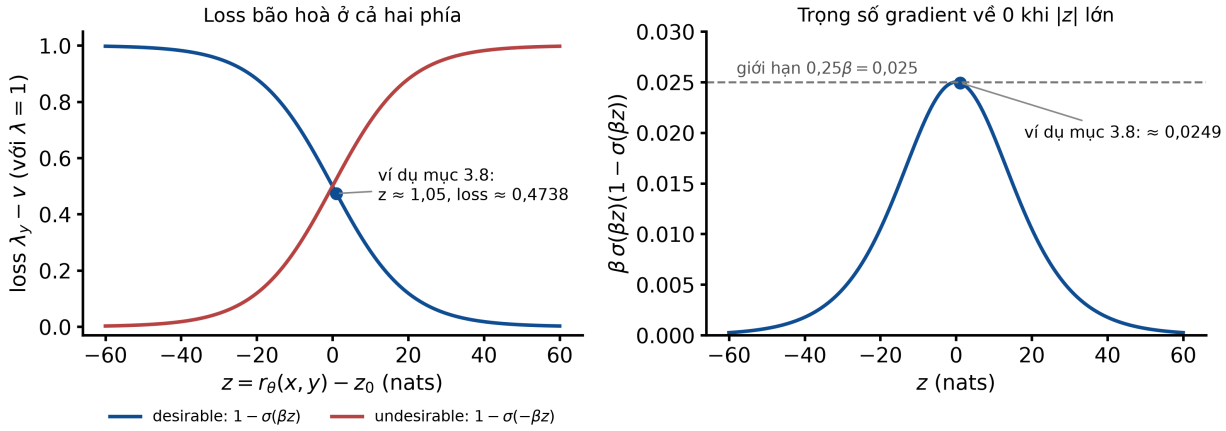
$$r_\theta(x,y) = \log \frac{0.03}{0.01} = \log 3 \approx 1.0986$$

$$z = r_\theta - \hat{z}_0 = 1.0986 - 0.05 = 1.0486, \qquad \beta z = 0.10486$$

$$\sigma(\beta z) = \frac{1}{1+e^{-0.10486}} \approx \frac{1}{1+0.9004} \approx 0.5262$$

$$v(x,y) = \lambda_D \cdot 0.5262 = 0.5262, \qquad \ell_{KTO}(x,y) = \lambda_D - v(x,y) = 1 - 0.5262 = 0.4738$$

Gradient weight (hệ số trước $\nabla_\theta \log \pi_\theta(y|x)$, theo công thức 3.6): $d(y)\lambda_D \sigma(\beta z)(1-\sigma(\beta z))\beta = (-1)(1)(0.5262)(0.4738)(0.1) \approx -0.02494$. Dấu âm tức cập nhật gradient descent tương đương **tăng** $\log \pi_\theta(y|x)$ với hệ số ≈ 0.0249, gần trọng số tối đa lý thuyết $\sigma(1-\sigma)\beta \le 0.25 \times 0.1 = 0.025$ (đạt tại $\beta z = 0$). Nếu $\pi_\theta(y|x)$ tăng mạnh hơn nữa, z lớn hơn, trọng số giảm dần về 0 đúng theo Proposition 4.1.



Hình 62: Loss KTO cho ví dụ desirable và undesirable (trái) và trọng số gradient bão hoà theo z (phải), với beta 0,1 (minh hoạ, tự tính)

Cách đọc hình: z là chênh lệch giữa reward ngầm của câu trả lời và điểm tham chiếu z0. Loss của ví dụ desirable giảm khi z tăng và của ví dụ undesirable thì ngược lại, cả hai đều phẳng ở hai đầu. Trọng số gradient đạt cực đại 0,025 tại z bằng 0 và tiến về 0 khi z lớn theo cả hai phía, nên các ví dụ quá dễ hoặc quá khó (kể cả nhãn nhiễu) bị hạ trọng số. Điểm đánh dấu là ví dụ số ở mục 3.8.

4. Thí nghiệm và kết quả chính

Thiết lập: Pythia-{1.4B, 2.8B, 6.9B, 12B} và Llama-{7B, 13B, 30B}; dữ liệu Anthropic-HH, OpenAssistant, SHP (dạng preference, chuyển thành desirable/undesirable bằng cách coi y_w desirable, y_l undesirable); đánh giá bằng GPT-4-0613 so với baseline "khó đánh bại": chính output từng dùng làm target SFT.

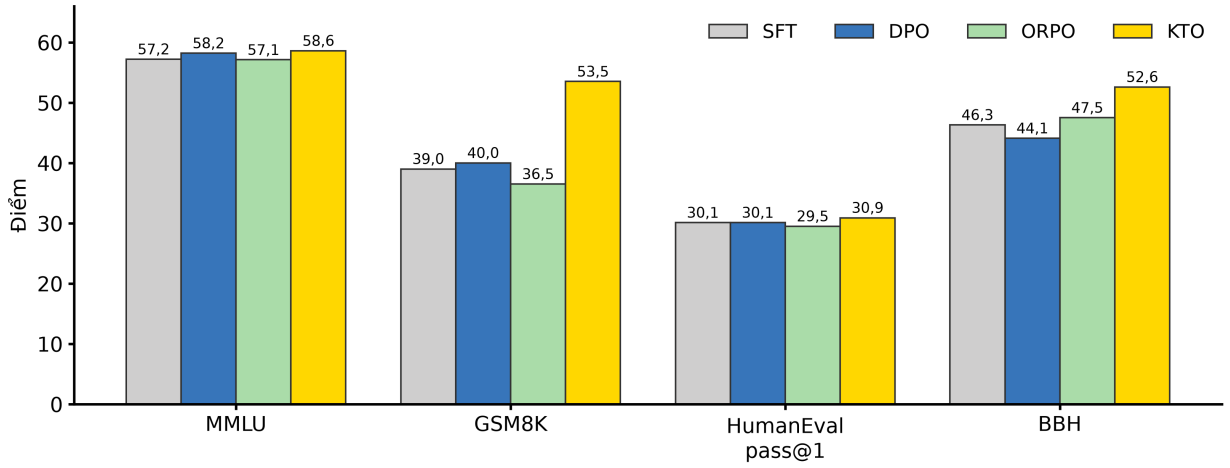
HALO vs non-HALO (Hình 2): HALO (DPO, PPO offline dummy reward ± 1) khớp hoặc vượt non-HALO (SLiC, CSFT) ở mọi quy mô; khác biệt có ý nghĩa ($p < 0.05$, hiệu chỉnh Holm) chỉ từ 13B+; chỉ Llama-{13B,30B} align bằng HALO đạt winrate $\geq 50\%$ so với target SFT.

KTO $\geq$ DPO (Hình 3): ở mọi quy mô 1B-30B, KTO khớp hoặc vượt DPO; với Llama, KTO một mình đã khớp hiệu năng SFT+DPO, vượt DPO đơn lẻ có ý nghĩa ($p < 0.01$ ở 7B và 30B); ở Pythia không có khác biệt có ý nghĩa, gợi ý cần năng lực model tối thiểu.

Bảng kết quả benchmark (Zephyr-β-SFT, align trên UltraFeedback 1 epoch, Bảng 2 rút gọn):

Phương pháp	MMLU	GSM8K	HumanEval pass@1	BBH
SFT	57.2	39.0	30.1	46.3
DPO	58.2	40.0	30.1	44.1
ORPO ($\lambda = 0.1$)	57.1	36.5	29.5	47.5
KTO ($\beta = 0.1, \lambda_D = 1$)	**58.6**	**53.5**	**30.9**	**52.6**

Đổi DPO sang KTO (cùng dữ liệu UltraFeedback) cải thiện GSM8K **13.5 điểm**. Ngay cả khi KTO chỉ thấy một trong hai output mỗi preference (one-y-per-x, giảm 50% dữ liệu), nó vẫn vượt DPO.



Hình 63: SFT, DPO, ORPO và KTO trên MMLU, GSM8K, HumanEval và BBH sau khi căn chỉnh trên UltraFeedback (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi cụm cột là một benchmark, bốn cột là bốn phương pháp xuất phát từ cùng checkpoint SFT. KTO dẫn đầu ở cả bốn benchmark, khoảng cách lớn nhất ở GSM8K (53,5 so với 40,0 của DPO). MMLU và HumanEval gần như không đổi giữa các phương pháp.

Mistral-7B trên OpenAssistant (Bảng 3, winrate vs SFT target, CI 90%): chưa align 0.525±0.037; +DPO 0.600±0.037; +KTO (toàn bộ y) 0.652±0.036; +KTO (one-y-per-x, giảm 72% dữ liệu) 0.631±0.036; Mistral-7B-Instruct chính thức 0.621±0.031. KTO one-y-per-x vẫn vượt cả DPO và bản instruct chính thức.

Chịu mất cân bằng dữ liệu (Hình 5): trên Llama-7B, bỏ tới 90% ví dụ desirable (tỉ lệ 1:1 xuống 1:10, $\lambda_U = 1, \lambda_D = 13.33$ theo công thức 9) vẫn giữ KTO vượt DPO.

Không cần SFT ở quy mô lớn: Llama-{13B,30B} align bằng KTO không qua SFT vẫn cạnh tranh với SFT+KTO, phương pháp align duy nhất thể hiện điều này; DPO không qua SFT khiến model lan man, bịa cả đoạn hội thoại (Hình 4), KTO vẫn giữ độ dài phản hồi ổn định.

5. Phân tích / ablation

- **Bỏ z_0:** giảm 3.6 điểm BBH và 4.0 điểm GSM8K.
- **Bỏ tính đối xứng của hàm giá trị** (đổi $1-\sigma(\cdot)$ phía undesirable thành $-\log\sigma(\cdot)$, tức lõm mọi nơi như DPO): giảm 9.4 điểm BBH, 11.0 điểm GSM8K.
- **Risk-neutral** (hàm giá trị = identity, bỏ tính bão hoà): BBH sụp hoàn toàn (6.1 so với 46.3+ của các biến thể khác).
- **Không cần** π_{ref} (giả định π_{ref} uniform, rút gọn $r_\theta - z_0$ thành $\log\pi_\theta(y|x) - H(\pi_\theta)$, $\lambda_D = 1.75$): tốt hơn DPO ở một số benchmark, kém ở số khác, vẫn thua KTO chuẩn; nhạy hơn với λ_D, λ_U; vẫn vượt ORPO (baseline reference-free khác).
- **Tỉ lệ loss aversion đề xuất (công thức 9):** $\frac{\lambda_D n_D}{\lambda_U n_U} \in [1, \frac{3}{2}]$ với n_D, n_U là số ví dụ desirable/undesirable; ưu tiên gain sensitivity hơn loss sensitivity cho hầu hết benchmark (trừ bài toán cần giảm downside như chống độc hại, nên đảo lại).
- **Vì sao KTO $\geq$ DPO trên cùng dữ liệu (Section 4.4):** (1) **Proposition 4.1**: gradient bão hoà về 0 cho ví dụ có reward quá cực đoan, nên KTO tự động bỏ qua dữ liệu nhiễu/gán nhãn sai, phổ biến ở feedback người thật. (2) **Theorem 4.2**: hai reward $r^*_a, r^*_b = r^*_a + h(x)$ cùng equivalence class (cùng policy tối ưu, cùng phân phối Bradley-Terry) nhưng khi giữ z_0 cố định lại tạo ra phân phối giá trị prospect khác nhau; preference log-likelihood không xác định được giá trị prospect, giải thích vì sao khoảng cách KTO-DPO lớn hơn trong đánh giá người so với đánh giá tự động. (3) **Theorem 4.3** (chứng minh Appendix B): với 2 output $\{y_a, y_b\}$, preference mâu thuẫn tỉ lệ $p \in (0.5, 1)$ cho $y_a \succ y_b$ và $1-p$ cho $y_b \succ y_a$. Nếu $p^{1/\beta}\pi_{ref}(y_a|x) < (1-p)^{1/\beta}\pi_{ref}(y_b|x)$, policy DPO tối ưu gán xác suất cao hơn cho y_b (thiểu số), dù $r_\theta(x, y_a) > r_\theta(x, y_b)$; cập nhật KTO loss-neutral ($\lambda_D = \lambda_U$) thì luôn tăng xác suất tương đối của y_a (đa số). Lập luận worst-case này cho thấy KTO bền hơn với dữ liệu **intransitive**.

6. Giới hạn

Tác giả nêu:

- Hàm giá trị Kahneman-Tversky gốc được ước lượng từ canh bạc tiền bạc, gần như chắc chắn khác với cách con người cảm nhận giá trị văn bản; chưa rõ hàm giá trị/điểm tham chiếu nào mô tả tốt nhất việc cảm nhận ngôn ngữ.
- Việc gán $y_w \to$ desirable, $y_l \to$ undesirable là "giả định ngây thơ, vì đơn giản"; phân rã preference phức tạp hơn có thể cho kết quả tốt hơn, để lại cho tương lai.
- $\hat{z}_0$ là ước lượng thiên lệch; $\beta, \lambda_D, \lambda_U$ cần tinh chỉnh theo tác vụ, learning rate tối ưu (2-10 lần DPO) cũng khác biệt.
- Với dữ liệu preference ít nhiễu/ít intransitive, DPO vẫn có thể tốt hơn (nguy cơ KTO underfit); giảm nhẹ bằng β nhỏ hơn và train nhiều epoch hơn.
- Không giải quyết công bằng (fairness) khi dữ liệu mâu thuẫn: KTO nghiêng về nhãn đa số, không phù hợp với nhiều lý thuyết công bằng (ví dụ Rawlsian).

Nhận xét:

- Theorem 4.3 chỉ là lập luận worst-case cho **2 output**, chưa định lượng tỉ lệ nhiễu/intransitive thực tế trong HH/SHP/OASST để kiểm chứng trực tiếp giả thuyết "vì sao KTO thắng DPO trên cùng dữ liệu"; biện luận hợp lý nhưng chưa đo thực nghiệm.
- Bảng ablation (mục 5) cho thấy biến động điểm số lớn (lệch hơn 10 điểm BBH/GSM8K) chỉ từ thay đổi nhỏ hàm giá trị hoặc λ_D, cho thấy KTO khá nhạy hyperparameter hơn những gì mục 4.2 ngụ ý.

- Kết quả "chịu được bỏ 90% dữ liệu desirable" (Hình 5) chỉ đo trên Llama-7B/OpenAssistant, chưa rõ giữ được ở mô hình lớn hơn hay bộ dữ liệu khác.
- So sánh KTO vs DPO dùng optimizer mặc định của DPO (RMSProp) để công bằng, trong khi tác giả khuyến nghị KTO dùng AdamW 5e-6 trong thực tế; các con số winrate trong bài có thể thấp hơn khả năng thực sự của KTO khi dùng đúng hyperparameter khuyến nghị.

7. Liên hệ với các bài khác

- **Bài 29 (DPO):** KTO dùng lại nghiệm KL-constrained và Lemma equivalence class của DPO để chứng minh Theorem 4.2; ngược lại, KTO cho thấy **DPO chính là một HALO** (Theorem 3.5), một cách diễn giải lý thuyết mới cho thành công của DPO mà bài 29 không đề cập.
- **Bài 30 (IPO):** cả hai bài chỉ ra điểm yếu của DPO/BT nhưng từ hai góc khác nhau: IPO lo ngại overfit khi preference gần xác định (Ψ không bị chặn); KTO lo ngại nhạy cảm với nhiễu và intransitivity (Theorem 4.3), khắc phục bằng hàm giá trị bão hoà (Proposition 4.1) chứ không đổi Ψ. Hai cơ chế bổ trợ nhau khi đánh giá độ bền của DPO.
- **Bài 32 (SimPO):** KTO và ORPO (baseline ở Bảng 2) đều hướng tới bỏ preference ghép cặp hoặc bỏ mô hình tham chiếu để giảm chi phí; SimPO đi theo hướng bỏ π_{ref} bằng chuẩn hoá độ dài, khác với KTO (vẫn giữ π_{ref} ở bản chuẩn).
- **Bài 33 (DPO vs PPO):** cùng phân tích hạn chế lý thuyết của DPO, nhưng bài 33 tập trung exploitation OOD response, còn KTO tập trung độ bền với nhiễu/mâu thuẫn trong nhãn.

8. Ứng dụng thực tế (suy luận)

Mức liên quan: **cao**, đặc biệt khi nguồn dữ liệu feedback thực tế là nhị phân.

- **Khi phản hồi tự nhiên là nhị phân** (like/dislike, accept/reject) thay vì preference ghép cặp, dùng KTO trực tiếp tiết kiệm chi phí thu thập dữ liệu so với việc ép tạo cặp so sánh.
- **Mất cân bằng dữ liệu:** điều chỉnh λ_D, λ_U theo công thức 9 ($\lambda_D n_D / (\lambda_U n_U) \in [1, 1.5]$) thay vì để mất cân bằng ảnh hưởng ngầm tới loss.
- **Cài đặt:** cần microbatch size ≥ 2 để ước lượng $\hat{z}_0$ qua cặp lệch; nếu SFT chạy trước trên cùng tập desirable, có thể đặt $\hat{z}_0 = 0$ để giảm một forward pass.
- **Khi nào vẫn nên dùng DPO:** nếu dữ liệu preference sẵn có, sạch, ít mâu thuẫn, tác giả khuyến nghị DPO (mục 4.5); KTO đáng dùng hơn khi nghi ngờ dữ liệu nhiễu/intransitive hoặc muốn mở rộng khối lượng bằng phản hồi nhị phân rẻ hơn.

9. Thí nghiệm gợi ý

1. Lập trình lại ví dụ 3.8: quét $r_\theta - z_0$ từ -5 đến 5, vẽ trọng số gradient, xác nhận hình dạng bão hoà hai phía của Proposition 4.1.
2. Cài $\hat{z}_0$ bằng kỹ thuật cặp lệch trên batch nhỏ (m=4-8), so sánh với KL ước lượng bằng sampling đầy đủ, đo độ lệch thực nghiệm.
3. Tạo dữ liệu tổng hợp 2 output $\{y_a, y_b\}$ với tỉ lệ preference mâu thuẫn p kiểm soát được, kiểm chứng dự đoán của Theorem 4.3.
4. Trên tập accept/reject tự thu thập mất cân bằng (ví dụ 1:5), quét λ_D theo công thức 9, xem khoảng $[1, 1.5]$ có tối ưu trên bài toán của mình không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Hàm giá trị Kahneman-Tversky	$(z - z_0)^\alpha$ nếu $z \geq z_0$, $-\lambda(z_0 - z)^\alpha$ nếu $z < z_0$	$\alpha = 0.88, \lambda = 2.25$ (median thực nghiệm)
HALO	$\mathbb{E}[a_{x,y} v_{x,y}(r_\theta - \mathbb{E}_Q[r_\theta])] + C_D$	DPO, PPO-Clip thoả; CSFT, SLiC không
Reward ngầm định	$r_\theta(x, y) = l(y) \log \frac{\pi_\theta(y\|x)}{\pi_{ref}(y\|x)}$	$l = \beta$ cho DPO, $l = 1$ cho KTO
KTO loss	$\mathbb{E}_{x,y}[\lambda_y - v(x, y)]$	$v = \lambda_D \sigma(\beta(r_\theta - z_0))$ hoặc $\lambda_U \sigma(\beta(z_0 - r_\theta))$
Điểm tham chiếu	$z_0 = KL(\pi_\theta \| \pi_{ref})$	ước lượng $\hat{z}_0$ qua cặp lệch trong microbatch
Gradient KTO	$d(y)\lambda_y \sigma(\beta z)(1 - \sigma(\beta z))\beta \nabla_\theta \log \pi_\theta(y\|x)$	bão hoà về 0 khi $z \to \pm\infty$ (Proposition 4.1)
Tỉ lệ loss aversion	$\lambda_D n_D / (\lambda_U n_U) \in [1, 1.5]$	bù mất cân bằng desirable:undesirable
Theorem 4.3	$p^{1/\beta} \pi_{ref}(y_a) < (1 - p)^{1/\beta} \pi_{ref}(y_b) \Rightarrow$ DPO lệch về thiểu số	KTO loss-neutral luôn nghiêng về đa số

11. Kết luận ngắn

Bài này dùng lý thuyết prospect để chỉ ra rằng các loss alignment phổ biến (DPO, PPO-Clip) đã ngầm định một hàm giá trị kiểu con người (HALO), giải thích một phần vì sao chúng hoạt động tốt hơn cross-entropy thường. Từ đó, KTO được xây trực tiếp từ hàm giá trị Kahneman-Tversky, dùng nhãn nhị phân desirable/undesirable không ghép cặp, điểm tham chiếu là KL-divergence ước lượng qua cặp lệch trong batch, và gradient bão hoà giúp tự động hạ trọng số các ví dụ quá dễ/quá khó (nhiễu). Thực nghiệm trên Pythia và Llama (1B-30B) cho thấy KTO khớp hoặc vượt DPO ở mọi quy mô, chịu được mất cân bằng dữ liệu tới 90%, và ở quy mô lớn không cần bước SFT trước. Tuy nhiên tác giả cũng nhấn mạnh không có HALO nào vượt trội tuyệt đối: lựa chọn phụ thuộc vào đặc điểm nhiễu và tính nhất quán của dữ liệu phản hồi trong từng bài toán cụ thể.

Link gốc: https://arxiv.org/abs/2402.01306

Bài 32

SimPO: Simple Preference Optimization with a Reference-Free Reward (2024)

Tác giả: Yu Meng (University of Virginia), Mengzhou Xia, Danqi Chen (Princeton University). **Venue / arXiv:** NeurIPS 2024; arXiv:2405.14734 (v3, 1/11/2024). **Nguồn:** https://arxiv.org/abs/2405.14734 **Chủ đề:** thay reward ngầm định của DPO (tỉ lệ log xác suất policy/reference) bằng average log-likelihood theo độ dài (length-normalized), bỏ hẳn mô hình tham chiếu, cộng thêm target margin γ vào mô hình Bradley-Terry; đơn giản hơn DPO, rẻ hơn, và vượt DPO trên nhiều benchmark chat. **Vị trí trong lộ trình:** đọc sau bài 29 (DPO), 30 (IPO), 31 (KTO); SimPO so sánh trực tiếp với cả ba cùng CPO, ORPO, RRHF, SLiC-HF, R-DPO. Cuối giai đoạn 8, trước bài 33 (DPO vs PPO).
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

DPO (bài 29) tham số hoá lại reward bằng nghiệm đóng của mục tiêu KL-constrained:

$$r(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

Nhưng khi **sinh văn bản** (decoding, beam search, so sánh multiple-choice), model không hề dùng π_{ref}: tiêu chí thực tế guide việc sinh gần với **average log-likelihood** $p_\theta(y|x) = \frac{1}{|y|}\log \pi_\theta(y|x)$. Vậy có một **khoảng lệch (discrepancy)** giữa reward dùng để huấn luyện (phụ thuộc tỉ lệ với π_{ref}) và metric dùng để sinh (không phụ thuộc π_{ref}). Thực nghiệm đo trên tập train UltraFeedback: trong số các cặp có $r_\theta(x,y_w) > r_\theta(x,y_l)$ theo DPO, có tới **gần một nửa** lại có $p_\theta(y_w|x) < p_\theta(y_l|x)$, tức ranking theo reward train và ranking theo likelihood sinh **không khớp nhau**.

2. Ý tưởng chính

Sửa đồng thời hai điều: (1) đổi hẳn reward ngầm định thành **chính average log-likelihood** (nhân β), loại bỏ khoảng lệch trên, và đồng thời **không cần mô hình tham chiếu** nữa (tiết kiệm bộ nhớ, tính toán); (2) thêm **target reward margin** γ vào mô hình Bradley-Terry, ép $r(x,y_w) - r(x,y_l)$ phải vượt quá γ (không chỉ > 0), giống vai trò margin trong các bộ phân loại biên lớn (SVM), cải thiện khả năng tổng quát hoá. Kết quả: **SimPO loss**, đơn giản hơn DPO (không forward pass qua π_{ref}) nhưng vượt DPO và các biến thể trên nhiều benchmark chat.

3. Phương pháp chi tiết

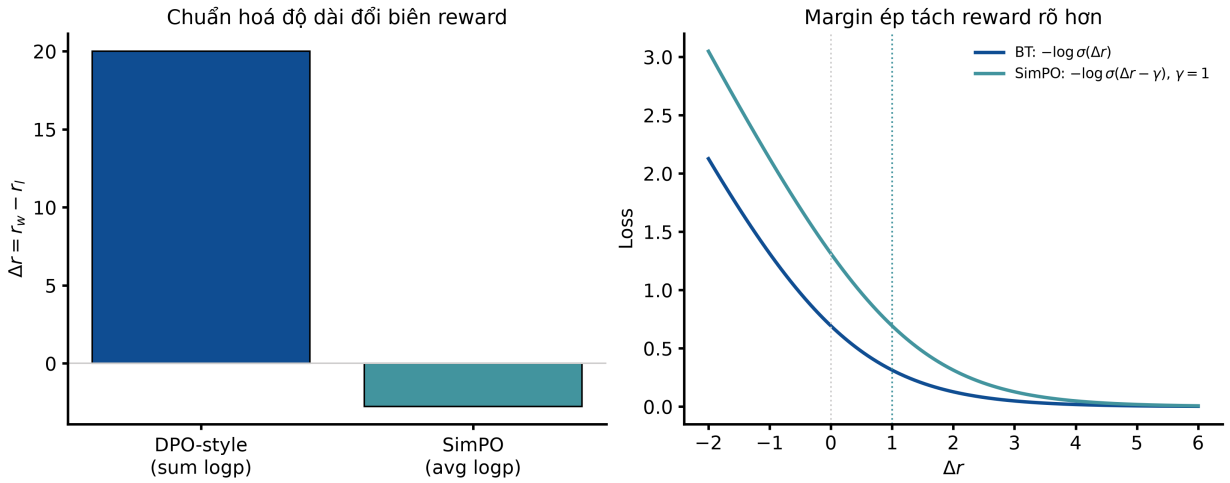
3.1 Nhắc lại DPO (bài 29)

$$\mathcal{L}_{DPO}(\pi_\theta;\pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right]$$

3.2 Reward length-normalized, không cần reference model (Section 2.2)

Một lựa chọn là dùng tổng log-prob $\log \pi_\theta(y|x)$ làm reward, nhưng tổng log-prob **thiên lệch theo độ dài** (sequence dài hơn có tổng log-prob nhỏ hơn một cách tự nhiên). Nếu y_w dài hơn y_l, tối ưu tổng log-prob buộc

model "thối phồng" xác suất của sequence dài để đạt reward cao hơn, dễ gây suy biến (degeneration). Giải pháp: dùng **average log-likelihood** (công thức 3, cũng là metric thường dùng để rank trong beam search và multiple-choice):



SimPO: avg-logprob (length-norm) và margin γ trong BT so với DPO-style sum (minh hoạ, tự tính)

$$p_\theta(y|x) = \frac{1}{|y|}\log \pi_\theta(y|x) = \frac{1}{|y|}\sum_{i=1}^{|y|}\log \pi_\theta(y_i|x, y_{<i})$$

Reward length-normalized, tham chiếu-tự do (reference-free) (công thức 4):

$$\boxed{r_{SimPO}(x,y) = \frac{\beta}{|y|}\log \pi_\theta(y|x) = \frac{\beta}{|y|}\sum_{i=1}^{|y|}\log \pi_\theta(y_i|x, y_{<i})}$$

Chuẩn hoá theo độ dài là **thiết yếu** (xem mục 5): bỏ nó dẫn đến thiên lệch tạo sequence dài hơn nhưng chất lượng kém hơn. Vì reward không còn phụ thuộc π_{ref}, SimPO loại bỏ hoàn toàn nhu cầu giữ mô hình tham chiếu trong bộ nhớ khi huấn luyện.

3.3 Target reward margin (Section 2.3, công thức 5)

$$p(y_w \succ y_l|x) = \sigma(r(x,y_w) - r(x,y_l) - \gamma), \qquad \gamma > 0$$

γ ép khoảng cách reward giữa thắng/thua tối thiểu bằng γ (gọi là "home advantage" trong mô hình Bradley-Terry cổ điển). Thực nghiệm (mục 5): tăng γ ban đầu cải thiện chất lượng sinh, nhưng γ quá lớn làm giảm chất lượng (xem phân tích trade-off ở mục 5). IPO (bài 30) cũng có một dạng margin tương tự (τ^{-1} cố định) nhưng cả objective IPO kém hiệu quả hơn SimPO trong thực nghiệm (mục 4).

3.4 SimPO objective (công thức 6)

Thế (3.2) vào (3.3):

$$\boxed{\mathcal{L}_{SimPO}(\pi_\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log\pi_\theta(y_l|x) - \gamma\right)\right]}$$

Không có ràng buộc KL rõ ràng với π_{ref}, nhưng tác giả quan sát thực nghiệm KL vẫn nhỏ nhờ ba yếu tố: learning rate nhỏ, dữ liệu preference đa dạng miền/tác vụ, và tính bền vững nội tại của LLM khi học dữ liệu

mới mà không quên kiến thức cũ (xem mục 5).

3.5 Phân tích gradient (Appendix F, công thức 7)

$$\nabla_\theta \mathcal{L}_{SimPO}(\pi_\theta) = -\beta\, \mathbb{E}_{(x,y_w,y_l)}\Big[s_\theta \cdot \Big(\frac{1}{|y_w|}\nabla_\theta \log \pi_\theta(y_w|x) - \frac{1}{|y_l|}\nabla_\theta \log \pi_\theta(y_l|x)\Big)\Big]$$

$$\nabla_\theta \mathcal{L}_{DPO}(\pi_\theta) = -\beta\, \mathbb{E}_{(x,y_w,y_l)}[d_\theta \cdot (\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x))]$$

với trọng số gradient $s_\theta = \sigma(\frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) + \gamma)$ và $d_\theta = \sigma(\beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} - \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)})$. Hai khác biệt chính: (1) s_θ **không cần** π_{ref}, diễn giải trực tiếp: trọng số lớn hơn cho các ví dụ model hiện tại gán nhầm likelihood cao hơn cho y_l so với y_w; (2) gradient của SimPO trên y_w, y_l được **chuẩn hoá theo độ dài** ($\frac{1}{|y_w|}, \frac{1}{|y_l|}$), còn DPO thì không, nên **DPO cập nhật mạnh hơn cho sequence dài hơn** dù không nên, khớp với quan sát thực nghiệm DPO dễ bị length exploitation hơn SimPO.

3.6 Các baseline cùng dạng (Table 3, tóm lược)

Phương pháp	Đặc điểm chính
RRHF, SLiC-HF	Ranking loss (max-margin) + loss SFT phụ trợ trên y_w
IPO (bài 30)	Margin cố định τ^{-1}, vẫn cần π_{ref}
CPO	Dùng sequence likelihood làm reward + train cùng SFT loss, không cần π_{ref}
KTO (bài 31)	Học từ nhãn desirable/undesirable không ghép cặp
ORPO	Log odds ratio trực tiếp trên π_θ + loss SFT, không cần π_{ref}
R-DPO	DPO cộng thêm số hạng regularize chống length exploitation
SimPO	Reward = avg log-likelihood (không π_{ref}) + margin γ, không cần loss SFT phụ trợ

3.7 Thuật toán (tổng hợp)

```
Input: D = {(x, yw, yl)}, beta, gamma
1. Lay minibatch (x, yw, yl)
2. Tinh avg log-likelihood: p_w = (1/|yw|) * log pi_theta(yw|x), p_l = (1/|yl|) * log
↪  pi_theta(yl|x)
3. logit = beta * p_w - beta * p_l - gamma
4. loss = mean( -log sigmoid(logit) )
5. Cap nhat theta bang gradient descent tren loss (KHONG can forward pass qua pi_ref)
```

3.8 Ví dụ minh hoạ bằng số (tự tính)

Xét x cố định, $\beta = 2.0$, $\gamma = 0.5$ (trong khoảng được bài báo đề xuất $\beta \in [2.0, 2.5]$, $\gamma \in [0.5, 1.5]$, mục 4). Giả sử y_w có 2 token với log-prob $(-0.5, -0.3)$, y_l có 3 token với log-prob $(-0.6, -0.7, -0.5)$:

$$p_\theta(y_w|x) = \frac{-0.5 - 0.3}{2} = -0.4, \qquad p_\theta(y_l|x) = \frac{-0.6 - 0.7 - 0.5}{3} = -0.6$$

$$r(x, y_w) = \beta \cdot (-0.4) = -0.8, \qquad r(x, y_l) = \beta \cdot (-0.6) = -1.2$$

$$\text{logit} = r(x, y_w) - r(x, y_l) - \gamma = -0.8 - (-1.2) - 0.5 = -0.1$$

$$\mathcal{L}_{SimPO} = -\log \sigma(-0.1) = \log(1 + e^{0.1}) \approx \log(2.1052) \approx 0.7444$$

Trọng số gradient: $s_\theta = \sigma(r(x, y_l) - r(x, y_w) + \gamma) = \sigma(-1.2 - (-0.8) + 0.5) = \sigma(0.1) \approx 0.5250$. Vì logit âm (margin thực tế $0.4 < \gamma = 0.5$, chưa đạt mục tiêu), $s_\theta > 0.5$: cặp này còn nhận trọng số gradient khá lớn, đẩy tiếp $\log \pi_\theta(y_w|x)/|y_w|$ lên và $\log \pi_\theta(y_l|x)/|y_l|$ xuống để đạt margin γ.

4. Thí nghiệm và kết quả chính

Setup: Llama-3-8B và Mistral-7B, mỗi mô hình 2 kiểu: **Base** (SFT trên UltraChat-200k rồi align trên UltraFeedback) và **Instruct** (dùng checkpoint instruct có sẵn làm SFT, dữ liệu preference được regenerate on-policy bằng chính SFT model, chấm bằng PairRM). Hyperparameter tối ưu theo Table 8: Mistral-Base $\beta = 2.0, \gamma = 1.6$; Mistral-Instruct $\beta = 2.5, \gamma = 0.3$; Llama-3-Base $\beta = 2.0, \gamma = 1.0$; Llama-3-Instruct $\beta = 2.5, \gamma = 1.4$ (mỗi thiết lập cần dò γ riêng, không có công thức chung). Đánh giá trên AlpacaEval 2 (LC và raw win rate), Arena-Hard, MT-Bench, giám khảo GPT-4/GPT-4-Turbo.

Kết quả chính (Table 4, rút gọn AlpacaEval2 LC % và Arena-Hard WR %):

Setting	DPO	Baseline tốt nhất khác	**SimPO**
Mistral-Base (AlpacaEval2 LC)	15.1	17.4 (R-DPO)	**21.5**
Mistral-Instruct (AlpacaEval2 LC)	26.8	27.3 (R-DPO)	**32.1**
Llama-3-Base (AlpacaEval2 LC)	18.2	18.2	**22.0**
Llama-3-Instruct (AlpacaEval2 LC)	40.3	41.1 (R-DPO)	**44.7**
Mistral-Base (Arena-Hard WR)	10.4	11.7 (R-DPO)	**16.6**
Llama-3-Instruct (Arena-Hard WR)	32.6	33.1 (R-DPO)	**33.8**

SimPO vượt baseline tốt nhất 3.6-4.8 điểm LC trên AlpacaEval2 ở mọi setting; cải thiện tới 6.4 điểm AlpacaEval2 và 7.5 điểm Arena-Hard so với DPO cụ thể (Hình 1). MT-Bench ít phân biệt được giữa các phương pháp (quy mô đánh giá nhỏ, chấm single-instance).

Leaderboard: mô hình tốt nhất, Gemma-2-9B-it-SimPO (dùng mô hình phần thưởng mạnh hơn, ArmoRM-Llama3-8B, để tạo dữ liệu), đạt **72.4%** LC win rate và 65.9% raw WR trên AlpacaEval2, **59.1%** WR trên Arena-Hard, đứng đầu các mô hình dưới 10B tham số trên Chatbot Arena (bỏ phiếu người dùng thật), leo từ hạng 36 lên hạng 25 so với Gemma-2-9B-it gốc.

5. Phân tích / ablation

Cả hai thiết kế đều cần thiết (Table 5, Mistral-Base/Instruct): bỏ length normalization (w/o LN) gây sụt giảm mạnh nhất (AlpacaEval2 LC Mistral-Base: 21.5 → 11.9), sinh ra văn bản dài, lặp, suy biến. Đặt $\gamma = 0$ cũng giảm hiệu năng (21.5 → 16.8), cho thấy γ mặc định không phải 0 là tối ưu.

Length normalization chống length exploitation: hệ số tương quan Spearman giữa average log-likelihood và độ dài phản hồi trên tập giữ lại: SimPO **0.34** (gần với SFT), DPO **0.59**, SimPO không có LN **0.82**. Không chuẩn hoá độ dài khiến reward tương quan dương mạnh với độ dài, tạo động lực sinh câu dài hơn bất kể chất lượng.

Trade-off của γ: tăng γ làm tăng đều **reward accuracy** (tỉ lệ cặp có $r(y_w) > r(y_l)$ trên tập giữ lại), nhưng win rate AlpacaEval2 tăng rồi **giảm** khi γ quá lớn; γ lớn làm giảm log-likelihood trung bình của câu thắng, có thể dẫn tới suy biến. Đây là trade-off giữa khớp đúng thứ hạng reward và giữ likelihood được hiệu chỉnh tốt, chưa được giải quyết triệt để (để future work).

DPO reward lệch khỏi likelihood sinh: trong số cặp DPO xếp đúng theo reward ($r_\theta(y_w) > r_\theta(y_l)$), **gần một nửa** lại có $p_\theta(y_w) < p_\theta(y_l)$ theo average log-likelihood; SimPO loại bỏ hoàn toàn khoảng lệch này vì reward chính là p_θ (nhân β).

Reward accuracy: SimPO đạt reward accuracy cao hơn DPO ở cả Mistral-Base và Mistral-Instruct (trình bày dạng biểu đồ cột, bài báo không cho số cụ thể).

KL divergence: dù không áp KL penalty tường minh, KL thực nghiệm (đo trên y_w, tập giữ lại) của SimPO vẫn nhỏ; tăng β giảm KL ở cả DPO và SimPO, DPO giảm mạnh hơn khi β lớn. Ở setting Mistral-Base, β nhỏ

hơn đôi khi cho win rate tốt hơn dù KL cao hơn, gợi ý ràng buộc chặt vào mô hình tham chiếu yếu không hẳn có lợi.

Hiệu quả tính toán: trên Llama-3-Base, 8xH100, so với cài đặt DPO thông thường (vanilla, không tách riêng forward pass của π_{ref}), SimPO giảm khoảng **20% thời gian chạy** (ước lượng từ số liệu khoảng 73 phút xuống 60 phút) và khoảng **10% bộ nhớ GPU đỉnh** (khoảng 77GB xuống 69GB), nhờ bỏ hẳn forward pass qua mô hình tham chiếu.

6. Giới hạn

Tác giả nêu (Appendix A):

- Cần phân tích lý thuyết sâu hơn, chặt chẽ hơn để hiểu đầy đủ các yếu tố làm SimPO hiệu quả; γ là hyperparameter thêm vào, hiện phải dò tay, chưa có cách xác định tự động.
- Không xem xét rõ ràng khía cạnh an toàn/trung thực (safety, honesty); dữ liệu UltraFeedback chủ yếu tập trung vào tính hữu ích (helpfulness). Dù vậy, SimPO vẫn đạt hiệu năng TruthfulQA cao so với các mục tiêu khác trong thực nghiệm phụ (Table 9), gợi ý tiềm năng cho alignment an toàn.
- **Giảm hiệu năng trên toán/reasoning nặng** (ví dụ GSM8K): các phương pháp preference optimization nói chung, và SimPO đôi khi cụ thể, có thể làm giảm downstream task performance ở các tác vụ mà chỉ đổi một token là lật nhãn đúng/sai (ví dụ "2+2=4" thành "2+2=5"); đây là hiện tượng đã biết từ nghiên cứu khác, chưa có giải pháp tích hợp sẵn vào SimPO.
- So sánh với PPO/online RL để lại cho nghiên cứu tương lai.

Nhận xét:

- Không có ràng buộc KL rõ ràng khiến SimPO về lý thuyết có nguy cơ reward hacking/suy biến (tác giả tự nhận là "caveat"), dù thực nghiệm với hyperparameter đã tune không thấy sụp đổ.
- γ tối ưu dao động khá rộng theo thiết lập (0.3 đến 1.6, Table 8), không có quy tắc chọn rõ ràng ngoài dò thực nghiệm, mâu thuẫn phần nào với tên gọi "Simple" (giống vấn đề nhiều hyperparameter của KTO ở bài 31, dù SimPO có ít hyperparameter hơn tổng thể vì không cần π_{ref}).
- Các con số reward accuracy (Hình 4c) chỉ trình bày dạng biểu đồ cột, không có bảng số liệu chính xác để trích dẫn trực tiếp.
- Hiệu quả tính toán (20% thời gian, 10% bộ nhớ) được đo trên một cấu hình phần cứng cụ thể (8xH100, Llama-3-Base), có thể không tổng quát hoá đều cho mọi quy mô model/batch size.

7. Liên hệ với các bài khác

- **Bài 29 (DPO):** SimPO sửa trực tiếp phần reward reparameterization của DPO (bỏ π_{ref}, chuẩn hoá độ dài) nhưng vẫn giữ khung Bradley-Terry và cấu trúc loss $-\log\sigma(\cdot)$.
- **Bài 30 (IPO):** IPO dùng margin cố định $\tau^{-1}/2$ trong một loss hồi quy bình phương; SimPO dùng margin γ tunable trong loss logistic. Thực nghiệm của bài này cho thấy objective IPO đầy đủ (vẫn cần π_{ref}) kém hiệu quả hơn SimPO trên AlpacaEval2/Arena-Hard.
- **Bài 31 (KTO):** cả KTO và SimPO đều nhằm giảm chi phí DPO nhưng theo hai hướng khác nhau: KTO bỏ yêu cầu dữ liệu ghép cặp (vẫn giữ π_{ref}), SimPO giữ dữ liệu ghép cặp nhưng bỏ π_{ref}. Bảng so sánh (Table 4) cho thấy SimPO vượt KTO trên cả 4 setting.
- **Bài 33 (DPO vs PPO):** phát hiện "reward train lệch khỏi likelihood sinh" ở SimPO liên quan tới các thảo luận rộng hơn về hạn chế lý thuyết của reward ngầm định DPO mà bài 33 sẽ phân tích từ góc OOD response.

8. Ứng dụng thực tế (suy luận)

Mức liên quan: **cao**, đặc biệt cho ai cần giảm chi phí tính toán/bộ nhớ của quy trình DPO.

- **Khi hạn chế bộ nhớ/compute** (không muốn giữ thêm một bản copy mô hình tham chiếu trong GPU), SimPO là lựa chọn thay thế trực tiếp cho DPO, tiết kiệm khoảng 10-20% thời gian/bộ nhớ theo đo đạc của bài báo.
- **Dò γ cẩn thận theo model/dataset:** không có công thức chung (γ dao động 0.3-1.6 giữa các thiết lập trong bài báo); nên quét một khoảng nhỏ và theo dõi cả reward accuracy và win rate thực tế (tránh chỉ nhìn reward accuracy, vì nó luôn tăng theo γ ngay cả khi chất lượng sinh giảm).
- **Kiểm tra downstream reasoning task riêng** (toán, code) sau khi align bằng bất kỳ phương pháp preference optimization nào bao gồm SimPO, vì hiện tượng giảm hiệu năng GSM8K đã được quan sát.

9. Thí nghiệm gợi ý

1. Lập trình lại ví dụ 3.8, quét γ từ 0 đến 2 với cùng cặp (y_w, y_l), vẽ s_θ và loss để thấy rõ cơ chế margin.
2. Train SimPO và DPO trên một subset nhỏ UltraFeedback, đo hệ số Spearman giữa avg log-likelihood và độ dài phản hồi, kiểm chứng con số 0.34 (SimPO) vs 0.59 (DPO) có tái lập được không.
3. Quét γ trên một thiết lập nhỏ, vẽ đồng thời reward accuracy và win rate thực tế để quan sát trực tiếp trade-off ở mục 5.
4. Đo runtime/memory thực tế của SimPO so với DPO trên phần cứng hiện có, xác nhận mức tiết kiệm có khớp khoảng 10-20% như bài báo báo cáo không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Reward DPO	$\beta \log \frac{\pi_\theta(y\|x)}{\pi_{ref}(y\|x)} + \beta \log Z(x)$	cần π_{ref}, không chuẩn hoá độ dài
Average log-likelihood	$p_\theta(y\|x) = \frac{1}{\|y\|} \log \pi_\theta(y\|x)$	metric decoding dùng để sinh
Reward SimPO	$r_{SimPO}(x,y) = \frac{\beta}{\|y\|} \log \pi_\theta(y\|x)$	reference-free, length-normalized
Target margin	$p(y_w \succ y_l\|x) = \sigma(r(x,y_w) - r(x,y_l) - \gamma)$	$\gamma > 0$, cần dò theo setup
SimPO loss	$-\mathbb{E}[\log \sigma(\frac{\beta}{\|y_w\|} \log \pi_\theta(y_w\|x) - \frac{\beta}{\|y_l\|} \log \pi_\theta(y_l\|x) - \gamma)]$	không cần π_{ref}
Trọng số gradient SimPO	$s_\theta = \sigma(\frac{\beta}{\|y_l\|} \log \pi_\theta(y_l\|x) - \frac{\beta}{\|y_w\|} \log \pi_\theta(y_w\|x) + \gamma)$	không phụ thuộc π_{ref}
Hyperparameter khuyến nghị	$\beta \in [2.0, 2.5]$, $\gamma \in [0.5, 1.5]$	dao động theo model/setup (Table 8)
Hiệu quả	giảm ≈20% runtime, ≈10% GPU memory	so với DPO vani, Llama-3-Base 8xH100

11. Kết luận ngắn

SimPO giải quyết một khoảng lệch cụ thể trong DPO: reward dùng để huấn luyện (dựa trên tỉ lệ log-prob với mô hình tham chiếu) không khớp với metric dùng để sinh văn bản (average log-likelihood thuần). Bằng cách đặt chính average log-likelihood (chuẩn hoá độ dài) làm reward và thêm một target margin γ vào mô hình Bradley-Terry, SimPO loại bỏ hoàn toàn nhu cầu giữ mô hình tham chiếu, đơn giản và rẻ hơn DPO, đồng thời vượt DPO và nhiều biến thể (IPO, KTO, CPO, ORPO, R-DPO) trên AlpacaEval2 và Arena-Hard ở cả bốn setting Mistral/Llama-3, Base/Instruct, mà không khai thác quá mức độ dài phản hồi. Hạn chế chính nằm ở việc thiếu phân tích lý thuyết chặt chẽ, γ phải dò tay theo từng thiết lập, và hiệu năng có thể giảm trên các tác vụ reasoning nặng như GSM8K, một vấn đề chung của các phương pháp preference optimization offline.

Link gốc: https://arxiv.org/abs/2405.14734

Bài 33

Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study (2024)

Tác giả: Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, Yi Wu (Tsinghua University, OpenPsi Inc., Shanghai Qi Zhi Institute). **Venue / arXiv:** arXiv:2404.10719 (v3, 10/10/2024); bài báo không nêu rõ venue chính thức trong phần văn bản đọc được. **Nguồn:** https://arxiv.org/abs/2404.10719 **Chủ đề:** chứng minh lý thuyết DPO có không gian nghiệm rộng hơn PPO (dễ khai thác response ngoài phân phối dữ liệu preference), xác định 3 yếu tố thực nghiệm then chốt giúp PPO vượt DPO (advantage normalization, batch size lớn, EMA cho mô hình tham chiếu), và benchmark PPO vs DPO trên dialogue (HH-RLHF, SafeRLHF) và code generation (APPS, CodeContest). **Vị trí trong lộ trình:** đọc sau bài 29 (DPO), 30 (IPO), 31 (KTO), 32 (SimPO); bài này khép giai đoạn 8 bằng quan điểm đối lập: trong khi bài 30-32 cải tiến các phương pháp offline/reward-free, bài này cho thấy PPO (online, reward-based, bài 00) vẫn vượt trội nếu tune đúng.
Chương nền liên quan: nền 07: RLHF, mô hình thưởng, phạt KL và DPO

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RLHF có hai nhóm phương pháp: **reward-based** (học mô hình phần thưởng rồi dùng actor-critic như PPO để tối ưu, cách ChatGPT và Claude dùng) và **reward-free** (DPO, RRHF, PRO, bỏ hẳn mô hình phần thưởng, học trực tiếp từ dữ liệu preference). Nghịch lý quan sát được: các ứng dụng thực tế thành công nhất dùng PPO, nhưng kết quả SOTA trên benchmark học thuật lại thường đến từ DPO. Bài này đặt hai câu hỏi: (1) DPO có thực sự vượt trội PPO trong RLHF? (2) Hiệu năng PPO có thể cải thiện đáng kể trên các benchmark RLHF phổ biến không?

2. Ý tưởng chính

Về lý thuyết, với một reward thật r^* và preference dataset D cố định, bài báo chứng minh tập các policy mà PPO có thể tạo ra (Π_{PPO}: train mô hình phần thưởng trên D rồi tối ưu bằng PPO) là **tập con thực sự (proper subset)** của tập policy DPO có thể tạo ra (Π_{DPO}: minimize DPO loss trên D). Nghĩa là mọi nghiệm PPO cũng là nghiệm DPO, nhưng DPO còn có **thêm** các nghiệm khác, trong đó có các nghiệm "lệch" đặt xác suất cao cho response **ngoài phân phối (OOD)** của preference data, điều PPO không bao giờ làm vì luôn bị neo vào π_{ref} qua nghiệm đóng của ràng buộc KL. Thực nghiệm (tổng hợp và trên SafeRLHF thật) xác nhận DPO rất nhạy với **distribution shift** giữa mô hình tham chiếu và dữ liệu preference. Đồng thời, bài xác định 3 kỹ thuật PPO ít được chú ý trong RLHF (advantage normalization, batch size lớn, cập nhật π_{ref} bằng EMA) giúp PPO vượt DPO toàn diện trên dialogue và đặc biệt trên code generation khó (APPS, CodeContest), đạt kết quả SOTA.

3. Phương pháp chi tiết

3.1 Nền tảng RLHF (công thức 2-7)

$$J_r(\pi_\theta) = \mathbb{E}_{x\sim p_{data}, y\sim\pi_\theta}\left[r(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}\right]$$

PPO: học mô hình phần thưởng Bradley-Terry $P_\phi(y_w \succ y_l|x) = \sigma(r_\phi(x,y_w) - r_\phi(x,y_l))$ bằng cực tiểu $\mathcal{L}_R(r_\phi) = -\mathbb{E}_D[\log\sigma(r_\phi(x,y_w) - r_\phi(x,y_l))]$, rồi tối ưu J_{r_ϕ} bằng PPO.

DPO: nghiệm đóng $\pi^*(y|x) = \frac{1}{Z(x)}\pi_{ref}(y|x)\exp(\frac{1}{\beta}r(x,y))$ (công thức 5) suy ra reward ngầm định $r_\phi(x,y) = \beta\log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} + C(x)$ (công thức 6, giống bài 29), dẫn tới DPO loss:

$$\mathcal{L}_{DPO}(\pi_\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\log\sigma\Big(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\Big)\Big]$$

3.2 Theorem 4.1: Π_{PPO} là tập con thực sự của Π_{DPO}

Phần $\Pi_{PPO} \subseteq \Pi_{DPO}$: định nghĩa song ánh $f(\pi)(x,y) = \beta\log\frac{\pi(y|x)}{\pi_{ref}(y|x)}$ (bỏ qua $C(x)$). Vì $\min_\pi \mathcal{L}_{DPO}(\pi) = \min_\pi \mathcal{L}_R(f(\pi)) \geq \min_r \mathcal{L}_R(r)$, và với r^* là minimizer của $\mathcal{L}_R$, dựng π^* từ công thức 5 thì $\mathcal{L}_{DPO}(\pi^*) = \mathcal{L}_R(r^*) = \min_r \mathcal{L}_R(r)$, nên $\min_\pi \mathcal{L}_{DPO}(\pi) \leq \min_r \mathcal{L}_R(r)$. Hai giá trị min bằng nhau, vậy với mọi $\pi_{PPO} \in \Pi_{PPO}$ (tạo từ một r_ϕ đạt min $\mathcal{L}_R$), thế π_{PPO} (công thức 8, cùng dạng công thức 5) vào $\mathcal{L}_{DPO}$ cũng đạt đúng giá trị min, nên $\pi_{PPO} \in \Pi_{DPO}$.

Phần tập con thực sự (phản ví dụ, Table 1): 3 hành động $\{y_1, y_2, y_3\}$ (bài toán stateless), $\pi_{ref} = (0.5, 0.5, 0)$ (không bao giờ sinh y_3). Preference dataset chỉ có **một** cặp: $y_w = y_1, y_l = y_2$ (không nói gì về y_3). Vì $\pi_{ref}(y_1) = \pi_{ref}(y_2) = 0.5$ bằng nhau, đặt $a = \pi_{DPO}(y_1), b = \pi_{DPO}(y_2)$, DPO loss rút gọn (suy luận lại từ phát biểu "minimized as long as $b = 0$" của bài báo, vì bản trích xuất text bị lỗi OCR ở công thức phân số):

$$\mathcal{L}_{DPO} = \log\left(1 + (b/a)^\beta\right)$$

Loss này đạt **cực tiểu toàn cục (0) bất cứ khi nào $b = 0$, bất kể a bằng bao nhiêu**. Một nghiệm cực tiểu hợp lệ của DPO: $\pi_{DPO} = (0.1, 0.0, 0.9)$, đặt **90% khối lượng xác suất vào** y_3, một hành động hoàn toàn không xuất hiện trong preference data và có $\pi_{ref}(y_3) = 0$. PPO **không thể** tạo ra nghiệm này: theo công thức 8, $\pi_{PPO}(y|x) \propto \pi_{ref}(y|x)\exp(r^*(x,y)/\beta)$, vì $\pi_{ref}(y_3) = 0$ nên $\pi_{PPO}(y_3) = 0$ luôn, bất kể reward học được cho y_3 là gì.

Cơ chế gốc: DPO loss không đặt ràng buộc nào lên các response không xuất hiện (hoặc xuất hiện ít) trong preference data, nên có thể tự do gán xác suất cao cho chúng miễn không vi phạm các so sánh đã quan sát; PPO luôn neo vào π_{ref} qua cấu trúc nhân của nghiệm đóng, nên không thể.

3.3 Kiểm tra số (tự tính)

Kiểm tra cả 2 policy trong Table 1 đều đạt DPO loss tối thiểu $= 0$:

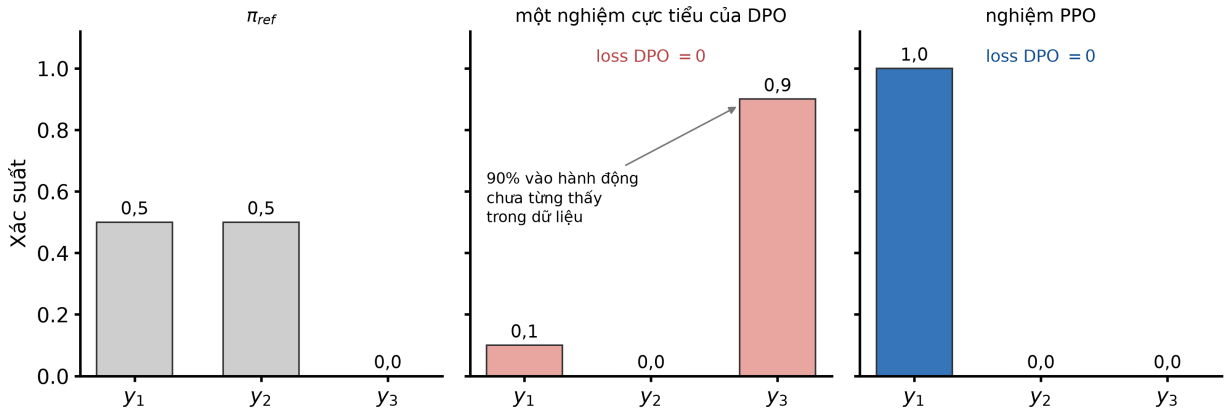
- $\pi_{DPO} = (0.1, 0.0, 0.9)$: $a = 0.1, b = 0 \Rightarrow \mathcal{L}_{DPO} = \log(1 + (0/0.1)^\beta) = \log(1+0) = 0$. ✓
- $\pi_{PPO} = (1, 0, 0)$: $a = 1, b = 0 \Rightarrow \mathcal{L}_{DPO} = \log(1 + (0/1)^\beta) = \log(1+0) = 0$. ✓

Hai policy hoàn toàn khác nhau (một đặt 90% xác suất vào hành động chưa từng xuất hiện trong dữ liệu, một giữ nguyên trong hỗ trợ của π_{ref}) đều đạt **cùng** giá trị DPO loss tối thiểu: minimize DPO loss không đủ để phân biệt và chọn ra nghiệm "an toàn" trong số các nghiệm cực tiểu.

Cách đọc hình: dữ liệu chỉ có một cặp y1 thắng y2, và y3 không xuất hiện trong dữ liệu lẫn trong pi_ref. Cả hai policy ở giữa và bên phải đều có loss DPO bằng 0, nhưng nghiệm ở giữa dồn 0,9 xác suất vào y3. PPO luôn giữ xác suất của y3 bằng 0 vì nghiệm của nó nhân với pi_ref.

3.4 Kiểm chứng bằng mô phỏng tổng hợp (Section 4.2)

Không gian prompt và response rời rạc, mỗi loại kích thước 8; policy và mô hình phần thưởng là MLP nhận one-hot, xuất phân phối categorical; response tối ưu được ấn định theo đường chéo (diagonal); preference dataset random, chỉ phủ một phần các cặp cho mỗi input. Kết quả (Hình 1): cả mô hình phần thưởng học được và policy DPO đều gán giá trị/xác suất cao cho các response OOD (không thuộc phần dataset phủ tới, đánh dấu khoanh tròn); ví dụ cụ thể bài báo nêu: DPO tăng xác suất một response từ 0.11 (theo π_{ref}) lên



Hình 64: Phản ví dụ ba hành động: pi_ref không bao giờ sinh y3, nhưng một nghiệm cực tiểu của DPO đặt 90% xác suất vào y3 còn nghiệm PPO thì không (minh hoạ, tự tính)

0.23 dù response đó không được dataset đề cập. Reward model của PPO cũng bị misspecify tương tự trên các response này, nhưng **KL regularization tường minh** của PPO giữ xác suất thấp trên các response đó và học được policy gần với tối ưu hơn.

3.5 Thực nghiệm thật xác nhận distribution shift (mục 4.3, Table 2, SafeRLHF)

SFT(Alpaca) làm base và reference ban đầu → DPO chỉ đạt an toàn (S.R.) **55.4%**, hữu ích (ΔHelp) **-4.19** (kém), vì Alpaca (dữ liệu SFT) và SafeRLHF (dữ liệu preference) có phân phối khác nhau. Ba cách khắc phục được thử:

1. **SFT thêm trên response an toàn** của SafeRLHF (SFT(Safe)) làm reference mới → DPO: S.R. **71.8%**, Help **-1.62** (cải thiện rõ so với 55.4%/-4.19).
2. **Lọc cặp preference "dual-unsafe"/"dual-safe"** (cả hai response trong cặp cùng nhãn an toàn, gây nhiễu) → lọc dual-unsafe: S.R. **80.8%**, Help **2.46**; lọc thêm cả dual-safe: S.R. **95.8%** nhưng Help giảm về **-2.86** (loại dữ liệu "tốt" làm giảm tính hữu ích).
3. **DPO lặp (DPO-Iter):** dùng SFT(Safe) sinh response mới, mô hình phần thưởng gán nhãn, cập nhật reference mỗi vòng → sau 4 vòng, S.R. đạt **99.9%** (gần bằng PPO), nhưng Help vẫn thấp (**-2.96**) so với PPO (**+1.69** đến **+4.47** tuỳ reference, Table 2/10).

Dù khắc phục được distribution shift, DPO/DPO-Iter vẫn không đạt mức helpfulness của PPO.

3.6 Ba yếu tố then chốt cho PPO (mục 5)

1. **Advantage normalization:** chuẩn hoá advantage (từ GAE) giúp ổn định huấn luyện.
2. **Batch size lớn:** giảm phương sai gradient, đặc biệt quan trọng cho code generation.
3. **EMA cho mô hình tham chiếu:** cập nhật dần π_{ref} theo exponential moving average của actor đang train, thay vì giữ cố định; lý do: actor thay đổi nhanh, nếu π_{ref} đứng yên, KL penalty ràng buộc actor quá chặt về SFT cũ, hại hiệu năng ở các task khó cần thay đổi nhiều.

3.7 Thuật toán (tổng hợp)

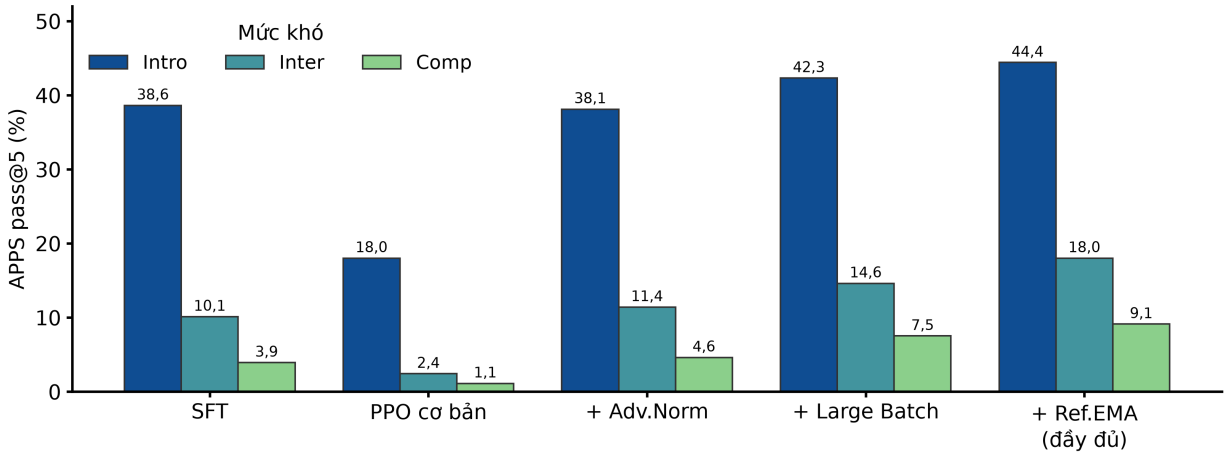
```
Input: SFT model pi_SFT, reward model r_phi, beta, batch size lon
1. pi_theta = pi_SFT, pi_ref = pi_SFT
2. repeat moi iteration:
     Lay batch lon (vd 512) prompt, rollout response tu pi_theta
     Tinh reward r_phi(x,y), advantage A qua GAE, CHUAN HOA advantage (Adv.Norm)
     Cap nhat pi_theta bang PPO-Clip voi KL penalty beta*log(pi_theta/pi_ref)
     Cap nhat pi_ref <- EMA(pi_ref, pi_theta)  (Ref.EMA)
```

4. Thí nghiệm và kết quả chính

4.1 Ablation 3 yếu tố PPO (Table 3, HH-RLHF trên Llama2-7B, APPS/CodeContest trên CodeLlama-34B)

Biến thể	OpenAssistant Reward (HH)	APPS Intro	APPS Inter	APPS Comp
SFT	0.532	38.6%	10.1%	3.9%
PPO cơ bản	0.706	18.0%	2.4%	1.1%
+ Adv.Norm	0.716	38.1%	11.4%	4.6%
+ Large Batch	0.716	42.3%	14.6%	7.5%
+ Ref.EMA (đầy đủ)	**0.718**	**44.4%**	**18.0%**	**9.1%**

PPO cơ bản (batch nhỏ) **tệ hơn SFT** trên APPS; thêm advantage normalization đưa PPO vượt SFT; batch lớn mang lại cải thiện lớn nhất (đặc biệt code); EMA reference cho thêm cải thiện nhất quán ở mọi chỉ số. Hình 2 (quét batch size trên APPS, CodeLlama-13B) cho thấy hiệu năng tăng đơn điệu theo batch size ở mọi mức khó; batch nhỏ (64) khiến PPO cơ bản **tệ hơn cả SFT** (33.7% ở mức introductory).



Hình 65: Ablation ba yếu tố của PPO trên APPS (CodeLlama-34B, pass@5) theo ba mức khó (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: các nhóm cột đi từ SFT, PPO cơ bản rồi thêm lần lượt chuẩn hoá advantage, batch lớn và EMA cho mô hình tham chiếu. PPO cơ bản thấp hơn SFT ở mọi mức khó, chuẩn hoá advantage đưa nó về ngang SFT, rồi batch lớn và Ref.EMA đẩy lên cao hơn rõ rệt. Mức khó Comp tăng từ 3,9 (SFT) lên 9,1 (đầy đủ).

4.2 Benchmark HH-RLHF (Llama2-7B)

OpenAssistant reward: RRHF 0.523, PRO 0.529, DPO 0.611, DPO-Iter 0.678, **PPO 0.718**. Win rate GPT-4 "Tested vs Chosen" (win/tie/lose %): PPO 57/21/22 tốt nhất trong 5 phương pháp; "Tested vs SFT": PPO 58/29/13. So sánh trực tiếp GPT-4: PPO vs DPO 42 win/28 tie/30 lose (nghiêng về PPO); PPO vs DPO-Iter 36/36/28. **Đánh giá người** (Appendix C.4): PPO vs DPO 45 win/26 tie/29 lose; PPO vs DPO-Iter 38/29/33; người đồng thuận với GPT-4 ở mức 60-61%. PPO thắng cả đánh giá GPT-4 và con người.

4.3 SafeRLHF (Llama1-7B, Llama2-7B, Table 6)

Llama2-7B: SFT ΔHelp -2.12, Harm 0.00, S.R. 52.1%; DPO -2.86/-6.82/95.8%; DPO-Iter -2.96/-11.07/99.9%; **PPO +1.69/-12.08/99.5%**. Beaver (model chính thức, Dai et al.) có Harm -6.59, S.R. 89.6%

(không công bố ΔHelp). Cả DPO/PPO đều giảm harm so với SFT, nhưng chỉ PPO giữ được helpfulness **dương** (+1.69 đến +4.47 tuỳ reference), còn DPO/DPO-Iter helpfulness luôn âm.

4.4 Code generation: APPS và CodeContest

APPS, CodeLlama-34B (Table 7, pass@5): SFT 38.6/10.1/3.9% (Intro/Inter/Comp); DPO-Iter 34.2/9.3/3.7% (kém hơn SFT ở mọi mức); **PPO 44.4/18.0/9.1%** (vượt SFT rõ rệt, và cải thiện mạnh hơn khi mô hình lớn hơn, trong khi DPO-Iter không cải thiện theo scale).

CodeContest (Table 8, test set 10@1k): AlphaCode-41B không clustering 15.6%, có clustering 16.4%; CodeLlama-34B: SFT 15.2%, **DPO 0.0%**, DPO-Iter 3.2%, **PPO 22.4%** (vượt AlphaCode-41B, khớp tuyên bố ở abstract: cải thiện 10@1k từ 16.4% lên 22.4%). DPO trên CodeContest cho pass rate **0%** sau 1 epoch; model sinh "nhiều đoạn code vô nghĩa" (meaningless code snippets), thất bại hoàn toàn trên code generation khó.

5. Phân tích / ablation

- **Đổi mô hình tham chiếu (Appendix C.1, Table 9-10):** trên APPS, DPO với reference Codellama-13B-Pretrain (xa dữ liệu preference) chỉ đạt avg pass@5 **0.24%**; đổi sang Codellama-13B-SFT (gần dữ liệu preference hơn) tăng vọt lên **12.8%**. PPO ít nhạy hơn nhiều: 13.8% → 15.1%. Trên SafeRLHF: DPO Help -4.19 → -1.62 khi đổi reference (biên độ lớn), PPO 1.69 → 4.47 (biên độ nhỏ hơn tương đối). DPO nhạy cảm hơn nhiều với distribution shift của mô hình tham chiếu.
- **Quét β (Appendix C.2, Table 11):** β quá lớn (0.2) hại hiệu năng cả hai phương pháp; $\beta = 0.1$ nhất quán tốt trên cả HH-RLHF và APPS cho cả DPO và PPO.
- **Quét độ phủ dữ liệu preference (Appendix C.3, Table 12-13):** huấn luyện trên subset "helpful-base" (hẹp hơn) khiến DPO rơi gần về mức SFT (0.544 so với SFT 0.532, full set 0.615); PPO tụt từ 0.718 xuống 0.602, ít hơn về tỉ lệ tương đối so với DPO. PPO cũng nhạy với lọc dữ liệu (lọc dual-unsafe tăng Help lên 5.88, nhưng lọc tiếp dual-safe khiến model quá bảo thủ, từ chối trả lời để luôn an toàn, Help giảm về -8.04, cùng hiện tượng quan sát ở DPO mục 3.5), nhưng S.R. của PPO luôn giữ trên 92% ở mọi biến thể lọc, ổn định hơn DPO.

6. Giới hạn

Tác giả nêu:

- Chưa đi sâu vào cách train mô hình phần thưởng tốt (dù mô hình phần thưởng quan trọng cho cả PPO và DPO-Iter), để lại cho nghiên cứu tương lai.
- Với bài toán thi code, PPO dùng ground-truth reward (pass/fail từ test case) còn DPO-Iter phải tự nhãn preference bằng kết quả test case; tác giả cho rằng sự khác biệt loại tín hiệu này không ảnh hưởng tới kết luận chung, nhưng đây là một điểm chưa hoàn toàn công bằng giữa 2 phương pháp.

Nhận xét:

- Công thức DPO loss trong phản ví dụ Table 1 bị lỗi OCR khi trích xuất (hiển thị dạng phân số không rõ tử/mẫu); đã suy luận lại dạng đúng $\log(1 + (b/a)^{\beta})$ dựa trên khẳng định "minimized as long as $b = 0$" của chính bài (mục 3.2), nhưng đây là diễn giải lại của phần phân tích, không phải trích dẫn chính xác công thức gốc.
- Bài không so sánh với các cải tiến offline mới hơn (IPO, KTO, SimPO, bài 30-32) mà chỉ so với RRHF, PRO, DPO/DPO-Iter; chưa rõ nếu dùng SimPO (bỏ hẳn π_{ref} khỏi reward) thì Theorem 4.1 (dựa trên công thức 6 kiểu DPO cụ thể) còn áp dụng tương tự không.
- Phần Human Evaluation (Table 14) không nêu rõ số lượng mẫu đánh giá trong văn bản đọc được, chỉ nêu mỗi cặp được 4 người đánh giá và tỉ lệ đồng thuận với GPT-4 (60-61%); bài báo không nêu cỡ mẫu chính xác.

7. Liên hệ với các bài khác

- **Bài 29 (DPO):** Theorem 4.1 dùng trực tiếp công thức 5-7 của DPO; Table 1 là phản ví dụ cho thấy Lemma 1 (reward equivalence class, bài 29) không đảm bảo DPO luôn tìm nghiệm "tốt": equivalence class chỉ chặt khi reward/policy có cùng hỗ trợ với π_{ref}; với response ngoài hỗ trợ đó, lý thuyết tương đương không còn ràng buộc đủ mạnh.
- **Bài 30 (IPO):** cũng cảnh báo DPO "quá tự tin" khi preference gần xác định, nhưng theo cơ chế khác (Ψ không bị chặn trong ΨPO); bài 33 chỉ ra cơ chế khác hẳn (khai thác response ngoài phân phối dữ liệu, không liên quan trực tiếp tới độ xác định của preference).
- **Bài 31 (KTO):** Theorem 4.3 của KTO phân tích dữ liệu preference **mâu thuẫn** (cùng cặp, nhãn ngược nhau); Theorem 4.1 ở đây phân tích response **chưa từng xuất hiện** trong dữ liệu (thiếu phủ, không phải nhiễu nhãn). Hai dạng "thiếu dữ liệu" bổ trợ nhau khi đánh giá độ bền của DPO.
- **Bài 32 (SimPO):** SimPO bỏ hẳn π_{ref} khỏi công thức reward, nên về lý thuyết không còn cơ chế "$\pi_{ref}(y) = 0 \Rightarrow \pi(y) = 0$" mà PPO dựa vào ở bài này; suy luận riêng (paper SimPO không thảo luận) là SimPO có thể dễ bị kiểu khai thác OOD tương tự hoặc hơn DPO, một câu hỏi mở đáng kiểm tra thêm.

8. Ứng dụng thực tế (suy luận)

Mức liên quan: **cao**, đặc biệt khi chọn giữa online RL (PPO) và offline preference optimization (DPO và họ hàng) cho một bài toán alignment cụ thể.

- **Khi preference dataset có độ phủ hẹp** so với không gian response thực tế của model (rất phổ biến với LLM, không gian response gần như vô hạn), nên ưu tiên PPO, hoặc tối thiểu kiểm tra kỹ phân phối response sau train DPO để tránh model "khai thác" response hiếm/chưa từng thấy.
- **Nếu buộc phải dùng DPO:** (1) đảm bảo reference/SFT model gần với phân phối response trong preference data (tinh chỉnh thêm trên preference data trước khi dùng làm reference, như SFT(Safe) ở mục 3.5); (2) xem xét DPO lặp (iterative, tái tạo dữ liệu và relabel bằng mô hình phần thưởng) nếu dữ liệu tĩnh ban đầu lệch phân phối.
- **Nếu đủ hạ tầng train PPO:** áp dụng 3 kỹ thuật advantage normalization, batch size lớn (đặc biệt cho code/reasoning), EMA cho mô hình tham chiếu, thay vì dùng PPO "vani" (dễ cho kết quả kém hơn cả SFT, như Table 3 cho thấy).
- **Với tác vụ có tín hiệu reward "cứng" rõ ràng** (test case, exact match, như code hoặc OCR exact match), ưu tiên mạnh PPO hơn offline preference methods: kết quả CodeContest cho thấy DPO gần như thất bại hoàn toàn (pass rate ~0%) trong khi PPO đạt SOTA.

9. Thí nghiệm gợi ý

1. Lập trình lại phản ví dụ Table 1 (3 hành động), train cả DPO loss và một PPO đơn giản trên cùng $\pi_{ref} = (0.5, 0.5, 0)$, xác nhận DPO có thể hội tụ về nghiệm đặt xác suất vào y_3 còn PPO không thể.
2. Dựng synthetic experiment tương tự mục 3.4 (prompt/response space nhỏ, MLP) với độ phủ preference dataset khác nhau (50%, 25%, 10%), đo mức tăng xác suất OOD của DPO theo độ phủ.
3. Trên một bài toán nhỏ có reward dạng pass/fail rõ ràng (ví dụ unit test đơn giản), so sánh PPO cơ bản vs PPO+3 kỹ thuật (Adv.Norm, batch lớn, Ref.EMA) vs DPO/DPO-Iter, xem thứ hạng có khớp Table 3/7/8 không.
4. Quét batch size cho PPO trên một tác vụ nhỏ, vẽ đường hiệu năng theo batch size, kiểm chứng xu hướng đơn điệu tăng như Hình 2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
$\Pi_{PPO} \subsetneq \Pi_{DPO}$	Theorem 4.1	mọi nghiệm PPO là nghiệm DPO, nhưng không ngược lại
Phản ví dụ OOD	$\mathcal{L}_{DPO} = \log(1 + (b/a)^{\beta})$, cực tiểu khi $b = 0$ bất kể a	DPO có thể đặt xác suất vào action chưa từng thấy
Nghiệm PPO	$\pi_{PPO}(y\|x) \propto \pi_{ref}(y\|x) \exp(r^*/\beta)$	$\pi_{ref}(y) = 0 \Rightarrow \pi_{PPO}(y) = 0$ luôn
3 yếu tố PPO	advantage normalization, batch size lớn, Ref.EMA	Table 3: cải thiện lớn nhất từ batch lớn
Distribution shift	DPO nhạy hơn PPO khi đổi reference model	Table 9-10, biên độ thay đổi của DPO lớn hơn
HH-RLHF	PPO 0.718 reward vs DPO 0.611, DPO-Iter 0.678	Table 4
SafeRLHF	PPO Help dương (+1.69 đến +4.47); DPO/DPO-Iter luôn âm	Table 2, 6, 10
CodeContest	PPO 22.4% (10@1k) vs DPO 0.0%, vượt AlphaCode-41B (16.4%)	Table 8

11. Kết luận ngắn

Bài này chứng minh DPO có không gian nghiệm rộng hơn PPO một cách chặt chẽ (Theorem 4.1): mọi nghiệm PPO cũng là nghiệm DPO, nhưng DPO còn chứa các nghiệm "lệch" đặt xác suất cao cho response ngoài phân phối dữ liệu preference, một rủi ro không thể xảy ra với PPO nhờ ràng buộc KL tường minh qua π_{ref}. Thực nghiệm trên dữ liệu tổng hợp và SafeRLHF thật xác nhận DPO nhạy cảm hơn hẳn với distribution shift giữa mô hình tham chiếu và preference data. Đồng thời, ba kỹ thuật PPO ít được chú ý trong tài liệu RLHF (advantage normalization, batch size lớn, EMA cho mô hình tham chiếu) giúp PPO vượt DPO toàn diện trên cả dialogue (HH-RLHF, SafeRLHF) và đặc biệt trên code generation khó (APPS, CodeContest), đạt SOTA vượt cả AlphaCode-41B trên CodeContest, trong khi DPO gần như thất bại hoàn toàn trên tác vụ này. Kết luận thực tiễn: PPO, nếu được tune đúng, vẫn là lựa chọn mạnh hơn DPO cho các bài toán có dữ liệu preference độ phủ hẹp hoặc cần tín hiệu reward chính xác cao.

Link gốc: https://arxiv.org/abs/2404.10719

Bài 34

STaR: Bootstrapping Reasoning With Reasoning (2022)

Tác giả: Eric Zelikman, Yuhuai Wu, Jesse Mu, Noah D. Goodman (Stanford University; Yuhuai Wu đồng thời tại Google Research).
Venue / arXiv: arXiv:2203.14465 (v2, 20/05/2022). Bài báo không nêu venue trong bản text.
Nguồn: https://arxiv.org/abs/2203.14465
Chủ đề: Dùng chính mô hình ngôn ngữ để tự sinh "rationale" (lời giải từng bước) cho một tập câu hỏi lớn, lọc lại những rationale dẫn tới đáp án đúng, rồi tinh chỉnh (SFT) trên dữ liệu đó, lặp lại nhiều vòng; thêm kỹ thuật "rationalization" (gợi ý đáp án đúng để mô hình suy ngược ra lý do) để không bị kẹt ở những câu nó luôn trả lời sai.
Vị trí trong lộ trình: Đọc sau bài 00 (mục 3.6 policy gradient/REINFORCE, 3.7 baseline) và sau nhóm RLHF cơ bản (23-26) để có nền. STaR là bài mở đầu cho nhóm "RL/self-training cho reasoning có thể kiểm tra đúng/sai" (34-50 trong lộ trình): không có mô hình phần thưởng, không có PPO/critic, chỉ có bộ lọc đúng/sai và SFT lại, nhưng tác giả chỉ ra đây là một xấp xỉ của policy gradient. Đọc tiếp bài 35 (PRM, giám sát từng bước thay vì chỉ đáp án cuối), bài 36 (DeepSeekMath/GRPO) và bài 39 (DeepSeek-R1) để thấy ý tưởng "lọc theo đúng/sai" được tổng quát hoá thành RL thật với nhiều mẫu mỗi prompt và chuẩn hoá advantage.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Rationale (lời giải trung gian): một chuỗi suy luận bằng ngôn ngữ tự nhiên đặt trước câu trả lời cuối cùng, kiểu "chain-of-thought". Các nghiên cứu trước bài báo này cho thấy sinh rationale trước khi trả lời giúp LLM giải toán, suy luận thường thức tốt hơn, nhưng có hai cách tạo dữ liệu rationale, cả hai đều có vấn đề:

- **Tập dữ liệu rationale do người viết:** chính xác, nhưng tốn công, không thể làm cho mọi bài toán mới (paper dẫn lại ví dụ CoS-E, chỉ có vài nghìn mẫu do người gán nhãn).
- **Few-shot prompting với vài ví dụ rationale:** không cần dữ liệu lớn, cải thiện so với prompt "trả lời trực tiếp", nhưng độ chính xác vẫn thấp hơn rõ so với một mô hình được tinh chỉnh trực tiếp (không rationale) trên toàn bộ dữ liệu có nhãn. Ví dụ paper đo được trên CommonsenseQA: few-shot CoT GPT-J chỉ 36.6%, trong khi tinh chỉnh trực tiếp đạt 60.0%.

Paper đặt câu hỏi: có cách nào lấy được cả hai lợi ích, "sinh rationale" và "dùng được toàn bộ dữ liệu lớn", mà không cần người viết rationale thủ công?

Thuật ngữ cần biết trước khi đọc:

- **In-context learning / few-shot prompting:** đưa vài ví dụ (câu hỏi, rationale, đáp án) vào prompt để mô hình học "theo mẫu" mà không cập nhật tham số.
- **Tinh chỉnh trực tiếp (Direct):** huấn luyện mô hình dự đoán thẳng đáp án từ câu hỏi, không có rationale.
- **Filtering (lọc theo đáp án đúng):** chỉ giữ lại các mẫu (câu hỏi, rationale sinh ra, đáp án) khi đáp án sinh ra khớp đáp án đúng trong tập dữ liệu.
- **Expert Iteration (ExIt):** một kỹ thuật RL cũ (self-play rồi học bắt chước "expert", sau đó thay expert bằng bản mới) mà tác giả dùng làm nguồn cảm hứng; khác biệt chính so với STaR là ExIt cần một "expert" riêng (ví dụ cây tìm kiếm), còn STaR dùng expert cố định là chính phép lọc đúng/sai.

2. Ý tưởng chính

Lặp một vòng ngoài (outer loop) gồm 3 bước: (1) few-shot prompt mô hình hiện tại để sinh rationale và đáp án cho một lượng lớn câu hỏi chưa có rationale; (2) lọc, chỉ giữ các cặp (câu hỏi, rationale, đáp án) mà đáp án sinh ra khớp đáp án đúng trong tập dữ liệu; (3) tinh chỉnh lại mô hình **gốc** (không tinh chỉnh tiếp từ checkpoint vòng trước, để tránh overfit) trên toàn bộ dữ liệu đã lọc từ đầu tới giờ, rồi lặp lại với mô hình mới.

Vấn đề: vòng lặp này chỉ học được từ câu mô hình **đã trả lời đúng**; với câu nó luôn trả lời sai, không có tín hiệu học nào, nên hiệu năng dừng lại khi mô hình hết khả năng giải thêm câu mới. Paper thêm bước **rationalization**: với câu trả lời sai, đưa thêm đáp án đúng vào prompt như một "gợi ý" (hint) và yêu cầu mô hình sinh rationale **suy ngược** từ đáp án đó. Rationale này được thêm vào dữ liệu huấn luyện **như thể** mô hình tự nghĩ ra mà không có gợi ý (bỏ hint khi lưu lại). Nhờ vậy mô hình được tiếp xúc với cả những bài khó nó chưa tự giải được.

Về mặt lý thuyết, tác giả diễn giải cả quy trình (không có rationalization) là một xấp xỉ của **policy gradient** với phần thưởng 0/1 (đúng/sai) trên biến ẩn là rationale.

3. Phương pháp chi tiết

3.1 Rationale generation bootstrapping (chưa có rationalization)

Cho một LLM đã pretrain M và tập dữ liệu $D = \{(x_i, y_i)\}_{i=1}^{D}$ (câu hỏi, đáp án đúng), cùng một tập prompt nhỏ $P = \{(x_i^p, r_i^p, y_i^p)\}_{i=1}^{P}$ gồm vài ví dụ có rationale viết sẵn ($P \ll D$, ví dụ $P = 10$). Ghép P vào trước mỗi câu hỏi x_i để tạo prompt đầy đủ, mô hình sinh rationale $\hat{r}_i$ rồi đáp án $\hat{y}_i$. Lọc: chỉ giữ $(x_i, \hat{r}_i, y_i)$ khi $\hat{y}_i = y_i$. Tinh chỉnh M gốc trên tập đã lọc, dùng mô hình mới sinh lại rationale cho cả D, lặp lại tới khi hiệu năng bão hoà.

3.2 STaR như một xấp xỉ của policy gradient (diễn giải lại từng bước)

Paper coi M là một mô hình biến ẩn rời rạc: trước khi trả lời, M lấy mẫu một rationale ẩn r, rồi sinh đáp án y dựa trên r:

$$p_M(y \mid x) = \sum_r p(r \mid x)\, p(y \mid x, r).$$

Định nghĩa phần thưởng là hàm chỉ số $\mathbb{1}(\hat{y} = y)$ (đúng = 1, sai = 0) và mục tiêu là tổng kỳ vọng phần thưởng trên toàn tập dữ liệu:

$$J(M, X, Y) = \sum_i \mathbb{E}_{\hat{r}_i, \hat{y}_i \sim p_M(\cdot \mid x_i)} \left[\mathbb{1}(\hat{y}_i = y_i)\right].$$

Lấy gradient, dùng đúng "log-derivative trick" như REINFORCE (bài 00, mục 3.6): với $f(\theta) = \mathbb{E}_{z \sim p_\theta}[g(z)] = \sum_z p_\theta(z) g(z)$,

$$\nabla_\theta f = \sum_z \nabla_\theta p_\theta(z)\, g(z) = \sum_z p_\theta(z)\, \nabla_\theta \log p_\theta(z)\, g(z) = \mathbb{E}_{z \sim p_\theta}\left[g(z)\, \nabla_\theta \log p_\theta(z)\right]$$

(dùng $\nabla p_\theta = p_\theta \nabla \log p_\theta$). Áp cho $z = (\hat{r}_i, \hat{y}_i)$, $g = \mathbb{1}(\hat{y}_i = y_i)$:

$$\boxed{\nabla_\theta J(M, X, Y) = \sum_i \mathbb{E}_{\hat{r}_i, \hat{y}_i \sim p_M(\cdot \mid x_i)} \Big[\mathbb{1}(\hat{y}_i = y_i) \cdot \nabla_\theta \log p_M(\hat{y}_i, \hat{r}_i \mid x_i)\Big]}$$

(diễn giải của phần phân tích, bám đúng công thức (1)-(2) của bài báo). Đây đúng dạng REINFORCE: trọng số của $\nabla \log p_M$ là reward 0/1, không có baseline. Hàm chỉ số làm **triệt tiêu hoàn toàn** gradient của mọi mẫu sai (trọng số 0): đó chính là bước "lọc" trong STaR. Paper nêu rõ hai xấp xỉ mà STaR thực hiện so với policy gradient "đúng nghĩa":

1. **Giảm phương sai bằng cách lấy mẫu greedy/ít mẫu** (không lấy nhiều mẫu Monte Carlo cho mỗi x_i) đổi lại có thể làm thiên lệch việc "khám phá" các rationale khác nhau.
2. **Lấy nhiều bước gradient trên cùng một batch dữ liệu** (giống cách một số thuật toán policy gradient tái sử dụng rollout), tức là không sinh lại rationale mới sau mỗi bước cập nhật mà dùng lại dữ liệu đã lọc cho cả epoch tinh chỉnh.

Vì hai xấp xỉ này, paper gọi STaR là một kỹ thuật đơn giản, cài đặt được bằng công cụ huấn luyện LLM tiêu chuẩn (SFT), và đề nghị nghiên cứu sau làm rõ hơn liên hệ với mục tiêu RL.

3.3 Rationalization

Với câu hỏi mô hình trả lời sai, cho thêm đáp án đúng y_i vào prompt như một **hint** (ví dụ: "(b) grocery cart" được đưa vào trước khi hỏi), yêu cầu mô hình sinh rationale theo đúng văn phong few-shot nhưng được phép "suy luận ngược" từ đáp án đã biết. Khi lưu vào dữ liệu huấn luyện, **bỏ hint đi**, giữ lại $(x_i, \hat{r}_i^{rat}, y_i)$ như thể mô hình tự nghĩ ra mà không có gợi ý. Paper diễn giải thêm (mục 5): rationale generation lấy mẫu từ $p(r \mid x)$, còn rationalization lấy mẫu từ $p(r \mid x, y)$ (điều kiện thêm trên đáp án đúng): một phân phối khác hẳn, "dễ" hơn vì đã biết đích đến. Paper gọi đây là "một ước lượng off-policy của mục tiêu ở công thức (1), lấy mẫu từ mô hình đã có hint làm phân phối đề xuất (proposal distribution)".

Diễn giải thêm của phần phân tích (bài báo không viết rõ phần này): nếu coi đúng nghĩa importance sampling thì lẽ ra cần nhân thêm trọng số $p(r \mid x)/p(r \mid x, y)$ để sửa lệch giữa phân phối lấy mẫu (có hint) và phân phối mục tiêu (không hint); STaR không tính trọng số này, chỉ coi rationale rationalize được là một mẫu "sạch" để SFT tiếp. Đây là một xấp xỉ bias (lệch) thêm, bên cạnh hai xấp xỉ ở mục 3.2; bài báo không phân tích ảnh hưởng của việc bỏ qua trọng số này.

3.4 Thuật toán đầy đủ

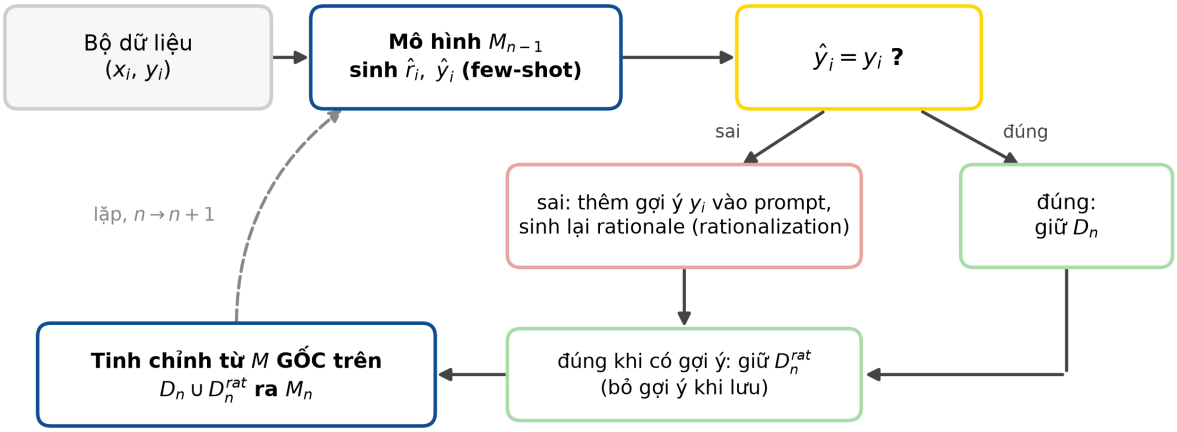
```
Input: M (LLM pretrained), D = {(x_i, y_i)}, few-shot prompt set P
M_0 <- M
for n in 1..N:                                    # outer loop
    for i in 1..|D|:
        (r_hat_i, y_hat_i)      <- M_{n-1}(x_i)                 # rationale generation
        (r_hat_i^rat, y_hat_i^rat) <- M_{n-1}(add_hint(x_i, y_i))  # rationalization
    D_n     = { (x_i, r_hat_i, y_i)     : y_hat_i     == y_i }              # lọc đúng
    D_n^rat = { (x_i, r_hat_i^rat, y_i) : y_hat_i != y_i  AND  y_hat_i^rat == y_i }  #
    ↪  lọc rationalize
    M_n <- train(M, D_n union D_n^rat)             # fine-tune từ M GỐC, không phải
    ↪  M_{n-1}
```

(Bỏ hai dòng liên quan rationalize thì đây chính là STaR không có rationalization.)

Cách đọc hình: mô hình hiện tại sinh rationale và đáp án cho từng câu hỏi, câu đúng đi thẳng vào tập huấn luyện D_n. Câu sai được hỏi lại với đáp án đúng làm gợi ý, nếu lần này đúng thì rationale được giữ lại (không kèm gợi ý). Mô hình mới luôn được tinh chỉnh từ mô hình gốc trên toàn bộ dữ liệu lọc, rồi vòng lặp tiếp tục.

3.5 Ví dụ nhỏ minh hoạ cơ chế lọc (tự tính)

Giả sử ở vòng n, mô hình được few-shot prompt để giải 4 câu hỏi cộng hai số một chữ số, sinh rationale rồi đáp án: $\{7+2 \to 9\ (\text{đúng}),\ 5+6 \to 12\ (\text{sai, đúng là } 11),\ 3+4 \to 7\ (\text{đúng}),\ 8+1 \to 10\ (\text{sai, đúng là } 9)\}$. Không rationalization: D_n chỉ gồm 2 mẫu đúng (7+2, 3+4); 2 mẫu sai bị loại hoàn toàn, không đóng góp gradient. Với rationalization: 2 câu sai được hỏi lại với hint "đáp án là 11" và "đáp án là 9"; nếu mô hình sinh



Không dùng rationalization: bỏ nhánh "sai", chỉ giữ D_n (tương đương REINFORCE với reward 0/1, không baseline)

Hình 66: Một vòng ngoài của STaR: sinh rationale, lọc theo đáp án, rationalization cho câu sai, tinh chỉnh lại từ mô hình gốc (minh hoạ, tự tính)

được rationale hợp lý dẫn tới đúng 2 đáp án đó, D_n^{rat} thêm 2 mẫu nữa, dữ liệu huấn luyện tăng gấp đôi (từ 2 lên 4 mẫu) mà không cần gán nhãn người.

4. Thí nghiệm và kết quả chính

Mô hình gốc: GPT-J (6B tham số, 28 lớp decoder-only, embedding 1024, 16 head attention kích thước 256, FFN hidden 16384, pretrain trên The Pile, vocab 50.4K). **Hyperparameter:** batch size 8 chuỗi, độ dài 1024 (có packing các mẫu ngắn), không weight decay, train/sample trên 1 node TPU-v3, tìm learning rate trong $[10^{-7}, 10^{-4}]$ với Adam, tốt nhất là 10^{-6}; warmup 100 bước rồi giữ hằng số; số bước tinh chỉnh ở vòng ngoài đầu tiên là 40, tăng 20% mỗi vòng. Riêng arithmetic: với rationalization, có thể bắt đầu ngay ở 300 bước và tăng thêm 20 bước (không phải 20%) mỗi vòng, vì rationalization cung cấp đủ dữ liệu đa dạng; không có rationalization, bắt đầu ở 300 bước làm mô hình overfit vào phép cộng 1 chữ số (tác giả phải dùng lại lịch 40 bước +20%/vòng cho trường hợp này).

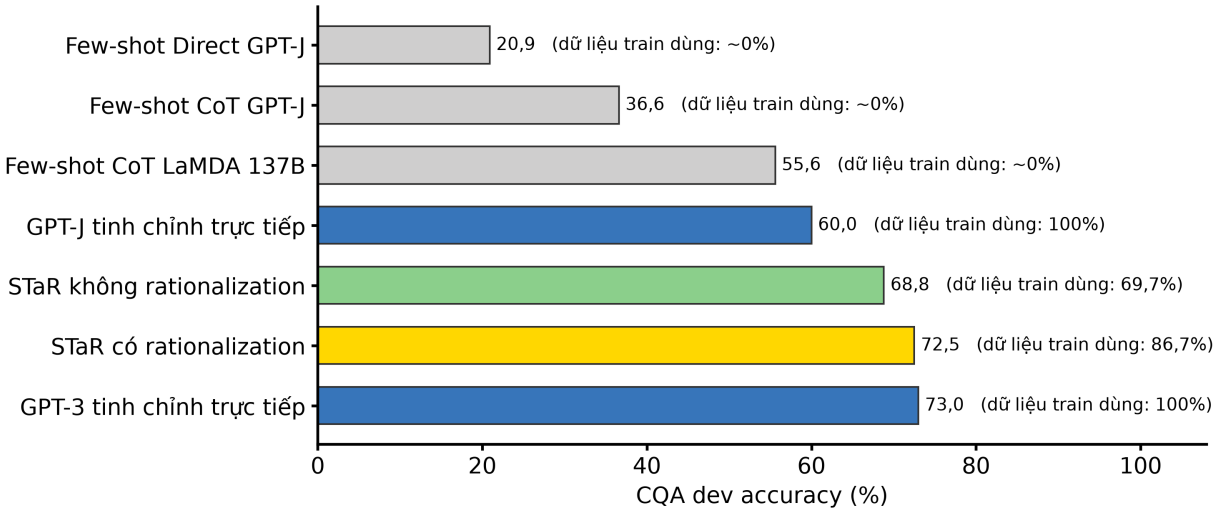
Ba miền thử nghiệm:

1. **Arithmetic (cộng hai số n chữ số, có scratchpad từng bước cộng + carry).** Few-shot thắng rất kém: cộng 2 chữ số đúng dưới 1%. Sau 16 vòng STaR, độ chính xác tổng thể đạt **89.5%** (bài báo không gắn rõ nhãn có/không rationalization cho số này ngay trong câu, nhưng 16 vòng khớp trục hoành của Hình 4b, tức panel "with rationalization"); baseline tinh chỉnh trực tiếp trên 10.000 mẫu không rationale, 5.000 bước, chỉ đạt **76.3%**. Có rationalization, 2 chữ số nhảy từ dưới 1% lên 32% chỉ sau 1 vòng tinh chỉnh; không rationalization, việc học diễn ra theo từng chữ số (n chữ số phải tốt trước khi học (n+1) chữ số).
2. **CommonsenseQA (CQA, 5 lựa chọn, 9.741 câu train).**

Phương pháp	CQA Dev Accuracy (%)	% dữ liệu train dùng
GPT-3 fine-tune trực tiếp (Xu et al. 2021)	73.0	100
Few-shot Direct GPT-J	20.9	~0
Few-shot CoT GPT-J	36.6	~0
Few-shot CoT LaMDA 137B	55.6	~0

Phương pháp	CQA Dev Accuracy (%)	% dữ liệu train dùng
GPT-J fine-tune trực tiếp	60.0	100
STaR không rationalization	68.8	69.7
STaR có rationalization	**72.5**	86.7

STaR (có rationalization) gần bằng GPT-3 tinh chỉnh trực tiếp dù GPT-3 lớn hơn khoảng 30 lần, và dùng ít hơn 100% dữ liệu train.



Hình 67: Độ chính xác CommonsenseQA dev theo phương pháp, kèm tỉ lệ dữ liệu huấn luyện được dùng (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: các thanh xám là few-shot không tinh chỉnh, các thanh xanh là tinh chỉnh trực tiếp, thanh xanh lá và vàng là STaR không và có rationalization. STaR có rationalization đạt 72,5, gần mức 73,0 của GPT-3 tinh chỉnh trực tiếp và cao hơn 60,0 của GPT-J tinh chỉnh trực tiếp. Hai cách STaR chỉ dùng phần dữ liệu train mà mô hình tự giải được hoặc giải được nhờ gợi ý.

3. **GSM8K (toán đố tiểu học, 7.473 train / 1.319 test).**

Phương pháp	GSM8K Test Accuracy (%)	% dữ liệu train dùng
Few-shot Direct GPT-J	3.0	~0
Few-shot CoT GPT-J	3.1	~0
GPT-J fine-tune trực tiếp	5.8	100
STaR không rationalization	10.1	25.0
STaR có rationalization	10.7	28.7

Đánh giá con người (CQA): so sánh rationale của few-shot CoT, STaR, và người viết (Rajani et al.) trên 50 câu cả hai (few-shot và STaR) đều trả lời đúng, 20 crowdworker trên Prolific xếp hạng. Người tham gia có xu hướng xếp STaR cao hơn few-shot CoT 30% nhiều hơn ($p=.039$), và cao hơn rationale do người viết 74% nhiều hơn ($p<.001$); tác giả nhấn mạnh không có nghĩa STaR viết tốt hơn người nói chung, mà cho thấy việc khai thác rationale chất lượng cao từ người vốn khó.

5. Phân tích / ablation

- **Đưa few-shot prompt vào chuỗi tinh chỉnh (CQA):** giữ nguyên các ví dụ few-shot ở đầu mỗi chuỗi huấn luyện (không cắt bỏ) cải thiện rõ: không rationalization 60.9% → 68.8%; có rationalization 69.9% → 72.5%.
- **Thay rationalization bằng sampling nhiệt độ cao hơn:** tác giả thử dùng temperature 0.5-0.7 để tăng đa dạng mẫu thay cho rationalization, nhưng nhất quán **kém hơn**: nhiệt độ cao làm tăng khả năng "đúng đáp án nhờ may" với lý luận sai, và học trên lý luận sai cản tổng quát hoá, đặc biệt rõ ở arithmetic (scratchpad suy biến thành vô nghĩa). Ngoài ra sinh N mẫu tốn khoảng N lần thời gian (sinh tự hồi quy), đắt hơn một lần rationalization.
- **GSM8K, số bước tính toán:** rationalization chỉ cải thiện nhẹ (10.1→10.7%) so với CQA/arithmetic. Số bước "calculator" mô hình sinh ra khớp với số bước người viết trong 53-57% trường hợp qua các vòng; khi khác, mô hình thường dùng **ít bước hơn** (có khi bỏ qua thông tin dư, có khi tìm được đường giải ngắn hơn hợp lệ).
- **Phân tích lỗi (case study, không có số liệu thống kê, mang tính minh hoạ):** các lỗi phổ biến là các "ngụy biện logic" tiêu chuẩn: nêu thông tin liên quan nhưng không thực sự là lý do cho đáp án; coi câu hỏi tự nó là bằng chứng cho đáp án; đặc biệt ở giai đoạn đầu train, mô hình trả lời như đang nói về một cá nhân cụ thể thay vì phát biểu tổng quát (ví dụ "lâu đài của nhà vua là nơi ông ấy cảm thấy an toàn" thay vì "lâu đài là nơi vua chúa cảm thấy an toàn").

6. Giới hạn

Tác giả nêu:

- Vòng đầu của STaR cần hiệu năng few-shot **trên mức ngẫu nhiên**, nghĩa là mô hình gốc phải đủ lớn/đủ năng lực suy luận: GPT-2 không thể tự khởi động (bootstrap) được ngay cả trên arithmetic.
- Các tác vụ có tỷ lệ đúng ngẫu nhiên cao (ví dụ quyết định nhị phân) sẽ sinh ra nhiều rationale "đúng nhờ may mắn" lẫn vào dữ liệu lọc, làm nhiễu quá trình; tác giả coi việc lọc bỏ các rationale "đúng nhưng dở" này là một bài toán mở.

Nhận xét:

- Không có cơ chế kiểm soát chất lượng rationale ngoài đáp án cuối: một rationale sai hoàn toàn về logic nhưng tình cờ ra đúng đáp án vẫn được giữ lại và dùng để tinh chỉnh (rõ nhất ở CQA, 5 lựa chọn, đúng ngẫu nhiên ~20%).
- Hai xấp xỉ nêu ở mục 3.2 (ít mẫu, nhiều bước gradient trên một batch) cùng với việc bỏ qua trọng số importance sampling ở rationalization (mục 3.3) khiến STaR chỉ là một **xấp xỉ khá xa** của policy gradient thật; không có so sánh trực tiếp với một thuật toán RL đầy đủ (PPO/REINFORCE có baseline) trên cùng tác vụ trong bài báo này.
- Chỉ thử nghiệm trên các tác vụ có đáp án **kiểm tra được tự động** (số, lựa chọn, kết quả toán); không rõ cách áp dụng khi không có cách chấm đúng/sai rõ ràng (bài báo không nêu).
- Chỉ dùng một kích thước mô hình (GPT-J 6B); không có thí nghiệm scaling theo kích thước mô hình.

7. Liên hệ với các bài khác

- **Bài 00 (mục 3.6-3.7):** công thức (1)-(2) của STaR chính là REINFORCE với phần thưởng 0/1, **không có baseline**: mọi mẫu đúng được trọng số 1 bằng nhau, mọi mẫu sai bị bỏ hoàn toàn (trọng số 0) chứ không bị phạt âm như khi có baseline. So với bài 00 mục 3.7 (bandit, baseline giảm phương sai), STaR chọn một baseline "cực đoan": ngưỡng cứng 0/1 thay vì trừ trung bình.
- **Bài 23-26 (RLHF cơ bản):** hướng khác hẳn: RLHF học một mô hình phần thưởng liên tục từ sở thích con người rồi chạy PPO; STaR dùng phần thưởng nhị phân có sẵn từ việc so khớp đáp án, không cần mô hình phần thưởng, không cần critic.

- **Bài 35 (Let's Verify Step by Step):** STaR chỉ lọc theo đáp án **cuối cùng** đúng/sai (giống outcome supervision ở bài 35); bài 35 cho thấy giám sát theo **từng bước** (process supervision) định vị lỗi chính xác hơn và cho mô hình phần thưởng mạnh hơn khi dùng để tìm kiếm (best-of-N); một hướng mở rộng tự nhiên của ý tưởng "lọc theo đúng/sai" mà STaR dùng.
- **Bài 36 (DeepSeekMath/GRPO), bài 39 (DeepSeek-R1):** theo mô tả của lộ trình học, các bài này tổng quát hoá ý tưởng "sinh nhiều mẫu, giữ mẫu đúng" của STaR thành RL thật (nhiều mẫu mỗi prompt, advantage chuẩn hoá theo nhóm, cập nhật bằng policy gradient có clipping) chứ không chỉ SFT lại trên mẫu lọc; cần đọc hai bài đó để xác nhận chi tiết quan hệ.
- **Llm 07 (InstructGPT):** quy trình SFT → RM → PPO của InstructGPT và quy trình "sinh → lọc → SFT lại" của STaR đều bắt đầu từ ý tưởng dùng chính mô hình để tạo dữ liệu huấn luyện tốt hơn, nhưng InstructGPT cần phản hồi con người liên tục còn STaR chỉ cần một bộ kiểm tra đáp án tự động.

8. Ứng dụng thực tế (suy luận)

Mức liên quan với việc huấn luyện/post-train mô hình bằng RL: **trung bình**, chủ yếu như một kỹ thuật tiền-RL rẻ.

- STaR cho thấy khi bài toán có cách **kiểm tra đúng/sai tự động, rẻ** (so khớp đáp án, pass unit test, v.v., giống tinh thần RLVR ở bài 00 mục 4.5), có thể lấy được một phần lợi ích của RL mà không cần cài PPO/GRPO: chỉ cần sinh mẫu, lọc, rồi SFT lại bằng quy trình huấn luyện tiêu chuẩn. Chi phí thấp hơn nhiều so với rollout + cập nhật policy-gradient đầy đủ, vì không cần giữ policy cũ, không cần tính advantage, không cần nhiều epoch PPO trên mỗi batch rollout.
- Rủi ro chính khi áp dụng: với tác vụ có tỷ lệ đúng-ngẫu-nhiên cao hoặc tiêu chí "đúng" lỏng (ví dụ chỉ so khớp một phần chuỗi), dữ liệu lọc sẽ lẫn nhiều mẫu "đúng nhờ may" và làm hỏng vòng tinh chỉnh tiếp theo (bài báo nêu rõ đây là vấn đề mở, không có giải pháp).
- Rationalization chỉ hoạt động được khi có thể "đưa đáp án đúng làm gợi ý" một cách tự nhiên trong prompt; với tác vụ sinh tự do (không phải trắc nghiệm/toán) việc này không rõ cách làm (bài báo không bàn).

9. Thí nghiệm gợi ý

1. So sánh ba cách huấn luyện trên cùng tập GSM8K con (ví dụ 1.000 câu): tinh chỉnh trực tiếp, STaR không rationalization, STaR có rationalization; đo không chỉ accuracy cuối mà cả số vòng outer-loop cần để bão hoà.
2. Đo tỷ lệ "đúng nhờ may" trong dữ liệu lọc của STaR trên CQA bằng cách cho người (hoặc GPT-4 làm giám khảo) chấm thủ công một mẫu 50 rationale được lọc vào D_n, xem bao nhiêu phần trăm rationale thực sự hợp lý dẫn tới đáp án đúng.
3. Thử một biến thể có baseline: thay $\mathbb{1}(\hat{y} = y)$ bằng $\mathbb{1}(\hat{y} = y) - c$ với c là tỷ lệ đúng trung bình của batch (giống baseline ở bài 00 mục 3.7), xem liệu phạt nhẹ các mẫu sai (gradient âm) thay vì bỏ hẳn (gradient 0) có cải thiện tốc độ học không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu RL ẩn	$J = \sum_i \mathbb{E}_{\hat{r},\hat{y}\sim p_M}[\mathbb{1}(\hat{y}_i = y_i)]$	reward 0/1 theo đáp án đúng
Gradient	$\nabla J = \sum_i \mathbb{E}[\mathbb{1}(\hat{y}_i = y_i)\nabla \log p_M(\hat{y}_i, \hat{r}_i \mid x_i)]$	REINFORCE không baseline
Lọc (filtering)	giữ $(x_i, \hat{r}_i, y_i)$ khi $\hat{y}_i = y_i$	chính là gradient 0/1 ở trên
Rationalization	lấy mẫu từ $p(r \mid x, y)$ (có hint), bỏ hint khi lưu	ước lượng off-policy, không có trọng số IS

Khái niệm	Công thức / ý	Ghi chú
Fine-tune lại từ M gốc	không tiếp tục từ M_{n-1}	tránh overfit tích lũy qua các vòng
Arithmetic	89.5% sau 16 vòng (khớp trục Hình 4b, có rat.) vs 76.3% baseline direct	scratchpad từng chữ số
CQA	72.5% (có rat.) vs 60.0% direct vs 73.0% GPT-3	86.7% dữ liệu được dùng 30x lớn hơn
GSM8K	10.7% (có rat.) vs 5.8% direct	cải thiện ít hơn CQA/arithmetic

11. Kết luận ngắn

STaR biến một mô hình pretrain thành một mô hình biết "tự dạy mình suy luận": sinh rationale bằng few-shot, lọc theo đáp án đúng, tinh chỉnh lại, lặp lại; rationalization giải quyết điểm nghẽn "không học được từ câu sai" bằng cách cho mô hình suy ngược từ đáp án đúng. Trên GPT-J 6B, kỹ thuật này cải thiện rõ so với tinh chỉnh trực tiếp trên cả ba miền (arithmetic, CommonsenseQA, GSM8K), và tác giả diễn giải toàn bộ quy trình như một xấp xỉ đơn giản, rẻ để cài đặt, của policy gradient với phần thưởng nhị phân, báo trước hướng RLVR/GRPO sẽ tổng quát hoá đầy đủ hơn ở các bài sau trong lộ trình.

Link gốc: https://arxiv.org/abs/2203.14465

Bài 35

Let’s Verify Step by Step (2023)

Tác giả: Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, Karl Cobbe (OpenAI).
Venue / arXiv: arXiv:2305.20050 (v1, 31/05/2023). Bài báo không nêu venue.
Nguồn: https://arxiv.org/abs/2305.20050
Chủ đề: So sánh hai cách huấn luyện mô hình phần thưởng (RM) cho suy luận toán nhiều bước: **outcome supervision** (ORM, chỉ chấm đáp án cuối) và **process supervision** (PRM, chấm đúng/sai từng bước); đánh giá RM bằng khả năng tìm lời giải đúng qua best-of-N; công bố PRM800K, tập 800K nhãn cấp-bước.
Vị trí trong lộ trình: Đọc sau bài 34 (STaR: lọc theo đáp án cuối đúng/sai, tương đương “outcome supervision” nhưng dùng để SFT lại chứ không phải để huấn luyện RM) và sau bài 00 mục 4.2 (quy trình RLHF, Bradley-Terry) và 4.6 (Goodhart/reward hacking). Bài báo này **không** huấn luyện lại generator bằng RL; nó chỉ huấn luyện RM và dùng RM để tìm kiếm (best-of-N) trên các lời giải có sẵn. Đọc tiếp bài 36 (DeepSeekMath/GRPO) và bài 39 (DeepSeek-R1) để xem PRM được dùng (hoặc bị bỏ) trong một quy trình RL thật như thế nào.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu “tự tính” được tính từ số liệu đã nêu. Mục “Ứng dụng ...” là suy luận.

1. Bối cảnh

LLM giải toán nhiều bước bằng chuỗi suy luận (“chain-of-thought”) vẫn thường **ảo giác** (hallucinate): bịa một bước tính toán hoặc một “sự thật” không đúng, và vì các bước sau phụ thuộc bước trước, một lỗi logic ở giữa làm sập toàn bộ lời giải. Để lọc ra lời giải đúng trong nhiều lời giải ứng viên, người ta huấn luyện một **mô hình phần thưởng (RM)** để chấm điểm lời giải, rồi dùng RM đó để làm **best-of-N** (sinh N lời giải, chọn lời giải RM chấm cao nhất) hoặc làm tín hiệu cho RL.

Uesato et al. (2022) là công trình gần nhất, đặt tên hai cách huấn luyện RM:

- **Outcome-supervised RM (ORM):** chỉ được dạy từ **kết quả cuối cùng** của chuỗi suy luận (đúng/sai so với đáp án).
- **Process-supervised RM (PRM):** được dạy từ nhãn đúng/sai gán cho **từng bước** trong chuỗi suy luận.

Uesato et al. tìm thấy ORM và PRM cho hiệu năng cuối **tương đương nhau** trên GSM8K (toán tiểu học). Bài báo này làm lại so sánh với ba khác biệt: mô hình nền mạnh hơn (GPT-4), lượng phản hồi người nhiều hơn hẳn, và test trên **MATH** (khó hơn GSM8K nhiều).

Thuật ngữ cần biết: **best-of-N**, **generator** (mô hình sinh lời giải, cố định trong suốt bài báo này, không được huấn luyện lại bằng RL), **active learning** (chọn mẫu “đáng dán nhãn nhất” để tối ưu hoá chi phí gán nhãn người), **credit assignment** (xác định chính xác bước nào trong một chuỗi dài là nguyên nhân gây sai).

2. Ý tưởng chính

Paper cố định một **generator** (không cải thiện nó bằng RL, mục 2.1 nói rõ đây là lựa chọn có chủ đích), và chỉ tập trung huấn luyện RM tốt nhất có thể để **xếp hạng** các lời giải do generator sinh ra. So sánh ORM và PRM ở hai quy mô: **large-scale** (tinh chỉnh từ GPT-4, dùng 800K nhãn người PRM800K, nhưng tập train của ORM và PRM không thể so sánh trực tiếp) và **small-scale** (dùng chính PRM lớn nhất làm “người gán nhãn” tự động cho các mô hình nhỏ, để cách ly các yếu tố nhiễu và so sánh công bằng outcome vs process). Kết quả

chính: PRM vượt trội ORM ở cả hai quy mô, và paper còn chỉ ra active learning (chọn lời giải "trông có vẻ đúng nhưng thực ra sai" để dán nhãn) giúp tăng hiệu quả dữ liệu 2.6 lần.

3. Phương pháp chi tiết

3.1 Mô hình nền và dữ liệu MathMix

Mô hình large-scale tinh chỉnh từ GPT-4 (chỉ pretrain next-token, chưa qua RLHF). Mô hình small-scale cùng kiến trúc nhưng pretrain với khoảng **200 lần ít compute hơn**. Cả hai được tinh chỉnh thêm trên **MathMix**: khoảng 1.5B token liên quan toán (bài toán + lời giải, văn bản thảo luận toán, dữ liệu tổng hợp), tương tự cách Lewkowycz et al. (Minerva) làm, giúp tăng năng lực toán trước khi vào giai đoạn RM. Với thí nghiệm lớn: train MathMix khoảng 2 epoch (~3B token); với thí nghiệm nhỏ: dùng biến thể nhỏ hơn của MathMix chỉ còn **1B token** (bỏ phần "critiques" ~500M token), train 6 epoch (~6B token, paper ghi "roughly 6.6B").

3.2 Generator: định dạng lời giải theo từng dòng

Generator được few-shot sinh lời giải cho các bài MATH training, lọc giữ lời giải ra đúng đáp án, rồi tinh chỉnh mô hình nền trên tập đó **một epoch duy nhất**. Mục đích của bước này **không phải** dạy kỹ năng mới, chỉ để dạy generator sinh lời giải ở **định dạng xuống dòng từng bước** (mỗi dòng mới = một bước), giúp việc tách bước để chấm điểm dễ dàng.

3.3 Thu thập nhãn người: PRM800K

Người gán nhãn xem từng bước trong lời giải (sinh bởi generator lớn) và gán một trong ba nhãn: **positive** (đúng và hợp lý), **negative** (sai hoặc không hợp lý), **neutral** (mơ hồ, ví dụ gây hiểu nhầm nhẹ hoặc là một gợi ý không hay nhưng về mặt kỹ thuật vẫn đúng). Nhãn neutral cho phép paper **trì hoãn quyết định**: ở test-time có thể coi neutral là positive hoặc negative và thử cả hai (mục F.2). Paper quyết định chỉ giám sát **tới bước sai đầu tiên**: với lời giải đúng, cả ORM và PRM đều biết "mọi bước đúng"; với lời giải sai, cả hai đều biết "có ít nhất 1 lỗi", còn PRM biết thêm **vị trí chính xác** của lỗi đó. Việc này giữ chi phí gán nhãn giữa hai phương pháp tương đương (xác định đúng/sai một lời giải tương đương với tìm lỗi đầu tiên).

Dữ liệu thô và lọc (Appendix B): thu thập **1.085.590** nhãn cấp-bước trên **101.599** mẫu lời giải. Sau khi loại nhãn kiểm soát chất lượng và nhãn không hoàn thành, còn **~800.000** nhãn trên **~75.000** lời giải của **12.000** bài toán, công bố dưới tên **PRM800K**. Tự tính: $800\text{K}/75\text{K} \approx 10.7$ bước mỗi lời giải; $75\text{K}/12\text{K} = 6.25$ lời giải mỗi bài toán.

Thu thập chia 2 giai đoạn:

- **Phase 1** (~5% dữ liệu, ~40.000 nhãn): cho người gán nhãn nhiều lựa chọn thay thế ở mỗi bước, cồng kềnh và lặp lại; nếu mọi lựa chọn đều negative, người gán nhãn **tự viết bước mới**.
- **Phase 2** (phần lớn dữ liệu, 10 "generation"): mỗi generation, sinh N lời giải/bài, dùng PRM tốt nhất hiện có xếp hạng, chỉ đưa cho người gán nhãn các lời giải **sai đáp án nhưng được PRM chấm cao** ("convincing wrong-answer"), huấn luyện lại PRM sau mỗi generation (active learning, mục 3.4 bên dưới). Bảng tỉ lệ đúng:

	Phase 1	Phase 2	Gộp
% lời giải đúng đáp án	85.1	13.2	14.2
% bước đúng (positive)	58.6	74.1	73.1

Người gán nhãn phải qua 30 câu hỏi kiểm soát chất lượng, chỉ được nhận nếu khớp nhãn "gold" ≥75% thời gian; mỗi generation sau đó còn chèn thêm 10-20 câu kiểm soát để theo dõi liên tục.

Để tránh dữ liệu test bị lẫn vào train, paper đưa 4.500 bài MATH test vào tập train PRM800K, và chỉ đánh giá trên **500 bài test còn lại** (lấy ngẫu nhiên đều, phân phối độ khó/chủ đề đại diện cho toàn bộ test MATH, theo Hình 5 phụ lục C).

3.4 Chiến lược chọn mẫu để dán nhãn (active learning)

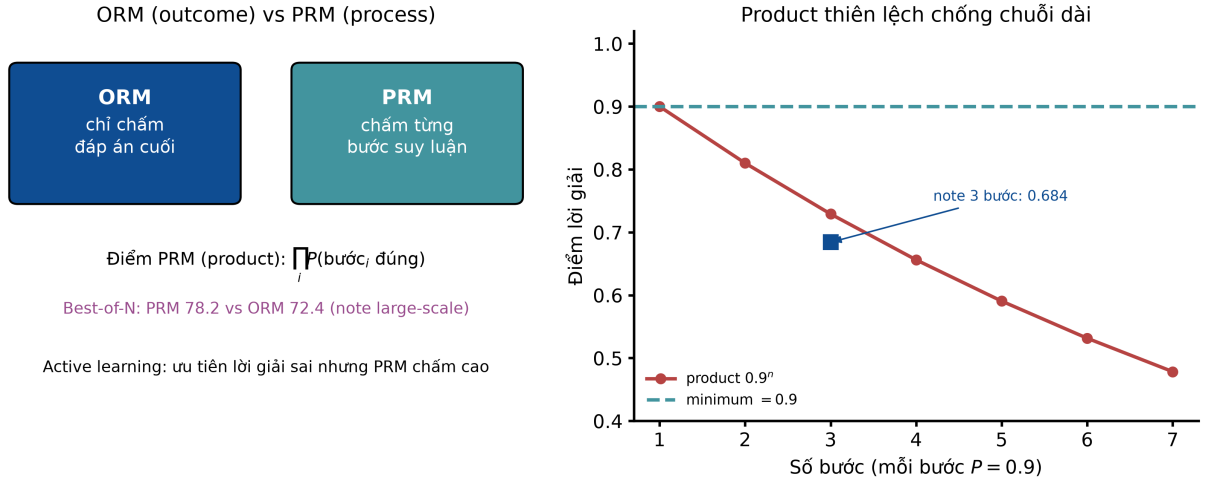
Ý tưởng: dán nhãn ngẫu nhiên lãng phí, vì nhiều lời giải sai "lộ rõ" ngay không cần gán nhãn người mới biết là sai. Paper ưu tiên dán nhãn các lời giải **"convincing wrong-answer"**: được PRM hiện tại chấm cao (convincing) nhưng thực ra ra đáp án sai (wrong-answer); các lời giải này chắc chắn chứa ít nhất một lỗi mà PRM hiện tại chưa phát hiện, nên mang nhiều thông tin hơn mỗi lần gán nhãn. Mục 4.2 huấn luyện một PRM nhỏ $\text{PRM}_{\text{selector}}$ trên 1 mẫu/bài, dùng nó chấm 1000 mẫu/bài, rồi chọn N mẫu để gán nhãn/chấm bằng $\text{PRM}_{\text{large}}$ sao cho 80% là "convincing wrong-answer" nhất và 20% là mẫu convincing còn lại (đúng hoặc sai) để tránh lệch hoàn toàn về phía sai. So sánh độ dốc đường fit giữa có và không có active learning: **ước lượng hiệu quả dữ liệu cao hơn khoảng 2.6 lần**.

3.5 Huấn luyện ORM

Theo phương pháp của Cobbe et al. (2021): lấy mẫu đều (uniform) từ generator, dán nhãn đúng/sai theo kiểm tra đáp án tự động (false positive có thể xảy ra: lời giải sai logic nhưng ra đúng số), huấn luyện RM dự đoán đúng/sai tại **mọi token**, lấy điểm ở **token cuối** làm điểm toàn chuỗi khi test. Large-scale ORM dùng **100 mẫu/bài** từ generator, hoàn toàn không trùng với PRM800K và lớn hơn PRM800K một bậc độ lớn; bài báo có thử trộn thêm mẫu đều vào PRM800K để huấn luyện ORM nhưng **không cải thiện**.

3.6 Huấn luyện PRM và cách chấm điểm lời giải

PRM được tinh chỉnh từ mô hình MathMix để dự đoán nhãn (positive/negative/neutral) ngay sau token cuối của mỗi bước đã gán nhãn, dưới dạng một token đơn, tối đa hoá log-likelihood; nghĩa là **không cần thay đổi kiến trúc hay quy trình huấn luyện LM thông thường**. Ở test-time, chỉ cần **một lượt forward** qua cả lời giải để lấy điểm mọi bước.



ORM vs PRM; điểm product thiên lệch chống lời giải dài (ví dụ $0.684 \to 0.616$) (minh hoạ, tự tính)

Để so sánh nhiều lời giải cần một **điểm số duy nhất** cho cả lời giải. Paper định nghĩa điểm PRM của một lời giải là **xác suất mọi bước đều đúng**, cài đặt bằng **tích (product)** các xác suất "đúng" cấp-bước. Có hai lựa chọn độc lập (Appendix F.2):

$$\boxed{\text{score}(\text{solution}) = \text{reduce}_{t=1}^{T}\, P(\text{step}_t = \text{positive})}$$

- Coi nhãn **neutral** là positive hay negative khi tính $P(\text{step} = \text{positive})$.

- Dùng **product** hay **minimum** làm hàm `reduce` trên các bước.

reduce neutral	= positive	= negative
product	**78.2%** (tốt nhất)	77.4%
minimum	77.6%	77.8%

(Best-of-1860, large-scale; khác biệt giữa 4 cách khá nhỏ.) Paper chọn **product + neutral = positive** cho toàn bộ phần còn lại, và ghi nhận dùng product tạo một **thiên lệch nhẹ chống lại lời giải nhiều bước** (chuỗi dài hơn có nhiều số hạng <1 nhân vào nên điểm càng thấp dù từng bước đều tốt). Ví dụ tự tính: lời giải 3 bước với $P(\text{đúng}) = [0.9, 0.95, 0.8]$ cho điểm product $= 0.9 \times 0.95 \times 0.8 = 0.684$ trong khi minimum $= 0.8$; nếu kéo dài thêm một bước tốt 0.9 nữa, product giảm tiếp xuống 0.616 còn minimum không đổi, minh hoạ đúng thiên lệch mà bài báo nêu.

Khi huấn luyện trực tiếp PRM từ task ngôn ngữ (dự đoán token tiếp theo) sang task phân loại (dự đoán nhãn), đây là một distribution shift lớn, nên cần **learning rate thấp** để ổn định; tất cả PRM huấn luyện **2 epoch** (với dữ liệu nhỏ 2 epoch tốt hơn 1; với dữ liệu lớn lợi ích giảm nhưng vẫn giữ 2 epoch cho nhất quán).

4. Thí nghiệm và kết quả chính

4.1 Large-scale: ORM vs PRM vs majority voting

Đánh giá bằng best-of-N trên 1860 lời giải/bài sinh sẵn cho mỗi bài test:

Phương pháp	% giải được (best-of-1860)
ORM	72.4
PRM	**78.2**
Majority voting	69.6

PRM vượt cả ORM và majority voting ở **mọi** giá trị N, và khoảng cách **nới rộng khi N tăng** (PRM tìm kiếm hiệu quả hơn trên nhiều lời giải). ORM chỉ nhích hơn majority voting một chút. Thử kết hợp PRM với majority voting bằng "RM-weighted voting" nhưng **không cải thiện đáng kể**.

4.2 Small-scale: cách ly nhiễu để so sánh công bằng

Large-scale ORM và PRM không so sánh trực tiếp được vì: (a) tập train khác nhau một bậc độ lớn, PRM800K lại bị lệch nặng về lời giải sai-đáp-án do active learning; (b) nhãn ORM qua kiểm tra đáp án tự động có false positive (lời giải sai logic nhưng đúng số). Để cách ly, bài báo dùng $\text{PRM}_{\text{large}}$ (PRM lớn nhất ở mục 4.1) làm "oracle" gán nhãn cho ba kiểu giám sát **trên cùng một tập dữ liệu mẫu từ generator nhỏ**: process supervision từ $\text{PRM}_{\text{large}}$, outcome supervision từ $\text{PRM}_{\text{large}}$ (coi đúng nếu mọi bước được $\text{PRM}_{\text{large}}$ chấm đúng), và outcome supervision từ kiểm tra đáp án cuối (final-answer checking). Kết quả (best-of-500): **process supervision vượt rõ cả hai dạng outcome supervision ở mọi mức dữ liệu**; outcome từ $\text{PRM}_{\text{large}}$ tốt hơn outcome từ final-answer checking (vì $\text{PRM}_{\text{large}}$ phát hiện được các lời giải "đúng nhờ sai số" mà final-answer checking bỏ lọt).

4.3 OOD generalization

Đánh giá trên **224 câu STEM** mới (lấy từ đề AP Calculus, AP Chemistry, AP Physics, AMC10/12 ra sau khi dữ liệu pretrain đã đóng băng), best-of-100:

Môn	ORM	PRM	Majority Vote	Số bài
AP Calculus	68.9%	86.7%	80.0%	45
AP Chemistry	68.9%	80.0%	71.7%	60
AP Physics	77.8%	86.7%	82.2%	45
AMC10/12	49.1%	53.2%	32.8%	84
Aggregate	63.8%	72.9%	61.3%	234

PRM vẫn vượt ORM và majority voting trên dữ liệu chưa từng thấy, cho thấy lợi thế không chỉ do PRM "quá fit" vào format MATH.

5. Phân tích / ablation

- **Vai trò của active learning (Hình 4a):** 4 chuỗi RM huấn luyện với chiến lược dán nhãn khác nhau, so trên nhiều cỡ tập train; PRM + active learning luôn nằm trên các đường khác, ước lượng **2.6 lần hiệu quả dữ liệu hơn** dán nhãn ngẫu nhiên đều.
- **Thử retrain $\text{PRM}_{\text{selector}}$ liên tục giữa các generation:** paper quan sát **bất ổn định không giải thích được**, và kết quả không tốt hơn cách cố định; coi đây là hướng để lại cho tương lai, không có kết luận chắc.
- **Phân tích theo độ khó (Appendix G):** chia MATH test thành 5 nhóm theo tỉ lệ generator giải được; khoảng cách PRM-ORM **xuất hiện ở mọi mức độ khó**, không chỉ bài khó. Ở nhóm dễ nhất, hiệu năng ORM còn **giảm nhẹ** khi tăng số mẫu N (dễ bị mẫu "giả mạo" đánh lừa ở N lớn), còn PRM ổn định; tăng N giúp nhiều nhất ở nhóm khó nhất (hợp lý, vì cần nhiều mẫu hơn để tìm được một lời giải đúng và thuyết phục).

6. Giới hạn

Tác giả nêu:

- **Ô nhiễm dữ liệu test (test set contamination):** một số bài MATH test có thể đã xuất hiện trên mạng và lẫn vào dữ liệu pretrain; paper lọc bằng string-matching nhưng không đảm bảo tuyệt đối. Tác giả cho rằng nếu có ô nhiễm, nó ảnh hưởng tương đối giống nhau giữa các phương pháp, nên so sánh tương đối (ORM vs PRM) vẫn hợp lệ; kết quả OOD (mục 4.3, đề thi ra sau khi đóng băng dữ liệu) được dùng để tăng độ tin cậy cho lập luận này.
- Không rõ kết quả có tổng quát ra ngoài toán học hay không (paper chỉ thử nghiệm trên MATH và STEM, mục 6.2 tự nhận "unknown how broadly these results will generalize beyond the domain of math").

Nhận xét:

- Paper **chủ động không** đưa generator vào vòng lặp RL (mục 2.1): mọi kết luận chỉ về chất lượng RM khi dùng để **tìm kiếm** (best-of-N), không phải khi dùng làm reward cho một policy đang được cập nhật; hành vi của PRM khi bị chính policy "tối ưu hoá chống lại" (giống Goodhart ở bài 27) chưa được kiểm tra trong bài báo này.
- Large-scale ORM và PRM **không so sánh trực tiếp được** (tập train khác nhau hẳn một bậc độ lớn và khác phân phối); phần small-scale (mục 4.2) tuy cách ly được nhiễu nhưng lại phải dùng $\text{PRM}_{\text{large}}$ làm "sự thật" thay người, nên về bản chất đang so sánh "bắt chước $\text{PRM}_{\text{large}}$ theo hai cách" chứ không hoàn toàn là "outcome vs process supervision của con người".
- Chi phí gán nhãn người cho PRM800K (hơn 1 triệu nhãn thô) rất lớn; bài báo không ước tính chi phí tiền/giờ công cụ thể, nên khó so sánh ROI với việc chỉ tăng dữ liệu cho ORM.

7. Liên hệ với các bài khác

- **Bài 00 (mục 4.2, 4.6):** RM ở đây huấn luyện bằng cross-entropy trên nhãn rời rạc (không phải Bradley-Terry trên cặp so sánh như RLHF cổ điển), nhưng mối lo "RM chỉ đúng gần phân phối nó được train" (mục 4.6, Goodhart) áp dụng tương tự: PRM vẫn có thể bị "đánh lừa" như ở các ví dụ False Positive trong Appendix I.3 của bài báo (lỗi đếm/suy luận tinh vi mà PRM bỏ lọt).
- **Bài 27 (Scaling Laws for Reward Model Overoptimization):** paper trích dẫn trực tiếp ở mục 7.2 ("Synthetic Supervision"), tự nhận cách dùng $\text{PRM}_{\text{large}}$ để giám sát mô hình nhỏ (mục 4.2) có cùng tinh thần với cách bài 27 dùng một "gold RM" để thay nhãn người. Khác biệt: bài 27 đo đường cong gold-vs-proxy reward khi **tối ưu hoá** chống lại proxy (RL/BoN), còn bài này chỉ dùng $\text{PRM}_{\text{large}}$ để **gán nhãn tĩnh**, không có vòng tối ưu hoá lặp lại.
- **Bài 34 (STaR):** STaR lọc dữ liệu SFT theo đáp án cuối đúng/sai, tương đương một dạng "outcome supervision" nhưng áp cho chính generator (tái huấn luyện generator) chứ không train RM riêng; bài báo này cho thấy nếu mục tiêu là có một bộ **chấm điểm** tốt (để tìm kiếm hoặc làm reward), giám sát theo từng bước (PRM) định vị lỗi chính xác hơn và cho hiệu năng cao hơn so với chỉ nhìn đáp án cuối.
- **Bài 00 mục 4.5 (RLVR):** lập luận "RLVR không có RM để bị khai thác" áp dụng rõ nếu generator sau này được train bằng RL trực tiếp trên đáp án đúng/sai (reward kiểm chứng được) thay vì dùng PRM làm reward; theo mô tả của lộ trình học, đây gần với lý do bài 39 (DeepSeek-R1) được cho là bỏ PRM ở quy trình RL cuối, cần đọc bài đó để xác nhận chi tiết.
- **Bài 36 (DeepSeekMath/GRPO):** theo lộ trình, GRPO có thể kết hợp với giám sát theo từng bước dạng PRM làm reward cho mỗi bước thay vì chỉ reward cuối chuỗi như RLVR thuần; cần đọc bài 36 để xác nhận cách họ dùng process reward cụ thể ra sao.

8. Ứng dụng thực tế (suy luận)

Mức liên quan với việc huấn luyện/post-train mô hình bằng RL: **trung bình**, chủ yếu như bài học về **thiết kế reward/verifier** hơn là một công thức áp dụng thẳng.

- Bài học chính có thể mang sang RLVR cho các tác vụ khác: khi tác vụ có nhiều bước trung gian, một verifier chỉ nhìn **kết quả cuối** (giống ORM) sẽ gặp bài toán credit-assignment khó (không biết lỗi nằm ở đâu trong một chuỗi dài), trong khi verifier/reward theo **từng bước** cho tín hiệu đặc hơn, dù chi phí gán nhãn ban đầu cao hơn nhiều (ở đây 800K nhãn thủ công).
- **Active learning (ưu tiên dán nhãn các mẫu "trông đúng nhưng sai")** là ý tưởng tái dùng được bất cứ khi nào việc tạo nhãn verifier đắt: ưu tiên ngân sách gán nhãn vào các mẫu mà RM/verifier hiện tại **tự tin nhưng sai**, thay vì dán nhãn ngẫu nhiên đều.
- Rủi ro khi áp dụng ngoài toán: cần một cách chấm "bước nào đúng/sai" rõ ràng; với các tác vụ không có cấu trúc bước rành mạch (ví dụ OCR, nơi lỗi là ký tự sai rải rác chứ không phải "bước suy luận"), khái niệm PRM không chuyển thẳng sang được, bài báo không bàn tới trường hợp này.

9. Thí nghiệm gợi ý

1. Trên một tập con GSM8K/MATH nhỏ, tự dựng một ORM và một PRM mini (mô hình nhỏ hơn nhiều so với GPT-4) dùng chính một LLM mạnh có sẵn làm "oracle" gán nhãn từng bước (giống mục 4.2), so best-of-N giữa hai cách.
2. Đo thực nghiệm sự khác biệt giữa `product` và `minimum` khi reduce điểm PRM trên các lời giải có độ dài khác nhau; kiểm tra có đúng là `product` phạt nặng lời giải dài như phần 3.6 chỉ ra không.
3. Thử active learning đơn giản: train một verifier nhỏ trên 10% dữ liệu, dùng nó chọn ra các mẫu "tự tin nhưng sai" trong 90% còn lại để ưu tiên gán nhãn/kiểm tra kỹ hơn, so hiệu năng với gán nhãn ngẫu nhiên cùng lượng.
4. Đo hiệu năng PRM theo quintile độ khó (giống Appendix G) trên một benchmark khác MATH, kiểm tra PRM có luôn tốt hơn ORM ở mọi mức khó hay chỉ ở một số mức.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
ORM	chấm đúng/sai theo đáp án cuối	token cuối = điểm toàn chuỗi
PRM	chấm đúng/sai từng bước	1 forward pass, dự đoán 3 lớp (pos/neg/neutral)
Điểm PRM của lời giải	score $= \prod_t P(\text{step}_t = \text{positive})$	product, neutral = positive (tốt nhất)
Best-of-1860	ORM 72.4%, **PRM 78.2%**, majority vote 69.6%	large-scale
Active learning	chọn mẫu "convincing wrong-answer" để dán nhãn	~2.6x hiệu quả dữ liệu
PRM800K	800K nhãn / 75K lời giải / 12K bài	lọc từ 1.085.590 nhãn thô / 101.599 mẫu
Giám sát tới bước sai đầu tiên	cả ORM, PRM biết "có lỗi"; PRM biết thêm vị trí	giữ chi phí gán nhãn ngang nhau
OOD (STEM mới)	PRM 72.9% > ORM 63.8% > majority 61.3%	234 bài theo Table 1 (text ghi 224)

11. Kết luận ngắn

Trên bài toán suy luận nhiều bước (MATH), huấn luyện mô hình phần thưởng bằng nhãn **từng bước** (PRM) cho kết quả tìm-kiếm-đúng (best-of-N) tốt hơn rõ rệt so với chỉ dùng nhãn **đáp án cuối** (ORM), ở cả quy mô lớn (tinh chỉnh GPT-4, 800K nhãn người) và quy mô nhỏ (dùng chính PRM lớn làm oracle để cách ly nhiễu). Active learning, ưu tiên dán nhãn các lời giải "trông thuyết phục nhưng sai", giúp tăng hiệu quả dữ liệu khoảng 2.6 lần. Paper chủ động giữ generator cố định, nên các kết luận chỉ nói về chất lượng của RM-làm-verifier, không phải về việc dùng PRM làm reward cho một vòng lặp RL thật, một câu hỏi được các bài sau trong lộ trình (36, 39) tiếp tục trả lời.

Link gốc: https://arxiv.org/abs/2305.20050

Bài 36

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (2024)

Phạm vi ghi chú: paper gồm cả quy trình thu thập corpus toán (120B token từ Common Crawl), SFT, và RL, cộng nhiều bảng benchmark. Ghi chú này **tập trung vào GRPO** (mục 4 của bài báo: từ PPO tới GRPO, advantage theo nhóm, outcome/process supervision, KL estimator, iterative RL) và phần phân tích **"khung thống nhất" (unified paradigm)** ở mục 5.2 (so sánh SFT/RFT/DPO/PPO/GRPO qua data source, reward function, gradient coefficient; phát hiện Maj@K vs Pass@K). Phần quy trình dữ liệu toán và SFT chỉ tóm tắt ngắn ở mục 5.
Chương nền liên quan: nền 03: Policy gradient và baseline, nền 08: RL cho mô hình lập luận

Tác giả: Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, Daya Guo (DeepSeek-AI; Tsinghua University; Peking University).
Venue / arXiv: arXiv:2402.03300 (v3, 27/04/2024); bài báo không nêu venue hội nghị (chỉ có link GitHub chính thức). **Nguồn:** https://arxiv.org/abs/2402.03300 **Chủ đề:** DeepSeekMath 7B (tiếp tục pretrain DeepSeek-Coder-Base-v1.5 7B trên corpus toán 120B token) và GRPO (Group Relative Policy Optimization), một biến thể của PPO bỏ hẳn mô hình giá trị, dùng trung bình phần thưởng của một nhóm lời giải cho cùng câu hỏi làm baseline. **Vị trí trong lộ trình:** Đọc sau bài 00 (MDP, policy gradient theorem, REINFORCE, baseline, GAE, importance sampling, trust region, PPO, KL penalty, RLHF, RLVR, DPO), bài 07/23 (PPO gốc, RLHF from human preferences), bài 29 (DPO). Nên đọc cùng nhóm với bài 34 (STaR) và 35 (Let's Verify Step by Step, process mô hình phần thưởng/PRM) vì GRPO process supervision dùng reward dạng PRM. GRPO là điểm khởi đầu cho cả nhánh "reasoning RL" phía sau: bài 39 (DeepSeek-R1 dùng GRPO trực tiếp, không mô hình giá trị, cho suy luận dài), bài 41 (DAPO sửa GRPO ở quy mô lớn), bài 42 (Dr. GRPO phê bình độ lệch do chuẩn hoá theo std/độ dài), bài 43 (GSPO chuyển advantage từ mức token sang mức chuỗi).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Toán học là tác vụ suy luận có cấu trúc, cần nhiều bước chính xác, nên các LLM mã nguồn mở thường thua xa các mô hình đóng (GPT-4, Gemini Ultra) trên benchmark MATH cấp thi đấu (Hendrycks et al., 2021). Hai hướng cải thiện mà paper kết hợp:

1. **Pretrain tiếp trên corpus toán lớn, sạch** (DeepSeekMath Corpus, 120B token), thay vì chỉ SFT trên bộ nhỏ.
2. **RL sau SFT** để "nâng" một mô hình đã biết giải toán lên mức chính xác hơn. Paper dẫn hai công trình trước đó (Luo et al., 2023; Wang et al., 2023b) cho thấy RL giúp cải thiện suy luận toán, và đề xuất **GRPO** làm thuật toán RL rẻ hơn PPO.

Khái niệm nền cần nhớ (đã có ở bài 00, 07, 23, nhắc lại ngắn):

- **Actor-critic / value function:** PPO cần một mạng giá trị V_ψ học song song với policy để làm baseline cho advantage (GAE, bài 05).
- **Reward model (RM) r_φ:** mạng được huấn luyện để chấm điểm một câu trả lời, thường học từ dữ liệu so sánh cặp (preference pairs) giống RLHF (bài 23-25).
- **Reference model π_{ref} và KL penalty:** mô hình SFT ban đầu, dùng để giữ policy không đi quá xa (tránh reward hacking), xem bài 00.

- **Process mô hình phần thưởng (PRM):** chấm điểm từng *bước* suy luận (không chỉ đáp án cuối), xem bài 35 (Let's Verify Step by Step).
- **Maj@K / Pass@K:** với K mẫu sinh ra cho một câu hỏi, Pass@K = tỉ lệ câu hỏi có *ít nhất một* mẫu đúng; Maj@K = tỉ lệ câu hỏi mà đáp án *xuất hiện nhiều nhất* (bầu chọn theo đa số, "majority voting") là đúng.

2. Ý tưởng chính

PPO dùng mô hình giá trị để tính baseline cho advantage, nhưng mô hình giá trị này cùng cỡ với policy, nên vừa tốn bộ nhớ/compute vừa khó huấn luyện chính xác vì trong bối cảnh LLM, reward thường chỉ được gán ở token cuối cùng của câu trả lời (sparse reward), khiến việc học giá trị *theo từng token* không chính xác. **GRPO bỏ hẳn mô hình giá trị**: với mỗi câu hỏi q, lấy mẫu một *nhóm* G câu trả lời $\{o_1, \ldots, o_G\}$ từ policy, chấm điểm từng câu trả lời, rồi dùng **trung bình phần thưởng của cả nhóm làm baseline** để tính advantage (độ lệch so với "mức trung bình của nhóm"). Cách này khớp tự nhiên với bản chất của mô hình phần thưởng (thường được huấn luyện trên dữ liệu *so sánh* các câu trả lời cho cùng câu hỏi), và tiết kiệm đáng kể tài nguyên huấn luyện so với PPO. Thành phần KL-penalty cũng được tách ra: GRPO cộng trực tiếp KL divergence vào loss thay vì nhúng vào reward như PPO/RLHF chuẩn.

3. Phương pháp chi tiết: GRPO

3.1. Từ PPO đến GRPO

PPO tối ưu hoá:

$$J_{PPO}(\theta) = \mathbb{E}_{q\sim P(Q),\, o\sim\pi_{\theta_{old}}(O|q)} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \min\Big(\rho_t A_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\, A_t \Big) \right], \quad \rho_t = \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}$$

với A_t là advantage tính bằng GAE dựa trên phần thưởng $\{r_{\geq t}\}$ và value function V_ψ. Chuẩn RLHF (Ouyang et al., 2022) nhúng KL penalty vào reward từng token:

$$r_t = r_\varphi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}$$

Vấn đề: value function V_ψ nặng tương đương policy (tốn bộ nhớ/compute gấp đôi), và vì mô hình phần thưởng trong LLM chủ yếu chỉ chấm ở token cuối, việc học giá trị *chính xác ở mọi token* khó khăn.

GRPO (Hình 4 của bài báo): với mỗi câu hỏi q, lấy mẫu nhóm $\{o_1, \ldots, o_G\} \sim \pi_{\theta_{old}}$, rồi tối ưu:

$$\boxed{J_{GRPO}(\theta) = \mathbb{E}_{q,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \Big(\min\left(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}\right) - \beta\, D_{KL}[\pi_\theta \| \pi_{ref}] \Big) \right]}$$

với $\rho_{i,t} = \dfrac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}$, và $\hat{A}_{i,t}$ là advantage tính **chỉ dựa trên phần thưởng tương đối trong nhóm** (mục 3.2). Khác biệt chính so với PPO: (1) không có mô hình giá trị, baseline lấy từ trung bình nhóm; (2) KL cộng trực tiếp vào loss (không nhúng vào reward), tránh làm phức tạp công thức advantage.

3.2. Advantage theo nhóm: outcome vs process supervision

Outcome supervision (chỉ chấm đáp án cuối): RM chấm G câu trả lời ra $\mathbf{r} = \{r_1, \ldots, r_G\}$, chuẩn hoá theo trung bình/độ lệch chuẩn của *nhóm*, rồi gán advantage đó cho **mọi token** trong câu trả lời:

$$\boxed{\hat{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}, \qquad \hat{A}_{i,t} = \hat{r}_i \ \ \forall t}$$

Process supervision (PRM chấm từng bước, theo Wang et al., 2023b): với mỗi câu trả lời o_i có K_i bước, PRM cho reward $r_i^{\text{index}(j)}$ ở token cuối bước j. Paper chuẩn hoá các reward này theo trung bình/độ lệch chuẩn của **toàn bộ tập R** (gộp reward mọi bước của mọi câu trả lời trong nhóm, không chỉ từng câu riêng):

$$\tilde{r}_i^{\text{index}(j)} = \frac{r_i^{\text{index}(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}, \qquad \hat{A}_{i,t} = \sum_{\text{index}(j) \geq t} \tilde{r}_i^{\text{index}(j)}$$

tức advantage của token t là tổng các reward-to-go của những bước *từ sau vị trí t trở đi* (giống reward-to-go trong REINFORCE nhưng rời rạc theo bước chứ không theo token).

3.3. KL estimator không chệch

Thay vì nhúng $\beta \log(\pi_\theta / \pi_{ref})$ vào reward, GRPO ước lượng trực tiếp $D_{KL}[\pi_\theta \| \pi_{ref}]$ bằng estimator không chệch (Schulman, 2020):

$$\boxed{D_{KL}[\pi_\theta \| \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_\theta(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_\theta(o_{i,t}|q, o_{i,<t})} - 1}$$

Vì sao luôn dương (tự tính): đặt $x = \log \dfrac{\pi_{ref}}{\pi_\theta}$, biểu thức trên bằng $e^x - x - 1$. Vì hàm e^x lồi và tiếp tuyến tại $x = 0$ là $1 + x$, ta có $e^x \geq 1 + x \ \forall x \in \mathbb{R}$ (dấu bằng chỉ khi $x = 0$, tức $\pi_\theta = \pi_{ref}$). Suy ra $e^x - x - 1 \geq 0$ luôn đúng, nên estimator này không âm, ngược với $\log(\pi_\theta / \pi_{ref})$ thô (có thể âm, phương sai cao hơn khi lấy mẫu một điểm).

3.4. Iterative RL với GRPO

Khi RL tiến triển, mô hình phần thưởng cũ có thể không còn đủ tốt để giám sát policy hiện tại (phân phối output đã đổi). Paper đề xuất lặp lại: cứ sau một số bước, đặt $\pi_{ref} \leftarrow \pi_\theta$ và huấn luyện tiếp RM bằng dữ liệu mới lấy mẫu từ chính policy, trộn 10% dữ liệu lịch sử (replay) để tránh quên.

```
Algorithm 1: Iterative GRPO
Input: policy khởi tạo pi_theta_init, reward model r_phi, tập prompt D, hyperparam eps,
↪  beta, mu
pi_theta <- pi_theta_init
for iteration = 1..I:
    pi_ref <- pi_theta                      # cập nhật reference = policy hiện tại
    for step = 1..M:
        lấy batch D_b từ D
        pi_theta_old <- pi_theta
        với mỗi câu hỏi q trong D_b: lấy mẫu G câu trả lời {o_i} ~ pi_theta_old(.|q)
        tính reward {r_i} bằng r_phi cho từng o_i
        tính A_hat_{i,t} theo outcome hoặc process supervision (muc 3.2)
        for grpo_iter = 1..mu:
            cập nhật pi_theta bằng cách tối đa hoá J_GRPO (muc 3.1)
```

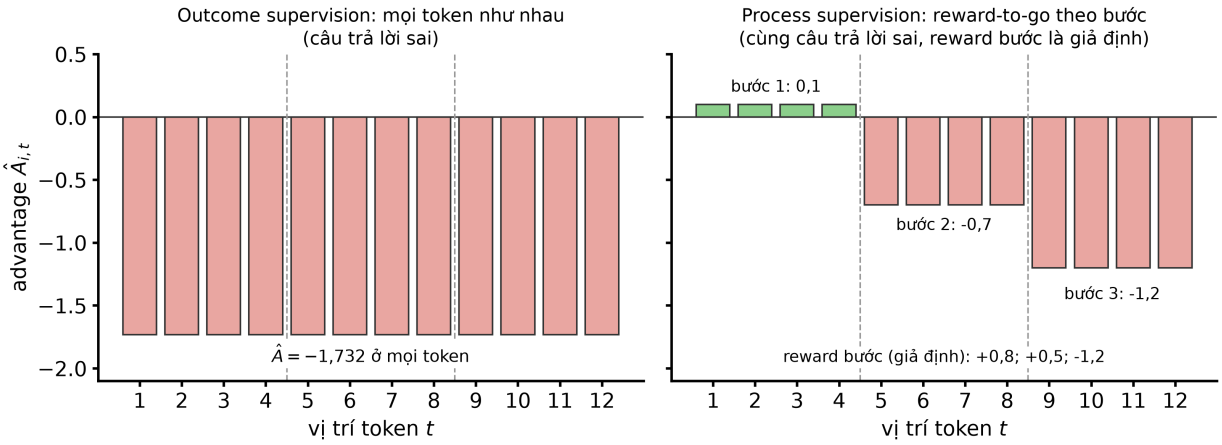
```
    cập nhật r_phi bằng continual training (dùng replay 10% dữ liệu cũ)
Output: pi_theta
```

3.5. Ví dụ tính advantage nhóm (tự tính)

Giả sử $G = 4$ câu trả lời cho cùng một câu hỏi, dùng outcome supervision với reward đúng/sai (1/0): $\mathbf{r} = [1, 1, 1, 0]$ (3 đúng, 1 sai).

- mean$(\mathbf{r}) = 0.75$.
- Phương sai (dùng công thức dân số): $\frac{1}{4}\left[3 \times (0.25)^2 + (-0.75)^2\right] = \frac{1}{4}(0.1875 + 0.5625) = 0.1875$, nên std$(\mathbf{r}) = \sqrt{0.1875} \approx 0.4330$.
- Advantage cho mỗi câu **đúng**: $\hat{r} = (1 - 0.75)/0.4330 \approx +0.577$.
- Advantage cho câu **sai**: $\hat{r} = (0 - 0.75)/0.4330 \approx -1.732$.

Nhận xét: cả 3 câu đúng nhận advantage dương *bằng nhau* (không phân biệt câu nào "đúng hay" hơn vì chỉ có reward 0/1), còn câu sai bị phạt nặng hơn nhiều về độ lớn tuyệt đối. Đây chính là điểm khác Online RFT (mục 4): Online RFT chỉ dùng gradient coefficient $\in \{0, 1\}$ (không phạt câu sai, hệ số 0 nghĩa là bỏ qua hoàn toàn), còn GRPO gán advantage âm rõ ràng cho câu sai, "đẩy" xác suất của nó xuống thay vì chỉ thờ ơ.



Hình 68: Advantage theo token của cùng một câu trả lời sai: outcome supervision (trái) và process supervision với reward bước giả định (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, theo ví dụ nhóm [1, 1, 1, 0] ở mục 3.5, mọi token của câu sai nhận cùng advantage -1,732. Bên phải, advantage của mỗi token là tổng reward-to-go của các bước từ vị trí đó trở đi, nên các token ở bước sai bị phạt nặng hơn các token ở bước đầu. Reward bước +0,8, +0,5, -1,2 là giả định để minh hoạ công thức, không phải số của paper.

4. Khung thống nhất (unified paradigm)

Paper (mục 5.2.1) đưa ra một công thức chung cho gradient của nhiều phương pháp huấn luyện:

$$\nabla_\theta J_{\mathcal{A}}(\theta) = \mathbb{E}_{(q,o)\sim D}\left[\frac{1}{|o|}\sum_{t=1}^{|o|} GC_{\mathcal{A}}(q, o, t, \pi_{rf})\,\nabla_\theta \log \pi_\theta(o_t|q, o_{<t})\right]$$

Ba thành phần: **Data Source** D (dữ liệu huấn luyện từ đâu), **Reward Function** π_{rf} (nguồn tín hiệu phần thưởng), và **Algorithm** $\mathcal{A}$ (biến tín hiệu đó thành **Gradient Coefficient** GC, hệ số khuếch đại/triệt tiêu gradient log-prob của mỗi token).

Phương pháp	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	(không có, là người chọn)	$GC = 1$
RFT	$q \sim P_{sft}(Q)$, $o \sim \pi_{sft}(O\|q)$	Rule (đúng/sai)	$GC = \mathbb{1}[o \text{ đúng}]$
Online RFT	$q \sim P_{sft}(Q)$, $o \sim \pi_\theta(O\|q)$	Rule	$GC = \mathbb{1}[o \text{ đúng}]$ (giống RFT, nhưng o lấy mẫu real-time)
DPO	$q \sim P_{sft}(Q)$, $o^+, o^- \sim \pi_{sft}(O\|q)$	Rule (hoặc preference người)	$GC_{DPO} = \sigma\Big(\beta \log \frac{\pi_\theta(o^-)}{\pi_{ref}(o^-)} - \beta \log \frac{\pi_\theta(o^+)}{\pi_{ref}(o^+)}\Big)$
PPO	$q \sim P_{sft}(Q)$, $o \sim \pi_\theta(O\|q)$	Model (RM đã huấn luyện)	$GC_{PPO} = A_t$ (từ GAE + value model)
GRPO	$q \sim P_{sft}(Q)$, $\{o_i\}_{i=1}^G \sim \pi_\theta(O\|q)$	Model (RM đã huấn luyện)	$GC_{GRPO} = \hat{A}_{i,t} + \beta\Big(\frac{\pi_{ref}(o_{i,t})}{\pi_\theta(o_{i,t})} - 1\Big)$

(Bảng 10 của bài báo; công thức GC lấy từ Phụ lục A.1, giả định $\pi_{\theta_{old}} = \pi_\theta$ để đơn giản hoá, khớp với cách huấn luyện thật của DeepSeekMath-RL là chỉ cập nhật policy một lần sau mỗi lượt thu thập dữ liệu.) Hệ số GC_{DPO} ở trên dùng tính đối xứng $\sigma(-x) = 1 - \sigma(x)$ nên tương đương dạng quen thuộc "trọng số bằng xác suất mô hình vẫn xếp hạng sai cặp preference" đã gặp ở bài 29.

Quan sát về Data Source (online vs offline): RFT và DPO là **offline** (dữ liệu lấy một lần từ π_{sft} ban đầu); Online RFT và GRPO là **online** (lấy mẫu real-time từ π_θ đang huấn luyện). Hình 5 của bài báo (DeepSeekMath-Instruct 1.3B tiếp tục train bằng RFT/Online RFT/GRPO+OS/GRPO+PS) cho thấy Online RFT vượt rõ RFT ở giai đoạn sau của huấn luyện: ban đầu actor gần giống SFT model nên hai cách lấy mẫu tương tự nhau, nhưng càng huấn luyện actor càng lệch xa SFT model, nên lấy mẫu real-time (online) mang nhiều thông tin hơn.

Quan sát về Gradient Coefficient (Rule vs Model): Online RFT chỉ có hệ số $\{0, 1\}$, tức *đồng nhất tăng cường mọi câu đúng ở cùng một mức* và *bỏ qua hoàn toàn* câu sai (không phạt). GRPO (xem mục 3.5) điều chỉnh hệ số liên tục theo giá trị reward, nên có thể **phạt khác nhau tuỳ mức độ sai/đúng**. Theo Hình 5, GRPO vượt Online RFT; và GRPO+PS (process supervision) vượt GRPO+OS (outcome supervision), cho thấy lợi ích của tín hiệu gradient mịn, theo từng bước. Iterative RL (2 vòng lặp, Hình 6) cải thiện thêm, rõ nhất ở vòng lặp đầu tiên (bài báo không cho số liệu chính xác trong văn bản, chỉ trình bày qua đồ thị).

Phát hiện Maj@K vs Pass@K (mục 5.2.2, "Why RL Works?"): so Pass@K (tồn tại ít nhất 1/K mẫu đúng) và Maj@K (đáp án đa số trong K mẫu đúng) giữa DeepSeekMath-Instruct và DeepSeekMath-RL trên GSM8K/MATH, với $K \in \{1, 4, 8, 16, 32, 64\}$. Kết quả: **RL cải thiện rõ Maj@K nhưng hầu như không cải thiện Pass@K**. Diễn giải của tác giả: RL không mở rộng *năng lực nền tảng* (tập các đáp án mà mô hình có khả năng sinh ra), mà làm phân phối đầu ra "mạnh mẽ hơn" (robust), tức khuếch đại xác suất của đáp án đúng vốn đã nằm trong top-K khả dĩ, nên khi bầu chọn theo đa số (Maj@K) đáp án đúng dễ thắng hơn. Đây là quan sát quan trọng cho cả các bài RLVR về sau (bài 39, 45): RL theo kiểu này giống "định hình lại" phân phối hơn là "học kỹ năng mới".

Bài báo cũng nêu hướng cải thiện tương lai cho từng thành phần: Data Source (thử sampling ngoài phân phối, decoding kiểu tree-search); Algorithm (thuật toán bền với reward noise, vì ngay cả PRM800K được annotate kỹ vẫn có ~20% nhãn sai); Reward Function (tăng khả năng tổng quát hoá của RM, mô hình hoá độ không chắc chắn của RM, xây PRM chất lượng cao hơn).

5. Pipeline dữ liệu toán học và SFT (tóm tắt)

DeepSeekMath Corpus: quy trình lặp thu thập trang web toán từ Common Crawl, khởi đầu bằng seed corpus OpenWebMath, huấn luyện fastText để lọc trang "giống toán", sau đó mở rộng seed bằng cách tìm các domain có >10% trang được giữ lại ở vòng trước (gán nhãn "math-related"). Sau 4 vòng lặp: 35.5M trang, 120B token; decontaminate bằng khớp n-gram (10-gram, hoặc 3-gram với đoạn ngắn) với GSM8K/MATH/CMATH/AGIEval. So với các corpus toán khác trên cùng mô hình thử nghiệm DeepSeek-LLM 1.3B (150B token mỗi corpus): DeepSeekMath Corpus (120.2B) cho GSM8K 23.8% / MATH 13.6%, vượt Proof-Pile-2 (51.9B, 14.3%/11.2%), OpenWebMath (13.6B, 11.5%/8.9%), MathPile (8.9B, 2.7%/3.3%).

DeepSeekMath-Base 7B: tiếp tục pretrain DeepSeek-Coder-Base-v1.5 7B trong 500B token (56% DeepSeek-Math Corpus, 4% AlgebraicStack, 10% arXiv, 20% code GitHub, 10% dữ liệu tự nhiên Anh/Trung). Kết quả: MATH 36.2% (vượt Llemma 34B 10%+ tuyệt đối, vượt cả Minerva 540B 33.6% dù nhỏ hơn 77 lần).

SFT: 776K ví dụ CoT/PoT (program-of-thought)/tool-integrated, tiếng Anh (GSM8K, MATH có giải tool-integrated, một phần MathInstruct, Lila-OOD) và tiếng Trung (bài toán K-12, 76 chủ đề con). Huấn luyện 500 bước, batch 256, learning rate hằng số 5e-5, ra DeepSeekMath-Instruct 7B.

6. Thí nghiệm và kết quả chính

Setup RL: khởi tạo từ DeepSeekMath-Instruct 7B; dữ liệu huấn luyện RL là tập câu hỏi dạng CoT liên quan GSM8K/MATH trích từ dữ liệu SFT (~144K câu hỏi, **chủ động loại các câu hỏi SFT khác** để kiểm tra RL có tổng quát hoá ra ngoài domain huấn luyện hay không). Reward model huấn luyện trên DeepSeekMath-Base 7B, learning rate 2e-5 (theo cách xây dựng của Wang et al., 2023b). Hyperparameter GRPO: learning rate policy 1e-6, hệ số KL $\beta = 0.04$, **64 output được lấy mẫu cho mỗi câu hỏi** ($G = 64$), độ dài tối đa 1024, batch size huấn luyện 1024, chỉ cập nhật policy **một lần** sau mỗi lượt khám phá (khớp với giả định $\pi_{\theta_{old}} = \pi_\theta$ ở mục 4).

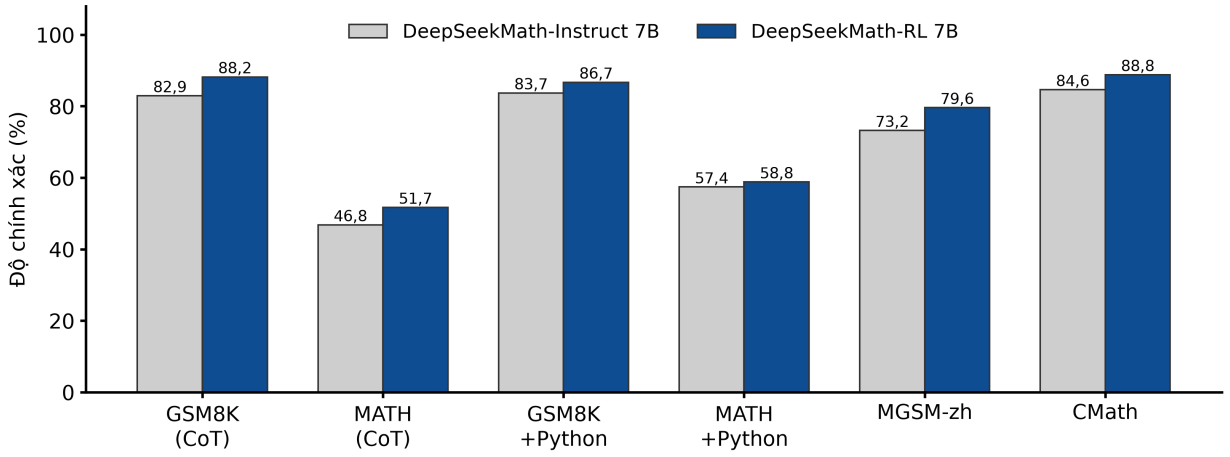
Model	GSM8K (CoT)	MATH (CoT)	GSM8K+Python	MATH+Python	MGSM-zh	CMath
DeepSeekMath-Instruct 7B	82.9%	46.8%	83.7%	57.4%	73.2%	84.6%
DeepSeekMath-RL 7B	**88.2%**	**51.7%**	**86.7%**	**58.8%**	**79.6%**	**88.8%**

(Trích Bảng 5 của bài báo; MGSM-zh/CMath là benchmark tiếng Trung, đo chain-of-thought.) DeepSeekMath-RL 7B vượt mọi mô hình mã nguồn mở từ 7B đến 70B và đa số mô hình đóng (trừ GPT-4, Gemini Ultra), mặc dù chỉ được RL trên dữ liệu GSM8K/MATH, nó **vẫn cải thiện cả trên benchmark tiếng Trung** (MGSM-zh, CMath) chưa từng xuất hiện trong dữ liệu RL, cho thấy một mức tổng quát hoá nhất định. Self-consistency 64 mẫu đạt 60.9% trên MATH (theo Abstract).

Cách đọc hình: mỗi cặp cột là một benchmark, cột xám là trước RL và cột xanh là sau RL bằng GRPO. RL cải thiện cả sáu benchmark, mức tăng lớn nhất ở GSM8K CoT (82,9 lên 88,2) và MGSM-zh (73,2 lên 79,6). MGSM-zh và CMath là benchmark tiếng Trung không nằm trong dữ liệu RL.

7. Phân tích / ablation

- **RFT vs Online RFT vs GRPO+OS vs GRPO+PS** (Hình 5, mô hình 1.3B): thứ tự hiệu quả tăng dần đúng như trình bày ở mục 4: RFT < Online RFT < GRPO+OS < GRPO+PS.
- **Iterative RL** (Hình 6, 2 vòng lặp trên GSM8K/MATH với DeepSeekMath-Instruct 7B): cải thiện rõ nhất ở vòng lặp đầu; bài báo không cho số liệu chính xác trong văn bản (chỉ đồ thị).



Hình 69: DeepSeekMath-Instruct 7B so với DeepSeekMath-RL 7B trên sáu benchmark toán (vẽ lại từ số liệu trong ghi chú)

- **Maj@K/Pass@K** (Hình 7): đã trình bày ở mục 4, RL nâng Maj@K ở mọi $K \in \{1, \ldots, 64\}$ nhưng gần như không đổi Pass@K.
- Paper (mục 5.1, không thuộc phạm vi GRPO của ghi chú này) còn có phân tích riêng về việc huấn luyện code cải thiện suy luận toán (continue training trên code trước khi học toán giúp cả hai task toán có/không dùng tool); người đọc muốn sâu hơn nên đọc trực tiếp mục 5.1 của bài báo.

8. Giới hạn

Tác giả nêu: RL quy trình hiện chỉ dùng câu hỏi từ giai đoạn instruction-tuning và sampling nucleus đơn giản, nên chỉ cải thiện Maj@K (không mở rộng Pass@K); định hướng tương lai là sampling ngoài phân phối và decoding kiểu tree-search. Thuật toán hiện tại tin tưởng tuyệt đối vào tín hiệu reward, trong khi mô hình phần thưởng không hoàn hảo (PRM800K annotate kỹ vẫn ~20% sai nhãn), cần thuật toán bền với nhiễu kiểu weak-to-strong (chưa làm trong bài báo này).

Nhận xét:

- GRPO bỏ mô hình giá trị nhưng **vẫn cần một mô hình phần thưởng** được huấn luyện riêng; không phải "reward-free". Cách loại bỏ chi phí này khác với RLOO (bài 37), nơi baseline leave-one-out lấy trực tiếp từ reward thô của nhiều mẫu, không cần RM riêng cho advantage.
- Chi phí chuyển từ "một forward value model mỗi bước" sang "$G = 64$ lần rollout tự hồi quy mỗi câu hỏi": vẫn tốn kém, chỉ là tốn ở pha inference (dễ song song hoá hơn pha update theo bài báo, nhưng không miễn phí).
- Advantage chuẩn hoá bằng chia cho std($\mathbf{r}$): nếu mọi reward trong nhóm bằng nhau (câu hỏi quá dễ hoặc quá khó so với policy hiện tại), std $\to 0$ và advantage không xác định/bùng nổ số học. Bài báo không đề cập cách xử lý trường hợp này; đây chính là một trong các điểm mà Dr. GRPO (bài 42) và DAPO (bài 41) sau này chỉ ra và sửa.
- $G = 64$ là khá lớn; với G nhỏ hơn, ước lượng mean/std trong nhóm nhiễu hơn nhiều (xem thí nghiệm gợi ý).

9. Liên hệ với các bài khác

- **00 (preface), 05 (GAE), 07 (PPO):** GRPO = PPO bỏ GAE/value function, thay bằng baseline theo nhóm; cần hiểu PPO/GAE trước khi đọc mục 3.
- **23-25 (RLHF gốc):** nguồn gốc objective reward + KL penalty mà GRPO sửa cách tính KL (đưa ra

ngoài reward, dùng estimator không chệch).

- **29 (DPO):** xuất hiện trong bảng khung thống nhất (mục 4), offline, "Rule" hoặc preference người làm reward.
- **34 (STaR), 35 (Let's Verify Step by Step):** STaR là một cách self-improve bằng rationale đúng (không phải RL chính thức) cùng tinh thần với RFT; 35 là nguồn của PRM dùng trong GRPO process supervision.
- **37 (RLOO):** hướng khác để bỏ mô hình giá trị, dùng baseline leave-one-out trực tiếp trên reward thô, không cần RM riêng biệt cho advantage theo nhóm như GRPO.
- **38 (Tulu 3 RLVR):** thay mô hình phần thưởng bằng verifier (RLVR), nhưng vẫn dùng PPO đầy đủ (có mô hình giá trị), khác hướng với GRPO.
- **39 (DeepSeek-R1):** dùng GRPO gần như nguyên bản (không mô hình giá trị) ở quy mô lớn hơn, kết hợp reward dạng RLVR (accuracy + format) thay mô hình phần thưởng học được.
- **41 (DAPO), 42 (Dr. GRPO), 43 (GSPO):** các bài chỉnh sửa GRPO khi scale lên lớn: clip bất cân xứng, sửa độ lệch do chuẩn hoá std/độ dài, chuyển advantage từ mức token sang mức chuỗi.

10. Ứng dụng thực tế (suy luận)

- **Khi nào nên chọn GRPO:** khi reward (hoặc verifier) *rẻ để chạy nhiều lần* trên cùng một câu hỏi (sampling G output), và việc duy trì một mô hình giá trị cùng cỡ policy là gánh nặng bộ nhớ không đáng (ví dụ GPU hạn chế hoặc policy đã lớn). Đổi lại, chi phí chuyển sang pha rollout: cần G lần sinh tự hồi quy mỗi câu hỏi, có thể song song hoá bằng batch inference nhưng vẫn là chi phí thật.
- **Siêu tham số quan trọng:** G (số lượng mẫu/nhóm) quyết định độ ổn định của ước lượng mean/std; G nhỏ (4-8) nhiễu hơn nhiều so với $G = 64$ của bài báo nhưng rẻ hơn tuyến tính; β (hệ số KL) kiểm soát độ lệch khỏi policy ban đầu, và vì GRPO cộng KL trực tiếp vào loss (không trộn vào advantage) nên thường dễ tune hơn so với nhúng vào reward; chọn outcome vs process supervision đổi lấy gradient mịn hơn (PS) với chi phí cần reward/PRM chấm theo từng bước (đắt hơn để tạo).
- **Failure mode cần lưu ý:** nếu mọi output trong nhóm cùng đúng hoặc cùng sai, $\text{std}(\mathbf{r}) = 0$, advantage không xác định/bùng nổ số học, mất tín hiệu học cho câu hỏi đó; cần cơ chế lọc hoặc thêm hằng số nhỏ vào mẫu số khi tự triển khai (bài báo gốc không đề cập điều này).

11. Thí nghiệm gợi ý

1. Trên một tập con GSM8K nhỏ, cài outcome-supervision GRPO với reward 0/1 (exact match); so sánh độ lệch (variance) của advantage ước lượng được giữa $G \in \{4, 16, 64\}$.
2. So PPO đầy đủ (GAE + mô hình giá trị) với GRPO trên cùng compute budget (đo wall-clock, GPU memory) cho một mô hình nhỏ (ví dụ 1.3B) trên GSM8K.
3. Thử outcome supervision vs process supervision (tạo PRM giả bằng heuristic khớp kết quả trung gian của từng bước) trên vài trăm câu MATH, đo tốc độ hội tụ của verifiable reward theo bước huấn luyện.
4. Đo lại Maj@K và Pass@K trước/sau GRPO trên một benchmark nhỏ hơn (ví dụ 200-500 câu) để kiểm tra phát hiện "RL cải thiện Maj@K không cải thiện Pass@K" có còn đúng ở quy mô nhỏ.

12. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
PPO objective	$\mathbb{E}\big[\min(\rho_t A_t, \text{clip}(\rho_t, 1 \pm \varepsilon) A_t)\big]$	A_t từ GAE + value model
GRPO objective	như PPO nhưng trung bình trên nhóm G output, trừ βD_{KL}	không value model
Outcome advantage	$\hat{A}_{i,t} = (r_i - \text{mean}(\mathbf{r}))/\text{std}(\mathbf{r})$	hằng số theo mọi token
Process advantage	tổng reward-to-go theo bước, chuẩn hoá theo cả nhóm $\mathbf{R}$	PRM chấm từng bước

Khái niệm	Công thức / ý	Ghi chú
KL estimator GRPO	$\pi_{ref}/\pi_\theta - \log(\pi_{ref}/\pi_\theta) - 1$	luôn ≥ 0 (Schulman 2020)
Gradient tổng quát	$GC_{\mathcal{A}}(q,o,t) \cdot \nabla_\theta \log \pi_\theta(o_t \| q, o_{<t})$	khung thống nhất SFT/RFT/DPO/PPO/GRPO
Maj@K vs Pass@K	RL tăng Maj@K, không tăng Pass@K	RL "định hình lại" phân phối, không mở rộng năng lực nền
Hyperparam RL chính	lr 10^{-6}, $\beta = 0.04$, $G = 64$, batch 1024	DeepSeekMath-RL 7B

13. Kết luận ngắn

GRPO giữ nguyên ý tưởng PPO (clip tỉ lệ xác suất, KL giữ gần reference) nhưng thay mô hình giá trị đắt đỏ bằng một cách tính baseline rẻ hơn: trung bình phần thưởng của một nhóm câu trả lời cho cùng câu hỏi, khớp tự nhiên với cách mô hình phần thưởng được huấn luyện (so sánh theo cặp/nhóm). Paper còn đưa ra khung thống nhất rất hữu ích để nhìn SFT/RFT/DPO/PPO/GRPO qua ba trục Data Source, Reward Function, Gradient Coefficient, và một quan sát quan trọng: RL (kiểu GRPO, dữ liệu hạn chế) cải thiện Maj@K nhưng không cải thiện Pass@K, tức làm phân phối đầu ra "sắc nét" hơn quanh đáp án đúng đã có, hơn là dạy mô hình năng lực hoàn toàn mới. Đây là nền tảng toán học trực tiếp cho GRPO trong DeepSeek-R1 và các biến thể sửa lỗi sau đó (DAPO, Dr. GRPO, GSPO).

Link gốc: https://arxiv.org/abs/2402.03300

Bài 37

Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs (2024)

Tác giả: Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, Sara Hooker (Cohere, Cohere For AI). **Venue / arXiv:** arXiv:2402.14740 (v2, 26/02/2024); bài báo không nêu venue hội nghị. **Nguồn:** https://arxiv.org/abs/2402.14740 **Chủ đề:** Lập luận rằng nhiều thành phần của PPO (value network + GAE, clipping, mô hình hoá từng token như một action) là không cần thiết trong RLHF cho LLM, vì policy khởi đầu từ một mô hình đã pretrain rất mạnh. Đề xuất dùng lại REINFORCE đơn giản (coi cả câu trả lời là một action) với baseline, và REINFORCE Leave-One-Out (RLOO) là bản mở rộng nhiều mẫu của REINFORCE. **Vị trí trong lộ trình:** Đọc sau bài 00 (policy gradient theorem, REINFORCE, baseline, GAE, trust region, PPO, KL penalty, RLHF, DPO), bài 05 (GAE), bài 07/23-25 (PPO, RLHF from human preferences). Đọc trước hoặc cùng bài 28 (N+ Implementation Details of RLHF with PPO, quan điểm đối lập: chi tiết triển khai PPO quan trọng) và bài 29-33 (họ DPO, một hướng khác để tránh PPO). Bài này là một "phản biện" cho việc dùng PPO đầy đủ trong RLHF, và RLOO còn xuất hiện lại gián tiếp qua so sánh với GRPO (bài 36) và với PPO trong RLVR (bài 38).
Chương nền liên quan: nền 03: Policy gradient và baseline, nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Pipeline RLHF chuẩn (Ziegler et al., 2020) gồm ba giai đoạn: (1) **SFT** trên dữ liệu (prompt, completion do người viết); (2) **Reward Model (RM)** huấn luyện như bộ phân loại nhị phân trên cặp preference (x, y^+, y^-); (3) **RL**, thường dùng **PPO**, để tối ưu $\max_\theta \mathbb{E}_{y\sim\pi_\theta}[r_\varphi(x,y) - \beta\,\mathrm{KL}[\pi_\theta \| \pi_{ref}]]$. PPO được xem là lựa chọn "mặc định" cho bước (3), nhưng tốn compute (cần thêm value network, GAE) và nhạy hyperparameter.

Paper đặt câu hỏi: PPO được thiết kế cho RL cổ điển, nơi lo ngại chính là **phương sai cao** và **không gian action/state không ổn định** khi khởi tạo từ chính sách ngẫu nhiên hoặc yếu. Trong RLHF cho LLM, policy khởi tạo từ một **mô hình đã pretrain rất mạnh**, và luôn có **prompt điều kiện hoá**. Liệu các lý do ban đầu để dùng PPO (và các thành phần giảm phương sai của nó) còn áp dụng được không?

Khái niệm nền (nhắc lại ngắn, đã có ở bài 00):

- **Actor-critic (PPO):** học đồng thời policy và một value function $V_\psi(s)$ dùng làm baseline cho advantage (GAE, bài 05).
- **REINFORCE (Monte-Carlo policy gradient):** ước lượng gradient không chệch $\mathbb{E}[R\cdot\nabla \log \pi]$, không cần baseline học riêng nhưng phương sai cao hơn nếu không giảm phương sai bằng cách khác.
- **Bandit vs MDP đầy đủ:** nếu toàn bộ phần thưởng chỉ xuất hiện ở *cuối* chuỗi và môi trường xác định (token sinh ra quyết định hoàn toàn trạng thái tiếp theo), bài toán có thể quy về **bandit** (một action = toàn bộ chuỗi sinh ra) thay vì MDP token-by-token.
- **RAFT (Reward rAnked FineTuning):** lấy k mẫu online, chỉ giữ mẫu điểm cao nhất, tinh chỉnh bằng cross-entropy (giống rejection sampling / "bandit-to-supervised conversion").
- **DPO:** tối ưu trực tiếp trên cặp preference, không cần RM/rollout riêng (xem bài 29).

2. Ý tưởng chính

Paper rà soát từng thành phần của PPO và lập luận rằng chúng không cần thiết trong RLHF: (1) các estimator advantage giảm phương sai bằng cách đánh đổi bias (GAE với $\lambda < 1$) là không cần, vì phương sai trong RLHF đã thấp nhờ khởi tạo mạnh; (2) clipping (để tránh update quá xa so với policy cũ) hiếm khi được kích hoạt, nên hầu như vô tác dụng; (3) mô hình hoá từng token là một action riêng (MDP đầy đủ) là không cần thiết vì phần thưởng chỉ gán cho EOS. Từ đó, bài báo đề xuất dùng lại **REINFORCE** (coi toàn bộ câu trả lời là một action, bài toán bandit) với baseline để giảm phương sai, và đẩy xa hơn bằng **REINFORCE Leave-One-Out (RLOO)**: khi có k mẫu online cho cùng một prompt, dùng trung bình phần thưởng của $k-1$ mẫu còn lại làm baseline cho mỗi mẫu, tạo ra một "value function không tham số" được ước lượng tại chỗ mỗi bước huấn luyện, không cần network giá trị riêng. Thực nghiệm trên TL;DR Summarize và Anthropic-HH cho thấy REINFORCE/RLOO vượt PPO, DPO, và RAFT.

3. Phương pháp chi tiết

3.1. Mục tiêu RLHF chung

Với π_{ref} là reference policy (thường là SFT model) và β kiểm soát độ lệch:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim D,\, y\sim\pi_\theta(\cdot|x)}[r_\varphi(x,y) - \beta\,\mathrm{KL}[\pi_\theta(\cdot|x)\|\pi_{ref}(\cdot|x)]] \equiv \max_{\pi_\theta} \mathbb{E}\Big[R(x,y)\Big], \quad R(x,y) = r_\varphi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$

3.2. PPO trong khung token-level MDP

Paper mô hình mỗi token là một action, mỗi chuỗi con đã sinh là một state, discount $\gamma = 1$. Chỉ token `<EOS>` mang phần thưởng từ mô hình phần thưởng (kết hợp với KL); mọi token khác chỉ có thành phần KL (không có tín hiệu "thực" từ RM). Objective huấn luyện:

$$\min\Big(f(y_t|s_t)\hat{A}_\lambda(y_t,s_t),\ \mathrm{clip}_{1-\epsilon}^{1+\epsilon}(f(y_t|s_t))\,\hat{A}_\lambda(y_t,s_t)\Big), \qquad f(y_t|s_t) = \frac{\pi_\theta(y_t|s_t)}{\pi_{old}(y_t|s_t)}$$

với $\hat{A}_\lambda$ là advantage GAE ($\lambda \in [0,1]$ kiểm soát trade-off bias/variance: $\lambda \to 1$ phương sai cao, không chệch; $\lambda \to 0$ phương sai thấp, chệch).

3.3. REINFORCE: coi cả câu trả lời là một action

Vì $r_\varphi(x,y)$ chỉ được tính ở cuối toàn chuỗi, bài báo đề xuất mô hình hoá **toàn bộ generation là một action duy nhất** (bandit), cho phép dùng trực tiếp REINFORCE:

$$\mathbb{E}_{x,\,y\sim\pi_\theta}[R(x,y)\,\nabla_\theta \log \pi_\theta(y|x)]$$

Giảm phương sai bằng baseline b (không đổi kỳ vọng vì b không phụ thuộc y, nên $\mathbb{E}_y[\nabla \log \pi_\theta(y|x)] = 0$):

$$\boxed{\mathbb{E}_{x,\,y\sim\pi_\theta}[(R(x,y) - b)\,\nabla_\theta \log \pi_\theta(y|x)]}$$

Baseline đơn giản nhất: **moving average** của mọi reward đã gặp trong suốt huấn luyện, $b_{MA} = \frac{1}{S}\sum_s R(x_s,y_s)$ (S = số bước đã huấn luyện).

3.4. RLOO: baseline leave-one-out (dẫn lại)

Khi có k mẫu online $y^{(1)}, \dots, y^{(k)} \overset{iid}{\sim} \pi_\theta(\cdot|x)$ cho **cùng một prompt** x, có thể xây baseline tốt hơn b_{MA}: dùng chính $k-1$ mẫu còn lại của *cùng prompt đó* làm baseline cho mẫu thứ i:

$$\boxed{\frac{1}{k}\sum_{i=1}^{k}\Big[R(y^{(i)},x) - \frac{1}{k-1}\sum_{j\neq i} R(y^{(j)},x)\Big]\nabla_\theta \log \pi_\theta(y^{(i)}|x)}$$

Vì sao không chệch (tự tính, theo lập luận baseline chuẩn): cố định k mẫu đã lấy. Với mẫu i, baseline $b_i = \frac{1}{k-1}\sum_{j\neq i} R(y^{(j)},x)$ không phụ thuộc trực tiếp vào $y^{(i)}$ (chỉ phụ thuộc các mẫu khác), nên khi lấy kỳ vọng riêng theo $y^{(i)} \sim \pi_\theta(\cdot|x)$ (giữ các mẫu khác cố định), $\mathbb{E}_{y^{(i)}}[b_i \nabla_\theta \log \pi_\theta(y^{(i)}|x)] = b_i\, \mathbb{E}_{y^{(i)}}[\nabla_\theta \log \pi_\theta(y^{(i)}|x)] = b_i \cdot 0 = 0$ (tính chất cơ bản của score function). Vậy trừ b_i không làm lệch kỳ vọng của gradient cho từng i; lấy trung bình trên $i = 1, \dots, k$ vẫn giữ tính không chệch. Khác với b_{MA} (một số vô hướng cố định cho mọi prompt, cập nhật chậm theo toàn bộ lịch sử huấn luyện), b_i ở đây được tính **riêng cho từng prompt, tại chỗ, mỗi bước huấn luyện**, nên bám sát "độ khó" thực tế của chính prompt đó hơn, giống một value function không tham số nhưng không cần học network nào. Đánh đổi: tốn thêm k lần sampling mỗi prompt (so với 1 mẫu của REINFORCE/b_{MA}).

3.5. RAFT và DPO (nền so sánh)

RAFT (Dong et al., 2023): lấy k mẫu online, chỉ tinh chỉnh bằng cross-entropy trên mẫu có $R(x,y)$ (hoặc $r(x,y)$) cao nhất; **không dùng hết** k mẫu (chỉ top-1), khác RLOO (dùng toàn bộ k mẫu để xây baseline + gradient).

DPO (Rafailov et al., 2023): bỏ hẳn bước RM + rollout, tối ưu trực tiếp trên cặp preference:

$$-\log \sigma\Big(\beta \log \frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)} - \beta \log \frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)}\Big)$$

3.6. Ba lập luận "bỏ PPO" (mục 3 của paper)

1. **Không cần estimator phương sai thấp (GAE $\lambda < 1$).** GAE đánh đổi bias để giảm phương sai; paper lập luận trong RLHF, khởi tạo mạnh + điều kiện hoá theo prompt làm xác suất tập trung vào ít token mỗi bước sinh, nên cảnh quan tối ưu ổn định, update hiếm khi "nổ". **Thực nghiệm:** sweep $\lambda \in \{0, 0.5, 0.95, 1.0\}$ (Hình 1), reward giảm đơn điệu khi λ giảm từ 1; $\lambda = 1$ (không chệch, "Vanilla PG", xem định nghĩa dưới) cho kết quả tốt nhất.
2. **Clipping hiếm khi cần.** Loss chỉ bị clip $< 5\%$ số lần mỗi batch trên mọi cặp dataset/model, cho thấy chế độ huấn luyện gần như "on-policy". Tắt hẳn clip và tỉ lệ $f(y_t|s_t)$ (đưa PPO về Vanilla PG) còn cho kết quả **nhích lên** một chút.
3. **Không cần mô hình hoá partial completion.** Vì động lực môi trường là xác định (chọn token nào quyết định hoàn toàn trạng thái kế tiếp) và chỉ EOS mang reward "có ý nghĩa", bài toán có thể quy về bandit (một action = cả chuỗi).

Vanilla PG là trường hợp giới hạn $\lambda = 1$ của PPO/GAE (không chệch, không clip), nhưng **vẫn mô hình hoá partial completion**: dùng return-to-go từ token t trừ một baseline học riêng theo state $b_\phi(s_t)$ (huấn luyện bằng MLE, giống value network nhưng chỉ hồi quy baseline, không dùng để tính advantage theo GAE):

$$\sum_{i=t}^{T} \gamma^{T-i-1} R_i(x, y_i) - b_\phi(s_t)$$

Khác biệt mấu chốt để kiểm chứng ở mục 5: **Vanilla PG/PPO** mô hình từng token (partial completion) còn **REINFORCE/RLOO** mô hình cả chuỗi là một action.

4. Thí nghiệm và kết quả chính

Dữ liệu: TL;DR Summarize (Stiennon et al., 2020, 116K hướng dẫn, 93K cặp preference) và Anthropic Helpful & Harmless (HH, 112K cặp preference, qua bản tiền xử lý Dahoas/full-hh-rlhf).

Mô hình: Pythia-6.9B cho cả hai dataset; thêm Llama-7B + Anthropic-HH để kiểm tra ảnh hưởng của chất lượng mô hình nền.

Setup: context length 512 cho cả SFT và RM; policy và RM khởi tạo từ checkpoint SFT. SFT: 2 epoch, lr 2×10^{-5} (Pythia); Llama chỉ cần 1 epoch. RM: 1 epoch, lr 1×10^{-5}. Giai đoạn RL: lr hằng số 1×10^{-6} (chọn sau khi sweep $\{10^{-6}, 10^{-5}, 2 \times 10^{-5}\}$ cho RAFT/RLOO và $\{10^{-6}, 10^{-5}\}$ cho PPO/Vanilla PG), 2 gradient step mỗi batch, TL;DR: 600 bước (rollout batch 512, step batch 256, $\beta = 0.03$); HH (Pythia): 393 bước, cùng cấu hình batch; HH (Llama): rollout/step batch 2048, 2 epoch, $\beta = 0.10$.

Đánh giá: phần thưởng trung bình trên 1000 mẫu test (dùng RM huấn luyện, đo mục tiêu "nội tại"); win-rate mô phỏng bằng GPT-4 làm giám khảo theo khung AlpacaFarm (mục tiêu "ngoại tại", so với completion gốc); và các chỉ số "alignment tax": perplexity, độ dài, đa dạng n-gram (Diversity-1/2).

Bảng 1 (win-rate cuối, trích):

Phương pháp	TL;DR	HH (Pythia)	HH (Llama)
RLOO (k=4)	**77.9**	43.7	**64.1**
RAFT (k=4)	73.2	42.1	63.3
RLOO (k=2)	74.2	**47.6**	62.2
RAFT (k=2)	72.1	37.7	58.4
REINFORCE w/ baseline	70.7	37.9	55.3
Vanilla PG	70.4	36.4	52.3
PPO	67.6	29.2	32.0
DPO	66.6	39.0	61.9

RLOO (k=4) vượt PPO 10.3 / 14.5 / 32.1 điểm trên TL;DR / HH(Pythia) / HH(Llama) tương ứng. REINFORCE w/ baseline xấp xỉ Vanilla PG trên Pythia (70.7 vs 70.4 TL;DR; 37.9 vs 36.4 HH) và **vượt** Vanilla PG trên HH/Llama (55.3 vs 52.3), ủng hộ luận điểm "không cần mô hình partial completion". Trung bình trên cả 3 cặp dataset/model: RLOO đạt win-rate 61.3 (k=2) và 61.9 (k=4), RAFT chỉ 56.1 (k=2) và 59.5 (k=4); RLOO(k=2) thường bằng hoặc vượt RAFT(k=4) dù dùng **nửa** sampling budget.

Cách đọc hình: mỗi panel là một cặp tác vụ và mô hình, các thanh xếp theo thứ tự của bảng, thanh đỏ là PPO. RLOO (k=4) đứng đầu ở TL;DR và HH (Llama), còn RLOO (k=2) cao nhất ở HH (Pythia). PPO thấp nhất ở hai panel HH, đặc biệt là HH (Llama) với 32,0.

5. Phân tích / ablation

Bảng 2 (alignment tax, Anthropic-HH, Llama):

Phương pháp	Độ dài	PPL	Diversity-1	Diversity-2	Reward-Var.
RLOO (k=4)	60.6	27.6	0.10	0.43	3.1
RAFT (k=4)	62.4	30.1	0.10	0.43	3.2
RLOO (k=2)	58.6	29.2	0.11	0.44	3.0

Phương pháp	Độ dài	PPL	Diversity-1	Diversity-2	Reward-Var.
RAFT (k=2)	52.8	28.9	0.12	0.47	3.1
REINFORCE w/ baseline	47.2	27.2	0.13	0.50	2.7
Vanilla PG	39.1	39.0	0.15	0.54	3.7
PPO	16.5	40.4	0.34	0.60	2.3
DPO	104.4	33.8	0.08	0.39	paper không nêu

PPO sinh câu **ngắn nhất** (16.5 token, có dấu hiệu khai thác lỗ hổng reward bằng câu ngắn) và PPL cao (kém trôi chảy hơn); DPO sinh câu **dài nhất** (104.4 token, "over-verbose"). RLOO/RAFT/REINFORCE w/ baseline có PPL thấp hơn rõ so với PPO/Vanilla PG; REINFORCE w/ baseline có **phương sai reward thấp hơn Vanilla PG khoảng 27%** trong khi vẫn tối ưu reward bằng hoặc hơn.

Cách đọc hình: mỗi điểm là một phương pháp, trục ngang là độ dài trung bình và trục dọc là perplexity, càng thấp càng trôi chảy. PPO và Vanilla PG nằm ở vùng perplexity cao, PPO có completion ngắn nhất (16,5). DPO có completion dài nhất (104,4), còn nhóm RLOO, RAFT và REINFORCE với baseline tập trung ở vùng perplexity thấp.

Bền với KL penalty cao (Hình 5, HH Pythia, $k = 2$): sweep $\beta \in \{0.1, 0.25, 0.5, 1.0\}$. Ở β thấp, RLOO và RAFT hội tụ về KL tương đương nhưng RLOO đạt reward cao hơn; ở β cao, RAFT **vừa tối ưu reward kém hơn vừa lệch xa reference hơn**, còn RLOO ổn định hơn.

Bền với nhiễu reward (Hình 6): thêm nhiễu Gauss $\epsilon \sim \mathcal{N}(0, \sigma^2)$ vào logit của RM, $\sigma \in \{1, 3, 5\}$. RAFT giảm reward (thật, không nhiễu) rõ rệt hơn RLOO ở $\sigma = 3, 5$, vì RAFT chỉ dựa vào thứ hạng của 1 mẫu tốt nhất (nhiễu dễ làm sai thứ hạng), còn RLOO lấy trung bình trên nhiều mẫu nên bền hơn.

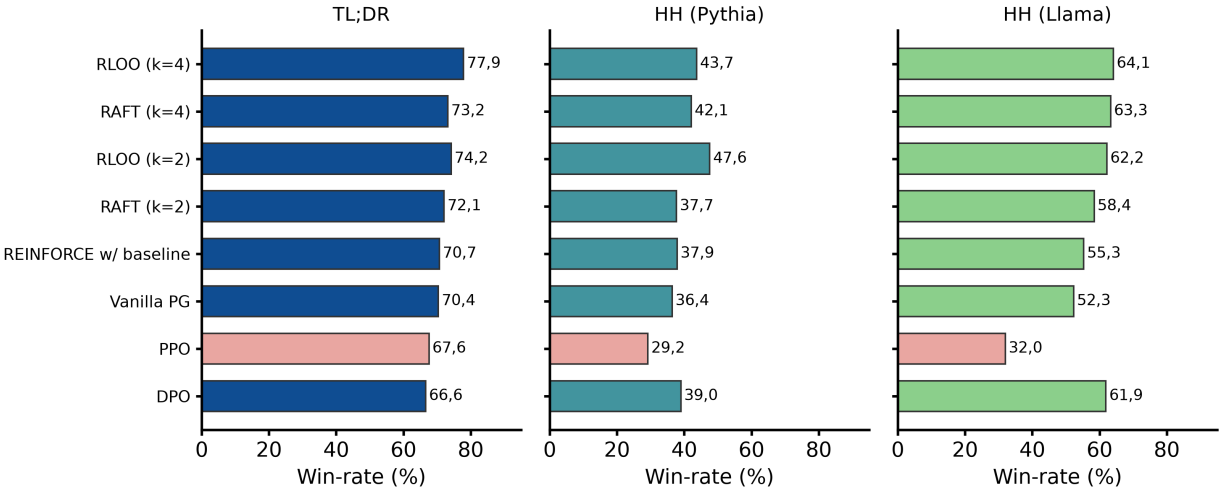
6. Giới hạn

Tác giả nêu (mục 7, Limitations): chưa nghiên cứu **mô hình phần thưởng over-optimization** (phần thưởng proxy lệch khỏi "gold" reward thật, Gao et al., 2022), kể cả cho RAFT; chưa khai thác LOO baseline trong khung mô hình hoá partial sequence (chỉ chứng minh partial sequence là *không cần thiết* trong bối cảnh RLHF này, chưa thử LOO ở mức token); chỉ đo win-rate mô phỏng bằng LLM giám khảo (GPT-4), chưa đối chiếu với đánh giá con người thật; chưa thử RL với reward khác (ROUGE, BLEU, hay các metric NLP khác).

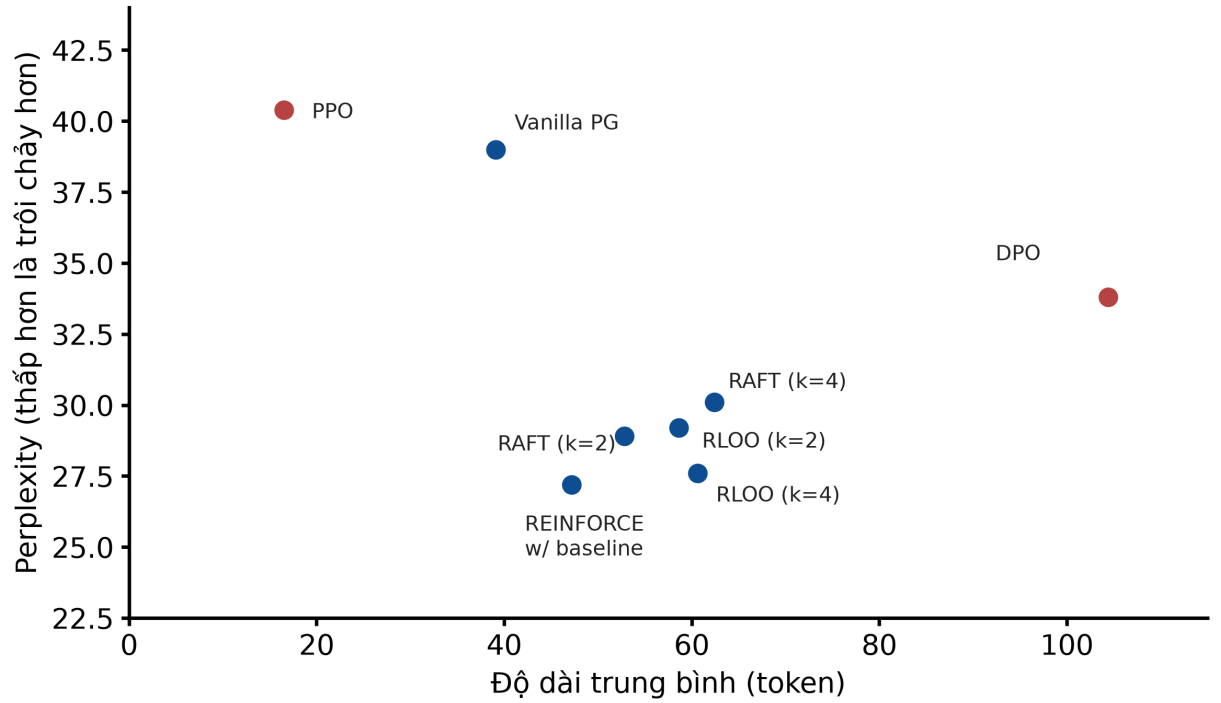
Nhận xét: thực nghiệm chỉ trên mô hình 6.9B-7B (Pythia, Llama-7B) và hai tác vụ (tóm tắt, hội thoại helpful/harmless); chưa rõ kết luận "các thành phần PPO không cần thiết" có còn đúng ở mô hình lớn hơn hoặc tác vụ nặng suy luận nhiều bước (nơi các bài RLVR sau này, 36/38/39, vẫn chọn PPO hoặc GRPO đầy đủ với KL/value phức tạp hơn). Paper chỉ thử $k \in \{2, 4\}$; chưa rõ RLOO scale ra sao với k lớn hơn nhiều (GRPO ở bài 36 dùng $G = 64$) hoặc chi phí sampling tuyến tính theo k có còn đáng hay không ở quy mô đó.

7. Liên hệ với các bài khác

- **00 (preface):** cần nắm REINFORCE, baseline, GAE, PPO, KL penalty trước khi đọc bài này.
- **05 (GAE):** RLOO phản biện trực tiếp lập luận "giảm phương sai bằng đánh đổi bias" của GAE trong bối cảnh RLHF cụ thể.
- **07/23-25 (PPO, RLHF gốc):** RLOO là phản đề cho việc dùng PPO đầy đủ (actor-critic, value network, clipping) trong RLHF.
- **28 (N+ Implementation Details of RLHF with PPO):** góc nhìn gần như đối lập, nhấn mạnh chi tiết triển khai PPO quan trọng; nên đọc đối chiếu.
- **29-33 (DPO, IPO, KTO, SimPO, DPO vs PPO):** cùng mục tiêu "rẻ hơn PPO" nhưng theo hướng offline (DPO-family); RLOO vẫn là online RL (cần rollout + RM) nhưng rẻ hơn PPO.



Hình 70: Win-rate cuối (%) của tám phương pháp trên TL;DR, HH với Pythia và HH với Llama (vẽ lại từ số liệu trong ghi chú)



Hình 71: Độ dài completion và perplexity của các phương pháp trên Anthropic-HH với Llama (vẽ lại từ số liệu trong ghi chú)

- **36 (GRPO):** cùng tinh thần bỏ mô hình giá trị, nhưng khác cách tạo baseline: GRPO chuẩn hoá (trừ mean, chia std) theo nhóm, gán *cùng* advantage cho mọi token (outcome) hoặc theo bước (process); RLOO dùng mean leave-one-out thô (không chia std) trên toàn completion, và baseline của mẫu i loại trừ chính mẫu i ra khỏi nhóm.
- **38 (Tulu 3 RLVR):** dùng PPO đầy đủ (mô hình giá trị khởi tạo từ RM) cho bước RLVR, ngược hướng với đề xuất "bỏ PPO" của RLOO; là một gợi ý thí nghiệm tự nhiên để thử thay RLOO vào RLVR.

8. Ứng dụng thực tế (suy luận)

- **Khi nào nên chọn RLOO:** khi có thể lấy $k = 2$-4 mẫu/prompt với chi phí sampling chấp nhận được, và muốn tránh tốn bộ nhớ GPU cho một value network + GAE riêng; đặc biệt phù hợp khi reward chỉ có ở cuối chuỗi (rất phổ biến trong RLHF/RLVR hiện nay).
- **Siêu tham số quan trọng:** k (paper chỉ thử 2 và 4, càng lớn baseline càng ổn định nhưng tốn sampling tuyến tính); β (RAFT nhạy hơn RLOO khi β lớn, theo mục 5); learning rate (bài báo dùng 10^{-6} sau khi sweep, khá nhỏ so với SFT).
- **Failure mode cần tránh:** RAFT (chỉ giữ $1/k$ mẫu tốt nhất) nhạy với nhiễu trong mô hình phần thưởng và với KL penalty cao hơn RLOO; nếu buộc phải chọn giữa hai, RLOO ổn định hơn với cùng k.
- **Chi phí:** RLOO cần k lần rollout mỗi prompt mỗi bước (tăng thời gian sampling tuyến tính theo k) nhưng không cần network giá trị riêng, giảm bộ nhớ GPU và đơn giản hoá hạ tầng huấn luyện so với PPO.

9. Thí nghiệm gợi ý

1. Trên một bộ prompt nhỏ với reward tự chế (ví dụ độ dài chuỗi hợp lệ, hoặc exact-match), cài REINFORCE với b_{MA} và RLOO ($k = 2, 4$), so sánh phương sai gradient đo qua nhiều lần lấy mẫu lại cùng một batch.
2. So sánh RLOO ($k = 4$) với PPO đầy đủ trên cùng compute budget cho một mô hình nhỏ, đo cả win-rate và wall-clock/GPU memory.
3. Lặp lại thí nghiệm "bền với nhiễu reward" (thêm nhiễu Gauss vào logit RM) cho RLOO vs RAFT trên một bài toán nhỏ để kiểm tra lại claim RLOO bền hơn ở quy mô khác.
4. Thử thay RLOO cho PPO trong bước RLVR (giống cấu trúc bài 38) trên GSM8K, so sánh độ phức tạp cài đặt và chất lượng kết quả.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu RLHF	$\max \mathbb{E}[r_\varphi(x,y) - \beta\,\mathrm{KL}[\pi_\theta \Vert \pi_{ref}]]$	chung cho PPO/REINFORCE/RLOO
REINFORCE (bandit)	$\mathbb{E}[R(x,y)\nabla \log \pi_\theta(y\Vert x)]$	coi cả câu trả lời là 1 action
REINFORCE w/ baseline	$\mathbb{E}[(R-b)\nabla \log \pi_\theta(y\Vert x)]$	b không đổi kỳ vọng
Baseline moving-average	$b_{MA} = \frac{1}{S}\sum_s R(x_s, y_s)$	đơn giản, không cần mẫu thêm
RLOO gradient	$\frac{1}{k}\sum_i \big[R(y^{(i)}) - \frac{1}{k-1}\sum_{j\neq i} R(y^{(j)})\big]\nabla \log \pi_\theta(y^{(i)}\Vert x)$	cần k mẫu/prompt, không mạng giá trị
Vanilla PG	GAE với $\lambda = 1$, vẫn mô hình partial completion	baseline $b_\phi(s_t)$ học bằng MLE
RAFT	cross-entropy trên mẫu tốt nhất trong k	chỉ dùng $1/k$ mẫu
DPO	$-\log \sigma(\beta \log \frac{\pi_\theta(y^+)}{\pi_{ref}(y^+)} - \beta \log \frac{\pi_\theta(y^-)}{\pi_{ref}(y^-)})$	offline, không rollout

11. Kết luận ngắn

Paper lập luận có cơ sở thực nghiệm rằng, trong RLHF cho LLM (khác RL cổ điển khởi tạo yếu), các thành phần giảm phương sai của PPO (GAE với $\lambda < 1$, clipping, mô hình hoá token-level) không mang lại lợi ích tương xứng, và một REINFORCE đơn giản với baseline tốt đã đủ mạnh. **RLOO**, bản mở rộng nhiều mẫu của REINFORCE dùng baseline leave-one-out thay cho network giá trị, vượt PPO, DPO, và RAFT về win-rate trên TL;DR Summarize và Anthropic-HH, đồng thời bền hơn RAFT khi KL penalty cao hoặc reward bị nhiễu. Đây là một lựa chọn "rẻ và đơn giản hơn" đáng xem xét trước khi lao vào cài PPO/GRPO đầy đủ, dù paper mới kiểm chứng ở quy mô 7B và hai tác vụ cụ thể.

Link gốc: https://arxiv.org/abs/2402.14740

Bài 38

Tulu 3: Pushing Frontiers in Open Language Model Post-Training (2024)

Phạm vi ghi chú: đây là báo cáo kỹ thuật rất dài (data curation, SFT, preference tuning, RLVR, hạ tầng, đánh giá hàng chục benchmark, phần "insights from the unfruitful"). Ghi chú này tóm tắt **cấu trúc recipe 4 giai đoạn** (data curation, SFT, preference tuning/DPO, RLVR) và **tập trung sâu vào RLVR** (định nghĩa reward có thể kiểm chứng, thiết lập PPO, khởi tạo mô hình giá trị, kết quả, ablation). Phần data curation chi tiết và SFT chỉ tóm tắt ngắn; phần đánh giá (Tulu 3 Eval) chỉ nêu khái quát.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

Tác giả: Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Xinxi Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, Hannaneh Hajishirzi (Allen Institute for AI; University of Washington). **Venue / arXiv:** arXiv:2411.15124 (v5, 14/04/2025); bài báo không nêu venue hội nghị. **Nguồn:** https://arxiv.org/abs/2411.15124 **Chủ đề:** Recipe post-training mở hoàn toàn (data, code, checkpoint) cho Llama 3.1 ở 8B/70B/405B: 4 giai đoạn (curation dữ liệu, SFT, preference tuning bằng length-normalized DPO, và Reinforcement Learning with Verifiable Rewards - RLVR) cùng bộ eval dev/unseen để tránh quá khớp. **Vị trí trong lộ trình:** Đọc sau bài 00 (PPO, GAE, KL penalty, RLHF, RLVR, DPO), bài 07/23-25 (PPO, RLHF gốc), bài 28 (N+ Implementation Details of RLHF with PPO, các trick triển khai mà Tulu 3 áp dụng trực tiếp), bài 29/32 (DPO, SimPO, nền cho length-normalized DPO), và cùng nhóm với bài 36 (GRPO) và 37 (RLOO) vì cả ba đều là các cách "rẻ hoá" RL cho LLM, dù Tulu 3 lại chọn giữ PPO đầy đủ. Đọc trước bài 39 (DeepSeek-R1), 40 (Kimi k1.5): cả hai dùng ý tưởng RLVR tương tự nhưng với GRPO và cho suy luận dài hơn.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Các recipe post-training mạnh nhất (GPT-4o, Claude, Llama 3.1 Instruct...) gần như không công bố dữ liệu, code, hay chi tiết huấn luyện. Open post-training trước đó (Tulu 2, Zephyr-β) có kết quả tốt trên chat eval (AlpacaEval, Arena-Hard) nhưng còn thua xa ở các năng lực cốt lõi như MATH, GSM8K, IFEval. **Tulu 3** (Allen Institute for AI) đặt mục tiêu công bố toàn bộ recipe (data, code, checkpoint, báo cáo chi tiết) cho các mô hình Llama 3.1 8B/70B/405B, đạt kết quả vượt Llama 3.1 Instruct, Qwen 2.5 Instruct, Mistral, Nous Hermes 3, Nemotron Llama 3.1, và cả một số mô hình đóng (GPT-3.5-Turbo, GPT-4o-mini, tiến gần Claude 3.5 Haiku).

Khái niệm nền (nhắc ngắn, đã có ở bài 00/07/23-25/29):

- **SFT, RM (Bradley-Terry), PPO, DPO:** xem lại nhanh ở mục 4.
- **RLVR (Reinforcement Learning with Verifiable Rewards):** ý tưởng mới của bài báo này: với các tác vụ có đáp án *kiểm chứng được tự động* (toán có đáp án số, hoặc chỉ dẫn có ràng buộc kiểm tra được bằng heuristic), thay mô hình phần thưởng học được bằng một **hàm verifier xác định** (deterministic): đúng cho điểm cố định, sai cho 0 điểm.
- **Dev vs Unseen eval suite:** Tulu 3 Eval có hai bộ, "development" (dùng để chọn hyperparameter/data mix trong lúc phát triển) và "unseen" (chỉ dùng để đánh giá cuối, tránh quá khớp vào quá trình phát triển).

2. Ý tưởng chính

Tulu 3 chia post-training thành 4 giai đoạn nối tiếp: **curation dữ liệu** (prompt từ nguồn mở + sinh tổng hợp theo persona, khử nhiễm với eval), **SFT**, **preference tuning** bằng length-normalized DPO (chọn qua ablation so với PPO/SimPO), và **RLVR**. Ý tưởng cốt lõi của RLVR: với một số kỹ năng có thể kiểm chứng tự động (toán, instruction-following có ràng buộc kiểm được), không cần tốn công xây mô hình phần thưởng học từ preference người mà có thể **cho điểm thẳng bằng một hàm verifier**: điểm α cố định nếu câu trả lời đúng, 0 nếu sai; sau đó tối ưu bằng PPO *nguyên bản* (chỉ đổi nguồn reward), tận dụng toàn bộ kinh nghiệm triển khai PPO đã có từ RLHF chuẩn (khởi tạo mô hình giá trị từ RM, tắt dropout, chuẩn hoá advantage...). Điểm thú vị: trong một ablation kiểm soát riêng (không liên quan RLVR), PPO chỉ **xấp xỉ** DPO (hơi thấp hơn) với chi phí compute cao hơn nhiều (~28 giờ 2 node so với ~4 giờ 1 node của DPO) khi tối ưu cùng một mô hình phần thưởng chung; vì vậy Tulu 3 dùng DPO cho preference tuning tổng quát, và **chỉ dùng PPO cho RLVR** nơi verifier rẻ, rõ ràng, nhắm đúng kỹ năng cụ thể (GSM8K, MATH, IFEval), và thực sự cải thiện các benchmark đó.

3. Cấu trúc recipe 4 giai đoạn: data curation và SFT (tóm tắt)

Giai đoạn 1: Data curation (mục 3). Khảo sát và tuyển chọn hàng loạt dataset mở (WildChat, OpenAssistant, No Robots, FLAN v2, UltraFeedback, SciRIFF, TableGPT, OpenMathInstruct 2, NuminaMath-TIR, Evol CodeAlpaca, Aya, CoCoNot, WildJailbreak, WildGuardMix) cộng dữ liệu tổng hợp theo phương pháp **persona-driven** (Chan et al., 2024): điều kiện hoá ~250K persona từ Persona Hub để sinh prompt nhắm đúng kỹ năng (toán, code, instruction-following chính xác). Tổng cộng khảo sát 23,327,961 prompt (Bảng 7), dùng 939,344 cho SFT và khoảng 425,145 cho DPO. **Khử nhiễm (decontamination):** khớp 8-gram giữa prompt huấn luyện và prompt eval; một instance train bị coi là nhiễm nếu >50% token của nó khớp 8-gram với cùng một instance test; nếu loại bỏ cả dataset không ảnh hưởng hiệu năng thì loại cả dataset, ngược lại chỉ loại instance trùng (ví dụ Evol CodeAlpaca mất 3.5% do trùng HumanEval, NuminaMath-TIR mất 11.3% do trùng MATH).

Giai đoạn 2: SFT (mục 4). Cân bằng tỉ lệ dữ liệu giữa các kỹ năng (general chat, kiến thức, toán, code, an toàn, multilingual, instruction-following chính xác) qua nhiều vòng ablation mixture và kỹ thuật model merging, được dẫn hướng bằng eval suite development, ra Tulu 3-SFT.

(Hai giai đoạn này không phải trọng tâm ghi chú; chi tiết đầy đủ nằm ở mục 3-4 của bài báo.)

4. Giai đoạn 3: Preference Tuning

Nền tảng (mục 5.1): RM huấn luyện theo Bradley-Terry chuẩn $\max_{r_\varphi} \mathbb{E}_{(x,y_c,y_r)}[\log \sigma(r_\varphi(x,y_c) - r_\varphi(x,y_r))]$; mục tiêu RLHF chung $\max_{\pi_\theta} \mathbb{E}_{y\sim\pi_\theta}[r_\varphi(x,y) - \beta\,\mathrm{KL}[\pi_\theta \| \pi_{ref}]]$, có thể tối ưu bằng **PPO** (online) hoặc **DPO** (offline, trực tiếp trên cặp preference):

$$\max_{\pi_\theta} \mathbb{E}_{y_c,y_r\sim D}\Big[\log\sigma\big(\beta\log\tfrac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \beta\log\tfrac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\big)\Big]$$

Paper thử DPO, SimPO, và **length-normalized DPO** (DPO-norm, theo Meng et al., 2024), chuẩn hoá log-prob theo độ dài để giảm thiên lệch ưu tiên câu dài:

$$\boxed{\max_{\pi_\theta} \mathbb{E}_{y_c,y_r\sim D}\Big[\log\sigma\Big(\frac{\beta}{|y_c|}\log\frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \frac{\beta}{|y_r|}\log\frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\Big)\Big]}$$

Ablation thuật toán (trên checkpoint SFT sớm + UltraFeedback, Bảng 18, trích): SFT nền 55.7; SimPO tốt nhất 52.9; DPO ($\beta = 0.1$) 55.2; PPO ($\beta = 0.05$) 55.5; **DPO-norm tốt nhất** ($\beta = 5$, 1 epoch) **57.3**. Chỉ DPO-norm vượt rõ baseline SFT, nên được chọn và tune tiếp.

Dữ liệu preference on-policy: mở rộng quy trình UltraFeedback (Cui et al., 2023): sinh completion từ Tulu 3-SFT và các mô hình khác cho cùng prompt, lấy nhãn preference qua so sánh cặp, ra 354,192 instance.

Hyperparameter DPO cuối (Bảng 20): 8B: lr 5×10^{-7}; 70B: lr 2×10^{-7}; cả hai: batch hiệu dụng 128, max token 2048, $\beta = 5$, warmup 0.1, 1 epoch. 8B train 10 giờ/8×H100; 70B train 19 giờ/64×H100.

So sánh PPO vs DPO có kiểm soát (mục 5.4.1): cùng prompt, cùng RM (huấn luyện trên UltraFeedback, lấy logit ở token EOS làm reward). Kết quả: PPO đạt điểm trung bình **xấp xỉ nhưng thấp hơn** DPO; PPO tốn ~28 giờ (2 node) so với ~4 giờ (1 node) của DPO. Do hạn chế compute và RM khó đánh giá đáng tin cậy, Tulu 3 **chọn DPO cho preference tuning tổng quát**, và dành PPO riêng cho RLVR (mục 5-8). Hai tối ưu hạ tầng để scale DPO lên 70B: (1) **cache log-prob tham chiếu** trước, không giữ bản copy mô hình tham chiếu trong bộ nhớ; (2) **forward pass riêng** cho chosen/rejected (tránh ghép đôi làm tăng gấp đôi batch).

5. Giai đoạn 4: RLVR, định nghĩa và dữ liệu

Mục tiêu RLVR: giữ nguyên dạng mục tiêu RLHF chuẩn, chỉ thay r_φ bằng hàm verifier xác định v:

$$\boxed{\max_{\pi_\theta} \mathbb{E}_{y\sim\pi_\theta(x)}\big[R_{RLVR}(x,y)\big] = \mathbb{E}\big[v(x,y) - \beta\,\mathrm{KL}[\pi_\theta(y|x)\|\pi_{ref}(y|x)]\big]}, \qquad v(x,y) = \begin{cases} \alpha & \text{nếu đúng} \\ 0 & \text{nếu sai} \end{cases}$$

Paper đặt $\alpha = 10$ theo thí nghiệm thử nghiệm ban đầu, **không tune thêm**. Huấn luyện bằng **PPO** (Schulman et al., 2017), sau giai đoạn preference tuning (khởi điểm là checkpoint DPO).

Dữ liệu RLVR (Bảng 22):

Nguồn	Số lượng	Cách kiểm chứng
GSM8K train	7,473	Khớp chính xác đáp số trích xuất (8-shot CoT)
MATH train	7,500	Khớp chính xác theo logic đánh giá "flex" (3-shot CoT)
IF verifiable	14,973	Verifier riêng theo từng loại ràng buộc (lấy từ taxonomy IFEval, Zhou et al. 2023)
Tổng	**29,946**	

(IF verifiable: lấy ngẫu nhiên instruction từ mix SFT Tulu 2, kết hợp với ràng buộc có thể kiểm tra, ví dụ "trả lời đúng 3 đoạn".)

6. RLVR: chi tiết triển khai PPO

Paper thừa nhận RL/RLHF có nhiều chi tiết triển khai ảnh hưởng mạnh tới ổn định huấn luyện, và áp dụng trực tiếp 5 "best practice" từ Huang et al. (2024a) (= bài 28, N+ Implementation Details of RLHF with PPO):

1. **Khởi tạo mô hình giá trị từ một RM chung** (không khởi tạo ngẫu nhiên hay từ policy), theo thiết lập chuẩn RLHF.
2. **Tắt dropout** khi huấn luyện RM/PPO, để log-prob được tính xác định (deterministic) qua cả 2 pha (rollout và learning); nếu không, log-prob hai pha lệch nhau, tỉ lệ ρ có thể bị clip hết, gradient triệt tiêu.
3. **Train trên dữ liệu SFT, xáo trộn (shuffle) giữa các epoch:** PPO có thể train nhiều episode hơn số prompt sẵn có, tức lặp qua dữ liệu nhiều epoch; thí nghiệm ablation train khoảng $100{,}000/7{,}473 \approx 13$ epoch.
4. **Phạt không có EOS:** nếu response không kết thúc bằng token EOS trong giới hạn token, gán phạt -10, khuyến khích mô hình luôn hoàn thành câu trả lời.

5. **Chuẩn hoá (whitening) advantage:** trừ mean, chia std, như trong các tài liệu PPO implementation chuẩn.

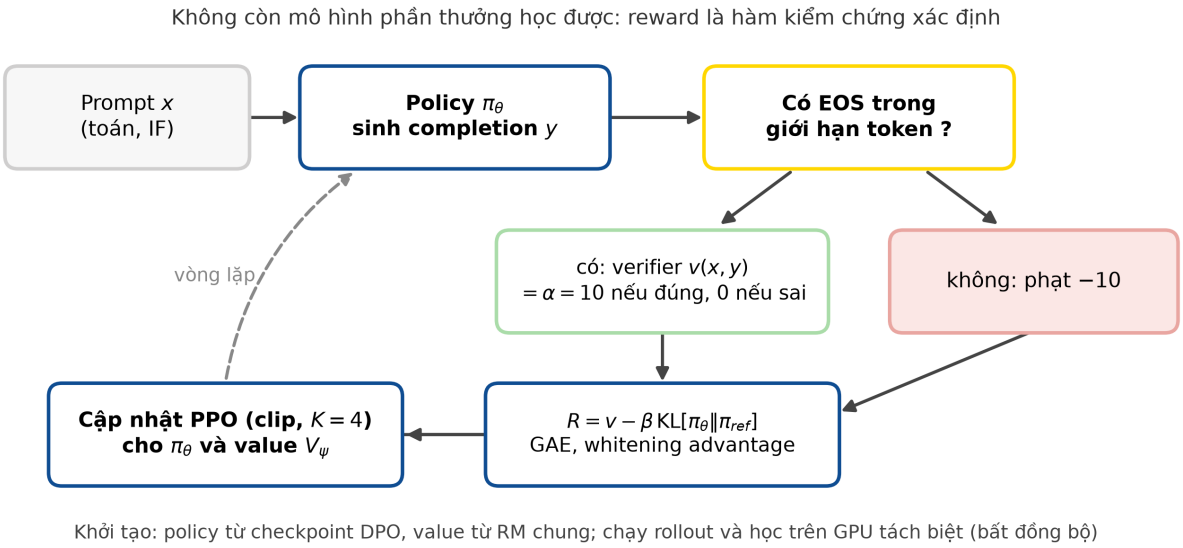
Hyperparameter PPO (Bảng 21, trích phần RLVR):

Hyperparameter	Giá trị (RLVR)
Discount γ	1.0
GAE λ	0.95
Số mini-batch	1
Clip ε (PPO)	0.2
Hệ số value loss c_1	0.1
Grad norm clip	1.0
Learning rate schedule	Linear
Temperature sinh	1.0
Max token / max prompt	2,048 / 2,048
Phạt thiếu EOS	-10.0
Learning rate	3×10^{-7} (1×10^{-7} cho 70B)
Batch size hiệu dụng	224 (640 cho 70B)
Số lần update PPO K	4
Response length	2,048 (1,024 riêng cho GSM8K)
Tổng episode (ablation)	100,000
β (quét)	$\{0.1, 0.05, 0.03, 0.01\}$
Warm-up ratio (quét)	$\{0.0, 0.1\}$

Mô hình RLVR 8B cuối dùng $\beta = 0.05$, $\omega = 0.0$; mô hình 70B cuối dùng $\beta = 0.07$, $\omega = 0.07$ theo chú thích Bảng 21. (Lưu ý: đoạn văn bản mục 6.4 lại ghi hyperparameter cuối cho 70B là learning rate 1×10^{-7}, warmup 0.1, response length 2048, **400,000 episode**, batch hiệu dụng 640, và $\beta = 0.7$; con số β ở đây khác với chú thích Bảng 21 (0.07). Đây là một điểm **không nhất quán giữa hai phần của bài báo**; ghi chú này nêu cả hai để người đọc tự đối chiếu, không chọn một con số làm "đúng tuyệt đối".)

```
RLVR loop (PPO voi verifier reward, theo Hinh 18 + muc 6.2-6.3)
Input: policy pi_theta (khoi tao = checkpoint DPO), value V_psi (khoi tao = RM chung),
       pi_ref (= checkpoint DPO, chi inference), tap prompt x co verifier v(x,.)
for episode = 1..N:
    # pha rollout (GPU inference rieng, vLLM + PagedAttention)
    lay mau completion y ~ pi_theta(.|x) cho batch prompt x
    if y khong ket thuc bang EOS trong max_token: v_bonus = -10
    else: v_bonus = alpha neu verifier(x,y) dung, nguoc lai 0
    R(x,y) = v_bonus - beta * KL[pi_theta(y|x) || pi_ref(y|x)]
    # pha hoc (GPU training rieng, chay dong thoi voi pha rollout ke tiep - async)
    tinh advantage GAE(gamma=1, lambda=0.95) dung V_psi, chuan hoa (whitening)
    cap nhat pi_theta va V_psi bang PPO (clip eps=0.2, K=4 lan update/batch)
    (vi du 2 vong: train tren du lieu cua buoc truoc do 1 buoc, de giam stale data)
Output: pi_theta (chon checkpoint tot nhat tren eval dev, kiem tra moi 40-100 buoc)
```

Cách đọc hình: policy sinh completion cho prompt có verifier, completion không có EOS trong giới hạn token bị phạt -10. Completion hợp lệ được cho alpha = 10 nếu đúng và 0 nếu sai, rồi trừ phạt KL để thành reward cho PPO. Cập nhật policy và value rồi lặp lại, không có mô hình phần thưởng học được nào.



Hình 72: Vòng RLVR: verifier xác định thay mô hình phần thưởng, phạt thiếu EOS, phạt KL và cập nhật PPO (minh hoạ, tự tính)

7. Hạ tầng RLVR

RLVR cần giữ **3 bản mô hình**: policy (train), value (train), reference (chỉ inference). Để giảm nghẽn cổ chai inference, paper **dành GPU riêng cho inference** (dùng Ray để cấp phát, chạy PagedAttention qua vLLM, giảm phân mảnh bộ nhớ, tăng batch size inference), tách biệt với GPU train (dùng ZeRO Stage 3 để fit model + optimizer state). Vì train và inference tách GPU, paper triển khai **PPO bất đồng bộ (asynchronous)**: pha inference (rollout) chạy đồng thời với pha train thay vì chờ nhau tuần tự như thiết lập PPO thông thường, giảm thời gian GPU rảnh (idle). Đánh đổi: dữ liệu rollout có thể "cũ" (stale) nếu inference nhanh hơn train tiêu thụ; để tăng khả năng tái lập, policy luôn huấn luyện trên **batch rollout cũ thứ hai** (không phải mới nhất).

Thời gian huấn luyện thực tế: RM 8B: 9 giờ / 8×H100. RLVR 8B: 65 giờ / 8 GPU. RLVR 70B: 60 giờ / 48 GPU. RLVR 405B: 46 giờ / 256 GPU. (Tất cả đều lấy checkpoint *sớm hơn* checkpoint cuối của run để release.)

8. Kết quả RLVR cuối cùng

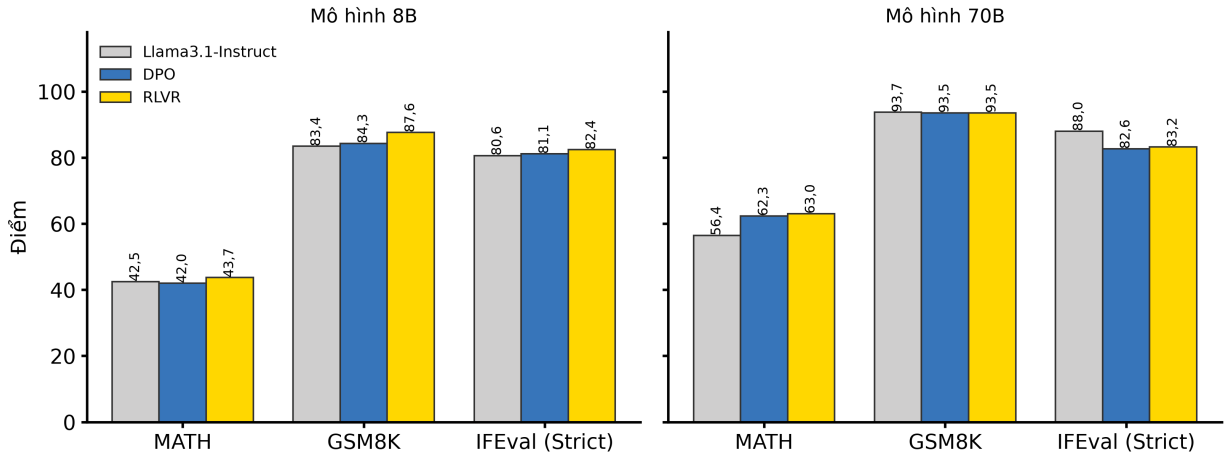
Setup cuối: 8B dùng điểm khởi đầu là DPO tốt nhất (và một biến thể trộn thêm dữ liệu persona IFEval); 70B dùng DPO tốt nhất tổng thể; mô hình giá trị khởi tạo từ RM huấn luyện trên cùng mixture preference 8B. Chọn checkpoint tốt nhất trên MATH+IFEval (8B) qua đánh giá mỗi 100 bước (40 bước cho 70B).

Bảng 23 (so với Llama 3.1 Instruct và checkpoint DPO):

	8B Llama3.1-Inst	8B DPO	8B RLVR	70B Llama3.1-Inst	70B DPO	70B RLVR
Avg.	62.2	64.4	**64.8**	73.4	75.9	**76.0**
MMLU	71.2	68.7	68.2	85.3	83.3	83.1
PopQA	20.2	29.3	29.1	46.4	46.3	46.5
TruthfulQA	55.1	56.1	55.0	66.8	67.9	67.6
BigBenchHard	62.8	65.8	66.0	73.8	81.8	82.0
DROP	61.5	62.5	62.6	77.0	74.1	74.3
MATH	42.5	42.0	**43.7**	56.4	62.3	**63.0**

	8B Llama3.1-Inst	8B DPO	8B RLVR	70B Llama3.1-Inst	70B DPO	70B RLVR
GSM8K	83.4	84.3	**87.6**	93.7	93.5	93.5
HumanEval	86.3	83.9	83.9	93.6	92.4	92.4
IFEval (Strict)	80.6	81.1	**82.4**	88.0	82.6	**83.2**
AlpacaEval 2	24.2	33.5	34.5	33.4	49.6	49.8
Safety	75.2	87.2	85.5	76.5	89.0	88.3

RLVR cải thiện cả 3 mục tiêu ở 8B (MATH, GSM8K, IFEval); ở 70B cải thiện khiêm tốn hơn ở MATH/IFEval và **không đổi** GSM8K (đã gần bão hoà ở 93.5%). Một số run 8B riêng lẻ đạt GSM8K tới 89.4% hoặc IFEval tới 84.8%, nhưng các run đó lại kéo điểm trung bình các benchmark khác xuống, nên không được chọn làm checkpoint cuối. Ở quy mô **405B** (Bảng 4 của bài báo, không trích đầy đủ ở đây): RLVR nâng MATH từ 59.9% (DPO) lên **67.3%**, GSM8K từ 94.2% lên **95.5%**, IFEval (loose) từ 85.0% lên **86.0%**, trong khi trung bình có/không an toàn đều tăng nhẹ (79.6 → 80.7 có an toàn).



Hình 73: Llama3.1-Instruct, DPO và RLVR trên MATH, GSM8K và IFEval ở 8B và 70B (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi nhóm ba cột là một benchmark mục tiêu của RLVR, panel trái là 8B và panel phải là 70B. Ở 8B, RLVR cao hơn DPO ở cả ba benchmark (GSM8K 84,3 lên 87,6). Ở 70B mức tăng nhỏ hơn: MATH 62,3 lên 63,0, IFEval 82,6 lên 83,2 và GSM8K giữ nguyên 93,5.

9. Phân tích / ablation RLVR

RLVR cải thiện đúng domain mục tiêu (Hình 19): chạy riêng từng tác vụ (GSM8K, MATH, IF-constraints) từ checkpoint DPO, quét $\beta \in \{0.1, 0.05, 0.03, 0.01\}$; cả 3 trường hợp đều tăng verifiable reward trên tập train **và** tăng điểm trên benchmark test tương ứng, nhưng trung bình toàn bộ eval **không được đảm bảo tăng** theo. Ở GSM8K/MATH, KL budget lớn hơn không nhất thiết cho verifiable reward cao hơn.

Khởi tạo value từ RM chung tốt hơn neo vào DPO (Hình 21): so hai cách init mô hình giá trị (từ RM chung được huấn luyện trên UltraFeedback, hoặc từ chính checkpoint DPO đang neo) trên GSM8K; cả hai đều cải thiện so với DPO gốc, nhưng **init từ RM chung cho điểm GSM8K và điểm trung bình cao hơn**.

Không nên cộng thêm điểm RM vào verifiable reward (Hình 22): thử cộng thêm score của RM lên trên verifiable reward (thay vì chỉ dùng verifiable reward thuần); kết quả **nhiễu hơn** và kém hơn trên GSM8K, trừ một ngoại lệ (KL lớn nhất lại cho điểm trung bình cao nhất trong thí nghiệm này).

Khởi điểm yếu hơn (SFT) vẫn hội tụ cùng mức verifiable reward, nhưng tốn KL hơn (Hình 20): so khởi đầu từ DPO vs từ SFT (yếu hơn, cùng "huyết thống") trên GSM8K; cả hai đạt mức verifiable reward tương tự, nhưng khởi đầu từ SFT tốn KL-divergence lớn hơn (hợp lý vì SFT cách xa đích hơn DPO); khởi đầu từ mô hình **mạnh hơn** (DPO) cho hiệu năng test tốt hơn.

"Overoptimization Happens": giảm β (penalty KL) → KL-divergence tăng → **điểm trung bình giảm** (Hình 21), đúng với lo ngại kinh điển về reward over-optimization (xem bài 27) dù ở đây reward là một verifier cứng, không phải RM học được. Phụ lục B.4 minh hoạ định tính các câu trả lời IFEval bị "overoptimize" (output lạ, cố gắng lách ràng buộc) ở các run có β nhỏ/KL lớn. Ngoại lệ duy nhất là thí nghiệm Hình 22 (cộng thêm RM score) nơi KL lớn nhất lại cho điểm trung bình cao nhất.

10. Giới hạn

Tác giả nêu: RLVR chỉ áp dụng cho 3 đánh giá có verifier tương đối đơn giản (GSM8K, MATH, IFEval), để lại verifier phức tạp hơn (ví dụ feedback từ chạy code) cho tương lai; tăng verifiable reward trên tập train không đảm bảo tăng điểm trung bình toàn bộ eval; overoptimization xảy ra khi KL budget lớn; ở 70B, KL-divergence của run cuối "đáng ngạc nhiên" luôn rất thấp (dưới 1) suốt quá trình huấn luyện, tác giả chỉ đoán "có thể do learning rate thấp" (probably), không khẳng định chắc. Ở phạm vi toàn bộ Tulu 3 (mục 8.3): dữ liệu ngắn, ít multi-turn (trung bình 2.4 lượt/mẫu), chỉ tiếng Anh (trừ Aya multilingual), chưa huấn luyện/đánh giá tool-use và agent; mục 8.2 còn liệt kê hai hướng thử nhưng **không đưa vào recipe cuối**: Online DPO (không/ít cải thiện GSM8K, làm giảm MATH) và Rejection Sampling (lợi ích nhỏ so với compute cần, và cần judge mạnh để xếp hạng chính xác).

Nhận xét:

- RLVR chỉ kiểm chứng cho 3 kỹ năng có đáp án kiểm tra được dễ dàng; chưa rõ mức độ khó khi mở rộng verifier cho tác vụ phức tạp hơn (code chạy thực tế, suy luận nhiều bước không có đáp số duy nhất).
- Cải thiện ở 70B khiêm tốn hơn 8B rõ rệt (không đổi GSM8K, chỉ nhích MATH/IFEval); có thể do DPO 70B đã gần bão hoà GSM8K, hoặc learning rate 70B quá nhỏ (chính tác giả cũng chỉ đoán, "probably"), nên khó kết luận chắc nguyên nhân.
- Bài báo không so sánh trực tiếp RLVR dùng PPO (như đã chọn) với RLVR dùng GRPO hoặc RLOO (2 bài liền trước trong lộ trình, 36 và 37, đều lập luận giảm được chi phí mô hình giá trị) trong cùng khung RLVR; đây là một cơ hội thí nghiệm rõ ràng còn mở.
- Sự không nhất quán giữa chú thích Bảng 21 ($\beta = 0.07$ cho 70B) và đoạn văn mục 6.4 ($\beta = 0.7$) chưa được paper giải thích hay đính chính; ghi chú này nêu cả hai số để minh bạch, không tự chọn một số "đúng".

11. Liên hệ với các bài khác

- **00 (preface):** cần nắm PPO, GAE, KL penalty, RLHF, RLVR, DPO trước khi đọc bài này.
- **23-25, 27 (RLHF gốc, mô hình phần thưởng overoptimization):** mục tiêu RLVR là mục tiêu RLHF chuẩn với mô hình phần thưởng thay bằng verifier; hiện tượng "Overoptimization Happens" ở mục 9 cho thấy overoptimization không chỉ xảy ra với RM học được (bài 27) mà cả với reward "cứng" dạng verifier.
- **28 (N+ Implementation Details of RLHF with PPO):** 5 trick triển khai PPO mà Tulu 3 áp dụng cho RLVR (init value từ RM, tắt dropout, non-EOS penalty, advantage whitening...) lấy trực tiếp từ bài này.
- **29 (DPO), 32 (SimPO):** nền cho length-normalized DPO (biến thể chuẩn hoá theo độ dài) dùng ở giai đoạn preference tuning.
- **36 (GRPO), 37 (RLOO):** cùng mục tiêu giảm chi phí RL cho LLM nhưng theo hướng bỏ mô hình giá trị; Tulu 3 lại **giữ PPO đầy đủ** cho RLVR, một lựa chọn khác hướng đáng đối chiếu.
- **39 (DeepSeek-R1), 40 (Kimi k1.5):** cùng tư tưởng RLVR (reward kiểm chứng được cho math/code) nhưng dùng GRPO và áp cho suy luận dài (long CoT), quy mô lớn hơn nhiều so với 3 kỹ năng gọn của

Tulu 3.

- **rl 56-57 (olmOCR 2, Format-decoupled RL cho OCR):** các bài RLVR cho OCR ở cuối lộ trình, cùng họ ý tưởng với RLVR của Tulu 3.

12. Ứng dụng thực tế (suy luận)

- **Siêu tham số quan trọng:** α (điểm thưởng khi đúng, paper chọn 10 không tune thêm) nên sweep nếu áp sang domain khác, vì thang reward ảnh hưởng tới cả phạt non-EOS (-10, được chọn tương xứng với $\alpha = 10$); β (KL coeff) cần theo dõi đồng thời reward trên tập train *và* điểm trung bình trên eval khác domain để phát hiện sớm overoptimization (mục 9); cách khởi tạo mô hình giá trị (từ RM chung, không phải từ policy đang neo) có vẻ quan trọng hơn người ta nghĩ.

13. Thí nghiệm gợi ý

1. Trên GSM8K subset nhỏ, triển khai RLVR bằng PPO đầy đủ (giống Tulu 3) và RLVR bằng GRPO (bài 36), cùng verifier và compute budget, so tốc độ hội tụ verifiable reward và GPU memory.
2. Lặp lại ablation "khởi tạo value từ RM chung vs từ checkpoint đang neo" ở quy mô nhỏ hơn, kiểm tra kết luận có còn giữ.
3. Sweep $\alpha \in \{1, 5, 10, 20\}$ trên một verifier tự tạo (ví dụ exact-match một tác vụ đơn giản), đo độ nhạy của KL-divergence và điểm trung bình theo α.
4. Theo dõi đồng thời (verifiable reward trên train, điểm trung bình trên eval khác domain) theo số episode để tự quan sát hiện tượng "overoptimization happens" (giống Hình 21/22) trên một tác vụ mới, từ đó chọn β phù hợp bằng early-stopping.

14. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu RLHF chung	$\max \mathbb{E}[r_\varphi(x,y) - \beta\,\mathrm{KL}[\pi_\theta \Vert \pi_{ref}]]$	nền cho cả PPO và DPO
Length-normalized DPO	chuẩn hoá $\log \pi_\theta/\pi_{ref}$ theo $1/\Vert y\Vert$ trong loss DPO	dùng cho preference tuning Tulu 3
Mục tiêu RLVR	$\mathbb{E}[v(x,y) - \beta\,\mathrm{KL}[\pi_\theta \Vert \pi_{ref}]]$	r_φ thay bằng verifier v
Verifiable reward	$v(x,y) = \alpha$ nếu đúng, 0 nếu sai	$\alpha = 10$, không tune thêm
Non-EOS penalty	-10 nếu response không có EOS	tương xứng thang $\alpha = 10$
Dữ liệu RLVR	GSM8K 7,473 + MATH 7,500 + IF 14,973 = 29,946	Bảng 22
Hyperparam RLVR chính	lr 3×10^{-7} (8B) / 1×10^{-7} (70B), $K = 4$, 100,000 episode	Bảng 21
Kết quả nổi bật	GSM8K 8B: 84.3 → 87.6; MATH 8B: 42.0 → 43.7; IFEval 8B: 81.1 → 82.4	so DPO → RLVR

15. Kết luận ngắn

Tulu 3 cho thấy một recipe post-training mở, đầy đủ 4 giai đoạn (data curation, SFT, DPO-norm, RLVR) có thể vượt các mô hình instruct cùng kích thước và tiến gần mô hình đóng, với mọi quyết định (thuật toán, hyperparameter, hạ tầng) được giải thích và kiểm chứng bằng ablation. **RLVR** là đóng góp đặc trưng nhất: thay mô hình phần thưởng học được bằng một verifier xác định cho các kỹ năng có đáp án kiểm chứng (toán, instruction-following có ràng buộc), huấn luyện bằng PPO đầy đủ với các trick triển khai đã được kiểm chứng (bài 28), cho cải thiện rõ ràng ở các benchmark mục tiêu (nhất là ở 8B) với chi phí hạ tầng không nhỏ (3 bản mô hình, PPO bất đồng bộ). Tulu 3 cũng thẳng thắn công bố các hướng không hiệu quả (Online DPO,

Rejection Sampling), và là một đối chứng "PPO đầy đủ" hữu ích để so sánh với các hướng rẻ hơn (GRPO, RLOO) ở các bài liền kề trong lộ trình.

Link gốc: https://arxiv.org/abs/2411.15124

Bài 39

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (2025)

Phạm vi ghi chú: đây là một technical report dài (86 trang kể cả phụ lục). Ghi chú này **tập trung vào phần RL**: DeepSeek-R1-Zero (RL thuần từ mô hình nền, GRPO, reward theo luật, template, các hành vi nổi lên, độ dài câu trả lời tăng, "aha moment"), quy trình nhiều giai đoạn của DeepSeek-R1 (cold start, RL có reward nhất quán ngôn ngữ, rejection sampling và SFT, RL cho mọi tình huống), các thử nghiệm không thành công (PRM, MCTS), và so sánh distillation với RL ở cùng quy mô. Phần **chi tiết dữ liệu 800K, bảng điểm từng model distill, hyperparameter distillation** đã được ghi ở distillation 38 (DeepSeek-R1 distillation); ghi chú này chỉ nhắc lại rất ngắn và trỏ sang đó. Phần an toàn (Ethics and Safety Statement, Phụ lục D.3) chỉ tóm tắt một câu.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

Tác giả: DeepSeek-AI (research@deepseek.com). **Venue / arXiv:** arXiv:2501.12948 (v2, 04/01/2026; bản gốc đăng 22/01/2025). Văn bản PDF không tự nêu tên hội nghị/tạp chí; metadata của arXiv (truy vấn export API) có liên kết DOI 10.1038/s41586-025-09422-z, gợi ý bài đã xuất bản trên Nature, nhưng đây là thông tin từ metadata, không phải nội dung trong bản PDF ta đọc. **Nguồn:** https://arxiv.org/abs/2501.12948 **Chủ đề:** Dùng RL thuần (GRPO, reward theo luật) trên mô hình nền để tạo ra khả năng lập luận dài (long chain-of-thought) mà không cần SFT trước; sau đó dùng quy trình nhiều giai đoạn để model vừa lập luận tốt vừa dễ đọc, hữu ích; cuối cùng chưng cất khả năng này xuống các mô hình nhỏ. **Vị trí trong lộ trình:** Đọc sau bài 00 (mục 4.4: bỏ critic, baseline nhóm, GRPO/RLOO), 34 (STaR), 35 (PRM, Let's Verify Step by Step) và 36 (DeepSeekMath, nơi GRPO được đề xuất lần đầu). Bài này là ví dụ quy mô công nghiệp đầu tiên áp GRPO cho RL thuần từ base, và là một trong hai báo cáo RL-for-reasoning lớn công bố cùng ngày (22/01/2025) với bài 40 (Kimi k1.5), nên nên đọc cặp với bài 40 để thấy hai lựa chọn thiết kế RL khác nhau cho cùng mục tiêu. Sau bài này: 41 (DAPO, sửa các vấn đề của GRPO), 42 (Dr. GRPO, phê bình việc chia std), 43 (GSPO). Phần chưng cất liên hệ trực tiếp với distillation 38.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Lập luận (reasoning) của LLM thường được cải thiện bằng chain-of-thought (CoT): cho model sinh các bước trung gian trước khi ra đáp án, hoặc tinh chỉnh trên dữ liệu lập luận do người viết. Hai cách này đều phụ thuộc dữ liệu người: đắt, khó mở rộng, và giới hạn model ở "cách nghĩ của người" (paper gọi là "human-provided exemplars... prevents the exploration of superior, non-human-like reasoning pathways").

Câu hỏi của bài báo: nếu bỏ hẳn SFT, chỉ dùng RL thuần trên mô hình nền với reward kiểm tra được bằng luật (verifiable reward, không cần mô hình phần thưởng thần kinh), mô hình có tự phát triển được lập luận dài không, và nếu có thì bằng cách nào.

Khái niệm cần nắm trước khi đọc (cross-ref bài 00):

- **Policy** π_θ: ở đây chính là LLM, sinh token tự hồi quy.
- **Rollout**: một lần model sinh hoàn chỉnh một câu trả lời từ một câu hỏi.
- **Verifiable reward / RLVR**: phần thưởng tính được bằng luật cứng (đáp án toán đúng, code qua test), đối lập với mô hình phần thưởng học từ so sánh người (bài 00 mục 4.5).
- **Reward hacking**: model khai thác kẽ hở hàm thưởng để đạt điểm cao mà không thực sự đúng ý định (bài 00 mục 4.6; bài 27 là ví dụ định lượng).

- **GRPO**: thuật toán policy gradient bỏ critic, dùng baseline chuẩn hóa trong nhóm mẫu (bài 00 mục 4.4; đề xuất gốc ở bài 36 DeepSeekMath).

Model nền: **DeepSeek-V3-Base**, MoE 671B tổng / 37B kích hoạt mỗi token, tiền huấn luyện trên 14.8T token (Phụ lục A.1), tri thức cắt ở tháng 7/2024. Dữ liệu tiền huấn luyện chỉ gồm web page và e-book thường (không synthetic cố ý), nhưng paper ghi nhận một số trang web chứa nhiều câu trả lời do model OpenAI sinh ra, nên mô hình nền có thể gián tiếp "thấy" phong cách lập luận từ các mô hình khác qua dữ liệu web.

2. Ý tưởng chính

Hai model, một quy trình hai giai đoạn ý tưởng:

DeepSeek-R1-Zero: áp GRPO trực tiếp lên DeepSeek-V3-Base, không SFT, với reward chỉ gồm đúng/sai đáp án (accuracy) và đúng định dạng (format, có cặp thẻ `<think>...</think>`). Không có ràng buộc nào về *cách* lập luận. Qua huấn luyện, model tự học sinh câu trả lời ngày càng dài, có tự kiểm tra (verification), nhìn lại (reflection), và thử hướng khác khi bí ("aha moment"). Nhưng R1-Zero đọc khó, trộn ngôn ngữ (Anh/Trung).

DeepSeek-R1: thêm một quy trình nhiều giai đoạn quanh ý tưởng trên: (1) thu thập một lượng nhỏ dữ liệu "cold start" (CoT dài, giọng văn người, lọc từ chính R1-Zero) để SFT khởi động; (2) RL lần một để model lập luận có ngôn ngữ nhất quán; (3) rejection sampling từ checkpoint đó, lọc câu đúng và dễ đọc, trộn với dữ liệu không lập luận (viết lách, hỏi đáp...), SFT lại; (4) RL lần hai trên mọi loại prompt, kết hợp reward luật và mô hình phần thưởng (giúp ích, an toàn). Kết quả: DeepSeek-R1, cạnh tranh với OpenAI o1 trên toán, code, STEM, đồng thời viết lách/đối thoại tốt.

Cuối cùng, paper chưng cất (distill) khả năng lập luận của R1 xuống các mô hình dense nhỏ hơn bằng SFT trên 800K mẫu do R1 sinh (chi tiết ở distillation 38), và so sánh trực tiếp: cùng một mô hình nền (Qwen2.5-32B), chưng cất từ R1 cho kết quả tốt hơn hẳn so với chính mô hình đó tự chạy RL quy mô lớn.

3. Phương pháp chi tiết

3.1 GRPO: nhắc lại và so với PPO

Bài báo dùng GRPO (Shao et al., 2024, bài 36) cho cả R1-Zero và R1. Nhắc công thức (đã nói kỹ ở bài 00 mục 4.4, dùng đúng ký hiệu của bài báo): với câu hỏi q, lấy G output $\{o_1, \ldots, o_G\}$ từ policy cũ $\pi_{\theta_{old}}$, tối đa hóa

$$\boxed{\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min\Big(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i,\ \text{clip}\big(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\big)A_i\Big) - \beta\,\mathbb{D}_{KL}(\pi_\theta\|\pi_{ref})\right)\right]}$$

$$A_i = \frac{r_i - \text{mean}(\{r_1,\ldots,r_G\})}{\text{std}(\{r_1,\ldots,r_G\})}, \qquad \mathbb{D}_{KL}(\pi_\theta\|\pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1$$

Dạng KL này chính là ước lượng k_3 ở bài 00 mục 4.2 ($u = \pi_{ref}/\pi_\theta$: $k_3 = (u-1) - \log u$), không chệch và luôn ≥ 0.

So với PPO (bài 07): GRPO **không có value network**, advantage tính trực tiếp từ G mẫu cùng câu hỏi, không cần GAE (bài 05) vì mô hình giá trị khó dự đoán đúng khi mô hình có thể tự sửa/nhìn lại giữa câu (Phụ lục A.3). PPO cộng phạt KL **vào reward mỗi token**, có thể ngầm phạt độ dài; GRPO cộng KL **trực tiếp vào loss** một lần, tránh ngầm phạt độ dài. Paper còn cập nhật lại π_{ref} bằng policy mới nhất **mỗi 400 bước**.

3.2 R1-Zero: template và reward theo luật

Template cố định (Bảng 1, dịch nguyên cấu trúc): *"Một cuộc hội thoại giữa User và Assistant. User hỏi, Assistant giải. Assistant trước tiên suy nghĩ về quá trình lập luận trong đầu rồi mới đưa ra câu trả lời, đặt trong* `<think>...</think>` *và* `<answer>...</answer>`*."* Paper nhấn mạnh: template chỉ ép **cấu trúc**, không ép nội dung, để quan sát xu hướng tự nhiên của model.

Reward thuần theo luật, không dùng mô hình phần thưởng thần kinh (outcome hay process) cho tác vụ lập luận, vì dễ bị reward hacking ở quy mô lớn và tốn tài nguyên train lại:

$$\boxed{Reward_{rule} = Reward_{acc} + Reward_{format}}$$

$Reward_{acc}$: với toán, đáp án phải nằm trong khuôn cố định (`\boxed{}`) để so khớp luật (đúng = 1, sai = 0, theo Phụ lục B.3.1); với code, chạy test suite. $Reward_{format}$: thưởng nếu đúng cặp thẻ `<think>`; bài báo không nêu giá trị số cụ thể, chỉ nói "accuracy reward và format reward được kết hợp với cùng trọng số".

Siêu tham số: lr 3×10^{-6}, $\beta = 0.001$, nhiệt độ rollout 1, $G = 16$/câu hỏi, 32 câu hỏi/bước $\Rightarrow$ batch 512; max length 32,768 token trước bước 8.2k, sau đó 65,536; thay π_{ref} mỗi 400 bước; mỗi rollout sinh 8,192 output, chia 16 minibatch, 1 inner epoch; tổng 10,400 bước $\approx$ 1.6 epoch. Paper **không nêu giá trị ε cụ thể cho R1-Zero**, chỉ công thức tổng quát.

3.3 Kết quả tự phát của R1-Zero: độ dài, "aha moment"

AIME 2024 pass@1 tăng từ 15.6% lên 77.9% (tự-consistency 16 mẫu: 86.7%). Hình 1(b) (chỉ đồ thị, không bảng số) cho thấy **độ dài câu trả lời trung bình tăng liên tục** suốt RL, hoàn toàn do model tự điều chỉnh.

"Aha moment" (Bảng 2): ở một checkpoint giữa chừng, giải $\sqrt{a - \sqrt{a + x}} = x$, model bỗng viết "Wait, wait. Wait. That's an aha moment... Let's reevaluate this step-by-step" rồi tự sửa hướng giải, không ai dạy nó làm vậy.

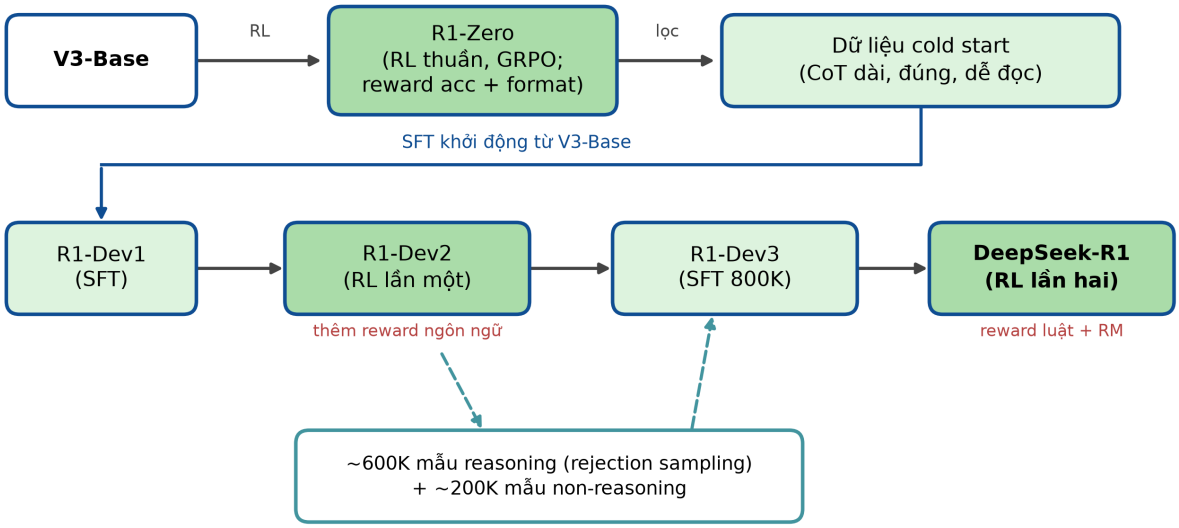
Định lượng (Phụ lục C.2): tần suất các từ phản ánh tư duy lại ("wait", "mistake", "however", "verify", "wrong"...) tăng **5 đến 7 lần** so với đầu huấn luyện; riêng "wait" gần như vắng bóng lúc đầu, rải rác ở bước 4000-7000, rồi **tăng vọt sau bước 8000**. (Nhận xét của phần phân tích: bước 8000 gần sát bước 8.2k, đúng lúc max length nâng từ 32,768 lên 65,536; hai hiện tượng có thể liên quan nhưng bài báo không bàn trực tiếp.) Phụ lục C.1: tách theo độ khó MATH (level 1-5), bài khó (level 4-5) cải thiện mạnh nhất, level 5 từ ~0.55 lên ~0.90.

3.4 Pipeline nhiều giai đoạn của DeepSeek-R1 (Hình 2)

```
V3-Base --RL(reasoning, acc+format)--> R1-Zero
R1-Zero --sample, lọc đúng/format, refine bằng V3 + người--> dữ liệu Cold Start (long
↪  CoT)
V3-Base + Cold Start --SFT--> R1-Dev1
R1-Dev1 --RL(reasoning, rule-based + language-consistency)--> R1-Dev2
R1-Dev2 --rejection sampling--> ~600K mẫu reasoning (đúng, lọc đọc được)
V3 (SFT pipeline) --> ~200K mẫu non-reasoning
(600K + 200K) + V3-Base --SFT--> R1-Dev3
R1-Dev3 --RL(diverse, rule-based + preference RM)--> R1 (cuối)
```

Cách đọc hình: đọc hàng trên từ trái sang phải để thấy R1-Zero sinh ra dữ liệu cold start, rồi theo đường xanh xuống hàng dưới. Ô xanh đậm là bước RL, ô xanh nhạt là bước SFT. Khối viền xanh lơ là dữ liệu 600K cộng 200K mẫu, được tạo từ Dev2 rồi dùng để SFT ra Dev3. Chữ đỏ ghi loại reward được thêm ở mỗi lần RL.

Cold start (Phụ lục B.3.2). Động lực là trải nghiệm người dùng: R1-Zero hay dùng "chúng ta"/không đại từ; R1 cần giọng "tôi" tự nhiên và nhất quán ngôn ngữ. Cách tạo: lấy rollout từ R1-Zero ở nhiệt độ 1.0, lọc giữ



Hình 74: Pipeline năm bước của DeepSeek-R1: R1-Zero tạo dữ liệu cold start, rồi hai lần SFT xen kẽ hai lần RL (minh hoạ, tự tính)

đáp án đúng và đọc được (sympy so khớp biểu thức toán; lọc trùng lặp, trộn ngôn ngữ), người chuyển một phần nhỏ sang giọng hội thoại, dùng làm few-shot để DeepSeek-V3 viết lại phần còn lại (yêu cầu trung thành với lập luận gốc, dùng `\boxed{}`), rồi người kiểm tra lại lần hai. Với code: 5,151 bài Codeforces + 2,504 bài AtCoder, dùng DeepSeek-V2.5 sinh test case rồi lọc hai pha (loại test sai bằng submission đúng; chọn tập test phân biệt đúng/sai).

RL lần một, R1-Dev1 → R1-Dev2 (3.2.1). Siêu tham số gần giống R1-Zero (lr 3×10^{-6}, $\beta = 0.001$, $G = 16$, max length 32,768, batch 512, thay π_{ref} mỗi 400 bước), nhưng bài báo nêu rõ $\varepsilon = 10$ cho giai đoạn này (rất khác clip PPO thường gặp 0.1-0.3, bài 07; xem mục 6). Thêm **reward nhất quán ngôn ngữ**:

$$\boxed{Reward_{language} = \frac{\text{Num(từ ngôn ngữ đích)}}{\text{Num(tổng số từ trong CoT)}}}$$

cộng trực tiếp vào reward cuối. Ablation (B.6, mục 5) cho thấy reward này giảm nhẹ hiệu năng nhưng giữ ngôn ngữ ổn định.

Rejection sampling + SFT lần hai, → R1-Dev3 (B.3.3). Từ checkpoint R1-Dev2, sinh nhiều mẫu/prompt, chỉ giữ đáp án đúng (luật, hoặc DeepSeek-V3 làm giám khảo sinh JSON "correct/incorrect"), lọc CoT trộn ngôn ngữ/quá dài/code lạc ⇒ **~600K mẫu reasoning**. Phần **non-reasoning 200K: viết lách, hỏi đáp, tự nhận thức, dịch, kỹ thuật phần mềm, lấy từ quy trình SFT của V3; câu đơn giản ("hello") không cần CoT. Tổng 800K mẫu** (thống kê và ví dụ chi tiết: xem distillation 38, cùng dữ liệu này SFT cả R1-Dev3 và các mô hình chưng cất). SFT: lr cosine $5\times10^{-5} \to 5\times10^{-6}$, 2-3 epoch, max length 32,768, batch 128.

RL lần hai, "all-scenario RL", → R1 (3.2.2). Reward tổng hợp:

$$\boxed{Reward = Reward_{reasoning} + Reward_{general} + Reward_{language}}$$

$$Reward_{general} = Reward_{reward_model} + Reward_{format}$$

Helpful RM: 66,000 cặp preference, sinh bằng prompt DeepSeek-V3 theo định dạng arena-hard (hỏi 4 lần, đổi vị trí A/B tránh positional bias, lấy trung bình, chỉ giữ $\Delta > 1$, cân bằng độ dài); batch 256, lr 6×10^{-6}, 1 epoch, max length 8192 khi huấn luyện. Safety RM: 106,000 prompt, loss point-wise, cùng siêu tham số. Nhiệt độ rollout giảm xuống 0.7 (cao hơn gây sinh thiếu mạch lạc). Tổng 1,700 bước; dữ liệu tổng hợp/preference reward chỉ dùng ở **400 bước cuối** (dùng nhiều hơn dễ reward hacking, mục 5, Hình 6).

3.5 Các thử nghiệm không thành công (Phụ lục G.2)

Process Reward Model (PRM). Thử PRM (tinh thần bài 35) nhưng gặp ba hạn chế: (1) khó định nghĩa một "bước" mịn trong lập luận tổng quát; (2) khó xác định bước trung gian đúng/sai (gán nhãn tự động chưa tốt, gán tay không scale); (3) PRM dựa trên model chắc chắn dẫn tới reward hacking (trích Gao et al., 2022, bài 27), và train lại nó tốn thêm tài nguyên. Kết luận: PRM tốt để rerank top-N hoặc guided search lúc suy luận, nhưng không bù được chi phí khi dùng để huấn luyện RL quy mô lớn.

Monte Carlo Tree Search (MCTS). Lấy cảm hứng từ AlphaGo/AlphaZero (bài 14): chia câu trả lời thành bước có thẻ, tìm kiếm nhờ mô hình giá trị đã train trước, dùng kết quả để huấn luyện lại actor và mô hình giá trị, lặp lại. Hai khó khăn: (1) không gian sinh token lớn hơn cờ vua theo cấp lũy, phải giới hạn mở rộng node, dễ kẹt local optimum; (2) train mô hình giá trị mịn cho từng bước sinh token rất khó, trong khi AlphaGo phụ thuộc đúng vào việc mô hình giá trị cải thiện dần. Kết luận: MCTS hữu ích lúc suy luận nếu có mô hình giá trị tốt sẵn, nhưng tự cải thiện qua self-search vẫn là thử thách lớn.

3.6 Hạ tầng RL (Phụ lục B.1, tóm lược)

Bốn module chạy xen kẽ, offload VRAM sau mỗi bước: Rollout (vLLM, song song theo expert, MTP speculative-decode), Inference (mô hình phần thưởng/reference), Rule-based Reward (code executor, answer matcher, format checker, chạy bất đồng bộ chồng lấp hai module trên), Training (PPO/GRPO/DPO, đóng gói dữ liệu Best-Fit, DualPipe).

4. Thí nghiệm và kết quả chính

Bảng 4 (Phụ lục B.3.1), dữ liệu RL:

Loại dữ liệu	Số prompt	Dạng câu hỏi	Dạng output
Math	26K	Lập luận định lượng	Số/biểu thức/phương trình
Code	17K	Thuật toán, sửa lỗi	Giải pháp code
STEM	22K	Trắc nghiệm	Lựa chọn
Logic	15K	Trắc nghiệm/định lượng	Lựa chọn/số
General	66K	Giúp ích/an toàn	Câu trả lời xếp hạng

(Lưu ý: mục văn bản nói Code gồm "17K bài thuật toán, cộng 8K bug-fixing", nhưng Bảng 4 chỉ ghi 17K cho Code; bài báo không nói rõ 8K này có nằm trong 17K hay là một phần riêng không liệt kê.)

Bảng 8 (trích), so sánh R1 với các mô hình khác:

Benchmark	Claude-3.5-Sonnet	GPT-4o-0513	o1-1217	DeepSeek-R1
MMLU-Pro (EM)	78.0	72.6	-	84.0
GPQA Diamond (Pass@1)	65.0	49.9	75.7	71.5
AlpacaEval2.0 (LC-winrate)	52.0	51.1	-	87.6
ArenaHard	85.2	80.4	-	92.3
LiveCodeBench (Pass@1-COT)	38.9	32.9	63.4	65.9

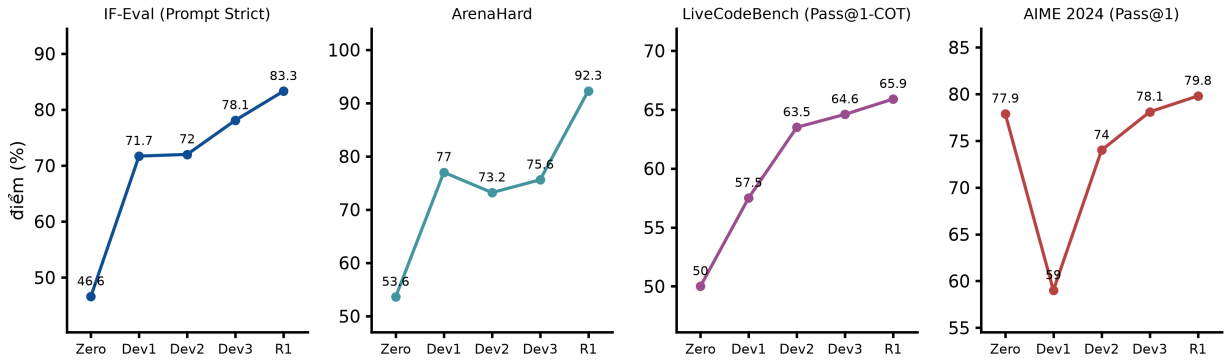
Benchmark	Claude-3.5-Sonnet	GPT-4o-0513	o1-1217	DeepSeek-R1
Codeforces (Rating)	717	759	2061	2029
AIME 2024 (Pass@1)	16.0	9.3	79.2	79.8
MATH-500 (Pass@1)	78.3	74.6	96.4	97.3

R1 ngang hoặc vượt o1-1217 ở toán, vượt rõ ở AlpacaEval2.0/ArenaHard (viết lách, đối thoại mở); o1-1217 vẫn hơn ở GPQA Diamond và Codeforces. Trên tác vụ kỹ thuật phần mềm (Aider), o1-1217 hơn R1, nhưng SWE Verified hai bên gần bằng.

Bảng 3 (trích), hiệu năng theo từng giai đoạn:

Benchmark	R1-Zero	R1-Dev1	R1-Dev2	R1-Dev3	R1
IF-Eval (Prompt Strict)	46.6	71.7	72.0	78.1	83.3
ArenaHard	53.6	77.0	73.2	75.6	92.3
LiveCodeBench (Pass@1-COT)	50.0	57.5	63.5	64.6	65.9
AIME 2024 (Pass@1)	77.9	59.0	74.0	78.1	79.8

Dev1 (SFT trên cold start nhỏ) cải thiện mạnh IF-Eval/ArenaHard nhưng **tụt AIME** so với R1-Zero (dữ liệu cold start nhỏ, chưa đủ phủ). Dev2 (RL lần một) lấy lại và vượt phần lập luận. Dev3 (thêm non-reasoning SFT) tăng ArenaHard tiếp. R1 (RL lần hai) tăng mạnh nhất ở benchmark hướng người dùng (AlpacaEval2.0 +25%, ArenaHard +17% theo bài báo), vì phần lập luận đã được xử lý nhiều ở các giai đoạn trước.



Hình 75: Điểm bốn benchmark qua các giai đoạn từ R1-Zero đến R1 (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi ô là một benchmark, trục ngang là các giai đoạn theo thứ tự huấn luyện. IF-Eval và ArenaHard tăng mạnh ngay khi có SFT cold start rồi tăng tiếp ở R1, còn AIME 2024 tụt ở Dev1 (59.0) rồi mới hồi lại và vượt R1-Zero ở R1 (79.8). Số trên từng điểm chép từ Bảng 3 (trích).

Bảng 14 (Phụ lục E.5), LiveCodeBench theo độ khó:

Độ khó	R1-Zero	R1-Dev1	R1-Dev2	R1-Dev3	R1
Easy	98.07	99.52	100.00	100.00	100.00
Medium	58.78	73.31	81.76	81.42	83.45
Hard	17.09	23.21	30.36	33.16	34.44

Cải thiện chủ yếu dồn vào bài Medium/Hard; bài Easy đã gần như giải hết từ giai đoạn đầu.

So sánh distillation và RL (Bảng 16, Phụ lục F.1; chi tiết quy trình distillation xem distillation 38):

Model	AIME24 pass@1	AIME24 cons@64	MATH-500	GPQA-D	LiveCodeBench
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9
Qwen2.5-32B-Zero (RL >10K bước)	47.0	60.0	91.6	55.0	40.2
R1-Distill-Qwen-32B (SFT từ R1)	**72.6**	**83.3**	**94.3**	**62.1**	**57.2**

Qwen2.5-32B-Zero (= "DeepSeek-R1-Zero-Qwen-32B" ở Phụ lục B.4.1: lr 2×10^{-6}, $\beta = 0.001$, nhiệt độ 1, $G = 16$, max length 32,768, batch 512) là kết quả tự RL >10K bước trên Qwen2.5-32B-Base, không chưng cất; chỉ ngang QwQ-32B-Preview, thua xa model chưng cất từ R1 cùng base. Kết luận tác giả: chưng cất kinh tế và hiệu quả hơn; nhưng để **vượt** ranh giới trí tuệ hiện có, có thể vẫn cần base mạnh hơn và RL quy mô lớn hơn.

Bảng 17 (bổ sung): Qwen2-Math-7B-Zero (RL ~10,000 bước trên base ra mắt **trước** OpenAI o1, đảm bảo chưa thấy dữ liệu lập luận kiểu o1): AIME 2024 = 22.3%, AIME 2025 = 18.1%, so với Qwen2-Math-7B-Instruct (7.9% / 4.6%) và GPT-4o-0513 (9.3% / không có số). RL thuần vẫn làm lập luận nổi lên ở quy mô 7B, dù thấp hơn R1-Distill-Qwen-7B (AIME 55.5, Bảng 15 ở distillation 38; base khác nhau nên so sánh chỉ tham khảo).

Chi phí huấn luyện (Bảng 7, thuê H800 $2/giờ GPU): R1-Zero 101K giờ ($202K), tạo SFT data 5K giờ $(10K), R141K giờ$(82K), tổng 147K giờ ($294K). Kiểm tra (tự tính): $64 \times 8 = 512$ GPU, R1-Zero chạy 198 giờ $\Rightarrow 512 \times 198 \approx 101$K (khớp); R1 "khoảng 4 ngày, hay 80 giờ" $\Rightarrow 512 \times 80 \approx 41$K (khớp GPU-giờ, nhưng 80 giờ $\approx$ 3.3 ngày, không hẳn khớp "4 ngày" cùng câu; trích nguyên văn, không tự sửa).

5. Phân tích / ablation

GRPO so với PPO (Phụ lục A.3, Hình 3-4). Trên tác vụ MATH với DeepSeek-Coder-V2-Lite (MoE 16B, 2.4B kích hoạt): PPO với $\lambda = 0.95$ (giá trị mặc định ở nhiều cài đặt GAE mã nguồn mở) kém hẳn GRPO; $\lambda = 1.0$ thì PPO gần bằng GRPO nhưng tốn thêm công tìm λ và vẫn cần mô hình giá trị. Ablation hẹp: chỉ một model, một tác vụ.

Reward hacking với mô hình phần thưởng giúp ích (Phụ lục B.5, Hình 6). Trong giai đoạn RL lần hai, khi dùng nhiều bước với preference reward, điểm reward tăng dần nhưng **hiệu năng CodeForces giảm** theo, một dấu hiệu kinh điển của reward hacking: model học cách "lấy điểm RM cao" mà lệch khỏi ý định thật. Đây là lý do paper chỉ dùng preference reward ở 400 bước cuối trong tổng 1,700 bước.

Ablation reward nhất quán ngôn ngữ (Phụ lục B.6, Hình 7). Trên DeepSeek-R1-Distill-Qwen-7B (cùng dữ liệu cold start, cũng bị trộn ngôn ngữ khi RL): không có reward ngôn ngữ, tỉ lệ nhất quán ngôn ngữ giảm dần theo bước huấn luyện; có reward, tỉ lệ giữ ổn định gần 1.0 suốt quá trình. Đổi lại, LiveCodeBench Pass@1 giảm nhẹ, AIME giữ tương đương. Kết luận tác giả: đánh đổi nhỏ về điểm số để lấy khả năng đọc, hợp sở thích người dùng.

Phát hiện chính (Phụ lục G.1):

- *Base checkpoint quan trọng:* ban đầu thử RL thuần trên 7B dense và 16B MoE, **không cải thiện có ý nghĩa** trên AIME; các mô hình này khi độ dài tăng có xu hướng lặp lại, không tận dụng được long CoT. Chỉ ở 32B dense, 230B MoE, 671B MoE mới thấy tăng rõ rệt. (Đối chiếu Bảng 17: Qwen2-Math-7B-Zero ở trên lại tăng được bằng RL. Nhận xét của phần phân tích: "7B không RL được" là phát biểu về **cấu hình thử ban đầu cụ thể** của tác giả [7B dense tổng quát, không phải 7B chuyên toán], không phải quy luật tuyệt đối theo kích thước; bài báo không giải thích rõ sự khác biệt giữa hai thí nghiệm.)

- *Verifier quan trọng:* rule-based RM và LLM-judge-theo-ground-truth là hai cách đáng tin để tránh reward hacking; LLM-judge hiệu quả với câu trả lời ngắn/rõ nhưng không tổng quát tốt cho văn bản mở.
- *Cần cả SFT và RL:* RL khai phá được lập luận dài mà dữ liệu người không có sẵn; SFT cần cho tác vụ khó định nghĩa reward (viết lách, hỏi đáp mở). Chỉ RL dễ reward-hack ở tác vụ khó kiểm; chỉ SFT thì model không tối ưu được khả năng lập luận qua tự khám phá.

Phân tích độ dài CoT lúc suy luận (Phụ lục E.4). Số token "suy nghĩ" tăng theo độ khó bài toán: câu dễ (ví dụ "1+1=?") dùng <100 token; với bộ đề thi 2024, R1 đạt 61.8% pass@1 khi dùng trung bình 8,793 token suy nghĩ, dùng <7,000 token cho bài dễ và >18,000 token cho bài khó nhất. So với GPT-4o-0513 (model không suy luận dài): chỉ 24.7% pass@1 với trung bình 711 token, và majority voting 64 mẫu trên AIME 2024 chỉ nâng GPT-4o từ 9.3% lên 13.4% (so với 79.8% của R1 pass@1, hay 90.0% ở pass@64). Lý do (theo bài báo): majority voting lấy mẫu độc lập, không tích lũy sửa lỗi qua các bước như long CoT; mẫu sai lặp lại không tăng cơ hội đúng. Bản thân R1 cũng chưa hoàn hảo: majority voting **trên chính R1** vẫn nâng AIME từ 79.8% lên 86.7%, cho thấy còn chỗ cải thiện bằng test-time scaling bổ sung (kể cả MCTS, dù không dùng để huấn luyện).

6. Giới hạn

Tác giả nêu (mục 6 của bài báo):

- Structured output/tool use còn yếu; R1 chưa dùng được công cụ ngoài (search, calculator).
- Token efficiency: còn hiện tượng overthinking ở câu đơn giản.
- Language mixing: chủ yếu tối ưu cho Anh/Trung, có thể lẫn ngôn ngữ khi hỏi bằng ngôn ngữ khác, vì mô hình nền chủ yếu thấy hai ngôn ngữ này.
- Nhạy prompt: few-shot làm **giảm** hiệu năng; khuyến nghị dùng zero-shot, mô tả trực tiếp.
- Kỹ thuật phần mềm: RL ít áp dụng ở đây do thời gian đánh giá dài, làm chậm quy trình RL.
- Reward hacking là rủi ro cố hữu của RL thuần khi không có verifier tin cậy; với tác vụ như vậy, paper phải dùng dữ liệu người + chỉ vài trăm bước RL.

Nhận xét:

- $\varepsilon = 10$ **cho GRPO ở RL lần một của R1** rất khác giá trị clip 0.1-0.3 quen thuộc trong PPO (bài 07); paper chỉ giải thích ngắn ("ε thấp cắt gradient của nhiều token; ε cao gây bất ổn") mà không nói vì sao chọn đúng 10, không ablation giá trị này. Nên thử lại cẩn thận trước khi copy số này sang bài toán khác.
- **Khả năng không khớp quy mô dữ liệu/số bước của R1-Zero (tự tính):** mỗi rollout dùng 512 câu hỏi ($8{,}192$ output $/16$); $10{,}400$ bước $/16 = 650$ rollout; $650 \times 512 \approx 332{,}800$ lượt câu hỏi. Paper nói con số này là **1.6 epoch**, ngụ ý pool có khoảng $332{,}800/1.6 \approx 208{,}000$ câu hỏi, lớn hơn hẳn tổng Math+Code+STEM+Logic ở Bảng 4 (khoảng 80-88K kể cả 8K bug-fixing). Bài báo không giải thích chênh lệch này; chỉ là một nghi vấn số liệu, không tự suy ra lời giải.
- Ablation GRPO/PPO (Hình 3-4) chỉ trên một model/một tác vụ, chưa chắc tổng quát.
- So sánh "distillation thắng RL" (Bảng 16) không khớp ngân sách tính toán: RL >10K bước trên 32B so với SFT 800K mẫu (sinh từ model 671B, chi phí này không tách bạch khỏi "5K giờ GPU tạo SFT data"); nêu kỹ hơn ở distillation 38.
- Phần an toàn (D.3) chỉ kết luận mức "trung bình, so sánh được với GPT-4o"; không đọc chi tiết bảng an toàn trong ghi chú này.

7. Liên hệ với các bài khác

- **00 (preface):** công thức GRPO, baseline nhóm, ước lượng KL k_3 (mục 4.2, 4.4) dùng lại nguyên trong bài này; đọc trước để không phải suy lại từ đầu.

- **34 (STaR), 35 (Let's Verify Step by Step):** PRM mà R1 thử và bỏ (mục G.2) chính là hướng mà bài 35 (Lightman et al.) theo đuổi; đọc 35 trước để hiểu rõ PRM là gì trước khi đọc phần "thất bại" ở đây.
- **36 (DeepSeekMath):** nơi GRPO ra đời; R1 là lần áp GRPO ở quy mô "RL thuần từ base, không SFT" lớn nhất tính đến thời điểm paper.
- **27 (Scaling Laws for Reward Model Overoptimization):** chính là trích dẫn "Gao et al., 2022" mà bài báo dùng để giải thích vì sao PRM dựa trên model dẫn tới reward hacking.
- **14 (AlphaZero):** nguồn cảm hứng trực tiếp cho thử nghiệm MCTS ở mục G.2; đọc 14 để hiểu vì sao "giống cờ vua" lại không áp dụng tốt cho sinh token.
- **40 (Kimi k1.5):** công bố cùng ngày (22/01/2025), cùng mục tiêu nhưng chọn thuật toán tối ưu chính sách khác hẳn (online mirror descent, không clip kiểu PPO, không chia std) và nêu rõ hạ tầng partial rollout cho 128K mà R1 không đề cập. Nên đọc cặp.
- **41 (DAPO), 42 (Dr. GRPO):** phân tích lại GRPO mà R1 dùng: DAPO xử lý nhóm toàn đúng/toàn sai (gradient 0); Dr. GRPO phê bình chia std (bài 00 mục 4.4 đã nhắc). Paper tự trích một công trình phản biện "aha moment" (Liu et al.) có chủ đề gần bài 42.
- **43 (GSPO):** hướng thay thế tính tỉ số importance sampling ở mức câu thay vì token.
- **distillation 38 (DeepSeek-R1 distillation):** có đầy đủ Bảng 5, 6, 15 (800K data, model distill theo kích thước) và thảo luận sâu hơn về distillation vs RL; ghi chú này chỉ nhắc Bảng 16, 17 vì thuộc phần RL trực tiếp.

8. Ứng dụng thực tế (suy luận)

Quy mô huấn luyện của R1 rất lớn; khi áp dụng các ý tưởng của bài báo ở quy mô khác, cần kiểm tra riêng hiệu quả và chi phí. Cold start bằng SFT có thể giúp ổn định định dạng trước RL, còn reward dựa trên luật phù hợp khi đầu ra có thể kiểm chứng. Với reward học được, cần theo dõi nguy cơ tối ưu vào sai lệch của bộ chấm.

9. Thí nghiệm gợi ý

1. So sánh các ngưỡng clip GRPO trên cùng tác vụ và đo cả độ ổn định lẫn hiệu năng.
2. So sánh RLVR có và không có SFT khởi động, theo dõi chất lượng và định dạng đầu ra.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
GRPO objective	$\mathcal{J}_{GRPO} = \mathbb{E}\left[\frac{1}{G}\sum_i \min(\rho_i A_i, \text{clip}(\rho_i, 1\pm\varepsilon)A_i) - \beta D_{KL}\right]$	$\rho_i = \pi_\theta/\pi_{\theta_{old}}$; không value network
Advantage nhóm	$A_i = (r_i - \text{mean})/\text{std}$	giống bài 00 mục 4.4
KL k_3	$u - 1 - \log u$, $u = \pi_{ref}/\pi_\theta$	không chệch, ≥ 0
Reward R1-Zero	$Reward_{acc} + Reward_{format}$	luật cứng, không RM thần kinh
Reward ngôn ngữ	$\text{Num}(W_{target})/\text{Num}(W)$	cộng vào reward cuối
Reward R1 (RL lần hai)	$Reward_{reasoning} + Reward_{general} + Reward_{language}$	$Reward_{general} = RM + format$
Pipeline R1	Cold start → SFT → RL1 → rejection sampling+SFT → RL2	5 bước, Hình 2
Thất bại: PRM	khó định nghĩa bước, khó gán nhãn, dễ bị hack	Phụ lục G.2
Thất bại: MCTS	không gian token quá lớn, value model mịn khó train	Phụ lục G.2
Distillation vs RL	SFT từ R1 (72.6 AIME) > tự RL Qwen2.5-32B (47.0 AIME)	Bảng 16

11. Kết luận ngắn

DeepSeek-R1-Zero cho thấy RL thuần (GRPO, reward theo luật, không SFT, không PRM/MCTS) đủ để một mô hình nền đủ lớn tự phát triển lập luận dài, có tự kiểm tra và "aha moment", nhưng phải đánh đổi khả năng đọc và tính nhất quán ngôn ngữ. DeepSeek-R1 sửa điều đó bằng một quy trình năm bước (cold start, hai lượt RL, một lượt rejection-sampling SFT), đạt kết quả ngang OpenAI o1 ở toán/code mà vẫn giữ khả năng viết lách/đối thoại. Khi so sánh trực tiếp ở cùng mô hình nền, chưng cất từ R1 vượt xa việc tự chạy RL quy mô lớn trên mô hình nhỏ, một kết luận quan trọng cho ai cần đưa khả năng lập luận xuống mô hình nhỏ với ngân sách hạn chế, nhưng không khẳng định RL là vô dụng: vượt ra ngoài biên giới trí tuệ hiện có, theo tác giả, vẫn cần RL quy mô lớn trên base đủ mạnh.

Link gốc: https://arxiv.org/abs/2501.12948

Bài 40

Kimi k1.5: Scaling Reinforcement Learning with LLMs (2025)

Phạm vi ghi chú: technical report công bố cùng ngày với DeepSeek-R1 (bài 39). Ghi chú này **tập trung vào phần RL**: mở rộng ngữ cảnh 128k, thuật toán tối ưu chính sách (biến thể online policy mirror descent, đạo hàm lại từng bước), length penalty, chiến lược lấy mẫu (curriculum, prioritized), long2short, và hạ tầng partial rollout. Phần **tiền huấn luyện và dữ liệu đa phương thức (Phụ lục B)** chỉ tóm tắt rất ngắn; các bảng benchmark đầy đủ (Phụ lục C, D) không trích lại toàn bộ.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

Tác giả: Kimi Team. Bài báo không nêu tên công ty/affiliation trong văn bản; danh sách đóng góp cá nhân ở Phụ lục A theo vai trò. **Venue / arXiv:** arXiv:2501.12599 (v4, 03/06/2025; bản gốc đăng 22/01/2025, cùng ngày với DeepSeek-R1). Tự gọi là "Technical Report", không phải hội nghị bình duyệt; bài báo không nêu venue khác. **Nguồn:** https://arxiv.org/abs/2501.12599 **Chủ đề:** Scale RL cho LLM đa phương thức bằng cách mở rộng ngữ cảnh tới 128k, dùng một biến thể tối ưu chính sách không cần clip kiểu PPO/GRPO và không cần value network, cộng hạ tầng partial rollout để huấn luyện hiệu quả với CoT rất dài, mà không cần MCTS, process mô hình phần thưởng, hay value function. **Vị trí trong lộ trình:** Đọc sau bài 00 (mục 3.11 trust region/PPO, mục 4.3 nghiệm KL-regularized/DPO, mục 4.4 bỏ critic/GRPO/RLOO) để có nền so sánh. Đọc cặp với bài 39 (DeepSeek-R1): cùng mục tiêu (RL tạo long CoT, bỏ PRM/MCTS/mô hình giá trị), hai lời giải toán học khác nhau. Sau bài này: 41 (DAPO), 42 (Dr. GRPO), 43 (GSPO) đều là các biến thể tối ưu chính sách khác cho cùng bài toán.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Pretraining với next-token prediction bị giới hạn bởi lượng dữ liệu chất lượng cao có sẵn (paper trích Kaplan 2020, Villalobos 2024). RL mở ra một trục mở rộng khác: model học bằng cách **khám phá với reward**, không bị giới hạn vào một tập dữ liệu tĩnh. Kimi k1.5 là một LLM đa phương thức huấn luyện bằng RL theo hướng này, bốn ý tưởng chính (paper tự liệt kê): mở rộng ngữ cảnh (dùng partial rollout để không phải sinh lại toàn bộ trajectory dài mỗi lần), tối ưu chính sách cải tiến (một biến thể online mirror descent), khung đơn giản (không MCTS/value function/PRM), và đa phương thức.

Khái niệm cần nắm trước (cross-ref bài 00):

- **Chain-of-thought (CoT)** $z = (z_1, \ldots, z_m)$: các bước trung gian giữa câu hỏi x và đáp án y, sinh tự hồi quy $z_t \sim \pi_\theta(\cdot|x, z_{<t})$.
- **Planning algorithm**: thuật toán tìm kiếm có cây (search tree), mỗi node là một lời giải bộ phận, dùng critic v chấm điểm để chọn hướng mở rộng tiếp theo (tư duy kiểu MCTS/AlphaGo, bài 14).
- **KL-regularized policy optimization, nghiệm dạng mũ**: bài 00 mục 4.3 đã suy nghiệm $\pi^*(y|x) = \pi_{ref}(y|x)\exp(r(x,y)/\beta)/Z(x)$ cho mục tiêu $\max_\pi \mathbb{E}[r] - \beta\,\mathrm{KL}(\pi\|\pi_{ref})$; Kimi dùng lại chính dạng này nhưng với π_{ref} **thay đổi theo từng iteration** (xem mục 3.2).
- **Baseline nhóm, không critic** (bài 00 mục 4.4: GRPO, RLOO): Kimi cũng bỏ value network, nhưng theo cách khác GRPO/RLOO.

2. Ý tưởng chính

Paper đóng khung việc sinh CoT như một dạng **tìm kiếm ẩn (implicit search)**: một thuật toán planning cổ điển (cây tìm kiếm + critic) có thể được "làm phẳng" thành một chuỗi token duy nhất, nơi số lượng thought (token) đóng vai trò ngân sách tính toán mà thuật toán planning vốn cần. Nếu vậy, một model với ngữ cảnh đủ dài có thể **học để xấp xỉ cả quá trình tìm kiếm đó bằng chính việc sinh tự hồi quy**, thay vì cài cứng một cây tìm kiếm + mô hình giá trị ở lúc suy luận. Hệ quả trực tiếp: mở rộng độ dài ngữ cảnh chính là mở rộng "ngân sách tìm kiếm", và CoT học được sẽ tự thể hiện tính chất lập kế hoạch, nhìn lại, tự sửa.

Để huấn luyện theo ý tưởng này, Kimi dùng một biến thể **online policy mirror descent**: mỗi vòng ngoài, giải một bài toán con KL-regularized mà π_{ref} chính là policy ở đầu vòng đó (không phải một model SFT cố định), thu được nghiệm dạng mũ, rồi biến nó thành một bài toán hồi quy bình phương (regression) có thể tối ưu bằng mẫu off-policy. Không có value network, không clip kiểu PPO; thay vào đó là một số hạng phạt L_2 trên log-tỉ lệ chính sách. Cộng thêm length penalty (chống "overthinking"), hai chiến lược lấy mẫu (curriculum, prioritized), và hạ tầng partial rollout để huấn luyện hiệu quả ở ngữ cảnh 128k, Kimi đạt hiệu năng ngang OpenAI o1 mà không cần MCTS/PRM/mô hình giá trị. Paper còn trình bày 4 cách "long2short" để ép kiến thức CoT dài vào một model CoT ngắn, tiết kiệm token lúc suy luận.

3. Phương pháp chi tiết

3.1 Bài toán: CoT như tìm kiếm ẩn (2.3.1)

Cho $\mathcal{D} = \{(x_i, y_i^*)\}$, mục tiêu là π_θ sao cho $y \sim \pi_\theta(\cdot|x, z)$ đúng với y^*. Một thuật toán planning cổ điển duy trì cây $\mathcal{T}$, node $s = (x, z_{1:|s|})$, dùng critic $v(x, z_{1:|s|})$ chấm điểm, chọn node tốt nhất để mở rộng, lặp lại tới khi có lời giải đầy đủ. Vì cả thought và feedback đều có thể viết dưới dạng token, paper "làm phẳng" toàn bộ thông tin trong cây vào một ngữ cảnh duy nhất, nhìn planning algorithm như một hàm $\mathcal{A}(\cdot|z_1, z_2, \dots)$ tác động trực tiếp lên dãy bước lập luận. Ý tưởng: **huấn luyện model xấp xỉ chính** $\mathcal{A}$ bằng RL, với số token đóng vai trò ngân sách tính toán, thay vì cài cứng một bộ tìm kiếm tường minh lúc suy luận.

Mục tiêu RL: với reward $r(x, y, y^*) \in \{0, 1\}$ (luật cho bài toán kiểm chứng được, hoặc mô hình phần thưởng cho đáp án tự do),

$$\max_\theta \ \mathbb{E}_{(x,y^*)\sim\mathcal{D},\,(y,z)\sim\pi_\theta}[r(x, y, y^*)]$$

3.2 Tối ưu chính sách: biến thể online policy mirror descent (2.3.2, đạo hàm lại)

Bước 1 (bài toán con có regularization). Ở vòng ngoài thứ i, lấy π_{θ_i} hiện tại làm *reference*, giải

$$\boxed{\max_\theta \ \mathbb{E}_{(x,y^*)\sim\mathcal{D}}\,\mathbb{E}_{(y,z)\sim\pi_\theta}[r(x, y, y^*)] - \tau\,\mathrm{KL}(\pi_\theta(x)\|\pi_{\theta_i}(x))}$$

với $\tau > 0$ là hệ số regularization. Đây **đúng dạng bài toán KL-regularized ở bài 00 mục 4.3** ($\beta \to \tau$), chỉ khác: ở DPO, π_{ref} là một model SFT cố định; ở đây $\pi_{ref} = \pi_{\theta_i}$ **thay đổi mỗi vòng ngoài** (giống việc GRPO/R1 cập nhật lại π_{ref} mỗi 400 bước, bài 39 mục 3.1, nhưng Kimi làm việc này ở mọi vòng).

Bước 2 (nghiệm đóng dạng mũ). Áp lại y hệt phép biến đổi ở bài 00 mục 4.3:

$$\boxed{\pi^*(y, z|x) = \pi_{\theta_i}(y, z|x)\exp(r(x, y, y^*)/\tau)/Z}, \qquad Z = \sum_{y',z'} \pi_{\theta_i}(y', z'|x)\exp(r(x, y', y^*)/\tau)$$

Bước 3 (đảo ngược thành ràng buộc trên mọi mẫu). Lấy log hai vế:

$$r(x, y, y^*) - \tau \log Z = \tau \log \frac{\pi^*(y, z|x)}{\pi_{\theta_i}(y, z|x)} \quad \text{đúng cho mọi } (y, z)$$

Bước 4 (biến thành hồi quy bình phương). Vì muốn $\pi_\theta \approx \pi^*$, thay π^* bằng π_θ ở vế phải và tối thiểu hóa bình phương sai số trên mẫu lấy từ π_{θ_i} (off-policy, không cần $\theta = \theta_i$):

$$\mathcal{L}(\theta) = \mathbb{E}_{(x,y^*)}\, \mathbb{E}_{(y,z)\sim\pi_{\theta_i}} \left[\left(r(x, y, y^*) - \tau \log Z - \tau \log \frac{\pi_\theta(y, z|x)}{\pi_{\theta_i}(y, z|x)} \right)^2 \right]$$

Bước 5 (xấp xỉ $\tau \log Z$, tự tính). Khi $\tau \to \infty$, $\exp(r/\tau) \approx 1 + r/\tau$, nên $\tau \log Z = \tau \log \mathbb{E}_{\pi_{\theta_i}}[\exp(r/\tau)] \approx \tau \log(1 + \mathbb{E}[r]/\tau) \approx \mathbb{E}_{\pi_{\theta_i}}[r]$: $\tau \log Z$ tiến về **reward kỳ vọng dưới** π_{θ_i}. Paper ước lượng nó bằng trung bình mẫu $\bar{r} = \text{mean}(r_1, \ldots, r_k)$ của k rollout lấy từ π_{θ_i}.

Bước 6 (gradient). Lấy gradient của $\mathcal{L}$ theo θ (chỉ số hạng log-tỉ lệ phụ thuộc θ), rồi hấp thụ hằng số 2τ vào learning rate, được công thức cuối (paper, eq. 3):

$$\boxed{\frac{1}{k} \sum_{j=1}^{k} \left[\nabla_\theta \log \pi_\theta(y_j, z_j|x)\, (r_j - \bar{r}) - \frac{\tau}{2} \nabla_\theta \left(\log \frac{\pi_\theta(y_j, z_j|x)}{\pi_{\theta_i}(y_j, z_j|x)} \right)^2 \right]}$$

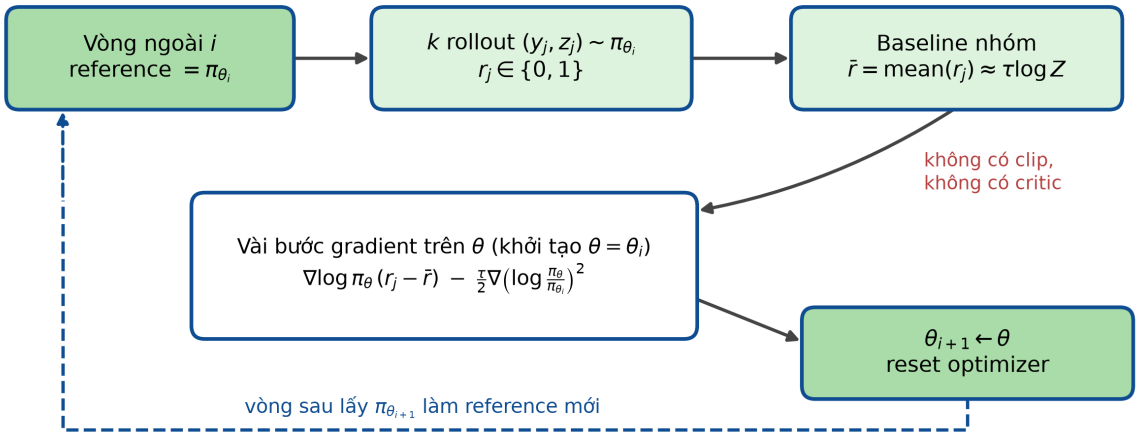
Đọc công thức: số hạng đầu là **REINFORCE với baseline trung bình nhóm** (giống bài 00 mục 4.4, GRPO/RLOO, nhưng không chia std như GRPO); số hạng hai là một **phạt** L_2 **mềm** trên log-tỉ lệ $\pi_\theta/\pi_{\theta_i}$, thay cho clip kiểu PPO/GRPO. Ngay tại đầu mỗi vòng ($\theta = \theta_i$), số hạng hai bằng 0, nên bước đầu tiên là REINFORCE thuần; các bước sau trong cùng vòng bị kéo nhẹ về lại π_{θ_i}. **Sau mỗi vòng ngoài, optimizer được reset** (vì bài toán con đổi reference). Không có importance-sampling ratio tường minh như PPO/GRPO: sai lệch off-policy được xử lý hoàn toàn qua số hạng phạt L_2.

Diễn giải lại thành pseudo-code (tự tính, không phải trích nguyên văn paper):

```
cho i = 0, 1, 2, ... (vòng ngoài):
    reset optimizer                       # bài toán con đổi reference mỗi vòng
    cho mỗi batch câu hỏi (x, y*):
        lấy k rollout (y_j, z_j) ~ π_θi(.|x)   # θ = θ_i lúc này
        r_j = reward(x, y_j, y*)  với j = 1..k
        r_bar = mean(r_1, ..., r_k)        # xấp xỉ τ log Z
        cho vài bước gradient trên θ (khởi tạo θ = θ_i):
            loss = -(1/k) Σ_j [ log π_θ(y_j,z_j|x) * (r_j - r_bar)
                                - (τ/2) * (log(π_θ(y_j,z_j|x)/π_θi(y_j,z_j|x)))^2 ]
            θ <- θ - lr * ∇_θ loss
    θ_{i+1} <- θ                          # vòng ngoài tiếp theo dùng θ_{i+1} làm
    ↪  reference
```

Cách đọc hình: đi theo mũi tên đen từ ô vòng ngoài sang rollout, baseline nhóm, rồi vài bước gradient. Điểm cốt lõi là đường nét đứt xanh: sau mỗi vòng, tham số mới trở thành reference của vòng kế tiếp, nên phạt L_2 luôn kéo về chính sách đầu vòng. Không có clip kiểu PPO và không có mô hình giá trị.

Vì sao bỏ value network (lập luận của bài báo). Giả sử mô hình đã sinh một CoT bộ phận $(z_1, \ldots, z_t)$ và có hai bước tiếp theo khả dĩ z_{t+1} (dẫn thẳng tới đáp án đúng) và z'_{t+1} (có vẻ sai). Credit assignment kiểu value-function chuẩn sẽ **phạt** z'_{t+1} vì advantage âm. Nhưng nếu model học cách **đi theo** z'_{t+1} **rồi tự phục hồi** và vẫn ra đáp án đúng, đó chính là hành vi lập luận dài có giá trị (thử, sai, sửa). Vì reward chỉ đặt ở đáp án cuối (không phân rã per-token theo advantage), mô hình được tự do học mẫu hình "thử và sửa" này.



Hình 76: Vòng ngoài của biến thể online policy mirror descent: reference đổi theo từng vòng (minh hoạ, tự tính)

3.3 Length penalty (2.3.3, tự tính ví dụ)

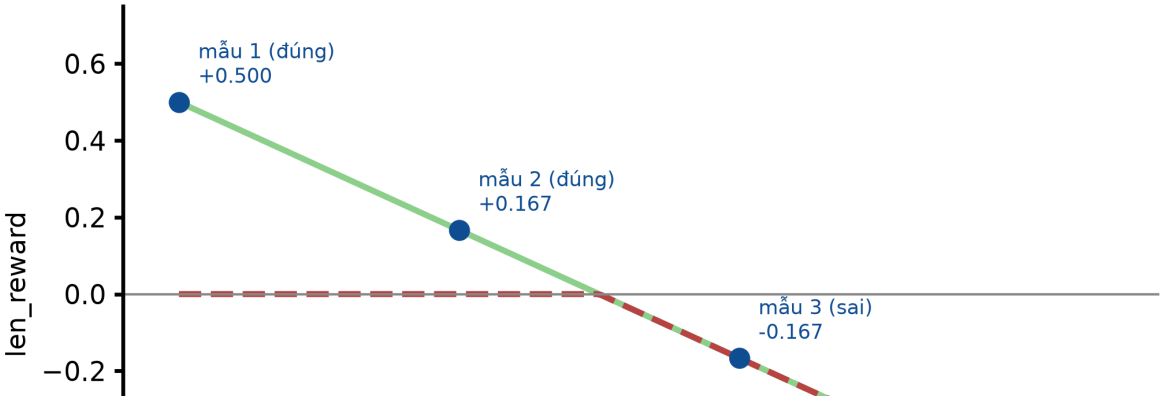
Hiện tượng "overthinking": độ dài câu trả lời tăng mạnh trong RL (giống bài 39), tốt cho điểm số nhưng tốn chi phí huấn luyện/suy luận. Với k mẫu (y_i, z_i) cùng câu hỏi, gọi $\text{len}(i)$ là độ dài, $\text{min_}len, \text{max_}len$ là min/max trong nhóm:

$$\boxed{\text{len_reward}(i) = \begin{cases} \lambda & \text{nếu } r(x, y_i, y^*) = 1 \\ \min(0, \lambda) & \text{nếu } r(x, y_i, y^*) = 0 \end{cases}}, \qquad \lambda = 0.5 - \frac{\text{len}(i) - \text{min_}len}{\text{max_}len - \text{min_}len}$$

(nếu $\text{max_}len = \text{min_}len$, đặt reward độ dài bằng 0 cho mọi mẫu). **Ví dụ tự tính**, $k = 4$, độ dài $[1000, 2000, 3000, 4000]$, đúng/sai $= [1, 1, 0, 0]$ ($\text{min_}len = 1000, \text{max_}len = 4000$, khoảng $= 3000$):

i	len	λ	đúng/sai	len_reward
1	1000	$0.5 - 0/3000 = 0.5$	đúng	0.5
2	2000	$0.5 - 1000/3000 = 0.167$	đúng	0.167
3	3000	$0.5 - 2000/3000 = -0.167$	sai	$\min(0, -0.167) = -0.167$
4	4000	$0.5 - 3000/3000 = -0.5$	sai	$\min(0, -0.5) = -0.5$

Đọc bảng: đáp án đúng ngắn nhất được thưởng nhiều nhất ($+0.5$); đáp án sai ngắn (λ>0, không xảy ra ở ví dụ này) không bị phạt thêm; đáp án sai dài nhất bị phạt nặng nhất. Reward này cộng vào reward gốc với một trọng số; paper nhận thấy áp ngay từ đầu làm chậm huấn luyện, nên **warm up**: chạy chính sách chuẩn không length penalty trước, rồi thêm length penalty hằng số cho phần còn lại.



3.4 Chiến lược lấy mẫu: curriculum và prioritized (2.1, 2.3.4)

Độ khó mỗi prompt được ước lượng bằng cách cho SFT model sinh 10 lần ở nhiệt độ cao, tỉ lệ đúng (pass rate) thấp = khó; cách này còn dùng để **lọc trước** các câu quá dễ trước khi vào RL.

- **Curriculum sampling:** huấn luyện trên câu dễ trước, chuyển dần sang câu khó, dùng nhãn độ khó có sẵn (ví dụ cấp lớp của đề toán). Lý do: model ban đầu yếu, dồn tính toán vào câu quá khó cho ít mẫu đúng, hiệu quả huấn luyện thấp.
- **Prioritized sampling:** theo dõi tỉ lệ thành công s_i của từng câu hỏi qua các lần lấy mẫu lại, sau đó lấy mẫu tỉ lệ với $1 - s_i$, dồn nỗ lực vào câu model còn yếu.

Để tránh reward hacking khi chọn prompt: loại câu trắc nghiệm/đúng-sai/chứng minh (dễ đúng đáp án mà sai lập luận); với câu hỏi mở, loại câu mà model đoán đúng trong $N = 8$ lần thử **không cần CoT** ("dễ hack").

3.5 Long2short: ép CoT dài vào model CoT ngắn (2.4)

Bốn phương pháp, không huấn luyện lại từ đầu:

1. **Model merging:** trộn trọng số (trung bình) của model CoT dài và model CoT ngắn, không cần train thêm.
2. **Shortest rejection sampling:** lấy mẫu cùng câu hỏi $n = 8$ lần, giữ **câu trả lời đúng ngắn nhất** để SFT.
3. **DPO:** câu đúng ngắn nhất làm positive; câu sai (bất kỳ độ dài) hoặc câu đúng nhưng dài hơn $1.5\times$ positive làm negative; train DPO chuẩn (bài 29).
4. **Long2short RL:** chọn checkpoint CoT dài cân bằng tốt giữa hiệu năng/hiệu quả token, tiếp tục RL với length penalty (mục 3.3) và **giảm mạnh max rollout length**.

3.6 Reward modeling cho toán và code (2.3.5, tóm lược)

Code: sinh test case tự động bằng thư viện CYaRon vì nhiều bài trên mạng không có test case công khai. Với mỗi bài: sinh 50 test case ứng viên, chạy cùng 10 submission đúng đã biết, giữ test case nếu $\geq 7/10$ submission cho kết quả khớp; đưa bài vào tập huấn luyện nếu $\geq 9/10$ submission qua được toàn bộ tập test đã chọn. Từ 1,000 bài thi mẫu: 614 không cần special judge, 463 generator sinh được ≥ 40 test case hợp lệ, cuối cùng **323 bài** vào tập huấn luyện.

Toán: hai loại mô hình phần thưởng, cùng 800K mẫu mỗi loại. Classic RM (kiểu InstructGPT, value-head, nhận câu hỏi+đáp án mẫu+câu trả lời, xuất 1 scalar) đạt độ chính xác spot-check **84.4**. Chain-of-Thought RM (sinh lập luận từng bước trước khi chấm, xuất JSON) đạt **98.5**. Paper chọn CoT RM cho huấn luyện RL vì tin cậy hơn hẳn (quan trọng vì $a^2 - 4$ và $(a + 2)(a - 2)$ là cùng một đáp án viết khác dạng, Classic RM dễ nhầm).

3.7 Hạ tầng: partial rollout (2.6, tóm lược)

Hệ thống RL đồng bộ lặp (Hình 3a): rollout worker (điều phối bởi một master) sinh trajectory, lưu vào replay buffer, trainer worker lấy ra để cập nhật trọng số. **Partial rollout** (2.6.2) là kỹ thuật chính cho long-CoT: đặt một ngân sách token cố định cho mỗi rollout; nếu trajectory chưa xong khi hết ngân sách, phần chưa hoàn thành được lưu vào replay buffer và **tiếp tục ở iteration sau** thay vì sinh lại từ đầu; chỉ đoạn thuộc iteration hiện tại cần tính on-policy, các đoạn trước tái dùng từ buffer. Nhờ vậy, rollout worker không bị "kẹt" chờ một trajectory siêu dài, các worker khác vẫn xử lý trajectory ngắn song song (bất đồng bộ). Hệ thống còn phát hiện lặp chuỗi và dừng sớm, phạt thêm cho lặp.

Hybrid deployment (2.6.3): Megatron (train) và vLLM (infer) chia sẻ GPU trong cùng pod qua container sidecar; một "checkpoint-engine" chuyển trọng số qua RDMA (Mooncake), chuyển từ train sang infer dưới 1 phút, ngược lại khoảng 10 giây. **Code sandbox (2.6.4):** dùng `crun` thay Docker (khởi động 0.04s so với 0.12s; 120 so với 27 container/giây trên máy 16 lõi), cgroup tái dùng, overlay filesystem với tmpfs.

3.8 Dữ liệu đa phương thức (tóm tắt rất ngắn)

Dữ liệu vision RL gồm ba nhóm: (1) dữ liệu thực tế (câu hỏi khoa học có hình, đoán vị trí, đọc biểu đồ); (2) dữ liệu tổng hợp (ảnh/cảnh sinh có kiểm soát để luyện quan hệ không gian, hình học); (3) dữ liệu văn bản-render-thành-ảnh (chuyển văn bản/code thành ảnh để model xử lý nhất quán dù input là text hay ảnh). Bài báo không cho số liệu quy mô cụ thể cho riêng phần RL này.

4. Thí nghiệm và kết quả chính

Ba nhóm benchmark: Text (MMLU, IF-Eval, CLUEWSC, C-Eval), Reasoning (LiveCodeBench, Codeforces, AIME 2024, MATH-500, HumanEval-Mul), Vision (MMMU, MathVision, MathVista).

Bảng 2 (trích), k1.5 long-CoT so với model hàng đầu:

Benchmark	QwQ-32B-Preview	OpenAI o1-mini	OpenAI o1	Kimi k1.5
MATH-500 (EM)	90.6	90.0	94.8	96.2
AIME 2024 (Pass@1)	50.0	63.6	74.4	77.5
Codeforces (Percentile)	62	88	94	94
LiveCodeBench (Pass@1)	40.6	53.1	67.2	62.5
MathVista-Test (Pass@1)	-	-	71.0	74.9
MMMU-Val (Pass@1)	-	-	77.3	70.0

Lưu ý đọc kỹ: k1.5 vượt o1 ở MATH-500/AIME/MathVista nhưng **thua** o1 ở LiveCodeBench (62.5 so với 67.2) và MMMU-Val (70.0 so với 77.3); không phải "vượt mọi benchmark" như phần tóm tắt dễ gây hiểu nhầm.

Bảng 3 (trích), k1.5 short-CoT so với model hàng đầu:

Benchmark	Qwen2.5-72B-Inst.	DeepSeek-V3	GPT-4o-0513	Kimi k1.5
MATH-500 (EM)	80.0	90.2	74.6	94.6
AIME 2024 (Pass@1)	23.3	39.2	9.3	60.8
LiveCodeBench (Pass@1)	31.1	40.5	33.4	47.3
MathVista-Test (Pass@1)	-	-	63.8	70.1

k1.5-short vượt rõ các mô hình CoT ngắn khác, kể cả DeepSeek-V3 (không suy luận dài) ở mọi dòng trên.

Chi phí huấn luyện: paper **không có bảng GPU-giờ/USD** tương đương Bảng 7 của bài 39, nên không thể so sánh trực tiếp chi phí hai hướng tiếp cận.

5. Phân tích / ablation

Độ dài tăng cùng hiệu năng (Hình 5, 6). Trên một model nội bộ nhỏ hơn k1.5, độ dài phản hồi và độ chính xác tăng cùng nhau qua các iteration; benchmark khó hơn có độ dốc tăng độ dài lớn hơn (ví dụ slope AIME2024 $\approx 3.4 \times 10^{-5}$, GPQA $\approx 4.2 \times 10^{-5}$, đọc từ chú thích Hình 6, cao hơn các benchmark dễ). Mô hình cuối (k1.5) scale tới 128k và vẫn cải thiện trên benchmark khó.

Quy mô model so với độ dài ngữ cảnh (Hình 8, mục 3.5 "Scaling of model size and context length"). Hai model kích thước khác nhau, cùng dữ liệu: mô hình lớn khởi đầu tốt hơn, nhưng **mô hình nhỏ có thể đạt hiệu năng tương đương nhờ CoT dài hơn** qua RL; tuy vậy mô hình lớn vẫn có **hiệu suất token tốt hơn** nói chung. Kết luận của bài báo: muốn hiệu năng tối đa, scale ngữ cảnh của mô hình lớn có cận trên cao hơn và

hiệu quả token hơn; nếu ngân sách suy luận bị giới hạn, train mô hình nhỏ với ngữ cảnh dài hơn có thể là lựa chọn khả thi.

Tác dụng của negative gradient, so với ReST (Hình 10). ReST (Gulcehre et al., 2023) chỉ fit vào mẫu tốt nhất lấy từ model hiện tại, **không** phạt mẫu sai (không có negative gradient). Phương pháp của Kimi có **độ phức tạp mẫu tốt hơn rõ rệt** so với ReST trên nhiều benchmark (OMNI-MATH500, MATH500, AIME2024, GPQA...). Kết luận: với long-CoT, việc phạt mẫu sai (negative gradient) quan trọng hơn hẳn so với các domain RL khác (theo Gulcehre et al., khoảng cách ReST với RL thường không lớn ở domain khác).

Curriculum sampling (Hình 9). So với baseline lấy mẫu đều (uniform) trên hỗn hợp dễ/khó, curriculum (đều trước, chuyển sang tập khó ở iteration 24) cho đường cong hiệu năng cao hơn rõ rệt và ổn định hơn baseline qua các iteration.

Long2short (Hình 7). k1.5-short-w/-rl đạt AIME2024 Pass@1 **60.8** (trung bình 8 lần chạy) chỉ dùng **3,272 token** trung bình; k1.5-shortest đạt MATH500 Pass@1 **88.2** với số token xấp xỉ các mô hình ngắn khác. Trong 4 phương pháp long2short, **long2short RL hiệu quả token nhất**, hơn DPO và model merging.

6. Giới hạn

Tác giả nêu:

- Kết luận chương (mục 4): "cách scale và cải thiện hiệu quả của long-context RL training vẫn là câu hỏi quan trọng" (chưa coi là giải quyết xong).
- Kiến trúc model (Phụ lục B.3): tác giả nói rõ các thí nghiệm scale kiến trúc "nằm ngoài phạm vi báo cáo này, sẽ công bố sau".
- Hình 5, 8 dùng "một model nội bộ nhỏ hơn hẳn k1.5" (chú thích Hình 5), không phải chính model cuối, nên các đường cong scaling không hẳn đại diện cho k1.5 thật.
- Bộ lọc chống reward-hacking cho câu hỏi mở (đoán đúng trong $N = 8$ lần không CoT thì loại) là một **heuristic**, không có gì đảm bảo; "xây dựng verification model tốt hơn vẫn là hướng mở".

Nhận xét:

- Paper **không nêu**: kích thước model k1.5 chính thức, số mẫu k mỗi câu hỏi khi RL, giá trị τ, trọng số length penalty trong reward tổng, learning rate của RL, số iteration RL của model cuối, kích thước "model nội bộ nhỏ hơn" dùng ở Hình 5/8/9/10. Đây là khác biệt rõ với bài 39 (DeepSeek-R1 nêu đầy đủ lr, β, batch, số bước cho từng giai đoạn).
- So sánh với ReST (Hình 10) chỉ chạy trên mô hình nhỏ nội bộ, không phải k1.5 cuối; không có so sánh trực tiếp với GRPO/PPO-clip ở cùng quy mô như bài 39 làm với GRPO/PPO (Hình 3-4 của bài 39).
- Bảng 2 cho thấy k1.5 **không** vượt o1 ở LiveCodeBench và MMMU-Val; phần tóm tắt đầu paper dễ gây cảm giác "vượt toàn diện", cần đọc bảng kỹ.
- Không có bảng chi phí GPU-giờ/USD như Bảng 7 của bài 39, nên không thể đối chiếu "chi phí RL của Kimi so với DeepSeek-R1" bằng số liệu công khai.
- Long2short RL (Hình 7) không nêu rõ ngân sách huấn luyện so với model merging (không cần train) hay DPO/rejection sampling, nên "hiệu quả token lúc suy luận" chưa hẳn phản ánh đúng "hiệu quả tổng ngân sách tính toán".

7. Liên hệ với các bài khác

- **00 (preface):** nghiệm KL-regularized dạng mũ (mục 4.3) chính là nền của phép đạo hàm ở mục 3.2; baseline trung bình nhóm không chia std (mục 4.4, so với GRPO/RLOO) giúp thấy rõ Kimi khác GRPO ở điểm nào.
- **39 (DeepSeek-R1):** công bố cùng ngày, cùng kết luận bỏ PRM/MCTS (Phụ lục G.2 của bài 39 so với lựa chọn thiết kế ở đây), nhưng chọn GRPO (clip PPO-style, chia std, cập nhật π_{ref} mỗi 400 bước) thay

vì online mirror descent (phạt L_2, không chia std, đổi π_{ref} mỗi vòng ngoài). Bài 39 công bố chi phí GPU-giờ, bài này không.

- **05 (GAE), 07 (PPO):** nền so sánh cho "credit assignment per-token" mà Kimi lập luận không hợp với long CoT (mục 3.2).
- **29 (DPO):** long2short dùng chính DPO (mục 3.5, phương pháp 3) với cách chọn positive/negative đặc biệt theo độ dài.
- **37 (RLOO):** baseline trung bình (không trừ-một-ra-ngoài) gần với cách Ahmadian et al. mà Kimi trích dẫn cho mean-baseline.
- **41 (DAPO), 43 (GSPO):** các bài sau tiếp tục sửa GRPO (không phải hướng của Kimi) nhưng cùng vấn đề cốt lõi: làm sao tối ưu chính sách ổn định cho long CoT mà không cần value network.
- **llm 46 (PagedAttention), llm 48 (speculative decoding):** partial rollout (mục 3.7) có tinh thần gần việc tái dùng/không tính lại trạng thái đã có, dù áp dụng cho huấn luyện RL, không phải serving.

8. Ứng dụng thực tế (suy luận)

Kimi k1.5 gợi ý các hướng phân bổ tài nguyên cho RLVR: chọn mẫu theo tỉ lệ thành công, phát hiện chuỗi lặp và tái sử dụng partial rollout. Hiệu quả của từng lựa chọn phụ thuộc vào độ dài đầu ra và chi phí sinh mẫu; cần đánh giá trên tác vụ cụ thể. Phần thưởng độ dài cũng cần được kiểm tra để tránh ưu tiên câu trả lời ngắn nhưng thiếu nội dung.

9. Thí nghiệm gợi ý

1. So sánh lấy mẫu theo tỉ lệ thành công với lấy mẫu đều, đo hiệu năng theo độ khó.
2. Thử phát hiện lặp trong rollout và đo tác động tới chất lượng lẫn thời gian sinh.
3. So sánh có và không có phần thưởng độ dài trên cùng dữ liệu.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu KL-regularized	$\max_\theta \mathbb{E}[r] - \tau\,\mathrm{KL}(\pi_\theta(x)\|\pi_{\theta_i}(x))$	$\pi_{ref} = \pi_{\theta_i}$ đổi mỗi vòng
Nghiệm dạng mũ	$\pi^* = \pi_{\theta_i}\exp(r/\tau)/Z$	giống bài 00 mục 4.3
Surrogate loss	$\mathbb{E}\big[(r - \tau\log Z - \tau\log\frac{\pi_\theta}{\pi_{\theta_i}})^2\big]$	hồi quy bình phương, off-policy
$\tau\log Z \approx \bar{r}$	trung bình mẫu của k rollout	xấp xỉ khi $\tau \to \infty$
Gradient cuối	$\frac{1}{k}\sum_j[\nabla\log\pi_\theta\,(r_j - \bar{r}) - \frac{\tau}{2}\nabla(\log\frac{\pi_\theta}{\pi_{\theta_i}})^2]$	REINFORCE + phạt L_2, không clip
Length reward	$\lambda = 0.5 - \frac{\text{len}-\min}{\max-\min}$; đúng nhận λ, sai nhận $\min(0,\lambda)$	chống overthinking
Curriculum / prioritized	dễ→khó theo nhãn; lấy mẫu $\propto (1-s_i)$	s_i = tỉ lệ thành công
Long2short (4 cách)	merge, shortest-RS, DPO, long2short-RL	RL hiệu quả token nhất
Partial rollout	cắt theo ngân sách token, lưu phần dở vào buffer, tiếp ở iteration sau	chỉ đoạn mới cần on-policy

11. Kết luận ngắn

Kimi k1.5 đạt hiệu năng ngang OpenAI o1 bằng cách xem việc sinh CoT dài như một dạng tìm kiếm ẩn mà model tự học xấp xỉ, huấn luyện bằng một biến thể online mirror descent (nghiệm KL-regularized dạng mũ, biến thành hồi quy off-policy, không value network, không clip kiểu PPO) cộng length penalty và hai chiến lược lấy mẫu để kiểm soát độ dài và hiệu quả huấn luyện. Hạ tầng partial rollout là mảnh ghép kỹ thuật

cho phép mở rộng ngữ cảnh tới 128k mà không phải sinh lại toàn bộ trajectory dài mỗi lần. So với bài 39 (DeepSeek-R1, công bố cùng ngày), đây là một lời giải toán học khác cho cùng bài toán "RL tạo long CoT mà không cần PRM/MCTS/value model", đáng đọc song song để thấy GRPO không phải lựa chọn duy nhất.

Link gốc: https://arxiv.org/abs/2501.12599

Bài 41

DAPO: An Open-Source LLM Reinforcement Learning System at Scale (2025)

Tác giả: ByteDance Seed, Institute for AI Industry Research (AIR, Tsinghua), The University of Hong Kong, SIA-Lab of Tsinghua AIR và ByteDance Seed (danh sách đầy đủ ở mục Contributions, project lead: Qiying Yu). **Venue / arXiv:** arXiv:2503.14476 (v2, 20/05/2025); bài báo không nêu venue (chỉ có "Date: March 17, 2025" và project page https://dapo-sia.github.io/). **Nguồn:** https://arxiv.org/abs/2503.14476 **Chủ đề:** Một hệ thống RL mã nguồn mở quy mô lớn cho LLM reasoning dạng long Chain-of-Thought (CoT), với 4 kỹ thuật sửa lỗi của GRPO: Clip-Higher, Dynamic Sampling, Token-level Policy Gradient Loss, Overlong Reward Shaping. **Vị trí trong lộ trình:** đọc sau bài 00 (PPO, GAE, GRPO, KL penalty), bài 36 (DeepSeekMath, nguồn gốc GRPO), bài 39 (DeepSeek-R1, mục tiêu tái lập), bài 40 (Kimi k1.5). DAPO là bài đầu tiên trong nhóm "sửa GRPO cho long-CoT RLVR" (41-44): 42 (Dr. GRPO) phân tích bias chuẩn hoá theo hướng khác, 43 (GSPO) thay hẳn đơn vị importance ratio, 44 (ScaleRL) đánh giá định lượng cả ba dưới một khung chung. **Chương nền liên quan:** nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Test-time scaling (OpenAI o1, DeepSeek R1) cho thấy RL quy mô lớn có thể tạo ra các hành vi reasoning phức tạp (tự kiểm tra, tinh chỉnh lặp lại) giúp LLM giải toán thi (AIME) và lập trình thi (Codeforces) tốt hơn nhiều. Nhưng các báo cáo kỹ thuật của o1/R1 giữ kín chi tiết thuật toán, nên cộng đồng gặp khó khi tái lập (paper dẫn một loạt công trình cố tái lập R1 nhưng không thành công đầy đủ).

Nhóm tác giả bắt đầu bằng GRPO "naive" (nguyên bản, xem lại bài 36) trên Qwen2.5-32B base: chỉ đạt **30 điểm** AIME 2024, thấp hơn nhiều so với DeepSeek-R1-Zero-Qwen-32B (**47 điểm**). Phân tích sâu phát hiện ba vấn đề: **entropy collapse** (entropy của policy giảm nhanh, các response trong một group gần như giống nhau, tức mất khả năng khám phá), **reward noise** (nhiễu trong tín hiệu thưởng) và **training instability**.

Khái niệm nhắc lại nhanh (xem đầy đủ ở bài 00, 36):

- **PPO** (bài 07): tối ưu mục tiêu surrogate có clip theo importance ratio $r_t(\theta) = \frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}$, với advantage $\hat{A}_t$ ước lượng bằng GAE (bài 05):

$$J_{PPO}(\theta) = \mathbb{E}_{(q,a)\sim D,\, o_{\le t}\sim\pi_{\theta_{old}}}\Big[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),1-\varepsilon,1+\varepsilon)\hat{A}_t\right)\Big]$$

- **GRPO** (bài 36): bỏ hẳn value function, lấy mẫu G response $\{o_i\}_{i=1}^{G}$ cho mỗi câu hỏi q, chuẩn hoá reward theo nhóm để ra advantage:

$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^{G})}{\text{std}(\{R_j\}_{j=1}^{G})}$$

  và tối ưu

$$J_{GRPO}(\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}(r_{i,t}(\theta),1-\varepsilon,1+\varepsilon)\hat{A}_{i,t}\right) - \beta D_{KL}(\pi_\theta\|\pi_{ref})\Big]$$

  Điểm quan trọng paper nhấn mạnh: GRPO tính loss ở **sample-level**, nghĩa là **trước tiên lấy trung bình**

theo token trong một response ($\frac{1}{|o_i|}\sum_t$), **rồi mới lấy trung bình giữa các response** ($\frac{1}{G}\sum_i$). Hệ quả của cách cộng dồn này là trọng tâm của kỹ thuật thứ ba (§3.3).

- **Entropy collapse:** entropy của phân phối token do actor sinh ra giảm nhanh, response trong cùng group gần giống nhau, hạn chế khám phá.
- **Reward hacking:** mô hình khai thác kẽ hở hàm thưởng để lấy điểm cao mà không thực sự đúng.

Vì mô hình phần thưởng học từ dữ liệu dễ bị *reward hacking*, bài báo dùng **rule-based reward** (kiểu outcome reward, xem lại RLVR ở bài 34, 36, 39):

$$R(\hat{y}, y) = \begin{cases} 1, & \text{is_equivalent}(\hat{y}, y) \\ -1, & \text{khác} \end{cases}$$

Về **KL penalty** (βD_{KL} trong GRPO): trong RLHF chuẩn (bài 07, 23), KL giữ policy gần π_{ref} để không "trôi" khỏi vùng mà mô hình phần thưởng còn đáng tin. Nhưng khi huấn luyện long-CoT reasoning, phân phối của policy *cần* trôi xa khỏi mô hình khởi tạo (để học cách suy luận mới), nên nhóm tác giả quyết định **bỏ hẳn hạng tử KL** khỏi mục tiêu của mình (§2.3).

2. Ý tưởng chính

DAPO = **D**ecoupled Clip và **D**ynamic s**A**mpling **P**olicy **O**ptimization, xây trên GRPO không-KL với 4 kỹ thuật, mỗi kỹ thuật nhắm vào một triệu chứng cụ thể quan sát được trong quá trình huấn luyện:

1. **Clip-Higher**: decouple clip-range dưới/trên ($\varepsilon_{low} \neq \varepsilon_{high}$) để chống entropy collapse.
2. **Dynamic Sampling**: lọc bỏ và lấy mẫu lại các prompt có accuracy = 0 hoặc = 1 (advantage = 0, gradient = 0) để giữ batch luôn có gradient hiệu quả.
3. **Token-level Policy Gradient Loss**: đổi cách cộng dồn loss từ sample-level sang token-level, để response dài không bị "giảm trọng số" một cách không mong muốn.
4. **Overlong Reward Shaping**: giảm nhiễu từ các response bị cắt (truncate) do vượt độ dài tối đa, bằng cách mask loss hoặc phạt mềm theo độ dài.

Kết quả: từ Qwen2.5-32B base, DAPO đạt **50 điểm** AIME 2024 avg@32, vượt DeepSeek-R1-Zero-Qwen-32B (47 điểm) chỉ với **50% số bước** huấn luyện.

3. Phương pháp chi tiết

3.1 Mục tiêu DAPO (công thức tổng)

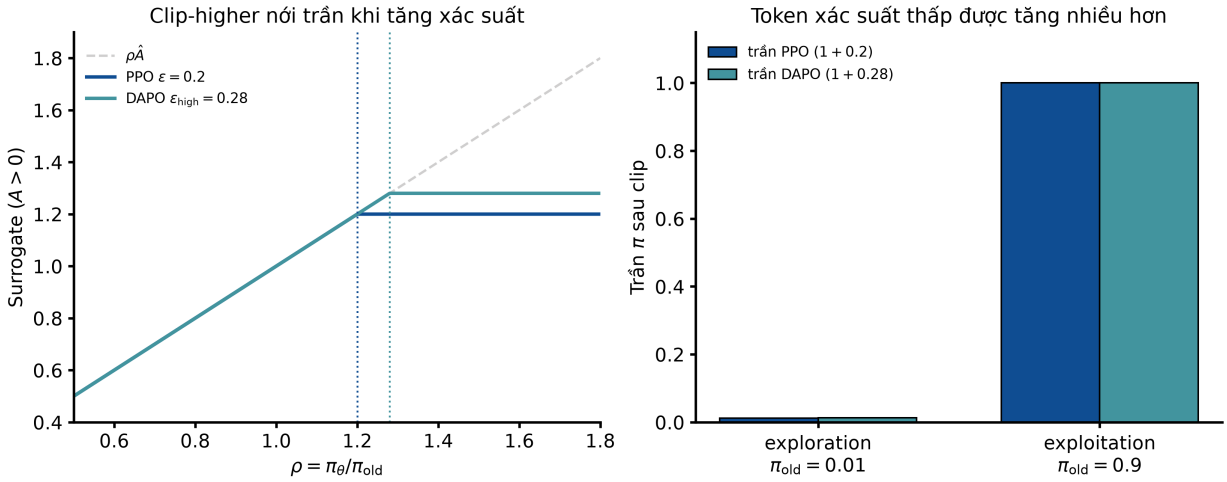
$$J_{DAPO}(\theta) = \mathbb{E}_{(q,a)\sim D,\, \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}(r_{i,t}(\theta), 1-\varepsilon_{low}, 1+\varepsilon_{high})\hat{A}_{i,t}\right) \right]$$

$$\text{s.t. } 0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G, \qquad r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^{G})}{\text{std}(\{R_j\}_{j=1}^{G})}$$

So với J_{GRPO}: (a) không còn hạng tử KL; (b) clip tách $\varepsilon_{low}/\varepsilon_{high}$; (c) mẫu số đổi từ $\frac{1}{G}\sum_i \frac{1}{|o_i|}$ (sample-level) sang $\frac{1}{\sum_i |o_i|}$ (token-level); (d) ràng buộc "0 < số response đúng < G" (không cho phép group toàn đúng hoặc toàn sai).

3.2 Raise the Ceiling: Clip-Higher

Quan sát (Hình 2 trong bài báo): khi dùng clip đối xứng $\varepsilon = 0.2$ (giá trị mặc định ở hầu hết thuật toán), accuracy AIME và entropy của actor đều giảm dần theo bước huấn luyện ("w/o Clip-Higher"). Phân tích nguyên nhân:



Clip-higher $\varepsilon_{\text{high}} = 0.28$: token $\pi_{\text{old}} = 0.01$ được tăng trần cao hơn PPO (minh hoạ, tự tính)

Khi $\hat{A}_{i,t} > 0$ (hệ thống muốn tăng xác suất token đó) và $\pi_{\theta_{old}}(o_i|q)$ nhỏ (token "exploration", ít khả năng), trần trên của xác suất mới bị giới hạn bởi $\pi_{\theta_{old}} \cdot (1 + \varepsilon)$. Ví dụ paper đưa ra với $\varepsilon = 0.2$:

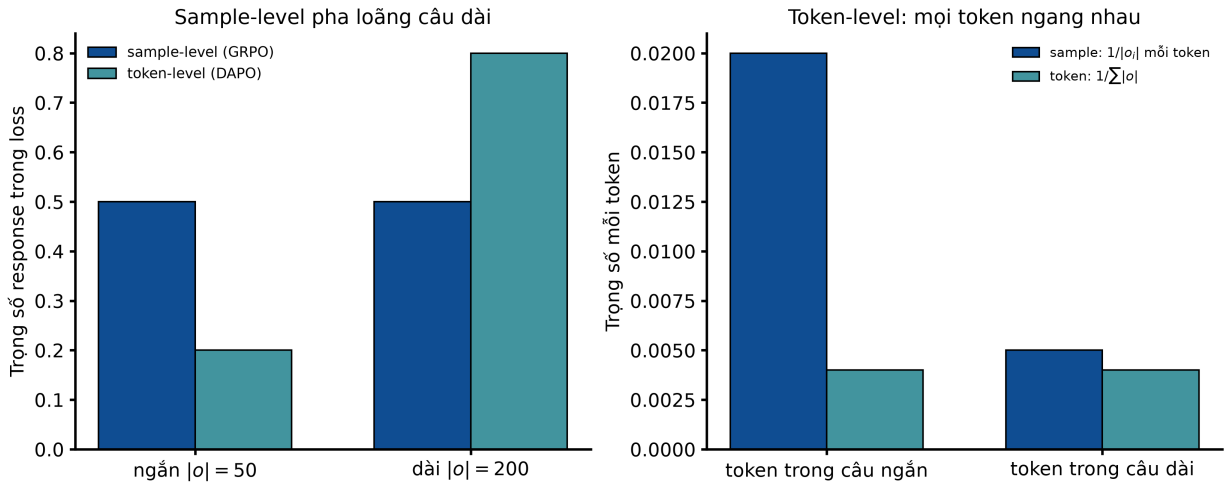
$\pi_{\theta_{old}}(o_i \mid q)$	Trần trên π_θ sau 1 bước clip
0.01 (token "exploration")	0.012
0.9 (token "exploitation")	1.08 (vô nghĩa vì > 1, tức không bị chặn thực tế)

Nghĩa là token đã có xác suất cao ("exploitation", ví dụ 0.9) không hề bị trần chặn (vì $0.9 \times 1.2 = 1.08 > 1$), có thể tăng lên gần 0.999 dễ dàng; còn token xác suất thấp ("exploration", ví dụ 0.01) bị trần $1 + \varepsilon$ bó rất chặt, khó tăng đáng kể dù muốn. Thực nghiệm xác nhận: xác suất trung bình của các token bị "up-clip" (bị chặn ở trần trên) luôn < 0.2 (Hình 3a), đúng như suy luận: chính các token xác suất thấp mới hay bị kẹp trần, làm giảm khả năng khám phá của hệ thống, góp phần vào entropy collapse.

Giải pháp: decouple ε_{low} và ε_{high}, **tăng** ε_{high} để chừa chỗ cho các token xác suất thấp tăng mạnh hơn, **giữ nguyên** ε_{low} (nếu tăng ε_{low} thì các token sẽ bị đẩy xác suất về gần 0, thu hẹp không gian sample, phản tác dụng). Giá trị dùng trong thực nghiệm: $\varepsilon_{low} = 0.2$, $\varepsilon_{high} = 0.28$ (§4.1).

3.3 The More the Merrier: Dynamic Sampling

Vấn đề "gradient biến mất": nếu mọi response $\{o_i\}_{i=1}^{G}$ của một prompt đều đúng (accuracy = 1) thì R_i bằng nhau $\Rightarrow \hat{A}_{i,t} = 0$ cho toàn bộ group $\Rightarrow$ gradient của nhóm đó bằng 0. Thực nghiệm cho thấy tỉ lệ các group có accuracy = 1 **tăng dần** theo huấn luyện (Hình 3b), nghĩa là càng về sau, càng nhiều prompt trong batch không đóng góp gradient $\Rightarrow$ số prompt "hiệu quả" trong batch giảm dần $\Rightarrow$ phương sai gradient tăng, tín hiệu học yếu đi.



Sample-level pha loãng câu dài vs token-level trọng số đều mỗi token (minh hoạ, tự tính)

Giải pháp: over-sample và lọc bỏ các prompt có accuracy bằng 0 hoặc bằng 1, tiếp tục sample thêm cho đến khi batch được lấp đầy bởi các prompt có accuracy nằm giữa 0 và 1 (đó chính là ràng buộc $0 < |\{o_i : \text{đúng}\}| < G$ trong công thức §3.1). Chi phí sample trên mỗi batch trở nên **động** (dynamic): có thể cần sample nhiều hơn số prompt dự kiến ban đầu.

Paper lưu ý thời gian tổng không bị ảnh hưởng nhiều, vì thời gian sinh của một batch thường bị chi phối bởi các mẫu "đuôi dài" (long-tail) nếu hệ thống đồng bộ và không quy trình sinh; thêm vào đó, với Dynamic Sampling số bước cần để đạt cùng hiệu năng còn **giảm** (Hình 6), nên thời gian huấn luyện tổng thể không tăng mà có thể giảm.

3.4 Rebalancing Act: Token-Level Policy Gradient Loss

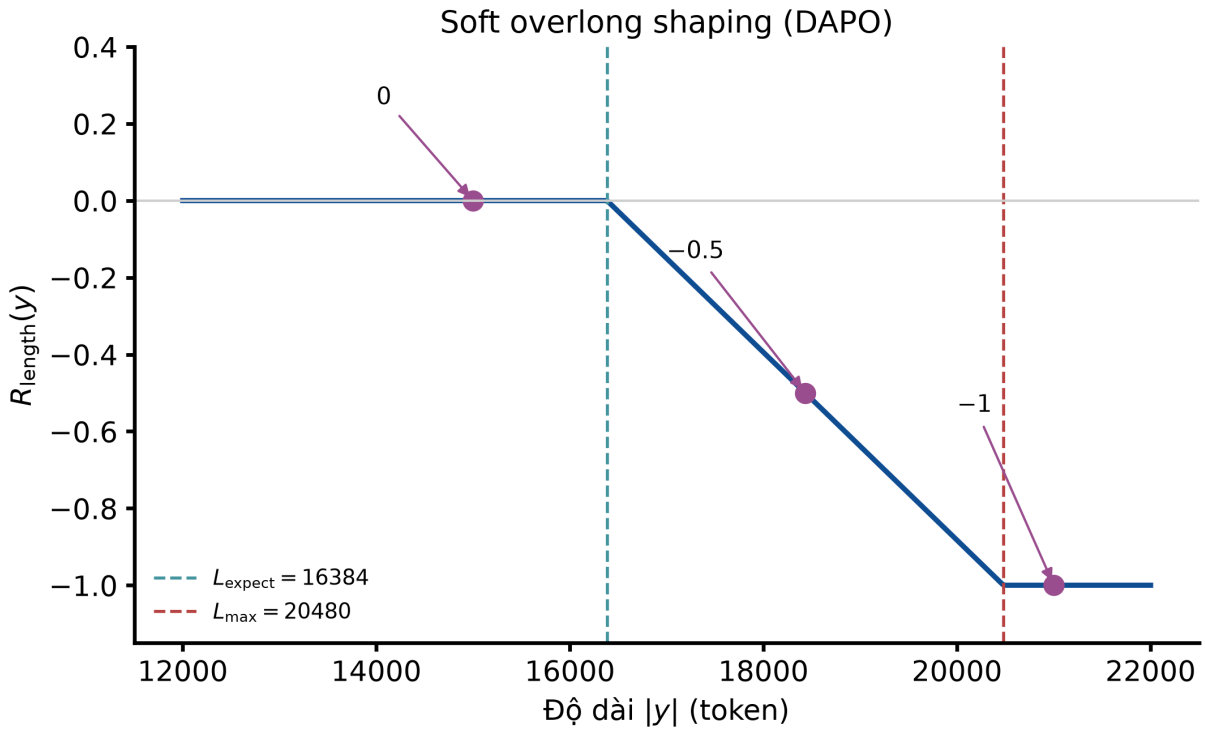
GRPO gốc: $\frac{1}{G}\sum_i \frac{1}{|o_i|}\sum_t(\cdot)$, tức mỗi **sample** (response) có trọng số bằng nhau trong loss cuối cùng, bất kể độ dài. Hệ quả: token trong response dài (nhiều token hơn) bị "pha loãng" đóng góp vào loss tổng. Hai tác động xấu quan sát được:

1. Với response dài và chất lượng cao: mô hình khó học các mẫu hình reasoning quan trọng bên trong (vì trọng số mỗi token của nó nhỏ).
2. Response quá dài thường có các mẫu hình chất lượng thấp (gibberish, lặp từ); vì sample-level loss không phạt hiệu quả các mẫu hình xấu trong response dài, entropy và độ dài response cùng tăng một cách "không lành mạnh" (Hình 4a-4b).

Giải pháp: đổi cách cộng dồn thành token-level, $\frac{1}{\sum_i |o_i|}\sum_i\sum_t(\cdot)$ (đã viết trong công thức §3.1). Hệ quả: response dài hơn có ảnh hưởng **lớn hơn** lên gradient tổng (tỉ lệ thuận với số token), và xét ở mức từng token: nếu một mẫu hình sinh ra khiến reward tăng/giảm, nó sẽ được khuyến khích/triệt tiêu **ngang nhau**, không phụ thuộc độ dài response chứa nó.

3.5 Hide and Seek: Overlong Reward Shaping

Khi sinh vượt độ dài tối đa, response bị cắt (truncate). Nếu gán phần thưởng phạt mặc định cho các mẫu bị cắt, có thể phạt oan một chuỗi suy luận đúng chỉ vì nó dài, gây nhiễu reward và làm rối huấn luyện.



Soft overlong: $R_{\text{length}} = 0$ tới 16384, tuyến tính tới -1 ở 20480; ví dụ -0.5 tại 18432 (minh hoạ, tự tính)

Bước 1, Overlong Filtering: mask loss của các sample bị truncate (không tính gradient từ chúng). Thực nghiệm (Hình 5): ổn định và tăng hiệu năng rõ rệt.

Bước 2, Soft Overlong Punishment: thay vì chỉ mask, phạt mềm theo độ dài trong một khoảng "đệm" (cache):

$$R_{length}(y) = \begin{cases} 0, & |y| \le L_{max} - L_{cache} \\ \dfrac{(L_{max} - L_{cache}) - |y|}{L_{cache}}, & L_{max} - L_{cache} < |y| \le L_{max} \\ -1, & L_{max} < |y| \end{cases}$$

Phần thưởng phạt này được **cộng thêm** vào reward đúng/sai gốc.

Ví dụ tính tay (tự tính), dùng các số liệu huấn luyện thực tế của bài báo ($L_{max} - L_{cache} = 16384$ token "kỳ vọng", $L_{cache} = 4096$ token đệm, do đó $L_{max} = 20480$):

- $|y| = 15000$ (dưới ngưỡng kỳ vọng): $R_{length} = 0$.
- $|y| = 18432$ (đúng giữa khoảng đệm): $R_{length} = \dfrac{16384 - 18432}{4096} = \dfrac{-2048}{4096} = -0.5$.
- $|y| = 21000$ (vượt L_{max}): $R_{length} = -1$.

3.6 Thuật toán tổng hợp

```
Input: policy ban đầu pi_theta; reward R(.); tập prompt D; sieu tham so eps_low, eps_high
for step = 1 .. M:
    lay 1 batch prompt Db tu D
    pi_theta_old <- pi_theta
    for q trong Db:
        sinh G output {o_i} ~ pi_theta_old(.|q)                  # rollout
        tinh reward {r_i} cho tung o_i bang R(.)
        # Dynamic Sampling (3.2): loai bo neu accuracy(q) == 0 hoac == 1,
        # dua phan con lai vao buffer; sample them q moi neu buffer chua day
```

```
    if |buffer| < N: tiep tuc sample
    cho moi o_i trong buffer: tinh A_hat_i,t theo cong thuc GRPO-advantage (sec 3.1)
    for inner_iter = 1 .. mu:
        cap nhat pi_theta bang cach maximize J_DAPO       # token-level loss (3.3),
                                                           # clip Clip-Higher (3.1),
                                                           # loss cua sample bi truncate
                                                           ↪  da mask (3.4)
Output: pi_theta
```

(Dịch lại Algorithm 1 của bài báo sang pseudo-code có chú thích; nội dung giữ đúng thứ tự bước của bản gốc.)

3.7 Biến đổi dữ liệu (Dataset Transformation)

Đáp án toán thường ở dạng biểu thức/công thức (ví dụ $a + c\sqrt{b}$), khó viết rule chính xác để so khớp. Bài báo dùng LLM để **biến đổi đề bài** sao cho đáp án trở thành một **số nguyên** dễ parse (ví dụ yêu cầu tính $a + b + c$ thay vì so khớp biểu thức $a + c\sqrt{b}$ trực tiếp). Kết quả là **DAPO-Math-17K**: 17.000 prompt, mỗi prompt có đáp án là một số nguyên. Quy trình biến đổi dùng CoT 4 bước: trích format đáp án, viết lại đề, giải đề đã viết lại, đưa ra đáp án số nguyên cuối (few-shot/hướng dẫn cho từng bước).

4. Thí nghiệm và kết quả chính

Mô hình: Qwen2.5-32B base. **Framework:** verl. **Baseline:** naive GRPO (dùng chuẩn hoá reward theo nhóm).

Siêu tham số (§4.1): AdamW, learning rate hằng số 1×10^{-6}, warm-up tuyến tính 20 bước rollout; prompt batch size 512; 16 response/prompt khi rollout; mini-batch huấn luyện 512 (= 16 lượt cập nhật gradient mỗi bước rollout); Overlong Reward Shaping: độ dài kỳ vọng tối đa 16.384 token + 4.096 token đệm phạt mềm $\Rightarrow$ tổng tối đa sinh 20.480 token; Clip-Higher: $\varepsilon_{low} = 0.2$, $\varepsilon_{high} = 0.28$. Đánh giá trên AIME: lặp lại 32 lần, báo cáo avg@32; nhiệt độ sinh (temperature) = 1.0, top-p = 0.7.

Kết quả chính (Hình 1): DAPO đưa Qwen2.5-32B base từ gần 0% lên **50%** AIME 2024 avg@32, vượt DeepSeek-R1-Zero-Qwen-32B (47%) với chỉ **50% số bước** huấn luyện.

Bảng 1 (bảng ablation tích lũy, đúng theo bài báo, đơn vị AIME24 avg@32):

Mô hình / cấu hình	AIME24 avg@32
DeepSeek-R1-Zero-Qwen-32B	47
Naive GRPO	30
+ Overlong Filtering	36
+ Clip-Higher	38
+ Soft Overlong Punishment	41
+ Token-level Loss	42
+ Dynamic Sampling (DAPO)	50

Lưu ý quan trọng về cách đọc bảng: thứ tự cộng dồn trong Bảng 1 (Overlong Filtering → Clip-Higher → Soft Overlong Punishment → Token-level Loss → Dynamic Sampling) **không giống** thứ tự trình bày trong §3 (Clip-Higher §3.2 → Dynamic Sampling §3.3 → Token-level Loss §3.4 → Overlong Reward Shaping §3.5). Vì đây là ablation tích lũy (mỗi dòng = dòng trước + 1 kỹ thuật), không nên suy ra "đóng góp riêng lẻ" của mỗi kỹ thuật bằng cách trừ trực tiếp hai dòng liên tiếp nếu không biết chắc thứ tự cộng dồn; bài báo không cung cấp ablation "loại từng kỹ thuật khỏi cấu hình đầy đủ" (leave-one-out) để đối chiếu.

Token-level loss mang lại ít điểm hơn các kỹ thuật khác (42 so với 41 trước đó, +1 điểm) nhưng theo bài báo nó **ổn định hoá huấn luyện** và khiến độ dài response tăng "lành mạnh" hơn (không phải là cải thiện điểm số, mà là cải thiện chất lượng quá trình). Dynamic Sampling đóng góp lớn nhất trong bảng (+8 điểm, từ 42 lên 50) nhưng paper cũng nói **thời gian huấn luyện tổng** không tăng đáng kể, nhờ hội tụ nhanh hơn (Hình 6).

5. Phân tích / ablation

Mục 4.3 của bài báo mô tả các **chỉ số giám sát then chốt** trong suốt training (không phải ablation số liệu mà là hướng dẫn thực hành vận hành hệ thống RL quy mô lớn):

- **Độ dài response sinh ra:** liên quan chặt tới ổn định và hiệu năng; độ dài tăng giúp không gian khám phá lớn hơn, nhưng không phải lúc nào cũng tăng liên tục, có giai đoạn chững hoặc giảm (hiện tượng này cũng từng thấy ở DeepSeek-R1). Nên dùng kết hợp với accuracy trên validation để đánh giá thực nghiệm có đang "xấu đi" hay không.
- **Động lực của reward:** trong đa số thực nghiệm của nhóm, reward tăng ổn định trên tập train, nhưng **reward trên train hầu như không tương quan** với accuracy trên validation, cho thấy có hiện tượng overfit vào tập train.
- **Entropy của actor và xác suất sinh:** cần giữ entropy trong một khoảng hợp lý: quá thấp = mất khả năng khám phá; quá cao = sinh gibberish/lặp lại quá mức. Clip-Higher (§3.2) là công cụ chính để chống entropy quá thấp; theo kinh nghiệm nhóm, xu hướng entropy tăng chậm (không tăng vọt, không giảm về 0) đi cùng với hiệu năng cải thiện (Hình 7c, 7d).

Case study (mục 4.4): paper quan sát định tính rằng các mẫu hình reasoning (ví dụ hành vi "reflect and backtrack": tự kiểm tra lại bước giải trước) hầu như không xuất hiện ở giai đoạn đầu huấn luyện nhưng dần xuất hiện khi training tiến triển (Bảng 2, một ví dụ cụ thể về bài toán hình học không gian). Đây là quan sát định tính minh hoạ, **không có số liệu định lượng** (ví dụ tỉ lệ % mẫu có hành vi reflect) kèm theo trong bài.

6. Giới hạn

Tác giả nêu: thực nghiệm tập trung hoàn toàn vào bài toán toán học (math), dù khẳng định phương pháp "có thể chuyển sang các tác vụ khác" nhưng không có số liệu minh chứng cho tác vụ khác (ví dụ code) trong bài này. Phần hệ thống hoá (4.3) cũng nhấn mạnh RL cho LLM là "bài toán kỹ thuật hệ thống phức tạp" với các thay đổi nhỏ có thể gây hậu quả khó dự đoán, tức thừa nhận bản thân recipe cần theo dõi/điều chỉnh liên tục, không phải "lắp vào là chạy".

Nhận xét:

- Baseline "naive GRPO" chỉ đạt 30 điểm, thấp hơn hẳn 47 điểm của DeepSeek-R1-Zero-Qwen-32B dù cùng kiến trúc gốc (Qwen2.5-32B); bài báo không làm rõ naive GRPO của họ khác biệt thế nào (data, hyperparameter khác) so với cấu hình thực tế DeepSeek dùng, nên khó tách bạch phần cải thiện thực sự đến từ 4 kỹ thuật DAPO và phần đến từ khác biệt dữ liệu/hạ tầng huấn luyện.
- Không có ablation tách riêng hiệu ứng của ε_{high} ở các giá trị khác 0.28 (chỉ có một cấu hình cuối cùng), nên chưa rõ độ nhạy với lựa chọn chính xác này.
- Soft Overlong Punishment và Overlong Filtering được giới thiệu như hai bước của cùng một ý tưởng, nhưng Bảng 1 chỉ cho số liệu tích lũy, không so sánh trực tiếp "chỉ Overlong Filtering" với "chỉ Soft Overlong Punishment" ở cùng điều kiện khác.
- Bài 42 (Dr. GRPO) sau này chỉ ra rằng ngay cả việc chuẩn hoá theo `std(R)` trong advantage (vẫn được DAPO giữ nguyên) cũng có bias riêng (ưu tiên câu hỏi có độ khó cực đoan); DAPO không bàn tới vấn đề này, cho thấy 4 kỹ thuật của DAPO chưa phải "sửa triệt để" mọi bias của GRPO.

7. Liên hệ với các bài khác

- **Bài 00:** nền tảng PPO, GAE, GRPO, KL penalty cần nắm trước khi đọc bài này.

- **Bài 36 (DeepSeekMath, GRPO):** DAPO xây trực tiếp trên GRPO, đây là baseline được cải tiến.
- **Bài 39 (DeepSeek-R1), 40 (Kimi k1.5):** mục tiêu tái lập/vượt kết quả reasoning RL quy mô lớn mà hai báo cáo này không công khai đủ chi tiết.
- **Bài 42 (Dr. GRPO):** cùng nhận diện vấn đề "độ dài response ảnh hưởng không công bằng tới gradient", nhưng theo hướng khác: DAPO giữ `std(R)` trong advantage và sửa cách **cộng dồn loss** (sample-level → token-level, §3.4 ở trên); Dr. GRPO thì **loại bỏ hẳn** cả `1/|o_i|` và `std(R)`, coi cả hai là nguồn bias cần xoá, không phải chỉ đổi cách cộng dồn. Hai hướng giải quyết cùng một triệu chứng (length bias) theo hai cơ chế khác nhau, nên hai bài nên đọc nối tiếp để thấy sự khác biệt.
- **Bài 43 (GSPO):** đi xa hơn nữa, cho rằng chính **importance ratio ở mức token** (vẫn được DAPO giữ nguyên, chỉ sửa clip-range và cách cộng dồn) là "ill-posed" về lý thuyết importance sampling, và chuyển hẳn sang ratio ở mức sequence.
- **Bài 44 (ScaleRL):** dùng **chính GRPO + asymmetric DAPO-clip** làm "base algorithm" xuất phát điểm cho nghiên cứu scaling của họ, và so sánh định lượng loss DAPO với GSPO, CISPO trong cùng một khung đo (sigmoid compute-performance); DAPO clip-range (ε_{max}) được ScaleRL chỉ ra là rất nhạy cảm, ảnh hưởng tới cả asymptote hiệu năng, không chỉ tốc độ hội tụ.

8. Ứng dụng thực tế (suy luận)

- **Clip-Higher** là kỹ thuật rẻ nhất để thử đầu tiên trong bất kỳ quy trình RLVR/GRPO nào: chỉ cần tách 1 siêu tham số (ε_{high}) thành giá trị lớn hơn ε_{low}, không cần thay đổi hạ tầng. Nếu quan sát entropy giảm nhanh và các response trong group giống nhau bất thường, đây là dấu hiệu nên thử Clip-Higher trước.
- **Dynamic Sampling** đổi chi phí sample (tốn thêm compute sinh mẫu) để lấy lại gradient hiệu quả; đáng dùng khi tập dữ liệu có nhiều câu "quá dễ" hoặc "quá khó" khiến accuracy nhanh bão hoà về 0/1.
- **Token-level loss** quan trọng hơn khi phân phối độ dài response rất lệch (một số response rất dài so với phần còn lại); nếu độ dài tương đối đồng đều, khác biệt với sample-level loss sẽ nhỏ.
- Overlong Reward Shaping phù hợp với bất kỳ tác vụ sinh văn bản dài có giới hạn độ dài cứng (hard max length), ví dụ OCR cho tài liệu nhiều trang nếu từng ghép nhiều trang vào một lần sinh; cơ chế "phạt mềm trong khoảng đệm" giúp tránh phạt oan các đầu ra dài nhưng đúng.

9. Thí nghiệm gợi ý

1. Trên một tác vụ RLVR nhỏ (ví dụ GSM8K với mô hình 1-3B), sweep $\varepsilon_{high} \in \{0.2, 0.24, 0.28, 0.32\}$ (giữ $\varepsilon_{low} = 0.2$), đo entropy theo bước và accuracy cuối, kiểm chứng lại quan sát "tăng ε_{high} giúp entropy không collapse".
2. So sánh Dynamic Sampling với GRPO thường trên cùng tác vụ nhỏ: đo số bước đến khi đạt một ngưỡng accuracy cố định, và tổng wall-clock time (bù chi phí sample thêm), kiểm tra lại tuyên bố "không chậm hơn, có thể nhanh hơn" của bài báo.
3. Cài Soft Overlong Punishment, thử các giá trị L_{cache} khác nhau (ví dụ 1/8, 1/4, 1/2 của $L_{max} - L_{cache}$) trên một tác vụ có nhiều response bị cắt, đo mức độ nhiễu reward (ví dụ phương sai reward của các response đúng nhưng dài).
4. Trên một dataset có phương sai độ dài response cao, so sánh trực tiếp token-level loss và sample-level loss (giữ mọi thứ khác cố định), đo entropy drift và tỉ lệ response "lặp từ/gibberish" theo thời gian huấn luyện.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
GRPO advantage	$\hat{A}_{i,t} = \dfrac{R_i - \text{mean}(R)}{\text{std}(R)}$	chuẩn hoá theo nhóm G response
DAPO objective	xem §3.1, mẫu số $\sum_i \lvert o_i \rvert$	token-level, không KL

Khái niệm	Công thức / ý	Ghi chú
Clip-Higher	$\text{clip}(r, 1-\varepsilon_{low}, 1+\varepsilon_{high})$, $\varepsilon_{low}=0.2, \varepsilon_{high}=0.28$	chống entropy collapse
Dynamic Sampling	$0 < \lvert\{o_i : \text{đúng}\}\rvert < G$	lọc group toàn đúng/toàn sai
Token-level loss	$\frac{1}{\sum_i \lvert o_i\rvert}\sum_i\sum_t(\cdot)$ thay cho $\frac{1}{G}\sum_i\frac{1}{\lvert o_i\rvert}\sum_t(\cdot)$	response dài ảnh hưởng gradient nhiều hơn
Overlong Filtering	mask loss các sample bị truncate	bước 1
Soft Overlong Punishment	xem công thức $R_{length}(y)$ ở §3.5	$L_{max}-L_{cache}=16384$, $L_{cache}=4096$
Rule-based reward	$R=1$ nếu đúng, -1 nếu sai	tránh reward hacking từ RM học
Kết quả chính	AIME24 avg@32: 30 (GRPO) → 50 (DAPO)	Qwen2.5-32B base, 50% số bước so với DeepSeek-R1-Zero-Qwen-32B (47)

11. Kết luận ngắn

DAPO cho thấy GRPO "trần trụi" không đủ để huấn luyện long-CoT RLVR ổn định ở quy mô lớn: entropy collapse, gradient biến mất ở các group đã bão hoà, length bias trong cách cộng dồn loss, và nhiễu từ các response bị cắt đều là những rào cản thực tế. Bốn kỹ thuật (Clip-Higher, Dynamic Sampling, Token-level Loss, Overlong Reward Shaping), mỗi kỹ thuật tương đối đơn giản để cài, cộng dồn lại đưa Qwen2.5-32B base từ 30 lên 50 điểm AIME 2024 avg@32, vượt DeepSeek-R1-Zero-Qwen-32B với một nửa số bước huấn luyện, và toàn bộ code/data được mở để cộng đồng tái lập.

Link gốc: https://arxiv.org/abs/2503.14476

Bài 42

Understanding R1-Zero-Like Training: A Critical Perspective (2025)

Tác giả: Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, Min Lin (Sea AI Lab; National University of Singapore; Singapore Management University). **Venue / arXiv:** arXiv:2503.20783 (v2, 06/10/2025); "Published as a conference paper at COLM 2025". **Nguồn:** https://arxiv.org/abs/2503.20783 **Chủ đề:** Phản biện hai thành phần của huấn luyện kiểu R1-Zero (RL trực tiếp trên mô hình nền, không SFT): (1) mô hình nền vốn đã có sẵn nhiều năng lực được cho là "do RL tạo ra"; (2) GRPO có hai bias làm response dài ra một cách giả tạo. Đề xuất **Dr. GRPO** (GRPO Done Right) để sửa bias thứ hai. **Vị trí trong lộ trình:** đọc sau bài 00, bài 36 (GRPO gốc), bài 39 (DeepSeek-R1-Zero, đối tượng phản biện chính). Đọc cùng nhóm với bài 41 (DAPO, cùng than phiền về length bias nhưng sửa bằng cách khác) và bài 43 (GSPO, sửa theo hướng đổi hẳn đơn vị importance ratio); bài 44 (ScaleRL) sau này kiểm định lại claim về chuẩn hoá `std` ở quy mô lớn hơn.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

DeepSeek-R1-Zero (bài 39) gây tiếng vang vì áp RL **trực tiếp** lên mô hình nền (không cần bước SFT trung gian), và trong quá trình đó độ dài response **tăng liên tục**, đi kèm "Aha moment" (hành vi tự phản tư/self-reflection xuất hiện). Nhiều người coi hai hiện tượng này là bằng chứng RL "dạy" mô hình biết suy luận dài hơn và biết tự kiểm tra. Bài này đặt câu hỏi phản biện cho cả hai vế.

Khái niệm cần nắm trước:

- **Template:** định dạng prompt ép mô hình nền (vốn chỉ học next-token prediction, $p_\theta(x)$) hoạt động như một policy hỏi-đáp $\pi_\theta(\cdot|q)$. Paper thử 3 loại: *R1 template* (có `<think>...</think><answer>...</answer>`, giống DeepSeek-R1), *Qwen-Math template* (chat template chuẩn, yêu cầu đưa đáp án vào `\boxed{}`), *No template* (chỉ đưa câu hỏi thô).
- **pass@k:** xác suất ít nhất 1 trong k lần sample ra đáp án đúng; dùng để đo khả năng *khám phá* (exploration ability) của base policy: nếu pass@k của mô hình nền = 0, RL không có tín hiệu reward nào để học.
- **MDP token-level cho sinh văn bản:** trạng thái $s_t = [q; o_{<t}]$ (câu hỏi nối với các token đã sinh), hành động o_t là token kế tiếp, chuyển trạng thái $s_{t+1} = [s_t; o_t]$ là tất định; return $R(q, o) = \sum_t r(s_t, o_t)$.
- **Mục tiêu RL có KL** (dạng tổng quát của RLHF, bài 00, 23):

$$J(\pi_\theta) = \mathbb{E}_{q\sim p_Q}\Big[\mathbb{E}_{o\sim\pi_\theta(\cdot|q)}[R(q,o)] - \beta D_{KL}[\pi_\theta(\cdot|q)\|\pi_{ref}(\cdot|q)]\Big]$$

  Khi reward là **rule-based verifier** (không phải mô hình phần thưởng học từ dữ liệu), không còn lo ngại "drift ra khỏi vùng RM còn tin được" như trong RLHF cổ điển, nên có thể đặt $\beta = 0$ (bỏ KL) để tiết kiệm bộ nhớ/compute cho π_{ref} và (theo một số quan sát được paper trích dẫn) còn cho hiệu năng tốt hơn. Bài này giả định $\beta = 0$ xuyên suốt.
- **GRPO** (nhắc lại bài 36): với group G response $\{o_1, \ldots, o_G\}$ và return $R = \{R_1, \ldots, R_G\}$,

$$\hat{A}_{i,t} = \frac{R(q,o_i) - \text{mean}(\{R(q,o_1), \ldots, R(q,o_G)\})}{\text{std}(\{R(q,o_1), \ldots, R(q,o_G)\})}$$

2. Ý tưởng chính

Hai phần phân tích độc lập, cộng lại thành một bức tranh phản biện:

Phần 1 (mô hình nền): mô hình nền, ngay cả khi chưa qua RL, đã có thể "trả lời câu hỏi" tốt nếu dùng template phù hợp, và một số mô hình nền (kể cả DeepSeek-V3-Base, mô hình gốc của R1-Zero thật) **đã sẵn có hành vi self-reflection** trước khi RL. Riêng họ Qwen2.5-Math còn đạt kết quả tốt nhất khi **không dùng template nào cả**, gợi ý rằng nó có thể đã được pretrain trên dữ liệu dạng câu hỏi-đáp ghép liền.

Phần 2 (RL/GRPO): bản thân công thức GRPO có **hai bias hệ thống** khiến độ dài response tăng một cách giả tạo (đặc biệt với response sai), dễ bị nhầm là "RL dạy mô hình suy luận dài hơn". Sửa bằng cách bỏ hai hạng tử chuẩn hoá gây bias ($1/|o_i|$ và $\text{std}(R)$) để được **Dr. GRPO**, một ước lượng không chệch (unbiased) của chính sách gradient PPO với baseline Monte Carlo.

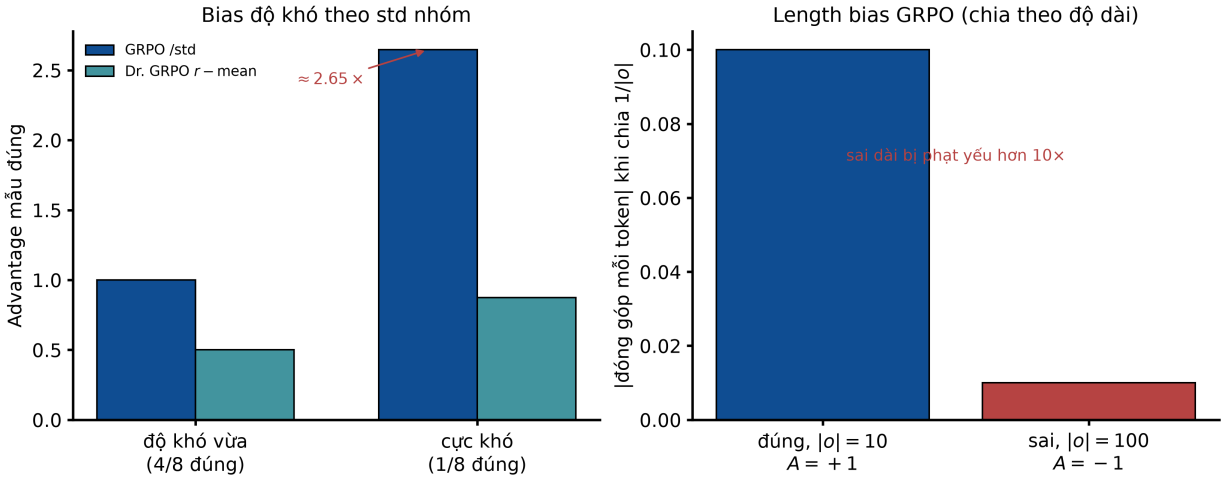
Gộp hai insight, nhóm đưa ra một "recipe tối giản": Qwen2.5-Math-7B + Dr. GRPO + câu hỏi MATH mức độ 3-5 + Qwen-Math template, đạt SOTA 43.3% AIME 2024 chỉ với 27 giờ compute trên 8×A100.

3. Phương pháp chi tiết

3.1 Hai bias trong GRPO (mục 3.1 paper)

Viết lại mục tiêu PPO chuẩn (không KL, $\beta = 0$) để làm chuẩn so sánh:

$$J^{PPO}(\pi_\theta) = \mathbb{E}_{q\sim p_Q,\, o\sim\pi_{\theta_{old}}} \sum_{t=1}^{|o|} \min\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}\hat{A}_t,\ \text{clip}(\cdot, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)$$



Difficulty bias: GRPO phóng đại câu cực khó (≈ 2.65); length bias khi chia $1/|o|$ (minh hoạ, tự tính)

và mục tiêu GRPO:

$$J^{GRPO}(\pi_\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(w_{i,t}(\theta)\hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\right)\Big]$$

So với PPO (không có $1/G$ và $1/|o_i|$ cho từng sample riêng, chỉ có $\sum_t$ với baseline hợp lệ), GRPO đưa thêm hai phép chia:

(a) Response-level length bias, từ việc chia cho $|o_i|$: khi $\hat{A}_{i,t} > 0$ (response đúng), gradient của mỗi token bị chia cho $|o_i|$, nên **response đúng ngắn hơn nhận gradient update lớn hơn mỗi token** $\Rightarrow$ chính sách bị

đẩy về phía "trả lời đúng thì ngắn gọn". Ngược lại, khi $\hat{A}_{i,t} < 0$ (response sai), response **dài hơn bị phạt nhẹ hơn mỗi token** (vì cũng chia cho $|o_i|$ lớn) $\Rightarrow$ chính sách có xu hướng **thích response sai dài hơn** (ít bị phạt tương đối). Đây là cơ chế cụ thể khiến độ dài trung bình của các response **sai** tăng dần theo huấn luyện, dễ bị nhầm là "mô hình đang học suy luận sâu hơn".

(b) Question-level difficulty bias, từ việc chia cho $\text{std}(\{R_1, \ldots, R_G\})$: các câu hỏi có std thấp (quá dễ hoặc quá khó, tức accuracy gần toàn 1 hoặc toàn 0) nhận **trọng số update lớn hơn** so với câu hỏi có độ khó "vừa" (accuracy gần 0.5, std gần cực đại). Chuẩn hoá advantage là một trick phổ biến trong RL (ví dụ chuẩn hoá advantage theo *toàn batch*), nhưng ở đây GRPO chuẩn hoá **theo từng câu hỏi riêng lẻ**, khiến trọng số khác nhau giữa các câu hỏi trong cùng batch, chứ không phải một phép co giãn chung vô hại.

Ví dụ minh hoạ bias (b), tự tính: so sánh 2 câu hỏi trong cùng batch, $G = 8$ response mỗi câu.

- Câu hỏi "độ khó vừa": $R = \{1, 1, 1, 1, 0, 0, 0, 0\}$, mean $= 0.5$, std $= 0.5$. Advantage GRPO của 1 response đúng: $(1 - 0.5)/0.5 = 1$; của 1 response sai: $(0 - 0.5)/0.5 = -1$.
- Câu hỏi "cực khó" (chỉ 1/8 đúng): $R = \{1, 0, 0, 0, 0, 0, 0, 0\}$, mean $= 0.125$, std $= \sqrt{0.125 \times 0.875} \approx 0.330$. Advantage của response đúng duy nhất: $(1 - 0.125)/0.330 \approx 2.65$; của response sai: $(0 - 0.125)/0.330 \approx -0.379$.

$\Rightarrow$ response đúng của câu hỏi cực khó được khuếch đại gradient gấp **2.65 lần** so với response đúng của câu hỏi độ khó vừa, hoàn toàn do std nhỏ, không liên quan gì tới "chất lượng" của response đó.

Length bias cũng tồn tại trong nhiều framework PPO mã nguồn mở. Paper khảo sát (Bảng 2 trong bài báo) và thấy trl, OpenRLHF, verl, SimpleRL-Zero, Open-Reasoner-Zero **đều** chuẩn hoá loss theo độ dài response (`mask.sum(axis=dim)` thay vì một hằng số cố định), đi ngược định nghĩa PPO gốc. Paper suy đoán nguồn gốc: thói quen từ giai đoạn pretraining (chia loss theo context length cố định để ổn định số học), bị mang sang giai đoạn RL-tuning một cách không chủ ý (ở RL, độ dài response không còn là hằng số).

```
def masked_mean(tensor, mask, dim):
    # bi-bias (cach lam cua nhieu repo PPO mo nguon mo, vd OpenRLHF / trl / verl):
    return (tensor * mask).sum(axis=dim) / mask.sum(axis=dim)   # chia theo do dai THUC TE,
    ↪  thay doi theo sample
    # khong-bias (Dr. GRPO):
    # return (tensor * mask).sum(axis=-1) / MAX_TOKENS          # chia theo hang so co dinh
```

3.2 Dr. GRPO: GRPO Done Right

Giải pháp: **bỏ hẳn** hai hạng tử gây bias, $1/|o_i|$ và $\text{std}(\{R_1, \ldots, R_G\})$, chỉ giữ lại advantage trung tâm hoá (centered) đơn giản:

$$\tilde{A}_{i,t} = R(q, o_i) - \text{mean}(\{R(q, o_1), \ldots, R(q, o_G)\})$$

và dùng đúng mục tiêu PPO gốc (sum theo t, không chia $|o_i|$):

$$\boxed{J^{\text{Dr.GRPO}}(\pi_\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\big(w_{i,t}(\theta)\tilde{A}_{i,t},\ \text{clip}(w_{i,t}(\theta), 1-\epsilon, 1+\epsilon)\tilde{A}_{i,t}\big)\Big]}$$

Về mặt thực hành, chỉ cần thay `mask.sum(axis=dim)` trong `masked_mean` bằng một **hằng số cố định** (ví dụ generation budget tối đa) như trong đoạn code ở trên.

Vì sao đây là "không chệch" (unbiased): Phụ lục A của bài báo dẫn lại đạo hàm policy gradient Monte Carlo cổ điển (REINFORCE với baseline, xem lại bài 00):

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{q,o}\Big[\sum_{t=1}^{|o|}\nabla_\theta \log \pi_\theta(o_t|q, o_{<t})\Big(\sum_{t'=t}^{|o|} r(q, o_{\le t'}) - B(q, o_{<t})\Big)\Big]$$

với $B(q, o_{<t})$ là baseline bất kỳ **không phụ thuộc** o_t, nên không làm lệch kỳ vọng gradient (chứng minh bằng $\sum_{o_t} \pi_\theta(o_t|\cdot)\nabla_\theta \log \pi_\theta(o_t|\cdot) = \nabla_\theta \sum_{o_t} \pi_\theta(o_t|\cdot) = \nabla_\theta 1 = 0$). Với outcome reward, $\sum_{t' \geq t} r(q, o_{\leq t'}) = R(q, o)$ (không phụ thuộc t), và chọn $B(q, o_{<t}) = \text{mean}(\{R_1, \ldots, R_G\})$ (baseline Monte Carlo ước lượng bằng trung bình nhóm) cho đúng $\tilde{A}_{i,t}$ ở trên: **không hề cần chia cho** $|o_i|$ **hay** $\text{std}(R)$ để có một ước lượng hợp lệ.

Liên hệ với RLOO (bài 37): paper chỉ ra $\tilde{A}_{i,t}$ tương đương (sai khác một hệ số nhân, có thể gộp vào learning rate) với advantage của REINFORCE Leave-One-Out:

$$\frac{G}{G-1}\tilde{A}_{i,t} = R(q,o_i) - \frac{1}{G-1}\sum_{j=1}^{G} R(q,o_j) = \frac{G}{G-1}R(q,o_i) - \frac{1}{G-1}\sum_{j\neq i} R(q,o_j) - \frac{1}{G-1}R(q,o_i) = \hat{A}_{i,t}^{RLOO}$$

(biến đổi đại số trực tiếp từ định nghĩa, paper trình bày ở Appendix A).

3.3 Bias length trong góc nhìn gradient per-token (tự tính, minh hoạ thêm)

Giả sử response A đúng, dài 10 token, advantage trước chuẩn hoá $= +1$; response B sai, dài 100 token, advantage trước chuẩn hoá $= -1$. Với GRPO (chia $1/|o_i|$), **đóng góp trung bình trên mỗi token** vào gradient tổng xấp xỉ $1/10 = 0.1$ cho A và $-1/100 = -0.01$ cho B: response sai dài bị phạt **10 lần yếu hơn mỗi token** so với mức được khen của response đúng ngắn. Với Dr. GRPO (không chia $|o_i|$), mỗi token của A và mỗi token của B đóng góp cùng biên độ ± 1 (trước khi nhân learning rate), không còn thiên vị theo độ dài.

4. Thí nghiệm và kết quả chính

Khung thực nghiệm chính: framework Oat (tự phát triển, tối ưu cho online RL), Qwen2.5-1.5B base + R1 template, huấn luyện trên tập MATH, reward nhị phân $\{0, 1\}$ (dùng Math-Verify để kiểm đáp án), so sánh GRPO và Dr. GRPO trên 5 benchmark: AIME2024, AMC, MATH500, Minerva Math, OlympiadBench.

Siêu tham số (Bảng 6, dùng cho mọi thực nghiệm): max response length 3000 token; temperature 1.0; (top-p, top-k) = (1.0, -1); 8 response/câu hỏi; AdamW, $(\beta_1, \beta_2) = (0.9, 0.95)$; weight decay 0.0; gradient clip 1.0; learning rate hằng số 1×10^{-6}; 1 epoch nội bộ (inner proximal update); hệ số KL = 0.0; clip $\epsilon = 0.2$. Hạ tầng: 8×A100, ~1 ngày cho mỗi thực nghiệm.

Kết quả (Hình 5): cả GRPO và Dr. GRPO đều cho reward và độ dài tăng cùng nhịp giai đoạn đầu (giống DeepSeek-R1-Zero), nhưng GRPO **tiếp tục** tăng độ dài ngay cả khi reward đã chững lại, còn Dr. GRPO **ngăn** độ dài "nổ" không kiểm soát. Trên benchmark đánh giá, độ dài của các response **sai** giảm mạnh với Dr. GRPO so với baseline, cho thấy bớt hiện tượng overthinking.

Recipe tối giản (minimalist recipe): Qwen2.5-Math-7B + Dr. GRPO + câu hỏi MATH mức 3-5 + Qwen-Math template → **43.3% AIME 2024**, SOTA tại thời điểm công bố, chỉ với **27 giờ compute trên 8×A100**.

Bảng 4 (trích, so sánh scale 1.5B/3B/7B, đơn vị % trên AIME24/AMC/MATH500/Minerva/OlympiadBench/Avg):

Mô hình	AIME24	AMC	MATH500	Minerva	OlympiadBench	Avg
Qwen2.5-Math-1.5B (4-shot)	20.0	32.5	33.0	12.5	22.8	24.2
Oat-Zero-1.5B	20.0	53.0	74.2	25.7	37.6	42.1
R1-Distill-Qwen-1.5B @3k	2.5	21.7	52.2	16.3	17.3	22.0
Llama-3.2-3B	0.0	2.4	6.4	6.3	1.3	3.3
+ RL Dr. GRPO	3.3	7.2	10.0	11.0	2.2	6.8
Llama-3.2-3B-FineMath	0.0	3.6	18.4	5.9	2.2	6.0
+ RL Dr. GRPO	3.3	10.8	38.0	12.9	9.0	14.8

Mô hình	AIME24	AMC	MATH500	Minerva	OlympiadBench	Avg
Llama-3.2-3B-NuminaQA	0.0	0.0	0.6	0.0	0.1	0.14
+ RL Dr. GRPO (Oat-Zero-3B)	6.7	18.1	50.0	14.3	14.7	20.7
Qwen2.5-Math-7B (no template*)	0.2	45.8	69.0	21.3	34.7	38.2
SimpleRL-Zero-7B	26.7	60.2	78.2	27.6	40.3	46.6
PRIME-Zero-7B	16.7	62.7	83.8	36.0	40.9	48.0
OpenReasoner-Zero-7B @8k	13.3	54.2	82.4	31.6	47.9	45.9
Oat-Zero-7B (Dr. GRPO, recipe tối giản)	**43.3**	62.7	80.0	30.1	41.0	**51.4**
R1-Distill-Qwen-7B @8k	33.3	68.4	88.1	35.9	47.7	54.7
Qwen2.5-Math-7B-Instruct	16.7	53.0	83.6	29.8	42.7	45.1

(* "no template" = dùng template tốt nhất cho mô hình nền để phản ánh đúng năng lực nền, không phải cấu hình RL cuối.) Lưu ý: Oat-Zero-7B có AIME24 cao nhất trong nhóm "R1-Zero-like, train từ base" (43.3), nhưng **avg tổng thể vẫn thấp hơn** R1-Distill-Qwen-7B (một mô hình **distill** từ R1 chứ không phải train từ đầu bằng RL, avg 54.7 so với 51.4 của Oat-Zero-7B).

5. Phân tích / ablation

Ablation hai bias riêng biệt (Hình 8, mục C phụ lục): so 4 biến thể trên Qwen2.5-1.5B, 3K câu hỏi hỗn hợp (ASDiv, MATH, AIME trước 2023): Dr. GRPO, GRPO không length-norm, GRPO không std-norm, GRPO gốc. Cả Dr. GRPO và biến thể "không length-norm" sinh response ngắn hơn rõ rệt so với 2 biến thể còn lại, xác nhận length-norm (`1/|o_i|`) là nguồn chính của việc "nổ" độ dài. Về hiệu năng, bỏ bias nào (length hoặc std) cũng cải thiện so với GRPO gốc.

Ý nghĩa thống kê (Hình 9, 3 lần lặp độc lập): Dr. GRPO nhất quán vượt GRPO cả về token-efficiency và accuracy cuối, không phải do may mắn một seed.

Template × độ phủ câu hỏi (mục 3.3): thử R1/Qwen-Math/No template × 4 bộ câu hỏi độ khó/độ phủ khác nhau (Bảng 3: ORZ 57K đa dạng, MATH 12K, GSM 8K đơn giản, ASDiv 2K cơ bản). Quan sát: (1) template quyết định điểm khởi đầu của policy, nhưng RL có thể nâng mọi policy lên mức tương đương ~40% nếu chọn đúng bộ câu hỏi; (2) với R1 template, độ phủ câu hỏi ảnh hưởng mạnh tới trần hiệu năng cuối (phủ hẹp → trần thấp); (3) với Qwen-Math template, huấn luyện trên **GSM-8K đơn giản hơn** lại cho kết quả cuối **tốt nhất** trên benchmark khó hơn (gần gấp đôi so với một số lựa chọn khác), cho thấy khi mô hình nền đã có sẵn năng lực, RL trên dữ liệu dễ/ít liên quan vẫn có thể "khơi lại" hành vi suy luận có sẵn, không cần dữ liệu mới.

Pretraining miền toán nâng trần RL (mục 3.4): Llama-3.2-3B (yếu về toán) chỉ cải thiện rất ít sau RL (avg 3.3 → 6.8). Sau khi continual-pretrain trên FineMath (6.0 → 14.8 sau RL) hoặc trên dữ liệu ghép câu hỏi-đáp kiểu NuminaQA (gần 0 → 20.7 sau RL với Dr. GRPO, mô hình "Oat-Zero-3B"), trần RL được nâng rõ rệt. Đồng thời, GRPO gốc trên Llama vẫn thể hiện "double-increase" (độ dài và reward cùng tăng) dễ bị hiểu nhầm là long-CoT "nổi lên", nhưng Dr. GRPO cho thấy phần tăng độ dài đó phần lớn là bias, không phải năng lực mới.

Phần phân tích mô hình nền (mục 2, tóm tắt): DeepSeek-V3-Base (mô hình nền gốc thật của R1-Zero) đã thể hiện self-reflection (từ khoá "Aha", "wait") **trước khi RL**. Qwen2.5-Math đạt accuracy cao nhất khi **không dùng template** (Bảng 1: no-template cải thiện trung bình ~60% so với 4-shot prompting), gợi ý khả năng được pretrain trên dữ liệu QA ghép liền. Khi phân tích tương quan self-reflection và accuracy trên DeepSeek-R1-Zero thật (Hình 15), **gần một nửa** các câu hỏi có response tự phản tư không đạt accuracy cao hơn các response không tự phản tư cho cùng câu hỏi đó, nghĩa là self-reflection không hiển nhiên đồng nghĩa

với chính xác hơn khi suy luận (dù có thể vẫn hữu ích cho *khám phá* trong quá trình huấn luyện, điều nằm ngoài phạm vi phân tích này của bài báo).

6. Giới hạn

Tác giả nêu: phân loại "có trả lời câu hỏi hay không" và "có self-reflection hay không" dựa một phần vào GPT-4o-mini làm giám khảo, có thể gây nhiễu; bộ từ khoá self-reflection được chọn lọc chủ ý nhỏ (chỉ các từ mạnh như "recheck", "reevaluate"...) để giảm false positive, nhưng có thể bỏ sót hành vi tự phản tư ngầm không dùng từ khoá rõ ràng. Compute hạn chế ở 8×A100/khoảng 1 ngày mỗi thực nghiệm.

Nhận xét:

- Giả thuyết "Qwen2.5-Math pretrain trên văn bản câu hỏi-đáp ghép liền" chỉ là suy đoán hợp lý từ hiện tượng quan sát (no-template tốt hơn template), Qwen không công bố dữ liệu pretraining để xác nhận trực tiếp.
- Kết quả SOTA 43.3% AIME24 là **một lần chạy lớn** (không thấy nói rõ số seed cho riêng con số headline này ở quy mô 7B), trong khi phần ablation hai bias (Hình 9) có 3 seed nhưng ở quy mô 1.5B nhỏ hơn; mức độ ổn định của số 43.3% qua nhiều seed ở 7B không được báo cáo.
- Dr. GRPO chỉ kiểm chứng tới quy mô 7B, chưa có bằng chứng ở quy mô hàng trăm tỷ tham số (như DeepSeek-R1 thật, 671B) nơi hai bias có thể biểu hiện khác đi.
- Bài 44 (ScaleRL) sau này so sánh 3 kiểu chuẩn hoá advantage (prompt-level std như GRPO, batch-level std, không chuẩn hoá std như Dr. GRPO) ở quy mô lớn hơn nhiều (16.000 GPU-giờ mỗi lần chạy) và thấy **cả ba cho hiệu năng tương tự nhau trong sai số**, chọn batch-level std chủ yếu vì "lý thuyết chặt hơn một chút" chứ không phải vì khác biệt lớn như claim gốc của Dr. GRPO. Điều này gợi ý bias (b) (question-level difficulty, do std theo-từng-câu-hỏi) có thể quan trọng hơn ở quy mô nhỏ/sớm trong huấn luyện, và ít quyết định hơn khi đã có nhiều cơ chế ổn định khác (ScaleRL dùng nhiều kỹ thuật cộng dồn, không chỉ một mình chuẩn hoá advantage).

7. Liên hệ với các bài khác

- **Bài 00, 36:** nền tảng PPO/GRPO cần nắm trước.
- **Bài 39 (DeepSeek-R1-Zero):** đối tượng phản biện trực tiếp của cả 2 phần phân tích trong bài này.
- **Bài 41 (DAPO):** cùng nhận ra length bias trong GRPO (DAPO: "sample-level loss khiến response dài bị pha loãng trọng số"), nhưng chọn hướng **đổi cách cộng dồn** (sample-level → token-level, vẫn giữ `std(R)`) thay vì loại bỏ hẳn `1/|o_i|` và `std(R)` như Dr. GRPO. Nên đọc nối tiếp hai bài để thấy rõ hai cách tiếp cận khác nhau cho cùng một triệu chứng.
- **Bài 37 (RLOO):** $\tilde{A}_{i,t}$ của Dr. GRPO tương đương (sai khác hệ số nhân) với advantage RLOO, một liên hệ toán học trực tiếp được chứng minh ở Phụ lục A của bài này.
- **Bài 43 (GSPO):** vẫn giữ token-level importance ratio (như Dr. GRPO), nhưng cho rằng vấn đề nằm sâu hơn ở chính đơn vị của ratio (token vs sequence), không chỉ ở cách chuẩn hoá advantage.
- **Bài 44 (ScaleRL):** kiểm định lại vai trò của chuẩn hoá std ở quy mô lớn hơn (xem phần Giới hạn ở trên); dùng "không chuẩn hoá std" của Dr. GRPO như một trong ba lựa chọn so sánh, không coi đó là lựa chọn bắt buộc tốt nhất.

8. Ứng dụng thực tế (suy luận)

- Nếu đang chạy một quy trình RLVR/GRPO và thấy **độ dài response tăng liên tục** mà accuracy không tăng tương xứng, đừng vội coi đó là "reasoning dài hơn đang hình thành": trước tiên kiểm tra xem có đang dùng chuẩn hoá `1/|o_i|` ở sample-level hay không, và thử tắt nó đi (hoặc dùng một hằng số cố định) để xem độ dài có còn tăng theo cùng cách không.
- Trước khi RL-tune một mô hình nền, nên thử **nhiều template khác nhau** (bao gồm "không template") trên một mẫu nhỏ, đo pass@k ở nhiệt độ khác nhau: nếu mô hình nền không exploratory dưới template

bạn chọn, RL sẽ không có gì để học.

- Chuẩn hoá advantage theo `std` trong một nhóm nhỏ ($G = 8, 16$ response) khiến các câu hỏi cực dễ/cực khó bị khuếch đại trọng số; nếu dữ liệu train có độ khó rất không đồng đều, cân nhắc chuẩn hoá theo **toàn batch** thay vì theo từng câu hỏi riêng (đây cũng là lựa chọn ScaleRL áp dụng, xem bài 44).

9. Thí nghiệm gợi ý

1. Trên một tác vụ RLVR nhỏ (ví dụ một tập con GSM8K, mô hình 0.5-1.5B), chạy song song GRPO và Dr. GRPO, theo dõi riêng độ dài trung bình của response **đúng** và response **sai** theo bước huấn luyện, kiểm chứng lại Hình 5 của bài báo ở quy mô nhỏ hơn nhiều.
2. Tái lập ablation Hình 8 ở quy mô tối thiểu: tách riêng "chỉ bỏ length-norm" và "chỉ bỏ std-norm" để xem cái nào ảnh hưởng độ dài response nhiều hơn trên dữ liệu đang xét.
3. Kiểm tra độ nhạy template: cùng một mô hình nền và cùng cấu hình RLVR, chạy với R1 template so với không template, đo pass@8 ban đầu và accuracy cuối, xem mức độ "mismatch model-template" có phá huỷ năng lực ban đầu rồi RL phải "xây lại" như paper quan sát trên Qwen2.5-Math-1.5B không.
4. Trên log huấn luyện RLVR đang xét, đo tương quan giữa (có/không có từ khoá tự phản tư trong response) và (đúng/sai), xem có tái lập được quan sát "self-reflection không tương quan rõ với accuracy" (Hình 15 của bài báo) hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
GRPO advantage	$\hat{A}_{i,t} = \dfrac{R_i - \text{mean}(R)}{\text{std}(R)}$	hai phép chia gây bias
Bias (a): length	chia $1/\lvert o_i\rvert$ trong loss	ưu tiên đúng-ngắn, ít phạt sai-dài
Bias (b): difficulty	chia $1/\text{std}(R)$ theo từng câu hỏi	khuếch đại câu hỏi cực dễ/cực khó
Dr. GRPO advantage	$\tilde{A}_{i,t} = R_i - \text{mean}(R)$	không chia $\lvert o_i\rvert$, không chia std
Dr. GRPO objective	PPO gốc + Monte Carlo baseline, không mẫu số $\lvert o_i\rvert$	tương đương REINFORCE + baseline
Liên hệ RLOO	$\frac{G}{G-1}\tilde{A}_{i,t} = \hat{A}^{RLOO}_{i,t}$	sai khác hệ số nhân (gộp vào lr)
Recipe tối giản	Qwen2.5-Math-7B + Dr. GRPO + MATH lv.3-5 + Qwen-Math template	43.3% AIME24, 27 giờ/8×A100
Template	R1 / Qwen-Math / No template	base model khác nhau cần template khác nhau

11. Kết luận ngắn

Bài này tách bạch hai câu chuyện thường bị gộp chung khi nói về "R1-Zero training": một phần những gì tưởng là "RL tạo ra" (self-reflection, năng lực hỏi-đáp) thực ra **đã có sẵn** trong mô hình nền nếu dùng đúng template; và một phần tăng độ dài response quan sát được trong lúc train **không hoàn toàn** phản ánh reasoning sâu hơn, mà một phần là hệ quả của hai bias toán học cụ thể trong công thức GRPO (chia theo độ dài response, chia theo độ lệch chuẩn nhóm). Dr. GRPO sửa phần bias này bằng một thay đổi rất nhỏ (bỏ hai phép chia), cho kết quả hiệu quả token tốt hơn mà không giảm accuracy, và khi kết hợp với lựa chọn mô hình nền/template phù hợp, đạt SOTA 43.3% AIME 2024 với compute khiêm tốn (27 giờ, 8×A100).

Link gốc: https://arxiv.org/abs/2503.20783

Bài 43

Group Sequence Policy Optimization (2025)

Tác giả: Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, Junyang Lin (Qwen Team, Alibaba Inc.). **Venue / arXiv:** arXiv:2507.18071 (v2, 28/07/2025); bài báo không nêu venue (chỉ có mốc thời gian "2025-07-29" ở đầu trang 1). **Nguồn:** https://arxiv.org/abs/2507.18071 **Chủ đề:** Chỉ ra importance ratio ở mức token của GRPO/PPO là "ill-posed" về lý thuyết importance sampling; đề xuất GSPO, một importance ratio ở mức **sequence** (có chuẩn hoá theo độ dài), với clip/reward/optimize đồng bộ ở mức sequence. Ổn định hoá huấn luyện Mixture-of-Experts (MoE), loại bỏ nhu cầu "Routing Replay". **Vị trí trong lộ trình:** đọc sau bài 00, bài 36 (GRPO gốc). Đọc cùng nhóm với bài 41 (DAPO, cùng than phiền về token-level ratio nhưng sửa clip-range/cách cộng dồn, giữ nguyên đơn vị token) và bài 42 (Dr. GRPO, sửa chuẩn hoá advantage, vẫn giữ đơn vị token). Bài 44 (ScaleRL) đánh giá định lượng GSPO cùng DAPO, CISPO dưới một khung chung và nêu một số hạn chế thực nghiệm của GSPO ở quy mô lớn hơn.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Scale RL compute cho LLM (mô hình lớn hơn, sparsity cao hơn như MoE, response dài hơn) đòi hỏi rollout batch lớn để dùng hết hardware. Để tăng hiệu quả mẫu, thực hành chuẩn là chia một batch rollout lớn thành nhiều mini-batch để cập nhật gradient nhiều lần, điều này tự động tạo ra tình huống **off-policy**: response y được sample từ $\pi_{\theta_{old}}$ (chính sách "cũ" tại lúc rollout) nhưng mục tiêu lại tối ưu π_θ (chính sách đang cập nhật trong các mini-batch sau đó). Đây chính là lý do PPO/GRPO cần cơ chế **clip** để chặn các mẫu "quá lệch pha" (off-policy quá xa) tham gia ước lượng gradient.

Nhưng tác giả cho rằng clip chỉ là "chữa triệu chứng", còn **vấn đề gốc** nằm ở chính cách định nghĩa importance ratio của GRPO: nó bị áp dụng sai nguyên lý importance sampling. Khi train các mô hình MoE lớn, GRPO (theo bài báo, dẫn kinh nghiệm thực tế từ Qwen và MiniMax) thường gây **sụp đổ mô hình** (model collapse) nghiêm trọng và **không hồi phục được**.

Nhắc lại các khái niệm cần có:

- **Likelihood của một response:** $\pi_\theta(y|x) = \prod_{t=1}^{|y|} \pi_\theta(y_t|x, y_{<t})$.
- **PPO** (bỏ KL cho ngắn gọn, theo quy ước của bài này):

$$J^{PPO}(\theta) = \mathbb{E}_{x\sim D, y\sim\pi_{\theta_{old}}}\Big[\frac{1}{|y|}\sum_{t=1}^{|y|}\min\left(w_t(\theta)\hat{A}_t,\ \text{clip}(w_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\Big], \quad w_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{old}}(y_t|x, y_{<t})}$$

- **GRPO**: thay mô hình giá trị bằng advantage chuẩn hoá theo nhóm G response cùng một query x:

$$J^{GRPO}(\theta) = \mathbb{E}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\Big(w_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}(w_{i,t}(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\Big)\Bigg].$$

$$\hat{A}_{i,t} = \hat{A}_i = \frac{r(x, y_i) - \text{mean}(\{r(x, y_i)\}_{i=1}^{G})}{\text{std}(\{r(x, y_i)\}_{i=1}^{G})}.$$

(mọi token trong y_i dùng chung advantage $\hat{A}_i$, chỉ khác nhau ở $w_{i,t}$.)

- **Nguyên lý importance sampling (nhắc lại, bài 00):** để ước lượng $\mathbb{E}_{z\sim\pi_{tar}}[f(z)]$ bằng mẫu từ π_{beh}, dùng $\mathbb{E}_{z\sim\pi_{tar}}[f(z)] = \mathbb{E}_{z\sim\pi_{beh}}[\frac{\pi_{tar}(z)}{\pi_{beh}(z)}f(z)]$. Điểm mấu chốt: công thức này **cần lấy trung bình trên nhiều mẫu** ($N \gg 1$) từ π_{beh} để trọng số $\pi_{tar}(z)/\pi_{beh}(z)$ thực sự điều chỉnh đúng sự lệch phân phối.

2. Ý tưởng chính

GRPO áp dụng trọng số importance $w_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x,y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t}|x,y_{i,<t})}$ tại **từng vị trí token**. Nhưng trọng số này được tính từ **một mẫu duy nhất** $y_{i,t}$ rút ra từ phân phối $\pi_{\theta_{old}}(\cdot|x,y_{i,<t})$ tại đúng vị trí đó, nó **không phải** trung bình trên nhiều mẫu như nguyên lý importance sampling yêu cầu. Vì vậy, ratio này **không thực hiện đúng vai trò hiệu chỉnh phân phối**, mà ngược lại **bơm nhiễu phương sai cao** vào gradient, nhiễu này **tích luỹ dọc theo chuỗi dài** và **bị khuếch đại thêm bởi cơ chế clip**, dẫn tới sụp đổ mô hình (model collapse) đã quan sát thực tế, đôi khi không thể hồi phục dù reset checkpoint, chỉnh clip-range, kéo dài generation, hay đổi query.

Nguyên tắc rút ra: đơn vị của mục tiêu tối ưu nên **khớp** với đơn vị của reward. Vì reward chỉ được cấp **một lần cho toàn bộ sequence** (outcome reward), áp dụng hiệu chỉnh off-policy ở mức **token** là không khớp đơn vị. GSPO (**G**roup **S**equence **P**olicy **O**ptimization) chuyển hẳn sang định nghĩa importance ratio, clip, reward, và tối ưu đều ở mức **sequence**.

3. Phương pháp chi tiết

3.1 Định nghĩa GSPO

Importance ratio ở mức sequence, có chuẩn hoá theo độ dài (length-normalized geometric mean của token-ratio):

$$\boxed{s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{old}}(y_i|x)}\right)^{1/|y_i|} = \exp\left(\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\log\frac{\pi_\theta(y_{i,t}|x,y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t}|x,y_{i,<t})}\right)}$$

Advantage: giữ **giống GRPO**, chuẩn hoá theo nhóm:

$$\hat{A}_i = \frac{r(x,y_i) - \text{mean}(\{r(x,y_i)\}_{i=1}^G)}{\text{std}(\{r(x,y_i)\}_{i=1}^G)}$$

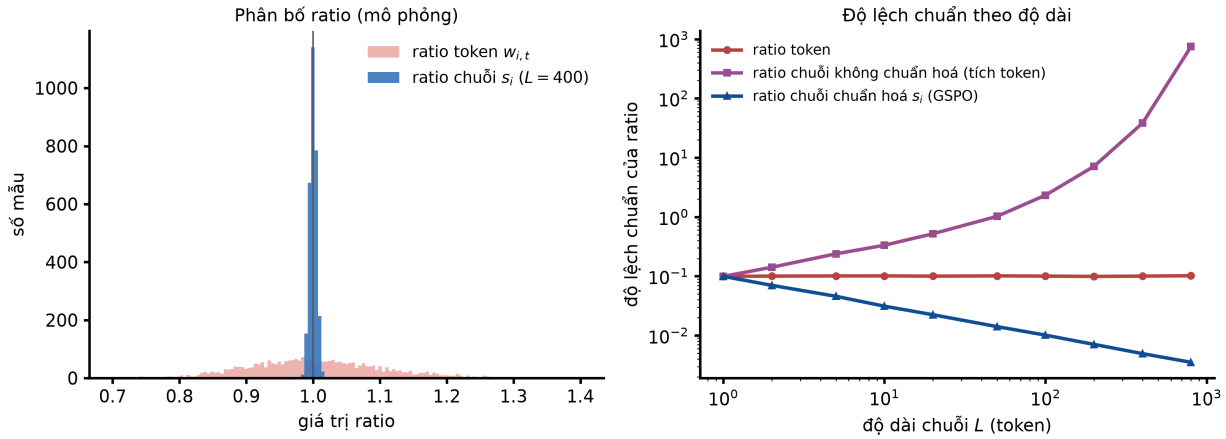
Mục tiêu GSPO:

$$J^{GSPO}(\theta) = \mathbb{E}_{x\sim D,\{y_i\}_{i=1}^G\sim\pi_{\theta_{old}}(\cdot|x)}\left[\frac{1}{G}\sum_{i=1}^{G}\min\left(s_i(\theta)\hat{A}_i,\ \text{clip}(s_i(\theta),1-\epsilon,1+\epsilon)\hat{A}_i\right)\right]$$

Khác biệt cốt lõi so với GRPO: **clip áp dụng cho toàn bộ response** (loại hẳn response "quá off-policy" khỏi gradient, không phải loại từng token riêng lẻ), khớp cả về reward (cấp cho cả response) và optimize (cùng một đơn vị sequence).

Vì sao cần chuẩn hoá theo độ dài ($1/|y_i|$) trong định nghĩa $s_i(\theta)$: nếu không chuẩn hoá (dùng trực tiếp $\pi_\theta(y_i|x)/\pi_{\theta_{old}}(y_i|x)$, tích của $|y_i|$ số hạng), chỉ cần vài token thay đổi likelihood nhẹ cũng khiến tích này dao động rất mạnh (do nhân dồn qua nhiều bước), và response dài/ngắn khác nhau sẽ cần clip-range khác nhau để so sánh công bằng. Lấy trung bình hình học (geometric mean, tức mũ $1/|y_i|$) giữ $s_i(\theta)$ trong một khoảng số trị ổn định, không phụ thuộc độ dài.

Cách đọc hình: mô phỏng giả định log-ratio của mỗi token độc lập, chuẩn 0.1, không phải số của paper. Bên trái, ratio token trải rộng còn s_i của chuỗi dài 400 token co sát quanh 1. Bên phải (thang log), tích token không



Hình 78: Mô phỏng độ dao động của ratio ở mức token, mức chuỗi không chuẩn hoá và mức chuỗi chuẩn hoá theo độ dài (minh hoạ, tự tính)

chuẩn hoá bùng nổ theo độ dài, còn trung bình hình học s_i giảm dần như $1/\sqrt{L}$, nên ổn định hơn khi chuỗi dài.

Về độ lớn clip-range: vì $s_i(\theta)$ là trung bình hình học của nhiều tỉ số gần 1 (mỗi tỉ số token riêng dao động quanh 1, rồi lấy trung bình log), bản thân $s_i(\theta)$ nằm gần 1 hơn nhiều so với một tỉ số token đơn lẻ $w_{i,t}(\theta)$. Do đó **clip-range của GSPO khác GRPO cả về bậc độ lớn**: thực nghiệm của bài báo dùng $\epsilon_{low} = 3 \times 10^{-4}$, $\epsilon_{high} = 4 \times 10^{-4}$ cho GSPO, so với $\epsilon_{low} = 0.2$, $\epsilon_{high} = 0.27$ cho baseline GRPO (đã được paper "tinh chỉnh cẩn thận" để so sánh công bằng).

3.2 Phân tích gradient (so sánh trực tiếp GSPO và GRPO)

Đạo hàm J^{GSPO} (bỏ clip để dễ nhìn, dùng $\nabla_\theta s_i(\theta) = s_i(\theta)\nabla_\theta \log s_i(\theta)$ và $\nabla_\theta \log s_i(\theta) = \frac{1}{|y_i|}\sum_t \nabla_\theta \log \pi_\theta(y_{i,t}|x, y_{i,<t})$ vì $\pi_{\theta_{old}}$ không phụ thuộc θ):

$$\nabla_\theta J^{GSPO}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} s_i(\theta)\hat{A}_i \cdot \frac{1}{|y_i|}\sum_{t=1}^{|y_i|} \nabla_\theta \log \pi_\theta(y_{i,t}|x, y_{i,<t})\right]$$

So sánh với đạo hàm J^{GRPO}:

$$\nabla_\theta J^{GRPO}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \cdot \frac{1}{|y_i|}\sum_{t=1}^{|y_i|} w_{i,t}(\theta)\, \nabla_\theta \log \pi_\theta(y_{i,t}|x, y_{i,<t})\right]$$

Khác biệt bản chất: ở GRPO, mỗi token t trong y_i được nhân với trọng số importance **riêng của chính nó**, $w_{i,t}(\theta)$, có thể nằm bất kỳ đâu trong $(0, 1+\epsilon]$ (khi $\hat{A}_i > 0$) hoặc $[1-\epsilon, +\infty)$ (khi $\hat{A}_i < 0$); các trọng số này **không đồng nhất** giữa các token, và hiệu ứng cộng dồn của chúng qua một sequence dài là khó dự đoán, có thể tích luỹ thành nhiễu lớn. Ở GSPO, **mọi token trong cùng một response chia sẻ đúng một trọng số vô hướng** $s_i(\theta)\hat{A}_i$ (không đổi theo t), loại bỏ hoàn toàn nguồn bất ổn "trọng số khác nhau giữa các token trong cùng câu".

3.3 GSPO-token: biến thể cho advantage theo từng token

Một số tình huống (ví dụ multi-turn RL) cần advantage khác nhau cho từng token, không chỉ một giá trị chung cho cả response. GSPO-token dùng kỹ thuật "stop-gradient" (`sg[.]`, tương đương `.detach()` trong

PyTorch) để tách phần giá trị số và phần lấy gradient:

$$s_{i,t}(\theta) = \text{sg}[s_i(\theta)] \cdot \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\text{sg}[\pi_\theta(y_{i,t}|x, y_{i,<t})]}$$

$$J^{GSPO\text{-}token}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\left(s_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}(s_{i,t}(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\right)\right]$$

Vì $\frac{\pi_\theta(y_{i,t}|\cdot)}{\text{sg}[\pi_\theta(y_{i,t}|\cdot)]}$ có **giá trị số bằng 1** (tử và mẫu bằng nhau về giá trị, chỉ khác ở việc mẫu bị chặn gradient), $s_{i,t}(\theta)$ **bằng số** với $s_i(\theta)$ (không đổi giá trị clip/reward), nhưng khi lấy đạo hàm, gradient lại "chảy" qua từng token riêng biệt. Paper chứng minh: nếu đặt $\hat{A}_{i,t} = \hat{A}_i$ cho mọi t (advantage chung cho cả response, như GSPO gốc), GSPO-token **giống hệt về số trị** mục tiêu, điều kiện clip, và gradient lý thuyết với GSPO; GSPO-token chỉ thêm linh hoạt khi cần advantage khác nhau theo từng token.

4. Thí nghiệm và kết quả chính

Thiết lập: mô hình cold-start tinh chỉnh từ Qwen3-30B-A3B-Base (MoE). Benchmark theo dõi: AIME'24 (avg pass@1 trên 32 lần sample), LiveCodeBench (giai đoạn 202410-202502, avg pass@1 trên 8 lần sample), CodeForces (Elo rating). Mỗi batch rollout chia thành 4 mini-batch để cập nhật gradient. GSPO: $\epsilon_{low} = 3 \times 10^{-4}$, $\epsilon_{high} = 4 \times 10^{-4}$. Baseline GRPO: $\epsilon_{low} = 0.2$, $\epsilon_{high} = 0.27$ (đã tinh chỉnh cẩn thận để so sánh công bằng), **và cần thêm chiến lược Routing Replay** (xem mục 5) để hội tụ bình thường cho MoE, còn GSPO không cần.

Kết quả (Hình 1, mô tả định lượng từ paper): GSPO huấn luyện ổn định suốt quá trình; hiệu năng cải thiện liên tục khi tăng compute, cập nhật đều tập query, và kéo dài generation length; GSPO đạt training reward, AIME'24, LiveCodeBench, CodeForces tốt hơn GRPO (w/ Routing Replay) ở **cùng mức compute và số query đã dùng**. GSPO sau đó được dùng để huấn luyện các mô hình Qwen3 mới nhất.

Lưu ý về mức độ cụ thể của số liệu: paper **không nêu giá trị số cuối cùng chính xác** (ví dụ "AIME'24 đạt X%") trong văn bản chính, chỉ trình bày qua đồ thị đường cong (Hình 1) với trục tung dạng khoảng (ví dụ AIME'24 khoảng 70-80, LiveCodeBench khoảng 55-65, CodeForces khoảng 1800-2000 Elo) và mô tả định tính ("vượt trội", "cải thiện liên tục"). Đây là một khác biệt rõ với bài 41 (DAPO) và 42 (Dr. GRPO), hai bài có bảng số liệu tường minh.

5. Phân tích / ablation

Tỉ lệ token bị clip (mục 5.2, Hình 2): paper quan sát **khác biệt hai bậc độ lớn** giữa tỉ lệ token bị clip của GSPO và GRPO (ngay cả khi đổi clip-range, khác biệt bậc độ lớn này không đổi). Paper nêu rõ một số, tỉ lệ clip của GRPO khoảng **0.0013** (0.13%); tỉ lệ của GSPO cao hơn khoảng hai bậc độ lớn so với số này theo mô tả bằng lời của bài báo (giá trị chính xác của GSPO chỉ thể hiện trên đồ thị, không được nêu bằng số trong văn bản, nên người ghi chú không khẳng định con số chính xác, chỉ dẫn lại mức "hai bậc độ lớn" mà paper phát biểu). Điểm đáng chú ý: **mặc dù GSPO clip bỏ nhiều token hơn hẳn** (dùng ít dữ liệu hơn cho mỗi lượt cập nhật gradient), GSPO **vẫn** đạt hiệu quả huấn luyện cao hơn GRPO. Paper diễn giải điều này là bằng chứng cho thấy gradient per-token của GRPO "vốn đã nhiễu và không hiệu quả", nên việc GSPO clip bỏ nhiều hơn nhưng học tốt hơn không hề nghịch lý: các token mà GSPO loại bỏ phần lớn là "nhiễu không đáng tin" chứ không phải tín hiệu học có giá trị.

Ổn định huấn luyện MoE (mục 5.3): với mô hình MoE (48 lớp, Qwen3-30B-A3B-Base), sau **mỗi lần cập nhật gradient**, khoảng **10% expert** được kích hoạt cho cùng một response lại **khác nhau** giữa π_θ (mới) và $\pi_{\theta_{old}}$ (cũ). Điều này làm token-level ratio $w_{i,t}(\theta)$ dao động mạnh hơn nữa (vì hai mạng con được kích hoạt có thể khác nhau, không chỉ trọng số thay đổi), khiến GRPO token-level **mất hiệu lực** nghiêm trọng hơn ở MoE so với mô hình dense.

Cách xử lý trước đây (Routing Replay): lưu lại expert routing của $\pi_{\theta_{old}}$ và "replay" đúng các expert đó khi tính π_θ cho ratio, để tử và mẫu của $w_{i,t}(\theta)$ dùng **chung một mạng con được kích hoạt**, phục hồi tính hợp lệ của token-ratio. Hình 3 của bài báo cho thấy nếu bỏ Routing Replay, GRPO với MoE **không hội tụ bình thường** (reward dao động thấp, không tăng ổn định).

Vì sao GSPO không cần Routing Replay: $s_i(\theta)$ chỉ phụ thuộc **likelihood của toàn sequence** $\pi_\theta(y_i|x)$ (năng lực ngôn ngữ tổng thể của mô hình), không nhạy với việc expert nào được kích hoạt ở từng bước riêng lẻ (năng lực ngôn ngữ tổng thể của MoE không dao động mạnh dù routing có thay đổi cục bộ). Routing Replay tốn thêm bộ nhớ/giao tiếp và có thể hạn chế năng lực thực tế của MoE (vì ép dùng lại routing cũ); GSPO loại bỏ hẳn nhu cầu này.

Lợi ích cho hạ tầng RL (mục 5.4): vì engine train (ví dụ Megatron) và engine infer (ví dụ SGLang, vLLM) thường có sai lệch số học nhỏ khi tính likelihood, cần "tính lại" likelihood bằng engine train cho $\pi_{\theta_{old}}$ trong thực hành thông thường. GSPO chỉ cần likelihood ở mức **sequence**, khoan dung hơn với các sai lệch số nhỏ cấp token, nên **có thể dùng trực tiếp** likelihood do engine infer trả về mà không cần tính lại bằng engine train, có lợi cho partial rollout, multi-turn RL, và kiến trúc train/infer tách rời (disaggregated).

6. Giới hạn

Tác giả nêu: không trình bày bảng số liệu cuối cùng cụ thể cho các benchmark (chỉ có đồ thị, xem mục 4); không định lượng mức cải thiện mang lại cho các mô hình Qwen3 thực tế đã phát hành (chỉ nói "góp phần vào cải thiện vượt trội"); GSPO-token được giới thiệu về mặt lý thuyết (mục 4.3) nhưng không có bảng thực nghiệm riêng so sánh GSPO-token với GSPO gốc trong bài.

Nhận xét:

- Đây là một bài rất ngắn (7 trang nội dung chính), mức độ minh bạch số liệu thấp hơn hẳn 41 (DAPO) và 42 (Dr. GRPO): chỉ có đúng một con số cứng trong toàn bài (tỉ lệ clip của GRPO, 0.0013); mọi khẳng định khác ("hai bậc độ lớn", "vượt trội", "cải thiện liên tục") đều ở dạng định tính hoặc chỉ qua đồ thị.
- Quan sát "~10% expert khác nhau sau mỗi update" chỉ là một đo đạc trên **một** mô hình cụ thể (Qwen3-30B-A3B-Base, 48 lớp), chưa rõ con số này thay đổi thế nào theo số lớp, số expert, hay kích thước mô hình.
- Bài 44 (ScaleRL), khi triển khai lại GSPO ở quy mô 8B dense của riêng họ, phát hiện **clip-range mặc định 10^{-4} của bài GSPO không hoạt động tốt**, phải dùng bậc $\sim$ 4-5 $\times$ 10^{-3} mới ổn định; họ cũng quan sát **GSPO phân kỳ giữa chừng quá trình train** ở nhiều lần thử, và với mô hình lớn hơn (Scout, MoE 17B×16) việc reset về checkpoint ổn định **không khắc phục được** sự bất ổn này, trái với ấn tượng "ổn định vượt trội" mà bài GSPO đưa ra. Đây là tín hiệu quan trọng: tính ổn định của GSPO có thể **phụ thuộc nhiều vào việc chọn đúng clip-range cho từng quy mô mô hình cụ thể**, không "cắm vào là chạy" như có thể hiểu nhầm từ bài gốc.

7. Liên hệ với các bài khác

- **Bài 00, 36:** nền tảng PPO, GRPO.
- **Bài 41 (DAPO):** cùng than phiền về entropy/model collapse khi huấn luyện long-CoT RLVR, nhưng DAPO giữ nguyên **đơn vị token** cho importance ratio, chỉ sửa clip-range (decoupled) và cách cộng dồn loss (token-level thay sample-level). GSPO đi xa hơn: cho rằng chính **đơn vị token** của ratio là vấn đề gốc, không chỉ là cách chọn clip-range hay cách cộng dồn.
- **Bài 42 (Dr. GRPO):** sửa bias trong cách **chuẩn hoá advantage** ($1/|o_i|$, std(R)), nhưng vẫn giữ token-level importance ratio. GSPO không đụng tới cách chuẩn hoá advantage (giữ nguyên kiểu GRPO, có `mean`/`std`), mà thay hẳn **loại ratio** dùng để clip và tính gradient.
- **Bài 44 (ScaleRL):** đánh giá định lượng GSPO cùng DAPO và CISPO (một loss khác, truncated importance sampling + REINFORCE thuần) trong cùng khung đo sigmoid compute-performance; thấy

GSPO và CISPO đều vượt DAPO về asymptote hiệu năng, nhưng GSPO kém robust hơn CISPO với lựa chọn clip-range và có vấn đề phân kỳ ở quy mô lớn (xem mục Giới hạn). ScaleRL cuối cùng chọn CISPO làm loss mặc định cho recipe của họ, không chọn GSPO.

- Liên hệ kỹ thuật MoE: bài 27 (Switch Transformer), 28 (Mixtral) về cơ chế routing chuyên gia, hữu ích để hiểu rõ hơn vì sao "expert-activation volatility" (biến động routing) là một vấn đề đặc thù của MoE mà GSPO giải quyết gián tiếp.

8. Ứng dụng thực tế (suy luận)

- Lập luận cốt lõi "token-level importance ratio chỉ dựa trên một mẫu, về lý thuyết không đúng vai trò hiệu chỉnh importance sampling, và nhiễu này tích luỹ theo độ dài sequence" là một cảnh báo chung hữu ích cho **bất kỳ** quy trình RLVR có sequence dài, không riêng MoE: nếu rollout trở nên dài, nên cân nhắc theo dõi tỉ lệ clip theo từng token so với cả sequence để phát hiện sớm nhiễu tích luỹ.
- Nếu có ý định thử sequence-level ratio (GSPO) thay cho token-level (GRPO/DAPO), cần nhớ **clip-range của GSPO cần tinh chỉnh lại hoàn toàn** (bậc 10^{-4}-10^{-3}), hoàn toàn khác bậc 0.2-0.3 quen thuộc của GRPO/DAPO; đây là một cái bẫy dễ gặp nếu copy nguyên giá trị mặc định từ bài GSPO.

9. Thí nghiệm gợi ý

1. Trên một tác vụ RLVR nhỏ, triển khai cả token-level ratio (GRPO) và sequence-level ratio (GSPO) cho cùng rollout, ghi lại phân phối của $w_{i,t}(\theta)$ (token) so với $s_i(\theta)$ (sequence) qua vài bước cập nhật, để thấy trực quan luận điểm "ratio token biến động mạnh hơn, kém ổn định hơn ratio sequence" của bài báo.
2. Sweep clip-range của GSPO trên thiết lập đang xét trong khoảng $\{10^{-4}, 10^{-3}, 4\times10^{-3}, 10^{-2}\}$ trước khi tin vào giá trị mặc định 3-4×10^{-4} của bài gốc, vì bài 44 (ScaleRL) đã cho thấy giá trị này không tổng quát hoá sang quy mô khác.
3. Nếu có huấn luyện bất kỳ mô hình MoE nào bằng RLVR, đo tỉ lệ % expert kích hoạt khác nhau giữa π_θ và $\pi_{\theta_{old}}$ sau mỗi lần cập nhật gradient (như paper làm với Qwen3-30B-A3B, ra ~10%), để quyết định có cần Routing Replay (nếu dùng GRPO) hay chuyển sang GSPO.
4. So sánh GSPO với GSPO-token trên một tác vụ có reward theo từng bước/lượt (ví dụ multi-turn tool-use), đo xem advantage theo-từng-token có mang lại cải thiện thực sự so với advantage chung cho cả response hay không (bài báo không tự làm thí nghiệm này).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Importance sampling (chuẩn)	$\mathbb{E}_{\pi_{tar}}[f] = \mathbb{E}_{\pi_{beh}}[\frac{\pi_{tar}}{\pi_{beh}} f]$	cần nhiều mẫu từ π_{beh} để hiệu chỉnh đúng
Token-level ratio (GRPO)	$w_{i,t}(\theta) = \pi_\theta(y_{i,t} \mid \cdot)/\pi_{\theta_{old}}(y_{i,t} \mid \cdot)$	1 mẫu/vị trí, "ill-posed" theo paper
Sequence-level ratio (GSPO)	$s_i(\theta) = (\pi_\theta(y_i \mid x)/\pi_{\theta_{old}}(y_i \mid x))^{1/\lvert y_i\rvert}$	length-normalized geometric mean
GSPO objective	$\frac{1}{G}\sum_i \min(s_i\hat{A}_i, \text{clip}(s_i, 1-\epsilon, 1+\epsilon)\hat{A}_i)$	clip/reward/optimize cùng mức sequence
GSPO-token	$s_{i,t}(\theta) = \text{sg}[s_i(\theta)] \cdot \pi_\theta(y_{i,t})/\text{sg}[\pi_\theta(y_{i,t})]$	bằng số với $s_i(\theta)$, gradient theo token
Clip-range	GSPO: $\epsilon_{low} = 3\times10^{-4}$, $\epsilon_{high} = 4\times10^{-4}$; GRPO: $0.2/0.27$	khác bậc độ lớn, không hoán đổi được
Routing Replay	cache + tái dùng expert routing cũ khi tính ratio	cần cho GRPO+MoE, GSPO không cần

Khái niệm	Công thức / ý	Ghi chú
Tỉ lệ clip (Fig.2)	GRPO ≈ 0.0013; GSPO cao hơn ~2 bậc độ lớn (paper không nêu số GSPO)	nhiều token bị clip hơn nhưng học hiệu quả hơn

11. Kết luận ngắn

GSPO lập luận rằng vấn đề bất ổn của GRPO khi scale RL (đặc biệt với MoE) không chỉ nằm ở việc chọn sai clip-range hay cách cộng dồn loss, mà ở chính việc áp dụng importance sampling **sai đơn vị**: reward cấp theo sequence nhưng ratio và clip lại tính theo token, một ratio dựa trên một mẫu duy nhất không thể làm đúng vai trò hiệu chỉnh phân phối. Chuyển hẳn sang ratio mức sequence (có chuẩn hoá độ dài) giúp mọi token trong một response chia sẻ cùng một trọng số ổn định, loại bỏ nguồn nhiễu tích luỹ, và đặc biệt hữu ích với MoE vì sequence-likelihood không nhạy với biến động routing cục bộ như token-likelihood. Dù ý tưởng đơn giản và thuyết phục, bài báo khá ngắn và ít số liệu định lượng cụ thể; các đánh giá độc lập sau đó (bài 44) cho thấy lợi ích ổn định của GSPO phụ thuộc nhiều vào việc tinh chỉnh đúng clip-range cho từng quy mô, và ở một số quy mô lớn GSPO vẫn gặp phân kỳ giữa huấn luyện.

Link gốc: https://arxiv.org/abs/2507.18071

Bài 44

The Art of Scaling Reinforcement Learning Compute for LLMs (2025)

Tác giả: Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, Rishabh Agarwal (Meta; UT Austin; UCL; UC Berkeley; Harvard University; Periodic Labs). **Venue / arXiv:** arXiv:2510.13786 (v1, 15/10/2025); bài báo không nêu venue. **Nguồn:** https://arxiv.org/abs/2510.13786 **Chủ đề:** Xây khung dự đoán quy mô (scaling framework) cho RL compute của LLM bằng đường cong sigmoid giữa compute và pass rate; khảo sát hơn 400.000 GPU-giờ các lựa chọn thiết kế RL để tách bạch "cái gì nâng trần hiệu năng (asymptote A)" khỏi "cái gì chỉ tăng tốc hội tụ (B, C_{mid})"; gộp các lựa chọn tốt nhất thành recipe **ScaleRL** và kiểm định bằng một lần chạy 100.000 GPU-giờ. **Vị trí trong lộ trình:** đọc sau bài 00, 36 (GRPO), và đặc biệt sau 41 (DAPO), 42 (Dr. GRPO), 43 (GSPO), vì ScaleRL dùng cả ba làm "nguyên liệu" được đánh giá định lượng trong một khung đo chung. Đây là bài tổng kết của giai đoạn 9 (RLVR cho LLM reasoning) trước khi sang giai đoạn 10 (điều gì thực sự thay đổi khi scale RL: entropy mechanism, high-entropy minority tokens, RL có thực sự "dạy reasoning mới" hay không, v.v., bài 45 trở đi).
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RL compute cho LLM đang tăng rất nhanh: DeepSeek-R1-Zero dùng 100.000 giờ GPU H800 cho RL, tương đương **3.75%** compute pretraining của nó; bước nhảy từ o1 lên o3 tăng hơn **10 lần**; Grok-3 lên Grok-4 cũng tương tự. Nhưng khác với pretraining (đã có Chinchilla/Kaplan scaling laws để dự đoán trước khi chi tiền chạy full-scale), lĩnh vực RL **chưa có một "khoa học" tương tự**: các recipe hiện có (DAPO, Magistral, MiniMax-M1...) là giải pháp "may đo" cho từng bối cảnh cụ thể, không cho biết cách suy ra trước liệu một lựa chọn thiết kế có "scale" tốt hay không trước khi đổ compute lớn vào.

Khái niệm cần nắm trước:

- **Generator-trainer split (async RL):** tách GPU thành nhóm *generator* (chạy kernel inference tối ưu để rollout) và nhóm *trainer* (chạy backend FSDP để cập nhật tham số); π_θ^{gen} và π_θ^{train} là cùng một mô hình nhưng chạy trên hai backend khác nhau.
- **Off-policy thiết lập:** $\pi_{\theta_{old}}^{gen}$ sinh rollout, π_θ^{train} được cập nhật dựa trên các rollout đó (đã "cũ" một phần so với θ hiện tại).
- **CISPO** (truncated importance sampling + REINFORCE thuần, không có cơ chế `min`/clip hai phía như PPO): xem công thức ở §3.4.
- **Zero-variance filtering:** loại khỏi batch các prompt mà mọi rollout đều cùng đúng hoặc cùng sai (advantage = 0, gradient = 0), **khác** với Dynamic Sampling của DAPO (bài 41): DAPO sample bù thêm để lấp đầy batch, còn ở đây chỉ loại bỏ, không bù.
- **No-Positive-Resampling:** loại vĩnh viễn khỏi các epoch sau những prompt đã có tỉ lệ đúng lịch sử (pass rate) ≥ 0.9.
- **Sigmoid fit** vs **power-law fit:** xem công thức ở §3.1.

2. Ý tưởng chính

Định nghĩa một hàm fit giữa compute C (GPU-giờ) và phần thưởng kỳ vọng R_C trên một **tập validation trong-phân-phối** (iid với dữ liệu train), dạng sigmoid:

$$R_C - R_0 = \underbrace{(A - R_0)}_{\text{Asymptotic Reward Gain}} \times \underbrace{\frac{1}{1 + (C_{mid}/C)^B}}_{\text{Compute Efficiency}} \tag{1}$$

với $0 \le A \le 1$ là **pass rate tiệm cận** (trần hiệu năng ở compute vô hạn), $B > 0$ là **độ dốc/hiệu suất scale**, C_{mid} là điểm compute đạt được một nửa mức tăng tổng ($R_{C_{mid}} - R_0 = (A - R_0)/2$).

Dùng khung này để tách bạch hai câu hỏi cho mỗi lựa chọn thiết kế RL: (a) nó **nâng trần** A hay chỉ (b) nó **tăng tốc** đạt tới trần đã có (B, C_{mid})? Ba nguyên tắc rút ra sau khảo sát hơn 400.000 GPU-giờ:

1. **Trần hiệu năng không phổ quát:** các phương pháp khác nhau gặp trần A khác nhau khi scale compute; trần này bị dịch chuyển bởi những lựa chọn như loss type, batch size.
2. **"Bitter lesson":** phương pháp có vẻ tốt ở compute nhỏ có thể tệ hơn khi ngoại suy lên compute lớn; có thể nhận diện phương pháp "scale tốt" sớm bằng cách fit (A, B) từ giai đoạn đầu huấn luyện.
3. **Nhiều "common wisdom" chỉ ảnh hưởng hiệu suất (B), không ảnh hưởng trần (A)** (ví dụ loss aggregation, data curriculum, length penalty, advantage normalization).

Gộp các lựa chọn tốt nhất (không phát minh kỹ thuật mới) thành **ScaleRL**: async PipelineRL, forced length interruption, CISPO loss, prompt-level loss averaging, batch-level advantage normalization, FP32 precision ở logits, zero-variance filtering, No-Positive-Resampling. Mỗi thành phần được kiểm định bằng leave-one-out ablation, 16.000 GPU-giờ mỗi lần chạy.

3. Phương pháp chi tiết

3.1 Vì sao dùng sigmoid, không dùng power-law như pretraining

Pretraining thường fit dạng power-law $R_C = A - D/C^B$ ($C \ge C_0$). Paper chỉ ra 3 lý do chọn sigmoid cho RL: (a) với các chỉ số bị chặn như pass rate/accuracy, sigmoid cho fit dự đoán tốt hơn; (b) power-law không bị chặn ở compute thấp, cần bỏ qua một đoạn đầu C_0, mà RL vốn có ít điểm đánh giá hơn pretraining (ví dụ chỉ ~75 điểm đánh giá trong lần chạy 100k GPU-giờ của chính paper), bỏ thêm dữ liệu sớm lại càng làm fit kém tin cậy; (c) thực nghiệm trực tiếp cho thấy power-law **nhạy cảm với vùng fit** còn sigmoid thì không.

Ví dụ cụ thể paper đưa ra (lần chạy 100k GPU-giờ, mô hình 8B dense, Hình 1): fit power-law trên khoảng (1.5k, 50k) GPU-giờ dự đoán $A = 1.0$ (rõ ràng sai, vì đường cong thực tế bão hoà gần 0.65); fit power-law trên (5k, 50k) lại cho $A = 0.74$ (một kết quả khác hẳn chỉ vì đổi vùng fit); trong khi fit sigmoid cho $A = 0.645$ **ổn định** dù đổi vùng fit.

Vì sao sigmoid vẫn "chứa" power-law ở vùng compute cao (tự suy lại theo khai triển của bài báo): viết $R_C = R_0 + \frac{A - R_0}{1 + (C_{mid}/C)^B}$. Khi $C \gg C_{mid}$, $(C_{mid}/C)^B \to 0$, nên $\frac{1}{1+(C_{mid}/C)^B} \approx 1 - (C_{mid}/C)^B$ (xấp xỉ bậc 1), suy ra

$$R_C \approx R_0 + (A - R_0)\left(1 - \frac{C_{mid}^B}{C^B}\right) = A - \frac{(A - R_0)C_{mid}^B}{C^B} = A - \frac{D}{C^B}, \quad D := (A - R_0)C_{mid}^B$$

đúng dạng power-law. Vậy ở vùng compute đủ lớn, hai dạng fit trùng nhau; sự khác biệt chỉ nằm ở vùng compute thấp/trung bình, đúng nơi mà các quyết định "có nên scale tiếp phương pháp này không" cần được đưa ra sớm.

Cách fit (Phụ lục A.5): grid search $A \in \{0.450, 0.455, \ldots, 0.800\}$ và $C_{mid} \in [100, 40000]$ (100 giá trị chia đều), với mỗi cặp (A, C_{mid}) fit B bằng SciPy `curve_fit`; chọn bộ tốt nhất theo tổng bình phương sai số dư

(SSR). Bỏ qua ~1.5k GPU-giờ đầu (quan sát hành vi hai giai đoạn: tăng gần tuyến tính trong ~1 epoch đầu, sau đó mới vào vùng sigmoid). **Sai số ước lượng:** 3 lần chạy ScaleRL độc lập cho A dao động tối đa ± 0.015; bài báo dùng ± 0.02 làm biên sai số thống kê khi so sánh các phương pháp.

3.2 Thuật toán RL "nền" (baseline xuất phát)

$$\hat{A}_i = r_i - \text{mean}(\{r_j\}_{j=1}^G), \qquad \hat{A}_i^G = \hat{A}_i / (\text{std}(\{r_j\}_{j=1}^G) + \epsilon)$$

$$\rho_{i,t}(\theta) = \frac{\pi_\theta^{train}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{old}}^{gen}(y_{i,t} \mid x, y_{i,<t})}, \qquad \text{clip}_{asym}(\rho, \epsilon^-, \epsilon^+) = \text{clip}(\rho, 1-\epsilon^-, 1+\epsilon^+)$$

$$J(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\left(\rho_{i,t}(\theta)\hat{A}_i^G,\ \text{clip}_{asym}(\rho_{i,t}(\theta), \epsilon^-, \epsilon^+)\hat{A}_i^G\right)\right]$$

tức GRPO không KL + clip bất đối xứng kiểu DAPO (bài 41), **sample-level loss aggregation** (chia $1/|y_i|$ rồi $1/G$, giống GRPO gốc), kiểm soát độ dài sinh bằng **forced interruption** (chèn cụm từ kết thúc suy luận, ví dụ `</think>`, khi sinh quá dài) thay cho length-penalty.

3.3 Lựa chọn hạ tầng async: PPO-off-policy-k vs PipelineRL-k (mục 3.1 paper)

- **PPO-off-policy-k:** $\pi_{\theta_{old}}^{gen}$ sinh một batch B prompt; mỗi lượt cập nhật gradient xử lý mini-batch $\hat{B}$ prompt, cho $k = B/\hat{B}$ lượt cập nhật/batch. Thực nghiệm cố định $\hat{B} = 48$ prompt (16 generation/prompt), thay đổi $k \in \{1, 8\}$.
- **PipelineRL-k** (Piche et al., dùng trong Magistral): generator sinh liên tục kiểu streaming; mỗi khi trainer xong một lượt cập nhật, trọng số mới được đẩy ngay cho generator, generator **tiếp tục sinh phần còn lại** của các trace đang sinh với trọng số mới (nhưng KV-cache "cũ"). k ở đây là **độ off-policy tối đa** cho phép (trainer chờ nếu vượt k bước so với generator).

Kết quả (Hình 4a-4b): PipelineRL và PPO-off-policy đạt **asymptote** A **tương đương**, nhưng PipelineRL cải thiện **hiệu suất** B rõ rệt (đạt trần nhanh hơn), nhờ giảm thời gian "chờ" (idle time) trong vòng lặp train/generate. $k = 8$ tối ưu cho PipelineRL (so với $k = 1, 4$). Paper nhấn mạnh: đây là **một trong rất ít** trục thiết kế ảnh hưởng tới cả asymptote (không chỉ hiệu suất), khiến lựa chọn thuật toán off-policy trở thành một trong những quyết định **quan trọng nhất**. ScaleRL chọn PipelineRL-8 làm baseline cập nhật cho các khảo sát tiếp theo.

3.4 Sáu trục thiết kế thuật toán (mục 3.2 paper)

(a) Loss type: so sánh DAPO (asymmetric clip, §3.2 trên) với GSPO (bài 43, đổi ratio token-level $\rho_{i,t}$ thành ratio sequence-level $\rho_i(\theta) = \pi^{train}(y_i|x,\theta)/\pi^{gen}(y_i|x,\theta_{old})$) và **CISPO** (kết hợp truncated importance sampling với policy gradient thuần, sg là stop-gradient):

$$J^{CISPO}(\theta) = \mathbb{E}\left[\frac{1}{T}\sum_{i=1}^{G}\sum_{t=1}^{|y_i|}\text{sg}(\min(\rho_{i,t}, \epsilon_{max}))\,\hat{A}_i\,\log\pi^{train}(y_{i,t} \mid x, y_{i,<t}, \theta)\right]$$

(không có cơ chế `min` hai phía như PPO, chỉ **chặn trên** $\rho_{i,t}$ rồi dừng gradient lan qua trọng số đó, phần gradient chỉ còn chảy qua $\log\pi^{train}$, giống REINFORCE có trọng số importance bị cắt ngọn). Kết quả: GSPO và CISPO **vượt rõ** DAPO về asymptote A; CISPO có xu hướng tăng gần tuyến tính kéo dài và nhích hơn GSPO về cuối, **ít nhạy với** ϵ_{max} **hơn** $\Rightarrow$ chọn CISPO.

(b) FP32 precision ở LM head: engine train và engine infer dùng kernel khác nhau, gây lệch số nhỏ trong xác suất token, ảnh hưởng trực tiếp tới IS ratio; lệch này tập trung nhiều nhất ở lớp cuối (LM head). Fix: tính FP32 ở đây cho cả generator và trainer. **Số liệu chính xác (Hình 5b): nâng asymptote** A **từ 0.52 lên 0.61.** $\Rightarrow$ đưa vào ScaleRL.

(c) Loss aggregation: sample-average (GRPO, mỗi rollout trọng số bằng nhau) vs prompt-average (DAPO, mỗi prompt trọng số bằng nhau, bất kể số token) vs token-average (mọi token trong batch trọng số bằng nhau, không phân nhóm). Prompt-average cho asymptote cao nhất ⇒ chọn.

(d) Advantage normalization: prompt-level std (GRPO, chuẩn hoá theo std của riêng từng prompt) vs batch-level std (chuẩn hoá theo std trên **toàn batch**) vs không chuẩn hoá std (centered reward thuần, như Dr. GRPO, bài 42). Ba cách cho hiệu năng **tương tự nhau trong sai số**; chọn batch-level vì "có cơ sở lý thuyết chặt hơn và nhích hơn một chút". (Đây là một điểm đáng đối chiếu với bài 42: ở quy mô 16k GPU-giờ của ScaleRL, khác biệt giữa "chuẩn hoá std" và "không chuẩn hoá std" **không lớn** như claim gốc của Dr. GRPO, có thể vì ScaleRL đã dùng nhiều kỹ thuật ổn định khác cộng dồn.)

(e) Zero-variance filtering: loại khỏi loss các prompt có mọi rollout cùng đúng/cùng sai (advantage = 0). **Khác Dynamic Sampling của DAPO** (bài 41): DAPO sample bù thêm để lấp đầy batch tới số prompt mong muốn; ở đây chỉ loại bỏ, không bù lại. Cải thiện asymptote rõ (Hình 6a) ⇒ chọn.

(f) No-Positive-Resampling (adaptive prompt filtering): theo dõi lịch sử pass rate của từng prompt; loại **vĩnh viễn** khỏi các epoch sau những prompt có pass rate ≥ 0.9 (ý tưởng: prompt đã "quá dễ" với policy hiện tại tốn compute nhưng không còn gradient hữu ích). Cải thiện cả khả năng scale và asymptote (Hình 6b) ⇒ chọn.

3.5 Công thức ScaleRL đầy đủ

$$J^{ScaleRL}(\theta) = \mathbb{E}\left[\frac{1}{\sum_{g=1}^{G}|y_g|}\sum_{i=1}^{G}\sum_{t=1}^{|y_i|}\text{sg}(\min(\rho_{i,t},\epsilon))\,\hat{A}_i^{norm}\log\pi^{train}(y_{i,t})\right]$$

$$\rho_{i,t} = \frac{\pi_\theta^{train}(y_{i,t})}{\pi_{\theta_{old}}^{gen}(y_{i,t})}, \qquad \hat{A}_i^{norm} = \hat{A}_i/\hat{A}_{std} \text{ với } 0 < \text{mean}(\{r_j\}_{j=1}^{G}) < 1 \text{ và pass_rate}(x) < 0.9$$

($\hat{A}_{std}$: độ lệch chuẩn của mọi advantage $\hat{A}_i$ **trong cả batch**, tức batch-level normalization; hai điều kiện sau dấu phẩy chính là zero-variance filter và No-Positive-Resampling.) Forced interruption dùng câu: *"Okay, time is up. Let me stop thinking and formulate a final answer now. `</think>`"*.

Ví dụ minh hoạ $\hat{A}_i^{norm}$ (tự tính): batch có 2 prompt, mỗi prompt 4 rollout. Prompt 1: reward $\{1,1,0,0\}$ (pass rate 0.5, qua được filter vì $0 < 0.5 < 1$). Prompt 2: reward $\{1,1,1,1\}$ (pass rate 1.0, **bị loại** bởi zero-variance filter vì không thoả mean < 1). Chỉ còn 4 rollout của Prompt 1 tham gia loss: $\hat{A}_i = r_i - 0.5 \Rightarrow \{0.5, 0.5, -0.5, -0.5\}$; $\hat{A}_{std} = \text{std}(\{0.5, 0.5, -0.5, -0.5\}) = 0.5$; $\hat{A}_i^{norm} = \{1, 1, -1, -1\}$.

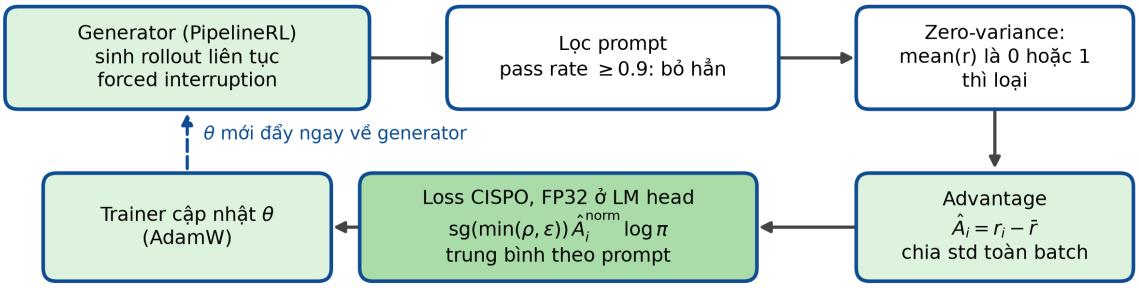
3.6 Pseudo-code tổng hợp (diễn giải của phần phân tích, dựa trên các công thức ở §3.3-3.5)

```
Input: policy pi_theta (chia pi_gen / pi_train qua generator-trainer split)
       du lieu D (prompt), nguong eps (CISPO), nguong pass_rate 0.9

history_pass_rate = {}                       # cho No-Positive-Resampling
loop (PipelineRL, k=8 buoc off-policy toi da):
    generator (dang pi_gen_theta_old) sinh lien tuc rollout cho prompt x ~ D
        neu x co history_pass_rate[x] >= 0.9: bo qua x (No-Positive-Resampling)
        neu do dai vuot nguong: chen cau "Okay, time is up..." (forced interruption)
    khi co 1 batch day: gui cho trainer

trainer, moi khi nhan 1 batch:
    cho tung prompt x trong batch, G rollout {y_i}, reward {r_i}:
        cap nhat history_pass_rate[x]
        neu mean(r_i) == 0 hoac == 1: loai ca nhom nay khoi loss   # zero-variance
        ↪  filtering
```

```
    nguoc lai: A_i = r_i - mean(r)                              # advantage, chua
    ↪ chuan hoa
A_std = std(TOAN BO A_i con lai trong batch)                    # batch-level
↪ normalization
A_norm_i = A_i / A_std
rho_i,t = pi_train_theta(y_i,t) / pi_gen_theta_old(y_i,t)
loss = -(1/sum_i|y_i|) * sum_i sum_t stopgrad(min(rho_i,t, eps)) * A_norm_i * log
↪ pi_train_theta(y_i,t)
cap nhat theta bang AdamW tren loss; day theta moi ve generator ngay (PipelineRL)
```



Hình 79: Một bước huấn luyện của ScaleRL: generator, hai bộ lọc, advantage chuẩn hoá theo batch, loss CISPO và cập nhật (minh hoạ, tự tính)

Cách đọc hình: theo mũi tên từ generator qua hai bộ lọc prompt, xuống bước tính advantage rồi loss CISPO và cập nhật bởi trainer. Nét đứt xanh là trọng số mới được đẩy ngay về generator (PipelineRL). Dòng chữ đỏ phía dưới nhắc cách đo: tách trần A khỏi hiệu suất B, C_{mid}.

4. Thí nghiệm và kết quả chính

Thiết lập chính: mô hình 8B dense; Polaris-53K (dữ liệu toán) cho hầu hết thực nghiệm, Deepcoder cho thực nghiệm math+code; batch 768 (48 prompt × 16 generation); sequence length 16.384 token (12.288 "thinking" + 2.048 solution + 2.048 prompt). SFT trước RL: batch 2M token, max length 12.288, lr 3×10^{-5}, AdamW, 32 node H100, ~4 epoch, 32B token. RL: lr hằng số 5×10^{-7}, AdamW $\epsilon = 10^{-15}$ (tránh gradient clip do underflow), weight decay 0.01, warmup 100 bước, reward ± 1. Hạ tầng: 80 GPU Nvidia GB200/lần chạy (64 generator/16 trainer); budget 3.5-4k GPU-giờ cho ablation nhỏ (§3), 16k cho leave-one-out (§4), 30k-100k cho các lần chạy lớn (§5).

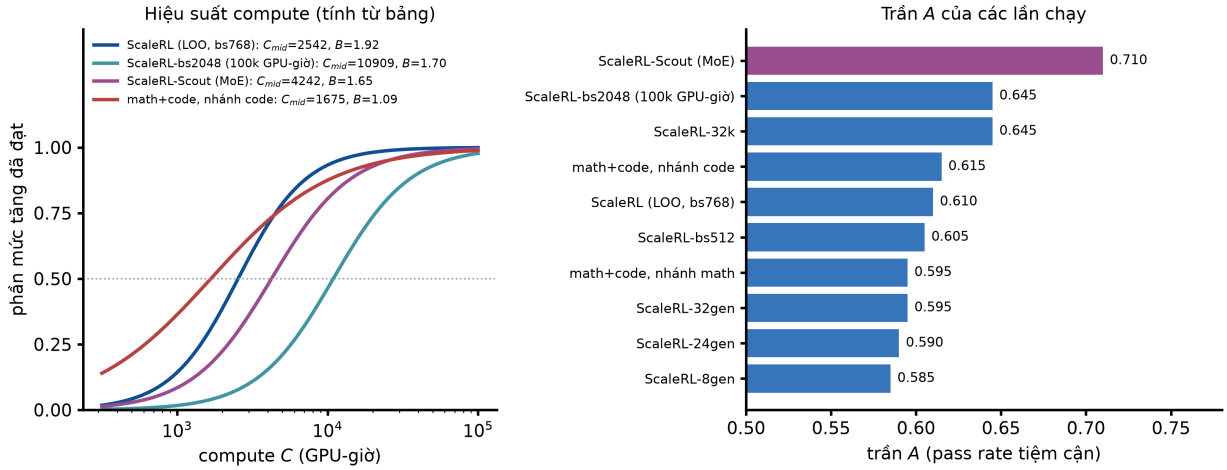
So sánh với các recipe phổ biến (Hình 2, cùng quy mô fit): DeepSeek (GRPO, $\epsilon_{min} = \epsilon_{max} = 0.2$, sample-average, PPO-off-policy-8) **mất ổn định sau ~6k GPU-giờ** do tỉ lệ truncation tăng cao. Qwen2.5 (DAPO) và MiniMax (CISPO + FP32 + PPO-off-policy, cả hai được cho batch lớn hơn 1280 để bù việc drop 0-variance prompt, "có lợi cho DAPO/MiniMax" theo bài báo tự nhận). Magistral giống DAPO nhưng dùng PipelineRL. **ScaleRL đạt asymptote cao nhất,** $A = 0.61$, vượt tất cả.

Leave-one-out (Hình 7, 16k GPU-giờ/lần): mọi biến thể LOO đạt A **tương tự ScaleRL trong sai số** ± 0.02; khác biệt chủ yếu ở hiệu suất B. Ví dụ cụ thể bài báo nêu: loo-dapo (quay lại loss DAPO) có $B = 1.77$ so với $B = 2.01$ của ScaleRL (CISPO); dù asymptote gần như ngang nhau trong cấu hình kết hợp, CISPO vẫn được giữ vì **bền với siêu tham số hơn** DAPO khi tách riêng (xem §5).

Lần chạy lớn 100k GPU-giờ (Hình 1): 8B dense, batch 2048, 7.400 bước (gấp **3.5 lần** ProRL theo bài báo); fit trên (1.5k, 50k) GPU-giờ, ngoại suy khớp sát với điểm thực tế tại 100k. Scout MoE (Llama-4 17B×16), 50k GPU-giờ, 7.100 bước, cũng khớp dự đoán; Scout đạt asymptote cao hơn 8B dense **chỉ với 1/6 compute RL** của 8B. Downstream AIME-24 (Hình 1b): xu hướng scale nhất quán, mô hình lớn hơn nâng cả downstream và asymptote.

Bảng các giá trị (C_{mid}, B, A) cho các lần chạy lớn ở §5 (Bảng 1 phụ lục, số liệu đúng theo bài báo):

Thực nghiệm	C_{mid}	B	A
ScaleRL (quy mô LOO, bs768)	2542	1.92	0.610
ScaleRL-32k (context dài)	11272	1.89	0.645
ScaleRL-8gen	2542	2.44	0.585
ScaleRL-24gen	3054	2.22	0.590
ScaleRL-32gen	2936	2.07	0.595
ScaleRL-Scout (MoE 17B×16)	4242	1.65	0.710
ScaleRL-bs512	2818	1.77	0.605
ScaleRL-bs2048 (lần chạy 100k GPU-giờ)	10909	1.70	0.645
ScaleRL-math+code, nhánh math	2896	2.05	0.595
ScaleRL-math+code, nhánh code	1675	1.09	0.615



Hình 80: Hiệu suất compute tính từ (C_{mid}, B) và trần A của các lần chạy ở bảng (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: bên trái vẽ phần $1/(1+(C_{mid}/C)^B)$ cho bốn lần chạy, tính từ C_{mid} và B trong bảng; đường dịch phải nghĩa là cần nhiều compute hơn để đạt cùng một phần mức tăng. Bên phải là trần A của cả mười dòng, xếp tăng dần: Scout (MoE) cao nhất (0.710), hai lần chạy context dài và bs2048 cùng 0.645.

5. Phân tích / ablation

Truncation và bất ổn (Phụ lục A.15): ở batch 768, tỉ lệ truncation 10-15% thường **gây mất ổn định không hồi phục** (minh chứng: lần chạy GRPO mở rộng ở Hình 2). ScaleRL giữ truncation **dưới 5% trong hơn 90%** thời gian train ở 8B/bs768; ở bs2048 truncation hơi cao hơn (đôi khi ~7%) nhưng vẫn ổn vì effective batch sau lọc vẫn lớn; với context dài hơn (34k, bs768), truncation chỉ bật lên ~4% trong giây lát rồi tụt xuống dưới 2%. Mô hình lớn hơn (Scout) càng ổn định: truncation dưới 2% hầu hết thời gian, dưới 1% ở >90% số bước.

Entropy không dự đoán hiệu năng (Phụ lục A.12): so sánh bs768 và bs2048, quỹ đạo entropy theo **bước** huấn luyện gần như giống nhau, nhưng bs2048 có hiệu năng downstream tốt hơn **rõ rệt** ở mọi giai đoạn. Kết luận của bài báo: entropy cao không đồng nghĩa khái quát hoá tốt hơn; không nên dùng riêng entropy làm chỉ báo hiệu năng.

Độ nhạy siêu tham số clip (Phụ lục A.17): GRPO/DAPO rất nhạy ϵ_{max}: tăng ϵ_{max} làm A tăng tới một điểm tối ưu rồi **giảm trở lại**, nghĩa là ϵ_{max} ảnh hưởng tới **chính asymptote**, không chỉ tốc độ hội tụ (một hiệu ứng "lạ" so với đa số siêu tham số khác). CISPO: thử $\epsilon_{max} \in \{4, 5, 8\}$ khác biệt **không đáng kể**. GSPO: giá trị mặc định 10^{-4} từ bài gốc (bài 43) **không hoạt động tốt** ở quy mô 8B của ScaleRL; phải dùng bậc $4\text{-}5 \times 10^{-3}$ mới ổn, và một khi đúng bậc thì cũng khá bền với các giá trị lân cận; tuy vậy GSPO **phân kỳ giữa chừng training** nhiều lần, với 8B có thể hồi phục bằng reset checkpoint, nhưng với Scout (MoE lớn hơn) **reset không khắc phục được**. $\Rightarrow$ CISPO được chọn vì cân bằng tốt nhất giữa ổn định và ít nhạy siêu tham số.

Các trục scale khác (mục 5 chính văn):

- **Context length (14k → 32k token thinking):** chậm hơn ban đầu (B thấp hơn, C_{mid} cao hơn, xem dòng ScaleRL-32k trong bảng) nhưng **asymptote cao hơn** (0.645 so với 0.610), xác nhận long-context RL là công cụ "nâng trần", không chỉ đánh đổi tốc độ.
- **Batch size (prompt):** batch nhỏ có vẻ tốt sớm trên validation trong-phân-phối nhưng "chững" ở downstream; batch lớn tránh được hiện tượng này và đạt asymptote cao hơn (bs2048: $A = 0.645$ so với bs512: $A = 0.605$).
- **Generations/prompt (8, 16, 24, 32, giữ tổng batch cố định):** fit gần như không đổi ở quy mô vừa (so hàng ScaleRL-8gen/24gen/32gen trong bảng, A dao động 0.585-0.595, trong biên sai số); có thể khác biệt rõ hơn ở batch rất lớn (2k+), paper để ngỏ cho tương lai.
- **Multi-task (math + code):** hai domain có đường sigmoid **song song riêng biệt** (math $A = 0.595$, code $A = 0.615$), không giao thoa tiêu cực, cho thấy khung đo tổng quát hoá được sang multi-task.

6. Giới hạn

Tác giả nêu: framework chỉ fit trên validation **trong-phân-phối** (cùng phân phối với dữ liệu train, Polaris-53k); mối liên hệ với khả năng khái quát hoá ra ngoài phân phối được quan sát có tương quan nhưng chưa được đặc tả đầy đủ; khác biệt giữa các mức generations/prompt chưa rõ ở quy mô vừa, có thể cần batch rất lớn (>2k) mới thấy; chưa có "luật scale" chính thức nối compute pretraining, kích thước mô hình, và compute RL; phạm vi domain chủ yếu toán (và một phần code), chưa thử agentic/multi-turn dài.

Nhận xét:

- Sai số thống kê ± 0.02 cho A chỉ dựa trên 3 lần lặp độc lập; với số lần lặp nhỏ như vậy, nên đọc các so sánh gần biên (ví dụ CISPO vs GSPO) với một mức độ dè dặt.
- Hình 2 (DeepSeek/DAPO/Magistral/MiniMax-M1) so sánh dựa trên **triển khai lại** các recipe trong codebase riêng của nhóm, không chạy trực tiếp code gốc; tác giả tự nhận đã "cho DAPO một lợi thế" (tăng batch để bù hiệu ứng drop prompt). Kết quả so sánh phản ánh cách ScaleRL hiểu và cài lại các recipe, không hẳn đúng 100% hiệu năng mà bài báo gốc (bài 41, 43) công bố.
- Kết luận "GSPO kém ổn định ở quy mô lớn" dựa trên triển khai lại với siêu tham số khác bản gốc; có thể một phần do chưa tìm đúng clip cho quy mô cụ thể (8B/Scout) hơn là hạn chế bản chất của thuật toán, nên nên đọc như một điểm dữ liệu độc lập cần cân nhắc, không phải bằng chứng dứt điểm.
- Chưa có ablation tách bạch forced-interruption so với length-penalty ở quy mô 100k GPU-giờ (chỉ so ở 16k, Phụ lục A.10), nên chưa rõ lựa chọn này còn quan trọng ở compute rất lớn hay không.

7. Liên hệ với các bài khác

- **Bài 00, 36:** nền tảng PPO, GAE, GRPO.

- **Bài 41 (DAPO):** nguồn gốc của asymmetric clip dùng trong baseline §3.2, và của Dynamic Sampling (so sánh/khác biệt với zero-variance filtering ở §3.4e); DAPO loss là 1 trong 3 loss type so sánh ở §3.4a, thua GSPO/CISPO về asymptote nhưng được dùng làm điểm xuất phát.
- **Bài 42 (Dr. GRPO):** "không chuẩn hoá std" là 1 trong 3 lựa chọn advantage normalization ở §3.4d; ScaleRL thấy khác biệt nhỏ hơn claim gốc khi đặt trong một recipe đầy đủ, và ở quy mô batch-level thay vì prompt-level.
- **Bài 43 (GSPO):** là 1 trong 3 loss type so sánh; ScaleRL tái khẳng định GSPO vượt DAPO về asymptote (đúng như claim gốc) nhưng phát hiện thêm vấn đề về độ bền siêu tham số và ổn định ở quy mô lớn mà bài gốc không thấy rõ (xem mục Giới hạn).
- Liên hệ phương pháp luận (không phải RL): tinh thần "đo lường có kiểm soát ở quy mô nhỏ trước khi scale, dùng fit để ngoại suy" tương tự với các nghiên cứu compute-optimal scaling trong pretraining (Chinchilla) mà paper trích dẫn trực tiếp làm nguồn cảm hứng.
- Các bài tiếp theo trong roadmap (rl 45+: liệu RL có thực sự "dạy reasoning mới" ngoài mô hình nền, entropy mechanism, high-entropy minority token...) tiếp tục đặt câu hỏi "điều gì thực sự thay đổi khi scale RL", một câu hỏi mà ScaleRL trả lời ở góc độ "hiệu năng đo được", còn các bài 45+ trả lời ở góc độ "cơ chế bên trong".

8. Ứng dụng thực tế (suy luận)

- Trước khi đổ compute lớn vào một recipe RLVR mới, nên chạy vài cấu hình ở quy mô nhỏ hơn nhiều, fit đường sigmoid, và **nhìn A (trần) trước khi tối ưu B/C_{mid} (tốc độ)**: một phương pháp "tăng nhanh ban đầu" không đảm bảo thắng ở compute lớn.
- Theo dõi **tỉ lệ truncation** như cảnh báo sớm bất ổn: nếu vượt ~10% và có xu hướng tăng, recipe nhiều khả năng sẽ sụp đổ trước khi đạt compute mục tiêu, nên dừng sớm để điều chỉnh (tăng max length, thêm forced interruption, hoặc giảm batch).
- Fix **FP32 ở lớp cuối (LM head)** là thay đổi rẻ, nên làm đầu tiên nếu quy trình có train engine và infer engine khác nhau (Megatron + vLLM/SGLang), vì ảnh hưởng trực tiếp độ chính xác IS ratio.
- Đừng early-stop hoặc đánh giá recipe chỉ dựa trên vài nghìn bước đầu ở downstream: batch nhỏ có thể "trông ổn" ban đầu rồi chững, batch lớn khởi đầu chậm hơn nhưng vượt lên sau.

9. Thí nghiệm gợi ý

1. Ở quy mô nhỏ (ví dụ vài trăm GPU-giờ hoặc tương đương trên 1-2 GPU trong vài ngày), chạy một baseline GRPO và 1-2 biến thể (ví dụ +FP32 fix ở lớp cuối, +CISPO) trên một tác vụ RLVR tự chọn, fit sigmoid ở nửa đầu budget và kiểm tra xem dự đoán có khớp với nửa sau không, để tự kiểm chứng tính dự đoán được của framework ở quy mô nhỏ hơn 44 rất nhiều.
2. Đo tỉ lệ truncation theo bước huấn luyện trên tác vụ RLVR đang xét, xem có tương quan với các dấu hiệu mất ổn định (reward dao động mạnh, entropy nhảy đột ngột) như paper mô tả không.
3. So sánh trực tiếp prompt-average và sample-average loss aggregation trên cùng một baseline nhỏ, cố gắng ước lượng A bằng fit sớm (không chỉ nhìn điểm cuối cùng) để xem khác biệt nằm ở trần hay ở tốc độ.
4. Thử batch-level và prompt-level chuẩn hoá std trên cùng một baseline nhỏ, so hiệu năng cuối, tự kiểm chứng claim "khác biệt nằm trong sai số" của ScaleRL (đối chiếu với claim ngược lại của Dr. GRPO ở bài 42).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Sigmoid scaling curve	$R_C - R_0 = (A - R_0)/(1 + (C_{mid}/C)^B)$	A = trần, B, C_{mid} = hiệu suất

Khái niệm	Công thức / ý	Ghi chú
Xấp xỉ power-law ở C lớn	$R_C \approx A - D/C^B$, $D = (A - R_0)C_{mid}^B$	tự suy từ khai triển sigmoid
CISPO loss	$\text{sg}(\min(\rho, \epsilon_{max}))\hat{A} \log \pi_{train}$	truncated IS + REINFORCE, không `min` 2 phía
ScaleRL objective	xem công thức đầy đủ ở §3.5	prompt-avg + batch-std-norm + CISPO + 2 filter
Zero-variance filtering	loại prompt có mean$(r) \in \{0, 1\}$	khác Dynamic Sampling (không bù lại)
No-Positive-Resampling	loại vĩnh viễn prompt có pass_rate ≥ 0.9	theo dõi lịch sử qua các epoch
FP32 ở LM head	$A : 0.52 \rightarrow 0.61$	sửa lệch số train/infer engine
Async setup tốt nhất	PipelineRL-8	nâng B, A không đổi nhiều so với PPO-off-policy
Kết quả lớn nhất	100k GPU-giờ, 8B, bs2048: $A = 0.645$; Scout MoE 50k GPU-giờ: $A = 0.710$	khớp dự đoán ngoại suy từ sớm

11. Kết luận ngắn

ScaleRL mang lại cho RL-cho-LLM một thứ mà pretraining đã có từ lâu: một cách **đo lường có kiểm soát ở quy mô nhỏ để dự đoán quy mô lớn**, thông qua fit đường sigmoid tách bạch "trần hiệu năng" (A) khỏi "tốc độ đạt trần" (B, C_{mid}). Khảo sát hơn 400.000 GPU-giờ cho thấy phần lớn "common wisdom" (loss aggregation, curriculum, length penalty, advantage normalization) chủ yếu ảnh hưởng tốc độ, không ảnh hưởng trần; còn thuật toán off-policy, loss type, và độ chính xác số học (FP32 ở LM head) là những quyết định hiếm hoi **thực sự nâng trần**. Recipe tổng hợp ScaleRL (không phát minh kỹ thuật mới, chỉ chọn lọc và phối hợp) scale ổn định và dự đoán được tới 100.000 GPU-giờ, vượt các recipe phổ biến khác (bao gồm cách triển khai lại của DAPO và GSPO) về cả trần hiệu năng và hiệu suất compute.

Link gốc: https://arxiv.org/abs/2510.13786

Bài 45

Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? (2025)

Tác giả: Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue (hai người tên "Yang Yue" khác chữ Hán, không phải cùng một người), Shiji Song, Gao Huang (LeapLab, Tsinghua University; Shanghai Jiao Tong University). **Venue / arXiv:** arXiv:2504.13837 (v5, 24/11/2025). Bài báo không nêu venue hội nghị cụ thể. **Nguồn:** https://arxiv.org/abs/2504.13837 **Chủ đề:** Dùng pass@k ở k lớn để đo "biên năng lực reasoning" (reasoning capacity boundary) của LLM, so sánh mô hình gốc (base) với mô hình đã huấn luyện RLVR trên toán, code, và visual reasoning. Kết luận: RLVR hiện tại chủ yếu làm tăng hiệu quả lấy mẫu (sampling efficiency) trên những bài mô hình nền đã giải được, không sinh ra đường suy luận mới; ngược lại, distillation từ teacher mạnh hơn mới thực sự mở rộng biên năng lực. **Vị trí trong lộ trình:** Đọc sau 36 (DeepSeekMath, GRPO), 39 (DeepSeek-R1, zero-RL), 40 (Kimi k1.5), 41 (DAPO), 42 (Dr. GRPO, bài gốc tạo ra checkpoint Oat-Zero được dùng làm model thử nghiệm ở đây), 43 (GSPO), 44 (ScaleRL). Bài này mở nhóm "RL thực sự thay đổi gì ở model?" (45-48, và nối sang 49-52): 46 (ProRL) phản biện trực tiếp kết luận của bài này (gọi đích danh là tài liệu tham khảo [13]); 47 (1-shot RLVR) và 48 (Spurious Rewards) cùng hướng, cung cấp thêm bằng chứng rằng RLVR khai thác năng lực đã có sẵn trong mô hình nền hơn là dạy năng lực mới.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Từ OpenAI-o1 và DeepSeek-R1 (bài 39), "RL với phần thưởng kiểm chứng được" (RLVR, Reinforcement Learning with Verifiable Rewards) trở thành công thức chuẩn để dạy LLM suy luận: cho model sinh chuỗi suy nghĩ (CoT) rồi thưởng nhị phân 0/1 dựa trên việc đáp số cuối có đúng không (verifier xác định bằng quy tắc, không cần mô hình phần thưởng học). Niềm tin phổ biến: giống RL cổ điển (AlphaGo, Atari), RLVR giúp model tự cải thiện liên tục, học được chiến lược suy luận mới vượt ra ngoài những gì mô hình nền (model chỉ pretrain, chưa RL) đã biết.

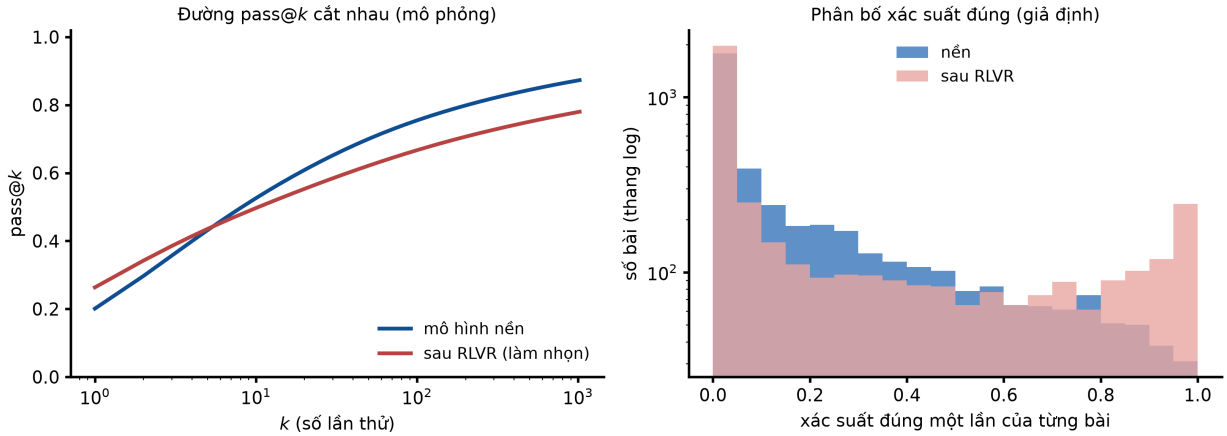
Bài này đặt câu hỏi ngược: RLVR có thực sự dạy năng lực mới, hay chỉ "định hình lại" phân phối đầu ra để model hay chọn đúng câu trả lời nó *đã* có khả năng sinh ra?

Khái niệm cần nắm trước khi đọc (xem lại ở bài 00 nếu cần):

- **Zero-RL:** áp RL trực tiếp lên mô hình nền, không SFT trung gian (cách DeepSeek-R1-Zero làm, bài 39). Bài báo này dùng zero-RL cho toán để cách ly ảnh hưởng của RLVR; với code và vision thì dùng mô hình đã instruct-tune làm điểm xuất phát vì zero-RL kém ổn định ở hai domain đó.
- **Policy gradient:** PPO/GRPO (bài 00, 07, 36) tối đa hóa log-likelihood của mẫu có thưởng cao, giảm log-likelihood mẫu thưởng thấp; học hoàn toàn từ mẫu on-policy (tự model sinh ra).
- **pass@k:** cho model sinh k câu trả lời độc lập cho một câu hỏi; tính 1 nếu ít nhất một trong k câu đúng, 0 nếu không; trung bình trên cả tập dữ liệu. Khác với **best-of-N** hay **majority voting** (chọn một câu trả lời cụ thể bằng verifier/voting), pass@k đo "biên khả năng" (model *có thể* giải được bài này trong bao nhiêu lần thử), tức là một chỉ số đo *tiềm năng*, không phải *hiệu năng thực dụng*.

2. Ý tưởng chính

Nếu chỉ nhìn pass@1 (độ chính xác trung bình một lần sinh), RLVR luôn thắng mô hình nền, đúng như kỳ vọng. Nhưng khi tăng k lên hàng chục, hàng trăm, hàng nghìn, mô hình nền dần bắt kịp rồi **vượt** model RLVR ở hầu hết benchmark toán, code, và visual reasoning đã thử. Nghĩa là: mô hình nền có "độ phủ" (coverage) các bài giải được rộng hơn; RLVR chỉ làm tăng xác suất model chọn đúng path nó hay đi, đổi lại thu hẹp các path khác. Phân tích perplexity (model RLVR sinh ra các path mà chính mô hình nền cũng có xác suất sinh ra không nhỏ) và phân tích tập bài giải được (set bài RLVR giải được gần như là tập con của set mô hình nền giải được) cùng ủng hộ giả thuyết: **năng lực reasoning của RLVR bị chặn trên bởi mô hình nền**. Ngược lại, **distillation** từ một teacher mạnh hơn (ví dụ DeepSeek-R1) đưa vào kiến thức mới thực sự, nên có thể vượt qua biên của mô hình nền.



Hình 81: Mô phỏng đường pass@k cắt nhau khi RLVR làm nhọn xác suất đúng của từng bài (minh hoạ, tự tính)

Cách đọc hình: mô phỏng giả định xác suất đúng mỗi bài của mô hình nền theo phân phối Beta, còn RLVR được mô hình hoá là kéo bài dễ lên và bài khó xuống (không phải số của paper). Bên trái, đường RLVR cao hơn ở k nhỏ rồi bị mô hình nền vượt ở k lớn. Bên phải cho thấy nguyên nhân: sau làm nhọn, nhiều bài dồn về xác suất 0 hoặc gần 1.

3. Phương pháp chi tiết

3.1 Công thức RLVR (nhắc lại ngắn)

Với policy π_θ, prompt $x \sim D$, verifier nhị phân $V(x, y) \in \{0, 1\}$ (đúng/sai đáp số cuối, có thể cộng thêm format reward), mục tiêu RL:

$$J(\theta) = \mathbb{E}_{x\sim D}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[\, r\,], \qquad r = V(x, y)$$

PPO tối ưu bằng surrogate clip: $\mathcal{L}^{CLIP} = \mathbb{E}[\min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$ với $r_t(\theta) = \pi_\theta(y_t|x, y_{<t})/\pi_{\theta_{old}}(y_t|x, y_{<t})$. Chi tiết đầy đủ PPO/GRPO xem bài 00, 07, 36.

3.2 Đo biên năng lực: pass@k với estimator không chệch

Đo pass@k trực tiếp bằng cách sinh đúng k mẫu rồi kiểm tra rất nhiều phương sai (ít mẫu, dễ "ăn may" hoặc "xui"). Theo cách của Chen et al. (2021, bài gốc pass@k cho code generation), với mỗi câu hỏi x_i sinh $n \geq k$ mẫu, đếm số mẫu đúng c_i, thì:

$$\boxed{\text{pass@k} := \mathbb{E}_{x_i \sim D}\left[1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}}\right]}$$

Vì sao công thức này đúng và không chệch: $\binom{n-c_i}{k}/\binom{n}{k}$ là xác suất khi *chọn ngẫu nhiên không lặp* k mẫu trong n mẫu đã sinh, **không có mẫu đúng nào** nằm trong k mẫu đó (chọn toàn bộ k mẫu từ $n - c_i$ mẫu sai). Vậy 1 trừ đi xác suất đó chính là xác suất *có ít nhất một mẫu đúng* trong k mẫu, đúng định nghĩa pass@k, nhưng tính trên toàn bộ $\binom{n}{k}$ cách chọn con (không chỉ một lần rút mẫu), nên phương sai thấp hơn nhiều so với rút đúng k mẫu một lần. Bài báo dùng n là giá trị k lớn nhất trên biểu đồ (128, 256, hoặc 1024 tùy benchmark).

Ví dụ nhỏ (tự tính): giả sử sinh $n = 5$ mẫu cho một câu hỏi, trong đó $c = 2$ mẫu đúng. Muốn biết pass@2:

$$\text{pass@2} = 1 - \frac{\binom{5-2}{2}}{\binom{5}{2}} = 1 - \frac{\binom{3}{2}}{\binom{5}{2}} = 1 - \frac{3}{10} = 0.7$$

Diễn giải: nếu chỉ được thử 2 lần (chọn ngẫu nhiên 2 trong 5 mẫu đã có), có 70% khả năng trúng ít nhất một câu đúng. Việc này tính trên toàn bộ $\binom{5}{2} = 10$ cách chọn thay vì chỉ sinh 2 mẫu rồi may rủi.

Vấn đề "đoán mò": ở toán, với k rất lớn, mô hình có thể ra đáp số đúng từ CoT sai (đoán trúng). Paper xử lý bằng cách **kiểm tra tay** một tập con CoT ở các câu khó nhất (accuracy cơ sở $< 5\%$): trên GSM8K, mô hình nền giải đúng 25 câu loại này, 24/25 có ít nhất một CoT hợp lệ; RLVR giải đúng 25 câu, 23/25 có CoT hợp lệ. Trên AIME24 (khó hơn): base giải 7 câu, 5/6 CoT hợp lệ (bỏ 1 trường hợp mơ hồ); RLVR giải 6 câu, 4/6 CoT hợp lệ. Trên visual reasoning: 7/8 có CoT hợp lệ cho cả hai loại model. Kết luận: pass@k không chỉ là đoán mò, phần lớn là giải thật.

3.3 Sampling Efficiency Gap (Δ_{SE})

Để so sánh các thuật toán RL một cách định lượng, paper định nghĩa:

$$\boxed{\Delta_{SE} = \text{pass@}1_{\text{RL}} - \text{pass@}k_{\text{base}} \;\; (k = 256)}$$

Δ_{SE} nhỏ (càng gần 0 càng tốt) nghĩa là RL đã khai thác gần hết "trần" mô hình nền đặt ra bằng pass@1 (một lần sinh). Δ_{SE} càng âm (hoặc rất dương) nghĩa là RL còn cách xa mức tối ưu lý thuyết khi coi mô hình nền làm chuẩn trên.

3.4 Perplexity: kiểm tra path của RLVR có "thuộc" phân phối base không

$$PPL_m(Y|x) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P_m(y_t \mid x, y_1, \ldots, y_{t-1})\right)$$

$PPL_{Base}(Y_{RL}|x)$ thấp nghĩa là mô hình nền vẫn "dễ" sinh ra đúng những câu mà model RLVR sinh, tức là các path đó nằm trong vùng xác suất cao của mô hình nền, không phải path hoàn toàn mới.

4. Thí nghiệm và kết quả chính

Thiết lập (Bảng 1 trong bài báo): temperature 0.6, top-p 0.95, tối đa 16384 token sinh, không dùng few-shot cho mô hình nền (để tránh gà con confound).

Lĩnh vực	Model khởi điểm	Framework RL	Thuật toán	Benchmark
Toán	LLaMA-3.1-8B, Qwen2.5-7B/14B/32B-Base, Qwen2.5-Math-7B	SimpleRLZoo, Oat-Zero, DAPO	GRPO	GSM8K, MATH500, Minerva, Olympiad, AIME24, AMC23
Code	Qwen2.5-7B-Instruct, DeepSeek-R1-Distill-Qwen-14B	Code-R1, DeepCoder	GRPO	LiveCodeBench, HumanEval+
Visual reasoning	Qwen2.5-VL-7B	EasyR1	GRPO	MathVista, MathVision
Phân tích sâu	Qwen2.5-7B-Base/Instruct, DeepSeek-R1-Distill-Qwen-7B	VeRL	PPO, GRPO, Reinforce++, RLOO, ReMax, DAPO	Omni-Math-Rule, MATH500

Toán (3.1): trên mọi benchmark (GSM8K, MATH500, Minerva, Olympiad, AIME24, AMC23) và mọi family model (Qwen2.5-7B/14B/32B, LLaMA-3.1-8B), cùng một kiểu đường cong: *ở* k nhỏ (ví dụ $k = 1$) RLVR thắng rõ; khi k tăng lên vài chục đến vài trăm, mô hình nền bắt kịp rồi vượt. Ví dụ cụ thể: trên Minerva với model 32B, base vượt RL khoảng **9 điểm %** *ở* $k = 128$ (tự diễn giải: nghĩa là base giải được thêm ~9% câu hỏi mà RL không giải được trong 128 lần thử). Với Oat-Zero-7B và DAPO-32B (hai checkpoint rất mạnh trên AIME24), RL vượt base tới ~30% *ở* k nhỏ nhưng **vẫn bị base vượt lại** khi k đủ lớn.

Code (3.2): CodeR1-Zero-Qwen2.5-7B (huấn luyện trên 12K bài LeetCode/TACO, 832 step) và DeepCoder-14B (từ DeepSeek-R1-Distill-Qwen-14B) trên LiveCodeBench v5, HumanEval+, MBPP+: cùng xu hướng crossover như toán. Vì code dùng unit test để chấm, "đoán mò" gần như không xảy ra, nên pass@k ở đây đáng tin hơn.

Visual reasoning (3.3): Qwen2.5-VL-7B train bằng EasyR1 trên Geometry3K, test trên MathVista/MathVision (đã lọc câu trắc nghiệm): cùng pattern.

Bảng phủ bài giải được (Table 2, tự trích từ paper):

	Base	SimpleRLZoo (RL)	AIME24 (k=1024)	MATH500 (k=128)
✓	✓		63.3%	92.4%
✓	✗		13.3%	3.6%
✗	✓		0.0%	1.0%
✗	✗		23.3%	3.0%

Đọc bảng: gần như không có câu nào RL giải được mà base không giải được (0.0% và 1.0%), nhưng có 13.3% / 3.6% số câu base giải được mà RL lại **không** giải được. Tức tập bài RL giải được gần như là **tập con** của tập base giải được, không mở rộng ra ngoài.

5. Phân tích / ablation

Phân tích perplexity (4.1): lấy 2 câu AIME24, cho Qwen2.5-7B-Base và SimpleRL-Qwen2.5-7B sinh 16 câu trả lời mỗi câu, và OpenAI-o1 sinh 8 câu trả lời (coi như "ground-truth style"). Phân phối $PPL_{Base}(Y_{RL}|x)$ trùng khít với **phần dưới** (perplexity thấp, tức dễ sinh) của phân phối $PPL_{Base}(Y_{Base}|x)$: nghĩa là những gì RL sinh ra cũng là những gì mô hình nền vẫn hay sinh (chỉ là base ít chọn thường xuyên hơn). Theo thời gian train, $PPL_{Base}(Y_{RL}|x)$ giảm dần, tức RLVR ngày càng "làm nhọn" (sharpen) một vùng đã có sẵn trong base, chứ không mở rộng ra vùng mới.

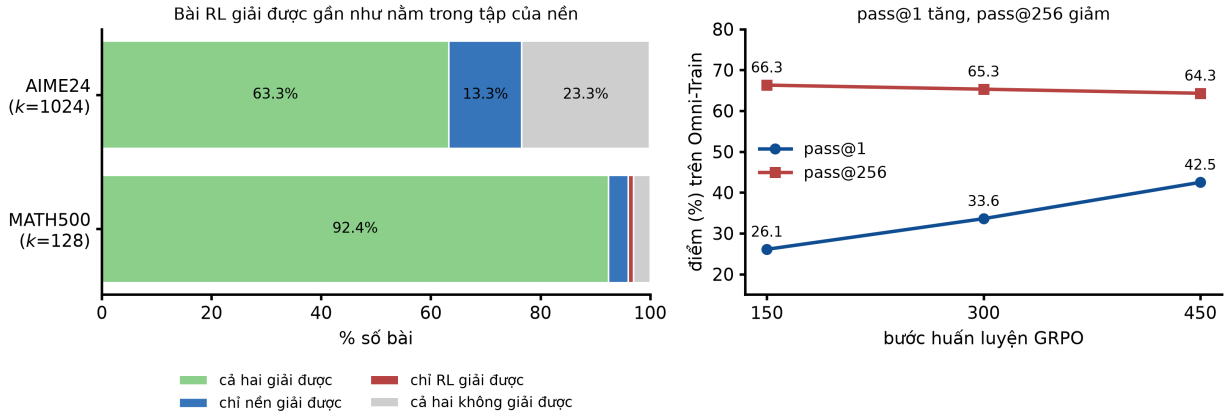
Distillation mở rộng biên, RLVR thì không (4.2): so sánh DeepSeek-R1-Distill-Qwen-7B (distill từ DeepSeek-R1 vào Qwen2.5-Math-7B) với base Qwen2.5-Math-7B, với bản RLVR (Oat-Zero) của chính base đó, và với bản Instruct. Trên Minerva, đường pass@k của model **distill** nằm **cao hơn** base ở mọi k (không bị base bắt kịp), trong khi model RLVR vẫn bị crossover như các thí nghiệm trước. Diễn giải của tác giả: distillation bơm vào kiến thức từ một teacher khác, còn RLVR chỉ tinh chỉnh lại xác suất trong không gian mà chính mô hình nền đã biết.

So sánh 6 thuật toán RL (4.3, Bảng 3 trong bài báo): train lại công bằng bằng VeRL trên Omni-MATH-Rule (2000 train / 821 test in-domain), test thêm out-of-domain trên MATH500. Không dùng KL penalty (theo convention của DAPO/Oat-Zero).

Model	Omni-Train pass@1/pass@256	Omni-Test pass@1/pass@256	MATH500 pass@1/pass@256
Qwen2.5-7B (base)	9.9 / 67.2	10.2 / 69.1	34.5 / 96.2
GRPO	26.1 / 66.3	25.1 / 68.3	74.4 / 97.2
PPO	27.2 / 65.8	26.8 / 69.2	75.2 / 97.2
ReMax	24.4 / 65.5	23.8 / 67.5	73.5 / 96.6
RLOO	28.6 / 66.4	28.1 / 69.2	75.0 / 97.4
Reinforce++	28.2 / 67.7	28.0 / 69.7	75.4 / 96.8
DAPO	31.4 / 66.1	26.5 / 67.0	75.6 / 96.4

Δ_{SE} (Omni-Test, $k = 256$): GRPO = 43.9, RLOO tốt nhất = 42.6. **Tất cả 6 thuật toán đều có** $\Delta_{SE} > 40$ dù khác biệt tuyệt đối giữa chúng nhỏ: không thuật toán nào áp sát "trần" lý thuyết (pass@256 của base). Tác giả cho rằng cần paradigm RL mới, không chỉ tinh chỉnh thuật toán hiện có.

Theo số bước train (4.4, Bảng 4): GRPO step 150 → 300 → 450 trên Omni-Train: pass@1 tăng đơn điệu $26.1 \rightarrow 33.6 \rightarrow 42.5$ (RL càng train càng "tự tin" hơn trên tập train), nhưng pass@256 lại **giảm dần** $66.3 \rightarrow 65.3 \rightarrow 64.3$: train lâu hơn làm RL càng thu hẹp biên reasoning, không mở rộng nó. Tăng số rollout mỗi prompt ($n = 8 \rightarrow 32$) chỉ cải thiện pass@k rất nhẹ, RL vẫn bị base vượt. Thêm KL penalty ($\beta = 0.001$) giữ pass@1 tương đương GRPO không-KL nhưng pass@128 thấp hơn nhiều.



Hình 82: Độ phủ bài giải được của nền và RL (Bảng 2), và pass@1, pass@256 của GRPO theo số bước (Bảng 4) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: bên trái, phần màu đỏ (chỉ RL giải được) gần như không thấy, còn phần xanh dương (chỉ nền giải được) lớn hơn rõ. Bên phải, pass@1 tăng đều theo số bước nhưng pass@256 giảm nhẹ, nghĩa là train lâu hơn làm biên khả năng thu hẹp. Số liệu chép từ hai bảng trong ghi chú.

Entropy (4.5): tăng temperature lúc sinh của model RLVR để khớp entropy của mô hình nền ở $T = 0.6$: pass@k cải thiện đôi chút nhưng **vẫn không đuổi được base**. Nghĩa là entropy thấp (do RL làm phân phối nhọn hơn) chỉ là một phần nguyên nhân thu hẹp biên, không phải toàn bộ.

Scale mô hình lớn hơn (4.6): dùng Magistral-Medium (RLVR, train thuần RL từ Mistral-Medium-3) so với base Mistral-Medium-3 trên AIME24/AIME25: cùng pattern "RL hơn ở k nhỏ, bị base bắt kịp ở k lớn" vẫn giữ nguyên ở quy mô gần frontier, dù kích thước model không được công bố.

6. Giới hạn

Tác giả nêu: nhiều model/quy trình RLVR mạnh nhất (ví dụ GPT-o1, Qwen3-235B train qua nhiều giai đoạn gồm cả SFT CoT dài) không cô lập được ảnh hưởng của riêng RLVR nên không đưa vào so sánh trực tiếp; DeepSeek-R1-Zero tự host chỉ chạy được ~50 token/giây ở ngữ cảnh 32k nên pass@k không khả thi về compute; RL cho LLM đang phát triển rất nhanh, kỹ thuật mới có thể giảm bớt hạn chế đã quan sát.

Nhận xét:

- Phần toán dùng zero-RL (RL thẳng từ base, không SFT), còn thực tế các quy trình RLVR công nghiệp thường SFT trước; kết luận có thể không áp dụng y nguyên cho setting SFT → RL.
- "Đoán mò" ở k lớn chỉ kiểm tra tay một mẫu nhỏ câu hỏi khó nhất (25, 7, 8 câu), chưa phải toàn bộ; vẫn có khả năng một phần cải thiện pass@k ở k rất lớn đến từ đoán đúng số.
- Không có thí nghiệm riêng về ProRL-style "RL kéo dài" (train rất nhiều bước, reset reference policy định kỳ) trước khi kết luận RL "không mở rộng biên" là tổng quát; bài 46 (ProRL) chỉ ra chính xác điểm này và tranh luận ngược lại một phần.
- Δ_{SE} so sánh pass@1 của RL với pass@256 của base; đây là một lựa chọn chuẩn khá nghiêm khắc (coi base ở $k = 256$ là "trần"), có thể khiến mọi thuật toán RL nhìn "giống nhau" hơn thực tế vì cùng bị giới hạn bởi cùng một base.

7. Liên hệ với các bài khác

- **39 (DeepSeek-R1), 40 (Kimi k1.5), 41 (DAPO):** nguồn gốc quy trình zero-RL/RLVR mà bài này đánh giá lại; DAPO cũng là một trong 6 thuật toán so sánh ở mục 4.3.
- **36 (DeepSeekMath, GRPO):** GRPO là thuật toán chính được dùng để huấn luyện phần lớn model thử nghiệm.
- **42 (Dr. GRPO):** checkpoint "Oat-Zero" dùng trong thí nghiệm AIME24 chính là sản phẩm huấn luyện của nhóm tác giả Dr. GRPO (Liu et al., trích dẫn [Liu et al., 2025b] trong bài này); hai bài cùng mạch phê bình cách đo lường RLVR.
- **46 (ProRL):** phản biện trực tiếp, trích dẫn bài này là tài liệu tham khảo [13], cho rằng kết luận "RL không mở rộng biên" chỉ đúng khi huấn luyện quá ngắn (bài báo này dùng tối đa vài trăm bước) và quá tập trung vào toán (domain đã bị overtrain ở cả pretrain và post-train).
- **47 (1-shot RLVR):** cùng hướng với bài này, lập luận rằng mô hình nền "đã có" năng lực reasoning từ trước, RLVR chỉ cần một tín hiệu rất nhỏ để "kích hoạt" nó.
- **48 (Spurious Rewards):** cùng hướng, cho thấy ngay cả phần thưởng ngẫu nhiên/sai cũng cải thiện Qwen2.5-Math vì RLVR khai thác hành vi đã có sẵn trong pretraining (code reasoning), không học năng lực mới; dùng chung giả thuyết "RLVR khai thác, không khám phá" với bài này.
- **49-52 (entropy mechanism, high-entropy minority tokens, RL finetunes small subnetworks, RL's razor):** các bài sau tiếp tục phân tích *cơ chế* bên trong RLVR (entropy, token nào thay đổi, phần nào của model thay đổi) để giải thích thêm hiện tượng "thu hẹp biên" quan sát ở đây.

8. Ứng dụng thực tế (suy luận)

- **Chọn chỉ số đánh giá đúng việc cần đo:** nếu mục tiêu là "model thực dụng, trả lời đúng ngay lần đầu", pass@1/accuracy trung bình vẫn là chỉ số hợp lý, và RLVR vẫn cải thiện nó rõ rệt (không có gì sai khi dùng RLVR cho mục đích này). Chỉ khi câu hỏi là "RL có dạy năng lực mới không" thì mới cần pass@k ở k lớn; đo pass@1 rồi khẳng định "model đã học reasoning mới" là kết luận quá đà theo bài báo này.
- **Khi cần model đa dạng về chiến lược giải (ví dụ để làm teacher cho distillation, hoặc để lấy nhiều lời giải khác nhau cho một bài khó), RLVR chuẩn (ngắn, ít bước) có xu hướng thu hẹp đa dạng; nếu mục tiêu là coverage/đa dạng, cần cân nhắc giữ entropy cao hơn (temperature cao khi sinh, hoặc kỹ thuật như ProRL ở bài 46) hoặc dùng mô hình nền sinh nhiều mẫu rồi lọc, thay vì chỉ tin vào pass@1 của model RLVR.**
- Δ_{SE} là một cách hữu ích để tự kiểm tra quy trình RLVR của mình: nếu pass@1 sau RL còn cách pass@256 của base rất xa, nghĩa là RL chưa khai thác hết "trần" sẵn có, có thể do explore kém (ít rollout, nhiệt độ sinh thấp) hơn là do thuật toán.
- **Không có khoảng cách lớn giữa 6 thuật toán RL (PPO, GRPO, Reinforce++, RLOO, ReMax, DAPO)** trong thí nghiệm ở đây: với một quy trình RLVR cỡ nhỏ/vừa, việc chọn thuật toán có thể ít quan trọng hơn việc chọn dữ liệu, cơ chế khám phá (exploration), hoặc thời gian train (xem bài 46 để thấy train dài hơn + đa dạng task mới là yếu tố thay đổi kết luận).

9. Thí nghiệm gợi ý

1. Lặp lại thí nghiệm pass@k crossover (mục 4.1) trên một mô hình nhỏ tự có sẵn (ví dụ Qwen2.5-1.5B-Base vs bản RLVR của nó trên GSM8K), dùng công thức estimator không chệch ở mục 3.2, với $n = 256$, để tự kiểm tra hiện tượng crossover có lặp lại ở quy mô nhỏ.
2. Đo Δ_{SE} (mục 3.3) cho một checkpoint RLVR tự train (ví dụ GRPO trên một bộ toán nhỏ) ở nhiều mốc step khác nhau, vẽ cả pass@1 và pass@256 theo step để xem có tái hiện mẫu hình "pass@1 tăng, pass@256 giảm" của Bảng 4.
3. Tính $PPL_{Base}(Y_{RL}|x)$ so với $PPL_{Base}(Y_{Base}|x)$ (mục 3.4) cho một tập nhỏ câu hỏi, để kiểm tra claim "path của RL vẫn nằm trong vùng xác suất cao của base" có đúng với model/bộ dữ liệu khác không.
4. Thử tăng temperature sinh của model RLVR để khớp entropy của base (như mục 4.5) và đo lại pass@k, xem mức độ "cứu được" bao nhiêu phần trăm khoảng cách crossover.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
pass@k (estimator không chệch)	$1 - \binom{n-c_i}{k} / \binom{n}{k}$, $n \geq k$	phương sai thấp hơn rút k mẫu trực tiếp
Sampling Efficiency Gap	$\Delta_{SE} = \text{pass@1}_{RL} - \text{pass@}k_{base}$ (k=256)	đo RL còn cách "trần" bao xa
Perplexity	$PPL_m(Y\|x) = \exp(-\frac{1}{T}\sum \log P_m(y_t\|\cdot))$	thấp = dễ sinh dưới model m
Crossover pass@k	RL thắng ở k nhỏ, base thắng ở k lớn	quan sát xuyên suốt toán/code/vision
Zero-RL	RL thắng từ base, không SFT	dùng cho toán để cô lập hiệu ứng RLVR
Distillation vs RLVR	distill vượt biên base, RLVR không	Hình 7, Minerva
6 thuật toán so sánh	PPO, GRPO, ReMax, RLOO, Reinforce++, DAPO	Δ_{SE} đều > 40

11. Kết luận ngắn

Dùng pass@k ở k lớn với estimator không chệch, bài này cho thấy RLVR (zero-RL, GRPO và 5 biến thể khác) trên toán/code/visual reasoning chủ yếu làm tăng xác suất model chọn đúng câu trả lời nó đã "biết" từ pretraining, chứ hiếm khi tạo ra con đường suy luận hoàn toàn mới; biên năng lực đo bằng pass@k ở k lớn thường **thu hẹp** sau RLVR, trong khi distillation từ teacher mạnh hơn mới thực sự mở rộng biên. Đây là một lời cảnh báo về cách diễn giải kết quả RLVR (đừng chỉ nhìn pass@1), nhưng kết luận "RL không bao giờ mở rộng biên" bị bài 46 (ProRL) phản biện khi huấn luyện RL đủ lâu và đủ đa dạng task.

Link gốc: https://arxiv.org/abs/2504.13837

Bài 46

ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models (2025)

Tác giả: Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, Yi Dong (NVIDIA). **Venue / arXiv:** arXiv:2505.24864 (v1, 30/5/2025). Bài báo không nêu venue hội nghị cụ thể (ghi "Preprint. Under review."). **Nguồn:** https://arxiv.org/abs/2505.24864 **Chủ đề:** Đề xuất **ProRL**, công thức RL "kéo dài" (>2000 bước, ~16k giờ GPU) kết hợp KL penalty + reset reference policy định kỳ + các mẹo của DAPO (decoupled clip, dynamic sampling) + dữ liệu đa domain (toán, code, STEM, logic puzzle, instruction following). Claim: nếu train đủ lâu và đủ đa dạng, RL *thực sự* mở rộng biên reasoning (pass@k lớn) so với mô hình nền, trái với kết luận của bài 45. **Vị trí trong lộ trình:** Đọc sau 36 (DeepSeekMath, GRPO, nền tảng thuật toán), 41 (DAPO, nguồn của decoupled clip và dynamic sampling), và 45 (bài liền trước trong cùng cluster "RL học được gì", mà bài này trích dẫn trực tiếp làm tài liệu tham khảo [13] và phản biện). Đọc tiếp 47, 48 để có cái nhìn đầy đủ về ba hướng tranh luận khác nhau trong cùng câu hỏi.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Bài 45 (và hai nghiên cứu khác được trích ở đây: Dang et al., đo chặn trên pass@k; Zhao et al., "Echo Chamber") cùng đưa ra lập luận: RLVR không dạy LLM năng lực mới, chỉ khai thác lại cái đã có trong mô hình nền, dựa trên việc pass@k ở k lớn giảm sau RL. Nhóm tác giả ProRL (NVIDIA) cho rằng kết luận đó có thể chỉ là **giả tạo do giới hạn thực nghiệm**, không phải giới hạn bản chất của RL, vì hai lý do:

1. **Quá tập trung vào domain đã "bị học nhiều"**: toán là domain mà cả pretraining và post-training đều train rất kỹ, nên mô hình nền đã gần như bão hòa, dư rất ít "không gian" để RL khám phá.
2. **Train RL quá ngắn**: các nghiên cứu phê bình RLVR thường chỉ train vài trăm bước. Nhóm ProRL cho rằng cần train lâu hơn nhiều (ở đây là >2000 bước) để mô hình có thời gian khám phá và "thẩm thấu" (internalize) chiến lược suy luận mới.

Thuật toán nền: **GRPO** (Group Relative Policy Optimization, bài 36). Không cần critic/value network như PPO, ước lượng advantage bằng cách chuẩn hóa điểm số trong một nhóm k rollout của cùng một prompt:

$$A(\tau) = \frac{R_\tau - \text{mean}(\{R_i\}_{i\in G(\tau)})}{\text{std}(\{R_i\}_{i\in G(\tau)})}$$

$$\mathcal{L}_{GRPO}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\min\big(r_\theta(\tau)A(\tau),\ \text{clip}(r_\theta(\tau),1-\epsilon,1+\epsilon)A(\tau)\big)\Big], \quad r_\theta(\tau) = \frac{\pi_\theta(\tau)}{\pi_{\theta_{old}}(\tau)}$$

Vấn đề khi huấn luyện GRPO rất lâu: **entropy collapse**. Phân phối đầu ra của model trở nên quá nhọn rất sớm trong quá trình huấn luyện, khiến các rollout trong một nhóm ít đa dạng, advantage ước lượng kém, policy update bị lệch, và việc train bị "tắc" (stagnate). Tăng temperature lúc rollout chỉ **trì hoãn** entropy collapse, không ngăn được nó.

2. Ý tưởng chính

ProRL = GRPO + ba thành phần giúp train ổn định rất lâu:

1. **Các mẹo của DAPO** (bài 41): decoupled clip (tách ngưỡng clip dưới/trên, ngưỡng trên cao hơn để "nới" cho token xác suất thấp tăng lên, chống entropy collapse) và dynamic sampling (lọc bỏ prompt mà model luôn đúng hoặc luôn sai, vì không mang tín hiệu học).
2. **KL penalty + reference policy reset**: giữ một hệ số phạt KL-divergence giữa policy hiện tại và một policy tham chiếu để ổn định training và tránh overfit vào reward giả (spurious reward signal); khi KL quá lớn làm update "tắc", **reset cứng** policy tham chiếu về một checkpoint gần đây của chính policy đang train, rồi train tiếp.
3. **Dữ liệu đa domain** (136K mẫu: toán, code, STEM, logic puzzle, instruction following) để tránh model chỉ tối ưu một domain đã bão hòa.

Với công thức này, nhóm tác giả train ra **Nemotron-Research-Reasoning-Qwen-1.5B**, claim là model 1.5B reasoning tốt nhất tại thời điểm công bố, vượt xa base DeepSeek-R1-Distill-Qwen-1.5B và sánh ngang/vượt DeepSeek-R1-Distill-Qwen-7B (7 lần tham số) trên nhiều benchmark. Quan trọng hơn cho câu hỏi "RL học được gì": đo lại bằng pass@k ở k lớn (tới 256), nhóm tác giả tìm thấy nhiều task mà **mô hình nền luôn luôn sai dù thử bao nhiêu lần (pass@k ~ 0 mọi k)**, còn model ProRL đạt pass rate gần 100%, một bằng chứng khó chối rằng RL (ở quy mô train này) đã học được năng lực hoàn toàn vắng mặt ở base.

3. Phương pháp chi tiết

3.1 Decoupled clip (DAPO)

GRPO gốc dùng một ngưỡng clip ϵ đối xứng. DAPO tách thành hai ngưỡng riêng:

$$\text{clip}(r_\theta(\tau),\, 1-\epsilon_{low},\, 1+\epsilon_{high})$$

ProRL dùng $\epsilon_{low} = 0.2$, $\epsilon_{high} = 0.4$. Đặt ϵ_{high} lớn hơn ("clip-higher") để các token đang có xác suất thấp dễ tăng mạnh hơn khi nhận advantage dương, giữ entropy cao hơn, chống việc model "khóa cứng" vào một vài token sớm.

3.2 Dynamic sampling (DAPO)

Lọc khỏi batch những prompt mà cả nhóm rollout đều đúng (accuracy = 1) hoặc đều sai (accuracy = 0): advantage chuẩn hóa theo nhóm sẽ bằng 0 với các prompt này (vì mọi R_i như nhau), không có tín hiệu gradient. Giữ lại các prompt độ khó trung bình giúp signal học luôn "có ích".

3.3 KL penalty và reference policy reset

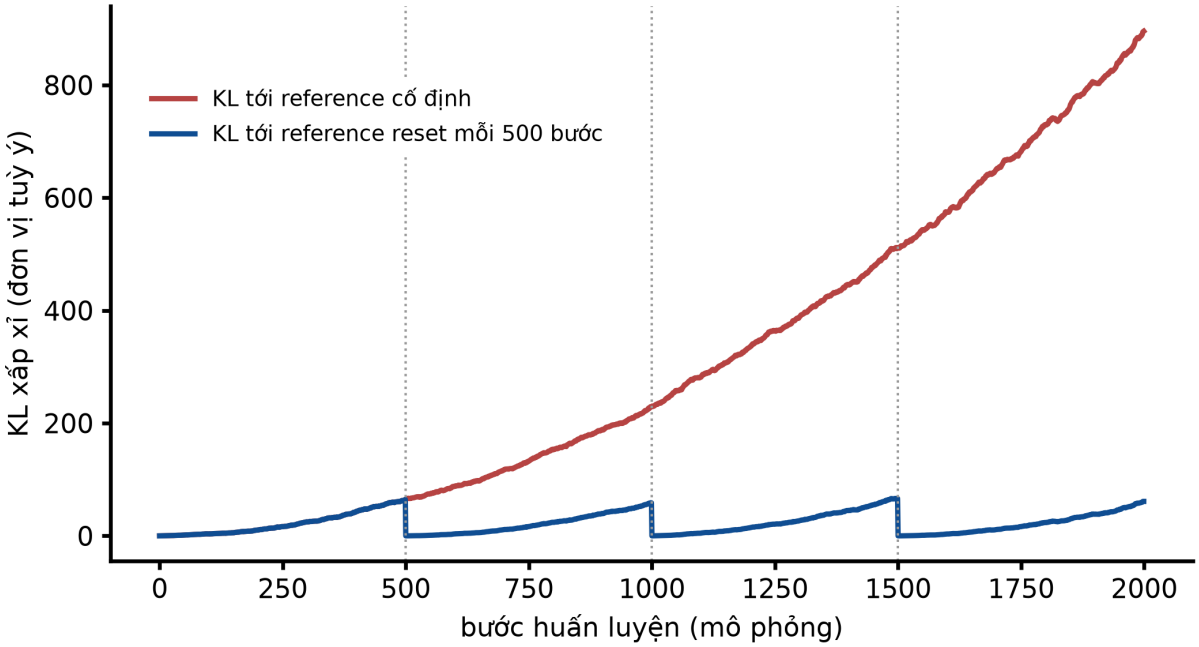
$$\mathcal{L}_{KL\text{-}RL}(\theta) = \mathcal{L}_{GRPO}(\theta) - \beta\, D_{KL}(\pi_\theta \| \pi_{ref})$$

Khác với DAPO/Oat-Zero (bỏ hẳn KL để không hạn chế model học, như bài 45 cũng làm theo convention này), ProRL **giữ** KL penalty vì model khởi điểm (DeepSeek-R1-Distill-Qwen-1.5B) đã là một checkpoint đã SFT trên CoT dài, có sẵn một "văn phong suy luận" hợp lý; KL giúp model không trôi quá xa khỏi hành vi hợp lý đó trong khi vẫn học.

Vấn đề mới nảy ra: train càng lâu, số hạng KL ngày càng chiếm ưu thế trong loss (vì π_θ càng ngày càng khác π_{ref} cố định), khiến update ngày càng nhỏ, training "tắc" dần. Giải pháp:

Reference policy reset: định kỳ gán $\pi_{ref} \leftarrow \pi_\theta$ (checkpoint gần đây), khởi tạo lại optimizer state

Việc reset vừa phục hồi độ ổn định của training (KL quay lại giá trị nhỏ), vừa cho phép model đi xa hơn tổng thể so với π_{ref} ban đầu (vì mỗi lần reset lại "dời mốc" tham chiếu đi theo).



Hình 83: Mô phỏng KL tới reference khi có và không có reset định kỳ (minh hoạ, tự tính)

Cách đọc hình: mô phỏng random walk có độ trôi với giả định tự đặt, không phải số của paper. Đường đỏ là KL tới reference cố định và tăng mãi, nên số hạng KL ngày càng lấn át loss. Đường xanh là KL khi reset mỗi 500 bước (các vạch chấm): KL về gần 0 sau mỗi lần reset trong khi chính sách vẫn đi xa dần so với điểm xuất phát.

3.4 Pseudo-code huấn luyện ProRL

```
Input: base checkpoint pi_theta0 (DeepSeek-R1-Distill-Qwen-1.5B), reward verifiers 5
↪  domain,
       KL coeff beta, clip eps_low=0.2, eps_high=0.4, rollout n=16, temp=1.2, ctx=8k

pi_ref <- pi_theta0
repeat:
    sample batch 256 prompts (math/code/STEM/logic/instruct, blend 136K)
    rollout n=16 responses/prompt tu pi_theta (temperature cao de giu entropy)
    loai prompt co accuracy group == 0 hoac == 1   (dynamic sampling)
    tinh advantage group-normalized A(tau)
    L = E[min(r*A, clip(r, 1-eps_low, 1+eps_high)*A)] - beta * KL(pi_theta || pi_ref)
    update theta (AdamW, lr=2e-6, mini-batch 64 => 4 gradient update/rollout step)
    theo doi validation blend (AIME24, Codeforces, GPQA-diamond, IFEval, graph_color)
    if validation stagnate hoac degrade:
        pi_ref <- pi_theta (hard reset); reset optimizer state
        (co the dieu chinh them: them data, reward shaping, tang rollout n, tang ctx
        ↪  window)
until du ~2000+ step (~16k GPU-hour, 4 node 8xH100)
```

Thực tế recipe trải qua 8 "run" (Phụ lục E của bài báo), mỗi lần reset đi kèm một thay đổi nhỏ: thêm instruction-following data (run 3), thêm reward phạt câu trả lời không kết thúc đúng <eos> (run 4-5, vì model lặp câu không dừng), tăng rollout n từ 16 lên 32 (run 6-7), rồi mở rộng context window từ 8k lên 16k ở giai đoạn cuối

(run 8, model thích nghi nhanh dù phần lớn thời gian train ở 8k).

4. Thí nghiệm và kết quả chính

Dữ liệu train (136K mẫu, Bảng 4 paper):

Domain	Loại reward	Số lượng	Nguồn
Toán	Nhị phân	40K	DeepScaleR dataset
Code	Liên tục (tỉ lệ test case qua)	24K	Eurus-2-RL dataset
STEM	Nhị phân	25K	SCP-116K
Logic puzzle	Liên tục	37K	Reasoning Gym
Instruction following	Liên tục	10K	Llama-Nemotron

Setup: verl framework, AdamW, lr 2×10^{-6} cố định, batch 256 / mini-batch 64 (4 update/step), rollout $n = 16$, context 8096 (mở lên 16k ~200 step cuối), temperature rollout 1.2, 4 node 8×H100-80GB, ~16k giờ GPU tổng.

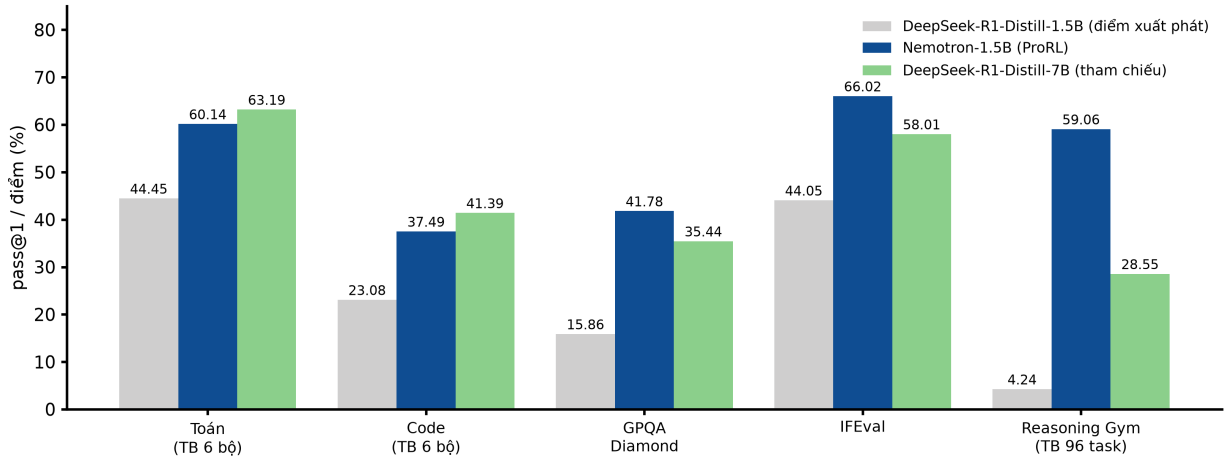
Kết quả pass@1 so với DeepSeek-R1-Distill-Qwen-1.5B (từ Bảng 1-3 paper):

Domain	Benchmark	DS-R1-Distill-1.5B	DeepScaleR/DeepCoder-1.5B (chuyên biệt)	Nemotron-1.5B (ProRL)	DS-R1-Distill-7B (tham chiếu)
Toán (avg 6 bộ)	AIME24/25, AMC, MATH, Minerva, Olympiad	44.45	54.54 (DeepScaleR)	**60.14**	63.19
Code (avg 6 bộ)	apps/cc/cf/taco/human/ICPC	27.08	30.96 (DeepCoder)	**37.49**	41.39
STEM (GPQA)	GPQA-Diamond	15.86	-	**41.78**	35.44
Instruction (IFEval)	IFEval	44.05	-	**66.02**	58.01
Logic puzzle (Reasoning Gym)	96 task trung bình	4.24	-	**59.06**	28.55

Nhận xét số liệu: model 1.5B sau ProRL **vượt** DeepSeek-R1-Distill-Qwen-7B (gấp ~5 lần tham số) ở GPQA, IFEval, Reasoning Gym, và code; chỉ thấp hơn một chút ở toán (60.14 so với 63.19). So với hai baseline chuyên biệt hóa (DeepScaleR-1.5B chỉ train toán, DeepCoder-1.5B chỉ train code), ProRL (generalist, train đa domain) vẫn **thắng cả hai domain đó**: +4.6% toán, +6.5% code, dù không chuyên biệt.

Cách đọc hình: mỗi nhóm cột là một domain, ba cột là ba mô hình. Cột xanh đậm (ProRL, 1.5B) vượt cột xám (điểm xuất phát) ở mọi domain và vượt cả mô hình 7B ở GPQA, IFEval, Reasoning Gym và gần ngang ở code. Ở toán, 7B vẫn nhỉnh hơn (63.19 so với 60.14).

Tổng hợp mức tăng trung bình (nguyên văn abstract, bài báo có hai cách tính hơi lệch nhau): phần tóm tắt (Abstract) ghi "+14.7% toán, +13.9% code, +54.8% logic puzzle, +25.1% STEM, +18.1% instruction-following", còn đoạn văn ngay trước Bảng 1 (thân bài) ghi "+15.7% toán, +14.4% code, +25.9% STEM, +22.0% instruction, +54.8% logic puzzle" (khớp với số tự tính từ Bảng 1: $60.14 - 44.45 = 15.69 \approx 15.7$). Người ghi chú trích nguyên cả hai, không tự diễn giải lý do lệch (có thể abstract tính trung bình theo cách khác, ví dụ trung bình % cải thiện tương đối theo từng benchmark con).



Hình 84: pass@1 theo domain của DeepSeek-R1-Distill-1.5B, Nemotron-1.5B (ProRL) và DeepSeek-R1-Distill-7B (vẽ lại từ số liệu trong ghi chú)

OOD (ngoài phân phối train, Bảng 3): ba task Reasoning Gym không có trong 37K mẫu train: `acre`, `boxnet`, `game_of_life_halting`. DeepSeek-R1-Distill-1.5B đạt 5.99 / 0.00 / 3.49 (hầu như không giải được); DeepSeek-R1-Distill-7B đạt 20.21 / 1.71 / 12.94; Nemotron-1.5B (ProRL) đạt **58.57 / 7.91 / 52.29**, vượt cả model 7B lớn hơn nhiều trên task nó chưa từng thấy.

5. Phân tích / ablation

"Yếu lúc đầu, lợi nhiều từ ProRL" (4.1): đo tương quan giữa pass@128 ban đầu của mô hình nền trên từng task và mức cải thiện sau ProRL (dùng 256 mẫu để đánh giá lại, chọn ngẫu nhiên 18/96 task Reasoning Gym do giới hạn compute). Kết quả: **tương quan âm rõ rệt**: task mà mô hình nền đã mạnh sẵn (pass@128 cao) cải thiện rất ít hoặc còn giảm sau RL; task mà mô hình nền yếu (pass@128 thấp) cải thiện mạnh nhất. Tác giả kiểm chứng thêm bằng **Creativity Index** (đo độ trùng lặp với corpus pretrain DOLMA): các task cải thiện ít có Creativity Index thấp, nghĩa là mô hình nền đã "thấy" dữ liệu tương tự nhiều trong pretrain, không còn nhiều chỗ để RL khai phá thêm.

Ba kiểu diễn biến pass@k theo training (4.2, chia theo quan sát benchmark cụ thể):

- **Diminish (thu hẹp):** chủ yếu ở toán. pass@1 tăng nhưng pass@128 giảm, giống hiện tượng bài 45 quan sát. Giải thích: các task này có pass@128 cơ sở đã cao, RL chỉ "làm nhọn" phân phối, đánh đổi lấy explore.
- **Plateau (bão hòa sớm):** cả pass@1 và pass@128 tăng, nhưng tăng chủ yếu ở giai đoạn đầu; so sánh checkpoint giữa và checkpoint cuối cho thấy train thêm gần như không lợi thêm.
- **Sustained (tăng liên tục):** đặc biệt ở code, model tiếp tục cải thiện pass@128 suốt quá trình train dài, ngụ ý cần explore nhiều instance khác nhau mới tổng quát hóa được, và train dài thực sự "trả công".

OOD boxnet (4.3): mô hình nền đạt pass@k ≈ 0 ở **mọi** giá trị k (kể cả $k = 256$, chưa từng giải được), còn model ProRL cuối đạt pass@k tiến gần 1.0 khi k tăng (checkpoint trung gian thấp hơn checkpoint cuối, cho thấy train dài hơn **tiếp tục** khuếch đại khả năng mới này). Đây là bằng chứng mạnh nhất của bài báo: mô hình nền **không thể** giải được task này dù cho bao nhiêu lần thử (khác hẳn pattern "base bắt kịp ở k lớn" của bài 45), nên cải thiện không thể giải thích bằng "chỉ làm nhọn lại phân phối cũ".

Diễn biến phân phối pass@1 (4.4): tác giả nhắc công thức chặn trên pass@k của Dang et al.: $\mathbb{E}[\text{pass@}k] \leq 1 - \left((1 - \mathbb{E}[\rho_x])^2 + \text{Var}(\rho_x)\right)^{k/2}$ với ρ_x là pass@1 từng task (ρ_x tăng thì chặn trên tăng, nhưng Var(ρ_x) tăng thì chặn trên giảm). ProRL cho thấy pass@1 tăng đủ mạnh để bù lại hiệu ứng phương sai này: ví dụ task `family_relationships` chuyển từ phân phối dồn gần 0 sang dồn gần 1 (hầu hết prompt đều giải được) sau

train dài.

Reasoning Gym theo subcategory (Bảng 5, Phụ lục F): model base yếu đến mức chỉ 0.73-10.90 điểm ở các mục dễ như algebra/arithmetic (lý do: base quen dùng `\boxed{}` thay vì tag `<answer></answer>` theo yêu cầu, lỗi format chứ không hẳn lỗi reasoning); sau ProRL, algebra lên 97.21, arithmetic lên 82.81; các mục khó hơn (arc, code, cognition, games) cải thiện ít hơn nhiều (ví dụ arc chỉ 1.53 → 2.52), cho thấy giới hạn của RL vẫn tồn tại ở task đòi hỏi kỹ năng lõi mà model chưa có.

6. Giới hạn

Tác giả nêu (Phụ lục A): chi phí tính toán rất lớn (nhiều giai đoạn train, reset định kỳ, rollout CoT dài); chưa rõ công thức có scale tốt lên mô hình lớn hơn 1.5B hay không; việc phải hard-reset reference policy + optimizer định kỳ làm quy trình phức tạp, kết quả có thể kém ổn định hơn các phương pháp train đơn giản; dữ liệu train dù đa domain vẫn chỉ là một phần nhỏ trong không gian task reasoning khả dĩ, không đảm bảo tổng quát hóa tới mọi domain OOD khác.

Nhận xét:

- Bằng chứng mạnh nhất (boxnet, task hoàn toàn mới) là OOD so với *dữ liệu train* của chính ProRL, nhưng đây vẫn là một puzzle "trong họ" Reasoning Gym (cùng kiểu task tổng hợp, luật rõ ràng), khác hẳn domain mở như toán cạnh tranh; chưa rõ "mở rộng biên" có giữ nguyên trên domain hoàn toàn khác cấu trúc (ví dụ agentic, multi-turn) hay không.
- Phần toán vẫn rơi vào nhóm "Diminish" giống bài 45 quan sát; ProRL không phủ nhận hiện tượng đó ở toán, chỉ chỉ ra rằng hiện tượng **không phổ quát** cho mọi domain, đặc biệt domain mà mô hình nền còn yếu.
- Hai cách tính trung bình cải thiện (abstract vs. thân bài) không khớp nhau (mục 4 ở trên), bản thân paper chưa giải thích rõ công thức trung bình nào được dùng ở đâu.
- Model so sánh chính (DeepSeek-R1-Distill-Qwen-1.5B) đã là một checkpoint **đã distill** từ DeepSeek-R1 (không phải mô hình nền "thuần" như Qwen2.5-Base trong bài 45); việc giữ KL với reference ban đầu cũng dựa trên giả định base này "đã biết suy luận hợp lý". Kết luận "RL mở rộng biên" ở đây có thể phần nào nhờ điểm khởi đầu đã tốt từ một distillation trước đó, chứ không hẳn chỉ do kỹ thuật ProRL.

7. Liên hệ với các bài khác

- **36 (DeepSeekMath, GRPO):** thuật toán nền.
- **41 (DAPO):** nguồn trực tiếp của decoupled clip và dynamic sampling, hai thành phần ProRL mượn nguyên.
- **45 (bài trước trong cluster):** ProRL trích dẫn trực tiếp là tài liệu tham khảo [13], phản biện ở hai điểm: domain quá hẹp (chỉ toán) và train quá ngắn (vài trăm bước). ProRL không bác bỏ số liệu của bài 45 trên toán (vẫn thấy "Diminish" ở đó), chỉ mở rộng phạm vi quan sát sang domain khác để cho thấy kết luận "RL luôn thu hẹp biên" không tổng quát.
- **38 (DeepSeek-R1 distillation, bên folder distillation), 42 (Dr. GRPO):** model khởi điểm của ProRL (DeepSeek-R1-Distill-Qwen-1.5B) chính là sản phẩm của loại distillation ở bài 38; so sánh ProRL (RL thêm vào sau distill) với distillation thuần giúp thấy rõ hai kỹ thuật bổ sung cho nhau theo chiều dọc (distill trước, RL kéo dài sau).
- **47 (1-shot RLVR), 48 (Spurious Rewards):** cùng cluster "RL học được gì" nhưng dùng thí nghiệm đối lập: 47/48 cho thấy RLVR có thể cải thiện mạnh ngay cả với tín hiệu rất ít/rất kém (1 ví dụ, hoặc reward ngẫu nhiên), ủng hộ giả thuyết "chủ yếu khai thác lại cái đã có"; ProRL thì cho thấy nếu train đủ lâu, đủ đa dạng, RL vẫn có thể vượt ra khỏi "cái đã có". Ba bài này nên đọc cùng nhau để thấy debate chưa ngã ngũ, phụ thuộc nhiều vào domain/thời gian train/model.
- **49 (entropy mechanism):** phân tích sâu hơn vai trò của entropy trong RL reasoning, liên quan trực tiếp đến vấn đề entropy collapse mà ProRL phải xử lý bằng DAPO tricks + KL + reset.

8. Ứng dụng thực tế (suy luận)

- **Muốn RL thực sự "dạy điều mới" (không chỉ làm nhọn phân phối cũ), ba điều kiện quan trọng theo bài báo này:** (1) chọn domain mà mô hình nền còn yếu (pass@k cơ sở thấp), không chỉ domain đã bão hòa; (2) train đủ lâu (hàng nghìn bước, không phải hàng trăm); (3) trộn nhiều domain/reward structure khác nhau, không chỉ một loại task.
- **Entropy collapse là vấn đề thực tế khi scale thời gian train RL:** nếu tự train RLVR dài hơn vài trăm bước, nên theo dõi entropy/response length theo thời gian (giống Hình 2 của bài báo) để phát hiện sớm hiện tượng "tắc" và cân nhắc decoupled clip + KL + reset reference, thay vì chỉ tăng temperature rollout (cách này chỉ trì hoãn, không giải quyết).
- **Reset reference policy** là một kỹ thuật đơn giản, rẻ để thử ngay cả ở quy mô nhỏ: nếu validation RLVR dừng cải thiện hoặc dao động, thử reset mô hình tham chiếu về checkpoint hiện tại và tiếp tục train, trước khi kết luận "RL đã bão hòa".

9. Thí nghiệm gợi ý

1. Với một checkpoint RLVR nhỏ tự train (ví dụ GRPO trên một bộ task tổng hợp dễ kiểm chứng), đo pass@128 của mô hình nền trước khi huấn luyện, chọn ra vài task base yếu nhất và vài task base mạnh nhất, so sánh mức cải thiện pass@k sau train, kiểm tra lại tương quan âm mà mục 4.1 nêu.
2. Ablation riêng hai thành phần: (a) chỉ GRPO; (b) GRPO + decoupled clip + dynamic sampling (DAPO); (c) thêm KL + reset. Theo dõi entropy và response length theo step để xem thành phần nào trì hoãn entropy collapse hiệu quả nhất, tái lập tinh thần Hình 2 paper ở quy mô nhỏ.
3. Train trên 2-3 domain khác nhau trộn lẫn (ví dụ toán + một bộ logic puzzle tự tạo) so với train riêng từng domain, đo pass@k trên domain OOD thứ ba, kiểm tra claim "đa dạng domain giúp tổng quát hóa OOD" (mục 4.3, boxnet).
4. Thử reset reference policy định kỳ (mỗi N bước cố định, không cần đợi validation xấu đi) so với không reset, trên cùng một quy trình GRPO + KL, đo xem reset có thực sự "mở khóa" được nhiều bước train thêm hữu ích hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
GRPO advantage	$A(\tau) = (R_\tau - \text{mean}(G))/\text{std}(G)$	chuẩn hóa theo nhóm rollout cùng prompt
Decoupled clip (DAPO)	$\text{clip}(r, 1-\epsilon_{low}, 1+\epsilon_{high})$, $\epsilon_{low} = 0.2, \epsilon_{high} = 0.4$	"clip-higher" chống entropy collapse
Dynamic sampling	loại prompt accuracy nhóm = 0 hoặc 1	tránh gradient = 0
KL-regularized loss	$\mathcal{L}_{GRPO} - \beta D_{KL}(\pi_\theta \Vert \pi_{ref})$	giữ ổn định, chống overfit reward
Reference policy reset	$\pi_{ref} \leftarrow \pi_\theta$ định kỳ + reset optimizer	chống KL "tắc" update khi train dài
3 kiểu diễn biến pass@k	Diminish / Plateau / Sustained	phụ thuộc task và base model đã mạnh sẵn chưa
Creativity Index	độ trùng lặp response với corpus pretrain (DOLMA)	thấp = base đã "thấy" nhiều, RL ít chỗ khai phá
Chặn trên pass@k (Dang et al.)	$\leq 1 - ((1 - \mathbb{E}[\rho_x])^2 + \text{Var}(\rho_x))^{k/2}$	ρ_x = pass@1 từng task

11. Kết luận ngắn

ProRL cho thấy nếu RL được train **đủ lâu** (>2000 bước, ~16k giờ GPU), **đủ ổn định** (KL penalty + reference policy reset + mẹo DAPO chống entropy collapse), và **đủ đa dạng** (5 domain khác nhau), thì nó có thể vượt

qua giới hạn "chỉ làm nhọn lại phân phối cũ" mà bài 45 quan sát, đặc biệt trên các task mà mô hình nền ban đầu hoàn toàn không giải được (pass@k ~ 0 ở mọi k). Hiệu ứng mở rộng biên mạnh nhất ở domain mô hình nền còn yếu và ở các task cần explore nhiều (ví dụ code); ở toán, domain đã "bão hòa" từ pretrain, ProRL vẫn thấy hiện tượng thu hẹp biên giống bài 45. Hai bài không hoàn toàn mâu thuẫn: khác biệt chính nằm ở **domain được chọn** và **thời gian train**, chưa phải một câu trả lời chung cuộc cho câu hỏi "RL có dạy LLM năng lực mới không".

Link gốc: https://arxiv.org/abs/2505.24864

Bài 47

Reinforcement Learning for Reasoning in Large Language Models with One Training Example (2025)

Tác giả: Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, Yelong Shen (University of Washington; Microsoft; USC; UC Santa Cruz; Georgia Tech). **Venue / arXiv:** arXiv:2504.20571 (v3, 24/10/2025). Venue: 39th Conference on Neural Information Processing Systems (NeurIPS 2025). **Nguồn:** https://arxiv.org/abs/2504.20571 **Chủ đề:** Chỉ cần **một** ví dụ toán (1-shot RLVR) để RLVR cải thiện Qwen2.5-Math-1.5B từ 36.0% lên 73.6% trên MATH500, ngang với huấn luyện trên 1.2K ví dụ. Phát hiện "post-saturation generalization" (độ chính xác trên tập test vẫn tăng dù accuracy trên ví dụ train đã bão hòa từ rất sớm), và chỉ ra policy gradient loss (không phải "grokking"/weight decay) là nguồn cải thiện chính. **Vị trí trong lộ trình:** Đọc sau 00 (GRPO, policy gradient, PPO), 36 (DeepSeekMath/GRPO). Cùng cluster "RL học được gì" với 45, 46, 48: bài này củng cố giả thuyết của 45 ("base model đã có sẵn năng lực reasoning, RLVR chỉ cần một cú đẩy nhỏ để lộ ra") bằng một thí nghiệm cực đoan ở chiều dữ liệu (1 ví dụ) thay vì chiều pass@k; nên đọc cùng 48 (Spurious Rewards, cực đoan ở chiều chất lượng reward: reward sai/ngẫu nhiên) để thấy hai cách khác nhau "làm RLVR cực tiểu" vẫn ra hiệu quả bất ngờ.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Phần lớn nghiên cứu RLVR tập trung cải tiến **thuật toán** (PPO, GRPO, DAPO...). Khía cạnh **dữ liệu** ít được khai thác: cần bao nhiêu ví dụ để RLVR hiệu quả? Công trình gần nhất, LIMR, dùng một chỉ số "learning impact measurement" để giảm số ví dụ train 6 lần mà vẫn giữ hiệu năng, nhưng chưa thử giảm cực đoan. Bài này đặt câu hỏi cực đoan hơn: **có thể giảm xuống còn đúng 1 ví dụ không?**

Nhắc lại GRPO (bài 00, 36): với mỗi prompt, sample k rollout, thưởng nhị phân $r \in \{0, 1\}$ theo đúng/sai đáp số, chuẩn hóa advantage theo nhóm. Loss gồm 3 phần: **policy gradient loss** (tăng log-prob mẫu tốt hơn trung bình nhóm, giảm mẫu kém hơn), **KL loss** (giữ gần policy tham chiếu), **entropy loss** (hệ số âm, khuyến khích entropy cao hơn để đa dạng khám phá; verl đưa vào mặc định, không bắt buộc với GRPO).

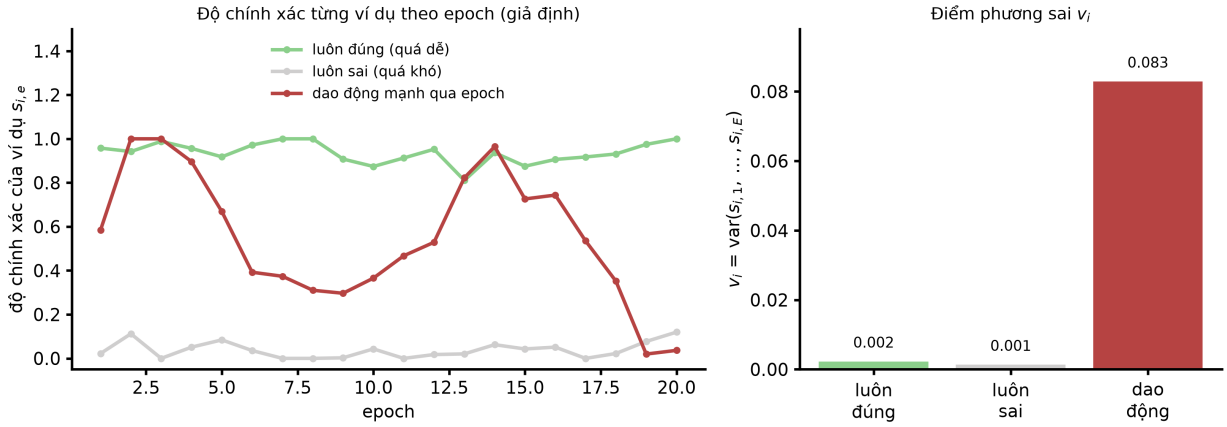
2. Ý tưởng chính

Dùng đúng **một** ví dụ toán (chọn theo một tiêu chí thống kê đơn giản) làm toàn bộ dữ liệu train RLVR cho Qwen2.5-Math-1.5B: độ chính xác MATH500 tăng từ 36.0% lên 73.6% (vượt cả format reward baseline 8.6 điểm %), trung bình 6 benchmark toán tăng từ 17.6% lên 35.7%, **ngang với huấn luyện trên 1209 ví dụ (DeepScaleR subset, DSR-sub)**. Hai ví dụ cho kết quả còn nhỏ nhoi tốt hơn train full dataset. Hiện tượng lặp lại trên nhiều model (Qwen2.5-Math-7B, Llama-3.2-3B-Instruct, DeepSeek-R1-Distill-Qwen-1.5B) và nhiều thuật toán (GRPO, PPO). Đi kèm là một loạt hiện tượng thú vị: độ chính xác test vẫn tăng dài sau khi độ chính xác trên chính ví dụ train đã đạt gần 100% ("post-saturation generalization"); model tự khái quát hóa sang chủ đề khác (train Hình học, cải thiện cả Đại số, Số học...); tần suất "tự kiểm lại" (self-reflection) tăng theo training; và cuối cùng, chính **policy gradient loss** (không phải weight decay như "grokking") mới là động lực chính.

3. Phương pháp chi tiết

3.1 Chọn ví dụ bằng “historical variance score”

Ý tưởng: ví dụ tốt để huấn luyện 1-shot RLVR là ví dụ mà model **dao động** nhiều về độ chính xác qua các epoch train (không phải luôn đúng, không phải luôn sai), vì phương sai của reward liên quan trực tiếp đến chất lượng tín hiệu gradient (advantage chuẩn hóa theo nhóm cần có cả mẫu đúng/sai để không triệt tiêu).



Hình 85: Mô phỏng điểm phương sai lịch sử của ba kiểu ví dụ: luôn đúng, luôn sai, dao động (minh hoạ, tự tính)

Cách đọc hình: mô phỏng giả định, không phải số của paper. Ví dụ luôn đúng hoặc luôn sai cho độ chính xác gần như phẳng nên phương sai gần 0 (advantage nhóm triệt tiêu). Ví dụ dao động mạnh qua epoch có v_i lớn nhất, nên được chọn trước theo tiêu chí này.

Quy trình: train model trên **toàn bộ** dataset E epoch, với mỗi ví dụ $i \in [N]$ ghi lại dãy accuracy theo epoch $L_i = [s_{i,1}, \ldots, s_{i,E}]$, tính:

$$v_i := \mathrm{var}(s_{i,1}, \ldots, s_{i,E})$$

Sắp xếp giảm dần theo v_i, định nghĩa hoán vị π sao cho $v_{\pi(1)} \geq \cdots \geq v_{\pi(N)}$; ví dụ có phương sai lớn nhất gọi là π_1 (viết tắt π_1). Paper nhấn mạnh: **tiêu chí này không nhất thiết tối ưu**; Bảng 3 của bài báo cho thấy nhiều ví dụ có phương sai trung bình/thấp vẫn tạo cải thiện tốt khi dùng làm 1-shot, nên hiện tượng có thể tổng quát hơn bản thân tiêu chí chọn.

3.2 Thiết lập huấn luyện

Dataset gốc: 1209 mẫu lấy ngẫu nhiên từ DeepScaleR-Preview (gọi là DSR-sub). Base model mặc định Qwen2.5-Math-1.5B. GRPO qua verl; hệ số KL $\beta = 0.001$, hệ số entropy $\alpha = -0.001$ (mặc định), temperature rollout 0.6, batch/mini-batch 128, 8 rollout/prompt (8 gradient update/rollout step), prompt dài tối đa 1024 token, response tối đa 3072 token. Vì dataloader của verl yêu cầu dataset $\geq$ batch size, ví dụ chọn được **nhân bản** tới 128 mẫu để tạo dataset hợp lệ (không phải 128 ví dụ khác nhau, vẫn cùng một bài toán).

3.3 Ví dụ π_1: không hề khó

π_1 là bài toán áp suất gió: $P = kAV^3$; cho $V = 8$, $A = 2$, $P = 4 \Rightarrow k = 1/256$; hỏi V khi $A = 4$, $P = 32$. Giải: $32 = \frac{1}{256} \cdot 4 \cdot V^3 \Rightarrow V^3 = 2048 \Rightarrow V = 2048^{1/3} \approx 12.699$. Ground truth trong DSR-sub ghi **12.8** (xấp xỉ, số chính xác hơn là 12.7). Điều thú vị: **mô hình nền, không train gì cả, đã giải đúng mọi bước trừ phép tính căn bậc ba cuối cùng với xác suất cao**; trong 128 lần sinh, 57.8% ra “12.7”/“12.70”, 6.3% ra

"12.8", 6.3% ra "13". Tức là bài toán này gần như đã "nằm sẵn" trong mô hình nền, chỉ thiếu bước chốt cuối chính xác.

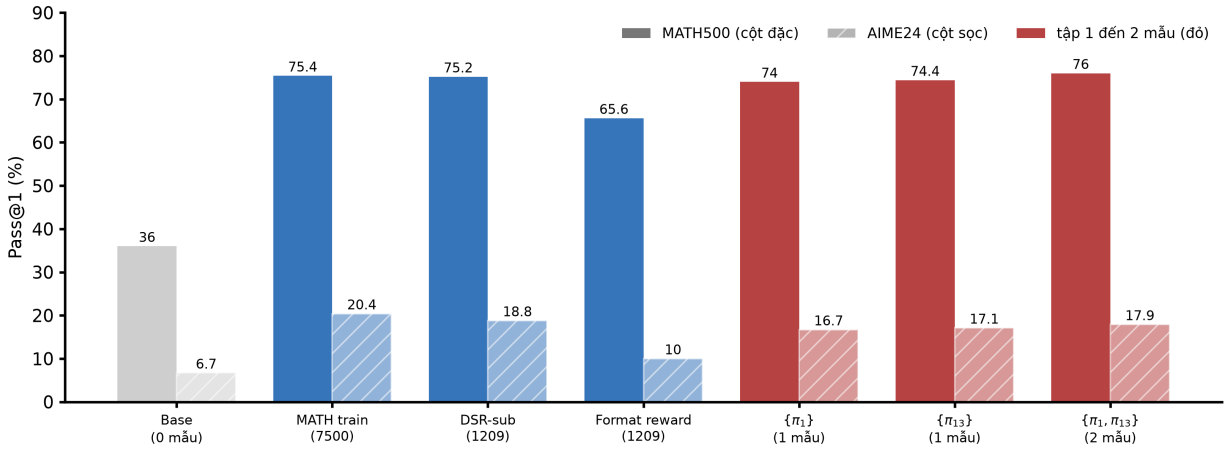
4. Thí nghiệm và kết quả chính

Benchmark: MATH500, AIME24, AMC23, Minerva Math, OlympiadBench, AIME25 (trung bình 6 bộ), thêm ARC-Easy/ARC-Challenge (phi-toán, kiểm tra tổng quát hóa chéo domain).

Bảng chính (trích Bảng 1, model Qwen2.5-Math-1.5B + GRPO):

Dataset	Size	MATH500	AIME24	Avg (6 bộ)
Base	0	36.0	6.7	17.6
MATH train	7500	75.4	20.4	-
DSR-sub	1209	75.2	18.8	35.9
Format reward	1209	65.6	10.0	-
$\{\pi_1\}$	**1**	**74.0**	**16.7**	**35.7**
$\{\pi_{13}\}$	1	74.4	17.1	-
$\{\pi_1, \pi_{13}\}$	2	76.0	17.9	36.6

Cột "Avg" là trung bình trên đủ 6 benchmark toán (MATH500, AIME24, AMC23, Minerva, OlympiadBench, AIME25), không chỉ hai cột MATH500/AIME24 hiển thị ở đây; dấu "-" là ô bài báo không tách riêng trong đoạn trích dùng cho bảng này.



Hình 86: MATH500 và AIME24 của Qwen2.5-Math-1.5B theo tập huấn luyện RLVR (Bảng 1, trích) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: cột đặc là MATH500, cột sọc là AIME24; cột đỏ là các tập chỉ 1 đến 2 mẫu. Một mẫu (π_1 hoặc π_{13}) đã đưa MATH500 từ 36.0 lên 74.0 đến 74.4, gần bằng DSR-sub 1209 mẫu (75.2) và MATH 7500 mẫu (75.4). Ô trống trong ghi chú (dấu "-") không được vẽ.

Tổng quát hóa phi-toán (Bảng ARC, mục 3.2): 1-shot RLVR với toán (π_1 hoặc π_{13}) vẫn cải thiện ARC-Easy/ARC-Challenge, **tốt hơn cả full-set RLVR với MATH (7500 mẫu)** ở một vài trường hợp:

Dataset	Size	ARC-Easy	ARC-Challenge
Base	-	48.0	30.2
MATH (7500)	7500	51.6	32.8

Dataset	Size	ARC-Easy	ARC-Challenge
DSR-sub	1209	42.2	29.9
$\{\pi_1\}$	1	52.0	32.2
$\{\pi_{13}\}$	1	**55.8**	**33.4**
$\{\pi_1, \pi_{13}\}$	2	52.1	32.4

Nhiều ví dụ khác cũng hoạt động (Bảng 3, mục 3.2.3): thử 1-shot với 17 ví dụ phương sai cao, 2 ví dụ phương sai trung, 9 ví dụ phương sai thấp từ các chủ đề khác nhau. Hầu hết cho cải thiện $\geq$ 30% trên MATH500 so với base (33.9%), ngoại trừ π_{1207} (label sai) và π_{1208} (quá khó, model gần như không bao giờ sample đúng). Thú vị: **ví dụ cùng chủ đề với test set không chắc cho kết quả tốt hơn**; ví dụ π_{11} (Number Theory) lại cho điểm Number Theory tương đối thấp so với dùng π_{605} (Precalculus).

1/few-shot trên model/thuật toán khác (Bảng 4, mục 3.3):

Model + Algo	Dataset	Size	Avg (6 bộ)
Qwen2.5-Math-7B + GRPO	base	-	22.4
	DSR-sub	1209	42.8
	$\{\pi_1\}$	1	40.2
	$\{\pi_1,\pi_2,\pi_{13},\pi_{1209}\}$	4	42.5
Llama-3.2-3B-Instruct + GRPO	base	-	17.5
	DSR-sub	1209	19.8
	$\{\pi_1,\pi_{13}\}$	2	21.0
Qwen2.5-Math-1.5B + **PPO**	base	-	17.6
	DSR-sub	1209	35.4
	$\{\pi_1\}$	1	33.8
DeepSeek-R1-Distill-Qwen-1.5B + GRPO	base	-	44.9
	DSR-sub	1209	48.6
	$\{\pi_1,...,\pi_{16}\}$	16	48.3

Nhận xét: hiệu ứng vẫn giữ nguyên qua PPO (không chỉ GRPO), qua mô hình đã distill CoT dài (DeepSeek-R1-Distill), và qua một model family khác (Llama). Khoảng cách giữa few-shot và full-set RLVR lớn hơn với mô hình đã distill sẵn (DeepSeek-R1-Distill), gợi ý hiệu ứng "1 ví dụ là đủ" rõ nhất với model **chưa** qua giai đoạn CoT training dài.

5. Phân tích / ablation

Post-saturation generalization (3.2.2): accuracy trên chính ví dụ train π_1/π_{13} đạt gần 100% trước bước 100 (dễ hiểu, chỉ có 1 ví dụ), nhưng accuracy trên **test set** vẫn tiếp tục tăng: π_1 tăng thêm 3.4 điểm % từ bước 100 đến 1540; π_{13} tăng thêm 9.9 điểm % từ bước 500 đến 2000. Ngược lại, train full-set DSR-sub (1209 mẫu) accuracy train **chưa bão hòa** sau 2000 bước nhưng test performance đã ngừng cải thiện (thậm chí giảm) sau bước 1000. Overfitting trên ví dụ train chỉ xảy ra rất muộn (π_1 sau bước 1400, π_{13} sau bước 1800, tức sau **hàng triệu lượt rollout** vì mỗi bước sample 1024 lần). Ở giai đoạn overfit, output cho **chính ví dụ train** biến thành văn bản đa ngôn ngữ vô nghĩa pha lẫn phép tính đúng, nhưng output cho **test set** vẫn mạch lạc và chính xác, chứng tỏ post-saturation generalization không phải ảo giác do overfit thông thường.

Tần suất self-reflection tăng (3.2.4): đếm số câu trả lời chứa "rethink", "recheck", "recalculate" trên 6 benchmark theo từng checkpoint: số lượng tăng dần, đặc biệt sau ~1250 bước, trùng với lúc response length

và entropy loss tăng mạnh (model thử nhiều cách diễn đạt/giải khác nhau cho cùng ví dụ train). Ngược lại, train full-set DSR-sub có xu hướng **giảm** tần suất self-reflection khi huấn luyện tiến triển.

Ablation loss (Bảng 5, mục 4.1): cố định ví dụ π_1, Qwen2.5-Math-1.5B.

Dòng	Policy grad	Weight decay	KL loss	Entropy loss	Label	Converge train	MATH500	AIME24
1					12.8	KHÔNG	39.8	7.5
2	+				12.8	CÓ	71.8	15.4
3	+	+			12.8	CÓ	71.4	16.3
4	+	+	+		12.8	CÓ	70.8	15.0
5 (full)	+	+	+	+	12.8	CÓ	**74.8**	**17.5**
6	+	+	+	+(-0.003)	12.8	CÓ	73.6	15.4
7	+			+	12.8	CÓ	75.6	17.1
8		+	+		12.8	KHÔNG	39.0	10.0
9		+	+	+	12.8	KHÔNG	65.4	7.1
10				+	12.8	KHÔNG	63.4	8.8

Đọc bảng: **chỉ riêng policy gradient loss** (dòng 2) đã cho kết quả gần với full loss (dòng 5); thêm weight decay/KL (dòng 3, 4) hầu như không đổi; thêm entropy loss đúng hệ số (dòng 5 so với dòng 4) cho thêm +4.0 điểm % MATH500 và +2.5 AIME24, nhưng hệ số quá lớn (dòng 6, -0.003 thay vì -0.001) làm train **kém ổn định hơn** chứ không tốt hơn. Vì "grokking" bị ảnh hưởng mạnh bởi weight decay còn ở đây weight decay gần như vô ảnh hưởng (dòng 8: chỉ weight decay + KL, không converge, kết quả tệ gần bằng base), tác giả kết luận **1-shot RLVR không phải grokking**, chủ yếu do policy gradient loss.

Chỉ dùng entropy loss, không reward đúng/sai (dòng 9, 10; mục 4.2): vẫn cải thiện so với base (base MATH500 36.0 → entropy-loss-only 63.4/65.4 tùy cấu hình) nhưng **kém hơn** format-reward baseline (65.0-65.8). Hiệu ứng này cũng xuất hiện trên Qwen2.5-Math-7B và Llama-3.2-3B-Instruct dù yếu hơn. Nghĩa là: ngay cả khi loại bỏ hoàn toàn tín hiệu đúng/sai, chỉ khuyến khích entropy cao (đa dạng output) đã mang lại một phần cải thiện, độc lập (một phần) với việc "sửa format".

Độ chính xác của label (dòng 5, 11-13): so sánh label đúng ("12.7", dòng 11) với label gốc hơi lệch ("12.8", dòng 5): kết quả gần như nhau (74.8 vs 73.4 MATH500). Label sai nhưng model **có thể overfit được** ("4", dòng 12): kết quả **tệ hơn hẳn** (57.0 MATH500). Label sai và **không thể guess/overfit** ("9292725", dòng 13): kết quả 64.4, tốt hơn trường hợp label "4" nhưng vẫn kém label đúng/gần đúng, và giống với trường hợp chỉ dùng entropy loss (dòng 10). Thứ tự xếp hạng: đúng ≈ gần đúng > sai-không-đoán-được > sai-mà-đoán-được. Tức label sai lệch nhẹ không hại nhiều, nhưng label sai lệch nặng mà vẫn "guessable" (model dễ hội tụ về nó) lại hại hơn cả một label hoàn toàn vô nghĩa.

Khuyến khích explore giúp post-saturation generalization tốt hơn (Hình 5): không entropy loss, cải thiện trung bình 6 bộ chững lại sau bước 150 (đúng lúc accuracy train bão hòa); thêm entropy loss: +2.3 điểm %; tăng thêm temperature rollout lên 1.0: +0.8 điểm % nữa.

6. Giới hạn

Tác giả nêu (Phụ lục D.1): chưa thử mô hình lớn hơn (ví dụ Qwen2.5-32B) do giới hạn compute, dù các mô hình 1.5B/7B đã cho kết quả đáng chú ý trên benchmark khó như OlympiadBench; chỉ tập trung domain toán, chưa thử domain verifiable khác (code); mục tiêu chính là nêu một **quan sát mới**, không phải đề xuất phương pháp tốt hơn để tiết kiệm compute (1-shot RLVR **không** tiết kiệm, có thể còn cần nhiều compute hơn vì phải sample cùng một ví dụ hàng triệu lần); π_1 được chọn dựa trên historical variance của riêng Qwen2.5-Math-

1.5B, không chắc là lựa chọn tốt nhất cho model khác; few-shot (ví dụ 16 ví dụ) thường ổn định hơn 1-shot trên các mô hình khác.

Nhận xét:

- "Historical variance score" cần huấn luyện trên **toàn bộ dataset** trước (500 bước) để tính được, nên về bản chất không phải một cách "rẻ" để tìm ví dụ tốt ngay từ đầu; đây là một hạn chế thực dụng không được nhấn mạnh trong phần giới hạn.
- Nhiều kết luận (post-saturation generalization, self-reflection tăng) chỉ minh họa rõ trên π_1/π_{13} với model 1.5B; mức độ tổng quát trên mô hình lớn hơn ít được kiểm chứng sâu (Bảng 4 chỉ cho số cuối, không có đường cong chi tiết cho mọi model).
- Bảng 3 của bài báo (chi tiết theo chủ đề MATH500) báo cáo kết quả ở checkpoint "tốt nhất" cho từng dataset (cột "Step" khác nhau cho mỗi dòng), nên các số trong bảng đó không phải một lần đánh giá duy nhất tại cùng một mốc step; cần đọc kèm chú thích "best checkpoint" khi so sánh giữa các dòng.

7. Liên hệ với các bài khác

- **00, 36 (GRPO):** thuật toán nền và nơi tra lại định nghĩa policy gradient/KL/entropy loss đầy đủ.
- **45 (Does RL really incentivize reasoning):** cùng giả thuyết nền "base model đã có sẵn năng lực reasoning"; bài 45 chứng minh bằng pass@k ở k lớn (chiều đo lường), bài này chứng minh bằng việc giảm cực đoan số lượng dữ liệu train (chiều dữ liệu) mà vẫn đạt hiệu quả tương đương full-set.
- **46 (ProRL):** khác góc nhìn: ProRL cho rằng cần train rất lâu + đa domain để RL vượt base; bài 1-shot này cho thấy ngược lại, ở domain toán, **rất ít dữ liệu** cũng đủ để "kích hoạt" cải thiện tương đương full-set, nhưng không bàn tới việc có vượt biên mô hình nền hay không (không đo pass@k ở k lớn).
- **48 (Spurious Rewards):** liên hệ trực tiếp, cùng nhóm tác giả tham chiếu lẫn nhau; bài 48 còn chạy lại thí nghiệm 1-shot RLVR của bài này trên các mô hình ngoài Qwen và thấy tín hiệu "1 ví dụ" **không tổng quát tốt** sang model family khác (Llama, OLMo2), giống cách spurious reward cũng chỉ hiệu quả trên Qwen. Hai bài cùng đưa ra lời cảnh báo: kết luận rút ra chủ yếu từ Qwen2.5-Math cần kiểm chứng lại trên model khác trước khi tổng quát hóa.
- **07 (InstructGPT), 08 (LoRA):** liên hệ xa hơn về ý tưởng "rất ít dữ liệu/tham số có thể đủ" (LoRA ít tham số, 1-shot RLVR ít dữ liệu), tuy cơ chế khác nhau hoàn toàn.

8. Ứng dụng thực tế (suy luận)

- **Khi dữ liệu reward-verifiable khan hiếm** (ví dụ chỉ có vài chục cặp câu hỏi/đáp án đã kiểm chứng cho một tác vụ hẹp), kết quả này gợi ý RLVR vẫn có thể đáng thử với rất ít ví dụ, miễn ví dụ đó không quá dễ (model luôn đúng ngay từ đầu, advantage sẽ gần 0) và không quá khó (model không bao giờ đúng để lấy được tín hiệu).
- **Entropy loss với hệ số nhỏ, dương tính hợp lý (không quá lớn) là một "nút chỉnh" rẻ để cải thiện exploration**, đặc biệt khi dữ liệu train ít: mục 4.1 cho thấy chênh lệch 2-4 điểm % chỉ từ việc có/không có entropy loss, và hệ số quá lớn gây bất ổn, nên cần dò nhỏ ($\sim 10^{-3}$) chứ không mặc định bật/tắt.
- **Cẩn trọng với nhãn (label) gần đúng nhưng sai:** mục 5 cho thấy nhãn sai lệch nhẹ (ví dụ do làm tròn số) gần như không hại, nhưng nhãn sai lệch nhiều mà "dễ đoán/dễ hội tụ" lại gây hại hơn một nhãn hoàn toàn ngẫu nhiên, điều cần nhớ khi tự gán nhãn bằng majority voting hoặc heuristic cho RLVR không giám sát.
- **Bài học tổng quát cho debug RLVR:** nếu accuracy trên tập train đã bão hòa sớm mà performance validation vẫn tăng, đừng vội dừng train hoặc coi là dấu hiệu overfit xấu; mục 5 cho thấy "post-saturation generalization" là hiện tượng thật, cần theo dõi riêng validation set thay vì chỉ nhìn train accuracy.

9. Thí nghiệm gợi ý

1. Lặp lại quy trình chọn "historical variance score" trên một mô hình nhỏ tự có (train 500 bước trên một bộ task tổng hợp dễ kiểm tra), rồi thử 1-shot RLVR với ví dụ phương sai cao nhất vs. thấp nhất, kiểm tra Bảng 3 có tái lập (ví dụ phương sai thấp vẫn có thể cải thiện tốt).
2. Ablation lại Bảng 5 ở quy mô nhỏ: tắt/mở từng thành phần loss (policy gradient, KL, entropy) trên một ví dụ 1-shot tự chọn, đo accuracy test theo step để xem đường cong "post-saturation generalization" có lặp lại.
3. Thử 1-shot RLVR với 3 loại label: đúng, sai nhẹ (gần đúng), sai nặng nhưng dễ đoán, để kiểm tra lại thứ tự xếp hạng ở mục 5 trên một bài toán/model khác.
4. Đếm tần suất từ khóa tự kiểm lại ("rethink", "recheck"...) theo checkpoint cho một lần train 1-shot của riêng mình, xem có tăng theo bước train tương tự Hình 4 của bài báo không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Historical variance score	$v_i = \mathrm{var}(s_{i,1}, \ldots, s_{i,E})$	chọn ví dụ dao động accuracy nhiều qua epoch
1-shot RLVR	train RLVR chỉ với 1 ví dụ (nhân bản đủ batch size)	đạt hiệu năng ngang 1.2K ví dụ (DSR-sub)
Post-saturation generalization	train accuracy bão hòa sớm, test accuracy vẫn tăng dài	khác grokking (ít phụ thuộc weight decay)
Policy gradient loss	thành phần chính tạo cải thiện	loại weight decay/KL gần như không đổi kết quả
Entropy loss	hệ số âm nhỏ ($\alpha = -0.001$), khuyến khích đa dạng	tăng thêm 2-4 điểm %, quá lớn thì bất ổn
Xếp hạng label	đúng ≈ gần đúng > sai-khó-đoán > sai-dễ-đoán	sai lệch nhẹ ít hại, sai lệch mà dễ hội tụ hại nhiều
Self-reflection words	"rethink", "recheck", "recalculate"	tần suất tăng theo bước train 1-shot

11. Kết luận ngắn

Chỉ một ví dụ toán (được chọn bằng phương sai accuracy lịch sử, hoặc thậm chí nhiều ví dụ khác không cần chọn kỹ) đủ để RLVR cải thiện Qwen2.5-Math-1.5B tương đương huấn luyện trên hàng nghìn ví dụ, kèm các hiện tượng lạ (post-saturation generalization, tổng quát hóa chéo chủ đề, tăng self-reflection) mà nguồn gốc chính là policy gradient loss (không phải "grokking"). Kết quả này củng cố thêm cho nhóm giả thuyết "RLVR chủ yếu kích hoạt lại năng lực sẵn có trong base model" (cùng hướng với bài 45, 48), nhưng cũng cho thấy hiệu ứng khá nhạy với model family (giảm nhiều ở Llama, tăng ít hơn rõ rệt so với Qwen) nên cần kiểm chứng rộng hơn trước khi coi là quy luật phổ quát của RLVR.

Link gốc: https://arxiv.org/abs/2504.20571

Bài 48

Spurious Rewards: Rethinking Training Signals in RLVR (2025)

Tác giả: Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, Luke Zettlemoyer (University of Washington; Allen Institute for AI; UC Berkeley). **Venue / arXiv:** arXiv:2506.10947 (v2, 25/02/2026). Bài báo không nêu venue hội nghị cụ thể (ghi "Preprint. February 26, 2026."). **Nguồn:** https://arxiv.org/abs/2506.10947 **Chủ đề:** RLVR với GRPO cải thiện mạnh Qwen2.5-Math-7B trên MATH-500 ngay cả khi dùng reward **ngẫu nhiên** hoặc **cố ý sai** (random/incorrect reward), gần bằng ground-truth reward; hiệu ứng này phụ thuộc mạnh vào model family (không xảy ra với Llama3, OLMo2). Giải thích bằng "clipping bias" trong GRPO và hành vi "code reasoning" đã có sẵn trong Qwen2.5-Math. **Vị trí trong lộ trình:** Đọc sau 00, 36 (GRPO). Khép lại cluster "RL học được gì" (45-48): cùng hướng với 45 và 47 (RLVR chủ yếu khai thác lại năng lực đã có trong mô hình nền), nhưng đưa ra bằng chứng cực đoan nhất (reward hoàn toàn không mang thông tin vẫn hiệu quả) kèm giải thích cơ chế toán học cụ thể (clipping bias) mà hai bài trước không có. Nên đọc sau cả 45, 46, 47 để thấy bức tranh đầy đủ trước khi sang 49-52 (cơ chế entropy, token, subnetwork).
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế (suy luận)" là suy luận.

1. Bối cảnh

Qwen2.5-Math là "model mặc định" trong rất nhiều nghiên cứu RLVR mã nguồn mở (test-time training, 1-shot RLVR ở bài 47, và nhiều paper khác được chính bài báo này liệt tên). Nếu một phương pháp RLVR mới cho kết quả tốt trên Qwen2.5-Math, cộng đồng thường coi đó là bằng chứng phương pháp "hoạt động". Bài này thử một thí nghiệm kiểm định: **thay reward đúng/sai bằng reward không mang thông tin gì** (ngẫu nhiên, hoặc cố tình sai), xem RLVR còn cải thiện được không. Nếu có, nghĩa là việc một phương pháp "hoạt động" trên Qwen2.5-Math chưa chắc chứng minh được phương pháp đó thực sự dạy reasoning đúng, mà có thể chỉ đang khai thác một đặc điểm riêng của mô hình đó.

Nhắc GRPO (bài 00, 36): advantage $\hat{A}(x,y) = r(x,y) - \mathbb{E}_{y'\sim\pi_{old}}[r(x,y')]$ (trừ trung bình phần thưởng trong nhóm rollout cùng prompt); loss có số hạng **clip** kiểu PPO để hạn chế bước update quá lớn:

$$J(\theta) = \mathbb{E}_{x,y\sim\pi_{old}} \sum_{t=1}^{|y|} \min\Big(\rho_t(y;\theta)\hat{A}(x,y),\ \text{clip}(\rho_t(y;\theta), 1-\epsilon_c, 1+\epsilon_c)\hat{A}(x,y)\Big), \quad \rho_t = \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{old}(y_t|x,y_{<t})}$$

2. Ý tưởng chính

Thiết kế một dải reward "ngày càng giả": từ ground truth (chuẩn), majority vote (nhãn giả bằng bỏ phiếu đa số trên 64 lần sample), format reward (chỉ cần có `\boxed{}`, không cần đúng), incorrect reward (cố tình thưởng **đáp án sai**, nhưng là đáp án mà model dễ sample ra), đến random reward (thưởng 1 với xác suất cố định $\gamma = 0.5$, hoàn toàn không liên quan tới câu trả lời). Kết quả bất ngờ: trên **Qwen2.5-Math-7B**, ngay cả random reward cũng cải thiện MATH-500 **+21.4 điểm %** (so với +29.1 của ground truth); nhưng trên **Llama3.1-8B-Instruct, OLMo2-7B**, các reward giả này gần như không cải thiện, có khi còn làm giảm điểm.

Giải thích gồm hai phần: (1) **clipping bias** trong GRPO khiến cơ chế clip tự khuếch đại token đã có xác suất cao dưới policy hiện tại, dù reward hoàn toàn ngẫu nhiên; (2) Qwen2.5-Math có sẵn một hành vi hữu ích từ pretraining ("**code reasoning**": viết code Python để hỗ trợ suy luận toán, dù không có công cụ thực thi code), reward giả nào cũng tình cờ khuếch đại hành vi này lên, kéo theo accuracy tăng.

3. Phương pháp chi tiết

3.1 Năm loại reward (định nghĩa chính xác theo paper)

1. **Ground Truth:** nhãn đúng, chuẩn upper bound.
2. **Majority Vote:** lấy mẫu 64 rollout/prompt trước RL, gán nhãn giả = đáp án xuất hiện nhiều nhất (có thể sai), rồi thưởng theo nhãn giả này suốt training.
3. **Format Reward:** thưởng 1 nếu response chứa `\boxed{...}` không rỗng, bất kể đúng/sai.
4. **Random Reward:** $r = 1$ với xác suất γ (mặc định $\gamma = 0.5$ trong thí nghiệm chính, không phụ thuộc gì vào x, y); $\gamma = 0$ cho gradient kỳ vọng bằng 0 (kiểm chứng được bằng giải tích).
5. **(Majority-voted) Incorrect Reward:** gán nhãn bằng majority vote, **chọn những prompt mà nhãn đa số này SAI** so với ground truth, rồi chỉ thưởng khi response khớp đúng cái nhãn sai đó. Mục đích: tách biệt "nhãn có khả năng đúng cao" (majority vote thường đúng) khỏi "nhãn là output xác suất cao của model" (nhãn sai nhưng vẫn là thứ model thường sample ra).

3.2 Giải thích clipping bias cho random reward (mục 4, suy ra ở Phụ lục B)

Khi tắt hẳn số hạng clip và sample độc lập với reward ngẫu nhiên, kỳ vọng gradient update bằng 0 (không học được gì có ý nghĩa) được kiểm chứng bằng 3 biến thể "no-clip": tắt clip trong code; tăng mini-batch size bằng đúng rollout size (để $\pi_\theta = \pi_{old}$ mỗi lần update, $\rho_t \equiv 1$); giảm rollout size để chỉ có 1 gradient update/rollout (cùng lý do $\rho_t \equiv 1$). Cả ba cách đều cho kết quả **không ổn định/không cải thiện**. Khi **bật lại clip**, random reward cải thiện ổn định trở lại.

Phân tích gradient (Phụ lục B, công thức (2) trong bài báo, bỏ các hằng số không âm):

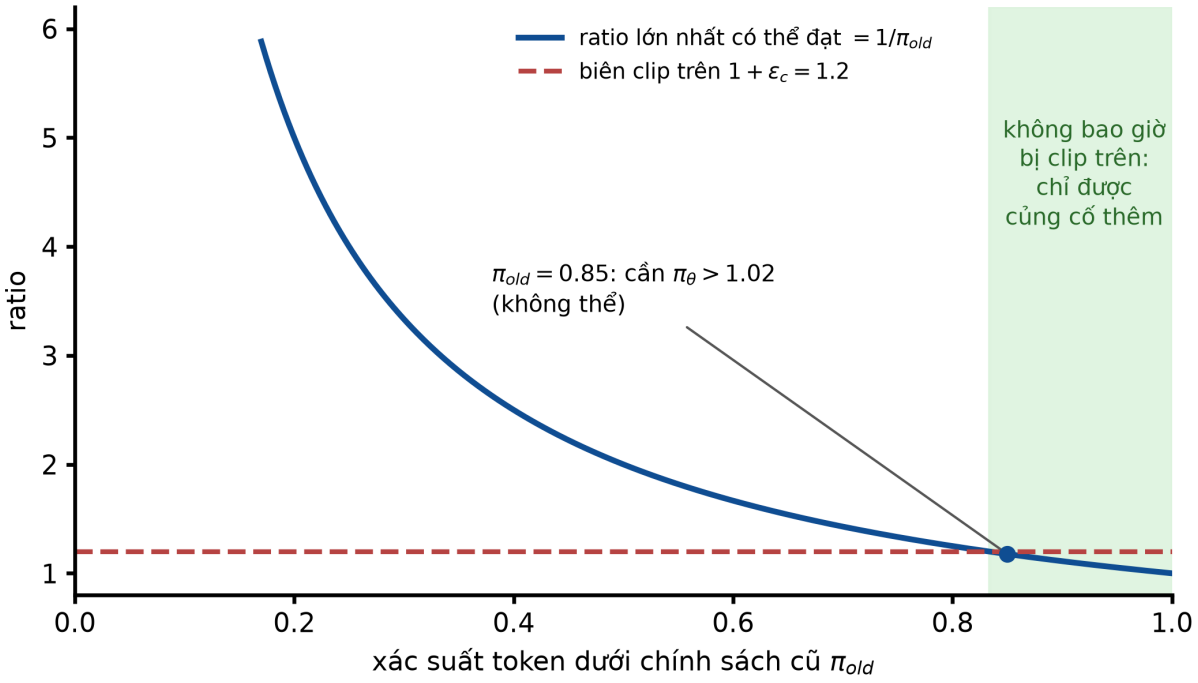
$$\mathbb{E}[\nabla_\theta J(\theta)] \;\propto\; \mathbb{E}_{x,y}\begin{cases} \nabla_\theta \log \pi_{\theta,x}(y_t), & R_\theta < 1-\epsilon_c \\ 0, & |R_\theta - 1| \le \epsilon_c \\ -\nabla_\theta \log \pi_{\theta,x}(y_t), & R_\theta > 1+\epsilon_c \end{cases}, \qquad R_\theta = \frac{\pi_{\theta,x}(y_t)}{\pi_{old,x}(y_t)}$$

Ví dụ minh họa của bài báo (diễn giải lại): cho $\epsilon_c = 0.2$. Nếu $\pi_{old,x}(y_t) = 0.85$ (token xác suất cao), để bị clip trên (nhánh thứ 3, trừ gradient) cần $\pi_{\theta,x}(y_t) > 0.85 \times 1.2 = 1.02$, điều **không thể xảy ra** vì xác suất ≤ 1. Vậy token này chỉ có thể rơi vào nhánh 1 hoặc 2, **không bao giờ bị kéo xuống**, chỉ có thể được **củng cố thêm**. Ngược lại nếu $\pi_{old,x}(y_t) = 0.02$ (token hiếm), chỉ cần $\pi_{\theta,x}(y_t) > 0.024$ là đã bị clip trên (trừ gradient), một ngưỡng rất dễ vượt qua bằng một bước update nhỏ. Kết quả bất đối xứng: **clipping luôn có xu hướng khuếch đại token đã có xác suất cao và dìm token hiếm**, bất kể reward có mang thông tin gì hay không. Đây chính là cách random reward "tạo ra" một tín hiệu học có cấu trúc dù về lý thuyết reward độc lập hoàn toàn với (x, y).

Cách đọc hình: đường xanh là ratio tối đa mà một token có thể đạt (vì xác suất không vượt 1). Với $\epsilon_c = 0.2$, token có $\pi_{old} > 0.833$ (vùng nền xanh lá) không bao giờ chạm biên clip trên, nên khi advantage dương nó chỉ được củng cố thêm. Điểm $\pi_{old} = 0.85$ là ví dụ trong ghi chú, cần $\pi_\theta > 1.02$, điều không thể xảy ra.

4. Thí nghiệm và kết quả chính

Setup (Phụ lục A): GRPO trên verl/OpenRLHF-style quy trình, 8 GPU, lr 5×10^{-7}, mini-batch 128, rollout batch 64, 16 rollout/prompt, sample temperature $\tau = 1$ lúc train, không KL, không entropy loss. Train trên dữ



Hình 87: Ratio lớn nhất có thể đạt $1/\pi_{old}$ so với biên clip trên $1 + \epsilon_c$ (minh hoạ, tự tính)

liệu DeepScaleR. Đánh giá: pass@1 trên MATH-500 (temperature 0), avg@8 trên AMC (temperature 0.6); thêm AIME24/25 ở phụ lục.

Kết quả chính, Qwen2.5-Math-7B, sau 300 bước (Hình 1-2 trong bài báo, trích số liệu nêu rõ bằng chữ trong văn bản):

Reward	Δ MATH-500 so với base	Δ AMC so với base
Ground Truth	+29.1	xấp xỉ 27-29 (nêu chung với Majority Vote)
Majority Vote	xấp xỉ 27-29 (nêu chung với Ground Truth)	xấp xỉ 27-29
Format	paper không nêu số cụ thể cho MATH-500 (chỉ thấy trên biểu đồ Hình 2)	+13.8
Incorrect (majority-voted sai)	+24.1	+24.1
Random ($\gamma = 0.5$)	+21.4	+21.4

Nguyên văn đoạn so sánh AMC: “training on format, incorrect, or random rewards yields a gain of 13.8%, 24.1%, or 21.4% respectively... approaching the ∼27%-29% improvement gained from training on majority voted and ground truth labels.” Paper nêu rõ bằng số cho MATH-500 chỉ với ground truth (+29.1), incorrect (+24.1) và random (+21.4); số của majority vote và format trên MATH-500 chỉ xuất hiện trên biểu đồ Hình 2 (không có trong văn bản), nên các ô tương ứng được để trống. Qwen2.5-Math-1.5B cũng cải thiện tương tự, nhưng **random reward hội tụ chậm hơn nhiều** (phải sau 100 bước mới thấy cải thiện) và cải thiện AMC yếu hơn (chỉ +4.9%, theo văn bản).

Trên AIME24/25 (Phụ lục D, nêu trong thân bài): format reward +10.3% (gần ground truth +15.3%); incorrect/random đạt +10.2%/+10.2%. Trên AIME25 (đề sau knowledge cutoff của mọi mô hình được thử), ground truth có lợi thế rõ rệt hơn, các reward khác chỉ đạt từ -0.4% đến $+4.5\%$: gợi ý hiệu ứng “reward giả

vẫn hiệu quả" yếu đi khi đề bài thực sự mới, không có trong phân phối pretrain.

Không tổng quát sang model khác (Hình 3, mục 3): thử thêm Qwen2.5-7B, Qwen2.5-1.5B, Llama3.1-8B(-Instruct), Llama3.2-3B(-Instruct), OLMo2-7B, OLMo2-7B-SFT, cùng quy trình/reward. Hai quan sát:

- **Model trong cùng họ hành xử giống nhau:** mọi reward (trừ random) cải thiện Qwen2.5 (bản thường, không chuyên toán); OLMo2 gần như đứng yên (flat) với reward giả, chỉ tăng rõ với ground truth.
- **Model nhỏ hưởng lợi ít hơn từ reward giả** (ví dụ random): tác giả suy đoán mô hình lớn hơn giữ lại nhiều "prior" từ pretrain hơn, nên có nhiều hành vi để clipping bias khuếch đại.

Cảnh báo thực hành (trích nguyên văn ý paper): các phương pháp RLVR "không giám sát" công bố gần đây, ví dụ **test-time training** (TTRL, Zuo et al.) và **1-shot RL** (bài 47), khi chạy lại trên model ngoài Qwen đều cho kết quả tương tự: hiệu quả mạnh trên Qwen2.5-Math/Qwen2.5, nhưng yếu/không có trên model khác.

5. Phân tích / ablation

Code reasoning: hành vi có sẵn, khuếch đại dễ dàng (mục 5.1, Bảng 1): Qwen2.5-Math-7B viết code Python trong 65.0% response trên MATH-500 **trước khi RL**, dù model không có công cụ thực thi code thật (tự "chạy" code bằng cách sinh tiếp văn bản mô phỏng kết quả). Response có code đạt accuracy 60.9%, cao hơn hẳn response chỉ dùng ngôn ngữ tự nhiên (35.0%).

Model	Code Freq. (trước RL)	Acc. khi có code	Acc. khi chỉ ngôn ngữ
Qwen2.5-Math-7B	65.0%	60.9%	35.0%
Qwen2.5-Math-1.5B	53.6%	52.6%	17.2%
Qwen2.5-7B	92.2%	39.9%	61.5%
OLMo2-7B-SFT	98.0%	21.0%	40.0%

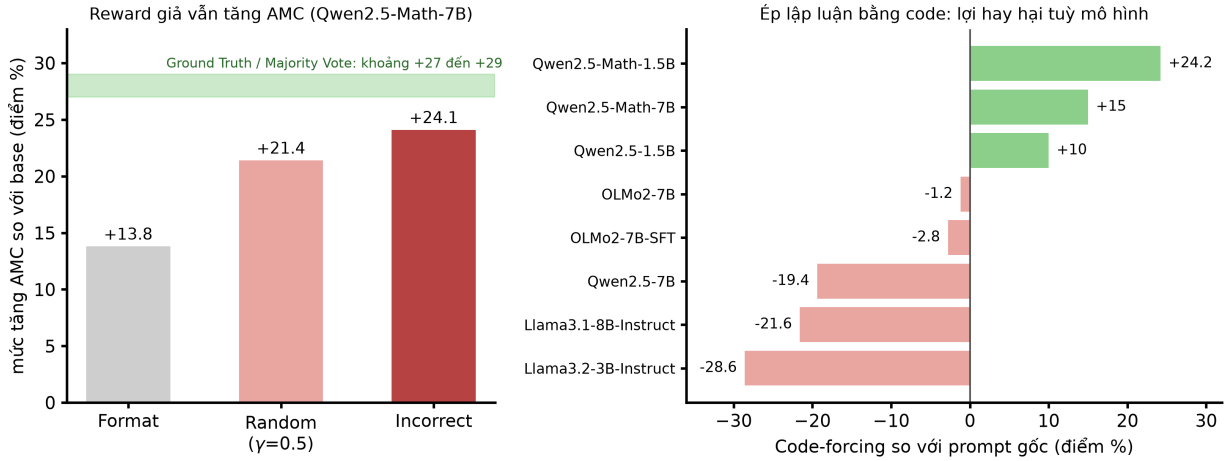
Phân loại: **Qwen2.5-Math** = "code reasoning tốt" (code giúp đúng hơn); **Qwen2.5-7B, OLMo2-7B-SFT** = "Bad-Code" (sinh code nhiều nhưng code lại làm giảm accuracy); các mô hình còn lại (Llama, OLMo2-7B thường, Qwen2.5-1.5B) **không bao giờ** sinh code. Code reasoning hiệu quả chỉ là đặc điểm riêng của Qwen2.5-Math.

Tần suất code reasoning tăng cùng accuracy khi huấn luyện bằng reward yếu/giả (Hình 6): với format/incorrect/random reward, tần suất code reasoning của Qwen2.5-Math-7B tăng nhanh lên ~90% chỉ sau 15 bước, tới 95.6% với random reward, đi kèm accuracy tăng. Với **ground truth reward**, code frequency cũng tăng ban đầu rồi **giảm dần** khi reasoning ngôn ngữ tự nhiên cải thiện, gợi ý ground truth dạy được cách học vượt ra ngoài việc chỉ dựa vào code.

Can thiệp trực tiếp vào code reasoning (mục 5.3, Bảng 2): ép câu đầu tiên của response là "Let's solve this using Python." (code-forcing prompt):

Model	Prompt gốc	Code-forcing	Chênh lệch
Qwen2.5-Math-1.5B	36.2%	60.4%	**+24.2%**
Qwen2.5-Math-7B	49.4%	64.4%	**+15.0%**
Qwen2.5-1.5B	3.0%	13.0%	+10.0%
Qwen2.5-7B	41.6%	22.2%	-19.4%
Llama3.2-3B-Instruct	36.8%	8.2%	-28.6%
Llama3.1-8B-Instruct	36.8%	15.2%	-21.6%
OLMo2-7B	9.0%	7.8%	-1.2%
OLMo2-7B-SFT	21.4%	18.6%	-2.8%

Ép code reasoning **chỉ giúp** nhóm Qwen2.5-Math (và một phần Qwen2.5 thường); **làm hại** các mô hình không có "prior" code reasoning tốt. Dùng RLVR với "Python reward" (thưởng 1 nếu response chứa chuỗi "python") cho kết quả tương tự: Qwen2.5-Math-7B đạt >99% code frequency chỉ sau 20 bước và cải thiện rõ, các mô hình khác cải thiện rất hạn chế.



Hình 88: Mức tăng AMC theo loại reward giả, và hiệu ứng ép lập luận bằng code lên tám mô hình (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: bên trái, format, random và incorrect reward vẫn tăng AMC (+13.8, +21.4, +24.1), gần dải +27 đến +29 của Ground Truth và Majority Vote theo văn bản ghi chú. Bên phải là Bảng 2: ép câu mở đầu bằng code giúp nhóm Qwen2.5-Math và Qwen2.5-1.5B (xanh lá) nhưng hại các mô hình còn lại (đỏ), cho thấy hiệu ứng phụ thuộc mô hình.

6. Giới hạn

Tác giả nêu: nhấn mạnh rõ (lặp lại nhiều lần trong bài) rằng spurious reward **không phải phương pháp nên dùng** để phát triển năng lực model thật, chỉ dùng cho mục đích phân tích; code reasoning chỉ là **một trường hợp đại diện** để minh họa giả thuyết "hành vi pretrain sẵn có quyết định hiệu quả RLVR", không phải lời giải thích đầy đủ (Phụ lục G còn nêu thêm "generation without repetition" là một hành vi khác cũng tương quan với hiệu năng); kết quả ban đầu của Qwen2.5-Math-7B còn nhạy với cách viết prompt đánh giá (Phụ lục I), nên một phần hiệu ứng quan sát có thể lẫn với yếu tố này.

Nhận xét:

- Thí nghiệm chính dùng $\gamma = 0.5$ cho random reward; bài báo có kiểm tra thêm $\gamma \in \{0.001, 0.3, 0.7\}$ ở phụ lục và báo "cải thiện tương tự với tốc độ hội tụ khác nhau", nhưng không phải mọi γ được phân tích sâu bằng $\gamma = 0.5$ trong thân bài.
- Chỉ chạy tới 300 bước; không rõ nếu train lâu hơn (theo tinh thần bài 46, ProRL) thì model dùng random reward có "sụp" (model collapse, lặp từ, mất khả năng ngôn ngữ) hay tiếp tục cải thiện, vì không có reward thật để neo giữ chất lượng output về lâu dài.
- Thí nghiệm chỉ trên toán (MATH-500, AMC, AIME); chưa rõ hiện tượng clipping bias + code reasoning có lặp lại trên domain verifiable khác (code generation, hoặc OCR/structured extraction) hay không.

7. Liên hệ với các bài khác

- **00, 36 (GRPO):** công thức GRPO/clip được phân tích sâu thêm ở đây (clipping bias), bổ sung một góc nhìn toán học cho cơ chế mà bài 00/36 chỉ giới thiệu như một kỹ thuật ổn định (giống PPO).

- **45 (Does RL really incentivize reasoning):** cùng kết luận tổng thể ("RLVR khai thác lại cái đã có, không dạy mới"), nhưng bài 45 dùng pass@k để đo ở mức model, còn bài này đưa ra cơ chế **vì sao** điều đó xảy ra (clipping bias khuếch đại hành vi có sẵn bất kể reward).
- **46 (ProRL):** bài 46 cố tình train rất lâu, đa domain, có KL + reset để **tránh** chính loại hiện tượng này (khuếch đại hành vi cũ một cách "giả tạo", không kiểm soát); phát hiện clipping bias ở bài này cho thấy việc giữ KL penalty của ProRL cũng có thể đóng vai trò hạn chế hiệu ứng "khuếch đại vô căn cứ" mà clipping riêng lẻ có thể gây ra.
- **47 (1-shot RLVR):** cùng nhóm tác giả tham chiếu lẫn nhau, bài này chạy lại thí nghiệm 1-shot RLVR trên model ngoài Qwen và xác nhận hiệu ứng không tổng quát tốt, giống cách spurious reward cũng chỉ hiệu quả trên Qwen; hai bài cùng đi tới lời khuyên "kiểm chứng trên nhiều model family trước khi tin một phương pháp RLVR mới".
- **41 (DAPO), 43 (GSPO):** cả hai cũng chỉnh sửa/thay thế cơ chế clip của PPO/GRPO (DAPO: decoupled clip; GSPO: đổi đơn vị importance ratio từ token sang sequence); phát hiện clipping bias ở bài này gợi ý rằng việc chỉnh sửa clip không chỉ ảnh hưởng tới ổn định train mà còn tới việc liệu RL có đang học từ reward hay đang khuếch đại ngẫu nhiên hành vi cũ.

8. Ứng dụng thực tế (suy luận)

- **Luôn dùng dummy baseline (random reward, format reward) khi đánh giá một phương pháp reward/RLVR mới**, đặc biệt nếu mô hình nền là Qwen2.5-Math: nếu dummy baseline đã đạt gần hiệu năng của phương pháp đề xuất, phương pháp đó có thể chưa chứng minh được gì vượt ngoài việc khuếch đại hành vi sẵn có.
- **Kiểm tra trên ít nhất hai model family khác nhau** trước khi kết luận một kỹ thuật RLVR "hoạt động": hiệu ứng mạnh trên Qwen không đảm bảo lặp lại trên Llama/OLMo2 (hoặc tổng quát hơn, trên một model không cùng pretraining data/behavior).
- **Hiểu clipping bias giúp debug RL không hội tụ hoặc hội tụ "sai hướng":** nếu thấy model RLVR cải thiện accuracy benchmark nhưng khi đọc kỹ output lại thấy hành vi lạ tăng lên không liên quan tới "học đúng/sai" (ví dụ chỉ tăng độ dài, tăng lặp code, tăng dùng một cấu trúc câu cố định), rất có thể đó là clipping bias khuếch đại một prior có sẵn, không phải model đang "học" theo reward.

9. Thí nghiệm gợi ý

1. Lặp lại thí nghiệm "no-clip ablation" (mục 3.2, Hình 4 paper) trên một quy trình GRPO nhỏ tự có: so sánh random reward với clip bật/tắt, kiểm tra xem có tái lập hiện tượng "clip bật thì học được, clip tắt thì không" ở quy mô nhỏ.
2. Thử format reward và random reward trên một model khác Qwen (ví dụ một mô hình nhỏ mã nguồn mở không thuộc họ Qwen) cho một tác vụ verifiable tự chọn, đo xem hiệu ứng "reward giả vẫn cải thiện" có xuất hiện hay không, đối chiếu với claim "phụ thuộc model family" ở mục 4.
3. Với một model bất kỳ, đo trước RL xem có hành vi nào xuất hiện với tần suất cao bất thường trong response (tương tự code reasoning của Qwen-Math); sau đó train bằng ground truth reward và bằng random reward, theo dõi tần suất hành vi đó theo step để kiểm tra giả thuyết "reward giả khuếch đại hành vi có sẵn" (tương tự Hình 6 paper).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
5 loại reward	Ground Truth, Majority Vote, Format, Incorrect, Random	dải từ chuẩn tới hoàn toàn không mang thông tin
Random reward	$r = 1$ với xác suất γ, độc lập (x, y)	$\gamma = 0$: gradient kỳ vọng = 0 (kiểm chứng giải tích)

Khái niệm	Công thức / ý	Ghi chú
Clipping bias (GRPO)	khuếch đại token $R_\theta < 1 - \epsilon_c$, dìm token $R_\theta > 1 + \epsilon_c$	bất đối xứng theo $\pi_{old,x}(y_t)$ ban đầu
Code reasoning	sinh code Python để hỗ trợ suy luận, không thực thi	hành vi có sẵn, chỉ hiệu quả ở Qwen2.5-Math
Tương quan hiệu quả	reward giả tăng accuracy cùng lúc tăng code frequency	ground truth: code frequency tăng rồi giảm
Model-dependence	Qwen2.5 (cả họ) nhạy reward giả; Llama3/OLMo2 không	cùng pretraining prior quyết định kết quả

11. Kết luận ngắn

RLVR chuẩn với GRPO trên Qwen2.5-Math-7B cải thiện gần bằng ground truth reward dù dùng reward hoàn toàn ngẫu nhiên hoặc cố ý sai, do hai nguyên nhân cộng hưởng: **clipping bias** trong công thức GRPO tự khuếch đại token xác suất cao bất kể reward, và Qwen2.5-Math có sẵn một hành vi hữu ích từ pretraining ("code reasoning") dễ bị khuếch đại theo cách đó. Hiệu ứng này **không tổng quát** sang Llama3/OLMo2, cho thấy nhiều kết luận RLVR rút ra chỉ từ Qwen cần được kiểm chứng lại trên model khác. Kết hợp với bài 45 và 47, ba bài này tạo thành bộ chứng cứ khá nhất quán rằng phần lớn hiệu quả RLVR hiện tại nằm ở việc khai thác lại năng lực/hành vi đã có sẵn trong mô hình nền, chứ chưa phải dạy năng lực hoàn toàn mới, trừ khi huấn luyện đủ lâu và đủ đa dạng như bài 46 (ProRL) đề xuất.

Link gốc: https://arxiv.org/abs/2506.10947

Bài 49

The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models (2025)

Tác giả: Ganqu Cui, Yuchen Zhang, Jiacheng Chen (đóng góp bằng nhau), Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, Ning Ding (Shanghai AI Laboratory; Tsinghua University; UIUC; Peking University; Nanjing University; CUHK). **Venue / arXiv:** arXiv:2505.22617 (v1, 28/05/2025). Bản text ghi "Preprint", bài báo không nêu venue chính thức. **Nguồn:** https://arxiv.org/abs/2505.22617 **Chủ đề:** Giải thích tại sao policy entropy sụp đổ (collapse) khi dùng RL huấn luyện LLM lý luận, đưa ra quy luật thực nghiệm giữa entropy và hiệu năng, suy ra cơ chế động học entropy từ hiệp phương sai (covariance), và hai kỹ thuật kiểm soát entropy (Clip-Cov, KL-Cov). **Vị trí trong lộ trình:** Đọc sau bài 00 (policy gradient, PPO, GRPO, KL penalty), bài 07/41 (PPO, DAPO và clip-higher) và bài 36 (DeepSeekMath, GRPO). Bài này mở đầu nhóm các bài "phân tích cơ chế RLVR" (45-52): 45 hỏi RL có vượt mô hình nền không, 46 (ProRL) kéo dài huấn luyện, 48 hỏi phần thưởng có cần đúng hay không, còn 49-52 nhìn vào *những gì thật sự thay đổi trong policy* khi RL. Đọc tiếp bài 50 (cùng nhóm tác giả Qwen, xác định token nào gây ra entropy cao) để thấy hai bài bổ sung cho nhau: 49 giải thích *vì sao* entropy giảm, 50 chỉ ra *token nào* quan trọng.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

RL luôn phải cân bằng **exploitation** (khai thác chiến lược đã biết là tốt) và **exploration** (tìm chiến lược mới). Đại lượng đo mức độ "còn thăm dò" của policy là **policy entropy**:

$$H(\pi_\theta, D) = -\mathbb{E}_{D,\pi_\theta}[\log \pi_\theta(y_t \mid y_{<t})]$$

tính trung bình trên các token của các rollout lấy mẫu từ tập prompt huấn luyện D. Trong RL cổ điển, nhiều thuật toán chủ động giữ entropy cao bằng **maximum-entropy RL** (ví dụ SAC, bài 12) hoặc regularizer entropy trong loss (A3C, PPO). Với LLM suy luận (RLVR: RL với phần thưởng kiểm chứng được, ví dụ đúng/sai một bài toán), các quy trình phổ biến (GRPO, DAPO) thường **không** có entropy bonus, và tác giả quan sát một hiện tượng nhất quán: entropy tụt gần 0 chỉ sau vài trăm bước huấn luyện, kéo theo hiệu năng chững lại (performance saturation). Bài này hỏi: quy luật của sự sụp đổ này là gì, và cơ chế toán học nào gây ra nó?

Khái niệm nền:

- **Policy gradient (Williams 1992):** $\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}) A_t\right]$, với A_t là advantage.
- **GRPO (bài 36):** chuẩn hóa phần thưởng theo nhóm K mẫu cùng prompt: $A_t = \dfrac{r(y) - \text{mean}(r(y^{1:K}))}{\text{std}(r(y^{1:K}))}$.
- **PPO surrogate loss (bài 07):** $L(\theta) = \mathbb{E}_t\left[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)\right]$, $\rho_t = \pi_\theta / \pi_{\theta_{old}}$.
- **Natural policy gradient (Kakade 2001):** biến thể policy gradient có nhân thêm nghịch đảo ma trận Fisher, cho bước cập nhật "đều" hơn theo không gian phân phối (ít dùng trực tiếp cho LLM vì tốn bậc hai, nhưng hữu ích để so sánh lý thuyết).
- **Softmax policy:** $\pi_\theta(a \mid s) = \exp(z_{s,a}) / \sum_{a'} \exp(z_{s,a'})$, $z_{s,a}$ là logit.

2. Ý tưởng chính

Hai phát hiện liên kết với nhau:

1. **Quy luật thực nghiệm:** không can thiệp (không entropy loss, không KL penalty), hiệu năng R và entropy H của policy tuân theo một hàm mũ đơn giản

$$\boxed{R = -a\exp(H) + b}$$

   với a, b là hệ số fit theo từng mô hình/dữ liệu. Điều này nghĩa là policy **đánh đổi** entropy để lấy performance theo một đường cong cố định, và trần hiệu năng ($H = 0 \Rightarrow R = -a + b$) có thể đoán được sớm.
2. **Cơ chế động học:** với policy dạng softmax, *thay đổi* entropy giữa hai bước xấp xỉ bằng **âm hiệp phương sai** giữa log-xác suất hành động và sự thay đổi logit của hành động đó. Dưới policy gradient, thay đổi logit tỉ lệ với advantage, nên hiệp phương sai này chính là hiệp phương sai giữa (xác suất hành động) và (advantage của hành động). Hiệp phương sai dương (hành động xác suất cao đi kèm advantage dương) sẽ kéo entropy đi xuống; paper đo thực nghiệm thấy hiệp phương sai này luôn dương trong RLVR, giải thích vì sao entropy giảm đơn điệu.

Từ cơ chế này, tác giả đề xuất **Clip-Cov** và **KL-Cov**: hạn chế cập nhật đúng những token có hiệp phương sai cao nhất (chỉ khoảng 10^{-4} đến 10^{-3} số token), nhờ đó giữ entropy cao hơn mà vẫn cải thiện hiệu năng.

3. Phương pháp chi tiết

3.1 Động học entropy của softmax policy (Lemma 1)

Với mỗi trạng thái s, logit $z_{s,a}$ là tham số riêng (tabular softmax). Khai triển Taylor bậc một quanh bước cập nhật $z^{k+1} - z^k$:

$$H(\pi_{\theta^{k+1}} \mid s) \approx H(\pi_{\theta^k} \mid s) + \langle \nabla H(\pi_{\theta^k} \mid s),\, z^{k+1} - z^k \rangle$$

Vì $\nabla_\theta H(\pi_\theta \mid s) = -\mathbb{E}_{a\sim\pi}[\log \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s)]$ và đạo hàm softmax $\partial \log \pi_\theta(a \mid s)/\partial\theta_{s,a'} = \mathbb{1}\{a = a'\} - \pi_\theta(a' \mid s)$, khai triển tích vô hướng rồi dùng định nghĩa hiệp phương sai (phần nhân với $\pi(a'|s)$ chính là số hạng trừ trung bình) cho ra:

$$\boxed{H(\pi_{\theta^{k+1}} \mid s) - H(\pi_{\theta^k} \mid s) \approx -\operatorname{Cov}_{a\sim\pi_{\theta^k}(\cdot|s)}\left[\log \pi_{\theta^k}(a \mid s),\; z^{k+1}_{s,a} - z^k_{s,a}\right]}$$

Trực giác: nếu hành động a đang có xác suất cao (log-prob lớn, ít âm) và logit của nó *còn tăng tiếp* sau cập nhật, hiệp phương sai dương, entropy giảm.

3.2 Dưới Policy Gradient và Natural Policy Gradient

Với policy gradient vanilla (Proposition 1, dùng $\mathbb{E}_{a\sim\pi}[A(s,a)] = 0$ vì advantage đã chuẩn hóa quanh baseline):

$$z^{k+1}_{s,a} - z^k_{s,a} = \eta\, \pi_\theta(a \mid s)\, A(s,a)$$

Thế vào Lemma 1 (Theorem 1):

$$\boxed{H(\pi_{\theta^{k+1}} \mid s) - H(\pi_{\theta^k} \mid s) \approx -\eta \operatorname{Cov}_{a\sim\pi_{\theta^k}}[\log \pi_{\theta^k}(a \mid s),\; \pi_{\theta^k}(a \mid s)\, A(s,a)]}$$

Với natural policy gradient (Theorem 2, dùng $z_{s,a}^{k+1} - z_{s,a}^{k} = \eta A(s,a)$ trực tiếp, không nhân thêm $\pi(a|s)$):

$$H(\pi_{\theta^{k+1}} \mid s) - H(\pi_{\theta^k} \mid s) \approx -\eta \, \mathrm{Cov}_{a\sim\pi_{\theta^k}}[\log \pi_{\theta^k}(a \mid s),\ A(s,a)]$$

Kết luận chung: tương quan dương giữa xác suất hành động và advantage của nó làm entropy giảm; tương quan âm (hành động hiếm nhưng advantage cao) làm entropy tăng.

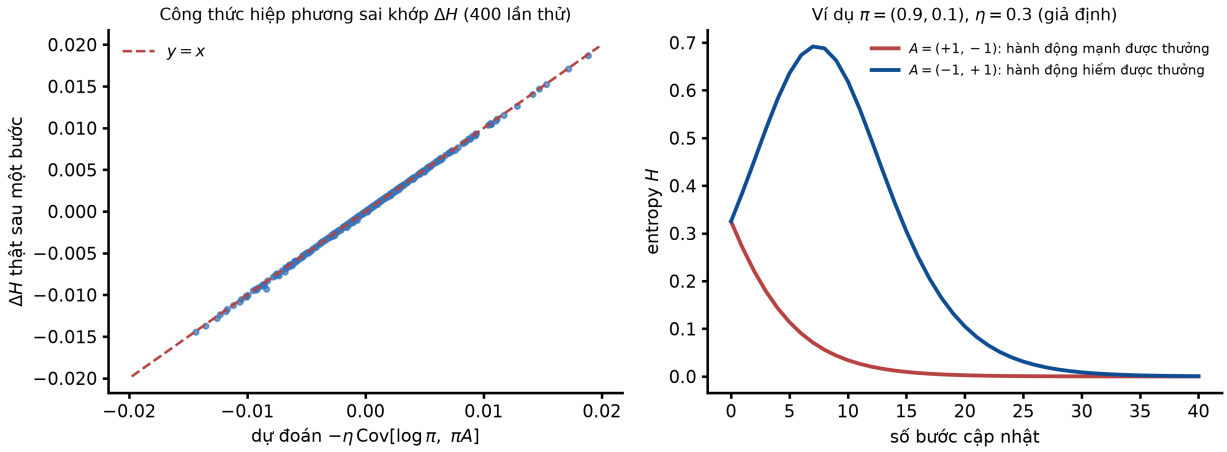
3.3 Ví dụ nhỏ (tự tính)

Minh họa Theorem 1 với 2 hành động, $\pi = (0.9,\ 0.1)$ (hành động a chiếm 90%, b chiếm 10%), $\log 0.9 = -0.105$, $\log 0.1 = -2.303$.

Trường hợp 1 (hành động đang mạnh được thưởng thêm): $A(a) = +1,\ A(b) = -1$. $\pi A = (0.9,\ -0.1)$; $\mathbb{E}[\log \pi] = 0.9(-0.105) + 0.1(-2.303) = -0.325$; $\mathbb{E}[\pi A] = 0.9(0.9) + 0.1(-0.1) = 0.80$; $\mathbb{E}[\log \pi \cdot \pi A] = 0.9(-0.105 \cdot 0.9) + 0.1(-2.303 \cdot -0.1) = -0.085 + 0.023 = -0.062$; Cov $= -0.062 - (-0.325)(0.80) = -0.062 + 0.260 = +0.198 > 0$. $\Rightarrow \Delta H = -\eta \cdot 0.198 < 0$: **entropy giảm** (policy tự tin thêm vào a đã mạnh).

Trường hợp 2 (hành động hiếm được thưởng): $A(a) = -1,\ A(b) = +1$ (đổi dấu advantage). Bằng tính toán tương tự, Cov $= -0.198 < 0 \Rightarrow \Delta H = +\eta \cdot 0.198 > 0$: **entropy tăng** (policy bớt tự tin vào a, chia sẻ xác suất cho b).

Hai trường hợp đối xứng này khớp đúng trực giác của bài báo: "hành động xác suất cao + advantage cao → entropy giảm; hành động hiếm + advantage cao → entropy tăng".



Hình 89: Mô phỏng softmax: ΔH thật khớp $-\eta$ Cov, và entropy giảm hay tăng theo dấu của hiệp phương sai (minh hoạ, tự tính)

Cách đọc hình: mô phỏng giả định, seed cố định. Bên trái, mỗi chấm là một lần thử với logit và advantage ngẫu nhiên; các chấm nằm trên đường $y = x$ nghĩa là công thức hiệp phương sai dự đoán đúng thay đổi entropy. Bên phải, thưởng cho hành động đang mạnh (đỏ) làm entropy giảm ngay; thưởng cho hành động hiếm (xanh) làm entropy tăng lúc đầu rồi giảm khi chính sách đổi sang hành động kia.

3.4 Clip-Cov và KL-Cov

Với một batch N token rollout, định nghĩa hiệp phương sai *token-wise* (ước lượng theo mẫu, công thức (10)):

$$\mathrm{Cov}(y_i) = \left(\log \pi_\theta(y_i) - \tfrac{1}{N}\textstyle\sum_j \log \pi_\theta(y_j)\right)\left(A(y_i) - \tfrac{1}{N}\textstyle\sum_j A(y_j)\right)$$

Clip-Cov: chọn ngẫu nhiên $\lfloor r \cdot N \rfloor$ token có $\text{Cov}(y_i) \in [\omega_{low}, \omega_{high}]$ (hai ngưỡng này đặt cao hơn trung bình hơn 500 lần), rồi **detach gradient** của các token đó khỏi loss PPO:

$$L_{\text{Clip-Cov}} = \begin{cases} \mathbb{E}_t \left[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t) \right], & t \notin I_{clip} \\ 0, & t \in I_{clip} \end{cases}$$

KL-Cov: xếp hạng theo Cov, lấy top $k \cdot N$ ($k \ll 1$) token có hiệp phương sai lớn nhất, áp **KL penalty** (so với policy rollout $\pi_{\theta_{old}}$) chỉ lên các token đó:

$$L_{\text{KL-Cov}} = \begin{cases} \mathbb{E}_t[\rho_t A_t], & t \notin I_{KL} \\ \mathbb{E}_t[\rho_t A_t - \beta\, D_{KL}(\pi_{\theta_{old}} \| \pi_\theta)], & t \in I_{KL} \end{cases}$$

```
def compute_policy_loss(old_log_prob, log_prob, advantages, select_ratio, method, **args):
    ratio = exp(log_prob - old_log_prob)
    pg_losses1 = -ratio * advantages
    covs = (log_prob - log_prob.mean()) * (advantages - advantages.mean())
    select_num = int(select_ratio * len(pg_losses1))
    if method == "clip_cov":
        pg_losses2 = -clip(ratio, clip_lb, clip_ub) * advantages
        clip_idx = random_select(covs[(covs > cov_lb) & (covs < cov_ub)], num=select_num)
        pg_losses1[clip_idx].detach_(); pg_losses2[clip_idx].detach_()
        pg_loss = maximum(pg_losses1, pg_losses2).mean()
    if method == "kl_cov":
        kl_penalty = (log_prob - old_log_prob).abs()
        select_idx = topk(covs, k=select_num, largest=True)
        pg_losses1[select_idx] += kl_coef * kl_penalty[select_idx]
        pg_loss = pg_losses1.mean()
    return pg_loss
```

Chỉ cần vài dòng thay đổi so với loss PPO/GRPO chuẩn.

4. Thí nghiệm và kết quả chính

Phạm vi thực nghiệm: 4 họ mô hình, 11 mô hình nền (0.5-32B): Qwen2.5 (0.5/1.5/3/7/32B), Mistral (7B, Nemo-12B, Small-24B), LLaMA3 (3B/8B), DeepSeek-Math-7B-Base. Toán và code, 8 benchmark công khai (MATH500, AIME24/25, AMC, OlympiadBench, OMNI-MATH cho toán; Eurus-2-RL-Code, KodCode cho code). 4 thuật toán RL: GRPO, REINFORCE++, PRIME, RLOO. Huấn luyện kiểu "Zero" (RL trực tiếp từ mô hình nền) với veRL; lr 5×10^{-7} (policy), 10^{-6} (PRM của PRIME); batch 256, micro-batch 128; rollout 512 prompt x 8 mẫu; hệ số KL tham chiếu = 0; $\epsilon = 0.2$.

Entropy collapse rất nhanh: trên 2400 bước GRPO của 11 mô hình, **73% lượng entropy tiêu hao và 76% điểm hiệu năng tăng thêm xảy ra trong 200 bước đầu** (1/12 tổng số bước); 800 bước đầu (1/3 tổng) chiếm **hơn 93% hiệu năng tăng thêm cùng 94% entropy mất đi**. Hơn 2/3 số bước còn lại chỉ mang lại lợi ích nhỏ.

Quy luật $R = -a\exp(H) + b$: fit chính xác trên hơn 200 điểm dữ liệu mỗi thí nghiệm, chỉ cần 2 hệ số, đúng cho mọi kích thước/họ mô hình/toán và code (Hình 3, 4).

Dự đoán sớm: fit hàm số chỉ với 36 bước đầu (Qwen2.5, 15% tổng số bước) rồi dự đoán 200 bước tiếp theo: RMSE trung bình 0.9% (toán) và 1.2% (code) cho toàn bộ quá trình dự đoán, 0.5%/1.9% cho điểm số bước cuối.

Hệ số theo kích thước mô hình: a, b biến đổi log-tuyến tính theo số tham số (Qwen2.5 family), nên

có thể ngoại suy hệ số của mô hình lớn từ mô hình nhỏ cùng họ. Hệ số không đổi đáng kể giữa GRPO/RLOO/PRIME/REINFORCE++ (Hình 6): a, b là tính chất nội tại của (mô hình, dữ liệu), không phải của thuật toán.

Bảng 2 (trích, avg@32 cho AIME, benchmark còn lại là điểm chuẩn):

Phương pháp	AIME24	AIME25	AMC	MATH-500	OMNI-MATH	OlympiadBench	Minerva	Avg
Qwen2.5-7B, GRPO	21.2	9.6	58.7	78.8	27.9	40.7	36.7	38.6
w. Clip-higher (bài 41)	18.1	11.5	56.6	79.2	29.8	43.3	40.4	38.8
w. Clip-Cov	22.1	15.8	58.2	80.4	30.5	44.1	41.1	**40.4**
w. KL-Cov	22.6	12.9	61.4	80.8	29.1	42.6	38.2	**40.6**
Qwen2.5-32B, GRPO	21.8	16.2	69.7	84.2	35.2	43.6	45.5	45.8
w. Clip-higher	35.6	22.3	69.5	77.2	35.1	42.5	43.0	47.2
w. Clip-Cov	32.3	22.7	67.2	87.0	42.0	57.2	46.0	**50.3**
w. KL-Cov	36.8	30.8	74.5	84.6	39.1	49.0	46.3	**52.2**

So với GRPO gốc: +2.0 điểm trung bình trên 7B, **+6.4 điểm trên 32B**; trên AIME24/25 của 32B, KL-Cov hơn GRPO tới +15.0/+14.6 điểm. Entropy của KL-Cov duy trì **hơn 10 lần** cao hơn GRPO gốc khi GRPO đã bão hòa; độ dài phản hồi tăng dần cùng lúc hiệu năng không bão hòa (Hình 11).

5. Phân tích / ablation

Entropy loss và KL regularization chuẩn đều không giúp: thêm entropy bonus $L_{ent} = L - \alpha H$ rất nhạy hệ số: α nhỏ (0.0001, 0.001) gần như không đổi, $\alpha = 0.01$ gây **bùng nổ entropy** (entropy explosion), $\alpha = 0.005$ ổn định được entropy nhưng **không vượt** baseline về hiệu năng. Thêm KL penalty với mô hình tham chiếu giữ entropy ổn định nhưng khiến hiệu năng **giảm**. Vì vậy các quy trình RLVR gần đây (DAPO, PRIME, Open-Reasoner-Zero...) thường bỏ cả hai.

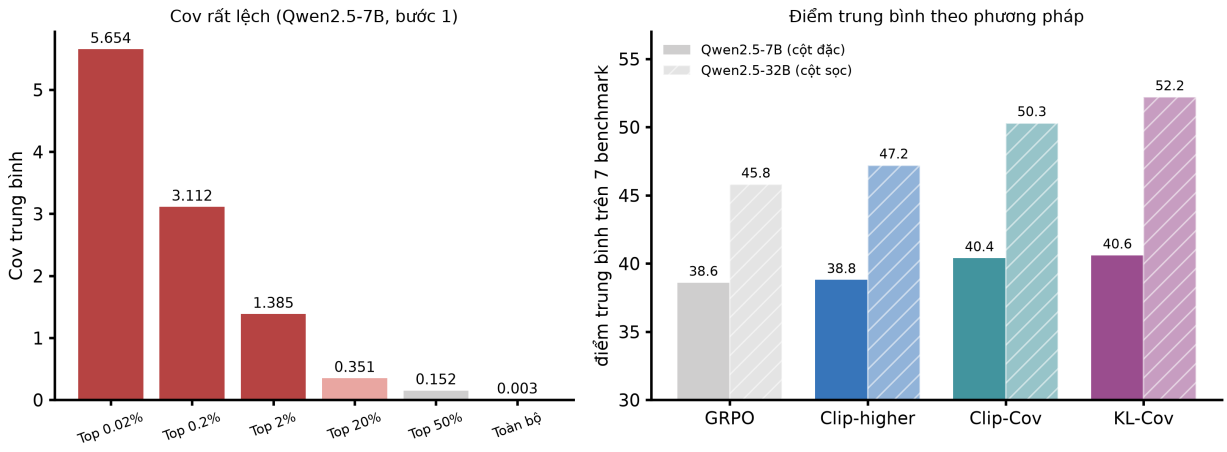
Phân bố hiệp phương sai rất lệch (Bảng 1, Qwen2.5-7B, bước 1):

Nhóm token	Giá trị Cov trung bình
Top 0.02%	5.654
Top 0.2%	3.112
Top 2%	1.385
Top 20%	0.351
Top 50%	0.152
Toàn bộ	0.003

Một phần token rất nhỏ chiếm hiệp phương sai vượt xa trung bình; Clip-Cov/KL-Cov chỉ can thiệp 10^{-4} đến 10^{-3} số token mà đổi hẳn đường cong entropy, cho thấy có "token then chốt" (pivotal tokens).

Cách đọc hình: bên trái, Cov trung bình của nhóm top 0.02% (5.654) lớn hơn toàn bộ token (0.003) hàng nghìn lần, nên chỉ cần can thiệp vài token. Bên phải, Clip-Cov và KL-Cov vượt GRPO ở cả hai cỡ mô hình, rõ nhất ở 32B (45.8 lên 50.3 và 52.2). Cột sọc là Qwen2.5-32B.

Kiểm soát entropy theo tham số: tăng tỉ lệ clip r (Clip-Cov) hoặc hệ số KL β (KL-Cov) làm entropy cao hơn một cách đơn điệu (Hình 12); KL-Cov cho đường cong entropy **ổn định hơn** Clip-Cov.



Hình 90: Hiệp phương sai theo nhóm token (Bảng 1) và điểm trung bình của bốn phương pháp (Bảng 2) (vẽ lại từ số liệu trong ghi chú)

Liên hệ với clip-higher (DAPO, bài 41): nâng ngưỡng trên ϵ_{high} của PPO cũng gián tiếp đưa thêm token xác suất thấp/advantage dương (hiệp phương sai trung bình khoảng -0.03, tức hiệp phương sai *thấp*) vào gradient, nên có tác dụng tương tự nhưng **gián tiếp**; Clip-Cov/KL-Cov dùng trực tiếp hiệp phương sai làm ngưỡng nên kiểm soát entropy chính xác hơn. Thực nghiệm cho thấy clip-higher cải thiện sớm nhưng về sau mất ổn định, hiệu năng bão hòa rồi giảm, trong khi Clip-Cov/KL-Cov giữ đường cong ổn định hơn trong suốt quá trình (Hình 11).

6. Giới hạn

Tác giả nêu: quy luật $R = -a\exp(H) + b$ không phổ quát: các công trình dùng policy model khác hoặc dữ liệu off-policy quan sát mẫu hình entropy khác (dẫn Luo et al. 2025, Yan et al. 2025); không có relationship rõ ràng giữa entropy *bị can thiệp* và hiệu năng cuối cùng; giá trị entropy "tối ưu" để cân bằng exploration/stability vẫn là câu hỏi mở; tác giả thừa nhận "trần" hiệu năng không phải do hạn chế nội tại của RL mà do cơ chế entropy của LLM, nhưng chưa chứng minh được cách phá trần đó ngoài việc trì hoãn sụp đổ entropy.

Nhận xét:

- Toàn bộ thực nghiệm dùng **phần thưởng kiểm chứng được** (toán/code, không có reward hacking); không rõ cơ chế hiệp phương sai còn đúng với mô hình phần thưởng học được (RLHF) hay không.
- $\omega_{low}, \omega_{high}$ (Clip-Cov) và k, β (KL-Cov) đều được chọn thực nghiệm riêng cho 7B/32B ($k = 2 \times 10^{-3}$ và 2×10^{-4} tương ứng); chưa có công thức chọn ngưỡng tự động theo kích thước mô hình.
- Theorem 1/2 chứng minh cho **tabular softmax policy** (mỗi (s,a) một logit riêng), còn LLM chia sẻ tham số qua hàng tỉ cặp (s,a); đây là xấp xỉ hợp lý nhưng chưa có bằng chứng lý thuyết chặt cho mạng neural thực tế.
- Bài không so sánh Clip-Cov/KL-Cov với các cách kiểm soát entropy khác (ví dụ chính bài 50 sắp đọc, chọn token theo entropy thay vì hiệp phương sai).

7. Liên hệ với các bài khác

- **Bài 00, 07 (PPO), 36 (GRPO/DeepSeekMath):** nền tảng cho advantage, surrogate loss, clip ϵ mà Clip-Cov/KL-Cov sửa đổi trực tiếp.
- **Bài 41 (DAPO, clip-higher):** baseline chính được so sánh; bài 49 giải thích *vì sao* clip-higher giúp tăng entropy (gián tiếp qua token advantage dương, xác suất thấp).
- **Bài 12 (SAC):** đối chiếu triết lý entropy-regularized RL cổ điển (entropy bonus trong reward) với RLVR cho LLM, nơi entropy bonus thất bại.

- **Bài 45 (Does RL really incentivize reasoning beyond the mô hình nền?):** chính bài báo này trích dẫn trực tiếp (Yue et al. 2025) cho luận điểm "trần hiệu năng"; bài 49 cung cấp cơ chế toán học cho hiện tượng mà bài 45 quan sát qua pass@k.
- **Bài 46 (ProRL):** huấn luyện kéo dài cố gắng vượt trần entropy; đọc cùng bài 49 để hiểu vì sao kéo dài đơn thuần có thể không đủ nếu không kiểm soát hiệp phương sai.
- **Bài 48 (Spurious rewards):** cùng mạch "chất vấn cơ chế RLVR"; 48 hỏi nội dung tín hiệu thưởng, 49 hỏi cơ chế entropy, bổ sung góc nhìn.
- **Bài 50 (High-entropy minority tokens):** đọc ngay sau; 49 là cơ chế tổng quát (hiệp phương sai), 50 là phát hiện thực nghiệm cụ thể hơn (20% token mang hầu hết tín hiệu entropy hữu ích). Hai bài dùng cùng ý tưởng lọc token nhưng tiêu chí khác: 49 lọc theo hiệp phương sai (loại token nguy hiểm), 50 lọc theo entropy tuyệt đối (giữ token có ích).

8. Ứng dụng thực tế (suy luận)

- **Theo dõi entropy ngay từ đầu huấn luyện RLVR:** vì hơn 70-90% biến động entropy/hiệu năng xảy ra trong 1/3 đầu tiến trình, log entropy mỗi bước là chẩn đoán rẻ và sớm cho biết liệu RL có đang "đốt" hết khả năng thăm dò quá nhanh.
- **Tránh entropy bonus/KL-to-reference ngây thơ:** paper cho bằng chứng thực nghiệm rõ ràng là cả hai cách kiểm soát entropy "kiểu RL cổ điển" đều thất bại hoặc có hại với LLM; nếu cần kiểm soát entropy, Clip-Cov/KL-Cov (sửa vài dòng loss, không thêm forward pass) là lựa chọn rẻ hơn.
- **Chi phí cài đặt thấp:** chỉ cần tính hiệp phương sai token-wise (một phép nhân elementwise), không cần rollout thêm hay mô hình phụ; phù hợp với ràng buộc GPU hạn chế.
- Bài báo không đề cập VLM hay OCR; mọi liên hệ trên là suy luận, cần thực nghiệm riêng để xác nhận.

9. Thí nghiệm gợi ý

1. Chạy GRPO trên một mô hình nhỏ (ví dụ Qwen2.5-0.5B) trên một tập toán nhỏ, log H và R mỗi 4 bước, fit $R = -a\exp(H) + b$ chỉ với 15% số bước đầu, kiểm tra RMSE dự đoán phần còn lại (tái lập Hình 5 ở quy mô nhỏ).
2. Cài KL-Cov trên quy trình GRPO sẵn có (chỉ cần thêm 5-6 dòng theo Listing 1), so entropy và test accuracy với GRPO gốc trên cùng tập.
3. Nếu có log rollout của một lần RLVR unit-test-reward (OCR hoặc tác vụ khác), tính phân phối hiệp phương sai token-wise như Bảng 1 để xem liệu cùng hiện tượng "một nhóm token nhỏ chiếm hiệp phương sai lớn" có xảy ra hay không.
4. So sánh entropy ban đầu (mô hình nền, trước RL) của một tác vụ đóng (OCR, có 1 đáp án đúng) với một tác vụ mở (toán, có nhiều cách giải): kiểm tra giả thuyết "tác vụ đóng sẽ có hệ số a lớn hơn, tức sụp entropy nhanh hơn".

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Policy entropy	$H = -\mathbb{E}_{D,\pi_\theta}[\log \pi_\theta(y_t \mid y_{<t})]$	trung bình token-level
Quy luật thực nghiệm	$R = -a\exp(H) + b$	2 hệ số khớp hơn 200 điểm
Trần hiệu năng	$H = 0 \Rightarrow R = -a + b$	dự đoán được sớm
Entropy diff (softmax)	$\Delta H \approx -\mathrm{Cov}_a[\log \pi(a),\, \Delta z_a]$	Lemma 1
Dưới policy gradient	$\Delta H \approx -\eta\, \mathrm{Cov}[\log \pi(a),\, \pi(a)A(a)]$	Theorem 1
Dưới natural PG	$\Delta H \approx -\eta\, \mathrm{Cov}[\log \pi(a),\, A(a)]$	Theorem 2
Clip-Cov	detach gradient của token Cov cao	$r \approx 2 \times 10^{-4}$

Khái niệm	Công thức / ý	Ghi chú
KL-Cov	KL penalty riêng cho top-k Cov	$k \approx 2 \times 10^{-3}$-$2 \times 10^{-4}$, $\beta = 1$
Kết quả chính	Avg GRPO 38.6/45.8 (7B/32B) $\rightarrow$ KL-Cov 40.6/52.2	Bảng 2

11. Kết luận ngắn

Bài này cho thấy entropy collapse trong RLVR không phải nhiễu mà tuân theo quy luật mũ có thể fit và dự đoán sớm, và cơ chế gây ra nó là hiệp phương sai dương giữa xác suất hành động và advantage dưới cập nhật policy gradient. Từ đó, hai kỹ thuật rất rẻ (Clip-Cov, KL-Cov) chỉ can thiệp một phần token cực nhỏ nhưng giữ entropy cao hơn hẳn và cải thiện hiệu năng, đặc biệt rõ ở mô hình lớn (32B). Thông điệp chính cho người thực hành: theo dõi entropy sớm, và nếu cần kiểm soát, nhắm vào đúng những token có hiệp phương sai cao thay vì áp entropy bonus đồng loạt.

Link gốc: https://arxiv.org/abs/2505.22617

Bài 50

Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning (2025)

Tác giả: Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, Junyang Lin (Qwen Team, Alibaba; LeapLab, Tsinghua University). **Venue / arXiv:** arXiv:2506.01939 (v2, 13/11/2025; bản gốc 02/06/2025). Accepted to NeurIPS 2025. **Nguồn:** https://arxiv.org/abs/2506.01939 **Chủ đề:** Phân tích RLVR qua lăng kính entropy *từng token*: chỉ một thiểu số token có entropy cao đóng vai trò "ngã ba" (forking tokens) định hướng lý luận; giới hạn gradient chính sách chỉ vào 20% token entropy cao nhất giữ hoặc vượt hiệu năng so với dùng toàn bộ token, và lợi ích càng rõ khi mô hình càng lớn. **Vị trí trong lộ trình:** Đọc sau bài 49 (cơ chế entropy chung qua hiệp phương sai) và bài 41 (DAPO, công thức policy gradient mà bài này sửa trực tiếp). Hai bài 49 và 50 bổ sung nhau: 49 trả lời "vì sao entropy giảm", 50 trả lời "token nào thực sự mang lại ích lợi khi cho entropy cao". Đọc tiếp bài 51, 52 để thấy thêm hai góc nhìn khác về việc RL chỉ thay đổi một phần nhỏ của mô hình/phân phối. **Chương nền liên quan:** nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Các bài trước (ví dụ bài 49) phân tích entropy **gộp** trên toàn bộ token của một rollout. Bài này đặt câu hỏi ở mức *từng vị trí token*: trong một chuỗi Chain-of-Thought (CoT), token nào có entropy cao, chúng đóng vai trò gì, và RLVR có "chăm sóc" chúng khác với các token khác không?

Khái niệm nền:

- **Token entropy:** $H_t = -\sum_{j=1}^{V} p_{t,j} \log p_{t,j}$, với $p_t = \text{Softmax}(z_t/T)$ là phân phối tại vị trí t (không phải entropy của một token cụ thể được lấy mẫu, mà entropy của *phân phối* tại vị trí đó).
- **PPO / GRPO / DAPO:** ba công thức policy gradient chuẩn cho RLVR, DAPO (bài 41) bỏ KL penalty, thêm **clip-higher** ($\epsilon_{high} > \epsilon_{low}$), dynamic sampling, loss theo token, và overlong reward shaping.
- **Forking token (thuật ngữ đề xuất trong bài):** token entropy cao đóng vai trò "ngã ba" dẫn tới các nhánh lý luận khác nhau (ví dụ "wait", "however", "suppose").

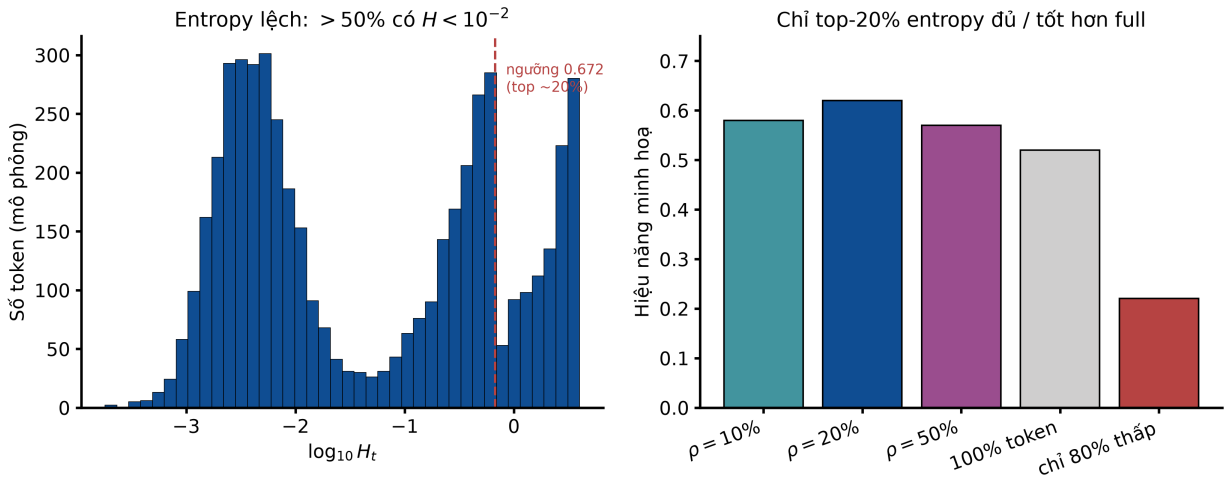
2. Ý tưởng chính

Ba quan sát liên kết với nhau:

1. **Mẫu hình entropy trong CoT:** đa số token có entropy rất thấp (hoàn thiện cấu trúc câu/code có sẵn), chỉ một thiểu số có entropy cao (các điểm rẽ nhánh lý luận).
2. **RLVR giữ nguyên mẫu hình của mô hình nền:** RL chủ yếu *chỉnh* entropy của những token *đã* có entropy cao từ trước, ít thay đổi token entropy thấp.
3. Từ hai quan sát trên, tác giả **giới hạn gradient chính sách chỉ vào top 20% token entropy cao nhất** trong mỗi batch (mask 80% còn lại), và thấy hiệu năng giữ nguyên hoặc **vượt** full-gradient, đặc biệt ở mô hình lớn (32B, 14B); huấn luyện chỉ trên 80% token entropy thấp thì hiệu năng sụp mạnh. Điều

này đi ngược "quy tắc 80/20" thông thường (ít nguyên nhân mang lại nhiều kết quả, nhưng ở đây 20% token mang gần như *toàn bộ* lợi ích, phần còn lại gần như vô ích hoặc có hại).

3. Phương pháp chi tiết



Phân phối entropy lệch; huấn luyện top ~20% forking tokens đủ/tốt hơn full (minh hoạ, tự tính)

3.1 Công thức DAPO và việc lọc theo entropy

DAPO tối ưu (ký hiệu $r_t^i(\theta) = \pi_\theta(o_t^i \mid q, o_{<t}^i)/\pi_{\theta_{old}}(o_t^i \mid q, o_{<t}^i)$):

$$J_{DAPO}(\theta) = \mathbb{E}\left[\frac{1}{\sum_i |o^i|}\sum_{i=1}^{G}\sum_{t=1}^{|o^i|}\min(r_t^i(\theta)\hat{A}_t^i,\ \text{clip}(r_t^i(\theta), 1-\epsilon_{low}, 1+\epsilon_{high})\hat{A}_t^i)\right]$$

Bài này đổi thành $J_{HighEnt}$, chỉ sửa **hai chỗ**: nhân thêm chỉ báo $\mathbb{I}[H_t^i \geq \tau_\rho^B]$ vào số hạng advantage, và đổi mẫu số chuẩn hóa chỉ còn đếm số token thỏa điều kiện:

$$J_{HighEnt}(\theta) = \mathbb{E}\left[\frac{1}{\sum_i\sum_t \mathbb{I}[H_t^i \geq \tau_\rho^B]}\sum_{i=1}^{G}\sum_{t=1}^{|o^i|}\mathbb{I}[H_t^i \geq \tau_\rho^B]\cdot\min(r_t^i\hat{A}_t^i,\ \text{clip}(r_t^i, 1-\epsilon_{low}, 1+\epsilon_{high})\hat{A}_t^i)\right]$$

với τ_ρ^B là ngưỡng entropy trong batch B sao cho đúng tỉ lệ ρ (ví dụ $\rho = 20\%$) token có $H_t \geq \tau_\rho^B$. Token entropy thấp bị loại hoàn toàn khỏi gradient (không chỉ giảm trọng số).

3.2 Thí nghiệm nhiệt độ có kiểm soát để kiểm tra vai trò "ngã ba"

Để kiểm chứng forking token *gây ra* hướng lý luận (không chỉ tương quan), tác giả áp nhiệt độ khác nhau cho hai nhóm token khi decode:

$$p_t' = \text{Softmax}\left(\frac{z_t}{T_t'}\right), \qquad T_t' = \begin{cases} T_{high}, & H_t > h_{threshold} \\ T_{low}, & \text{ngược lại} \end{cases}$$

với $h_{threshold} = 0.672$ (percentile 80 của 10^6 token mẫu từ Qwen3-8B trên AIME). Giảm T_{high} (ép forking token "chắc tay" hơn) làm điểm AIME giảm mạnh hơn hẳn so với giảm T_{low}; tăng T_{high} cải thiện điểm rõ rệt

hơn tăng T_{low} (tăng quá mạnh T_{low} còn sinh ra văn bản vô nghĩa). Điều này xác nhận: **forking token cần được giữ entropy cao để hoạt động đúng vai trò của nó**.

3.3 Đo entropy qua các giai đoạn RLVR

Với mô hình sau RLVR, lấy top 20% vị trí entropy cao nhất tại mỗi checkpoint, tính tỉ lệ trùng lặp (overlap) vị trí đó với mô hình nền (bước 0) và với mô hình cuối (bước 1360):

$$\text{overlap} = \frac{|\text{top-20\% của checkpoint } \cap \text{ top-20\% của mô hình tham chiếu}|}{|\text{top-20\%}|}$$

4. Thí nghiệm và kết quả chính

Thiết lập phân tích entropy (§3): Qwen3-8B, chế độ "thinking" bắt buộc, sinh trên AIME'24/25 ở $T = 1.0$, thu thập $> 10^6$ token.

Mẫu hình 1: hơn **50.64%** token có entropy $< 10^{-2}$; chỉ **20%** token có entropy > 0.672. Phân phối rất lệch (Hình 2a).

Mẫu hình 2: 100 token tần suất cao có entropy trung bình cao nhất là các từ nối logic: "wait", "however", "unless" (chuyển ý/đối lập), "thus", "also" (tiếp diễn), "since", "because" (nhân quả), hay "suppose", "assume", "given", "define" trong toán học. 100 token entropy thấp nhất là hậu tố từ, mảnh code, thành phần công thức, có độ xác định cao (Hình 2b-c).

Bảng 1 (RLVR giữ mẫu hình entropy của mô hình nền, Qwen3-14B, DAPO):

So với	Step 0	Step 112	Step 480	Step 864	Step 1280	Step 1360
Base model	100%	98.70%	93.02%	87.45%	87.09%	86.67%
RLVR model (step 1360)	86.67%	86.83%	90.65%	96.61%	97.34%	100%

Overlap với mô hình nền vẫn **trên 86%** khi hội tụ; Hình 4 cho thấy token entropy khởi điểm cao có mức tăng entropy sau RLVR **lớn hơn hẳn** (log-scale) so với token entropy khởi điểm thấp, nghĩa là RLVR chủ yếu "chỉnh" đúng nhóm token đã có entropy cao.

Bảng 2 (trích, DAPO toàn bộ token vs DAPO chỉ 20% token entropy cao nhất, Acc@16 là độ chính xác trung bình trên 16 lần sinh):

Base model	Benchmark	All tokens Acc@16 (Len@16)	Forking tokens Acc@16 (Len@16)	ΔAcc
Qwen3-32B	AIME'24	55.83 (9644)	**63.54** (12198)	**+7.71**
Qwen3-32B	AIME'25	45.63 (9037)	**56.67** (11842)	**+11.04**
Qwen3-32B	Trung bình 6 bộ	66.59 (5849)	**70.69** (7227)	+4.10
Qwen3-14B	AIME'24	45.21 (7945)	**50.42** (11814)	+5.21
Qwen3-14B	AIME'25	38.13 (7057)	**42.92** (12060)	+4.79
Qwen3-14B	Trung bình 6 bộ	61.40 (4752)	**64.39** (7628)	+2.99
Qwen3-8B	AIME'24	33.33 (6885)	34.58 (9494)	+1.25
Qwen3-8B	Trung bình 6 bộ	53.71 (4022)	**54.23** (5508)	+0.53

Mô hình 32B huấn luyện chỉ với 20% token entropy cao nhất đạt **63.5 AIME'24 / 56.7 AIME'25**, SOTA cho mô hình RLVR trực tiếp từ base dưới 600B tham số (tại thời điểm công bố). Mở rộng độ dài phản hồi tối đa

từ 20480 lên 29696 nâng AIME'24 lên **68.1**. Mọi cấu hình (chỉ 20% token) đều cho độ dài phản hồi dài hơn full-token.

Tổng quát hóa ngoài phân phối: huấn luyện trên DAPO-Math-17K (toán), chỉ dùng top 10% hoặc 20% token entropy cao nhất vẫn vượt rõ vanilla DAPO (100% token) trên LiveCodeBench (code, ngoài phân phối).

Mô hình khác Qwen: Llama-3.1-8B base hầu như không học được trực tiếp (độ chính xác ~1%, sinh lặp từ); sau cold-start SFT bằng dữ liệu Qwen3-32B sinh ra, DAPO chỉ-forking-token vẫn vượt vanilla DAPO, nhưng điểm tuyệt đối thấp nên tác giả tự nhận "kém thuyết phục hơn" kết quả trên Qwen3.

5. Phân tích / ablation

Tỉ lệ ρ: thay đổi $\rho \in \{10\%, 20\%, 50\%, 100\%\}$. Với Qwen3-8B, hiệu năng gần như không đổi theo ρ. Với 14B/32B, giảm ρ từ 20% xuống 10% làm giảm nhẹ hiệu năng; tăng mạnh lên 100% (= full-token) làm giảm **rõ rệt**. Giữ $\rho \approx 20\%$ cân bằng tốt nhất giữa exploration và ổn định (Hình 7).

80% token entropy thấp nhất gây hại: chỉ huấn luyện trên 80% token entropy thấp (loại bỏ 20% cao nhất) làm hiệu năng **sụp mạnh**, dù nhóm này chiếm 80% tổng số token dùng để huấn luyện. Chứng tỏ token entropy thấp góp rất ít, có khi cản trở việc học.

Entropy tổng thể theo ρ: $\rho = 20\%$ giữ entropy tổng thể cao nhất trong suốt giai đoạn trước khi hội tụ; $\rho = 10\%/50\%/100\%$ hoặc nhóm 80% thấp đều giảm entropy tổng thể nhanh hơn. Tác giả suy ra: lợi ích chính của forking token nằm ở việc **duy trì exploration**.

Quy mô hóa theo kích thước mô hình: lợi ích của chỉ-dùng-forking-token tăng dần theo kích thước (32B > 14B > 8B); tác giả suy đoán mô hình nhỏ có năng lực hạn chế nên không khai thác được exploration tăng thêm.

6. Giới hạn

Tác giả nêu: thử nghiệm ngoài họ Qwen (Llama) không đủ mạnh để thuyết phục; phạm vi dữ liệu chủ yếu là toán (DAPO-Math-17K), chưa mở rộng sang code hay ARC-AGI; tỉ lệ $\rho = 20\%$ tối ưu trong thí nghiệm này có thể cần điều chỉnh cho thiết lập RLVR khác.

Nhận xét:

- Ngưỡng entropy $h_{threshold} = 0.672$ và tỉ lệ ρ được xác định từ phân phối của *một* mô hình (Qwen3-8B) trên *một* tập dữ liệu (AIME), rồi áp dụng như một hằng số chung; chưa rõ độ nhạy khi miền dữ liệu có phân phối entropy khác hẳn (ví dụ CoT ngắn, hay văn bản không phải toán).
- Kết quả trên Qwen3-8B hầu như không có lợi (Acc@16 trung bình chỉ +0.53), và một chỉ số (AMC'23) thậm chí giảm nhẹ (-0.625); "lợi ích tăng theo kích thước" dựa trên 3 điểm dữ liệu (8B/14B/32B) cùng họ mô hình, chưa kiểm chứng ở họ khác.
- Thí nghiệm nhiệt độ (§3.3 bài này, mục 3.2 ghi chú) là can thiệp ở **suy luận**, không phải huấn luyện; chứng minh forking token quan trọng lúc decode, nhưng không trực tiếp chứng minh *tại sao* chỉ cần gradient của chúng lúc train là đủ.
- Bài không so sánh trực tiếp với Clip-Cov/KL-Cov (bài 49): cả hai đều lọc token nhưng theo tiêu chí khác (entropy tuyệt đối so với hiệp phương sai); chưa rõ hai cách lọc có trùng nhau phần lớn hay bổ sung nhau.

7. Liên hệ với các bài khác

- **Bài 41 (DAPO):** nền trực tiếp; công thức $J_{HighEnt}$ chỉ là J_{DAPO} có thêm mặt nạ entropy, dùng y nguyên clip-higher, dynamic sampling, overlong reward shaping của DAPO.

- **Bài 49 (Entropy mechanism):** cùng nhóm tác giả gần (Qwen/Shanghai AI Lab) nghiên cứu entropy, nhưng góc nhìn bổ sung: 49 giải thích *cơ chế* (hiệp phương sai) khiến entropy giảm; 50 chỉ ra token nào mang lại ích lợi khi giữ entropy cao. Có thể hình dung: 49 nói "đừng cho hiệp phương sai cao kéo entropy về 0 quá nhanh", 50 nói "khi phân bổ ngân sách exploration, hãy ưu tiên 20% token forking".
- **Bài 36 (DeepSeekMath/GRPO), 39 (DeepSeek-R1):** RLVR nền; bài này dùng DAPO (tiến hóa từ GRPO) làm baseline.
- **Bài 00:** PPO/GRPO/token-level policy gradient là nền cho công thức (6).
- Thảo luận trong bài (không nằm trong lộ trình này, chỉ nhắc để biết nguồn) trích Chu et al. 2025 ("SFT memorizes, RL generalizes") để lý giải vì sao forking-token entropy cao có thể là một phần nguyên nhân khiến RL tổng quát hóa tốt hơn SFT; đáng đối chiếu khi đọc bài 51 và 52 (cùng chủ đề SFT/RL khác nhau thế nào), vì cả hai bài đó cũng trích dẫn Chu et al. 2025.

8. Ứng dụng thực tế (suy luận)

- **Chi phí cài đặt gần như bằng không:** chỉ cần tính entropy từng token (đã có log-prob sẵn trong rollout), chọn ngưỡng theo percentile trong batch, nhân vào advantage và đổi mẫu số chuẩn hóa. Không cần rollout thêm hay mô hình phụ, dễ ghép vào bất kỳ quy trình GRPO/DAPO sẵn có.
- **Hiểu đúng giới hạn:** lợi ích rõ nhất ở mô hình lớn (14B, 32B) trên bài toán có không gian lý luận đa dạng (nhiều cách giải). Với mô hình nhỏ hoặc bài toán gần như tất định, lợi ích có thể bằng 0 hoặc âm nhẹ (giống kết quả Qwen3-8B/AMC'23 trong bài).
- Không nên áp dụng số liệu toán học (ngưỡng 0.672, $\rho = 20\%$) nguyên bản cho OCR; đây chỉ là gợi ý hướng thử nghiệm, bài báo không đề cập OCR hay VLM.

9. Thí nghiệm gợi ý

1. Với một checkpoint OCR (trước hoặc sau RLVR), tính phân phối entropy từng token như Hình 2a; kiểm tra xem bao nhiêu phần trăm token có entropy > 0.672 (ngưỡng của bài) để so sánh độ "lệch" với toán học.
2. Nếu huấn luyện lại GRPO cho OCR: thử $\rho \in \{5\%, 10\%, 20\%\}$, log entropy tổng thể và CER trên tập validation, xem liệu quy tắc "20% tối ưu" của toán học có giữ đúng hay cần ρ nhỏ hơn.
3. Trên một tập văn bản dài sinh bởi mô hình reasoning, gắn nhãn thủ công (hoặc bằng từ khóa như bài này: "wait", "however"...) các forking token rồi so khớp với vị trí entropy cao thực đo, để kiểm chứng mẫu hình 2 trên ngôn ngữ/miền khác tiếng Anh toán học.
4. Kết hợp bộ lọc entropy (bài này) với Clip-Cov/KL-Cov (bài 49) trên cùng một lần chạy GRPO, xem hai bộ lọc trùng nhau bao nhiêu phần trăm token và kết hợp có cộng dồn lợi ích hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Token entropy	$H_t = -\sum_j p_{t,j} \log p_{t,j}$	entropy của phân phối tại t
Forking token	H_t thuộc top 20%	"ngã ba" lý luận
Mẫu hình CoT	$> 50\%$ token $H < 10^{-2}$; 20% token $H > 0.672$	Qwen3-8B, AIME
Mục tiêu lọc entropy	$J_{HighEnt}$: nhân $\mathbb{I}[H_t \geq \tau_\rho^B]$ vào advantage, đổi mẫu số	chỉ 2 thay đổi so với DAPO
Overlap entropy base/RLVR	$\geq 86.67\%$ ở hội tụ (Qwen3-14B)	RLVR giữ mẫu hình base
Kết quả chính	AIME'24/25 Qwen3-32B: 55.83/45.63 → 63.54/56.67	$\rho = 20\%$
Mở rộng response length	20480 → 29696 token: AIME'24 63.54 → 68.12	cùng cấu hình 32B

Khái niệm	Công thức / giá trị	Ghi chú
Nhiệt độ có kiểm soát	$T'_t = T_{high}$ nếu $H_t > 0.672$	xác nhận vai trò forking

11. Kết luận ngắn

Trong CoT của LLM lý luận, chỉ một thiểu số token (khoảng 20%) có entropy cao và đóng vai trò ngã ba định hướng lý luận; RLVR chủ yếu chỉnh entropy của đúng nhóm token này, còn giữ nguyên mẫu hình entropy thấp của phần còn lại. Giới hạn gradient policy gradient chỉ vào 20% token entropy cao nhất giữ hoặc vượt hiệu năng full-token, với lợi ích tăng rõ theo kích thước mô hình (tới +11 điểm AIME'25 ở 32B) trong khi huấn luyện trên 80% token entropy thấp làm hiệu năng sụp. Thông điệp cho người thực hành: không phải mọi token đều đáng để "chi" gradient ngang nhau trong RLVR; biết token nào là forking token (và tỉ lệ phù hợp cho miền dữ liệu cụ thể, không mặc định 20%) có thể là một cách rẻ để cải thiện RLVR.

Link gốc: https://arxiv.org/abs/2506.01939

Bài 51

Reinforcement Learning Finetunes Small Subnetworks in Large Language Models (2025)

Tác giả: Sagnik Mukherjee, Lifan Yuan, Dilek Hakkani-Tür, Hao Peng (University of Illinois Urbana-Champaign). **Venue / arXiv:** arXiv:2505.11711 (v2, 18/12/2025; bản gốc tháng 5/2025). 39th Conference on Neural Information Processing Systems (NeurIPS 2025). **Nguồn:** https://arxiv.org/abs/2505.11711 **Chủ đề:** RL tinh chỉnh LLM thực chất chỉ cập nhật một **subnetwork** rất nhỏ (5-30% tham số), phần còn lại gần như không đổi; subnetwork này gần như giống nhau giữa seed/dữ liệu/thuật toán khác nhau, và chỉ tinh chỉnh riêng subnetwork đó cũng tái tạo được mô hình full tinh chỉnh. **Vị trí trong lộ trình:** Đọc sau bài 00 (PPO, GRPO, DPO), bài 29 (DPO), bài 36 (GRPO/PRIME). Cùng nhóm "phân tích cơ chế RLVR" với bài 49, 50, 52. Đọc cùng lúc hoặc ngay sau bài 52 (RL's Razor): hai bài tranh luận trực tiếp về nguyên nhân thật của hiện tượng "RL thay đổi ít hơn SFT" (xem mục 6 và 7 bên dưới).
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Full fine-tuning (cập nhật mọi tham số, không đóng băng gì) là cách phổ biến để chạy RL (PPO, GRPO, DPO...) sau pretraining, với giả định ngầm: vì hành vi mong muốn (lý luận, alignment, an toàn) khác biệt lớn so với mô hình pretrain, cần thay đổi nhiều tham số. Bài này kiểm tra trực tiếp giả định đó: **RL với full fine-tuning có thực sự cập nhật mọi tham số không?**

Khái niệm nền:

- **Update sparsity:** $\text{sparsity}(\theta_0, \theta_1) := 1 - \|\theta_1 - \theta_0\|_0 / n$, tỉ lệ tham số **không đổi** (dưới một ngưỡng dung sai số học, mặc định 10^{-5} cho bfloat16).
- **Lottery Ticket Hypothesis (LTH, bài 08 bên pruning):** mạng dày chứa mạng con thưa ("winning ticket") mà huấn luyện riêng từ *đúng khởi tạo ban đầu* đạt hiệu năng tương đương mạng đầy đủ.
- **In-distribution data:** dữ liệu huấn luyện gần với phân phối hiện tại của policy (ví dụ rollout on-policy, hoặc SFT trên dữ liệu đã được chính policy đó tạo ra).
- **Rank của ma trận cập nhật:** số chiều độc lập mà $\theta_1 - \theta_0$ (coi như ma trận) thực sự trải ra, so với rank tối đa có thể.

2. Ý tưởng chính

Qua 7 thuật toán RL (PPO, GRPO, ORPO, KTO, DPO, SimPO, PRIME) và 10 LLM khác nhau, tác giả quan sát nhất quán: RL để lại **ít nhất 68.5%** tham số không đổi, trong khi SFT cùng kiểu mô hình chỉ có 6-15% không đổi (SFT cập nhật dày đặc). Hiện tượng này xuất hiện **không cần** bất kỳ regularizer thưa hóa, ràng buộc kiến trúc, hay LoRA nào, mọi mô hình đều train bằng full fine-tuning thông thường. Hơn nữa, update sparse này **gần như full-rank** (không nằm trong không gian con hạng thấp như LoRA), và việc tinh chỉnh riêng **chỉ** subnetwork được xác định sau khi RL xong (đóng băng phần còn lại) tái tạo gần như **chính xác** mô hình full tinh chỉnh (không chỉ hiệu năng, mà cả giá trị tham số). Subnetwork này còn có overlap đáng kể giữa các seed/dữ liệu/thuật toán khác nhau. Nguyên nhân chính được đề xuất: huấn luyện trên dữ liệu **gần phân phối hiện tại của policy** (in-distribution), không phải do KL regularization hay gradient clipping.

3. Phương pháp chi tiết

3.1 Đo sparsity trên checkpoint công khai

So sánh checkpoint trước/sau một giai đoạn (SFT hoặc RL) trên Hugging Face, coi hai giá trị bfloat16 bằng nhau nếu $|\theta_1 - \theta_0| \leq 10^{-5}$ (dung sai mặc định của PyTorch gradient checking).

Bảng 1 (trích, update sparsity theo thuật toán):

Thuật toán	Init → RL model	Sparsity	On-policy	KL	Online
DPO	Tulu-3-8B-SFT → -DPO	81.4%	không	có	không
DPO	Tulu-3-70B-SFT → -DPO	95.2%	không	có	không
GRPO	deepseek-math-7b-instruct → -rl	68.5%	có	có	có
GRPO	DeepSeek v3 base → R1-Zero	86.0%	có	có	có
ORPO	mistral-7B-v0.1 → mistral-orpo-beta	76.9%	không	không	không
KTO	Eurus-7b-sft → -kto	96.0%	không	có	không
PPO	mistral-7b-sft → math-shepherd-rl	80.8%	có	có	có
SimPO	Llama-3-8B-Instruct → -SimPO	86.5%	không	không	không
PRIME	Eurus-2-7b-sft → -PRIME	77.0%	có	không	có

So sánh: SFT (Tulu-3-8B, Tulu-3-70B, Eurus 7B...) chỉ đạt **6-15% sparsity** (Hình 1 của bài báo), tức SFT cập nhật dày đặc hơn RL rất nhiều lần.

3.2 Sparsity theo layer và rank của update

Phân tích theo từng lớp/ma trận tham số (Q, K, V...) của DPO và PRIME: sparsity **tương đối đồng đều** giữa các lớp, chỉ riêng **layer normalization** gần như không được cập nhật. Tính rank trung bình của từng ma trận update, so với rank tối đa có thể:

Bảng 2 (rank của ma trận update, % so với rank tối đa):

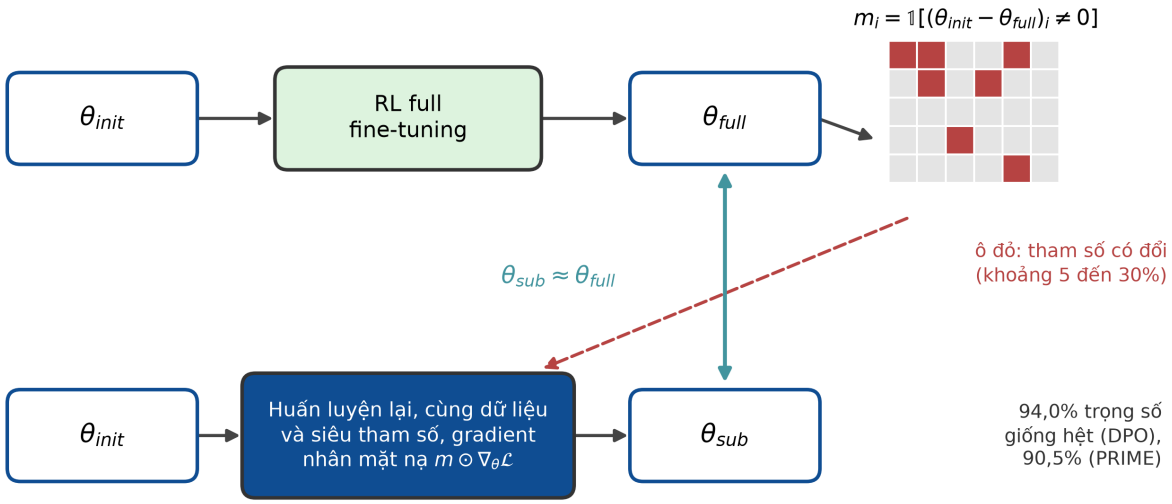
Mô hình, thuật toán	Rank (%)
Tulu 8B (DPO)	99.8
Eurus 7B (PRIME)	99.5
Llama-3 8B (KTO)	99.2
DeepSeek Math 7B (GRPO)	99.4

$\Rightarrow$ update **thưa nhưng gần như full-rank**: khác hẳn LoRA (bài 08 bên llm), nơi update bị ép nằm trong không gian con hạng thấp cố định; ở đây, một **tập con tham số** được chọn để cập nhật, nhưng trong không gian con đó, update vẫn trải khắp các chiều có thể.

3.3 Fine-tune chỉ subnetwork (Conjecture 1)

Gọi θ_{full} là tham số sau RL full fine-tuning từ θ_{init}. Định nghĩa mặt nạ nhị phân $m_i = \mathbb{1}[(\theta_{init} - \theta_{full})_i \neq 0]$. Huấn luyện lại một mô hình thứ hai từ θ_{init}, **cùng** dữ liệu/siêu tham số/số bước, nhưng tại mỗi bước nhân mặt nạ vào gradient trước khi cập nhật: $m \odot \nabla_\theta \mathcal{L}(\theta)$. Kết quả θ_{sub}.

Conjecture 1: $\theta_{sub} \approx \theta_{full}$ (cả về test accuracy và giá trị tham số)



Hình 91: Thí nghiệm mặt nạ của Conjecture 1: lấy mặt nạ từ chênh lệch giữa mô hình khởi tạo và mô hình sau RL, rồi huấn luyện lại chỉ phần được chọn (minh hoạ, tự tính)

Cách đọc hình: hàng trên là lượt RL đầy đủ, từ đó rút ra mặt nạ m (ô đỏ là tham số có thay đổi). Hàng dưới huấn luyện lại từ cùng điểm khởi tạo nhưng gradient bị nhân với m, nên phần ô xám đứng yên. Mũi tên hai đầu màu xanh là điều Conjecture 1 khẳng định: hai kết quả gần như trùng nhau.

3.4 Chỉ số overlap giữa hai subnetwork

Với hai subnetwork có chỉ số cập nhật I_1, I_2 và sparsity s_1, s_2:

$$o_1 = \frac{|I_1 \cap I_2|}{|I_1|} = \frac{|I_1 \cap I_2|}{1 - s_1}, \qquad o_2 = \frac{|I_1 \cap I_2|}{1 - s_2}$$

So với baseline **random guessing** (chọn ngẫu nhiên đúng số lượng tham số tương ứng): kỳ vọng overlap ngẫu nhiên là $O_{1,random} = (1 - s_2)/100$ (tự tính, suy từ Phụ lục E: hai mặt nạ độc lập ngẫu nhiên có xác suất giao đúng bằng tích mật độ của nhau).

4. Thí nghiệm và kết quả chính

RQ2/RQ3 (Bảng 3, tinh chỉnh riêng subnetwork có tái tạo được full tinh chỉnh không): với DPO, 94.0% trọng số **giống hệt nhau** giữa θ_{full} và θ_{sub} (dung sai 10^{-5}); với PRIME, 90.5%. Ở dung sai 10^{-4}, hai mô hình **giống 100%**.

Task (DPO)	θ_{full}	θ_{sub}	Δ
AGIEval LSAT-AR	21.3	24.8	+3.5
AGIEval LSAT-LR	53.1	54.7	+1.6
AGIEval LogiQA-en	43.5	45.5	+2.0
GPQA	32.8	32.8	+0.0
MMLU Pro Math	50.8	51.6	+0.8
Trung bình	40.3	**41.9**	**+1.6**

MATH500 level (PRIME)	θ_{full}	θ_{sub}	Δ
1	93.0	93.0	0.0
2	85.6	85.6	0.0
3	82.9	83.8	+0.9
4	71.1	74.2	+3.1
5	40.3	45.5	+5.2
Toàn bộ	69.8	**72.2**	**+2.4**

θ_{sub} **không hề thua** θ_{full}, thậm chí nhích hơn ở mọi task; loss huấn luyện của θ_{sub} cũng **thấp hơn** θ_{full} một cách nhất quán (Hình 4).

RQ4 (Bảng 4, overlap giữa subnetwork):

Biến đổi	o_1	o_2	Sparsity
Đổi seed (42 vs 123)	60.5%	60.6%	63.3%
Đổi dữ liệu (Tulu vs PRIME data)	26.7%	67.1%	63.3% / 85.4%
Đổi cả seed + dữ liệu + thuật toán (DPO vs PRIME)	59.1%	33.2%	87.1% / 77.0%

Mọi trường hợp đều overlap **cao hơn hẳn** random guessing (baseline ngẫu nhiên tương ứng chỉ bằng $1 - s$), cho thấy subnetwork phần nào mang tính cấu trúc của mô hình pretrain, không phải ngẫu nhiên hoàn toàn.

RQ5 (nguyên nhân, Bảng 5 + Hình 5/6): gradient clipping và KL regularization gần như không ảnh hưởng sparsity (GRPO có KL: 69.8% vs không KL: 68.8%); SFT trước RL không bắt buộc (DeepSeek-R1-Zero không SFT vẫn 86% sparsity sau 8K bước); yếu tố quan trọng nhất là **dữ liệu trong phân phối**:

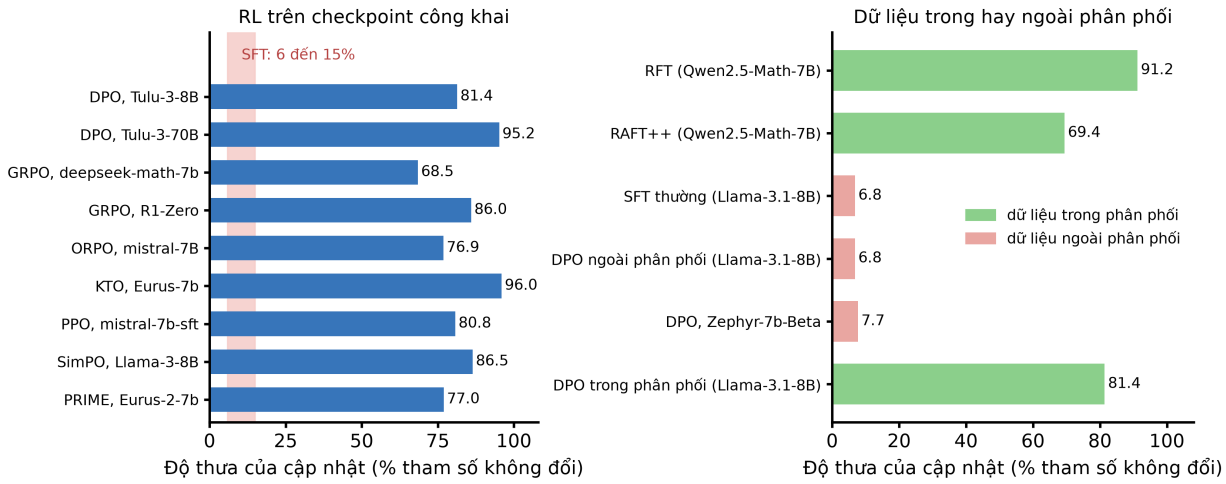
Mô hình	Phương pháp	Sparsity	SFT/RL	In-dist
Qwen2.5-Math-7B	RFT (rejection sampling)	91.2%	SFT	có
Qwen2.5-Math-7B	RAFT++ (lặp RFT)	69.4%	SFT	có
Llama-3.1-8B-SFT	SFT thường	6.8%	SFT	không
Llama-3.1-8B-SFT	DPO (ngoài phân phối)	6.8%	RL	không
Zephyr-7b-Beta	DPO	7.7%	RL	không
Llama-3.1-8B-DPO	DPO (trong phân phối)	81.4%	RL	có

$\Rightarrow$ ngay cả **SFT** trên dữ liệu trong phân phối (rejection sampling) cũng cho sparsity cao; ngay cả **DPO** (RL) trên dữ liệu ngoài phân phối cũng cho update dày đặc. Tính "trong/ngoài phân phối" dự đoán sparsity tốt hơn việc gọi tên thuật toán là SFT hay RL.

Cách đọc hình: panel trái cho thấy mọi lượt RL trên checkpoint công khai đều để lại từ 68,5% đến 96,0% tham số không đổi, trong khi dải đỏ là mức 6 đến 15% của SFT thường. Panel phải đổi tiêu chí phân loại: màu xanh là dữ liệu trong phân phối, màu đỏ là ngoài phân phối. Độ thưa cao đi theo màu xanh, dù thuật toán gọi là SFT hay DPO.

5. Phân tích / ablation

Động học theo số bước (Hình 5, 6, Phụ lục D): sparsity của PRIME **giảm dần** rồi **hội tụ** quanh ~80% (không tiến về 0), trung bình mỗi bước chỉ **~7%** tham số nhận gradient khác 0 so với bước liền trước; khoảng **8.5%** tham số từng được cập nhật tại một thời điểm nào đó rốt cuộc nằm ngoài subnetwork cuối cùng (gradient của



Hình 92: Độ thưa của cập nhật theo thuật toán (Bảng 1) và theo việc dữ liệu có nằm trong phân phối của policy hay không (Bảng 5) (vẽ lại từ số liệu trong ghi chú)

chúng "triệt tiêu" lẫn nhau qua thời gian, không tích lũy net). DeepSeek-R1-Zero huấn luyện 8K bước (20 lần nhiều hơn PRIME) vẫn giữ sparsity cao (86%), nên tác giả không cho rằng đơn giản là "huấn luyện chưa đủ lâu".

Phụ lục A, Hình 2 (phân tích riêng PRIME): trong subnetwork, khoảng **20%** tham số thực sự cập nhật nhất quán, **8%** nhận gradient khác 0 nhưng triệt tiêu, **72%** hoàn toàn không đổi suốt quá trình.

6. Giới hạn

Tác giả nêu: chỉ thay đổi một yếu tố mỗi lần (không kiểm soát đầy đủ tương tác nhiều yếu tố); không thể chạy ablation đầy đủ vì RL tốn tính toán, nên một phần dựa vào checkpoint công khai không rõ hết cấu hình; chỉ thử trên LLM văn bản, chưa thử đa phương thức/diffusion; tác giả còn tự trích dẫn khả năng gây nhiễu từ ProRL (Liu et al. 2025a, **chính là bài 46** trong lộ trình này) như một yếu tố chưa loại trừ hết (huấn luyện rất dài có thể thay đổi kết luận).

Người ghi chú nhận xét, và điểm tranh luận quan trọng với bài 52: bài 52 (RL's Razor, đọc ngay sau) kiểm tra lại chính giả thuyết này và phát hiện: lý do quan sát được sparsity trong thực nghiệm của họ là do **giới hạn độ chính xác số học của bfloat16** (mantissa hữu hạn khiến cập nhật rất nhỏ, như của RL, không vượt ngưỡng biểu diễn được và bị "làm tròn về 0"); khi huấn luyện bằng **float32**, bài 52 không còn thấy sparsity nào ở các mô hình họ thử, dù hiệu năng giống nhau. Bản thân bài 51 cũng đã **chủ động thừa nhận và lập luận trước** về khả năng này (chú thích 5: "nếu backprop chính xác tuyệt đối trên giấy, update sẽ dày đặc; sparsity quan sát được là do dấu phẩy động"), nhưng vẫn giữ kết luận vì theo họ *không* giải thích được tại sao SFT (cũng chạy bfloat16) lại luôn dày đặc còn RL luôn thưa, nếu chỉ là hiệu ứng làm tròn đơn thuần không liên quan gì đến thuật toán. Đây là một tranh luận **chưa ngã ngũ** giữa hai bài cùng lộ trình: bfloat16 có thể là nguyên nhân *trực tiếp cơ chế* (update nhỏ bị làm tròn), nhưng câu hỏi *vì sao* update của RL (in-distribution) lại nhỏ hơn SFT (out-of-distribution) ngay từ đầu vẫn cần lời giải, và cả hai bài đều đồng thuận ở điểm này: **dữ liệu trong phân phối** khiến cập nhật cần thiết nhỏ hơn.

- Ngưỡng dung sai 10^{-5} (và cả 10^{-4} dùng để nói "giống 100%") là lựa chọn chủ quan; Phụ lục của bài có thử các ngưỡng khác (Bảng 6) nhưng không trình bày đầy đủ trong phần chính.
- Kết luận "LoRA ép hạng thấp, còn đây là chọn tập con nhưng vẫn full-rank" dựa trên **rank trung bình** của ma trận update; không rõ liệu các chiều "không được chọn" có mang thông tin bị bỏ sót quan trọng cho tác vụ khác không.

7. Liên hệ với các bài khác

- **Bài 00, 07 (PPO), 29 (DPO), 36 (GRPO/PRIME):** các thuật toán được đo sparsity trực tiếp.
- **Pruning 08 (Lottery Ticket Hypothesis):** Conjecture 1 mở rộng trực tiếp từ LTH nhưng khác ba điểm mà chính tác giả nêu: (1) LTH tìm ticket bằng cắt theo độ lớn rồi reset về khởi tạo, còn ở đây subnetwork **tự xuất hiện tự nhiên** từ RL, không cần thuật toán tìm kiếm; (2) LTH chỉ chứng minh lại được *hiệu năng*, còn ở đây tái tạo gần như *chính xác giá trị tham số*; (3) LTH huấn luyện từ đầu (from scratch), còn đây là tinh chỉnh từ mô hình đã pretrain.
- **Llm 08 (LoRA):** đối lập về "kiểu" hiệu quả tham số: LoRA ép cập nhật vào không gian con hạng thấp cố định trước khi huấn luyện; RL "tự chọn" một tập con tham số nhưng cập nhật trong đó vẫn gần full-rank. Hai cách tiết kiệm tham số khác nhau về bản chất.
- **Pruning 16 (Movement Pruning):** cùng ý tưởng "trọng số nào di chuyển nhiều khi fine-tune mới quan trọng", nhưng Movement Pruning *chủ động* dùng độ di chuyển làm tiêu chí cắt, còn bài này *quan sát* độ di chuyển tự nhiên phát sinh từ RL mà không can thiệp.
- **Bài 46 (ProRL):** được chính tác giả bài này trích dẫn như một nguồn nhiễu khả dĩ (huấn luyện rất dài, chưa rõ sparsity có giảm thêm không).
- **Bài 52 (RL's Razor):** tranh luận trực tiếp về nguồn gốc sparsity (xem mục 6); cả hai bài đồng ý RL/dữ liệu trong phân phối làm thay đổi tham số ít hơn SFT/dữ liệu ngoài phân phối, nhưng khác nhau ở cơ chế đề xuất (sparsity do bfloat16 vs sparsity là thật) và ở cách đo (tham số thô vs KL-divergence đầu ra).

8. Ứng dụng thực tế (suy luận)

Cần phân biệt độ thưa của cập nhật với cắt tỉa cấu trúc. Trong nghiên cứu này, kiến trúc được giữ nguyên và chỉ một phần tham số thay đổi đáng kể; cắt tỉa cấu trúc loại bỏ các thành phần của mạng. Vì vậy, không thể dùng trực tiếp mạng con quan sát được để quyết định cần cắt bộ phận nào.

Kết quả cũng gợi ý khảo sát fine-tuning trên tập tham số được chọn, nhưng lợi ích về bộ nhớ và khả năng khái quát cần được đo trong từng thiết lập. Bài báo không trực tiếp thử phục hồi mô hình sau cắt tỉa.

9. Thí nghiệm gợi ý

1. Đo độ thưa của cập nhật với nhiều ngưỡng và cả bfloat16 lẫn float32; đối chiếu cách giải thích trong bài 52.
2. Xác định mạng con theo mặt nạ m, sau đó tinh chỉnh riêng mạng con ấy và so sánh với fine-tuning toàn bộ.
3. Thử đóng băng các tham số layer normalization và đo ảnh hưởng tới chất lượng cuối.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Update sparsity	$1 - \|\theta_1 - \theta_0\|_0/n$	dung sai 10^{-5} (bf16)
Sparsity RL	68.5%-96.0%	7 thuật toán, 10 mô hình
Sparsity SFT	6%-15%	dày đặc hơn RL rõ rệt
Rank update	≈99.2-99.8% rank tối đa	thưa nhưng gần full-rank
Conjecture 1	$\theta_{sub} \approx \theta_{full}$	mask $m_i = \mathbb{1}[(\theta_{init} - \theta_{full})_i \neq 0]$
Overlap	$o_1 = \|I_1 \cap I_2\|/(1 - s_1)$	so với random guessing
Cấu tạo subnetwork (PRIME)	20% cập nhật nhất quán, 8% triệt tiêu, 72% không đổi	Hình 2

Khái niệm	Công thức / giá trị	Ghi chú
Nguyên nhân chính	dữ liệu trong phân phối (in-distribution)	không phải KL/clipping
Kết quả chính	θ_{sub} bằng hoặc hơn θ_{full} trên mọi task thử	Bảng 3a/3b

11. Kết luận ngắn

RL full fine-tuning trên LLM không thực sự cập nhật mọi tham số: chỉ 5-30% tham số (một subnetwork khá nhất quán giữa seed/dữ liệu/thuật toán, và gần như full-rank trong phần được chọn) thực sự thay đổi, phần còn lại gần như đứng yên; tinh chỉnh riêng subnetwork đó tái tạo gần như chính xác mô hình full tinh chỉnh. Nguyên nhân chính được đề xuất là huấn luyện trên dữ liệu gần phân phối hiện tại của policy, không phải các cơ chế ổn định như KL hay gradient clipping. Khả năng tận dụng độ thưa để tiết kiệm tài nguyên là một giả thuyết cần kiểm chứng; bài báo không trực tiếp đánh giá phục hồi sau cắt tỉa.

Link gốc: https://arxiv.org/abs/2505.11711

Bài 52

RL's Razor: Why Online Reinforcement Learning Forgets Less (2025)

Tác giả: Idan Shenfeld, Jyothish Pari (đóng góp bằng nhau), Pulkit Agrawal (Improbable AI Lab, MIT).
Venue / arXiv: arXiv:2509.04259 (v1, 04/09/2025). Bài báo không nêu venue chính thức (preprint). **Nguồn:** https://arxiv.org/abs/2509.04259 **Chủ đề:** RL tinh chỉnh LLM/robot quên ít hơn SFT ở cùng mức hiệu năng tác vụ mới; mức độ quên được dự đoán chính xác bởi KL-divergence thuận (forward KL) giữa policy đã tinh chỉnh và policy gốc, đo trên chính tác vụ mới; lý thuyết giải thích vì sao policy gradient on-policy tự nhiên thiên về nghiệm KL-minimal. **Vị trí trong lộ trình:** Đọc sau bài 00 (policy gradient, KL penalty), bài 07 (PPO), bài 29/31/32 (DPO/KTO/SimPO, dùng làm baseline offline ở đây). Đọc cùng nhóm với bài 49-51 (phân tích cơ chế RLVR); đặc biệt đọc **sau bài 51** vì bài này trực tiếp kiểm tra lại một phần giả thuyết của 51 (xem mục 6, 7). Đây là bài cuối của nhóm 4 bài phân tích cơ chế (49-52) trong giai đoạn này.
Chương nền liên quan: nền 08: RL cho mô hình lập luận

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Mô hình nền tảng (foundation model) lý tưởng nên học thêm kỹ năng mới liên tục mà không **quên thảm khốc** (catastrophic forgetting) những gì đã biết. Các cách giảm quên truyền thống (EWC, ràng buộc đặc trưng, regularize output...) xử lý *triệu chứng* chứ không chỉ ra *nguyên nhân* khiến các thuật toán khác nhau quên khác nhau. Bài này so sánh trực tiếp SFT và RL ở cùng mức hiệu năng tác vụ mới, rồi tìm một biến số duy nhất giải thích mức độ quên.

Khái niệm nền:

- **SFT loss:** $\mathcal{L}_{SFT}(\pi) = -\mathbb{E}_{x\sim D,\, y\sim\pi_\beta}[\log \pi(y \mid x)]$, học từ nhãn cố định π_β (bên ngoài).
- **RL loss (policy gradient):** $\mathcal{L}_{RL}(\pi) = -\mathbb{E}_{x\sim D,\, y\sim\pi}[A(x,y)\log \pi(y \mid x)]$, mẫu y lấy từ **chính policy đang học**.
- **Forward KL:** $\mathrm{KL}(\pi_0\|\pi) = \mathbb{E}_{y\sim\pi_0} \log \frac{\pi_0(y)}{\pi(y)}$ (khác reverse KL $\mathrm{KL}(\pi\|\pi_0)$, xem bài 28 MiniLLM bên distillation).
- **I-projection / M-projection (information geometry):** chiếu một phân phối lên một tập ràng buộc sao cho tối thiểu hóa KL theo một chiều cố định; I-projection tối thiểu $\mathrm{KL}(q\|p)$ (ứng với rejection sampling), M-projection tối thiểu $\mathrm{KL}(q\|\pi)$ trong họ Π (ứng với một bước policy gradient).

2. Ý tưởng chính

Thực nghiệm trên LLM (toán, Q&A khoa học, dùng công cụ) và robot (gắp vật) cho thấy: *ở cùng độ chính xác tác vụ mới*, **RL giữ lại hiệu năng tác vụ cũ cao hơn SFT rất rõ** (Hình 2). Tìm biến số dự đoán được mức quên qua nhiều biến thử (thay đổi trọng số, thay đổi biểu diễn, sparsity/rank, khoảng cách phân phối), chỉ một biến vượt trội: **forward KL giữa policy sau tinh chỉnh và policy gốc, đo trên tác vụ mới** $\mathbb{E}_{x\sim\tau}\mathrm{KL}(\pi_0\|\pi)$. Biến này dự đoán mức quên **như nhau cho cả RL và SFT** khi vẽ trên cùng một đồ thị (Hình 3 giữa): không phải thuật toán nào quyết định mức quên, mà là khoảng cách phân phối. **RL's Razor:** trong số nhiều nghiệm cùng giải được tác vụ mới, cập nhật on-policy (lấy mẫu từ chính policy) tự nhiên thiên về nghiệm **gần policy gốc nhất theo KL**, vì mỗi bước chỉ tái cân trọng số trên các mẫu policy *đã* cho xác suất không nhỏ, còn SFT có thể bị kéo tới bất kỳ phân phối nào mà nhãn bên ngoài chỉ định, xa tùy ý.

3. Phương pháp chi tiết

3.1 Bốn lớp thuật toán theo hai tiêu chí

	Không dùng negative examples	Có dùng negative examples
On-policy	1-0 REINFORCE ($A = 1$ nếu đúng, 0 nếu sai; tương đương SFT trên câu trả lời đúng do chính model sinh ra)	GRPO (advantage chuẩn hóa, có cả dấu âm)
Offline	SFT	SimPO: $\mathcal{L}_{SimPO} = -\mathbb{E}[\log \sigma(\log \pi(y_w) - \log \pi(y_l) - 1)]$

Thí nghiệm trên Science Q&A cho thấy: 1-0 REINFORCE hành xử giống GRPO (ít quên), SimPO hành xử giống SFT (quên nhiều); **yếu tố quyết định là on-policy hay không, không phải có negative example hay không** (Hình 4).

3.2 Lý thuyết: RL như một thuật toán EM với I-projection

Với reward nhị phân $R : \mathcal{Y} \to \{0, 1\}$:

Bổ đề 5.1 (rejection sampling = I-projection): rejection sampling từ p với điều kiện nhận $R(y) = 1$ cho ra phân phối $q_{RS} = p(\cdot \mid R = 1)$, và đây **chính là** nghiệm của

$$q_{RS} = \arg\min_q \mathrm{KL}(q\|p) \quad \text{s.t.} \quad \mathbb{E}_{y\sim q}[R(y)] = 1$$

(Chứng minh: với q hỗ trợ trên $S = \{y : R(y) = 1\}$, $\mathrm{KL}(q\|p) = \mathrm{KL}(q\|p(\cdot \mid S)) - \log p(S)$; số hạng thứ hai là hằng số, nên tối thiểu $\mathrm{KL}(q\|p)$ tương đương tối thiểu $\mathrm{KL}(q\|p(\cdot \mid S))$, đạt 0 tại $q = p(\cdot \mid S) = q_{RS}$ nhờ tính lồi chặt của KL.)

Bổ đề về M-projection: lấy một bước policy gradient trên mục tiêu RL tương đương lấy một bước gradient trên $\mathrm{KL}(q\|\pi)$ với $q(y) = \pi(y)R(y)/Z$ (phân phối re-weight theo reward), tức chính là **M-projection** của π hiện tại lên tập Π.

Định lý 5.2 (hội tụ về nghiệm KL-minimal): với $\Pi \subseteq \Delta(\mathcal{Y})$ là một họ lồi (ví dụ họ hàm mũ), $P^* = \{q : \mathbb{E}_q[R] = 1\}$ tập nghiệm tối ưu, dưới điều kiện chính quy phù hợp, policy gradient hội tụ về

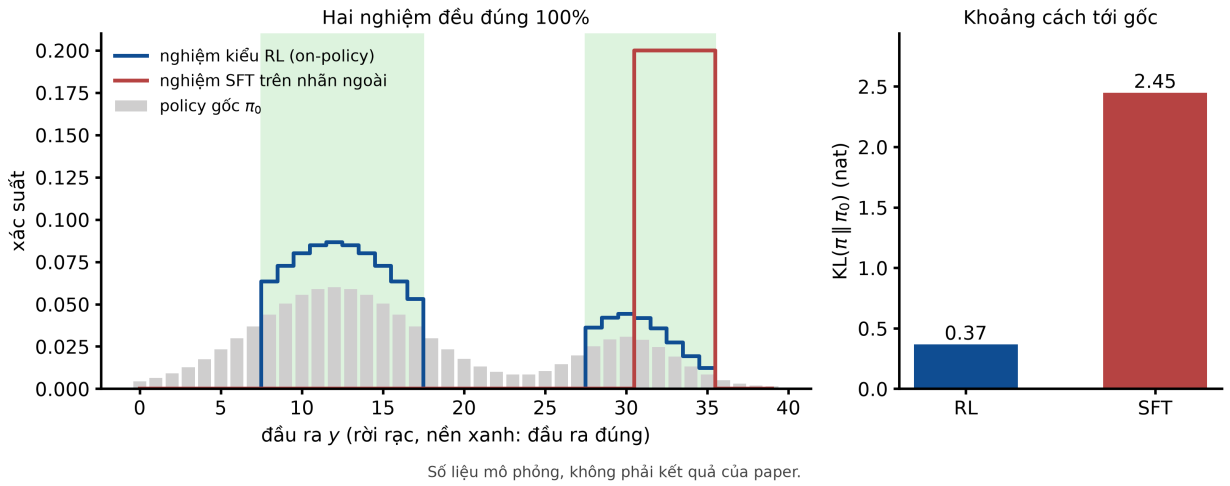
$$\boxed{\pi^\dagger = \arg\min_{\pi \in P^* \cap \Pi} \mathrm{KL}(\pi\|\pi_0)}$$

tức là **trong số mọi nghiệm tối ưu biểu diễn được, policy gradient chọn nghiệm gần policy khởi tạo nhất theo KL**. (Chứng minh đầy đủ bằng cách xen kẽ I-projection (bước sample/reject) và M-projection (bước gradient), giống thuật toán EM, với I-projection luôn chính xác còn M-projection có thể chỉ gần đúng (GEM); xem Phụ lục A của bài báo.)

Cách đọc hình: cột xám là policy gốc π_0 trên 40 đầu ra rời rạc, nền xanh đánh dấu các đầu ra đúng. Đường xanh dương là nghiệm kiểu RL, chỉ tái cân π_0 trên tập đúng; đường đỏ là nghiệm SFT dồn hết xác suất vào một nhãn ngoài, cũng đúng nhưng xa π_0. Panel phải đo $\mathrm{KL}(\pi\|\pi_0)$ của hai nghiệm; số liệu là mô phỏng với seed cố định, không phải kết quả của paper.

3.3 Xây dựng "oracle SFT" để kiểm chứng

Trên bài toán đồ chơi ParityMNIST (dự đoán chẵn/lẻ của chữ số MNIST, nhiều nhãn số cụ thể đều đúng miễn đúng tính chẵn lẻ), vì bài toán đơn giản, tác giả **giải tích** được phân phối nhãn tối thiểu hóa KL với mô hình



Hình 93: Mô phỏng đồ chơi: hai nghiệm cùng đúng 100% nhưng khác khoảng cách KL tới policy gốc (minh hoạ, tự tính)

gốc trong số mọi phân phối đạt 100% chính xác (do KL lồi, ràng buộc tuyến tính $\Rightarrow$ có nghiệm dạng đóng, xem Phụ lục B.3). SFT trên chính phân phối oracle này được dùng để kiểm tra trực tiếp "nếu SFT cũng nhắm đúng nghiệm KL-minimal, nó có quên ít như RL (hoặc ít hơn) không?"

4. Thí nghiệm và kết quả chính

4 miền: LLM Toán (Qwen2.5-3B-Instruct, dữ liệu Open-Reasoner-Zero), LLM Khoa học Q&A (Qwen2.5-3B-Instruct, SciKnowEval Chemistry L-3), LLM Dùng công cụ (Qwen2.5-3B-Instruct, ToolAlpaca), Robot gắp-đặt (OpenVLA-7B, SimplerEnv, nhiệm vụ gắp lon). Đo hiệu năng tác vụ cũ trên Hellaswag, TruthfulQA, MMLU, IFEval, Winogrande, HumanEval (LLM) hoặc mở/đóng ngăn kéo (robot). RL dùng GRPO, chỉ phần thưởng đúng/sai nhị phân, **không** KL regularization rõ ràng. Mỗi phương pháp quét hàng chục cấu hình siêu tham số, lấy **Pareto frontier** giữa hiệu năng tác vụ mới và tác vụ cũ.

Hình 2 (Pareto frontier): trên cả 4 miền, khi tăng hiệu năng tác vụ mới, **RL giữ hiệu năng tác vụ cũ gần như không đổi**, còn **SFT đánh đổi**: tăng tác vụ mới luôn kèm giảm tác vụ cũ, rõ nhất ở Toán (chỉ cần một chút tăng ở tác vụ mới đã làm giảm mạnh tác vụ cũ).

ParityMNIST (Hình 3): lặp lại được khoảng cách RL/SFT ở quy mô nhỏ, hội tụ đầy đủ. Vẽ forgetting theo KL-divergence: cả RL và SFT **rơi trên cùng một đường cong duy nhất**, fit bậc hai đạt $R^2 = 0.96$; trên LLM thật, $R^2 = 0.71$ (yếu hơn do nhiễu ước lượng KL/accuracy, nhưng residual trung bình 0, Hình 11 phụ lục). Lặp lại với **hai cách gán nhãn SFT khác nhau**: Pareto frontier khác nhau nhưng đường cong forgetting-KL **trùng khớp**, càng xác nhận KL là biến quyết định, không phải nhãn cụ thể.

Oracle SFT (Hình 3 trái): SFT trên phân phối tối thiểu KL đạt **trade-off tốt nhất quan sát được**, vượt cả RL. Thêm thí nghiệm: SFT trên dữ liệu **do chính một mô hình đã RL sinh ra** (distillation từ RL teacher) tái tạo đúng trade-off của RL (Hình 9: SFT student khớp RL teacher trong khoảng nhiễu, cả về accuracy tác vụ mới và FashionMNIST). $\Rightarrow$ **không phải bản thân thuật toán RL**, mà **phân phối đích** mới là thứ quyết định mức quên.

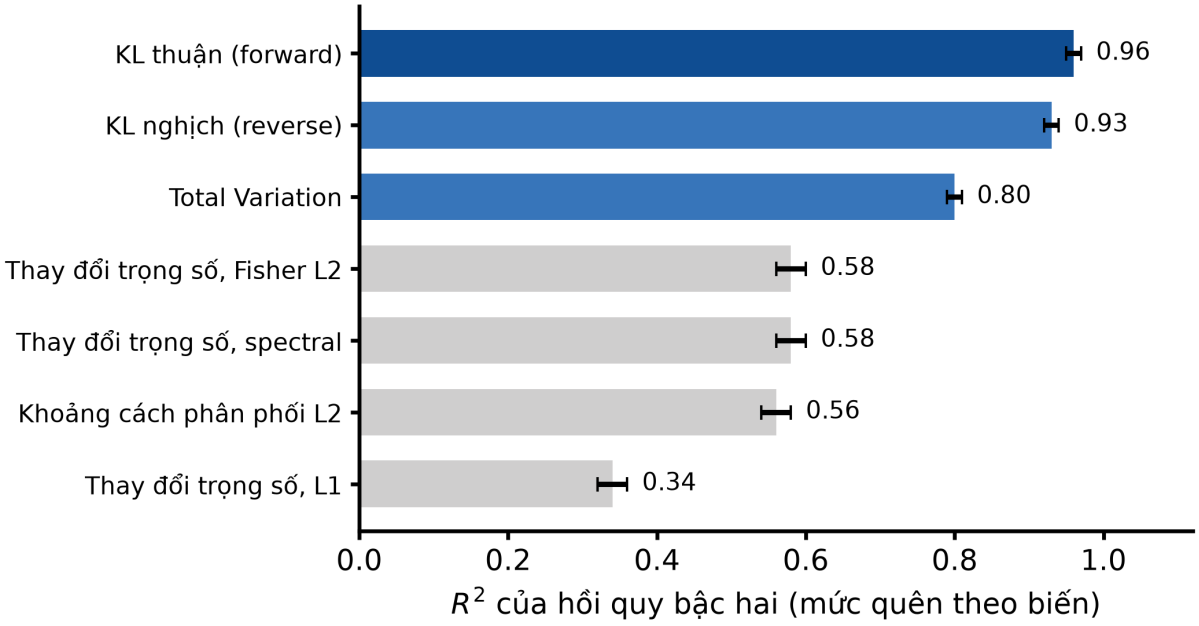
Lớp thuật toán (Hình 4): GRPO và 1-0 REINFORCE (cả hai on-policy) đạt accuracy tương đương với KL-shift **nhỏ hơn hẳn** SimPO và SFT (offline) ở cùng mức accuracy.

5. Phân tích / ablation

Bảng 1 (sức dự đoán của các biến thay thế, R^2 fit bậc hai, MNIST):

Biến	R^2
KL thuận (forward)	$\mathbf{0.96 \pm 0.01}$
KL nghịch (reverse)	0.93 ± 0.01
Total Variation	0.80 ± 0.01
Khoảng cách phân phối L2	0.56 ± 0.02
Thay đổi trọng số, L1	0.34 ± 0.02
Thay đổi trọng số, Fisher-weighted L2	0.58 ± 0.02
Thay đổi trọng số, spectral norm	0.58 ± 0.02
Sparsity của update trọng số	không áp dụng
Rank của update trọng số	không áp dụng
Thay đổi activation, L1/L2	0.52 / 0.55

Điểm đáng chú ý nhất (liên hệ trực tiếp bài 51): tác giả thử lại giả thuyết "RL update thưa, SFT update dày" (trích rõ Mukherjee et al. 2025, chính là bài 51), và phát hiện nguyên nhân sparsity trong thiết lập của họ là do **bfloat16**: mantissa hữu hạn khiến các update rất nhỏ (như của RL) bị làm tròn về 0 và không được biểu diễn; train lại bằng **float32** cho hiệu năng giống nhau nhưng **không còn sparsity nào**; rank của update luôn full-rank ở mọi thuật toán thử. Vì vậy sparsity/rank không được dùng làm biến dự đoán số liệu ("N/A" trong bảng).



Hình 94: Sức dự đoán mức quên (R^2 của hồi quy bậc hai) của từng biến, theo Bảng 1 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi thanh là một biến dùng để dự đoán mức quên trên MNIST, thanh càng dài thì biến đó giải thích mức quên càng tốt. KL thuận vượt xa các biến đo thay đổi trọng số. Các thanh sparsity và rank không có mặt vì bảng ghi chúng là không áp dụng.

CKA/CKNNA (biểu diễn, Hình 7): so sánh hình học embedding (Wikipedia, không liên quan tác vụ tinh chỉnh) giữa mô hình gốc và mô hình đã tinh chỉnh ở **cùng** accuracy tác vụ mới: RL giữ CKNNA ≈ 0.94 (gần không đổi), SFT chỉ còn ≈ 0.56 (trôi mạnh).

Theo quy mô mô hình (Hình 8, Qwen2.5 3B/7B/14B): trade-off SFT accuracy-mới/accuracy-cũ **không thu hẹp** theo kích thước, trái với một số công trình trước cho rằng mô hình lớn ít quên hơn.

Động học tối ưu (Hình 10): tương quan dương giữa KL-shift mỗi bước và độ tương đồng cosine giữa update đó với "hướng gây quên" (gradient loss trên tác vụ cũ): bước gây dịch KL lớn cũng là bước gây quên nhiều.

6. Giới hạn

Tác giả nêu: chưa có giải thích **cơ chế** vì sao KL lớn trên tác vụ mới lại phá hoại kiến thức cũ (can thiệp biểu diễn? giới hạn dung lượng ngầm?); chưa kiểm chứng ở quy mô biên giới (frontier scale) hay miền đa dạng hơn; không thử các thuật toán **online nhưng off-policy**; lý thuyết giả định họ hàm mũ (e-flat) cho Π nhưng mạng neural thực tế không đúng dạng đó, nên Định lý 5.2 chỉ mang tính "giải thích có nguyên tắc", không phải chứng minh chặt cho mạng neural. Tác giả còn chỉ ra một công trình cùng thời (Lai et al. 2025) kết luận RL quên ít hơn SFT nhưng **gán nguyên nhân cho negative examples**, trái với kết quả Hình 4 của bài này (yếu tố quyết định là on-policy, không phải dấu của advantage); đây là một tranh luận **chưa ngã ngũ hoàn toàn** trong tài liệu.

Nhận xét:

- Mọi "tác vụ mới" trong bài đều có phần thưởng gần như **nhị phân** (đúng/sai một bài toán, định dạng tool call đúng/sai, gắp được/không) nên tập nghiệm tối ưu P^* khá rõ ràng; chưa rõ RL's Razor có còn đúng với mô hình phần thưởng học được, có nhiều mức điểm liên tục (như RLHF cổ điển) hay không.
- $R^2 = 0.71$ trên LLM thật yếu hơn hẳn $R^2 = 0.96$ trên MNIST đồ chơi; bài quy cho nhiễu đo KL/accuracy nhưng không loại trừ khả năng còn biến số khác đóng vai trò ở quy mô LLM thực.
- Thí nghiệm robot chỉ có **một** tác vụ mới (gắp lon) và một cặp tác vụ cũ (mở/đóng ngăn kéo); kết luận tổng quát cho robot còn mỏng so với LLM.

7. Liên hệ với các bài khác

- **Bài 00, 07 (PPO):** nền cho policy gradient, KL penalty; bài này giải thích vì sao PPO/GRPO *tự* có xu hướng gần policy gốc dù không cần thêm KL penalty rõ ràng.
- **Bài 29 (DPO), 31 (KTO), 32 (SimPO):** SimPO dùng làm đại diện "offline + negative examples" trong Hình 4; kết quả cho thấy cùng là offline, SimPO vẫn quên nhiều như SFT dù có dùng negative examples, nên các phương pháp preference-offline này không tự động có lợi thế "giữ kiến thức cũ" như RL on-policy.
- **Llm 07 (InstructGPT):** PPO-ptx (trộn loss pretraining) là một cách *tường minh* để giảm quên; bài 52 cho một lời giải thích khác: ngay cả PPO thường (không ptx) cũng đã quên ít hơn SFT nhờ cơ chế KL-minimal nội tại, PPO-ptx là lớp bảo vệ bổ sung.
- **Bài 51 (RL finetunes small subnetworks):** tranh luận trực tiếp (xem mục 6); cả hai đồng thuận RL thay đổi mô hình ít hơn SFT ở cùng hiệu năng, nhưng khác cơ chế đề xuất (subnetwork rời rạc do bfloat16 và dữ liệu trong phân phối, so với khoảng cách KL liên tục do bản chất on-policy của cập nhật).
- **Distillation 28 (MiniLLM), 29 (GKD):** cả hai dùng **on-policy sampling** (rollout từ student) khi distill, với lý do tương tự: tránh exposure bias/giữ student gần phân phối của chính nó. RL's Razor cho một khung lý thuyết tổng quát hơn: on-policy sampling (không chỉ trong RL mà cả trong distillation) tự nhiên thiên về nghiệm gần policy gốc theo KL, nên các phương pháp này "quên ít hơn" KD offline không phải do may rủi mà có nền tảng toán học chung.
- **Distillation 47 (ShortOPD):** phương pháp dùng rollout của student và tín hiệu từ teacher; liên hệ với lập luận về dữ liệu on-policy ở đây là một cách diễn giải, cần kiểm chứng riêng về khả năng giữ kiến thức.

8. Ứng dụng thực tế (suy luận)

Khi cần học thêm một kỹ năng mà vẫn giữ năng lực cũ, kết quả gợi ý so sánh dữ liệu do chính mô hình sinh với dữ liệu từ nguồn ngoài. Cần đo cả hiệu năng trên tác vụ mới, khả năng giữ các tác vụ cũ và độ lệch phân phối so với mô hình ban đầu; không nên giả định một phương pháp luôn tránh được quên.

9. Thí nghiệm gợi ý

1. So sánh RL on-policy và SFT ở mức hiệu năng tương đương trên tác vụ mới; đo KL và hiệu năng trên các tác vụ cũ.
2. So sánh SFT trên dữ liệu được lọc từ rollout của mô hình với SFT trên dữ liệu ngoài.
3. Đo độ tương đồng biểu diễn trước và sau huấn luyện, theo cách phân tích ở Hình 7.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Luật quên thực nghiệm	mức quên $\approx f(\mathbb{E}_{x\sim\tau}\mathrm{KL}(\pi_0\|\pi))$	cùng đường cong cho RL và SFT
RL's Razor	$\pi^\dagger = \arg\min_{\pi\in P^*\cap\Pi}\mathrm{KL}(\pi\|\pi_0)$	nghiệm tối ưu gần gốc nhất theo KL
I-projection (rejection sampling)	$q_{RS} = \arg\min_q \mathrm{KL}(q\|p)$ s.t. $\mathbb{E}_q[R]=1$	Bổ đề 5.1
M-projection (1 bước PG)	chiếu $\pi\, R/Z$ lên Π	tương đương gradient step
Oracle SFT	nhãn tối thiểu KL với π_0, 100% đúng	vượt cả RL về giữ kiến thức cũ
Biến dự đoán tốt nhất	forward KL, $R^2 = 0.96$ (MNIST) / 0.71 (LLM)	hơn hẳn weight/activation change
Sparsity (phản biện bài 51)	biến mất khi dùng float32	hiệu ứng làm tròn bfloat16
CKA biểu diễn	RL 0.94 vs SFT 0.56	cùng accuracy tác vụ mới

11. Kết luận ngắn

Mức độ quên thảm khốc khi tinh chỉnh không do thuật toán (RL hay SFT) quyết định, mà do khoảng cách KL-divergence thuận giữa mô hình sau tinh chỉnh và mô hình gốc, đo trên tác vụ mới; RL thường quên ít hơn SFT đơn giản vì bản chất on-policy của nó (lấy mẫu từ chính mình rồi tái cân theo thưởng) tự nhiên thiên về nghiệm gần gốc nhất trong số các nghiệm giải đúng tác vụ mới, một kết luận có nền tảng lý thuyết (RL như EM với I/M-projection) và được xác nhận bằng oracle-SFT vượt cả RL khi SFT cũng nhắm đúng mục tiêu KL-minimal. Với distillation on-policy (MiniLLM, GKD, và ShortOPD), bài này gợi ý một cách lý giải có thể kiểm chứng về tác động của việc học từ rollout của chính mô hình.

Link gốc: https://arxiv.org/abs/2509.04259

Bài 53

Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning (2025)

Tác giả: Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Ö. Arık, Dong Wang, Hamed Zamani, Jiawei Han (University of Illinois Urbana-Champaign; University of Massachusetts Amherst; Google Cloud AI Research).
Venue / arXiv: arXiv:2503.09516 (v5, 05/08/2025); văn bản ghi "Published as a conference paper at COLM 2025" (Conference on Language Modeling 2025).
Nguồn: https://arxiv.org/abs/2503.09516
Chủ đề: RL đa lượt (multi-turn) cho LLM tự quyết định khi nào gọi search engine trong lúc suy luận, xen kẽ suy luận và truy xuất bằng token đặc biệt, huấn luyện bằng PPO hoặc GRPO có che mất mát (loss masking) cho token truy xuất, phần thưởng chỉ dựa trên kết quả cuối (outcome reward).
Vị trí trong lộ trình: Đọc sau bài 00 (PPO, GRPO, KL penalty), bài 07 (PPO gốc) và bài 36 (DeepSeekMath, GRPO gốc), bài 39 (DeepSeek-R1, RLVR với phần thưởng rule-based). Search-R1 tự nhận là mở rộng của DeepSeek-R1-Zero sang bài toán có **tool dùng ngoài** (search engine), không chỉ suy luận nội tại. Cùng nhóm "agent" với bài 54 (khảo sát Agentic RL), nơi Search-R1 được liệt kê như ví dụ "Search & Research Agent, external search".

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Có hai cách chính để LLM dùng kiến thức bên ngoài: **RAG** (truy xuất đoạn văn theo câu hỏi, nhét vào context rồi sinh câu trả lời; có thể lặp nhiều vòng bằng prompting như IRCoT, nhưng mô hình không được huấn luyện để tương tác tối ưu với search engine), và **tool use** (LLM được prompt/SFT để tự gọi search khi cần, như ReAct, Toolformer; cần dữ liệu quỹ đạo chất lượng cao, tốn công gán nhãn, khó mở rộng quy mô).

RL đã cho thấy hiệu quả tăng khả năng suy luận của LLM chỉ với outcome reward: OpenAI-o1, DeepSeek-R1 (bài 39) dùng PPO (bài 07)/GRPO (bài 36), học được self-verification, self-correction chỉ từ outcome reward. Áp RL cho "suy luận + search" gặp ba thách thức tác giả nêu: (1) chưa rõ cách đưa search engine vào vòng lặp RL mà vẫn ổn định khi phải xử lý nội dung truy xuất chèn vào rollout; (2) mô hình lý tưởng phải tự quyết định khi nào search, bao nhiêu lần, tùy độ khó câu hỏi; (3) chưa rõ outcome reward đơn giản có đủ dạy hành vi search hợp lý hay không.

Khái niệm nền: **rollout** (chuỗi hành động/token hoàn chỉnh, gồm cả suy luận, lệnh search, nội dung truy xuất, dùng để tính phần thưởng); **loss masking** (chỉ tính gradient trên một tập con token, ở đây bỏ token search engine trả về); **Exact Match (EM)** (1 nếu câu trả lời khớp đáp án vàng, 0 nếu không); **GAE** (bài 05, cách ước lượng advantage trong PPO, cần value function/critic riêng).

2. Ý tưởng chính

Search-R1 coi **search engine là một phần của environment**: rollout của policy không còn là một chuỗi văn bản liên tục do một mình LLM sinh, mà xen kẽ giữa đoạn LLM tự sinh (bọc trong token đặc biệt `<think>`, `<search>`, `<answer>`) và đoạn do search engine trả về (bọc trong `<information>`), ký hiệu $y \sim \pi_\theta(\cdot \mid x; R) = \pi_\theta(\cdot \mid x) \otimes R$ (ký hiệu $\otimes$ của bài báo chỉ phép xen kẽ truy xuất-suy luận). Cả PPO và GRPO đều được mở rộng để tối ưu trực tiếp trên loại rollout này, với một mẹo mấu chốt: **che mất mát token truy xuất**,

chỉ lan truyền gradient qua token do chính LLM sinh ra, không qua nội dung search engine chèn vào. Phần thưởng chỉ là outcome reward rule-based (EM), không cần mô hình phần thưởng huấn luyện riêng, không cần format reward. Kết quả: LLM tự học được khi nào cần search, viết query gì, và tổng hợp nhiều lượt truy xuất để trả lời, vượt các baseline RAG/tool-use trên 7 bộ dữ liệu hỏi đáp.

3. Phương pháp chi tiết

3.1 Mục tiêu RL với search engine

Mục tiêu gốc (công thức 1 của bài báo):

$$\max_{\pi_\theta} \mathbb{E}_{x\sim D,\, y\sim\pi_\theta(\cdot|x;R)}\Big[r_\phi(x,y) - \beta\, D_{KL}[\pi_\theta(y \mid x;R) \,\|\, \pi_{ref}(y \mid x;R)]\Big]$$

So với mục tiêu RLHF/RLVR chuẩn (bài 00, bài 39) nơi $y \sim \pi_\theta(\cdot \mid x)$ chỉ do policy sinh, ở đây y còn phụ thuộc vào search engine R: quỹ đạo gồm cả token suy luận và token truy xuất chèn vào theo thời gian thực, nên công thức RLHF chuẩn (giả định toàn bộ y do policy sinh) phải viết lại tường minh với R (Phụ lục A).

3.2 Không gian hành động và rollout đa lượt

Hành động gồm token văn bản tự do và các "lệnh gọi" được đánh dấu bằng token đặc biệt: `<search>query</search>` để gọi search engine, kết quả trả về được chèn vào giữa `<information>...</information>`, suy luận trung gian bọc trong `<think>...</think>`, câu trả lời cuối bọc trong `<answer>...</answer>`. Thuật toán rollout (Algorithm 1, giữ nguyên logic của bài báo, diễn đạt lại ngắn gọn):

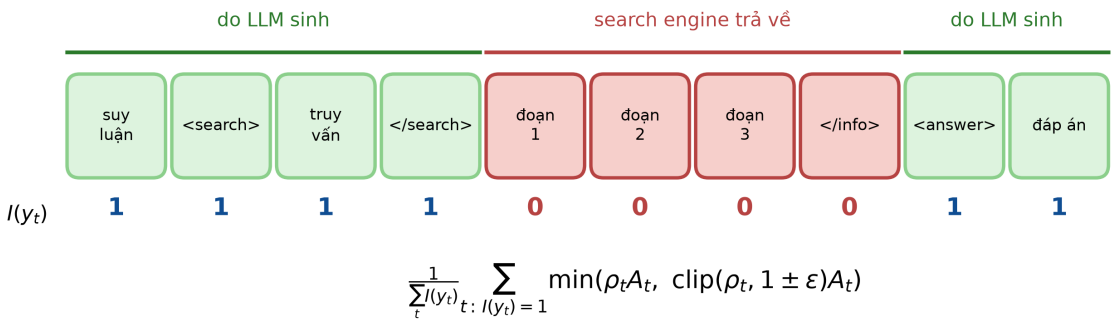
```
Input: query x, policy pi_theta, search engine R, max action budget B
y <- {} ; b <- 0
while b < B:
    yb <- sinh token tu pi_theta(.|x,y+yb) den khi gap </search>, </answer> hoac <eos>
    y <- y + yb
    if "<search>...</search>" trong yb:
        d <- R(parse_query(yb))                # truy xuat top-k doan van
        y <- y + "<information>" + d + "</information>"
    elif "<answer>...</answer>" trong yb:
        return y
    else:
        y <- y + "My action is not correct. Let me rethink."
    b <- b + 1
return y
```

Rollout dừng khi mô hình sinh `<answer>` hoặc hết ngân sách hành động B (thực nghiệm $B = 4$, tức tối đa 4 lượt search trước khi buộc trả lời).

Cách đọc hình: mỗi ô là một token của ví dụ 10 token ở mục 3.7. Ô xanh lá do LLM sinh nên có $I = 1$ và được tính vào tổng lẫn mẫu số; ô đỏ là nội dung truy xuất có $I = 0$ nên bị loại khỏi cả hai. Vì vậy mẫu số của công thức là 6, không phải 10.

3.3 Che mất mát token truy xuất (retrieved token loss masking)

Ký hiệu $I(y_t) = 1$ nếu y_t do LLM sinh, $I(y_t) = 0$ nếu y_t là token truy xuất (trong `<information>...</information>`). Khi tính loss, cả tử và mẫu của trung bình chỉ chạy trên token $I(y_t) = 1$; token truy xuất bị loại hoàn toàn khỏi tổng và khỏi chuẩn hoá độ dài. Lý do (diễn giải): nếu không che, policy gradient sẽ cố "giải thích" cả xác suất sinh đoạn văn do search engine trả về, dù mô hình không thực sự tự hồi quy sinh ra chúng, gây học lệch. Mục 5 cho thấy bỏ masking làm hiệu năng giảm rõ rệt.



Hình 95: Rollout xen kẽ token do LLM sinh và token do search engine trả về, kèm chỉ báo $I(y_t)$ của việc che mất mát (minh hoạ, tự tính)

3.4 PPO với search engine (công thức 2)

$$J_{PPO}(\theta) = \mathbb{E}_{x\sim D,\, y\sim\pi_{old}(\cdot|x;R)} \left[\frac{1}{\sum_{t=1}^{|y|} I(y_t)} \sum_{t:I(y_t)=1} \min\Big(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t\Big) \right]$$

với $\rho_t = \dfrac{\pi_\theta(y_t \mid x, y_{<t}; R)}{\pi_{old}(y_t \mid x, y_{<t}; R)}$. A_t là advantage ước lượng bằng GAE (bài 05) dựa trên phần thưởng tương lai và một value function V_ϕ học riêng (critic). Đây là mở rộng trực tiếp của PPO chuẩn (bài 07): khác biệt duy nhất là tổng và phép chuẩn hoá chỉ chạy trên các vị trí $I(y_t) = 1$.

3.5 GRPO với search engine (công thức 3)

Lấy mẫu G câu trả lời $\{y_1, \ldots, y_G\}$ từ π_{old} cho cùng một câu hỏi x:

$$J_{GRPO}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{\sum_t I(y_{i,t})} \sum_{t:I(y_{i,t})=1} \min\Big(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\Big) - \beta D_{KL}[\pi_\theta \| \pi_{ref}] \right]$$

với advantage chuẩn hoá theo nhóm (giống bài 36):

$$\hat{A}_{i,t} = \frac{R(s_{i,t}, a_{i,t}) - \text{mean}(R^{(1)}, \ldots, R^{(G)})}{\text{std}(R^{(1)}, \ldots, R^{(G)})}$$

Khác với công thức (1), ở GRPO số hạng KL **không** nằm trong phần thưởng mà được cộng trực tiếp vào loss (cùng cách DeepSeekMath trình bày, bài 36); loss masking cũng áp dụng khi tính số hạng KL này.

3.6 Mẫu huấn luyện và phần thưởng (Bảng 1, công thức 4)

Mẫu huấn luyện (Bảng 1, rút gọn): yêu cầu mô hình suy luận trong `<think>`, gọi search bằng `<search>query</search>` khi thiếu kiến thức (kết quả trả về trong `<information>`), được search bao nhiêu lần tùy ý, và trả lời trong `<answer>`. Tác giả **chủ ý** chỉ ràng buộc cấu trúc đầu ra, không ép nội dung suy

luận hay ví dụ few-shot, để hành vi học được phản ánh đúng động lực RL (cùng tinh thần DeepSeek-R1-Zero, bài 39).

Phần thưởng (công thức 4): $\boxed{r_\phi(x, y) = \text{EM}(a_{pred}, a_{gold})}$, với a_{pred} trích từ tag `<answer>`. Không có format reward (khác DeepSeek-R1 gốc, vì mô hình đã tuân thủ cấu trúc tốt); không dùng mô hình phần thưởng neural vì nhạy cảm và tốn kém ở quy mô lớn.

3.7 Ví dụ nhỏ minh hoạ loss masking và advantage (tự tính)

Giả sử rollout có 10 token: 4 token do LLM sinh ($I = 1$), 4 token `<information>` do search engine trả ($I = 0$), rồi 2 token trả lời do LLM sinh ($I = 1$). Mẫu số chuẩn hoá của J_{PPO} là 6 (số token $I = 1$), không phải 10; 4 token truy xuất ở giữa không xuất hiện trong tổng hay mẫu số.

Ví dụ GRPO với $G = 3$, phần thưởng $R = \{1, 0, 0\}$: mean $= 0{,}333$, std $\approx 0{,}471$. Advantage rollout đúng $\hat{A} \approx 1{,}415$; hai rollout sai $\hat{A} \approx -0{,}707$. Token $I = 0$ trong rollout sai không nhận gradient dù advantage âm, chỉ token $I = 1$ bị đẩy xác suất xuống.

4. Thí nghiệm và kết quả chính

Dữ liệu: huấn luyện trên tập train hợp nhất của NQ và HotpotQA; đánh giá EM trên 7 bộ: General QA: NQ, TriviaQA, PopQA; Multi-hop QA: HotpotQA, 2WikiMultiHopQA, Musique, Bamboogle (NQ, HotpotQA in-domain, 5 bộ còn lại out-of-domain). **Retriever:** E5 (dense retriever) trên Wikipedia dump 2018, top-$k = 3$ đoạn văn mỗi lần search (theo Lin et al. 2023, để công bằng giữa các phương pháp). **Mô hình:** Qwen2.5-3B/7B Base và Instruct; phụ lục còn thử Qwen2.5-14B. **Baseline:** Direct Inference, CoT, IRCoT, Search-o1, RAG, SFT, "R1" (RL kiểu DeepSeek-R1 không search, huấn luyện lại trên cùng dữ liệu), Rejection Sampling (5 mẫu/prompt, giữ mẫu đúng để tinh chỉnh). **Siêu tham số chính (Appendix B.2):** PPO: lr policy 10^{-6}, lr value 10^{-5}, 500 bước, GAE $\lambda = 1, \gamma = 1$, batch 512/mini-batch 256, max sequence length 4096, max response length 500, max độ dài truy xuất 500, $\beta = 0{,}001$, $\epsilon = 0{,}2$, rollout qua vLLM (temperature 1,0, top-p 1,0), 1 node 8×H100. GRPO: lr 10^{-6}, group size 5, cùng cấu hình batch/độ dài/β/ϵ. $B = 4$ (tối đa 4 lượt search).

Bảng 2 (trích): kết quả chính, EM trên các bộ dữ liệu

Phương pháp	NQ†	TriviaQA⋆	HotpotQA†	Bamboogle⋆	Avg (7 bộ)
Qwen2.5-7B RAG	0,349	0,585	0,299	0,208	0,304
R1 (base, không search)	0,297	0,539	0,242	0,296	0,276
Search-R1-base	**0,480**	**0,638**	**0,433**	**0,432**	**0,431**
Search-R1-instruct	0,393	0,610	0,370	0,368	0,385
Qwen2.5-3B RAG	0,348	0,544	0,255	0,080	0,270
R1 (base, không search)	0,226	0,455	0,201	0,224	0,229
Search-R1-base	**0,406**	**0,587**	0,284	0,088	**0,303**
Search-R1-instruct	0,341	0,545	**0,324**	**0,264**	**0,325**

(†: in-domain, ⋆: out-of-domain; bảng trích 4 cột trong 7, và bỏ hai baseline yếu "Direct Inference"/"Rejection Sampling" để gọn; xem Bảng 2 gốc cho đầy đủ.) Qwen2.5-14B cũng được thử (Appendix C, Bảng 5): Search-R1-base đạt Avg 0,479 so với RAG 0,281 và R1-base 0,357, lợi thế vẫn giữ khi tăng kích thước mô hình.

Mức cải thiện trung bình được công bố ở hai nơi khác nhau trong bài báo: tóm tắt (Abstract) và §4.4 ghi "cải thiện 24% (Qwen2.5-7B) và 20% (Qwen2.5-3B) so với baseline RAG"; nhưng phần giới thiệu (danh sách contribution) lại ghi "cải thiện trung bình 41% và 20%". Đây là một mâu thuẫn trong chính văn bản paper (có thể do cách tính trung bình khác nhau giữa các baseline so sánh, bài báo không nêu rõ công thức); ở đây nêu

cả hai con số, không tự chọn số nào là "đúng". (Tự tính để tham khảo: $(0{,}431 - 0{,}304)/0{,}304 \approx 41{,}8\%$ khi so Search-R1-base 7B với dòng RAG trong Bảng 2, khớp khá sát với số "41%"; còn $(0{,}385 - 0{,}304)/0{,}304 \approx 26{,}6\%$ với bản instruct, gần nhưng không khớp đúng số "24%" của tóm tắt, nên cách tính chính xác vẫn chưa rõ.)

5. Phân tích / ablation

PPO vs GRPO (§5.1, Hình 2a, Bảng 3): GRPO hội tụ nhanh hơn PPO ở mọi trường hợp (PPO cần "khởi động" critic trước khi cập nhật hiệu quả), nhưng GRPO bị **sụp đổ phần thưởng (reward collapse)** sau nhiều bước, còn PPO ổn định hơn suốt quá trình; phần thưởng huấn luyện cuối cùng của hai phương pháp nhìn chung tương đương. Số liệu Bảng 3 (Avg 7 bộ): Qwen2.5-7B-base, GRPO = 0,350 so với PPO = 0,431 (PPO hơn hẳn); Qwen2.5-3B-base, GRPO = 0,312 so với PPO = 0,303 (GRPO nhỉnh hơn). Tác giả chọn PPO làm mặc định vì ổn định hơn, dù không phải lúc nào cũng cho điểm cao nhất.

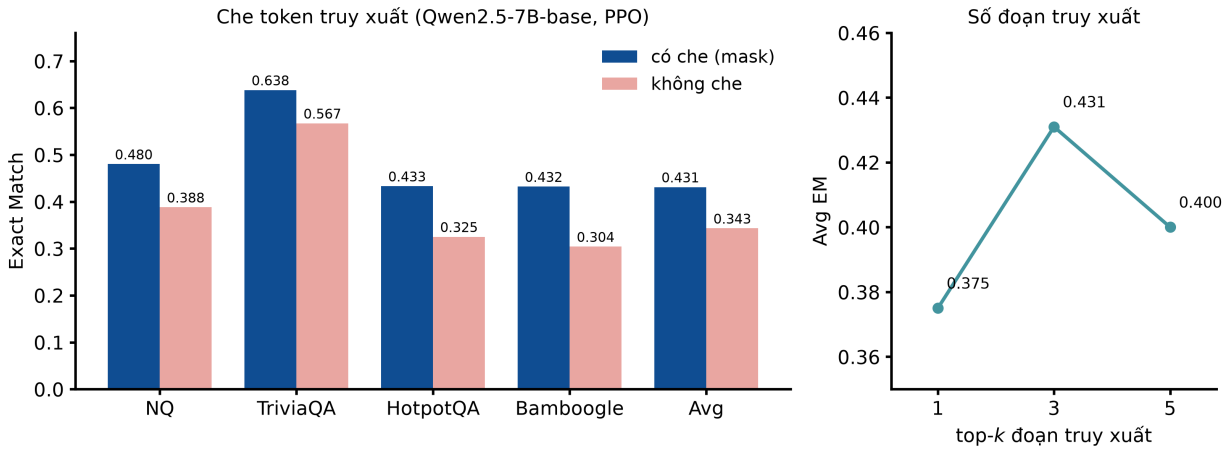
Base vs Instruct (§5.2, Hình 2b): instruct hội tụ nhanh hơn, điểm khởi đầu cao hơn, nhưng phần thưởng cuối cùng gần như bằng base, tức RL có thể "bù" khoảng cách ban đầu theo thời gian.

Động lực độ dài phản hồi và số lượt search (§5.3, Hình 2c-d): độ dài phản hồi giảm mạnh ~100 bước đầu (bỏ từ đệm thừa), rồi tăng mạnh và ổn định ở giai đoạn sau (gọi search thường xuyên hơn, phản hồi dài ra vì chứa đoạn truy xuất, phần thưởng tăng đáng kể); số lượt search hợp lệ tăng dần theo huấn luyện.

Che mất mát token truy xuất (§5.4, Bảng 4, Qwen2.5-7B-base, PPO):

	NQ	TriviaQA	HotpotQA	Bamboogle	Avg
Có mask	0,480	0,638	0,433	0,432	0,431
Không mask	0,388	0,567	0,325	0,304	0,343

Chênh lệch Avg khoảng 8,8 điểm phần trăm, khẳng định masking là thành phần quan trọng, không chỉ là chi tiết kỹ thuật phụ.



Hình 96: Ablation che token truy xuất (Bảng 4) và ảnh hưởng của số đoạn truy xuất top k lên Avg EM (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái so sánh có che và không che trên từng bộ dữ liệu; thanh xanh đậm luôn cao hơn thanh hồng. Panel phải cho Avg EM theo k (1, 3, 5) với Qwen2.5-7B-base và PPO, đạt cao nhất ở $k = 3$. Các số đều lấy từ mục 5 của ghi chú.

Số đoạn truy xuất top-k (Appendix G, Qwen2.5-7B-base, PPO): $k = 1$: Avg 0,375 (recall thấp); $k = 3$: Avg 0,431 (tốt nhất); $k = 5$: Avg 0,400 (đoạn nhiễu làm giảm độ chính xác và cả độ ổn định huấn luyện RL). $k = 5$ hội tụ nhanh lúc đầu nhưng kém ổn định về sau; $k = 3$ chậm hơn lúc đầu nhưng vượt lên ở bước 500.

Kích thước nhóm GRPO (Appendix H, Qwen2.5-7B-base): size $\in \{1, 3, 5\}$ (size=1 tức GRPO suy biến về REINFORCE thuần). Avg: size=1: 0,410; size=3: 0,363; size=5: 0,350. Size lớn hội tụ nhanh hơn nhưng dễ sụp đổ; size nhỏ ổn định và tổng quát hoá tốt hơn trên dữ liệu ngoài phân phối, trade-off tương tự PPO vs GRPO ở trên.

Case study (Appendix I-J): Search-R1 thường search thêm một lượt để "tự kiểm chứng" (self-verification) ngay cả khi đã đủ thông tin, tương tự hành vi quan sát ở DeepSeek-R1 (bài 39) nhưng trong bối cảnh có search thật. Ví dụ thất bại: khi đoạn truy xuất không liên quan hoặc gây hiểu lầm (nhầm tên diễn viên, nhầm bộ phim), mô hình lặp lại truy vấn gần giống nhau nhiều lần mà không đổi chiến lược, cuối cùng vẫn trả lời sai.

Quan sát thêm của phần phân tích (bài báo không thảo luận): với Qwen2.5-3B-base, Search-R1-base chỉ đạt Bamboogle 0,088, **thấp hơn** cả R1-base không search (0,224); thêm search không giúp, thậm chí làm giảm ở quy mô 3B-base trên bộ này. Hiện tượng này không xuất hiện ở bản instruct (Search-R1-instruct 3B: 0,264, tốt hơn R1-instruct 0,192), gợi ý lợi ích của Search-R1 phụ thuộc năng lực nền của mô hình.

6. Giới hạn

Tác giả nêu: hướng tương lai gồm thiết kế phần thưởng phức tạp hơn, điều chỉnh truy xuất động theo độ bất định (uncertainty), kết hợp đa dạng tool hơn (không chỉ search), và mở rộng sang suy luận đa phương thức (multimodal).

Nhận xét:

- Retriever (E5) và nguồn tri thức (Wikipedia dump 2018, tĩnh) không phải search engine thực trên web, chỉ là truy vấn một kho dữ liệu cố định; theo bài 54, các công trình sau (WebSailor, DeepResearcher...) mới hướng tới web thật.
- Dữ liệu huấn luyện chỉ gồm NQ và HotpotQA (câu hỏi thực thể, multi-hop entity), khá hẹp; chưa kiểm chứng trên dạng câu hỏi khác hẳn (toán, code). Cần chạy retriever sống trong rollout RL, thêm độ trễ/hạ tầng (vLLM + FSDP + server truy xuất) mà bài báo không đo overhead.
- Phần thưởng EM nhị phân có thể phạt nhầm câu trả lời đúng nhưng diễn đạt khác (hạn chế chung của EM, bài báo không thảo luận).
- "GRPO sụp đổ" được quan sát nhưng chưa giải thích cơ chế; các bài 41 (DAPO), 42 (Dr. GRPO), 43 (GSPO) có thể liên quan (bias độ dài, chuẩn hoá cấp token vs câu) nhưng đây chỉ là liên hệ suy luận, cần đọc trực tiếp để xác nhận.
- Mâu thuẫn số liệu "24%/41%" (mục 4) và lợi ích search không nhất quán ở 3B-base trên Bamboogle (mục 5) cho thấy cần cẩn trọng khi trích dẫn số liệu bài báo này.

7. Liên hệ với các bài khác

- **Bài 00, 07 (PPO):** PPO gốc, chỉ thêm loss masking theo $I(y_t)$; GAE, clip ϵ giữ nguyên.
- **Bài 36 (DeepSeekMath, GRPO):** advantage chuẩn hoá theo nhóm dùng lại nguyên bản, chỉ thêm masking.
- **Bài 39 (DeepSeek-R1):** Search-R1 tự nhận là mở rộng của DeepSeek-R1-Zero; baseline "R1" trong bài chính là RL kiểu DeepSeek-R1 không search, huấn luyện lại trên cùng dữ liệu.
- **Bài 41 (DAPO), 42 (Dr. GRPO), 43 (GSPO):** biến thể GRPO xuất hiện sau, có thể giải thích vì sao GRPO "sụp đổ" (suy luận liên hệ).
- **Bài 54 (khảo sát Agentic RL):** xếp Search-R1 vào nhóm "External" trong Bảng 4 "search and research agent", cùng nhóm với R1-Searcher, ReSearch, DeepResearcher, StepSearch (ngoài lộ trình số), trái

hướng ZeroSearch (search giả lập bằng LLM); cũng lấy "retrieved-token masking" của Search-R1 làm ví dụ cho "RL cho tool use".

8. Ứng dụng thực tế (suy luận)

- **Loss masking là bài học tổng quát nhất**, không riêng cho search: tăng gần 9 điểm Avg (mục 5) cho thấy bất cứ khi nào rollout RL chứa văn bản policy không thực sự tự sinh có kiểm soát (kết quả tool, đoạn truy xuất, log thực thi code), nên loại các token đó khỏi cả tử và mẫu của loss chính sách (và loss KL), nếu không dễ gây học lệch.
- **Group size là siêu tham số rẻ cần thử trước:** size nhỏ (thậm chí 1, suy biến về REINFORCE) ổn định và tổng quát tốt hơn trên dữ liệu ngoài phân phối, đổi lại chậm hơn, chỉ cần đổi một siêu tham số.
- **Phần thưởng tối giản là điểm khởi đầu hợp lý:** chỉ EM thuần, không format reward, không mô hình phần thưởng, vẫn học được hành vi phức tạp; nên thử outcome reward đơn giản nhất trước khi thêm process reward/mô hình phần thưởng.
- **Chi phí hạ tầng:** 8×H100/500 bước khiêm tốn so với chuẩn LLM RL, nhưng cần server truy xuất/tool chạy song song rollout, cộng vLLM/FSDP, vượt ngoài một quy trình RLVR một lượt (ví dụ bài 43).

9. Thí nghiệm gợi ý

1. **Ablation loss masking trên một kịch bản tool đơn giản:** dựng thiết lập 2 lượt nơi mô hình OCR "gọi" tool zoom trả về token của vùng ảnh crop lại (có thể giả lập); so sánh GRPO có/không che mất mát token tool trả về, đo phương sai phần thưởng và CER cuối, đối chiếu mức chênh ~9 điểm Avg ở Bảng 4.
2. **Lặp lại khảo sát kích thước nhóm GRPO (1, 3, 5)** trên thiết lập RLVR của bài 43 (lr 4e-5, beta 0,01) thay cho 28 rollout cố định, xem nhóm nhỏ có đánh đổi tốc độ để lấy khả năng tổng quát tốt hơn trên tài liệu chưa thấy, giống hiện tượng ở Search-R1.
3. **Tái hiện đồ thị động lực độ dài phản hồi** (giảm rồi tăng rồi ổn định) cho quy trình tài liệu có tính agentic (cho phép đọc lại một dòng khi không chắc), kiểm tra mẫu 3 giai đoạn có xuất hiện không.
4. **So sánh PPO vs GRPO về độ ổn định ở quy mô 3B** trên một tác vụ search-augmented doc QA tương tự, trước khi đầu tư cài đặt critic PPO đầy đủ, vì lựa chọn tốt nhất ở đây phụ thuộc quy mô (GRPO thắng 3B-base, PPO thắng 7B-base).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Mục tiêu RL + search	$\max \mathbb{E}[r_\phi - \beta D_{KL}]$, $y \sim \pi_\theta(\cdot \mid x; R)$	công thức (1)
Loss masking	$I(y_t) = 1$ nếu LLM sinh, $= 0$ nếu truy xuất; tổng/mẫu chỉ trên $I = 1$	mấu chốt ổn định, +8,8 điểm Avg
PPO w/ search	J_{PPO}: clip ratio, A_t từ GAE, có masking	công thức (2)
GRPO w/ search	J_{GRPO}: advantage theo nhóm, KL cộng trực tiếp vào loss, có masking	công thức (3)
Advantage nhóm	$\hat{A} = (R - \text{mean}(R))/\text{std}(R)$ trên G rollout	giống bài 36
Reward	$r = \text{EM}(a_{pred}, a_{gold})$, rule-based, không format reward	công thức (4)
Template	`<think>` / `<search>` / `<information>` / `<answer>`	Bảng 1
Rollout	lặp: sinh → search hoặc answer → lặp tới B lượt	Algorithm 1, $B = 4$
Hyperparam chính	PPO: $\beta = 0{,}001$, $\epsilon = 0{,}2$, lr $10^{-6}/10^{-5}$; GRPO: group size 5	Appendix B.2, 8×H100

Khái niệm	Công thức / ý	Ghi chú
PPO vs GRPO	GRPO hội tụ nhanh hơn, dễ sụp đổ hơn; PPO ổn định hơn	phụ thuộc quy mô mô hình

11. Kết luận ngắn

Search-R1 mở rộng khung RLVR kiểu DeepSeek-R1 sang bài toán có công cụ ngoài bằng cách coi search engine là một phần của environment, xen kẽ suy luận và truy xuất qua token đặc biệt, và tối ưu bằng PPO hoặc GRPO với một kỹ thuật tưởng chừng nhỏ nhưng quyết định: che mất mát token truy xuất khỏi gradient. Với phần thưởng outcome đơn giản (exact match), mô hình 3B/7B tự học được khi nào cần search, viết query gì và tự kiểm chứng câu trả lời, vượt rõ các baseline RAG và tool-use trên 7 bộ hỏi đáp. Những phân tích đi kèm (PPO vs GRPO, kích thước nhóm, số đoạn truy xuất, ablation masking) cho thấy phần lớn lợi ích nằm ở các lựa chọn kỹ thuật ổn định hoá cụ thể hơn là ở một kiến trúc RL mới, một bài học hữu ích cho bất kỳ ai định huấn luyện RL đa lượt có tool.

Link gốc: https://arxiv.org/abs/2503.09516

Bài 54

The Landscape of Agentic Reinforcement Learning for LLMs: A Survey (2025)

Phạm vi ghi chú: đây là một survey rất dài (tác giả nói tổng hợp hơn 500 công trình, ~48 trang nội dung chính). Ghi chú tập trung vào năm mảng: (1) hình thức hóa toán học của bước chuyển từ PBRFT sang Agentic RL (MDP suy biến một bước sang POMDP đa lượt); (2) phân loại theo năng lực agent (planning, tool use, memory, self-improvement, reasoning, perception); (3) phân loại theo miền nhiệm vụ (search, code, math, GUI, vision, embodied, multi-agent, khác) cùng ma trận phụ thuộc năng lực x ứng dụng; (4) tổng quan môi trường và framework RL; (5) các thách thức mở (an toàn, scaling, debate về cơ chế RL). Ghi chú KHÔNG liệt kê chi tiết hàng trăm phương pháp con trong mục "Task Perspective" của bài báo; các bảng chỉ trích một phần đại diện bảng gốc, có ghi rõ khi bị cắt bớt.

Tác giả: Guibin Zhang, Hejia Geng, Xiaohang Yu (đóng góp bằng nhau), cùng danh sách rất dài đồng tác giả (Oxford, Shanghai AI Laboratory, NUS, UIUC, Brown, UCL...); tác giả liên hệ: Lei Bai (Shanghai AI Laboratory).
Venue / arXiv: arXiv:2509.02547 (v5, 17/04/2026). Venue: Transactions on Machine Learning Research (TMLR), theo header PDF "Published in Transactions on Machine Learning Research (01/2026)"; review công khai trên OpenReview.
Nguồn: https://arxiv.org/abs/2509.02547
Chủ đề: Survey hình thức hóa "Agentic RL" (RL cho LLM-agent đa lượt, tương tác môi trường/tool/memory) như một paradigm khác biệt với RL truyền thống cho LLM (PBRFT), rồi đưa ra bản đồ hai chiều (năng lực agent x miền nhiệm vụ) cùng tổng quan môi trường, framework, thách thức mở.
Vị trí trong lộ trình: nên đọc sau bài 00 (nền toán RL), bài 07 (PPO), bài 36 (GRPO/DeepSeekMath), bài 39 (DeepSeek-R1), bài 53 (Search-R1, ví dụ cụ thể chính survey liệt kê). Bài 54 không đưa ra phương pháp mới; nó là bản đồ định vị mọi bài RL khác trong lộ trình (41, 42, 43, 46, 55...) vào một khung lý thuyết chung, phù hợp đọc ở giữa/cuối lộ trình, trước khi đi sâu vào bài ứng dụng cụ thể.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng thực tế" là suy luận.

1. Bối cảnh

Hai dòng nghiên cứu đã phát triển gần như song song trước khi survey này ra đời. Dòng thứ nhất là **RL cho LLM alignment** (PBRFT: preference-based RL fine-tuning): dùng PPO (bài 07) hoặc GRPO (bài 36) để tối ưu một phản hồi (response) duy nhất cho một prompt, dựa trên reward từ mô hình phần thưởng học theo sở thích con người (RLHF) hoặc từ verifier có thể kiểm chứng (RLVR, như DeepSeek-R1, bài 39). Dòng thứ hai là **LLM agent**: ReAct, Toolformer, Reflexion, VOYAGER... cho LLM gọi tool, giữ memory, tự phản tư (reflect) qua nhiều lượt, nhưng ban đầu chủ yếu dựa vào prompting hoặc SFT trên quỹ đạo người gán nhãn, thiếu cơ chế thích nghi qua trải nghiệm.

Tác giả quan sát: khi RL được áp cho agent (nhiều lượt, tương tác môi trường động, quan sát không đầy đủ), các giả định của PBRFT (một bước, quan sát đầy đủ, dữ liệu cố định) không còn đúng. Survey này hình thức hóa sự khác biệt đó bằng lý thuyết MDP/POMDP, rồi dùng khung đó để tổ chức lại toàn bộ tài liệu đã có (survey tự nhận tổng hợp hơn 500 bài) theo hai trục: **năng lực** (agent học làm gì) và **miền nhiệm vụ** (agent được áp dụng ở đâu).

Khái niệm nền cần nắm trước khi đọc: **MDP** (bộ $\langle S, A, P, R, \gamma \rangle$, bài 00); **POMDP** (agent chỉ thấy quan sát

$o_t = O(s_t)$, một hàm có thể mất thông tin của trạng thái thật s_t); **MDP suy biến một bước** (horizon $T = 1$, episode kết thúc ngay sau một hành động); **outcome reward** (chỉ cho ở cuối episode, ví dụ EM của bài 53) **vs step-wise/process reward** (cho ở từng bước trung gian); **reward hacking** (policy lách reward để đạt điểm cao mà không đạt mục tiêu thật).

Theo Hình 1, cấu trúc tổng thể: mục 2 (hình thức hóa), mục 3 (năng lực), mục 4 (miền nhiệm vụ), mục 5 (môi trường, framework), mục 6 (thách thức mở).

2. Ý tưởng chính

Ý tưởng trung tâm: RL truyền thống cho LLM (PBRFT) là một MDP suy biến (một prompt, một response, một reward scalar, không có "trạng thái kế tiếp" thực sự), còn **Agentic RL** là một POMDP đầy đủ với horizon $T > 1$ (agent nhận quan sát từng phần, chọn hành động gồm cả văn bản tự do và hành động có cấu trúc, trạng thái chuyển động ngẫu nhiên, reward có thể rời rạc theo bước); PBRFT chỉ là trường hợp đặc biệt của Agentic RL khi $T = 1$ (chứng minh ở mục 3). Dựa trên hình thức hóa này, survey xây dựng taxonomy hai chiều (năng lực x miền nhiệm vụ), tổng hợp môi trường/framework thực tế, và chỉ ra các thách thức mở của cả lĩnh vực.

3. Phương pháp chi tiết: từ PBRFT đến Agentic RL

3.1 Hình thức hóa MDP và POMDP

Paper định nghĩa MDP tổng quát cho quá trình tinh chỉnh RL là bộ bảy phần tử $\langle S, O, A, P, R, T, \gamma \rangle$: S không gian trạng thái, O không gian quan sát, A không gian hành động, P xác suất chuyển trạng thái, R hàm reward, T horizon nhiệm vụ, γ hệ số chiết khấu.

PBRFT là MDP suy biến:

$$\boxed{\langle S_{trad},\ A_{trad},\ P_{trad},\ R_{trad},\ T{=}1,\ \gamma{=}1 \rangle}$$

Agentic RL là một POMDP:

$$\boxed{\langle S_{agent},\ A_{agent},\ P_{agent},\ R_{agent},\ \gamma,\ O \rangle}, \quad o_t = O(s_t)$$

Bảng 1 của bài báo (rút gọn, giữ đúng nội dung từng dòng):

Khái niệm	PBRFT truyền thống	Agentic RL
S: không gian trạng thái	$\{s_0\}$ (một prompt duy nhất); episode kết thúc ngay	$s_t \in S_{agent}$; $o_t = O(s_t)$; horizon $T > 1$
A: không gian hành động	Chuỗi văn bản thuần	$A_{text} \cup A_{action}$
P: chuyển trạng thái	Chuyển tất định đến trạng thái kết thúc	Hàm chuyển động $P(s_{t+1} \mid s_t, a_t)$
R: reward	Một scalar duy nhất $r(a)$	$R(s_t, a_t)$ theo bước, kết hợp sparse (nhiệm vụ) và dense (sub-reward)
$J(\theta)$: mục tiêu	$\mathbb{E}_{a\sim\pi_\theta}[r(a)]$	$\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \gamma^t R(s_t, a_t)\right]$

3.2 Không gian hành động, động lực chuyển trạng thái, hàm reward

Không gian hành động của Agentic RL:

$$\boxed{A_{agent} = A_{text} \cup A_{action}}$$

A_{text} là token ngôn ngữ tự nhiên sinh tự hồi quy; A_{action} là hành động phi ngôn ngữ, đánh dấu bằng token đặc biệt `<action_start>`/`<action_end>`, có thể gọi tool (`call("search", "Einstein")`) hoặc tương tác môi trường (`move("north")`). A_{action} có tính đệ quy: một hành động có thể là một chuỗi hành động nguyên tố, hợp nhất hành động đơn và phức trong cùng một không gian.

Động lực chuyển trạng thái: PBRFT là tất định $P(s_1 \mid s_0, a) = 1$; Agentic RL là ngẫu nhiên $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$, với $a_t \in A_{text} \cup A_{action}$.

Hàm reward: PBRFT là $R_{trad}(s_0, a) = r(a)$, một scalar không có phản hồi trung gian. Agentic RL là phân đoạn theo bước:

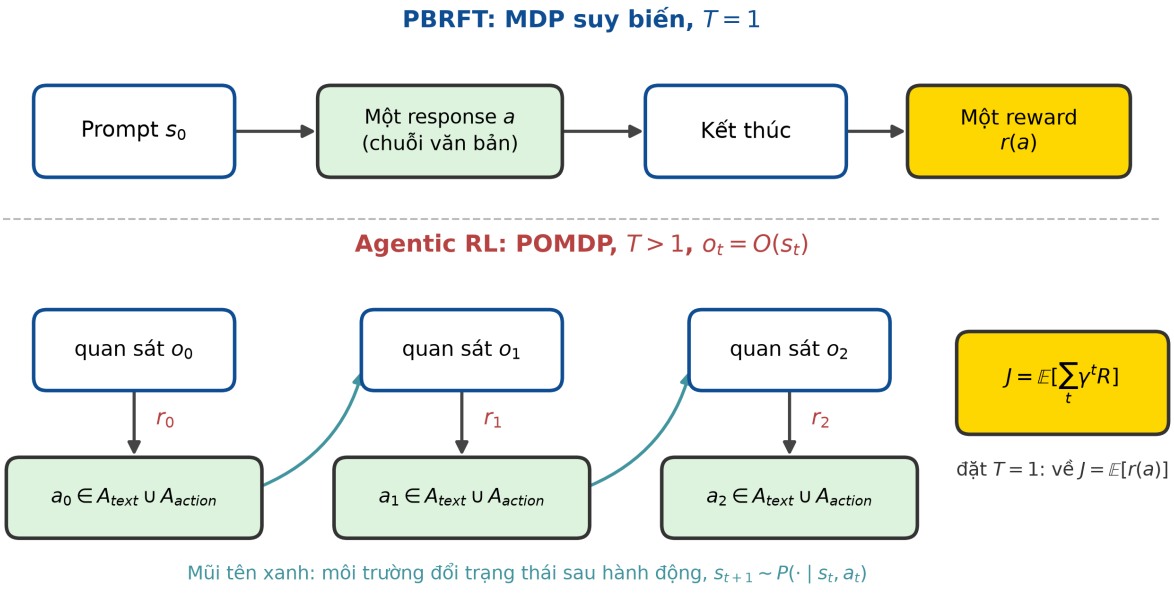
$$R_{agent}(s_t, a_t) = \begin{cases} r_{task} & \text{khi hoàn thành nhiệm vụ} \\ r_{sub} & \text{tiến triển theo bước (step-level)} \\ 0 & \text{còn lại} \end{cases}$$

cho phép reward dense, sparse, hoặc học được (ví dụ pass unit-test, verifier ký hiệu).

Mục tiêu tối ưu: PBRFT là $J_{trad}(\theta) = \mathbb{E}_{a\sim\pi_\theta}[r(a)]$ (không cần chiết khấu); Agentic RL là

$$\boxed{J_{agent}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t R_{agent}(s_t, a_t)\right], \quad 0 < \gamma < 1}$$

Tự tính: nếu đặt $T = 1$ vào J_{agent}, tổng chỉ còn một hạng tử $\gamma^0 R(s_0, a_0) = R(s_0, a_0)$, và nếu coi $R(s_0, a_0) = r(a)$ thì J_{agent} suy biến đúng về $J_{trad} = \mathbb{E}[r(a)]$. Điều này xác nhận cách paper diễn đạt: PBRFT là một trường hợp riêng, suy biến, của Agentic RL khi horizon bằng 1, chứ không phải hai framework tách biệt nhau.



Hình 97: So sánh PBRFT một bước với RL có tác tử nhiều bước trên POMDP (minh hoạ, tự tính)

Cách đọc hình: hàng trên là PBRFT, một prompt sinh một response rồi nhận đúng một reward. Hàng dưới là RL có tác tử, mỗi bước tác tử nhận quan sát o_t, chọn hành động a_t và nhận r_t, còn môi trường đổi trạng thái giữa các bước. Khi $T = 1$, mục tiêu J nhiều bước co về $\mathbb{E}[r(a)]$ của PBRFT.

3.3 Bốn thuật toán RL nền tảng

Paper trình bày lại bốn thuật toán kinh điển rồi so sánh các biến thể phổ biến (Bảng 2).

REINFORCE (policy gradient nền tảng):

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{s_0}\Big[\frac{1}{N}\sum_{i=1}^{N}\big(R(s_0, a^{(i)}) - b(s_0)\big)\nabla_\theta \log \pi_\theta(a^{(i)} \mid s_0)\Big]}$$

$b(s_0)$ là baseline giảm phương sai (thường là value function $V(s)$). Nhược điểm: phương sai gradient cao, cần nhiều mẫu, nhạy learning rate, không có critic.

PPO (bài 07), mục tiêu clip:

$$\boxed{L_{PPO}(\theta) = \frac{1}{N}\sum_i \min\Big(\rho_i A(s_i, a_i),\ \text{clip}(\rho_i, 1{-}\epsilon, 1{+}\epsilon)A(s_i, a_i)\Big)}, \quad \rho_i = \frac{\pi_\theta(a_i \mid s_i)}{\pi_{\theta_{old}}(a_i \mid s_i)}$$

với advantage $A(s_t, a_t) = R(s_t, a_t) - V(s_t)$, V_θ là value function học bằng critic riêng (thường cùng cỡ với policy). Nhược điểm chính: cần critic riêng, tăng đáng kể số tham số cần huấn luyện.

DPO, bỏ hẳn mô hình phần thưởng, viết lại bài toán tối ưu reward có ràng buộc KL thành hàm hợp lý trên dữ liệu ưu tiên $D = \{(y_w, y_l)\}$:

$$\boxed{L_{DPO}(\pi_\theta; \pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\Big)\Big]}$$

Nhược điểm: hiệu quả phụ thuộc chất lượng và độ bao phủ của bộ dữ liệu ưu tiên tĩnh.

GRPO (bài 36), dùng reward tương đối trong một nhóm G output để tính advantage, bỏ hẳn critic tuyệt đối:

$$\boxed{L_{GRPO} = \frac{1}{G}\sum_{g=1}^{G}\min\Big(\rho_g \hat{A}(s_t, a_t),\ \text{clip}(\rho_g, 1{-}\epsilon, 1{+}\epsilon)\hat{A}(s_t, a_t)\Big)}$$

$$\hat{A}(s_t, a_t) = \frac{R(s_t, a_t) - \text{mean}(R^{(1)}, \ldots, R^{(G)})}{\text{std}(R^{(1)}, \ldots, R^{(G)})}$$

Ưu điểm: hiệu quả mẫu cao, giảm chi phí tính toán; nhược điểm: advantage theo nhóm dễ bị phương sai cao và kém chính xác, dẫn đến hàng loạt biến thể cải tiến (Bảng 2).

Bảng 2 (trích, bài báo có danh sách dài hơn nhiều, chỉ giữ các hàng liên quan tới roadmap này):

Phương pháp	Loại mục tiêu	Cơ chế chính
PPO	Policy gradient	Clip tỉ lệ policy
DPO	Preference optimization	Reward ngầm liên hệ với policy
GRPO	Policy gradient (reward theo nhóm)	Reward tương đối trong nhóm, bỏ value estimate
DAPO (bài 41)	Surrogate của GRPO	Decoupled clip + dynamic sampling
GSPO (bài 43)	Surrogate của GRPO	Importance ratio theo cả câu, clip/reward/update ở mức câu
Dr.GRPO (bài 42)	GRPO không chệch	Loại bỏ độ chệch trong tối ưu của GRPO
ProRL (bài 46)	Giống GRPO	Reset policy tham chiếu (reference policy reset)

Bài báo không cho pseudo-code; diễn giải lại từ mục 3.1-3.3 (tự tổng hợp): ở mỗi bước t, agent nhận $o_t = O(s_t)$, sinh $a_t \sim \pi_\theta(\cdot \mid o_t)$ với $a_t \in A_{text} \cup A_{action}$; nếu $a_t \in A_{action}$ thì thực thi (gọi tool/tác động môi

trường) và $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, ngược lại $s_{t+1} = s_t$; nhận $r_t = R_{agent}(s_t, a_t)$; lặp tới T rồi tính advantage (PPO qua critic V, GRPO chuẩn hóa theo nhóm rollout cùng prompt) và cập nhật θ.

4. Phân loại theo năng lực agent

Paper chia Agentic RL theo sáu năng lực cốt lõi (mục 3.1-3.7), coi chúng là các policy liên kết với nhau có thể được tối ưu đồng thời, không phải quy trình tĩnh.

Năng lực	Vai trò của RL	Ví dụ tiêu biểu	Thách thức còn mở
Planning	RL hướng dẫn ngoài (huấn luyện value/heuristic dẫn MCTS) hoặc là động cơ trong (tối ưu trực tiếp policy sinh kế hoạch)	RAP, LATS, VOYAGER, ETO, Planner-R1	Hợp nhất tìm kiếm có cấu trúc với policy nội tại
Tool use	Từ bắt chước tĩnh (ReAct, SFT) sang tối ưu outcome-driven, agent tự khám phá khi nào/dùng tool nào	ToolRL, ReTool, ASPO, DeepEyes, ARTIST	Temporal credit assignment cho tool-use đa lượt
Memory	Từ bộ nhớ ngoài tĩnh (RAG) sang RL kiểm soát ghi/xóa/truy xuất, tới bộ nhớ dạng token	Memory-R1 (PPO/GRPO: ADD/UPDATE/DELETE/NOOP), MemAgent	RL cho bộ nhớ có cấu trúc (graph) hầu như chưa khai thác
Self-improvement	Từ tự sửa lỗi bằng lời (Reflexion) sang nội tại hóa bằng RL (KnowSelf) rồi tự huấn luyện lặp (self-play)	Reflexion, KnowSelf, R-Zero, Absolute Zero, TTRL	Meta-policy quyết định khi nào/cách nào nên tự phản tư
Reasoning	Phân biệt fast (System 1) và slow reasoning (System 2); RLVR huấn luyện slow reasoning đáng tin cậy hơn	OpenAI o1/o3, DeepSeek-R1 (bài 39), rStar-Math	Overthinking, cần chiến lược lai (adaptive test-time scaling)
Perception	Từ nhận thức bị động sang chủ động: grounding, tool thị giác, tự sinh ảnh/sketch khi suy luận	Visual-RFT/Visual-ARFT (bài 55, GRPO + IoU), GRIT, GoT-R1	Mở rộng sang audio/3D/video với reward verifiable

Bảng 3 của bài báo (mục Memory, trích các hàng tiêu biểu nhất mỗi loại; dấu † nghĩa là phương pháp trực tiếp dùng RL):

Phương pháp	Loại	Đặc điểm chính
MemoryBank, MemGPT, HippoRAG	External Store	Bộ nhớ ngoài tĩnh, quy tắc lưu/truy xuất định trước, không RL
Memory-R1†	RL-guided Retrieval	RL (PPO/GRPO) điều khiển ADD/UPDATE/DELETE/NOOP của bộ nhớ
Mem-α†, Memory-as-action†	RL-guided Retrieval/Management	RL hướng dẫn agent truy xuất/quản lý bộ nhớ end-to-end
MemAgent†, MEM1†	Explicit Token	RL quyết định giữ/ghi đè token ngôn ngữ tự nhiên trong pool nhớ
MemoryLLM, M+	Latent Token	Token latent (không đọc được) tích hợp và cập nhật lặp lại, không RL
Zep, A-MEM, G-Memory, Mem0	Structured Memory	Đồ thị tri thức/bộ nhớ nguyên tử có cấu trúc, quản lý bằng heuristic, chưa có RL (hướng mở, mục 3.3)

5. Phân loại theo miền nhiệm vụ

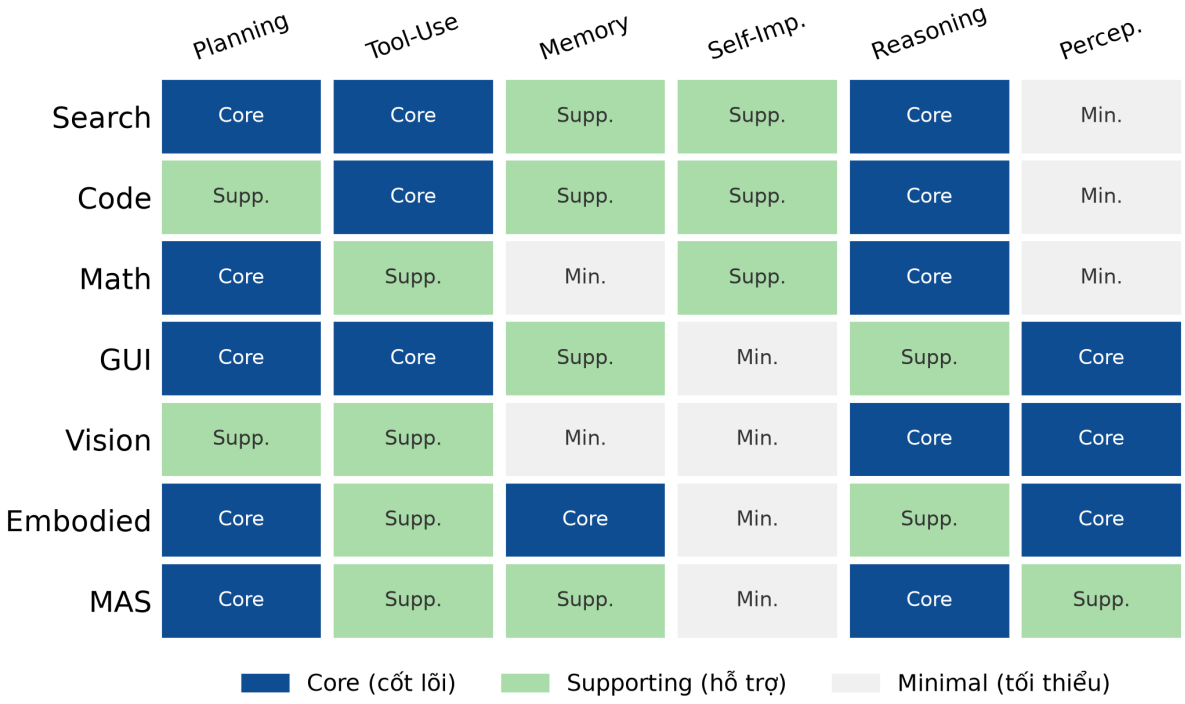
Paper khảo sát tám miền nhiệm vụ (mục 4.1-4.8). Tổng hợp lại (tự tổng hợp từ phần mở đầu mỗi mục, không liệt kê hết các phương pháp con):

Miền	RL hay dùng	Ví dụ tiêu biểu	Môi trường/benchmark chính
Search & Research	GRPO/PPO trên search engine thật (External) hoặc giả lập từ tri thức nội tại LLM (Internal)	DeepRetrieval, Search-R1 (bài 53), R1-Searcher, ZeroSearch, SSRL	Open-domain multi-hop QA
Code	Outcome reward (pass@k/unit test) hoặc process reward (feedback compile/test theo bước)	AceCoder, DeepCoder-14B, StepCoder, PRLCoder	SWE-bench, LiveCodeBench, BigCodeBench
Math	Informal reasoning (outcome/process reward) hoặc formal reasoning (proof assistant)	ToRL, TTRL, 1-shot RLVR; DeepSeek-Prover, Kimina-Prover	định lý hình thức (Lean), benchmark toán tự nhiên
GUI	Từ VLM zero-shot, qua SFT tĩnh, đến RL môi trường tĩnh rồi tương tác thật	GUI-R1, UI-TARS, ComputerRL	AndroidWorld, OSWorld, WindowsAgentArena
Vision (ảnh/video/3D)	GRPO với reward kiểm chứng theo tác vụ (IoU, mAP, reward điểm ảnh)	Visual-RFT/Visual-ARFT (bài 55), VLM-R1, TW-GRPO	benchmark detection/segmentation/video
Embodied	RL post-training cho mô hình vision-language-action theo chỉ dẫn ngôn ngữ	mô hình VLA theo chỉ dẫn	môi trường mô phỏng vật lý/robot
Multi-Agent (MAS)	RL tối ưu điều phối phi tham số (topology, agent cố định) hoặc policy agent/MARL end-to-end	GPTSwarm, MaAS, G-Designer	benchmark hợp tác đa agent
Khác	Text game, Text-to-SQL, time series, QA tổng quát, agent xã hội	ARIA, GiGPO, RAGEN; SkyRL-SQL; Time-R1; Sotopia-RL	môi trường theo từng tác vụ

Bảng 9 (ma trận phụ thuộc năng lực x ứng dụng, giữ đầy đủ; dấu chấm biểu thị mức phụ thuộc định tính: • Core (cốt lõi), ◦ Supporting (hỗ trợ), "-" Minimal (tối thiểu)):

Ứng dụng	Planning	Tool-Use	Memory	Self-Imp.	Reasoning	Percep.
Search	•	•	◦	◦	•	-
Code	◦	•	◦	◦	•	-
Math	•	◦	-	◦	•	-
GUI	•	•	◦	-	◦	•
Vision	◦	◦	-	-	•	•
Embodied	•	◦	•	-	◦	•
MAS	•	◦	◦	-	•	◦

Bảng này liên kết trục năng lực (mục 4) với trục ứng dụng (mục 5): ví dụ với Vision, Perception và Reasoning là Core, Memory/Self-Improvement chỉ Minimal, nên khi thiết kế RL cho agent thị giác, nên ưu tiên reward/kiến trúc cho hai năng lực đó trước.



Hình 98: Ma trận phụ thuộc giữa ứng dụng và năng lực của tác tử (Bảng 9) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi hàng là một ứng dụng, mỗi cột là một năng lực; màu đậm là Core, xanh lá nhạt là Supporting, xám là Minimal. Ví dụ hàng Vision có Reasoning và Perception là Core, còn Memory và Self-Imp. chỉ Minimal. Dùng ma trận này để chọn nơi đầu tư reward trước.

6. Môi trường mô phỏng và framework RL

Môi trường (Bảng 10, trích một phần đại diện; paper liệt kê khoảng 37 môi trường, ký hiệu năng lực: ① Reasoning, ② Planning, ③ Tool Use, ④ Memory, ⑤ Collaboration, ⑥ Self-Improve):

Môi trường	Năng lực	Miền nhiệm vụ	Modal
ALFWorld, TextWorld, AgentGym	②①	Embodied, Text Games	Text
WebShop, WebArena	②③	Web	Text
VisualWebArena	①②③	Web	Text, Visual
AndroidWorld, OSWorld	②③	GUI, App/OS	Text, Visual
SWE-bench, R2E-Gym	①②③	SWE	Text
TheAgentCompany	②③⑤	SWE	Text
MLE-Bench, PaperBench	②①③	MLE	Text
Crafter / Craftax, SMAC	②④⑤	Game	Visual
InternBootcamp	①	General, Coding, Logic	Text

Framework (Bảng 11, trích đại diện ba nhóm; paper liệt kê ~23 framework):

Framework	Nhóm	Đặc điểm chính
Verifiers, AREAL, EasyR1	Agentic RL	Môi trường kiểm chứng được, huấn luyện bất đồng bộ, hỗ trợ đa phương thức
Agent Lightning	Agentic RL	RL phân tách theo cấp, gần như không cần sửa code agent
AWorld, ROLL	Agentic RL	Rollout song song trên nhiều cluster/GPU, quy mô lớn
VerlTool	Agentic RL	Rollout tích hợp tool, xây trên Verl
OpenRLHF, TRL, trlX	RLHF/LLM fine-tuning	RLHF hiệu năng cao/baseline Hugging Face/phân tán mô hình lớn
HybridFlow, SLiMe	RLHF/LLM fine-tuning	Quản lý thí nghiệm gọn gàng; kết hợp Megatron + SGLang cho Async RL
RLlib, Tianshou	RL tổng quát	Thư viện RL quy mô sản xuất; nền tảng PyTorch thuần hiệu năng cao
Stable Baselines3, PFRL	RL tổng quát	Cài đặt thuật toán model-free tin cậy, dùng tạo prototype nhanh

7. Phân tích

Mục 6.4 trình bày cuộc tranh luận về cơ chế thật sự khiến RL cải thiện suy luận của LLM, hai trường phái: **"Amplifier"** (RL với reward kiểm chứng được, như GRPO, chủ yếu định hình lại phân phối đầu ra: lấy mẫu nhiều quỹ đạo, khen thưởng quỹ đạo đúng, tập trung khối xác suất vào đường suy luận đã tới được, cải thiện pass@1 nhưng không mở rộng không gian giải pháp; bằng chứng: pass@k với k lớn thường cho thấy mô hình gốc bắt kịp hoặc vượt mô hình đã tinh chỉnh RL) và **"New-knowledge"** (RL có thể cài đặt tính toán mới về chất, khai thác tín hiệu outcome-level thưa và test-time computation dài hơn; lý thuyết cho thấy RL giúp tổng quát hóa trên bài toán mà next-token training đơn thuần không khả thi, ví dụ parity; thực nghiệm cho thấy RL cải thiện tổng quát hóa ngoài phân phối và tạo hành vi nhận thức mới như tự kiểm tra, quay lại, đặt mục tiêu con). Paper nêu rõ câu hỏi này vẫn còn mở.

Case study toán học: khoảng 2/3 nghiên cứu RL cho toán/code chủ yếu cải thiện pass@1 (nhất quán "amplifier"), khoảng 1/3 mở rộng biên pass@k (nhất quán "new-knowledge"). Hành vi "năng lực mới" xuất hiện đáng tin cậy nhất khi có đồng thời ba điều kiện: (i) reward tin cậy cao, thực thi/kiểm chứng được; (ii) cấu trúc nhiều bước có thể chấm điểm từng phần; (iii) mô hình gốc ở vùng "trung gian" (không gần ngẫu nhiên, không gần trần năng lực).

So sánh bốn thuật toán ở mục 3: PPO ổn định nhưng cần critic riêng (tăng tham số); GRPO nhẹ hơn nhưng advantage theo nhóm dễ phương sai cao, sinh ra cả họ biến thể (Bảng 2) chỉ để sửa lỗi cụ thể của GRPO; DPO bỏ hẳn mô hình phần thưởng nhưng phụ thuộc hoàn toàn chất lượng dữ liệu ưu tiên tĩnh.

8. Giới hạn

Tác giả nêu (mục 6, "Open Challenges and Future Directions"), ba nhóm thách thức lớn:

- **Trustworthiness:** RL biến agent từ nạn nhân bị động của tấn công thành chủ thể tích cực khai thác lỗ hổng để tối đa reward (reward hacking: agent học dùng tool độc hại/lách an toàn nếu có lợi cho reward). RLVR outcome-only có thể khuếch đại hallucination ("hallucination tax": mất khả năng từ chối câu hỏi không có lời giải). RLHF có thể làm trầm trọng sycophancy vì mô hình phần thưởng học từ nhãn người có thiên hướng ưa phản hồi dễ chịu. Giải pháp gợi ý: sandbox nghiêm ngặt, reward theo tiến trình, huấn luyện đối kháng, giám sát bất thường sau triển khai.
- **Scaling up Agentic Training:** mở rộng compute (ProRL/ProRLv2: huấn luyện kéo dài mở rộng biên suy luận ngoài mô hình gốc), mô hình lớn hơn dễ entropy collapse, dữ liệu đa domain vừa hỗ trợ vừa gây nhiễu lẫn nhau, và hiệu quả huấn luyện (calibrate độ khó, lấy mẫu đa dạng).

- **Scaling up Agentic Environments:** môi trường phổ biến hiện tại (ALFWorld, ScienceWorld...) được cho là không đủ cho agent tổng quát; xu hướng mới là "đồng tiến hóa" môi trường: agent phụ tự sinh mô hình phần thưởng, hoặc LLM sinh curriculum thích nghi theo điểm yếu hiện tại của agent (ví dụ EnvGen).

Ngoài ba nhóm chính: cuộc tranh luận cơ chế RL chưa ngã ngũ (mục 6.4, đã trình bày ở mục 7); các mẫu hình kiến trúc triển khai thực tế ngoài vòng lặp RL (guardrail, human-in-the-loop, điều phối theo cấp, giao thức liên-agent, mục 6.5); và tác động xã hội rộng hơn, gồm rủi ro dual-use như "sleeper agent" (mô hình an toàn lúc huấn luyện nhưng kích hoạt chính sách ẩn gây hại lúc triển khai, kể cả sau SFT/RLHF, mục 6.6).

Nhận xét:

- Đây là survey nên "giới hạn" là thách thức mở của cả lĩnh vực, không phải giới hạn thực nghiệm của một phương pháp cụ thể, không có bảng số liệu mới để kiểm tra.
- Hình 7, 8 (so sánh hiệu năng RL cho code/toán) là ảnh biểu đồ, pdftotext không lấy được số liệu bên trong: "paper không nêu bằng số" ở dạng text.
- Tổng hợp hơn 500 công trình ở mức khái quát nên các mô tả một dòng có thể bỏ sót chi tiết/điều kiện gốc; đối chiếu với bài 53 (Search-R1) trong lộ trình, điểm mâu thuẫn 24%/41% trong chính bài Search-R1 không được survey nhắc tới, nên trước khi dùng một số liệu cụ thể từ survey, nên tra lại bài gốc.
- Taxonomy hai chiều là cách tổ chức riêng của nhóm tác giả; việc đặt một phương pháp vào "Others" ở trục nào mang tính chủ quan nhất định.

9. Liên hệ với các bài khác

- **Bài 00, 07, 36, 39:** nền toán (MDP, POMDP, PPO, GRPO) và các ví dụ RLVR kinh điển mà survey dùng để minh họa, chất xúc tác cho họ biến thể GRPO trong Bảng 2.
- **Bài 41 (DAPO), 42 (Dr.GRPO), 43 (GSPO), 46 (ProRL):** đều được nêu tên cụ thể trong Bảng 2 (họ GRPO); ProRL/ProRLv2 còn là bằng chứng chính cho luận điểm "mở rộng compute training" ở mục 6.2.
- **Bài 53 (Search-R1):** được chính Bảng 4 của survey liệt kê trong nhóm "External" (search agent dùng search engine thật), cùng các phương pháp họ hàng ngoài lộ trình như R1-Searcher, ReSearch, DeepResearcher, StepSearch, ZeroSearch, SSRL.
- **Bài 55 (Visual-RFT):** nêu tên tường minh ở mục 3.6/4.5 như phương pháp sớm nhất áp GRPO cho thị giác, dùng IoU kèm độ tin cậy làm reward cho bounding box. Survey cũng nhắc "Visual-ARFT" (agent tự tạo tool thị giác, cùng nhóm tác giả) là bài liên quan, không phải cùng bài với 55.

10. Ứng dụng thực tế (suy luận)

- **Danh sách kiểm tra thiết kế:** trước khi gọi một hệ thống là "Agentic RL", tự trả lời theo khung Bảng 1: O là gì, hành động có chia A_{text}/A_{action} không, reward outcome hay step-wise, T có thật sự lớn hơn 1. Nếu bài toán suy biến về PBRFT, nên huấn luyện như RLVR một bước (rẻ, ổn định hơn) thay vì xây hạ tầng rollout đa lượt không cần thiết.
- **Chọn biến thể GRPO theo lỗi quan sát được:** mỗi biến thể trong Bảng 2 sửa một lỗi cụ thể (độ dài, clip mức câu, advantage collapse, reset reference policy); nên tra cột "Cơ chế chính" để chọn biến thể khớp với dấu hiệu thực tế trong training curve, không chọn theo độ phổ biến.
- **Dùng Bảng 9 để ưu tiên reward:** với agent thị giác (OCR), Perception/Reasoning là Core, Memory/Self-Improvement là Minimal; nên đầu tư reward vào tính đúng nội dung đọc được trước khi đầu tư module bộ nhớ.

11. Thí nghiệm gợi ý

1. **Kiểm tra hình thức hóa cho quy trình hiện có:** viết lại tường minh bộ $\langle S, O, A, P, R, T, \gamma \rangle$ cho quy trình RLVR/GRPO hiện tại theo mẫu Bảng 1, rồi tính (tự tính) T thực sự bằng 1 (PBRFT) hay lớn hơn 1 (Agentic RL thật).
2. **So sánh một biến thể GRPO mục tiêu:** chọn biến thể trong Bảng 2 giải quyết đúng lỗi quan sát được trong log huấn luyện (Dr.GRPO nếu nghi độ chệch độ dài, DAPO nếu nghi mẫu không hiệu quả), so sánh đường reward/entropy với GRPO gốc.
3. **Tự chấm ma trận phụ thuộc cho task riêng:** tự cho điểm Core/Supporting/Minimal như Bảng 9 cho sáu năng lực trên tác vụ đang xét, kiểm tra recipe reward hiện tại có phân bổ tương xứng không.

12. Tóm tắt công thức/khái niệm

Khái niệm	Công thức/ý	Ghi chú
Bộ PBRFT	$\langle S_{trad}, A_{trad}, P_{trad}, R_{trad}, T{=}1, \gamma{=}1 \rangle$	MDP suy biến một bước
Bộ Agentic RL	$\langle S_{agent}, A_{agent}, P_{agent}, R_{agent}, \gamma, O \rangle$	POMDP, $T > 1$
Không gian hành động	$A_{agent} = A_{text} \cup A_{action}$	văn bản tự do vs hành động có cấu trúc
Mục tiêu PBRFT	$J = \mathbb{E}[r(a)]$	không chiết khấu
Mục tiêu Agentic RL	$J = \mathbb{E}\big[\sum_t \gamma^t R(s_t, a_t)\big]$	đa bước, có chiết khấu
REINFORCE	$\nabla J = \mathbb{E}[(R - b)\nabla \log \pi]$	baseline giảm phương sai
PPO	$\min(\rho A,\ \text{clip}(\rho, 1{\pm}\epsilon)A)$	cần critic $V(s)$
DPO	hàm hợp lý trên cặp ưu tiên (y_w, y_l)	không cần reward model tường minh
GRPO	$\hat{A} = (R - \text{mean}(R^{(1..G)}))/\text{std}(R^{(1..G)})$	chuẩn hóa theo nhóm, không cần critic
Ma trận phụ thuộc (Bảng 9)	•Core / ◦Supporting / -Minimal	nối trục năng lực với trục miền nhiệm vụ
Debate cơ chế RL	“amplifier” (định hình lại phân phối) vs “new-knowledge” (tạo tính toán mới)	chưa có kết luận dứt khoát theo paper

13. Kết luận ngắn

Survey này không đề xuất phương pháp RL mới, mà đóng góp một khung lý thuyết (MDP suy biến vs POMDP đa lượt) và một bản đồ có cấu trúc (taxonomy năng lực x miền nhiệm vụ, cùng danh mục môi trường và framework) để tổ chức lại một lĩnh vực phát triển rất nhanh và phân tán. Giá trị lớn nhất cho người học là bộ từ vựng chung (PBRFT vs Agentic RL, A_{text} vs A_{action}, Core/Supporting/Minimal) giúp định vị bất kỳ bài RL-cho-LLM cụ thể (36, 39, 41, 42, 43, 46, 53, 55 trong lộ trình này) vào đúng vị trí trong bức tranh tổng thể, cũng như nhận diện các thách thức mở mà từng bài riêng lẻ thường không đặt ra.

Link gốc: https://arxiv.org/abs/2509.02547

Bài 55

Visual-RFT: Visual Reinforcement Fine-Tuning (2025)

Tác giả: Ziyu Liu, Zeyi Sun (đóng góp bằng nhau), Yuhang Zang (tác giả liên hệ), Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, Jiaqi Wang (tác giả liên hệ). Shanghai Jiao Tong University, Shanghai Artificial Intelligence Laboratory, The Chinese University of Hong Kong. **Venue / arXiv:** arXiv:2503.01785 (v1, 03/03/2025); bài báo không nêu venue. **Nguồn:** https://arxiv.org/abs/2503.01785 **Chủ đề:** Mở rộng RLVR (reinforcement learning with verifiable rewards) sang các tác vụ thị giác (nhận diện, phân loại, grounding) bằng cách thiết kế reward có thể kiểm chứng bằng luật (IoU reward, CLS reward) thay vì mô hình phần thưởng học từ sở thích người dùng, tối ưu bằng GRPO trên LVLM (Qwen2-VL). **Vị trí trong lộ trình:** Đọc trước: bài 00 (MDP, policy gradient, GRPO, RLVR), 36 (DeepSeekMath, nơi GRPO ra đời), 39 (DeepSeek-R1, nguồn cảm hứng trực tiếp: RLVR + GRPO không cần SFT cho suy luận). Bài này là bài đầu tiên của nhóm "RL cho VLM/agent" (53-57) chuyển trọng tâm từ suy luận văn bản thuần sang **nhận thức thị giác** (perception): nó định nghĩa khuôn mẫu reward có thể kiểm chứng cho các tác vụ thị giác cổ điển (IoU, độ chính xác phân loại), là nền cho 56 và 57 áp dụng ý tưởng tương tự vào OCR tài liệu.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

RFT (Reinforcement Fine-Tuning) là tên OpenAI đặt cho chương trình tinh chỉnh o1 bằng RL với chỉ vài chục đến vài nghìn mẫu, khác hẳn SFT (supervised fine-tuning) vốn cần dữ liệu lớn vì học theo kiểu "ghi nhớ" câu trả lời đúng cho sẵn. RFT đánh giá câu trả lời của model và điều chỉnh theo đúng/sai, tức học kiểu thử-sai, nên hợp với miền dữ liệu khan hiếm.

RLVR (Reinforcement Learning with Verifiable Rewards): thay vì dùng mô hình phần thưởng huấn luyện từ dữ liệu sở thích (preference data, như RLHF cổ điển), RLVR dùng **hàm kiểm chứng trực tiếp** (rule-based) để tính điểm đúng/sai, dựa trên bản chất "có đáp án đúng khách quan" của bài toán (toán, code). DeepSeek-R1-Zero (bài 39) cho thấy RLVR kết hợp GRPO có thể dạy suy luận mà **không cần giai đoạn SFT**. Nhưng các nghiên cứu RLVR trước Visual-RFT hầu như chỉ dừng ở miền ngôn ngữ (toán, code); việc áp dụng cho LVLM (large vision-language model) còn hạn chế, RL cho LVLM trước đó chủ yếu nhắm vào giảm ảo giác hoặc alignment theo sở thích người dùng, chứ chưa nhắm vào **nhận thức thị giác** (object detection, phân loại, grounding).

GRPO (Group Relative Policy Optimization, nguồn: DeepSeekMath, bài 36**):** khác PPO ở chỗ không cần một critic model riêng để ước lượng giá trị trạng thái. Với một câu hỏi q, lấy mẫu G phản hồi $\{o_1, ..., o_G\}$ từ policy hiện tại $\pi_{\theta_{old}}$, tính thưởng $\{r_1, ..., r_G\}$ cho từng phản hồi, rồi **chuẩn hoá theo nhóm** (trừ trung bình, chia độ lệch chuẩn của chính nhóm đó) để ra lợi thế (advantage) tương đối, không cần baseline học riêng.

Thuật ngữ cần biết: **Policy model** π_θ: LVLM đang huấn luyện, nhận ảnh + câu hỏi, sinh chuỗi gồm quá trình suy luận và câu trả lời. **Reference model** π_{ref}: bản sao cố định của policy trước khi RL, dùng để tính phạt KL giữ policy không trôi quá xa. **Few-shot:** chỉ có vài đến vài trăm ảnh gán nhãn cho một hạng mục. **Open-vocabulary detection:** phát hiện vật thể thuộc các lớp (category) chưa từng xuất hiện lúc huấn luyện.

2. Ý tưởng chính

Visual-RFT giữ nguyên khung RLVR + GRPO của DeepSeek-R1 nhưng thay hàm thưởng nhị phân "đúng/sai" (vốn chỉ hợp với toán/code) bằng các **hàm thưởng có thể kiểm chứng dành riêng cho nhận thức thị giác**: thưởng IoU (Intersection over Union) cho object detection, thưởng độ chính xác phân loại

(CLS) cho fine-grained classification, cộng với thưởng định dạng (format) để ép model xuất đúng khuôn `<think>...</think><answer>...</answer>`. Với mỗi ảnh và câu hỏi, policy model sinh G phản hồi có kèm quá trình suy luận, mỗi phản hồi được chấm điểm tự động bằng hàm thưởng rule-based (không cần mô hình phần thưởng, không cần người gán nhãn preference), rồi GRPO cập nhật model theo lợi thế tương đối trong nhóm, có phạt KL so với policy tham chiếu. Vì reward rẻ và tự động, Visual-RFT chỉ cần **10-1000 mẫu** huấn luyện, và vượt trội SFT rõ rệt trên 4 nhóm tác vụ: phân loại ảnh ít mẫu, phát hiện vật thể ít mẫu, grounding theo suy luận, và phát hiện vật thể từ vựng mở, đặc biệt khi dữ liệu cực ít (SFT có thể còn làm **giảm** điểm so với baseline).

3. Phương pháp chi tiết

3.1 Mục tiêu RLVR (nhắc lại khung chung)

Với câu hỏi q (ảnh + chỉ dẫn), policy π_θ sinh phản hồi o, nhận thưởng kiểm chứng được:

$$\max_{\pi_\theta} \mathbb{E}_{o\sim\pi_\theta(q)}\big[R_{RLVR}(q,o)\big] = \mathbb{E}\Big[R(q,o) - \beta\,\mathrm{KL}\big[\pi_\theta(o|q)\,\|\,\pi_{ref}(o|q)\big]\Big]$$

Dạng tổng quát (toán/code) của R chỉ là so khớp đáp án:

$$R(q,o) = \begin{cases} 1, & o = \text{ground truth} \\ 0, & \text{khác} \end{cases}$$

Visual-RFT giữ nguyên vế phạt KL này, chỉ **thay** R bằng các hàm thưởng thị giác bên dưới (3.3, 3.4). Bài báo không cho giá trị cụ thể của β dùng trong thí nghiệm.

3.2 GRPO: lợi thế chuẩn hoá theo nhóm

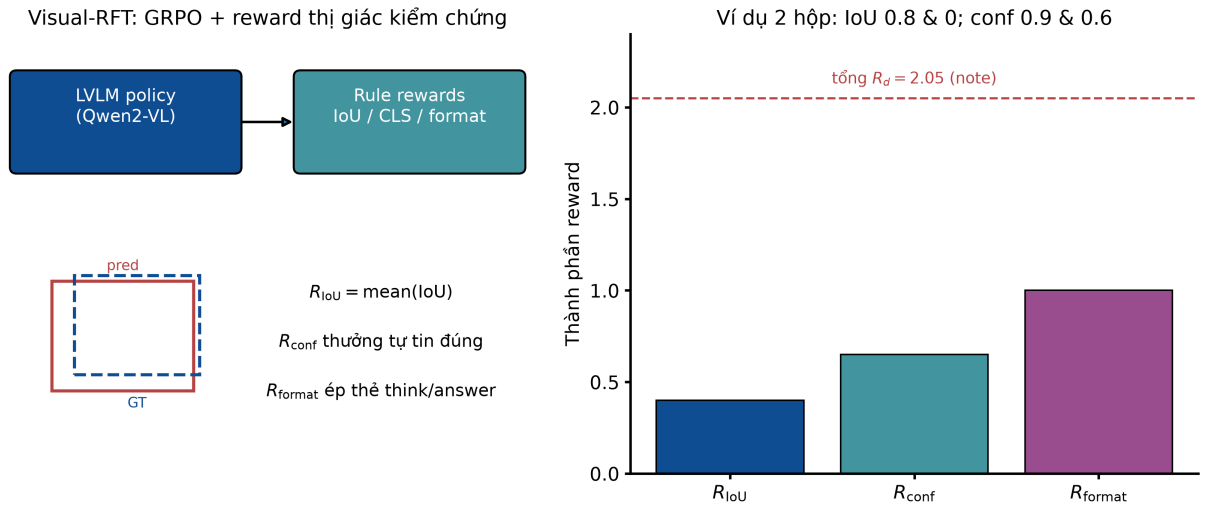
Với G phản hồi $\{o_1, ..., o_G\}$ lấy mẫu từ $\pi_{\theta_{old}}$ và thưởng tương ứng $\{r_1, ..., r_G\}$:

$$\boxed{A_i = \frac{r_i - \text{mean}(\{r_1, ..., r_G\})}{\text{std}(\{r_1, ..., r_G\})}}$$

A_i dương nghĩa là phản hồi i tốt hơn trung bình nhóm, GRPO khuyến khích policy tăng xác suất sinh các phản hồi có A_i cao. Bài báo không nêu giá trị G (số mẫu mỗi nhóm) dùng trong thí nghiệm, không nêu learning rate, optimizer, số bước huấn luyện chi tiết theo từng cấu hình (trừ hai con số ở 3.5), và không nêu phần cứng/GPU dùng để huấn luyện.

3.3 IoU reward cho object detection

Đầu ra model gồm danh sách hộp giới hạn (bounding box) kèm độ tin cậy (confidence). Sắp các hộp dự đoán theo confidence giảm dần $\{b_1, ..., b_n\}$, khớp từng b_i với hộp ground truth gần nhất $\{b_{g1}, ..., b_{gm}\}$, tính IoU; đặt ngưỡng τ (bài báo không cho giá trị τ): IoU dưới τ hoặc không khớp được coi là 0. Kết quả là tập $\{iou_1 : c_1, ..., iou_n : c_n\}$. Thưởng tổng:



Visual-RFT: $R_{\text{IoU}} + R_{\text{conf}} + R_{\text{format}}$; ví dụ tổng $R_d = 2.05$ (minh hoạ, tự tính)

$$\boxed{R_d = R_{IoU} + R_{conf} + R_{format}}$$

$$R_{IoU} = \frac{iou_1 + iou_2 + \cdots + iou_n}{n}$$

Thưởng tin cậy R_{conf} khuyến khích model "biết mình đúng/sai": nếu hộp khớp thành công ($iou_i \neq 0$) thì thưởng chính là độ tin cậy đã khai báo (càng tự tin khi đúng càng tốt); nếu không khớp ($iou_i = 0$) thì thưởng là $1 - c_i$ (càng **không** tự tin khi sai càng tốt):

$$r_{c_i} = \begin{cases} c_i, & iou_i \neq 0 \\ 1 - c_i, & iou_i = 0 \end{cases} \qquad R_{conf} = \frac{1}{n}\sum_{i=1}^{n} r_{c_i}$$

R_{format} là thưởng nhị phân ép đúng khuôn HTML-tag `<think>...</think><answer>...</answer>`; bài báo không mô tả công thức/giá trị cụ thể, chỉ nói mục đích.

Ví dụ nhỏ (tự tính, không có trong bài báo): model xuất 2 hộp, hộp 1 có $c_1 = 0.9$ khớp GT với $iou_1 = 0.8$; hộp 2 có $c_2 = 0.6$ không khớp GT nào ($iou_2 = 0$). $R_{IoU} = (0.8 + 0)/2 = 0.4$. $r_{c_1} = 0.9$ (đúng, tự tin cao là tốt), $r_{c_2} = 1 - 0.6 = 0.4$ (sai nhưng khá tự tin nên thưởng thấp); $R_{conf} = (0.9 + 0.4)/2 = 0.65$. Nếu định dạng đúng ($R_{format} = 1$): $R_d = 0.4 + 0.65 + 1 = 2.05$.

3.4 CLS reward cho phân loại

$$\boxed{R_{cls} = R_{acc} + R_{format}}, \qquad R_{acc} = \begin{cases} 1, & \text{lớp dự đoán = lớp đúng} \\ 0, & \text{khác} \end{cases}$$

3.5 Prompt và dữ liệu

Prompt detection yêu cầu model trả về hộp (toạ độ 0-1000, số nguyên) kèm confidence (0-1, 2 chữ số thập phân), hoặc `'No Objects'` nếu không có; prompt classification yêu cầu tên loài. Cả hai đều ép khuôn `<think>...</think><answer>...</answer>`. Huấn luyện theo kiểu **few-shot**: với phân loại và phát hiện ít mẫu, lấy mẫu trực tiếp làm tập train RL (10-1000 ảnh); với phát hiện từ vựng mở, lấy 6K annotation từ 65 lớp cơ sở của COCO. Số bước huấn luyện cụ thể: **200 bước** cho các thí nghiệm few-shot detection (COCO 8

lớp, LVIS 6 lớp hiếm), **500 bước** cho LISA reasoning grounding (239 ảnh train). Bài báo không nêu số bước cho open-vocabulary detection và fine-grained classification.

```
Pseudo-code (suy từ mô tả mục 3.1-3.2, paper không cho code):
for mỗi (ảnh, câu hỏi) q trong tập train (10-1000 mẫu):
    sample G phản hồi {o_1..o_G} ~ pi_theta_old(.|q)   # mỗi o chứa
    ↪  <think>..</think><answer>..</answer>
    for i in 1..G:
        parse o_i -> dự đoán (list hộp+confidence, hoặc nhãn lớp)
        if detection: r_i = R_IoU + R_conf + R_format   (khớp hộp theo 3.3)
        if classification: r_i = R_acc + R_format        (3.4)
    A_i = (r_i - mean(r_1..r_G)) / std(r_1..r_G)
    cập nhật theta: tối đa hoá E[A_i] - beta * KL(pi_theta || pi_ref)   (kiểu PPO, không
    ↪  critic)
```

4. Thí nghiệm và kết quả chính

Model nền: Qwen2-VL-2B và Qwen2-VL-7B. **Baseline so sánh:** mô hình gốc (chưa tinh chỉnh) và SFT trên cùng tập dữ liệu ít mẫu.

Bảng 2: Phân loại ảnh ít mẫu (4 bộ: Flower102, Pets37, FGVC-Aircraft, Cars196; Qwen2-VL-2B; cột "Trung bình" là trung bình 4 bộ).

Thiết lập	Trung bình	Flower102	Pets37	FGVC	Cars196
Baseline	56.0	54.8	66.4	45.9	56.8
1-shot +SFT	51.7 (**-4.3**)	56.6	54.7	65.3	30.0
1-shot +Visual-RFT	80.3 (**+24.3**)	70.8	84.1	72.5	93.8
4-shot +SFT	55.6	58.5	55.5	67.9	40.5
4-shot +Visual-RFT	81.9 (+25.9)	71.4	86.1	74.8	95.3
16-shot +SFT	64.0	66.8	71.6	76.1	41.5
16-shot +Visual-RFT	85.3 (+29.3)	79.2	87.1	79.4	95.3

Ở 1-shot, SFT **giảm** so với baseline (-4.3%) trong khi Visual-RFT tăng +24.3%. Chênh lệch Visual-RFT so baseline tăng dần theo số shot (+24.3 → +29.3 từ 1 đến 16-shot).

Bảng 3 (rút gọn, Qwen2-VL-2B, COCO 8 lớp: bus, train, fire hydrant, stop sign, cat, dog, bed, toilet; cột mAP):

Shot	Baseline mAP	+SFT mAP	+Visual-RFT mAP	Δ (RFT - baseline)
1-shot	19.6	19.5	33.6	+14.0
2-shot	19.6	21.0	41.5	+21.9
4-shot	19.6	25.2	40.6	+21.0
8-shot	19.6	30.2	47.4	+27.8
16-shot	19.6	31.3	46.8	+27.2

(Qwen2-VL-7B, baseline mAP 43.0, 4-shot +SFT 44.1, +Visual-RFT 54.3, Δ +11.3.)

Bảng 4 (LVIS 6 lớp hiếm: horse buggy, die, kitchen table, omelet, papaya, stepladder; 10-shot xấp xỉ):

Model	Baseline mAP	+SFT mAP	+Visual-RFT mAP	Δ
Qwen2-VL-2B	4.0	10.0	19.4	+15.4
Qwen2-VL-7B	15.4	27.6	33.8	+18.4

Đáng chú ý ở 2B: lớp "stepladder" từ 0.0 (baseline và SFT) lên **10.9** (Visual-RFT); SFT hoàn toàn không học được lớp này từ 10 ảnh trong khi RL học được.

Bảng 5 (MG - Monster Girls, 5 lớp ngoài phân phối gốc, anime, Qwen2-VL-2B): baseline trung bình mAP 20.6; 4-shot +SFT 26.8, +Visual-RFT **61.8** (+41.2 so baseline, SFT chỉ +6.2); 16-shot +SFT 51.3, +Visual-RFT 63.4. Dù dữ liệu hoàn toàn khác phân phối pretraining, RL vẫn khái quát tốt hơn SFT nhiều.

Bảng 6 (LISA reasoning grounding, 239 ảnh train, 500 bước):

Model	mIoU_test	mIoU_val	gIoU_test
OV-Seg / X-Decoder / GroundedSAM (chuyên dụng)	26.2-28.5	28.6-30.5	21.3-26.1
Qwen2-VL-2B baseline	26.9	30.1	25.3
+SFT	28.3	29.7	25.3
+Visual-RFT	**37.6** (+10.7)	**34.4** (+4.3)	**34.4** (+9.1)
Qwen2-VL-7B baseline	40.4	45.2	38.0
+Visual-RFT	**43.9** (+3.5)	47.1 (+1.9)	**43.7** (+5.6)

gIoU đo bằng cách dùng hộp Qwen2-VL dự đoán làm prompt cho SAM để sinh mask phân đoạn, rồi so gIoU. Visual-RFT vượt cả các hệ thống grounding chuyên dụng (GroundedSAM...) dù chỉ huấn luyện trên 239 ảnh.

Bảng 7 (Open-vocabulary detection COCO, train 65 lớp cơ sở, test 15 lớp mới):

Model	mAP_novel	mAP_base	mAP_all
Qwen2-VL-2B baseline	9.8	6.0	6.7
+SFT	13.6	7.8	8.9
+Visual-RFT	**31.3** (+21.5)	**20.6** (+14.6)	**22.6** (+15.9)
Qwen2-VL-7B baseline	26.3	17.5	19.2
+Visual-RFT	**35.8** (+9.5)	**25.4** (+7.9)	**27.4** (+8.2)

Bảng 8 (test trên 13 lớp hiếm LVIS, cùng model train ở Bảng 7): mAP trung bình Qwen2-VL-2B: baseline 2.7 → +SFT 7.6 → +Visual-RFT **20.7** (+18.0). Một số lớp hiếm SFT giữ nguyên 0.0 AP (egg roll, mallet) trong khi Visual-RFT đạt 2.0 và 11.1; GroundingDINO-B (hệ chuyên detection) đạt mAP 23.9, gần bằng Visual-RFT-2B (20.7) dù Visual-RFT chỉ tinh chỉnh LVLM tổng quát trên 65 lớp khác hẳn 13 lớp test.

5. Phân tích / ablation

Bài báo không có mục ablation tách riêng từng thành phần reward (R_{IoU} so với R_{conf}, có/không R_{format}), không có ablation số shot liên tục hay β/G. Những gì quan sát được trực tiếp từ các bảng:

- **SFT có thể gây hại ở chế độ cực ít dữ liệu:** 1-shot classification SFT thấp hơn cả baseline chưa tinh chỉnh (-4.3%), trong khi Visual-RFT luôn tăng. Khoảng cách này thu hẹp khi tăng shot (SFT bắt đầu vượt baseline từ 8-shot) nhưng Visual-RFT luôn dẫn trước nhiều.

- **"Nhảy từ 0":** trên các lớp hiếm LVIS/rare categories, mô hình gốc và SFT thường giữ nguyên AP = 0 (không nhận dạng được), trong khi Visual-RFT đưa một số lớp lên hai chữ số (ví dụ egg roll, futon, stepladder). Tác giả diễn giải đây là bằng chứng Visual-RFT giúp model **hiểu nhiệm vụ thật sự** thay vì chỉ ghi nhớ; cần lưu ý đây vẫn là suy diễn định tính, không có thước đo "hiểu" tách biệt khỏi điểm số.
- **Khái quát miền ngoài phân phối (MG, anime):** chênh lệch Visual-RFT so SFT ở đây lớn hơn nhiều dạng ảnh thường (ví dụ +41.2 so baseline ở 4-shot), gợi ý RL ít phụ thuộc "ăn khớp phong cách ảnh" hơn SFT.
- **Mô hình lớn hơn vẫn hưởng lợi nhưng ít hơn tương đối:** 7B luôn có Δ (Visual-RFT - baseline) nhỏ hơn 2B ở mọi bảng có cả hai (ví dụ open-vocab mAP_all: 2B +15.9 so 7B +8.2), phù hợp trực giác mô hình lớn đã có baseline cao hơn nên "còn ít chỗ để tăng".

6. Giới hạn

Tác giả nêu: Không có mục giới hạn riêng trong bài (báo cáo ngắn, khuôn CVPR, không có "Limitations"/"Future Work" tường minh).

Nhận xét:

- Không báo cáo seed, số lần lặp thí nghiệm, hay khoảng tin cậy/độ lệch chuẩn cho bất kỳ con số nào trong 7 bảng kết quả; chênh lệch nhỏ (ví dụ Δ 1-2 điểm) khó biết có ý nghĩa thống kê không.
- Không nêu learning rate, optimizer, batch size, G (số mẫu mỗi nhóm GRPO), β (hệ số KL), ngưỡng τ (IoU reward), và không nêu phần cứng/thời gian huấn luyện cho bất kỳ thí nghiệm nào, nên **không thể tái lập chính xác** từ bài báo.
- Không ablation riêng từng thành phần reward (R_{IoU}, R_{conf}, R_{format}) nên không biết phần nào đóng góp bao nhiêu vào kết quả tổng.
- So sánh SFT có thể chưa được tinh chỉnh kỹ (không nêu learning rate/epoch cho SFT), nên khoảng cách SFT vs Visual-RFT một phần có thể do baseline SFT yếu hơn mức tối ưu, không chỉ do ưu thế nội tại của RLVR.
- Prompt yêu cầu parse chặt theo khuôn `<answer>[...]</answer>`; tỉ lệ phản hồi hỏng định dạng (parse fail) xử lý thế nào, bài báo không nêu.
- Benchmark detection dùng mAP; cách tính (ngưỡng IoU 0.5:0.95 hay single-threshold) bài báo không nêu chi tiết.

7. Liên hệ với các bài khác

- **00 (RLVR, GRPO, policy gradient):** khung lý thuyết nền; Visual-RFT chỉ thay hàm thưởng.
- **36 DeepSeekMath:** nguồn gốc công thức GRPO dùng nguyên văn ở đây (advantage chuẩn hoá std theo nhóm).
- **39 DeepSeek-R1 / R1-Zero:** cảm hứng trực tiếp "RL thuần, không cần SFT, dùng reward kiểm chứng được"; Visual-RFT là bước chuyển sang thị giác.
- **42 Dr. GRPO:** phê bình chuẩn hoá advantage theo std (công thức mục 3.2) vì có thể gây thiên lệch độ dài/độ khó; Visual-RFT dùng đúng công thức này nên đáng đọc bài 42 để hiểu rủi ro.
- **43 GSPO, 41 DAPO:** các cải tiến GRPO sau này (importance sampling mức chuỗi, dynamic sampling loại nhóm có advantage=0) không được dùng ở đây; nhóm có mọi phản hồi đúng/sai như nhau (ví dụ ảnh dễ, mọi o_i đều IoU cao) sẽ cho $A_i \approx 0$ với mọi i, lãng phí rollout, nhưng bài báo không bàn vấn đề này.
- **53 Search-R1, 54 agentic RL survey:** cùng nhóm "RL cho LVLM/agent" của lộ trình; Visual-RFT hẹp hơn (một bước suy luận, không multi-turn).
- **56 olmOCR 2, 57 FD-RL (hai bài tiếp theo):** áp dụng chính tinh thần "reward kiểm chứng được bằng luật" của Visual-RFT vào OCR tài liệu; olmOCR 2 dùng unit test nhị phân (tương tự R_{format}/R_{acc} ở

đây nhưng cho văn bản dài), FD-RL tách reward theo loại nội dung (plain text/formula/table) giống tinh thần $R_d = R_{IoU} + R_{conf} + R_{format}$ (tổng nhiều thành phần theo loại lỗi).

- **llm 37 Qwen2-VL:** kiến trúc nền dùng xuyên suốt bài này (2B và 7B).

8. Ứng dụng thực tế (suy luận)

Visual-RFT cho thấy các tác vụ thị giác có nhãn kiểm chứng được có thể dùng reward dựa trên IoU, lớp hoặc định dạng. Các kết quả ít mẫu và ngoài phân phối là động lực để thử cách này, nhưng không bảo đảm rằng RL luôn thay thế được SFT. Cần kiểm tra chất lượng của bộ kiểm chứng và chi phí rollout trong từng tác vụ.

9. Thí nghiệm gợi ý

1. So sánh RLVR với SFT trên cùng tập ví dụ cho một tác vụ phát hiện đối tượng.
2. Tách ảnh hưởng của reward IoU, confidence và định dạng, đo riêng trên các nhóm lỗi.
3. Đo tỉ lệ nhóm rollout có phần thưởng đồng nhất để đánh giá khả năng áp dụng dynamic sampling.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
RLVR	$\max \mathbb{E}[R(q,o) - \beta\,\mathrm{KL}(\pi_\theta \Vert \pi_{ref})]$	reward kiểm chứng bằng luật, không reward model
GRPO advantage	$A_i = (r_i - \text{mean})/\text{std}$ theo nhóm G	không cần critic
IoU reward	$R_{IoU} = \frac{1}{n}\sum iou_i$	khớp hộp theo confidence, ngưỡng τ (không nêu giá trị)
Confidence reward	$r_{c_i} = c_i$ nếu khớp, $1 - c_i$ nếu không	phạt tự tin sai
Detection reward	$R_d = R_{IoU} + R_{conf} + R_{format}$	
CLS reward	$R_{cls} = R_{acc} + R_{format}$	$R_{acc} \in \{0, 1\}$
Dữ liệu	10-1000 mẫu (few-shot), 6K annotation (open-vocab)	200 bước (detection), 500 bước (LISA)
Kết quả nổi bật	1-shot CLS: +24.3% (SFT -4.3%); OV-det COCO novel: +21.5 mAP (2B)	

11. Kết luận ngắn

Visual-RFT chứng minh rằng công thức RLVR + GRPO vốn thành công cho suy luận toán/code (DeepSeek-R1) có thể mở rộng sang nhận thức thị giác chỉ bằng cách thiết kế lại hàm thưởng (IoU reward, CLS reward) sao cho kiểm chứng được tự động, không cần mô hình phần thưởng hay dữ liệu preference. Với chỉ 10-1000 mẫu, Visual-RFT vượt SFT rõ rệt trên phân loại ít mẫu, phát hiện vật thể ít mẫu, grounding theo suy luận và phát hiện từ vựng mở, kể cả trên dữ liệu ngoài phân phối gốc, trong khi SFT ở chế độ cực ít dữ liệu đôi khi còn làm giảm hiệu năng. Đây là bài học trực tiếp cho hai bài tiếp theo trong lộ trình (olmOCR 2, FD-RL), nơi "hàm thưởng kiểm chứng được" được thiết kế lại cho OCR tài liệu thay vì detection/classification.

Link gốc: https://arxiv.org/abs/2503.01785

Bài 56

olmOCR 2: Unit Test Rewards for Document OCR (2025)

Tác giả: Jake Poznanski, Luca Soldaini, Kyle Lo (Allen Institute for AI). **Venue / arXiv:** arXiv:2510.19817 (v1, 22/10/2025); bài báo không nêu venue. **Nguồn:** https://arxiv.org/abs/2510.19817 **Chủ đề:** RLVR cho OCR tài liệu dùng **unit test nhị phân** làm reward; quy trình sinh dữ liệu tổng hợp (render trang thật thành HTML, rồi tự động trích unit test từ HTML) để scale số lượng test đủ cho RL; GRPO trên Qwen2.5-VL-7B, model souping nhiều seed. **Vị trí trong lộ trình:** Đọc sau bài 55 (Visual-RFT), llm 42 (olmOCR) và bài 00 (GRPO, RLVR). Bài 57 đánh giá olmOCR 2 trên OmniDocBench và bàn về một thiết kế reward khác.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

olmOCR (llm 42) tinh chỉnh Qwen2-VL-7B bằng SFT trên nhãn "bạc" do GPT-4o sinh, và giới thiệu **olmOCR-Bench**: bộ ~7.000 **unit test** nhị phân (pass/fail) kiểm tra từng tính chất cụ thể của output OCR, thay vì dùng edit-distance liên tục so với một bản "ground truth" duy nhất. olmOCR 2 đặt câu hỏi: nếu unit test đã là cách đánh giá tốt, **tại sao không dùng chính nó làm phần thưởng để huấn luyện RL?** Trở ngại: unit test gốc của olmOCR-Bench được người làm thủ công, tốn hàng giờ mỗi bộ, không scale được cho RL (cần hàng chục nghìn mẫu).

Thuật ngữ cần biết: **RLVR (RL with Verifiable Rewards):** phần thưởng tính bằng hàm kiểm chứng rule-based, không cần mô hình phần thưởng học từ preference. **GRPO:** lấy G mẫu mỗi prompt, chuẩn hoá thưởng theo nhóm thành advantage, không cần critic (chi tiết xem bài 00, 36). **Unit test (olmOCR-Bench):** 6 loại kiểm tra nhị phân trên một trang: *Text Presence* (đoạn văn phải xuất hiện), *Text Absence* (đoạn văn, ví dụ header/footer/số trang, phải **không** xuất hiện), *Natural Reading Order* (câu A phải đứng trước câu B), *Table Accuracy* (quan hệ vị trí tương đối giữa các ô có giá trị cho trước), *Math Formula Accuracy* (LaTeX dự đoán phải **render giống hệt** bằng KaTeX so với LaTeX gốc, so bằng vị trí ký hiệu chứ không so chuỗi), *Baseline Robustness* (không có n-gram lặp dài, không lẫn ký tự ngôn ngữ sai).

Vì sao không dùng edit distance làm reward? Hai lý do tác giả nêu, minh hoạ bằng 2 ví dụ trong bài báo: (1) *Đối xử công bằng với "hoà":* các phần tử nổi (bảng, hình, chú thích) không có một vị trí "đúng duy nhất"; ví dụ chú thích hình có thể hợp lệ dù đặt trước hay sau đoạn văn chính, nhưng edit distance phạt khác nhau tuỳ vị trí dù cả hai đều đúng về mặt đọc hiểu. (2) *Điểm liên tục không đo đúng "tính đúng":* với công thức toán, hai chuỗi LaTeX có edit distance khác xa nhau (ví dụ 37,84 so với 9,33 ký tự lệch) có thể **render ra hình ảnh giống hệt nhau** (cùng vị trí ký hiệu tương đối) hoặc ngược lại chuỗi gần giống về ký tự lại render sai; unit test theo kiểu "render rồi so vị trí ký hiệu bằng KaTeX" bắt đúng bản chất hơn là so chuỗi.

2. Ý tưởng chính

olmOCR 2 gồm hai phần bổ sung cho nhau: (1) **quy trình dữ liệu tổng hợp** lấy trang PDF thật, dùng một VLM tổng quát (Claude Sonnet 4) sinh ra bản HTML "gần giống hệt" trang đó, rồi từ cấu trúc ngữ nghĩa của HTML đó **tự động sinh unit test** (không cần người duyệt tay từng test); (2) áp dụng **GRPO** lên model olmOCR nền, dùng chính các unit test tổng hợp này làm phần thưởng nhị phân/phân số (tỉ lệ test pass). Kết hợp thêm nhiều cải tiến hạ tầng khác (temperature động, sửa lỗi prompt, đổi trainer, đổi định dạng đầu ra JSON → YAML, ảnh lớn hơn, đổi mô hình nền, souping nhiều seed), olmOCR 2 đạt **82,4 ± 1,1** trên olmOCR-Bench (phiên bản benchmark mới nhất tại thời điểm viết bài), tăng **+14,2 điểm** so với olmOCR bản đầu (6 tháng trước), và tác giả công bố đầy đủ model, data, code dưới giấy phép mở.

3. Phương pháp chi tiết

3.1 Scale hoá việc tạo unit test cho RL

Nguồn PDF: chọn tài liệu khó OCR có chủ đích theo loại test muốn tăng cường, ví dụ lấy paper arXiv nặng công thức toán để tạo nhiều test công thức, thay vì chỉ vài template dựng sẵn, nên đa dạng về bố cục.

Chuyển PDF → HTML (3 bước, lặp lại bằng cùng một VLM tổng quát):

1. *Phân tích bố cục:* đưa ảnh trang, hỏi VLM về số cột, có bảng/ảnh không, có header/footer không... để định hướng việc sinh HTML sau (tăng độ phủ các loại unit test).
2. *Sinh nội dung:* đưa lại output bước 1 cùng ảnh trang, yêu cầu "render trang này thành HTML sạch, ngữ nghĩa" khớp kích thước ảnh gốc.
3. *Tinh chỉnh đầu ra:* render HTML vừa sinh thành ảnh, đưa ảnh đó + ảnh gốc + HTML cho VLM, yêu cầu sửa HTML cho khớp hơn.

Sinh unit test từ HTML: vì bước 1 đã yêu cầu header/footer nằm trong thẻ `<header>`/`<footer>`, có thể tự động tạo test "Text Absence" cho các thẻ đó; công thức toán trong HTML render bằng KaTeX nên trích trực tiếp làm test công thức; bảng trích từ HTML, chọn ngẫu nhiên ô tạo test bảng.

Chi phí và độ bền: dùng `claude-sonnet-4-20250514`, chi phí **≈ 0,12 USD/trang**. Vì unit test chỉ dựa vào HTML (không dựa vào việc Claude tự OCR trang), **quy trình bền với ảo giác** của VLM tổng quát: dù Claude đọc sai một chữ khi mô tả, miễn cấu trúc HTML đúng thì test vẫn đúng. Kết quả: **olmOCR2-synthmix-1025**: 2.186 trang PDF, tổng **30.381 test case**.

Song song, SFT mix cũng được làm mới: **olmOCR-mix-1025** (267.962 trang từ hơn 100.000 PDF, gồm 9.828 trang từ national archives), xử lý lại bằng GPT-4.1 thay GPT-4o, định dạng công thức nhất quán hơn (`\[`, `\(`), bảng dùng HTML, có alt text cho ảnh. Tinh chỉnh 1 epoch riêng từng mix cho kết quả gần như ngang nhau trên olmOCR-Bench (olmOCR-mix-0225: **78,5 ± 1,1**; olmOCR-mix-1025: **78,3 ± 1,2**), tức mix mới không tự nó cải thiện điểm, giá trị của nó nằm ở chất lượng định dạng cho các bước sau.

3.2 Huấn luyện RL

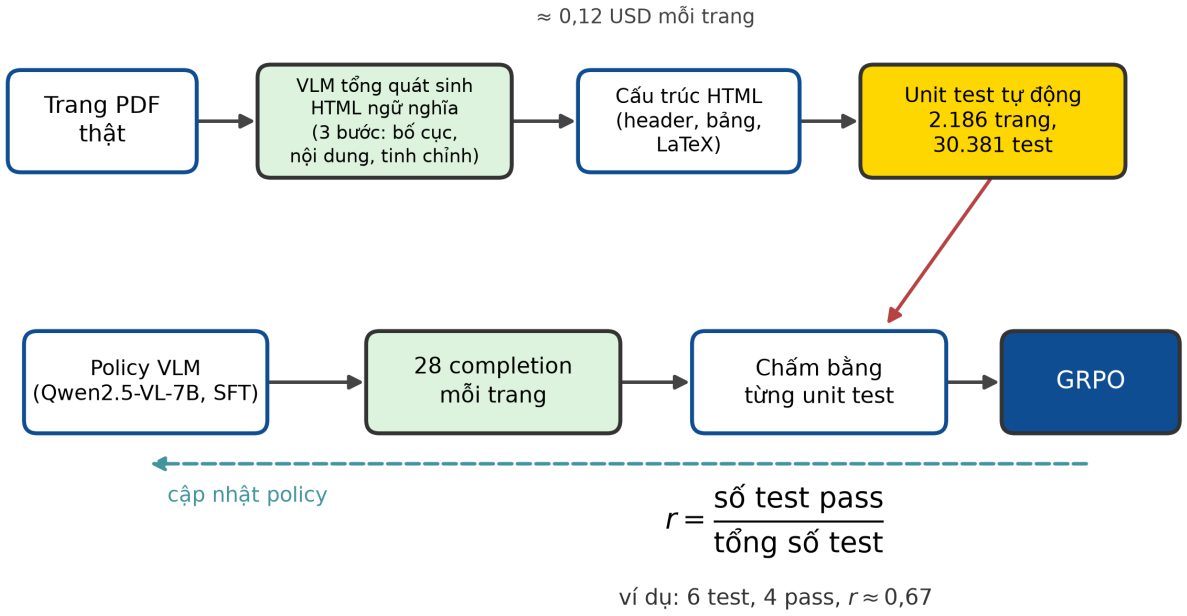
Xuất phát từ Qwen2.5-VL-7B-Instruct đã SFT trên olmOCR-mix-1025 (theo công thức llm 42). Train **1 epoch** trên olmOCR2-synthmix-1025, trên **1 node 8×H100**. Với mỗi tài liệu, sinh **28 completion**. Mỗi completion chấm bằng đúng luật chấm điểm của olmOCR-Bench: mỗi test case pass/fail, **phần thưởng chính = tỉ lệ test pass** (từ 0,0 đến 1,0).

$$\boxed{r = \frac{\text{số unit test pass}}{\text{tổng số unit test của trang}}}$$

Ví dụ minh hoạ trong bài báo (Hình 4): trang có 6 test, 4 pass → $r = 4/6 \approx 0,67$.

Cách đọc hình: hàng trên là khâu tạo dữ liệu, không cần người duyệt từng test vì test được rút từ cấu trúc HTML. Hàng dưới là vòng huấn luyện: mỗi trang sinh 28 completion, mỗi completion được chấm bằng các unit test, tỉ lệ đạt trở thành reward cho GRPO. Các con số 2.186 trang, 30.381 test và 0,12 USD lấy từ mục 3.1.

Ngoài phần thưởng chính, cộng thêm **hai phần thưởng phụ để ép đúng định dạng đầu ra**: (i) thưởng nhị phân cho việc completion có kết thúc bằng token EOS hay không (chống lặp vô hạn); (ii) thưởng từ 0 đến 1 cho việc mô hình có xuất đúng metadata tài liệu ở đầu phản hồi hay không (ngôn ngữ chính, hệ số xoay trang...). Paper **không nêu trọng số/cách cộng gộp cụ thể** giữa 3 thành phần thưởng này (có cộng trực tiếp hay có hệ số riêng, không nói rõ).



Hình 99: Quy trình olmOCR 2: từ trang PDF thật sinh HTML, rút unit test tự động rồi dùng tỉ lệ test đạt làm phần thưởng GRPO (minh hoạ, tự tính)

Dùng thư viện **Hugging Face TRL**, **hệ số KL** $\beta = 0,01$. Để tối đa hiệu năng, tác giả **huấn luyện 6 model với 6 seed ngẫu nhiên khác nhau rồi "soup"** (lấy trung bình trọng số) ở cuối: 3 lần dùng importance sampling **mức token**, 3 lần dùng importance sampling **mức chuỗi** (sequence-level, tham chiếu Zheng et al. 2025 = bài 43 GSPO của lộ trình này).

```
Pseudo-code huấn luyện RL (suy từ mô tả 3.1-3.2, paper không cho code):
model = SFT(Qwen2.5-VL-7B-Instruct, olmOCR-mix-1025)     # điểm khởi đầu
for seed in 1..6:
    theta = copy(model)
    for page, unit_tests in olmOCR2-synthmix-1025:          # 1 epoch, 2.186 trang
        sample 28 completions {o_1..o_28} ~ pi_theta(.|page)
        for o_i:
            r_i = pass_fraction(o_i, unit_tests)             # reward chính
                + reward_eos(o_i) + reward_metadata(o_i)      # 2 reward phụ, trọng số
                ↪  không nêu
        A_i = (r_i - mean(r_1..r_28)) / std(r_1..r_28)        # GRPO, beta=0.01
        cập nhật theta (token-level IS cho 3/6 seed, sequence-level IS cho 3/6 seed còn
        ↪  lại)
    luu checkpoint_seed[seed] = theta
olmOCR-2-7B-1025 = trung_bình_trọng_số(checkpoint_seed[1..6])   # souping
```

4. Thí nghiệm và kết quả chính

Bảng 1 (trích, olmOCR-Bench, phiên bản benchmark tại thời điểm viết bài; cột điểm tổng ± CI; * = số do chính tác giả mô hình đó công bố, còn lại tác giả bài báo tự đo lại):

Model	Overall ± CI	Ngày công bố
Qwen 2 VL 7B	31,5 ± 0,9	08/2024
GPT-4o	68,9 ± 1,1	05/2024

Model	Overall ± CI	Ngày công bố
Gemini Flash 2	57,8 ± 1,1	12/2024
Qwen 2.5 VL 7B	65,5 ± 1,2	02/2025
olmOCR (bản đầu)	68,2 ± 1,1	02/2025
Mistral OCR API	72,0 ± 1,1	03/2025
MinerU 1.3.10	61,5 ± 1,1	04/2025
Nanonets OCR S	64,5 ± 1,1	06/2025
MonkeyOCR Pro 3B	75,8 ± 1,0*	06/2025
dots.OCR	79,1 ± 1,0*	07/2025
Marker 1.10.1	76,1 ± 1,1	09/2025
MinerU 2.5.4	75,2 ± 1,1*	09/2025
PaddleOCR-VL	80,0 ± 1,0*	10/2025
DeepSeek-OCR	75,7 ± 1,0	10/2025
Infinity-Parser 7B	82,5 ± ?*	10/2025
Chandra OCR 0.1.0	83,1 ± 0,9*	10/2025
olmOCR 2	**82,4 ± 1,1**	10/2025

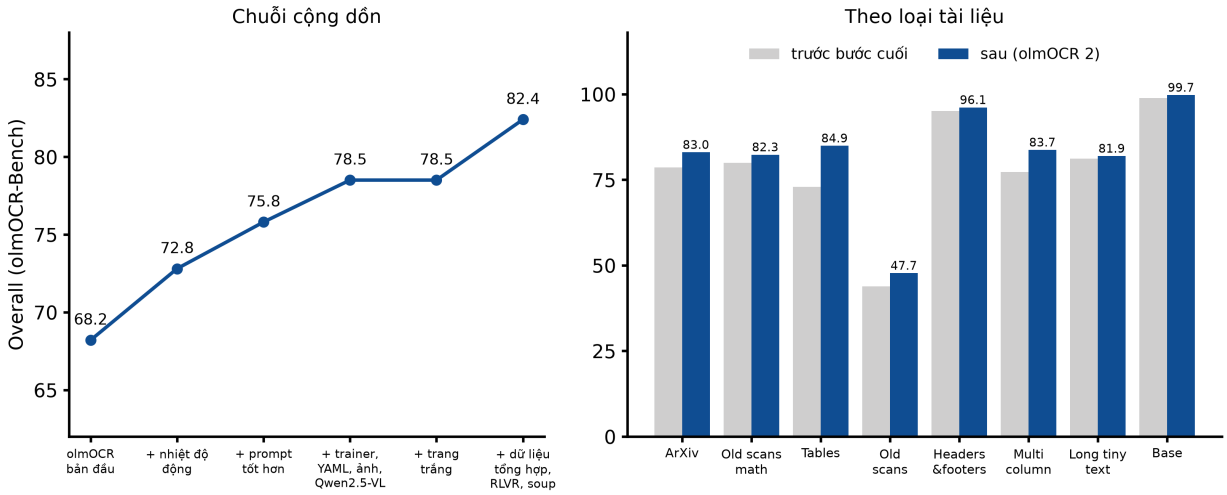
(Lưu ý: điểm olmOCR bản đầu ở đây là **68,2**, khác với **75,5** trong chính bài gốc llm 42, vì bài này dùng **phiên bản olmOCR-Bench mới hơn**, không so trực tiếp được giữa hai bài.)

Bảng 3 (chuỗi ablation từ olmOCR bản đầu lên olmOCR 2, cùng đo trên olmOCR-Bench; cột: ArXiv, Old scans math, Tables, Old scans, Headers&footers, Multi column, Long tiny text, Base, Overall):

Bước	AR	OSM	TA	OS	HF	MC	LTT	Base	Overall
olmOCR (bản đầu)	63,3	67,5	62,3	38,6	93,4	67,6	54,8	97,9	68,2 ± 1,1
+ Dynamic temp scaling	71,4	73,1	65,6	40,5	93,2	76,6	64,9	96,7	72,8 ± 1,2
+ Better prompting	76,3	76,0	70,2	43,2	94,1	77,5	71,9	96,8	75,8 ± 1,0
+ New trainer, YAML, resize ảnh, Qwen2.5-VL	78,8	77,5	71,9	45,4	94,2	78,6	81,4	99,8	78,5 ± 1,1
+ Xử lý trang trắng	78,6	79,9	72,9	43,9	95,1	77,3	81,2	98,9	78,5 ± 1,1
+ Synth data, RLVR, souping (olmOCR 2)	**83,0**	**82,3**	**84,9**	**47,7**	96,1	**83,7**	81,9	99,7	**82,4 ± 1,1**

Cải thiện lớn nhất ở bước RLVR cuối rơi vào **Tables (+12,0: 72,9 → 84,9), ArXiv math (+4,4), Old Scans Math (+2,4), Multi-column (+6,4)**, đúng như tác giả mô tả "cải thiện lớn nhất ở công thức toán, bảng, bố cục nhiều cột". Old Scans (OS) gần như không đổi (43,9 → 47,7).

Cách đọc hình: panel trái là điểm Overall sau mỗi thay đổi cộng dồn, từ 68,2 lên 82,4. Panel phải tách hai dòng cuối của Bảng 3 theo loại tài liệu; Tables tăng nhiều nhất, Old scans gần như không đổi. Bước cuối gộp cả dữ liệu tổng hợp, RLVR và souping nên không tách riêng được phần của RLVR.



Hình 100: Chuỗi ablation của olmOCR 2 (Bảng 3), và so sánh từng loại tài liệu trước và sau bước cuối (vẽ lại từ số liệu trong ghi chú)

5. Phân tích / ablation

Bảng 3 ở trên **chính là ablation chain** của bài: mỗi dòng cộng dồn một thay đổi. Các điểm đáng chú ý:

- **Temperature động (bắt đầu 0,1, tăng dần 0,2/0,3/.../0,8 mỗi khi model không sinh EOS, tức có nguy cơ lặp vô hạn):** +4,6 điểm tổng so với nhiệt độ cố định 0,8 ở bản đầu. Tận dụng được lợi thế nhiệt độ thấp (bám sát phân phối, ít lỗi) mà vẫn có "van an toàn" khi mô hình có dấu hiệu lặp.
- **Sửa lỗi prompt (thứ tự ảnh/text không khớp giữa train và inference):** +3,0 điểm, cải thiện lớn dù chỉ là bugfix, cho thấy **chi tiết vận hành** (không phải kiến trúc/thuật toán) đôi khi quan trọng hơn cả thay đổi mô hình.
- **Đổi JSON → YAML:** không đổi điểm benchmark trực tiếp, nhưng giảm mạnh tỉ lệ cần retry (ít lặp vô hạn hơn, vì model không cần nhớ số dấu ngoặc kép đang mở) → tăng hiệu quả suy luận (không đo bằng điểm mà bằng chi phí vận hành).
- **Xử lý trang trắng (bugfix data loader bỏ sót toàn bộ trang trắng khi huấn luyện):** không đổi điểm benchmark (78,5 → 78,5), dù tác giả kỳ vọng tránh được ảo giác trên trang trắng trong thực tế.
- **Bước cuối (synth data + RLVR + souping) gộp chung +3,9 điểm (78,5 → 82,4):** đây là bước **không tách được RLVR và model souping** riêng, vì cả hai được áp dụng cùng lúc trong một dòng bảng. Người ghi chú lưu ý: không có cách nào từ bảng này biết souping một mình đóng góp bao nhiêu so với RLVR một mình.

6. Giới hạn

Tác giả nêu: Muốn mở rộng quy trình dữ liệu tổng hợp sang loại tài liệu và unit test phức tạp hơn; muốn khám phá sâu hơn sự khác biệt giữa unit test nhị phân và điểm liên tục kiểu edit distance (bao gồm cả so sánh với các RL reward khác như Infinity Parser).

Nhận xét:

- **RLVR và souping bị gộp trong một bước ablation** (mục 5): không tách được đóng góp riêng của RLVR.
- **Không nêu trọng số giữa 3 thành phần thưởng** (tỉ lệ pass unit test, EOS, metadata), nên khó biết thành phần nào chi phối gradient.
- **Không nêu learning rate, optimizer, epsilon clip** của GRPO, cũng không nêu kích thước batch rollout; chỉ biết TRL, β=0,01, 28 completion/tài liệu, 8×H100, 1 epoch.

- **So sánh giữa các phiên bản olmOCR-Bench:** bài này báo điểm olmOCR bản đầu là 68,2, trong khi bài gốc báo 75,5. Cần ghi rõ phiên bản benchmark và cách đánh giá trước khi so sánh.
- **Reward hacking tiềm ẩn chưa được kiểm tra chéo trong chính bài này:** bài 57 (FD-RL, đọc sau) đánh giá lại olmOCR 2 trên **OmniDocBench** (benchmark khác) và đo được **80,03** tổng, thấp hơn cả olmOCR-7B **bản đầu** (81,79) theo đúng bảng của bài 57. Nghĩa là RLVR bằng unit test olmOCR-Bench có thể **tối ưu quá khớp phong cách của chính benchmark dùng để thiết kế reward**, và **không chắc chuyển hoá thành cải thiện trên benchmark khác**; olmOCR 2 không tự đánh giá trên OmniDocBench nên không tự phát hiện vấn đề này.

7. Liên hệ với các bài khác

- **llm 42 olmOCR:** bài tiền nhiệm trực tiếp, nguồn olmOCR-Bench và mô hình nền (Qwen2-VL-7B → nay đổi Qwen2.5-VL-7B).
- **00, 36 DeepSeekMath:** nền tảng GRPO.
- **43 GSPO:** chính bài này trích dẫn "Zheng et al. 2025" (= GSPO) khi dùng importance sampling mức chuỗi cho 3/6 seed; đọc 43 để hiểu khác biệt token-level vs sequence-level IS mà olmOCR 2 chỉ nhắc chứ không giải thích.
- **55 Visual-RFT:** cùng tinh thần "thưởng kiểm chứng rule-based thay reward model", nhưng 55 dùng IoU/CLS cho detection/classification còn bài này dùng unit test nhị phân cho văn bản dài; thưởng ở đây là **phân số liên tục** (tỉ lệ pass) trong khi R_{acc} ở 55 là nhị phân thuần, nên có thể cho gradient tín hiệu mịn hơn trên mỗi rollout.
- **57 FD-RL (bài tiếp theo):** đánh giá chéo olmOCR 2 trên OmniDocBench (80,03, xem mục 6); đồng thời đề xuất một thiết kế reward khác (tách theo loại nội dung text/formula/table) thay vì unit test tổng hợp.
- **41 DAPO:** ý tưởng dynamic sampling (loại nhóm có mọi reward giống nhau) có thể liên quan khi reward là phân số liên tục: nếu mọi rollout của một trang đều đạt cùng tỉ lệ pass (ví dụ trang rất dễ, mọi completion đều pass 100%), advantage = 0 cho cả nhóm, lãng phí rollout; olmOCR 2 không bàn vấn đề này.

8. Ứng dụng thực tế (suy luận)

Bài học tổng quát cho bất kỳ ai cần thiết kế reward RLVR cho một tác vụ sinh có cấu trúc (không chỉ OCR): **dùng một VLM/LLM tổng quát để sinh "đại diện có cấu trúc" (ở đây là HTML) của dữ liệu thật, rồi trích unit test tự động từ cấu trúc đó** có thể tạo nhiều test tự động từ dữ liệu tổng hợp. Độ tin cậy của test phụ thuộc vào cấu trúc HTML và quá trình sinh dữ liệu, nên vẫn cần kiểm tra chất lượng.

Phần thưởng theo tỉ lệ unit test đạt yêu cầu cho phép phân biệt các đầu ra đúng một phần. Tuy nhiên, chất lượng của bộ kiểm thử và mức độ bao phủ lỗi quyết định ý nghĩa của phần thưởng. Cần đánh giá trên một benchmark độc lập với bộ dùng thiết kế reward.

9. Thí nghiệm gợi ý

1. Tạo một tập unit test nhỏ từ tài liệu tổng hợp và đo số lượng, chất lượng và chi phí tạo test.
2. So sánh RLVR với và không có model souping để tách đóng góp của từng thành phần.
3. Đo tỉ lệ nhóm rollout có cùng phần thưởng để xem việc lọc nhóm kiểu DAPO có thể hữu ích hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Reward chính	$r =$ số unit test pass / tổng unit test

Khái niệm	Công thức / giá trị
Reward phụ	nhị phân EOS + thưởng metadata [0,1] (trọng số không nêu)
Pipeline dữ liệu	PDF thật → HTML (3 bước, Claude Sonnet 4, 0,12 USD/trang) → unit test tự động
Dữ liệu RL	olmOCR2-synthmix-1025: 2.186 trang, 30.381 test
Dữ liệu SFT mới	olmOCR-mix-1025: 267.962 trang, >100.000 PDF
Huấn luyện RL	1 epoch, 8×H100, 28 completion/tài liệu, TRL, β=0,01, GRPO
Souping	6 seed (3 token-level IS, 3 sequence-level IS kiểu GSPO)
Kết quả	olmOCR-Bench 82,4 ± 1,1 (so olmOCR bản đầu 68,2 ± 1,1, cùng phiên bản bench)
Cảnh báo chéo	OmniDocBench (bài 57): olmOCR 2 chỉ 80,03, thấp hơn olmOCR bản đầu 81,79

11. Kết luận ngắn

olmOCR 2 chứng minh rằng unit test nhị phân (vốn dùng để **đánh giá** OCR trong olmOCR-Bench) có thể **scale thành phần thưởng RL** nếu có quy trình tự động sinh unit test từ dữ liệu tổng hợp (render HTML), không cần người duyệt tay. Kết hợp RLVR với nhiều cải tiến vận hành (temperature động, sửa prompt, đối định dạng, souping đa seed), olmOCR 2 đạt 82,4 trên olmOCR-Bench, cải thiện rõ nhất ở công thức toán, bảng và bố cục nhiều cột. Tuy nhiên bài không tách được đóng góp riêng của RLVR khỏi souping, và đánh giá chéo từ bài 57 cho thấy cải thiện này **không nhất thiết chuyển hoá sang benchmark khác** (OmniDocBench), một lời nhắc quan trọng khi áp dụng công thức này trong thực tế: luôn đánh giá trên benchmark độc lập với benchmark dùng thiết kế reward.

Link gốc: https://arxiv.org/abs/2510.19817

Bài 57

Reading or Reasoning? Format Decoupled Reinforcement Learning for Document OCR (2026)

Tác giả: Yufeng Zhong, Lei Chen (đóng góp bằng nhau), Zhixiong Zeng (project lead), Xuanle Zhao, Deyang Jiang, Liming Zheng, Jing Huang, Haibo Qiu, Peng Shi, Siqi Yang, Lin Ma (tác giả liên hệ). Meituan. **Venue / arXiv:** arXiv:2601.08834 (v1, 11/12/2025); bài báo không nêu venue. **Nguồn:** https://arxiv.org/abs/2601.08834 **Chủ đề:** Quan sát: văn bản có định dạng (công thức, bảng) có entropy đầu ra cao hơn hẳn văn bản thuần; từ đó đề xuất **FD-RL**: huấn luyện hai giai đoạn SFT-rồi-RL, giai đoạn RL dùng **lọc dữ liệu theo entropy** và **reward tách theo loại nội dung** (text/formula/table), đạt SOTA trong nhóm model OCR end-to-end trên OmniDocBench. **Vị trí trong lộ trình:** Đọc trước: 50 (High-Entropy Minority Tokens, nguồn cảm hứng trực tiếp được trích dẫn trong bài: token entropy cao là "ngã ba" dẫn hướng suy luận), 55 (Visual-RFT), 56 (olmOCR 2, cũng RLVR cho OCR nhưng reward là unit test thay vì NED/BLEU/TEDS theo loại nội dung). Bài này khép lại nhóm "RL cho OCR" của lộ trình, và tự đánh giá lại chính olmOCR 2 (bài 56) trên một benchmark khác với kết quả bất ngờ (mục 6-7).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" được tính từ số liệu đã nêu. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Các model OCR VLM end-to-end SOTA gần đây (DeepSeek-OCR, dots.ocr, PaddleOCR-VL...) chủ yếu cải thiện bằng cách đầu tư vào **kỹ thuật dữ liệu cho SFT**: thu thập nhiều hơn, làm sạch kỹ hơn. Tác giả bài này quan sát một hiện tượng bị bỏ qua: dù mô hình đã SFT tốt, **entropy đầu ra của văn bản có định dạng (công thức, bảng) vẫn cao hơn văn bản thuần rõ rệt, đôi khi chênh một bậc độ lớn**. Thí nghiệm minh hoạ (Hình 1, dùng Qwen3-VL): chia tài liệu theo tỉ lệ nội dung có định dạng thành 5 nhóm (20/40/60/80/100%), tính entropy đầu ra trung bình lúc suy luận cho mỗi nhóm; tỉ lệ định dạng càng cao, entropy càng cao.

Tác giả lập luận: token entropy cao đóng vai trò "ngã ba" dẫn hướng model tới các đường suy luận khác nhau (trích dẫn trực tiếp bài 50, "Beyond the 80/20 Rule"). Văn bản định dạng có nhiều cách đọc **đều đúng ngữ nghĩa** nhưng khác chuỗi ký tự (phân số viết `1/2` hay `\frac{1}{2}` đều đúng). SFT ép model học đúng **một** chuỗi mục tiêu, còn RL có thể thưởng cho **nhiều đường đọc hợp lệ khác nhau**. Các công trình RL-OCR trước đó (Infinity Parser, olmOCR 2 = bài 56, Logics-Parsing) tự xây reward hỗn hợp theo kiểu thực nghiệm, chưa nhắm có chủ đích vào nội dung entropy cao.

Thuật ngữ cần biết: **CDM (Character Detection Matching):** so khớp công thức bằng render rồi so vị trí ký hiệu, không so chuỗi LaTeX trực tiếp (giống tinh thần unit test công thức ở bài 56). **TEDS (Tree Edit Distance based Similarity):** chỉ số chuẩn cho cấu trúc bảng (so cây HTML dự đoán với ground truth); TEDS-S chỉ so cấu trúc, bỏ nội dung ô. **NED (Normalized Edit Distance):** edit distance chuẩn hoá theo độ dài, dùng cho văn bản thuần. **OmniDocBench:** benchmark 1.355 trang, 9 loại tài liệu, 2 ngôn ngữ, chấm điểm liên tục theo 3 chỉ số (khác hẳn unit test nhị phân của olmOCR-Bench, bài 42/56).

2. Ý tưởng chính

FD-RL huấn luyện **hai giai đoạn**. Giai đoạn 1 (SFT): xây một model OCR nền tảng mạnh từ Qwen3-VL-4B trên kho dữ liệu lớn, đa nguồn (mở + PDF thật + tổng hợp), đóng băng vision encoder và projector, chỉ cập nhật LLM. Giai đoạn 2 (RL): hai cải tiến chính. (a) **Lọc dữ liệu theo entropy:** dùng chính mô hình đã SFT để suy luận trên tập ứng viên, tính entropy đầu ra trung bình từng mẫu, chỉ giữ lại các mẫu entropy cao (nội dung định dạng, khó, nhiều đường đọc hợp lệ) cho RL, bỏ bớt mẫu dễ/entropy thấp. (b) **Reward tách theo**

loại nội dung (format decoupled reward): tách output và ground truth thành văn bản thuần / công thức / bảng bằng regex, tính reward **riêng cho từng loại** bằng chỉ số phù hợp (NED cho text, BLEU sau khi chuẩn hoá LaTeX cho công thức, TEDS cho bảng), rồi lấy trung bình trên các loại có mặt trong ground truth, tối ưu bằng GRPO. Kết quả: **90,41** điểm tổng trên OmniDocBench, SOTA trong nhóm model **end-to-end** (vượt dots.ocr +2,0, DeepSeek-OCR +3,4), và vượt xa các mô hình RL-OCR trước đó (olmOCR 2, Infinity Parser, Logics-Parsing) khi đo lại trên cùng benchmark này.

3. Phương pháp chi tiết

3.1 Kỹ thuật dữ liệu (3 nguồn, 566k mẫu SFT)

1. **Tăng chất lượng dữ liệu mở** (240k mẫu): gộp PDFA, DocStruct4M, DocGenome (văn phòng/form/cấu trúc) + IAM/ORAND-CAR/HME (chữ viết tay/công thức). Phát hiện lỗi phổ biến (thiếu nội dung, sai thứ tự đọc, lặp câu) → **gán nhãn lại toàn bộ** bằng model nhẹ GOT → lọc theo độ tương đồng nhãn gốc/nhãn GOT, chỉ giữ mẫu tương đồng cao. **Giữ nhãn gốc làm ground truth** (không dùng nhãn GOT) để tránh học lệch theo phong cách riêng của GOT.
2. **Dữ liệu PDF thật** (208k): *layout-aware* (cấp trang: MinerU + Mathjax + Text-Dedup; cấp vùng: Mathpix lấy OCR có toạ độ, khung màu, mẫu âm khung rỗng; cấp đa trang: 2-6 trang liên tiếp) và *content-aware* (dots.ocr chuyển công thức → LaTeX, bảng → HTML, sắp lại thứ tự đọc; hậu xử lý bỏ ảnh, xoá header/footer).
3. **Dữ liệu OCR tổng hợp** (118k): bài tập giáo dục (K12-sau đại học) + hỏi đáp StackExchange → template HTML (font/khoảng cách/màu) + MathJax + CSS → render bằng Playwright thành ảnh, ghép Markdown làm mẫu train.

(Phụ lục, Bảng 7-8): tổng 566k mẫu, trong đó academic papers 136k, books 108k, magazines 100k, newspapers 82k, synthetic 118k, slides 10k, notes 6k, financial reports 3k, exam papers 3k.

3.2 Giai đoạn SFT

Tinh chỉnh Qwen3-VL-4B thành `FD-RL(SFT)` bằng framework **LLaMA-Factory**, 2 node × 8 GPU, template Qwen3-VL (không dùng thinking token), độ dài chuỗi tối đa **8.192 token**, 128 worker tiền xử lý (batch 256), 128 worker dataloader, **lr = 1e-5**, cosine, warmup 10%, batch per-device = 1, **1 epoch**, BF16. **Đóng băng ViT và MLP**, chỉ full tinh chỉnh phần LLM.

3.3 Lọc dữ liệu theo entropy cho RL

Từ tập SFT, loại các loại tài liệu chủ yếu văn bản thuần (slides, magazines, books, newspapers), lọc tiếp bằng regex để chỉ giữ mẫu có công thức/bảng → tập ứng viên **16k mẫu**. Dùng chính model `FD-RL(SFT)` suy luận trên toàn bộ, lấy log-xác suất (logprob) từng token sinh ra, tính entropy trung bình mỗi mẫu:

$$D_{filtered} = \left\{ d \in D_{raw} \;\middle|\; -\frac{1}{N_d} \sum_{i=1}^{N_d} \log p_i(d) \;\geq\; \tau \right\}$$

với τ là ngưỡng entropy, N_d là số token của mẫu d, $p_i(d)$ là xác suất token thứ i. **Giữ top 50% mẫu entropy cao nhất** làm tập RL cuối cùng (≈ 8k mẫu, **tự tính**: 16k × 50%; bài báo không nêu số tuyệt đối sau lọc, chỉ nêu tỉ lệ). Bài báo không cho giá trị τ cụ thể, chỉ kiểm soát qua "tỉ lệ lọc" (xem ablation mục 5).

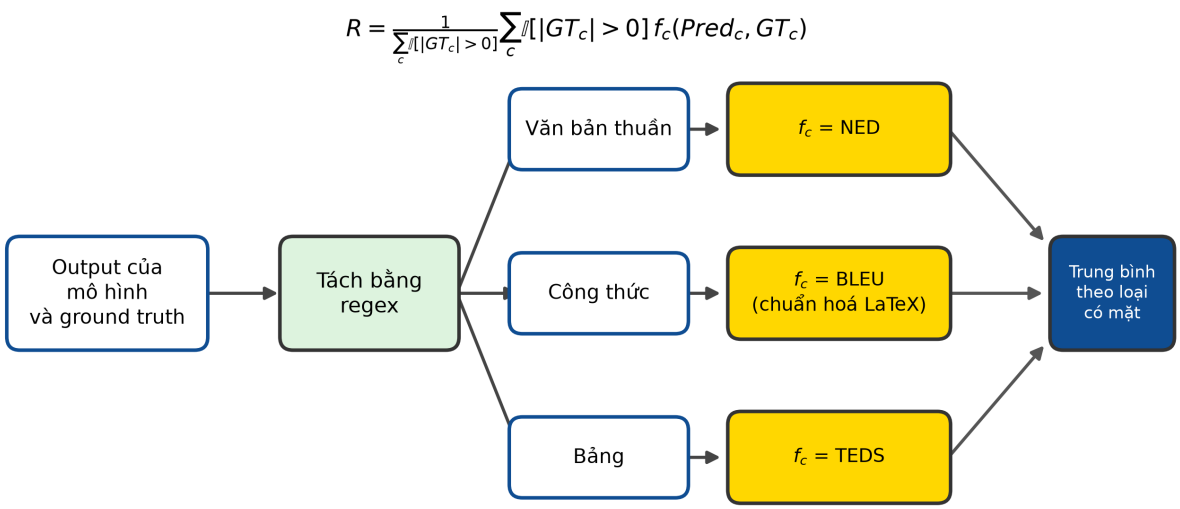
Lưu ý của phần phân tích: công thức (1) là **log-likelihood âm trung bình của các token đã lấy mẫu** (surprisal của chính chuỗi model sinh ra), **không phải entropy Shannon đầy đủ** trên toàn phân phối từ vựng (muốn vậy phải tính $-\sum_{v \in V} p(v) \log p(v)$, tốn hơn hẳn). Paper gọi cả phân tích Hình 1 (dùng Qwen3-VL) lẫn công thức lọc dữ liệu (1) (dùng mô hình đã SFT) là "entropy" dù hai phép đo không giống hệt nhau.

3.4 Reward tách theo loại nội dung (Format Decoupled Reward)

Tách công thức, bảng, văn bản thuần khỏi cả output model và ground truth bằng regex. Văn bản thuần: NED (so ký tự). Công thức: chuyển về LaTeX tương đương rồi tính BLEU. Bảng: TEDS (so cấu trúc cây). Reward tổng là **trung bình trên các loại nội dung có mặt trong ground truth**:

$$R = \frac{1}{\sum_{c=1}^{C} \mathbb{I}[|GT_c| > 0]} \sum_{c=1}^{C} \mathbb{I}[|GT_c| > 0] \cdot f_c(Pred_c, GT_c)$$

với C là tổng số loại nội dung, $\mathbb{I}[\cdot] = 1$ nếu ground truth loại c không rỗng (bỏ qua loại nội dung không xuất hiện trong trang, không phạt oan). f_c là hàm reward riêng cho loại c (NED/BLEU/TEDS).



Hình 101: Reward tách theo loại nội dung: mỗi loại được chấm bằng chỉ số riêng rồi lấy trung bình trên các loại có mặt (minh hoạ, tự tính)

Cách đọc hình: output và ground truth cùng được tách bằng regex thành văn bản thuần, công thức và bảng. Mỗi loại đi qua hàm f_c riêng (NED, BLEU, TEDS), rồi các điểm được trung bình. Loại nào không có trong ground truth thì không đóng góp và không bị phạt.

3.5 GRPO và cấu hình huấn luyện RL

GRPO chuẩn (tham chiếu DeepSeekMath = bài 36, cùng dạng công thức như bài 55/56): G phản hồi mỗi đầu vào, lợi thế chuẩn hoá theo nhóm $A_i = (R_i - \text{mean})/\text{std}$, tối ưu mục tiêu kiểu PPO-clip với ϵ (bài báo không cho giá trị). **Khác biệt đáng chú ý so với bài 56 (olmOCR 2):** FD-RL **loại bỏ hoàn toàn số hạng phạt KL** khỏi hàm loss ("để cho phép cập nhật policy linh hoạt hơn"), trong khi olmOCR 2 giữ $\beta = 0,01$.

Cấu hình (phụ lục 6.2.2): framework **EasyR1** (xây trên veRL); **rollout batch = train batch = 32**; nhiệt độ sinh = **1**; $G = 8$ **rollout/mẫu**; độ dài phản hồi tối đa **10.240 token**; tối ưu **lr = 1e-6**, weight decay **1e-2**; **toàn bộ tham số model (kể cả ViT, khác SFT) được mở khoá**; 1 epoch trên **1 node × 8 GPU**, mất **≈ 10 giờ**.

```
Pseudo-code FD-RL giai đoạn RL (tóm tắt 3.3-3.5, paper không cho code):
candidates = regex_filter(SFT_pool - {slides,magazines,books,newspapers}, có công
↪  thức/bảng)  # 16k
entropy[d]  = -1/N_d * sum(log p_i(d))  với p từ model_sft suy luận trên d  # mọi d
↪  trong candidates
RL_data = top_50%(candidates, theo entropy)                    # ~8k mẫu (tự tính)
```

```
theta = copy(model_sft); mở khoá toàn bộ tham số (kể cả ViT)
for x, GT in RL_data:                                          # 1 epoch, G=8, batch
↪  32, T=1
    sample {o_1..o_8} ~ pi_theta(.|x)
    R_i = trung_bình(NED(text), BLEU(latex), TEDS(table))      # chỉ loại có mặt trong
    ↪  GT
    A_i = (R_i - mean(R_1..R_8)) / std(R_1..R_8)
    cập nhật theta bằng J_GRPO (clip epsilon, KHÔNG có số hạng KL)
```

Chỉ số OmniDocBench: $\text{Overall} = \big((1 - \text{TextEdit}) \times 100 + \text{FormulaCDM} + \text{TableTEDS}\big)/3$.

4. Thí nghiệm và kết quả chính

Bảng 1 (OmniDocBench, trích các dòng tiêu biểu; Overall càng cao càng tốt, TextEdit càng thấp càng tốt):

Loại	Model	E2E	Overall	TextEdit	FormulaCDM	TableTEDS
Pipeline	MinerU2.5	✗	90,67	0,047	88,46	88,22
Pipeline	PaddleOCR-VL (0,9B)	✗	**92,56**	0,035	91,43	89,76
VLM tổng quát	Qwen2,5-VL-72B	✓	87,02	0,094	88,27	82,15
VLM tổng quát	Gemini-2,5 Pro	✓	88,03	0,075	85,82	85,71
End-to-end chuyên OCR	olmOCR-7B (v1, bài 42)	✓	81,79	0,096	86,04	68,92
End-to-end chuyên OCR	DeepSeek-OCR	✓	87,01	0,073	83,37	84,97
End-to-end chuyên OCR	dots.ocr	✓	88,41	0,048	83,22	86,78
End-to-end chuyên OCR	**FD-RL**	✓	**90,41**	**0,049**	**88,67**	**87,35**

FD-RL là **SOTA trong nhóm end-to-end** (vượt dots.ocr +2,0, DeepSeek-OCR +3,4), nhưng **hai hệ quy trình** (không phải end-to-end) là PaddleOCR-VL (92,56) và MinerU2.5 (90,67) vẫn cao hơn FD-RL về điểm tổng.

Bảng 2 (edit distance theo loại tài liệu, trích vài cột tiêu biểu, thấp hơn = tốt hơn):

Model	Slides	Exam Paper	Book	Notes	Academic Papers
dots.ocr	0,0290	0,0467	0,0433	0,1116	0,0231
olmOCR-7B	0,0497	0,0728	0,0539	0,1220	0,0365
FD-RL	**0,0235**	**0,0464**	0,0300	**0,0881**	0,0258

FD-RL đứng đầu 4/9 loại tài liệu (slides, exam paper, và 2 loại khác), đứng thứ nhì ở 5/9 loại còn lại (paper tự báo cáo).

Bảng 9 (so với các mô hình RL-OCR khác, cùng đo trên OmniDocBench, cùng protocol của bài này):

Model	Overall	TextEdit	FormulaCDM	TableTEDS
Infinity Parser	56,12	0,494	73,70	44,05
olmOCR 2 (bài 56)	**80,03**	0,163	84,00	72,40

Model	Overall	TextEdit	FormulaCDM	TableTEDS
Logics-Parsing	83,85	0,158	84,36	82,97
FD-RL	**90,41**	0,049	88,67	87,35

FD-RL vượt Logics-Parsing (SOTA trước đó trong nhóm RL-OCR) **+6,56 điểm**.

5. Phân tích / ablation

Bảng 3 (dữ liệu đa nguồn, cộng dồn từng nguồn): không dữ liệu (mô hình gốc) 46,06 → +OSD (dữ liệu mở đã làm sạch) 78,25 (+32,19) → +RWPD (PDF thật) 84,16 (+5,91) → +SOD (tổng hợp) 87,13 (+2,97) → +RLD (dữ liệu RL, tức toàn bộ giai đoạn RL) **90,41 (+3,28)**. Mỗi nguồn đều đóng góp dương, dữ liệu mở đóng góp lớn nhất (hợp lý vì là nền tảng đầu tiên).

Bảng 4 (hai giai đoạn SFT-rồi-RL):

SFT	RL	Overall	Formula	Table	Table-S
✗	✗	46,06	55,40	38,06	42,18
✗	✓	49,37 (+3,31)	62,03	32,08	34,32
✓	✗	87,13	85,60	81,27	84,91
✓	✓	**90,41**	**88,67** (+3,07)	**87,35** (+6,08)	**92,10** (+7,19)

RL trực tiếp lên model chưa SFT chỉ cải thiện +3,31 (chưa có năng lực OCR nền để RL "mài sắc"), trong khi SFT riêng cho +41,07; RL **sau** SFT cộng thêm +3,28, tập trung rõ nhất vào bảng (+6,08 TEDS) và công thức (+3,07 CDM), đúng kỳ vọng thiết kế.

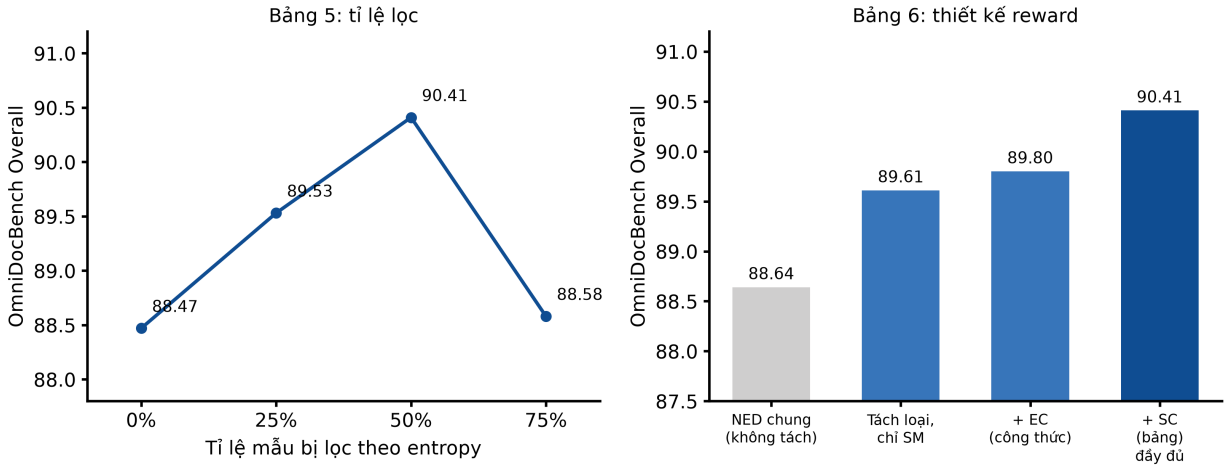
Bảng 5 (tỉ lệ lọc entropy): 0% (không lọc) 88,47 → 25% 89,53 → **50% 90,41 (tốt nhất)** → 75% 88,58. Dạng chữ U ngược: không lọc thì lẫn mẫu dễ/entropy thấp làm loãng tín hiệu RL; lọc quá tay (75%) bỏ phí dữ liệu có giá trị.

Bảng 6 (reward tách theo loại nội dung, FP = có tách loại hay không, SM/EC/SC = string match/expression correctness/structural coherence):

FP	SM	EC	SC	Overall	TextEdit	Formula	Table
✗	✓	✗	✗	88,64	0,044	87,04	83,27
✓	✓	✗	✗	89,61 (+0,97)	0,052	87,18	86,84
✓	✓	✓	✗	89,80	0,050	87,59	86,80
✓	✓	✓	✓	**90,41**	0,049	88,67	**87,35**

Dòng đầu (FP=✗, chỉ SM áp đều cho toàn bộ output, không tách loại nội dung) chính là **reward NED thống nhất, không phân biệt định dạng** ("unified NED reward"), cho 88,64 điểm, thấp nhất trong 4 cấu hình. Thêm tách loại nội dung (FP=✓, vẫn chỉ dùng SM nhưng tính riêng từng loại) tăng Table rõ rệt (83,27 → 86,84) nhưng **TextEdit lại xấu đi** (0,044 → 0,052): tách reward theo loại giúp bảng nhưng hơi hại văn bản thuần, điều bài báo không giải thích. Thêm EC (công thức) và SC (bảng) tiếp tục cải thiện, đạt tốt nhất ở cấu hình đầy đủ.

Cách đọc hình: panel trái cho thấy Overall đạt cao nhất khi lọc 50% mẫu, dạng chữ U ngược. Panel phải đi từ reward NED chung đến cấu hình đầy đủ, điểm tăng từ 88,64 lên 90,41. Bài báo không báo cáo seed hay khoảng tin cậy, nên các chênh lệch nhỏ cần đọc thận trọng.



Hình 102: Ablation tỉ lệ lọc entropy (Bảng 5) và thiết kế reward (Bảng 6) trên OmniDocBench (vẽ lại từ số liệu trong ghi chú)

6. Giới hạn

Tác giả nêu: Không có mục giới hạn/future-work riêng biệt ngoài câu kết luận ngắn; bài không thảo luận rõ về chi phí tính entropy lọc dữ liệu hay độ bền của việc tách nội dung bằng regex.

Nhận xét:

- **"SOTA" chỉ đúng trong nhóm end-to-end:** hai hệ quy trình-based (PaddleOCR-VL 92,56; MinerU2.5 90,67) vẫn cao hơn FD-RL (90,41) về điểm tổng trên chính Bảng 1 của bài; tiêu đề "SOTA" cần đọc kèm điều kiện này.
- **Nhầm lẫn thuật ngữ entropy:** công thức (1) là surprisal trung bình của chuỗi đã sinh, không phải entropy Shannon đầy đủ; Hình 1 dùng Qwen3-VL để khảo sát còn bước lọc dữ liệu RL dùng chính mô hình đã SFT, hai phép đo khác mục đích dùng chung một tên.
- **Không báo cáo seed/CI** cho bất kỳ bảng nào (1-9); chênh lệch nhỏ giữa các dòng ablation (ví dụ Bảng 6: 89,61 → 89,80, chỉ +0,19) khó biết có ý nghĩa hay chỉ là nhiễu.
- **Tách reward theo loại làm hại văn bản thuần** (Bảng 6, TextEdit 0,044 → 0,052, xem mục 5); bài báo không phân tích nguyên nhân.
- ϵ **(clip GRPO) và** τ **(ngưỡng entropy công thức 1) không có giá trị số cụ thể**; lọc dữ liệu chỉ kiểm soát qua "tỉ lệ" (percentile), không qua ngưỡng τ tuyệt đối như công thức định nghĩa.
- **Bỏ số hạng KL khỏi loss** là lựa chọn thiết kế khác hẳn $\beta = 0,01$ của bài 56, nhưng không có ablation chứng minh việc bỏ KL có lợi; rủi ro drift chính sách lớn hơn không được bàn.
- **Phát hiện chéo quan trọng nhất** (từ chính Bảng 9): olmOCR 2 (bài 56) đo được **80,03** trên OmniDocBench, **thấp hơn** chính olmOCR-7B bản đầu (v1, chưa RLVR, **81,79** ở Bảng 1). Điểm số này thấp hơn trên OmniDocBench dù olmOCR 2 cải thiện trên olmOCR-Bench; so sánh giữa các mô hình không tách riêng được tác động của RLVR. Kết quả này gợi ý RLVR bằng unit test có nguy cơ **tối ưu quá khớp phong cách benchmark thiết kế reward**.

7. Liên hệ với các bài khác

- **50 High-Entropy Minority Tokens:** nguồn cảm hứng trực tiếp (trích dẫn tường minh), lý do entropy cao = "ngã ba" đáng để RL khai thác.
- **49 Entropy mechanism of RL:** cùng chủ đề entropy-RL, đọc bổ sung để hiểu sâu hơn vai trò entropy trong huấn luyện RL cho reasoning.
- **36 DeepSeekMath, 00:** nguồn công thức GRPO (paper trích Shao et al. 2024 trực tiếp).

- **42 Dr. GRPO:** phê bình chuẩn hoá advantage theo std (công thức 3.5 ở đây) giống hệt phê bình áp dụng được cho bài 55 và 56.
- **55 Visual-RFT:** cùng ý tưởng "thưởng tách thành nhiều thành phần theo loại lỗi/loại nội dung" ($R_d = R_{IoU} + R_{conf} + R_{format}$ ở đó, R tách theo text/formula/table ở đây), nhưng Visual-RFT tách theo **thành phần lỗi trong một loại tác vụ** còn FD-RL tách theo **loại nội dung**.
- **56 olmOCR 2:** đánh giá chéo trực tiếp trong Bảng 9 (mục 4, 6); khác biệt thiết kế: olmOCR 2 dùng unit test nhị phân + giữ KL ($\beta = 0,01$), FD-RL dùng NED/BLEU/TEDS liên tục theo loại nội dung + bỏ KL hoàn toàn; hai bài cũng validate trên hai benchmark khác nhau (olmOCR-Bench vs OmniDocBench), và phát hiện chéo ở mục 6 cho thấy điều đó gây lệch khi so chéo.

8. Ứng dụng thực tế (suy luận)

Hai bài học thiết kế chung, áp dụng được ngoài phạm vi OCR: (1) **tách reward theo loại nội dung đầu ra** và dùng chỉ số phù hợp riêng cho từng loại, thay vì một chỉ số chung cho toàn bộ chuỗi sinh ra; (2) **dùng chính mô hình đã SFT để đo độ khó/entropy của từng mẫu**, rồi tập trung ngân sách RL (vốn đắt hơn SFT nhiều) vào các mẫu khó nhất, thay vì lấy mẫu đều.

Các kết quả gợi ý đánh giá reward theo từng loại nội dung, đồng thời theo dõi sự đánh đổi giữa văn bản, công thức và bảng. Chọn mẫu theo entropy là một hướng phân bổ ngân sách huấn luyện, nhưng cần kiểm tra xem entropy có thực sự tương ứng với độ khó trong dữ liệu đang xét.

9. Thí nghiệm gợi ý

1. So sánh phân phối entropy giữa trang văn bản và trang có bảng hoặc công thức.
2. So sánh reward NED thống nhất với reward tách theo loại nội dung, theo dõi riêng từng chỉ số.
3. Đánh giá trên nhiều benchmark độc lập với reward để kiểm tra khả năng tổng quát hóa.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Lọc entropy	$D_{filtered} = \{d : -\frac{1}{N_d}\sum \log p_i(d) \geq \tau\}$, giữ top 50%
Format decoupled reward	$R = \frac{1}{\sum \mathbb{I}[\cdot]}\sum \mathbb{I}[\cdot]\, f_c(Pred_c, GT_c)$, f_c=NED/BLEU/TEDS
GRPO	$A_i = (R_i - \text{mean})/\text{std}$; clip ϵ (không nêu giá trị); **không có số hạng KL**
OmniDocBench Overall	$\big((1 - \text{TextEdit}) \times 100 + \text{CDM} + \text{TEDS}\big)/3$
SFT	Qwen3-VL-4B, đóng băng ViT+MLP, lr 1e-5, 566k mẫu, 1 epoch
RL	mở khoá toàn bộ, lr 1e-6, $G = 8$, batch 32, nhiệt độ 1, max 10.240 token, ~10 giờ/8 GPU
Kết quả chính	OmniDocBench 90,41 (SOTA nhóm end-to-end); vượt Logics-Parsing +6,56
Cảnh báo chéo	olmOCR 2 (bài 56) đo lại: 80,03, thấp hơn olmOCR v1 (81,79)

11. Kết luận ngắn

FD-RL chỉ ra một hiện tượng cụ thể và hữu ích: nội dung có định dạng (công thức, bảng) có entropy đầu ra cao hơn văn bản thuần, nên (a) đáng ưu tiên cho RL thay vì SFT thuần, và (b) đáng có hàm thưởng riêng theo loại nội dung thay vì một chỉ số chung. Kết hợp lọc dữ liệu theo entropy và reward tách loại, FD-RL đạt SOTA trong nhóm end-to-end trên OmniDocBench (90,41), vượt các mô hình RL-OCR trước đó (olmOCR 2, Logics-Parsing, Infinity Parser) khi đo lại cùng điều kiện. Đóng góp quan trọng không kém với người đọc là phát hiện phụ: olmOCR 2 (vốn SOTA trên olmOCR-Bench, bài 56) lại **kém hơn chính bản thân nó trước RLVR** khi đo trên OmniDocBench, một lời nhắc rằng RLVR bằng reward tự thiết kế cần được kiểm chứng trên benchmark độc lập trước khi tin tưởng.

Link gốc: https://arxiv.org/abs/2601.08834